前言

Preface

2006 年，Geoffrey Hinton 等人发表了一篇论文，展示了如何训练能够以超过 98% 的当时最先进精度识别手写数字的深度神经网络。他们把这项技术称为“深度学习”（deep learning）。深度神经网络是大脑皮层的一个（非常）简化模型，由多层人工神经元堆叠而成。当时，人们普遍认为训练深度神经网络是不可能的，许多研究者在 20 世纪 90 年代末已经放弃了这个想法。这篇论文重新激发了科学界的兴趣；不久之后，大量论文证明深度学习不仅可行，而且能够取得其他机器学习（machine learning，ML）技术难以企及的惊人结果——当然，这依赖于巨大的计算能力和海量数据。此后，这种热情扩展到了机器学习的许多其他领域。

十年后，机器学习已经征服了许多行业：为网页结果排序、推荐视频和商品、对生产线上的物品进行分类，有时甚至用于驾驶汽车。机器学习经常登上新闻头条，例如 DeepMind 的 AlphaFold 机器学习系统解决了困扰研究人员数十年的蛋白质折叠问题。但大多数时候，机器学习只是在幕后安静地工作。又过了十年，人工智能助手兴起：2022 年出现了 ChatGPT，2023 年出现了 Gemini、Claude 和 Grok，此后又有许多其他系统。人工智能真正进入了高速发展阶段，正在迅速改变每一个行业；过去只存在于科幻作品中的事物，如今已经非常现实。

在项目中使用机器学习

你自然会对机器学习感到兴奋，也希望加入这场变革。也许你想给自制机器人装上自己的“大脑”，让它识别人脸，或学会四处走动？又或者，你的公司拥有大量数据（用户日志、财务数据、生产数据、机器传感器数据、客服热线统计、HR 报告等），只要知道该去哪里寻找，就很可能从中挖掘出隐藏的价值。使用机器学习，你可以完成以下任务以及更多事情：

- 将客户分群，并为每个群体找到最佳营销策略。
- 根据相似客户购买过的商品，为每位客户推荐产品。
- 检测哪些交易很可能是欺诈交易。

- 预测明年的收入。
- 预测工作负载峰值，并建议最佳人员配置水平。
- 构建一个帮助客户的聊天机器人。

无论原因是什么，你都决定学习机器学习，并把它应用到项目中。这是个好主意！

目标与方法

本书假定你几乎不了解机器学习。目标是为你提供实现“从数据中学习”的程序所需的概念、工具和直觉。

本书将介绍大量技术，从最简单、最常用的技术（例如线性回归）到经常赢得竞赛的深度学习技术。为此，我们使用 Python—-数据科学和机器学习领域的主流语言—-以及开源且可用于生产环境的 Python 框架：

- Scikit-Learn 易于使用，同时能够高效实现许多机器学习算法，是学习机器学习的优秀起点。它由 David Cournapeau 于 2007 年创建，后来由法国计算机科学与自动化研究所（Inria）的研究团队负责维护，最近则由 Probabl.ai 负责推动发展。
- PyTorch 是一个强大而灵活的深度学习库，可以高效训练和运行各种神经网络，并把计算分配到多个 GPU（图形处理单元）上。PyTorch 最初由 Facebook 的 AI Research 实验室（FAIR）开发，于 2016 年首次发布；它源自用 Lua 编写的旧框架 Torch。2022 年，PyTorch 转入 Linux 基金会旗下的 PyTorch Foundation，以促进社区驱动的开发。

本书还会在学习过程中使用以下开源机器学习库：

- 第 6 章使用 XGBoost 实现一种强大的梯度提升技术。
- 第 13 章和第 15 章使用 Hugging Face 库下载数据集和预训练模型，包括 Transformer 模型。Transformer 极其强大且用途广泛，如今几乎所有人工智能助手都以它为主要构件。
- 第 19 章使用 Gymnasium 学习强化学习，也就是训练自主智能体。

本书偏重动手实践：通过具体的、可运行的例子建立对机器学习的直觉理解，只介绍少量必要的理论。

> **提示** 你当然可以不打开笔记本电脑阅读本书，但我强烈建议你亲自运行代码示例并进行实验。

代码示例

本书中的所有代码示例都是开源的，并以 Jupyter notebook 的形式在线提供，地址为 https://github.com/ageron/handson-mlp。Jupyter notebook 是包含文本、图像和可执行代码片段的交互式文档（本书使用 Python）。最简单、最快的入门方式是使用 Google Colab 运行这些 notebook：这项免费服务允许你直接在线运行 Jupyter notebook，无需在计算机上安装任何东西，只需要一个 Web 浏览器和 Google 账户。

> **注** 本书假定你使用 Google Colab，但我也在 Kaggle 和 Binder 等其他在线平台上测试过这些 notebook，你可以按自己的偏好选择。也可以在本地安装所需的库和工具（或使用本书的 Docker 镜像），直接在自己的计算机上运行 notebook。安装说明见 https://homl.info/install-p。

本书旨在帮助你完成工作。如果你希望使用代码示例之外的额外内容，且这种使用超出了合理使用原则的范围（例如销售或分发 O'Reilly 图书中的内容，或将本书的大量材料纳入产品文档），请联系 O'Reilly 获取许可，邮箱为 **permissions@oreilly.com**。

我们感谢署名，但通常不强制要求。署名通常包括书名、作者、出版商和 ISBN。例如："Aurélien Géron 著《Hands-On Machine Learning with Scikit-Learn and PyTorch》。版权所有 © 2026 Aurélien Géron，979-8-341-60798-9。"

前置知识

本书假定你有一些 Python 编程经验。如果还不了解 Python，https://learnpython.org 是很好的起点，Python.org 的官方教程也很不错。

本书还假定你熟悉 Python 的主要科学计算库，尤其是 NumPy、pandas 和 Matplotlib。如果从未使用过这些库，不必担心，它们很容易学习；作者为每个库都准备了教程，见 https://homl.info/tutorials-p。

如果你希望完全理解机器学习算法的工作原理，而不仅仅是学会使用它们，那么至少需要掌握一些数学概念，特别是线性代数。具体来说，你应该知道向量和矩阵是什么，以及如何执行向量相加、矩阵转置和矩阵乘法等简单操作。如果需要快速了解线性代数（其实并没有那么难），可以参考 https://homl.info/tutorials-p 上的教程。那里还提供微积分教程，理解神经网络训练时可能会有帮助，但掌握重要概念并不完全依赖它。本书偶尔还会用到指数和对数、少量概率论以及一些基础统计学概念，但不会涉及太高级的内容。需要帮助时，可以访问提供许多优质免费数学课程的 Khan Academy。

路线图

本书分为两部分。第一部分“机器学习基础”涵盖以下主题：

- 机器学习是什么、它试图解决哪些问题，以及机器学习系统的主要类别和基本概念。
- 一个典型机器学习项目的步骤。
- 通过将模型拟合到数据来学习。
- 最小化成本函数，也就是预测误差的度量。
- 处理、清洗和准备数据。
- 选择和构造特征，也就是数据字段。
- 使用交叉验证选择模型和调整超参数，例如训练多个模型变体，再选择在未见过的数据上表现最好的模型。
- 机器学习的挑战，尤其是欠拟合、过拟合以及偏差–方差权衡。
- 最常见的学习算法：线性和多项式回归、逻辑回归、k 近邻、决策树、随机森林和集成方法。
- 降低训练数据的维度，以对抗“维度灾难”。
- 其他无监督学习技术，包括聚类、密度估计和异常检测。

第二部分“神经网络与深度学习”涵盖以下主题：

- 神经网络是什么，以及它们适合解决什么问题。
- 使用 PyTorch 构建和训练深度神经网络。
- 重要的神经网络架构：适用于表格数据的前馈神经网络；适用于计算机视觉的卷积网络；用于序列处理的循环网络和长短期记忆（LSTM）网络；用于自然语言处理、视觉等任务的编码器–解码器、Transformer、状态空间模型（SSM）和混合架构；以及用于生成式学习的自编码器、生成对抗网络（GAN）和扩散模型。
- 如何构建能够通过试错学会良好策略的智能体（例如游戏中的机器人），也就是使用强化学习。
- 如何高效加载和预处理大量数据。

第一部分主要使用 Scikit-Learn，第二部分主要使用 PyTorch。

> **注意** 不要过早跳入深水区。深度学习无疑是机器学习中最令人兴奋的领域之一，但你应该先掌握基础知识。许多问题使用随机森林和集成方法等更简单的技术就能解决得很好。深度学习最适合图像识别、语音识别或自然语言处理等复杂问题，通常还需要大量数据、计算能力和耐心（除非你能够利用预训练神经网络）。

如果你特别关注某个主题，并希望尽快学到它，图 P-1 会告诉你应该先阅读哪些章节、哪些章节可以暂时跳过。

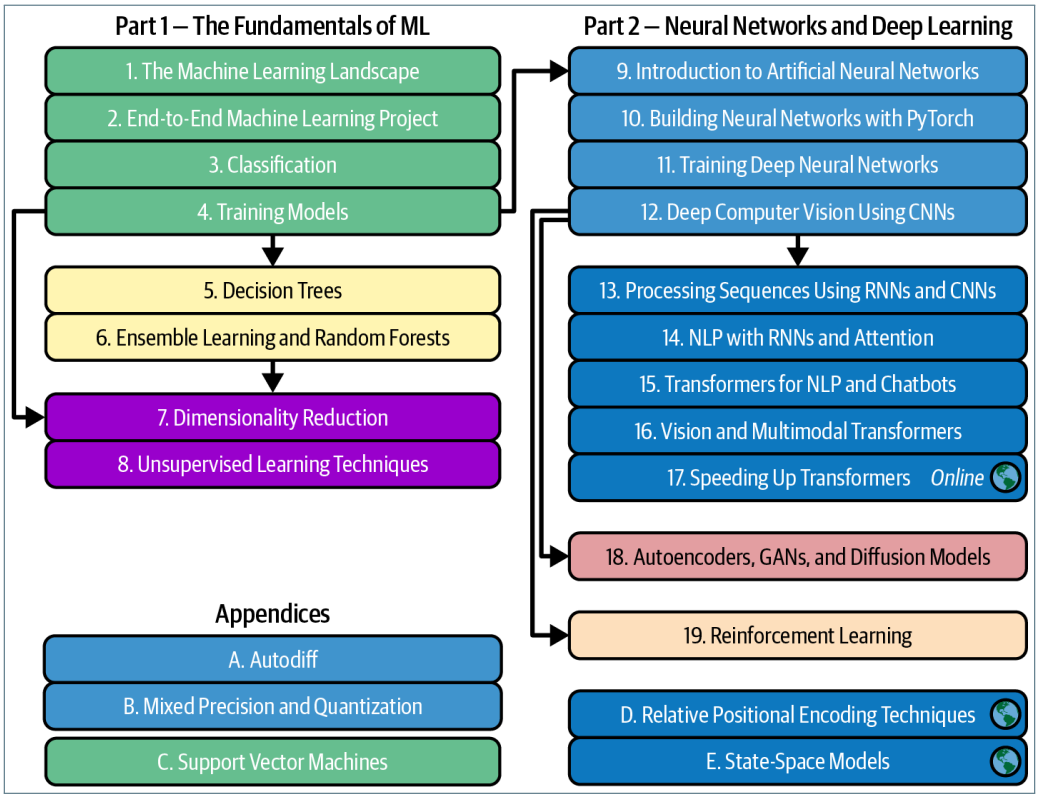

图 P-1：章节依赖关系（Chapter dependencies）

TensorFlow 版与 PyTorch 版的变化

作者曾为 O'Reilly 撰写过三版 TensorFlow (TF) 版本，分别于 2017、2019 和 2022 年出版。多年来，TF 一直是领先的深度学习库，被 Google 在内部使用并针对大规模生产进行了优化。但由于 PyTorch 简单、灵活且开放，它逐渐取得领先：如今它主导研究论文和开源项目，大多数新模型都优先提供 PyTorch 实现。因此，产业界也逐渐转向 PyTorch。

近几年，Google 减少了对 TensorFlow 的投入，转而重点发展 JAX—-另一个兼具研究和生产优势的优秀深度学习库。不过，与 PyTorch 相比，JAX 的采用率仍然较低。

这就是作者这次选择 PyTorch 的原因。O'Reilly 和作者决定把它作为新的 PyTorch 系列的第一版，而不是原系列的第四版。这也为将来的 JAX 系列，或者新的 TF 系列版本留下了空间。

如果你已经读过本书最新的 TensorFlow 版本，主要变化如下（详情见 https://homl.info/changes-p）：

- 全书代码更新到了较新的库版本。
- 第二部分的所有代码从 TensorFlow 和 Keras 迁移到了 PyTorch，这些章节都有较大变化。
- 删除了 TensorFlow 专属内容，包括原第 12、13 章以及原附录 C、D。
- 第 10 章现在介绍 PyTorch。
- 新增三章 Transformer 内容：第 15 章介绍自然语言处理中的 Transformer，包括如何构建聊天机器人；第 16 章介绍视觉 Transformer 和多模态 Transformer；第 17 章在线提供，讨论用于加速和扩展 Transformer 的高级技术，包括 FlashAttention、专家混合（MoE）和低秩适配（LoRA）等。
- 新增三个附录：附录 B 介绍如何缩小模型，使模型运行更快并适应更小的设备；“相对位置编码”介绍 Transformer 的高级位置编码技术；“状态空间模型（SSM）”介绍状态空间模型。
- 为新内容腾出空间，支持向量机（SVM）章节移至线上并改名为“支持向量机”；最后两个附录也在线提供，部署章节的一部分并入第 10 章。

TensorFlow/Keras 版本的三版图书分别简称为 **`homl1`**、**`homl2`** 和 **`homl3`**。本书是 PyTorch 版本的第一版，简称 **`homlp`**。试着连续说三遍吧。

> **注** 与最新 TensorFlow 版本相比，大多数变化集中在本书第二部分。如果你读过 **`homl3`**，不要期待第一部分有太大变化：机器学习的基本概念自 2022 年以来并没有改变。

其他资源

学习机器学习有许多优秀资源。例如，Andrew Ng 在 Coursera 上的机器学习课程非常出色，虽然需要投入大量时间。Scikit-Learn 的优秀用户指南、Dataquest 的互动教程，以及无数机器学习博客和 YouTube 频道也都值得参考。

其他适合入门的书籍包括：Joel Grus 的《Data Science from Scratch》第二版；Stephen Marsland 的《Machine Learning: An Algorithmic Perspective》第二版；Se-

bastian Raschka 的《Machine Learning with PyTorch and Scikit-Learn》;François Chollet 的《Deep Learning with Python》第三版；Andriy Burkov 的《The Hundred-Page Machine Learning Book》；Yaser S. Abu-Mostafa、Malik Magdon-Ismail 和 Hsuan-Tien Lin 的《Learning from Data》；Stuart Russell 和 Peter Norvig 的《Artificial Intelligence: A Modern Approach》第四版；Jeremy Howard 和 Sylvain Gugger 的《Deep Learning for Coders with fastai and PyTorch》；Andrew Ng 的《Machine Learning Yearning》；Lewis Tunstall、Leandro von Werra 和 Thomas Wolf 的《Natural Language Processing with Transformers》；以及 Jay Alammar 和 Maarten Grootendorst 的《Hands-On Large Language Models》。

最后，加入 Kaggle.com 等机器学习竞赛网站，可以在真实问题上练习技能，并获得优秀机器学习专业人士的帮助和见解。

本书使用的排版约定

斜体：表示新术语、URL、电子邮件地址、文件名和文件扩展名。

等宽字体：用于程序清单，以及段落中表示变量名、函数名、数据库、数据类型、环境变量、语句和关键字等程序元素。

等宽粗体：表示用户应按原样输入的命令或其他文本。

等宽斜体：表示应由用户提供的值，或根据上下文确定的值。

为避免混淆，本书始终把标点放在引号外。对此给讲究规范的读者表示歉意。

> **提示** 该元素表示一个提示或建议。

> **注** 该元素表示一般性说明。

> **警告** 该元素表示警告或注意事项。

O'Reilly 在线学习

四十多年来，O'Reilly Media 一直通过图书、文章和在线学习平台提供技术与商业培训、知识和洞见，帮助企业取得成功。O'Reilly 的在线学习平台提供按需直播课

程、深入学习路径、互动编码环境，以及来自 O'Reilly 和其他 200 多家出版商的大量文本与视频内容。详情见 https://oreilly.com。

联系我们

关于本书的意见和问题，请联系出版商：

O'Reilly Media, Inc.
141 Stony Circle, Suite 195
Santa Rosa, CA 95401
800-889-8969（美国或加拿大）
707-827-7019（国际或本地）
707-829-0104（传真）
support@oreilly.com
https://oreilly.com/about/contact.html

本书还有一个网页，用于发布勘误、示例和其他信息：https://oreil.ly/hands-on-machine-learning。有关 O'Reilly 图书和课程的新闻与信息，请访问 https://oreilly.com；也可以在 LinkedIn 关注 O'Reilly Media，或在 YouTube 观看 O'Reilly Media 的视频。

致谢

作者从未想过三版 TensorFlow 图书会拥有如此广泛的读者。作者收到了许多读者来信：有人提问，有人友善地指出勘误，更多人送来鼓励的话语。感谢所有读者给予的巨大支持。如果你在代码示例中发现错误（或只是想提问），欢迎在 GitHub 上提交 issue；如果在正文中发现错误，也欢迎提交勘误。

有些读者分享了本书如何帮助他们获得第一份工作，或如何帮助他们解决正在处理的具体问题。这些反馈极大地激励了作者。如果本书对你有帮助，也欢迎通过 LinkedIn 私下分享故事，或在 Amazon 上公开写下评论。

感谢所有慷慨抽出时间、凭借专业知识审阅本书并提出修改建议的人。他们让本书变得更好：Jeremy Howard、Haesun Park、Omar Sansevero、Lewis Tunstall、Leandro Von Werra 和 Sam Wittemeier 审阅了目录并帮助细化全书范围；Hesam Hassani、Ashu Jha、Meetu Malhotra 和 Ammar Mohanna 审阅了第一部分；Ulf Bissbort、Louis-François Bouchard、Luba Elliot、Thomas Lacombe、Tarun Narayanan、Marco Tabor

以及作者的弟弟 Sylvain 审阅了第二部分。特别感谢 Haesun Park 审阅每一章。你们都非常了不起!

当然，没有 O'Reilly 出色的工作人员，本书也不可能完成。作者尤其感谢审阅每一章并在整整一年里每周提供支持的 Michele Cronin；感谢负责本项目、帮助完善全书范围的 Nicole Butterfield；感谢制作团队，尤其是 Beth Kelly 和 Kristen Brown；感谢 Sonia Saruba 细致的文字编辑、Kate Dullea 让图表更加漂亮，也感谢 Susan Thompson 绘制封面上的美丽红毛猩猩。

最后，作者无限感谢深爱的妻子 Emmanuelle 和三个优秀的孩子 Alexandre、Rémi 与 Gabrielle。他们鼓励作者为本书努力工作；他们永不满足的好奇心弥足珍贵——向妻子和孩子解释书中最困难的一些概念，帮助作者澄清了自己的思路，也直接改善了书中的许多部分。此外，他们还不断送来饼干和咖啡。还有什么能比这更好呢?

目录

前言（Preface） i

第一部分：机器学习基础（Part I. The Fundamentals of Machine Learning） 1

第 1 章 机器学习全景（The Machine Learning Landscape） 2
1.1 什么是机器学习（What Is Machine Learning）？ 3
1.2 为什么要使用机器学习？（Why Use Machine Learning?） 3
1.3 机器学习的应用示例（Examples of Applications） 7
1.4 机器学习系统的类型（Types of Machine Learning Systems） 9
1.5 机器学习的主要挑战（The Main Challenges of Machine Learning） . . 28
1.6 测试与验证（Testing and Validating） 35
1.7 练习（Exercises） 38

第 2 章 端到端机器学习项目（End-to-End Machine Learning Project） 40
2.1 使用真实数据（Working with Real Data） 40
2.2 了解全局（Look at the Big Picture） 42
2.3 获取数据（Get the Data） 46
2.3.1 使用 Google Colab 运行代码示例（Running the Code Examples with Google Colab） 46
2.3.2 保存代码变更和数据（Saving Changes and Data） 49
2.3.3 交互性的力量与风险（The Power and Risk of Interactivity） . . 50
2.3.4 书中代码与 notebook 代码（Book Code and Notebook Code） 50
2.3.5 下载数据（Download the Data） 51
2.3.6 快速查看数据结构（Take a Quick Look at the Data Structure） 52
2.3.7 创建测试集（Create a Test Set） 56
2.4 探索数据并获得洞见（Discover and Visualize the Data to Gain Insights） 62
2.4.1 可视化地理数据（Visualizing Geographical Data） 62
2.4.2 查找相关性（Looking for Correlations） 65
2.4.3 试验属性组合（Experimenting with Attribute Combinations） . 69

2.5 为机器学习算法准备数据（Prepare the Data for Machine Learning Algorithms） . . . 70
2.5.1 清洗数据（Data Cleaning） . . . 71
2.5.2 Scikit-Learn 的设计（Scikit-Learn Design） . . . 72
2.5.3 处理文本和类别属性（Handling Text and Categorical Attributes） 74
2.5.4 特征缩放与变换（Feature Scaling and Transformation） . . . 78
2.5.5 自定义转换器（Custom Transformers） . . . 82
2.5.6 转换流水线（Transformation Pipelines） . . . 87
2.6 选择和训练模型（Select and Train a Model） . . . 93
2.6.1 在训练集上训练和评估（Train and Evaluate on the Training Set） 93
2.6.2 使用交叉验证获得更好的评估（Better Evaluation Using Cross-Validation） . . . 95
2.7 微调模型（Fine-Tune Your Model） . . . 98
2.7.1 网格搜索（Grid Search） . . . 98
2.7.2 随机搜索（Randomized Search） . . . 100
2.7.3 集成方法（Ensemble Methods） . . . 101
2.7.4 分析最佳模型及其错误（Analyze the Best Models and Their Errors） . . . 102
2.8 在测试集上评估系统（Evaluate the System on the Test Set） . . . 103
2.9 启动、监控和维护系统（Launch, Monitor, and Maintain Your System） 104
2.10 试试看！（Try It Out!） . . . 107
2.11 练习（Exercises） . . . 108

第 3 章 分类（Classification） 110
3.1 MNIST（The MNIST Dataset） . . . 110
3.2 训练二分类器（Training a Binary Classifier） . . . 114
3.3 性能指标（Performance Measures） . . . 115
3.3.1 使用交叉验证测量准确率（Measuring Accuracy Using Cross-Validation） . . . 115
3.3.2 混淆矩阵（The Confusion Matrix） . . . 116
3.3.3 精确率和召回率（Precision and Recall） . . . 119
3.3.4 精确率/召回率权衡（The Precision/Recall Trade-off） . . . 120
3.3.5 ROC 曲线（The ROC Curve） . . . 125
3.4 多分类（Multiclass Classification） . . . 129
3.5 误差分析（Error Analysis） . . . 132

3.6 多标签分类（Multilabel Classification） . . . 136
3.7 多输出分类（Multioutput Classification） . . . 138
3.8 练习（Exercises） . . . 141

第 4 章 训练模型（Training Models） 143
4.1 线性回归（Linear Regression） . . . 144
4.1.1 正规方程（The Normal Equation） . . . 146
4.1.2 计算复杂度（Computational Complexity） . . . 150
4.2 梯度下降（Gradient Descent） . . . 150
4.2.1 批量梯度下降（Batch Gradient Descent） . . . 154
4.2.2 随机梯度下降（Stochastic Gradient Descent） . . . 156
4.3 小批量梯度下降（Mini-Batch Gradient Descent） . . . 160
4.4 多项式回归（Polynomial Regression） . . . 162
4.5 学习曲线（Learning Curves） . . . 164
4.6 正则化线性模型（Regularized Linear Models） . . . 169
4.6.1 岭回归（Ridge Regression） . . . 169
4.6.2 Lasso 回归（Lasso Regression） . . . 171
4.6.3 弹性网络回归（Elastic Net Regression） . . . 174
4.6.4 提前停止（Early Stopping） . . . 175
4.7 逻辑回归（Logistic Regression） . . . 177
4.7.1 估计概率（Estimating Probabilities） . . . 177
4.7.2 训练与成本函数（Training and Cost Function） . . . 178
4.7.3 决策边界（Decision Boundaries） . . . 180
4.8 Softmax 回归（Softmax Regression） . . . 183
4.9 练习（Exercises） . . . 187

第 5 章 决策树（Decision Trees） 189
5.1 训练和可视化决策树（Training and Visualizing a Decision Tree） . . . 189
5.2 进行预测（Making Predictions） . . . 191
5.2.1 模型解释：白盒与黑盒（Model Interpretation: White Box versus Black Box） . . . 192
5.2.2 估计类别概率（Estimating Class Probabilities） . . . 193
5.2.3 CART 训练算法（The CART Training Algorithm） . . . 193
5.2.4 计算复杂度（Computational Complexity） . . . 194
5.2.5 基尼不纯度还是熵？（Gini Impurity or Entropy?） . . . 195

5.2.6 正则化超参数（Regularization Hyperparameters） 195
5.3 正则化决策树（Regularized Decision Trees） 197
5.4 回归（Regression） 198
5.5 对坐标轴方向的敏感性（Sensitivity to Axis Orientation） 200
5.6 决策树具有高方差（Decision Trees Have a High Variance） 202
5.7 练习（Exercises） 202

第 6 章 集成学习与随机森林（Ensemble Learning and Random Forests） 204
6.1 投票分类器（Voting Classifiers） 204
6.2 Bagging 与 Pasting（Bagging and Pasting） 208
6.2.1 Scikit-Learn 中的 Bagging 与 Pasting（Bagging and Pasting in Scikit-Learn） 210
6.2.2 袋外评估（Out-of-Bag Evaluation） 212
6.2.3 随机补丁和随机子空间（Random Patches and Random Subspaces） 213
6.3 随机森林（Random Forests） 213
6.3.1 极端随机树（Extra-Trees） 214
6.3.2 特征重要性（Feature Importance） 215
6.4 提升法（Boosting） 216
6.4.1 AdaBoost 217
6.5 梯度提升（Gradient Boosting） 220
6.5.1 基于直方图的梯度提升（Histogram-Based Gradient Boosting） 223
6.6 Stacking 225
6.7 练习（Exercises） 229

第 7 章 降维（Dimensionality Reduction） 232
7.1 维度灾难（The Curse of Dimensionality） 233
7.2 降维的主要方法（Main Approaches for Dimensionality Reduction） . . 234
7.3 PCA 239
7.4 随机投影（Random Projection） 248
7.5 LLE 251
7.6 其他降维技术（Other Dimensionality Reduction Techniques） 253
7.7 练习（Exercises） 254

第 8 章 无监督学习技术（Unsupervised Learning Techniques） 256

8.1 聚类算法：k-means 和 DBSCAN（Clustering Algorithms: k-means and DBSCAN） . . . 257
8.2 DBSCAN . . . 274
8.2.1 其他聚类算法（Other Clustering Algorithms） . . . 278
8.3 高斯混合（Gaussian Mixtures） . . . 279
8.3.1 使用高斯混合进行异常检测（Using Gaussian Mixtures for Anomaly Detection） . . . 283
8.4 选择簇数量（Selecting the Number of Clusters） . . . 285
8.4.1 似然函数（Likelihood Function） . . . 285
8.4.2 贝叶斯高斯混合模型（Bayesian Gaussian Mixture Models） . . . 287
8.5 其他异常检测与新颖性检测算法（Other Algorithms for Anomaly and Novelty Detection） . . . 288
8.6 练习（Exercises） . . . 289

第二部分：神经网络与深度学习（Part II. Neural Networks and Deep Learning） 291

第 9 章 人工神经网络简介（Introduction to Artificial Neural Networks） 292

9.1 从生物神经元到人工神经元（From Biological to Artificial Neurons） . 294
9.1.1 生物神经元（Biological Neurons） . . . 294
9.1.2 用神经元进行逻辑计算（Logical Computations with Neurons） 295
9.1.3 感知器（The Perceptron） . . . 296
9.1.4 多层感知器和反向传播（The Multilayer Perceptron and Back-propagation） . . . 301
9.2 使用 Scikit-Learn 构建和训练 MLP（Building and Training MLPs with Scikit-Learn） . . . 307
9.2.1 回归 MLP（Regression MLPs） . . . 307
9.2.2 分类 MLP（Classification MLPs） . . . 311
9.3 超参数调整指南（Hyperparameter Tuning Guidelines） . . . 316
9.3.1 隐藏层数量（Number of Hidden Layers） . . . 316
9.3.2 每个隐藏层的神经元数量（Number of Neurons per Hidden Layer） . . . 317
9.3.3 学习率（Learning Rate） . . . 318

9.3.4 批量大小（Batch Size） . . . 318
9.3.5 其他超参数（Other Hyperparameters） . . . 319
9.4 练习（Exercises） . . . 320

第 10 章 PyTorch 神经网络构建（Building Neural Networks with PyTorch） 323
10.1 PyTorch 基础（PyTorch Fundamentals） . . . 324
10.1.1 PyTorch 张量（PyTorch Tensors） . . . 324
10.1.2 硬件加速（Hardware Acceleration） . . . 327
10.1.3 Autograd . . . 329
10.2 实现线性回归（Implementing Linear Regression） . . . 333
10.2.1 使用张量和 Autograd 实现线性回归（Linear Regression Using Tensors and Autograd） . . . 333
10.2.2 使用 PyTorch 高级 API 实现线性回归（Linear Regression Using PyTorch's High-Level API） . . . 336
10.2.3 实现回归 MLP（Implementing a Regression MLP） . . . 339
10.2.4 使用 DataLoader 实现小批量梯度下降（Implementing Mini-Batch Gradient Descent Using DataLoaders） . . . 341
10.3 模型评估（Model Evaluation） . . . 343
10.4 使用自定义模块构建非顺序模型（Building Nonsequential Models Using Custom Modules） . . . 346
10.4.1 构建具有多个输入的模型（Building Models with Multiple Inputs） . . . 349
10.4.2 构建具有多个输出的模型（Building Models with Multiple Outputs） . . . 352
10.5 使用 PyTorch 构建图像分类器（Building an Image Classifier with PyTorch） . . . 355
10.5.1 使用 TorchVision 加载数据集（Using TorchVision to Load the Dataset） . . . 355
10.5.2 构建分类器（Building the Classifier） . . . 357
10.5.3 其他分类损失（Other Classification Losses） . . . 359
10.6 使用 Optuna 微调神经网络超参数（Fine-Tuning Neural Network Hyperparameters with Optuna） . . . 361
10.7 保存和加载 PyTorch 模型（Saving and Loading PyTorch Models） . . . 365
10.8 编译和优化 PyTorch 模型（Compiling and Optimizing a PyTorch Model） 367
10.9 练习（Exercises） . . . 370

第 11 章 训练深度神经网络（Training Deep Neural Networks） 372
11.1 梯度消失/爆炸问题（The Vanishing/Exploding Gradients Problems） . 373
11.1.1 Glorot 初始化与 He 初始化（Glorot Initialization and He Initialization） . . . 374
11.2 更好的激活函数（Better Activation Functions） . . . 376
11.2.1 Leaky ReLU . . . 377
11.2.2 ELU 和 SELU . . . 378
11.2.3 GELU、Swish、SwiGLU、Mish 和 ReLU2 . . . 380
11.3 批归一化（Batch Normalization） . . . 383
11.3.1 使用 PyTorch 实现批归一化（Implementing Batch Norm with PyTorch） . . . 385
11.3.2 BatchNorm1d、BatchNorm2d 和 BatchNorm3d . . . 387
11.3.3 层归一化（Layer Normalization） . . . 387
11.3.4 梯度裁剪（Gradient Clipping） . . . 388
11.4 重用预训练层（Reusing Pretrained Layers） . . . 389
11.4.1 使用 PyTorch 进行迁移学习（Transfer Learning with PyTorch） 391
11.5 无监督预训练（Unsupervised Pretraining） . . . 393
11.6 在辅助任务上预训练（Pretraining on an Auxiliary Task） . . . 395
11.7 更快的优化器（Faster Optimizers） . . . 395
11.7.1 动量（Momentum） . . . 396
11.7.2 Nesterov 加速梯度（Nesterov Accelerated Gradient） . . . 397
11.7.3 AdaGrad . . . 398
11.7.4 RMSProp . . . 400
11.7.5 Adam . . . 400
11.7.6 AdaMax、NAdam 和 AdamW . . . 401
11.8 学习率调度（Learning Rate Scheduling） . . . 403
11.8.1 指数调度（Exponential Scheduling） . . . 404
11.8.2 余弦退火（Cosine Annealing） . . . 405
11.8.3 基于性能的调度（Performance Scheduling） . . . 406
11.8.4 学习率预热（Warming Up the Learning Rate） . . . 407
11.8.5 带热重启的余弦退火（Cosine Annealing with Warm Restarts） 408
11.8.6 1cycle 调度（1cycle Scheduling） . . . 409
11.9 通过正则化避免过拟合（Avoiding Overfitting Through Regularization） 410
11.9.1 $\ell 1$ 和 $\ell 2$ 正则化（L1 and L2 Regularization） . . . 410

11.9.2 Dropout . . . 412
11.9.3 Monte Carlo Dropout . . . 415
11.9.4 Max-Norm 正则化（Max-Norm Regularization） . . . 418
11.10 实用指南（Practical Guidelines） . . . 419
11.11 练习（Exercises） . . . 420
11.11.1 参考文献 . . . 421

第 12 章 使用卷积神经网络进行深度计算机视觉（Deep Computer Vision Using Convolutional Neural Networks） 424

12.1 视觉皮层的结构（The Architecture of the Visual Cortex） . . . 424
12.2 卷积层（Convolutional Layers） . . . 426
12.2.1 滤波器（Filters） . . . 428
12.2.2 堆叠多个特征图（Stacking Multiple Feature Maps） . . . 429
12.2.3 使用 PyTorch 实现卷积层（Implementing Convolutional Layers with PyTorch） . . . 431
12.3 池化层（Pooling Layers） . . . 434
12.4 使用 PyTorch 实现池化层（Implementing Pooling Layers with PyTorch） 436
12.5 CNN 架构（CNN Architectures） . . . 439
12.5.1 LeNet-5 . . . 441
12.5.2 AlexNet . . . 442
12.5.3 数据增强（Data Augmentation） . . . 442
12.5.4 GoogLeNet . . . 443
12.5.5 ResNet . . . 446
12.5.6 Xception . . . 451
12.5.7 SENet . . . 452
12.5.8 其他值得注意的架构（Other Noteworthy Architectures） . . . 455
12.5.9 选择合适的 CNN 架构（Choosing the Right CNN Architecture） 457
12.5.10 GPU 内存需求：推理与训练（GPU RAM Requirements: Inference Versus Training） . . . 457
12.5.11 可逆残差网络（RevNets）（Reversible Residual Networks） . . . 459
12.6 使用 PyTorch 实现 ResNet-34 CNN（Implementing a ResNet-34 CNN Using PyTorch） . . . 460
12.7 使用 TorchVision 的预训练模型（Using TorchVision's Pretrained Models） . . . 462
12.8 用于迁移学习的预训练模型（Pretrained Models for Transfer Learning） 464

12.9 分类与定位（Classification and Localization） . . . 467
12.10 目标检测（Object Detection） . . . 470
12.10.1 全卷积网络（Fully Convolutional Networks） . . . 472
12.10.2 YOLO（You Only Look Once） . . . 474
12.10.3 平均精度均值（Mean Average Precision） . . . 474
12.11 目标跟踪（Object Tracking） . . . 476
12.12 语义分割（Semantic Segmentation） . . . 477
12.12.1 其他 PyTorch 卷积层（Other PyTorch Convolutional Layers） . 479
12.13 练习（Exercises） . . . 481
12.13.1 参考文献 . . . 482

第 13 章 序列处理：RNN 与 CNN（Processing Sequences Using RNNs and CNNs） 485

13.1 循环神经元和循环层（Recurrent Neurons and Layers） . . . 485
13.2 记忆单元（Memory Cells） . . . 488
13.3 输入和输出序列（Input and Output Sequences） . . . 489
13.4 训练 RNN（Training RNNs） . . . 490
13.5 预测时间序列（Forecasting a Time Series） . . . 491
13.6 ARMA 模型系列（The ARMA Model Family） . . . 497
13.7 SARIMA 模型 . . . 498
13.8 为机器学习模型准备数据（Preparing the Data for Machine Learning Models） . . . 499
13.9 使用线性模型进行预测（Forecasting Using a Linear Model） . . . 501
13.10 使用简单 RNN 进行预测（Forecasting Using a Simple RNN） . . . 502
13.11 使用深层 RNN 进行预测（Forecasting Using a Deep RNN） . . . 505
13.12 预测多变量时间序列（Forecasting Multivariate Time Series） . . . 505
13.13 预测未来多个时间步（Forecasting Several Time Steps Ahead） . . . 507
13.14 使用序列到序列模型进行预测（Forecasting Using a Sequence-to-Sequence Model） . . . 509
13.15 处理长序列（Handling Long Sequences） . . . 512
13.16 对抗不稳定梯度问题（Fighting the Unstable Gradients Problem） . . 512
13.17 应对短期记忆问题（Tackling the Short-Term Memory Problem） . . 513
13.18 LSTM 单元（LSTM cells） . . . 514
13.19 GRU 单元（GRU cells） . . . 516

13.20 使用一维卷积层处理序列（Using 1D convolutional layers to process sequences） . . . 518
13.21 WaveNet . . . 519
13.22 练习（Exercises） . . . 522
13.22.1 参考文献 . . . 523

第 14 章 NLP：RNN 与注意力（Natural Language Processing with RNNs and Attention） 525

14.1 使用字符 RNN 生成莎士比亚文本（Generating Shakespearean Text Using a Character RNN） . . . 526
14.1.1 创建训练数据集（Creating the Training Dataset） . . . 526
14.2 嵌入（Embeddings） . . . 529
14.3 构建和训练字符 RNN 模型（Building and Training the Char-RNN Model） . . . 532
14.3.1 生成伪莎士比亚文本（Generating Fake Shakespearean Text） . 533
14.4 使用 Hugging Face 库进行情感分析（Sentiment Analysis Using Hugging Face Libraries） . . . 535
14.5 使用 Hugging Face Tokenizers 库进行分词（Tokenization Using the Hugging Face Tokenizers Library） . . . 536
14.6 复用预训练 tokenizer（Reusing Pretrained Tokenizers） . . . 539
14.7 构建和训练情感分析模型（Building and Training a Sentiment Analysis Model） . . . 541
14.8 双向 RNN（Bidirectional RNNs） . . . 544
14.9 复用预训练嵌入和语言模型(Reusing Pretrained Embeddings and Language Models） . . . 545
14.10 任务专用类（Task-Specific Classes） . . . 548
14.11 Trainer API . . . 549
14.12 Hugging Face Pipelines . . . 550
14.13 用于神经机器翻译的编码器-解码器网络（An Encoder-Decoder Network for Neural Machine Translation） . . . 552
14.13.1 模型优化（Model Optimizations） . . . 558
14.14 Beam Search . . . 558
14.15 注意力机制（Attention Mechanisms） . . . 559
14.16 练习（Exercises） . . . 563
14.17 参考文献（References） . . . 563

第 15 章 Transformer：自然语言处理与聊天机器人（Transformers for Natural Language Processing and Chatbots） 566

15.1 注意力就是一切：原始 Transformer 架构（Attention Is All You Need: The Original Transformer Architecture） 567

15.1.1 位置编码（Positional Encodings） 569

15.1.2 多头注意力（Multi-Head Attention） 569

15.1.3 构建 Transformer 的其余部分（Building the Rest of the Transformer） 571

15.1.4 构建英语到西班牙语的 Transformer（Building an English-to-Spanish Transformer） 572

15.2 用于自然语言理解的仅编码器 Transformer（Encoder-Only Transformers for Natural Language Understanding） 574

15.2.1 BERT 的架构（BERT's Architecture） 574

15.2.2 BERT 预训练（BERT Pretraining） 574

15.2.3 使用 Transformers 训练 BERT（Training BERT with Transformers） 575

15.2.4 BERT 微调（BERT Fine-Tuning） 576

15.2.5 其他仅编码器模型（Other Encoder-Only Models） 578

15.3 仅解码器 Transformer（Decoder-Only Transformers） 581

15.3.1 GPT-1 架构和生成式预训练（GPT-1 Architecture and Generative Pretraining） 582

15.3.2 GPT-2 与零样本学习（GPT-2 and Zero-Shot Learning） 583

15.3.3 GPT-3、上下文学习、单样本学习和少样本学习（GPT-3, In-Context Learning, One-Shot Learning, and Few-Shot Learning） 583

15.3.4 使用 GPT-2 生成文本（Using GPT-2 to Generate Text） 583

15.3.5 使用 GPT-2 回答问题（Using GPT-2 for Question Answering） 585

15.3.6 下载并运行更大的模型：Mistral-7B（Downloading and Running a Larger Model: Mistral-7B） 585

15.4 把大语言模型变成聊天机器人（Turning a Large Language Model into a Chatbot） 588

15.5 提示工程（Prompt Engineering） 590

15.6 使用 SFT 和 RLHF 微调聊天模型（Fine-Tuning a Model for Chatting and Following Instructions Using SFT and RLHF） 591

15.6.1 监督微调（Supervised Fine-Tuning，SFT） 591

15.6.2 人类反馈微调：RLHF 和 DPO 592
15.6.3 使用 TRL 库进行微调（Fine-Tuning a Model Using the TRL Library） 593
15.7 从聊天模型到完整聊天机器人系统（From a Chatbot Model to a Full Chatbot System） 595
15.7.1 计算器（Calculator） 595
15.7.2 Web 搜索（Web Search） 595
15.7.3 检索增强生成（Retrieval-Augmented Generation，RAG） . . . 596
15.7.4 记忆与持久化上下文（Memory） 596
15.7.5 智能体行为（Agentic Behavior） 596
15.7.6 其他工具（Other Tools） 596
15.8 模型上下文协议（Model Context Protocol） 596
15.8.1 结构化生成（Structured Generation） 597
15.9 聊天机器人库和工具（Libraries and Tools） 597
15.10 编码器-解码器模型（Encoder-Decoder Models） 598
15.11 练习（Exercises） 599
15.12 参考文献（References） 599

第 16 章 视觉与多模态 Transformer（Vision and Multimodal Transformers） 603
16.1 视觉 Transformer（Vision Transformers） 604
16.1.1 具有视觉注意力的 RNN（RNNs with Visual Attention） 604
16.1.2 DETR：用于目标检测的 CNN-Transformer 混合模型（DETR: A CNN-Transformer Hybrid for Object Detection） 605
16.1.3 原始 ViT（The Original ViT） 605
16.1.4 从头实现 ViT（Implementing a ViT from Scratch Using PyTorch） 606
16.1.5 使用 Transformers 微调预训练 ViT（Fine-Tuning a Pretrained ViT Using the Transformers Library） 608
16.1.6 数据高效图像 Transformer（Data-Efficient Image Transformer, DeiT） 610
16.1.7 用于密集预测的金字塔视觉 Transformer（Pyramid Vision Transformer for Dense Prediction Tasks） 611
16.1.8 Swin Transformer：快速且通用的 ViT（The Swin Transformer: A Fast and Versatile ViT） 612
16.1.9 DINO：自监督视觉表示学习（DINO: Self-Supervised Visual Representation Learning） 613

16.1.10 其他主要视觉模型和技术（Other Major Vision Models and Techniques） . . . 615
16.2 多模态 Transformer（Multimodal Transformers） . . . 616
16.2.1 VideoBERT：文本加视频的 BERT 变体（VideoBERT: A BERT Variant for Text plus Video） . . . 617
16.2.2 ViL BERT：文本加图像的双流 Transformer（ViL BERT: A Dual-Stream Transformer for Text plus Image） . . . 618
16.2.3 CLIP：通过对比学习训练的图像-文本双编码器（CLIP: A Dual-Encoder Text plus Image Model Trained with Contrastive Pre-training） . . . 620
16.2.4 DALL·E：从文本生成图像（DALL·E: Generating Images from Text Prompts） . . . 623
16.2.5 Perceiver：用潜在空间连接高分辨率模态（Perceiver: Bridging High-Resolution Modalities with Latent Spaces） . . . 623
16.2.6 Perceiver IO：灵活的输出机制（Perceiver IO: A Flexible Output Mechanism） . . . 625
16.2.7 Flamingo：开放式视觉对话（Flamingo: Open-Ended Visual Dialogue） . . . 626
16.2.8 BLIP 和 BLIP-2（BLIP and BLIP-2） . . . 627
16.2.9 其他多模态模型（Other Multimodal Models） . . . 630
16.3 练习（Exercises） . . . 631
16.4 参考文献（References） . . . 632

第 17 章 加速 Transformer（Speeding Up Transformers） 637
17.1 推理时加速解码（Faster Decoding at Inference Time） . . . 637
17.1.1 键/值缓存（Key/Value Caching） . . . 638
17.1.2 推测解码（Speculative Decoding） . . . 640
17.1.3 并行化解码的主要方法（Main Approaches to Parallelize Decoding） . . . 643
17.2 加速多头注意力（Speeding Up Multi-Head Attention） . . . 644
17.2.1 稀疏注意力（Sparse Attention） . . . 644
17.2.2 近似注意力（Approximate Attention） . . . 649
17.2.3 在注意力头之间共享投影（Sharing Projections Across Attention Heads） . . . 659

17.2.4 FlashAttention：快速且精确的注意力实现（FlashAttention: A Fast and Accurate Attention Implementation） 663
17.3 使用专家混合扩展（Scaling Up with Mixture of Experts） 666
17.3.1 专家混合层（The Mixture of Experts Layer） 666
17.3.2 MoE 的优势（Benefits of MoE） 668
17.3.3 MoE 的挑战（The Challenges of MoEs） 668
17.4 更快地训练（Faster Training） 671
17.4.1 模型权重（Model Weights） 671
17.4.2 激活值（Activations） 671
17.4.3 梯度（Gradients） 671
17.4.4 优化器状态（Optimizer States） 671
17.4.5 参数高效微调（Parameter-Efficient Fine-Tuning，PEFT） . . . 672
17.4.6 激活检查点（Activation Checkpointing） 674
17.4.7 序列打包与分桶（Sequence Packing and Bucketing） 675
17.4.8 梯度累积（Gradient Accumulation） 676
17.5 使用并行化加速训练（Speeding Up Training Using Parallelism） 677
17.5.1 数据并行（Data Parallelism） 677
17.5.2 模型并行（Model Parallelism） 680
17.6 练习（Exercises） 683
17.7 原书超链接资源 683

第 18 章 自编码器、GAN 和扩散模型（Autoencoders, GANs, and Diffusion Models） 686
18.1 高效的数据表示（Efficient Data Representations） 687
18.1.1 使用欠完备线性自编码器执行 PCA（Performing PCA with an Undercomplete Linear Autoencoder） 689
18.2 堆叠自编码器（Stacked Autoencoders） 690
18.2.1 使用 PyTorch 实现堆叠自编码器（Implementing a Stacked Autoencoder Using PyTorch） 691
18.2.2 可视化重建结果（Visualizing the Reconstructions） 692
18.2.3 使用自编码器进行异常检测（Anomaly Detection Using Autoencoders） 693
18.2.4 可视化 Fashion MNIST 数据集（Visualizing the Fashion MNIST Dataset） 693

18.2.5 使用堆叠自编码器进行无监督预训练（Unsupervised Pretraining Using Stacked Autoencoders） . . . 695
18.2.6 权重绑定（Tying Weights） . . . 696
18.2.7 一次训练一个自编码器（Training One Autoencoder at a Time） 696
18.3 卷积自编码器（Convolutional Autoencoders） . . . 697
18.4 去噪自编码器（Denoising Autoencoders） . . . 698
18.5 稀疏自编码器（Sparse Autoencoders） . . . 700
18.6 变分自编码器（Variational Autoencoders） . . . 702
18.6.1 生成 Fashion MNIST 图像（Generating Fashion MNIST Images） 705
18.7 离散变分自编码器（Discrete Variational Autoencoders） . . . 707
18.8 生成式对抗网络（Generative Adversarial Networks） . . . 709
18.8.1 第一阶段：训练判别器 . . . 711
18.8.2 第二阶段：训练生成器 . . . 711
18.9 GAN 训练的困难（The Difficulties of Training GANs） . . . 713
18.10 扩散模型（Diffusion Models） . . . 714
18.10.1 方差调度（Variance Schedule） . . . 716
18.10.2 DDPM 模型 . . . 717
18.10.3 DDPM 与 DDIM 采样 . . . 718
18.11 潜在扩散模型（Latent Diffusion Models） . . . 720
18.12 练习（Exercises） . . . 721
18.13 原书超链接资源 . . . 722
18.14 参考文献说明 . . . 723

第 19 章 强化学习（Reinforcement Learning） 725
19.1 什么是强化学习？ . . . 725
19.2 策略梯度（Policy Gradients） . . . 727
19.3 Gymnasium 库简介（Introduction to the Gymnasium Library） . . . 729
19.4 神经网络策略（Neural Network Policies） . . . 732
19.5 评估动作：信用分配问题（Evaluating Actions: The Credit Assignment Problem） . . . 735
19.6 使用策略梯度解决 CartPole（Solving the CartPole Using Policy Gradients） . . . 736
19.7 基于价值的方法（Value-Based Methods） . . . 738
19.7.1 马尔可夫决策过程（Markov Decision Processes） . . . 739
19.7.2 时序差分学习（Temporal-Difference Learning） . . . 743

19.7.3 Q 学习（Q-Learning） . . . 744
19.7.4 探索策略（Exploration Policies） . . . 745
19.7.5 近似 Q 学习与深度 Q 学习（Approximate Q-Learning and Deep Q-Learning） . . . 746
19.7.6 实现深度 Q 学习（Implementing Deep Q-Learning） . . . 746
19.8 DQN 的改进（DQN Improvements） . . . 751
19.9 演员-评论家算法（Actor-Critic Algorithms） . . . 753
19.9.1 异步优势演员-评论家（A3C） . . . 755
19.9.2 优势演员-评论家（A2C） . . . 756
19.9.3 软演员-评论家（SAC） . . . 756
19.9.4 近端策略优化（PPO） . . . 756
19.10 使用 Stable-Baselines3 的 PPO 实现玩 Atari Breakout（Mastering Atari Breakout Using the Stable-Baselines3 PPO Implementation） . . . 756
19.11 一些流行的 RL 算法（Overview of Some Popular RL Algorithms） . . . 760
19.11.1 AlphaGo . . . 760
19.11.2 基于好奇心的探索（Curiosity-Based Exploration） . . . 761
19.11.3 开放式学习（Open-Ended Learning，OEL） . . . 761
19.12 总结 . . . 762
19.13 练习（Exercises） . . . 762
19.14 致谢（Thank You!） . . . 763
19.15 原书超链接资源 . . . 763
19.16 章节注释 . . . 764

附录 A：自动微分（Automatic Differentiation） 767

附录 B：混合精度与量化（Mixed Precision and Quantization） 774

术语索引（Index） 784

作者介绍（About the Author） 793

版本说明（Colophon） 794

原书超链接对照表（Original Link Catalog） 795

第一部分：机器学习基础

Part I. The Fundamentals of Machine Learning

第 1 章　机器学习全景

The Machine Learning Landscape

不久以前，如果你拿起手机让它告诉你回家的路，它会置之不理–而旁人可能会怀疑你的精神状态。但机器学习（machine learning，ML）早已不是科幻：每天有数十亿人使用它。事实上，几十年来，机器学习一直应用于某些专业领域，例如光学字符识别（optical character recognition，OCR）。

真正走入主流、改善了数亿人生活的第一个机器学习应用，在 20 世纪 90 年代悄然接管了世界：垃圾邮件过滤器。它当然不是什么具有自我意识的机器人，但从技术定义上看确实属于机器学习：它已经学得如此出色，以至于你几乎不再需要把邮件标记为垃圾邮件。

随后，大数据、硬件升级和一些算法创新相继出现，数百种机器学习应用随之而来，如今它们默默驱动着你经常使用的数百种产品和功能：语音提示、自动翻译、图像搜索、产品推荐，以及更多功能。最后，ChatGPT、Gemini（前身为 Bard）、Claude、Perplexity 等众多聊天机器人出现了：人工智能不再只是后台为服务提供动力，它本身就是服务。

机器学习从哪里开始，又在哪里结束？机器学会某件事究竟意味着什么？如果我下载了维基百科全部条目的副本，我的计算机真的学会了什么吗？它是否突然变得更聪明了？本章将先澄清机器学习（machine learning）是什么，以及你为什么可能想要使用它。

接着，在我们踏上探索机器学习大陆的旅程之前，我们会先看一看地图，了解其中的主要区域和最显著的地标：监督学习（supervised learning）与无监督学习（unsupervised learning）及其变体、在线学习（online learning）与批量学习（batch learning）、基于实例的学习（instance-based learning）与基于模型的学习（model-based learning）。然后，我们将了解典型机器学习项目的工作流程，讨论你可能遇到的主要挑战，并介绍如何评估和微调机器学习系统。

本章介绍许多每位数据科学家都应该熟记于心的基础概念（以及术语）。本章将进行高层次概览（这是全书代码最少的一章），内容都比较简单；但我的目标是确保在继续阅读全书之前，你已经对这些内容有了清晰透彻的理解。所以，拿杯咖啡，让我们开始吧！

> 提示：如果你已经熟悉机器学习基础知识，可以直接跳到第 2 章。如果你不确定自己是否掌握了这些知识，请在继续阅读之前，尝试回答本章末尾列出的所有问题。

1.1　什么是机器学习（What Is Machine Learning）？

机器学习是让计算机能够从数据中学习的编程科学（也是一门艺术）。

下面是一个略微更通用的定义：

> 【机器学习】是一个研究领域，它赋予计算机无需明确编程即可学习的能力。——Arthur Samuel，1959 年

下面是一个更偏工程的定义：

> 如果一个计算机程序能够针对某项任务 T 和某项性能度量 P，从经验 E 中学习，那么当且仅当它在任务 T 上的性能（由 P 衡量）随着经验 E 的积累而提高时，我们就称该程序学会了。——Tom Mitchell，1997 年

你的垃圾邮件过滤器（spam filter）是一个机器学习程序：它通过用户标记的垃圾邮件示例和普通邮件（非垃圾邮件，也称为“ham”）的示例，学会标记垃圾邮件。系统用来学习的示例称为训练集（training set）。每个训练示例称为训练实例（training instance，或样本）。机器学习系统中负责学习并进行预测的部分称为模型（model）。神经网络（neural network）和随机森林（random forest）都是模型的例子。

在这个例子中，任务 T 是为新邮件标记垃圾邮件，经验 E 是训练数据，而性能度量 P 需要具体定义；例如，可以使用被正确分类的邮件所占比例。这种特定的性能度量称为准确率（accuracy），常用于分类任务（第 3 章将讨论其他几种性能度量）。

如果你只是下载了维基百科所有条目的副本，你的计算机确实拥有了更多数据，但它并没有因此在任何任务上变得更好。这不属于机器学习。

1.2　为什么要使用机器学习？（Why Use Machine Learning?）

想一想，如果使用传统编程技术编写垃圾邮件过滤器，你会怎么做（图 1-1）：

1. 首先研究垃圾邮件通常是什么样子。你可能会注意到，某些单词或短语（例如“4U”、“credit card”、“free”和“amazing”）经常出现在主题行中。你也许还会在发件人姓名、邮件正文和邮件的其他部分发现一些模式。
2. 对你发现的每种模式分别编写检测算法；如果检测到其中若干模式，程序就把邮件标记为垃圾邮件。
3. 测试程序，反复执行步骤 1 和 2，直到它足够好，可以上线。

由于问题很难，你的程序很可能会变成一长串复杂规则–非常难维护。

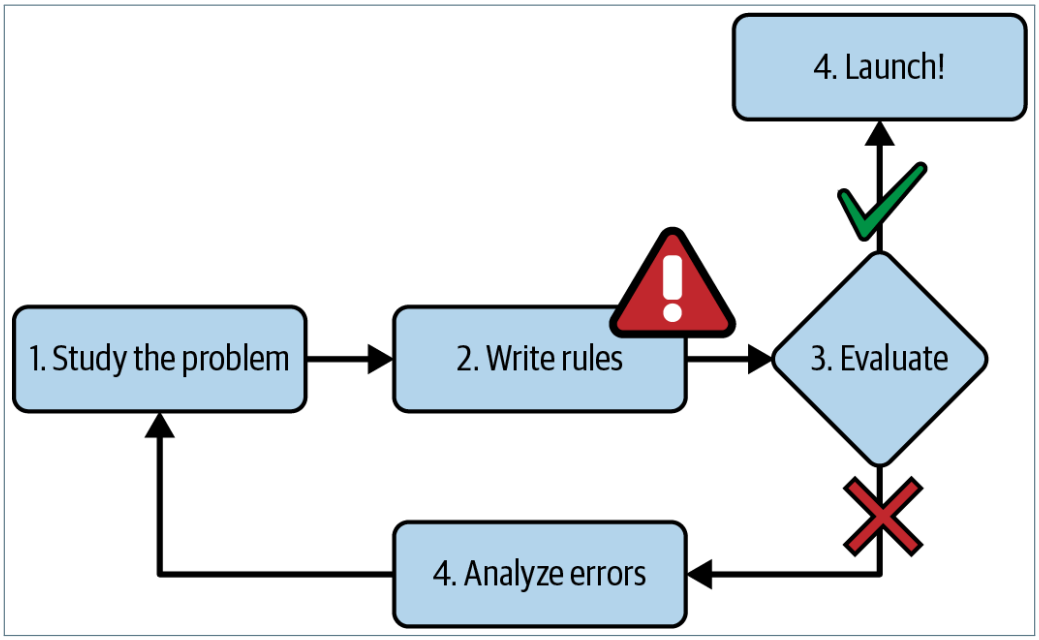

图 1-1: 传统方法（The traditional approach）

相比之下，基于机器学习技术的垃圾邮件过滤器会通过比较垃圾邮件示例和普通邮件示例中词语的异常频率，自动学会哪些单词和短语能够很好地预测垃圾邮件（图 1-2）。这样的程序短得多，也更易维护，而且很可能更准确。

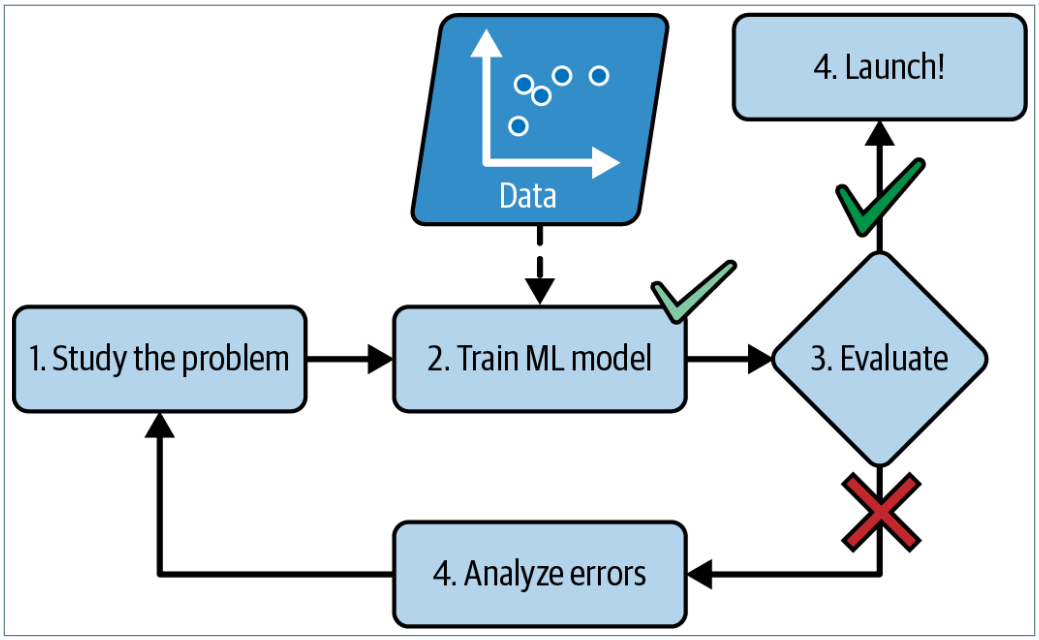

图 1-2: 机器学习方法（The machine learning approach）

如果垃圾邮件发送者注意到所有包含“4U”的邮件都会被拦截，他们可能会改用“For U”。采用传统编程技术的垃圾邮件过滤器需要更新规则，才能识别包含“For U”的邮件。如果垃圾邮件发送者不断绕过你的过滤器，你就得永远不断编写新规则。

相比之下，基于机器学习技术的垃圾邮件过滤器会自动注意到：在用户标记的垃圾邮件中，“For U”的出现频率异常升高，于是它会在无需人工干预的情况下开始将这类邮件标记为垃圾邮件（图 1-3）。

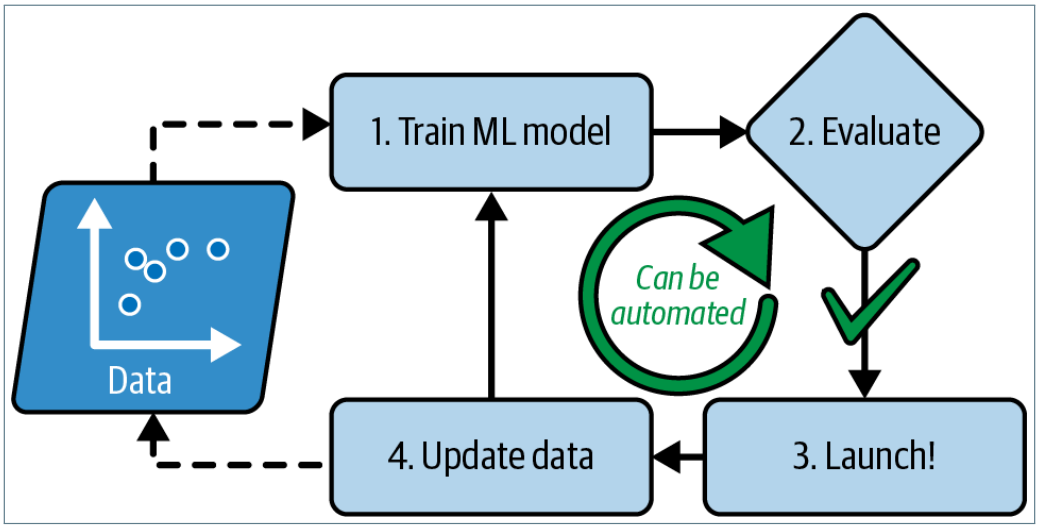

图 1-3: 自动适应变化（Automatically adapting to change）

机器学习大显身手的另一个领域，是那些对传统方法来说过于复杂，或者根本不存在已知算法的问题。以语音识别为例。假设你想从简单做起，编写一个能够区分单词“one”和“two”的程序。你可能会注意到，“two”以高音开头（“T”），于是可以硬编码一个算法来测量高音的强度，以此区分 one 和 two–但显然，这种技术无法扩展到数千个由数百万不同的人在嘈杂环境中、使用数十种语言说出的单词。

最好的解决方案（至少目前如此）是编写一种能够自行学习的算法，为每个单词提供大量示例录音。

最后，机器学习还能帮助人类学习（图 1-4）。我们可以检查 ML 模型，看看它们学到了什么（尽管对某些模型来说，这件事可能很棘手）。例如，垃圾邮件过滤器用足够多的垃圾邮件训练后，我们可以轻松检查它，找出它认为最能预测垃圾邮件的词语及其组合。有时，这会揭示出人们未曾察觉的相关性或新趋势，从而帮助我们更好地理解问题。从大量数据中挖掘隐藏模式的过程称为数据挖掘（data mining），而机器学习非常擅长这件事。

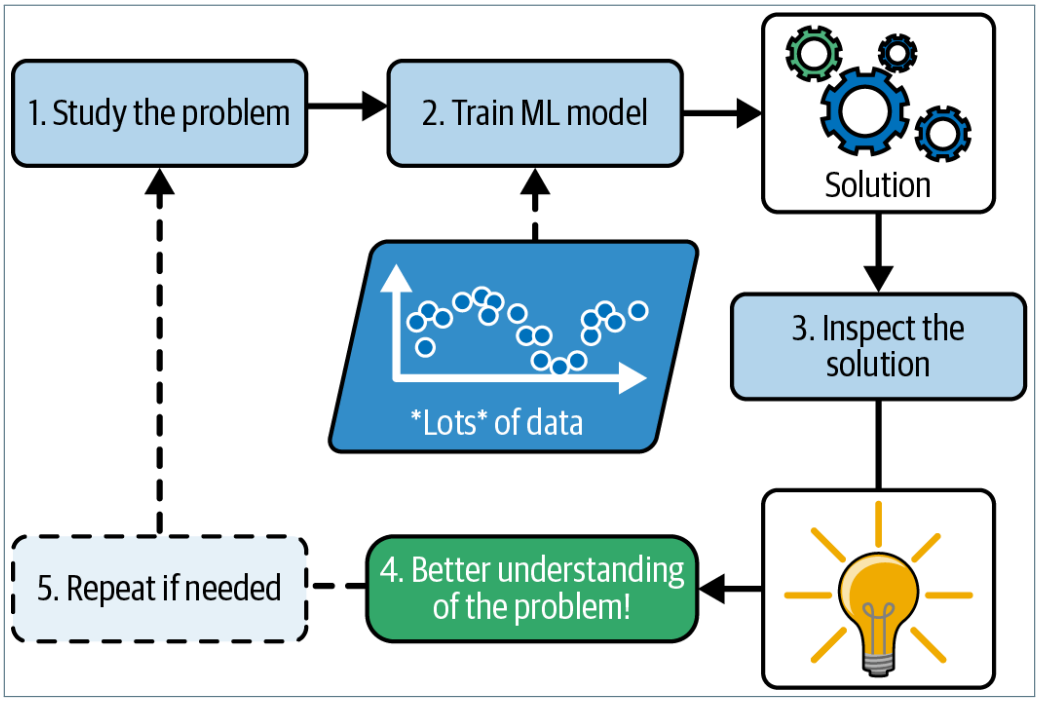

图 1-4: 机器学习可以帮助人类学习（Machine learning can help humans learn）

概括来说，机器学习特别适合以下问题：

- 现有解决方案需要大量工作和维护的问题，例如包含冗长规则列表的问题（机器学习模型通常可以简化代码，而且性能优于传统方法）。
- 使用传统方法会得不到好解决方案的复杂问题（最先进的机器学习技术或许能够找到解决方案）。
- 环境不断变化的问题（机器学习系统可以利用新数据轻松重新训练，从而始终保持最新）。
- 需要从复杂问题和大量数据中获取洞察的问题。

1.3　机器学习的应用示例（Examples of Applications）

下面来看一些具体的机器学习任务，以及能够解决它们的技术。

分析生产线上的产品图像并自动分类。这是图像分类（image classification），通常使用卷积神经网络（convolutional neural network，CNN；参见第 12 章）或视觉 Transformer（vision transformer；参见第 16 章）完成。

检测脑部扫描中的肿瘤。这是语义图像分割（semantic image segmentation）：图像中的每个像素都会被分类，因为我们需要确定肿瘤的确切位置和形状。这类任务通常使用 CNN 或视觉 Transformer。

自动分类新闻文章。这是自然语言处理（natural language processing，NLP），更具体地说是文本分类（text classification）。可以使用循环神经网络（recurrent neural network，RNN）和 CNN 处理，但 Transformer 的效果更好（参见第 15 章）。

自动标记讨论论坛中的冒犯性评论。这同样属于文本分类，可以使用相同的 NLP 工具。

自动总结长篇文档。这是 NLP 的一个分支，称为文本摘要（text summarization），同样可以使用上述工具。

通过分析很长的 DNA 序列，估计某人患某种疾病的遗传风险。这类任务需要在很长的序列中发现分散的模式，这正是状态空间模型（state-space model，SSM）特别擅长的领域（参见 https://homl.info 上的“State-Space Models (SSMs)”）。

创建聊天机器人或个人助理。这需要许多 NLP 组件，包括自然语言理解（natural language understanding，NLU）模块和问答模块。

根据多项绩效指标预测公司明年的收入。这是一个回归（regression）任务，也就是预测数值。可以使用任何回归模型来处理，例如线性回归（linear regression）或多项式回归模型（参见第 4 章）、回归支持向量机（support vector machine，SVM；参见 https://homl.info 上关于 SVM 的在线附录）、回归随机森林（参见第 6 章）或人工神经网络（artificial neural network，ANN；参见第 9 章）。如果还要考虑过去绩效指标的序列，可以使用 RNN、CNN 或 Transformer（参见第 13 至 15 章）。

让应用响应语音指令。这是语音识别（speech recognition），需要处理音频样本。由于音频样本是长而复杂的序列，通常使用 RNN、CNN 或 Transformer 处理（参见第 13 至 15 章）。

检测信用卡欺诈。这是异常检测（anomaly detection），可以使用孤立森林（isolation forest）、高斯混合模型（Gaussian mixture model；参见第 8 章）或自编码器（autoencoder；参见第 18 章）处理。

根据客户的购买行为对客户分组。这是聚类（clustering），可以使用 k-means、DBSCAN 等方法完成（参见第 8 章）。

将复杂的高维数据集表示为清晰且有洞察力的图表。这是数据可视化（data visualization），通常会用到降维（dimensionality reduction）技术（参见第 7 章）。

根据客户过去的购买记录推荐其可能感兴趣的产品。这是推荐系统（recommender system）。一种方法是将客户过去的购买记录（以及客户的其他信息）输入人工神经网络（artificial neural network，ANN；参见第 9 章），让它输出最可能的下一次购买行为。通常需要使用所有客户的历史购买序列来训练这个神经网络。

为游戏构建智能机器人。这通常使用强化学习（reinforcement learning，RL；参见第 19 章）处理。强化学习是机器学习的一个分支，它训练智能体（agent，例如游戏机器人）在给定环境（例如游戏）中选择能够让奖励随时间最大化的动作。著名的 AlphaGo 程序曾使用强化学习击败围棋世界冠军。

这个列表还可以继续列下去，但希望这些例子能让你感受到机器学习能够处理的任务范围有多么广泛、复杂，以及针对不同任务可以使用哪些类型的技术。

1.4　机器学习系统的类型（Types of Machine Learning Systems）

机器学习系统种类繁多，因此有必要根据以下标准将它们划分为几个宽泛的类别：

- 训练时如何获得引导（监督学习、无监督学习、半监督学习、自监督学习等）。
- 能否在运行过程中持续增量学习（在线学习与批量学习）。
- 是简单地将新数据点与已知数据点进行比较，还是像科学家那样检测训练数据中的模式并建立预测模型（基于实例的学习与基于模型的学习）。

这些标准并不互斥，你可以按照任意方式将它们组合起来。例如，一个先进的垃圾邮件过滤器可能使用深度神经网络模型在线学习，而模型又是利用人工提供的垃圾邮件和普通邮件示例训练的；这使它成为一个在线、基于模型的监督学习系统。

让我们更仔细地看看这些标准。

训练监督　机器学习系统可以根据训练时获得的监督数量和类型进行分类。监督的类别很多，但我们将讨论其中主要的几类：监督学习（supervised learning）、无监督学习（unsupervised learning）、自监督学习（self-supervised learning）、半监督学习（semi-supervised learning）和强化学习（reinforcement learning）。

监督学习　在监督学习中，你提供给算法的训练集包含期望的解决方案，这些解决方案称为标签（label；图 1-5）。

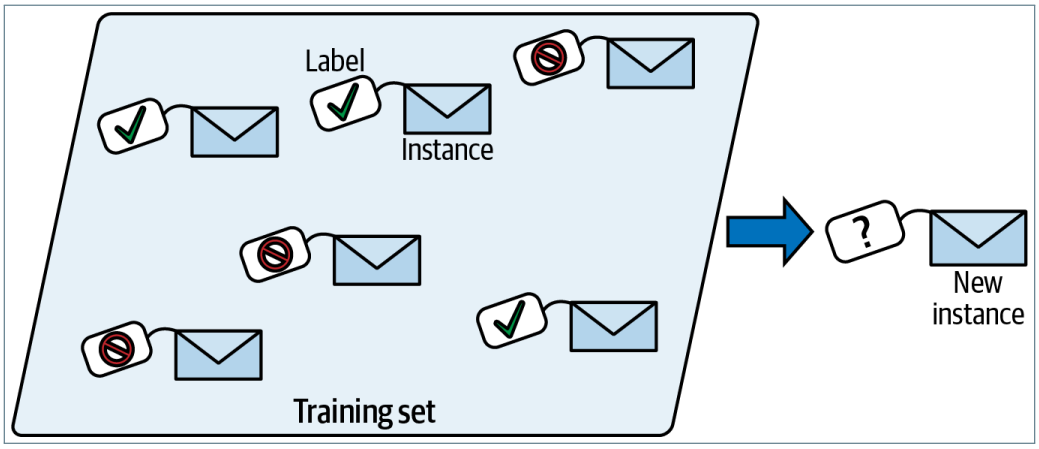

图 1-5: 用于垃圾邮件分类的带标签训练集（A labeled training set for spam classification；监督学习示例）

一个典型的监督学习任务是分类（classification）。垃圾邮件过滤器就是一个很好的例子：它使用大量带有类别（垃圾邮件或普通邮件）的示例邮件进行训练，并且必须学会对新邮件进行分类。

另一个典型任务是：给定一组特征（里程、车龄、品牌等），预测一个目标数值，例如汽车价格。这类任务称为回归（regression；图 1-6）。[1]要训练这样的系统，你需要向它提供许多汽车示例，其中既包括特征，也包括目标值（即价格）。

需要注意的是，一些回归模型也能用于分类，反之亦然。例如，逻辑回归（logistic regression）通常用于分类，因为它可以输出一个数值，表示样本属于某个类别的概率（例如，有 20% 的概率是垃圾邮件）。

[1]趣闻：这个听起来有些奇怪的名称来自统计学家 Francis Galton。他在研究高个子父母的孩子往往比父母矮这一现象时，将其称为“向均值回归”（regression to the mean）；后来，这一名称也被用于他分析变量之间相关性的方法。

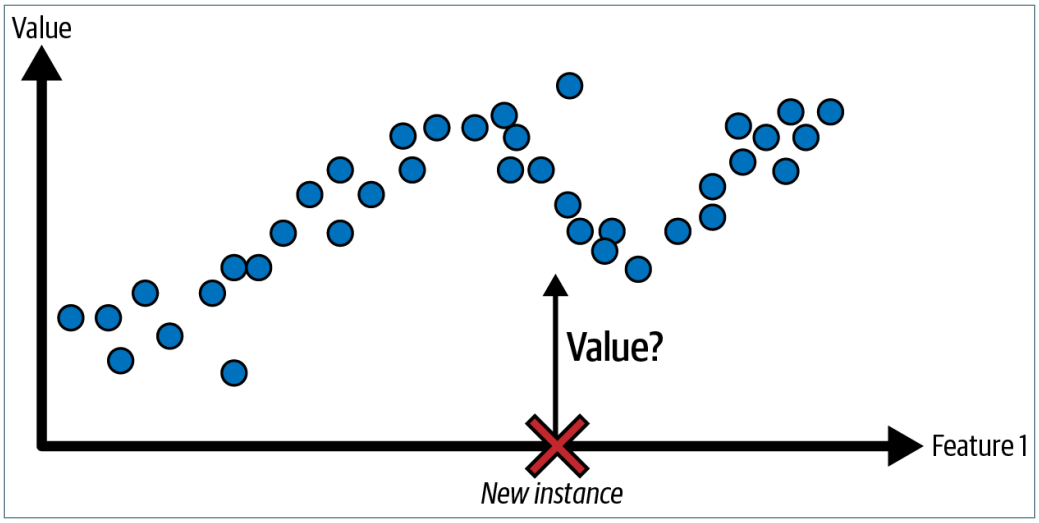

图 1-6: 回归问题：给定一个输入特征预测一个值（通常会有多个输入特征，有时也有多个输出值）（A regression problem）

> 说明：在监督学习中，目标（target）和标签（label）通常被视为同义词，不过 target 更常用于回归任务，label 更常用于分类任务。此外，特征（feature）有时也称为预测变量（predictor）或属性（attribute）。这些词既可以指单个样本中的值（例如“这辆车的里程特征为 15,000”），也可以指所有样本中的同一特征（例如“里程特征与价格高度相关”）。

无监督学习　顾名思义，在无监督学习（unsupervised learning）中，训练数据没有标签，系统尝试在没有老师指导的情况下学习。

例如，假设你拥有大量博客访问者数据。你可能希望运行聚类算法，尝试找出相似访问者组成的群组（图 1-7）。特征可能包括用户的年龄段、所在地区、兴趣、会话时长等。你从不告诉算法某位访问者属于哪个群组；它会自行发现这些联系。例如，它可能注意到 40% 的访问者是喜欢漫画、通常放学后阅读博客的青少年，而 20% 是喜欢科幻、通常周末访问的成年人。如果使用层次聚类（hierarchical clustering）算法，它还可能把每个群组继续细分为更小的群组。这有助于你针对不同群组定向设计文章。

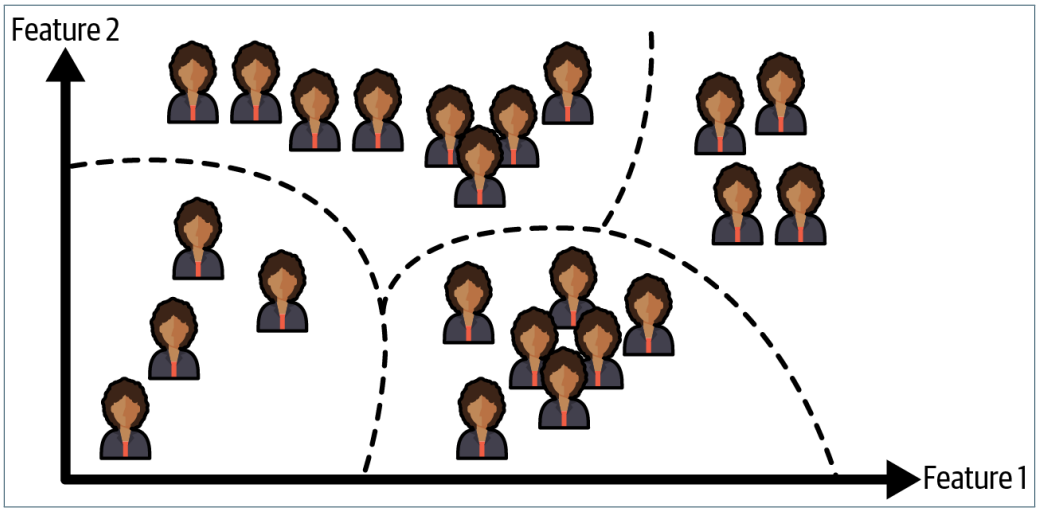

图 1-7: 聚类（Clustering）

可视化算法（visualization algorithm）也是无监督学习的好例子：向它们输入大量复杂且没有标签的数据，它们会输出便于绘制的二维或三维数据表示（图 1-8）。这些算法会尽量保留原始结构（例如尽量避免输入空间中彼此分离的聚类在可视化结果中发生重叠），从而帮助你理解数据的组织方式，并可能发现未曾察觉的模式。

另一个相关任务是降维（dimensionality reduction），其目标是在不过多损失信息的前提下简化数据。一种做法是把若干相关特征合并为一个特征。例如，汽车里程可能与车龄高度相关，因此降维算法可以把它们合并为一个表示汽车磨损程度的特征。这称为特征提取（feature extraction）。

> 提示：通常值得先用降维算法减少训练数据的维度，再将其输入另一个机器学习算法（例如监督学习算法）。这样算法会运行得快得多，数据占用的磁盘和内存空间也更少；在某些情况下，性能还会更好。

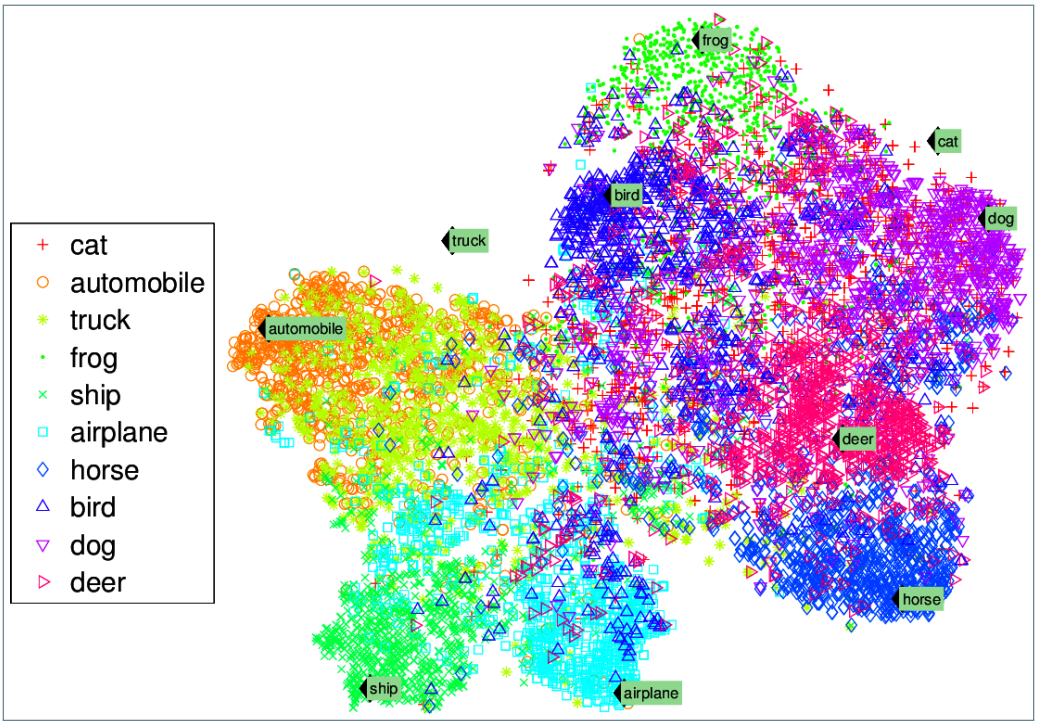

图 1-8: 突出显示语义聚类的 t-SNE 可视化示例（Example of a t-SNE visualization highlighting semantic clusters）

图中动物与车辆分得相当清楚；马与鹿彼此接近，却离鸟较远。[2]

另一个重要的无监督任务是异常检测（anomaly detection），例如检测异常信用卡交易以防止欺诈、发现制造缺陷，或在将数据集交给另一个学习算法前自动移除离群值。训练时，系统主要看到正常实例，因此会学会识别它们；当它看到新实例时，便可以判断该实例看起来是否正常，或是否可能属于异常（见图 1-9）。特征可能包括距住所的距离、一天中的时间、星期几、取款金额、商户类别、交易频率等。

一个非常相似的任务是新颖性检测（novelty detection）：它旨在检测与训练集中所有实例都不同的新实例。这要求训练集非常“干净”，不能含有你希望算法检测出来的实例。例如，如果你有数千张狗的照片，其中 1% 是吉娃娃，那么新颖性检测算法不应把新的吉娃娃照片视为新颖实例。另一方面，异常检测算法可能认为这些狗太少见、与其他狗差异太大，从而把它们判为异常（无意冒犯吉娃娃）。

[2]图片经 Richard Socher 等人许可转载，见“Zero-Shot Learning Through Cross-Modal Transfer”，Proceedings of the 26th International Conference on Neural Information Processing Systems 1（2013）：935-943。

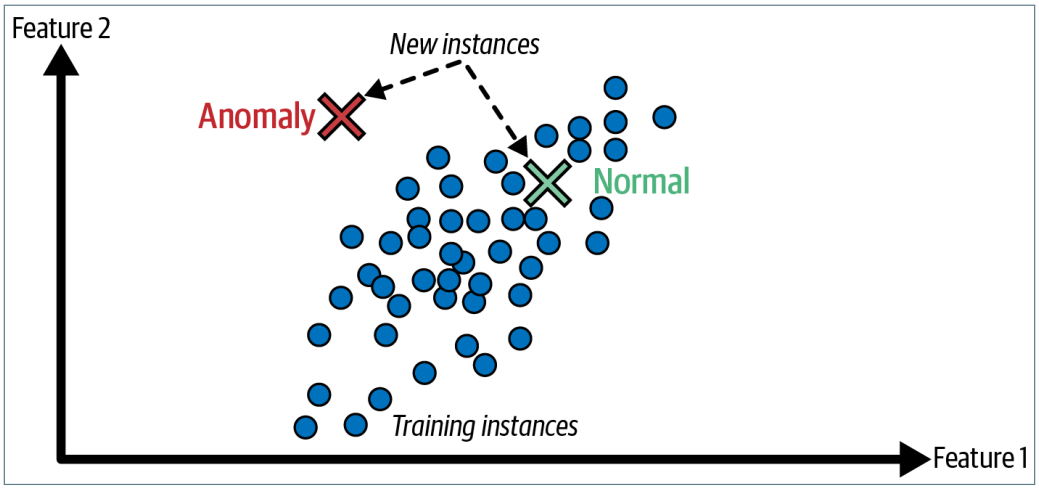

图 1-9: 异常检测（Anomaly detection）

最后，另一个常见的无监督任务是关联规则学习（association rule learning），其目标是从大量数据中发现属性之间有趣的关系。例如，假设你经营一家超市。对销售记录运行关联规则算法，可能会发现购买烧烤酱和薯片的人往往还会购买牛排。因此，你可能想把这些商品摆得近一些。

半监督学习 由于标注数据通常耗时且昂贵，你经常会拥有大量无标签实例，却只有少量带标签实例。有些算法能够处理部分标注的数据，这称为半监督学习（semi-supervised learning；图 1-10）。

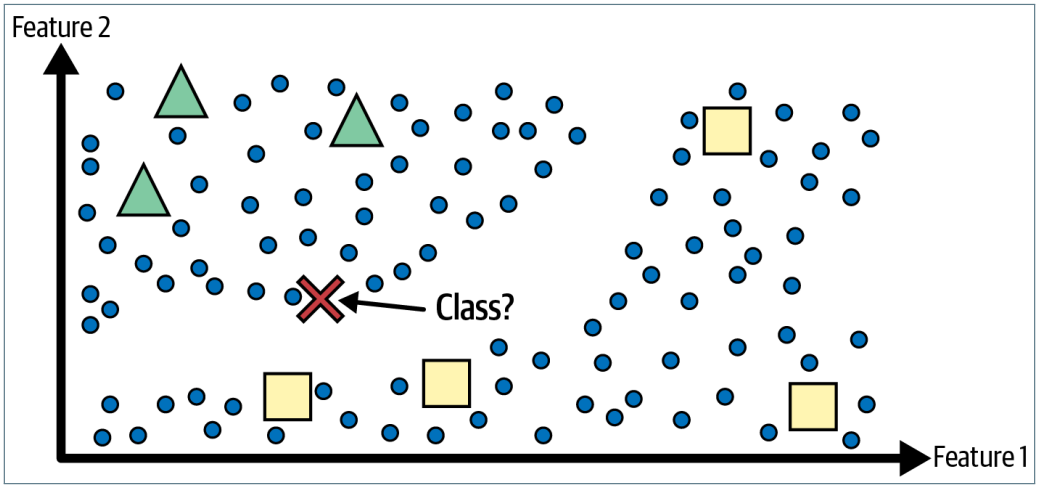

图 1-10: 包含两个类别（三角形与正方形）的半监督学习：无标签样本（圆形）有助于把新实例（叉号）归入三角形类别，而不是正方形类别，尽管它更靠近带标签的正方形（Semi-supervised learning）

一些照片托管服务（如 Google Photos）就是很好的例子。把所有家庭照片上传后，服务会自动识别出同一个人 A 出现在照片 1、5 和 11 中，而另一个人 B 出现在照片 2、5 和 7 中。这是算法的无监督部分（聚类）。接下来，系统只需要你告诉它这些人是谁。每个人只需添加一个标签，[3]系统就能在每张照片中标出所有人的姓名，这对照片搜索很有用。

多数半监督学习算法都结合了无监督算法和监督算法。例如，可以先用聚类算法把相似实例归为一组，再用该聚类中最常见的标签标注每个无标签实例。整个数据集都带上标签后，便可以使用任意监督学习算法。

自监督学习　另一种机器学习方法，是从完全没有标签的数据集中生成一个完整的带标签数据集。数据集全部带上标签后，同样可以使用任意监督学习算法。这种方法称为自监督学习（self-supervised learning）。

例如，假设你有一个大型无标签图像数据集，可以随机遮住每幅图像的一小部分，然后训练模型恢复原图（图 1-11）。训练时，遮挡后的图像作为模型输入，原始图像则作为标签。

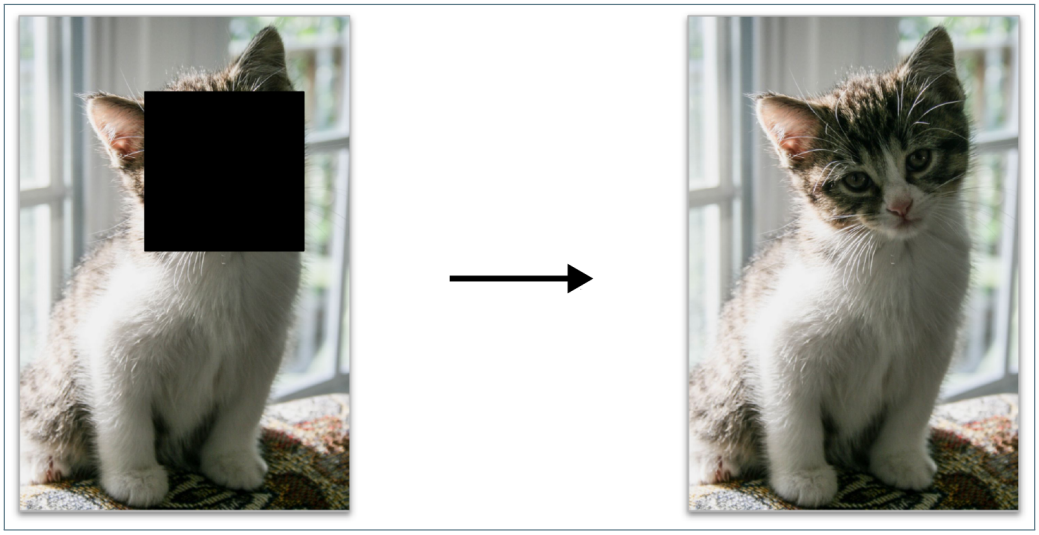

图 1-11: 自监督学习示例：输入（左）和目标（右）（Self-supervised learning example）

得到的模型本身可能就很有用，例如修复受损图像，或从照片中擦除不需要的对象。不过，大多数情况下，自监督学习训练出的模型并不是最终目标。通常还要针对另一个略有不同、真正关心的任务调整并微调模型。

[3]这是系统运行完美时的情况。实践中，它常常会为同一个人建立多个聚类，有时也会混淆长相相似的两个人，因此你可能需要为每个人提供几个标签，并手动清理某些聚类。

例如，假设你真正想要的是宠物分类模型：给它任何宠物照片，它都能判断宠物属于哪个物种。如果你有一个大型无标签宠物照片数据集，可以先通过自监督学习训练图像修复模型。当它表现良好后，应该已经能够区分不同宠物物种：修复一张脸部被遮挡的猫咪照片时，它必须知道不能补上一张狗脸。只要模型架构允许（多数神经网络架构都允许），随后就可以调整模型，让它预测宠物物种，而不是修复图像。最后一步是在带标签数据集上微调（fine-tune）模型：模型已经知道猫、狗以及其他宠物物种长什么样，因此这一步只需让它学习已知物种与预期标签之间的映射。

> 说明：将从一个任务学到的知识迁移到另一个任务，称为迁移学习（transfer learning）。这是当今机器学习中最重要的技术之一，尤其是在使用深度神经网络（即由多层神经元构成的神经网络）时。我们将在第二部分详细讨论。

如第 15 章将介绍的那样，大型语言模型（large language model，LLM）的训练方式非常相似：随机遮住巨大文本语料库中的词，训练模型预测缺失的词。得到的大型预训练模型随后可以针对从情感分析到聊天机器人的各种应用进行微调。

有些人认为自监督学习属于无监督学习，因为它处理的是完全没有标签的数据集。但自监督学习在训练中使用（生成的）标签，因此从这个角度看，它更接近监督学习。而且，“无监督学习”一词通常用于聚类、降维或异常检测等任务；自监督学习关注的却与监督学习相同，主要是分类和回归。简而言之，最好把自监督学习视为独立类别。

强化学习　强化学习（reinforcement learning）是一种截然不同的方法。在这种情境中，学习系统称为智能体（agent）。它能够观察环境、选择并执行动作，并获得奖励作为回报（或者得到以负奖励形式表示的惩罚，如图 1-12 所示）。随后，它必须自行学会能让长期累计奖励最大化的最佳策略（policy）。策略定义了智能体在给定情境下应选择什么动作。

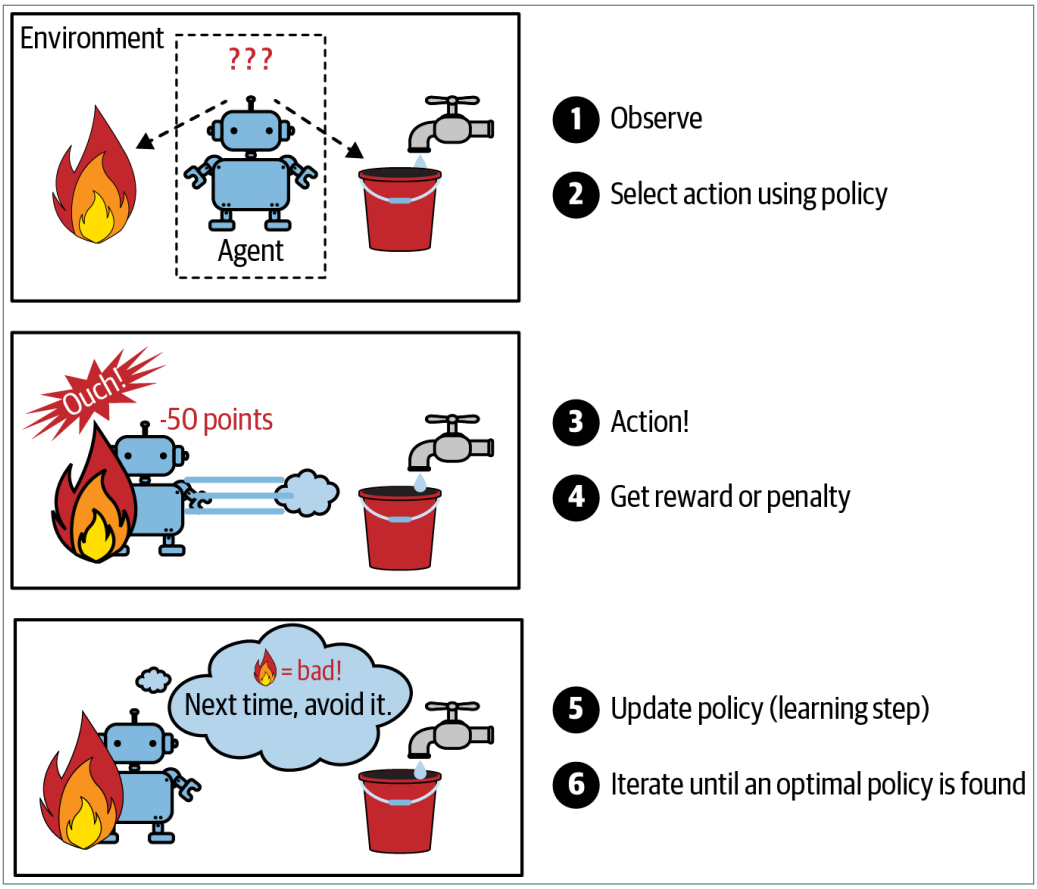

图 1-12: 强化学习（Reinforcement learning）

例如，许多机器人使用强化学习算法来学习行走。DeepMind 的 AlphaGo 程序也是强化学习的典型例子：2017 年 5 月，它击败了当时世界排名第一的围棋选手柯洁，登上新闻头条。AlphaGo 通过分析数百万盘棋局、再与自己对弈许多盘，学会了制胜策略。请注意，在它与冠军对弈时，学习功能已经关闭；AlphaGo 只是在应用已经学会的策略。正如下节将介绍的那样，这称为离线学习（offline learning）。

批量学习与在线学习　机器学习系统的另一个分类标准，是系统能否从持续到来的数据流中增量学习。例如，随机森林（参见第 6 章）只能在完整数据集上从头训练，这称为批量学习（batch learning）；其他模型则可以每次在一个数据批次上训练，例如使用梯度下降（参见第 4 章），这称为在线学习（online learning）。

批量学习　在批量学习中，系统必须使用所有可用数据进行训练。这通常需要大量时间和计算资源，因此一般在线下完成。系统先接受训练，然后部署到生产环境，此后不再继续学习，只应用已经学到的内容。这称为离线学习。

遗憾的是，模型性能往往会随着时间缓慢衰减，原因很简单：世界在不断变化，而模型保持不变。这种现象通常称为数据漂移（data drift，也称模型腐化 model rot）。解决办法是定期使用最新数据重新训练模型。重训频率取决于具体用例：如果模型用于区分猫和狗的图片，性能衰减会非常缓慢；但如果模型处理快速变化的系统，例如预测金融市场，性能就可能迅速衰减。

> 警告：即使模型只是对猫狗图片进行分类，也可能需要定期重新训练。不是因为猫狗会一夜之间发生变异，而是因为相机、图像格式、清晰度、亮度和宽高比都在不断变化。此外，明年人们可能喜欢不同的品种，或者决定给宠物戴上小帽子–谁知道呢？

如果希望批量学习系统掌握新数据（例如一种新的垃圾邮件），就必须在整个数据集上从头训练系统的新版本–不仅使用新数据，也要使用旧数据–然后用新模型替换旧模型。幸运的是，训练、评估和部署机器学习系统的整个过程都可以自动化（如图 1-3 所示），因此批量学习系统也能适应变化。只需更新数据，并按需要的频率从头训练系统的新版本即可。

这种方案简单而且通常效果良好，但使用完整数据集训练可能耗费数小时，因此一般每 24 小时甚至每周才训练一个新系统。如果系统需要适应快速变化的数据（例如预测股票价格），就需要反应更快的方案。

此外，在完整数据集上训练需要大量计算资源，包括 CPU、内存、磁盘空间、磁盘 I/O 和网络 I/O 等。如果数据很多，又把系统自动化为每天从头训练，最终会花费大量资金。如果数据量巨大且系统必须时刻保持最新，甚至可能根本无法使用批量学习。

最后，如果系统需要自主学习，而可用资源又很有限（例如智能手机应用或火星车），那么携带大量训练数据、每天占用大量资源训练数小时将成为无法接受的障碍。

在这些情形下，更好的选择是使用能够增量学习的算法。

在线学习　在线学习系统通过依次输入数据实例来进行增量训练；每次可以输入一个实例，也可以输入称为小批量（mini-batch）的小组实例。每一步学习都快速且廉价，因此系统可以在新数据到达时实时学习（见图 1-13）。目前最常见的在线算法是梯度下降（gradient descent），此外还有少数其他算法。

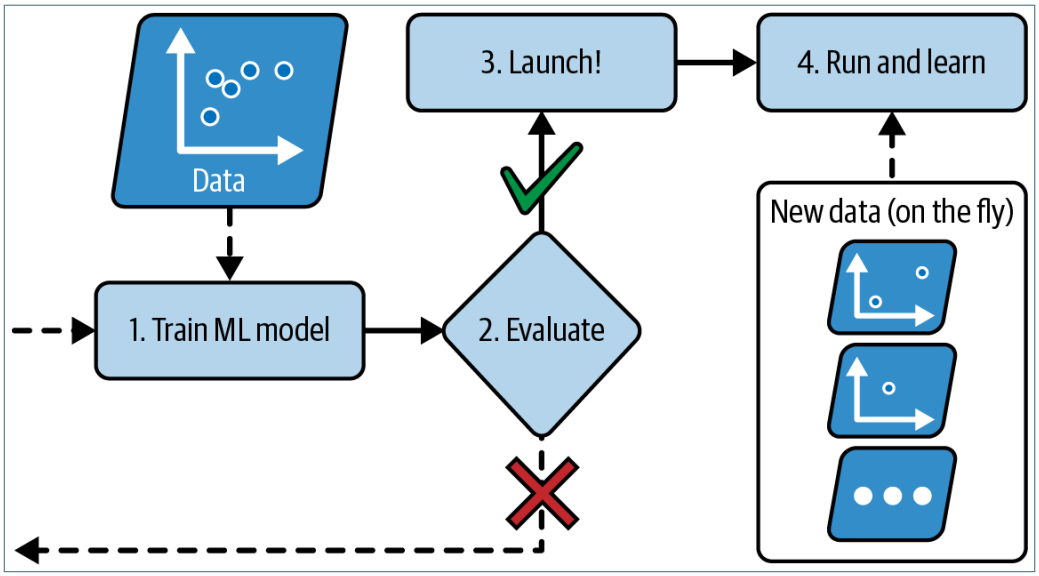

图 1-13: 在在线学习中，模型经过训练并部署到生产环境，此后会随着新数据到来持续学习（Online learning）

在线学习适用于需要极快适应变化的系统（例如检测股市中的新模式）。如果计算资源有限，例如模型在移动设备上训练，它也是不错的选择。

最重要的是，在线学习算法能够在大到无法装入单台计算机内存的数据集上训练模型，这称为核外学习（out-of-core learning）。算法加载一部分数据，在这部分数据上执行一个训练步骤，然后重复这一过程，直到处理完全部数据（见图 1-14）。

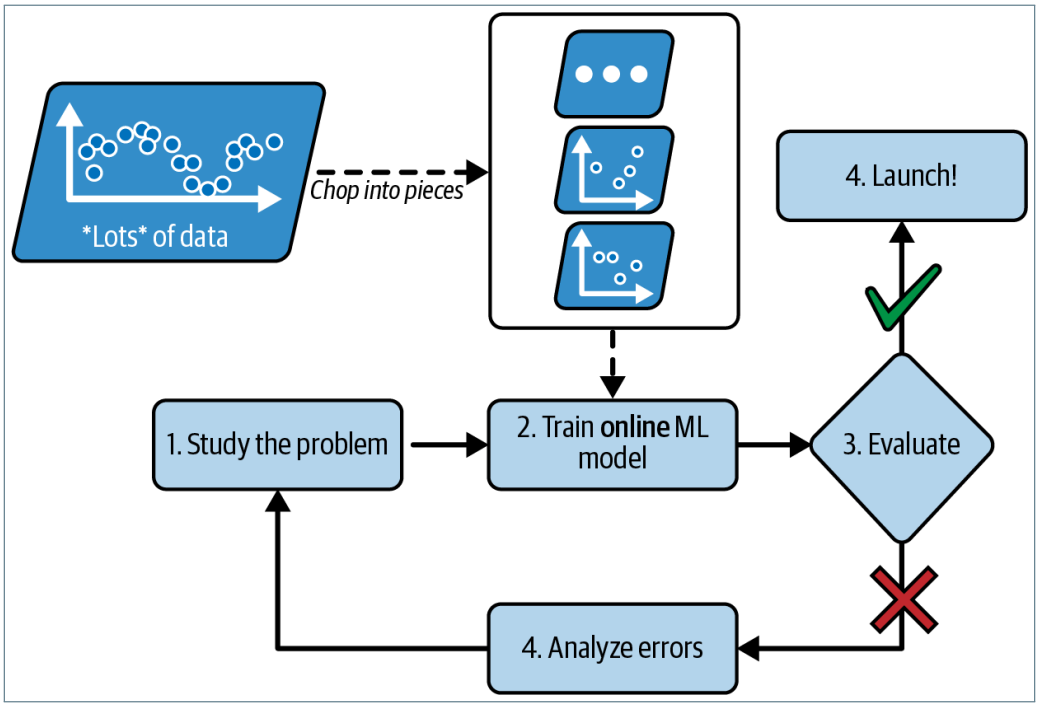

图 1-14: 使用在线学习处理巨大数据集（Using online learning to handle huge datasets）

在线学习系统的一个重要参数，是它适应数据变化的速度，这称为学习率（learning rate）。如果学习率很高，系统会迅速适应新数据，但也容易很快忘记旧数据；这称为灾难性遗忘（catastrophic forgetting，也称 catastrophic interference）。你当然不希望垃圾邮件过滤器只会识别最近见过的垃圾邮件！相反，如果学习率较低，系统的惯性就更大；也就是说，它学习得较慢，但对新数据中的噪声或连续出现的不具代表性数据点（离群值）也不那么敏感。

> 警告：核外学习通常在线下进行（即不在实时运行的系统上），所以“在线学习”这个名称可能令人困惑。可以把它理解为“增量学习”。此外，小批量经常简称为“批量”，因此“批量学习”同样容易令人困惑；可以把它理解为“在完整数据集上从头学习”。

在线学习面临的一项重大挑战是：如果系统接收到坏数据，性能就会下降，而且可能下降得很快，具体取决于数据质量和学习率。如果这是实时系统，客户会直接注意到。例如，坏数据可能来自程序缺陷（如机器人上发生故障的传感器），也可能来自试图操纵系统的人（如向搜索引擎灌入垃圾内容，企图提高搜索排名）。为了降低这种风险，需要密切监控系统；一旦检测到性能下降，就应立即停止学习，并且可

能要回退到先前能够正常工作的状态。还可以监控输入数据并对异常数据作出反应，例如使用异常检测算法（参见第 8 章）。

基于实例的学习与基于模型的学习　机器学习系统还可以按其泛化方式分类。多数机器学习任务都与预测有关：给定若干训练示例，系统需要对从未见过的示例作出良好预测，也就是泛化到新示例。在训练数据上取得良好的性能指标固然不错，但这还不够；真正的目标是在新实例上表现良好。

泛化主要有两种方法：基于实例的学习和基于模型的学习。

基于实例的学习　最简单的学习形式或许就是死记硬背。如果按这种方式构建垃圾邮件过滤器，它只会标记那些与用户已经标记过的邮件完全相同的邮件–这不是最差的方案，但肯定也不是最好的。

垃圾邮件过滤器不必只标记与已知垃圾邮件完全相同的邮件；还可以让它标记与已知垃圾邮件非常相似的邮件。这需要定义两封邮件之间的相似性度量（similarity measure）。一种非常基础的度量方法，是统计两封邮件共有多少个单词。如果一封邮件与某封已知垃圾邮件包含许多相同单词，系统就会把它标记为垃圾邮件。

这称为基于实例的学习（instance-based learning）：系统记住示例，再用相似性度量把新样本与学过的示例（或其中一部分）比较，从而泛化到新情况。例如，在图 1-15 中，与新实例最相似的多数实例属于三角形类别，因此新实例会被归为三角形。

基于实例的学习在小型数据集上往往表现出色，特别是在数据持续变化时；但它的扩展性不佳：需要把完整训练集副本部署到生产环境，预测时还要搜索相似实例，速度可能很慢，而且它不适合图像等高维数据。

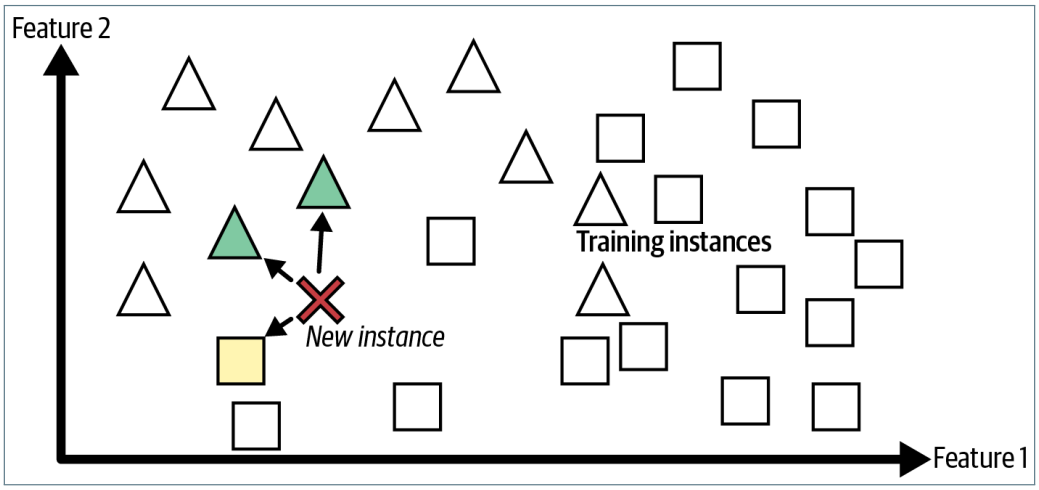

图 1-15: 基于实例的学习：本例考虑训练集中三个最近邻的类别（Instance-based learning）

基于模型的学习与典型机器学习工作流程　从一组示例进行泛化的另一种方法，是为这些示例建立模型，再使用该模型进行预测。这称为基于模型的学习（model-based learning；图 1-16）。

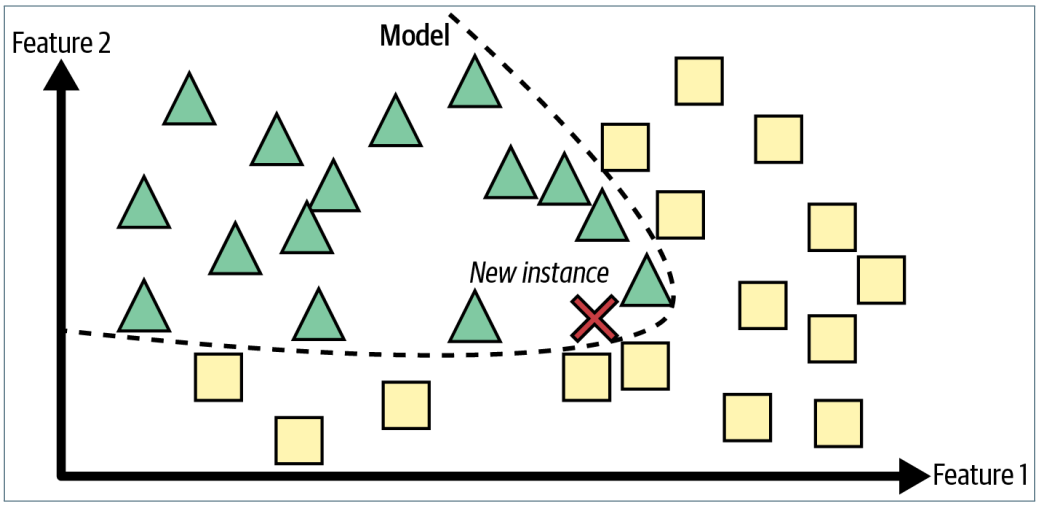

图 1-16: 基于模型的学习（Model-based learning）

例如，假设你想知道金钱能否让人更幸福，于是从经济合作与发展组织（OECD）网站下载“美好生活指数”（Better Life Index）数据，又从世界银行获取人均国内生产总值（GDP）统计数据。

接着，把两张表连接起来，并按人均 GDP 排序。表 1-1 展示了所得数据的一部分。

表 1-1: 金钱会让人更幸福吗?

国家	人均 GDP（美元）	生活满意度
土耳其	28,384	5.5
匈牙利	31,008	5.6
法国	42,026	6.5
美国	60,236	6.9
新西兰	42,404	7.3
澳大利亚	48,698	7.3
丹麦	55,938	7.6

下面绘制这些国家的数据（图 1-17）。

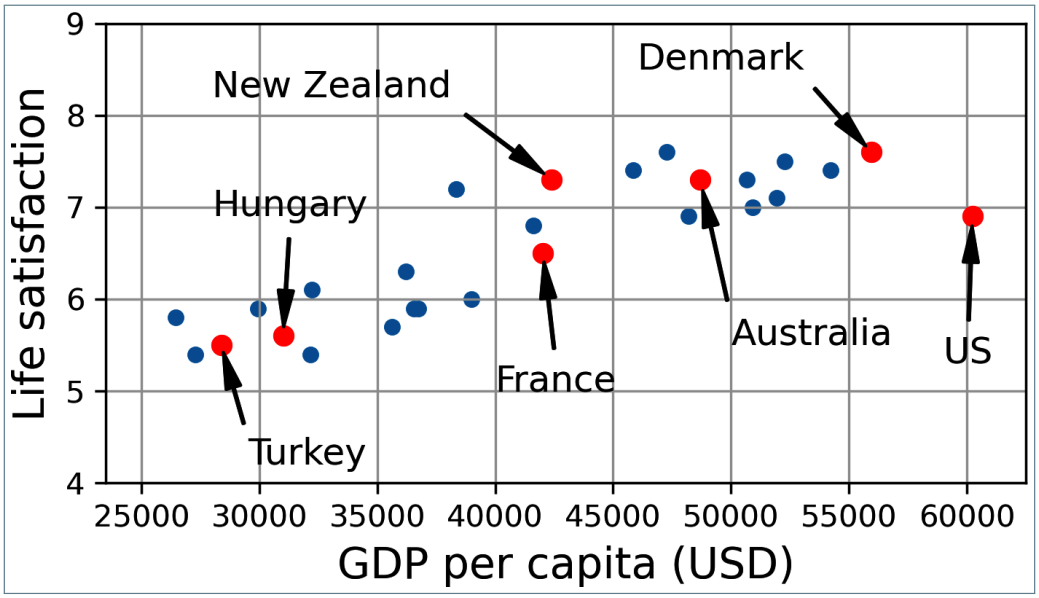

图 1-17: 你看出趋势了吗？（Do you see a trend here?）

数据似乎确实存在趋势！尽管其中有噪声（即部分随机性），但生活满意度看起来会随着人均 GDP 增长而大致呈线性上升。因此，你决定把生活满意度建模为人均 GDP 的线性函数，并假设偏离直线的部分只是随机噪声。这一步称为模型选择（model selection）：你选择了一个只使用人均 GDP 这一项属性的生活满意度线性模型（公式 1-1）。

$$\text{life_satisfaction} = \theta_0 + \theta_1 \times \text{GDP_per_capita} \tag{1-1}$$

公式 1-1　简单线性模型（A simple linear model）

这个模型有两个模型参数（model parameter）：θ_0 和 θ_1。[4]通过调整这些参数，可以让模型表示任意线性函数，如图 1-18 所示。

[4]按照惯例，希腊字母 θ（theta）经常用于表示模型参数。

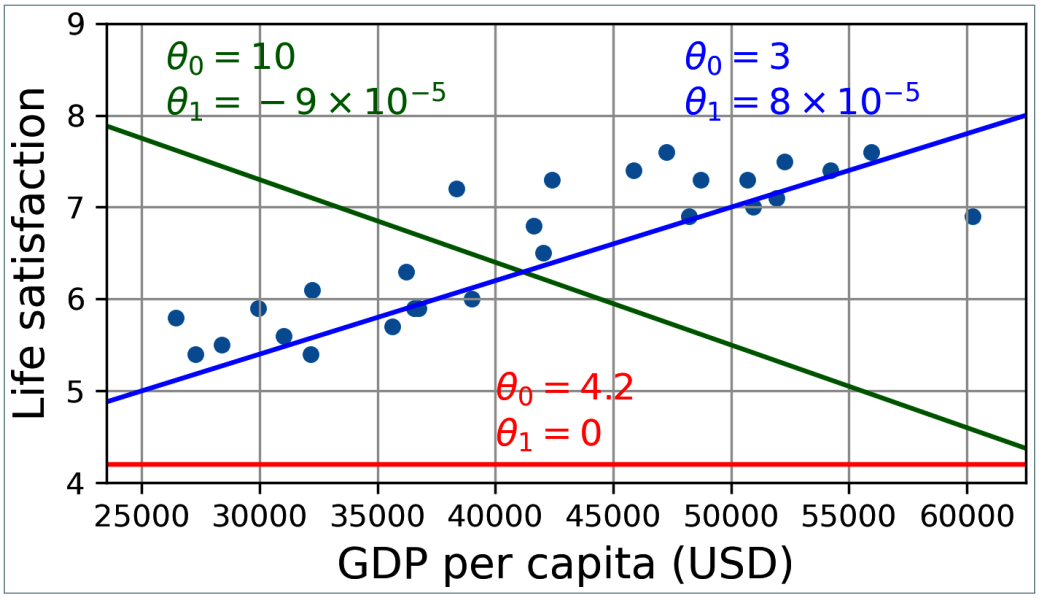

图 1-18: 几个可能的线性模型（A few possible linear models）

在使用模型之前，需要确定参数 θ_0 和 θ_1 的值。怎样才能知道哪些值会让模型表现最佳？为回答这个问题，需要指定性能度量。可以定义效用函数（utility function，也称适应度函数 fitness function）来衡量模型有多好，也可以定义成本函数（cost function，也称损失函数 loss function）来衡量模型有多差。对于线性回归问题，人们通常使用一种成本函数来测量线性模型预测结果与训练示例之间的距离，目标是让这个距离最小。

这正是线性回归算法发挥作用的地方：向算法输入训练示例，它就会找到最能让线性模型拟合数据的参数。这称为训练模型。在本例中，算法求得最优参数 $\theta_0 = 3.75$，$\theta_1 = 6.78 \times 10^{-5}$。

> 警告：容易引起混淆的是，“模型”一词既可能指模型类型（例如线性回归），也可能指已经完整指定的模型架构（例如一个输入、一个输出的线性回归），还可能指可以直接用于预测的最终已训练模型（例如一个输入、一个输出，且 $\theta_0 = 3.75$、$\theta_1 = 6.78 \times 10^{-5}$ 的线性回归）。模型选择是选择模型类型并完整指定其架构；训练模型则是运行算法，寻找最适合训练数据、并有望在新数据上作出良好预测的模型参数。

现在，这个模型已经在一个线性模型所能达到的范围内尽可能贴合训练数据，如图 1-19 所示。

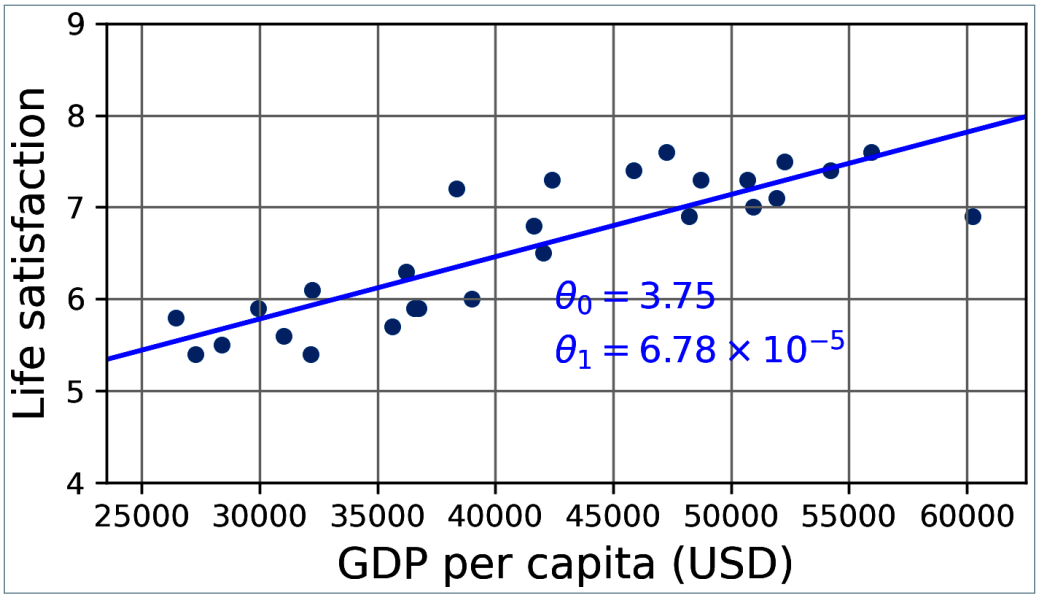

图 1-19: 最贴合训练数据的线性模型（The linear model that fits the training data best）

终于可以运行模型进行预测了。例如，假设你想知道波多黎各居民的幸福程度，但 OECD 数据没有给出答案。好在可以使用模型作出合理预测：查得波多黎各人均 GDP 为 33,442 美元，代入模型可得生活满意度约为 $3.75 + 33{,}442 \times 6.78 \times 10^{-5} = 6.02$。

为了让你先尝尝鲜，示例 1-1 给出了相应的 Python 代码：它加载数据，把输入 X 与标签 y 分开，绘制散点图以便可视化，然后训练线性模型并作出预测。[5]

[5] 即使你目前还不能完全理解这些代码也没关系；我将在后续章节介绍 Scikit-Learn。

示例 1-1：使用 Scikit-Learn 训练并运行线性模型

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

# Download and prepare the data
data_root = "https://github.com/ageron/data/raw/main/"
lifesat = pd.read_csv(data_root + "lifesat/lifesat.csv")
X = lifesat[["GDP per capita (USD)"]].values
y = lifesat[["Life satisfaction"]].values

# Visualize the data
lifesat.plot(kind='scatter', grid=True,
             x="GDP per capita (USD)", y="Life satisfaction")
plt.axis([23_500, 62_500, 4, 9])
plt.show()

# Select a linear model
model = LinearRegression()

# Train the model
model.fit(X, y)

# Make a prediction for Puerto Rico
X_new = [[33_442.8]]  # Puerto Rico' GDP per capita in 2020
print(model.predict(X_new)) # outputs [[6.01610329]]
```

说明：如果改用基于实例的学习算法，你会发现波兰的人均 GDP（32,238 美元）最接近波多黎各。OECD 数据显示，波兰人的生活满意度为 6.1，因此你也会预测波多黎各的生活满意度为 6.1。如果扩大范围，查看接下来两个最近的国家，会发现葡萄牙的生活满意度为 5.4，爱沙尼亚为 5.7。对这三个值取平均，得到 5.73，略低于基于模型的预测。这个简单算法称为 k 近邻回归（k-nearest neighbors regression；本例中 k = 3）。

在前面的代码中，把线性回归模型替换为 k 近邻回归，只需把下面两行：

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
```

替换为：

```
from sklearn.neighbors import KNeighborsRegressor
model = KNeighborsRegressor(n_neighbors=3)
```

如果一切顺利，模型将作出良好预测。如果效果不好，可能需要使用更多属性（就业率、健康状况、空气污染等），获取更多或质量更高的训练数据，或者选择更强大的模型（例如多项式回归模型）。

总结一下：

- 研究数据。
- 选择模型。
- 在训练数据上训练模型，也就是让学习算法搜索能够使成本函数最小的模型参数值。
- 最后，应用模型对新情况作出预测，这称为推理（inference），并希望模型能够很好地泛化。

这就是典型机器学习项目的基本样貌。第 2 章将通过一个端到端项目，让你亲自经历整个过程。

我们已经讨论了相当多的 ML 系统类别，但这个领域还有更多类型。例如，集成学习（ensemble learning）会训练多个模型，再把它们各自的预测组合成更好的预测（参见第 6 章）；联邦学习（federated learning）是一种去中心化方法，模型在多个设备（例如智能手机）上分别训练并针对每个用户调整，同时无需交换原始数据，从而保护用户隐私；元学习（meta-learning）是一种“学习如何学习”的方法，让模型能够凭借极少数据快速学会新任务。这样的类别还可以继续列下去！图 1-20 汇总了我们目前讨论过的各种 ML 系统分类。

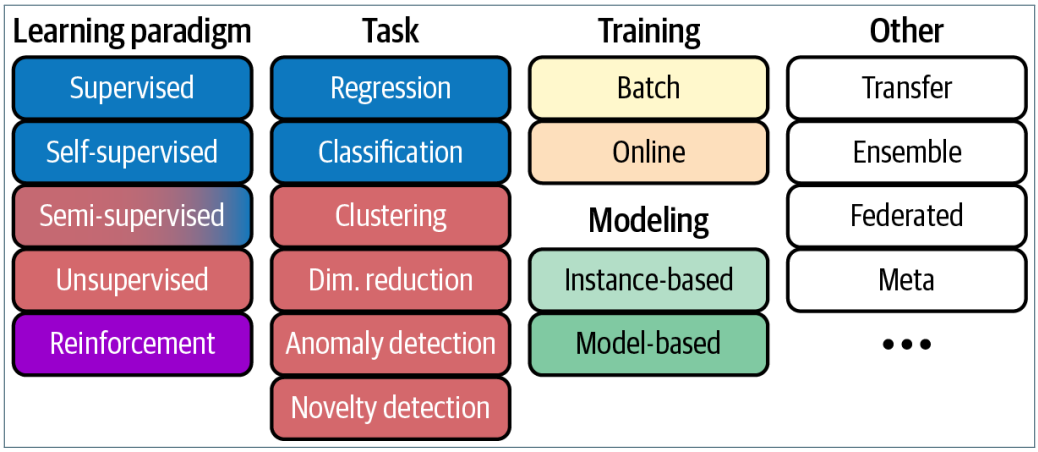

图 1-20: 机器学习类别概览（Overview of ML categories）

到目前为止，我们已经走过很长一段路：你现在知道机器学习究竟是什么、它为什么有用、最常见的机器学习系统类别有哪些，以及典型项目工作流程是什么样子。下面来看一看学习过程中可能出现哪些问题，导致系统无法作出准确预测。

1.5　机器学习的主要挑战（The Main Challenges of Machine Learning）

简而言之，由于主要任务是选择模型并用数据训练模型，可能出问题的两方面就是“坏模型”和“坏数据”。先从坏数据的例子开始。

训练数据数量不足　幼儿学习什么是苹果时，你只需指着一个苹果说“苹果”（也许重复几次）。随后，孩子便能识别各种颜色和形状的苹果。真是天才。

机器学习还没有达到这种程度；多数机器学习算法都需要大量数据才能正常工作。即使是非常简单的问题，通常也需要数千个示例；对于图像或语音识别等复杂问题，可能需要数百万个示例，除非能够复用现有模型的一部分，也就是使用迁移学习。

> 数据不可思议的有效性：在 2001 年发表的一篇著名论文中，微软研究员 Michele Banko 和 Eric Brill 表明，在获得足够数据后，差异很大的机器学习算法–包括相当简单的算法–在复杂的自然语言消歧问题[a]上取得了几乎相同的良好表现，如图 1-21 所示。
>
> [a]例如，根据上下文判断应写作“to”、“two”还是“too”。

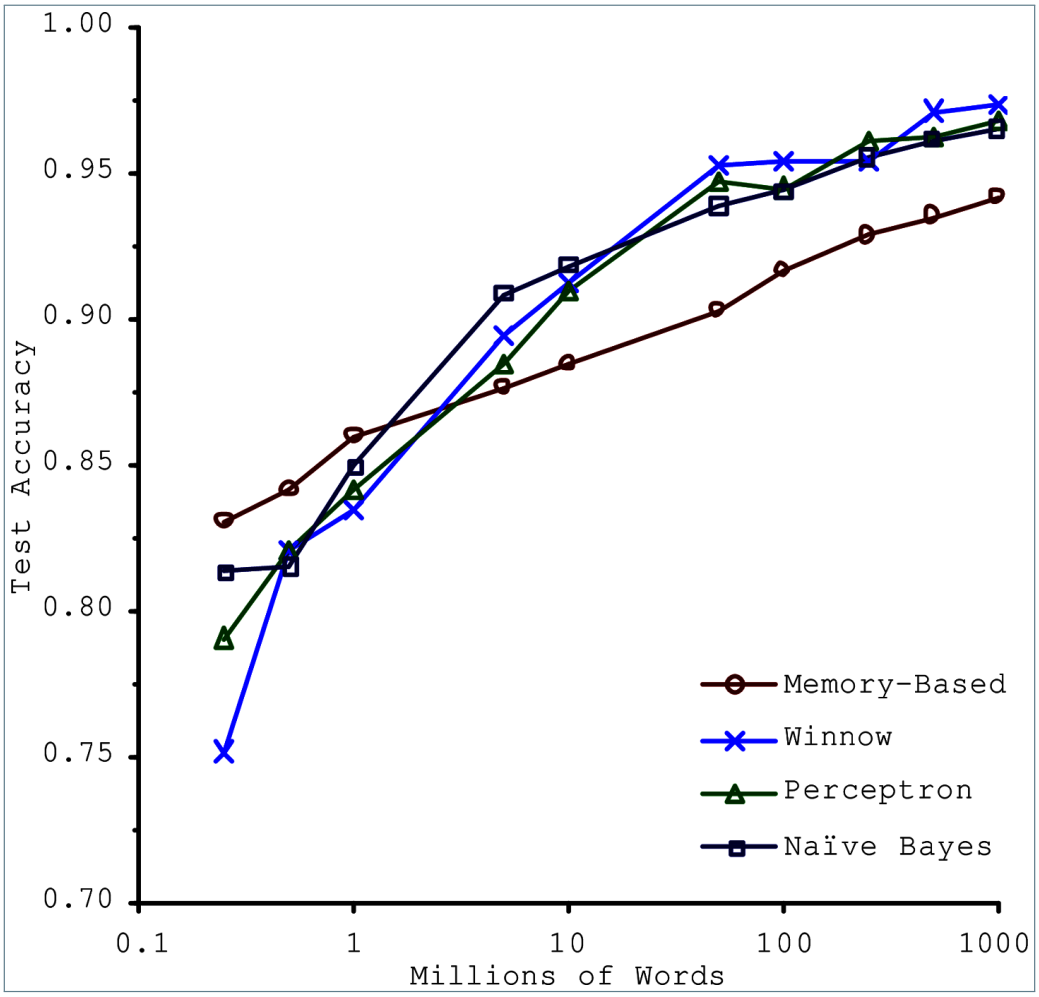

图 1-21: 数据与算法的重要性对比（The importance of data versus algorithms）

该图经原作者许可转载。[6]

正如两位作者所说，这些结果表明，我们或许应该重新考虑这样一种权衡：把时间和金钱投入算法开发，还是投入语料库建设。2009 年，Peter Norvig 等人在题为“The Unreasonable Effectiveness of Data”的论文中进一步推广了这样一种观点：对于复杂问题，数据比算法更重要。[7]不过需要注意，小型和中型数据集依然非常常见，获取额外训练数据并不总是容易或便宜–所以暂时还不能放弃算法。

[6]图片来源：Michele Banko 与 Eric Brill，“Scaling to Very Very Large Corpora for Natural Language Disambiguation”，Proceedings of the 39th Annual Meeting of the Association for Computational Linguistics (2001)：26-33。

[7]Peter Norvig 等，“The Unreasonable Effectiveness of Data”，IEEE Intelligent Systems 24，第 2 期 (2009)：8-12。

训练数据缺乏代表性　为了很好地泛化，训练数据必须能够代表希望泛化到的新情况，这一点至关重要。无论使用基于实例的学习还是基于模型的学习，都是如此。

例如，先前用于训练线性模型的国家集合并不完全具有代表性；其中没有任何人均 GDP 低于 23,500 美元或高于 62,500 美元的国家。图 1-22 展示了加入这些国家后的数据。

如果使用这些数据训练线性模型，会得到图中的实线，而旧模型用虚线表示。可以看出，加入几个缺失的国家不仅显著改变了模型，还清楚地表明：如此简单的线性模型可能永远无法取得良好效果。非常富裕的国家似乎并不比中等富裕的国家更幸福（事实上，它们看起来还略微不幸福一些）；反过来，一些贫穷国家似乎比许多富裕国家更幸福。

使用缺乏代表性的训练集，会训练出不太可能作出准确预测的模型，尤其是对于非常贫穷或非常富裕的国家。

训练集必须能够代表希望泛化到的情况。这往往说起来容易、做起来难：样本太小会产生抽样噪声（sampling noise），即由于偶然性造成数据缺乏代表性；但如果抽样方法存在缺陷，即使样本非常大，也可能不具代表性。这称为抽样偏差（sampling bias）。

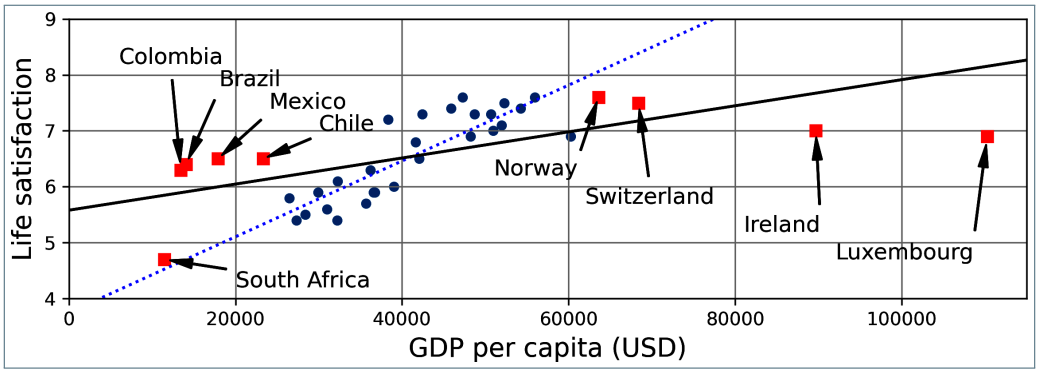

图 1-22: 更具代表性的训练样本（A more representative training sample）

抽样偏差的例子：最著名的抽样偏差案例之一发生在 1936 年美国总统选举期间，当时 Landon 与 Roosevelt 竞选。《文学文摘》（Literary Digest）开展了一项大规模民意调查，向大约 1,000 万人寄出问卷，收到 240 万份回复，并非常自信地预测 Landon 将获得 57% 的选票。然而，最终 Roosevelt 以 62% 的得票率胜出。问题出在《文学文摘》的抽样方法。第一，为取得寄送问卷的地址，它使用了电话簿、杂志订户名单和俱乐部会员名单等资料，这些名单偏向较富裕的

> 人，而他们更可能投票给共和党候选人 Landon。第二，收到问卷的人中只有不到 25% 作出了回复。这再次引入抽样偏差，因为不太关心政治、不喜欢《文学文摘》以及其他关键群体都可能被排除。这种特殊的抽样偏差称为无应答偏差（nonresponse bias）。另一个例子是，假设你要构建一个识别放克音乐视频的系统。一种建立训练集的方法是在 YouTube 上搜索“funk music”，并使用返回的视频。但这等于假设 YouTube 搜索引擎返回的视频能够代表平台上的全部放克音乐视频。实际上，搜索结果很可能偏向热门艺人；如果你住在巴西，还会看到许多与 James Brown 风格完全不同的“funk carioca”视频。可另一方面，还能怎样获得大型训练集呢？

低质量数据　显然，如果训练数据充满错误、离群值和噪声（例如来自质量低劣的测量），系统就更难检测潜在模式，因而不太可能取得良好表现。花时间清理训练数据通常非常值得。事实上，多数数据科学家都会把相当一部分时间花在这项工作上。下面是两个需要清理训练数据的例子：

- 如果某些实例显然属于离群值，直接丢弃它们或尝试手动修正错误可能会有所帮助。
- 如果某些实例缺少少量特征（例如 5% 的客户没有填写年龄），就必须决定是完全忽略这个属性、忽略这些实例、填补缺失值（例如使用年龄中位数），还是分别训练一个包含该特征的模型和一个不包含该特征的模型。

无关特征　俗话说：垃圾进，垃圾出（garbage in, garbage out）。只有当训练数据包含足够多的相关特征，又没有太多无关特征时，系统才能进行学习。机器学习项目能否成功，一个关键环节是设计出一组优质的训练特征。这个过程称为特征工程（feature engineering），包括以下步骤：

- 特征选择（feature selection）：从现有特征中选择最有用的特征进行训练。
- 特征提取（feature extraction）：组合现有特征以产生更有用的新特征；如前所述，降维算法可以提供帮助。
- 通过收集新数据创建新特征。

看过许多坏数据的例子后，下面再看几个坏算法的例子。

过拟合训练数据　假设你在国外旅行，出租车司机坑了你一笔钱。你可能忍不住断言，那个国家的所有出租车司机都是骗子。人类经常会过度概括；遗憾的是，如果不够

谨慎，机器也会掉进同一个陷阱。在机器学习中，这称为过拟合（overfitting）：模型在训练数据上表现良好，却不能很好地泛化。

图 1-23 展示了一个高阶多项式生活满意度模型，它严重过拟合了训练数据。尽管它在训练数据上的表现远好于简单线性模型，你真的会相信它的预测吗？

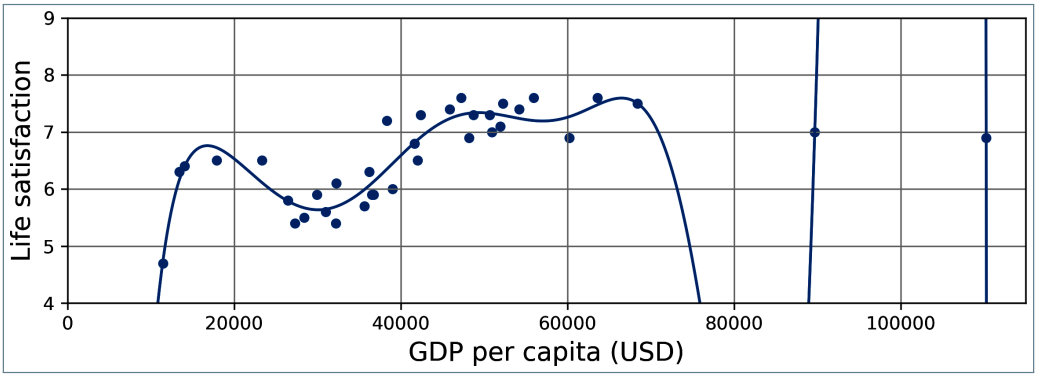

图 1-23: 过拟合训练数据（Overfitting the training data）

深度神经网络等复杂模型能够检测数据中的细微模式；但如果训练集带有噪声，或训练集太小而引入了抽样噪声，模型就可能检测到噪声本身的模式，就像前面的出租车司机例子一样。显然，这些模式无法泛化到新实例。

例如，假设你向生活满意度模型提供更多属性，其中包括国家名称之类没有信息价值的属性。复杂模型可能会发现这样一种模式：训练数据中，名称包含字母 w 的所有国家，其生活满意度都高于 7，包括 New Zealand（7.3）、Norway（7.6）、Sweden（7.3）和 Switzerland（7.5）。你有多大把握认为这条“w-满意度”规则能泛化到 Rwanda 或 Zimbabwe？显然，这种模式完全是偶然出现在训练数据中的，但模型无法判断模式是真实存在，还是仅仅由数据噪声造成。

> 警告：当模型相对于训练数据的规模和噪声程度过于复杂时，就会发生过拟合，模型开始学习训练数据中的随机模式。

可能的解决办法包括：

- 简化模型，例如选择参数更少的模型（用线性模型代替高阶多项式模型）、减少训练数据中的属性数量，或对模型施加约束。
- 收集更多训练数据。
- 降低训练数据中的噪声，例如修复数据错误并移除离群值。

约束模型，使其更简单并降低过拟合风险，称为正则化（regularization）。例如，前面定义的线性模型有两个参数 θ_0 和 θ_1。这给了学习算法两个自由度来使模型适应训练数据：它可以同时调整直线的高度（θ_0）和斜率（θ_1）。如果强制 $\theta_1 = 0$，算法就只有一个自由度，因而很难恰当地拟合数据：它只能上下移动直线，使其尽可能靠近训练实例，最终会停在均值附近。这个模型确实非常简单！

如果允许算法修改 θ_1，但强制它保持较小的值，那么学习算法实际上会拥有介于一个和两个之间的自由度。它生成的模型会比拥有两个自由度的模型简单，但比只有一个自由度的模型复杂。我们需要在完美拟合训练数据和保持模型足够简单、确保良好泛化之间找到适当平衡。

图 1-24 展示了三个模型。点线表示原始模型，它只在圆形所代表的国家上训练，不包括方形所代表的国家；实线表示使用所有国家（圆形与方形）训练的第二个模型；虚线表示使用与第一个模型相同的数据、但加入正则化约束训练的模型。可以看到，正则化迫使模型采用较小的斜率：它拟合训练数据（圆形）的效果不如第一个模型，却能更好地泛化到训练期间从未见过的新示例（方形）。

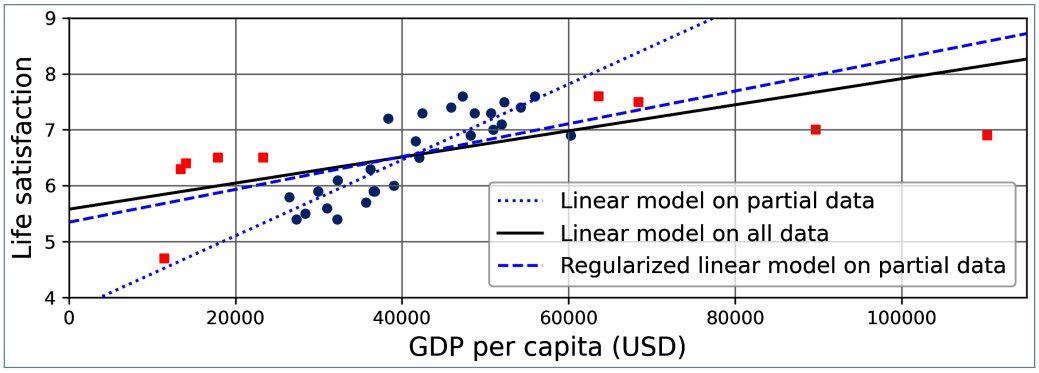

图 1-24: 正则化降低过拟合风险（Regularization reduces the risk of overfitting）

学习时施加多少正则化，可以通过超参数（hyperparameter）控制。超参数是学习算法的参数，而不是模型参数。因此，它不会受到学习算法本身的影响；必须在训练前设置，并在训练过程中保持不变。如果把正则化超参数设得非常大，得到的模型几乎是一条水平线（斜率接近零）。学习算法几乎肯定不会过拟合训练数据，但也更难找到良好解。调整超参数是构建机器学习系统的重要组成部分，下一章将给出一个详细示例。

欠拟合训练数据　不难猜到，欠拟合（underfitting）与过拟合相反：当模型过于简单，无法学习数据的潜在结构时，就会发生欠拟合。例如，用线性模型表示生活满意度

就很容易欠拟合；现实远比模型复杂，因此即使面对训练示例，它的预测也必然不够准确。

解决这个问题的主要方法包括：

- 选择参数更多、能力更强的模型。
- 向学习算法提供更好的特征，即进行特征工程。
- 减少对模型的约束，例如降低正则化超参数。

部署问题　即使拥有庞大而干净的数据集，也成功训练出了既不欠拟合也不过拟合的漂亮模型，部署时仍可能遇到问题。例如，模型可能过于复杂而难以维护，过大而无法装入内存，速度太慢，无法恰当扩展，存在安全漏洞，或因更新不够频繁而过时，等等。

简而言之，机器学习项目不仅仅涉及数据和模型。不过，解决这些运维问题所需的技能与数据建模技能差异很大，因此企业通常会设立专门的 MLOps 团队，也就是机器学习运维（machine learning operations）团队来处理这些工作。

退后一步看全局　到这里，你已经了解了许多机器学习知识。然而，我们一口气讲过这么多概念，你可能觉得有些迷失，所以让我们退后一步，看看全局：

- 机器学习的核心，是让机器从数据中学习，在某项任务上变得更好，而不必显式编写规则。
- ML 系统有许多不同类型：有监督或无监督、批量或在线、基于实例或基于模型。
- 在 ML 项目中，你收集数据组成训练集，再把训练集交给学习算法。如果算法基于模型，它会调整某些参数，让模型拟合训练集（即在训练集本身上作出良好预测），并希望模型随后也能对新情况作出良好预测。如果算法基于实例，它只会记住示例，再用相似性度量把新实例与已经学过的实例进行比较，从而泛化到新实例。
- 如果训练集太小、数据缺乏代表性、带有噪声或混入无关特征（垃圾进，垃圾出），系统就无法取得良好表现。模型既不能太简单（否则会欠拟合），也不能太复杂（否则会过拟合）。
- 最后，还必须仔细考虑部署约束。

还有最后一个重要主题需要讨论：模型训练完毕后，不能只是“希望”它能泛化到新情况。我们需要评估模型，并在必要时进行微调。下面来看具体做法。

1.6　测试与验证（Testing and Validating）

要知道模型能否很好地泛化到新情况，唯一的办法就是实际用新情况测试它。一种做法是把模型投入生产环境并监控其表现。这种方法行得通，但如果模型糟糕透顶，用户就会抱怨–这显然不是好主意。

更好的选择是把数据分成两个集合：训练集（training set）和测试集（test set）。顾名思义，使用训练集训练模型，再使用测试集测试模型。模型在新情况上的错误率称为泛化误差（generalization error，也称样本外误差 out-of-sample error）。在测试集上评估模型，可以得到这一误差的估计值。这个数值说明模型面对从未见过的实例时大概会表现如何。

如果训练误差很低（即模型在训练集上几乎不犯错），而泛化误差很高，就说明模型过拟合了训练数据。

> 提示：通常会使用 80% 的数据进行训练，并留出 20% 用于测试。不过，这取决于数据集规模。如果数据集包含 1,000 万个实例，那么只留出 1% 也能得到包含 100,000 个实例的测试集，这很可能已经足以准确估计泛化误差。

超参数调整与模型选择　评估模型很简单：使用测试集即可。但假设你在两种模型之间犹豫不决，例如线性模型和多项式模型，应如何选择？一种做法是分别训练两种模型，再使用测试集比较它们的泛化表现。

现在假设线性模型泛化得更好，但你希望加入一些正则化来避免过拟合。问题是：怎样选择正则化超参数的值？一种做法是使用 100 个不同的超参数值训练 100 个不同模型。

假设你找到了能够产生最低泛化误差的最佳超参数值，误差仅为 5%。你把这个模型部署到生产环境，遗憾的是，它的表现却不如预期，错误率达到 15%。究竟发生了什么？

问题在于，你在测试集上多次测量泛化误差，并不断调整模型和超参数，从而找到了最适合这个特定测试集的模型。这意味着它在新数据上的表现很可能不如在该测试集上那么好。

这个问题的一种常见解决方案称为留出验证（holdout validation；图 1-25）：从训练集中留出一部分数据，用于评估多个候选模型并选择最佳模型。这个新留出的集合称为验证集（validation set），也称开发集（development set 或 dev set）。具体来说，在缩减后的训练集（即完整训练集减去验证集）上，用不同超参数训练多个模型，再选择验证集上表现最佳的模型。完成留出验证后，在完整训练集（包括验证

集）上训练最佳模型，得到最终模型。最后，在测试集上评估这个最终模型，以估计泛化误差。

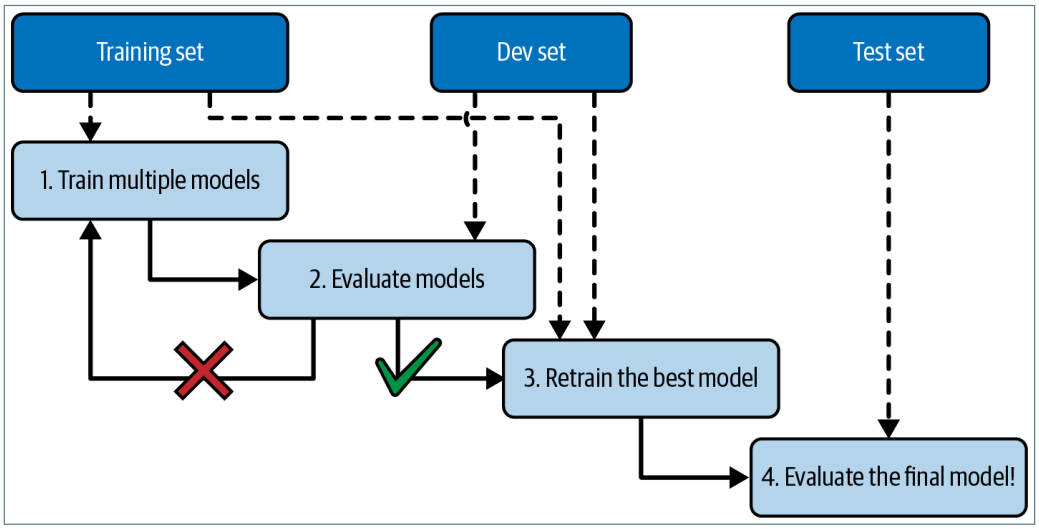

图 1-25: 使用留出验证进行模型选择（Model selection using holdout validation）

这种解决方案通常效果很好。不过，如果验证集太小，模型评估就不够精确，可能会误选次优模型。相反，如果验证集太大，剩余训练集就会远小于完整训练集。这为什么不好？因为最终模型将在完整训练集上训练，而把在小得多的训练集上训练的候选模型进行比较并不理想，就像挑选短跑最快的运动员去参加马拉松。

解决这个问题的一种办法是执行重复交叉验证（repeated cross-validation），使用许多小型验证集。每个模型在其余数据上训练后，会针对每个验证集各评估一次。对同一模型的所有评估结果取平均，可以更准确地衡量其性能。不过，这种方法也有缺点：训练时间会乘以验证集的数量。

数据不匹配 在某些情况下，很容易获得大量训练数据，但这些数据可能无法完全代表生产环境中实际使用的数据。例如，假设你要创建一个移动应用，用手机拍摄花朵并自动判断其种类。你可以轻松从网上下载数百万张花朵图片，但这些图片无法完全代表用户真正通过移动设备应用拍摄的照片。也许你只有 1,000 张有代表性的图片，也就是实际通过该应用拍摄的图片。

在这种情况下，最重要的规则是：验证集和测试集必须尽可能代表预期在生产环境中使用的数据。因此，它们应当完全由有代表性的图片组成。可以打乱这 1,000 张图片，一半放入验证集，另一半放入测试集，并确保重复或近似重复的图片不会同时出现在两个集合中。

在网络图片上训练模型后，如果发现模型在验证集上的表现令人失望，你将无法判断原因究竟是模型过拟合了训练集，还是网络图片与移动应用照片之间的数据不匹配。

一种解决办法是从训练图片（网络图片）中再留出一部分，组成 Andrew Ng 所称的训练-开发集（train-dev set；图 1-26）。模型训练完成后–训练时使用训练集而不是训练-开发集–可以在训练-开发集上评估它。如果表现不佳，就说明模型一定过拟合了训练集，因此应尝试简化模型或加强正则化、获取更多训练数据，并清理训练数据。

如果模型在训练-开发集上表现良好，就可以进一步在开发集上评估。如果表现不佳，问题必然来自数据不匹配。可以尝试预处理网络图片，让它们看起来更像移动应用将要拍摄的照片，然后重新训练模型。一旦得到在训练-开发集和开发集上都表现良好的模型，就可以最后在测试集上评估一次，以了解它在生产环境中可能达到的表现。

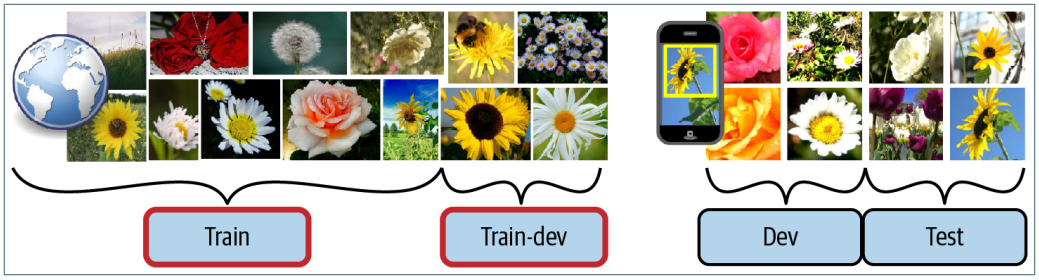

图 1-26: 当真实数据稀缺时（右），可以使用相似而充足的数据（左）进行训练，并留出一部分组成训练-开发集以评估过拟合；随后使用真实数据评估数据不匹配（开发集）和最终模型性能（测试集）（Using a train-dev set）

没有免费午餐定理：模型是数据的简化表示。简化的目的是舍弃不太可能泛化到新实例的多余细节。选择特定模型类型时，实际上也隐含地对数据作出了假设。例如，选择线性模型，就等于假设数据从根本上是线性的，而实例与直线之间的距离只是可以安全忽略的噪声。David Wolpert 在 1996 年的一篇著名论文中证明，[a]如果完全不对数据作任何假设，就没有理由偏爱某一种模型。这称为没有免费午餐（No Free Lunch，NFL）定理。对一些数据集而言，最佳模型是线性模型；对另一些数据集而言，最佳模型则是神经网络。没有任何模型能够先验地保证表现更好，这也正是该定理名称的由来。若想确切知道哪个模型最佳，唯一的方法是评估所有模型。由于这不可能做到，实践中只能对数据作

出一些合理假设，再评估少数几种合理模型。例如，对于简单任务，可以评估采用不同正则化强度的线性模型；对于复杂问题，则可以评估多种神经网络。

[a] David Wolpert, "The Lack of A Priori Distinctions Between Learning Algorithms", Neural Computation 8，第 7 期（1996）：1341-1390。

1.7　练习（Exercises）

本章介绍了机器学习中一些最重要的概念。接下来的章节将进一步深入，并编写更多代码；在此之前，请确保你能够回答以下问题：

1. 你会如何定义机器学习？
2. 请列举四类机器学习特别擅长的应用。
3. 什么是带标签训练集？
4. 最常见的两类监督学习任务是什么？
5. 请列举四类常见的无监督学习任务。
6. 如果要让机器人学会在各种未知地形中行走，应使用哪类算法？
7. 如果要把客户划分为多个群组，应使用哪类算法？
8. 你会把垃圾邮件检测定义为监督学习问题，还是无监督学习问题？
9. 什么是在线学习系统？
10. 什么是核外学习？
11. 哪类算法依靠相似性度量作出预测？
12. 模型参数与模型超参数有何区别？
13. 基于模型的算法会搜索什么？它们最常使用什么策略来获得成功？又如何作出预测？
14. 请列举机器学习面临的四项主要挑战。
15. 如果模型在训练数据上表现很好，却不能很好地泛化到新实例，发生了什么？请列出三种可能的解决方案。

16. 什么是测试集？为什么需要使用它？

17. 验证集的作用是什么？

18. 什么是训练-开发集？什么时候需要它？应如何使用？

19. 使用测试集调整超参数可能会出现什么问题？

这些练习的答案位于本章笔记本末尾：https://homl.info/colab-p。

第 2 章　端到端机器学习项目

End-to-End Machine Learning Project

本章将端到端地完成一个示例项目，假设你是一家房地产公司的新入职数据科学家。这个示例是虚构的；目的在于说明机器学习项目的主要步骤，而不是让你学习房地产行业知识。我们将依次完成以下主要步骤：

1. 了解全局。
2. 获取数据。
3. 探索并可视化数据，从中获得洞见。
4. 为机器学习算法准备数据。
5. 选择模型并训练模型。
6. 微调模型。
7. 展示解决方案。
8. 启动、监控并维护系统。

2.1　使用真实数据（Working with Real Data）

学习机器学习时，最好使用真实世界的数据进行实验，而不是人工构造的数据集。幸运的是，目前有成千上万个开放数据集可供选择，覆盖各种领域。下面是一些可以获取数据的热门开放数据仓库：

- Google Datasets Search
- Hugging Face Datasets
- OpenML.org
- Kaggle.com

- UC Irvine Machine Learning Repository
- Stanford Large Network Dataset Collection
- Amazon AWS datasets
- U.S. Government's Open Data
- Data Portals.org
- Wikipedia 的机器学习数据集列表

本章将使用 StatLib 仓库中的 California Housing Prices 数据集。[1]（见图 2-1）。该数据集基于 1990 年加利福尼亚州人口普查数据。它并不算新数据（当时湾区的一栋好房子仍然买得起），但它非常适合用来学习，因此我们暂且把它当作最新数据。为了教学，我添加了一个类别属性，并删除了几个特征。

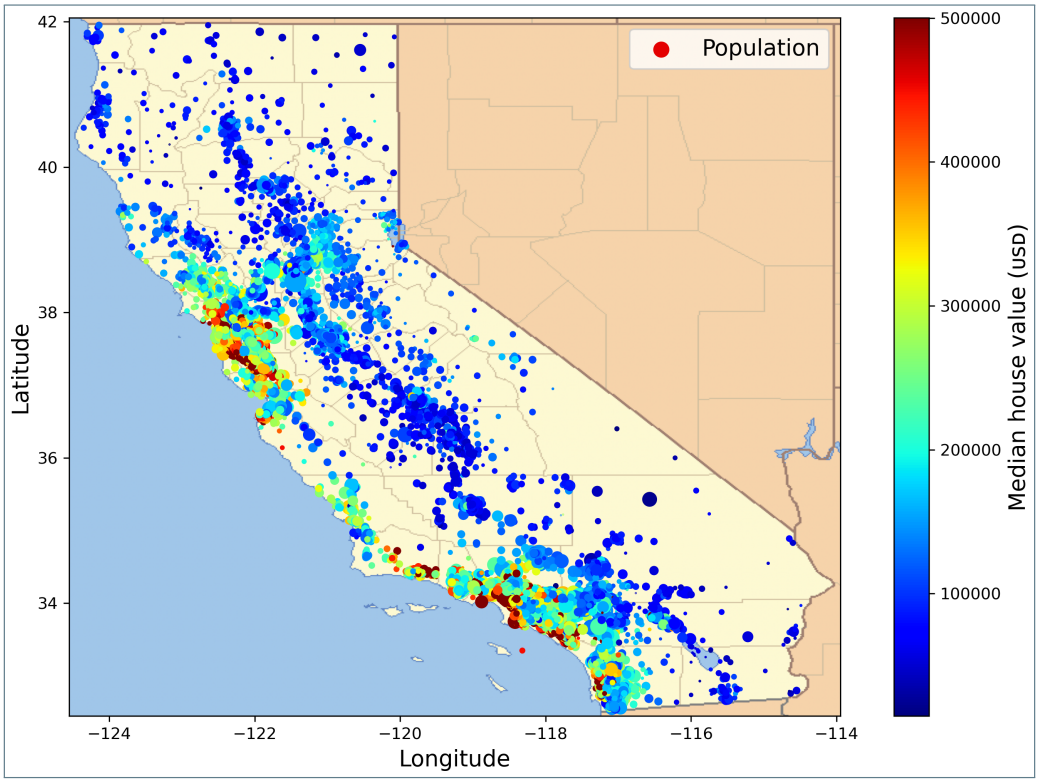

图 2-1: 加利福尼亚州房价（California housing prices）

[1]原始数据集见 R. Kelley Pace 和 Ronald Barry 的论文"Sparse Spatial Autoregressions"，Statistics & Probability Letters 33，第 3 期（1997）：291-297。

2.2　了解全局（Look at the Big Picture）

欢迎加入 Machine Learning Housing Corporation！你的第一项任务是使用加利福尼亚州的人口普查数据，建立一个预测该州房价的模型。这些数据包含加利福尼亚州每个街区组的指标，例如人口、中位数收入和中位数房价。街区组是美国人口普查局发布抽样数据时所使用的最小地理单位（一个街区组通常有 600 至 3,000 人）。为了简便起见，下面将它们称为“地区”。

你的模型应该从这些数据中学习，并且能够在给定某个地区其他所有指标的情况下，预测该地区的中位数房价。

> 提示：既然你是一名组织有序的数据科学家，第一件事应该是拿出机器学习项目检查清单。你可以从 https://homl.info/checklist 上的清单开始，它应该适用于大多数机器学习项目，但一定要根据自己的需要进行调整。本章会介绍其中很多清单项目，也会跳过一些项目，要么因为它们不言自明，要么因为会在后续章节中讨论。

划定问题　首先要问老板的问题是：业务目标究竟是什么？建立模型可能并不是最终目标。公司打算如何使用这个模型，又能从中获得什么收益？了解目标非常重要，因为它会决定你如何定义问题、选择哪些算法、使用什么性能指标评估模型，以及愿意花多少精力对模型进行调整。

老板回答说，模型的输出（对某个地区中位数房价的预测）对于判断是否值得投资某个地区至关重要。更具体地说，模型的输出会与其他信号一起，输入到另一个机器学习系统中（见图 2-2）。[2] 因此，我们必须尽可能准确地建立房价模型。

接下来要问老板的问题是：目前的解决方案是什么样的（如果存在的话）？现状通常能为性能提供参照，也能让你获得解决问题的启发。老板回答说，目前地区房价由专家人工估算：一个团队收集某个地区的最新信息；如果无法获得中位数房价，就使用复杂的规则估算。当他们后来设法知道实际的中位数房价时，往往会发现估计值偏差超过 30%。

[2] 输入机器学习系统的一条信息通常称为信号，这是借用了 Claude Shannon 的信息论术语。他在贝尔实验室发展了这套理论，用于改进电信系统。其核心思想是：应当具有较高的信噪比。

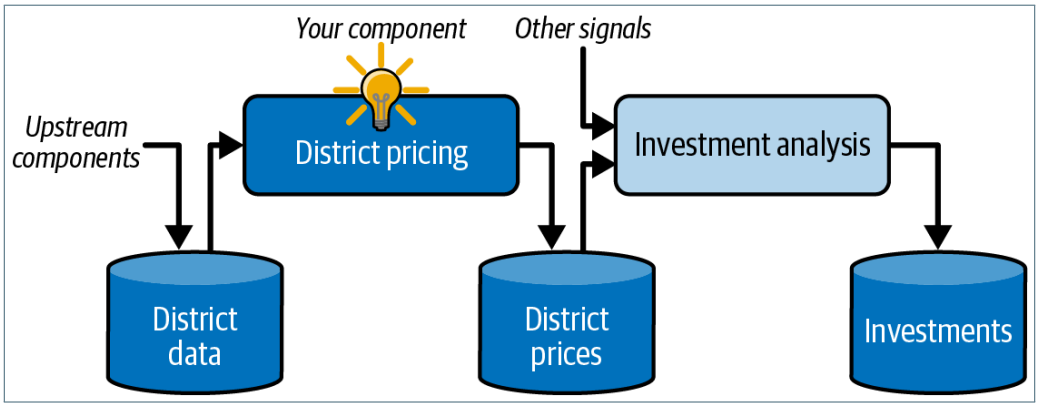

图 2-2: 房地产投资的机器学习流水线（A machine learning pipeline for real estate investments）

这种方法既昂贵又耗时，而且估计结果并不理想；当公司设法查到实际的中位数房价时，经常发现估计值偏差超过 30%。因此，公司认为，利用一个模型，根据某个地区的其他数据来预测其中位数房价，应该会有帮助。人口普查数据看起来很适合这个目的，因为它包含数千个地区的中位数房价以及其他相关数据。

数据处理流水线　一系列数据处理组件被称为数据流水线（data pipeline）。流水线在机器学习系统中非常常见，因为系统需要处理大量数据，并应用许多数据转换。

各组件通常是异步运行的。每个组件获取大量数据，处理这些数据，再把结果写入另一个数据存储。过一段时间后，流水线中的下一个组件读取这些数据，并输出自己的结果。每个组件都相当独立：组件之间的接口只是数据存储。借助数据流图，这种系统很容易理解，不同团队也可以分别关注不同组件。此外，如果某个组件发生故障，下游组件通常仍可以暂时使用该故障组件的最后一次输出，继续正常运行。因此，这种架构具有很强的稳健性。

另一方面，如果没有实现适当的监控，某个发生故障的组件可能长时间不被发现。数据会逐渐过时，整个系统的性能也会下降。

有了这些信息，你已经可以开始设计系统了。首先，确定模型需要哪一种训练监督：这是监督学习、无监督学习、半监督学习、自监督学习，还是强化学习任务？它是分类任务、回归任务，还是其他任务？应该使用批量学习还是在线学习技术？在继续阅读之前，先停下来尝试自己回答这些问题。

想好答案了吗？我们来看看。这显然是一个典型的监督学习任务，因为模型可以使用带标签的示例进行训练（每个实例都带有预期输出，也就是该地区的中位数房价）。这也是一个典型的回归任务，因为模型需要预测一个数值。更具体地说，这是一个多元回归问题，因为系统将使用多个特征进行预测（地区人口、中位数收入

等）。同时，它又是单变量回归问题，因为我们只想为每个地区预测一个值。如果要为每个地区预测多个值，那就属于多变量回归问题。最后，由于没有连续不断的数据流进入系统，没有特别需要快速适应数据变化，而且数据量足够小，可以装入内存，因此直接使用批量学习就足够了。

> 提示：如果数据量非常大，可以使用 MapReduce 技术，把批量学习工作分配到多台服务器上；也可以使用在线学习技术。

选择性能指标　下一步是选择性能指标。回归问题常用的性能指标是均方根误差（root mean squared error，RMSE）。它可以反映系统通常会产生多大的误差，并且会对较大的误差赋予更高权重。公式 2-1 给出了计算 RMSE 的数学公式。

$$\text{RMSE}(X, y, h) = \sqrt{\frac{1}{m}\sum_{i=1}^{m}\left(h\left(\mathbf{x}^{(i)}\right) - y^{(i)}\right)^2} \tag{2-1}$$

公式 2-1　均方根误差（root mean squared error，RMSE）

记号　这个公式引入了本书后面会反复使用的一些常见机器学习记号：

- m 是用于计算 RMSE 的数据集中的实例数量。
- 例如，如果要在包含 2,000 个地区的验证集上评估 RMSE，那么 $m = 2{,}000$。
- $\mathbf{x}^{(i)}$ 是数据集中第 i 个实例的所有特征值组成的向量（不包括标签），而 $y^{(i)}$ 是它的标签，也就是该实例期望的输出值。y 是包含数据集中所有实例标签的向量。
- 例如，如果数据集中的第一个地区位于经度 -118.29°、纬度 33.91°，有 1,416 名居民，中位数收入为 38,372 美元，中位数房价为 156,400 美元（暂时忽略其他特征），那么 $\mathbf{x}^{(1)} = (-118.29, 33.91, 1416, 38372)^\intercal$，而 $y^{(1)} = 156400$。
- $\mathbf{X}$ 是包含所有实例特征值（不包括标签）的矩阵。每个实例对应一行，第 i 行等于 $(\mathbf{x}^{(i)})^\intercal$。[3]
- 例如，对于上面描述的第一个地区，$\mathbf{X}$ 的第一行就是 $(-118.29, 33.91, 1416, 38372)$。
- h 是系统的预测函数，也称为假设（hypothesis）。当系统接收到某个实例的特征向量 $\mathbf{x}^{(i)}$ 时，会输出该实例的预测值 $\hat{y}^{(i)} = h(\mathbf{x}^{(i)})$。$\hat{y}$ 读作“y-hat”。

[3] 转置运算可以把列向量转换为行向量，反之亦然。

- 例如，如果系统预测第一个地区的中位数房价为 158,400 美元，那么 $\hat{y}^{(1)} = h(\mathbf{x}^{(1)}) = 158400$。该地区的预测误差为 $\hat{y}^{(1)} - y^{(1)} = 2000$。
- RMSE($\mathbf{X}, y, h$) 是使用假设 h 在一组示例上测得的代价函数。

标量值（例如 m 或 $y^{(i)}$）以及函数名（例如 h）使用小写斜体；向量（例如 $\mathbf{x}^{(i)}$）使用小写粗体；矩阵（例如 $\mathbf{X}$）使用大写粗体。

虽然 RMSE 通常是回归任务的首选性能指标，但在数据中存在许多异常值时，你可能更喜欢使用其他函数，因为 RMSE 对异常值非常敏感。此时可以考虑公式 2-2 所示的平均绝对误差（mean absolute error，MAE，也称平均绝对偏差）。

$$\text{MAE}(X, y, h) = \frac{1}{m}\sum_{i=1}^{m}\left|h\left(\mathbf{x}^{(i)}\right) - y^{(i)}\right| \tag{2-2}$$

公式 2-2　平均绝对误差（mean absolute error，MAE）

RMSE 和 MAE 都是在衡量两个向量之间的距离：预测值向量与目标值向量之间的距离。可以使用多种距离指标，也就是范数：

- 对平方和取平方根（RMSE）对应欧几里得范数，也就是我们最熟悉的距离概念。它也称为 ℓ_2 范数，记作 $\|\cdot\|_2$（或简写为 $\|\cdot\|$）。
- 取绝对值之和（MAE）对应 ℓ_1 范数，记作 $\|\cdot\|_1$。它有时也称为曼哈顿范数，因为当你只能沿着城市街区的正交道路行走时，它衡量的就是城市中两点之间的距离。
- 更一般地，包含 n 个元素的向量 $\mathbf{v}$ 的 ℓ_k 范数定义为 $\|\mathbf{v}\|_k = (|v_1|^k + |v_2|^k + \cdots + |v_n|^k)^{1/k}$。$\ell_0$ 给出向量中的非零元素数量，ℓ_∞ 给出绝对值的最大值。

范数的指标越高，就越关注较大的值，而忽略较小的值。这就是 RMSE 比 MAE 对异常值更敏感的原因。不过，当异常值呈指数级稀少（例如数据服从钟形曲线）时，RMSE 的表现非常好，通常也是首选指标。

检查假设　最后，最好列出并验证截至目前所作出的假设（无论这些假设是你自己提出的，还是来自其他人）。这有助于尽早发现严重问题。例如，你的系统输出的地区房价将被输入下游机器学习系统，而你假设这些房价会以数值形式被使用。但如果下游系统把房价转换成“便宜”“中等”或“昂贵”等类别，然后使用这些类别，而不是房价本身呢？在这种情况下，是否精确预测房价根本不重要；你的系统只需准确判断

类别即可。这样一来，问题就应该被定义为分类任务，而不是回归任务。你肯定不希望在回归系统上工作数月之后才发现这一点。

幸运的是，与下游系统负责团队沟通后，你确认他们确实需要实际房价，而不仅仅是类别。很好，所有条件都已具备，现在可以开始编码了！

2.3　获取数据（Get the Data）

是时候亲自动手了。不要犹豫，拿起笔记本电脑，跟着代码示例一起操作。正如前言中所说，本书中的所有代码示例都是开源的，并且以 Jupyter notebook 的形式在线提供。Jupyter notebook 是包含文本、图像和可执行代码片段的交互式文档（本书使用 Python）。本书假设你在 Google Colab 上运行这些 notebook。Google Colab 是一项免费服务，可以让你直接在线运行 Jupyter notebook，而无需在计算机上安装任何东西。如果希望使用其他在线平台（例如 Kaggle），或希望在自己的计算机上本地安装全部环境，请参阅本书 GitHub 页面上的说明。

2.3.1　使用 Google Colab 运行代码示例（Running the Code Examples with Google Colab）

首先，打开网页浏览器并访问 https://homl.info/colab-p。该网址会把你带到 Google Colab，并显示本书的 Jupyter notebook 列表（见图 2-3）。每章都有一个 notebook，此外还有一些关于 NumPy、Matplotlib、Pandas、线性代数和微积分的额外 notebook 与教程。例如，如果点击 **02_end_to_end_machine_learning_project.ipynb**，第 2 章的 notebook 就会在 Google Colab 中打开（见图 2-4）。

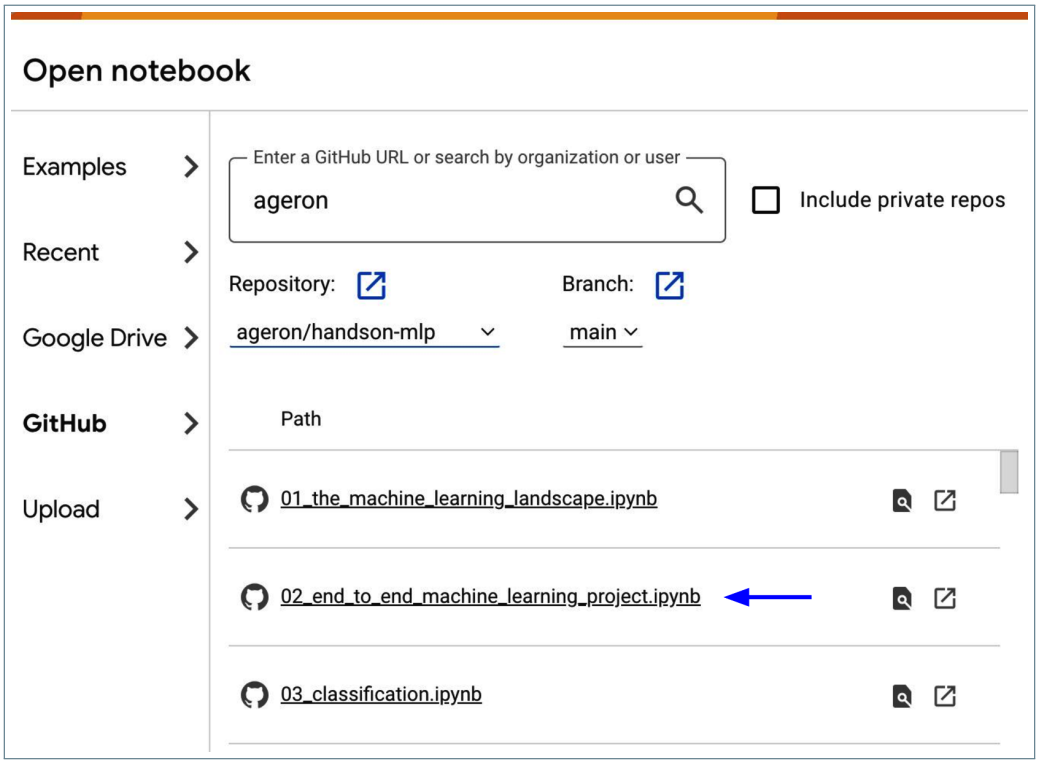

图 2-3: Google Colab 中的 notebook 列表（List of notebooks in Google Colab）

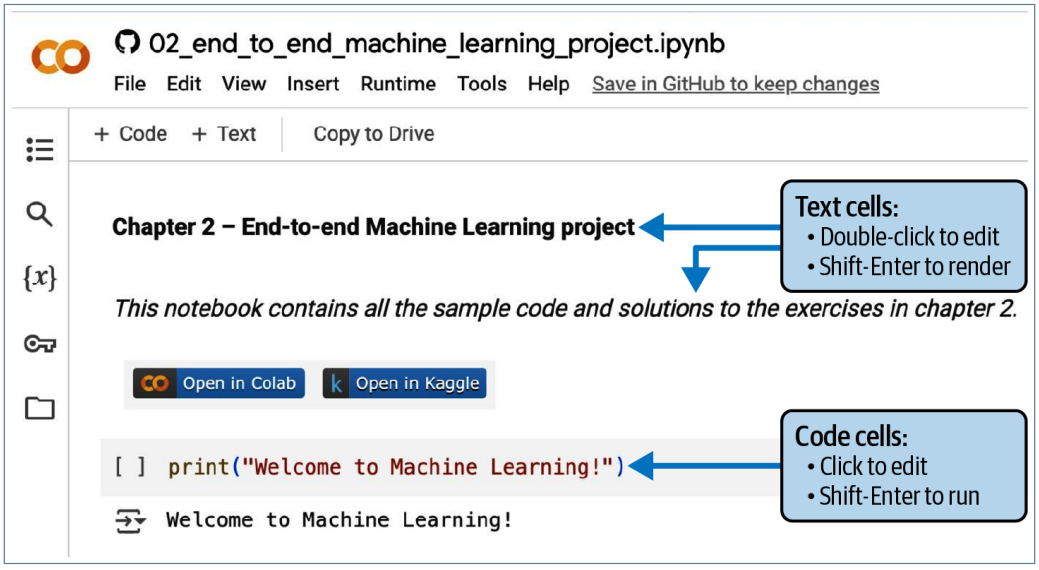

图 2-4: Google Colab 中的 notebook（Your notebook in Google Colab）

Jupyter notebook 由一系列单元格组成。每个单元格包含可执行代码或文本。试着双击第一个文本单元格（其中包含“Welcome to Machine Learning Housing Corp.!”

这句话）。这样会打开该单元格供编辑。注意，Jupyter notebook 使用 Markdown 语法进行格式化（例如 `**bold**`、`*italics*`、`# Title`、`[url](link text)` 等）。尝试修改这段文本，然后按 Shift-Enter 查看结果。

接下来，通过菜单中的 Insert → “Code cell”创建一个新的代码单元格。也可以点击工具栏中的“+ Code”按钮，或者将鼠标悬停在某个单元格底部，直到出现“+ Code”和“+ Text”，然后点击“+ Code”。在新代码单元格中输入一些 Python 代码，例如 `print("Hello World")`，然后按 Shift-Enter 运行代码（也可以点击单元格左侧的运行按钮）。

如果没有登录 Google 账户，此时系统会要求你登录。如果还没有 Google 账户，需要先创建一个。登录后，当你尝试运行代码时，会看到一条安全警告，提示该 notebook 未经 Google 认证。恶意用户可能创建一个 notebook，诱骗你输入 Google 凭据，从而访问个人数据。因此，在运行 notebook 之前，一定要确认信任其作者（或者逐个检查代码单元格在运行什么）。如果你信任作者（或者打算检查每个代码单元格），现在就可以点击“Run anyway”。

Colab 随后会为你分配一个运行时：这是一台位于 Google 服务器上的免费虚拟机，其中包含大量工具和 Python 库，包括大多数章节所需的全部内容（个别章节需要先运行命令安装额外库）。这需要几秒钟。接着，Colab 会自动连接到这个运行时，并使用它执行新的代码单元格。重要的是，代码运行在该运行时上，而不是你的计算机上。现在，你已经在 Colab 上运行了 Python 代码！

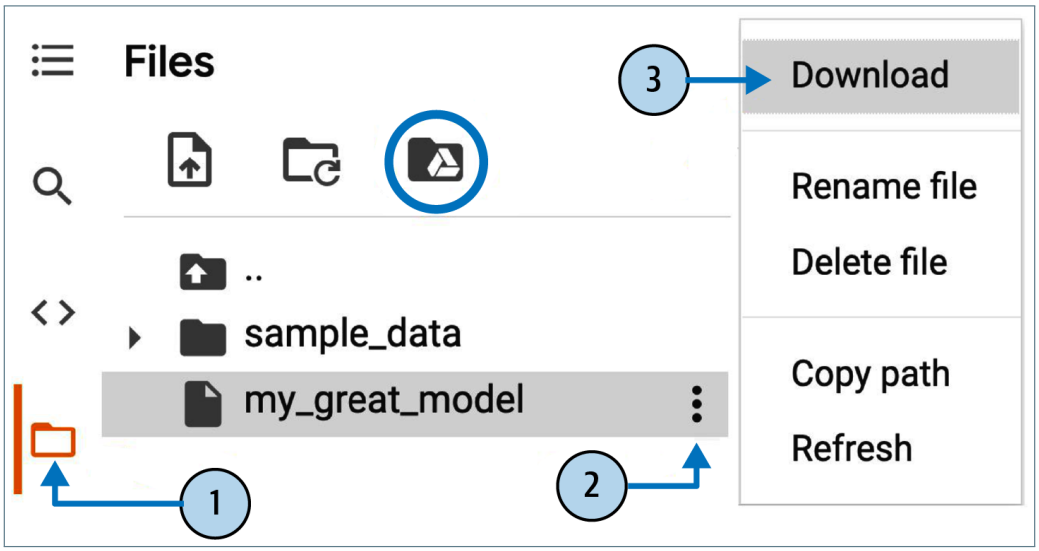

图 2-5: 从 Google Colab 运行时下载文件（步骤 1 至 3），或挂载 Google Drive（Download a file from a Google Colab runtime, or mount your Google Drive）

> 提示：插入新代码单元格还可以按 Ctrl-M（macOS 上为 Cmd-M），再按 A 将单元格插入活动单元格上方，或按 B 将单元格插入下方。还有许多其他键盘快捷键，可以先按 Ctrl-M（macOS 上为 Cmd-M），再按 H 查看和编辑快捷键。如果选择在 Kaggle 上运行 notebook，或者在自己的计算机上使用 JupyterLab 或带 Jupyter 扩展的 Visual Studio Code 运行 notebook，你会看到一些细微差异，例如运行时被称为内核，用户界面和键盘快捷键略有不同等。但从一个 Jupyter 环境切换到另一个环境并不困难。

熟悉 Colab 之后，就可以下载数据了。

2.3.2　保存代码变更和数据（Saving Changes and Data）

你可以修改 Colab notebook，并且只要浏览器标签页保持打开，修改就会一直存在。但一旦关闭标签页，修改就会丢失。为避免这种情况，应通过选择 File → “Save a copy in Drive”，把 notebook 的副本保存到 Google Drive。也可以通过 File → Download → “Download .ipynb”把 notebook 下载到计算机上。之后，可以访问 https://colab.research.google.com，从 Google Drive 中重新打开 notebook，或者从计算机上传 notebook。

默认情况下，Google Drive 会挂载到 `/content/drive/My Drive`。如果要备份数据文件，只需在代码单元格中运行下面的命令：

```
!cp /content/my_great_model /content/drive/MyDrive
```

以感叹号开头的命令会被当作 shell 命令，而不是 Python 代码。这里的 **cp** 是 Linux shell 中用于把文件从一个路径复制到另一个路径的命令。需要注意，Colab 运行时运行在 Linux（具体来说是 Ubuntu）上。

2.3.3　交互性的力量与风险（The Power and Risk of Interactivity）

Jupyter notebook 具有交互性，这是非常有用的特性：你可以逐个运行单元格，在任何地方暂停，插入单元格，随意修改，再回过头来重新运行某个单元格。我非常鼓励你这样做。如果只是按顺序运行单元格，从不动手尝试，学习速度会慢很多。

然而，这种灵活性也有代价：你很容易以错误的顺序运行单元格，或者忘记运行某个单元格。这样一来，后续代码单元格可能会失败。例如，每个 notebook 的第一个代码单元格都包含设置代码（例如导入库），所以一定要先运行它，否则后面的代码都无法工作。

> 提示：如果遇到奇怪的错误，可以尝试通过 Runtime → “Restart runtime”重启运行时，然后从头开始运行所有单元格。这通常可以解决问题。如果仍然无法解决，很可能是你所做的某项修改破坏了 notebook：只需恢复原始 notebook，再试一次。如果这样仍然不行，请在 GitHub 上提交 issue。

2.3.4　书中代码与 notebook 代码（Book Code and Notebook Code）

有时你会注意到，本书中的代码与 notebook 中的代码存在一些小差异，原因可能有以下几种：

- 当你读到这些内容时，某个库可能已经发生了轻微变化；或者，尽管我已经尽力，书中的代码仍可能存在错误。遗憾的是，我无法神奇地修复你手中这本书的代码（除非你阅读的是电子版，并且可以下载最新版本），但我可以修复 notebook。因此，如果你从书中复制代码后遇到错误，请在 notebook 中查找修正后的代码。我会尽力让 notebook 中的代码没有错误，并保持与最新库版本兼容。
- notebook 中包含一些用于美化图形的额外代码（添加标签、设置字体大小等），以及用于以高分辨率保存图形的代码。你可以放心忽略这些额外代码。

我对代码进行了可读性优化，使其尽可能线性、扁平，只定义很少的函数或类。目标是让你通常能在眼前直接看到正在运行的代码，而不是必须在多层抽象中寻找代码。这也让你更容易动手修改代码。为简单起见，错误处理比较有限；此外，我把一些不常用的导入放在实际需要它们的位置，而不是放在文件顶部（PEP 8 Python 风格指南建议把导入放在顶部）。不过，你的生产代码不会有太大不同：只需变得更加模块化，并增加测试和错误处理即可。

好了，熟悉 Colab 后，就可以下载数据了。

2.3.5　下载数据（Download the Data）

在典型环境中，数据可能位于关系数据库或其他常见数据存储中，并分散在多个表、文档或文件中。要访问这些数据，首先需要获得凭据和访问授权，并熟悉数据模式。[4] 但在本项目中，事情简单得多：只需下载一个压缩文件 **housing.tgz**，它包含一个名为 **housing.csv** 的逗号分隔值（CSV）文件，其中包含全部数据。

与其手动下载并解压，不如编写一个函数自动完成这些工作。如果数据会定期更新，这尤其有用：你可以编写一个小脚本，使用这个函数获取最新数据，或者设置定时任务，让它按固定间隔自动运行。需要把数据集安装到多台计算机上时，自动化获取数据的过程也很有帮助。

下面是获取并加载数据的函数：

```
from pathlib import Path
import pandas as pd
import tarfile
import urllib.request

def load_housing_data():
    tarball_path = Path("datasets/housing.tgz")
    if not tarball_path.is_file():
        Path("datasets").mkdir(parents=True, exist_ok=True)
        url = "https://github.com/ageron/data/raw/main/housing.tgz"
        urllib.request.urlretrieve(url, tarball_path)
        with tarfile.open(tarball_path) as housing_tarball:
            housing_tarball.extractall(path="datasets", filter="data")
    return pd.read_csv(Path("datasets/housing/housing.csv"))

housing_full = load_housing_data()
```

[4] 还可能需要检查法律限制，例如不应复制到不安全数据存储中的私人字段。

> 注意：如果使用 Python 3.12 或 3.13，应在 `extractall()` 方法的参数中添加 `filter='data'`。这样可以限制解压算法的能力，从而提高安全性；详见相关文档。

调用 `load_housing_data()` 时，它会查找 `datasets/housing.tgz` 文件。如果找不到，就在当前目录中创建 `datasets` 目录（在 Colab 中，当前目录默认为 `/content`），从 ageron/data GitHub 仓库下载 `housing.tgz`，并将其内容解压到 `datasets` 目录中；这会创建 `datasets/housing` 目录，其中包含 `housing.csv` 文件。最后，函数将这个 CSV 文件加载到包含全部数据的 Pandas DataFrame 对象中，并返回该对象。

2.3.6　快速查看数据结构（Take a Quick Look at the Data Structure）

首先，使用 DataFrame 的 `head()` 方法查看前五行数据（见图 2-6）。

```
housing.head()
```

	longitude	latitude	housing_median_age	median_income	ocean_proximity	median_house_value
0	-122.23	37.88	41.0	8.3252	NEAR BAY	452600.0
1	-122.22	37.86	21.0	8.3014	NEAR BAY	358500.0
2	-122.24	37.85	52.0	7.2574	NEAR BAY	352100.0
3	-122.25	37.85	52.0	5.6431	NEAR BAY	341300.0
4	-122.25	37.85	52.0	3.8462	NEAR BAY	342200.0

图 2-6: 数据集的前五行（Top five rows in the dataset）

每一行代表一个地区。数据共有 10 个属性（截图中没有全部显示）：`longitude`、`latitude`、`housing_median_age`、`total_rooms`、`total_bedrooms`、`population`、`households`、`median_income`、`median_house_value` 和 `ocean_proximity`。

`info()` 方法可以快速描述数据，尤其可以查看总行数、每个属性的类型以及非空值数量：

```
>>> housing_full.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           20640 non-null  float64
 1   latitude            20640 non-null  float64
 2   housing_median_age  20640 non-null  float64
```

```
 3   total_rooms         20640 non-null  float64
 4   total_bedrooms      20433 non-null  float64
 5   population          20640 non-null  float64
 6   households          20640 non-null  float64
 7   median_income       20640 non-null  float64
 8   median_house_value  20640 non-null  float64
 9   ocean_proximity     20640 non-null  object
dtypes: float64(9), object(1)
memory usage: 1.6+ MB
```

> 注意：本书中，如果代码示例像上面这样同时包含代码和输出，为了提高可读性，会按照 Python 解释器的格式排版：代码行前有 `>>>`（缩进代码块使用`...`），输出行没有前缀。

数据集共有 20,640 个实例。按照机器学习的标准，这个数据集相当小，但非常适合入门。你会注意到，`total_bedrooms` 属性只有 20,433 个非空值，这意味着有 207 个地区缺少这个特征。稍后需要处理这些缺失值。

除了 `ocean_proximity` 以外，所有属性都是数值型的。它的类型是 `object`，因此可以存放任意 Python 对象。但由于数据来自 CSV 文件，你知道它实际上应该是文本属性。

查看前五行时，你可能已经注意到，`ocean_proximity` 列中的值反复出现，这说明它很可能是一个类别属性。可以使用 `value_counts()` 方法找出有哪些类别，以及每个类别包含多少个地区：

```
>>> housing_full["ocean_proximity"].value_counts()
ocean_proximity
<1H OCEAN     9136
INLAND        6551
NEAR OCEAN    2658
NEAR BAY      2290
ISLAND           5
Name: count, dtype: int64
```

再看看其他字段。`describe()` 方法可以显示数值属性的摘要信息（见图 2-7）。

`housing.describe()`

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	median_house_value
count	20640.000000	20640.000000	20640.000000	20640.000000	20433.000000	20640.000000
mean	-119.569704	35.631861	28.639486	2635.763081	537.870553	206855.816909
std	2.003532	2.135952	12.585558	2181.615252	421.385070	115395.615874
min	-124.350000	32.540000	1.000000	2.000000	1.000000	14999.000000
25%	-121.800000	33.930000	18.000000	1447.750000	296.000000	119600.000000
50%	-118.490000	34.260000	29.000000	2127.000000	435.000000	179700.000000
75%	-118.010000	37.710000	37.000000	3148.000000	647.000000	264725.000000
max	-114.310000	41.950000	52.000000	39320.000000	6445.000000	500001.000000

图 2-7: 各数值属性的摘要（Summary of each numerical attribute）

`count`、`mean`、`min` 和 `max` 行的含义不难理解。注意，空值会被忽略（例如，`total_bedrooms` 的计数是 20,433，而不是 20,640）。`std` 行显示标准差，它衡量的是数据值的离散程度。[5]

`25%`、`50%` 和 `75%` 行显示相应的百分位数：百分位数表示一组观测值中，有给定百分比的观测值低于该数值。例如，25% 的地区 `housing_median_age` 低于 18，50% 低于 29，75% 低于 37。它们通常分别称为第 25 百分位数（或第一四分位数）、中位数和第 75 百分位数（或第三四分位数）。

想要大致了解数据类型，另一个快捷方法是为每个数值属性绘制直方图。直方图在纵轴上显示实例数量，在横轴上显示取值范围。可以一次绘制一个属性，也可以对整个数据集调用 `hist()` 方法，如下面的代码所示（见图 2-8）。

[5]标准差通常记作 σ，是方差的平方根，而方差是各个值与均值之差的平方的平均值。对于钟形的正态分布（也称高斯分布），适用“68-95-99.7”规则：约 68% 的值落在均值的 1σ 范围内，95% 落在 2σ 范围内，99.7% 落在 3σ 范围内。

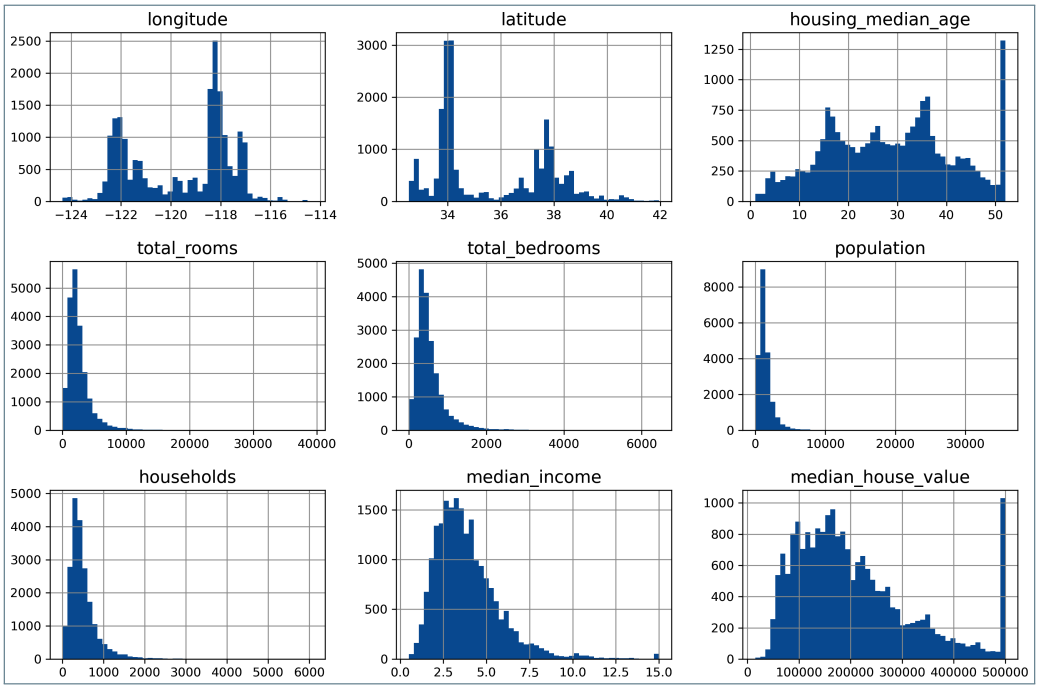

图 2-8: 每个数值属性的直方图（A histogram for each numerical attribute）

可以使用 **bins** 参数调整取值范围的数量；你可以尝试修改它，观察它如何影响直方图：

```
import matplotlib.pyplot as plt

housing_full.hist(bins=50, figsize=(12, 8))
plt.show()
```

观察这些直方图，可以发现几件事：

首先，**median_income** 属性看起来并不是用美元表示的。向收集数据的团队确认后，你得知这个数据已经经过缩放，并且对较高的中位数收入封顶为 15（准确值是 15.0001），对较低的中位数收入封底为 0.5（准确值是 0.4999）。这些数字大致代表数万美元，例如 3 实际上约等于 30,000 美元。在机器学习中，使用预处理过的属性很常见，这不一定是问题，但你应当尽力理解数据的计算方式。

housing_median_age 和 **median_house_value** 也经过了封顶处理。后者可能是一个严重问题，因为它是目标属性，也就是标签。机器学习算法可能会学到房价永远不会超过这个上限。你需要与客户团队（将使用系统输出的团队）确认这是否构成问题。如果他们需要精确预测超过 500,000 美元的房价，就有两个选择：

- 为标签被封顶的地区收集正确的标签。
- 从训练集中删除这些地区（同时也要从测试集中删除，因为如果系统预测出超过 500,000 美元的值，就不应因此得到较差的评估结果）。

这些属性的尺度差异很大。本章稍后讨论特征缩放时，会进一步说明这一点。

最后，许多直方图呈右偏分布：向右延伸的距离远大于向左延伸的距离。这可能使某些机器学习算法更难检测模式。稍后会尝试对这些属性进行变换，使它们具有更加对称、接近钟形的分布。

现在，你应该已经更加了解正在处理的数据类型了。

2.3.7　创建测试集（Create a Test Set）

在继续查看数据之前，需要先创建测试集，将它放在一边，并且永远不要查看它。在这个阶段主动留出部分数据似乎有些奇怪。毕竟，你目前只粗略看过数据，按理说应该在决定使用哪些算法之前进一步了解数据，对吧？这当然没错，但人的大脑非常擅长发现模式，也因此极易过拟合：如果查看测试集，可能会偶然发现测试数据中某个看似有趣的模式，从而选择某一类机器学习模型。当你使用测试集估计泛化误差时，估计结果会过于乐观，最终启动的系统也不会像预期那样表现良好。这种现象称为数据窥探偏差（data snooping bias）。

创建测试集在理论上很简单：随机选择一些实例，通常是数据集的 20%（如果数据集非常大，也可以少于 20%），然后把它们放在一边：

```
import numpy as np

def shuffle_and_split_data(data, test_ratio, rng):
    shuffled_indices = rng.permutation(len(data))
    test_set_size = int(len(data) * test_ratio)
    test_indices = shuffled_indices[:test_set_size]
    train_indices = shuffled_indices[test_set_size:]
    return data.iloc[train_indices], data.iloc[test_indices]
```

接下来可以这样使用这个函数：

```
>>> rng = np.random.default_rng()  # default random number generator
>>> train_set, test_set = shuffle_and_split_data(housing_full, 0.2, rng)
>>> len(train_set)
16512
>>> len(test_set)
4128
```

这段代码可以运行，但并不完美：每次重新运行程序，都会生成不同的测试集！随着时间推移，你（或者机器学习算法）最终会看到整个数据集，而这正是需要避免的。

一种解决办法是在第一次运行时保存测试集，之后每次运行都加载它。另一种选择是设置随机数生成器的种子，例如向 `default_rng()` 传入 `seed=42`，[6] 这样每次运行程序时都能生成相同的随机数序列。

不过，数据集更新后，这两种方案都会失效。为了在更新数据集后仍保持稳定的训练集/测试集划分，一种常见做法是使用每个实例的标识符来决定它是否进入测试集，前提是实例具有唯一且不可变的标识符。比如，可以计算每个实例标识符的哈希值，如果哈希值小于或等于最大哈希值的 20%，就把这个实例放入测试集。这样，即使刷新数据集，测试集仍会保持一致。新测试集会包含新实例中的 20%，但不会包含以前曾经出现在训练集中的任何实例。

下面是一种可能的实现：

```
from zlib import crc32

def is_id_in_test_set(identifier, test_ratio):
    return crc32(np.int64(identifier)) < test_ratio * 2**32

def split_data_with_id_hash(data, test_ratio, id_column):
    ids = data[id_column]
    in_test_set = ids.apply(lambda id_: is_id_in_test_set(id_, test_ratio))
    return data.loc[~in_test_set], data.loc[in_test_set]
```

不幸的是，housing 数据集没有标识符列。最简单的解决方案是使用行索引作为 ID：

```
housing_with_id = housing_full.reset_index()  # adds an `index` column
train_set, test_set = split_data_with_id_hash(housing_with_id, 0.2, "index")
```

如果使用行索引作为唯一标识符，就必须确保新数据被追加到数据集末尾，并且永远不会删除任何行。如果做不到这一点，可以尝试使用最稳定的特征来构造唯一标识符。例如，一个地区的经纬度在数百万年内都可以认为是稳定的，因此可以像下面这样把它们组合成 ID：[7]

[6] 你经常会看到人们把随机种子设为 42。这个数字没有特殊性质，只是因为它是《银河系漫游指南》中“生命、宇宙以及一切的终极答案”。

[7] 这里的地理位置信息实际上相当粗略，因此许多地区会拥有完全相同的 ID，最终被分配到同一个集合（测试集或训练集）中。这会引入一些不理想的抽样偏差。

```
housing_with_id["id"] = (housing_full["longitude"] * 1000
                         + housing_full["latitude"])
train_set, test_set = split_data_with_id_hash(housing_with_id, 0.2, "id")
```

Scikit-Learn 提供了若干函数，可以用不同方式把数据集拆分成多个子集。最简单的函数是 **train_test_split()**，它和前面定义的 **shuffle_and_split_data()** 基本相同，但增加了一些功能。首先，它有一个 **random_state** 参数，可以设置随机数生成器的种子。其次，可以向它传入多个行数相同的数据集，它会使用相同的索引进行划分。例如，当标签存放在单独的 DataFrame 中时，这非常有用：

```
from sklearn.model_selection import train_test_split

train_set, test_set = train_test_split(housing_full, test_size=0.2,
                                       random_state=42)
```

到目前为止，我们讨论的都是纯随机抽样方法。如果数据集足够大（尤其是相对于属性数量而言足够大），这通常没有问题；但如果数据集不够大，就可能引入显著的抽样偏差。调查公司员工决定打电话询问 1,000 个人时，不会只从电话簿中随机选 1,000 人。他们会尽量确保这 1,000 人能代表总体，尤其是在调查问题涉及的维度上。例如，根据美国人口普查局的数据，达到投票年龄的公民中，女性占 51.6%，男性占 48.4%。因此，一项组织良好的美国调查会尽量维持这一比例，抽取 516 名女性和 484 名男性，至少在答案可能因性别不同而变化时如此。这种方法叫作分层抽样（stratified sampling）：先把总体划分为若干同质子群，称为层（strata），再从每个层中抽取适当数量的实例，以保证测试集能够代表总体。如果调查人员采用纯随机抽样，那么抽到女性比例低于 49% 或高于 54% 的偏斜测试集的概率超过 10%。无论哪种情况，调查结果都很可能存在偏差。

假设你与几位专家交流后得知，中位数收入是预测中位数房价时非常重要的属性。你可能希望确保测试集能代表整个数据集中不同收入类别的分布。由于中位数收入是连续数值属性，首先需要创建一个收入类别属性。再看一眼图 2-8 中的中位数收入直方图：大多数中位数收入值聚集在 1.5 到 6 附近（即 15,000 至 60,000 美元），但有些中位数收入远高于 6。数据集中的每一层都必须有足够数量的实例，否则对该层重要性的估计可能存在偏差。这意味着不应设置太多层，并且每一层都应足够大。下面的代码使用 **pd.cut()** 创建一个包含 5 个类别的收入类别属性（标签为 1 至 5）；类别 1 的范围为 0 至 1.5（即低于 15,000 美元），类别 2 的范围为 1.5 至 3，以此类推：

```
housing_full["income_cat"] = pd.cut(housing_full["median_income"],
                                    bins=[0., 1.5, 3.0, 4.5, 6., np.inf],
                                    labels=[1, 2, 3, 4, 5])
```

图 2-9 展示了这些收入类别:

```
cat_counts = housing_full["income_cat"].value_counts().sort_index()
cat_counts.plot.bar(rot=0, grid=True)
plt.xlabel("Income category")
plt.ylabel("Number of districts")
plt.show()
```

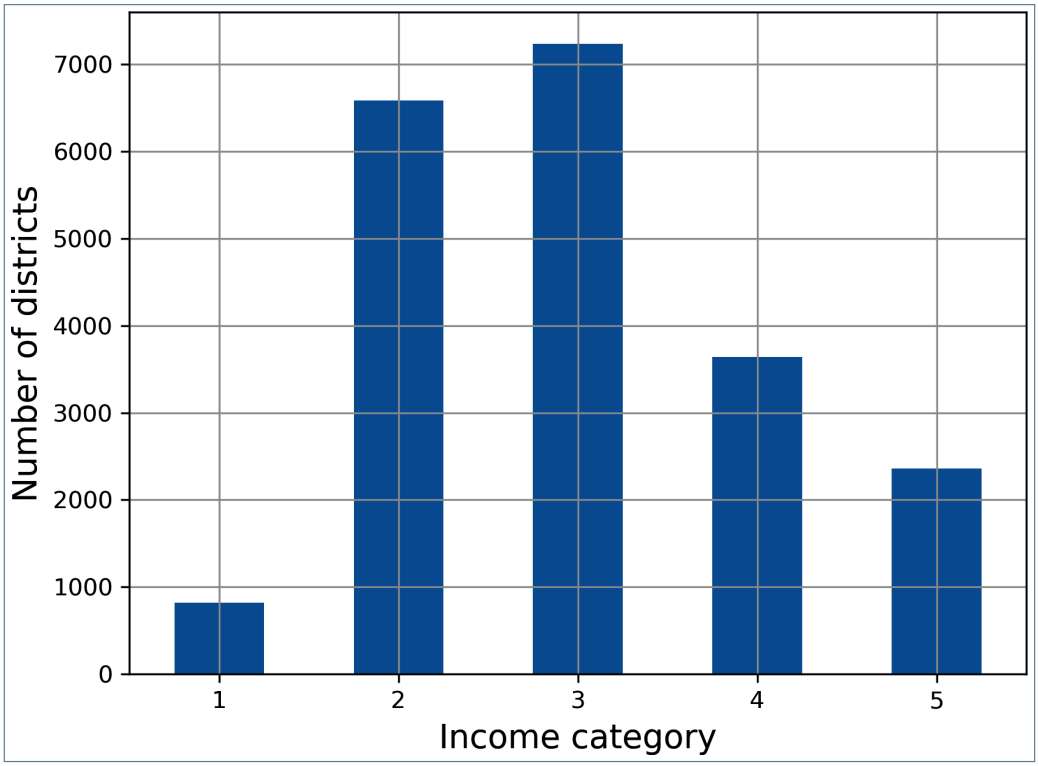

图 2-9: 收入类别直方图（Histogram of income categories）

现在可以根据收入类别进行分层抽样了。Scikit-Learn 在 **sklearn.model_selection** 包中提供了若干 splitter 类，用于实现不同的数据划分策略。每个 splitter 都有一个 **split()** 方法，返回同一数据集不同训练集/测试集划分的迭代器。

严格来说，**split()** 方法产生的是训练索引和测试索引，而不是数据本身。如果希望更好地估计模型性能，拥有多个划分会很有用，正如本章稍后讨论交叉验证时将看到的。例如，下面的代码为同一个数据集生成 10 个不同的分层划分：

```
from sklearn.model_selection import StratifiedShuffleSplit

splitter = StratifiedShuffleSplit(n_splits=10, test_size=0.2, random_state=42)
strat_splits = []
for train_index, test_index in splitter.split(housing_full,
                                              housing_full["income_cat"]):
    strat_train_set_n = housing_full.iloc[train_index]
    strat_test_set_n = housing_full.iloc[test_index]
    strat_splits.append([strat_train_set_n, strat_test_set_n])
```

目前只需使用第一个划分：

```
strat_train_set, strat_test_set = strat_splits[0]
```

或者，由于分层抽样相当常见，也可以使用带有 **stratify** 参数的 **train_test_split()** 函数，以更简洁地获得一个划分：

```
strat_train_set, strat_test_set = train_test_split(
    housing_full, test_size=0.2, stratify=housing_full["income_cat"],
    random_state=42)
```

下面确认结果是否符合预期。可以先查看测试集中各收入类别的比例：

```
>>> strat_test_set["income_cat"].value_counts() / len(strat_test_set)
income_cat
3    0.350533
2    0.318798
4    0.176357
5    0.114341
1    0.039971
Name: count, dtype: float64
```

使用类似代码，可以测量完整数据集中各收入类别的比例。图 2-10 比较了总体数据集、分层抽样生成的测试集以及纯随机抽样生成的测试集中各收入类别的比例。可以看到，分层抽样测试集的收入类别比例几乎与完整数据集一致，而纯随机抽样的测试集则出现了偏斜。

Income Category	Overall %	Stratified %	Random %	Strat. Error %	Rand. Error %
1	3.98	4.00	4.24	0.36	6.45
2	31.88	31.88	30.74	-0.02	-3.59
3	35.06	35.05	34.52	-0.01	-1.53
4	17.63	17.64	18.41	0.03	4.42
5	11.44	11.43	12.09	-0.08	5.63

图 2-10: 分层抽样与纯随机抽样的抽样偏差比较（Sampling bias comparison of stratified versus purely random sampling）

你不会再使用 `income_cat` 列，所以可以删除它，把数据恢复到原始状态：

```
for set_ in (strat_train_set, strat_test_set):
    set_.drop("income_cat", axis=1, inplace=True)
```

我们花了相当多时间生成测试集，这是有充分理由的：测试集生成往往被忽略，但它是机器学习项目中的关键环节。此外，后面讨论交叉验证时，还会用到其中许多思想。现在可以进入下一阶段：探索数据。

2.4　探索数据并获得洞见（Discover and Visualize the Data to Gain Insights）

到目前为止，你只是快速浏览数据，以便大致了解正在处理的数据类型。现在的目标是进一步深入。

首先，确认已经把测试集放在一边，并且只探索训练集。如果训练集非常大，可以抽取一个探索集，让探索阶段的各种操作更加容易、快速。在本例中，训练集相当小，因此可以直接使用完整训练集。

由于将要对完整训练集尝试各种变换，应该先复制一份原始数据，以便之后恢复：

```
housing = strat_train_set.copy()
```

2.4.1　可视化地理数据（Visualizing Geographical Data）

由于数据集包含地理信息（纬度和经度），绘制所有地区的散点图来可视化数据是一个好主意（见图 2-11）：

```
housing.plot(kind="scatter", x="longitude", y="latitude", grid=True)
plt.show()
```

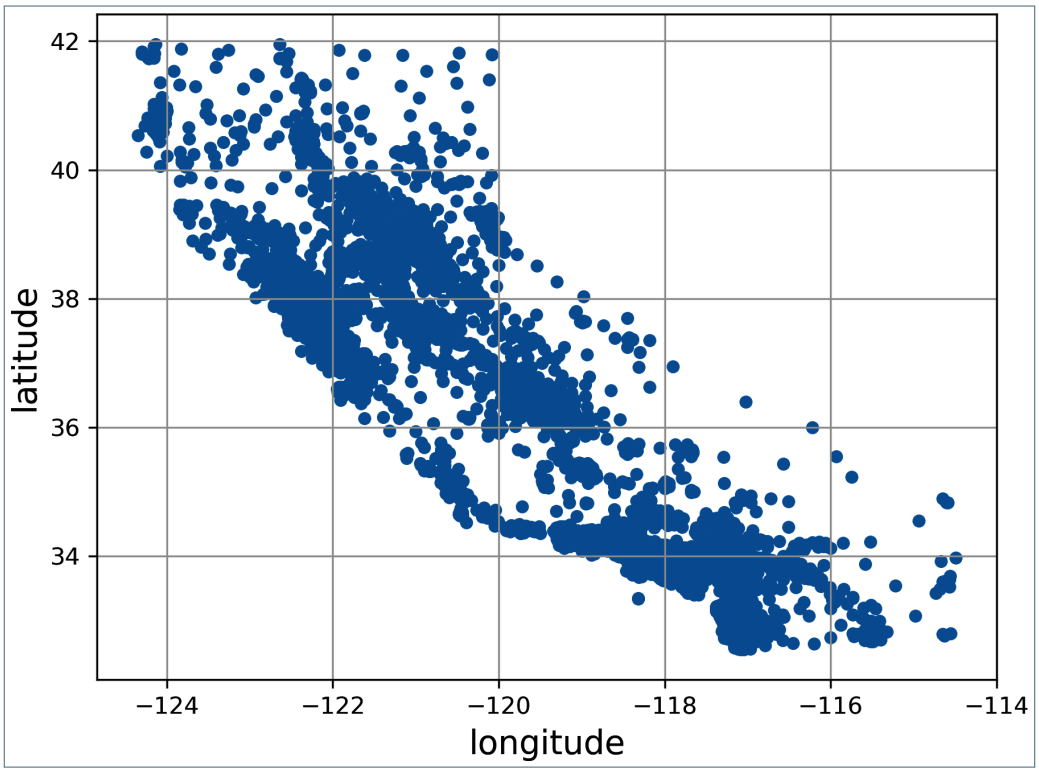

图 2-11: 数据的地理散点图（A geographical scatter plot of the data）

这看起来确实像加利福尼亚州，但除此之外很难看出什么特别的模式。把 `alpha` 选项设置为 0.2，可以更容易地看出数据点密度较高的区域（见图 2-12）：

```
housing.plot(kind="scatter", x="longitude", y="latitude", grid=True, alpha=0.2)
plt.show()
```

这好多了：可以清楚地看到高密度区域，也就是湾区、洛杉矶和圣迭戈周边，以及中央谷地中一条由许多高密度区域组成的长带（尤其是萨克拉门托和弗雷斯诺周边）。

人的大脑非常擅长从图像中发现模式，但你可能需要反复调整可视化参数，才能让模式更加明显。

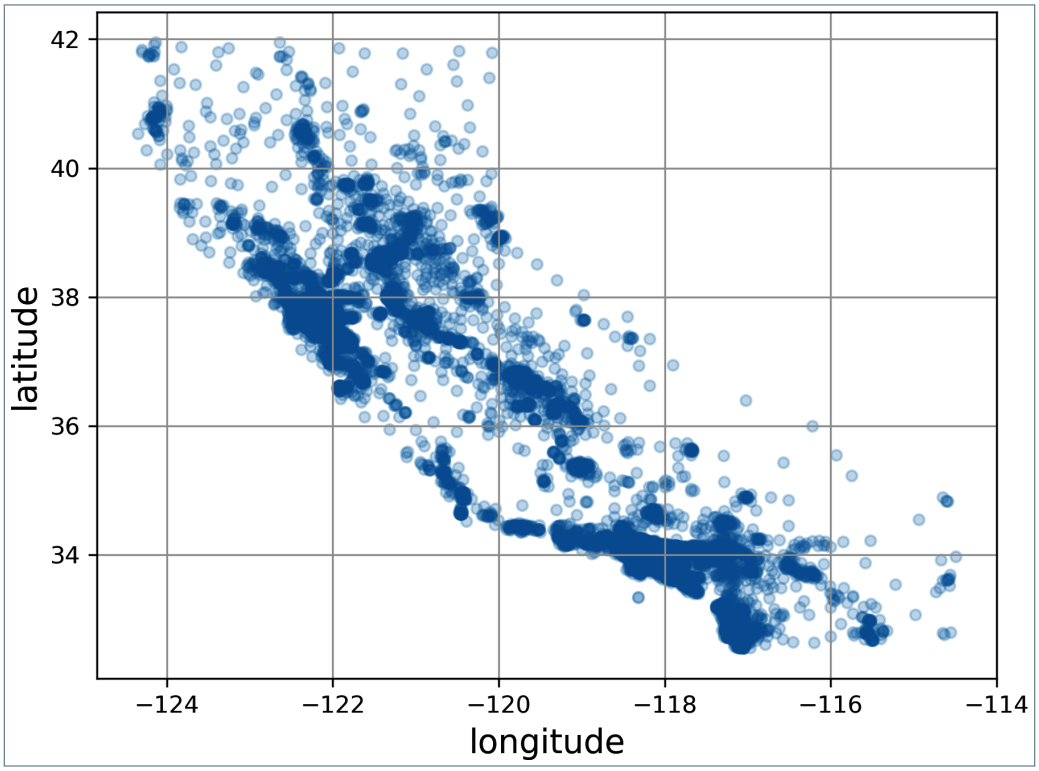

图 2-12: 突出高密度区域的更好可视化（A better visualization that highlights high-density areas）

接下来查看房价（见图 2-13）。每个圆的半径代表该地区的人口（**s** 选项），颜色代表房价（**c** 选项）。这里使用了预定义的 **jet** 颜色映射（**cmap** 选项），它从蓝色（低值）变化到红色（高房价）：[8]

[8]如果你阅读的是灰度版本，可以拿一支红笔，从湾区沿海岸线一路涂到圣迭戈附近；你大概会预料到这种结果。也可以在萨克拉门托周围涂一片黄色。

```
housing.plot(kind="scatter", x="longitude", y="latitude", grid=True,
             s=housing["population"] / 100, label="population",
             c="median_house_value", cmap="jet", colorbar=True,
             legend=True, sharex=False, figsize=(10, 7))
plt.show()
```

这张图说明，房价与地理位置（例如是否靠近海洋）以及人口密度有很大关系，这一点你可能已经猜到了。聚类算法应该有助于检测主要聚类，并添加衡量地区与聚类中心距离的新特征。**ocean_proximity** 属性可能也有用，不过在北加利福尼亚州，沿海地区房价并不算很高，因此不能简单地概括为“靠海就贵”。

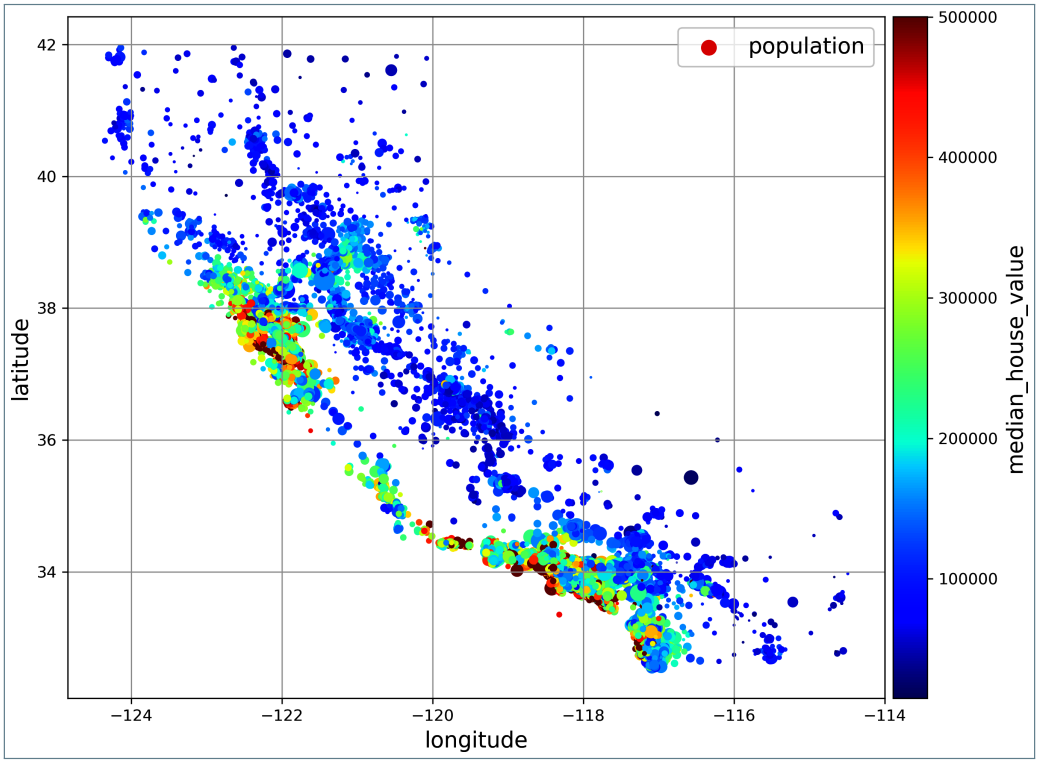

图 2-13: 加利福尼亚州房价：红色表示昂贵，蓝色表示便宜；圆越大表示人口越多（California housing prices: red is expensive, blue is cheap）

2.4.2　查找相关性（Looking for Correlations）

由于数据集不算大，可以使用 **corr()** 方法，轻松计算每一对数值属性之间的标准相关系数（也称 Pearson 的 r）：

```
corr_matrix = housing.corr(numeric_only=True)
```

现在可以查看各属性与 **median_house_value** 的相关程度：

```
>>> corr_matrix["median_house_value"].sort_values(ascending=False)
median_house_value    1.000000
median_income         0.688380
total_rooms           0.137455
housing_median_age    0.102175
households            0.071426
total_bedrooms        0.054635
population           -0.020153
longitude            -0.050859
latitude             -0.139584
Name: count, dtype: float64
```

相关系数的取值范围是 -1 到 1。接近 1 时，表示强正相关；例如，中位数收入增加时，中位数房价往往也会上升。接近 -1 时，表示强负相关；可以看到，纬度与中位数房价之间存在轻微负相关，也就是说，越往北走，房价有轻微下降趋势。最后，接近 0 的系数表示不存在线性相关。

检查属性之间相关性的另一种方式是使用 Pandas 的 **scatter_matrix()** 函数，它会绘制每个数值属性与其他每个数值属性的关系。现在有 9 个数值属性，因此会得到 9^2=81 张图，无法放在一页中。所以决定关注几个看起来与中位数房价最相关的属性（见图 2-14）：

```
from pandas.plotting import scatter_matrix

attributes = ["median_house_value", "median_income", "total_rooms",
              "housing_median_age"]
scatter_matrix(housing[attributes], figsize=(12, 8))
plt.show()
```

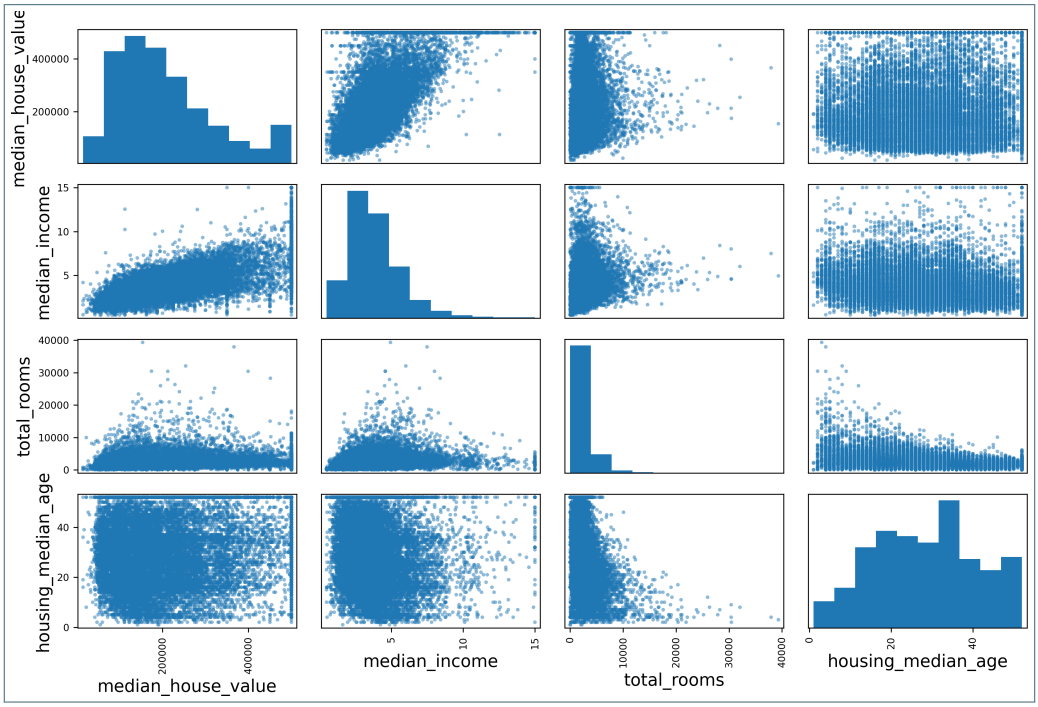

图 2-14: 数值属性散点矩阵（Scatter matrix of numerical attributes）

如果 Pandas 把每个变量与自身作图，主对角线会是一条条直线，这并没有多大用处。因此，Pandas 改为在对角线上显示每个属性的直方图（也有其他选项，详见 Pandas 文档）。

观察相关性散点图，最有希望用于预测中位数房价的属性似乎是 **median_income**，所以进一步放大这个散点图（见图 2-15）：

```
housing.plot(kind="scatter", x="median_income", y="median_house_value",
             alpha=0.1, grid=True)
plt.show()
```

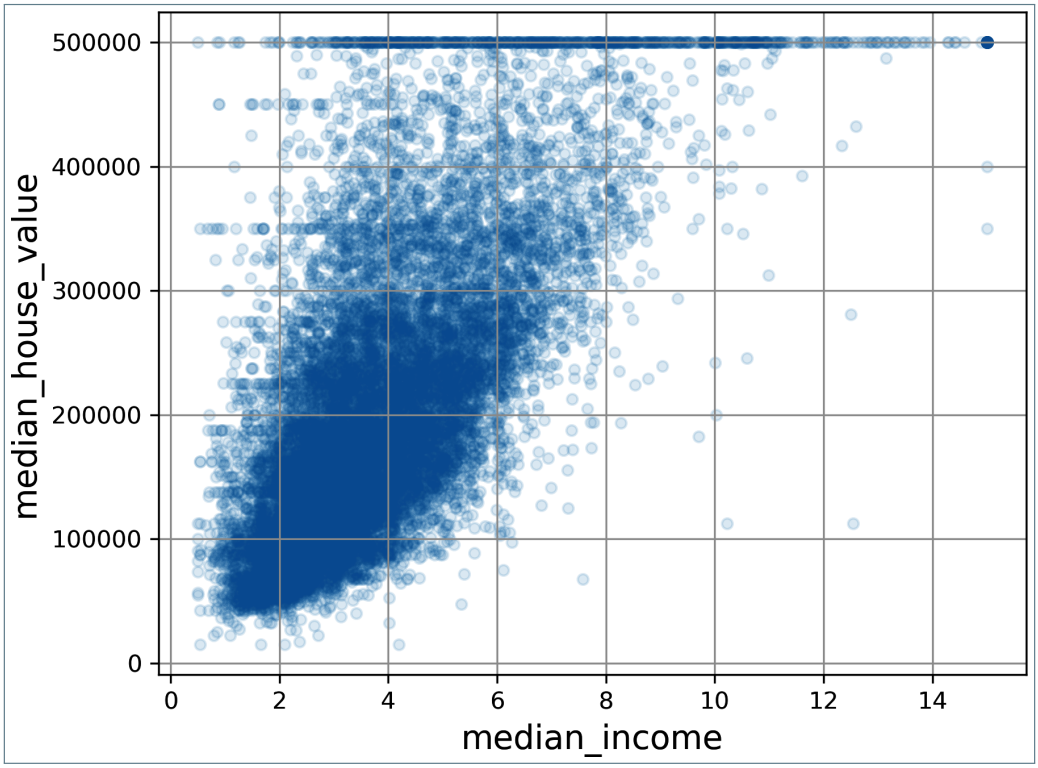

图 2-15: 中位数收入与中位数房价（Median income versus median house value）

这张图揭示了几件事。首先，相关性确实很强：虽然数据有噪声，但可以清楚看到向上的趋势。其次，前面注意到的价格上限清晰地表现为 500,000 美元处的一条水平线。但图中还显示出其他不那么明显的直线：大约 450,000 美元处一条，350,000 美元处另一条，可能还有一条位于 280,000 美元附近，下面还有几条。你可能希望删除相应的地区，防止算法学习并复现这些数据瑕疵。

注意，相关系数只衡量线性相关（“随着 x 增大，y 通常增大或减小”）。它可能完全漏掉非线性关系，例如“当 x 接近 0 时，y 通常上升”。图 2-16 展示了多个数据集及其相关系数。注意，底部一行的所有图，其相关系数都是 0，尽管坐标轴显然并不相互独立；它们就是非线性关系的例子。第二行还展示了相关系数等于 1 或 -1 的例子；注意这与斜率无关。例如，以英寸为单位的身高与以英尺或纳米为单位的身高，其相关系数都是 1。

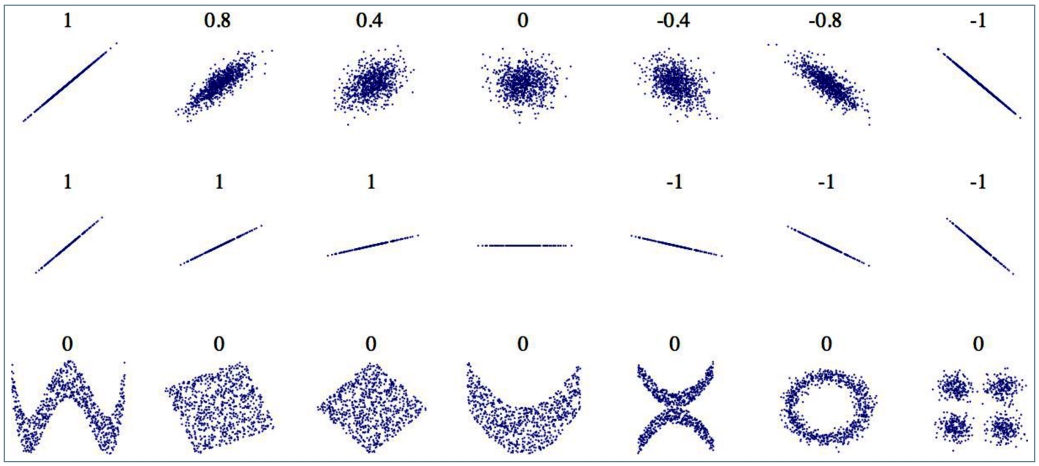

图 2-16: 不同数据集的标准相关系数（Standard correlation coefficient of various datasets）

2.4.3　试验属性组合（Experimenting with Attribute Combinations）

希望前面的内容已经让你了解了几种探索数据并获得洞见的方法。你发现了几处可能需要在把数据输入机器学习算法前清理的数据瑕疵，也发现了属性之间有趣的相关性，尤其是属性与目标属性之间的相关性。

你还注意到，有些属性的分布右偏，因此可能需要对它们进行变换，例如计算对数或平方根。当然，每个项目的实际情况都会有很大差异，但总体思路是相似的。

在为机器学习算法准备数据之前，还可以尝试各种属性组合。例如，如果不知道一个地区有多少户家庭，那么地区的房间总数并没有太大用处。真正需要的可能是每户的房间数。类似地，单独看卧室总数也不够有用；最好将它与房间总数进行比较。每户人口似乎也是一个值得研究的属性组合。可以按下面的方式创建这些新属性：

```
housing["rooms_per_house"] = housing["total_rooms"] / housing["households"]
housing["bedrooms_ratio"] = housing["total_bedrooms"] / housing["total_rooms"]
housing["people_per_house"] = housing["population"] / housing["households"]
```

然后再次查看相关矩阵：

```
>>> corr_matrix = housing.corr(numeric_only=True)
>>> corr_matrix["median_house_value"].sort_values(ascending=False)
median_house_value    1.000000
median_income         0.688380
rooms_per_house       0.143663
total_rooms           0.137455
housing_median_age    0.102175
households            0.071426
total_bedrooms        0.054635
population           -0.020153
people_per_house     -0.038224
longitude            -0.050859
latitude             -0.139584
bedrooms_ratio       -0.256397
Name: count, dtype: float64
```

效果不错！新建的 **bedrooms_ratio** 属性与中位数房价的相关性明显高于房间总数或卧室总数。它是强负相关，因此卧室与房间的比例较低的房子往往更加昂贵。每户房间数也比地区房间总数更有信息量，这很合理：房子越大，价格通常越高。

> 警告：创建组合特征时，要确保它们与现有特征不存在过强的线性相关性。共线性会给某些模型（例如线性回归）带来问题。尤其应避免对现有特征进行简单的加权求和。

这一轮探索不必绝对彻底；目的是迈出正确的第一步，迅速获得有助于构建一个相当不错的原型的洞见。但这是一个迭代过程：原型运行起来后，可以分析它的输出以获得更多洞见，再回到探索阶段。

2.5　为机器学习算法准备数据（Prepare the Data for Machine Learning Algorithms）

现在是时候为机器学习算法准备数据了。不要手动完成这些工作，而应该为此编写函数，理由有很多：

- 这样可以轻松地在任何数据集上重现这些变换，例如下一次拿到新数据集时。
- 可以逐渐建立一个转换函数库，在未来的项目中重复使用。
- 可以在实时系统中使用这些函数，先转换新数据，再把数据输入算法。

- 可以轻松尝试各种变换，观察哪种变换组合效果最好。

不过，首先要恢复一个干净的训练集（再次复制 **strat_train_set**）。还应该把预测变量和标签分开，因为你未必希望对预测变量和目标值应用相同的变换。注意，**drop()** 会创建数据副本，不会影响 **strat_train_set**：

```
housing = strat_train_set.drop("median_house_value", axis=1)
housing_labels = strat_train_set["median_house_value"].copy()
```

2.5.1　清洗数据（Data Cleaning）

大多数机器学习算法无法处理缺失特征，因此需要处理这些缺失值。例如，前面已经注意到 **total_bedrooms** 属性存在一些缺失值。可以通过三种方式解决：

1. 删除相应的地区。
2. 删除整个属性。
3. 将缺失值设置为某个值，例如 0、均值或中位数。这称为填补（imputation）。

可以使用 Pandas DataFrame 的 **dropna()**、**drop()** 和 **fillna()** 方法轻松完成这些操作：

```
housing.dropna(subset=["total_bedrooms"], inplace=True)  # option 1
housing.drop("total_bedrooms", axis=1, inplace=True)  # option 2
median = housing["total_bedrooms"].median()  # option 3
housing["total_bedrooms"] = housing["total_bedrooms"].fillna(median)
```

这里决定使用方案 3，因为它破坏性最小。但与其使用上面的代码，不如使用方便的 Scikit-Learn 类 **SimpleImputer**。它的优点是会存储每个特征的中位数，因此不仅可以对训练集填补缺失值，也可以对验证集、测试集以及之后输入模型的新数据进行填补。首先，创建一个 **SimpleImputer** 实例，并指定用每个属性的中位数替换缺失值：

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median")
```

由于只能对数值属性计算中位数，接着要创建一个只包含数值属性的数据副本（这会排除文本属性 **ocean_proximity**）：

```
housing_num = housing.select_dtypes(include=[np.number])
```

现在可以使用 `fit()` 方法，将填补器实例拟合到训练数据：

```
imputer.fit(housing_num)
```

填补器只计算了每个属性的中位数，并把结果存储在它的 `statistics_` 实例变量中。虽然目前只有 `total_bedrooms` 属性存在缺失值，但不能保证系统上线后新数据中不会出现其他缺失值，因此把填补器应用于全部数值属性会更加安全。

```
>>> imputer.statistics_
array([-118.51  ,   34.26  ,   29.    , 2125.    ,  434.    ,
       1167.    ,  408.    ,    3.5385])
>>> housing_num.median().values
array([-118.51  ,   34.26  ,   29.    , 2125.    ,  434.    ,
       1167.    ,  408.    ,    3.5385])
```

现在可以使用这个“训练好”的填补器转换训练集，用学习到的中位数替换缺失值：

```
X = imputer.transform(housing_num)
```

也可以用均值替换缺失值（`strategy="mean"`），用最频繁值替换（`strategy="most_frequent"`），或者用常量替换（`strategy="constant", fill_value=...`）。后两种策略支持非数值数据。

> 提示：`sklearn.impute` 包中还有更强大的填补器（都只用于数值特征）。`KNNImputer` 使用该特征的 k 个最近邻的均值替换每个缺失值，距离根据所有可用特征计算。`IterativeImputer` 为每个特征训练一个回归模型，根据其他所有可用特征预测缺失值；然后在更新后的数据上重新训练模型，并重复多次，在每轮迭代中不断改进模型和替换值。

2.5.2　Scikit-Learn 的设计（Scikit-Learn Design）

Scikit-Learn 的 API 设计得非常出色。以下是它的主要设计原则：[9]

[9]关于这些设计原则的更多细节，请参阅 Lars Buitinck 等人的论文“API Design for Machine Learning Software: Experiences from the Scikit-Learn Project”，arXiv 预印本 arXiv:1309.0238（2013）。

一致性　所有对象都共享一致且简单的接口：

估计器（estimators）　任何能够根据数据集估计某些参数的对象都称为估计器，例如 `SimpleImputer` 就是一个估计器。估计过程由 `fit()` 方法执行，该方法接收一个数据集作为参数；对于监督学习算法，还会接收第二个数据集，其中包含标签。用于指导估计过程的其他参数称为超参数，例如 `SimpleImputer` 的 `strategy`；它们必须设置为实例变量，通常通过构造函数参数进行设置。

转换器（transformers）　有些估计器（例如 `SimpleImputer`）还可以转换数据集，这些对象称为转换器。同样，转换过程由 `transform()` 方法执行，该方法接收待转换的数据集作为参数，并返回转换后的数据集。所有转换器还提供一个便利方法 `fit_transform()`，它等价于先调用 `fit()`、再调用 `transform()`（但有时 `fit_transform()` 经过了优化，运行速度更快）。

预测器（predictors）　有些估计器接收数据集后可以进行预测，它们称为预测器。例如上一章的 `LinearRegression` 模型就是预测器：给定一个国家的人均 GDP，它会预测生活满意度。预测器具有 `predict()` 方法，接收包含新实例的数据集，并返回相应的预测数据集；它还有一个 `score()` 方法，用测试集（以及监督学习算法对应的标签）衡量预测质量。[10]

可检查性　估计器的所有超参数都可以通过公开的实例变量直接访问，例如 `imputer.strategy`；估计器学习到的所有参数都可以通过带下划线后缀的公开实例变量访问，例如 `imputer.statistics_`。

不扩散自定义类　数据集用 NumPy 数组或 SciPy 稀疏矩阵表示，而不是使用自定义类。超参数只是普通的 Python 字符串或数字。

组合　尽可能重复使用已有的构件。例如，可以很容易地根据任意转换器序列和最终估计器创建一个 `Pipeline` 估计器，后面将看到这一点。

合理的默认值　Scikit-Learn 为大多数参数提供合理的默认值，因此可以快速创建一个能够工作的基线系统。

Scikit-Learn 的转换器即使接收 Pandas DataFrame 作为输入，也会输出 NumPy 数组（有时会输出 SciPy 稀疏矩阵）。[11] 因此，`imputer.transform(housing_num)` 的输出是一个 NumPy 数组：`X` 没有列名和索引。幸运的是，可以很容易地用 DataFrame 包装 `X`，并从 `housing_num` 中恢复列名和索引：

[10] 有些预测器还提供衡量预测置信度的方法。

[11] 如果运行 `sklearn.set_config(transform_output="pandas")`，所有转换器在接收 DataFrame 作为输入时都会输出 Pandas DataFrame，也就是“Pandas 输入，Pandas 输出”。

```
housing_tr = pd.DataFrame(X, columns=housing_num.columns,
                          index=housing_num.index)
```

2.5.3　处理文本和类别属性（Handling Text and Categorical Attributes）

到目前为止，我们只处理了数值属性，但数据中也可能包含文本属性。本数据集中只有一个文本属性：**ocean_proximity**。先查看前几个实例中的值：

```
>>> housing_cat = housing[["ocean_proximity"]]
>>> housing_cat.head(8)
      ocean_proximity
13096        NEAR BAY
14973       <1H OCEAN
3785            INLAND
14689           INLAND
20507       NEAR OCEAN
1286            INLAND
18078       <1H OCEAN
4396         NEAR BAY
```

这并不是任意文本，而是有限数量的可能取值，每个值代表一个类别。因此，这个属性是类别属性。大多数机器学习算法更喜欢处理数字，所以需要把这些文本类别转换成数字。为此可以使用 Scikit-Learn 的 **OrdinalEncoder** 类：

```
from sklearn.preprocessing import OrdinalEncoder

ordinal_encoder = OrdinalEncoder()
housing_cat_encoded = ordinal_encoder.fit_transform(housing_cat)
```

下面是 **housing_cat_encoded** 中前几个编码值的样子：

```
>>> housing_cat_encoded[:8]
array([[3.],
       [0.],
       [1.],
       [1.],
       [4.],
       [1.],
       [0.],
       [3.]])
```

可以使用 **categories_** 实例变量获取类别列表。它是一个列表，其中包含每个类别属性对应的一个一维数组（本例中只有一个类别属性，因此列表只含一个数组）：

```
>>> ordinal_encoder.categories_
[array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN'],
       dtype=object)]
```

这种表示方式存在一个问题：机器学习算法会假设两个相邻的数值比两个相距较远的数值更加相似。在有序类别中（例如“差”“一般”“好”“优秀”），这个假设可能成立；但对于 **ocean_proximity** 列显然不成立。例如，类别 0 和类别 4 明显比类别 0 和类别 1 更相似。解决这个问题的常见方法，是为每个类别创建一个二值属性：当类别为 **<1H OCEAN** 时，该属性为 1，否则为 0；对 **INLAND**、**ISLAND** 等类别依次如此。这叫作独热编码（one-hot encoding），因为只有一个属性为 1（hot），其他属性都为 0（cold）。新属性有时称为哑变量属性。Scikit-Learn 提供了 **OneHotEncoder** 类，将类别值转换为独热向量：

```
from sklearn.preprocessing import OneHotEncoder

cat_encoder = OneHotEncoder()
housing_cat_1hot = cat_encoder.fit_transform(housing_cat)
```

默认情况下，**OneHotEncoder** 的输出是 SciPy 稀疏矩阵，而不是 NumPy 数组：

```
>>> housing_cat_1hot
<Compressed Sparse Row sparse matrix of dtype 'float64'
 with 16512 stored elements and shape (16512, 5)>
```

稀疏矩阵是表示主要由零组成的矩阵的高效方式。当一个类别属性拥有数百或数千个类别时，独热编码会产生一个非常大的矩阵，除了每行只有一个 1 以外，其他位置全是 0。此时，稀疏矩阵正是所需要的表示方式：它可以节省大量内存，并加快计算。

稀疏矩阵基本上可以像普通二维数组一样使用。[12] 如果希望把它转换成（稠密的）NumPy 数组，只需调用 **toarray()** 方法：

```
>>> housing_cat_1hot.toarray()
array([[0., 0., 0., 1., 0.],
       [1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       ...,
       [0., 0., 0., 0., 1.],
       [1., 0., 0., 0., 0.],
       [0., 0., 0., 0., 1.]], shape=(16512, 5))
```

[12] 关于稀疏矩阵的更多细节，请参阅 SciPy 文档。

也可以在创建 OneHotEncoder 时设置 sparse_output=False，这样 transform() 方法就会直接返回普通的（稠密）NumPy 数组：

```
cat_encoder = OneHotEncoder(sparse_output=False)
housing_cat_1hot = cat_encoder.fit_transform(housing_cat)  # now a dense array
```

和 OrdinalEncoder 一样，可以使用编码器的 categories_ 实例变量获取类别列表：

```
>>> cat_encoder.categories_
[array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN'],
       dtype=object)]
```

Pandas 提供了一个名为 get_dummies() 的函数，也可以将每个类别特征转换成独热表示，为每个类别创建一个二值特征：

```
>>> df_test = pd.DataFrame({"ocean_proximity": ["INLAND", "NEAR BAY"]})
>>> pd.get_dummies(df_test)
   ocean_proximity_INLAND  ocean_proximity_NEAR BAY
0                    True                     False
1                   False                      True
```

它看起来简单又漂亮，为什么不使用它来替代 OneHotEncoder 呢？这是因为 OneHotEncoder 会记住训练时遇到的类别。这一点非常重要，因为模型投入生产后，输入的特征必须与训练时完全相同，不能多也不能少。

看看训练好的 cat_encoder 转换同一个 df_test 时会输出什么（使用 transform()，而不是 fit_transform()）：

```
>>> cat_encoder.transform(df_test)
array([[0., 1., 0., 0., 0.],
       [0., 0., 0., 1., 0.]])
```

看出区别了吗？get_dummies() 只看到了两个类别，因此输出两列；而 OneHotEncoder 会为学到的每个类别输出一列，并且顺序正确。此外，如果把包含未知类别（例如 <2H OCEAN）的 DataFrame 交给 get_dummies()，它会很自然地为该类别生成一列：

```
>>> df_test_unknown = pd.DataFrame({"ocean_proximity": ["<2H OCEAN", "ISLAND"]})
>>> pd.get_dummies(df_test_unknown)
   ocean_proximity_<2H OCEAN  ocean_proximity_ISLAND
0                       True                   False
1                      False                    True
```

但 **OneHotEncoder** 更聪明：它会检测到未知类别并抛出异常。如果愿意，可以把 **handle_unknown** 超参数设置为**"ignore"**，这样未知类别只会被表示为全 0：

```
>>> cat_encoder.handle_unknown = "ignore"
>>> cat_encoder.transform(df_test_unknown)
array([[0., 0., 0., 0., 0.],
       [0., 0., 1., 0., 0.]])
```

> 提示：如果类别属性有大量可能的类别（例如国家代码、职业或物种），独热编码会产生大量输入特征，这可能拖慢训练并降低性能。出现这种情况时，可以将类别输入替换为与类别相关的有用数值特征，例如用到海洋的距离替换 **ocean_proximity**，或者用国家人口和人均 GDP 替换国家代码。也可以使用 GitHub 上 category_encoders 包提供的编码器。处理神经网络时，还可以用称为嵌入（embedding）的可学习低维向量表示每个类别。这个例子属于表示学习，我们会在第 18 章看到更多例子。

当使用 DataFrame 拟合 Scikit-Learn 估计器时，估计器会把列名存储在 **feature_names_in_** 属性中。之后，Scikit-Learn 会确保输入该估计器的任何 DataFrame（例如传给 **transform()** 或 **predict()** 的 DataFrame）都拥有相同的列名。转换器还提供 **get_feature_names_out()** 方法，可以用它根据转换器的输出构建 DataFrame：

```
>>> cat_encoder.feature_names_in_
array(['ocean_proximity'], dtype=object)
>>> cat_encoder.get_feature_names_out()
array(['ocean_proximity_<1H OCEAN', 'ocean_proximity_INLAND',
       'ocean_proximity_ISLAND', 'ocean_proximity_NEAR BAY',
       'ocean_proximity_NEAR OCEAN'], dtype=object)
>>> df_output = pd.DataFrame(cat_encoder.transform(df_test_unknown),
...                          columns=cat_encoder.get_feature_names_out(),
...                          index=df_test_unknown.index)
...
```

这个功能有助于避免列不匹配问题，在调试时也非常有用。

2.5.4　特征缩放与变换（Feature Scaling and Transformation）

需要对数据应用的最重要变换之一就是特征缩放。除了少数例外，当输入数值属性的尺度差异很大时，机器学习算法通常表现不佳。房价数据正是如此：房间总数的范围约为 6 至 39,320，而中位数收入的范围只有 0 至 15。如果不缩放，大多数模型会偏向于忽略中位数收入，而把注意力放在房间数量上。

让所有属性拥有相同尺度的常见方法有两种：最小-最大缩放和标准化。

> 警告：和所有估计器一样，缩放器只能在训练数据上拟合：除了训练集以外，不要对任何数据调用 `fit()` 或 `fit_transform()`。训练好缩放器后，可以用它对包括验证集、测试集和新数据在内的其他数据调用 `transform()`。注意，训练集的值一定会被缩放到指定范围，但如果新数据包含异常值，缩放后的值可能落在该范围之外。如果想避免这种情况，只需把 `clip` 超参数设置为 `True`。

最小-最大缩放（很多人称为归一化）最简单：对每个属性，平移并重新缩放其值，使其范围变为 0 到 1。具体做法是从所有值中减去最小值，再除以最大值与最小值之差。Scikit-Learn 提供了 `MinMaxScaler` 转换器，并通过 `feature_range` 参数允许你修改范围，例如神经网络最适合零均值输入，因此可以使用 -1 到 1 的范围：

```
from sklearn.preprocessing import MinMaxScaler

min_max_scaler = MinMaxScaler(feature_range=(-1, 1))
housing_num_min_max_scaled = min_max_scaler.fit_transform(housing_num)
```

标准化有所不同：首先减去均值，使标准化后的值均值为 0；然后除以标准差，使标准化后的值标准差为 1。与最小-最大缩放不同，标准化不会把值限制在特定范围内。不过，标准化受异常值的影响小得多。例如，假设某个地区的中位数收入错误地记录为 100，而不是通常的 0 到 15。把它最小-最大缩放到 0 到 1，会把这个异常值映射为 1，并把其他所有值压缩到 0 到 0.15；标准化则不会受到太大影响。Scikit-Learn 提供了用于标准化的 **StandardScaler** 转换器：

```
from sklearn.preprocessing import StandardScaler

std_scaler = StandardScaler()
housing_num_std_scaled = std_scaler.fit_transform(housing_num)
```

> 提示：如果想缩放稀疏矩阵，又不想先将其转换成稠密矩阵，可以使用把 **with_mean** 超参数设为 **False** 的 **StandardScaler**。它只会除以标准差，不减去均值，因为减去均值会破坏稀疏性。

当特征分布具有厚尾时，也就是远离均值的值并不是呈指数级稀少时，最小-最大缩放和标准化都会把大多数值压缩到很小的范围内。机器学习模型通常不喜欢这种情况，后面会在第 4 章看到这一点。因此，在缩放特征之前，应该先对特征进行变换，缩小厚尾，并尽可能使分布大致对称。例如，对于右侧具有厚尾的正值特征，一种常见做法是用特征的平方根替换它，或者把特征提升到 0 到 1 之间的某个幂。如果特征的尾部特别长、特别厚，例如服从幂律分布，那么用它的对数替换可能会有所帮助。**population** 特征大致服从幂律分布：人口为 10,000 的地区只比人口为 1,000 的地区少约 10 倍，而不是呈指数级稀少。图 2-17 展示了计算对数后这个特征的改善效果：它非常接近高斯分布，也就是钟形分布。

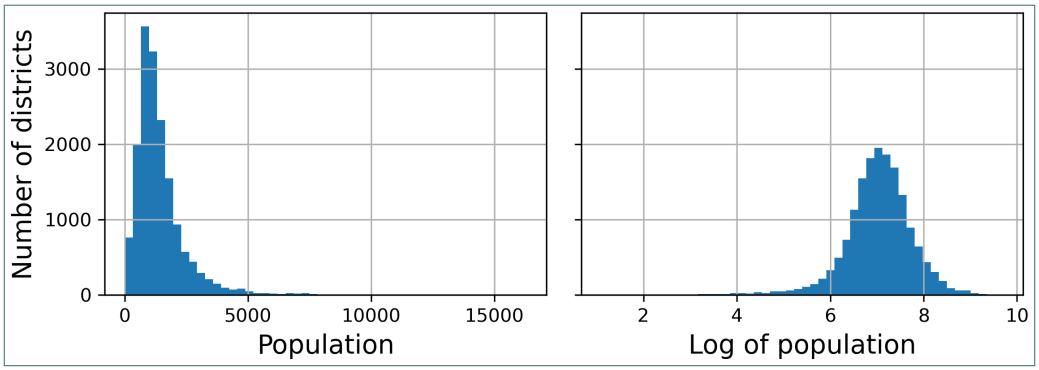

图 2-17: 将特征变换为更接近高斯分布（Transforming a feature to make it closer to a Gaussian distribution）

处理厚尾特征的另一种方法是对特征进行分桶。这意味着把分布切分成大小大致相等的桶，并用每个特征值所属桶的索引替换原值，有点像前面创建 **income_cat** 特征时所做的事情（不过当时只用于分层抽样）。例如，可以用每个值对应的百分位数替换原值。使用大小相等的桶进行分桶，会得到几乎均匀分布的特征，因此不需要进一步缩放；或者也可以除以桶的数量，把值强制缩放到 0 到 1 的范围内。

当特征具有多峰分布，也就是存在两个或更多清晰峰值（称为众数）时，例如 **housing_median_age**，分桶同样可能有帮助，但这次应把桶 ID 当作类别，而不是数值。也就是说，必须对桶索引进行编码，例如使用 **OneHotEncoder**，因此通常不应使用太多桶。这种方法可以让回归模型更容易为特征值的不同范围学习不同规则。例如，建造于大约 35 年前的房子可能具有某种独特风格，后来不再流行，因此价格低于仅根据房龄所作的判断。

转换多峰分布的另一种方法，是为每个峰值（至少为主要峰值）添加一个特征，用于表示房屋中位数房龄与该峰值的相似度。相似度通常使用径向基函数（radial basis function，RBF）计算。径向基函数是只依赖输入值与固定点之间距离的函数。最常用的 RBF 是高斯 RBF：当输入值远离固定点时，其输出值呈指数级衰减。例如，房龄 x 与 35 之间的高斯 RBF 相似度由 $\exp(-\gamma(x-35)^2)$ 给出。超参数 γ 决定相似度随 x 远离 35 而衰减的速度。使用 Scikit-Learn 的 **rbf_kernel()** 函数，可以创建一个新高斯 RBF 特征，衡量房屋中位数房龄与 35 的相似度：

```
from sklearn.metrics.pairwise import rbf_kernel

age_simil_35 = rbf_kernel(housing[["housing_median_age"]], [[35]], gamma=0.1)
```

图 2-18 将这个新特征表示为房龄的函数(实线),同时展示了使用更小的 gamma 值时特征的样子。可以看到，新建的房龄相似度特征在 35 岁处达到峰值，正好位于房龄分布的尖峰附近；如果这一房龄组确实与较低房价强相关，那么这个新特征很可能对模型有所帮助。

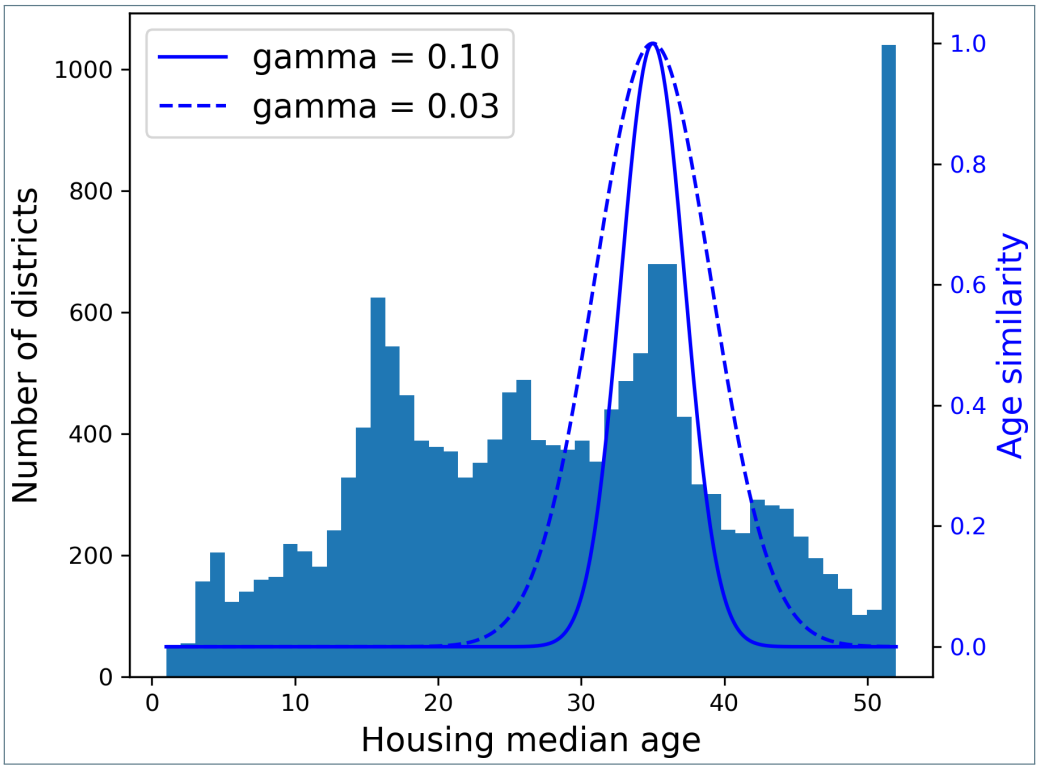

图 2-18: 衡量房屋中位数房龄与 35 岁相似度的高斯 RBF 特征（Gaussian RBF feature measuring similarity to housing median age 35）

到目前为止，我们只查看了输入特征，但目标值也可能需要变换。例如，如果目标分布具有厚尾，可以考虑用目标值的对数替换目标值。但这样一来，回归模型预测的就不再是中位数房价本身，而是中位数房价的对数。如果想得到预测的中位数房价，就需要计算模型预测值的指数。

幸运的是,Scikit-Learn 的大多数转换器都有 **`inverse_transform()`** 方法,因此很容易计算变换的逆变换。下面的代码示例展示了如何使用 **`StandardScaler`** 缩放标签

（与缩放输入一样），在缩放后的标签上训练一个简单的线性回归模型，然后对新数据进行预测，并使用训练好的缩放器的 `inverse_transform()` 方法，把预测值变换回原始尺度。注意，必须把标签从 Pandas Series 转换成 DataFrame，因为 `StandardScaler` 需要二维输入。另外，为了简单起见，本例只使用一个原始输入特征（`median_income`）训练模型：

```
from sklearn.linear_model import LinearRegression

target_scaler = StandardScaler()
scaled_labels = target_scaler.fit_transform(housing_labels.to_frame())
model = LinearRegression()
model.fit(housing[["median_income"]], scaled_labels)
some_new_data = housing[["median_income"]].iloc[:5]  # pretend this is new data
scaled_predictions = model.predict(some_new_data)
predictions = target_scaler.inverse_transform(scaled_predictions)
```

这段代码运行良好，但使用 `TransformedTargetRegressor` 会更加简单，也不容易出错，可以避免潜在的缩放不匹配问题。只需把回归模型和标签转换器传给它来构造对象，然后使用原始、未缩放的标签在训练集上拟合。它会自动使用转换器缩放标签，并在缩放后的标签上训练回归模型，就像前面的代码所做的那样。进行预测时，它会调用回归模型的 `predict()` 方法，再使用缩放器的 `inverse_transform()` 方法生成预测值：

```
from sklearn.compose import TransformedTargetRegressor

model = TransformedTargetRegressor(LinearRegression(),
                                   transformer=StandardScaler())
model.fit(housing[["median_income"]], housing_labels)
predictions = model.predict(some_new_data)
```

2.5.5　自定义转换器（Custom Transformers）

尽管 Scikit-Learn 提供了许多有用的转换器，但有时仍需要针对自定义转换、清理操作或特定属性组合编写自己的转换器。

对于不需要训练的转换，可以编写一个函数，接收 NumPy 数组并输出转换后的数组。例如，前面提到过，对于右侧具有厚尾的特征（假设特征为正值），用对数替换特征通常是个好主意。下面创建一个对数转换器，并将它应用于 `population` 特征：

```
from sklearn.preprocessing import FunctionTransformer

log_transformer = FunctionTransformer(np.log, inverse_func=np.exp)
log_pop = log_transformer.transform(housing[["population"]])
```

`inverse_func` 参数是可选的。它允许你指定逆变换函数，例如计划在 `TransformedTargetRegressor` 中使用这个转换器时。

转换函数还可以接收额外的超参数。例如，下面创建一个转换器，计算与前面相同的高斯 RBF 相似度：

```
rbf_transformer = FunctionTransformer(rbf_kernel,
                                      kw_args=dict(Y=[[35.]], gamma=0.1))
age_simil_35 = rbf_transformer.transform(housing[["housing_median_age"]])
```

RBF 核没有逆函数，因为对于固定点来说，除距离为 0 外，每个距离总有两个对应值。还要注意，`rbf_kernel()` 不会分别处理各个特征。如果传入含有两个特征的数组，它会使用二维欧几里得距离来衡量相似度。例如，下面的代码可以添加一个特征，用于衡量每个地区与旧金山的地理相似度：

```
sf_coords = 37.7749, -122.41
sf_transformer = FunctionTransformer(rbf_kernel,
                                     kw_args=dict(Y=[sf_coords], gamma=0.1))
sf_simil = sf_transformer.transform(housing[["latitude", "longitude"]])
```

自定义转换器对于组合特征也很有用。例如，下面的 `FunctionTransformer` 会计算输入特征 0 与特征 1 的比值：

```
>>> ratio_transformer = FunctionTransformer(lambda X: X[:, [0]] / X[:, [1]])
>>> ratio_transformer.transform(np.array([[1., 2.], [3., 4.]]))
array([[0.5 ],
       [0.75]])
```

`FunctionTransformer` 非常方便，但如果希望转换器能够通过 `fit()` 方法学习某些参数，并在之后的 `transform()` 方法中使用这些参数，该怎么办？此时需要编写自定义类。

> 注意：本节接下来的内容会说明如何定义自定义转换器类。具体来说，将定义一个自定义转换器，把地区分成 10 个地理聚类，然后计算每个地区与各聚类中心之间的距离，向数据中添加 10 个相应的 RBF 相似度特征。由于定义自定

> 义转换器类稍微有些高级，如果暂时不需要，可以先跳到下一节，之后再回来查看。

Scikit-Learn 依赖鸭子类型（duck typing）：[13] 因此，自定义转换器类不需要继承任何特定的基类。它们只需实现三个方法：`fit()`（必须返回 `self`）、`transform()` 和 `fit_transform()`。只需把 `TransformerMixin` 添加为基类，就可以免费获得 `fit_transform()`：默认实现会先调用 `fit()`，再调用 `transform()`。如果添加 `BaseEstimator` 作为基类（并且构造函数中不使用 `*args` 和 `**kwargs`），还会额外获得 `get_params()` 和 `set_params()` 两个方法。这些方法之后可用于自动调整超参数。

例如，下面的自定义转换器执行的功能类似于 `StandardScaler`：

```
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.utils.validation import check_array, check_is_fitted

class StandardScalerClone(BaseEstimator, TransformerMixin):
    def __init__(self, with_mean=True):  # no *args or **kwargs!
        self.with_mean = with_mean

    def fit(self, X, y=None):  # y is required even though we don't use it
        X = check_array(X)  # checks that X is an array with finite float values
        self.mean_ = X.mean(axis=0)
        self.scale_ = X.std(axis=0)
        self.n_features_in_ = X.shape[1]  # every estimator stores this in fit()
        return self  # always return self!

    def transform(self, X):
        check_is_fitted(self)  # looks for learned attributes (with trailing _)
        X = check_array(X)
        assert self.n_features_in_ == X.shape[1]
        if self.with_mean:
            X = X - self.mean_
        return X / self.scale_
```

这里有几件事值得注意：

- `sklearn.utils.validation` 包含几个可用于验证输入的函数。为了简单起见，本书后面的内容会跳过这些测试，但生产代码应该包含它们。

[13] 鸭子类型强调对象的方法和行为，而不是对象的类型："如果它看起来像鸭子，叫起来也像鸭子，那么它就是鸭子"。

- Scikit-Learn 流水线要求 `fit()` 方法接收两个参数 `X` 和 `y`，这就是即使不使用 `y`，也需要声明 `y=None` 参数的原因。
- 所有 Scikit-Learn 估计器都会在 `fit()` 方法中设置 `n_features_in_`，并确保传给 `transform()` 或 `predict()` 的数据具有相同数量的特征。
- `fit()` 方法必须返回 `self`。
- 这个实现并不完整。所有估计器在接收 DataFrame 时都应在 `fit()` 方法中设置 `feature_names_in_`。此外，所有转换器都应提供 `get_feature_names_out()` 方法；当转换可以逆转时，还应提供 `inverse_transform()` 方法。详情参见本章末尾的最后一道练习。

自定义转换器可以在实现中使用其他估计器。例如，下面的代码展示了一个自定义转换器：它在 `fit()` 方法中使用 `KMeans` 聚类器识别训练数据中的主要聚类，然后在 `transform()` 方法中使用 `rbf_kernel()` 衡量每个样本与各聚类中心的相似度：

```
from sklearn.cluster import KMeans

class ClusterSimilarity(BaseEstimator, TransformerMixin):
    def __init__(self, n_clusters=10, gamma=1.0, random_state=None):
        self.n_clusters = n_clusters
        self.gamma = gamma
        self.random_state = random_state

    def fit(self, X, y=None, sample_weight=None):
        self.kmeans_ = KMeans(self.n_clusters, random_state=self.random_state)
        self.kmeans_.fit(X, sample_weight=sample_weight)
        return self  # always return self!

    def transform(self, X):
        return rbf_kernel(X, self.kmeans_.cluster_centers_, gamma=self.gamma)

    def get_feature_names_out(self, names=None):
        return [f"Cluster {i} similarity" for i in range(self.n_clusters)]
```

> 提示：可以将一个估计器实例传给 `sklearn.utils.estimator_checks` 包中的 `check_estimator()`，检查自定义估计器是否遵循 Scikit-Learn 的 API。完整 API 请参阅 https://scikit-learn.org/stable/developers/。

如第 8 章将要介绍的，k-means 是一种用于定位数据中聚类的算法。例如，可以用它找出加利福尼亚州人口最密集的区域。k-means 要搜索的聚类数量由 **n_clusters** 超参数控制。**KMeans** 的 **fit()** 方法支持一个可选的 **sample_weight** 参数，可以让用户指定各样本的相对权重。例如，如果希望聚类偏向富裕地区，可以传入中位数收入作为权重。训练完成后，可以通过 **cluster_centers_** 属性访问聚类中心。k-means 是一种随机算法，依靠随机性定位聚类，因此如果希望得到可复现的结果，就必须设置 **random_state** 参数。虽然任务看起来复杂，但代码相当直接。现在使用这个自定义转换器：

```
cluster_simil = ClusterSimilarity(n_clusters=10, gamma=1., random_state=42)
similarities = cluster_simil.fit_transform(housing[["latitude", "longitude"]])
```

这段代码创建了一个 **ClusterSimilarity** 转换器，把聚类数量设置为 10。然后对训练集中每个地区的纬度和经度调用 **fit_transform()**（你可以尝试用每个地区的中位数收入作为权重，观察聚类如何变化）。该转换器使用 k-means 定位聚类，再计算每个地区与 10 个聚类中心之间的高斯 RBF 相似度。结果是一个矩阵，每个地区对应一行，每个聚类对应一列。下面查看前 3 行，并四舍五入到小数点后两位：

```
>>> similarities[:3].round(2)
array([[0.46, 0.  , 0.08, 0.  , 0.  , 0.  , 0.  , 0.98, 0.  , 0.  ],
       [0.  , 0.96, 0.  , 0.03, 0.04, 0.  , 0.  , 0.  , 0.11, 0.35],
       [0.34, 0.  , 0.45, 0.  , 0.  , 0.  , 0.01, 0.73, 0.  , 0.  ]])
```

图 2-19 展示了 k-means 找到的 10 个聚类中心。地区按照它们与最近聚类中心的地理相似度着色。注意，大多数聚类位于人口密集的区域。

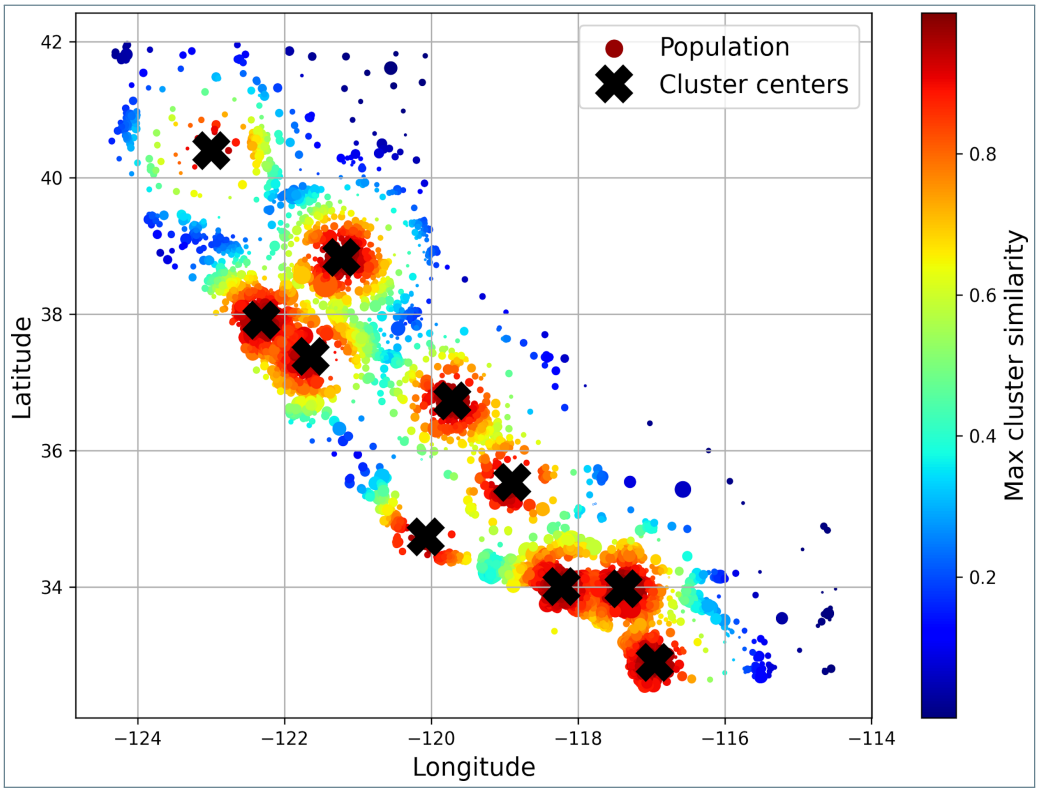

图 2-19: 与最近聚类中心的高斯 RBF 相似度（Gaussian RBF similarity to the nearest cluster center）

2.5.6　转换流水线（Transformation Pipelines）

如你所见，需要按照正确顺序执行许多数据转换步骤。幸运的是，Scikit-Learn 提供了 `Pipeline` 类，可以帮助处理这样的一系列转换。下面是一个用于数值属性的小型流水线，它会先填补缺失值，再缩放输入特征：

```
from sklearn.pipeline import Pipeline

num_pipeline = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("standardize", StandardScaler()),
])
```

`Pipeline` 构造函数接收一个名称/估计器对（2 元组）列表，用于定义步骤序列。名称可以任意选择，只要名称唯一，并且不含双下划线（`__`）。稍后讨论超参数调整时会用到这些名称。除了最后一个估计器以外，所有估计器都必须是转换器，也就是具有 `fit_transform()` 方法；最后一个估计器可以是任何类型，包括转换器、预测器或其他估计器。

> 提示：在 Jupyter notebook 中，如果导入 `sklearn` 并运行 `sklearn.set_config(display="diagram")`，所有 Scikit-Learn 估计器都会以交互式图表显示。这对于可视化流水线尤其有用。要可视化 `num_pipeline`，只需将 `num_pipeline` 作为一个单元格的最后一行。点击估计器可以查看更多细节。

如果不想给转换器命名，也可以使用方便的 `make_pipeline()` 函数。它接收转换器作为位置参数，并使用转换器的类名创建一个 `Pipeline`，类名会转换成小写并去掉下划线（例如`"simpleimputer"`）：

```
from sklearn.pipeline import make_pipeline

num_pipeline = make_pipeline(SimpleImputer(strategy="median"), StandardScaler())
```

如果多个转换器拥有相同名称，系统会在名称后附加索引，例如"foo-1"、"foo-2" 等。

调用流水线的 fit() 方法时，它会依次对每个转换器调用 fit_transform()，把每次调用的输出作为下一个调用的输入，直到到达最终估计器。对于最终估计器，只调用它的 fit() 方法。

流水线会暴露与最终估计器相同的方法。本例中的最终估计器是 StandardScaler，它是一个转换器，因此流水线也充当转换器。如果调用流水线的 transform() 方法，它会依次对数据应用所有变换。如果最终估计器是预测器而不是转换器，那么流水线会提供 predict() 方法；调用这个方法时，流水线会依次完成所有数据转换，然后把结果传给预测器的 predict() 方法。

现在调用流水线的 fit_transform() 方法，查看输出的前两行，并四舍五入到小数点后两位：

```
>>> housing_num_prepared = num_pipeline.fit_transform(housing_num)
>>> housing_num_prepared[:2].round(2)
array([[-1.42,  1.01,  1.86,  0.31,  1.37,  0.14,  1.39, -0.94],
       [ 0.6 , -0.7 ,  0.91, -0.31, -0.44, -0.69, -0.37,  1.17]])
```

如前所述，如果希望得到一个漂亮的 DataFrame，可以使用流水线的 get_feature_names_out() 方法：

```
df_housing_num_prepared = pd.DataFrame(
    housing_num_prepared, columns=num_pipeline.get_feature_names_out(),
    index=housing_num.index)
```

流水线支持索引操作。例如，pipeline[1] 返回流水线中的第二个估计器，pipeline[:-1] 返回包含除最后一个估计器之外所有估计器的 Pipeline 对象。还可以通过 steps 属性（一个名称/估计器对列表）或 named_steps 字典属性访问估计器，后者把估计器名称映射到估计器对象。例如，num_pipeline["simpleimputer"] 会返回名称为"simpleimputer" 的估计器。

到目前为止，我们分别处理了类别列和数值列。如果有一个能够处理所有列，并对各列应用相应变换的转换器，会更加方便。此时可以使用 ColumnTransformer。例

如，下面的 ColumnTransformer 会把前面定义的 num_pipeline 应用于数值属性，把 cat_pipeline 应用于类别属性：

```
from sklearn.compose import ColumnTransformer

num_attribs = ["longitude", "latitude", "housing_median_age", "total_rooms",
               "total_bedrooms", "population", "households", "median_income"]
cat_attribs = ["ocean_proximity"]
cat_pipeline = make_pipeline(
    SimpleImputer(strategy="most_frequent"),
    OneHotEncoder(handle_unknown="ignore"))
preprocessing = ColumnTransformer([
    ("num", num_pipeline, num_attribs),
    ("cat", cat_pipeline, cat_attribs),
])
```

首先导入 ColumnTransformer 类，然后定义数值列和类别列的名称列表，并为类别属性构建一个简单流水线。最后构造一个 ColumnTransformer。它的构造函数需要一个三元组列表，每个三元组包含名称（必须唯一且不能含双下划线）、转换器以及该转换器所要应用的列名（或索引）列表。

> 提示：如果希望删除某些列，可以指定字符串"drop" 代替转换器；如果希望原样保留某些列，可以指定"passthrough"。默认情况下，未列出的剩余列会被删除，但可以把 remainder 超参数设置为某个转换器（或"passthrough"），让这些列以不同方式处理。

由于列名列表不太方便，Scikit-Learn 提供了 make_column_selector 类，可以根据类型自动选择特征，例如选择数值特征或类别特征。可以把 selector 传给 ColumnTransformer，替代列名或索引。此外，如果不需要为转换器命名，还可以使用 make_column_transformer()，它会像 make_pipeline() 一样自动选择名称。例如，下面的代码创建了和前面相同的 ColumnTransformer，只是转换器被自动命名为"pipeline-1" 和"pipeline-2"，而不是"num" 和"cat"：

```
from sklearn.compose import make_column_selector, make_column_transformer

preprocessing = make_column_transformer(
    (num_pipeline, make_column_selector(dtype_include=np.number)),
    (cat_pipeline, make_column_selector(dtype_include=object)),
)
```

现在可以把这个 ColumnTransformer 应用于 housing 数据：

```
housing_prepared = preprocessing.fit_transform(housing)
```

很好！我们已经得到一个预处理流水线，它接收完整训练集，对适当的列应用每个转换器，然后沿水平方向拼接转换后的列（转换器绝不能改变行数）。这一次仍会返回 NumPy 数组，但可以使用 `preprocessing.get_feature_names_out()` 获取列名，再像前面一样把数据包装成漂亮的 DataFrame。

> 注意：`OneHotEncoder` 返回稀疏矩阵，而 `num_pipeline` 返回稠密矩阵。当稀疏矩阵和稠密矩阵混合存在时，`ColumnTransformer` 会估计最终矩阵的密度，也就是非零单元格的比例；当密度低于给定阈值（默认 `sparse_threshold=0.3`）时，它会返回稀疏矩阵。在本例中，它返回的是稠密矩阵。

项目进展得很顺利，马上就可以训练模型了！现在要创建一个单一流水线，完成目前为止尝试过的所有变换。先回顾一下这条流水线会做什么，以及为什么要这样做：

- 数值特征中的缺失值用中位数填补，因为大多数机器学习算法无法处理缺失值；类别特征中的缺失值用最频繁的类别填补。
- 类别特征进行独热编码，因为大多数机器学习算法只接受数值输入。
- 计算并添加几个比例特征：`bedrooms_ratio`、`rooms_per_house` 和 `people_per_house`。希望它们与中位数房价的相关性更好，从而帮助机器学习模型。
- 添加一些聚类相似度特征。这些特征可能比纬度和经度本身更有用。
- 用对数替换长尾特征，因为大多数模型更喜欢分布大致均匀或呈高斯分布的特征。
- 对所有数值特征进行标准化，因为大多数机器学习算法希望所有特征具有大致相同的尺度。

现在构建完成所有这些工作的流水线，代码应该已经很熟悉了：

```
def column_ratio(X):
    return X[:, [0]] / X[:, [1]]

def ratio_name(function_transformer, feature_names_in):
    return ["ratio"]  # feature names out

def ratio_pipeline():
    return make_pipeline(
        SimpleImputer(strategy="median"),
        FunctionTransformer(column_ratio, feature_names_out=ratio_name),
        StandardScaler())

log_pipeline = make_pipeline(
    SimpleImputer(strategy="median"),
    FunctionTransformer(np.log, feature_names_out="one-to-one"),
    StandardScaler())

cluster_simil = ClusterSimilarity(n_clusters=10, gamma=1., random_state=42)
default_num_pipeline = make_pipeline(SimpleImputer(strategy="median"),
                                     StandardScaler())
preprocessing = ColumnTransformer([
        ("bedrooms", ratio_pipeline(), ["total_bedrooms", "total_rooms"]),
        ("rooms_per_house", ratio_pipeline(), ["total_rooms", "households"]),
        ("people_per_house", ratio_pipeline(), ["population", "households"]),
        ("log", log_pipeline, ["total_bedrooms", "total_rooms", "population",
                               "households", "median_income"]),
        ("geo", cluster_simil, ["latitude", "longitude"]),
        ("cat", cat_pipeline, make_column_selector(dtype_include=object)),
    ],
    remainder=default_num_pipeline)  # one column remaining: housing_median_age
```

如果运行这个 **ColumnTransformer**，它会完成所有变换，并输出包含 24 个特征的 NumPy 数组：

```
>>> housing_prepared = preprocessing.fit_transform(housing)
>>> housing_prepared.shape
(16512, 24)
>>> preprocessing.get_feature_names_out()
array(['bedrooms__ratio', 'rooms_per_house__ratio',
       'people_per_house__ratio', 'log__total_bedrooms',
       'log__total_rooms', 'log__population', 'log__households',
       'log__median_income', 'geo__Cluster 0 similarity', [...],
       'geo__Cluster 9 similarity', 'cat__ocean_proximity_<1H OCEAN',
       'cat__ocean_proximity_INLAND', 'cat__ocean_proximity_ISLAND',
       'cat__ocean_proximity_NEAR BAY', 'cat__ocean_proximity_NEAR OCEAN',
       'remainder__housing_median_age'], dtype=object)
```

2.6　选择和训练模型（Select and Train a Model）

终于完成了问题定义、数据获取和探索，划分出了训练集和测试集，也编写了能够自动清理和准备数据的预处理流水线。现在可以选择并训练机器学习模型了。

2.6.1　在训练集上训练和评估（Train and Evaluate on the Training Set）

好消息是，得益于前面的所有步骤，接下来的工作会变得很简单！先训练一个非常基础的线性回归模型：

```
from sklearn.linear_model import LinearRegression

lin_reg = make_pipeline(preprocessing, LinearRegression())
lin_reg.fit(housing, housing_labels)
```

完成了！现在已经有了一个能工作的线性回归模型。先在训练集上试用它，查看前 5 个预测值，并与标签进行比较：

```
>>> housing_predictions = lin_reg.predict(housing)
>>> housing_predictions[:5].round(-2)  # -2 = rounded to the nearest hundred
array([246000., 372700., 135700.,  91400., 330900.])
>>> housing_labels.iloc[:5].values
array([458300., 483800., 101700.,  96100., 361800.])
```

模型确实能工作，但并不总是很好：第一个预测值偏差很大（超过 200,000 美元），另外几个预测值好一些，其中两个偏差约为 25%，两个偏差小于 10%。记住，选择的性能指标是 RMSE，因此需要使用 Scikit-Learn 的 **root_mean_squared_error()** 函数，在整个训练集上测量这个回归模型的 RMSE：

```
>>> from sklearn.metrics import root_mean_squared_error
>>> lin_rmse = root_mean_squared_error(housing_labels, housing_predictions)
>>> lin_rmse
68972.88910758484
```

> 提示：这里没有使用 **score()** 方法，因为它返回的是决定系数 R2，而不是 RMSE。决定系数表示模型能够解释的数据方差比例；越接近最大值 1，模型越好。如果模型始终只预测均值，就无法解释方差的任何部分，因此 R2 得分为 0。如果模型的表现比这还差，R2 得分可以为负值，甚至可以任意低。

这比什么都不做要好，但显然不算理想：大多数地区的 **median_house_value** 在 120,000 至 265,000 美元之间，因此典型预测误差达到 68,973 美元，实在令人不满意。

这是模型对训练数据欠拟合的例子。发生欠拟合可能意味着特征没有提供足够的信息来作出良好预测，也可能意味着模型不够强大。正如上一章所见，解决欠拟合的主要方法是选择更强大的模型、为训练算法提供更好的特征，或者减少对模型的约束。这个模型没有正则化，因此最后一种方法不适用。可以尝试添加更多特征，但先尝试更复杂的模型，看看效果如何。

决定尝试 **DecisionTreeRegressor**，因为它是一个相当强大的模型，能够找出复杂的非线性关系（第 5 章将更详细地介绍决策树）：

```
from sklearn.tree import DecisionTreeRegressor

tree_reg = make_pipeline(preprocessing, DecisionTreeRegressor(random_state=42))
tree_reg.fit(housing, housing_labels)
```

模型训练完成后，在训练集上评估它：

```
>>> housing_predictions = tree_reg.predict(housing)
>>> tree_rmse = root_mean_squared_error(housing_labels, housing_predictions)
>>> tree_rmse
0.0
```

等等，怎么会这样!？误差竟然为零？这个模型真的完美无缺吗？当然不是，更可能是模型严重过拟合了数据。如何确认这一点？正如前面所说，在准备好启动一个有把握的模型之前，不应触碰测试集，所以需要把训练集的一部分用于训练，另一部分用于模型验证。

2.6.2　使用交叉验证获得更好的评估（Better Evaluation Using Cross-Validation）

评估决策树模型的一种方法，是使用 `train_test_split()` 将训练集拆分成更小的训练集和验证集，然后在较小的训练集上训练模型，并在验证集上评估模型。这需要一些工作，但并不困难，而且效果应该相当不错。

一个很好的替代方案是使用 Scikit-Learn 的 k 折交叉验证功能。把训练集分成 k 个互不重叠的子集，称为折（fold）；然后训练和评估模型 k 次，每次选择不同的折进行评估（即验证折），并使用其余 k-1 个折进行训练。这个过程会产生 k 个评估分数（见图 2-20）。

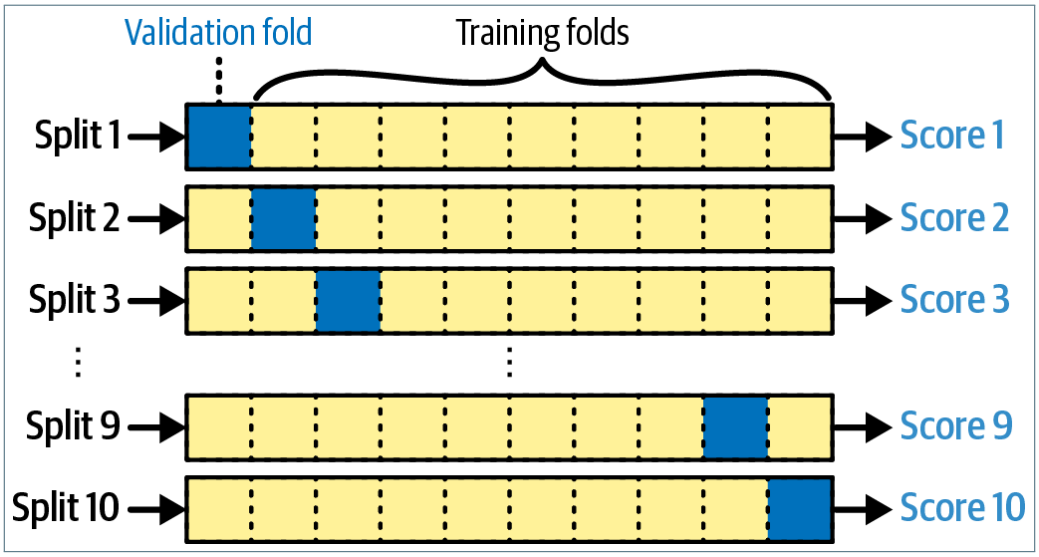

图 2-20: k 折交叉验证，k=10（k-fold cross-validation, with k = 10）

Scikit-Learn 提供了方便的 **cross_val_score()** 函数来完成这件事，并返回一个包含 k 个评估分数的数组。例如，使用 k=10 评估决策树回归器：

```
from sklearn.model_selection import cross_val_score

tree_rmses = -cross_val_score(tree_reg, housing, housing_labels,
                              scoring="neg_root_mean_squared_error", cv=10)
```

> 警告：Scikit-Learn 的交叉验证功能要求传入一个效用函数（越大越好），而不是代价函数（越小越好），因此评分函数实际上是 RMSE 的相反数。它是负值，所以需要把输出的符号切换过来，才能得到 RMSE 分数。

查看结果：

```
>>> pd.Series(tree_rmses).describe()
count       10.000000
mean     66573.734600
std       1103.402323
min      64607.896046
25%      66204.731788
50%      66388.272499
75%      66826.257468
max      68532.210664
dtype: float64
```

现在，决策树看起来不像刚才那么好了。事实上，它的表现似乎几乎和线性回归模型一样差！注意，交叉验证不仅可以估计模型的性能，还可以衡量这个估计的精确程度，也就是标准差。如果只使用一个验证集，就无法获得这些信息。不过，交叉验证需要多次训练模型，因此并不总是可行。

如果对线性回归模型计算相同的指标，会发现其平均 RMSE 为 70,003，标准差为 4,182。因此，决策树模型似乎比线性模型略好，但由于严重过拟合，差别很小。我们知道存在过拟合问题，是因为训练误差很低（实际上为零），而验证误差很高。

现在再尝试一个模型：**RandomForestRegressor**。正如第 6 章将要介绍的，随机森林通过在特征的随机子集上训练许多决策树，然后对它们的预测取平均来工作。这种由许多其他模型组成的模型称为集成模型（ensemble）。如果底层模型差异很大，它们的错误就不会高度相关，因此对预测值取平均可以平滑错误、减少过拟合，并改善总体性能。代码与之前非常相似：

```
from sklearn.ensemble import RandomForestRegressor

forest_reg = make_pipeline(preprocessing,
                           RandomForestRegressor(random_state=42))
forest_rmses = -cross_val_score(forest_reg, housing, housing_labels,
                                scoring="neg_root_mean_squared_error", cv=10)
```

查看分数：

```
>>> pd.Series(forest_rmses).describe()
count       10.000000
mean     47038.092799
std       1021.491757
min      45495.976649
25%      46510.418013
50%      47118.719249
75%      47480.519175
max      49140.832210
dtype: float64
```

太好了，这要好得多：随机森林看起来非常适合这个任务！不过，如果训练一个 **RandomForestRegressor**，然后在训练集上测量 RMSE，会得到约 17,551，明显更低，说明仍然存在相当严重的过拟合。可能的解决方案包括简化模型、约束模型（也就是正则化），或者获取更多训练数据。但在深入研究随机森林之前，还应该尝试来自不同机器学习算法类别的其他模型，例如使用不同核函数的多个支持向量机，可能还包括神经网络，而不要花太多时间调整超参数。目标是列出 2 至 5 个有希望的候选模型。

2.7　微调模型（Fine-Tune Your Model）

假设现在已经有了一个有希望的模型短名单。接下来需要对它们进行微调。下面介绍几种方法。

2.7.1　网格搜索（Grid Search）

一种方法是手动调整超参数，直到找到很好的超参数组合。这项工作十分枯燥，而且可能没有时间探索许多组合。

不如使用 Scikit-Learn 的 **GridSearchCV** 类来搜索。只需要告诉它要尝试哪些超参数以及哪些值，它就会使用交叉验证评估所有可能的超参数组合。例如，下面的代码搜索 **RandomForestRegressor** 的最佳超参数组合：

```
from sklearn.model_selection import GridSearchCV

full_pipeline = Pipeline([
    ("preprocessing", preprocessing),
    ("random_forest", RandomForestRegressor(random_state=42)),
])
param_grid = [
    {'preprocessing__geo__n_clusters': [5, 8, 10],
     'random_forest__max_features': [4, 6, 8]},
    {'preprocessing__geo__n_clusters': [10, 15],
     'random_forest__max_features': [6, 8, 10]},
]
grid_search = GridSearchCV(full_pipeline, param_grid, cv=3,
                           scoring='neg_root_mean_squared_error')
grid_search.fit(housing, housing_labels)
```

注意，可以引用流水线中任意估计器的超参数，即使这个估计器深藏在多层流水线和列转换器内部。例如，Scikit-Learn 看到`"preprocessing__geo__n_clusters"`时，会按双下划线拆分这个字符串，然后在流水线中查找名为`"preprocessing"`的估计器，找到预处理 `ColumnTransformer`。接着在这个 `ColumnTransformer` 中查找名为`"geo"`的转换器，找到应用于纬度和经度属性的 `ClusterSimilarity` 转换器，最后找到这个转换器的 `n_clusters` 超参数。类似地，`random_forest__max_features` 指的是名为`"random_forest"`的估计器的 `max_features` 超参数，这个估计器就是 `RandomForestRegressor` 模型（第 6 章会解释 `max_features` 超参数）。

> 提示：把预处理步骤包装在 Scikit-Learn 流水线中，就可以和模型超参数一起调整预处理超参数。这是件好事，因为它们往往会相互作用。例如，增加 `n_clusters` 可能需要同时增加 `max_features`。如果拟合流水线的转换器计算代价很高，可以把流水线的 `memory` 参数设置为缓存目录路径。第一次拟合流水线时，Scikit-Learn 会把拟合后的转换器保存到该目录；之后如果使用相同的超参数再次拟合流水线，就会直接加载缓存的转换器。

`param_grid` 中有两个字典，因此 `GridSearchCV` 会先评估第一个字典指定的 `n_clusters` 与 `max_features` 的全部 3 × 3 = 9 种组合，然后再尝试第二个字典中的 2 × 3 = 6 种组合。总计会探索 9 + 6 = 15 种超参数组合。由于使用 3 折交叉验证，每种组合都要训练流水线 3 次，总共要训练 15 × 3 = 45 轮！这可能需要一段时间，但完成后可以这样获取最佳参数组合：

```
>>> grid_search.best_params_
{'preprocessing__geo__n_clusters': 15, 'random_forest__max_features': 6}
```

在本例中，把 **n_clusters** 设置为 15、把 **max_features** 设置为 6，可以得到最佳模型。

> 提示：由于 15 是此次评估的 **n_clusters** 最大值，因此最好用更高的值再次搜索；得分可能还会继续改善。

可以使用 **grid_search.best_estimator_** 访问最佳估计器。如果用 **refit=True** 初始化 **GridSearchCV**（这是默认值），它会先使用交叉验证找到最佳估计器，然后在完整训练集上重新训练该估计器。这通常是个好主意，因为使用更多数据训练很可能会提高性能。

评估分数可以通过 **grid_search.cv_results_** 获取。这是一个字典，但把它包装成 DataFrame 后，就能得到一个漂亮的列表，其中包含每种超参数组合、每个交叉验证划分的测试分数，以及所有划分的平均测试分数。

```
>>> cv_res = pd.DataFrame(grid_search.cv_results_)
>>> cv_res.sort_values(by="mean_test_score", ascending=False, inplace=True)
>>> [...]  # change column names to fit on this page, and show rmse = -score
>>> cv_res.head()  # note: the 1st column is the row ID
    n_clusters  max_features  split0  split1  split2  mean_test_rmse
12          15             6   42725   43708   44335           43590
13          15             8   43486   43820   44900           44069
6           10             4   43798   44036   44961           44265
9           10             6   43710   44163   44967           44280
7           10             6   43710   44163   44967           44280
```

最佳模型的平均测试 RMSE 是 43,590，比前面使用默认超参数得到的 47,038 更好。恭喜你，已经成功微调了最佳模型！

2.7.2　随机搜索（Randomized Search）

当探索的组合相对较少时，网格搜索很合适；但当超参数搜索空间很大时，**RandomizedSearchCV** 往往更好。这个类的使用方式与 **GridSearchCV** 基本相同，但它不会尝试全部可能的组合，而是评估固定数量的组合，每次迭代为每个超参数随机选择一个值。听起来也许有些出人意料，但这种方法有几个优点：

- 如果某些超参数是连续的（或者虽然离散但有很多可能值），并让随机搜索运行 1,000 次，那么它会为这些超参数分别探索 1,000 个不同值；网格搜索却只能对每个超参数探索你列出的少数几个值。
- 假设某个超参数实际上影响不大，但你还不知道。如果它有 10 个可能值，并把它加入网格搜索，训练时间会增加 10 倍；但如果把它加入随机搜索，训练时间不会因此改变。
- 如果要探索 6 个超参数，每个超参数有 10 个可能值，网格搜索只能训练一百万次模型，而随机搜索可以按照你选择的任意迭代次数运行。

对于每个超参数，都可以提供一个可能值列表或一个概率分布：

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

param_distribs = {'preprocessing__geo__n_clusters': randint(low=3, high=50),
                  'random_forest__max_features': randint(low=2, high=20)}
rnd_search = RandomizedSearchCV(
    full_pipeline, param_distributions=param_distribs, n_iter=10, cv=3,
    scoring='neg_root_mean_squared_error', random_state=42)
rnd_search.fit(housing, housing_labels)
```

Scikit-Learn 还提供 **HalvingRandomSearchCV** 和 **HalvingGridSearchCV** 超参数搜索类。它们的目标是更高效地使用计算资源，要么加快训练速度，要么探索更大的超参数空间。它们的工作方式如下：第一轮生成许多超参数组合（称为候选项），生成方式可以是网格搜索或随机搜索。然后像往常一样，通过交叉验证训练和评估这些候选模型。不过，训练使用的是有限资源，所以第一轮会明显加快。默认情况下，“有限资源”意味着模型只在训练集的一小部分上训练，但也可以使用其他限制，例如当模型有需要设置的迭代次数超参数时，减少训练迭代次数。所有候选模型评估完成后，只有表现最好的候选模型进入第二轮，获得更多资源进行竞争。经过多轮之后，最终候选模型才会使用完整资源进行评估。这可能会节省调整超参数所需的时间。

2.7.3　集成方法（Ensemble Methods）

另一种微调系统的方法，是尝试组合表现最好的模型。集成模型通常会比其中最好的单个模型表现更好，就像随机森林通常比它所依赖的单棵决策树表现更好一样，尤其是当各个模型产生的错误类型差异很大时。例如，可以训练并微调一个 k 近

邻模型，然后创建一个集成模型，让它预测随机森林预测值与该模型预测值的平均值。第 6 章会更详细地介绍这个主题。

2.7.4　分析最佳模型及其错误（Analyze the Best Models and Their Errors）

检查最佳模型往往能让你对问题获得很好的洞见。例如，`RandomForestRegressor` 可以显示各个属性对于作出准确预测的相对重要性：

```
>>> final_model = rnd_search.best_estimator_  # includes preprocessing
>>> feature_importances = final_model["random_forest"].feature_importances_
>>> feature_importances.round(2)
array([0.07, 0.05, 0.05, 0.01, 0.01, 0.01, 0.01, 0.19, [...], 0.  , 0.01])
```

接下来把这些重要性分数按降序排列，并显示对应的属性名称：

```
>>> sorted(zip(feature_importances,
...            final_model["preprocessing"].get_feature_names_out()),
...        reverse=True)
...
[(np.float64(0.18599734460509476), 'log__median_income'),
 (np.float64(0.07338850855844489), 'cat__ocean_proximity_INLAND'),
 (np.float64(0.06556941990883976), 'bedrooms__ratio'),
 (np.float64(0.053648710076725316), 'rooms_per_house__ratio'),
 (np.float64(0.04598870861894749), 'people_per_house__ratio'),
 (np.float64(0.04175269214442519), 'geo__Cluster 30 similarity'),
 (np.float64(0.025976797232869678), 'geo__Cluster 25 similarity'),
 (np.float64(0.023595895886342255), 'geo__Cluster 36 similarity'),
 [...]
 (np.float64(0.0004325970342247361), 'cat__ocean_proximity_NEAR BAY'),
 (np.float64(3.0190221102670295e-05), 'cat__ocean_proximity_ISLAND')]
```

有了这些信息，可以尝试删除不太有用的特征。例如，似乎只有一个 `ocean_proximity` 类别真正有用，因此可以尝试删除其他类别。

> 提示：`sklearn.feature_selection.SelectFromModel` 转换器可以自动删除最不重要的特征。拟合它时，会训练一个模型（通常是随机森林），查看它的 `feature_importances_` 属性，选择最有用的特征。之后调用 `transform()`，它就会删除其他特征。

还应该查看系统产生的具体错误，然后尝试理解错误原因以及如何修复，例如添加额外特征、删除没有信息量的特征、清理异常值等。

现在也是检查模型公平性的好时机：模型不仅要在平均意义上表现良好，还应该在不同类别的地区上表现良好，无论这些地区是农村还是城市、富裕还是贫穷、北部还是南部、少数族裔聚居区还是其他地区。为此需要进行详细的偏差分析：为验证集的每个类别创建子集，并分析模型在这些子集上的表现。这是一项繁重的工作，但很重要。如果模型在某个完整类别的地区上表现很差，那么在问题解决之前，通常不应部署模型；至少不应将其用于该类别的预测，否则可能弊大于利。

2.8　在测试集上评估系统（Evaluate the System on the Test Set）

对模型进行一段时间的调整后，最终会得到一个表现足够好的系统。现在可以在测试集上评估最终模型了。这个过程没有什么特别之处：从测试集中获取预测变量和标签，运行 `final_model` 转换数据并进行预测，然后评估预测结果：

```
X_test = strat_test_set.drop("median_house_value", axis=1)
y_test = strat_test_set["median_house_value"].copy()
final_predictions = final_model.predict(X_test)
final_rmse = root_mean_squared_error(y_test, final_predictions)
print(final_rmse)  # prints 41445.533268606625
```

有时，仅仅给出这样一个泛化误差点估计还不足以让你决定启动系统：如果它只比当前生产模型好 0.1% 呢？你可能希望知道这个估计有多精确。为此，可以使用 `scipy.stats.bootstrap()` 计算泛化误差的 95% 置信区间。得到的区间是 39,521 至 43,702，之前的点估计 41,445 大致位于区间中间：

```
from scipy import stats

def rmse(squared_errors):
    return np.sqrt(np.mean(squared_errors))

confidence = 0.95
squared_errors = (final_predictions - y_test) ** 2
boot_result = stats.bootstrap([squared_errors], rmse,
                              confidence_level=confidence, random_state=42)
rmse_lower, rmse_upper = boot_result.confidence_interval
```

如果大量调整超参数，性能通常会略差于使用交叉验证测得的结果。这是因为系统最终会针对验证数据进行微调，以获得良好的表现；在未知数据集上，它很可能表现得没那么好。本例并非如此，因为测试 RMSE 低于验证 RMSE，但如果出现这

种情况，必须抵制为了让测试集上的数字好看而调整超参数的诱惑；这种改进不太可能泛化到新数据。

现在进入项目启动前阶段。能否有效地展示解决方案，是优秀数据科学家与普通数据科学家之间的区别。应该创建简洁的报告（Markdown、PDF、幻灯片），可视化关键洞见（例如使用 Matplotlib，或 Seaborn、Tableau 等其他工具），并根据受众调整表达方式：面向同行时使用技术性语言，面向利益相关者时使用高层次语言。提供有影响力且容易记忆的陈述，例如“中位数收入是房价的第一预测指标”。突出说明学到了什么、哪些方法有效、哪些方法无效、作出了哪些假设，以及系统有哪些局限性。

结果应该尽可能可复现：让团队能够访问代码（例如通过 GitHub），为技术人员提供结构化的 README 文件，指导其完成安装步骤；提供清晰的 notebook，其中包含代码、解释和结果；编写干净、注释良好的代码。定义 **`requirements.txt`** 或 **`environment.yml`** 文件，列出所有需要的库及其精确版本，或者创建 Docker 镜像。为随机数生成器设置种子，并消除其他可能造成结果差异的因素。

在这个加利福尼亚州房价示例中，系统的最终性能并没有比专家估价好太多，专家估价往往会偏差 30%；但启动系统可能仍然是个好主意，尤其是它能为专家节省时间，让他们投入更有趣、更有成效的工作。

2.9　启动、监控和维护系统（Launch, Monitor, and Maintain Your System）

太好了，你已经获准启动系统！现在需要让解决方案做好投入生产的准备，例如完善代码、编写文档和测试等，然后把模型部署到生产环境。最基本的方法是保存训练出的最佳模型，把文件传到生产环境，再加载它。可以使用 **`joblib`** 库保存模型：

```
import joblib

joblib.dump(final_model, "my_california_housing_model.pkl")
```

> 提示：通常最好保存每个实验过的模型，这样以后可以方便地回到任意一个模型。也可以保存交叉验证分数，以及验证集上的实际预测值。这样就能轻松比较不同模型类型的分数，也能比较它们产生的错误类型。

模型传到生产环境后，就可以加载并使用它。首先必须导入模型所依赖的自定义类和函数，也就是说，需要把代码一并传到生产环境；然后使用 **joblib** 加载模型并进行预测：

```
import joblib

[...]  # import KMeans, BaseEstimator, TransformerMixin, rbf_kernel, etc.

def column_ratio(X): [...]

def ratio_name(function_transformer, feature_names_in): [...]

class ClusterSimilarity(BaseEstimator, TransformerMixin): [...]

final_model_reloaded = joblib.load("my_california_housing_model.pkl")
new_data = [...]  # some new districts to make predictions for
predictions = final_model_reloaded.predict(new_data)
```

例如，模型可能会在一个网站中使用：用户输入新地区的信息，点击“Estimate Price”按钮。网站会把包含这些数据的查询发送给 Web 服务器，服务器再把查询转发给 Web 应用，最后代码只需调用模型的 **predict()** 方法即可（应当在服务器启动时加载模型，而不是每次使用模型时都加载）。

也可以把模型包装在一个专用的 Web 服务中，Web 应用通过 REST API 访问这个服务。[14] 见图 2-21。这种方式便于升级模型版本，而不会中断主应用。它还简化了扩展过程，因为可以启动所需数量的 Web 服务，并在这些服务之间对来自 Web 应用的请求进行负载均衡。此外，Web 应用可以使用任意编程语言，而不仅仅是 Python。

[14] 简单来说，REST（或 RESTful）API 是一种基于 HTTP 的 API，遵循一些约定，例如使用标准 HTTP 动词读取、更新、创建或删除资源（GET、POST、PUT 和 DELETE），并使用 JSON 作为输入和输出。

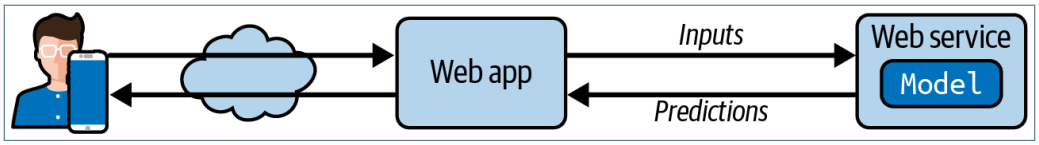

图 2-21: 部署为 Web 服务并由 Web 应用使用的模型（A model deployed as a web service and used by a web application）

另一种常见策略是把模型部署到云端，例如 Google 的 Vertex AI（以前称为 Google Cloud AI Platform 和 Google Cloud ML Engine）：只需使用 `joblib` 保存模型，将它上传到 Google Cloud Storage（GCS），然后进入 Vertex AI 创建一个新模型版本，并将其指向 GCS 中的文件。这样就完成了！Vertex AI 会提供一个简单的 Web 服务，为你处理负载均衡和扩展。它接收包含输入数据（例如某个地区信息）的 JSON 请求，并返回包含预测结果的 JSON 响应。之后可以在网站或正在使用的任何生产环境中调用这个 Web 服务。

但部署并不是故事的终点。还需要编写监控代码，定期检查系统的在线性能，并在性能下降时触发警报。性能可能会因为基础设施中的组件发生故障而迅速下降，但也要注意，它也可能缓慢衰退，很长时间都不容易被发现。这很常见，因为可能发生数据漂移：如果模型使用去年的数据训练，那么它可能不适应今天的数据。

因此，需要监控模型的在线性能。但应该如何监控呢？这要视情况而定。在某些情况下，可以从下游指标推断模型性能。例如，如果模型是推荐系统的一部分，会推荐用户可能感兴趣的产品，那么很容易监控每天售出的推荐产品数量。如果这个数量（与非推荐产品相比）下降，首要怀疑对象就是模型。这可能是因为数据流水线损坏，也可能是因为需要使用新数据重新训练模型。

不过，有时还需要人工分析来评估模型性能。例如，假设训练了一个图像分类模型（第 3 章将介绍这类模型），用于检测生产线上的各种产品缺陷。怎样才能在数千件有缺陷的产品被发给客户之前，及时发现模型性能下降并发出警报？一种解决方案是把模型分类过的所有图片中抽取一部分，发送给人工评分员（尤其是模型不太确定的图片）。根据任务不同，评分员可能需要是专家，也可能是不具备专业知识的人员，例如众包平台 Amazon Mechanical Turk 上的工作人员；在某些应用中，甚至可以让用户自己参与，例如通过调查问卷或改造后的验证码反馈。[15]

[15]验证码是一种用于确认用户不是机器人的测试。验证码经常被用作一种廉价方式，为训练数据添加标签。

无论采用哪种方式，都需要建立监控系统（可以有人工评分员，也可以没有），并建立相关流程，明确发生故障时应该做什么以及如何为故障做好准备。不幸的是，这可能是一项很大的工作。实际上，它往往比构建和训练模型本身还要耗费更多工作。

如果数据持续演化，就需要定期更新数据集并重新训练模型。整个过程应尽可能自动化。下面是一些可以自动化的工作：

- 定期收集新数据并为其添加标签，例如使用人工评分员。
- 编写脚本，自动训练模型并微调超参数。根据需要，这个脚本可以每天或每周自动运行。
- 再编写一个脚本，在更新后的测试集上评估新模型和旧模型；如果性能没有下降，就将模型部署到生产环境（如果性能下降，必须调查原因）。这个脚本还应该在测试集的不同子集上评估模型，例如贫穷或富裕地区、农村或城市地区等。

还要确保评估模型输入数据的质量。有时性能轻微下降，是因为输入信号质量较差，例如传感器发生故障并发送随机值，或者另一个团队的输出变得过时；但在系统性能下降到足以触发警报之前，可能需要一段时间。如果监控模型的输入，就可能更早捕获问题。例如，当越来越多的输入缺少某个特征、均值或标准差与训练集偏离过远，或者某个类别特征开始包含新类别时，可以触发警报。

最后，必须备份创建过的每个模型，并建立流程和工具，以便新模型因某种原因严重失败时迅速回滚到旧模型。保留备份还可以轻松比较新模型和旧模型。同样，也应备份数据集的每个版本。如果新数据被加入数据集后，结果发现其中充满异常值，数据集发生损坏，就可以回滚到以前的版本。数据集备份还可以让你用任意模型评估任意旧数据集。

可以看到，机器学习涉及相当多的基础设施。这是一个非常宽泛的主题，叫作机器学习运维（ML Operations，MLOps），值得单独写一本书。因此，如果第一个机器学习项目需要投入大量精力和时间才能构建并部署到生产环境，不要感到惊讶。好在一旦基础设施就绪，从想法走向生产的速度会快得多。

2.10　试试看！（Try It Out!）

希望本章让你了解机器学习项目是什么样的，也展示了一些可以用来训练优秀系统的工具。可以看到，大量工作都集中在数据准备阶段：构建监控工具、建立人工

评估流水线，以及自动化定期的模型训练。当然，机器学习算法很重要，但与其把所有时间都花在探索高级算法上，不如先熟悉完整流程，并深入掌握三四种算法。

如果还没有这样做，现在正是拿起笔记本电脑，选择一个感兴趣的数据集，尝试从头到尾走完整个流程的好时机。Kaggle 之类的竞赛网站是很好的起点：那里有可供操作的数据集、清晰的目标，以及可以分享经验的人。祝你玩得开心！

2.11　练习（Exercises）

下面的练习都基于本章使用的 housing 数据集：

1. 尝试使用支持向量机回归器（`sklearn.svm.SVR`），并尝试各种超参数，例如 `kernel="linear"`（尝试不同的 `C` 超参数值），或者 `kernel="rbf"`（尝试不同的 `C` 和 `gamma` 超参数值）。注意，支持向量机不适合扩展到大型数据集，因此最好只在训练集的前 5,000 个实例上训练模型，并且只使用 3 折交叉验证，否则可能需要数小时。暂时不必担心这些超参数的含义；在线 SVM 章节会解释它们，网址为 https://homl.info/。最佳 SVR 预测器的表现如何？
2. 尝试用 `RandomizedSearchCV` 替换 `GridSearchCV`。
3. 尝试在预处理流水线中添加 `SelectFromModel` 转换器，只选择最重要的属性。
4. 尝试创建一个自定义转换器，在其 `fit()` 方法中训练 k 近邻回归器（`sklearn.neighbors.KNeighborsRegressor`），并在其 `transform()` 方法中输出模型预测值。KNN 回归器只使用纬度和经度作为输入，并预测中位数收入。然后把这个新转换器添加到预处理流水线中。这将增加一个表示附近地区平滑中位数收入的特征。
5. 使用 `RandomizedSearchCV` 自动探索一些数据准备选项。
6. 尝试从头实现一次 `StandardScalerClone` 类，然后添加对 `inverse_transform()` 方法的支持：执行 `scaler.inverse_transform(scaler.fit_transform(X))` 应返回一个与 `X` 非常接近的数组。接着添加对特征名称的支持：如果输入是 DataFrame，就在 `fit()` 方法中设置 `feature_names_in_` 属性，该属性应为包含列名的 NumPy 数组。最后实现 `get_feature_names_out()` 方法，它应包含一个可选输入参数 `input_features=None`。如果传入该参数，方法应检查它的长度是否与 `n_features_in_` 相同，并且如果已经定义 `feature_names_in_`，还应检查两者是否一致；之后返回 `input_features`。如果 `input_features` 为 `None`，则

如果已经定义 `feature_names_in_`，返回它；否则返回长度为 `n_features_in_` 的 `np.array(["x0", "x1", ...])`。

7. 按照本章学习的流程，处理你选择的一个回归任务。例如，可以尝试 Vehicle 数据集，目标是根据二手车的车龄、行驶里程、品牌和型号等信息预测售价。另一个不错的数据集是 Bike Sharing 数据集：目标是根据星期几、时间和天气状况，预测某个时间段内租出的自行车数量（列名为 `cnt`）。

这些练习的答案位于本章 notebook 的末尾，网址为 https://homl.info/colab-p。

本章使用的术语和代码接口包括 `train-dev set`、`out-of-sample error`、`REST API`、`MLOps` 等，均按术语库保留英文对照，便于后续章节交叉阅读。

第 3 章　分类

Classification

在第 1 章中，我提到最常见的监督学习任务是回归（预测数值）和分类（预测类别）。在第 2 章中，我们探讨了一个预测房价的回归任务，使用了线性回归、决策树和随机森林等多种算法（后续章节会进一步详细介绍这些算法）。现在，我们把注意力转向分类系统。

3.1　MNIST（The MNIST Dataset）

本章将使用 MNIST 数据集。它包含 70,000 张由美国高中生和美国人口普查局员工手写的数字小图像，每张图像都标注了所代表的数字。这个数据集被研究得如此广泛，以至于常被称为机器学习的“Hello World”：每当有人提出一种新的分类算法，都会想看看它在 MNIST 上的表现；任何学习机器学习的人迟早都会接触这个数据集。

Scikit-Learn 提供了许多用于下载常见数据集的辅助函数，MNIST 就是其中之一。下面的代码从 OpenML.org 获取 MNIST 数据集：[1]

```
from sklearn.datasets import fetch_openml
mnist = fetch_openml('mnist_784', as_frame=False)
```

`sklearn.datasets` 包主要包含三类函数：`fetch_*` 函数（例如 `fetch_openml()`）用于下载真实世界的数据集；`load_*` 函数用于加载 Scikit-Learn 自带的小型玩具数据集，因此不需要通过互联网下载；`make_*` 函数用于生成适合测试的虚假数据集。生成的数据集通常以包含输入数据和目标值的 `(X, y)` 元组返回，两者都是 NumPy 数组。其他数据集则以 `sklearn.utils.Bunch` 对象返回，这类对象本质上是字典，但其条目也可以作为属性访问。它们通常包含以下条目：

- `"DESCR"`

[1]默认情况下，Scikit-Learn 会把下载的数据集缓存在用户主目录下名为 `scikit_learn_data` 的目录中。

数据集的描述。

- `"data"`

  输入数据，通常是二维 NumPy 数组。

- `"target"`

  标签，通常是一维 NumPy 数组。

`fetch_openml()` 有一点不同：默认情况下，它会把输入返回为 Pandas DataFrame，把标签返回为 Pandas Series（数据集稀疏时除外）。但 MNIST 数据集包含图像，DataFrame 并不理想，因此最好设置 `as_frame=False`，让数据以 NumPy 数组返回。下面来看看这些数组：

```
>>> X, y = mnist.data, mnist.target
>>> X
array([[0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       ...,
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0]])
>>> X.shape
(70000, 784)
>>> y
```

```
array(['5', '0', '4', ..., '4', '5', '6'], dtype=object)
>>> y.shape
(70000,)
```

共有 70,000 张图像，每张图像有 784 个特征。这是因为每张图像的尺寸为 28 x 28 像素，每个特征表示一个像素的强度，范围从 0（白色）到 255（黑色）。我们来看看数据集中的一张数字图像（见图 3-1）。只需取出一个实例的特征向量，将其重塑为 28 x 28 的数组，再使用 Matplotlib 的 `imshow()` 函数显示即可。这里使用 `cmap="binary"` 得到灰度色图，其中 0 为白色、255 为黑色：

```
import matplotlib.pyplot as plt
def plot_digit(image_data):
    image = image_data.reshape(28, 28)
    plt.imshow(image, cmap="binary")
    plt.axis("off")
some_digit = X[0]
plot_digit(some_digit)
plt.show()
```

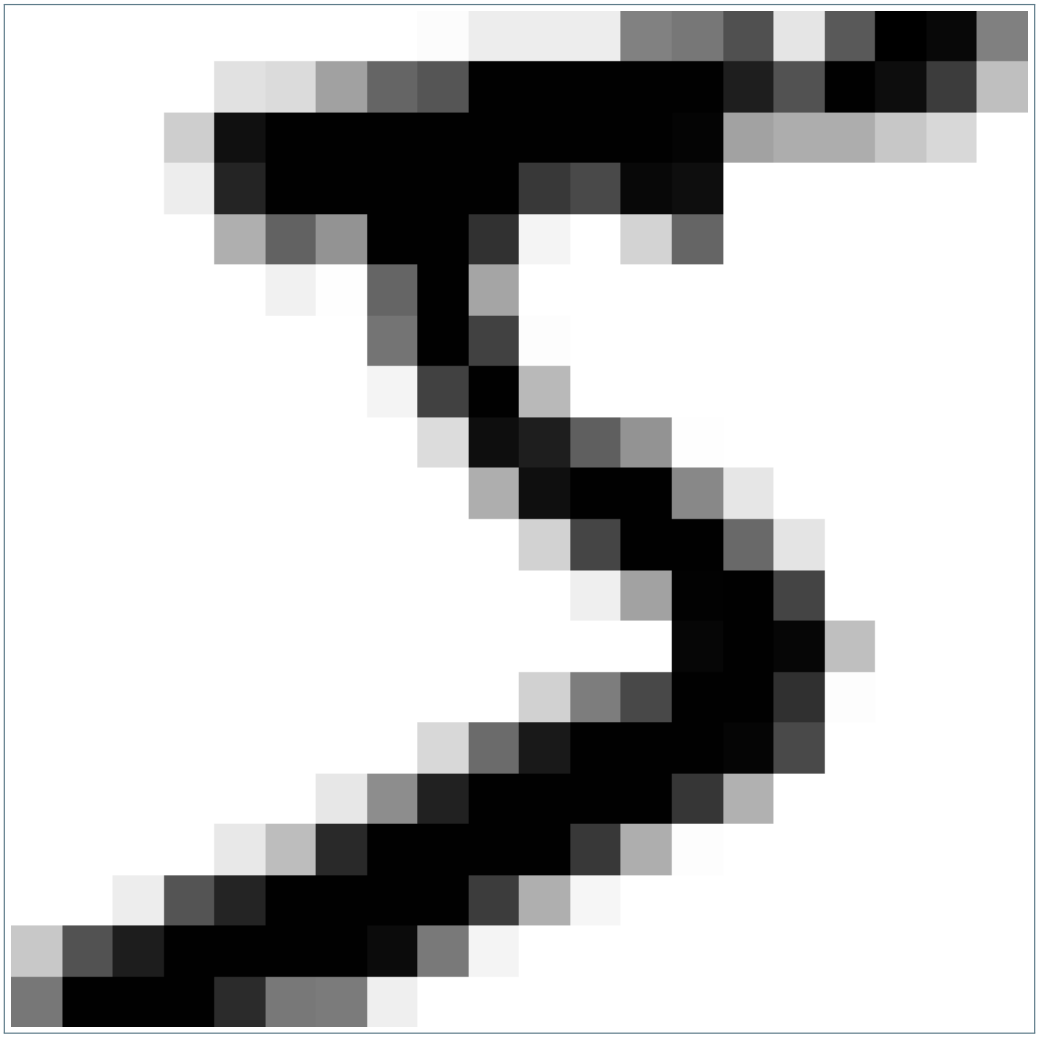

图 3-1: MNIST 图像示例（Example of an MNIST image）

这看起来像数字 5，标签也确实如此：

```
>>> y[0]
'5'
```

为了让你感受分类任务的复杂程度，图 3-2 展示了 MNIST 数据集中的更多图像。数字的形状差异相当大。不过，这些图像干净、居中，没有过度旋转，而且数字大小大致相同：这个数据集不需要太多预处理（现实世界的数据集通常不会这么友好）。

图 3-2: MNIST 数据集中的数字（Digits from the MNIST dataset）

但是先别急！在仔细查看数据之前，你总应该先创建一个测试集并将其搁置一旁。`fetch_openml()` 返回的 MNIST 数据集实际上已经分成训练集（前 60,000 张图像）和测试集（后 10,000 张图像）：[2]

```
X_train, X_test, y_train, y_test = X[:60000], X[60000:], y[:60000], y[60000:]
```

训练集已经替我们打乱，这很好，因为这样可以保证所有交叉验证折都比较相似（我们不希望某个折缺少某些数字）。此外，有些学习算法对训练实例的顺序比较敏感，如果连续遇到许多相似实例，表现就会很差。打乱数据集可以避免这种情况。[3]

3.2　训练二分类器（Training a Binary Classifier）

现在暂时简化问题，只识别一个数字，例如数字 5。这个“5 检测器”就是一个二分类器，只负责区分两个类别：5 和非 5。首先为这个分类任务创建目标向量：

```
y_train_5 = (y_train == '5')  # True for all 5s, False for all other digits
y_test_5 = (y_test == '5')
```

接下来选择一个分类器并训练它。一个很好的起点是随机梯度下降（stochastic gradient descent，SGD，也称 stochastic GD）分类器，使用 Scikit-Learn 的 `SGDClassifier` 类。这个分类器能够高效处理非常大的数据集，部分原因是 SGD 每次独立处理一个训练实例，这也使它很适合在线学习（后文会介绍）。现在创建一个 `SGDClassifier`，并在整个训练集上训练：

```
from sklearn.linear_model import SGDClassifier
sgd_clf = SGDClassifier(random_state=42)
sgd_clf.fit(X_train, y_train_5)
```

现在可以用它检测数字 5 的图像：

[2] 由 `fetch_openml()` 返回的数据集并不总是已经打乱或分割好的。

[3] 在某些场景中，打乱数据可能是坏主意，例如处理股票价格或天气状况等时间序列数据时。我们会在第 13 章中探讨这一点。

```
>>> sgd_clf.predict([some_digit])
array([ True])
```

分类器判断这张图像代表 5（True）。看起来这次猜对了！下面评估这个模型的性能。

3.3　性能指标（Performance Measures）

评估分类器通常比评估回归器棘手得多，因此本章会花很大篇幅讨论这个主题。可用的性能指标很多，准备好学习一批新的概念和缩写吧！

3.3.1　使用交叉验证测量准确率（Measuring Accuracy Using Cross-Validation）

评估模型的一种好方法是使用交叉验证，就像第 2 章中所做的那样。使用 cross_val_score() 函数，以三折交叉验证评估 SGDClassifier 模型。记住，三折交叉验证就是把训练集分成 3 个折，然后训练模型 3 次，每次留出不同的折进行评估（见第 2 章）：

```
>>> from sklearn.model_selection import cross_val_score
>>> cross_val_score(sgd_clf, X_train, y_train_5, cv=3, scoring="accuracy")
array([0.95035, 0.96035, 0.9604 ])
```

哇！所有交叉验证折的准确率（正确预测所占比例）都超过 95%？看起来很惊人，对吧？先别太兴奋，我们来看看一个虚拟分类器：它把每张图像都预测为出现频率最高的类别，也就是负类（非 5）：

```
from sklearn.dummy import DummyClassifier
dummy_clf = DummyClassifier()
dummy_clf.fit(X_train, y_train_5)
print(any(dummy_clf.predict(X_train)))  # prints False: no 5s detected
```

你猜这个模型的准确率是多少？来看看：

```
>>> cross_val_score(dummy_clf, X_train, y_train_5, cv=3, scoring="accuracy")
array([0.90965, 0.90965, 0.90965])
```

没错，它的准确率超过了 90%！这只是因为图像中只有约 10% 是数字 5，所以如果你永远猜图像不是 5，大约 90% 的时候都能猜对。真是胜过诺查丹玛斯。

这说明，对于分类器来说，准确率通常不是首选性能指标，尤其是在处理偏斜数据集时（即某些类别出现频率远高于其他类别）。更好的评估方式是查看混淆矩阵（confusion matrix，CM）。

实现交叉验证

有时你需要比 Scikit-Learn 开箱即用的功能更好地控制交叉验证过程。这时可以自行实现交叉验证。下面的代码大致完成了与 Scikit-Learn 的 **`cross_val_score()`** 函数相同的工作，并打印相同的结果：

```
from sklearn.model_selection import StratifiedKFold
from sklearn.base import clone
skfolds = StratifiedKFold(n_splits=3)  # add shuffle=True if the dataset is
                                       # not already shuffled
for train_index, test_index in skfolds.split(X_train, y_train_5):
    clone_clf = clone(sgd_clf)
    X_train_folds = X_train[train_index]
    y_train_folds = y_train_5[train_index]
    X_test_fold = X_train[test_index]
    y_test_fold = y_train_5[test_index]
    clone_clf.fit(X_train_folds, y_train_folds)
    y_pred = clone_clf.predict(X_test_fold)
    n_correct = sum(y_pred == y_test_fold)
    print(n_correct / len(y_pred))  # prints 0.95035, 0.96035, and 0.9604
```

`StratifiedKFold` 类执行分层抽样（第 2 章已解释），生成包含各类别代表性比例的折。每次迭代时，代码都会克隆一个分类器，在训练折上训练克隆对象，然后在测试折上进行预测。接着统计正确预测的数量，并输出正确预测的比例。

3.3.2　混淆矩阵（The Confusion Matrix）

混淆矩阵的基本思想，是针对所有 A/B 类别对，统计 A 类实例被分类为 B 类的次数。例如，要知道分类器把多少个数字 8 误判成 0，就查看混淆矩阵第 8 行、第 0 列的值。

要计算混淆矩阵，首先需要一组预测值，用来与实际目标进行比较。可以在测试集上进行预测，但目前最好保持测试集不变（记住，只有在项目末尾、分类器准备上线时，才使用测试集）。因此，可以使用 **cross_val_predict()** 函数：

```
from sklearn.model_selection import cross_val_predict
y_train_pred = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3)
```

与 **cross_val_score()** 一样，**cross_val_predict()** 会执行 k 折交叉验证，但它不返回评估分数，而是返回每个测试折产生的预测结果。这样，对于训练集中的每个实例，都能得到一个干净的预测（这里的“干净”是指“样本外”：模型从未在该数据上训练过）。

现在可以使用 **confusion_matrix()** 函数计算混淆矩阵。传入目标类别（**y_train_5**）和预测类别（**y_train_pred**）：

```
>>> from sklearn.metrics import confusion_matrix
>>> cm = confusion_matrix(y_train_5, y_train_pred)
>>> cm
array([[53892,   687],
       [ 1891,  3530]])
```

混淆矩阵的每一行代表一个实际类别，每一列代表一个预测类别。第一行表示非 5 图像（负类）：其中 53,892 张被正确分类为非 5（称为真负例），其余 687 张被错误分类为 5（称为假正例，也称 I 型错误）。第二行表示数字 5 图像（正类）：其中 1,891 张被错误分类为非 5（称为假负例，也称 II 型错误），其余 3,530 张被正确分类为 5（称为真正例）。完美分类器只会产生真正例和真负例，因此其混淆矩阵只有主对角线（从左上到右下）上有非零值：

```
>>> y_train_perfect_predictions = y_train_5  # pretend we reached perfection
>>> confusion_matrix(y_train_5, y_train_perfect_predictions)
array([[54579,     0],
       [    0,  5421]])
```

混淆矩阵包含大量信息，但有时你可能更喜欢一个简洁的指标。一个值得关注的指标是正类预测的准确率，这称为分类器的精确率（precision，见公式 3-1）。

$$\text{precision} = \frac{TP}{TP + FP} \tag{3-1}$$

公式 3-1　精确率（precision）

其中，TP 是真正例数量，FP 是假正例数量。

现在考虑一个只有在极其有把握时才做出正类预测的模型。把这种做法推向极端：假设模型总是预测负类，只有对它最有把握的一个实例做出正类预测。如果这一次预测正确，那么分类器的精确率就是 100%（精确率 = 1/1 = 100%）。显然，这样的分类器没有多大用处，因为它忽略了除一个实例之外的所有正类实例。因此，精确率通常要与另一个名为召回率（recall）的指标一起使用。召回率也称灵敏度（sensitivity）或真正例率（true positive rate，TPR），表示被分类器正确检测出的正类实例所占的比例（见公式 3-2）。

$$\text{recall} = \frac{TP}{TP + FN} \tag{3-2}$$

公式 3-2　召回率（recall）

当然，FN 是假负例数量。

如果你对混淆矩阵感到困惑，图 3-3 可能会有所帮助。

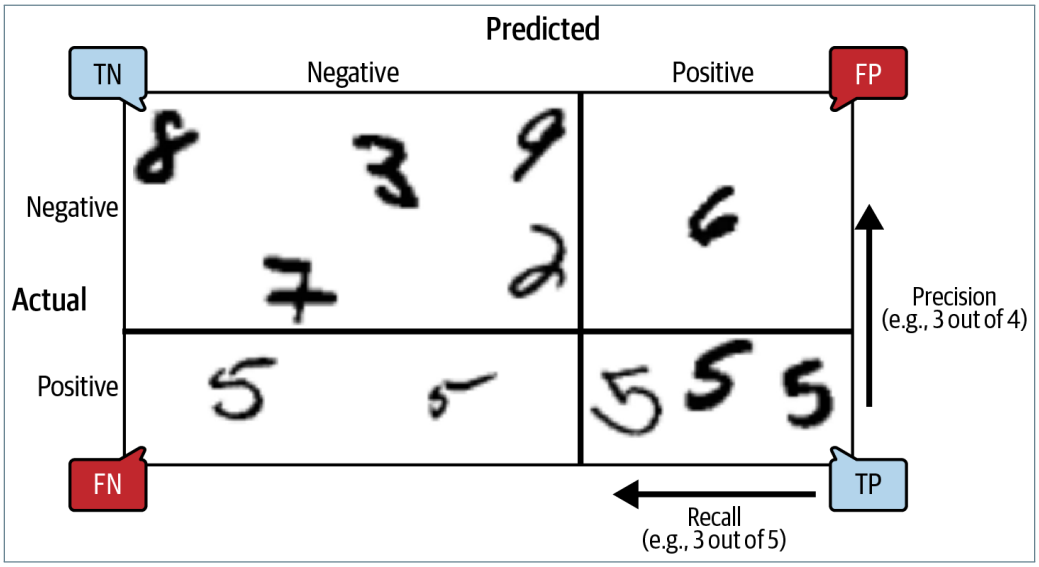

图 3-3: 展示真负例（左上）、假正例（右上）、假负例（左下）和真正例（右下）的混淆矩阵示意图（An illustrated confusion matrix showing examples of true negatives, false positives, false negatives, and true positives）

3.3.3　精确率和召回率（Precision and Recall）

Scikit-Learn 提供了多个计算分类器指标的函数，包括精确率和召回率：

```
>>> from sklearn.metrics import precision_score, recall_score
>>> precision_score(y_train_5, y_train_pred)  # == 3530 / (687 + 3530)
np.float64(0.8370879772350012)
>>> recall_score(y_train_5, y_train_pred)  # == 3530 / (1891 + 3530)
np.float64(0.6511713705958311)
```

现在，这个 5 检测器不像查看准确率时那么光鲜了。它声称某张图像代表 5 时，只有 83.7% 的概率是正确的；此外，它只能检测出 65.1% 的数字 5。

把精确率和召回率合并为一个称为 F1 分数的指标通常很方便，尤其是在需要用单个指标比较两个分类器时。F1 分数是精确率和召回率的调和平均值（见公式 3-3）。普通平均值会平等对待所有数值，而调和平均值会对较小的值赋予更大权重。因此，只有当召回率和精确率都很高时，分类器才能获得较高的 F1 分数。

$$F_1 = \frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} = \frac{TP}{TP + \frac{FN+FP}{2}} \tag{3-3}$$

公式 3-3　F1 分数（F1 score）

计算 F1 分数只需调用 `f1_score()` 函数：

```
>>> from sklearn.metrics import f1_score
>>> f1_score(y_train_5, y_train_pred)
0.7325171197343846
```

F1 分数偏好精确率和召回率相近的分类器，但这并不总是你想要的结果：有些场景更关注精确率，另一些场景则更关注召回率。例如，如果训练分类器检测适合儿童观看的视频，你可能更希望它拒绝许多优质视频（低召回率），但只保留安全视频（高精确率），而不是选择召回率更高却让少数真正糟糕的视频出现在产品中的分类器（这种情况下，甚至可以增加人工流程来检查分类器筛选出的视频）。另一方面，假设训练分类器检测监控图像中的商店扒窃者，那么即使精确率只有 30%，只要召回率达到 99% 也可能完全可以接受。保安会收到一些误报，但几乎所有扒窃者都会被抓住。类似地，医学诊断通常需要较高的召回率，以免漏掉任何重要情况；假正例可以通过后续医学检查排除。

遗憾的是，精确率和召回率不能两全：提高精确率会降低召回率，反之亦然。这称为精确率/召回率权衡。

3.3.4　精确率/召回率权衡（The Precision/Recall Trade-off）

为了理解这种权衡，来看看 `SGDClassifier` 如何做出分类决策。对于每个实例，它会根据一个决策函数计算分数。如果分数高于阈值，就把实例分到正类；否则分到负类。图 3-4 展示了几个数字，从左侧最低分到右侧最高分排列。假设决策阈值位于中央箭头处（两个 5 之间）：阈值右侧有 4 个真正例（实际为 5）和 1 个假正例（实际为 6）。因此，在这个阈值下，精确率为 80%（5 个中有 4 个），而 6 个实际为 5 的图像中只有 4 个被检测出来，所以召回率为 67%（6 个中有 4 个）。如果提高阈值（将阈值移动到右侧箭头处），假正例（6）会变成真负例，精确率随之提高（本例中达到 100%），但一个真正例会变成假负例，使召回率降至 50%。反过来，降低阈值会提高召回率并降低精确率。

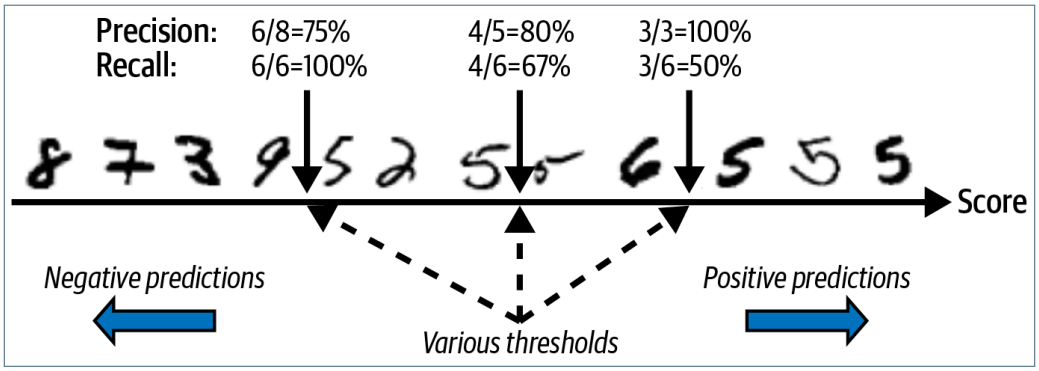

图 3-4: 精确率/召回率权衡：图像按分类器分数排列，高于所选决策阈值的图像被视为正类；阈值越高，召回率越低，但通常精确率越高（The precision/recall trade-off）

不要调用分类器的 `predict()` 方法，而是调用它的 `decision_function()` 方法，该方法会返回每个实例的分数。然后可以根据这些分数使用任意阈值做出预测：

```
>>> y_scores = sgd_clf.decision_function([some_digit])
>>> y_scores
array([2164.22030239])
>>> threshold = 0
>>> y_some_digit_pred = (y_scores > threshold)
>>> y_some_digit_pred
array([ True])
```

`SGDClassifier` 使用 0 作为阈值，因此上面的代码与 `predict()` 方法返回相同结果（即 `True`）。现在提高阈值：

```
>>> threshold = 3000
>>> y_some_digit_pred = (y_scores > threshold)
>>> y_some_digit_pred
array([False])
```

这证实了提高阈值会降低召回率。这张图像实际代表 5，阈值为 0 时分类器能检测出来，但阈值提高到 3,000 后就漏掉了它。

应该如何决定使用哪个阈值？一种方法是使用 `cross_val_predict()` 函数获取训练集中所有实例的分数，但这次指定返回决策分数而不是预测结果：

```
y_scores = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3,
                             method="decision_function")
```

利用这些分数，可以使用 **precision_recall_curve()** 函数计算所有可能阈值对应的精确率和召回率（该函数会额外添加一个精确率为 1、召回率为 0 的末尾点，对应无穷大的阈值）：

```
from sklearn.metrics import precision_recall_curve
precisions, recalls, thresholds = precision_recall_curve(y_train_5, y_scores)
```

最后，使用 Matplotlib 绘制精确率和召回率随阈值变化的曲线（见图 3-5）。这里显示所选的 3,000 阈值：

```
plt.plot(thresholds, precisions[:-1], "b--", label="Precision", linewidth=2)
plt.plot(thresholds, recalls[:-1], "g-", label="Recall", linewidth=2)
plt.vlines(threshold, 0, 1.0, "k", "dotted", label="threshold")
[...]  # beautify the figure: add grid, legend, axis, labels, and circles
plt.show()
```

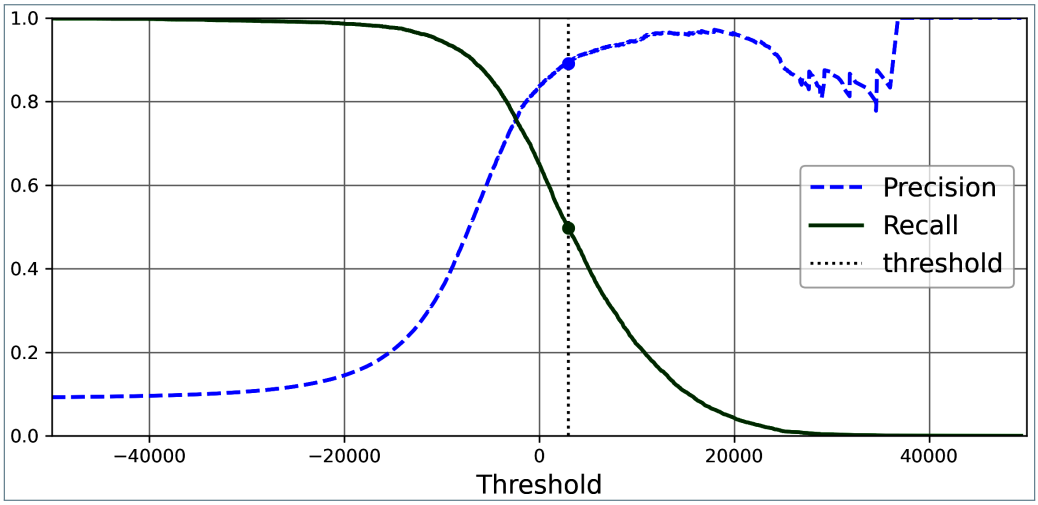

图 3-5: 精确率和召回率与决策阈值的关系（Precision and recall versus the decision threshold）

你可能会疑惑，为什么图 3-5 中的精确率曲线比召回率曲线更不平滑。原因是，提高阈值时精确率有时会下降（尽管总体上会提高）。回看图 3-4，从中央阈值向右移动一个数字时，精确率会从 4/5（80%）降到 3/4（75%）。另一方面，提高阈值时召回率只会下降，这就解释了为什么它的曲线看起来很平滑。

在这个阈值下，精确率接近 90%，召回率约为 50%。另一种选择合适精确率/召回率权衡的方法，是直接绘制精确率与召回率的关系图，如图 3-6 所示（图中也显示了相同阈值）：

```
plt.plot(recalls, precisions, linewidth=2, label="Precision/Recall curve")
[...]  # beautify the figure: add labels, grid, legend, arrow, and text
plt.show()
```

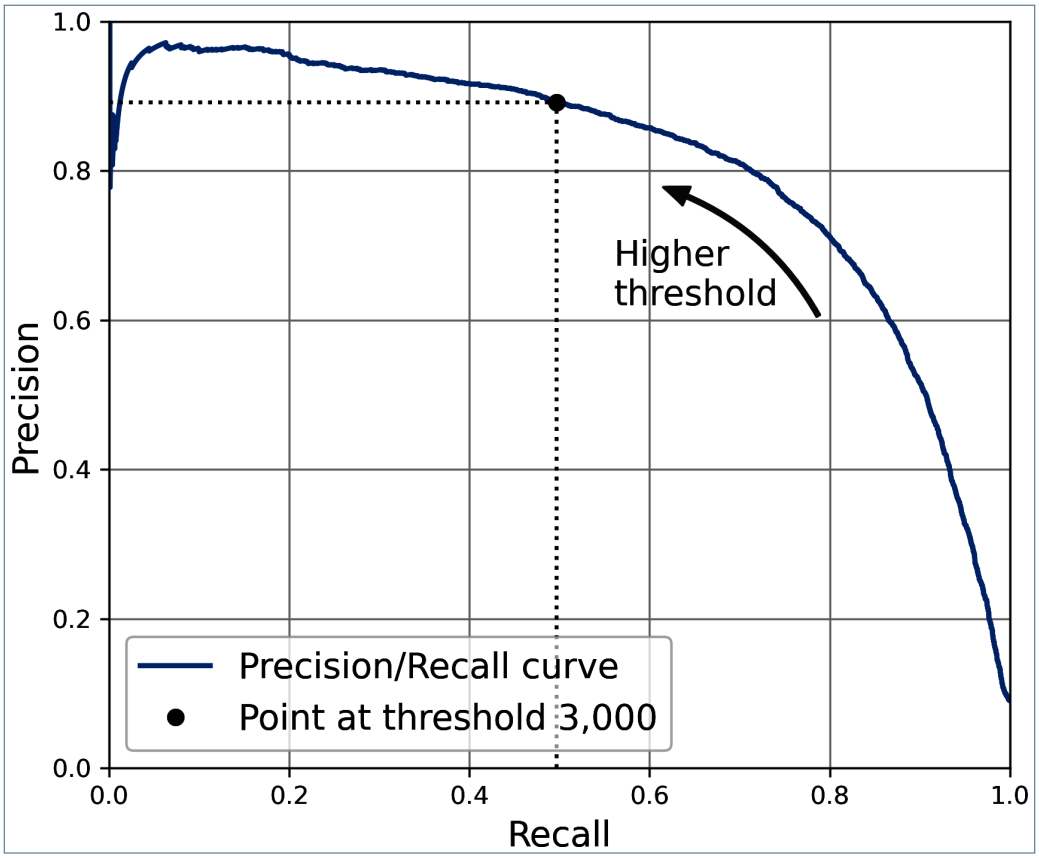

图 3-6: 精确率与召回率的关系（Precision versus recall）

可以看到，召回率达到约 80% 后，精确率开始急剧下降。你可能希望在下降之前选择一个精确率/召回率权衡点，例如召回率约为 60% 的位置。当然，最终选择取决于你的项目。

假设你决定以 90% 的精确率为目标。可以使用第一张图查找所需阈值，但这种方法不够精确。另一种方法是搜索能够达到至少 90% 精确率的最低阈值。可以使用 NumPy 数组的 `argmax()` 方法，它返回最大值的第一个索引，也就是第一个 `True` 值：

```
>>> idx_for_90_precision = (precisions >= 0.90).argmax()
>>> threshold_for_90_precision = thresholds[idx_for_90_precision]
>>> threshold_for_90_precision
np.float64(3370.0194991439557)
```

暂时在训练集上进行预测时，不要调用分类器的 **predict()** 方法，而是运行下面的代码：

```
y_train_pred_90 = (y_scores >= threshold_for_90_precision)
```

来检查这些预测的精确率和召回率：

```
>>> precision_score(y_train_5, y_train_pred_90)
0.9000345901072293
>>> recall_at_90_precision = recall_score(y_train_5, y_train_pred_90)
>>> recall_at_90_precision
0.4799852425751706
```

很好，你得到了一个精确率为 90% 的分类器。可以看到，创建一个具有几乎任意精确率的分类器相当容易：只要把阈值设置得足够高即可。但是先别急着下结论：如果召回率太低，高精确率分类器也没什么用！对许多应用来说，48% 的召回率一点也不理想。

> 提示：如果有人说“我们要达到 99% 的精确率”，你应该问：“召回率是多少？”

从 Scikit-Learn 1.5 开始，有两个新类可以更方便地调整决策阈值：

- **FixedThresholdClassifier** 类可以包装一个二分类器，并手动设置所需阈值。如果底层分类器有 **predict_proba()** 方法，阈值应为 0 到 1 之间的值（默认值为 0.5）；否则，阈值应为与模型 **decision_function()** 输出相当的决策分数（默认值为 0）。
- **TunedThresholdClassifierCV** 类使用 k 折交叉验证，自动为给定指标寻找最佳阈值。默认情况下，它尝试找到能最大化模型平衡准确率的阈值，即各类别召回率的平均值。不过，也可以选择其他要优化的指标（完整选项请参阅文档）。

3.3.5　ROC 曲线（The ROC Curve）

受试者工作特征（receiver operating characteristic，ROC）曲线是二分类器的另一个常用工具。它与精确率/召回率曲线非常相似，但 ROC 曲线绘制的不是精确率与召回率，而是真正例率（召回率的另一个名称）与假正例率（false positive rate，FPR）的关系。FPR（也称误报率）是错误分类为正类的负类实例所占的比例，等于 1 - 真负例率（true negative rate，TNR）；TNR 是被正确分类为负类的负类实例所占的比例，也称特异度（specificity）。因此，ROC 曲线绘制的是灵敏度（召回率）与 1 - 特异度的关系。

首先使用 **`roc_curve()`** 函数计算各种阈值对应的 TPR 和 FPR：

```
from sklearn.metrics import roc_curve
fpr, tpr, thresholds = roc_curve(y_train_5, y_scores)
```

然后用 Matplotlib 绘制 FPR 与 TPR 的关系。下面的代码生成图 3-7。要找到对应 90% 精确率的点，需要寻找目标阈值的索引。由于本例中的阈值按降序排列，第一行使用 **`<=`** 而不是 **`>=`**：

```
idx_for_threshold_at_90 = (thresholds <= threshold_for_90_precision).argmax()
tpr_90, fpr_90 = tpr[idx_for_threshold_at_90], fpr[idx_for_threshold_at_90]
plt.plot(fpr, tpr, linewidth=2, label="ROC curve")
plt.plot([0, 1], [0, 1], 'k:', label="Random classifier's ROC curve")
plt.plot([fpr_90], [tpr_90], "ko", label="Threshold for 90% precision")
[...]  # beautify the figure: add grid, legend, arrow, and text
plt.show()
```

同样存在权衡：召回率（TPR）越高，分类器产生的假正例（FPR）就越多。虚线表示纯随机分类器的 ROC 曲线；优秀的分类器应尽可能远离这条线，朝向左上角。

比较分类器的一种方法是测量曲线下面积（area under the curve，AUC）。完美分类器的 ROC AUC 等于 1，纯随机分类器的 ROC AUC 等于 0.5。Scikit-Learn 提供了估计 ROC AUC 的函数：

```
>>> from sklearn.metrics import roc_auc_score
>>> roc_auc_score(y_train_5, y_scores)
np.float64(0.9604938554008616)
```

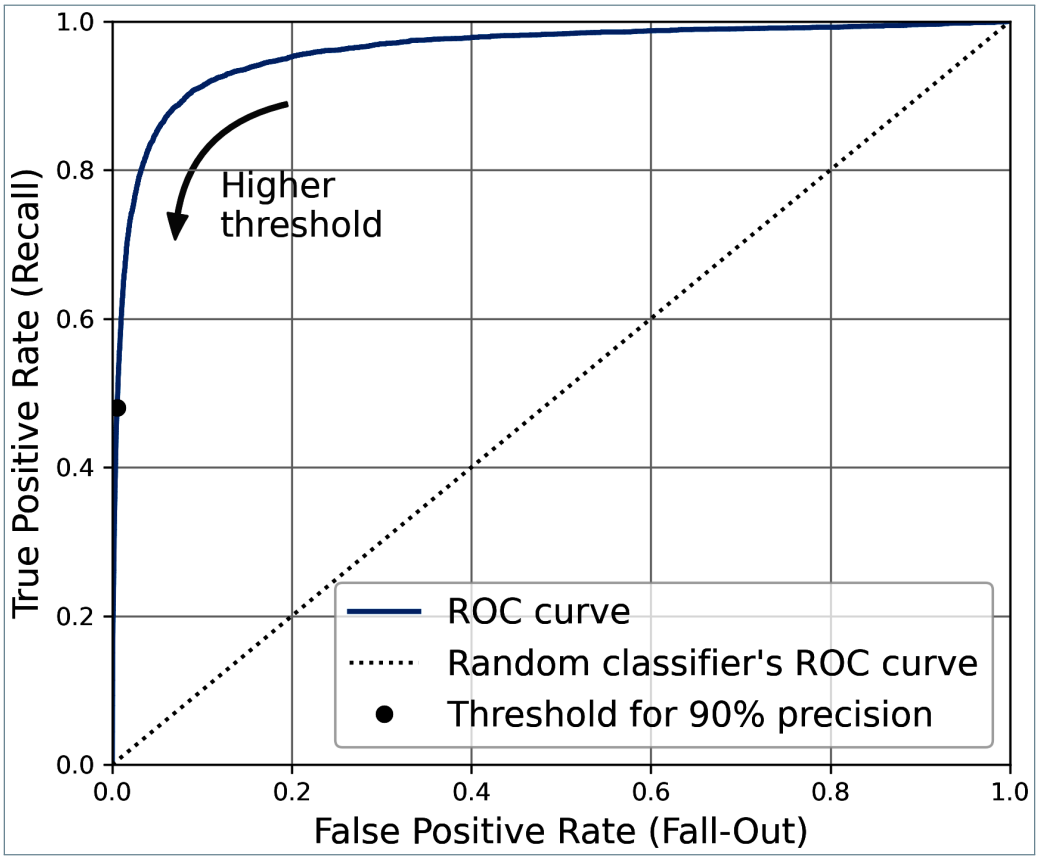

图 3-7: ROC 曲线：针对所有可能的阈值绘制假正例率与真正例率；黑色圆点标出所选比例（精确率为 90%、召回率为 48%）（A ROC curve plotting the false positive rate against the true positive rate）

提示：由于 ROC 曲线与精确率/召回率（PR）曲线非常相似，你可能会疑惑应该使用哪一条。经验法则是：当正类很少，或者你更关注假正例而不是假负例时，应优先使用 PR 曲线；否则使用 ROC 曲线。例如，观察前面的 ROC 曲线（以及 ROC AUC 分数），你可能会觉得分类器非常好。但这主要是因为正类（5）相对于负类（非 5）很少。相比之下，PR 曲线清楚地表明分类器仍有提升空间：曲线可以更接近右上角（再次参见图 3-6）。

现在创建一个 RandomForestClassifier，将它的 PR 曲线和 F1 分数与 SGDClassifier 的结果进行比较：

```
from sklearn.ensemble import RandomForestClassifier
forest_clf = RandomForestClassifier(random_state=42)
```

precision_recall_curve() 函数需要每个实例的标签和分数，因此要先训练随机森林分类器，并为每个实例分配一个分数。但 RandomForestClassifier 没有 decision_function() 方法，这是由它的工作方式决定的（第 6 章会介绍）。幸运的是，它有一个 predict_proba() 方法，可以返回每个实例的类别概率；只需把正类的概率作为分数，precision_recall_curve() 就可以使用。可以调用 cross_val_predict()，通过交叉验证训练 RandomForestClassifier，并为每张图像预测类别概率：[4]

```
y_probas_forest = cross_val_predict(forest_clf, X_train, y_train_5, cv=3,
                                    method="predict_proba")
```

来看看训练集中前两张图像的估计类别概率：

```
>>> y_probas_forest[:2]
array([[0.11, 0.89],
       [0.99, 0.01]])
```

模型预测第一张图像属于正类的概率为 89%，预测第二张图像属于负类的概率为 99%。由于每张图像要么属于正类，要么属于负类，因此每行的估计概率之和为 100%。

> 警告：这些是估计概率，而不是实际概率。例如，在模型估计正类概率介于 50% 和 60% 的所有图像中，大约 94% 实际上属于正类。因此，本例中模型的估计概率明显偏低；当然，模型也可能过度自信。sklearn.calibration 包中的 CalibratedClassifierCV 类可以使用交叉验证校准估计概率，使其更接近实际概率（notebook 中有代码示例）。这在医学诊断、金融风险评估或欺诈检测等场景中非常重要。

第二列包含正类的估计概率，因此把它传给 precision_recall_curve() 函数：

[4] Scikit-Learn 的分类器总是至少有 decision_function() 或 predict_proba() 其中一个方法，有时两者都有。

```
y_scores_forest = y_probas_forest[:, 1]
precisions_forest, recalls_forest, thresholds_forest = precision_recall_curve(
    y_train_5, y_scores_forest)
```

现在可以绘制 PR 曲线。为了进行比较，也把前面的 PR 曲线画出来（见图 3-8）：

```
plt.plot(recalls_forest, precisions_forest, "b-", linewidth=2,
         label="Random Forest")
plt.plot(recalls, precisions, "--", linewidth=2, label="SGD")
[...]  # beautify the figure: add grid, legend, and axis labels
plt.show()
```

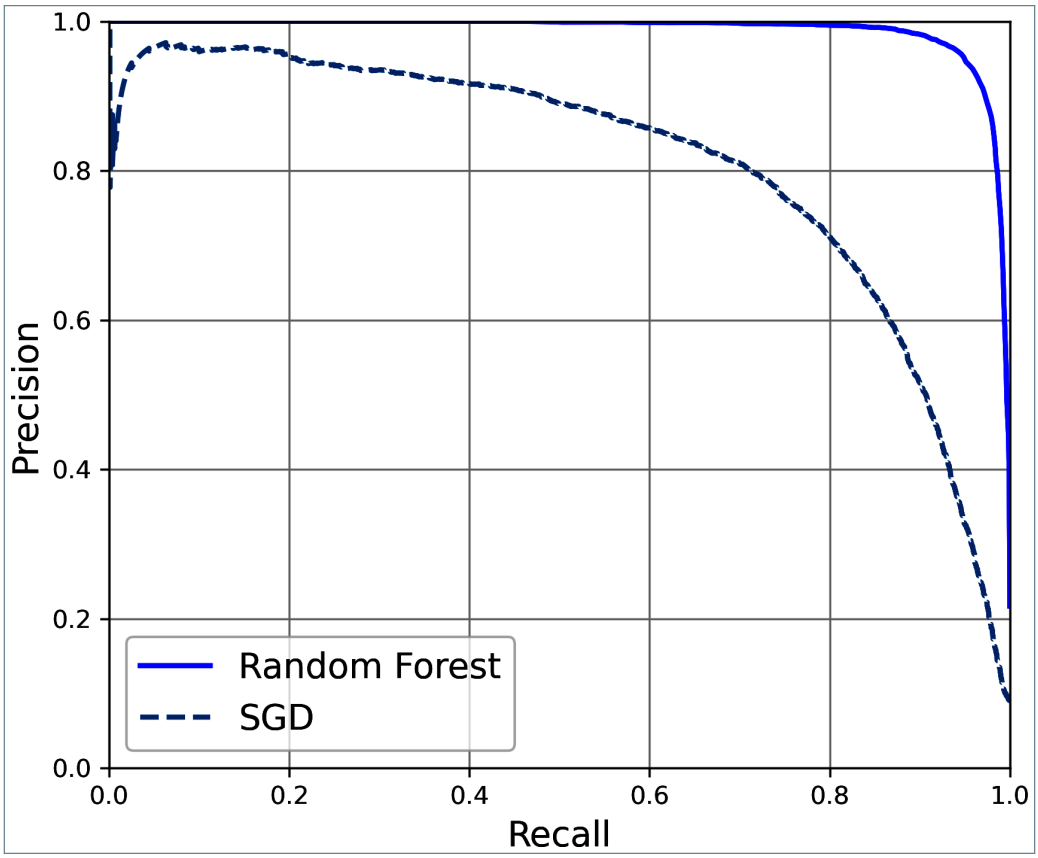

图 3-8: PR 曲线比较：随机森林分类器优于 SGD 分类器，因为它的 PR 曲线更接近右上角，且 AUC 更大（Comparing PR curves）

可以看到，`RandomForestClassifier` 的 PR 曲线明显优于 `SGDClassifier`，它更接近右上角。它的 F1 分数和 ROC AUC 分数也显著更好：

```
>>> y_train_pred_forest = y_probas_forest[:, 1] >= 0.5  # positive proba ≥ 50%
>>> f1_score(y_train_5, y_train_pred_forest)
0.9274509803921569
>>> roc_auc_score(y_train_5, y_scores_forest)
0.9983436731328145
```

再测量精确率和召回率：应该会得到约 99.0% 的精确率和 87.3% 的召回率。表现相当不错!

现在，你已经知道如何训练二分类器、为任务选择合适的指标、使用交叉验证评估分类器、选择符合需求的精确率/召回率权衡，并使用多种指标和曲线比较不同模型。接下来尝试检测不只是数字 5 的类别。

3.4　多分类（Multiclass Classification）

二分类器只能区分两个类别，而多分类器（也称多项式分类器）可以区分两个以上的类别。

有些 Scikit-Learn 分类器（例如 **`LogisticRegression`**、**`RandomForestClassifier`** 和 **`GaussianNB`**）可以原生处理多个类别。其他分类器则严格来说是二分类器（例如 **`SGDClassifier`** 和 **`SVC`**）。不过，可以通过多种策略，使用多个二分类器完成多分类。

创建一个能把数字图像分成 10 类（0 到 9）的系统，一种方法是训练 10 个二分类器，每个数字对应一个分类器（0 检测器、1 检测器、2 检测器，以此类推）。分类图像时，获取每个分类器对该图像的决策分数，并选择输出分数最高的类别。这称为一对多（one-versus-the-rest，OvR）策略，有时也称一对所有（one-versus-all，OvA）。

另一种策略是针对每一对数字训练一个二分类器：一个区分 0 和 1，一个区分 0 和 2，一个区分 1 和 2，以此类推。这称为一对一（one-versus-one，OvO）策略。如果有 N 个类别，就需要训练 N x (N - 1) / 2 个分类器。对于 MNIST，这意味着要训练 45 个二分类器！分类图像时，需要让图像通过全部 45 个分类器，看看哪个类别赢得最多对决。OvO 的主要优点是，每个分类器只需要在包含它要区分的两个类别的那部分训练集上训练。

有些算法（例如支持向量机分类器）随着训练集规模增大，扩展性很差。对于这些算法，OvO 更受欢迎，因为在小训练集上训练许多分类器比在大训练集上训练少数分类器更快。不过，对于大多数二分类算法，OvR 更受欢迎。

当你尝试使用二分类算法解决多分类任务时，Scikit-Learn 会自动检测出来，并根据算法自动运行 OvR 或 OvO。现在使用 **sklearn.svm.SVC** 类尝试支持向量机分类器（在线 SVM 章节见 https://homl.info）。这里只在前 2,000 张图像上训练，否则会花费很长时间：

```
from sklearn.svm import SVC
svm_clf = SVC(random_state=42)
svm_clf.fit(X_train[:2000], y_train[:2000])  # y_train, not y_train_5
```

这很简单！我们使用了 0 到 9 的原始目标类别（**y_train**），而不是 5 对其他类别的目标向量（**y_train_5**），训练了 **SVC**。由于有 10 个类别（多于 2 个），Scikit-Learn 使用 OvO 策略训练了 45 个二分类器。现在对一张图像做预测：

```
>>> svm_clf.predict([some_digit])
array(['5'], dtype=object)
```

预测正确！这段代码实际上做出了 45 次预测（每一对类别一次），并选择赢得最多对决的类别。[5] 如果调用 **decision_function()** 方法，会看到它为每个实例返回 10 个分数，每个类别一个。每个类别的分数等于它赢得的对决数，再加上或减去一个很小的调整值（最大为 +/-0.33），根据分类器分数打破平局：

```
>>> some_digit_scores = svm_clf.decision_function([some_digit])
>>> some_digit_scores.round(2)
array([[ 3.79,  0.73,  6.06,  8.3 , -0.29,  9.3 ,  1.75,  2.77,  7.21,
         4.82]])
```

最高分为 9.3，确实对应类别 5：

```
>>> class_id = some_digit_scores.argmax()
>>> class_id
np.int64(5)
```

分类器训练后，会把目标类别列表存储在 **classes_** 属性中，并按值排序。对于 MNIST，**classes_** 数组中每个类别的索引恰好与类别本身相同（例如，索引 5 处的类别碰巧就是**'5'**），但一般情况下不能指望这么幸运，需要像下面这样查找类别标签：

[5] 如果出现平局，则选择第一个类别；除非把 **break_ties** 超参数设置为 **True**，此时会使用 **decision_function()** 的输出打破平局。

```
>>> svm_clf.classes_
array(['0', '1', '2', '3', '4', '5', '6', '7', '8', '9'], dtype=object)
>>> svm_clf.classes_[class_id]
'5'
```

如果希望强制 Scikit-Learn 使用一对一或一对多，可以使用 **OneVsOneClassifier** 或 **OneVsRestClassifier** 类。创建一个实例并把分类器传给构造函数即可（它甚至不必是二分类器）。例如，下面的代码基于 **SVC**，使用 OvR 策略创建一个多分类器：

```
from sklearn.multiclass import OneVsRestClassifier
ovr_clf = OneVsRestClassifier(SVC(random_state=42))
ovr_clf.fit(X_train[:2000], y_train[:2000])
```

来做个预测，并检查训练出的分类器数量：

```
>>> ovr_clf.predict([some_digit])
array(['5'], dtype='<U1')
>>> len(ovr_clf.estimators_)
10
```

在多分类数据集上训练 **SGDClassifier** 并进行预测同样简单：

```
>>> sgd_clf = SGDClassifier(random_state=42)
>>> sgd_clf.fit(X_train, y_train)
>>> sgd_clf.predict([some_digit])
array(['3'], dtype='<U1')
```

糟糕，预测错误了。预测出错是会发生的！这一次，Scikit-Learn 在底层使用了 OvR 策略：由于有 10 个类别，它训练了 10 个二分类器。现在，**decision_function()** 方法会为每个类别返回一个值。来看看 SGD 分类器分配给每个类别的分数：

```
>>> sgd_clf.decision_function([some_digit]).round()
array([[-31893., -34420.,  -9531.,   1824., -22320.,  -1386., -26189.,
        -16148.,  -4604., -12051.]])
```

可以看到，分类器对自己的预测并不十分有把握：几乎所有分数都非常负，而类别 3 的分数为 +1,824，类别 5 的分数为 -1,386，并没有落后太多。当然，你会希望在不止一张图像上评估这个分类器。由于每个类别中的图像数量大致相同，准确率指标就很合适。和往常一样，可以使用 **cross_val_score()** 评估模型：

```
>>> cross_val_score(sgd_clf, X_train, y_train, cv=3, scoring="accuracy")
array([0.87365, 0.85835, 0.8689 ])
```

所有测试折的准确率都超过了 85.8%。如果使用随机分类器，准确率会是 10%，所以这个结果并不差，但仍然可以做得更好。只需缩放输入（第 2 章已经讨论过），准确率就能提高到 89.1% 以上：

```
>>> from sklearn.preprocessing import StandardScaler
>>> scaler = StandardScaler()
>>> X_train_scaled = scaler.fit_transform(X_train.astype("float64"))
>>> cross_val_score(sgd_clf, X_train_scaled, y_train, cv=3, scoring="accuracy")
array([0.8983, 0.891 , 0.9018])
```

3.5　误差分析（Error Analysis）

如果这是一个真实项目，现在应该按照机器学习项目检查清单执行后续步骤（见 https://homl.info/checklist）：探索数据准备方案，尝试多个模型，筛选出最好的模型，使用 `GridSearchCV` 微调超参数，并尽可能实现自动化。在这里，假设你已经找到了一个有潜力的模型，现在想办法改进它。一种方法是分析它产生的错误类型。

首先查看混淆矩阵。需要先使用 `cross_val_predict()` 生成预测，再像前面一样把标签和预测传给 `confusion_matrix()`。不过，现在有 10 个类别而不是 2 个，混淆矩阵会包含很多数字，可能很难阅读。

可以用彩色图更容易地分析混淆矩阵。下面使用 `ConfusionMatrixDisplay.from_predictions()` 函数绘制彩色图：

```
from sklearn.metrics import ConfusionMatrixDisplay
y_train_pred = cross_val_predict(sgd_clf, X_train_scaled, y_train, cv=3)
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred)
plt.show()
```

这会生成图 3-9 左侧的图。这个混淆矩阵看起来相当不错：大多数图像都位于主对角线上，这说明它们被正确分类。注意，第 5 行、第 5 列的对角线单元格比其他数字略暗。这可能是因为模型对 5 的错误更多，也可能是因为数据集中 5 的数量比其他数字少。因此，应该通过将每个值除以相应实际类别的图像总数（即除以该行的和），对混淆矩阵进行归一化。只需设置 `normalize="true"` 即可。还可以指定 `values_format=".0%"`，以不显示小数的百分比形式展示结果。下面的代码生成图 3-9 右侧的图：

```
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred,
                                        normalize="true", values_format=".0%")
plt.show()
```

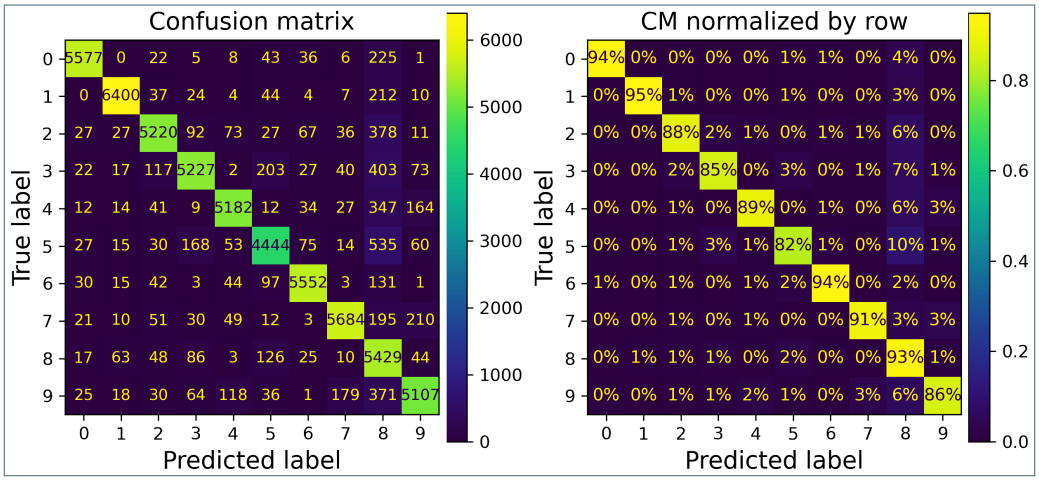

图 3-9: 混淆矩阵（左）以及按行归一化后的同一混淆矩阵（右）（Confusion matrix (left) and the same CM normalized by row (right)）

现在很容易看出，只有 82% 的数字 5 图像被正确分类。模型对数字 5 最常见的错误，是把它们误分类为 8：这发生在全部数字 5 的 10% 中。但只有 2% 的数字 8 被误分类为 5；混淆矩阵通常并不对称！仔细观察会发现，许多数字都被误分类为 8，但从这张图中并不容易看出来。如果希望错误更加突出，可以把正确预测的权重设为 0。下面的代码正是这样做的，并生成图 3-10 左侧的图：

```
sample_weight = (y_train_pred != y_train)
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred,
                                        sample_weight=sample_weight,
                                        normalize="true", values_format=".0%")
plt.show()
```

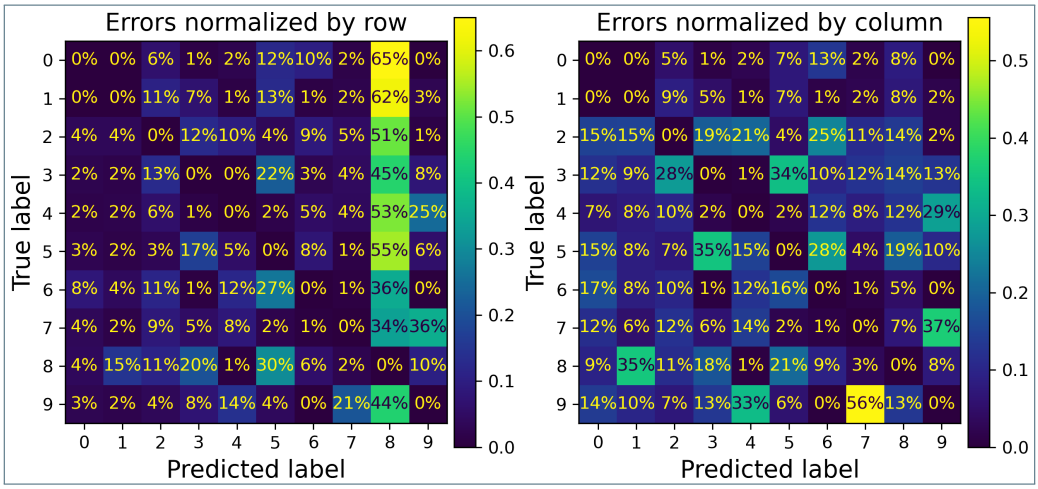

图 3-10: 只包含错误、按行归一化（左）和按列归一化（右）的混淆矩阵（Confusion matrix with errors only, normalized by row (left) and by column (right)）

现在可以更清楚地看到分类器产生的错误类型。类别 8 所在的列非常亮，这证实了许多图像都被误分类为 8。事实上，对于几乎所有类别来说，这是最常见的误分类。不过，解释这张图中的百分比时要小心：记住，我们排除了正确预测。例如，左侧网格第 7 行、第 9 列的 36%，并不意味着所有数字 7 中有 36% 被误分类为 9；它意味着模型对数字 7 产生的错误中，有 36% 是把它们误分类为 9。实际上，只有 3% 的数字 7 被误分类为 9，正如图 3-9 右侧所示。

也可以按列而不是按行对混淆矩阵进行归一化：设置 **`normalize="pred"`**，就会得到图 3-10 右侧的结果。例如，可以看到被误分类的数字 7 中有 56% 实际上是数字 9。

分析混淆矩阵通常能帮助你找到改进分类器的方法。从这些图来看，应该把精力放在减少“假 8”上。例如，可以尝试收集更多看起来像 8、但实际上不是 8 的数字的训练数据，让分类器学会把它们与真正的 8 区分开。也可以设计有助于分类器的新特征，例如编写算法统计闭合环的数量（8 有两个，6 有一个，5 没有）。还可以预处理图像（使用 Scikit-Image、Pillow 或 OpenCV），让闭合环等模式更加突出。

分析单个错误也能帮助你了解分类器在做什么，以及为什么会失败。例如，可以用混淆矩阵的形式绘制数字 3 和 5 的示例（见图 3-11）：

```
cl_a, cl_b = '3', '5'
X_aa = X_train[(y_train == cl_a) & (y_train_pred == cl_a)]
X_ab = X_train[(y_train == cl_a) & (y_train_pred == cl_b)]
X_ba = X_train[(y_train == cl_b) & (y_train_pred == cl_a)]
X_bb = X_train[(y_train == cl_b) & (y_train_pred == cl_b)]
[...]  # plot all images in X_aa, X_ab, X_ba, X_bb in a confusion matrix style
```

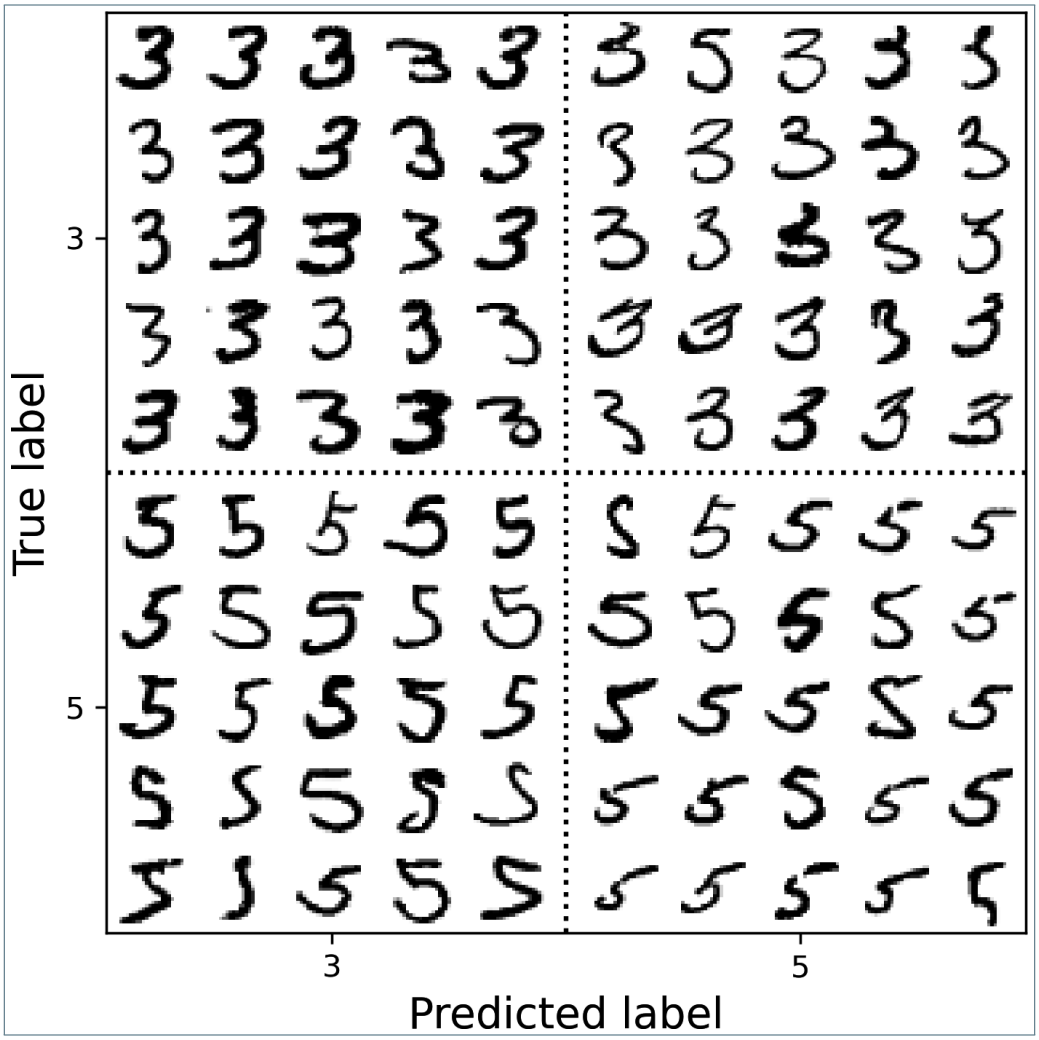

图 3-11: 以混淆矩阵形式排列的一些数字 3 和 5 图像（Some images of 3s and 5s organized like a confusion matrix）

可以看到，分类器判断错误的某些数字（即左下和右上方块中的数字）写得非常潦草，即使人类也很难分类。不过，大多数误分类图像在人看来都属于明显的错误。你可能很难理解分类器为什么会犯这些错误，但要记住，人脑是非常出色的模式识别系统；在任何信息进入意识之前，视觉系统已经完成了大量复杂的预处理。因此，任务感觉简单，并不意味着任务真的简单。回想一下，我们使用的是简单的`SGDClassifier`，它只是一个线性模型：它为每个像素分配一个类别权重，看到新图像时，只需把加权后的像素强度相加，为每个类别得到一个分数。由于 3 和 5 只在少数几个像素上有差异，这个模型很容易混淆它们。

3 和 5 的主要差异，在于连接顶部横线与底部弧线的小线段位置。如果画数字 3 时把连接处略微向左移动，分类器可能把它判断为 5，反之亦然。换句话说，这个分类器对图像的平移和旋转非常敏感。减少 3/5 混淆的一种方法，是对图像进行预处理，保证它们居中且没有过度旋转。不过，这并不容易，因为它要求先预测每张图像的正确旋转角度。更简单的方法，是用轻微平移和旋转后的图像变体扩充训练集。这会迫使模型学会对这些变化更加容忍。这称为数据增强（第 12 章会介绍；也请参阅本章末尾的练习 2）。

3.6　多标签分类（Multilabel Classification）

到目前为止，每个实例都只被分配到一个类别。但有时你希望分类器为每个实例输出多个类别。考虑一个人脸识别分类器：如果同一张图片中识别出多个人，它应该怎么做？它应该为识别出的每个人附加一个标签。假设分类器经过训练，可以识别 Alice、Bob 和 Charlie 三张脸。当它看到一张包含 Alice 和 Charlie 的照片时，应该输出`[True, False, True]`（分别表示“Alice 是，Bob 否，Charlie 是”）。这种输出多个二元标签的分类系统称为多标签分类系统。

我们暂时不讨论人脸识别，先看一个更简单的例子：

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
y_train_large = (y_train >= '7')
y_train_odd = (y_train.astype('int8') % 2 == 1)
y_multilabel = np.c_[y_train_large, y_train_odd]
knn_clf = KNeighborsClassifier()
knn_clf.fit(X_train, y_multilabel)
```

这段代码创建了一个`y_multilabel`数组，为每张数字图像包含两个目标标签：第一个表示数字是否较大（7、8 或 9），第二个表示数字是否为奇数。然后创建一个

支持多标签分类的 `KNeighborsClassifier` 实例（并非所有分类器都支持），并使用多个目标组成的数组训练模型。现在可以进行预测，注意它会输出两个标签：

```
>>> knn_clf.predict([some_digit])
array([[False,  True]])
```

它判断正确了！数字 5 确实不大（`False`），并且是奇数（`True`）。

评估多标签分类器有很多方法，选择合适的指标完全取决于你的项目。一种方法，是为每个独立标签测量 F1 分数（或前面讨论过的任意二分类器指标），然后简单地计算平均分。下面的代码计算所有标签的平均 F1 分数：

```
>>> y_train_knn_pred = cross_val_predict(knn_clf, X_train, y_multilabel, cv=3)
>>> f1_score(y_multilabel, y_train_knn_pred, average="macro")
0.976410265560605
```

这种方法假设所有标签同样重要，但实际情况可能并非如此。尤其是，如果 Alice 的图像远多于 Bob 或 Charlie 的图像，你可能希望根据每个目标标签所包含的实例数量（即支持度）为分类器分数加权。只需在调用 `f1_score()` 时设置 `average="weighted"` 即可。[6]

如果希望使用不原生支持多标签分类的分类器（例如 `SVC`），一种策略是为每个标签训练一个模型。不过，这种策略可能很难捕捉标签之间的依赖关系。例如，大数字（7、8 或 9）成为奇数的概率是偶数的两倍，但负责“奇数”标签的分类器不知道负责“大数字”标签的分类器预测了什么。为解决这个问题，可以将模型组织成链：一个模型做出预测后，下一个模型将输入特征与链中前面模型的所有预测一起作为输入。

Scikit-Learn 提供了一个名为 `ClassifierChain` 的类，正是用于实现这种功能。默认情况下，它在训练时使用真实标签，根据标签在链中的位置，把相应标签提供给每个模型。但如果设置 `cv` 超参数，它会使用交叉验证获取训练集中每个实例的“干净”（样本外）预测，再用这些预测训练链中后面的所有模型。注意，分类器在链中的顺序可能会影响最终性能。下面的例子使用交叉验证策略创建并训练 `ClassifierChain`。和前面一样，为了加快速度，只使用前 2,000 张图像：

```
from sklearn.multioutput import ClassifierChain
chain_clf = ClassifierChain(SVC(), cv=3, random_state=42)
chain_clf.fit(X_train[:2000], y_multilabel[:2000])
```

现在使用这个 `ClassifierChain` 做预测：

[6] Scikit-Learn 还提供了其他几种平均选项和多标签分类器指标，详情请参阅文档。

```
>>> chain_clf.predict([some_digit])
array([[0., 1.]])
```

3.7　多输出分类（Multioutput Classification）

最后一种要讨论的分类任务称为多输出-多分类（multi-output-multiclass）分类，简称多输出分类。它是多标签分类的推广，其中每个标签可以是多分类的（即可以有两个以上的可能值）。

为了说明这一点，构建一个去除图像噪声的系统。它接收一张有噪声的数字图像作为输入，并（希望能够）输出一张干净的数字图像；干净图像以像素强度数组表示，就像 MNIST 图像一样。注意，分类器的输出是多标签的（每个像素一个标签），并且每个标签可以有多个值（像素强度范围为 0 到 255）。因此，这是一个多输出分类系统的例子。

> 注意：分类与回归之间的界线有时很模糊，例如本例。严格来说，预测像素强度更像是回归而不是分类。此外，多输出系统并不局限于分类任务；你甚至可以构建一个为每个实例输出多个标签的系统，其中同时包含类别标签和值标签。

首先，使用随机数生成器的 `integers()` 方法，在 MNIST 图像的像素强度上添加噪声，创建训练集和测试集。目标图像仍然是原始图像：

```
rng = np.random.default_rng(seed=42)
noise_train = rng.integers(0, 100, (len(X_train), 784))
X_train_mod = X_train + noise_train
noise_test = rng.integers(0, 100, (len(X_test), 784))
X_test_mod = X_test + noise_test
y_train_mod = X_train
y_test_mod = X_test
```

我们来看看测试集中的第一张图像（见图 3-12）。没错，我们正在窥探测试数据，你现在应该皱起眉头。

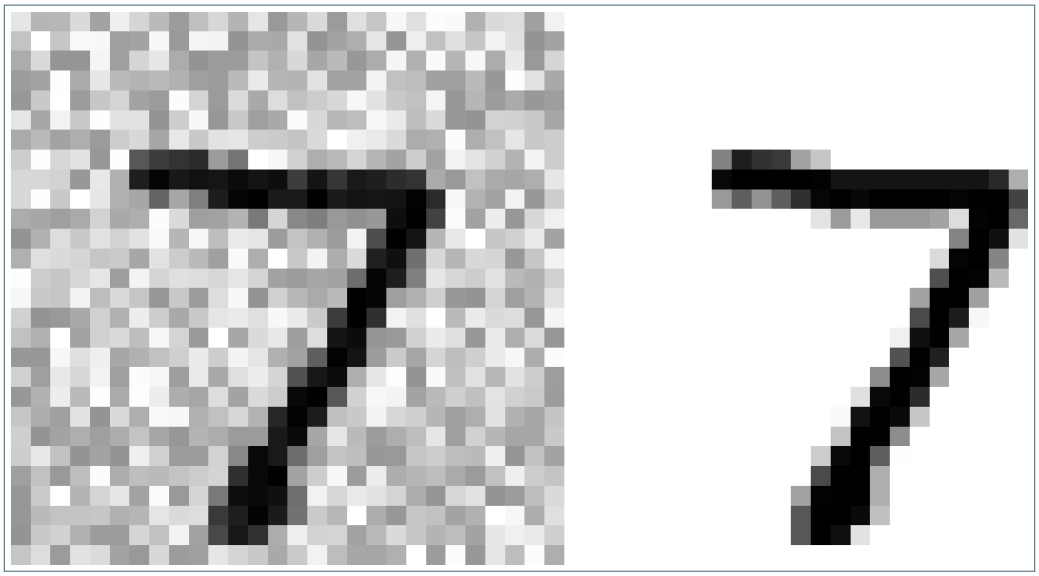

图 3-12: 有噪声的图像（左）和目标干净图像（右）（A noisy image (left) and the target clean image (right)）

左侧是有噪声的输入图像，右侧是干净的目标图像。现在训练分类器，让它清理这张图像（见图 3-13）：

```
knn_clf = KNeighborsClassifier()
knn_clf.fit(X_train_mod, y_train_mod)
clean_digit = knn_clf.predict([X_test_mod[0]])
plot_digit(clean_digit)
plt.show()
```

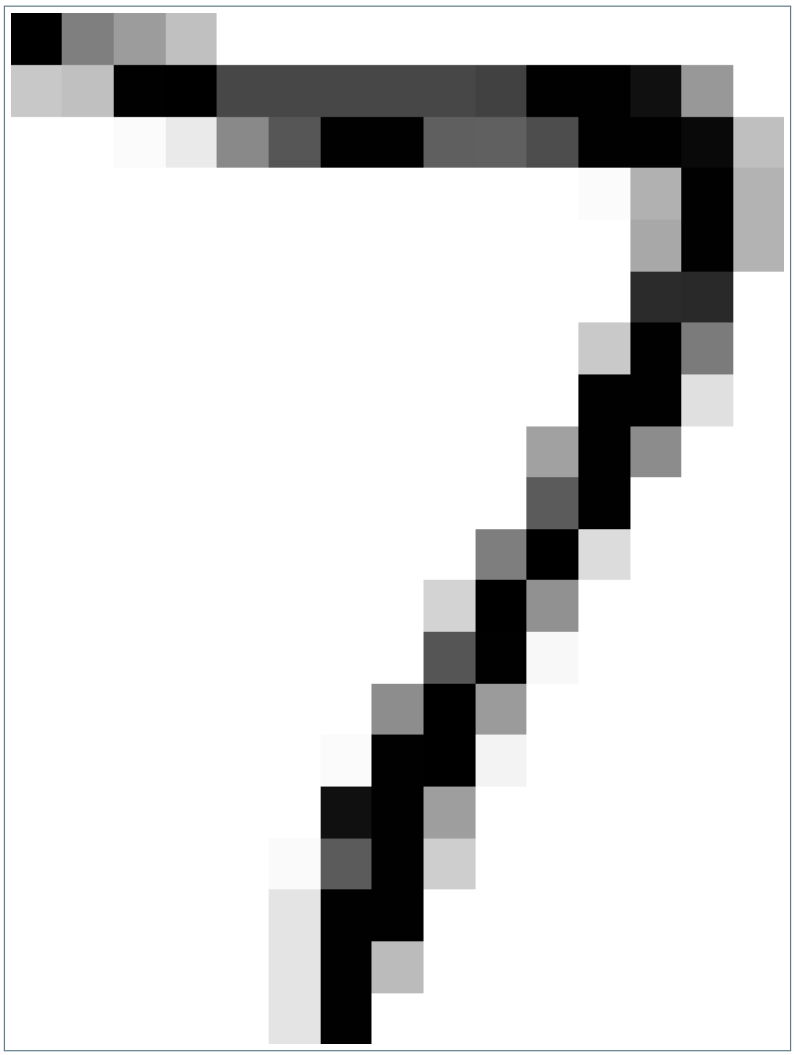

图 3-13: 清理后的图像（The cleaned-up image）

效果看起来足够接近目标图像！分类之旅到此结束。现在，你已经知道如何为分类任务选择合适的指标、选择恰当的精确率/召回率权衡、比较分类器，以及更一般地为各种任务构建良好的分类系统。在后续章节中，你将了解前面一直使用的这些机器学习模型实际上是如何工作的。

3.8　练习（Exercises）

1. 尝试为 MNIST 数据集构建一个在测试集上准确率超过 97% 的分类器。提示：`KNeighborsClassifier` 在这个任务上表现相当好，你只需要找到合适的超参数值（尝试对 `weights` 和 `n_neighbors` 超参数执行网格搜索）。

2. 编写一个函数，让 MNIST 图像可以向任意方向（左、右、上或下）移动一个像素。[7] 然后，为训练集中的每张图像创建 4 个平移副本（每个方向一个），并把它们添加到训练集。最后，在扩充后的训练集上训练最佳模型，并在测试集上测量准确率。你应该会发现，模型现在表现得更好！这种人为扩充训练集的技术称为数据增强或训练集扩展。

3. 处理 Titanic 数据集。Kaggle 是一个很好的起点。也可以从 https://homl.info/titanic.tgz 下载数据，并像第 2 章处理房价数据那样解压这个 tar 包。这样会得到两个 CSV 文件：`train.csv` 和 `test.csv`，可以使用 `pandas.read_csv()` 加载。目标是训练一个分类器，根据其他列预测 `Survived` 列。

[7] 可以使用 `scipy.ndimage.interpolation` 模块中的 `shift()` 函数。例如，`shift(image, [2, 1], cval=0)` 会让图像向下移动两个像素，向右移动一个像素。

4. 构建一个垃圾邮件分类器（更具挑战性的练习）：

a. 从 Apache SpamAssassin 的公共数据集下载垃圾邮件和正常邮件的示例，网址为 https://homl.info/spamassassin。

b. 解压数据集，熟悉数据格式。

c. 将数据分成训练集和测试集。

d. 编写数据准备流水线，把每封邮件转换成一个特征向量。你的准备流水线应该把邮件转换为一个（稀疏）向量，表示每个可能的单词是否出现。例如，如果所有邮件只包含四个单词："Hello""how""are""you"，那么邮件"Hello you Hello Hello you"应转换为 `[1, 0, 0, 1]`（表示 ["Hello"出现，"how"未出现，"are"未出现，"you"出现]）；如果更愿意统计每个单词的出现次数，也可以转换为 `[3, 0, 0, 2]`。

你可能希望给准备流水线增加超参数，用于控制是否去除邮件头、把每封邮件转换为小写、删除标点、把所有 URL 替换为`"URL"`、把所有数字替换为`"NUMBER"`，甚至执行词干提取（即去掉单词词尾；有可用的 Python 库可以完成这项工作）。

e. 最后尝试多种分类器，看看能否构建一个优秀的垃圾邮件分类器，同时拥有较高的召回率和精确率。

这些练习的答案位于本章 notebook 的末尾，网址为 https://homl.info/colab-p。

第 4 章　训练模型

Training Models

到目前为止，我们主要把机器学习模型及其训练算法当作黑盒来处理。如果你完成了前几章中的一些练习，可能会惊讶于：即使不了解模型内部的工作原理，也能完成很多事情。你优化了一个回归系统，改进了一个数字图像分类器，甚至从头构建了一个垃圾邮件分类器，而不必知道它们实际上是如何工作的。的确，在很多情况下，你并不需要了解实现细节。

不过，充分理解模型的工作方式，可以帮助你快速找到适合任务的模型、正确的训练算法以及合适的超参数组合。了解模型内部机制还有助于更高效地调试问题和执行误差分析。最后，本章讨论的大多数主题，对于理解、构建和训练神经网络也至关重要（本书第二部分将讨论神经网络）。

本章首先考察线性回归模型，这是最简单的模型之一。我们将讨论两种截然不同的训练方式：

- 使用“闭式”方程[1]：直接计算出最适合训练集的模型参数，也就是使训练集上的成本函数最小的参数。
- 使用称为梯度下降（gradient descent，GD）的迭代优化方法：逐步调整模型参数，使训练集上的成本函数最小化，最终收敛到与第一种方法相同的参数。我们还会研究几种梯度下降变体：批量 GD、小批量 GD 和随机 GD；在本书第二部分学习神经网络时还会反复使用它们。

接下来，我们将研究多项式回归，这是一种能够拟合非线性数据集的更复杂模型。由于它的参数多于线性回归，因此更容易过拟合训练数据。我们会探讨如何使用学习曲线检测过拟合，然后研究几种能够降低训练集过拟合风险的正则化技术。

最后，我们将考察另外两种常用于分类任务的模型：逻辑回归（logistic regression）和 Softmax 回归（softmax regression）。

[1] 闭式方程只由有限个常数、变量和标准运算组成，例如 a = sin(b - c)。它不包含无限求和、极限、积分等。

> 警告：本章会使用不少线性代数和微积分的基础数学公式。要理解这些公式，你需要熟悉向量和矩阵，包括如何转置、相乘和求逆，以及偏导数。如果对这些概念不熟悉，请复习在线补充材料中提供的线性代数和微积分入门 Jupyter notebook。如果你确实不喜欢数学，可以跳过公式；正文仍然能够帮助你掌握大多数概念。不过，学习数学形式化表达非常有用，因为它能让你读懂机器学习论文。虽然一开始可能有些吓人，但实际上并不难，本章中的代码也会帮助你理解这些公式。

4.1　线性回归（Linear Regression）

在第 1 章中，我们看过一个简单的生活满意度线性模型（公式 4-1）。

$$\text{life_satisfaction} = \theta_0 + \theta_1 \times \text{GDP_per_capita} \tag{4-1}$$

公式 4-1　一个简单的生活满意度线性模型（A simple linear model of life satisfaction）

这个模型只是输入特征 `GDP_per_capita` 的线性函数。`θ0` 和 `θ1` 是模型参数。

更一般地，线性模型通过计算输入特征的加权和，再加上一个称为偏置项（bias term，也称截距项）的常数来做出预测，如公式 4-2 所示。

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n \tag{4-2}$$

公式 4-2　线性回归模型预测（Linear regression model prediction）

在这个公式中：

- `ŷ` 是预测值。
- `n` 是特征数量。
- `xi` 是第 `i` 个特征值。
- `θj` 是第 `j` 个模型参数，包括偏置项 `θ0` 和特征权重 `θ1`、`θ2`、……、`θn`。

使用向量化形式可以把它写得简洁得多，如公式 4-3 所示。

$$\hat{y} = h_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta} \cdot \mathbf{x} \tag{4-3}$$

公式 4-3　线性回归模型预测的向量化形式（Vectorized form）

在这个公式中：

- **hθ** 是使用模型参数 **θ** 的假设函数。
- **θ** 是模型参数向量，包含偏置项 **θ0** 以及从 **θ1** 到 **θn** 的特征权重。
- **x** 是实例的特征向量，包含 **x0** 到 **xn**，其中 **x0** 始终等于 1。
- **θ** ·**x** 是向量 **θ** 和 **x** 的点积，等于 **θ0x0 + θ1x1 + θ2x2 + ... + θnxn**。

> 提示：在机器学习中，向量通常表示为列向量，即只有一列的二维数组。如果 **θ** 和 **x** 都是列向量，那么预测值是 **ŷ = theta^T x**，其中 **theta^T** 是 **θ** 的转置（一个行向量而不是列向量），**theta^T x** 是 **theta^T** 与 **x** 的矩阵乘法。预测值当然相同，只是现在表示为单元素矩阵，而不是标量。本书将使用这种记法，以避免在点积和矩阵乘法之间来回切换。

这就是线性回归模型—-但如何训练它呢？回想一下，训练模型就是设置模型参数，使模型尽可能拟合训练集。为此，首先需要衡量模型对训练数据的拟合程度。在第 2 章中我们看到，回归模型最常用的性能指标是均方根误差（root mean squared error，RMSE；公式 2-1）。因此，要训练线性回归模型，需要找到使 RMSE 最小的 **θ**。实际操作中，最小化均方误差（mean squared error，MSE）比最小化 RMSE 更简单，而且结果相同，因为使一个正函数最小的值，也会使它的平方根最小。

> 警告：学习算法在训练时经常优化与最终模型性能指标不同的损失函数。这通常是因为前者更容易优化，或者训练过程中需要额外项（例如正则化）。好的性能指标应尽可能接近最终业务目标；好的训练损失则应易于优化，并且与性能指标高度相关。例如，分类器通常使用对数损失这样的成本函数训练（本章稍后会介绍），但使用精确率和召回率评估。对数损失容易最小化，最小化它通常会改善精确率和召回率。

线性回归假设 **hθ** 在训练集 **X** 上的 MSE 使用公式 4-4 计算。

$$\mathrm{MSE}(X, \mathbf{y}, h_{\boldsymbol{\theta}}) = \frac{1}{m}\sum_{i=1}^{m}\left(\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}^{(i)} - y^{(i)}\right)^2 \tag{4-4}$$

公式 4-4　线性回归模型的 MSE 成本函数（MSE cost function）

本章的大多数记号在第 2 章中已经介绍过（见“记号”一节）。唯一的区别是，这里写作 `hθ`，而不只是 `h`，以明确模型由向量 `θ` 参数化。为了简化记号，我们将用 `MSE(θ)` 代替 `MSE(X, hθ)`。

4.1.1　正规方程（The Normal Equation）

为了找到使 MSE 最小的 `θ`，存在一个闭式解，也就是能直接给出结果的数学方程。这称为正规方程（normal equation；公式 4-5）。

$$\hat{\boldsymbol{\theta}} = (X^{\mathsf{T}}X)^{-1}X^{\mathsf{T}}\mathbf{y} \tag{4-5}$$

公式 4-5　正规方程（Normal equation）

在这个公式中：

- `theta_hat` 是使成本函数最小的 `θ` 值。
- `y` 是目标值向量，包含 `y(1)` 到 `y(m)`。

下面生成一些线性数据来测试这个方程（见图 4-1）：

```
import numpy as np
rng = np.random.default_rng(seed=42)
m = 200  # number of instances
X = 2 * rng.random((m, 1))  # column vector
y = 4 + 3 * X + rng.standard_normal((m, 1))  # column vector
```

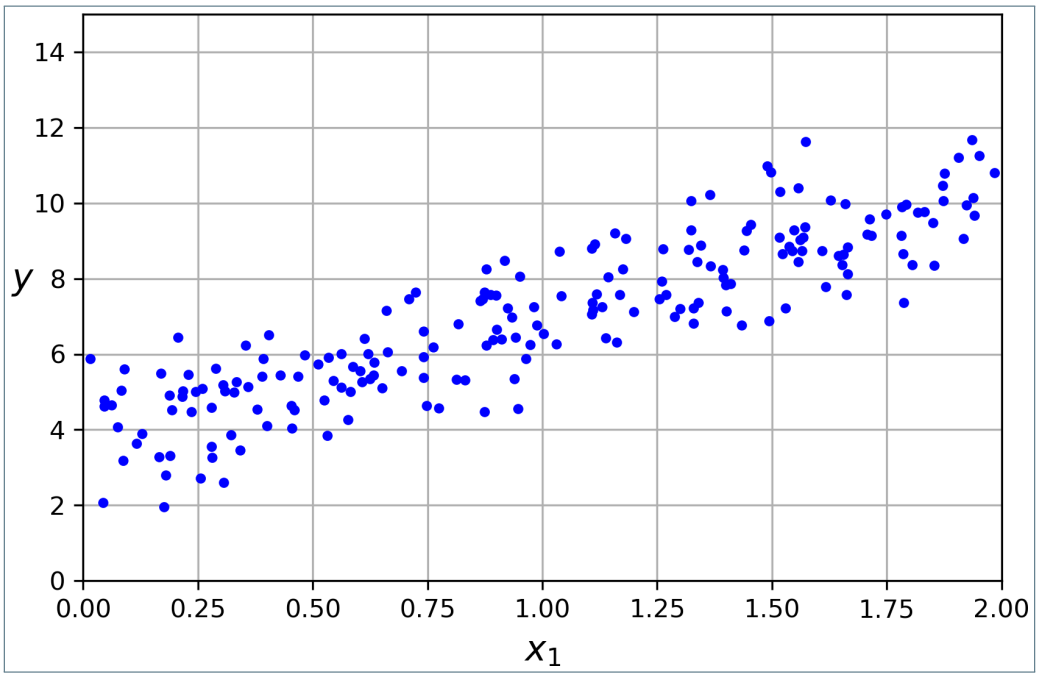

图 4-1: 随机生成的线性数据集（A randomly generated linear dataset）

现在使用正规方程计算 `theta_hat`。我们将使用 NumPy 线性代数模块（`np.linalg`）中的 `inv()` 函数计算矩阵的逆，并使用 `@` 运算符执行矩阵乘法：

```
from sklearn.preprocessing import add_dummy_feature
X_b = add_dummy_feature(X)  # add x0 = 1 to each instance
theta_best = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y
```

> 提示：`@` 运算符执行矩阵乘法。如果 `A` 和 `B` 是 NumPy 数组，那么 `A @ B` 等价于 `np.matmul(A, B)`。许多其他库（例如 TensorFlow、PyTorch 和 JAX）也支持 `@` 运算符。不过，不能对纯 Python 数组（即列表的列表）使用 `@`。

我们用来生成数据的函数是 `y = 4 + 3x1 +` 高斯噪声。看看方程找到了什么：

```
>>> theta_best
array([[3.69084138],
       [3.32960458]])
```

我们本来希望得到 `θ0 = 4` 和 `θ1 = 3`，而不是 `θ0 = 3.6908` 和 `θ1 = 3.3296`。不过已经足够接近；噪声使我们无法恢复原函数的精确参数。数据集越小、噪声越大，恢复精确参数就越困难。

现在可以使用 `theta_hat` 做预测：

```
>>> X_new = np.array([[0], [2]])
>>> X_new_b = add_dummy_feature(X_new)  # add x0 = 1 to each instance
>>> y_predict = X_new_b @ theta_best
>>> y_predict
array([[ 3.69084138],
       [10.35005055]])
```

绘制模型的预测结果（见图 4-2）：

```
import matplotlib.pyplot as plt
plt.plot(X_new, y_predict, "r-", label="Predictions")
plt.plot(X, y, "b.")
[...]  # beautify the figure: add labels, axis, grid, and legend
plt.show()
```

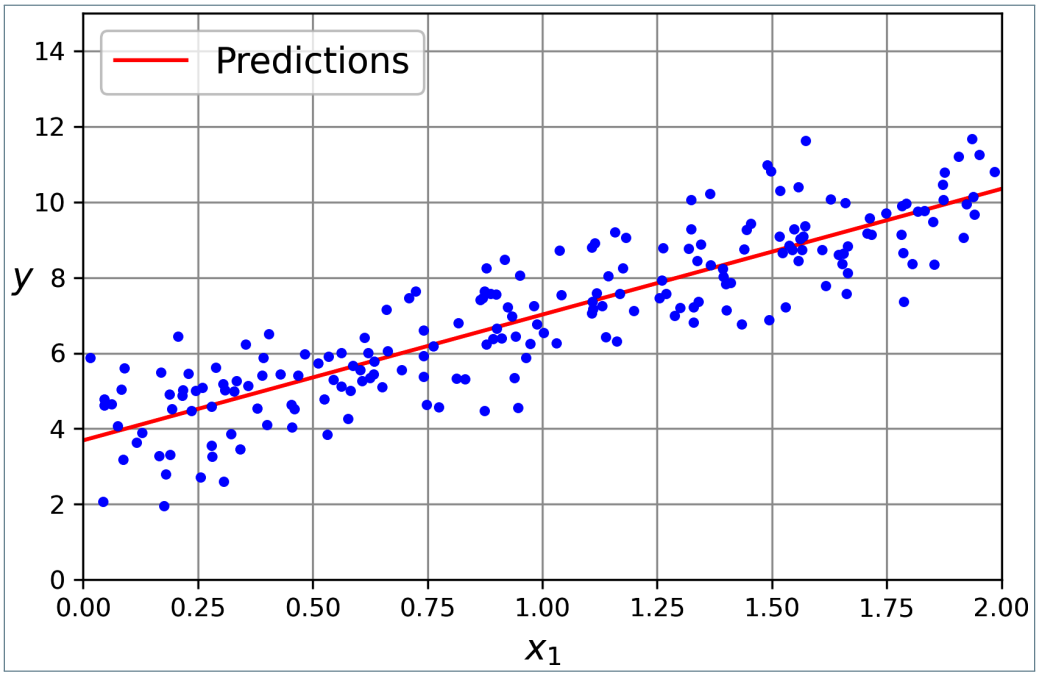

图 4-2: 线性回归模型的预测结果（Linear regression model predictions）

使用 Scikit-Learn 执行线性回归相对简单：

```
>>> from sklearn.linear_model import LinearRegression
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([3.69084138]), array([[3.32960458]]))
>>> lin_reg.predict(X_new)
array([[ 3.69084138],
       [10.35005055]])
```

注意，Scikit-Learn 将偏置项（`intercept_`）与特征权重（`coef_`）分开保存。`LinearRegression` 类基于 `scipy.linalg.lstsq()` 函数（名称代表“最小二乘”），也可以直接调用：

```
>>> theta_best_svd, residuals, rank, s = np.linalg.lstsq(X_b, y, rcond=1e-6)
>>> theta_best_svd
array([[3.69084138],
       [3.32960458]])
```

这个函数计算 `theta_hat = X+ y`，其中 `X+` 是 `X` 的伪逆（更具体地说，是 Moore-Penrose 逆）。可以使用 `np.linalg.pinv()` 直接计算伪逆：

```
>>> np.linalg.pinv(X_b) @ y
array([[3.69084138],
       [3.32960458]])
```

伪逆本身使用一种标准的矩阵分解技术计算，这种技术称为奇异值分解（singular value decomposition，SVD），它能将训练集矩阵 `X` 分解为三个矩阵的乘积 `U Sigma V^T`（见 `numpy.linalg.svd()`）。伪逆的计算公式是 `X+ = V Sigma+ U^T`。为了计算矩阵 `Sigma+`，算法会将 `Sigma` 中小于极小阈值的值设为零，把其余非零值替换为倒数，最后对结果矩阵转置。这种方法比计算正规方程更高效，还能很好地处理边界情况：如果矩阵 `X^T X` 不可逆（即奇异），正规方程可能失效，例如 `m < n` 或某些特征冗余时；而伪逆总是有定义的。

4.1.2　计算复杂度（Computational Complexity）

正规方程需要计算 `X^T X` 的逆，这是一个 `(n + 1) × (n + 1)` 矩阵，其中 `n` 是特征数量。求这种矩阵的逆，其计算复杂度通常约为 `O(n^2.4)` 到 `O(n^3)`，取决于具体实现。换句话说，如果特征数量翻倍，计算时间大约会增加到 `2^2.4 = 5.3` 到 `2^3 = 8` 倍。

Scikit-Learn 的 `LinearRegression` 类所使用的 SVD 方法复杂度约为 `O(n^2)`。如果特征数量翻倍，计算时间大约会变为原来的 4 倍。

> 警告：当特征数量很大时（例如 100,000 个），正规方程和 SVD 方法都会变得很慢。好的一面是，它们关于训练实例数量是线性的，即 `O(m)`，因此只要能放入内存，就能高效处理大型训练集。

训练好线性回归模型后（无论使用正规方程还是其他算法），预测会非常快：计算复杂度同时关于待预测实例数量和特征数量呈线性。换句话说，对两倍数量的实例（或两倍数量的特征）做预测，大约需要两倍时间。

现在来看看一种完全不同的线性回归训练方法，它更适合特征数量很多，或训练实例太多、无法全部放入内存的情况。

4.2　梯度下降（Gradient Descent）

梯度下降是一种通用的优化算法，能够为各种问题寻找最优解。其基本思想是迭代调整参数，使成本函数最小化。

假设你在浓雾中的山里迷路了，只能感受到脚下地面的坡度。要快速到达山谷底部，一个好策略是沿最陡的下坡方向前进。这正是梯度下降所做的事情：它计算误差函数相对于参数向量 θ 的局部梯度，然后沿梯度下降的方向前进。当梯度为零时，就到达了一个极小值。

实际操作中，先用随机值填充 θ（称为随机初始化），然后一步一步地改进它；每一步都试图降低成本函数（例如 MSE），直到算法收敛到某个极小值（见图 4-3）。

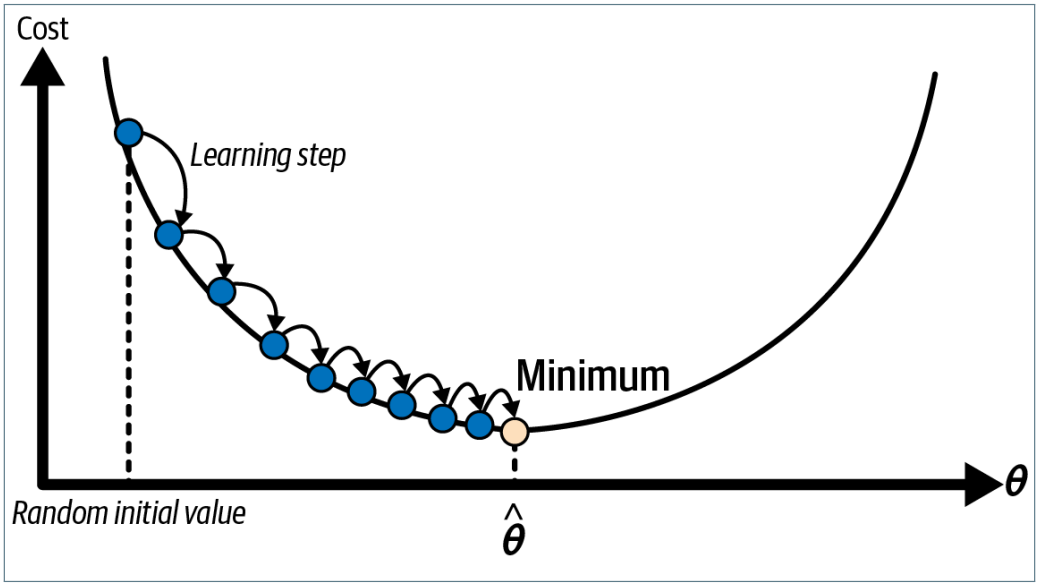

图 4-3: 梯度下降逐步逼近极小值（Gradient descent steps toward a minimum）

梯度下降中的一个重要参数是步长大小，它由学习率（learning rate）超参数决定。如果学习率太小，算法就必须经过很多次迭代才能收敛，需要很长时间（见图 4-4）。

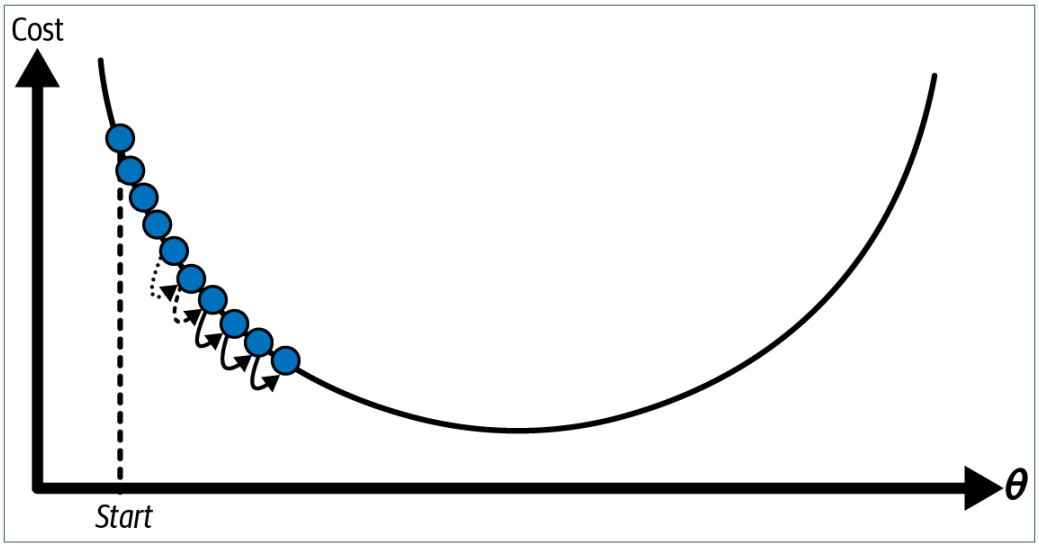

图 4-4: 学习率太小（Learning rate too small）

反过来，如果学习率太大，可能会跳过山谷，落到另一侧，甚至比原来更高的位置。这可能导致算法发散，数值越来越大，最终找不到好的解（见图 4-5）。

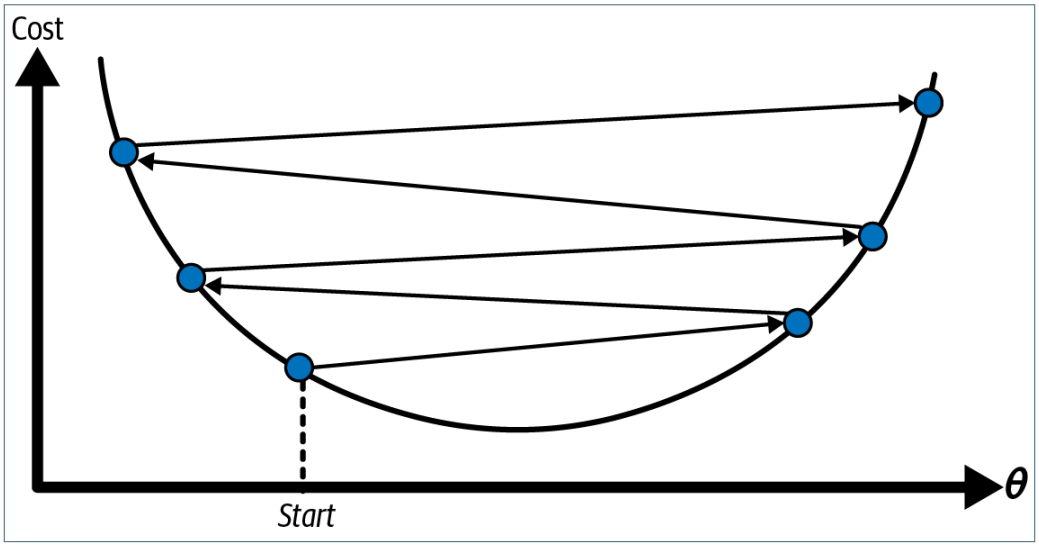

图 4-5: 学习率太大（Learning rate too high）

此外，并不是所有成本函数都像规则的碗。有些成本函数可能有洞、脊、平台以及各种不规则地形，使收敛到极小值变得困难。图 4-6 展示了梯度下降的两个主要挑战。如果随机初始化让算法从左侧开始，它会收敛到局部极小值，但这个极小值

不如全局极小值；如果从右侧开始，它需要很长时间才能穿过平台；如果过早停止，则永远无法到达全局极小值。

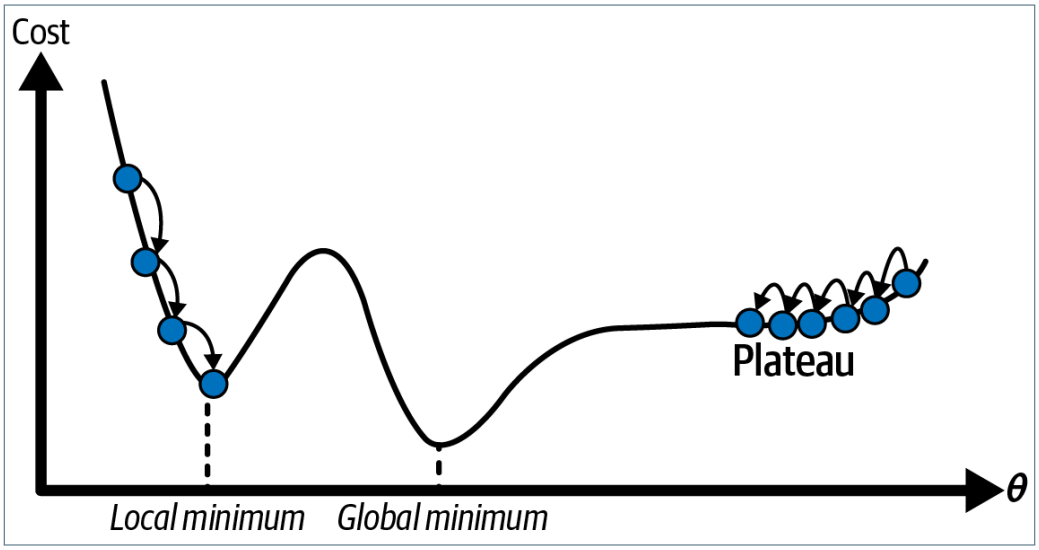

图 4-6: 梯度下降的陷阱（Gradient descent pitfalls）

幸运的是，线性回归模型的 MSE 成本函数恰好是凸函数。这意味着，在曲线上任意选择两个点，它们之间的线段都不会位于曲线下方。因此不存在局部极小值，只有一个全局极小值。它还是连续函数，斜率不会突然改变。[2] 这两个事实带来一个重要结论：只要等待足够长时间且学习率不过高，梯度下降一定能任意接近全局极小值。

不过，成本函数虽然呈碗状，但如果各特征的尺度差异很大，它可能是一个狭长的碗。图 4-7 左侧展示了特征 1 和特征 2 尺度相同的训练集上的梯度下降，右侧展示了特征 1 的值远小于特征 2 时的情况。[3]

从图中可以看到，左侧的梯度下降算法直奔极小值，很快就到达；右侧则首先沿几乎垂直于全局极小值方向的路径前进，最后沿一条几乎平坦的狭长谷底长途跋涉。它最终也会到达极小值，但需要很长时间。

这张图还说明，训练模型意味着在模型参数空间中寻找一组组合，使成本函数（在训练集上）最小。模型参数越多，这个空间的维度越高，搜索就越困难：在 300 维干草堆中找一根针，远比在 3 维空间中困难。幸运的是，在线性回归的情况下成本函数是凸的，因此这根针就在碗底。

[2] 严格地说，它的导数是 Lipschitz 连续的。

[3] 由于特征 1 较小，需要更大的 θ1 变化才能影响成本函数，因此这个碗沿 θ1 轴方向更加狭长。

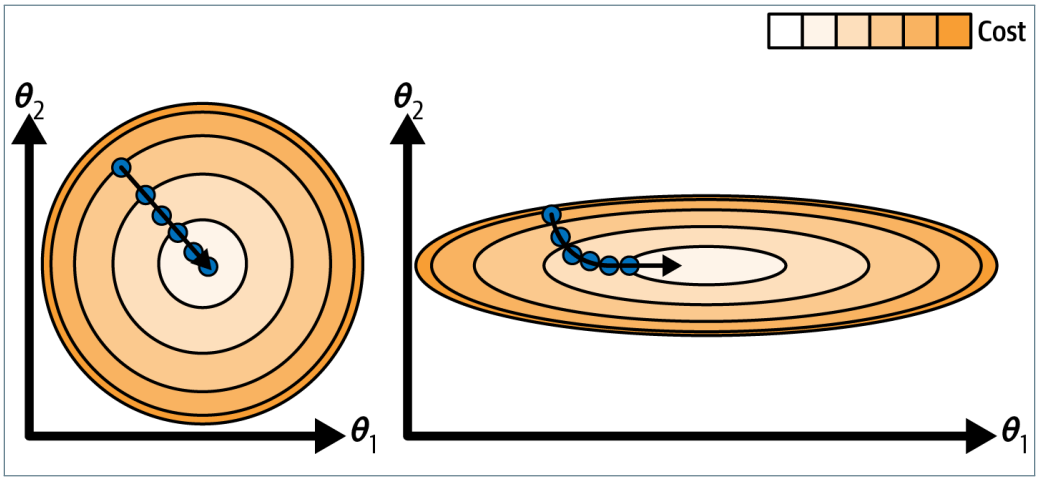

图 4-7: 使用特征缩放与不使用特征缩放时的梯度下降（Gradient descent with and without feature scaling）

> 警告: 使用梯度下降时, 应确保所有特征具有相近的尺度, 例如使用 Scikit-Learn 的 `StandardScaler` 类；否则收敛会慢得多。

4.2.1　批量梯度下降（Batch Gradient Descent）

大多数模型都有多个模型参数。因此，实现梯度下降时，需要计算成本函数关于每个模型参数 `θj` 的梯度。换句话说，需要计算 `θj` 改变一点点时成本函数会改变多少。这称为偏导数，就像先问“山向东的坡度是多少”，再面向北方问同一个问题（如果你能想象一个超过三维的宇宙，还要对其他所有维度重复）。公式 4-6 计算了 MSE 关于参数 `θj` 的偏导数，记作 `∂ MSE(θ) / ∂θj`。

$$\frac{\partial}{\partial \theta_j} \text{MSE}(\boldsymbol{\theta}) = \frac{2}{m} \sum_{i=1}^{m} \left(\boldsymbol{\theta}^\intercal \mathbf{x}^{(i)} - y^{(i)} \right) x_j^{(i)} \tag{4-6}$$

公式 4-6　成本函数的偏导数（Partial derivatives of the cost function）

与其分别计算这些偏导数，不如使用公式 4-7 一次性计算全部偏导数。梯度向量记作 `nabla_theta MSE(theta)`，包含成本函数的所有偏导数（每个模型参数对应一个）。

$$\nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta}) = \frac{2}{m} X^\intercal (X\boldsymbol{\theta} - \mathbf{y}) \tag{4-7}$$

公式 4-7　成本函数的梯度向量（Gradient vector of the cost function）

注意，这个公式在每一步梯度下降中都要遍历整个训练集 X！这就是算法称为批量梯度下降（batch gradient descent）的原因：每一步都使用全部训练数据组成的批次（严格来说，称为全量梯度下降可能更准确）。因此，在非常大的训练集上它会非常慢（稍后会介绍一些快得多的梯度下降算法）。不过，梯度下降关于特征数量的扩展性很好；当特征有数十万时，使用梯度下降训练线性回归模型，会比正规方程或 SVD 分解快得多。

梯度向量指向上坡方向，因此只要沿相反方向前进就能下坡，也就是从 θ 中减去 `nabla_theta MSE(theta)`。这就是学习率 η 发挥作用的地方：把梯度向量乘以 η，就能确定下坡步长（公式 4-8）。[4]

$$\boldsymbol{\theta}^{(\text{next step})} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta}) \tag{4-8}$$

公式 4-8　梯度下降步骤（Gradient descent step）

下面快速实现这个算法：

```
eta = 0.1  # learning rate
n_epochs = 1000
m = len(X_b)  # number of instances
rng = np.random.default_rng(seed=42)
theta = rng.standard_normal((2, 1))  # randomly initialized model parameters
for epoch in range(n_epochs):
    gradients = 2 / m * X_b.T @ (X_b @ theta - y)
    theta = theta - eta * gradients
```

这并不难！对训练集进行一次完整迭代称为一个 epoch。看看得到的 `theta`：

[4] Eta（η）是希腊字母表中的第七个字母。

```
>>> theta
array([[3.69084138],
       [3.32960458]])
```

看起来它与正规方程找到的结果完全相同！梯度下降工作得很完美。但如果使用不同的学习率 `eta` 呢？图 4-8 展示了使用三种不同学习率时梯度下降的前 20 步。每个图底部的线表示随机起点，之后每个 epoch 都用颜色更深的线表示。

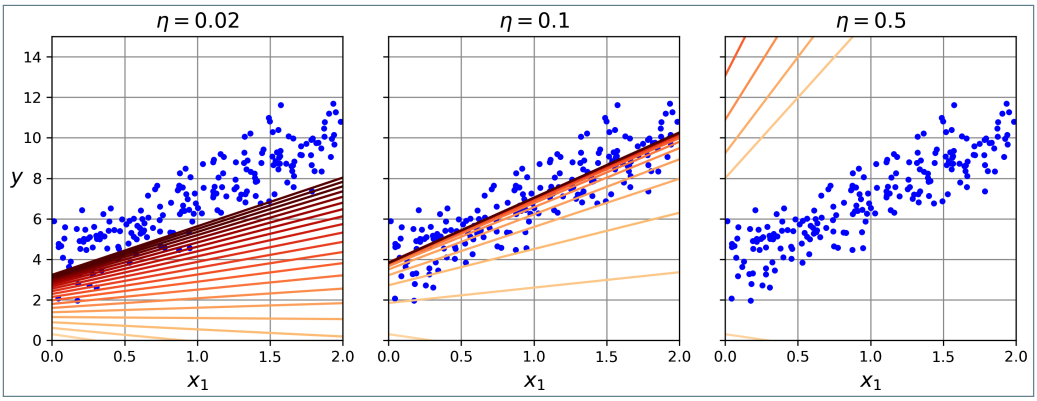

图 4-8: 使用不同学习率的梯度下降（Gradient descent with various learning rates）

左侧学习率太低：算法最终会到达解，但需要很长时间。中间的学习率看起来不错：仅仅几个 epoch 后就已经收敛到解。右侧学习率太高：算法发散，每一步都到处跳跃，实际上越来越远离解。

4.2.2　随机梯度下降（Stochastic Gradient Descent）

要找到合适的学习率，可以使用网格搜索（见第 2 章）。不过，可能需要限制 epoch 数量，让网格搜索排除那些收敛太慢的模型。

你可能会疑惑 epoch 数量该如何设置。如果太少，算法停止时仍然远离最优解；如果太多，当模型参数已经不再变化时又会浪费时间。一个简单的办法是设置很大的 epoch 数，但当梯度向量变得极小，也就是其范数小于一个很小的数 ε（称为容差）时，中断算法，因为这说明梯度下降已经（几乎）到达极小值。

> 收敛速度：当成本函数是凸函数且其斜率不会突然改变（MSE 成本函数就是如此）时，使用固定学习率的批量梯度下降最终会收敛到最优解，但可能需要等待一段时间：根据成本函数的形状，达到距离最优点 ε 范围内可能需要 $O(1/\varepsilon)$

> 次迭代。如果将容差缩小 10 倍以得到更精确的解，算法运行时间可能会增加约 10 倍。

批量梯度下降的主要问题是，每一步都使用整个训练集计算梯度，因此训练集较大时会很慢。与之相反，随机梯度下降（stochastic gradient descent，SGD）每一步从训练集中随机选择一个实例，只根据这一个实例计算梯度。每次只处理一个实例，数据操作量显然很小，因此算法快得多；同时，由于每次迭代只需将一个实例放入内存，也可以训练超大数据集（SGD 可以实现为核外算法；见第 1 章）。

另一方面，由于随机性，这个算法比批量梯度下降不规则得多：成本函数不是平缓下降到极小值，而是上下波动，只在平均意义上下降。随着时间推移，它会非常接近极小值，但到达附近后仍会继续波动，无法稳定下来（见图 4-9）。算法停止时，最终参数值会不错，但不一定是最优值。

当成本函数非常不规则（如图 4-6）时，这种随机性反而能帮助算法跳出局部极小值，因此随机梯度下降找到全局极小值的机会可能比批量梯度下降更大。

因此，随机性有助于逃离局部最优，但也意味着算法无法在极小值处稳定下来。解决这个矛盾的一种办法是逐渐降低学习率。开始时步长较大，有助于快速前进并逃离局部极小值；之后步长越来越小，使算法能够稳定到达全局极小值。这个过程类似模拟退火（simulated annealing），该算法受冶金中缓慢冷却熔融金属的过程启发。决定每次迭代学习率的函数称为学习率调度（learning schedule）。如果学习率下降太快，可能会卡在局部极小值，甚至在走到一半时冻结；如果下降太慢，可能会在极小值周围跳跃很久，如果过早停止训练，最终得到次优解。

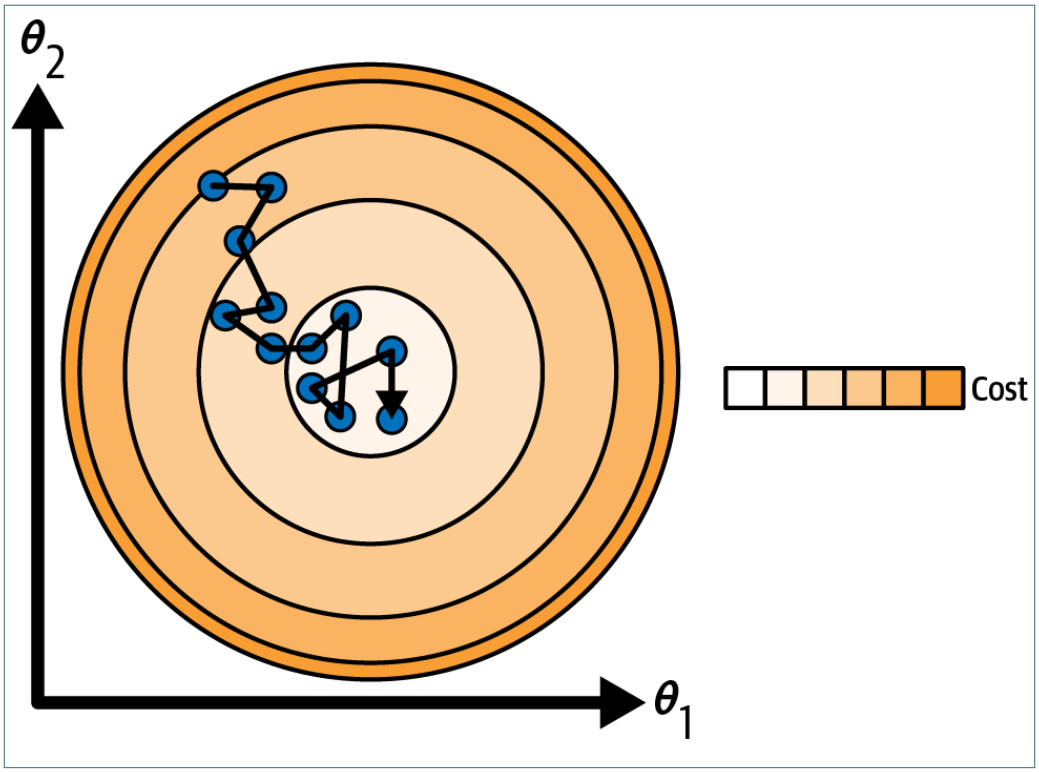

图 4-9: 随机梯度下降的每一步训练更快，但随机性也比批量梯度下降更强（Stochastic gradient descent steps）

下面的代码使用简单的学习率调度实现随机梯度下降：

```
n_epochs = 50
t0, t1 = 5, 50  # learning schedule hyperparameters
def learning_schedule(t):
    return t0 / (t + t1)
rng = np.random.default_rng(seed=42)
theta = rng.standard_normal((2, 1))  # randomly initialized model parameters
for epoch in range(n_epochs):
    for iteration in range(m):
        random_index = rng.integers(m)
        xi = X_b[random_index : random_index + 1]
        yi = y[random_index : random_index + 1]
        gradients = 2 * xi.T @ (xi @ theta - yi)  # for SGD, do not divide by m
        eta = learning_schedule(epoch * m + iteration)
        theta = theta - eta * gradients
```

按照惯例，我们以 `m` 次迭代为一轮；每一轮也称为一个 epoch，就像前面一样。批量梯度下降的代码遍历训练集 1,000 次，而这里的代码只遍历 50 次，就得到一个相当好的解：

```
>>> theta
array([[3.69826475],
       [3.30748311]])
```

图 4-10 展示了训练的前 20 步（注意这些步长很不规则）。

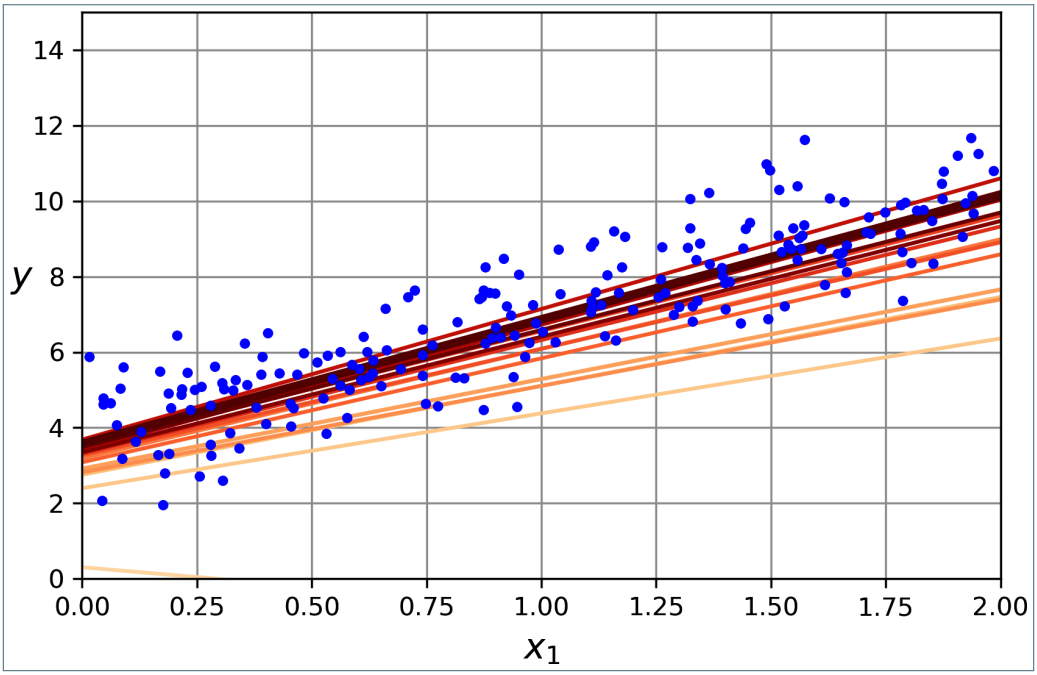

图 4-10: 随机梯度下降的前 20 步（The first 20 steps of stochastic gradient descent）

由于实例是随机选取的，每个 epoch 中有些实例可能被选中多次，而另一些可能一次也没有被选中。如果希望每个 epoch 都遍历每个实例，可以打乱训练集（确保输入特征和标签同步打乱），然后逐个遍历，再次打乱，如此循环。不过，这种方法更复杂，而且通常不会改善结果。

> 警告：使用随机梯度下降时，训练实例必须独立同分布（independent and identically distributed，IID），才能保证参数在平均意义上被拉向全局最优。一个简单的保证方法是在训练过程中打乱实例，例如随机选择每个实例，或在每个 epoch 开始时打乱训练集。如果不打乱实例，例如实例按标签排序，那么 SGD 会先针对一个标签优化，再针对下一个标签优化，最终不会稳定在全局最小值附近。

使用 Scikit-Learn，可以调用 **SGDRegressor** 类通过随机 GD 执行线性回归。它默认优化 MSE 成本函数。下面的代码最多运行 1,000 个 epoch（**max_iter**），或者当损失在 100 个 epoch（**n_iter_no_change**）内下降幅度小于 **10^-5**（**tol**）时停止。初始学习率为 0.01（**eta0**），使用默认学习率调度（与我们使用的不同）。最后，它不使用任何正则化（**penalty=None**；稍后会详细介绍）：

```
from sklearn.linear_model import SGDRegressor
sgd_reg = SGDRegressor(max_iter=1000, tol=1e-5, penalty=None, eta0=0.01,
                       n_iter_no_change=100, random_state=42)
sgd_reg.fit(X, y.ravel())  # y.ravel() because fit() expects 1D targets
```

又一次，得到的解与正规方程返回的结果非常接近：

```
>>> sgd_reg.intercept_, sgd_reg.coef_
(array([3.68899733]), array([3.33054574]))
```

> 提示：所有 Scikit-Learn 估计器都可以使用 **fit()** 方法训练，但有些估计器还提供 **partial_fit()** 方法，可以对一个或多个实例运行一轮训练（它会忽略 **max_iter** 或 **tol** 等超参数）。反复调用 **partial_fit()** 会逐渐训练模型，这在需要更多训练控制时很有用。另一些模型使用 **warm_start** 超参数（有些模型两者都有）：如果设置 **warm_start=True**，对已经训练过的模型调用 **fit()** 不会重置模型，而是从上次停止的地方继续训练，并遵守 **max_iter** 和 **tol** 等超参数。注意，**fit()** 会重置学习率调度使用的迭代计数器，而 **partial_fit()** 不会。

4.3　小批量梯度下降（Mini-Batch Gradient Descent）

最后一种要介绍的梯度下降算法称为小批量梯度下降（mini-batch gradient descent）。了解批量梯度下降和随机梯度下降后，它就很容易理解了：每一步不再像批量 GD 那样使用完整训练集，也不再像随机 GD 那样只使用一个实例，而是基于称为小批量（mini-batch）的小型随机实例集合计算梯度。小批量 GD 相对于随机 GD 的主要优势，是矩阵运算可以获得硬件加速，尤其是在使用图形处理器（GPU）时。

小批量 GD 在参数空间中的路径没有随机 GD 那么不规则，尤其是在小批量较大时。因此，小批量 GD 最终会比随机 GD 更接近极小值；但在存在局部极小值的问题中，它可能更难跳出局部极小值（MSE 成本函数的线性回归没有这个问题）。图 4-11 展示了三种梯度下降算法在训练期间于参数空间中走过的路径。它们最终都接近极

小值，但批量 GD 的路径确实停在极小值处，而随机 GD 和小批量 GD 会继续在周围移动。不要忘记，批量 GD 每走一步都要花很长时间；如果使用好的学习率调度，随机 GD 和小批量 GD 也能到达极小值。

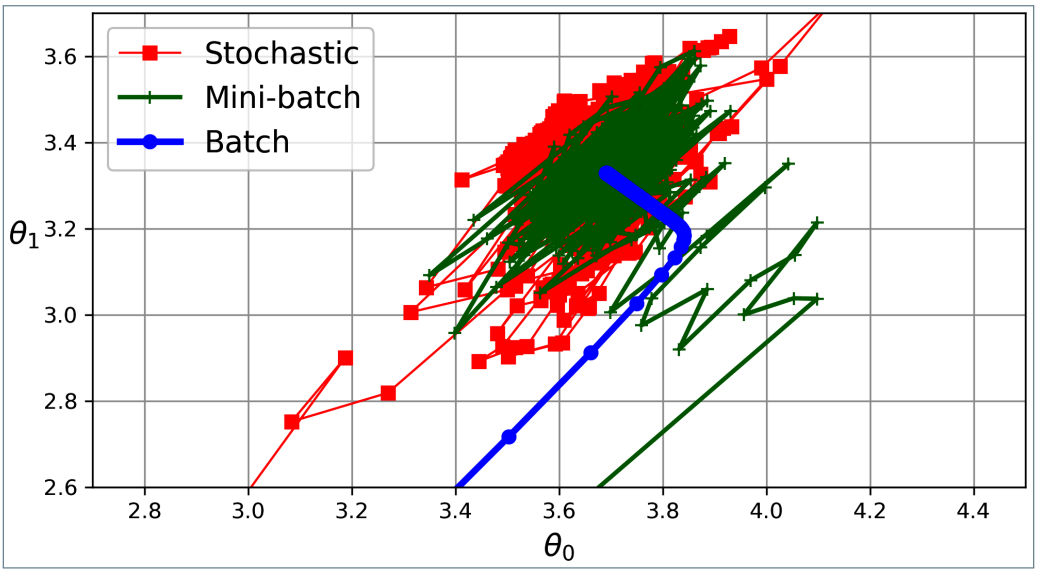

图 4-11: 参数空间中的梯度下降路径（Gradient descent paths in parameter space）

表 4-1 比较了目前讨论过的线性回归算法。[5] 其中 `m` 是训练实例数量，`n` 是特征数量。

表 4-1: 线性回归算法比较

算法	大 m	支持核外	大 n	超参数	需要缩放	Scikit-Learn
正规方程	快	否	慢	0	否	N/A
SVD	快	否	慢	0	否	LinearRegression
批量 GD	慢	否	快	2	是	N/A
随机 GD	快	是	快	≥2	是	SGDRegressor
小批量 GD	快	是	快	≥2	是	N/A

训练结束后几乎没有差异：所有这些算法最终得到的模型都非常相似，做预测时也完全相同。

[5] 正规方程只能执行线性回归，而梯度下降算法还可以训练许多其他模型，后面会看到。

4.4　多项式回归（Polynomial Regression）

如果数据比直线复杂怎么办？令人惊讶的是，仍然可以使用线性模型拟合非线性数据。一种简单方法是把每个特征的幂作为新特征添加进来，然后在扩展后的特征集上训练线性模型。这种技术称为多项式回归。

下面看一个例子。首先，根据一个简单的二次方程（即 $y = ax^2 + bx + c$）加上噪声，生成一些非线性数据（见图 4-12）：

```
rng = np.random.default_rng(seed=42)
m = 200  # number of instances
X = 6 * rng.random((m, 1)) - 3
y = 0.5 * X ** 2 + X + 2 + rng.standard_normal((m, 1))
```

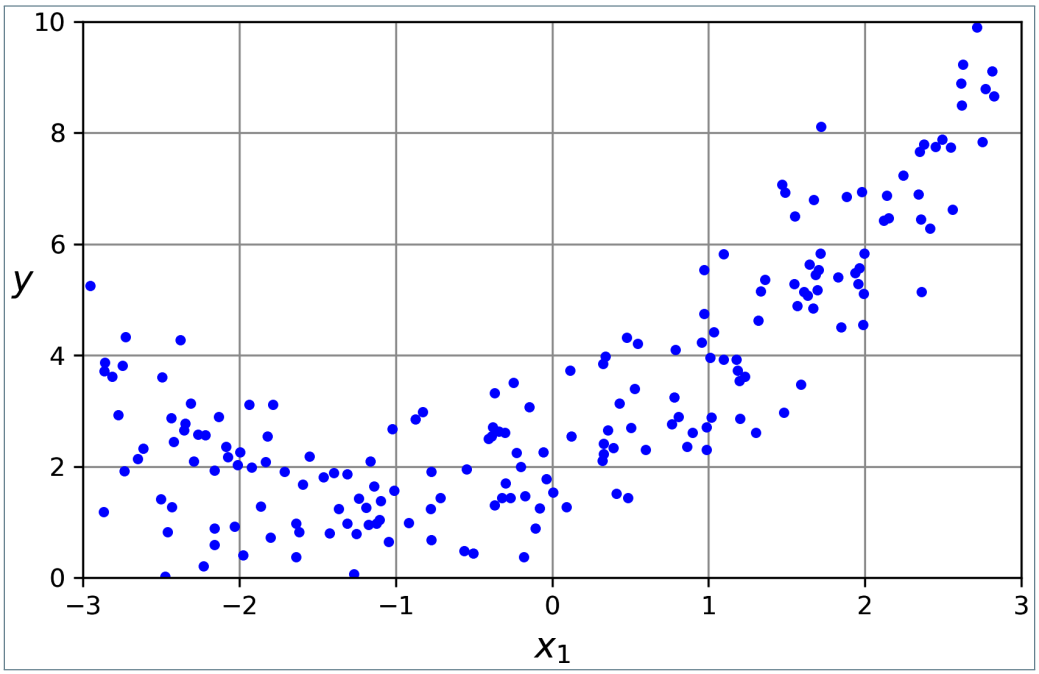

图 4-12: 生成的非线性含噪数据集（Generated nonlinear and noisy dataset）

显然，直线无法很好地拟合这些数据。因此，使用 Scikit-Learn 的 `PolynomialFeatures` 类转换训练数据，为训练集中的每个特征添加平方（即二次多项式）作为新特征（这里只有一个特征）：

```
>>> from sklearn.preprocessing import PolynomialFeatures
>>> poly_features = PolynomialFeatures(degree=2, include_bias=False)
>>> X_poly = poly_features.fit_transform(X)
>>> X[0]
array([1.64373629])
>>> X_poly[0]
array([1.64373629, 2.701869  ])
```

现在 `X_poly` 包含 `X` 的原始特征和该特征的平方。接着可以在扩展后的训练数据上拟合 `LinearRegression` 模型（见图 4-13）：

```
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X_poly, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([2.00540719]), array([[1.11022126, 0.50526985]]))
```

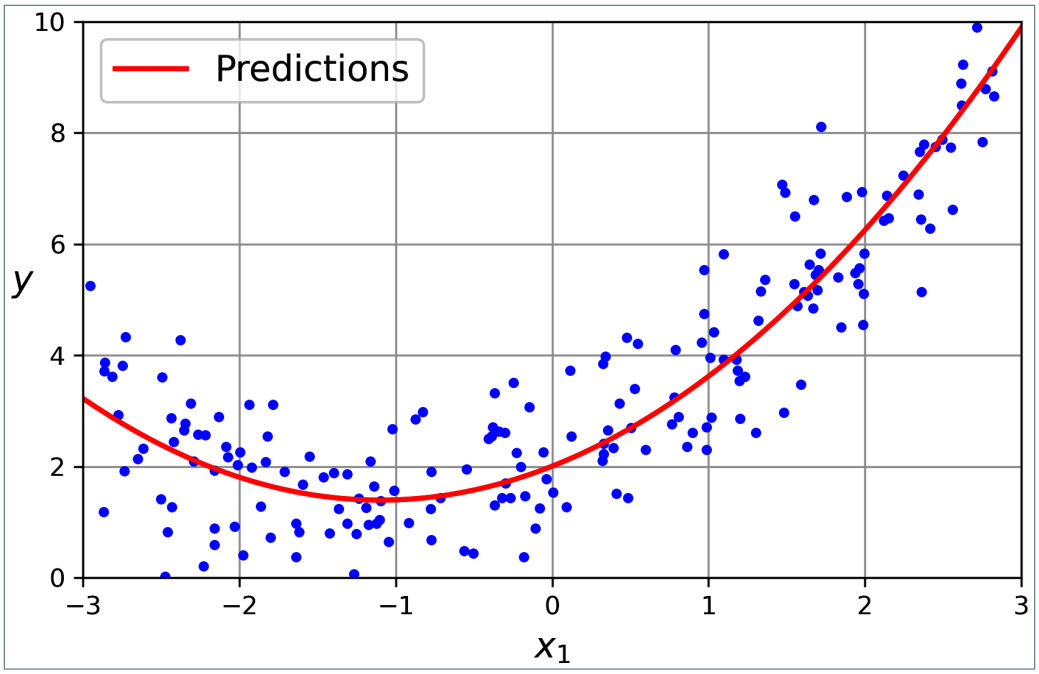

图 4-13: 多项式回归模型的预测结果（Polynomial regression model predictions）

结果不错：模型估计 $\hat{y} = 0.56x_1^2 + 0.93x_1 + 1.78$，而原函数实际为 $y = 0.5x_1^2 + 1.0x_1 + 2.0$ + 高斯噪声。

注意，当存在多个特征时，多项式回归还能找出特征之间的关系，而普通线性回归模型做不到这一点。这是因为 `PolynomialFeatures` 还会添加不超过给定次数的所

有特征组合。例如，如果有两个特征 a 和 b，degree=3 的 PolynomialFeatures 不仅会添加 a^2、a^3、b^2 和 b^3，还会添加 ab、a^2b 和 ab^2。

> 警告：PolynomialFeatures(degree=d) 会把包含 n 个特征的数组转换为包含 (n + d)! / d!n! 个特征的数组，其中 n! 是阶乘，等于 1 × 2 × 3 × ... × n。当心特征数量的组合爆炸！

4.5　学习曲线（Learning Curves）

如果执行高次多项式回归，通常会比普通线性回归更好地拟合训练数据。例如，图 4-14 对前面的训练数据应用 300 次多项式模型，并将结果与纯线性模型和二次模型进行比较。注意，300 次多项式模型会扭曲着尽可能贴近训练实例。

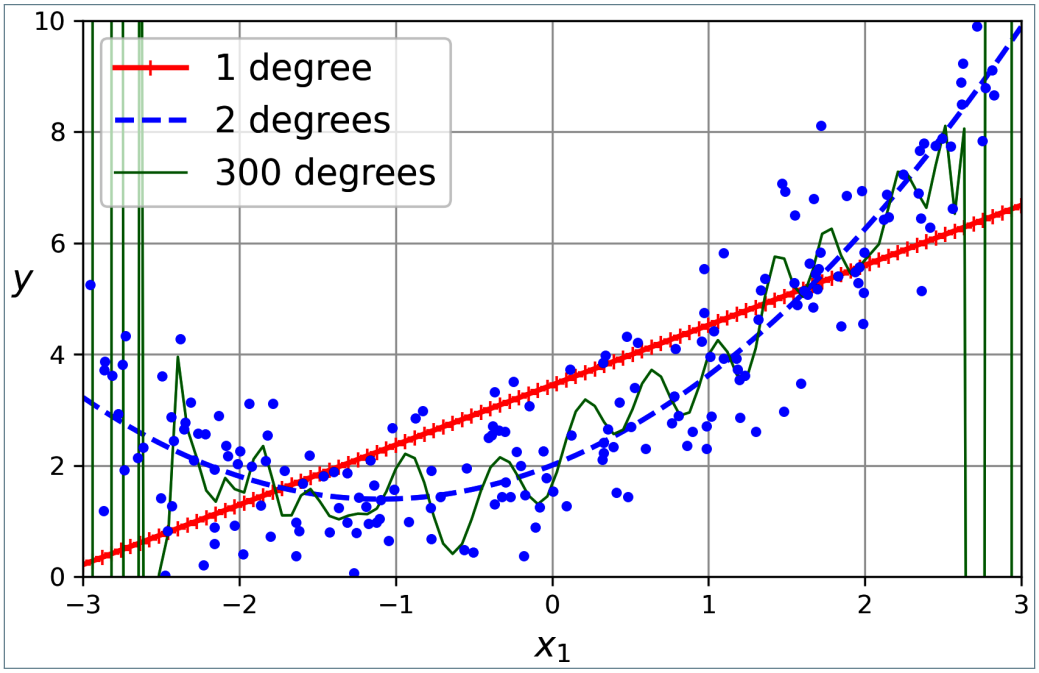

图 4-14: 高次多项式回归（High-degree polynomial regression）

这个高次多项式回归模型严重过拟合训练数据，而线性模型则欠拟合。这里泛化能力最好的模型是二次模型，因为数据正是由二次模型生成的。不过，一般情况下你不会知道生成数据的函数，那么该如何决定模型应该多复杂？如何判断模型是过拟合还是欠拟合？

在第 2 章中，你使用交叉验证估计模型的泛化性能。如果模型在训练数据上表现很好，但根据交叉验证指标泛化很差，那么模型过拟合了；如果在两者上都表现很差，那么模型欠拟合了。这是判断模型过于简单或过于复杂的一种方法。

另一种方法是查看学习曲线。学习曲线绘制模型训练误差和验证误差随训练迭代次数变化的情况：只需在训练过程中定期评估模型在训练集和验证集上的表现，然后绘图。如果模型无法增量训练（也就是不支持 **partial_fit()** 或 **warm_start**），就必须在训练集逐渐扩大的子集上多次训练模型。

Scikit-Learn 提供了有用的 **learning_curve()** 函数来完成这一工作：它使用交叉验证训练和评估模型。默认情况下，它会在逐渐增大的训练集子集上重新训练模型；如果模型支持增量学习，也可以设置 **exploit_incremental_learning=True**，让 **learning_curve()** 改为增量训练。该函数返回评估模型时使用的训练集大小，以及每个大小和每个交叉验证折对应的训练分数与验证分数。下面使用这个函数查看普通线性回归模型的学习曲线（见图 4-15）：

```
from sklearn.model_selection import learning_curve
train_sizes, train_scores, valid_scores = learning_curve(
    LinearRegression(), X, y, train_sizes=np.linspace(0.01, 1.0, 40), cv=5,
    scoring="neg_root_mean_squared_error")
train_errors = -train_scores.mean(axis=1)
valid_errors = -valid_scores.mean(axis=1)
plt.plot(train_sizes, train_errors, "r-+", linewidth=2, label="train")
plt.plot(train_sizes, valid_errors, "b-", linewidth=3, label="valid")
[...]  # beautify the figure: add labels, axis, grid, and legend
plt.show()
```

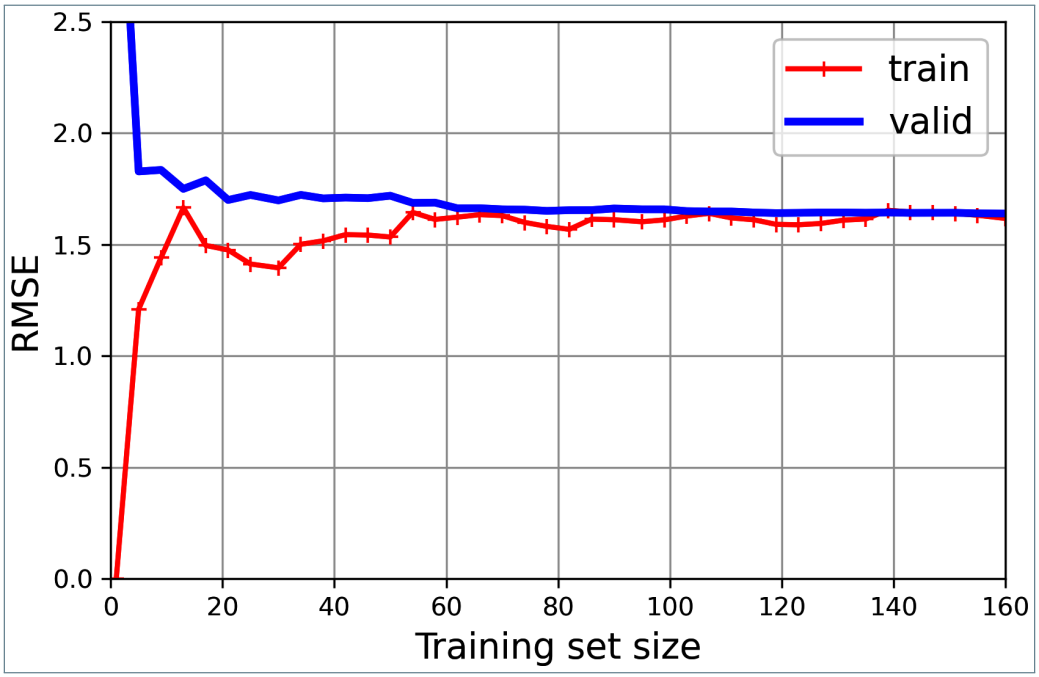

图 4-15: 学习曲线（Learning curves）

这个模型欠拟合，太简单，无法表达数据。怎么判断？看看训练误差即可。当训练集中只有一个或两个实例时，模型可以完美拟合它们，因此曲线从零开始。随着训练集加入更多实例，模型不可能继续完美拟合训练数据，既因为数据含噪，也因为数据根本不是线性的。因此，训练数据上的误差会逐渐上升，直到达到平台；此时继续添加训练实例，不会让平均误差明显变好或变坏。

再看验证误差。当模型只在极少数训练实例上训练时，它无法正确泛化，因此验证误差一开始很大。随着模型看到更多训练样本，它开始学习，验证误差逐渐下降。不过，直线仍然无法很好地建模这些数据，所以最终误差会达到一个平台，并且非常接近训练误差曲线。

这种学习曲线是欠拟合模型的典型特征：两条曲线都达到平台，彼此接近，而且都比较高。

> 提示：如果模型欠拟合训练数据，添加更多训练样本没有帮助。需要使用更好的模型，或者设计更好的特征。

现在查看相同数据上十次多项式模型的学习曲线（见图 4-16）：

```
from sklearn.pipeline import make_pipeline
polynomial_regression = make_pipeline(
    PolynomialFeatures(degree=10, include_bias=False),
    LinearRegression())
train_sizes, train_scores, valid_scores = learning_curve(
    polynomial_regression, X, y, train_sizes=np.linspace(0.01, 1.0, 40), cv=5,
    scoring="neg_root_mean_squared_error")
[...]  # same as earlier
```

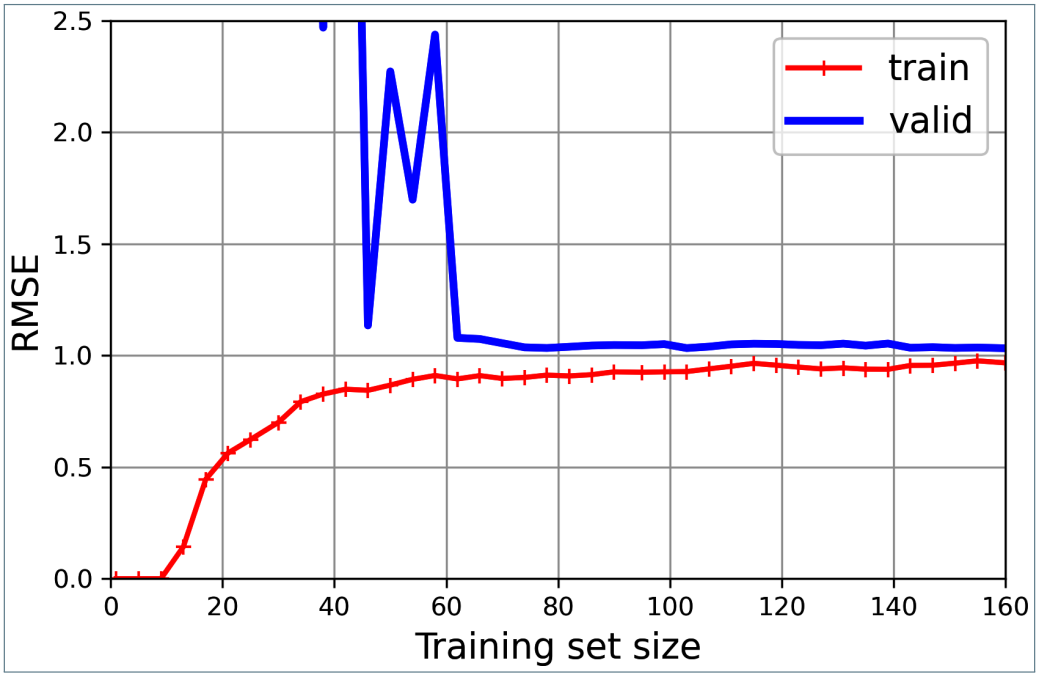

图 4-16: 十次多项式模型的学习曲线（Learning curves for the 10th-degree polynomial model）

这些学习曲线与前面的有些相似，但有两个非常重要的区别：

- 训练数据上的误差比之前低得多。
- 两条曲线之间存在间隙。这意味着模型在训练数据上的表现比在验证数据上好，这是过拟合模型的典型特征。不过，如果使用大得多的训练集，两条曲线会继续靠近。

> 偏差/方差权衡：统计学和机器学习中的一个重要理论结果是，模型的泛化误差可以表示为三个截然不同的误差之和。

- 偏差（bias）：泛化误差中由错误假设造成的部分，例如假设数据是线性的，而实际数据是二次的。高偏差模型最有可能欠拟合训练数据。[6]
- 方差（variance）：由模型对训练数据中的微小变化过度敏感造成的部分。具有很多自由度的模型（例如高次多项式模型）很可能方差较高，从而过拟合训练数据。
- 不可约误差（irreducible error）：由数据本身的噪声造成的部分。降低这部分误差的唯一办法是清理数据，例如修复数据源中的问题（传感器损坏等），或检测并移除异常值。

提高模型复杂度通常会增加方差、降低偏差；反过来，降低模型复杂度（或增加正则化）会增加偏差、降低方差（见图 4-17）。这就是所谓的权衡。

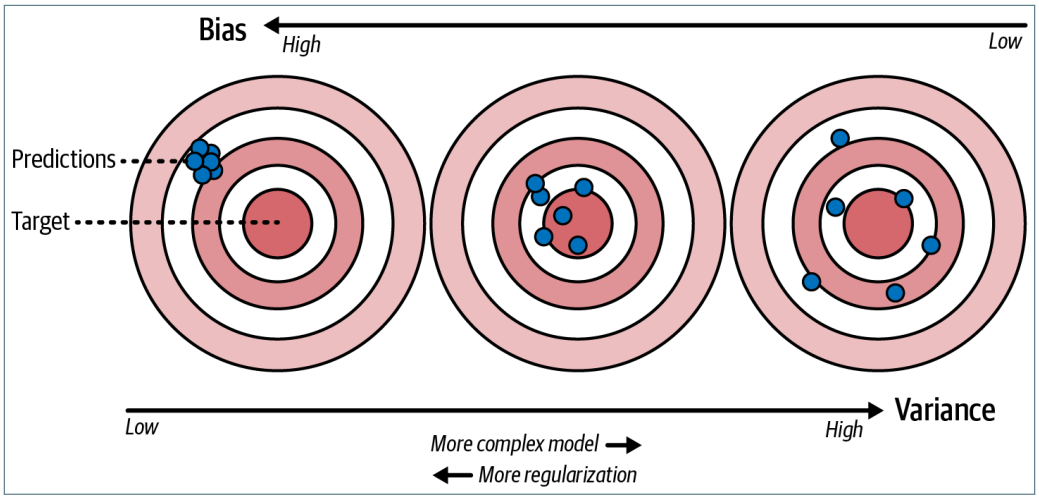

图 4-17：偏差/方差权衡（Bias/variance trade-off）

> 提示：改进过拟合模型的一种方法，是不断提供更多训练数据，直到验证误差足够接近训练误差。

[6] 这里的偏差概念不要与线性模型的偏置项混淆。

4.6　正则化线性模型（Regularized Linear Models）

正如第 1 章和第 2 章所见，降低过拟合的一种好方法是对模型进行正则化，也就是约束模型：自由度越少，就越难过拟合数据。正则化多项式模型的一种简单方法，是降低多项式次数。

线性模型也能正则化吗？你可能会疑惑为什么要这样做：线性模型难道还不够受约束吗？线性回归做出了一些假设，包括输入和输出之间的真实关系是线性的，噪声均值为零、方差恒定且与输入独立；此外，输入矩阵满秩，意味着输入之间不共线，并且样本数量至少与参数数量相同。[7] 实际中，这些假设并不总是完全成立。例如，有些输入可能接近共线，这会使线性回归在数值上不稳定，也就是说，训练集的极小差异可能对训练后的模型造成很大影响。正则化可以稳定线性模型并提高准确性。

那么，如何正则化线性模型？通常是约束模型的权重。本节将讨论岭回归、Lasso 回归和弹性网络回归，它们分别实现了三种不同的约束方式。

4.6.1　岭回归（Ridge Regression）

岭回归（也称 Tikhonov 正则化）是线性回归的正则化版本：在 MSE 中加入等于 α/m Σ(i=1 到 n) θi² 的正则化项。这会迫使学习算法不仅拟合数据，还要尽可能保持较小的模型权重。这个约束使模型灵活性降低，避免模型过度拉伸自己来拟合每个数据点，从而降低过拟合风险。注意，正则化项只应在训练期间加入成本函数；模型训练完毕后，应使用未正则化的 MSE（或 RMSE）评估模型性能。

超参数 α 控制正则化程度。如果 α = 0，岭回归就是线性回归；如果 α 很大，所有权重都会非常接近零，结果是一条穿过数据均值的水平线。公式 4-9 给出了岭回归成本函数。[8]

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \frac{\alpha}{m}\sum_{i=1}^{n}\theta_i^2 \tag{4-9}$$

公式 4-9　岭回归成本函数（Ridge regression cost function）

注意，偏置项 θ0 不进行正则化（求和从 i = 1 而不是 0 开始）。如果用 w 表示特征权重向量（θ1 到 θn），则正则化项等于 α(||w||2)² / m，其中 ||w||2 表示权重向量

[7] 当一个输入等于其他输入的线性组合时，输入之间就是共线的。例如，摄氏温度与华氏温度共线。

[8] 对于没有简短名称的成本函数，通常使用 J(θ) 这一记号；本书后面会经常使用它。上下文会明确当前讨论的是哪个成本函数。

的 ℓ2 范数。[9] 对于批量梯度下降，只需将 2αw / m 加到 MSE 梯度向量中对应特征权重的部分，不要加到偏置项的梯度上（见公式 4-7）。

> 警告：执行岭回归前必须缩放数据，例如使用 Scikit-Learn 的 StandardScaler，因为岭回归对输入特征的尺度敏感。大多数正则化模型也是如此。

图 4-18 展示了在非常嘈杂的线性数据上，使用不同 α 值训练的多个岭回归模型。左侧直接使用岭回归模型，得到线性预测；右侧先用 PolynomialFeatures(degree=10) 扩展数据，再使用 StandardScaler 缩放，最后对所得特征应用岭回归模型，也就是带岭正则化的多项式回归。注意，增加 α 会得到更平坦的预测（更不极端，也更合理），从而降低模型方差，但增加偏差。

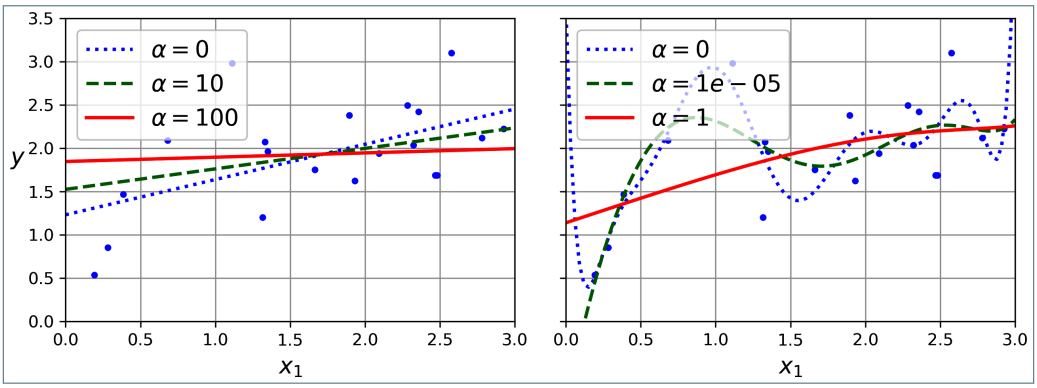

图 4-18: 使用不同岭正则化程度的线性模型（左）和多项式模型（右）（Linear and polynomial models with various levels of ridge regularization）

与线性回归一样，岭回归既可以通过计算闭式方程，也可以通过梯度下降执行；优缺点相同。公式 4-10 展示了闭式解，其中 A 是 (n + 1) × (n + 1) 单位矩阵，[10] 但左上角元素为 0，对应偏置项。

$$\hat{\boldsymbol{\theta}} = (X^{\mathsf{T}}X + \alpha A)^{-1}X^{\mathsf{T}}\mathbf{y} \tag{4-10}$$

公式 4-10　岭回归闭式解（Ridge regression closed-form solution）

下面使用闭式解（采用 André-Louis Cholesky 的矩阵分解技术变体）通过 Scikit-Learn 执行岭回归：

[9] 范数在第 2 章中讨论过。

[10] 除主对角线（从左上到右下）上的元素为 1 外，其余元素均为 0 的方阵。

```
>>> from sklearn.linear_model import Ridge
>>> ridge_reg = Ridge(alpha=0.1, solver="cholesky")
>>> ridge_reg.fit(X, y)
>>> ridge_reg.predict([[1.5]])
array([1.84414523])
```

也可以使用随机梯度下降：[11]

```
>>> sgd_reg = SGDRegressor(penalty="l2", alpha=0.1 / m, tol=None,
...                        max_iter=1000, eta0=0.01, random_state=42)
...
>>> sgd_reg.fit(X, y.ravel())  # y.ravel() because fit() expects 1D targets
>>> sgd_reg.predict([[1.5]])
array([1.83659707])
```

penalty 超参数设置要使用的正则化项类型。指定"l2" 表示让 SGD 在 MSE 成本函数中加入等于权重向量 $\ell 2$ 范数平方乘以 alpha 的正则化项。这与岭回归类似，只是这里没有除以 m；因此传入 alpha=0.1 / m，以获得与 Ridge(alpha=0.1) 相同的结果。

> 提示：RidgeCV 类也执行岭回归，并使用交叉验证自动调整超参数。它大致等价于使用 GridSearchCV，但针对岭回归进行了优化，运行速度快得多。其他一些估计器（主要是线性估计器）也有高效的 CV 变体，例如 LassoCV 和 ElasticNetCV。

4.6.2　Lasso 回归（Lasso Regression）

最小绝对收缩与选择算子回归（least absolute shrinkage and selection operator regression，通常简称 Lasso 回归）是线性回归的另一种正则化版本。与岭回归一样，它向成本函数添加正则化项，但使用权重向量的 $\ell 1$ 范数，而不是 $\ell 2$ 范数的平方（见公式 4-11）。注意，Lasso 中 $\ell 1$ 范数乘以 2α，而岭回归中的 $\ell 2$ 范数乘以 α/m。这些系数经过选择，确保最优 α 值与训练集大小无关：不同范数会导致不同系数。详见 Scikit-Learn issue #15657。

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + 2\alpha \sum_{i=1}^{n} |\theta_i| \tag{4-11}$$

[11] 也可以使用带"sag" 求解器的 Ridge 类。随机平均梯度下降（stochastic average GD）是随机 GD 的一种变体。更多细节请参阅 Mark Schmidt 等人（来自不列颠哥伦比亚大学）的论文《使用随机平均梯度算法最小化有限和》（https://homl.info/12）。

公式 4-11　Lasso 回归成本函数（Lasso regression cost function）

图 4-19 展示了与图 4-18 相同的情况，但将岭模型替换为 Lasso 模型，并使用了不同的 α 值。

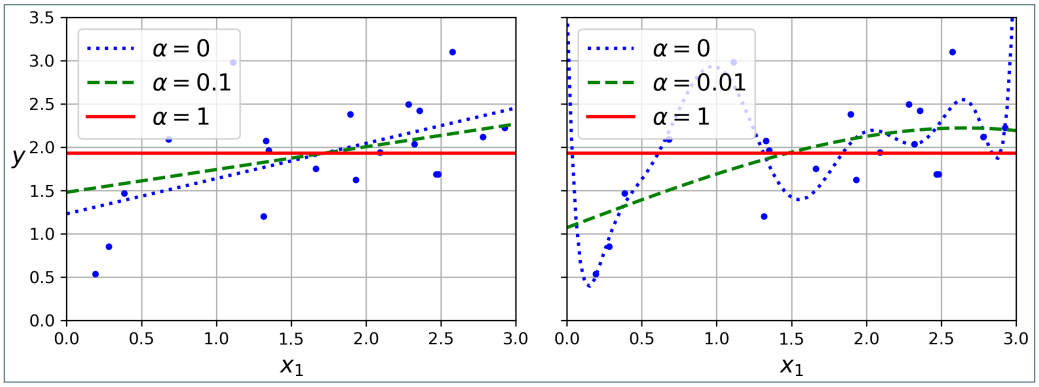

图 4-19: 使用不同 Lasso 正则化程度的线性模型（左）和多项式模型（右）（Linear and polynomial models with various levels of lasso regularization）

Lasso 回归的一个重要特征，是它倾向于消除最不重要特征的权重（即将其设为零）。例如，图 4-19 右侧图中虚线模型（α = 0.01）大致看起来像三次曲线：所有高次多项式特征的权重都等于零。换句话说，Lasso 回归会自动执行特征选择，并输出一个只有少量非零特征权重的稀疏模型。当然，这存在权衡：如果 α 增加太多，模型会非常稀疏，但性能会大幅下降。

要理解 ℓ1 范数为什么会导致稀疏性，可以看图 4-20：坐标轴代表两个模型参数，背景轮廓代表不同的损失函数。左上图中的轮廓表示 ℓ1 损失（|θ1| + |θ2|）；越接近任意一条坐标轴，损失下降得越快。例如，如果将模型参数初始化为 θ1 = 2、θ2 = 0.5，运行梯度下降会使两个参数等量减少（虚线黄色路径），因此由于 θ2 一开始更接近 0，它会先到达 0。之后，梯度下降会沿“沟槽”下滑，直到 θ1 = 0（由于 ℓ1 的梯度不会接近 0，而是每个参数取 -1 或 1，因此会有一些周围跳动）。

右上图中的轮廓表示 Lasso 回归的成本函数，即 MSE 成本函数加 ℓ1 损失。白色小圆圈表示梯度下降优化模型参数时的路径，初始参数大约为 θ1 = 0.25、θ2 = -1。注意，路径再次快速到达 θ2 = 0，随后沿沟槽下滑，在全局最优点（红色方块）附近来回跳动。如果增加 α，全局最优点会沿虚线黄色路径向左移动；如果减小 α，全局最优点会向右移动（在这个例子中，未正则化 MSE 的最优参数是 θ1 = 2、θ2 = 0.5）。

底部两图展示了使用 ℓ2 惩罚时的相同情况。左下图可以看到，越接近原点，ℓ2 损失下降得越多，因此梯度下降会沿直线朝向原点。右下图中的轮廓表示岭回归成

本函数，即 MSE 成本函数加 ℓ2 损失。可以看到，参数接近全局最优点时梯度变小，因此梯度下降会自然减速。这减少了周围跳动，有助于岭回归比 Lasso 更快收敛。还要注意，增加 α 时，最优参数（红色方块）会越来越接近原点，但不会被完全消除。

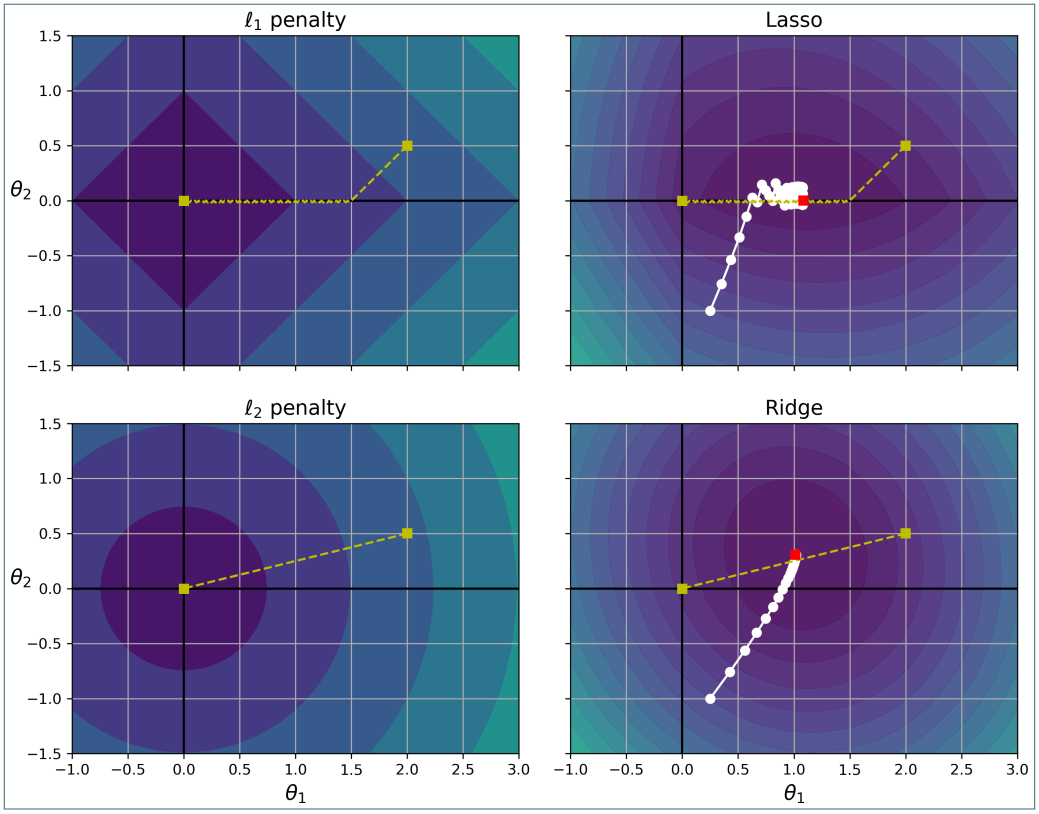

图 4-20: Lasso 与岭正则化的比较（Lasso versus ridge regularization）

> 提示：为了避免使用 Lasso 回归时梯度下降在最后阶段围绕最优点跳动，需要逐渐降低训练过程中的学习率。它仍会在最优点附近跳动，但步长越来越小，因此最终会收敛。

Lasso 成本函数在 θi = 0（i = 1, 2, ⋯, n）处不可微。不过，如果某个 θi = 0，使用次梯度向量（subgradient vector）[12]仍然可以执行梯度下降。公式 4-12 展示了可用于 Lasso 成本函数梯度下降的次梯度向量。

[12]在不可微点处，可以把次梯度向量理解为该点两侧梯度向量之间的一个中间向量。

$$g(\boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta}) + 2\alpha \begin{pmatrix} \text{sign}(\theta_1) \\ \text{sign}(\theta_2) \\ \vdots \\ \text{sign}(\theta_n) \end{pmatrix}, \quad \text{sign}(\theta_i) = \begin{cases} -1 & \text{if } \theta_i < 0 \\ 0 & \text{if } \theta_i = 0 \\ +1 & \text{if } \theta_i > 0 \end{cases} \tag{4-12}$$

公式 4-12　Lasso 回归次梯度向量（Lasso regression subgradient vector）

下面是使用 Scikit-Learn Lasso 类的一个小例子：

```
>>> from sklearn.linear_model import Lasso
>>> lasso_reg = Lasso(alpha=0.1)
>>> lasso_reg.fit(X, y)
>>> lasso_reg.predict([[1.5]])
array([1.87550211])
```

也可以使用 SGDRegressor(penalty="l1", alpha=0.1)。

4.6.3　弹性网络回归（Elastic Net Regression）

弹性网络回归介于岭回归和 Lasso 回归之间。正则化项是岭回归和 Lasso 回归正则化项的加权和，可以通过混合率 r 控制两者的比例。当 r = 0 时，弹性网络等价于岭回归；当 r = 1 时，等价于 Lasso 回归（公式 4-13）。

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + r\left(2\alpha \sum_{i=1}^{n} |\theta_i|\right) + (1-r)\left(\frac{\alpha}{m} \sum_{i=1}^{n} \theta_i^2\right) \tag{4-13}$$

公式 4-13　弹性网络成本函数（Elastic net cost function）

那么什么时候应该使用弹性网络、岭回归、Lasso 或普通线性回归（不带任何正则化）？几乎总是建议至少使用一点正则化，因此通常应避免普通线性回归。岭回归是不错的默认选择；但如果怀疑只有少数特征有用，应优先使用 Lasso 或弹性网络，因为它们倾向于将无用特征的权重降为零，如前面所述。一般来说，弹性网络优于 Lasso，因为当特征数量大于训练实例数量，或多个特征高度相关时，Lasso 可能表现不稳定。

下面是使用 Scikit-Learn ElasticNet 的简短例子（l1_ratio 对应混合率 r）：

```
>>> from sklearn.linear_model import ElasticNet
>>> elastic_net = ElasticNet(alpha=0.1, l1_ratio=0.5)
>>> elastic_net.fit(X, y)
>>> elastic_net.predict([[1.5]])
array([1.8645014])
```

4.6.4　提前停止（Early Stopping）

另一种正则化梯度下降等迭代学习算法的方法，是验证误差达到最小值时立即停止训练。这种常用技术称为提前停止。图 4-21 展示了使用批量梯度下降训练复杂模型的过程，这里的复杂模型是前面使用过的二次数据集上的高次多项式回归模型。随着 epoch 增加，算法逐渐学习，训练集上的预测误差（RMSE）下降，验证集上的预测误差也下降。

但是过了一段时间，验证误差停止下降并开始回升。这说明模型开始过拟合训练数据。使用提前停止时，只需在验证误差达到最小值时停止训练。这是一种如此简单而高效的正则化技术，以至于 Geoffrey Hinton 称它为“美丽的免费午餐”。[13] 不过，验证误差有时会在一段时间后再次下降，这称为双重下降（double descent）。它在大型神经网络中相当常见，也是一个活跃的研究领域。

[13] NeurIPS 2015 深度学习教程的第 63 页幻灯片（https://homl.info/freelunch）。

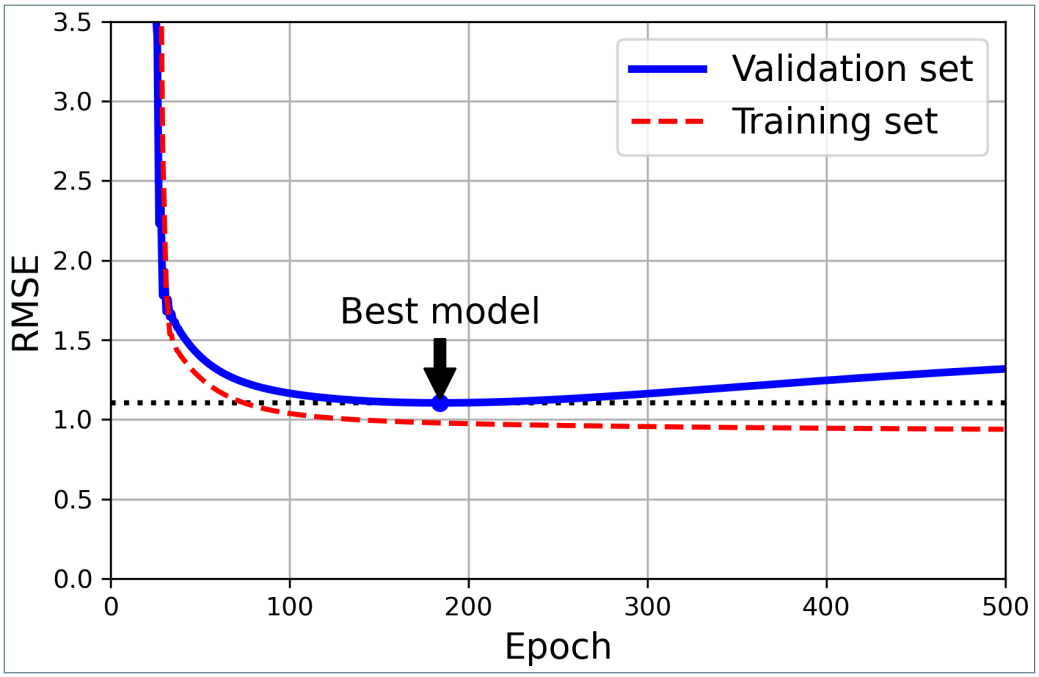

图 4-21: 提前停止正则化（Early stopping regularization）

下面是提前停止的基本实现：

```
from copy import deepcopy
from sklearn.metrics import root_mean_squared_error
from sklearn.preprocessing import StandardScaler
X_train, y_train, X_valid, y_valid = [...]  # split the quadratic dataset
preprocessing = make_pipeline(PolynomialFeatures(degree=90, include_bias=False),
                              StandardScaler())
X_train_prep = preprocessing.fit_transform(X_train)
X_valid_prep = preprocessing.transform(X_valid)
sgd_reg = SGDRegressor(penalty=None, eta0=0.002, random_state=42)
n_epochs = 500
best_valid_rmse = float('inf')
for epoch in range(n_epochs):
    sgd_reg.partial_fit(X_train_prep, y_train)
    y_valid_predict = sgd_reg.predict(X_valid_prep)
    val_error = root_mean_squared_error(y_valid, y_valid_predict)
    if val_error < best_valid_rmse:
        best_valid_rmse = val_error
        best_model = deepcopy(sgd_reg)
```

这段代码首先添加多项式特征并缩放所有输入特征，训练集和验证集都如此处理（代码假设你已经把原始训练集划分为较小的训练集和验证集）。然后创建一个不带正则化、学习率较小的 SGDRegressor 模型。在训练循环中，它调用 partial_fit() 而

不是 fit()，以执行增量学习。每个 epoch 都测量验证集上的 RMSE。如果低于目前见过的最低 RMSE，就把模型副本保存到 best_model 变量中。这个实现实际上不会停止训练，但训练结束后可以恢复最佳模型。注意，模型使用 copy.deepcopy() 复制，因为它同时复制模型超参数和已学习参数；相比之下，sklearn.base.clone() 只会复制模型超参数。

4.7　逻辑回归（Logistic Regression）

正如第 1 章所讨论的，有些回归算法可以用于分类，反过来也一样。逻辑回归（也称 logit 回归）通常用于估计实例属于某个类别的概率，例如一封电子邮件是垃圾邮件的概率。如果估计概率高于给定阈值（通常为 50%），模型就预测实例属于该类别（称为正类，标记为“1”）；否则预测它不属于该类别（属于负类，标记为“0”）。这就构成了一个二分类器。

4.7.1　估计概率（Estimating Probabilities）

逻辑回归是如何工作的？与线性回归模型一样，逻辑回归模型计算输入特征的加权和（加上偏置项），但不像线性回归模型那样直接输出结果，而是输出这个结果的逻辑函数值（见公式 4-14）。

$$\hat{p} = h_{\boldsymbol{\theta}}(\mathbf{x}) = \sigma(\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}) \tag{4-14}$$

公式 4-14　逻辑回归模型的估计概率（向量化形式）（Estimated probability, vectorized form）

逻辑函数记作 σ(·)，是一个 S 形函数，会输出 0 到 1 之间的数。它的定义如公式 4-15 和图 4-22 所示。

$$\sigma(t) = \frac{1}{1 + \exp(-t)} \tag{4-15}$$

公式 4-15　逻辑函数（Logistic function）

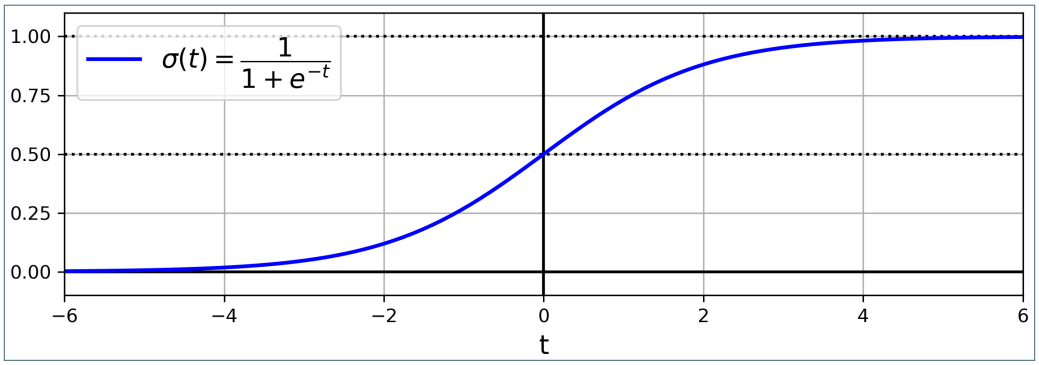

图 4-22: 逻辑函数（Logistic function）

逻辑回归模型估计实例 x 属于正类的概率 p_hat = hθ(x) 后，就能很容易地做出预测（见公式 4-16）。

$$\hat{y} = \begin{cases} 0 & \text{if } \hat{p} < 0.5 \\ 1 & \text{if } \hat{p} \geq 0.5 \end{cases} \tag{4-16}$$

公式 4-16　使用 50% 阈值概率的逻辑回归模型预测（Model prediction using a 50% threshold）

注意，当 t < 0 时 σ(t) < 0.5，当 t ⩾ 0 时 σ(t) ⩾ 0.5。因此，使用默认 50% 概率阈值的逻辑回归模型，在 theta^T x 为正时预测 1，在它为负时预测 0。

> 提示：分数 t 通常称为 logit。这个名称来自 logit 函数 logit(p) = log(p / (1 - p))，它是逻辑函数的逆函数。实际上，如果对估计概率 p 计算 logit，就会得到 t。logit 也称为对数几率（log-odds），因为它是正类估计概率与负类估计概率之比的对数。

4.7.2　训练与成本函数（Training and Cost Function）

现在已经知道逻辑回归模型如何估计概率和做出预测了。那么它如何训练？训练目标是设置参数向量 θ，使模型对正类实例（y = 1）估计较高概率，对负类实例（y = 0）估计较低概率。对于单个训练实例 x，公式 4-17 中的成本函数体现了这一思想。

$$c(\boldsymbol{\theta}) = \begin{cases} -\log(\hat{p}) & \text{if } y = 1 \\ -\log(1-\hat{p}) & \text{if } y = 0 \end{cases} \tag{4-17}$$

公式 4-17　单个训练实例的成本函数（Cost function of a single training instance）

这个成本函数很合理，因为当 t 接近 0 时，-log(t) 会变得非常大。因此，如果模型对正类实例估计出接近 0 的概率，或者对负类实例估计出接近 1 的概率，成本都会很大。反过来，当 t 接近 1 时，-log(t) 接近 0；如果负类实例的估计概率接近 0，或正类实例的估计概率接近 1，成本就接近 0，这正是我们希望的结果。

整个训练集上的成本是所有训练实例成本的平均值。它可以写成一个单一表达式，称为对数损失（log loss；公式 4-18）。

$$J(\boldsymbol{\theta}) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(\hat{p}^{(i)}\right) + (1-y^{(i)})\log\left(1-\hat{p}^{(i)}\right)\right] \tag{4-18}$$

公式 4-18　逻辑回归成本函数（对数损失）（Logistic regression cost function）

> 警告：对数损失并不是凭空提出的。数学上可以证明（使用贝叶斯推断），假设实例围绕各类别均值服从高斯分布，最小化该损失会得到最大似然意义下的最优模型。使用对数损失，就隐含地做出了这个假设。假设越不符合实际，模型偏差就越大。类似地，使用 MSE 训练线性回归模型时，隐含假设数据是纯线性的，并带有高斯噪声。如果数据不是线性的（例如是二次的），或者噪声不是高斯噪声（例如异常值并非极少见），模型就会产生偏差。

坏消息是，没有已知的闭式方程可以计算使这个成本函数最小的 θ，也就是没有正规方程的等价物。好消息是，这个成本函数是凸函数，因此梯度下降（或其他优化算法）一定能找到全局极小值，只要学习率不过大且等待足够长时间。成本函数关于第 j 个模型参数 θj 的偏导数见公式 4-19。

$$\frac{\partial}{\partial\theta_j}J(\boldsymbol{\theta}) = \frac{1}{m}\sum_{i=1}^{m}\left(\sigma\left(\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}^{(i)}\right) - y^{(i)}\right)x_j^{(i)} \tag{4-19}$$

公式 4-19　逻辑回归成本函数的偏导数（Logistic cost function partial derivatives）

这个公式与公式 4-6 非常相似：对于每个实例，计算预测误差，将其乘以第 j 个特征值，然后对所有训练实例取平均。得到包含所有偏导数的梯度向量后，就可以在

批量梯度下降算法中使用它。就是这样：现在你已经知道如何训练逻辑回归模型了。对于随机 GD，可以一次使用一个实例；对于小批量 GD，则一次使用一个小批量。

4.7.3　决策边界（Decision Boundaries）

我们可以使用 iris 数据集说明逻辑回归。这是一个著名的数据集，包含 150 朵鸢尾花的萼片和花瓣长度、宽度，花朵属于三个不同物种：Iris setosa、Iris versicolor 和 Iris virginica（见图 4-23）。[14]

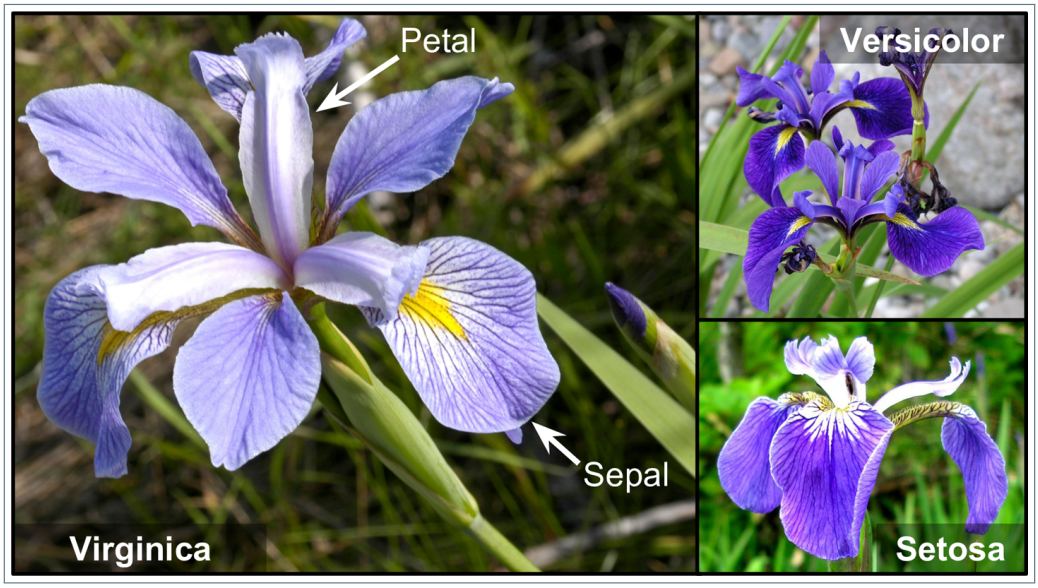

图 4-23: 三种鸢尾花（Flowers of three iris plant species）

尝试仅根据花瓣宽度构建一个分类器，检测 Iris virginica。第一步是加载数据并快速查看：

```
>>> from sklearn.datasets import load_iris
>>> iris = load_iris(as_frame=True)
>>> list(iris)
['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names',
 'filename', 'data_module']
>>> iris.data.head(3)
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
```

[14]照片取自相应的 Wikipedia 页面：Iris virginica 照片由 Frank Mayfield 提供（Creative Commons BY-SA 2.0，https://oreil.ly/O2fAq）；Iris versicolor 照片由 D. Gordon E. Robertson 提供（Creative Commons BY-SA 3.0，https://oreil.ly/pMbrK）；Iris setosa 照片属于公共领域。

```
0                5.1               3.5               1.4               0.2
1                4.9               3.0               1.4               0.2
2                4.7               3.2               1.3               0.2
>>> iris.target.head(3)  # note that the instances are not shuffled
0    0
1    0
2    0
Name: target, dtype: int64
>>> iris.target_names
array(['setosa', 'versicolor', 'virginica'], dtype='<U10')
```

接下来划分数据，并在训练集上训练逻辑回归模型：

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
X = iris.data[["petal width (cm)"]].values
y = iris.target_names[iris.target] == 'virginica'
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
log_reg = LogisticRegression(random_state=42)
log_reg.fit(X_train, y_train)
```

接着查看花瓣宽度从 0 cm 到 3 cm 的花朵的估计概率（见图 4-24）：[15]

[15] NumPy 的 reshape() 函数允许某一维为 -1，表示“自动”：该维的值会根据数组长度和其余维度推断。

```
X_new = np.linspace(0, 3, 1000).reshape(-1, 1)  # reshape to get a column vector
y_proba = log_reg.predict_proba(X_new)
decision_boundary = X_new[y_proba[:, 1] >= 0.5][0, 0]
plt.plot(X_new, y_proba[:, 0], "b--", linewidth=2,
         label="Not Iris virginica proba")
plt.plot(X_new, y_proba[:, 1], "g-", linewidth=2, label="Iris virginica proba")
plt.plot([decision_boundary, decision_boundary], [0, 1], "k:", linewidth=2,
         label="Decision boundary")
[...] # beautify the figure: add grid, labels, axis, legend, arrows, and samples
plt.show()
```

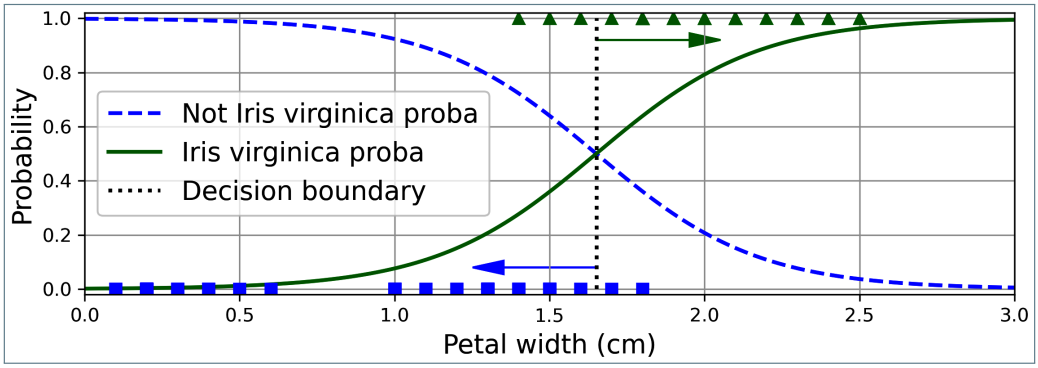

图 4-24: 估计概率与决策边界（Estimated probabilities and decision boundary）

Iris virginica 花朵的花瓣宽度（用三角形表示）范围为 1.4 cm 到 2.5 cm；其他鸢尾花（用方块表示）的花瓣宽度通常较小，范围为 0.1 cm 到 1.8 cm。注意两者有一些重叠。花瓣宽度超过约 2 cm 时，分类器非常确信这是 Iris virginica（对该类别输出高概率）；低于 1 cm 时，它非常确信不是 Iris virginica（对“Not Iris virginica”类别输出高概率）。在这两个极端之间，分类器并不确定。

不过，如果要求它预测类别（使用 predict() 方法，而不是 predict_proba()），它会返回概率最高的类别。因此，在大约 1.6 cm 处存在一条决策边界，此时两个概率都等于 50%：花瓣宽度大于 1.6 cm，分类器预测为 Iris virginica；否则预测为其他类别，即使它并不是非常确信：

```
>>> decision_boundary
np.float64(1.6516516516516517)
>>> log_reg.predict([[1.7], [1.5]])
array([ True, False])
```

图 4-25 展示了相同的数据集，但这次使用两个特征：花瓣宽度和花瓣长度。训练后，逻辑回归分类器可以根据这两个特征，估计一朵新花属于 Iris virginica 的概率。虚线表示模型估计概率为 50% 的点，这就是模型的决策边界。注意，它是一条线性边界。[16] 每条平行线表示模型输出某个特定概率的点，从左下方的 15% 到右上方的 90%。根据模型，位于右上方 90% 概率线之外的花朵，属于 Iris virginica 的概率超过 90%。

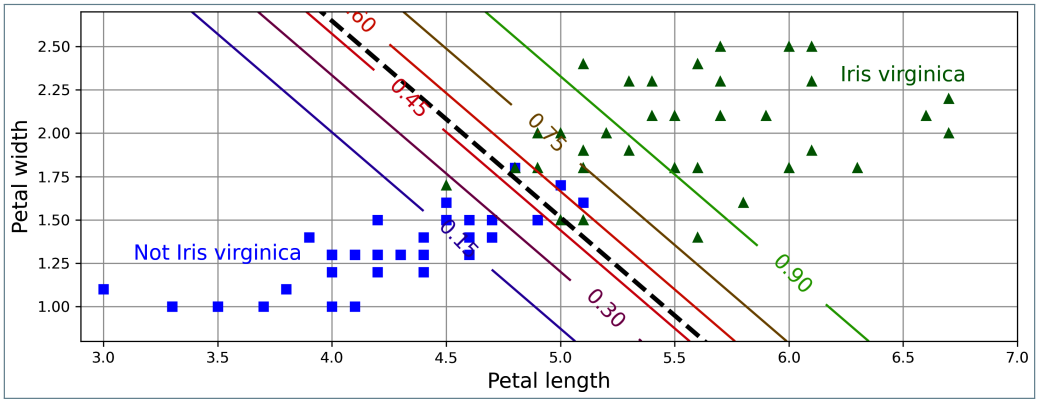

图 4-25: 线性决策边界（Linear decision boundary）

> 提示：控制 Scikit-Learn LogisticRegression 模型正则化强度的超参数不是其他线性模型使用的 alpha，而是它的倒数 C。C 越大，模型正则化越弱。

与其他线性模型一样，逻辑回归模型可以使用 ℓ1 或 ℓ2 惩罚进行正则化。Scikit-Learn 默认实际加入的是 ℓ2 惩罚。

4.8　Softmax 回归（Softmax Regression）

逻辑回归模型可以推广为直接支持多个类别，而不必像第 3 章讨论的那样训练并组合多个二分类器。这称为 Softmax 回归，也称多项逻辑回归（multinomial logistic regression）。

[16] 它是满足 θ0 + θ1x1 + θ2x2 = 0 的点集，该方程定义了一条直线。

思想很简单：给定实例 x 后，Softmax 回归模型首先为每个类别 k 计算一个分数 sk(x)，然后将这些分数输入 Softmax 函数（也称归一化指数函数），估计每个类别的概率。计算 sk(x) 的方程应该很熟悉，因为它与线性回归预测方程相同（见公式 4-20）。

$$s_k(\mathbf{x}) = \left(\boldsymbol{\theta}^{(k)}\right)^{\mathsf{T}} \mathbf{x} \tag{4-20}$$

公式 4-20　类别 k 的 Softmax 分数（Softmax score for class k）

注意，每个类别都有自己的专用参数向量 θ(k)。所有这些向量通常作为参数矩阵 Θ 的行保存。

计算出实例 x 对每个类别的分数后，就可以把这些分数输入 Softmax 函数，估计实例属于类别 k 的概率（公式 4-21）。该函数计算每个分数的指数，然后将它们归一化，也就是除以所有指数的和。分数通常称为 logits 或 log-odds，不过它们实际上是未经归一化的 log-odds。

$$\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp\left(s_k(\mathbf{x})\right)}{\sum_{j=1}^{K} \exp\left(s_j(\mathbf{x})\right)} \tag{4-21}$$

公式 4-21　Softmax 函数（Softmax function）

在这个公式中：

- K 是类别数量。
- s(x) 是一个向量，包含实例 x 对各类别的分数。
- σ(s(x))k 是根据该实例各类别分数估计的、实例 x 属于类别 k 的概率。

与逻辑回归分类器一样，Softmax 回归分类器默认预测估计概率最高的类别（这也就是分数最高的类别），如公式 4-22 所示。

$$\hat{y} = \arg\max_k \sigma(\mathbf{s}(\mathbf{x}))_k = \arg\max_k s_k(\mathbf{x}) = \arg\max_k \left(\left(\boldsymbol{\theta}^{(k)}\right)^{\mathsf{T}} \mathbf{x}\right) \tag{4-22}$$

公式 4-22　Softmax 回归分类器的预测（Softmax regression classifier prediction）

argmax 运算符返回使函数达到最大值的变量值。在这里，它返回使估计概率 σ(s(x))k 最大的 k。

> 提示：Softmax 回归分类器一次只预测一个类别，也就是说它是多分类而不是多输出分类，因此只能用于类别互斥的情况，例如不同植物物种。不能用它识别一张照片中的多个人。

现在已经知道模型如何估计概率并做出预测，下面看看如何训练。目标是让模型对目标类别估计出高概率（从而对其他类别估计出低概率）。最小化公式 4-23 所示的成本函数，应当能达到这个目标。该成本函数称为交叉熵（cross entropy），因为它会惩罚模型对目标类别估计低概率的情况。交叉熵经常用于衡量一组估计类别概率与目标类别的匹配程度。

$$J(\mathbf{\Theta}) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log\left(\hat{p}_k^{(i)}\right) \tag{4-23}$$

公式 4-23　交叉熵成本函数（Cross entropy cost function）

在这个公式中，y(k)(i) 是目标概率，表示第 i 个实例属于类别 k 的目标概率。一般来说，它等于 1 或 0，取决于该实例是否属于该类别。注意，当只有两个类别（K = 2）时，这个成本函数等价于逻辑回归成本函数（对数损失；见公式 4-18）。

> 交叉熵：交叉熵源自 Claude Shannon 的信息论。假设你想高效传输每天的天气信息。如果有 8 种可能（晴天、下雨等），可以用 3 个比特编码每种情况，因为 2^3 = 8。不过，如果认为几乎每天都是晴天，那么给“晴天”编码为一个比特（0），其他七种情况编码为以 1 开头的四个比特，会高效得多。交叉熵衡量每种情况实际发送的平均比特数。如果对天气的假设完全正确，交叉熵就等于天气本身的熵，也就是其内在不可预测性；但如果假设错误（例如经常下雨），交叉熵会多出一部分，称为 Kullback-Leibler（KL）散度。两个概率分布 p 和 q 之间的交叉熵定义为 H(p, q) = -Σx p(x) log q(x)（至少在分布离散时如此）。更多细节可以参考我关于该主题的视频。

成本函数关于 θ(k) 的梯度向量见公式 4-24。

$$\nabla_{\boldsymbol{\theta}^{(k)}} J(\mathbf{\Theta}) = \frac{1}{m}\sum_{i=1}^{m}\left(\hat{p}_k^{(i)} - y_k^{(i)}\right)\mathbf{x}^{(i)} \tag{4-24}$$

公式 4-24　类别 k 的交叉熵梯度向量（Cross entropy gradient vector for class k）

现在可以计算每个类别的梯度向量，然后使用梯度下降（或其他优化算法）寻找使成本函数最小的参数矩阵 Θ。

下面使用 Softmax 回归把鸢尾花分为三个类别。Scikit-Learn 的 LogisticRegression 分类器在三个以上类别上训练时，会自动使用 Softmax 回归（假设使用 solver="lbfgs"，这是默认值）。它默认还会应用 ℓ2 正则化，可以使用超参数 C 控制正则化强度：如前面所述，减小 C 会增强正则化。

```
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = iris["target"]
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
softmax_reg = LogisticRegression(C=30, random_state=42)
softmax_reg.fit(X_train, y_train)
```

下次遇到花瓣长度为 5 cm、宽度为 2 cm 的鸢尾花时，可以让模型判断其类型；模型会以 96% 的概率回答 Iris virginica（类别 2），以 4% 的概率回答 Iris versicolor：

```
>>> softmax_reg.predict([[5, 2]])
array([2])
>>> softmax_reg.predict_proba([[5, 2]]).round(2)
array([[0.  , 0.04, 0.96]])
```

图 4-26 展示了最终的决策边界，背景颜色表示不同区域。注意，任意两个类别之间的决策边界都是线性的。图中还用曲线表示 Iris versicolor 类别的概率，例如标记为 0.30 的曲线表示 30% 概率边界。注意，模型可能预测一个估计概率低于 50% 的类别。例如，在所有决策边界相交的点上，所有类别的估计概率都相等，都是 33%。

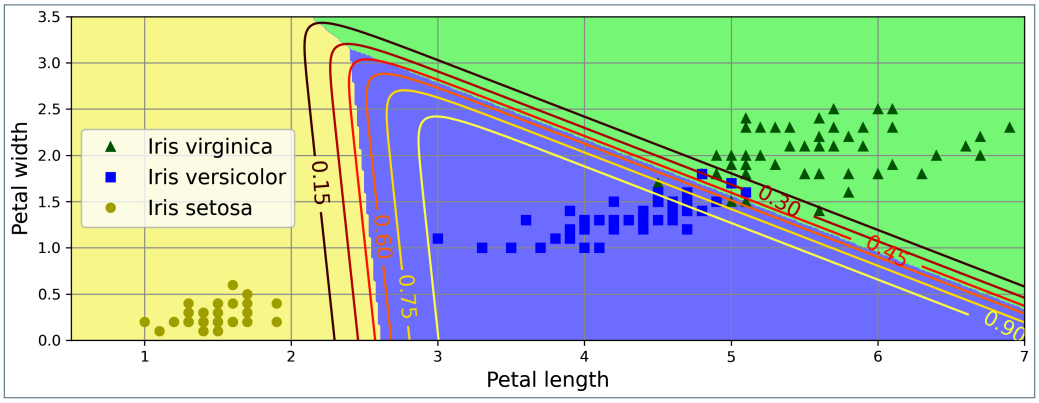

图 4-26: Softmax 回归决策边界（Softmax regression decision boundaries）

本章学习了训练线性模型的各种方法，包括回归和分类。你使用闭式方程求解线性回归，也学习了梯度下降；还学习了如何在训练期间向成本函数添加各种惩罚项，从而对模型进行正则化。在此过程中，你学会了绘制和分析学习曲线，以及实现提前停止。最后，你了解了逻辑回归和 Softmax 回归的工作原理。我们已经打开了第一个机器学习黑盒！接下来几章还会打开更多黑盒，首先是支持向量机。

4.9　练习（Exercises）

1. 如果训练集包含数百万个特征，可以使用哪种线性回归训练算法？
2. 假设训练集中的特征尺度差异很大。哪些算法可能受到影响？会怎样影响？可以采取什么措施？
3. 训练逻辑回归模型时，梯度下降会陷入局部极小值吗？
4. 只要让所有梯度下降算法运行足够长时间，它们都会得到同一个模型吗？
5. 假设使用批量梯度下降，并在每个 epoch 绘制验证误差。如果发现验证误差持续上升，可能发生了什么？如何修复？
6. 当验证误差上升时，是否应该立即停止小批量梯度下降？
7. 在讨论过的梯度下降算法中，哪一种会最快到达最优解附近？哪一种最终会收敛？如何让其他算法也收敛？
8. 假设使用多项式回归，绘制学习曲线后发现训练误差与验证误差之间有很大间隙。发生了什么？有哪三种解决方法？
9. 假设使用岭回归，发现训练误差和验证误差几乎相等且都比较高。你会说模型存在高偏差还是高方差？应该增加正则化超参数 α 还是减小它？
10. 为什么要使用：

    a. 岭回归，而不是普通线性回归（不带正则化）？

    b. Lasso，而不是岭回归？

    c. 弹性网络，而不是 Lasso 回归？

11. 假设要把图片分类为室内/室外以及白天/夜晚。应该实现两个逻辑回归分类器，还是一个 Softmax 回归分类器？

12. 不使用 Scikit-Learn、只使用 NumPy，实现带提前停止的批量梯度下降 Softmax 回归。将它用于一个分类任务，例如 iris 数据集。

这些练习的答案位于本章 notebook 的末尾，网址为 https://homl.info/colab-p。

第 5 章　决策树

Decision Trees

决策树（decision tree）是用途广泛的机器学习算法，既可以执行分类和回归任务，甚至还能执行多输出任务。它们功能强大，能够拟合复杂数据集。例如，在第 2 章中，你曾在加利福尼亚住房数据集上训练 **DecisionTreeRegressor** 模型，并把数据拟合得非常完美（实际上是过拟合）。

决策树也是随机森林（random forest）的基本组成部分（见第 6 章），而随机森林是当今最强大的机器学习算法之一。

本章首先讨论如何训练、可视化决策树，以及如何使用决策树进行预测。然后，我们将介绍 Scikit-Learn 使用的 CART 训练算法，并探讨如何正则化决策树以及如何将它们用于回归任务。最后，我们会讨论决策树的一些局限性。

5.1　训练和可视化决策树（Training and Visualizing a Decision Tree）

为了理解决策树，我们先构建一棵决策树，看看它是如何进行预测的。下面的代码在 iris 数据集上训练一个 **DecisionTreeClassifier**（见第 4 章）：

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
iris = load_iris(as_frame=True)
X_iris = iris.data[["petal length (cm)", "petal width (cm)"]].values
y_iris = iris.target
tree_clf = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_clf.fit(X_iris, y_iris)
```

你可以先使用 **export_graphviz()** 函数输出一个名为 **iris_tree.dot** 的图定义文件，从而将训练好的决策树可视化：

```
from sklearn.tree import export_graphviz
export_graphviz(
    tree_clf,
    out_file="iris_tree.dot",
    feature_names=["petal length (cm)", "petal width (cm)"],
    class_names=iris.target_names,
    rounded=True,
    filled=True
)
```

然后，可以使用 `graphviz.Source.from_file()` 将该文件加载并显示在 Jupyter notebook 中：

```
from graphviz import Source
Source.from_file("iris_tree.dot")
```

Graphviz 是一个开源图形可视化软件包，还包含一个 `dot` 命令行工具，可以将 `.dot` 文件转换为多种格式，例如 PDF 或 PNG。

你的第一棵决策树如图 5-1 所示。

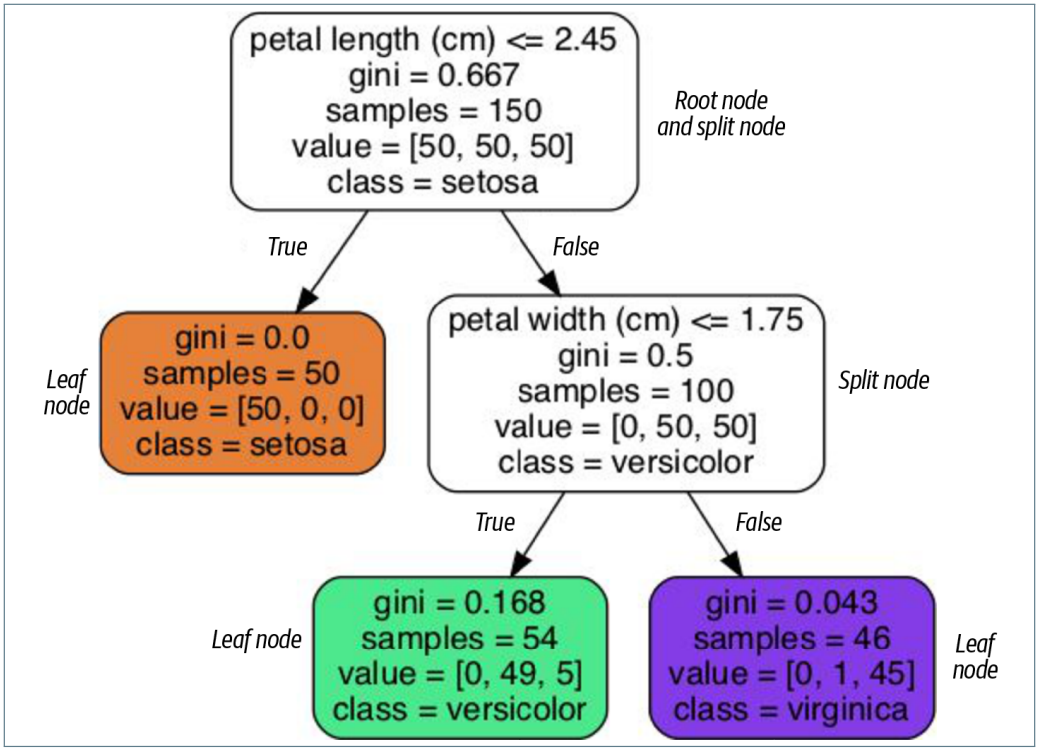

图 5-1: 鸢尾花决策树（Iris decision tree）

5.2　进行预测（Making Predictions）

我们来看看图 5-1 中的树如何进行预测。假设你发现了一朵鸢尾花，想根据它的花瓣对其分类。你从根节点（深度 0，即顶部节点）开始：该节点询问这朵花的花瓣长度是否小于 2.45 cm。如果是，就向下移动到根节点的左子节点（深度 1，左侧）。这里是一个叶节点（也就是说，它没有子节点），因此不会提出任何问题：只需查看该节点预测的类别，决策树就会预测你的花属于 Iris setosa（类别为 **`setosa`**）。

现在假设你发现了另一朵花，这次花瓣长度大于 2.45 cm。你同样从根节点开始，但这次向下移动到它的右子节点（深度 1，右侧）。这不是叶节点，而是一个分裂节点，因此它会提出另一个问题：花瓣宽度是否小于 1.75 cm？如果是，你的花很可能属于 Iris versicolor（深度 2，左侧）；否则，很可能属于 Iris virginica（深度 2，右侧）。就是这么简单。

> 注意：决策树的众多优点之一，是它们几乎不需要数据预处理。事实上，决策树完全不需要特征缩放或中心化。

节点的 **`samples`** 属性记录了该节点适用的训练实例数量。例如，有 100 个训练实例的花瓣长度大于 2.45 cm（深度 1，右侧）；其中有 54 个实例的花瓣宽度小于 1.75 cm（深度 2，左侧）。节点的 **`value`** 属性告诉你该节点适用的每个类别的训练实例数量：例如，右下角节点适用于 0 个 Iris setosa、1 个 Iris versicolor 和 45 个 Iris virginica。最后，节点的 **`gini`** 属性衡量其基尼不纯度（Gini impurity）：如果一个节点适用的所有训练实例都属于同一个类别，那么该节点是“纯”的（**`gini=0`**）。例如，由于深度 1 的左节点只适用于 Iris setosa 训练实例，因此其基尼不纯度为 0。相反，其他节点都适用于多个类别的实例，因此是“不纯的”。公式 5-1 展示了训练算法如何计算第 **`i`** 个节点的基尼不纯度。类别越多、混合越严重，不纯度就越高。例如，深度 2 的左节点的基尼不纯度为 **`1 - (0/54)^2 - (49/54)^2 - (5/54)^2 ≈ 0.168`**。

$$G_i = 1 - \sum_{k=1}^{n} p_{i,k}^2 \tag{5-1}$$

公式 5-1　基尼不纯度（Gini impurity）

在这个公式中：

- **`G_i`** 是第 **`i`** 个节点的基尼不纯度。
- **`p_i,k`** 是第 **`i`** 个节点中属于类别 **`k`** 的实例所占比例。

> 注意：Scikit-Learn 使用 CART 算法（稍后讨论），该算法只生成二叉树，也就是分裂节点始终恰好有两个子节点（即问题只有“是/否”两种答案）。不过，其他算法（例如 ID3）可以生成具有两个以上子节点的决策树。

图 5-2 展示了这棵决策树的决策边界。粗竖线表示根节点（深度 0）的决策边界：花瓣长度 = 2.45 cm。由于左侧区域是纯的（只有 Iris setosa），因此无法进一步分裂。不过，右侧区域是不纯的，所以深度 1 的右节点在花瓣宽度 = 1.75 cm 处将其分裂（用虚线表示）。由于 `max_depth` 被设置为 2，决策树就在这里停止。如果将 `max_depth` 设置为 3，那么两个深度 2 的节点都会各自增加一个决策边界（用两条竖直点线表示）。

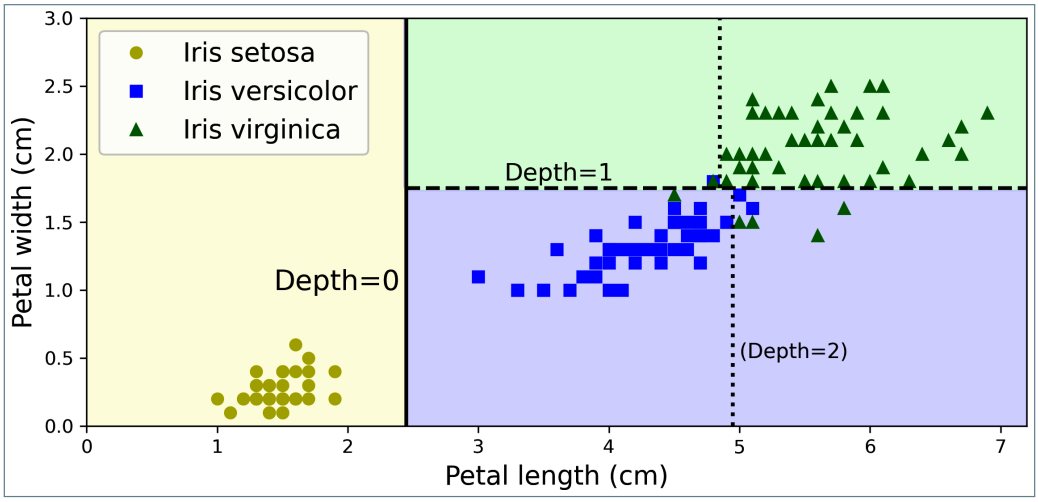

图 5-2: 决策树的决策边界（Decision tree decision boundaries）

> 提示：树的结构（包括图 5-1 中显示的全部信息）可以通过分类器的 `tree_` 属性获得。输入 `help(tree_clf.tree_)` 可查看详情，本章 notebook 中也有一个示例。

5.2.1　模型解释：白盒与黑盒（Model Interpretation: White Box versus Black Box）

决策树非常直观，容易解释其决策过程。这类模型通常称为白盒模型（white box model）。相比之下，正如你将看到的，随机森林和神经网络通常被视为黑盒模型

(black box model)。它们能够做出很好的预测，你也可以轻松检查它们为做出这些预测而执行的计算；不过，通常很难用简单的语言解释它们为什么会做出这些预测。

例如，如果神经网络判断某个人出现在一张图片中，你很难知道是什么因素促成了这个预测：模型识别出了这个人的眼睛、嘴巴、鼻子、鞋子，还是他坐着的沙发？反过来，决策树提供了简洁明了的分类规则，必要时甚至可以手工应用这些规则（例如进行花卉分类）。可解释机器学习（interpretable ML）领域致力于构建能够以人类理解的方式解释其决策的机器学习系统。这在许多领域都很重要：例如在医疗保健领域，让医生复核诊断；在金融领域，让分析师理解风险；在司法系统中，让人类做最终决定；在人力资源领域，确保决策不带偏见。

5.2.2　估计类别概率（Estimating Class Probabilities）

决策树还可以估计一个实例属于某个类别 `k` 的概率。首先，它遍历决策树，找到该实例对应的叶节点；然后返回属于类别 `k` 的实例在训练实例中所占的比例，这些训练实例也会到达同一个叶节点。

例如，假设你找到一朵花，花瓣长 5 cm、宽 1.5 cm。对应的叶节点是深度 2 的左节点，因此决策树会输出以下概率：Iris setosa 为 0%（0/54），Iris versicolor 为 90.7%（49/54），Iris virginica 为 9.3%（5/54）。如果要求它预测类别，它会输出 Iris versicolor（类别 1），因为该类别的概率最高。我们来验证一下：

```
>>> tree_clf.predict_proba([[5, 1.5]]).round(3)
array([[0.   , 0.907, 0.093]])
>>> tree_clf.predict([[5, 1.5]])
array([1])
```

完美！注意，在图 5-2 右下角的矩形区域内，无论实例位于何处，估计概率都完全相同。例如，即使花瓣长 6 cm、宽 1.5 cm，结果也一样（尽管在这种情况下，显然最可能属于 Iris virginica）。

5.2.3　CART 训练算法（The CART Training Algorithm）

Scikit-Learn 使用分类与回归树（Classification and Regression Tree，CART）算法训练决策树（也称“生长”决策树）。该算法首先使用一个特征 `k` 和一个阈值 `t_k` 将训练集分为两个子集（例如“花瓣长度 ⩽ 2.45 cm”）。它如何选择 `k` 和 `t_k`？算法会搜索能够生成最纯子集的 `(k, t_k)` 组合，同时按照子集大小加权。公式 5-2 给出了算法试图最小化的成本函数。

$$J(k, t_k) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}} \tag{5-2}$$

公式 5-2　CART 分类成本函数（CART cost function for classification）

CART 算法成功将训练集分为两个子集后，会使用相同的逻辑继续分裂子集，然后分裂子集的子集，如此递归进行。当达到最大深度（由 `max_depth` 超参数定义），或者找不到能够降低不纯度的分裂方式时，算法停止递归。其他一些超参数（稍后介绍）也会控制额外的停止条件，例如 `min_samples_split`、`min_samples_leaf`、`max_leaf_nodes` 等。

> 警告：如你所见，CART 算法是一种贪心算法：它在顶层贪心地搜索最优分裂，然后在每个后续层级重复这一过程。它不会检查该分裂是否会在更深的层级带来可能的最低不纯度。贪心算法通常会产生一个相当好的解决方案，但不能保证最优。

遗憾的是，寻找最优决策树是一个已知的 NP 完全问题。[1] 它需要 `O(exp(m))` 的时间，[2] 因此，即使训练集很小，这个问题也难以处理。这就是为什么训练决策树时，我们只能满足于“相当好”的解决方案。

5.2.4　计算复杂度（Computational Complexity）

进行预测需要从根节点遍历决策树，直到某个叶节点。决策树通常大致平衡，因此遍历一棵决策树大约需要经过 `O(log_2(m))` 个节点，其中 `m` 是训练实例数量，`log_2(m)` 是以 2 为底的对数，等于 `log(m) / log(2)`。由于每个节点只需检查一个特征的值，因此总体预测复杂度为 `O(log_2(m))`，与特征数量无关。所以，即使训练集很大，预测也非常快。

默认情况下，训练算法会在每个节点上比较所有样本的所有特征，因此训练复杂度为 `O(n × m log_2(m))`。

[1] P 是可以在多项式时间内解决的问题集合（也就是说，运行时间是数据集规模的多项式）。NP 是其解可以在多项式时间内验证的问题集合。NP-hard 问题可以在多项式时间内归约为某个已知的 NP-hard 问题。NP-complete 问题同时属于 NP 和 NP-hard。一个重要的开放数学问题是 P 是否等于 NP。如果 P ≠ NP（这似乎很可能），那么除非未来某天在量子计算机上实现，否则任何 NP-complete 问题都不会存在多项式算法。

[2] 这里的大 O 记号表示，随着 m（即训练实例数量）增大，计算时间与 m 的指数成正比（严格来说，它是一个上界，但我们会尽量把它做得小一些）。这告诉我们计算速度如何随着 m 增长；`O(exp(m))` 是非常快的。

可以使用 `max_depth` 超参数设置树的最大深度，也可以设置每个节点要考虑的最大特征数（特征会被随机选择）。这样做可以显著加快训练速度，也可以降低过拟合风险（但和往常一样，做得过头会导致欠拟合）。

5.2.5　基尼不纯度还是熵？（Gini Impurity or Entropy?）

默认情况下，`DecisionTreeClassifier` 类使用基尼不纯度指标，但你也可以将 `criterion` 超参数设置为`"entropy"`，从而选择熵不纯度指标。

熵的概念起源于热力学，用来衡量分子无序程度：当分子静止且排列有序时，熵趋近于零。后来，熵扩展到了许多领域，包括香农信息论；在信息论中，熵衡量消息的平均信息量，正如我们在第 4 章中看到的那样。当所有消息都相同时，熵为零。在机器学习中，熵经常被用作不纯度指标：当一个集合只包含一个类别的实例时，该集合的熵为零。公式 5-3 给出了第 `i` 个节点的熵定义。例如，图 5-1 中深度 2 的左节点的熵为 `-(49/54) log_2(49/54) - (5/54) log_2(5/54) ≈ 0.445`。

$$H_i = -\sum_{\substack{k=1 \\ p_{i,k} \neq 0}}^{n} p_{i,k} \log_2\left(p_{i,k}\right) \tag{5-3}$$

公式 5-3　熵（Entropy）

那么，应该使用基尼不纯度还是熵？事实上，大多数时候差别并不大：它们会得到相似的决策树。基尼不纯度的计算略快一些，因此是一个很好的默认选择。不过，二者结果不同时，基尼不纯度倾向于把最常见的类别隔离到自己的分支中，而熵倾向于生成稍微更平衡的树。[3]

5.2.6　正则化超参数（Regularization Hyperparameters）

与线性模型假设数据是线性的不同，决策树对训练数据几乎不作任何假设。如果不加约束，树的结构会适应训练数据，极其紧密地拟合它——事实上，很可能发生过拟合。这种模型通常称为非参数模型（nonparametric model），并不是因为它没有参数（它通常有很多参数），而是因为参数数量不会在训练之前确定，所以模型结构可以自由地贴合数据。

相比之下，线性模型等参数模型（parametric model）具有预先确定的参数数量，因此自由度受到限制，过拟合风险降低（但欠拟合风险增加）。

[3]参见 Sebastian Raschka 对此主题的有趣分析，以了解更多细节。

为了避免过拟合训练数据，你需要在训练期间限制决策树的自由度。正如你现在已经知道的，这称为正则化（regularization）。正则化超参数取决于所使用的算法，但通常至少可以限制决策树的最大深度。在 Scikit-Learn 中，这是由 `max_depth` 超参数控制的。默认值是 `None`，表示不受限制。减小 `max_depth` 会对模型进行正则化，从而降低过拟合风险。

`DecisionTreeClassifier` 类还有一些可以类似地限制决策树形状的参数：

- `max_features`：每个节点进行分裂时评估的最大特征数。
- `max_leaf_nodes`：最大叶节点数。
- `min_samples_split`：一个节点在可以分裂之前必须拥有的最少样本数。
- `min_samples_leaf`：创建一个叶节点所需的最少样本数。
- `min_weight_fraction_leaf`：与 `min_samples_leaf` 相同，但表示为全部加权实例的比例。
- `min_impurity_decrease`：只有当该分裂至少带来这么多不纯度降低时，才分裂节点。
- `ccp_alpha`：控制最小代价复杂度剪枝（minimal cost-complexity pruning，MCCP），也就是剪除相对于叶节点数量而言没有充分降低不纯度的子树；`ccp_alpha` 越大，剪枝越多，树越小（默认值为 0，表示不剪枝）。

为了限制模型复杂度并对模型进行正则化，可以增大 `min_*` 超参数或 `ccp_alpha`，也可以减小 `max_*` 超参数。调节 `max_depth` 通常是一个很好的默认选择：它能有效正则化，同时保持树较小、易于解释。设置 `min_samples_leaf` 也很有用，尤其对于小型数据集。处理高维数据集时，`max_features` 也很有帮助。

> 注意：其他算法会先不加限制地训练决策树，然后剪除不必要的节点。如果某个节点的所有子节点都是叶节点，并且它带来的纯度提升在统计上不显著，那么这个节点就被认为是不必要的。可以使用 χ^2 检验（卡方检验）等标准统计检验，估计这种提升完全由偶然因素导致的概率，这称为零假设（null hypothesis）。如果这个概率（称为 p 值）高于给定阈值（通常为 5%，由超参数控制），则认为该节点不必要，并删除它的子节点。剪枝会持续进行，直到所有不必要的节点都被剪除。

我们来使用 moons 数据集测试正则化：这是一个二分类玩具数据集，数据点呈两个相互交错的新月形（见图 5-3）。你可以使用 `make_moons()` 函数生成该数据集。

5.3　正则化决策树（Regularized Decision Trees）

我们将训练一棵不进行正则化的决策树，以及另一棵设置 `min_samples_leaf=5` 的决策树。下面是代码；图 5-3 展示了两棵树的决策边界：

```
from sklearn.datasets import make_moons
X_moons, y_moons = make_moons(n_samples=150, noise=0.2, random_state=42)
tree_clf1 = DecisionTreeClassifier(random_state=42)
tree_clf2 = DecisionTreeClassifier(min_samples_leaf=5, random_state=42)
tree_clf1.fit(X_moons, y_moons)
tree_clf2.fit(X_moons, y_moons)
```

左侧的非正则化模型显然过拟合了，而右侧的正则化模型很可能具有更好的泛化能力。我们可以在使用不同随机种子生成的测试集上评估两棵树来验证这一点：

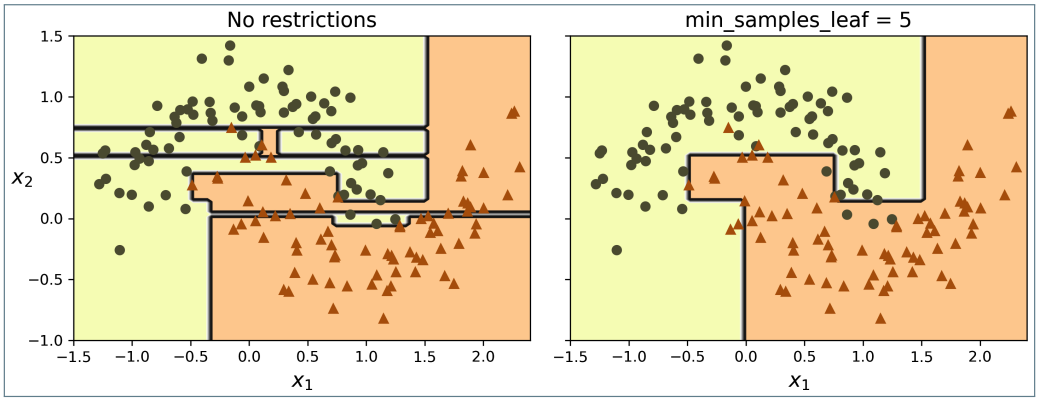

图 5-3: 未正则化树与正则化树的决策边界（Decision boundaries of an unregularized tree and a regularized tree）

```
>>> X_moons_test, y_moons_test = make_moons(n_samples=1000, noise=0.2,
...                                         random_state=43)
...
>>> tree_clf1.score(X_moons_test, y_moons_test)
0.898
>>> tree_clf2.score(X_moons_test, y_moons_test)
0.92
```

确实，第二棵树在测试集上的准确率更高。

5.4　回归（Regression）

决策树同样能够执行回归任务。线性回归只适合线性数据，而决策树可以拟合各种复杂数据集。我们使用 Scikit-Learn 的 **DecisionTreeRegressor** 类，在带噪声的二次数据集上训练一棵 **max_depth=2** 的回归树：

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor
rng = np.random.default_rng(seed=42)
X_quad = rng.random((200, 1)) - 0.5  # a single random input feature
y_quad = X_quad ** 2 + 0.025 * rng.standard_normal((200, 1))
tree_reg = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg.fit(X_quad, y_quad)
```

生成的树如图 5-4 所示。

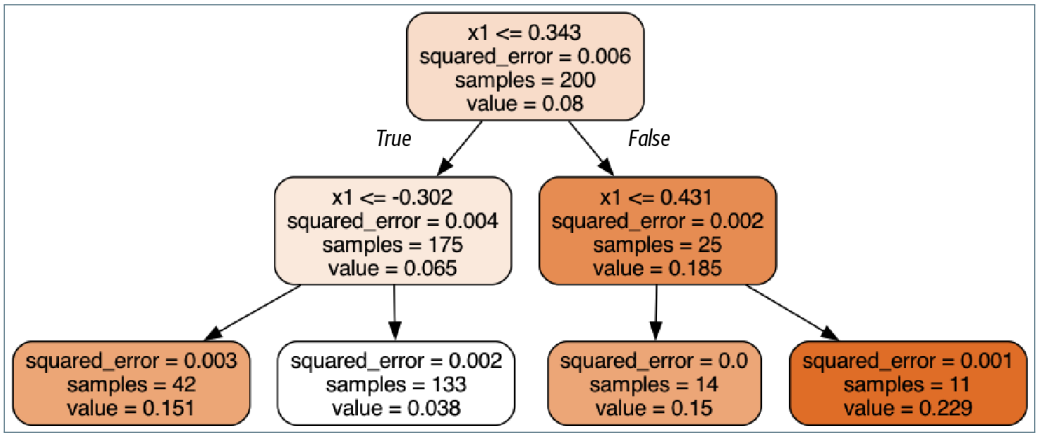

图 5-4: 用于回归的决策树（A decision tree for regression）

图 5-4 中的树与前面构建的分类树非常相似。主要区别在于，每个节点不再预测一个类别，而是预测一个数值。例如，假设你想对一个 **x_1 = 0.2** 的新实例进行预测。根节点询问 **x_1 ≤ 0.343** 是否成立。由于成立，算法移到左子节点；该节点询问 **x_1 ≤ -0.302** 是否成立。由于不成立，算法移到右子节点。这是一个叶节点，预测值为 **value=0.038**。这个预测值是与该叶节点关联的 133 个训练实例的目标值平均数，在这 133 个实例上得到的均方误差为 0.002。

该模型的预测结果显示在图 5-5 左侧。如果将 **max_depth** 设置为 3，就会得到右侧所示的预测结果。注意，每个区域的预测值始终是该区域实例目标值的平均值。算法会以一种方式分裂每个区域，使大多数训练实例尽可能接近预测值。

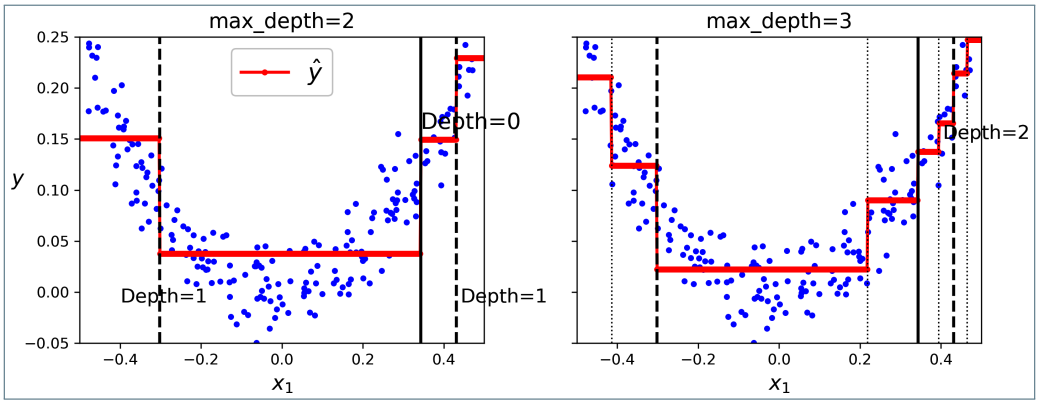

图 5-5: 两棵决策树回归模型的预测结果（Predictions of two decision tree regression models）

正如前面所述，CART 算法会尝试将训练集分裂为两个子集；但对于回归任务，它不是试图降低不纯度，而是试图降低 MSE。公式 5-4 展示了算法试图最小化的成本函数。

$$
\begin{aligned}
J(k, t_k) &= \frac{m_{\text{left}}}{m} \text{MSE}_{\text{left}} + \frac{m_{\text{right}}}{m} \text{MSE}_{\text{right}} \\
\text{MSE}_{\text{node}} &= \frac{1}{m_{\text{node}}} \sum_{i \in \text{node}} \left(\hat{y}_{\text{node}} - y^{(i)} \right)^2 \\
\hat{y}_{\text{node}} &= \frac{\sum_{i \in \text{node}} y^{(i)}}{m_{\text{node}}}
\end{aligned}
\tag{5-4}
$$

公式 5-4　CART 回归成本函数（CART cost function for regression）

与分类任务一样，决策树在执行回归任务时也容易过拟合。如果不进行任何正则化（即使用默认超参数），就会得到图 5-6 左侧所示的预测结果。这些预测显然严重过拟合了训练集。只需将 `min_samples_leaf=10`，就能得到合理得多的模型，如图 5-6 右侧所示。

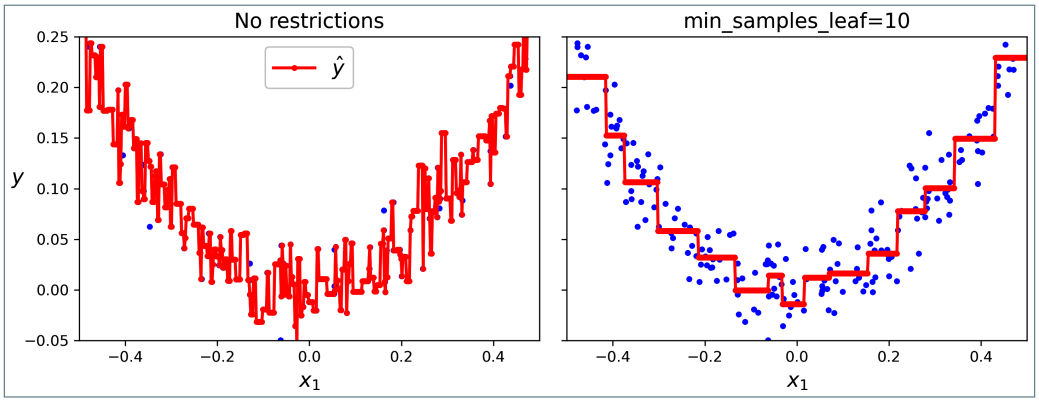

图 5-6: 未正则化回归树与正则化回归树的预测结果（Predictions of an unregularized regression tree and a regularized tree）

5.5　对坐标轴方向的敏感性（Sensitivity to Axis Orientation）

现在你应该已经相信，决策树有许多优点：相对容易理解和解释，使用简单，用途广泛，而且功能强大。不过，它们也有一些局限性。

首先，正如你可能已经注意到的，决策树喜欢正交的决策边界（所有分裂都垂直于某个坐标轴），因此对数据的方向很敏感。例如，图 5-7 展示了一个简单的线性可分数据集：左侧的决策树可以轻松将其分裂，而右侧的数据集旋转 45° 后，决策边界看起来就不必要地复杂。尽管两棵决策树都完美拟合了训练集，但右侧的模型很可能无法良好泛化。

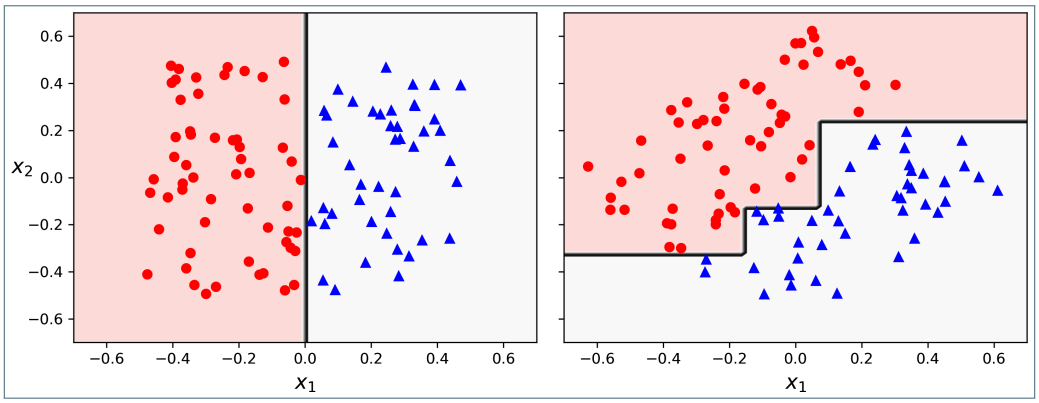

图 5-7: 对训练集旋转的敏感性（Sensitivity to training set rotation）

限制这一问题的一种方法是缩放数据，然后应用主成分分析（principal component analysis，PCA）变换。我们将在第 7 章详细讨论 PCA；现在只需知道，它会以降低特征之间相关性的方式旋转数据，而这通常（但并不总是）会让决策树更容易处理数据。

我们来创建一个小型流水线，使用 PCA 缩放并旋转数据，然后在处理后的数据上训练 `DecisionTreeClassifier`。图 5-8 展示了该决策树的决策边界：可以看到，旋转之后，只使用一个特征 `z_1`（它是原始花瓣长度和宽度的线性函数）就能很好地拟合数据集。下面是代码：

```
from sklearn.decomposition import PCA
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
pca_pipeline = make_pipeline(StandardScaler(), PCA())
X_iris_rotated = pca_pipeline.fit_transform(X_iris)
tree_clf_pca = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_clf_pca.fit(X_iris_rotated, y_iris)
```

> 提示：`DecisionTreeClassifier` 和 `DecisionTreeRegressor` 类都能原生支持缺失值，不需要插补器。

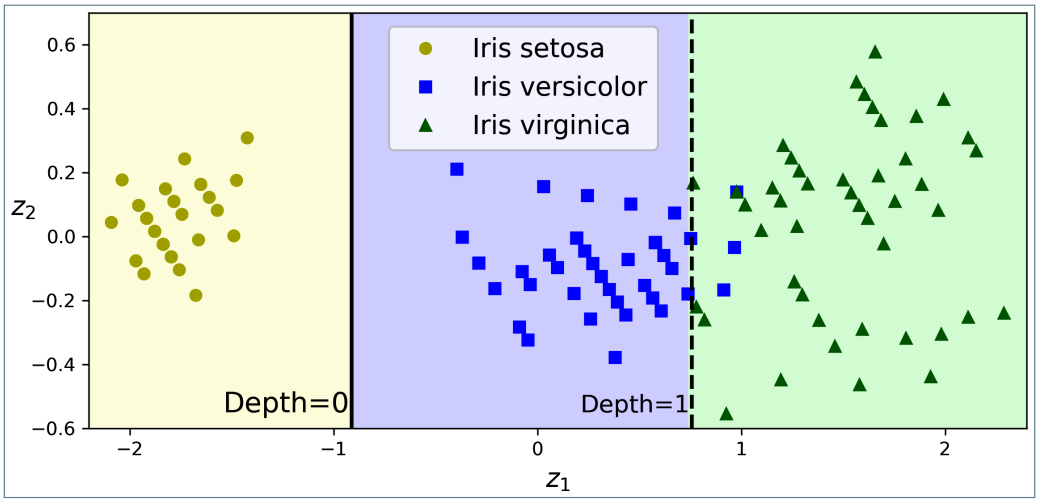

图 5-8：缩放并经 PCA 旋转后的鸢尾花数据集决策边界（A tree's decision boundaries on the scaled and PCA-rotated iris dataset）

5.6　决策树具有高方差（Decision Trees Have a High Variance）

更一般地说，决策树的主要问题是方差很高：超参数或数据稍有变化，就可能产生完全不同的模型。事实上，由于 Scikit-Learn 使用的训练算法具有随机性——它会在每个节点随机选择要评估的特征集合——即使在完全相同的数据上重新训练同一棵决策树，也可能得到非常不同的模型，例如图 5-9 所示的模型（除非你设置了 `random_state` 超参数）。可以看到，它与之前的决策树（图 5-2）看起来非常不同。

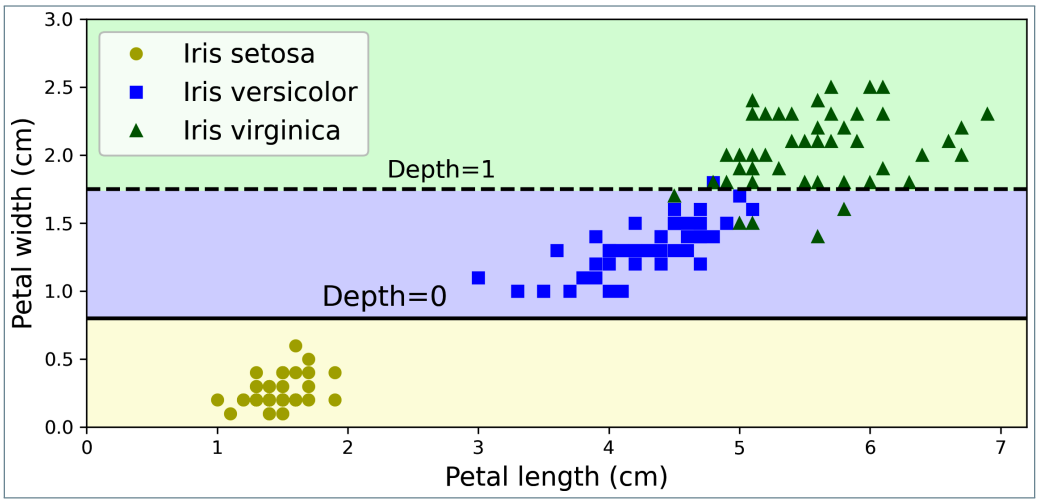

图 5-9: 重新训练同一模型可能产生非常不同的树（Retraining the same model may produce a very different tree）

幸运的是，通过对许多棵树的预测取平均，可以显著降低方差。这样的树集成称为随机森林（random forest），是当今最强大的模型类型之一，下一章将对此进行介绍。

5.7　练习（Exercises）

1. 在一个包含一百万个实例的训练集上训练一棵不受限制的决策树时，其近似深度是多少？
2. 一个节点的基尼不纯度通常比其父节点低还是高？它通常是更低/更高，还是始终更低/更高？
3. 如果一棵决策树正在过拟合训练集，尝试减小 `max_depth` 是好主意吗？

4. 如果一棵决策树正在欠拟合训练集，尝试缩放输入特征是好主意吗？

5. 如果在包含一百万个实例的训练集上训练一棵决策树需要 1 小时，那么在包含一千万个实例的训练集上训练另一棵决策树大约需要多长时间？提示：考虑 CART 算法的计算复杂度。

6. 如果在给定训练集上训练一棵决策树需要 1 小时，那么将特征数量加倍后大约需要多长时间？

7. 按照以下步骤，为 moons 数据集训练并微调一棵决策树：

- 使用 `make_moons(n_samples=10000, noise=0.4)` 生成 moons 数据集。
- 使用 `train_test_split()` 将数据集分为训练集和测试集。
- 使用交叉验证网格搜索（借助 `GridSearchCV` 类）寻找 `DecisionTreeClassifier` 的较好超参数值。提示：尝试 `max_leaf_nodes` 的各种值。
- 使用这些超参数在完整训练集上训练模型，并在测试集上测量模型性能。准确率应该约为 85% 至 87%。

8. 按照以下步骤构建一个森林：

- 延续上一个练习，随机生成 1,000 个训练集子集，每个子集包含随机选取的 100 个实例。提示：可以使用 Scikit-Learn 的 `ShuffleSplit` 类。
- 使用上一个练习找到的最佳超参数值，在每个子集上训练一棵决策树。在测试集上评估这 1,000 棵决策树。由于它们在更小的数据集上训练，这些决策树的表现很可能比第一棵决策树差，准确率只有约 80%。
- 现在见证奇迹。针对每个测试集实例，生成 1,000 棵决策树的预测结果，只保留出现频率最高的预测（可以使用 SciPy 的 `mode()` 函数）。这种方法会对测试集产生多数投票预测。
- 在测试集上评估这些预测结果：准确率应该略高于第一棵模型（高出约 0.5% 至 1.5%）。恭喜你，你已经训练出一个随机森林分类器！

解决方案位于本章 notebook 的末尾，网址为 https://homl.info/colab-p。

第 6 章　集成学习与随机森林

Ensemble Learning and Random Forests

假设你向成千上万名随机选取的人提出一个复杂问题，然后汇总他们的答案。在很多情况下，你会发现这个汇总答案比专家的答案更好。这种现象称为群众智慧（wisdom of the crowd）。类似地，如果汇总一组预测器（例如分类器或回归器）的预测结果，通常也能得到比其中最佳单个预测器更好的预测结果。一组预测器称为集成（ensemble）；因此，这种技术称为集成学习（ensemble learning），相应的算法称为集成方法（ensemble method）。

例如，你可以训练一组决策树分类器，每棵树都使用训练集的一个不同随机子集。然后收集所有单棵树的预测结果，把获得最多票数的类别作为集成的预测结果（见第 5 章末尾的练习）。这样的决策树集成称为随机森林（random forest）。尽管随机森林非常简单，但它是当今最强大的机器学习算法之一。

正如第 2 章所讨论的，你通常会在项目接近尾声时使用集成方法：先构建几个优秀的预测器，再把它们组合成一个更好的预测器。事实上，机器学习竞赛中的获胜方案通常会用到多个集成方法，最著名的例子是 Netflix Prize 竞赛。集成学习也有缺点：无论训练还是推理，集成学习都需要比单个模型多得多的计算资源；部署和管理可能更复杂；预测结果也更难解释。不过，集成学习的优点通常超过缺点。

本章将介绍最常见的集成方法，包括投票分类器、Bagging 与 Pasting 集成、随机森林、提升法（boosting）以及 Stacking 集成。

6.1　投票分类器（Voting Classifiers）

假设你训练了几个分类器，每个分类器的准确率大约为 80%。这些分类器可以包括逻辑回归分类器、SVM 分类器、随机森林分类器、k 最近邻分类器等（见图 6-1）。

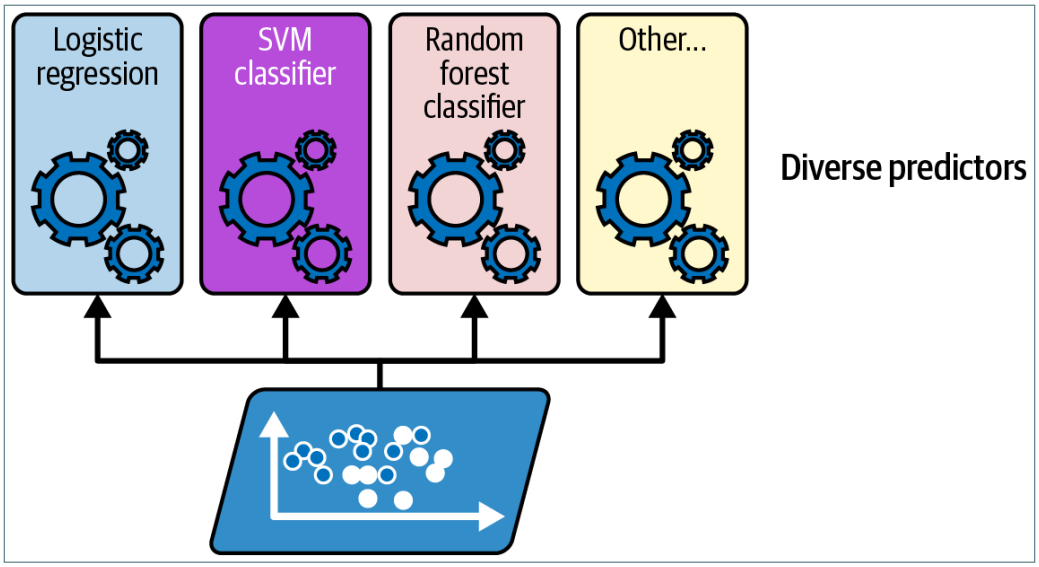

图 6-1: 训练多个多样化分类器（Training diverse classifiers）

一个非常简单的办法是汇总每个分类器的预测结果，把获得最多票数的类别作为集成的预测结果。这种多数投票分类器称为硬投票分类器（hard voting classifier），如图 6-2 所示。

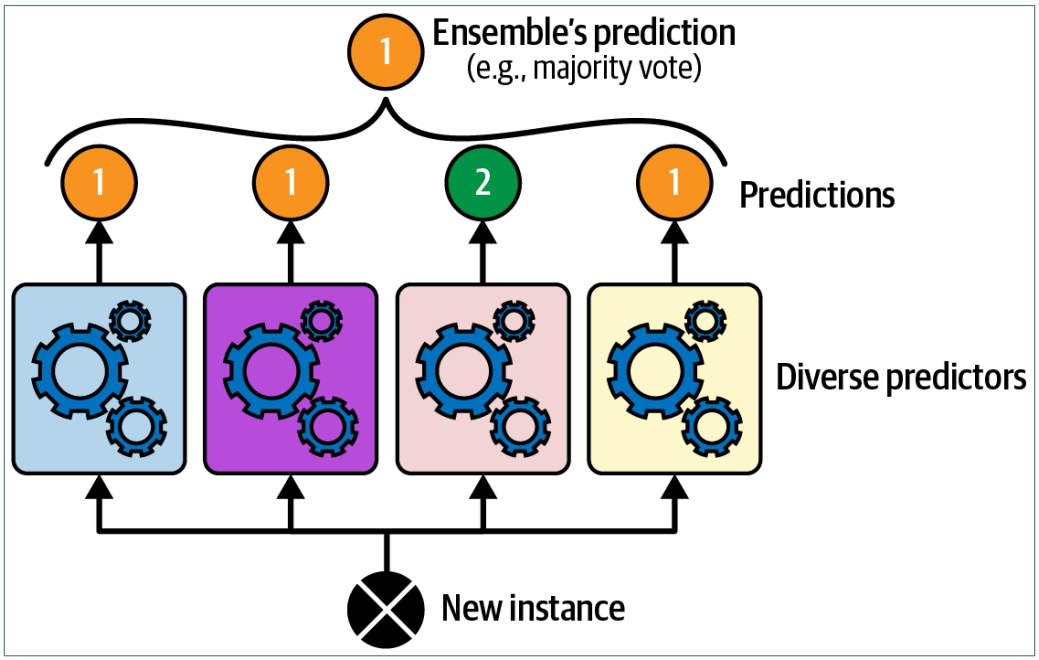

图 6-2: 硬投票分类器的预测结果（Hard voting classifier predictions）

有些令人意外的是，这种投票分类器的准确率往往高于集成中最好的单个分类器。事实上，即使每个分类器都是弱学习器（weak learner），也就是只比随机猜测略好一点，只要集成中有足够多的弱学习器，并且它们足够多样化，集成仍然可以成为强学习器（strong learner）。这里的“多样化”是指它们关注数据的不同方面，并犯下不同类型的错误。

为什么会这样？下面的类比可以帮助我们理解。假设一枚略有偏差的硬币正面朝上的概率为 51%，反面朝上的概率为 49%。如果抛掷它 1,000 次，通常会得到大约 510 次正面和 490 次反面，因此正面占多数。计算可知，抛掷 1,000 次后正面占多数的概率接近 75%。抛掷次数越多，这个概率越高，例如抛掷 10,000 次时，概率会超过 97%。这是大数定律（law of large numbers）的结果：不断抛掷硬币时，正面比例会越来越接近正面朝上的概率 51%。图 6-3 展示了 10 组有偏硬币的抛掷结果。随着抛掷次数增加，正面比例逐渐接近 51%；最终，10 组结果都会稳定地高于 50%。

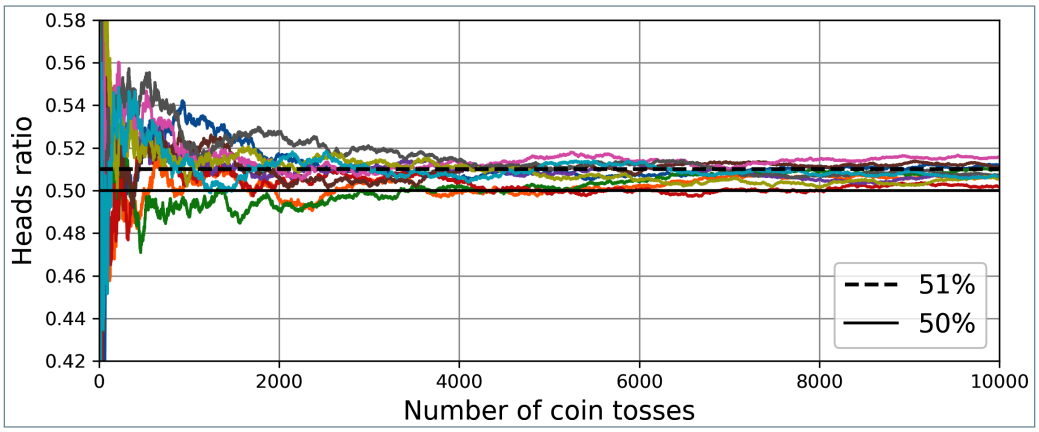

图 6-3: 大数定律（The law of large numbers）

同样地，假设你构建了一个包含 1,000 个分类器的集成，每个分类器单独预测正确的概率只有 51%，仅仅略好于随机猜测。如果采用多数投票，准确率最高可能达到 75%！不过，这只有在所有分类器完全独立、错误互不相关时才成立，而训练于同一数据集的分类器显然不会满足这一条件。它们很可能犯下相同类型的错误，于是错误类别也会得到许多多数票，降低集成的准确率。

> 提示：当各预测器尽可能彼此独立时，集成方法的效果最好。获得多样化分类器的一种方法是使用非常不同的算法训练它们，这会增加它们犯下不同类型错误的可能性，从而提高集成准确率。你也可以调整模型的超参数来得到多样化的模型，或者在不同的数据子集上训练模型，后文会介绍这种方法。

Scikit-Learn 提供了易于使用的 **VotingClassifier** 类：只需向它传入名称和预测器组成的列表，然后就可以像使用普通分类器一样使用它。下面在第 5 章介绍过的 moons 数据集上尝试投票分类器。我们先加载并划分 moons 数据集，再创建并训练一个由三个多样化分类器组成的投票分类器：

```
from sklearn.datasets import make_moons
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
X, y = make_moons(n_samples=500, noise=0.30, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
voting_clf = VotingClassifier(
    estimators=[
        ('lr', LogisticRegression(random_state=42)),
        ('rf', RandomForestClassifier(random_state=42)),
        ('svc', SVC(random_state=42))
    ]
)
voting_clf.fit(X_train, y_train)
```

调用 **VotingClassifier** 的 **fit()** 方法时，它会复制每个估计器并训练这些副本。原始估计器可以通过 **estimators** 属性访问，训练后的副本可以通过 **estimators_** 属性访问。如果你更喜欢使用字典而不是列表，也可以使用 **named_estimators** 或 **named_estimators_**。先看看每个训练后分类器在测试集上的准确率：

```
>>> for name, clf in voting_clf.named_estimators_.items():
...     print(name, "=", clf.score(X_test, y_test))
...
lr = 0.864
rf = 0.896
svc = 0.896
```

调用投票分类器的 **predict()** 方法时，它会执行硬投票。例如，测试集的第一个实例被预测为类别 1，因为三个分类器中有两个预测为该类别：

```
>>> voting_clf.predict(X_test[:1])
array([1])
>>> [clf.predict(X_test[:1]) for clf in voting_clf.estimators_]
[array([1]), array([1]), array([0])]
```

再看投票分类器在测试集上的性能：

```
>>> voting_clf.score(X_test, y_test)
0.912
```

可以看到，投票分类器的表现超过了所有单个分类器。

如果所有分类器都能估计类别概率，也就是都具有 `predict_proba()` 方法，那么通常应该让 Scikit-Learn 对所有分类器给出的类别概率取平均，并预测平均概率最高的类别。这称为软投票（soft voting）。与硬投票相比，软投票会给置信度较高的投票更大的权重，因此通常能取得更好的性能。只需要将投票分类器的 `voting` 超参数设为`"soft"`，并确保所有分类器都能估计类别概率即可。默认情况下，`SVC` 类不满足这一条件，因此需要将它的 `probability` 超参数设为 `True`。这会让 `SVC` 使用交叉验证来估计类别概率，训练会变慢，同时会增加一个 `predict_proba()` 方法。

```
>>> voting_clf.voting = "soft"
>>> voting_clf.named_estimators["svc"].probability = True
>>> voting_clf.fit(X_train, y_train)
>>> voting_clf.score(X_test, y_test)
0.92
```

仅仅使用软投票，准确率就达到了 92%，效果不错!

> 提示：当估计出的概率经过良好校准时，软投票效果最好。如果概率没有校准，可以使用 `sklearn.calibration.CalibratedClassifierCV` 进行校准（见第 3 章）。

6.2　Bagging 与 Pasting（Bagging and Pasting）

获得一组多样化分类器的一种方法是使用不同的训练算法。另一种方法是使用同一个训练算法，但在训练每个预测器时使用训练集的不同随机子集。带放回抽样时，这种方法称为 Bagging[1]，是 bootstrap aggregating 的缩写[2][3]；不带放回抽样时，称为 Pasting[4]。

[1]从一副牌中随机抽取一张牌，记下它，再把牌放回去后抽取下一张；因此同一张牌可能被抽到多次。

[2]Leo Breiman，“Bagging Predictors”，Machine Learning 24，第 2 期（1996）：123-140。

[3]在统计学中，带放回重采样称为 bootstrap。

[4]Leo Breiman，“Pasting Small Votes for Classification in Large Databases and On-Line”，Machine Learning 36，第 1-2 期（1999）：85-103。

换句话说，Bagging 和 Pasting 都允许一个训练实例在多个预测器之间被重复抽样，但只有 Bagging 允许同一个训练实例在同一个预测器中被重复抽到。图 6-4 展示了这种抽样和训练过程。

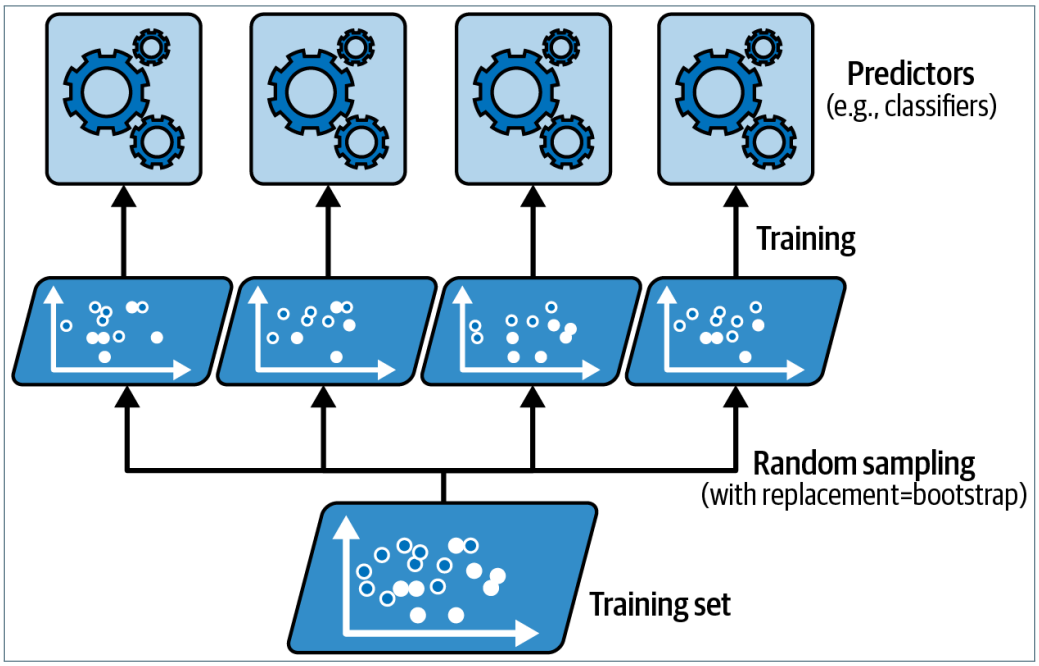

图 6-4: Bagging 与 Pasting 在训练集的不同随机样本上训练多个预测器（Bagging and pasting involve training several predictors on different random samples of the training set）

所有预测器训练完成后，集成可以简单地汇总所有预测器的预测结果，对新实例进行预测。对于分类任务，汇总函数通常是统计众数，也就是出现频率最高的预测结果，与硬投票分类器类似；对于回归任务，通常是平均值。每个单独预测器的偏差会高于在原始训练集上训练的情况，但汇总会同时降低偏差和方差[5]。

为了直观理解这一点，假设你训练了两个预测房价的回归器。第一个平均低估房价 40,000 美元，第二个平均高估房价 50,000 美元。假设两个回归器完全独立，预测结果服从正态分布，那么两个预测结果的平均值平均只会高估 `(-40,000 + 50,000) / 2 = 5,000` 美元，偏差大幅降低。类似地，如果两个预测器的标准差都是 10,000 美元，也就是方差为 100,000,000，那么平均预测的方差为 `(10,000² + 10,000²) / 2² = 50,000,000`，标准差为 7,071 美元，方差减半！

[5]偏差和方差在第 4 章介绍过。高偏差意味着平均预测离目标很远，高方差意味着预测结果分散程度很高；我们希望两者都低。

在实际中，与在原始训练集上训练的单个预测器相比，集成通常具有相近的偏差，但方差更低。因此，Bagging 和 Pasting 最适合高方差、低偏差的模型，例如决策树集成，而不适合线性回归器集成。

> 提示：当数据集含噪声或模型容易过拟合时（例如深度决策树），优先选择 Bagging。否则可以选择 Pasting，因为它避免了训练过程中的重复抽样，计算效率略高。

如图 6-4 所示，所有预测器都可以并行训练，可以使用不同的 CPU 核心，甚至不同的服务器；预测过程也可以并行。这是 Bagging 和 Pasting 很受欢迎的原因之一：它们具有很好的扩展性。

6.2.1　Scikit-Learn 中的 Bagging 与 Pasting（Bagging and Pasting in Scikit-Learn）

Scikit-Learn 为 Bagging 和 Pasting 提供了简单的 API：**`BaggingClassifier`** 类（回归任务对应 **`BaggingRegressor`**）。下面的代码训练一个包含 500 棵决策树分类器的集成[6]，每棵树都从训练集带放回地随机抽取 100 个训练实例。这里是 Bagging；如果想使用 Pasting，只需将 **`bootstrap=False`**。**`n_jobs`** 参数告诉 Scikit-Learn 使用多少个 CPU 核心，**`-1`** 表示使用所有可用核心：

[6] **`max_samples`** 也可以设为 0.0 到 1.0 之间的浮点数，此时最大抽样实例数等于训练集大小乘以 **`max_samples`**。

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
bag_clf = BaggingClassifier(DecisionTreeClassifier(), n_estimators=500,
                            max_samples=100, n_jobs=-1, random_state=42)
bag_clf.fit(X_train, y_train)
```

> 注意：如果基础分类器能够估计类别概率，也就是具有 `predict_proba()` 方法，那么 `BaggingClassifier` 会自动执行软投票，而不是硬投票。决策树分类器就满足这一条件。

图 6-5 比较了单棵决策树与由前面代码训练的 500 棵树组成的 Bagging 集成在 moons 数据集上的决策边界。可以看到，集成的预测结果很可能比单棵决策树具有更好的泛化能力：集成具有相近的偏差，但方差更小，在训练集上犯下的错误数量大致相同，而决策边界不那么不规则。

Bagging 会给每个预测器带来更多子集多样性，因此相比 Pasting，偏差会略高；但这种额外多样性也会让预测器之间的相关性降低，从而降低集成的方差。总体而言，Bagging 往往会得到更好的模型，这也是通常优先选择它的原因。不过，如果有足够的时间和 CPU 资源，可以使用交叉验证评估 Bagging 和 Pasting，再选择表现更好的方法。

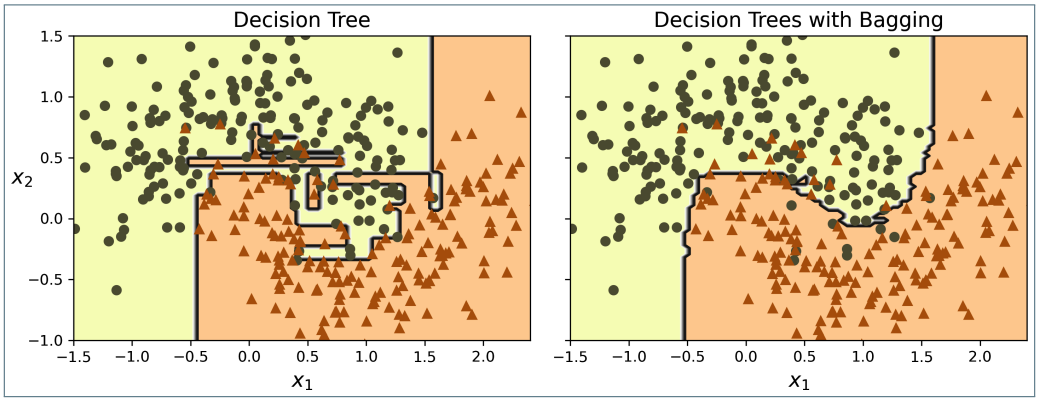

图 6-5: 单棵决策树（左）与 500 棵树组成的 Bagging 集成（右）（A single decision tree versus a bagging ensemble of 500 trees）

6.2.2　袋外评估（Out-of-Bag Evaluation）

在 Bagging 中，对于任意一个预测器，有些训练实例可能被抽到多次，而另一些实例可能一次也没有被抽到。默认情况下，**BaggingClassifier** 会带放回地抽取 **m** 个训练实例（**bootstrap=True**），其中 **m** 是训练集的大小。数学上可以证明，每个预测器平均只会抽到约 63% 的训练实例。其余未被抽到的 37% 训练实例称为袋外（out-of-bag，OOB）实例[7]。注意，对于不同预测器，这 37% 实例并不相同。

可以利用 OOB 实例评估 Bagging 集成，而不需要单独的验证集。只要预测器足够多，训练集中的每个实例很可能都是若干个预测器的 OOB 实例，因此可以用这些预测器为该实例生成公平的集成预测，然后计算集成的预测准确率或其他指标。

在 Scikit-Learn 中，创建 **BaggingClassifier** 时将 **oob_score=True**，即可在训练后自动进行 OOB 评估。评估结果可通过 **oob_score_** 属性访问：

```
>>> bag_clf = BaggingClassifier(DecisionTreeClassifier(), n_estimators=500,
...                             oob_score=True, n_jobs=-1, random_state=42)
...
>>> bag_clf.fit(X_train, y_train)
>>> bag_clf.oob_score_
0.896
```

根据 OOB 评估，这个 **BaggingClassifier** 在测试集上可能达到约 89.6% 的准确率。验证如下：

```
>>> from sklearn.metrics import accuracy_score
>>> y_pred = bag_clf.predict(X_test)
>>> accuracy_score(y_test, y_pred)
0.92
```

测试集上的准确率为 92.0%，因此 OOB 评估略显悲观。

每个训练实例的 OOB 决策函数也可以通过 **oob_decision_function_** 属性获取。由于基础估计器具有 **predict_proba()** 方法，该决策函数会返回每个训练实例的类别概率。例如，OOB 评估估计第一个训练实例属于正类的概率为 67.6%，属于负类的概率为 32.4%：

[7] 当 m 增大时，这个比例趋近于 **1 – exp(–1) ≈ 63%**。

```
>>> bag_clf.oob_decision_function_[:3]  # probas for the first 3 instances
array([[0.32352941, 0.67647059],
       [0.3375    , 0.6625    ],
       [1.        , 0.        ]])
```

6.2.3　随机补丁和随机子空间（Random Patches and Random Subspaces）

`BaggingClassifier` 还支持抽样特征。抽样由两个超参数控制：`max_features` 和 `bootstrap_features`。它们与 `max_samples` 和 `bootstrap` 的作用方式相同，只不过抽样对象从实例变成了特征。因此，每个预测器都会在输入特征的一个随机子集上训练。

当输入是高维数据，例如图像时，这种技术尤其有用，因为它可以显著加快训练。对训练实例和特征同时抽样，称为随机补丁（random patches）方法[8]。保留所有训练实例（将 `bootstrap=False` 且 `max_samples=1.0`），但抽样特征（将 `bootstrap_features` 设为 `True`，或者将 `max_features` 设为小于 1.0 的值），称为随机子空间（random subspaces）方法[9]。

抽样特征会带来更多预测器多样性，以略高的偏差换取更低的方差。

6.3　随机森林（Random Forests）

如前文所述，随机森林[10] 是决策树的集成，通常采用 Bagging 方法训练，有时也采用 Pasting，且 `max_samples` 通常设为训练集大小。与其创建一个 `BaggingClassifier` 并传入一个 `DecisionTreeClassifier`，不如直接使用针对决策树优化、更加方便的 `RandomForestClassifier` 类[11]。回归任务对应的类是 `RandomForestRegressor`。下面的代码训练一个包含 500 棵树的随机森林分类器，每棵树最多有 16 个叶节点，并使用所有可用 CPU 核心：

[8] Gilles Louppe 和 Pierre Geurts，“Ensembles on Random Patches”，Lecture Notes in Computer Science 7523（2012）：346-361。

[9] Tin Kam Ho，“The Random Subspace Method for Constructing Decision Forests”，IEEE Transactions on Pattern Analysis and Machine Intelligence 20，第 8 期（1998）：832-844。

[10] Tin Kam Ho，“Random Decision Forests”，Proceedings of the Third International Conference on Document Analysis and Recognition 1（1995）：278。

[11] 如果想集成决策树以外的模型，`BaggingClassifier` 类仍然很有用。

```
from sklearn.ensemble import RandomForestClassifier
rnd_clf = RandomForestClassifier(n_estimators=500, max_leaf_nodes=16,
                                 n_jobs=-1, random_state=42)
rnd_clf.fit(X_train, y_train)
y_pred_rf = rnd_clf.predict(X_test)
```

除少数例外，**RandomForestClassifier** 具有 **DecisionTreeClassifier** 的全部超参数，用于控制树的生长；同时还具有 **BaggingClassifier** 的全部超参数，用于控制集成本身。

RandomForestClassifier 在生长树时引入了额外的随机性：在节点分裂时，它不是在所有特征中寻找最佳特征，而是在特征的一个随机子集中寻找最佳特征（见第 5 章）。默认情况下，它会抽取 **sqrt(n)** 个特征，其中 **n** 是特征总数。该算法会得到更加多样化的树，以较高的偏差换取较低的方差，通常能得到更好的总体模型。因此，下面这个 **BaggingClassifier** 与前面的 **RandomForestClassifier** 等价：

```
bag_clf = BaggingClassifier(
    DecisionTreeClassifier(max_features="sqrt", max_leaf_nodes=16),
    n_estimators=500, n_jobs=-1, random_state=42)
```

6.3.1　极端随机树（Extra-Trees）

训练随机森林时，每个节点只考虑一个随机特征子集来进行分裂。还可以让树更加随机：对每个特征也使用随机阈值，而不是像普通决策树那样搜索可能的最佳阈值。创建 **DecisionTreeClassifier** 时，将 **splitter="random"** 即可实现。

由这种极端随机树组成的森林称为极端随机树（extremely randomized trees）[12]，简称 Extra-Trees。该技术再次以更高的偏差换取更低的方差，因此当数据集含噪声或维度很高、普通随机森林出现过拟合时，Extra-Trees 可能表现更好。

Extra-Trees 分类器的训练速度也比普通随机森林快得多，因为在随机森林的每个节点上寻找每个特征的最佳阈值，是生长决策树时最耗时的任务之一。

你可以使用 Scikit-Learn 的 **ExtraTreesClassifier** 类创建 Extra-Trees 分类器。它的 API 与 **RandomForestClassifier** 相同，只是 **bootstrap** 默认值为 **False**。同样，**ExtraTreesRegressor** 与 **RandomForestRegressor** 的 API 相同，只是 **bootstrap** 默认值为 **False**。

[12] Pierre Geurts 等，“Extremely Randomized Trees”，Machine Learning 63，第 1 期（2006）：3-42。

> 提示：与决策树类一样，最新版本的随机森林类和 Extra-Trees 类原生支持缺失值，不需要使用填充器。

6.3.2　特征重要性（Feature Importance）

随机森林的另一个优点是很容易测量各个特征的相对重要性。Scikit-Learn 会观察使用某个特征的树节点平均减少了多少不纯度，以此测量该特征的重要性。更准确地说，这是一个加权平均值，其中每个节点的权重等于与该节点关联的训练样本数（见第 5 章）。

Scikit-Learn 会在训练后自动计算每个特征的重要性，然后对结果进行缩放，使所有重要性之和为 1。可以通过 **`feature_importances_`** 变量访问这些结果。下面的代码在第 4 章介绍过的 iris 数据集上训练随机森林分类器，并输出每个特征的重要性。花瓣长度（44%）和花瓣宽度（42%）似乎是最重要的特征，而萼片长度和萼片宽度相对不重要（分别为 11% 和 2%）：

```
>>> from sklearn.datasets import load_iris
>>> iris = load_iris(as_frame=True)
>>> rnd_clf = RandomForestClassifier(n_estimators=500, random_state=42)
>>> rnd_clf.fit(iris.data, iris.target)
>>> for score, name in zip(rnd_clf.feature_importances_, iris.data.columns):
...     print(round(score, 2), name)
...
0.11 sepal length (cm)
0.02 sepal width (cm)
0.44 petal length (cm)
0.42 petal width (cm)
```

类似地，如果在第 3 章介绍过的 MNIST 数据集上训练随机森林分类器，并绘制每个像素的重要性，就会得到图 6-6。

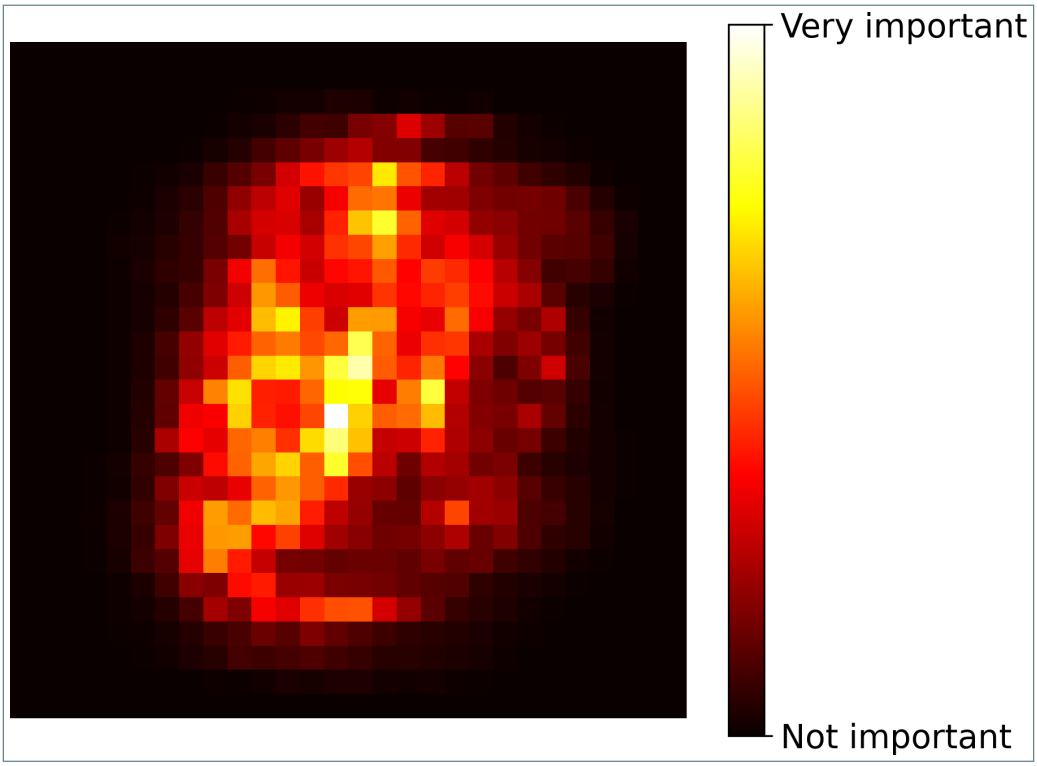

图 6-6: 随机森林分类器给出的 MNIST 像素重要性（MNIST pixel importance according to a random forest classifier）

随机森林对于快速了解哪些特征真正重要非常有用，尤其是在需要进行特征选择时。

6.4　提升法（Boosting）

提升法最初称为假设提升（hypothesis boosting），指的是任何能够把多个弱学习器组合成强学习器的集成方法。大多数提升方法的基本思想是按顺序训练预测器，每个预测器都试图纠正前一个预测器。提升方法有很多，最流行的是 AdaBoost[13]（adaptive boosting 的缩写）和梯度提升。先从 AdaBoost 开始。

[13] Yoav Freund 和 Robert E. Schapire，“A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting”，Journal of Computer and System Sciences 55，第 1 期（1997）：119-139。

6.4.1　AdaBoost

让新预测器纠正前一个预测器的一种方法，是更多关注前一个预测器拟合不佳的训练实例。这样，新预测器会越来越关注困难样本，这正是 AdaBoost 使用的技术。

例如，训练 AdaBoost 分类器时，算法首先训练一个基础分类器（例如决策树），并用它对训练集进行预测。然后提高被错误分类的训练实例的相对权重。接着使用更新后的权重训练第二个分类器，再次对训练集进行预测、更新实例权重，如此重复（见图 6-7）。

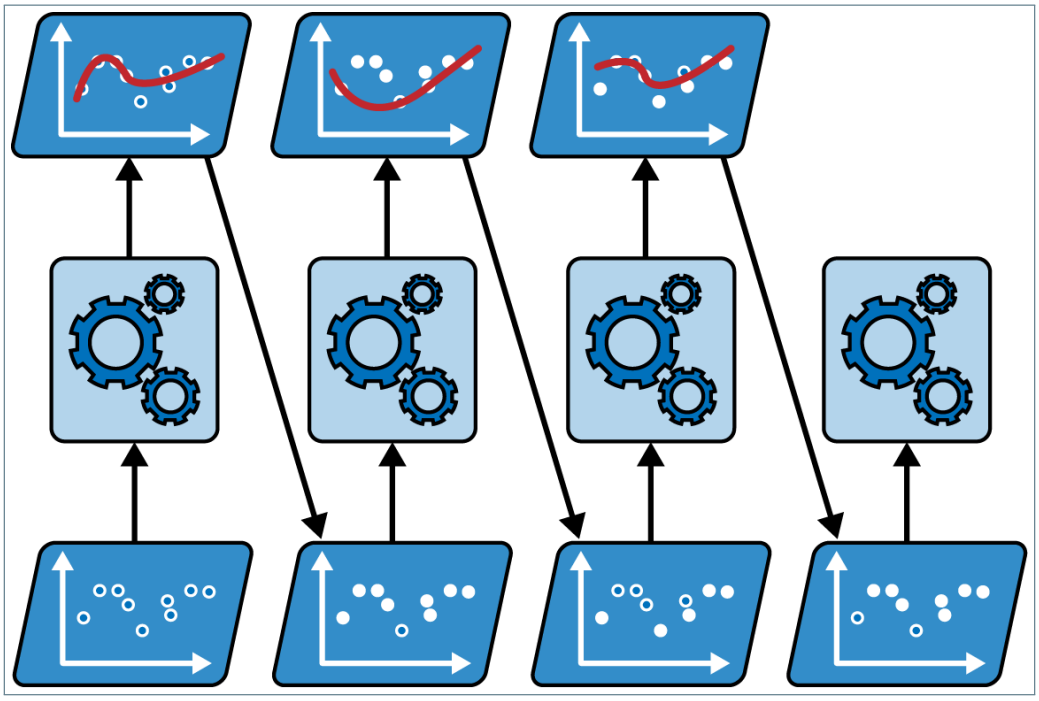

图 6-7: AdaBoost 按顺序训练并更新实例权重（AdaBoost sequential training with instance weight updates）

图 6-8 展示了 moons 数据集上连续五个预测器的决策边界。在这个例子中，每个预测器都是使用 RBF 核的高度正则化 SVM 分类器[14]。第一个分类器犯下许多错误，因此这些实例的权重被提高；第二个分类器因此能更好地处理这些实例，以此类推。右侧图展示了相同的预测器序列，但学习率减半，也就是每次迭代中错误分类实例的权重提升幅度更小。可以看到，这种顺序学习与梯度下降有些相似：梯度下降调

[14]这里只是为了说明问题。SVM 通常不是 AdaBoost 的理想基础预测器，因为它速度慢，而且与 AdaBoost 结合时往往不稳定。

整单个预测器的参数以最小化成本函数，而 AdaBoost 向集成中不断添加预测器，使集成逐步变好。

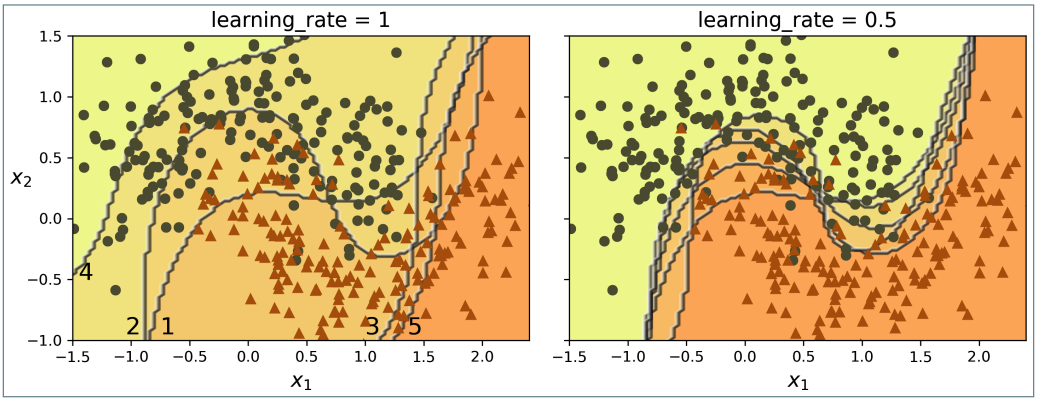

图 6-8: 连续预测器的决策边界（Decision boundaries of consecutive predictors）

> 警告：这种顺序学习有一个重要缺点：由于每个预测器必须等前一个预测器训练并评估完毕后才能训练，因此训练无法并行化。结果是，它的扩展性不如 Bagging 或 Pasting。

所有预测器训练完成后，集成的预测方式与 Bagging 或 Pasting 很相似，只是每个预测器的权重取决于它在加权训练集上的总体准确率。

具体来看 AdaBoost 算法。每个实例权重 `w^(i)` 初始设为 `1/m`，因此总和为 1。训练第一个预测器，并在训练集上计算它的加权错误率 `r_1`：

$$r_j = \sum_{i=1}^{m} w^{(i)} \mathbf{1}_{\left(\hat{y}_j^{(i)} \neq y^{(i)}\right)} \tag{6-1}$$

公式 6-1　第 j 个预测器的加权错误率（Weighted error rate of the jth predictor）

随后使用公式 6-2 计算预测器权重 `α_j`，其中 η 是学习率超参数，默认值为 1[15]。预测器越准确，权重越高；如果它只是在随机猜测，权重就接近 0；如果它大多数时候都预测错误，也就是比随机猜测还差，权重会变成负数。

$$\alpha_j = \eta \log\left(\frac{1 - r_j}{r_j}\right) \tag{6-2}$$

公式 6-2　预测器权重（Predictor weight）

[15] 原始 AdaBoost 算法不使用学习率超参数。

接下来使用公式 6-3 更新实例权重，提高被错误分类实例的权重，使下一个预测器更加关注它们：

$$w^{(i)} \leftarrow \begin{cases} w^{(i)} & \text{if } \hat{y}_j^{(i)} = y^{(i)} \\ w^{(i)} \exp(\alpha_j) & \text{if } \hat{y}_j^{(i)} \neq y^{(i)} \end{cases} \tag{6-3}$$

公式 6-3　权重更新规则（Weight update rule）

然后对所有实例权重进行归一化，确保它们的总和再次为 1，也就是除以 `Σ_{i=1}^m w^(i)`。

最后，使用更新后的权重训练新的预测器，并重复整个过程：计算新预测器的权重，更新实例权重，再训练另一个预测器，依此进行。达到所需的预测器数量，或找到完美预测器时，算法停止。

进行预测时，AdaBoost 计算所有预测器的预测结果，并使用预测器权重 `α_j` 对其加权。获得加权投票最多的类别就是预测类别：

$$\hat{y}(\mathbf{x}) = \arg\max_k \sum_{j=1}^{N} \alpha_j \mathbf{1}_{(\hat{y}_j(\mathbf{x})=k)} \tag{6-4}$$

公式 6-4　AdaBoost 预测（AdaBoost predictions）

Scikit-Learn 使用 AdaBoost 的多分类版本 SAMME[16]。当只有两个类别时，SAMME 等价于 AdaBoost。

下面的代码使用 Scikit-Learn 的 **`AdaBoostClassifier`** 类，训练一个由 30 个决策树桩组成的 AdaBoost 分类器。决策树桩是 **`max_depth=1`** 的决策树，也就是只有一个决策节点和两个叶节点的树，它是 **`AdaBoostClassifier`** 的默认基础估计器。回归任务对应的类是 **`AdaBoostRegressor`**：

```
from sklearn.ensemble import AdaBoostClassifier
ada_clf = AdaBoostClassifier(
    DecisionTreeClassifier(max_depth=1), n_estimators=30,
    learning_rate=0.5, random_state=42, algorithm="SAMME")
ada_clf.fit(X_train, y_train)
```

[16] SAMME 是 Stagewise Additive Modeling using a Multiclass Exponential loss function 的缩写。更多细节见 Ji Zhu 等，“Multi-Class AdaBoost”，Statistics and Its Interface 2，第 3 期（2009）：349-360。

> 提示：如果 AdaBoost 集成在训练集上过拟合，可以尝试减少预测器数量，或者对基础估计器进行更强的正则化。

6.5　梯度提升（Gradient Boosting）

另一种非常流行的提升算法是梯度提升[17]。与 AdaBoost 一样，梯度提升也会顺序地向集成中添加预测器，每个预测器都纠正前一个预测器。不过，它不像 AdaBoost 那样每次迭代都调整实例权重，而是尝试让新的预测器拟合前一个预测器产生的残差误差。

下面通过一个简单的回归示例来说明，基础预测器使用决策树，这种方法称为梯度树提升（gradient tree boosting），或梯度提升回归树（GBRT）。先生成一个带噪声的二次数据集，并在其上拟合 `DecisionTreeRegressor`：

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor
m = 100  # number of instances
rng = np.random.default_rng(seed=42)
X = rng.random((m, 1)) - 0.5
noise = 0.05 * rng.standard_normal(m)
y = 3 * X[:, 0] ** 2 + noise  # y = 3x² + Gaussian noise
tree_reg1 = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg1.fit(X, y)
```

接下来，让第二个 `DecisionTreeRegressor` 拟合第一个预测器产生的残差：

```
y2 = y - tree_reg1.predict(X)
tree_reg2 = DecisionTreeRegressor(max_depth=2, random_state=43)
tree_reg2.fit(X, y2)
```

然后让第三个回归器拟合第二个预测器产生的残差：

```
y3 = y2 - tree_reg2.predict(X)
tree_reg3 = DecisionTreeRegressor(max_depth=2, random_state=44)
tree_reg3.fit(X, y3)
```

现在集成中有三棵树。只需把所有树的预测结果相加，就可以对新实例进行预测：

[17] 梯度提升最早由 Leo Breiman 在 1997 年的论文“Arcing the Edge”中提出，并在 Jerome H. Friedman 1999 年的论文“Greedy Function Approximation: A Gradient Boosting Machine”中进一步发展。

```
>>> X_new = np.array([[-0.4], [0.], [0.5]])
>>> sum(tree.predict(X_new) for tree in (tree_reg1, tree_reg2, tree_reg3))
array([0.57356534, 0.0405142 , 0.66914249])
```

图 6-9 左侧展示三棵树的预测结果，右侧展示集成的预测结果。第一行中集成只有一棵树，因此预测结果与第一棵树完全相同；第二行中，第二棵树在第一棵树的残差上训练，右侧的集成预测结果等于前两棵树预测结果之和；第三行中，第三棵树继续拟合第二棵树的残差。随着树加入集成，预测结果逐渐变好。

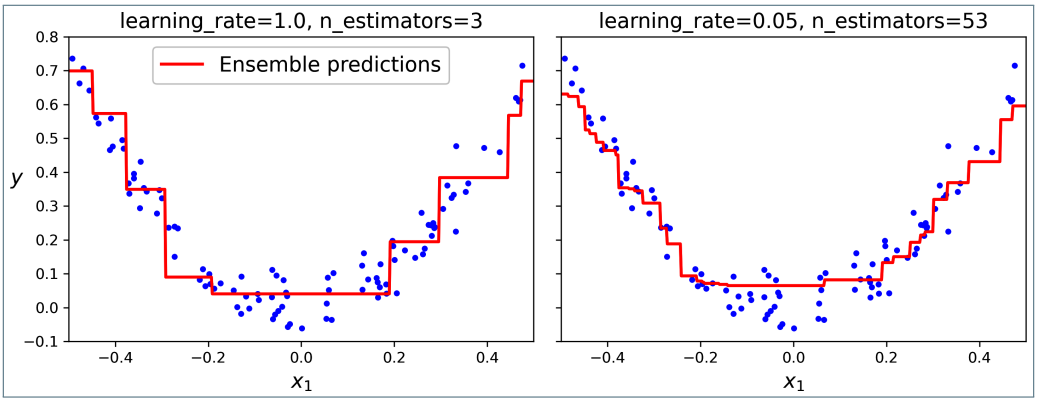

图 6-9: 梯度提升：后续预测器拟合前一个预测器的残差（Gradient boosting: later predictors fit the previous predictor's residuals）

可以使用 Scikit-Learn 的 **GradientBoostingRegressor** 类更方便地训练 GBRT 集成，分类任务对应 **GradientBoostingClassifier**。它与 **RandomForestRegressor** 类似，既有控制决策树生长的超参数（例如 **max_depth**、**min_samples_leaf**），也有控制集成训练的超参数，例如树的数量 **n_estimators**。下面的代码创建与前面相同的集成：

```
from sklearn.ensemble import GradientBoostingRegressor
gbrt = GradientBoostingRegressor(max_depth=2, n_estimators=3,
                                 learning_rate=1.0, random_state=42)
gbrt.fit(X, y)
```

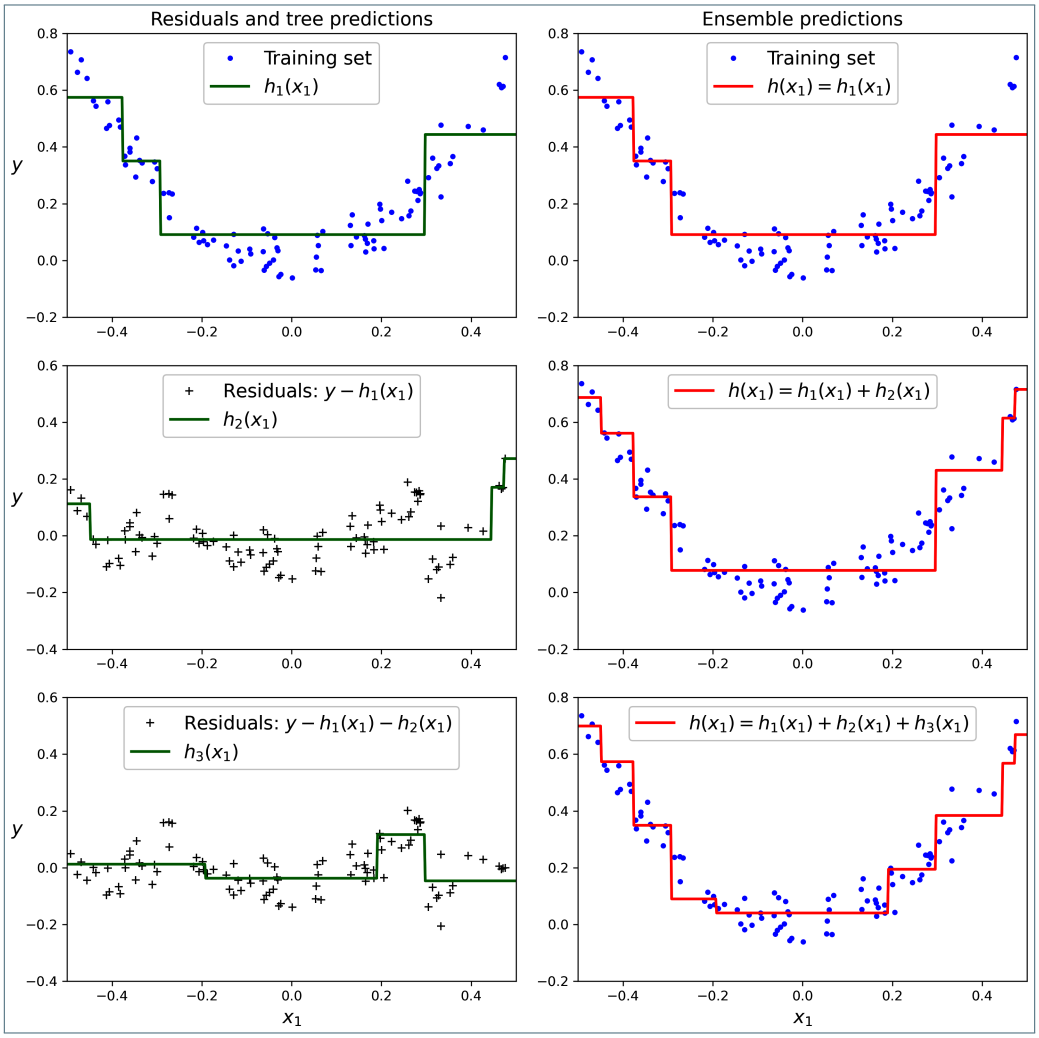

图 6-10: GBRT 集成：预测器数量不足与数量恰好合适（GBRT ensembles with not enough predictors and just enough）

learning_rate 超参数会缩放每棵树的贡献。如果设为较小值，例如 0.05，那么需要更多树才能拟合训练集，但预测结果通常会有更好的泛化能力。这是一种称为收缩（shrinkage）的正则化技术。图 6-10 比较了使用不同超参数训练的两个 GBRT 集成：左侧的树数量不足，无法拟合训练集；右侧的树数量大致合适。如果再添加

更多树，GBRT 就会开始对训练集过拟合。与往常一样，可以使用 `GridSearchCV` 或 `RandomizedSearchCV` 通过交叉验证寻找最佳学习率。

寻找最佳树数量也可以使用交叉验证，但还有更简单的方法：将 `n_iter_no_change` 超参数设为整数，例如 10。这样，如果最后 10 棵树没有带来帮助，`GradientBoostingRegressor` 会在训练过程中自动停止添加树。这就是提前停止，只不过多了一点耐心：在停止前允许若干次迭代没有进展。下面使用提前停止训练集成：

```
gbrt_best = GradientBoostingRegressor(
    max_depth=2, learning_rate=0.05, n_estimators=500,
    n_iter_no_change=10, random_state=42)
gbrt_best.fit(X, y)
```

如果 `n_iter_no_change` 太低，训练可能过早停止，模型会欠拟合；如果太高，则可能过拟合。这里将学习率设得较小，将预测器数量设得较大，但由于提前停止，训练后集成中的实际预测器数量会小得多：

```
>>> gbrt_best.n_estimators_
53
```

设置 `n_iter_no_change` 后，`fit()` 方法会自动把训练集划分为更小的训练集和验证集，从而在每增加一棵树时评估模型性能。验证集大小由 `validation_fraction` 超参数控制，默认值为 10%；`tol` 超参数决定仍然被视为可忽略的最大性能改善，默认值为 0.0001。

`GradientBoostingRegressor` 还支持 `subsample` 超参数，用于指定训练每棵树时使用的训练实例比例。例如，`subsample=0.25` 表示每棵树都使用随机选取的 25% 训练实例。这个技术以较高的偏差换取较低的方差，同时显著加快训练，称为随机梯度提升（stochastic gradient boosting）。

6.5.1　基于直方图的梯度提升（Histogram-Based Gradient Boosting）

Scikit-Learn 还提供了另一种针对大型数据集优化的 GBRT 实现，即基于直方图的梯度提升（HGB）。它会将输入特征分箱，并把特征替换为整数。分箱数量由 `max_bins` 超参数控制，默认值为 255，且不能设置得更高。分箱可以大幅减少训练算法需要评估的可能阈值数量；此外，使用整数可以采用更快、更节省内存的数据结构。分箱的构建方式还消除了训练每棵树时对特征排序的需要。

因此，这种实现的计算复杂度为 `O(b × m)`，而不是 `O(n × m × log(m))`，其中 `b` 是分箱数，`m` 是训练实例数，`n` 是特征数。实际中，在大型数据集上，HGB 的训练速

度可以比普通 GBRT 快数百倍。不过，分箱会造成精度损失，它会起到正则化器的作用：根据数据集的不同，这可能有助于减少过拟合，也可能造成欠拟合。

Scikit-Learn 提供两个 HGB 类：**HistGradientBoostingRegressor** 和 **HistGradientBoostingClassifier**。它们与 **GradientBoostingRegressor** 和 **GradientBoostingClassifier** 相似，但有几个值得注意的区别：

- 当实例数大于 10,000 时，提前停止会自动启用。可以将 **early_stopping** 设为 **True** 或 **False**，强制始终启用或始终关闭提前停止。
- 不支持子采样。
- **n_estimators** 更名为 **max_iter**。
- 可调整的决策树超参数只有 **max_leaf_nodes**、**min_samples_leaf**、**max_depth** 和 **max_features**。

HGB 类原生支持缺失值和类别特征，因此可以大幅简化预处理。不过，类别特征必须表示为从 0 到小于 **max_bins** 的整数，可以使用 **OrdinalEncoder** 完成这一转换。下面以第 2 章介绍过的加利福尼亚住房数据集为例，构建并训练完整流水线：

```
from sklearn.pipeline import make_pipeline
from sklearn.compose import make_column_transformer
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.preprocessing import OrdinalEncoder
hgb_reg = make_pipeline(
    make_column_transformer((OrdinalEncoder(), ["ocean_proximity"]),
                            remainder="passthrough",
                            force_int_remainder_cols=False),
    HistGradientBoostingRegressor(categorical_features=[0], random_state=42)
)
hgb_reg.fit(housing, housing_labels)
```

整个流水线几乎和导入语句一样短！不需要填充器、缩放器或独热编码器，非常方便。注意，**categorical_features** 必须设为类别列的索引，也可以传入布尔数组。不进行任何超参数调优时，该模型的 RMSE 约为 47,600，表现并不差。

总之，当数据集相当大，尤其包含类别特征和缺失值时，HGB 是很好的选择：性能良好，不需要太多预处理，训练速度快。不过，由于分箱，它的精度可能略低于 GBRT，因此可以同时尝试两者。

> 提示：Python 机器学习生态中还有几个经过优化的梯度提升实现，尤其是 XGBoost、CatBoost 和 LightGBM。这些库已经存在多年，专门用于梯度提升，API 与 Scikit-Learn 非常相似，并提供许多额外功能，包括使用 GPU 的硬件加速，值得了解。此外，Yggdrasil Decision Forests（YDF）也提供了多种随机森林算法的优化实现。

6.6　Stacking

本章要讨论的最后一种集成方法称为 Stacking，是 stacked generalization 的缩写[18]。它基于一个简单的想法：与其使用硬投票这样的简单函数汇总所有预测器的预测结果，为什么不训练一个模型来执行这种汇总？图 6-11 展示了一个对新实例执行回归任务的集成。底部三个预测器分别预测 3.1、2.7 和 2.9，最终预测器，也就是混合器（blender）或元学习器（meta-learner），以这些预测结果为输入，给出最终预测 3.0。

[18] David H. Wolpert, “Stacked Generalization”, Neural Networks 5, 第 2 期（1992）: 241-259。

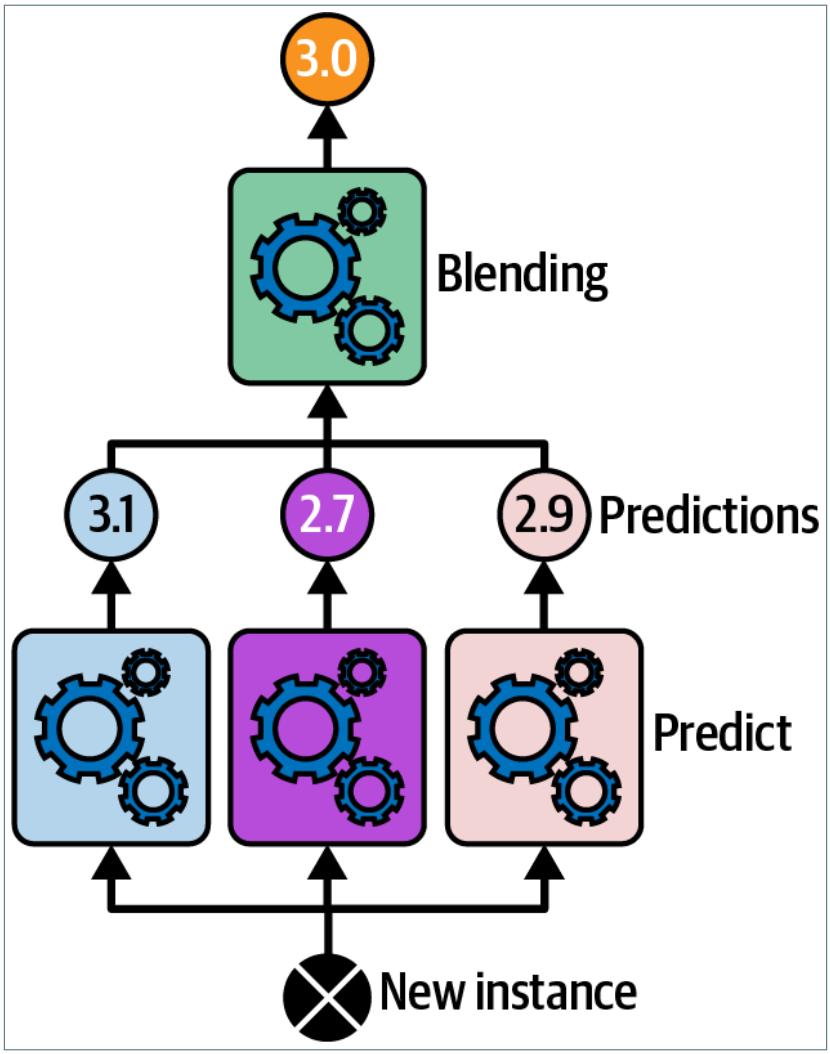

图 6-11: 使用混合预测器汇总预测结果（Aggregating predictions using a blending predictor）

要训练混合器，首先需要构建混合训练集。可以对集成中的每个预测器调用 `cross_val_predict()`，为原始训练集中的每个实例获得样本外预测（见图 6-12），并把这些预测作为混合器的输入特征；目标值可以直接从原始训练集复制。注意，无论原始训练集有多少个特征，混合训练集都会为每个预测器包含一个输入特征，本例中就是三个输入特征。

混合器训练完成后，必须在完整的原始训练集上重新训练基础预测器一次，因为 `cross_val_predict()` 不会保留训练好的估计器。

实际上，可以用这种方式训练多个不同的混合器，例如一个使用线性回归，另一个使用随机森林回归，从而得到一整层混合器；然后再在这一层上添加另一个混合

器，得到最终预测，如图 6-13 所示。这样可能再挤出一些性能，但会增加训练时间和系统复杂度。

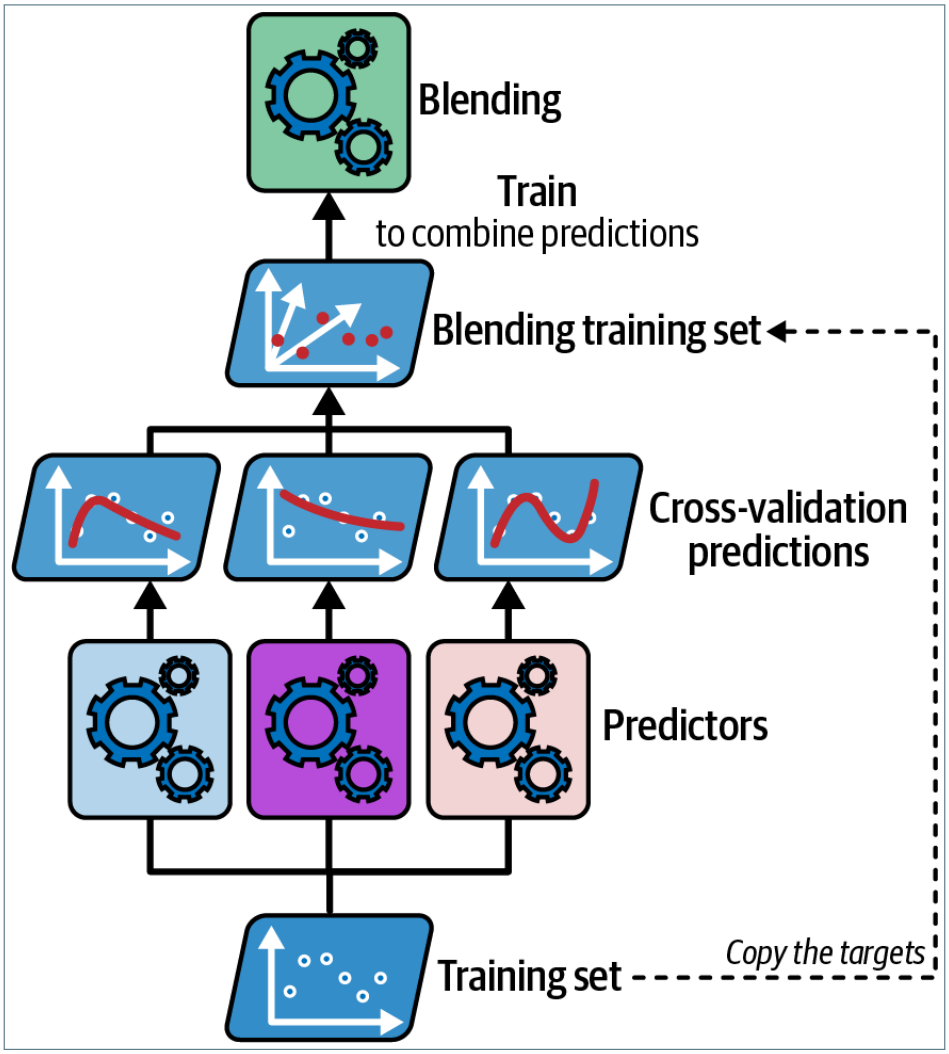

图 6-12: 训练 Stacking 集成中的混合器（Training the blender in a stacking ensemble）

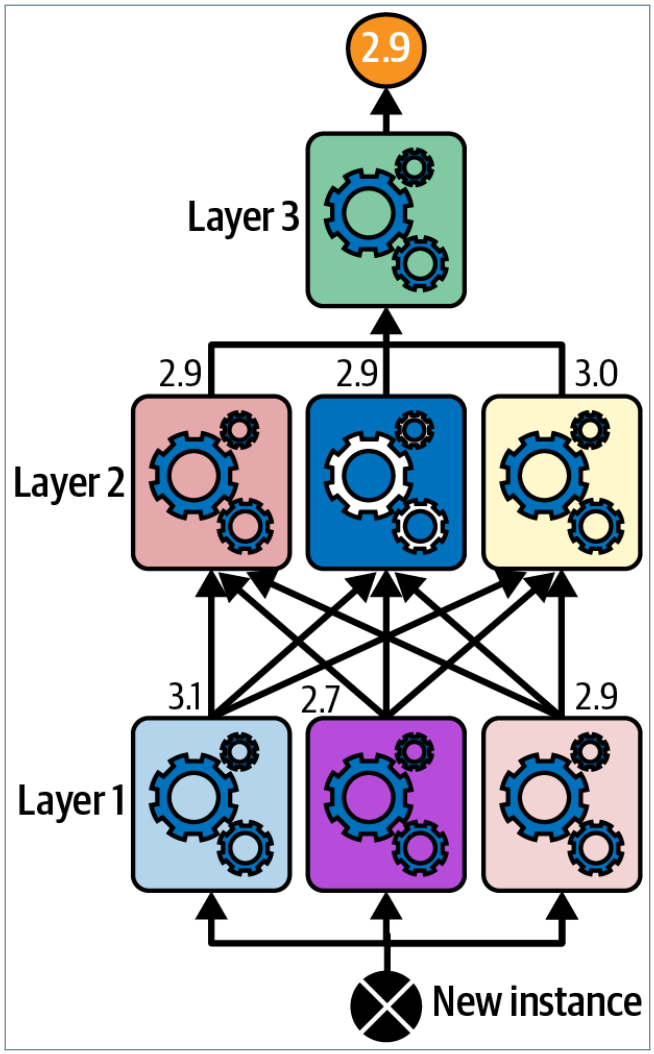

图 6-13: 多层 Stacking 集成中的预测结果（Predictions in a multilayer stacking ensemble）

Scikit-Learn 提供了两个 Stacking 集成类：`StackingClassifier` 和 `StackingRegressor`。例如，可以在本章开始使用的 moons 数据集上，用 `StackingClassifier` 替换之前的 `VotingClassifier`：

```
from sklearn.ensemble import StackingClassifier
stacking_clf = StackingClassifier(
    estimators=[
        ('lr', LogisticRegression(random_state=42)),
        ('rf', RandomForestClassifier(random_state=42)),
        ('svc', SVC(probability=True, random_state=42))
    ],
    final_estimator=RandomForestClassifier(random_state=43),
    cv=5  # number of cross-validation folds
```

```
)
stacking_clf.fit(X_train, y_train)
```

对于每个预测器，Stacking 分类器会在可用时调用 **predict_proba()**；如果没有该方法，就退回到 **decision_function()**，最后才调用 **predict()**。如果没有提供最终估计器，**StackingClassifier** 会使用 **LogisticRegression**，而 **StackingRegressor** 会使用 **RidgeCV**。

在测试集上评估这个 Stacking 模型，会得到 92.8% 的准确率，略高于使用软投票的投票分类器（92%）。是否值得付出额外的复杂度和计算成本，取决于具体应用，因为最终还要在所有其他模型之后再运行一个模型。

总之，集成方法灵活、强大，而且相当容易使用。只要不粗心，它们也可能过拟合，但任何强大的模型都有这个问题。表 6-1 总结了本章讨论的各种技术以及适用场景。

> 提示：对于大多数机器学习任务，尤其是异构表格数据，随机森林、AdaBoost、GBRT 和 HGB 都是应该优先尝试的模型。此外，由于它们几乎不需要预处理，非常适合快速构建原型并运行。

到目前为止，我们只讨论了监督学习技术。下一章将转向最常见的无监督学习任务：降维。

6.7　练习（Exercises）

1. 如果你在完全相同的训练数据上训练了五个不同的模型，并且它们的精确率都达到 95%，把这些模型组合起来还有可能得到更好的结果吗？如果可以，如何组合？如果不可以，为什么？
2. 硬投票分类器和软投票分类器有什么区别？
3. 可以通过把 Bagging 集成分布到多台服务器上来加快训练吗？Pasting 集成、提升集成、随机森林和 Stacking 集成呢？
4. 袋外评估有什么好处？
5. Extra-Trees 集成为什么比普通随机森林更随机？这种额外的随机性有什么帮助？Extra-Trees 分类器比普通随机森林更慢还是更快？

表 6-1: 各集成学习方法的适用场景

集成方法	适用场景	示例用例
硬投票	类别均衡、包含多个强大且多样的分类器的分类数据集。	垃圾邮件检测、情感分析、疾病分类
软投票	包含概率模型且置信度分数重要的分类数据集。	医学诊断、信用风险分析、假新闻检测
Bagging	具有高方差且模型容易过拟合的结构化或半结构化数据集。	金融风险建模、电商推荐
Pasting	需要更独立模型的结构化或半结构化数据集。	客户分群、蛋白质分类
随机森林	特征可能含噪声的高维结构化数据集。	客户流失预测、遗传数据分析、欺诈检测
极端随机树	特征很多且速度至关重要、同时希望降低方差的大型结构化数据集。	实时欺诈检测、传感器数据分析
AdaBoost	含弱学习器、噪声较少的小型至中型结构化数据集，且可解释性有帮助。	信用评分、异常检测、预测性维护
梯度提升	需要较高预测能力、即使需要额外调参也可以接受的中大型结构化数据集。	房价预测、风险评估、需求预测
基于直方图的梯度提升(HGB)	训练速度和可扩展性关键的大型结构化数据集。	点击率预测、排序算法、广告实时竞价
Stacking	通过组合多个多样模型来最大化准确率的复杂高维数据集。	推荐引擎、自动驾驶决策、Kaggle 竞赛

6. 如果 AdaBoost 集成在训练数据上欠拟合，应该调整哪些超参数，如何调整？

7. 如果梯度提升集成在训练集上过拟合，应该增大学习率还是减小学习率？

8. 加载第 3 章介绍过的 MNIST 数据集，并将其划分为训练集、验证集和测试集，例如训练集使用 50,000 个实例，验证集使用 10,000 个实例，测试集使用 10,000 个实例。然后训练各种分类器，例如随机森林分类器、Extra-Trees 分类器和 SVM 分类器。接着尝试使用软投票或硬投票把它们组合成一个在验证集上超过每个单独分类器的集成。找到这样的集成后，在测试集上测试它。与单个分类器相比，性能提升了多少？

9. 使用上一道练习中的单个分类器在验证集上进行预测，并使用这些预测创建一个新的训练集：每个训练实例都是该图像所有分类器预测结果组成的向量，目标值是图像类别。在这个新训练集上训练一个分类器。恭喜，你刚刚训练了一个混合器，它与这些分类器一起构成了一个 Stacking 集成！现在在测试集上评估该集成。对于测试集中的每幅图像，使用所有分类器进行预测，再把这些预测输入混合器，得到集成预测。它与之前训练的投票分类器相比如何？现在使用 `StackingClassifier` 再试一次。性能更好吗？如果更好，为什么？

本章练习的解答位于本章 notebook 的末尾：https://homl.info/colab-p。

本章脚注中的文献出处、示例说明和公式注释均已按原书顺序保留在对应正文位置。

第 7 章　降维

Dimensionality Reduction

许多机器学习问题中，每个训练实例都包含成千上万甚至数百万个特征。这些特征不仅会让训练极其缓慢，还会让找到良好解决方案变得更加困难。正如你将看到的，这个问题通常称为维度灾难（curse of dimensionality）。

幸运的是，在现实问题中，通常可以大幅减少特征数量，把一个难以处理的问题变成可处理的问题。例如，考虑第 3 章介绍过的 MNIST 图像：图像边缘的像素几乎总是白色的，因此可以从训练集中完全删除这些像素，而不会损失太多信息。上一章的图 6-6 证实，这些像素对于分类任务几乎完全不重要。此外，相邻像素往往高度相关：如果把它们合并成一个像素，例如取两个像素强度的平均值，也不会损失太多信息，还能消除冗余，有时也能消除噪声。

> 警告：降维也可能丢失有用信息，就像把图像压缩成 JPEG 可能降低图像质量一样；尤其是降维过度时，系统性能可能略有下降。此外，一些模型，例如神经网络，能够高效处理高维数据，并学习在保留任务所需有用信息的同时降低数据维度。总之，增加一个降维预处理步骤并不总是有帮助。

除了加快训练、可能提高模型性能之外，降维对数据可视化也极其有用。把维度减少到二维或三维，就可以在图表上绘制高维训练集的压缩视图，并通过观察聚类等模式获得重要洞见。此外，数据可视化对于向非数据科学家传达结论非常重要，尤其是要使用结果做决策的人。

本章首先讨论维度灾难，了解高维空间中的现象。然后介绍降维的两种主要方法：投影和流形学习。最后讨论三种最流行的降维技术：PCA、随机投影和局部线性嵌入（LLE）。

7.1　维度灾难（The Curse of Dimensionality）

我们已经习惯生活在三维空间中[1]，因此在想象高维空间时，直觉往往会失效。即使是一个基本的四维超立方体，也很难在脑海中想象（见图 7-1），更不用说弯曲在一千维空间中的二维百维椭球了。

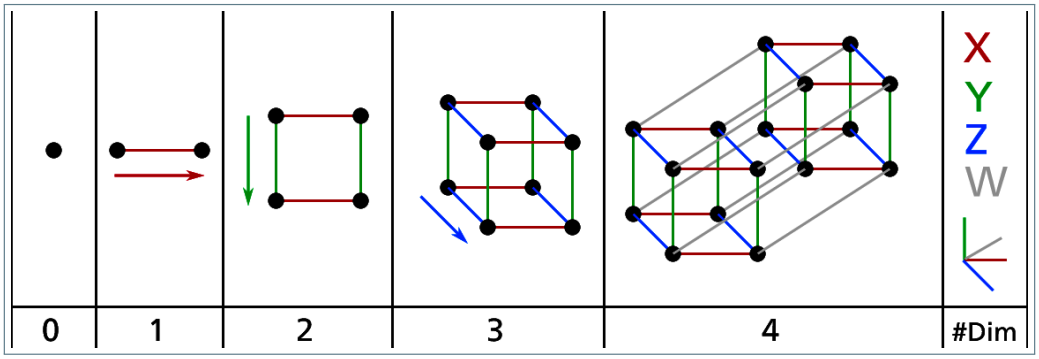

图 7-1: 点、线段、正方形、立方体和超立方体 tesseract（0D 至 4D 超立方体）（Point, segment, square, cube, and tesseract）

图 7-1 中旋转的 tesseract 投影示例见：https://homl.info/30。图像由 Wikipedia 用户 Nerd Boy1392 制作(Creative Commons BY-SA 3.0), 转载自：https://en.wikipedia.org/wiki/Tesseract。

许多事物在高维空间中的行为确实完全不同。例如，在单位正方形（1 × 1 正方形）中随机取一个点，它距离边界小于 0.001 的概率只有约 0.4%，换句话说，随机点几乎不可能沿任何维度处于“极端”位置。但在 10,000 维单位超立方体中，这个概率超过 99.999999%。高维超立方体中的大多数点都非常靠近边界[2]。

还有一个更棘手的差异：在单位正方形中随机取两个点，它们之间的平均距离大约为 0.52；在三维单位立方体中，两个随机点的平均距离约为 0.66。那么，在 1,000,000 维单位超立方体中随机取两个点呢？平均距离居然约为 408.25，大致等于 sqrt(1,000,000)。

这违反直觉：两个点明明都位于同一个单位超立方体中，为什么会相距这么远？这是因为高维空间拥有极其充足的空间。

因此，高维数据集通常非常稀疏：大多数训练实例彼此相距很远，所以基于距离或相似度的训练方法，例如 k 最近邻，效果会差得多。有些模型甚至完全不可用，因

[1]如果把时间也算进去，那就是四维；如果你是弦论学家，还要再加上几个维度。

[2]有趣的是，只要考虑足够多的维度，你认识的任何人很可能都会在至少一个维度上是“极端”的，例如他们放入咖啡中的糖量。

为它们随数据维度扩展得很差，例如 SVM 或稠密神经网络。新实例也可能距离所有训练实例都很远，使预测比低维情况下更不可靠，因为预测会依赖更大的外推。随着数据中的模式越来越难识别，模型会更容易拟合噪声；正则化会变得更加重要。最后，模型也会更难解释。

理论上，可以通过增加训练集规模，使训练实例密度达到足够程度来解决其中一些问题。不幸的是，实践中，要达到给定密度所需的训练实例数量会随着维度数量呈指数增长。即使只有 100 个特征，而且这已经显著少于 MNIST 问题中的特征，每个特征取值范围为 0 到 1，并且假设实例在所有维度上均匀分布，要使训练实例彼此平均距离不超过 0.1，所需实例数也会超过可观测宇宙中的原子总数。

7.2　降维的主要方法（Main Approaches for Dimensionality Reduction）

在深入具体的降维算法之前，先看看降维的两种主要方法：投影和流形学习。

投影（Projection）　在大多数现实问题中，训练实例并不是均匀分布在所有维度上的。许多特征几乎保持不变，而另一些特征高度相关，MNIST 就是这种情况。因此，所有训练实例都位于高维空间中一个低得多的子空间之内或附近。这个概念比较抽象，我们来看一个例子。图 7-2 中，三维数据集用小球表示；后文会多次引用这幅图。

注意，所有训练实例都靠近一个平面。这个平面是高维三维空间中的低维二维子空间。如果把每个训练实例垂直投影到这个子空间，也就是沿着连接实例与平面的短虚线投影，就会得到图 7-3 所示的新二维数据集。就这样，我们把数据集从三维降到了二维。注意，坐标轴对应新特征 z1 和 z2，它们是投影点在平面上的坐标。

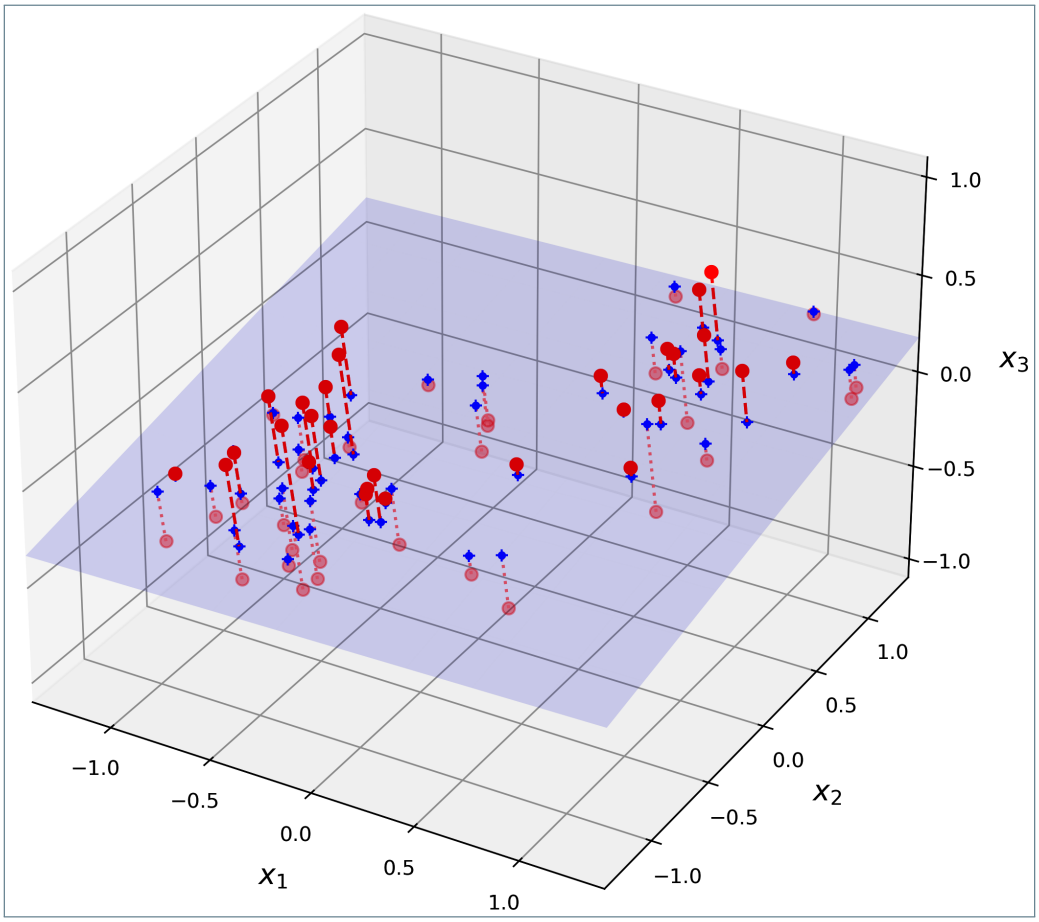

图 7-2: 接近二维子空间的三维数据集（A 3D dataset lying close to a 2D subspace）

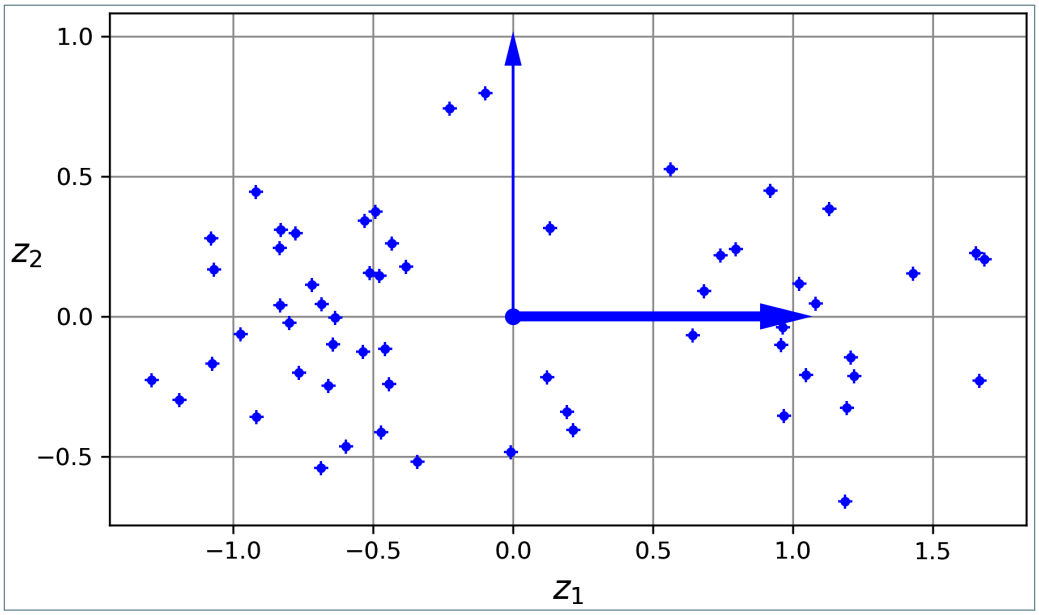

图 7-3: 投影后的新二维数据集（The new 2D dataset after projection）

流形学习（Manifold Learning） 虽然投影速度快，而且通常效果很好，但它并不总是降维的最佳方法。很多时候，子空间可能会发生扭曲，例如图 7-4 所示的 Swiss roll 数据集。这是一个由三维点组成、形状像瑞士卷的玩具数据集。

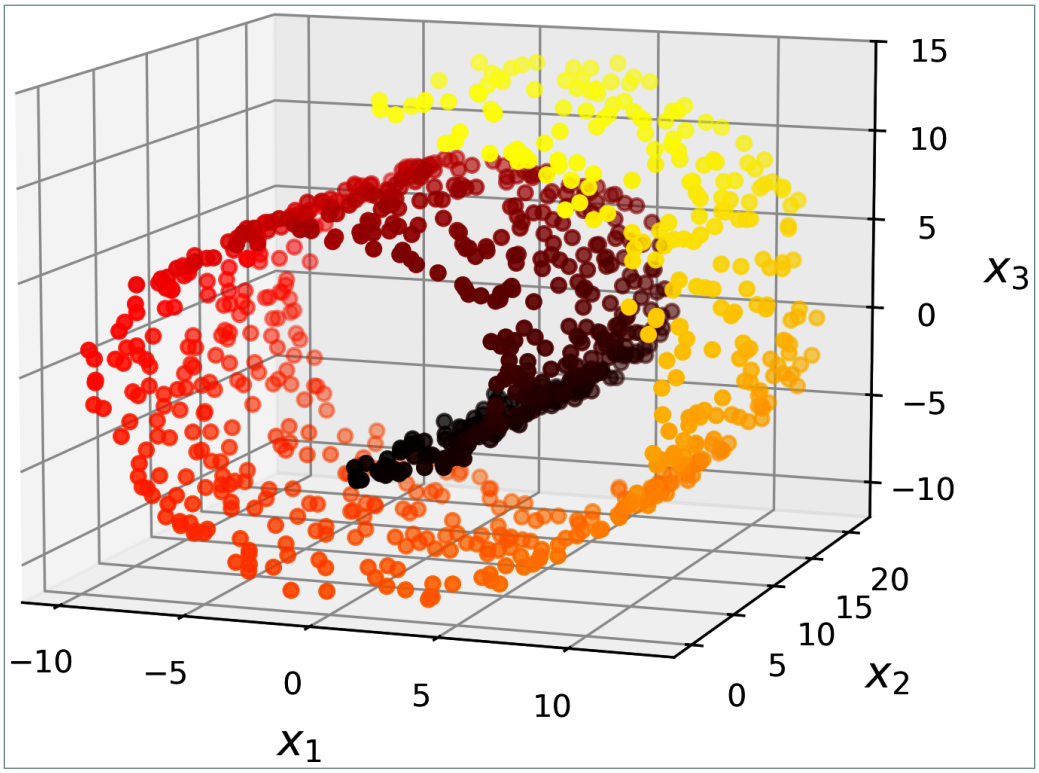

图 7-4: Swiss roll 数据集（Swiss roll dataset）

简单投影到一个平面上，例如丢弃 x3，会把 Swiss roll 的不同层压到一起，如图 7-5 左侧所示。更理想的做法是把 Swiss roll 展开，得到图 7-5 右侧的二维数据集。

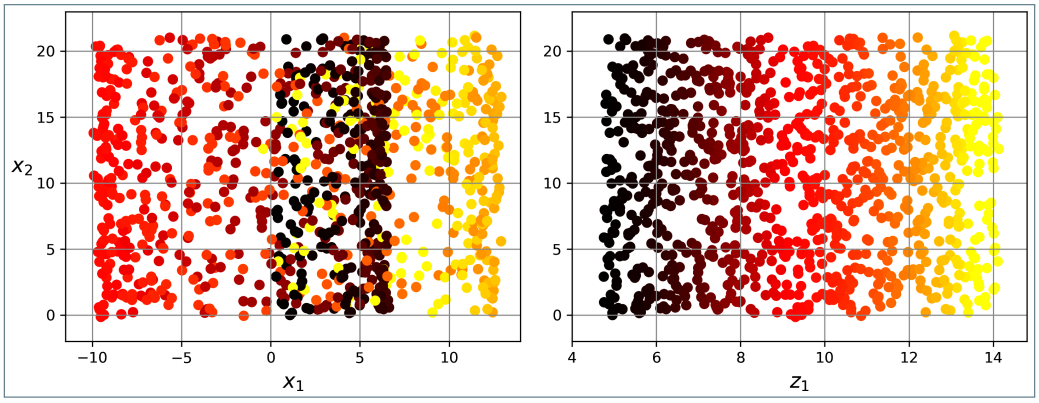

图 7-5: 投影到平面造成的压扁（左）与展开 Swiss roll（右）（Squashing by projection versus unrolling the Swiss roll）

Swiss roll 是二维流形（manifold）的一个例子。简单地说，二维流形是可以在高维空间中弯曲和扭转的二维形状。更一般地，d 维流形是 n 维空间（其中 d < n）的

一部分，它在局部看起来像一个 d 维超平面。对 Swiss roll 来说，d = 2、n = 3：它在局部看起来像二维平面，但沿第三个维度卷了起来。

许多降维算法，例如 LLE、Isomap、t-SNE 和 UMAP，都会对训练实例所在的流形进行建模，这称为流形学习。流形学习依赖流形假设（manifold assumption），也称流形假说，即大多数现实世界的高维数据集都接近某个低得多的维流形。这一假设经常得到经验观察的支持。

再次考虑 MNIST 数据集：所有手写数字图像都有一些相似性。它们由相连的线条构成，边缘是白色的，而且大致位于中心。如果随机生成图像，那么看起来像手写数字的图像只占极小比例。换句话说，生成数字图像时可用的自由度，远低于生成任意图像时可用的自由度。这些约束会把数据集挤压到低维流形中。

流形假设通常还伴随另一个隐含假设：如果把任务，例如分类或回归，表示在流形所在的低维空间中，任务会变得更简单。例如，图 7-6 上排中，Swiss roll 被分成两个类别。在三维空间中，决策边界相当复杂；但在二维展开流形空间中，决策边界是一条直线。

不过，这个隐含假设并不总是成立。例如，图 7-6 下排中，决策边界位于 x1 = 5。在原始三维空间中，这个决策边界很简单，是一个垂直平面；但在展开的流形中，它看起来更复杂，由四条相互独立的线段组成。

总之，在训练模型前降低训练集维度通常可以加快训练，但不一定得到更好或更简单的解决方案，一切取决于数据集。当数据集相对于特征数量较小、噪声较多，或者许多特征彼此高度相关，也就是存在冗余时，降维通常更有效。如果你了解生成数据的过程，并且知道这个过程比较简单，那么流形假设当然成立，降维也很可能有帮助。

希望现在你已经对维度灾难以及降维算法如何应对它有了较好理解，尤其是在流形假设成立时。接下来介绍一些最流行的降维算法。

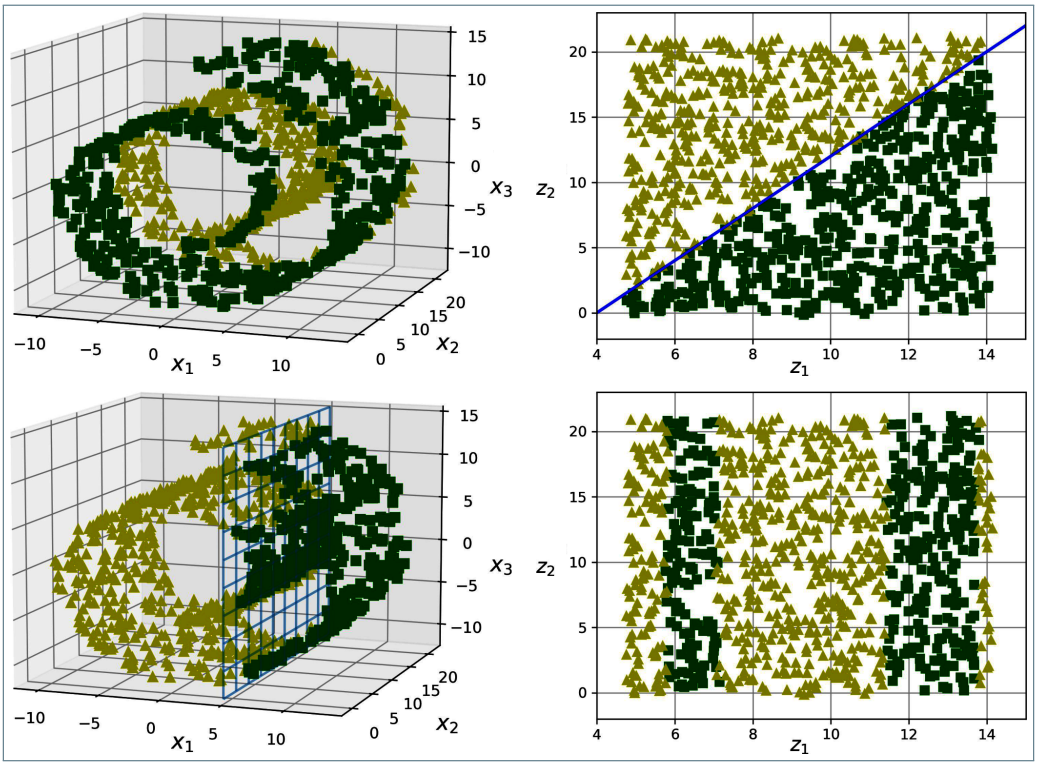

图 7-6: 降到低维后，决策边界不一定更简单（The decision boundary may not always be simpler with lower dimensions）

7.3　PCA

主成分分析（principal component analysis，PCA）是最流行的降维算法。它首先寻找最接近数据的超平面，然后把数据投影到该超平面上，如图 7-2 所示。

保留方差（Preserving the Variance）　在把训练集投影到低维超平面之前，首先需要选择正确的超平面。例如，图 7-7 左侧展示了一个简单二维数据集和三条不同的轴，也就是三个一维超平面；右侧展示了将数据投影到各条轴后的结果。可以看到，投影到实线轴上时保留的方差最大，投影到点线轴上时保留的方差很少，投影到虚线轴上时保留的方差居中。

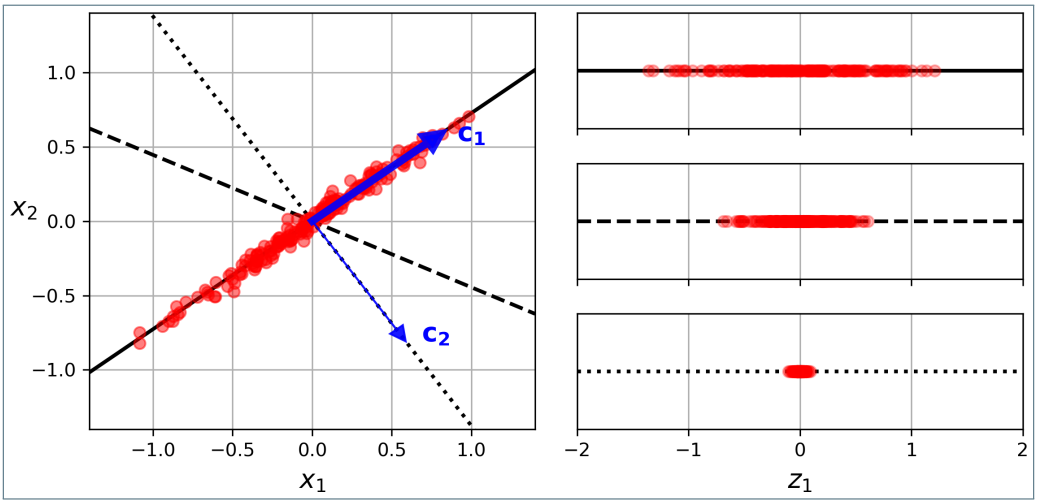

图 7-7: 选择投影所在的子空间（Selecting the subspace on which to project）

选择保留最大方差的轴似乎很合理，因为它很可能损失最少的信息。想象太阳正好位于头顶上方时你在地面上的影子：它只是一个小色块，完全不像你；而日出时你投射在墙上的影子更大，也更像你。选择最大化方差的轴还有另一个理由：它也是使原始数据集与投影之间的均方距离最小的轴。这正是 PCA 背后的简单思想，早在 1901 年就被提出[3]。

主成分（Principal Components） PCA 会寻找训练集中能够解释最大方差的轴。在图 7-7 中，这条轴就是实线。它还会寻找与第一条轴正交、并能解释剩余方差中最大部分的第二条轴。在这个二维例子中没有别的选择，就是点线。如果数据集维度更高，PCA 还会寻找与前两条轴都正交的第三条轴、第四条轴、第五条轴，以此类推，直到数据集的维度数量。

第 i 条轴称为数据的第 i 个主成分（principal component，PC）。在图 7-7 中，第一个主成分是向量 c1 所在的轴，第二个主成分是向量 c2 所在的轴。在图 7-2 中，前两个主成分位于投影平面上，第三个主成分是垂直于该平面的轴。投影后，在图 7-3 中，第一主成分对应 z1 轴，第二主成分对应 z2 轴。

> 注意：对于每个主成分，PCA 都会寻找一个零中心化的单位向量，使其指向该主成分的方向。不幸的是，方向并不保证稳定：如果对训练集稍作扰动再运行 PCA，单位向量可能会指向相反方向。当两个轴上的方差非常接近时，一对单位向量甚至可能旋转或交换。因此，如果把 PCA 作为模型前的预处理步骤，每

[3]Karl Pearson，“On Lines and Planes of Closest Fit to Systems of Points in Space”，The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science 2，第 11 期（1901）：559-572。

> 次更新 PCA 转换器时都必须重新训练整个模型；否则，如果 PCA 输出与之前的版本不一致，模型会非常困惑。

如何找到训练集的主成分？幸运的是，有一种标准的矩阵分解技术，称为奇异值分解（singular value decomposition，SVD）。它可以把训练集矩阵 X 分解为三个矩阵 UΣV^T 的乘积，其中 V 包含按正确顺序排列的单位向量，这些向量定义了我们要寻找的全部主成分，如公式 7-1 所示[4]。

$$V = \begin{pmatrix} | & | & & | \\ c_1 & c_2 & \cdots & c_n \\ | & | & & | \end{pmatrix} \tag{7-1}$$

公式 7-1　主成分矩阵（Principal components matrix）

下面的 Python 代码使用 NumPy 的 svd() 函数，获得图 7-2 所示三维训练集的全部主成分，然后提取定义前两个主成分的两个单位向量：

[4]证明 SVD 恰好给出 PCA 所需主成分，需要特征向量和协方差矩阵等先备数学知识。感兴趣的话，可以参考 Jonathon Shlens 在 2014 年发表的论文。

```
import numpy as np
X = [...]  # create a small 3D dataset
X_centered = X - X.mean(axis=0)
U, s, Vt = np.linalg.svd(X_centered)
c1 = Vt[0]
c2 = Vt[1]
```

> 警告：PCA 假设数据集以原点为中心。正如你将看到的，Scikit-Learn 的 PCA 类会为你完成中心化。如果自己实现 PCA，或使用其他库，记得先对数据进行中心化。

将数据投影到 d 个维度（Projecting Down to d Dimensions） 确定所有主成分后，可以把数据集降到 d 个维度：将数据投影到由前 d 个主成分定义的超平面上，后文会讨论如何选择维度数量 d。选择这个超平面可以确保投影尽可能保留方差。例如，在图 7-2 中，三维数据集投影到由前两个主成分定义的二维平面上，保留了数据集的大部分方差。因此，二维投影看起来与原始三维数据集非常相似。

要把训练集投影到超平面，得到维度为 d 的降维数据集 Xd-proj，需要将训练集矩阵 X 与矩阵 Wd 相乘。Wd 是由 V 的前 d 列组成的矩阵，如公式 7-2 所示。

$$X_{d\text{-proj}} = XW_d \tag{7-2}$$

公式 7-2　将训练集投影到 d 个维度（Projecting the training set down to d dimensions）

下面的 Python 代码把训练集投影到由前两个主成分定义的平面：

```
W2 = Vt[:2].T
X2D = X_centered @ W2
```

这样，你就可以把任意数据集投影到任意数量的维度，同时尽可能多地保留方差。

使用 Scikit-Learn（Using Scikit-Learn） Scikit-Learn 的 PCA 类使用 SVD 实现 PCA，与本章前面手动实现的方法相同。下面的代码应用 PCA，将数据集降到二维；注意，它会自动对数据进行中心化：

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
X2D = pca.fit_transform(X)
```

将 PCA 转换器拟合到数据集后，它的 components_ 属性包含 Wd 的转置：其中每一行对应前 d 个主成分中的一个。

方差解释率（Explained Variance Ratio） 另一个有用的信息是每个主成分的方差解释率（explained variance ratio），可以通过 explained_variance_ratio_ 变量获取。该比例表示数据集方差中沿着每个主成分方向的部分。

例如，看看图 7-2 所示三维数据集前两个主成分的方差解释率：

```
>>> pca.explained_variance_ratio_
array([0.82279334, 0.10821224])
```

这表示数据集约 82% 的方差沿着第一主成分，约 11% 沿着第二主成分。因此第三主成分只剩约 7%，可以合理地认为它携带的信息很少。

选择正确的维度数（Choosing the Right Number of Dimensions） 与其随意选择降到多少维，不如选择能够累积保留足够大方差比例的维度数量，例如 95%。当然，如果降维的目的是数据可视化，就应该把维度降到二维或三维。

下面的代码加载第 3 章介绍过的 MNIST 数据集，并划分训练集和测试集。它先执行不降维的 PCA，再计算保留训练集 95% 方差所需的最小维度数：

```
from sklearn.datasets import fetch_openml
mnist = fetch_openml('mnist_784', as_frame=False)
X_train, y_train = mnist.data[:60_000], mnist.target[:60_000]
X_test, y_test = mnist.data[60_000:], mnist.target[60_000:]
pca = PCA()
pca.fit(X_train)
cumsum = np.cumsum(pca.explained_variance_ratio_)
d = np.argmax(cumsum >= 0.95) + 1  # d equals 154
```

然后可以把 n_components 设为 d，再次运行 PCA，但还有更好的方法。不要指定要保留的主成分数量，而是把 n_components 设为 0.0 到 1.0 之间的浮点数，表示希望保留的方差比例：

```
pca = PCA(n_components=0.95)
X_reduced = pca.fit_transform(X_train)
```

实际主成分数量在训练过程中确定，并存储在 n_components_ 属性中：

```
>>> pca.n_components_
np.int64(154)
```

还可以绘制方差解释率与维度数量的关系图，只需绘制 cumsum 即可，见图 7-8。通常曲线中会出现一个肘部，此处方差解释率的增长速度开始明显变慢。在这个例子中，把维度降到约 100 似乎不会损失太多方差解释率。

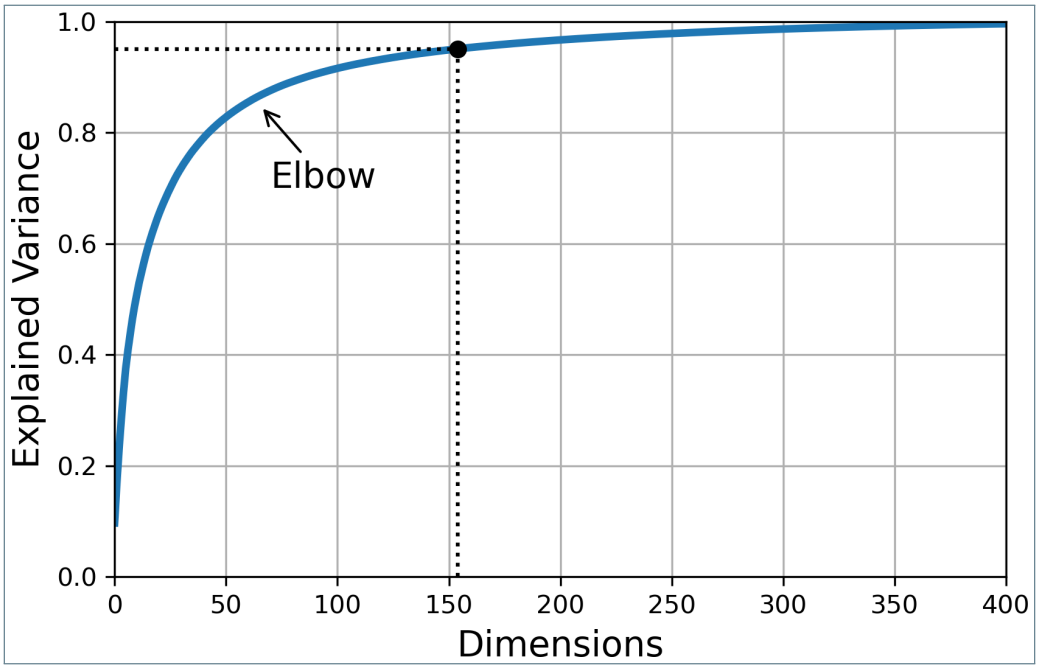

图 7-8: 方差解释率随维度数量的变化（Explained variance as a function of the number of dimensions）

如果把降维作为监督学习任务的预处理步骤，例如分类，那么可以像调节其他超参数一样调节维度数量（见第 2 章）。下面的示例创建一个两步流水线，先使用 PCA 降维，再使用随机森林进行分类；然后使用 RandomizedSearchCV 同时为 PCA 和随机森林分类器寻找较好的超参数组合。这里的搜索很快，只调节两个超参数，只在 1,000 个实例上训练，并运行 10 次迭代。如果有时间，可以执行更全面的搜索：

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV
from sklearn.pipeline import make_pipeline
clf = make_pipeline(PCA(random_state=42),
                    RandomForestClassifier(random_state=42))
param_distrib = {
    "pca__n_components": np.arange(10, 80),
    "randomforestclassifier__n_estimators": np.arange(50, 500)
}
rnd_search = RandomizedSearchCV(clf, param_distrib, n_iter=10, cv=3,
                                random_state=42)
rnd_search.fit(X_train[:1000], y_train[:1000])
```

看看找到的最佳超参数：

```
>>> print(rnd_search.best_params_)
{'randomforestclassifier__n_estimators': np.int64(475),
 'pca__n_components': np.int64(57)}
```

最佳主成分数量很低，这一点很有意思：我们把一个 784 维数据集降到了仅 57 个维度！这与使用随机森林有关，因为随机森林是相当强大的模型。如果改用线性模型，例如 SGDClassifier，搜索会发现需要保留更多维度，大约 75 个。

> 注意：关注的不一定只有模型性能，还可能关注模型大小和速度。维度越少，模型越小，训练和推理越快；但如果数据缩减过度，就会损失太多信号，模型会欠拟合。需要根据具体应用，在速度、大小和性能之间选择合适的平衡。

用 PCA 压缩（PCA for Compression） 降维后，训练集占用的空间会小得多。例如，对 MNIST 应用 PCA，同时保留 95% 的方差后，只剩 154 个特征，而不是原来的 784 个特征。数据集现在不到原始大小的 20%，而只损失了 5% 的方差！这种压缩比很合理，显然可以大幅加快分类算法。

也可以通过对 PCA 投影执行逆变换，把降维后的数据集解压回 784 个维度。由于投影损失了一部分信息，恢复结果不会与原始数据完全相同，但很可能接近原始数据。原始数据与重构数据之间的均方距离称为重构误差（reconstruction error）。

inverse_transform() 方法可以把降维后的 MNIST 数据集解压回 784 个维度：

```
X_recovered = pca.inverse_transform(X_reduced)
```

图 7-9 左侧是原始训练集中的一些数字，右侧是相应数字经过压缩和解压后的结果。可以看到图像质量略有损失，但数字仍然基本保持完整。

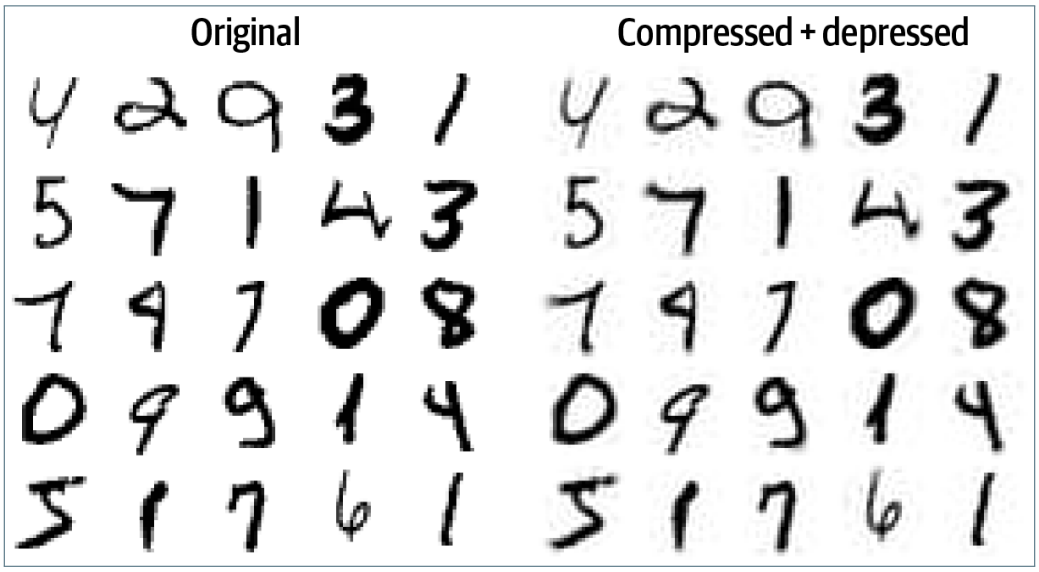

图 7-9: 保留 95% 方差的 MNIST 压缩（MNIST compression that preserves 95% of the variance）

逆变换公式见公式 7-3。

$$X_{\text{recovered}} = X_{d\text{-proj}} W_d^\intercal \tag{7-3}$$

公式 7-3　PCA 逆变换，恢复到原始维度数量（PCA inverse transformation, back to the original number of dimensions）

随机 PCA（Randomized PCA） 如果把 svd_solver 超参数设为"randomized"，Scikit-Learn 会使用一种称为随机 PCA 的随机算法，快速找到前 d 个主成分的近似值。它的计算复杂度为 $O(m \times d^2) + O(d^3)$，而完整 SVD 方法的复杂度为 $O(m \times n^2) + O(n^3)$。当 d 远小于 n 时，随机 PCA 会比完整 SVD 快得多：

```
rnd_pca = PCA(n_components=154, svd_solver="randomized", random_state=42)
X_reduced = rnd_pca.fit_transform(X_train)
```

> 提示：默认情况下，svd_solver 设为"auto"。如果输入数据特征数较少（n < 1,000），并且样本数至少是特征数的 10 倍（m > 10n），就使用"covariance_eigh" 求解器；否则，如果 max(m, n) > 500，并且 n_components 是 min(m, n) 的 80% 以下的整数，就使用"randomized" 求解器。在其他情况下使用完整 SVD。如果想强制使用完整 SVD，以计算时间换取略高精度，可以将 svd_solver 设为"full"。

增量 PCA（Incremental PCA） 前面几种 PCA 实现有一个问题：算法必须把整个训练集装入内存才能运行。幸运的是，已经有增量 PCA（Incremental PCA，IPCA）算法，可以把训练集拆成多个小批量，一次输入一个小批量。这对于大型训练集，以及在新实例到达时在线应用 PCA 很有用[5]。

下面的代码使用 NumPy 的 array_split() 函数把 MNIST 训练集拆成 100 个小批量，并将它们逐个输入 Scikit-Learn 的 IncrementalPCA 类，将 MNIST 数据集降到 154 个维度：

```
from sklearn.decomposition import IncrementalPCA
n_batches = 100
inc_pca = IncrementalPCA(n_components=154)
for X_batch in np.array_split(X_train, n_batches):
    inc_pca.partial_fit(X_batch)
X_reduced = inc_pca.transform(X_train)
```

注意，必须对每个小批量调用 partial_fit()，而不能对整个训练集调用 fit()。

还可以使用 NumPy 的 memmap 类，把磁盘上的大型二进制文件当作内存中的数组来操作；它只会在需要时把所需数据载入内存。先创建一个内存映射文件，把 MNIST 训练集复制进去，再调用 flush()，确保缓存中的数据写入磁盘。在实际应用中，X_train 通常无法装入内存，因此应逐块加载数据，把每块保存到 memmap 数组的正确位置：

[5] Scikit-Learn 使用 David A. Ross 等人在"Incremental Learning for Robust Visual Tracking"中描述的算法，论文发表于 International Journal of Computer Vision 77，第 1-3 期（2008）：125-141。

```
filename = "my_mnist.mmap"
X_mmap = np.memmap(filename, dtype='float32', mode='write', shape=X_train.shape)
X_mmap[:] = X_train  # could be a loop instead, saving the data chunk by chunk
X_mmap.flush()
```

接下来加载这个 memmap 文件，并像普通 NumPy 数组一样使用它。由于这个算法任意时刻只使用数组的一小部分，因此内存占用仍然可控，也可以使用常规 fit() 方法，而不必调用 partial_fit()：

```
X_mmap = np.memmap(filename, dtype="float32", mode="readonly").reshape(-1, 784)
batch_size = X_mmap.shape[0] // n_batches
inc_pca = IncrementalPCA(n_components=154, batch_size=batch_size)
inc_pca.fit(X_mmap)
```

> 警告：磁盘上只保存了原始二进制数据，因此重新加载数组时必须指定数据类型和形状。如果省略 shape，np.memmap() 会返回一维数组。

对于维度非常高的数据集，PCA 可能太慢。即使使用随机 PCA，计算复杂度仍为 $O(m \times d^2) + O(d^3)$，因此目标维度 d 不能太大。如果数据集有数万甚至更多特征，例如图像，训练可能会变得太慢，此时应考虑使用随机投影。

7.4　随机投影（Random Projection）

顾名思义，随机投影算法使用随机线性投影把数据投影到低维空间。这听起来很疯狂，但事实证明，随机投影很可能能够较好地保留距离，这是 William B. Johnson 和 Joram Lindenstrauss 在著名引理中数学证明的结果。因此，相似实例在投影后仍然相似，差异很大的实例仍然相差很远。

显然，删除的维度越多，损失的信息越多，距离失真也越严重。那么，如何选择最佳维度数量？Johnson 和 Lindenstrauss 给出了一个公式，它确定了为保证距离变化不超过给定容差、并且以较高概率保留距离所需的最小维度数量。例如，一个数据集包含 m = 5,000 个实例，每个实例包含 n = 20,000 个特征；如果不希望任意两个实例之间的平方距离变化超过 ε = 10%[6]，就应把数据投影到 d 个维度，其中 $d \geqslant 4\log(m) / (\frac{1}{2}\varepsilon^2 - \frac{1}{3}\varepsilon^3)$，计算得到 7,300 个维度。这个降维幅度相当可观！注意，这个公式不使用 n，只依赖 m 和 ε。该公式由 johnson_lindenstrauss_min_dim() 函数实现：

[6] ε 是希腊字母 epsilon，通常用于表示很小的值。

```
>>> from sklearn.random_projection import johnson_lindenstrauss_min_dim
>>> m, ε = 5_000, 0.1
>>> d = johnson_lindenstrauss_min_dim(m, eps=ε)
>>> d
7300
```

现在只需生成一个形状为 [d, n] 的随机矩阵 P，其中每个元素都从均值为 0、方差为 1 / d 的高斯分布中随机抽取，再利用它把数据从 n 维投影到 d 维：

```
n = 20_000
rng = np.random.default_rng(seed=42)
P = rng.standard_normal((d, n)) / np.sqrt(d)  # std dev = sqrt(variance)
X = rng.standard_normal((m, n))  # generate a fake dataset
X_reduced = X @ P.T
```

就是这么简单高效！创建随机矩阵所需的只是数据集的形状，完全不需要使用数据本身。因此，随机投影特别适合文本或基因组数据这类特征数达到数百万的超高维数据，以及非常稀疏的数据，因为随机 PCA 可能训练时间过长、内存需求过高。在推理阶段，随机投影和 PCA 一样快，也就是一次矩阵乘法。不过，随机投影不是万能方案：它比 PCA 多损失一点信号，因此需要在训练速度和性能之间权衡。

Scikit-Learn 提供 GaussianRandomProjection 类来完成同样的工作。调用 fit() 时，它使用 johnson_lindenstrauss_min_dim() 确定输出维度，然后生成随机矩阵，并把它存储在 components_ 属性中。调用 transform() 时，它用该矩阵执行投影。创建转换器时，可以设置 eps 来调节 ε，默认值为 0.1；也可以设置 n_components 强制指定目标维度 d。通常应使用交叉验证微调这些超参数。下面的代码与前面手动实现的结果相同，也可以验证 gaussian_rnd_proj.components_ 与 P 相等：

```
from sklearn.random_projection import GaussianRandomProjection
gaussian_rnd_proj = GaussianRandomProjection(eps=ε, random_state=42)
X_reduced = gaussian_rnd_proj.fit_transform(X)  # same result as above
```

Scikit-Learn 还提供了第二个随机投影转换器 SparseRandomProjection。它以相同方式确定目标维度，生成相同形状的随机矩阵并执行投影，主要区别是随机矩阵是稀疏的。因此它占用的内存少得多：在前面例子中约 25 MB，而不是接近 1.2 GB；生成随机矩阵和降维也更快，这个例子中大约快 50%。此外，如果输入是稀疏的，转换结果也会保持稀疏，除非设置 dense_output=True。它同样具有保留距离的性质，降维质量也相当，只是精度略低。总之，尤其对于大型或稀疏数据集，通常更适合使用这个转换器。

稀疏随机矩阵中非零元素的比例称为密度（density）。默认情况下，密度等于 1 / sqrt(n)。当特征数为 20,000 时，随机矩阵中约每 141 个单元格只有一个非零值，非常稀疏！如果需要，可以把 density 超参数设置为其他值。稀疏随机矩阵中的每个单元格以概率 r 非零；每个非零值为 -v 或 +v，且两者概率相同，其中 v = 1 / sqrt(d r)。

如果想执行逆变换，首先需要使用 SciPy 的 pinv() 函数计算 components_ 矩阵的伪逆，然后把降维数据与伪逆的转置相乘：

```
components_pinv = np.linalg.pinv(gaussian_rnd_proj.components_)
X_recovered = X_reduced @ components_pinv.T
```

> 警告：如果 components_ 矩阵很大，计算伪逆可能需要很长时间，因为 pinv() 的计算复杂度为 $O(dn^2)$（当 d < n 时），或者 $O(nd^2)$（其他情况）。

总之，随机投影是一种简单、快速、节省内存而且出人意料地强大的降维算法，尤其是在处理高维数据集时值得记住。

> 注意：随机投影并不总是用于降低大型数据集的维度。例如，Sanjoy Dasgupta 等人在 2017 年的论文中指出，果蝇的大脑实现了随机投影的一种模拟版本，把稠密低维嗅觉输入映射成稀疏的高维二值输出[a]。这与一种著名算法局部敏感哈希（locality-sensitive hashing，LSH）相似，搜索引擎通常使用 LSH 对相似文档分组（见第 17 章）。
>
> [a] Sanjoy Dasgupta 等，“A neural algorithm for a fundamental computing problem”，Science 358，第 6364 期（2017）：793-796。

7.5　LLE

局部线性嵌入（locally linear embedding，LLE）[7] 是一种非线性降维（nonlinear dimensionality reduction，NLDR）技术。它是一种不依赖投影的流形学习技术，与 PCA 和随机投影不同。简单来说，LLE 首先确定每个训练实例与最近邻之间的局部线性关系，然后寻找一个低维训练集表示，使这些局部关系得到尽可能好的保留。这个方法特别擅长展开扭曲的流形，尤其是在噪声不太多时。不过，它的扩展性不好，主要适用于小型或中型数据集。

下面的代码生成一个 Swiss roll，然后使用 Scikit-Learn 的 LocallyLinearEmbedding 类将其展开：

```
from sklearn.datasets import make_swiss_roll
from sklearn.manifold import LocallyLinearEmbedding
X_swiss, t = make_swiss_roll(n_samples=1000, noise=0.2, random_state=42)
lle = LocallyLinearEmbedding(n_components=2, n_neighbors=10, random_state=42)
X_unrolled = lle.fit_transform(X_swiss)
```

变量 t 是一个一维 NumPy 数组，包含每个实例沿卷曲轴的位置。这个示例不使用它，但可以把它作为非线性回归任务的目标。结果如图 7-10 所示。

[7] Sam T. Roweis 和 Lawrence K. Saul，“Nonlinear Dimensionality Reduction by Locally Linear Embedding”，Science 290，第 5500 期（2000）：2323-2326。

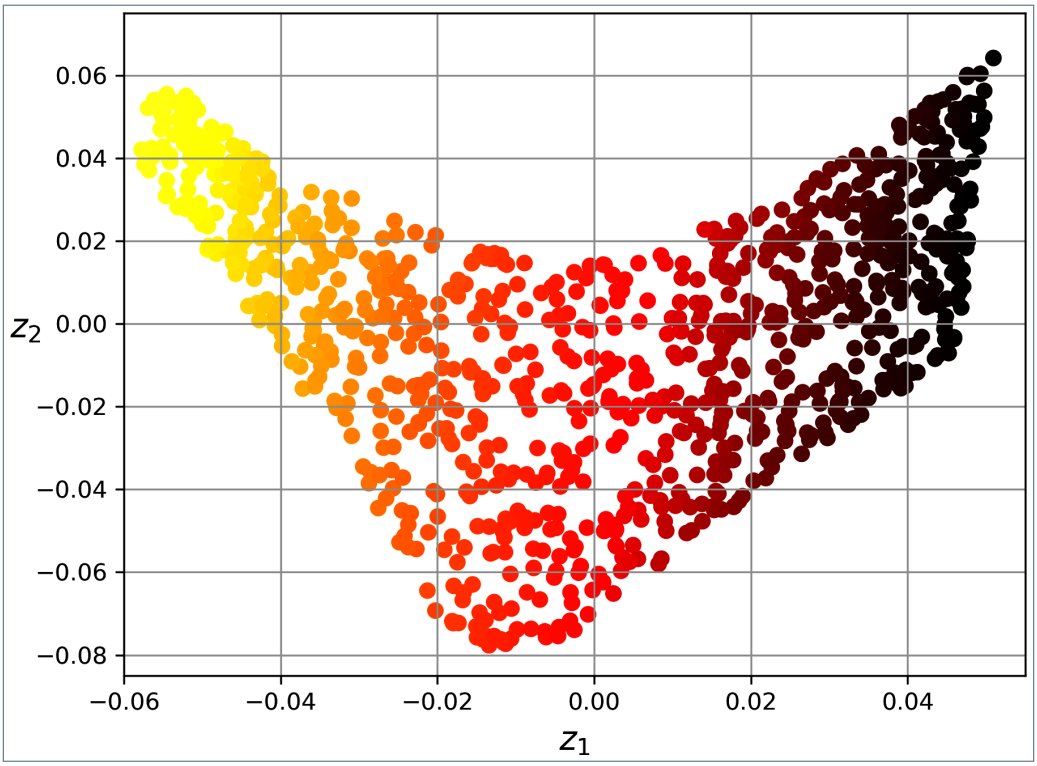

图 7-10: 使用 LLE 展开的 Swiss roll（Unrolled Swiss roll using LLE）

可以看到，Swiss roll 被完全展开，实例之间的距离在局部得到了很好的保留。不过，大尺度距离没有保留：展开后的 Swiss roll 应该是一个矩形，而不是这种拉伸、扭曲的带状结构。尽管如此，LLE 仍然很好地对流形进行了建模。

LLE 的工作方式如下：对于每个训练实例 x(i)，算法识别它的 k 个最近邻，在前面的代码中 k = 10，然后尝试把 x(i) 重构为这些邻居的线性函数。更具体地说，它寻找权重 wi,j，使 x(i) 与邻居加权和之间的平方距离尽可能小；如果 x(j) 不是 x(i) 的 k 个最近邻之一，则令 wi,j = 0。因此，LLE 的第一步是公式 7-4 所描述的受约束优化问题，其中 W 是包含全部 wi,j 的权重矩阵。第二个约束只是对每个训练实例 x(i) 的权重进行归一化。

$$\hat{W} = \arg\min_{W} \sum_{i=1}^{m} \left\| \mathbf{x}^{(i)} - \sum_{j=1}^{m} w_{i,j} \mathbf{x}^{(j)} \right\|^2 \quad \text{s.t.} \begin{cases} w_{i,j} = 0 & \text{if } j \notin \mathcal{N}_k(i) \\ \sum_{j=1}^{m} w_{i,j} = 1 & \text{for } i = 1, 2, \ldots, m \end{cases} \tag{7-4}$$

公式 7-4　LLE 第一步：线性建模局部关系（LLE step 1: linearly modeling local relationships）

这一步得到的权重矩阵 W-hat，也就是包含权重 w-hat-i,j 的矩阵，编码了训练实例之间的局部线性关系。第二步是把训练实例映射到 d 维空间（d < n），同时尽可能保留这些局部关系。如果 z(i) 是 x(i) 在这个 d 维空间中的映像，那么希望 z(i) 与邻居映像的加权和之间的平方距离尽可能小。这个思想得到公式 7-5 所描述的无约束优化问题。它与第一步非常相似，但第一步固定实例并寻找最佳权重，第二步则反过来固定权重并寻找实例映像在低维空间中的最佳位置。注意，Z 是包含所有 z(i) 的矩阵。

$$\hat{Z} = \underset{Z}{\arg\min} \sum_{i=1}^{m} \left\| \mathbf{z}^{(i)} - \sum_{j=1}^{m} \hat{w}_{i,j} \mathbf{z}^{(j)} \right\|^2 \tag{7-5}$$

公式 7-5　LLE 第二步：保留关系并降低维度 (LLE step 2: reducing dimensionality while preserving relationships)

Scikit-Learn 的 LLE 实现具有以下计算复杂度：寻找 k 个最近邻为 O(m log(m) n log(k))；优化权重为 O(m n k^3)；构建低维表示为 O(d m^2)。最后一项中的 m^2 使该算法无法很好地扩展到非常大的数据集。

如你所见，LLE 与投影技术非常不同，复杂度明显更高，但它也能构造更好的低维表示，尤其是在数据非线性时。

7.6　其他降维技术（Other Dimensionality Reduction Techniques）

在结束本章前，快速看看 Scikit-Learn 中另外几种流行的降维技术：

`sklearn.manifold.MDS` 多维尺度分析（multidimensional scaling，MDS）在降低维度的同时尝试保留实例之间的距离。随机投影也会对高维数据这样做，但在低维数据上效果不好。

`sklearn.manifold.Isomap` Isomap 通过把每个实例与最近邻连接来创建图，然后在降低维度的同时尝试保留实例之间的测地距离。图中两个节点之间的测地距离，是连接这两个节点的最短路径上的节点数量。当数据位于一个较平滑、低维且具有单一全局结构的流形上时，例如 Swiss roll，这种方法效果最好。

`sklearn.manifold.TSNE` t 分布随机邻域嵌入（t-distributed stochastic neighbor embedding，t-SNE）在降低维度时试图让相似实例靠近、不同实例远离。它主要用于

可视化，尤其是可视化高维空间中的实例聚类。例如，本章末尾的练习会使用 t-SNE 为 MNIST 图像生成二维地图。不过，t-SNE 不适合作为机器学习模型的预处理阶段。

`sklearn.discriminant_analysis.LinearDiscriminantAnalysis` 线性判别分析（linear discriminant analysis，LDA）是一种线性分类算法，在训练过程中学习类别之间最有区分度的轴。这些轴可以用来定义一个超平面，把数据投影到该平面上。这个方法的好处是投影会让类别之间尽可能远，因此 LDA 是运行其他分类算法前降低维度的好技术，除非单独使用 LDA 已经足够。

> 提示：统一流形近似与投影（Uniform Manifold Approximation and Projection，UMAP）是另一种流行的可视化降维技术。t-SNE 更擅长保留局部结构，尤其是聚类；UMAP 则尝试同时保留局部和全局结构，而且在大型数据集上扩展性更好。遗憾的是，UMAP 不在 Scikit-Learn 中，但 umap-learn 软件包提供了良好实现。

图 7-11 展示了使用 MDS、Isomap 和 t-SNE 把 Swiss roll 降到二维的结果。MDS 在保留全局曲率的同时展平了 Swiss roll；Isomap 则完全丢弃了曲率。根据下游任务的不同，保留大尺度结构可能有利，也可能有害。t-SNE 展开 Swiss roll 的效果不错，保留了一部分曲率，同时放大了聚类，把卷曲结构撕开。同样，这样做是否有利取决于下游任务。

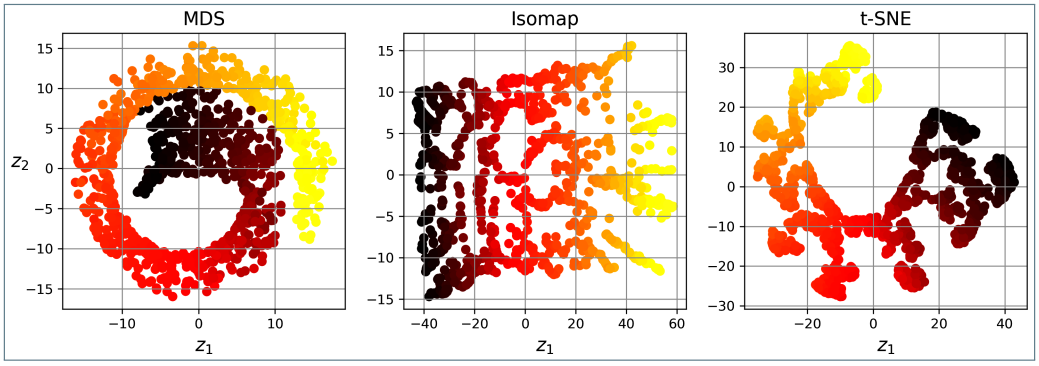

图 7-11: 使用各种技术将 Swiss roll 降到二维（Using various techniques to reduce the Swiss roll to 2D）

7.7　练习（Exercises）

1. 降低数据集维度的主要动机是什么？主要缺点是什么？

2. 什么是维度灾难？
3. 数据集维度降低后，能否反向执行这个操作？如果可以，如何做？如果不可以，为什么？
4. PCA 能否用于降低高度非线性数据集的维度？
5. 假设你在一个 1,000 维数据集上执行 PCA，并将方差解释率设为 95%。降维后的数据集有多少个维度？
6. 哪些情况下应该使用普通 PCA、增量 PCA、随机 PCA 或随机投影？
7. 如何在你的数据集上评估降维算法的性能？
8. 把两种不同的降维算法串联起来有意义吗？
9. 加载第 3 章介绍过的 MNIST 数据集，并划分训练集和测试集，前 60,000 个实例用于训练，剩余 10,000 个用于测试。先在数据集上训练随机森林分类器并计时，然后在测试集上评估模型。接着使用 PCA 将数据集降到保留 95% 方差的维度，再训练一个新的随机森林分类器并观察训练时间。训练是否快了很多？在测试集上评估分类器，与之前的分类器相比如何？再使用 SGDClassifier 试一次，此时 PCA 有多大帮助？
10. 使用 t-SNE 把 MNIST 数据集的前 5,000 幅图像降到二维，并使用 Matplotlib 绘制结果。可以用 10 种不同颜色的散点表示每幅图像的目标类别；也可以用图像类别，也就是 0 到 9 的数字，替代散点；还可以绘制图像本身的缩小版本。如果绘制所有数字，图像会过于拥挤，因此应随机抽样，或者只有在某个实例附近尚未绘制其他实例时才绘制它。你应该能得到一个聚类分离良好的数字可视化结果。再尝试 PCA、LLE 或 MDS 等其他降维算法，并比较得到的可视化结果。

本章练习的解答位于本章 notebook 的末尾：https://homl.info/colab-p。

附加说明：关于增量 PCA、随机投影和 LLE 的论文出处已在正文脚注中保留；原书对这些算法的复杂度、数据规模和适用条件的说明也已完整保留。

第 8 章　无监督学习技术

Unsupervised Learning Techniques

图灵奖得主、Meta 首席 AI 科学家 Yann LeCun 曾经说过：“如果把智能比作一块蛋糕，那么无监督学习就是蛋糕本身，监督学习是蛋糕上的糖霜，而强化学习是蛋糕上的樱桃。”（NeurIPS 2016）换句话说，无监督学习蕴含着巨大潜力，而我们才刚刚开始探索其中的一小部分。事实上，现有数据中绝大多数都没有标签：我们有输入特征 X，却没有标签 y。

假设你想创建一个系统，为生产线上的每种产品拍摄几张照片，并检测哪些产品有缺陷。创建一个每天自动拍摄成千上万张照片的系统并不难，几周内就能构建出相当大的数据集。但问题是没有标签！如果想训练一个普通的二分类器来预测产品是否有缺陷，就必须把每张图片标记为“有缺陷”或“正常”。这通常需要人类专家坐下来逐张查看所有图片，过程漫长、昂贵且枯燥，因此通常只能标记一小部分图片。结果是，带标签的数据集很小，分类器性能令人失望。而且每当公司对产品做出任何改变，都必须从头开始整个过程。如果算法能直接利用未标记数据，不需要人类给每张图片标注，那不是很好吗？这就是无监督学习。

第 7 章介绍了最常见的无监督学习任务：降维。本章将讨论更多无监督任务：

聚类（Clustering）

目标是把相似实例归入同一簇。聚类是数据分析、客户分群、推荐系统、搜索引擎、图像分割、半监督学习、降维等任务的有力工具。

异常检测，也称离群点检测（Anomaly detection / outlier detection）

目标是学习“正常”数据的样子，然后用它检测异常实例。这些实例称为异常或离群点，正常实例称为内点。异常检测可以应用于欺诈检测、制造业缺陷产品检测、识别时间序列中的新趋势，或者在训练其他模型前删除离群点，从而显著提高结果模型的性能。

密度估计（Density estimation）

这是估计生成数据集的随机过程的概率密度函数（PDF）的任务[1]。密度估计常用于异常检测：位于极低密度区域的实例很可能是异常。它对数据分析和可视化也很有用。

准备好吃蛋糕了吗？我们先从两个聚类算法 k-means 和 DBSCAN 开始，然后讨论高斯混合模型，并了解它们如何用于密度估计、聚类和异常检测。

8.1 聚类算法：k-means 和 DBSCAN（Clustering Algorithms: k-means and DBSCAN）

在山间徒步时，你偶然发现一种从未见过的植物。环顾四周，又看到几株类似的植物。它们并不完全相同，但相似程度足以让你判断它们很可能属于同一个物种，至少属于同一个属。也许需要植物学家告诉你它的具体物种，但你不需要专家就能识别外观相似的物体群。这称为聚类：识别相似实例，并将它们分配到由相似实例构成的簇或组中。

与分类一样，每个实例都会被分配到一个组。不过，聚类是无监督任务，没有标签，因此算法必须自行找出如何对实例分组。考虑图 8-1：左侧是第 4 章介绍过的 iris 数据集，每个实例的物种，也就是类别，用不同标记表示。这是一个带标签的数据集，适合使用逻辑回归、SVM 或随机森林分类器等分类算法。右侧是同一个数据集，但去除了标签，此时不能再使用分类算法。聚类算法可以很容易地发现左下方的簇；我们肉眼也能看出来，但右上方的簇由两个不同子簇组成，这一点就不那么明显了。数据集还包含未在图中表示的两个特征，即萼片长度和宽度；聚类算法可以利用所有特征，因此实际上能很好地识别出三个簇。例如，使用高斯混合模型时，150 个实例中只有 5 个被分配到错误的簇。

[1]如果不熟悉概率论，强烈推荐 Khan Academy 的免费在线课程。

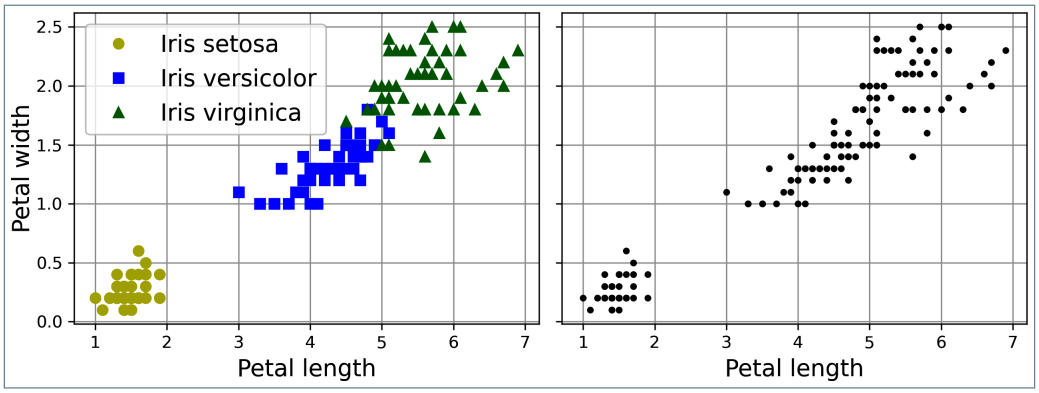

图 8-1: 分类（左）与聚类（右）：聚类没有标签，算法必须在没有指导的情况下识别簇（Classification versus clustering）

聚类有许多应用，包括：

客户分群

可以根据客户的购买记录和网站活动进行聚类。这有助于了解客户是谁、需要什么，从而针对每个群体调整产品和营销活动。例如，客户分群可以用于推荐系统，为用户推荐同一簇中其他用户喜欢的内容。

数据分析

分析新数据集时，先运行聚类算法，再分别分析每个簇，通常很有帮助。

降维

数据集聚类后，通常可以度量每个实例与每个簇的亲和度。亲和度是衡量实例与某个簇匹配程度的指标。之后可以把每个实例的特征向量 x 替换为它对各簇的亲和度向量。如果有 k 个簇，新向量就是 k 维的。它通常比原始特征向量的维度低得多，同时可能保留足够的信息供后续处理。

特征工程

簇亲和度经常可以作为额外特征。例如，在第 2 章中使用 k-means 为加利福尼亚住房数据集添加地理簇亲和度特征后，模型性能得到了提升。

异常检测，也称离群点检测

对所有簇的亲和度都很低的实例很可能是异常。例如，如果根据网站用户的行为进行聚类，就可以检测出每秒请求数异常高等行为异常的用户。

半监督学习

如果只有少量标签，可以先聚类，再把标签传播给同一簇中的所有实例。这种技术可以大幅增加后续监督学习算法可用的标签数量，从而改善性能。

搜索引擎 有些搜索引擎允许搜索与参考图像相似的图片。要构建这样的系统，首先对数据库中的所有图像应用聚类算法，使相似图像落入同一簇。当用户提供参考图像时，只需用训练好的聚类模型找到它所属的簇，再返回该簇中的所有图像。

图像分割 按照颜色对像素进行聚类，再把每个像素的颜色替换为其所在簇的平均颜色，可以大幅减少图像中不同颜色的数量。图像分割被用于许多目标检测和跟踪系统，因为它让检测每个物体的轮廓更容易。

对于簇并没有统一定义：它取决于上下文，不同算法会捕捉不同类型的簇。有些算法寻找围绕某个点，也就是质心的实例；另一些算法寻找由密集实例组成的连续区域，这种簇可以呈现任意形状；还有些算法是层次化的，会寻找簇的簇。可选方法还有很多。

本节将介绍两种流行的聚类算法 k-means 和 DBSCAN，并探索它们在非线性降维、半监督学习和异常检测中的一些应用。

k-Means 聚类（k-Means Clustering） 考虑图 8-2 所示的未标记数据集，你可以清楚看到五个实例团块。k-means 算法非常简单，可以快速高效地对这类数据集聚类，通常只需几次迭代。它由 Bell Labs 的 Stuart Lloyd 于 1957 年提出，最初用于脉冲编码调制，但直到 1982 年才在公司外发表[2]。1965 年，Edward W. Forgy 发表了几乎相同的算法，因此 k-means 有时也称为 Lloyd-Forgy 算法。

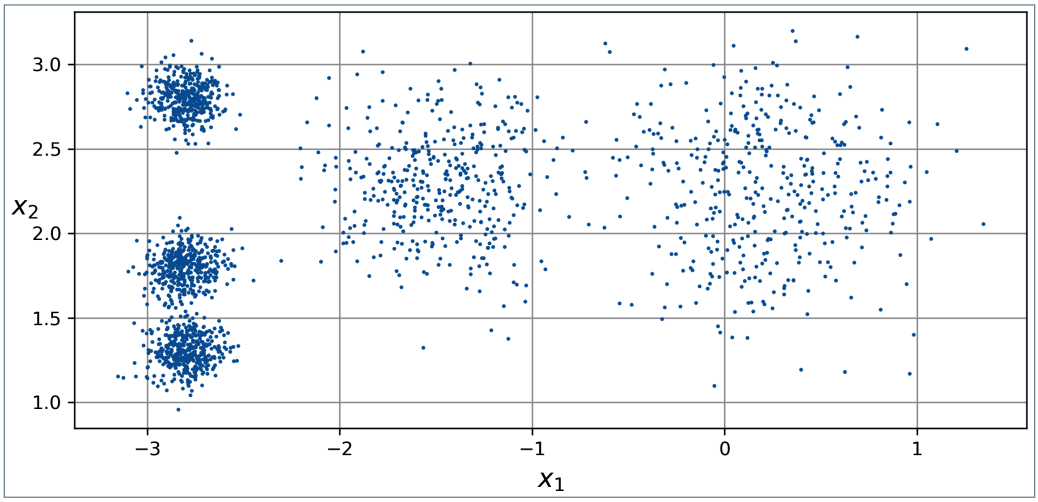

图 8-2: 由五个实例团块组成的未标记数据集（An unlabeled dataset composed of five blobs of instances）

[2]Stuart P. Lloyd, “Least Squares Quantization in PCM”, IEEE Transactions on Information Theory 28, 第 2 期（1982）：129-137。

在这个数据集上训练 k-means 聚类器。它会尝试找到每个团块的中心，并把每个实例分配给最近的团块：

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
X, y = make_blobs([...])  # make the blobs: y contains the cluster IDs, but we
                          # will not use them; that's what we want to predict
k = 5
```

```
kmeans = KMeans(n_clusters=k, random_state=42)
y_pred = kmeans.fit_predict(X)
```

必须指定算法要找出的簇数量 k。在这个例子中，观察数据即可看出 k 应设为 5，但一般情况下并不容易，我们稍后会讨论。

每个实例都会被分配到五个簇中的一个。在聚类语境中，实例的标签是算法为它分配的簇索引，不要把它与分类中的类别标签混淆，后者用作目标值；聚类是无监督学习任务。

KMeans 实例会保留训练实例的预测标签，可以通过 labels_ 实例变量访问：

```
>>> y_pred
array([4, 0, 1, ..., 2, 1, 0], dtype=int32)
>>> y_pred is kmeans.labels_
True
```

还可以查看算法找到的五个质心：

```
>>> kmeans.cluster_centers_
array([[-2.80389616,  1.80117999],
       [ 0.20876306,  2.25551336],
       [-2.79290307,  2.79641063],
       [-1.46679593,  2.28585348],
       [-2.80037642,  1.30082566]])
```

可以把新实例分配给质心最近的簇：

```
>>> import numpy as np
>>> X_new = np.array([[0, 2], [3, 2], [-3, 3], [-3, 2.5]])
>>> kmeans.predict(X_new)
array([1, 1, 2, 2], dtype=int32)
```

绘制簇的决策边界会得到 Voronoi 镶嵌：见图 8-3，其中每个质心用星号表示。

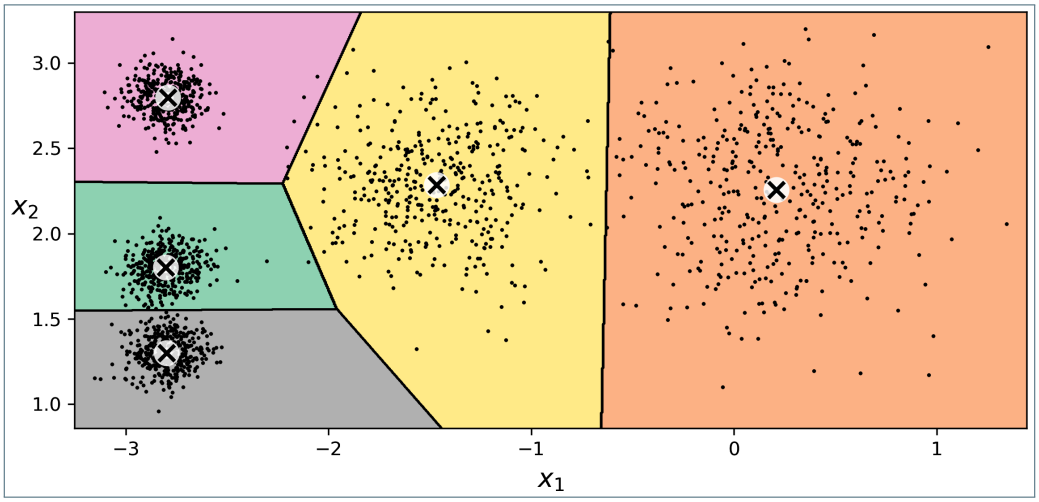

图 8-3: k-means 决策边界（Voronoi 镶嵌）（k-means decision boundaries）

绝大多数实例都被清楚地分配到了合适的簇，但有些实例可能被错误标记，尤其是左上方簇与中央簇之间的边界附近。当团块直径差异很大时，k-means 的表现确实不好，因为它把实例分配给簇时只关心实例到质心的距离。

与其把每个实例分配给单个簇，也就是硬聚类，不如为每个实例给出针对每个簇的分数，这称为软聚类。分数可以是实例与质心之间的距离，或者是相似度、亲和度，例如第 2 章使用过的高斯径向基函数。

在 KMeans 类中，transform() 方法计算每个实例到每个质心的距离：

```
>>> kmeans.transform(X_new).round(2)
array([[2.81, 0.33, 2.9 , 1.49, 2.89],
       [5.81, 2.8 , 5.85, 4.48, 5.84],
       [1.21, 3.29, 0.29, 1.69, 1.71],
       [0.73, 3.22, 0.36, 1.55, 1.22]])
```

在这个例子中，X_new 中第一个实例到第一个质心的距离约为 2.81，到第二个质心的距离为 0.33，到第三、第四和第五个质心的距离分别为 2.90、1.49 和 2.89。如果数据集是高维的，以这种方式转换后会得到一个 k 维数据集，这是一种非常高效

的非线性降维技术。还可以把这些距离作为额外特征，训练另一个模型，就像第 2 章所做的那样。

k-means 算法（The k-means Algorithm） 算法是如何工作的？假设给定质心，就可以把每个实例分配给最近质心所在的簇，从而轻松标记所有实例。反过来，如果给定所有实例的标签，也可以通过计算簇内实例的平均值轻松定位每个簇的质心。但现在标签和质心都未知，应该怎么办？

先随机放置质心，例如从数据集中随机选择 k 个实例，把它们的位置作为质心。然后为实例标记、更新质心，再标记实例、更新质心，如此循环，直到质心不再移动。算法保证在有限步骤内收敛，通常所需步骤很少。这是因为实例与最近质心之间的平均平方距离在每一步都不会增加，而它不可能为负，因此一定会收敛。

图 8-4 展示了算法的运行过程：先随机初始化质心（左上），然后给实例标记（右上），更新质心（左中），重新给实例标记（右中），以此类推。可以看到，仅经过三次迭代，算法就达到看起来接近最优的聚类结果。

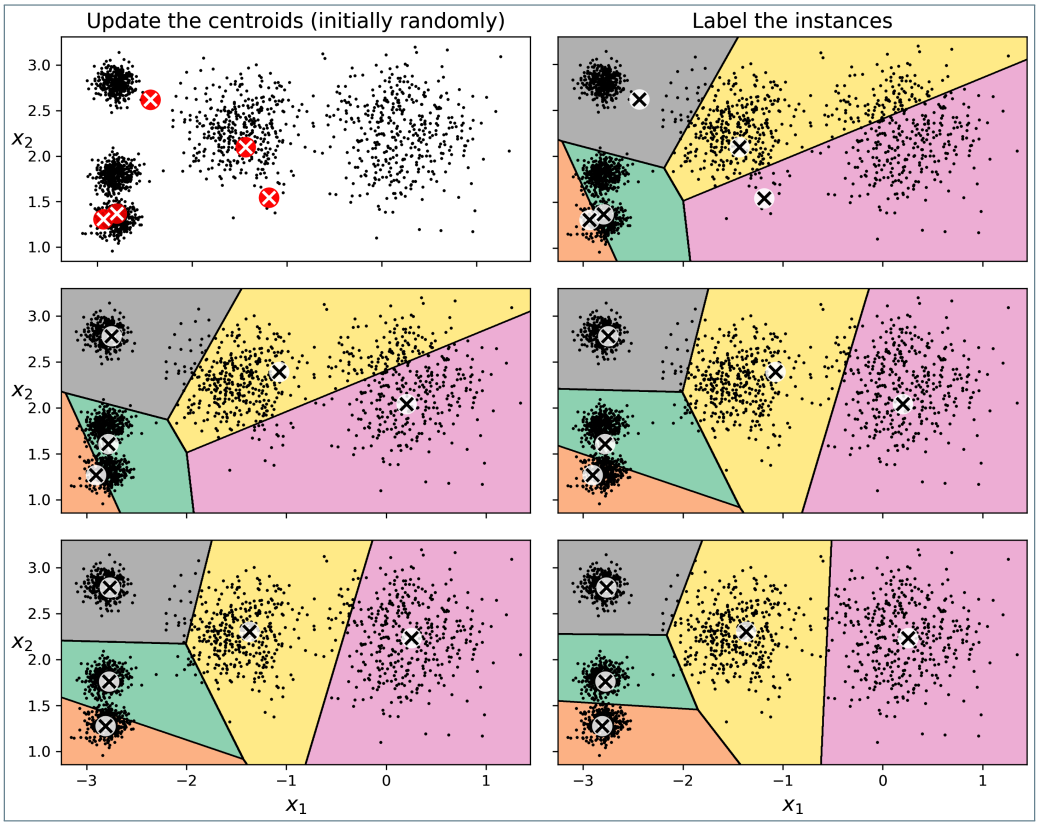

图 8-4: k-means 算法（The k-means algorithm）

> 注意：当数据具有聚类结构时，算法的计算复杂度通常与实例数量 m、簇数量 k 和维度数量 n 呈线性关系。如果数据没有聚类结构，最坏情况下复杂度可能随实例数量呈指数增长。实践中这种情况很少发生，k-means 通常仍是速度最快的聚类算法之一。

虽然算法保证收敛，但不一定收敛到正确的解，也就是可能收敛到局部最优；是否如此取决于质心初始化。图 8-5 展示了随机初始化不理想时，算法可能收敛到的两个次优解。

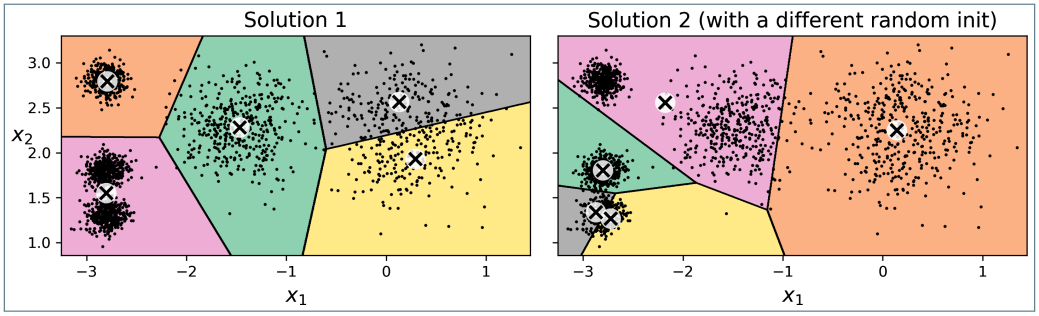

图 8-5: 质心初始化不理想导致的次优解（Suboptimal solutions due to unlucky centroid initializations）

下面看看几种通过改进质心初始化来降低这种风险的方法。

质心初始化方法（Centroid Initialization Methods） 如果大致知道质心应该在哪里，例如之前运行过另一个聚类算法，就可以把 init 超参数设为包含质心列表的 NumPy 数组：

```
good_init = np.array([[-3, 3], [-3, 2], [-3, 1], [-1, 2], [0, 2]])
kmeans = KMeans(n_clusters=5, init=good_init, random_state=42)
kmeans.fit(X)
```

另一种方法是使用不同的随机初始化运行算法多次，然后保留最佳解。随机初始化次数由 n_init 超参数控制。当 init="random" 时，默认值为 10，也就是调用 fit() 时运行前面描述的完整算法 10 次，Scikit-Learn 保留最佳解。但它如何知道哪个解最好？它使用一个性能指标，称为模型的惯性（inertia），定义见公式 8-1。

$$\text{inertia} = \sum_{i} \left\| \mathbf{x}^{(i)} - \mathbf{c}^{(i)} \right\|^2 \tag{8-1}$$

公式 8-1　模型惯性：实例到模型预测的最近质心之间的平方距离之和（A model's inertia）

模型在图 8-5 左侧的惯性约为 219.6，右侧约为 600.4，图 8-3 中的模型只有 211.6。KMeans 类运行 n_init 次初始化算法，并保留惯性最低的模型。在这个例子中，会选择图 8-3 中的模型，除非 n_init 次连续随机初始化都非常不走运。模型的惯性可以通过 inertia_ 实例变量访问：

```
>>> kmeans.inertia_
211.59853725816828
```

score() 方法返回负惯性。之所以为负，是因为预测器的 score() 方法必须遵守 Scikit-Learn 的"越大越好"规则：

```
>>> kmeans.score(X)
-211.59853725816828
```

2006 年，David Arthur 和 Sergei Vassilvitskii 在论文中提出了 k-means++[3]，改进了初始化步骤，使其倾向于选择彼此距离较远的质心。这一变化大幅提高了找到所有重要簇的可能性，降低了收敛到次优解的可能性，就像把捕鱼船分散开会增加发现更多鱼群的概率。虽然智能初始化需要额外计算，但非常值得，因为它能大幅减少找到最优解所需的算法运行次数。k-means++ 初始化算法如下：

1. 从数据集中均匀随机选择一个质心 c(1)。
2. 选择新质心 c(i)，从实例 x(i) 中抽样，其概率为 $D(x(i))^2 / \Sigma j=1m\ D(x(j))^2$。其中 D(x(i)) 是实例 x(i) 与已经选定质心之间的最近距离。这个概率分布确保远离现有质心的实例更可能被选为质心。
3. 重复上一步，直到选择出全部 k 个质心。

当设置 init="k-means++"，也就是默认值时，KMeans 类实际上使用一种称为贪心 k-means++ 的变体：每次迭代抽取多个候选质心，选择其中最佳者。使用该算法时，n_init 默认为 1。

[3]David Arthur 和 Sergei Vassilvitskii，"k-Means++: The Advantages of Careful Seeding"，Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms（2007）：1027-1035。

加速 k-means 与小批量 k-means（Accelerated k-means and Mini-Batch k-means） 2003 年，Charles Elkan 提出了 k-means 的另一种改进[4]。在包含许多簇的大型数据集上，可以通过避免许多不必要的距离计算来加速算法。Elkan 利用三角不等式，也就是直线总是两点之间最短距离[5]，并维护实例与质心之间距离的上下界。不过，Elkan 算法并不总能加快训练，有时甚至会显著拖慢训练，具体取决于数据集。如果想尝试，将 algorithm="elkan" 即可。

2010 年，David Sculley 提出了 k-means 的另一个重要变体[6]。算法不再每次迭代使用完整数据集，而是使用小批量，每次只让质心轻微移动。这样可以加快算法，并对无法装入内存的超大数据集进行聚类。Scikit-Learn 在 MiniBatchKMeans 类中实现了该算法，使用方式与 KMeans 类似：

```
from sklearn.cluster import MiniBatchKMeans
minibatch_kmeans = MiniBatchKMeans(n_clusters=5, random_state=42)
minibatch_kmeans.fit(X)
```

如果数据集无法装入内存，最简单的选择是使用第 7 章增量 PCA 中介绍过的 memmap 类。也可以一次向 partial_fit() 方法传入一个小批量，但这需要更多工作，因为必须自行执行多次初始化并选择最佳解。

找到最佳簇数量（Finding the Optimal Number of Clusters） 前面把簇数量 k 设为 5，是因为观察数据即可看出这是正确数量。但一般情况下并不容易，如果 k 设置错误，结果可能很差。图 8-6 展示了 k 设为 3 或 8 时的糟糕结果。

你可能会想到选择惯性最低的模型。不幸的是事情没这么简单。k=3 时惯性约为 653.2，远高于 k=5 时的 211.7；但 k=8 时惯性只有 127.1。惯性不是选择 k 的好指标，因为随着 k 增大，惯性会不断降低。簇越多，每个实例距离最近质心越近，惯性就越低。绘制惯性与 k 的关系图时，曲线通常包含一个称为肘部的拐点，见图 8-7。

[4] Charles Elkan，"Using the Triangle Inequality to Accelerate k-Means"，Proceedings of the 20th International Conference on Machine Learning（2003）：147-153。

[5] 三角不等式为 AC ≤ AB + BC，其中 A、B、C 是三个点，AB、AC、BC 是它们之间的距离。

[6] David Sculley，"Web-Scale K-Means Clustering"，Proceedings of the 19th International Conference on World Wide Web（2010）：1177-1178。

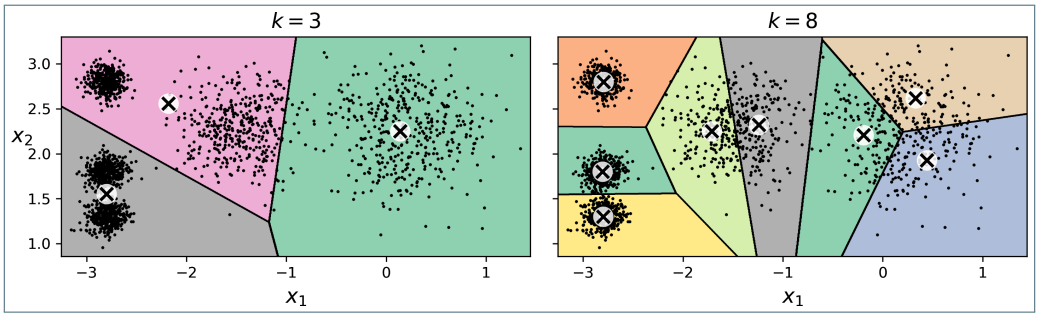

图 8-6: 簇数量选择错误：k 太小时合并独立簇，k 太大时把簇切成多个部分（Bad choices for the number of clusters）

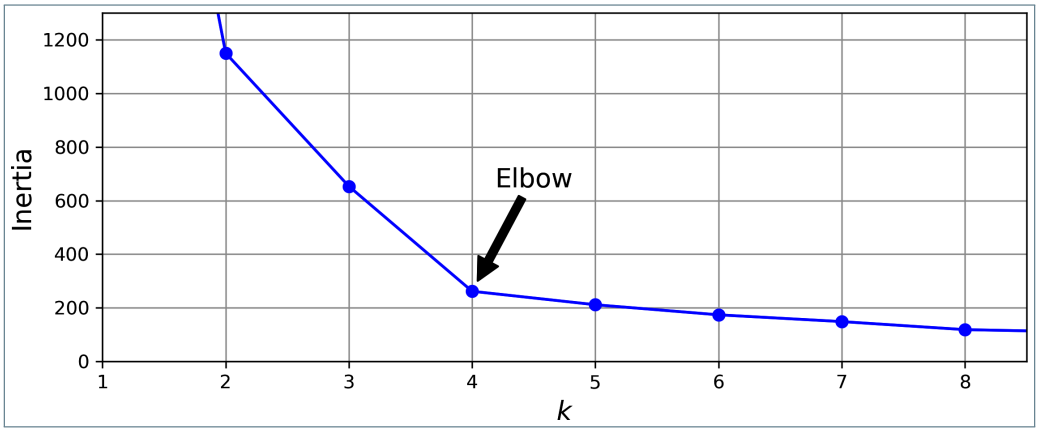

图 8-7: 将惯性绘制为簇数量 k 的函数（Plotting the inertia as a function of the number of clusters k）

可以看到，随着 k 增大到 4，惯性下降非常快；继续增加 k 后，下降速度明显变慢。曲线大致像一条手臂，肘部位于 k=4。因此，如果没有更多信息，4 似乎是一个不错的选择：更小的值会造成明显变化，更大的值帮助不大，反而可能把本来很好的簇无理由地一分为二。

这种选择最佳簇数量的方法比较粗略。一个更精确但计算成本更高的方法是使用轮廓分数（silhouette score），也就是所有实例的平均轮廓系数。一个实例的轮廓系数等于 (b - a) / max(a, b)，其中 a 是它到同一簇中其他实例的平均距离，即簇内平均距离；b 是它到最近其他簇的平均距离，也就是使 b 最小的那个簇，不包含实例自己的簇。轮廓系数范围为 -1 到 +1。接近 +1 表示实例处于自身簇内部，且远离其他簇；接近 0 表示实例靠近簇边界；接近 -1 表示实例可能被分到了错误的簇。

可以使用 Scikit-Learn 的 silhouette_score() 函数计算轮廓分数：

```
>>> from sklearn.metrics import silhouette_score
>>> silhouette_score(X, kmeans.labels_)
np.float64(0.655517642572828)
```

图 8-8 比较了不同簇数量的轮廓分数。

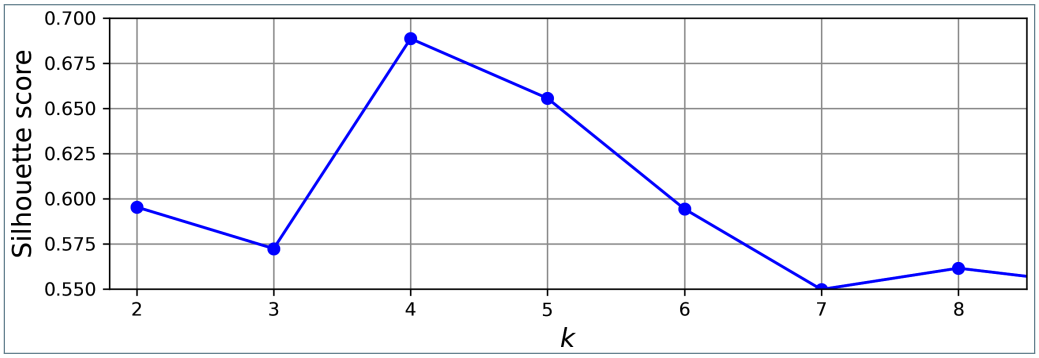

图 8-8: 使用轮廓分数选择簇数量 k（Selecting the number of clusters k using the silhouette score）

这个可视化比前面的惯性图更丰富：它确认 k=4 是很好的选择，同时也显示 k=5 也相当不错，而且明显优于 k=6 或 k=7。比较惯性时看不出这一点。

更有信息量的可视化是绘制每个实例的轮廓系数，并按其所属簇以及系数值排序，这称为轮廓图（silhouette diagram），见图 8-9。每个轮廓图中每个簇对应一个刀形区域。形状的高度表示簇中实例数量，宽度表示该簇内实例轮廓系数的排序结果，越宽越好。

竖直虚线表示每个簇数量对应的平均轮廓分数。如果一个簇中的大多数实例系数都低于该分数，也就是许多实例在虚线左侧结束，那么这个簇表现较差，因为实例离其他簇太近。图中 k=3 或 k=6 时会得到较差的簇；k=4 或 k=5 时，大多数实例都延伸到虚线右侧，接近 1.0，簇看起来很好。k=4 时，索引为 0 的簇位于底部且相当大；k=5 时，各簇大小相近。因此，虽然 k=4 的总体轮廓分数略高于 k=5，但选择 k=5 得到大小相近的簇似乎更合理。

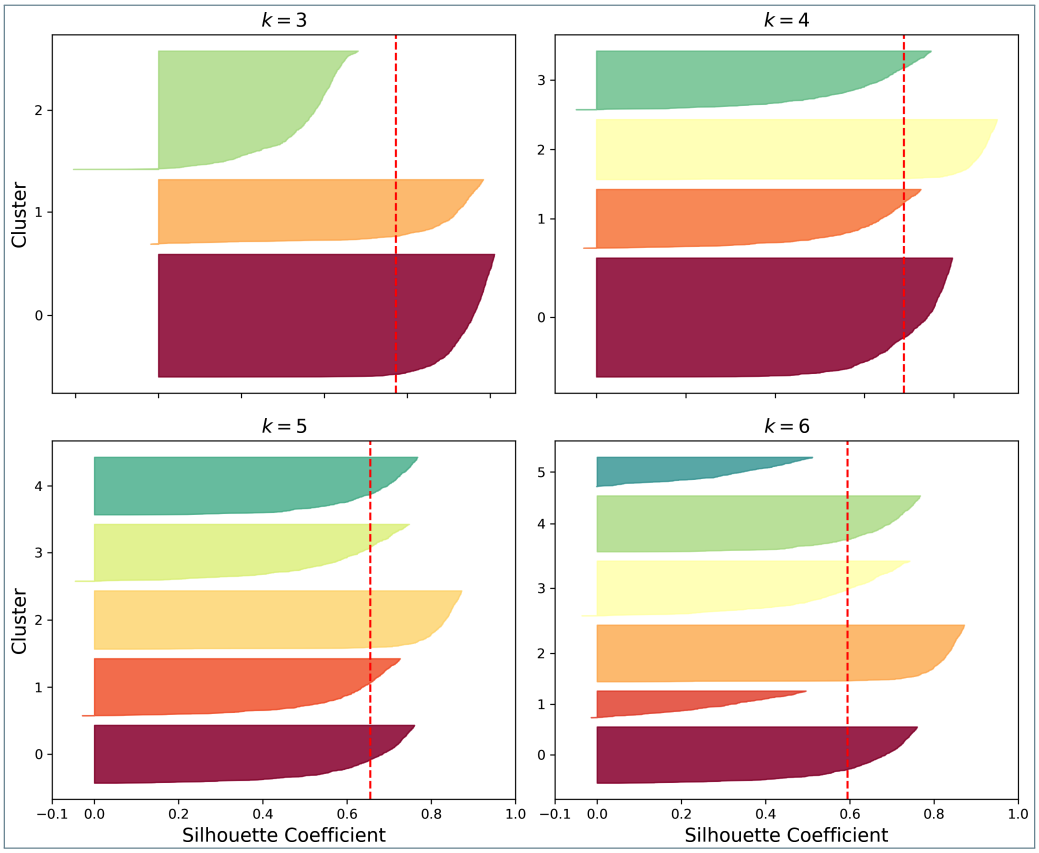

图 8-9: 分析不同 k 值的轮廓图（Analyzing the silhouette diagrams for various values of k）

k-means 的局限（Limits of k-Means） 尽管 k-means 有许多优点，尤其是速度快、扩展性好，但它并不完美。为了避免次优解，需要运行算法多次，而且还必须指定簇数量，这可能很麻烦。此外，当簇的大小不同、密度不同或形状不是球形时，k-means 表现不好。图 8-10 展示了 k-means 对三个椭圆簇进行聚类的结果，这些簇的维度、密度和方向各不相同。

如你所见，这两个结果都不好。左侧结果稍好，但仍然把中间簇的 25% 切掉，并把它分配给右侧簇；右侧结果非常糟糕，尽管它的惯性更低。因此，根据数据的不同，其他聚类算法可能表现更好。对于这类椭圆簇，高斯混合模型效果很好。

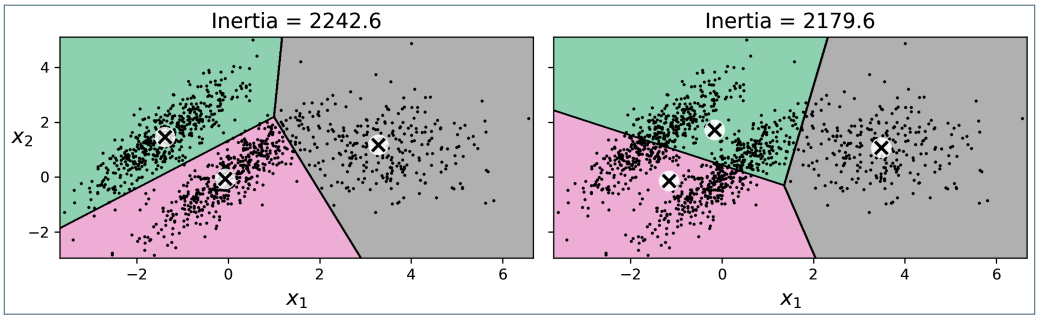

图 8-10: k-means 无法正确聚类椭圆团块（k-means fails to cluster these ellipsoidal blobs properly）

> 提示：运行 k-means 前务必缩放输入特征（见第 2 章），否则簇可能会被拉得很长，k-means 的表现会很差。缩放不能保证所有簇都变成球形，但通常会有帮助。

下面看看如何从聚类中获益。这里使用 k-means，但也可以尝试其他聚类算法。

使用聚类进行图像分割（Using Clustering for Image Segmentation） 图像分割是把一幅图像划分为多个片段的任务。它有几种变体：

- 在颜色分割中，颜色相似的像素被分配到同一片段。这已经足够应对许多应用。例如，如果要分析卫星图像，测量某个区域中的森林总面积，颜色分割就可能足够。
- 在语义分割中，属于同一物体类型的所有像素被分配到同一片段。例如，在自动驾驶汽车的视觉系统中，属于行人的所有像素都可以分配给“行人”片段。
- 在实例分割中，属于同一个独立物体的所有像素被分配到同一片段。此时每个行人都会有一个不同的片段。

目前，语义分割或实例分割的最先进方法使用基于卷积神经网络（见第 12 章）或视觉 Transformer（见第 16 章）的复杂架构。本章关注更简单的颜色分割任务，使用 k-means 完成。

先导入 Pillow 软件包，也就是 Python Imaging Library（PIL）的后继者，再加载 ladybug.png 图像，假定它位于 filepath：

```
>>> import PIL
>>> image = np.asarray(PIL.Image.open(filepath))
>>> image.shape
(533, 800, 3)
```

图像表示为三维数组。第一维是高度，第二维是宽度，第三维是颜色通道数，本例中为红、绿、蓝（RGB）。也就是说，每个像素都有一个三维向量，分别表示红、绿、蓝的强度，取值是 0 到 255 之间的无符号 8 位整数。有些图像通道更少，例如灰度图像只有一个通道；有些图像通道更多，例如包含透明度 alpha 通道的图像，或者包含红外等额外光频通道的卫星图像。

下面的代码把数组重塑为 RGB 颜色列表，然后使用包含 8 个簇的 k-means 对这些颜色进行聚类。它创建 segmented_img 数组，为每个像素保存最近簇中心，也就是该像素所在簇的平均颜色，最后把数组重塑为原始图像形状。第三行使用了 NumPy 的高级索引：

```
X = image.reshape(-1, 3)
kmeans = KMeans(n_clusters=8, random_state=42).fit(X)
segmented_img = kmeans.cluster_centers_[kmeans.labels_]
segmented_img = segmented_img.reshape(image.shape)
```

输出结果如图 8-11 右上角所示。可以尝试不同的簇数量。使用少于 8 个簇时，ladybug 的鲜红色不会得到独立簇，而会与周围颜色合并。这是因为 k-means 偏好大小相近的簇；瓢虫在图像中很小，远小于其他区域，因此即使颜色鲜艳，k-means 也不会专门为它分配一个簇。

图 8-11: 使用不同颜色簇数量进行 k-means 图像分割（Image segmentation using k-means with various numbers of color clusters）

这并不难，对吧？下面看看聚类的另一个应用。

使用聚类进行半监督学习（Using Clustering for Semi-Supervised Learning） 聚类的另一个用例是半监督学习：有大量未标记实例，但只有很少带标签的实例。例如，聚类可以帮助选择要额外标记的实例，例如选择靠近簇质心的实例；也可以把每个簇中最常见的标签传播给该簇中的未标记实例。

下面在 digits 数据集上尝试这些想法。它是一个类似 MNIST 的简单数据集，包含 1,797 幅灰度 8 × 8 图像，表示数字 0 到 9。先加载并划分数据集，它已经完成了随机打乱：

```
from sklearn.datasets import load_digits
X_digits, y_digits = load_digits(return_X_y=True)
X_train, y_train = X_digits[:1400], y_digits[:1400]
X_test, y_test = X_digits[1400:], y_digits[1400:]
```

假设我们只有 50 个实例的标签。作为基线，先用这 50 个带标签的实例训练逻辑回归模型：

```
from sklearn.linear_model import LogisticRegression
n_labeled = 50
log_reg = LogisticRegression(max_iter=10_000)
log_reg.fit(X_train[:n_labeled], y_train[:n_labeled])
```

在测试集上评估模型准确率，注意测试集必须有标签：

```
>>> log_reg.score(X_test, y_test)
0.7581863979848866
```

模型准确率只有 75.8%，不算好。如果在完整训练集上训练模型，准确率大约能达到 90.9%。下面看看如何改进。先把训练集聚成 50 个簇，然后在每个簇中找到最接近质心的图像，把它们称为代表性图像：

```
k = 50
kmeans = KMeans(n_clusters=k, random_state=42)
X_digits_dist = kmeans.fit_transform(X_train)
representative_digit_idx = X_digits_dist.argmin(axis=0)
X_representative_digits = X_train[representative_digit_idx]
```

图 8-12 展示了 50 幅代表性数字图像。

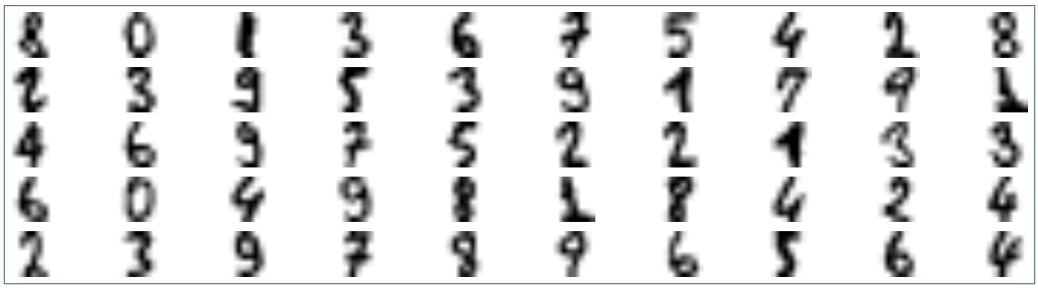

图 8-12: 每个簇一幅代表性数字图像，共 50 幅（Fifty representative digit images）

逐一查看图像并手工标记：

```
y_representative_digits = np.array([8, 0, 1, 3, 6, 7, 5, 4, 2, 8, ..., 6, 4])
```

现在有了一个只有 50 个带标签实例的数据集，但它们不是随机实例，而是各自簇的代表图像。再次评估性能：

```
>>> log_reg = LogisticRegression(max_iter=10_000)
>>> log_reg.fit(X_representative_digits, y_representative_digits)
>>> log_reg.score(X_test, y_test)
0.8337531486146096
```

准确率从 75.8% 提高到了 83.4%，虽然模型仍然只在 50 个实例上训练。由于标记实例通常成本高、过程痛苦，尤其需要专家手工完成时，与其随机选择实例，不如标记具有代表性的实例。

还可以更进一步：把标签传播给同一簇中的其他实例。这称为标签传播：

```
y_train_propagated = np.empty(len(X_train), dtype=np.int64)
for i in range(k):
    y_train_propagated[kmeans.labels_ == i] = y_representative_digits[i]
```

再次训练模型并查看性能：

```
>>> log_reg = LogisticRegression()
>>> log_reg.fit(X_train, y_train_propagated)
>>> log_reg.score(X_test, y_test)
0.8690176322418136
```

准确率又有了明显提升。还可以忽略距离簇中心最远的 50% 实例，以消除一些离群点。下面的代码首先计算每个实例到最近簇中心的距离，然后对每个簇把距离最大的 50% 实例设为 -1，最后构造一个不包含这些实例的数据集：

```
percentile_closest = 50
X_cluster_dist = X_digits_dist[np.arange(len(X_train)), kmeans.labels_]
for i in range(k):
    in_cluster = (kmeans.labels_ == i)
    cluster_dist = X_cluster_dist[in_cluster]
    cutoff_distance = np.percentile(cluster_dist, percentile_closest)
    above_cutoff = (X_cluster_dist > cutoff_distance)
    X_cluster_dist[in_cluster & above_cutoff] = -1
partially_propagated = (X_cluster_dist != -1)
X_train_partially_propagated = X_train[partially_propagated]
y_train_partially_propagated = y_train_propagated[partially_propagated]
```

在这个部分传播的数据集上再次训练模型：

```
>>> log_reg = LogisticRegression(max_iter=10_000)
>>> log_reg.fit(X_train_partially_propagated, y_train_partially_propagated)
>>> log_reg.score(X_test, y_test)
0.8841309823677582
```

只使用 50 个带标签实例，也就是平均每个类别只有 5 个示例，就达到了 88.4% 的准确率，非常接近在完整带标签 digits 数据集上得到的结果。这部分得益于删除了一些离群点，部分得益于传播的标签确实很准确。下面的代码显示，传播标签的准确率约为 98.9%：

```
>>> (y_train_partially_propagated == y_train[partially_propagated]).mean()
np.float64(0.9887798036465638)
```

> 提示：Scikit-Learn 还提供了 LabelSpreading 和 LabelPropagation 两个可以自动传播标签的类，位于 sklearn.semi_supervised 软件包中。两者都会构造所有实例之间的相似度矩阵，并迭代地把标签从带标签实例传播给相似的未标记实例。该软件包还提供 SelfTrainingClassifier：给它一个基础分类器，例如 RandomForestClassifier，它会先在带标签实例上训练，再用模型预测未标记样本的标签，然后把最有把握的标签加入训练集，重复训练和标记，直到不能再增加标签。这些技术不是万能的，但偶尔可以让模型得到一点提升。

主动学习（Active Learning） 要继续改进模型和训练集，下一步可以进行几轮主动学习。主动学习是指人类专家与学习算法交互：算法请求时，专家为特定实例提供标签。主动学习策略很多，最常见的一种称为不确定性抽样：

1. 使用目前收集到的带标签实例训练模型，再用模型对所有未标记实例进行预测。
2. 把模型最不确定的实例，也就是估计概率最低的实例交给专家标记。
3. 重复这一过程，直到性能提升不再值得付出标记成本。

其他主动学习策略包括：标记会导致模型变化最大、验证误差下降最大，或者不同模型意见不一致的实例，例如 SVM 和随机森林意见不一致的实例。

在进入高斯混合模型之前，先看看 DBSCAN。这是另一种流行的聚类算法，它采用基于局部密度估计的完全不同方法，可以识别任意形状的簇。

8.2　DBSCAN

基于密度的带噪声空间聚类（density-based spatial clustering of applications with noise，DBSCAN）把簇定义为连续的高密度区域。它的工作方式如下：

- 对于每个实例，算法统计位于其小距离 ε（epsilon）范围内的实例数量，这个区域称为该实例的 ε 邻域。
- 如果一个实例的 ε 邻域中至少有 min_samples 个实例，包括它自己，那么它就被视为核心实例。也就是说，核心实例位于密集区域。

- 核心实例邻域中的所有实例都属于同一个簇。这个邻域可能包含其他核心实例，因此一长串相邻核心实例会形成一个簇。
- 既不是核心实例，且邻域中没有核心实例的实例，被视为异常。

如果所有簇都被低密度区域良好分隔，这个算法效果很好。Scikit-Learn 的 DBSCAN 类使用起来非常简单。下面在第 5 章介绍过的 moons 数据集上尝试：

```
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons
X, y = make_moons(n_samples=1000, noise=0.05, random_state=42)
dbscan = DBSCAN(eps=0.05, min_samples=5)
dbscan.fit(X)
```

所有实例的标签现在可以通过 labels_ 实例变量访问：

```
>>> dbscan.labels_
array([ 0,  2, -1, -1,  1,  0,  0,  0,  2,  5, [...], 3,  3,  4,  2,  6,  3])
```

注意，有些实例的簇索引为 -1，表示它们被算法视为异常。核心实例的索引可以通过 core_sample_indices_ 访问，核心实例本身可以通过 components_ 访问：

```
>>> dbscan.core_sample_indices_
array([  0,   4,   5,   6,   7,   8,  10,  11, [...], 993, 995, 997, 998, 999])
>>> dbscan.components_
array([[-0.02137124,  0.40618608],
       [-0.84192557,  0.53058695],
       [...],
       [ 0.79419406,  0.60777171]])
```

图 8-13 左侧展示了这个聚类结果。可以看到，它识别出很多异常以及七个不同的簇，结果令人失望。幸运的是，把每个实例的邻域半径 eps 增加到 0.2 后，右侧的聚类结果看起来就完美了。下面继续使用这个模型。

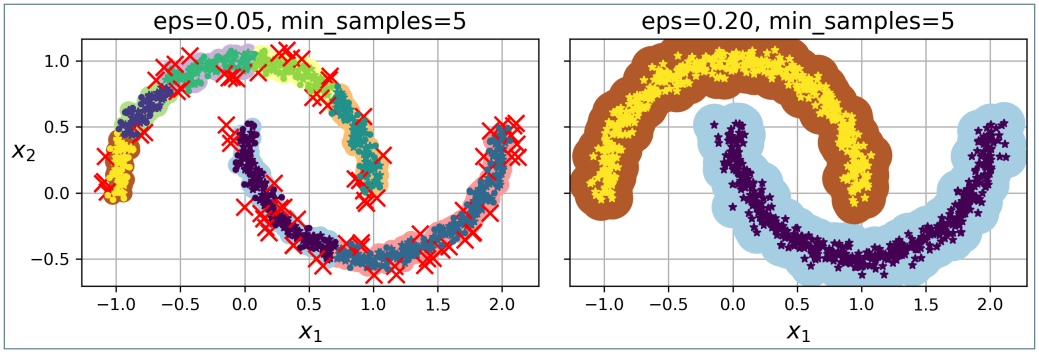

图 8-13: 使用两种不同邻域半径的 DBSCAN 聚类（DBSCAN clustering using two different neighborhood radiuses）

令人意外的是，DBSCAN 类没有 predict() 方法，虽然有 fit_predict() 方法。换句话说，它不能预测新实例属于哪个簇。原因是不同分类算法可能更适合不同任务，因此作者决定让用户选择使用哪种分类器。而且实现这个功能并不难。例如，训练一个 KNeighborsClassifier：

```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=50)
knn.fit(dbscan.components_, dbscan.labels_[dbscan.core_sample_indices_])
```

给定几个新实例后，可以预测它们最可能属于哪个簇，甚至估计属于各簇的概率：

```
>>> X_new = np.array([[-0.5, 0], [0, 0.5], [1, -0.1], [2, 1]])
>>> knn.predict(X_new)
array([1, 0, 1, 0])
>>> knn.predict_proba(X_new)
array([[0.18, 0.82],
       [1.  , 0.  ],
       [0.12, 0.88],
       [1.  , 0.  ]])
```

注意，我们只在核心实例上训练分类器，但也可以选择在所有实例上训练，或在除异常外的所有实例上训练，具体选择取决于最终任务。

图 8-14 展示了决策边界，叉号表示 X_new 中的四个实例。由于训练集中没有异常，分类器总会选择一个簇，即使某个簇距离很远。可以引入最大距离：此时，距离两个簇都很远的两个实例会被分类为异常。使用 KNeighborsClassifier 的 kneighbors() 方法即可。给定一组实例，它返回训练集中 k 个最近邻的距离和索引，也就是两个各有 k 列的矩阵：

```
>>> y_dist, y_pred_idx = knn.kneighbors(X_new, n_neighbors=1)
>>> y_pred = dbscan.labels_[dbscan.core_sample_indices_][y_pred_idx]
>>> y_pred[y_dist > 0.2] = -1
>>> y_pred.ravel()
array([-1,  0,  1, -1])
```

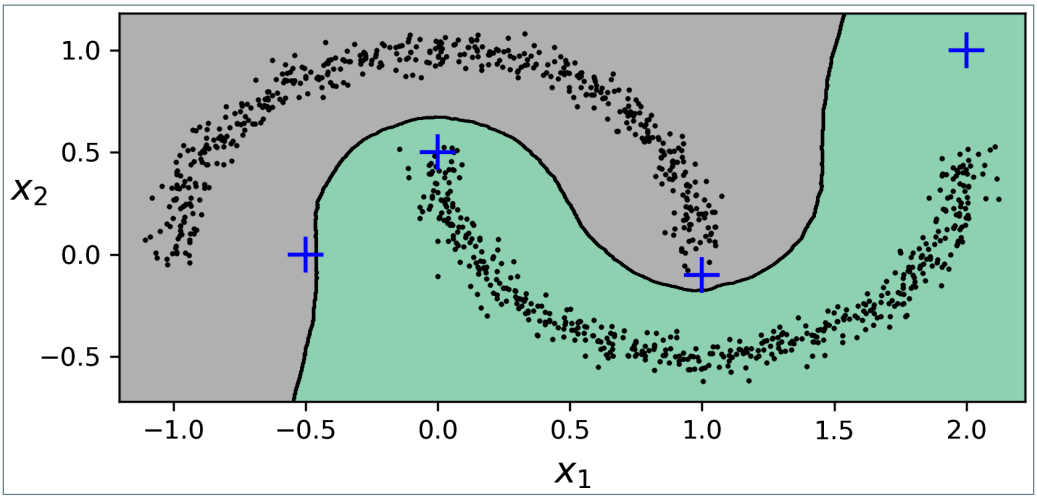

图 8-14: 两个簇之间的决策边界（Decision boundary between two clusters）

总之，DBSCAN 是非常简单但强大的算法，能够识别任意数量、任意形状的簇，对离群点具有鲁棒性，而且只有两个超参数 eps 和 min_samples。不过，如果各簇密度差异很大，或者某些簇周围没有足够的低密度区域，DBSCAN 可能无法正确捕捉所有簇。此外，它的计算复杂度大约为 $O(m^2n)$，无法很好地扩展到大型数据集。

> 提示：还可以尝试层次 DBSCAN（HDBSCAN），使用 sklearn.cluster.HDBSCAN。它通常比 DBSCAN 更擅长发现密度不同的簇。

8.2.1　其他聚类算法（Other Clustering Algorithms）

Scikit-Learn 还实现了几种聚类算法，值得了解。这里无法逐一详细介绍，下面给出简要概览：

凝聚聚类（Agglomerative clustering）

它从底向上构建簇的层次结构。想象许多漂浮在水面上的小气泡逐渐彼此连接，最终形成一个大群体。类似地，每次迭代中，凝聚聚类都会连接最近的一对簇，从单个实例开始。如果为每次簇合并画一条树枝，就会得到一棵簇的二叉树，叶子是单个实例。这种方法可以捕捉不同形状的簇，还会生成灵活且信息丰富的簇树，不必强迫用户选择一个特定簇尺度，并且可以使用任意成对距离。如果提供连接矩阵，也可以很好地扩展到大量实例。连接矩阵是一个稀疏的 m × m 矩阵，表示哪些实例互为邻居，例如由 sklearn.neighbors.kneighbors_graph() 返回；没有连接矩阵时，算法无法很好地扩展到大型数据集。

BIRCH

平衡迭代缩减与层次聚类（balanced iterative reducing and clustering using hierarchies，BIRCH）专门为大型数据集设计，只要特征数量不是太多（小于 20），它的速度可以比批量 k-means 更快，同时得到相似结果。训练过程中，它构建一个树结构，只保存快速把新实例分配给簇所需的信息，而不是把所有实例存储在树中，因此能在内存有限的情况下处理海量数据。

均值漂移（Mean-shift）

该算法先在每个实例上放置一个圆，然后对每个圆计算其中所有实例的平均值，把圆移动到平均值位置。接着不断执行均值漂移，直到所有圆都停止移动，也就是每个圆都以其中实例的平均位置为中心。均值漂移会把圆向更高密度方向移动，直到每个圆找到一个局部密度最大值。最后，圆停留在同一位置或相近位置的实例被分配给同一簇。均值漂移具有与 DBSCAN 相同的一些特性，例如能找到任意数量、任意

形状的簇，超参数很少，只有一个表示圆半径的带宽参数，并依赖局部密度估计。不过，内部密度有变化的簇可能被均值漂移切成多个部分。它的计算复杂度为 $O(m^2n)$，不适用于大型数据集。

亲和传播（Affinity propagation）

该算法让实例之间反复交换消息，直到每个实例都选择另一个实例，或者选择自己作为代表。被选中的实例称为代表点（exemplar）；每个代表点和选择它的实例形成一个簇。在现实政治中，人们通常想投票给意见与自己相似且有机会获胜的候选人，因此可能选择一个并不完全认同、但更受欢迎的候选人。亲和传播的工作方式类似，倾向于选择位于簇中心附近的代表点，与 k-means 相似。但与 k-means 不同，不需要事先选择簇数量，数量在训练过程中决定；它也能很好地处理大小不同的簇。不幸的是，其计算复杂度为 $O(m^2)$，不适用于大型数据集。

谱聚类（Spectral clustering）

该算法先根据实例之间的相似度矩阵创建低维嵌入，也就是降低矩阵维度，然后在低维空间中使用另一个聚类算法，Scikit-Learn 的实现使用 k-means。谱聚类可以捕捉复杂的簇结构，也可以用于切分图，例如识别社交网络中的朋友群体。它无法很好地扩展到大量实例，当簇大小差异很大时表现也不好。

下面进入高斯混合模型，它可以用于密度估计、聚类和异常检测。

8.3　高斯混合（Gaussian Mixtures）

高斯混合模型（Gaussian mixture model，GMM）是一种概率模型，假设实例由若干个高斯分布混合生成，而这些分布的参数未知。由同一个高斯分布生成的所有实例构成一个簇，该簇通常呈椭球形。每个簇都可以具有不同的椭球形状、大小、密度和方向，就像图 8-10 所示[7]。观察到一个实例时，可以知道它由某个高斯分布生成，但不知道是哪个分布，也不知道这些分布的参数。

GMM 有多种变体。最简单的变体由 GaussianMixture 类实现，必须事先知道高斯分布的数量 k。数据集 X 被假设通过以下概率过程生成：

- 对于每个实例，从 k 个簇中随机选择一个。选择第 j 个簇的概率是该簇的权重 $\varphi(j)$[8]。第 i 个实例所选簇的索引记为 $z(i)$。

[7] 与前面所见相反，k-means 隐含地假设所有簇大小和密度相近，而且大致呈圆形。

[8] Phi（phi 或 φ）是希腊字母表的第 21 个字母。

- 如果第 i 个实例被分配给第 j 个簇，也就是 z(i) = j，那么该实例的位置 x(i) 从均值 μ(j)、协方差矩阵 Σ(j) 的高斯分布中随机抽取，记为 x(i) ~ N(μ(j), Σ(j))。

给定数据集 X，通常希望估计权重 φ 以及从 μ(1) 到 μ(k)、从 Σ(1) 到 Σ(k) 的所有分布参数。Scikit-Learn 的 GaussianMixture 类可以轻松完成：

```
from sklearn.mixture import GaussianMixture
gm = GaussianMixture(n_components=3, n_init=10, random_state=42)
gm.fit(X)
```

查看算法估计出的参数：

```
>>> gm.weights_
array([0.40005972, 0.20961444, 0.39032584])
>>> gm.means_
array([[-1.40764129,  1.42712848],
       [ 3.39947665,  1.05931088],
       [ 0.05145113,  0.07534576]])
>>> gm.covariances_
array([[[ 0.63478217,  0.72970097],
        [ 0.72970097,  1.16094925]],
       [[ 1.14740131, -0.03271106],
        [-0.03271106,  0.95498333]],
       [[ 0.68825143,  0.79617956],
        [ 0.79617956,  1.21242183]]])
```

结果很好！三个簇中有两个各由 500 个实例生成，第三个簇只有 250 个实例，因此真实簇权重为 0.4、0.4 和 0.2，算法得到的结果与之大致相同，只是簇的顺序不同。真实均值和协方差矩阵也与算法找到的结果接近。

这是怎么做到的？GaussianMixture 类依赖期望最大化（expectation-maximization，EM）算法。它与 k-means 有许多相似之处：先随机初始化簇参数，然后重复两个步骤直到收敛，第一步是把实例分配给簇，称为期望步骤；第二步是更新簇，称为最大化步骤。熟悉吗？在聚类语境中，可以把 EM 看作 k-means 的推广：它不仅寻找簇中心 μ(1) 到 μ(k)，还会寻找簇的大小、形状、方向（Σ(1) 到 Σ(k)）以及相对权重（φ(1) 到 φ(k)）。

不过，与 k-means 不同，EM 使用软聚类分配，而不是硬聚类分配。在期望步骤中，算法根据当前簇参数估计每个实例属于每个簇的概率；在最大化步骤中，用数据集中的所有实例更新每个簇，每个实例的权重等于它属于该簇的估计概率。这些概率称为簇对实例的责任度。最大化步骤中，一个簇的更新主要受它承担责任最大的那些实例影响。

> 警告：与 k-means 一样，EM 可能最终收敛到较差的解，因此需要运行多次并只保留最佳解。这就是前面把 n_init 设为 10 的原因。注意，n_init 默认值为 1。

可以检查算法是否收敛以及使用了多少次迭代：

```
>>> gm.converged_
True
>>> gm.n_iter_
4
```

现在已经估计出每个簇的位置、大小、形状、方向和相对权重，模型可以轻松把实例分配给最可能的簇，也就是硬聚类；或者估计实例属于某个簇的概率，也就是软聚类。硬聚类使用 predict()，软聚类使用 predict_proba()：

```
>>> gm.predict(X)
array([2, 2, 0, ..., 1, 1, 1])
>>> gm.predict_proba(X).round(3)
array([[0.   , 0.023, 0.977],
       [0.001, 0.016, 0.983],
       [1.   , 0.   , 0.   ],
       ...,
       [0.   , 1.   , 0.   ],
       [0.   , 1.   , 0.   ],
       [0.   , 1.   , 0.   ]])
```

高斯混合模型是生成模型，也就是说可以从中抽取新实例。注意，实例会按簇索引排序：

```
>>> X_new, y_new = gm.sample(6)
>>> X_new
array([[-2.32491052,  1.04752548],
       [-1.16654983,  1.62795173],
       [ 1.84860618,  2.07374016],
       [ 3.98304484,  1.49869936],
       [ 3.8163406 ,  0.53038367],
       [ 0.38079484, -0.56239369]])
>>> y_new
array([0, 0, 1, 1, 1, 2])
```

还可以估计模型在任意位置的密度。score_samples() 方法会对给定的每个实例，估计该位置的概率密度函数（PDF）的对数。分数越大，密度越高：

```
>>> gm.score_samples(X).round(2)
array([-2.61, -3.57, -3.33, ..., -3.51, -4.4 , -3.81])
```

对这些分数取指数，就得到给定实例位置的 PDF 值。它们不是概率，而是概率密度，可以取任意正值，不局限于 0 到 1 之间。要估计实例落在某个区域内的概率，必须对该区域上的 PDF 积分；如果在所有可能的位置上积分，结果为 1。

图 8-15 展示了该模型的簇均值、决策边界和密度等高线。

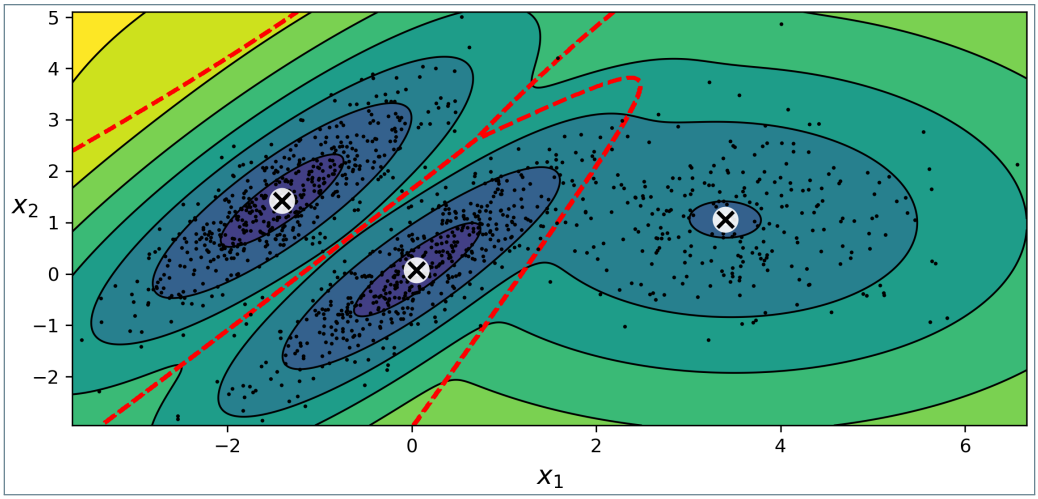

图 8-15: 训练好的高斯混合模型的簇均值、决策边界和密度等高线（Cluster means, decision boundaries, and density contours）

这个算法显然找到了很好的解。当然，生成数据时使用的是一组二维高斯分布，这让任务变得容易；现实数据并不总是高斯分布，也不总是低维数据。而且我们还向算法提供了正确的簇数量。当维度很多、簇很多或实例很少时，EM 可能难以收敛到最优解。此时需要通过限制模型必须学习的参数数量来降低任务难度，例如限制簇的形状和方向范围。可以把 covariance_type 超参数设为以下值之一：

“spherical”

所有簇都必须是球形，但直径，也就是方差，可以不同。

“diag”

簇可以具有任意椭球形状和大小，但椭球的轴必须与坐标轴平行，也就是协方差矩阵必须是对角矩阵。

“tied”

所有簇必须具有相同的椭球形状、大小和方向，也就是所有簇共享同一个协方差矩阵。

默认情况下，covariance_type 等于“full”，表示每个簇可以具有任意形状、大小和方向，每个簇都有自己的无约束协方差矩阵。图 8-16 展示了 covariance_type 设为“tied”和“spherical”时 EM 算法找到的结果。

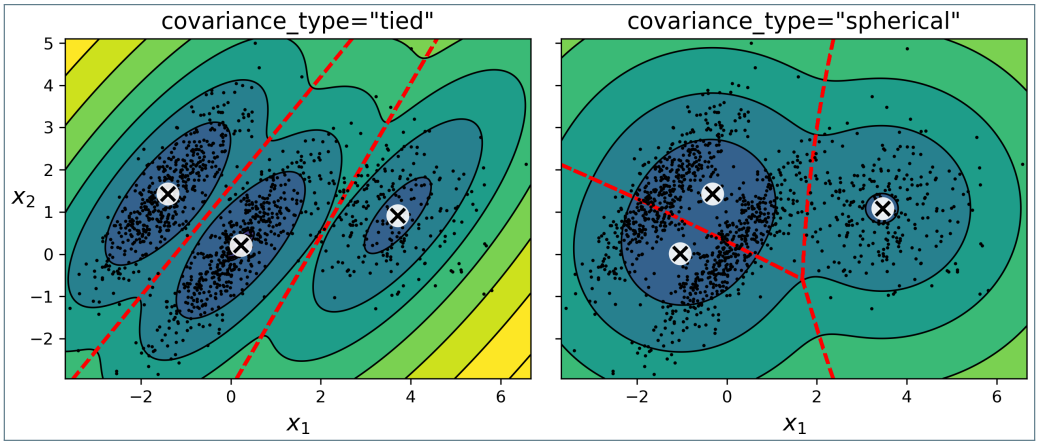

图 8-16: tied 簇（左）与球形簇（右）的高斯混合（Gaussian mixtures for tied clusters and spherical clusters）

> 注意：训练 GaussianMixture 模型的计算复杂度取决于实例数量 m、维度数量 n、簇数量 k 以及协方差矩阵的约束。当 covariance_type 为“spherical”或“diag”时，如果数据具有聚类结构，复杂度为 O(kmn)；当 covariance_type 为“tied”或“full”时，复杂度为 $O(kmn^2 + kn^3)$，因此无法扩展到特征数量很大的数据集。

高斯混合模型还可以用于异常检测，下一节将介绍。

8.3.1　使用高斯混合进行异常检测（Using Gaussian Mixtures for Anomaly Detection）

使用高斯混合模型进行异常检测非常简单：位于低密度区域的实例都可以视为异常。必须定义想使用的密度阈值。例如，一家制造企业想检测缺陷产品，且已知缺陷产品比例约为 2%。此时可以把密度阈值设为某个值，使约 2% 的实例位于低于该阈值的区域。如果假阳性太多，也就是正常产品被标记为缺陷，可以降低阈值；反之，如果假阴性太多，也就是缺陷产品没有被检测出来，可以提高阈值。这就是通常的

精确率/召回率权衡（见第 3 章）。下面以第 2 百分位的最低密度为阈值，也就是把约 2% 的实例标记为异常：

```
densities = gm.score_samples(X)
density_threshold = np.percentile(densities, 2)
anomalies = X[densities < density_threshold]
```

图 8-17 用星号表示这些异常。

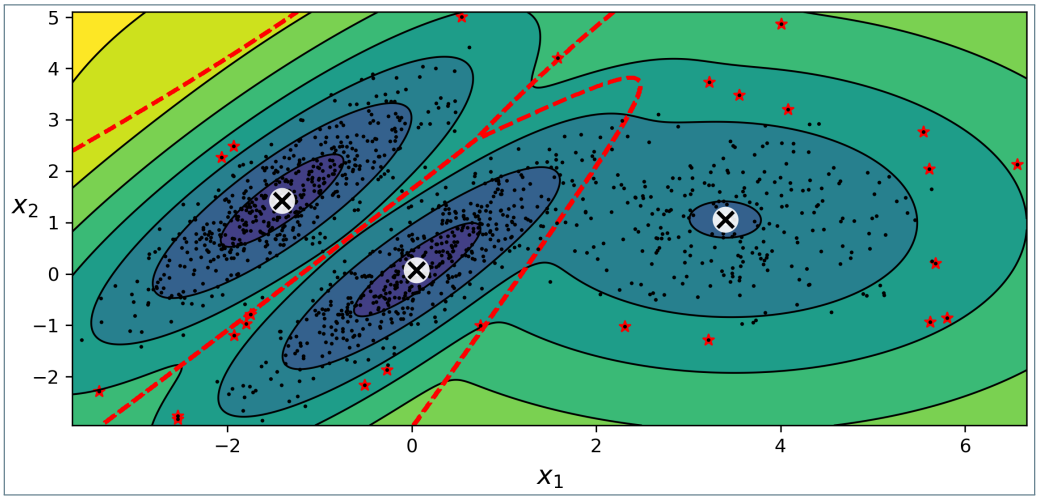

图 8-17: 使用高斯混合模型进行异常检测（Anomaly detection using a Gaussian mixture model）

与异常检测密切相关的是新颖性检测（novelty detection）。两者的区别在于，新颖性检测假设算法在“干净”数据集上训练，数据中没有离群点；异常检测不做这一假设。事实上，离群点检测经常用于清理数据集。

> 提示：高斯混合模型会尝试拟合所有数据，包括离群点。如果离群点太多，它们会影响模型对“正常”的理解，其中一些离群点可能会被错误视为正常。如果发生这种情况，可以先拟合模型，用它检测并删除最极端的离群点，再用清理后的数据集重新拟合模型。另一个方法是使用鲁棒协方差估计方法，例如 EllipticEnvelope 类。

GaussianMixture 算法和 k-means 一样，需要指定簇数量。那么如何找到这个数量呢？

8.4　选择簇数量（Selecting the Number of Clusters）

使用 k-means 时，可以用惯性或轮廓分数选择合适的簇数量。但对于高斯混合，这些指标在簇不是球形或大小不同时并不可靠。可以尝试寻找使理论信息准则最小的模型，例如贝叶斯信息准则（BIC）或赤池信息准则（AIC），定义见公式 8-2。

$$\text{BIC} = \log(m)p - 2\log(\hat{L}), \qquad \text{AIC} = 2p - 2\log(\hat{L}) \tag{8-2}$$

公式 8-2　贝叶斯信息准则（BIC）和赤池信息准则（AIC）（Bayesian information criterion and Akaike information criterion）

公式中的量含义如下：

- m 是实例数量。
- p 是模型学习的参数数量。
- L-hat 是模型似然函数的最大值。

BIC 和 AIC 都会惩罚需要学习更多参数的模型，例如簇更多的模型，同时奖励对数据拟合得好的模型。它们经常选择同一个模型。如果结果不同，BIC 选择的模型通常比 AIC 选择的模型更简单，参数更少，但对数据的拟合也可能不那么好，尤其是大型数据集。

8.4.1　似然函数（Likelihood Function）

“概率”和“似然”在日常语言中经常混用，但它们在统计学中含义不同。给定带有参数 θ 的统计模型，“概率”用于描述未来结果 x 的可信程度，也就是已知参数值 θ 时 x 的可能性；“似然”用于描述在结果 x 已知后，某组参数值 θ 的可信程度。

考虑一个由两个高斯分布组成的一维混合模型，两个分布中心位于 -4 和 +1。为简单起见，该玩具模型只有一个参数 θ，它控制两个分布的标准差。图 8-18 左上角的等高线图展示了整个模型 f(x; θ) 随 x 和 θ 的变化。要估计未来结果 x 的概率分布，必须先确定模型参数 θ。例如，将 θ 设为 1.3，也就是图中的水平线，就得到左下角的概率密度函数 f(x; θ=1.3)。若想估计 x 落在 -2 到 +2 之间的概率，需要对这个范围内的 PDF 积分，也就是计算阴影区域的面积。

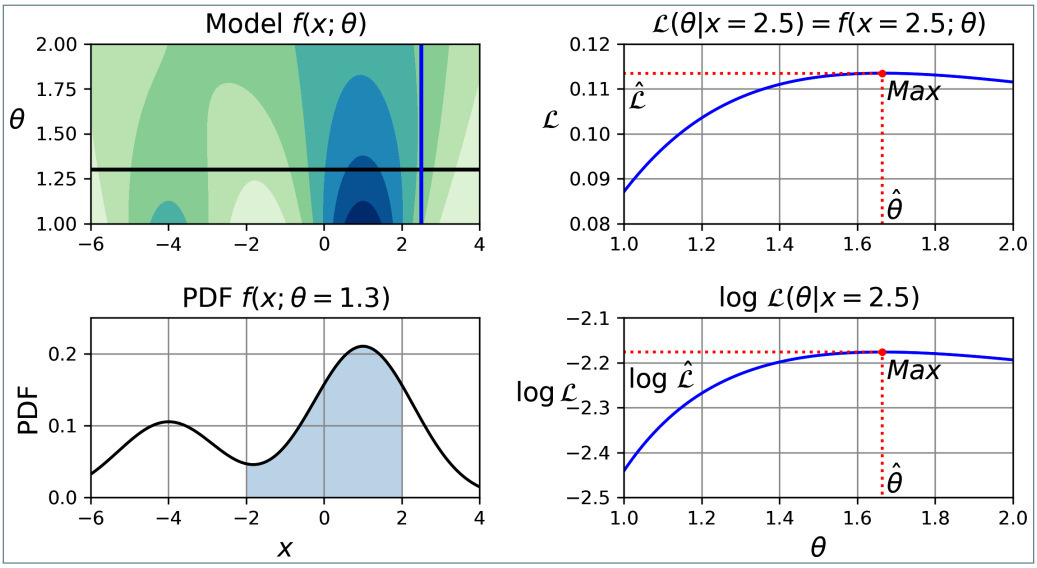

图 8-18: 模型的参数函数，以及由此得到的 PDF、似然函数和对数似然函数（A model's parametric function and some derived functions）

但是，如果不知道 θ，而是观察到一个实例 x=2.5，也就是图中竖直线所表示的位置，该怎么办？此时得到似然函数 L(θ|x=2.5)=f(x=2.5; θ)，如右上角图所示。

简而言之，PDF 是 x 的函数，θ 固定；似然函数是 θ 的函数，x 固定。必须理解，似然函数不是概率分布：对概率分布在所有可能 x 上积分，结果总是 1；但对似然函数在所有可能 θ 上积分，结果可以是任何正值。

给定数据集 X，一个常见任务是估计模型参数最可能的值。为此，需要寻找在给定 X 时最大化似然函数的参数值。在这个例子中，如果观察到一个实例 x=2.5，θ 的最大似然估计（MLE）约为 1.66。如果 θ 存在先验概率分布 g，就可以最大化 L(θ|x)g(θ)，而不是只最大化 L(θ|x)。这称为最大后验估计（MAP）。由于 MAP 会限制参数值，可以把它看作 MLE 的正则化版本。

注意，最大化似然函数等价于最大化它的对数，因为对数是严格递增函数。因此，如果 θ 最大化对数似然，也会最大化似然。最大化对数似然通常更容易。例如，观察到多个相互独立的实例 x(1) 到 x(m) 时，需要寻找最大化各个似然函数乘积的 θ；但同样可以最大化各个对数似然函数之和，因为对数会把乘法转换为加法：log(ab)=log(a)+log(b)。

一旦估计出使似然函数最大的 theta-hat，就可以计算 L-hat = L(theta-hat, X)，这是计算 AIC 和 BIC 时使用的值；可以把它看作模型拟合数据好坏的度量。调用 bic() 和 aic() 方法即可计算 BIC 和 AIC：

```
>>> gm.bic(X)
np.float64(8189.733705221636)
>>> gm.aic(X)
np.float64(8102.508425106598)
```

图 8-19 展示了不同簇数量 k 对应的 BIC。可以看到，BIC 和 AIC 都在 k=3 时达到最低值，因此 k=3 很可能是最佳选择。

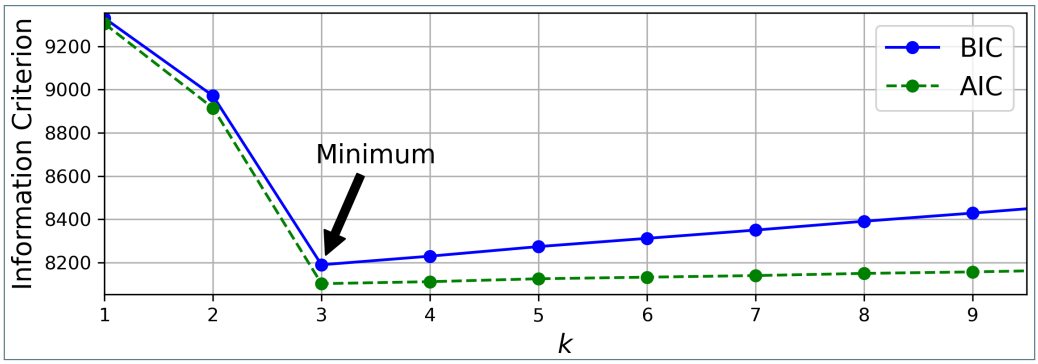

图 8-19: 不同簇数量 k 对应的 AIC 和 BIC（AIC and BIC for different numbers of clusters k）

8.4.2　贝叶斯高斯混合模型（Bayesian Gaussian Mixture Models）

与其手动搜索最佳簇数量，不如使用 BayesianGaussianMixture 类。它能够把不必要簇的权重设为零或接近零。将 n_components 设为一个你有充分理由认为大于最佳簇数量的值（这要求对问题有一些基本了解），算法就会自动消除不必要的簇。例如，将簇数量设为 10：

```
>>> from sklearn.mixture import BayesianGaussianMixture
>>> bgm = BayesianGaussianMixture(n_components=10, n_init=10, max_iter=500,
...                               random_state=42)
...
>>> bgm.fit(X)
>>> bgm.weights_.round(2)
array([0.4 , 0.21, 0.39, 0.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0.  ])
```

非常好：算法自动检测出只需要三个簇，得到的簇与图 8-15 中的几乎完全相同。

最后补充一点：高斯混合模型虽然非常适合椭球形簇，但面对形状差异很大的簇时表现不佳。例如，使用 Bayesian Gaussian Mixture 模型对 moons 数据集聚类时，会发生什么呢？

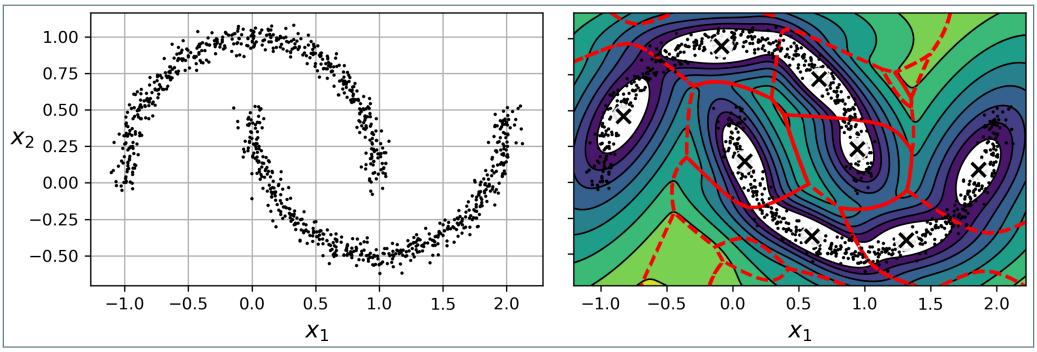

图 8-20: 使用高斯混合模型拟合非椭球形簇（Fitting a Gaussian mixture to nonellipsoidal clusters）

糟糕！算法拼命寻找椭球体，结果找到了八个不同的簇，而不是两个。它的密度估计并不算太差，因此或许仍可用于异常检测，但它没能识别出两个月牙形簇。最后，我们快速看一下几种能够处理任意形状簇的算法。

8.5　其他异常检测与新颖性检测算法（Other Algorithms for Anomaly and Novelty Detection）

Scikit-Learn 还实现了其他专门用于异常检测或新颖性检测的算法：

快速 MCD（最小协方差行列式，Fast-MCD）

该算法由 EllipticEnvelope 类实现，适合离群点检测，尤其适合清理数据集。它假设正常实例（内点）由单个高斯分布生成，而不是由混合分布生成；同时假设数据集被没有从该高斯分布生成的离群点污染。当算法估计高斯分布的参数，也就是内点周围椭圆包络的形状时，会谨慎地忽略最有可能是离群点的实例。因此，它可以更准确地估计椭圆包络，也更擅长识别离群点。

孤立森林（Isolation Forest）

这是一个高效的离群点检测算法，尤其适用于高维数据集。该算法构建一片随机森林，其中每棵决策树都以随机方式生长：在每个节点随机选择一个特征，再随机选择一个阈值（位于该特征的最小值和最大值之间），将数据集分成两部分。如此不断切分，直到所有实例都与其他实例隔离。异常通常远离其他实例，因此在所有决策树中取平均后，异常往往比正常实例用更少的步骤就能被隔离。

局部离群因子（Local Outlier Factor，LOF）

该算法同样适合离群点检测。它比较给定实例周围的实例密度与该实例邻居周围的密度。异常通常比其 k 个最近邻更加孤立。

单类 SVM（One-class SVM）

该算法更适合新颖性检测。回顾一下，核化 SVM 分类器首先（隐式地）将所有实例映射到高维空间，然后在这个高维空间中使用线性 SVM 分类器分隔两个类别（参见 SVM 在线章节：https://homl.info）。由于这里只有一个实例类别，单类 SVM 算法会尝试在高维空间中把这些实例与原点分隔开。在原始空间中，这相当于寻找一个包围全部实例的小区域。如果新实例落在这个区域之外，就会被视为异常。需要调整的超参数包括核化 SVM 的常规超参数，以及一个间隔超参数；它对应于把实际正常的新实例错误地视为新颖实例的概率。该算法在高维数据集上表现很好，但和所有 SVM 一样，无法扩展到大型数据集。

带有 inverse_transform() 方法的 PCA 及其他降维技术

如果比较正常实例和异常实例的重构误差，后者通常会大得多。这是一种简单且经常很有效的异常检测方法，本章练习中会给出示例。

8.6　练习（Exercises）

1. 你会如何定义聚类？能说出几种聚类算法吗？
2. 聚类算法有哪些主要应用？
3. 使用 k-means 时，有哪两种技术可以用来选择合适的簇数量？
4. 什么是标签传播？为什么要实现它，以及如何实现？
5. 说出两种能够扩展到大型数据集的聚类算法，以及两种寻找高密度区域的聚类算法。
6. 你能想到一个适合使用主动学习的场景吗？你会如何实现？
7. 异常检测和新颖性检测有什么区别？
8. 什么是高斯混合？可以用它完成哪些任务？
9. 使用高斯混合模型时，有哪两种技术可以用来确定合适的簇数量？
10. 经典的 Olivetti faces 数据集包含 400 张灰度人脸图像，每张图像大小为 64 × 64 像素。每张图像被展平为大小为 4,096 的一维向量。数据集中有 40 个不同的人，

每个人拍摄 10 次；通常的任务是训练一个模型，预测每张图像中是哪一个人。使用 `sklearn.datasets.fetch_olivetti_faces()` 函数加载数据集，然后将其划分为训练集、验证集和测试集（注意，数据集已经缩放到 0 到 1 之间）。由于数据集很小，你可能需要使用分层抽样，确保每个集合中每个人的图像数量相同。接着使用 k-means 对图像进行聚类，并使用本章介绍的某种技术确保簇数量合适。将簇可视化：每个簇中是否能看到相似的人脸？

11. 继续使用 Olivetti faces 数据集，训练一个分类器来预测每张图像中的人物，并在验证集上评估它。然后使用 k-means 作为降维工具，在降维后的数据集上训练分类器。搜索能让分类器获得最佳性能的簇数量：能达到什么性能？如果把降维后的特征追加到原始特征之后（再次搜索最佳簇数量），结果如何？

12. 在 Olivetti faces 数据集上训练一个高斯混合模型。为了加速算法，你可能需要先降低数据集的维度（例如使用 PCA 保留 99% 的方差）。使用模型生成一些新的人脸（调用 `sample()` 方法）并将其可视化（如果使用了 PCA，就需要调用它的 `inverse_transform()` 方法）。尝试修改一些图像（例如旋转、翻转、变暗），观察模型能否检测出异常，也就是比较正常图像与异常图像的 `score_samples()` 方法输出。

13. 某些降维技术也可以用于异常检测。例如，使用 Olivetti faces 数据集，并用 PCA 降维，同时保留 99% 的方差。然后计算每张图像的重构误差。接着取上一道练习中构造的一些修改图像，观察它们的重构误差：注意误差大了多少。如果绘制重构图像，就会明白原因：算法会尝试重构出一张正常的人脸。

这些练习的解答位于本章笔记本末尾：https://homl.info/colab-p。

脚注 7：与前面所见相反，k-means 隐含地假设簇大小和密度相近，并且大致呈圆形。

脚注 8：Phi（phi 或 φ）是希腊字母表中的第 21 个字母。

注：原 PDF 第 219 页是第二部分“神经网络与深度学习”的分隔页，不属于第 8 章正文。

第二部分：神经网络与深度学习

Part II. Neural Networks and Deep Learning

第 9 章　人工神经网络简介

Introduction to Artificial Neural Networks

鸟类启发了人类制造飞机，牛蒡启发了魔术贴，而大自然启发了无数发明。因此，研究大脑的结构，从中寻找构建智能机器的灵感，似乎顺理成章。这正是人工神经网络（artificial neural networks，ANN）的思想来源：人工神经网络是受大脑中生物神经元网络启发的机器学习模型。不过，飞机虽然受到鸟类启发，却不需要拍打翅膀才能飞行。同样，ANN 也逐渐变得与它们的生物近亲大不相同。一些研究人员甚至认为，我们应该完全抛开生物学类比，例如使用“单元（units）”而不是“神经元（neurons）”这个说法，以免把创造力限制在生物学上合理的系统之内[1]。

ANN 是深度学习的核心。它们通用、强大且可扩展，非常适合处理大型而复杂的机器学习任务，例如对数十亿张图像进行分类（如 Google 图片），支持语音识别服务（如 Apple 的 Siri 或 Google Assistant）和聊天机器人（如 ChatGPT 或 Claude），每天为数亿用户推荐最值得观看的视频（如 YouTube），或者学习蛋白质如何折叠（如 DeepMind 的 AlphaFold）。

本章介绍人工神经网络，先快速回顾最早的 ANN 架构，再逐步介绍如今广泛使用的多层感知器（multilayer perceptrons，MLP；后续章节还会介绍许多其他架构）。本章将使用 Scikit-Learn 实现简单的 MLP，帮助你入门；下一章将转而使用 PyTorch，因为它是更加灵活、高效的神经网络库。

现在，让我们回到人工神经网络的起源。

ANN 的早期成功让人们普遍相信，我们很快就能与真正智能的机器对话。但到了 20 世纪 60 年代，人们发现这一承诺还无法实现（至少短期内如此），研究资金转向了其他方向，ANN 进入了漫长的低谷期。20 世纪 80 年代初，人们发明了新的架构并开发出更好的训练技术，重新激发了对联结主义（connectionism，即对神经网络的研究）的兴趣。但进展依然缓慢。到了 20 世纪 90 年代，支持向量机等其他强大的机器学习技术出现了；与 ANN 相比，这些技术似乎效果更好，理论基础也更强，因此神经网络研究再次被搁置。

[1] 只要模型效果良好，就可以在接受生物学启发的同时，不惧怕创建并不符合生物学现实的模型，从而兼得两者的优点。

如今，我们正在经历新一轮对 ANN 的兴趣高峰。这一轮会不会像之前几轮那样消退？下面是一些理由，可以说明这一次有所不同，ANN 的复兴可能会对我们的生活产生更加深远的影响：

- 现在有大量数据可用于训练神经网络，而且在非常大型、复杂的问题上，ANN 经常优于其他机器学习技术。
- 自 20 世纪 90 年代以来，计算能力大幅提升，如今可以在合理的时间内训练大型神经网络。这一方面得益于摩尔定律（过去 50 年中，集成电路中的元件数量大约每两年翻一番），另一方面也得益于游戏产业。游戏产业推动了数百万个强大图形处理单元（GPU）的生产：GPU 卡最初用于加速图形处理，但神经网络执行的某些计算（例如大规模矩阵乘法）与图形处理类似，因此也可以使用 GPU 加速。此外，云平台让每个人都能使用这些计算能力。
- 训练算法得到了改进。严格来说，它们与 20 世纪 90 年代使用的算法只有细微差异，但这些相对较小的调整带来了巨大的积极影响。
- ANN 的一些理论局限在实践中证明并不严重。例如，许多人认为 ANN 训练算法很可能陷入局部最优，因此注定失败；但事实证明，在实践中这并不是大问题，尤其是对于更大的神经网络：局部最优通常与全局最优的表现几乎一样好。
- 2017 年 Transformer 架构的发明（见第 15 章）改变了整个局面：与早期更加专门化的架构不同，Transformer 可以处理和生成各种数据（例如文本、图像和音频），并且在从机器人到蛋白质折叠的各种任务上都有出色表现。此外，它的扩展性相当好，因此可以训练非常大型的基础模型，并将它们复用于许多不同任务；必要时只需稍作微调（这就是迁移学习），或者以正确方式提示模型（这就是上下文学习，即 ICL）。例如，可以给模型几个任务示例（这就是少样本学习，即 FSL），也可以要求它逐步推理（这就是思维链提示，即 CoT）。这是一个全新的世界！
- ANN 似乎已经进入了资金与进展相互促进的良性循环。基于 ANN 的惊人产品不断登上新闻头条，吸引越来越多的关注和资金，带来更多进展和更加惊人的产品。AI 不再只是隐藏在产品背后提供动力：自从 ChatGPT 等聊天机器人发布以来，公众已经开始每天直接与 AI 助手互动，各大科技公司也在激烈争夺这个巨大的市场，创新速度非常惊人。

9.1　从生物神经元到人工神经元（From Biological to Artificial Neurons）

9.1.1　生物神经元（Biological Neurons）

在讨论人工神经元之前，先快速了解一下生物神经元（如图 9-1 所示）。这是一种外形独特的细胞，主要存在于动物的大脑中。它由包含细胞核和大部分复杂细胞器的细胞体、许多称为树突的分支状延伸，以及一条很长的轴突组成。轴突的长度可能只是细胞体长度的几倍，也可能达到细胞体长度的数万倍。靠近末端时，轴突会分出许多称为轴突末梢的分支；这些分支的顶端有微小结构，称为突触末梢（或简称突触），它们与其他神经元的树突或细胞体相连[2]。生物神经元会产生称为动作电位（action potentials，AP；也简称信号）的短促电脉冲。这些脉冲沿轴突传播，并促使突触释放称为神经递质的化学信号。当神经元在几毫秒内接收到足够多的神经递质时，就会发出自己的电脉冲（具体情况取决于神经递质，因为其中一些会抑制神经元放电）。

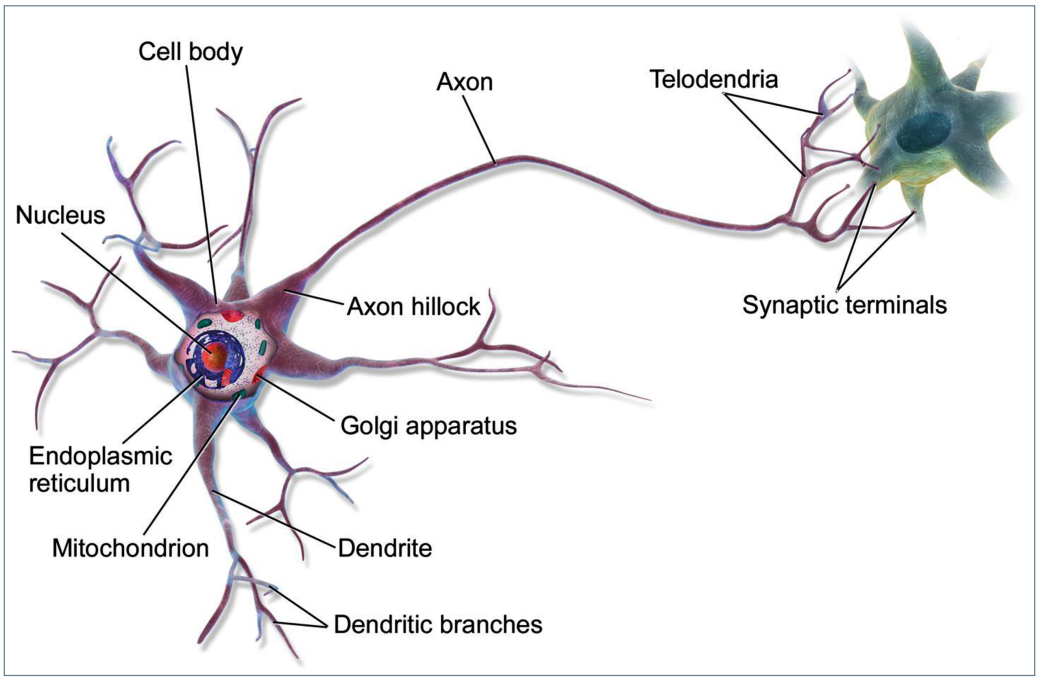

图 9-1: 生物神经元（A biological neuron）

[2]突触实际上并未直接接触，它们之间的距离很近，因此可以非常迅速地交换化学信号。

图 9-1 的图片来源见脚注[3]。

因此，单个生物神经元的行为似乎很简单，但它们组成了一个包含数十亿个神经元的庞大网络，每个神经元通常都与数千个其他神经元相连。正如复杂的蚁丘可以由简单蚂蚁的共同努力形成一样，高度复杂的计算也可以由大量相当简单的神经元网络完成。生物神经网络（biological neural networks，BNN）的架构仍是活跃的研究领域，但人们已经绘制出了大脑部分区域的结构。这些研究表明，神经元经常被组织成连续的层，尤其是在大脑外层的大脑皮层中，如图 9-2 所示[4][5]。

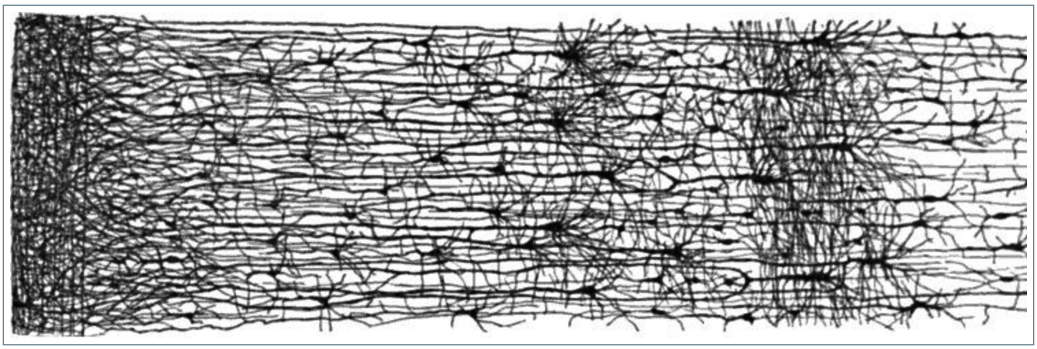

图 9-2: 生物神经网络中的多层结构（人类大脑皮层）（Multiple layers in a biological neural network, human cortex）

9.1.2　用神经元进行逻辑计算（Logical Computations with Neurons）

McCulloch 和 Pitts 提出了一个非常简单的生物神经元模型，这个模型后来被称为人工神经元：它有一个或多个二元（开/关）输入和一个二元输出。当超过一定数量的输入处于激活状态时，人工神经元就会激活其输出。在他们的里程碑论文《神经活动中内在思想的逻辑演算》（A Logical Calculus of Ideas Immanent in Nervous Activity）中[6]，McCulloch 和 Pitts 证明，即使使用如此简化的模型，也可以构建一个人工神经元网络，利用命题逻辑计算任意逻辑命题。这是第一个人工神经网络架构。为了了解这种网络的工作方式，我们来构建几个执行不同逻辑计算的 ANN（见图 9-3），并假设当一个神经元至少有两条输入连接处于激活状态时，它就会被激活。

[3]图片由 Bruce Blaus 创作，采用 Creative Commons 3.0 许可。图片重新制作用于本书，来源：https://en.wikipedia.org/wiki/Neuron。

[4]在机器学习语境中，“神经网络”通常指 ANN，而不是 BNN。

[5]图中展示的是 S. Ramon y Cajal 绘制的大脑皮层分层结构，属于公共领域。来源：https://en.wikipedia.org/wiki/Cerebral_cortex。

[6]Warren S. McCulloch 和 Walter Pitts，“A Logical Calculus of the Ideas Immanent in Nervous Activity”，The Bulletin of Mathematical Biology 5，第 4 期（1943）：115-113。

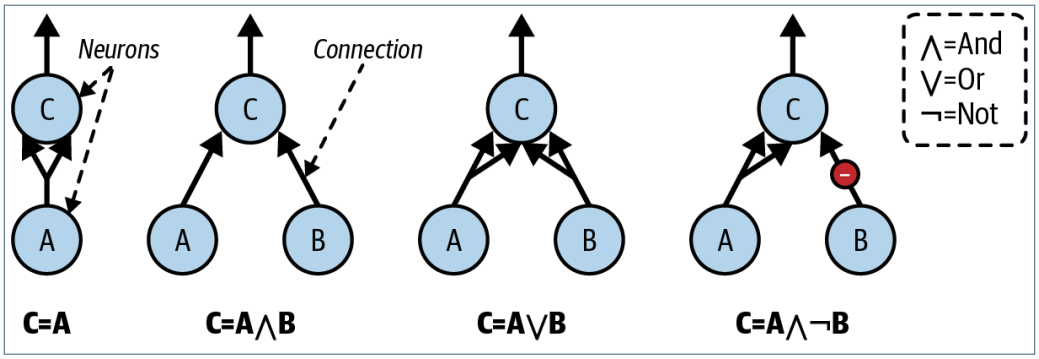

图 9-3: 执行简单逻辑计算的 ANN（ANNs performing simple logical computations）

来看看这些网络的功能：

- 左侧第一个网络是恒等函数：如果神经元 A 被激活，那么神经元 C 也会被激活（因为它从神经元 A 接收到两个输入信号）；但如果神经元 A 关闭，神经元 C 也会关闭。
- 第二个网络执行逻辑 AND：只有当神经元 A 和 B 都被激活时，神经元 C 才会被激活（单个输入信号不足以激活神经元 C）。
- 第三个网络执行逻辑 OR：只要神经元 A 或神经元 B 被激活（或者两者都被激活），神经元 C 就会被激活。
- 最后，假设一条输入连接可以抑制神经元的活动（生物神经元就是如此），那么第四个网络可以计算稍微复杂一些的逻辑命题：只有神经元 A 处于激活状态且神经元 B 关闭时，神经元 C 才会被激活。如果神经元 A 始终处于激活状态，就可以得到逻辑 NOT：神经元 B 关闭时神经元 C 激活，反之亦然。

你可以想象，这些网络还可以组合起来计算复杂的逻辑表达式（本章末练习中有一道相关示例）。

9.1.3　感知器（The Perceptron）

感知器（perceptron）是最简单的 ANN 架构之一，由 Frank Rosenblatt 于 1957 年发明。它基于一种略有不同的人工神经元（见图 9-4），称为阈值逻辑单元（threshold logic unit，TLU），有时也称为线性阈值单元（linear threshold unit，LTU）。其输入和输出是数值，而不是二元的开/关值；每条输入连接都关联一个权重。TLU 首先计算输入的线性函数：$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b = \mathbf{w}^\intercal \mathbf{x} + b$。然后对结果应用阶

跃函数：$h_{\mathbf{w},b}(\mathbf{x}) = \text{step}(z)$。因此，它与逻辑回归非常相似，只是使用阶跃函数而不是逻辑函数[7]。和逻辑回归一样，模型参数是输入权重 **w** 和偏置项 b。

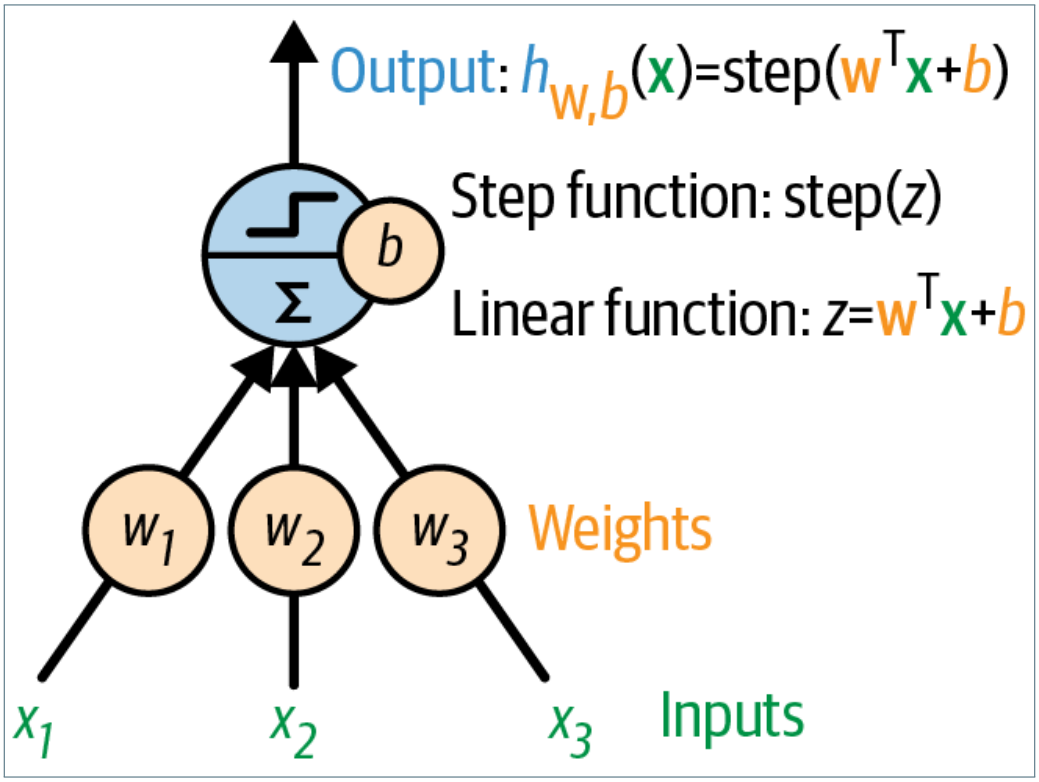

图 9-4: TLU：计算输入加权和 $\mathbf{w}^T\mathbf{x}$ 并加上偏置项 b，然后应用阶跃函数的人工神经元（TLU: an artificial neuron that computes a weighted sum of its inputs, plus a bias term, then applies a step function）

感知器中最常用的阶跃函数是 Heaviside 阶跃函数（见公式 9-1）。有时也会使用符号函数。

$$\text{heaviside}(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases} \qquad \text{sgn}(z) = \begin{cases} -1 & \text{if } z < 0 \\ 0 & \text{if } z = 0 \\ +1 & \text{if } z > 0 \end{cases} \tag{9-1}$$

公式 9-1　感知器中常用的阶跃函数（阈值设为 0）（Common step functions used in perceptrons, assuming threshold = 0）

[7] 逻辑回归和逻辑函数在第 4 章介绍；本章还会大量使用 Softmax、交叉熵、梯度下降、提前停止等概念，请先阅读第 4 章。

单个 TLU 可用于简单的线性二元分类。它计算输入的线性函数；如果结果超过某个阈值，就输出正类，否则输出负类。这可能会让你想到逻辑回归（第 4 章）或线性 SVM 分类（参见在线 SVM 章节：https://homl.info）。例如，可以使用单个 TLU 根据花瓣长度和宽度对鸢尾花进行分类。训练这样的 TLU，需要找出 w_1、w_2 和 b 的正确取值（训练算法很快就会介绍）。

感知器由一个或多个 TLU 组成，这些 TLU 被组织在单层中，并且每个 TLU 都与所有输入相连。这样的层称为全连接层（fully connected layer），也称为稠密层（dense layer）。输入构成输入层。由于 TLU 层产生最终输出，因此称为输出层。例如，一个包含两个输入和三个输出的感知器如图 9-5 所示。

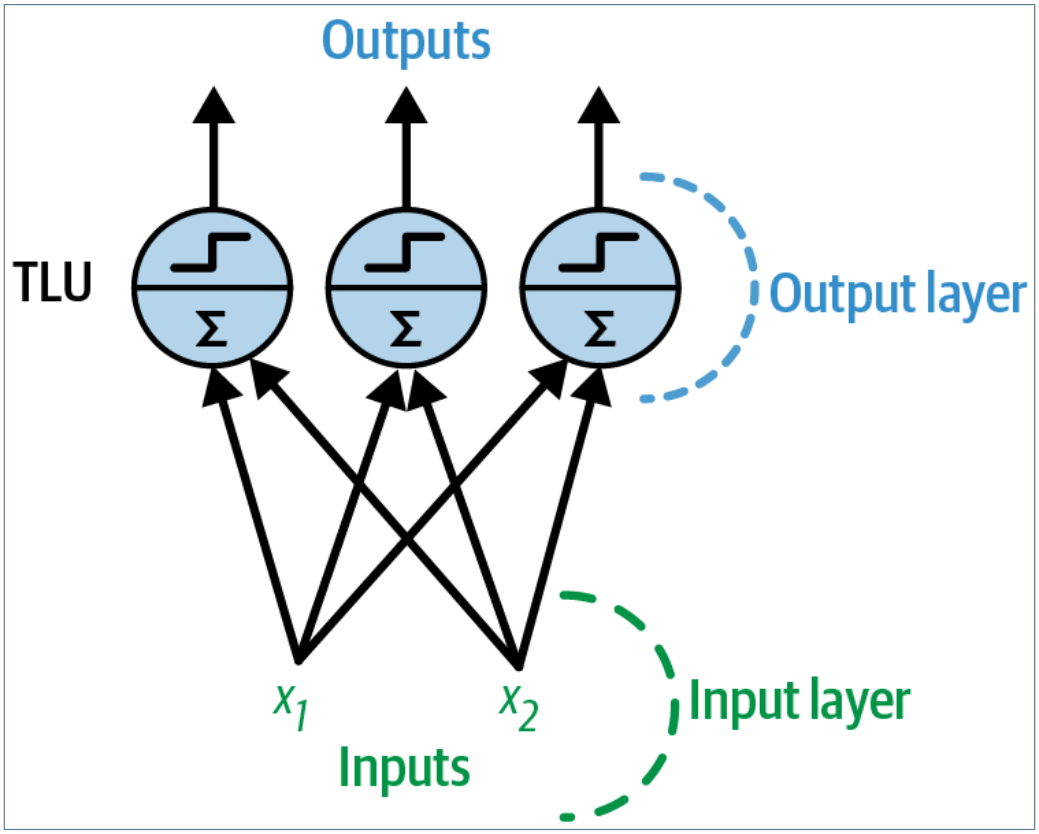

图 9-5: 含有两个输入和三个输出神经元的感知器架构（Architecture of a perceptron with two inputs and three output neurons）

这个感知器可以同时把实例分为三个不同的二元类别，因此它是一个多标签分类器，也可以用于多分类。

借助线性代数的力量，公式 9-2 可以高效地同时计算多个实例通过一层人工神经元后的输出。

$$\hat{Y} = \varphi(XW + b) \tag{9-2}$$

公式 9-2　计算全连接层的输出（Computing the outputs of a fully connected layer）

在这个公式中：

- $\hat{Y}$ 是输出矩阵。它每一行对应一个实例，每一列对应一个神经元。
- X 是输入矩阵。它每一行对应一个实例，每一列对应一个输入特征。
- 权重矩阵 W 包含所有连接权重。它每一行对应一个输入特征，每一列对应一个神经元[8]。
- 偏置向量 b 包含所有偏置项，每个神经元对应一个。
- 函数 φ 称为激活函数：当人工神经元是 TLU 时，它是阶跃函数（稍后还会讨论其他激活函数）。

> 注意：在数学中，矩阵与向量相加是没有定义的。不过在数据科学中，我们允许“广播”：把一个向量加到矩阵上，意味着把它加到矩阵的每一行。因此，$XW + b$ 会先将 X 与 W 相乘，得到一个矩阵，其中每行对应一个实例、每列对应一个输出；然后把向量 b 加到这个矩阵的每一行上，也就是把每个偏置项加到每个实例对应的输出上。最后，φ 会逐元素应用于结果矩阵中的每个元素。

那么，感知器如何训练？Rosenblatt 提出的感知器训练算法在很大程度上受到 Hebb 规则的启发。Donald Hebb 在 1949 年出版的《行为组织》（The Organization of Behavior，Wiley）一书中提出：当一个生物神经元经常触发另一个神经元时，这两个神经元之间的连接会变得更强。Siegrid Löwel 后来用一句朗朗上口的话概括了 Hebb 的观点：“一起放电的细胞，会连接在一起（Cells that fire together, wire together）”；也就是说，两个神经元同时放电时，它们之间的连接权重往往会增加。

后来，感知器使用了这条规则的一个变体，并把网络做出预测时产生的错误考虑在内；感知器学习规则会强化有助于减小错误的连接。更具体地说，感知器每次接收一个训练实例，并对每个实例做出预测。对于每个产生错误预测的输出神经元，它都会强化那些本应有助于得到正确预测的输入连接。公式 9-3 展示了这条规则。

[8] 在 PyTorch 等一些库中，权重矩阵会转置，因此每一行对应一个神经元，每一列对应一个输入特征。

$$w_{i,j}^{(\text{next step})} = w_{i,j} + \eta\left(y_j - \hat{y}_j\right) x_i \tag{9-3}$$

公式 9-3　感知器学习规则（权重更新）（Perceptron learning rule, weight update）

在这个公式中：

- $w_{i,j}$ 是第 i 个输入与第 j 个神经元之间的连接权重。
- x_i 是当前训练实例的第 i 个输入值。
- $\hat{y}_j$ 是当前训练实例通过第 j 个输出神经元得到的输出。
- y_j 是当前训练实例中第 j 个输出神经元的目标输出。
- η 是学习率（见第 4 章）。

每个输出神经元的决策边界都是线性的，因此感知器无法学习复杂模式（就像逻辑回归分类器一样）。不过，如果训练实例是线性可分的，Rosenblatt 证明了该算法会收敛到某个解[9]。这称为感知器收敛定理。Scikit-Learn 提供了一个 Perceptron 类，使用方式基本符合预期。例如，可以在第 4 章介绍过的 iris 数据集上使用它：

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron
iris = load_iris(as_frame=True)
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = (iris.target == 0)  # Iris setosa
per_clf = Perceptron(random_state=42)
per_clf.fit(X, y)
X_new = [[2, 0.5], [3, 1]]
y_pred = per_clf.predict(X_new)  # predicts True and False for these 2 flowers
```

你可能已经注意到，感知器学习算法与随机梯度下降（第 4 章介绍）非常相似。事实上，Scikit-Learn 的 Perceptron 类等价于使用以下超参数的 SGDClassifier：`loss="perceptron"`、`learning_rate="constant"`、`eta0=1`（学习率）和 `penalty=None`（不进行正则化）。

> 注意：与逻辑回归分类器不同，感知器不会输出类别概率。这是倾向于使用逻辑回归而不是感知器的一个原因。此外，感知器默认不使用任何正则化，而且当训练集上不再出现预测错误时就会停止训练，因此模型通常不如逻辑回归或

[9] 这个解并不唯一：当数据点线性可分时，可以把它们分开的超平面有无穷多个。

> 线性 SVM 分类器具有良好的泛化能力。不过，感知器的训练速度可能会快一些。

在 1969 年出版的专著《感知器》（Perceptrons）中，Marvin Minsky 和 Seymour Papert 指出了感知器的一些严重缺陷，尤其是它们无法解决一些看似简单的问题（例如异或（exclusive OR，XOR）分类问题；见图 9-6 左侧）。其他线性分类模型（例如逻辑回归分类器）也有同样的问题，但研究人员原本对感知器抱有更高期望，因此其中一些人非常失望，转而完全放弃神经网络，投入逻辑、问题求解和搜索等更加形式化的方法。实际应用的缺乏也没有帮助。

事实证明，通过叠加多个感知器，可以消除感知器的一些局限。这样得到的 ANN 称为多层感知器（multilayer perceptron，MLP）。

9.1.4　多层感知器和反向传播（The Multilayer Perceptron and Backpropagation）

MLP 可以解决 XOR 问题。你可以通过计算图 9-6 右侧 MLP 的输出来验证这一点：输入为（0，0）或（1，1）时，网络输出 0；输入为（0，1）或（1，0）时，网络输出 1。试着验证这个网络确实能解决 XOR 问题[10]！

[10]例如，当输入为（0，1）时，左下方神经元计算得到 0 × 1 + 1 × 1 - 3 / 2 = -1 / 2，这是负数，因此输出 0。右下方神经元计算得到 0 × 1 + 1 × 1 - 1 / 2 = 1 / 2，这是正数，因此输出 1。输出神经元接收前两个神经元的输出作为输入，因此计算得到 0 × (-1) + 1 × 1 - 1 / 2 = 1 / 2，这是正数，因此输出 1。

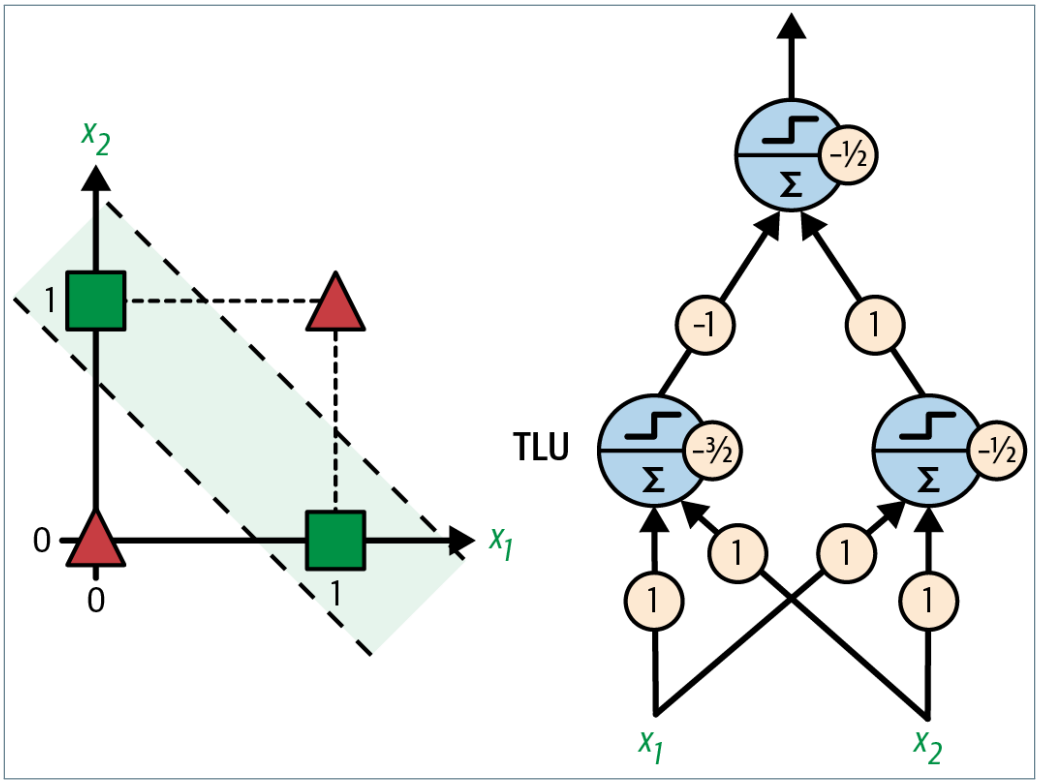

图 9-6: XOR 分类问题以及解决该问题的 MLP（XOR classification problem and an MLP that solves it）

MLP 由一个输入层、一个或多个由人工神经元（最初是 TLU）组成的层（称为隐藏层），以及一个由人工神经元组成的最终层（称为输出层）构成（见图 9-7）。靠近输入层的层通常称为低层，靠近输出的层通常称为高层。

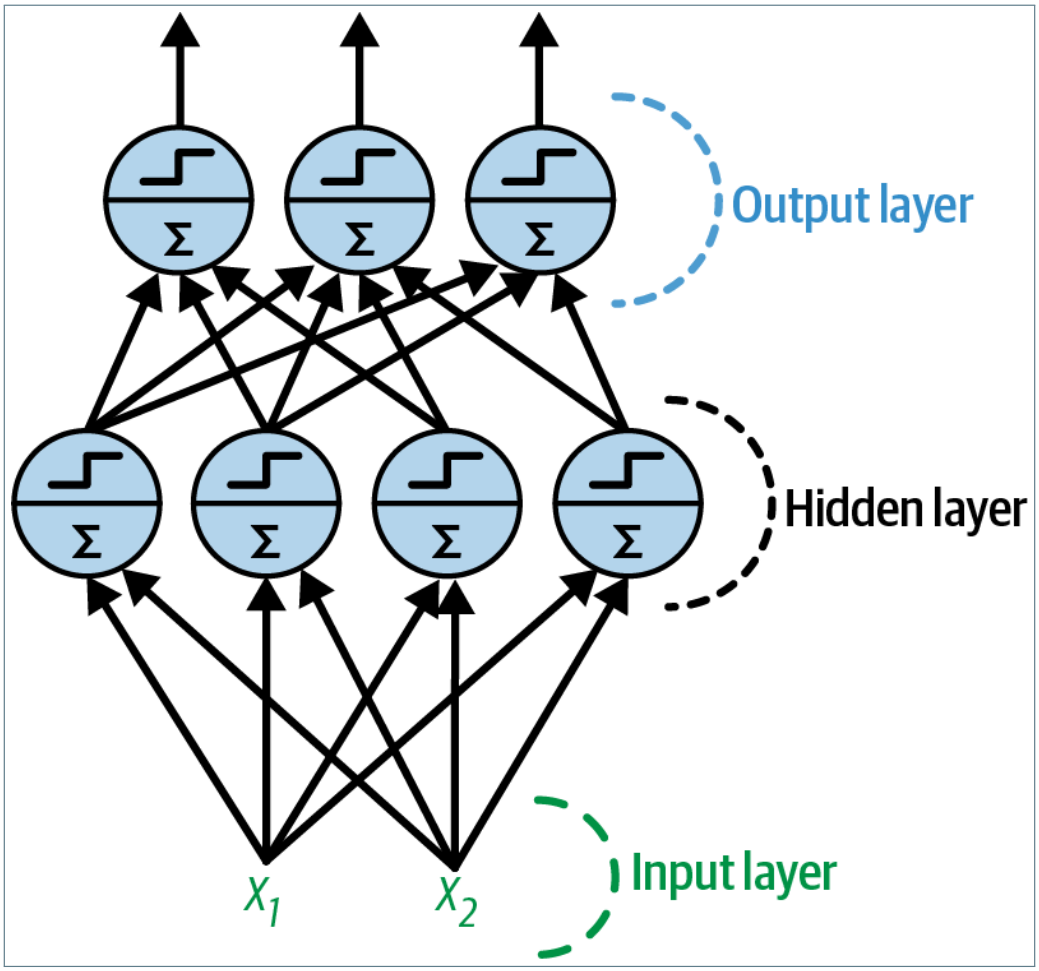

图 9-7: 含有两个输入、一个包含四个神经元的隐藏层和三个输出神经元的多层感知器架构（Architecture of a multilayer perceptron with two inputs, one hidden layer of four neurons, and three output neurons）

> 注意：信号只沿一个方向流动（从输入流向输出），因此这种架构是前馈神经网络（feedforward neural network，FNN）的一个示例。

当 ANN 包含由多个隐藏层组成的深层堆叠时[11]，就称为深度神经网络（deep neural network，DNN）。深度学习领域研究 DNN，更广义地说，它关注包含深层计算堆叠的模型。不过，只要涉及神经网络，很多人都会使用“深度学习”这个说法，即使网络并不深。

[11] 20 世纪 90 年代，包含两个以上隐藏层的 ANN 就被视为深度网络。如今，包含几十层甚至几百层的 ANN 已很常见，因此“深度”的定义相当模糊。

多年来，研究人员一直难以找到训练 MLP 的方法。20 世纪 60 年代初，一些研究人员讨论过使用梯度下降训练神经网络的可能性，但正如第 4 章所见，这需要计算模型误差相对于模型参数的梯度；在当时，人们并不清楚如何高效地处理这种包含大量参数的复杂模型，尤其是在当时的计算机条件下。

后来，1970 年，一位名为 Seppo Linnainmaa 的研究人员在硕士论文中提出了一种自动、高效计算所有梯度的技术。这种算法现在称为反向模式自动微分（reverse-mode automatic differentiation，简称 reverse-mode autodiff）。它只需对网络执行两次遍历（一次正向、一次反向），就能计算神经网络误差相对于每个模型参数的梯度。换句话说，它可以找出每个连接权重和偏置应该如何调整，才能减小神经网络的误差。随后可以利用这些梯度执行一次梯度下降。如果反复自动计算梯度并执行梯度下降，神经网络的误差就会逐渐降低，最终达到某个最小值。反向模式自动微分与梯度下降的结合，现在称为反向传播（backpropagation，简称 backprop）。

这里可以用一个类比来理解：假设你正在学习把篮球投进篮筐。你把球投出去（这就是正向传播），观察到篮球偏向右侧很远的位置（这就是计算误差），然后思考下一次怎样调整身体姿势，让球少偏向右侧一些（这就是反向传播）：你意识到手臂需要稍微逆时针旋转，上半身可能也要一起调整，这又意味着双脚也需要转动（注意，我们正在沿着“层”逐步向下追溯）。思考完毕后，你真正移动身体（这就是梯度下降步骤）。误差越小，调整幅度越小。你反复进行整个过程很多次，误差会逐渐变小；练习几个小时后，你就能每次都把球投进篮筐。做得好！

> 注意：自动微分有多种技术，各有优缺点。当待求导函数包含许多变量（例如连接权重和偏置）但输出很少（例如一个损失）时，反向模式自动微分非常适合。如果想进一步了解自动微分，请参见附录 A。

反向传播实际上可以应用于各种计算图，而不仅仅是神经网络；Linnainmaa 的硕士论文其实也不是关于神经网络的，而是更加一般的主题。反向传播在若干年后才开始用于训练神经网络，而且当时仍未成为主流。1985 年，David Rumelhart、Geoffrey Hinton 和 Ronald Williams 发表了一篇论文，分析反向传播如何让神经网络学习有用的内部表示[12]。他们的结果非常令人印象深刻，因此反向传播很快在该领域流行起来。40 多年过去了，它仍然是神经网络最常用的训练技术。

下面更详细地回顾反向传播的工作方式：

[12] David Rumelhart 等人，“Learning Internal Representations by Error Propagation”，国防技术信息中心技术报告，1985 年 9 月。

- 它每次处理一个小批量，并多次遍历完整训练集。如果每个小批量包含 32 个实例，每个实例有 100 个特征，那么该小批量将表示为一个包含 32 行、100 列的矩阵。每次遍历训练集称为一个 epoch。
- 对于每个小批量，算法使用公式 9-2 计算第一隐藏层中所有神经元的输出。如果该层有 50 个神经元，那么输出就是一个矩阵：每行对应小批量中的一个样本（例如 32），每列对应一个神经元（即 50 列）。然后把这个矩阵传给下一层，计算下一层的输出，再把结果传给下一层，如此继续，直到得到最后一层，也就是输出层的输出。这就是正向传播：它与进行预测完全相同，只是会保留所有中间结果，因为反向传播需要这些结果。
- 接下来，算法度量网络输出的误差，也就是使用一个损失函数比较网络期望的输出和实际输出，并返回某种误差度量。
- 然后，算法计算输出层的每个参数对误差的贡献。这个过程通过应用链式法则（微积分中最基本的规则之一）以解析方式完成，因此既快速又精确。结果是每个参数对应一个梯度。
- 接着，算法再次使用链式法则，计算这些误差贡献中有多少来自下一层的每条连接，并逐层向后计算，直到输入层。如前所述，这次反向遍历通过在网络中反向传播误差梯度，高效地计算所有连接权重和偏置的误差梯度。
- 最后，算法利用刚刚计算出的误差梯度执行梯度下降步骤，调整网络中的所有连接权重和偏置项。

> 警告：必须随机初始化所有隐藏层的连接权重，否则训练会失败。例如，如果把所有权重和偏置都初始化为 0，那么同一层中的所有神经元都会完全相同，反向传播也会以完全相同的方式影响它们，因此它们会一直保持相同。换句话说，尽管每层有数百个神经元，你的模型表现得却仿佛每层只有一个神经元，并不会太聪明。如果随机初始化权重，就能打破这种对称性，让反向传播训练出一组各不相同的神经元。

简而言之，反向传播先为一个小批量做出预测（正向传播），度量误差，然后反向遍历各层，计算每个参数的误差贡献（反向传播），最后调整连接权重和偏置，以减小误差（梯度下降步骤）。

为了让反向传播正常工作，Rumelhart 和同事对 MLP 架构做了一个关键改变：他们用逻辑函数 $\sigma(z) = 1/(1 + \exp(-z))$（也称 sigmoid 函数）替换了阶跃函数。这一

点至关重要，因为阶跃函数只有平坦线段，没有可用的梯度（梯度下降无法在平面上移动）；而 sigmoid 函数处处都有定义良好的非零导数，因此梯度下降每一步都能取得一定进展。事实上，反向传播算法可以很好地配合许多其他激活函数，而不只是 sigmoid 函数。下面是另外两种常用选择。

双曲正切函数]] [[MATH:\{}tanh(z)=2\{}sigma(2z)-1 与 sigmoid 函数一样，该激活函数呈 S 形、连续且可微，但其输出值范围是 -1 到 1（sigmoid 函数的范围是 0 到 1）。在训练开始时，这个范围往往能让每层输出大致以 0 为中心，从而加快收敛。

修正线性单元函数]] [[MATH:\{}operatorname{ReLU}(z)=\{}max(0,z) ReLU 函数是连续的，但遗憾的是在 $z = 0$ 处不可微（斜率会突然改变，可能导致梯度下降来回波动），并且当 $z < 0$ 时其导数为 0。不过在实践中，它的效果非常好，而且计算速度快，因此已经成为大多数架构的默认激活函数（Transformer 架构除外，见第 15 章）[13]。重要的是，它没有最大输出值，这有助于减少梯度下降过程中的一些问题（第 11 章还会回到这个话题）。

图 9-8 展示了这些常用激活函数及其导数。不过，等等！我们为什么一开始就需要激活函数？如果把多个线性变换串联起来，得到的仍然只是线性变换。例如，如果 $f(x) = 2x + 3$、$g(x) = 5x - 1$，那么串联这两个线性函数会得到另一个线性函数：$f(g(x)) = 2(5x - 1) + 3 = 10x + 1$。因此，如果层与层之间没有非线性，即使是层层堆叠的深层网络，也等价于一个单层网络，无法解决非常复杂的问题。反过来，具有非线性激活函数且规模足够大的 DNN，在理论上可以逼近任意连续函数。

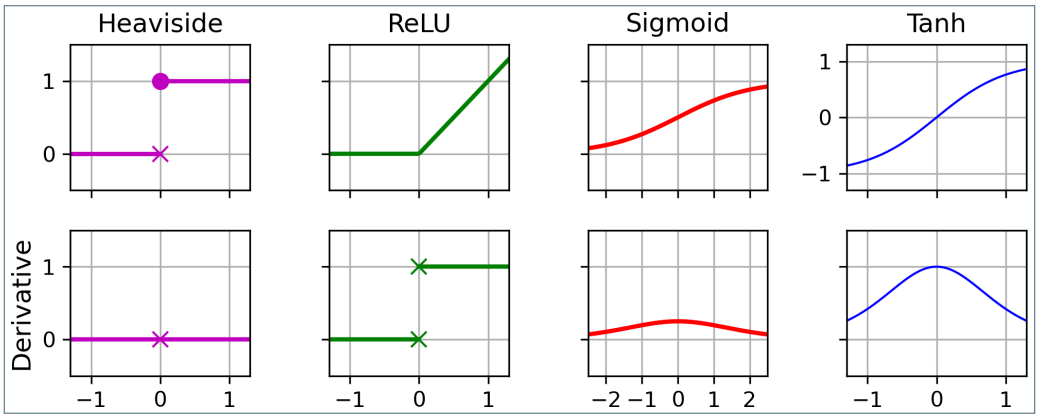

图 9-8: 激活函数（左）及其导数（右）（Activation functions, left, and their derivatives, right）

[13]生物神经元似乎实现了大致呈 sigmoid 形状的激活函数，因此研究人员长期坚持使用 sigmoid 函数。但事实证明，ReLU 通常在 ANN 中表现更好。这是生物学类比可能产生误导的例子之一。

好了，现在你已经了解神经网络的来源、MLP 的架构和它计算输出的方式，也学会了反向传播算法。下面该看看 MLP 的实际应用了！

9.2　使用 Scikit-Learn 构建和训练 MLP（Building and Training MLPs with Scikit-Learn）

MLP 可以处理各种任务，但最常见的任务是回归和分类。Scikit-Learn 可以同时支持这两类任务。先从回归开始。

9.2.1　回归 MLP（Regression MLPs）

如何为回归任务构建 MLP？如果想预测一个值（例如根据房屋的多个特征预测房价），只需要一个输出神经元，它的输出就是预测值。如果是多元回归（即同时预测多个值），则需要为每个输出维度设置一个输出神经元。例如，要定位图像中某个物体的中心，需要预测二维坐标，因此需要两个输出神经元。如果还要在物体周围放置边界框，就需要再预测两个数值：物体的宽度和高度。这样最终需要四个输出神经元。

Scikit-Learn 包含一个 MLPRegressor 类，因此我们使用它构建一个 MLP，其中有三个隐藏层，每层包含 50 个神经元，并在加利福尼亚住房数据集上训练。为简单起见，使用 Scikit-Learn 的 **`fetch_california_housing()`** 函数加载数据。这个数据集比第 2 章使用的数据集简单，因为它只包含数值特征（没有 **`ocean_proximity`** 特征），也没有缺失值。目标值也经过缩放：每个单位代表 100,000 美元。先导入所需内容：

```
from sklearn.datasets import fetch_california_housing
from sklearn.metrics import root_mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
```

接下来获取加利福尼亚住房数据集，并将其分为训练集和测试集：

```
housing = fetch_california_housing()
X_train, X_test, y_train, y_test = train_test_split(
    housing.data, housing.target, random_state=42)
```

现在创建一个 MLPRegressor 模型，其中有 3 个隐藏层，每层包含 50 个神经元。训练开始时，第一隐藏层的输入大小（即权重矩阵的行数）和输出层的输出大小（即权重矩阵的列数）会分别根据输入和目标的维度自动调整。模型在所有隐藏层中使用 ReLU 激活函数，在输出层中完全不使用激活函数。我们还设置 **verbose=True**，以查看模型训练进度：

```
mlp_reg = MLPRegressor(hidden_layer_sizes=[50, 50, 50], early_stopping=True,
                       verbose=True, random_state=42)
```

由于神经网络可能包含大量参数，因此很容易在训练集上过拟合。降低这种风险的一种方法是使用提前停止（第 4 章介绍）：设置 **early_stopping=True** 后，MLPRegressor 类会自动留出 10% 的训练数据，并在每个 epoch 对模型进行评估（可以通过设置 **validation_fraction** 调整验证集大小）。如果验证分数连续 10 个 epoch 没有改善，训练就会自动停止（可以通过设置 **n_iter_no_change** 调整 epoch 数量）。

现在创建一个流水线，在把输入特征送入 MLPRegressor 之前先进行标准化。这一点非常重要，因为正如第 4 章所见，当特征的尺度差异很大时，梯度下降无法很好地收敛。然后就可以训练模型了！MLPRegressor 类使用一种称为 Adam 的梯度下降变体（见第 11 章）来最小化均方误差，同时还使用少量 ℓ_2 正则化（可以通过 **alpha** 超参数控制正则化强度，默认值为 0.0001）：

```
>>> pipeline = make_pipeline(StandardScaler(), mlp_reg)
>>> pipeline.fit(X_train, y_train)
Iteration 1, loss = 0.85190332
Validation score: 0.534299
Iteration 2, loss = 0.28288639
Validation score: 0.651094
[...]
Iteration 45, loss = 0.12960481
Validation score: 0.788517
Validation score did not improve more than tol=0.000100 for 10 consecutive
epochs. Stopping.
```

这样就训练出了第一个 MLP！训练需要 45 个 epoch，可以看到训练损失在每个 epoch 都下降了。这个损失对应公式 4-9 除以 2 的结果，因此必须将它乘以 2 才能得到 MSE（但并不完全准确，因为损失中还包含 ℓ_2 正则化项）。验证分数总体上也

在逐步提高。在 Scikit-Learn 的所有回归器中，MLPRegressor 默认使用 R2 分数进行评估，这就是 **score()** 方法返回的结果。正如第 2 章所见，R2 分数衡量模型解释的方差比例。本例中，验证集上的分数接近 80%，对于这个任务来说相当不错：

```
>>> mlp_reg.best_validation_score_
0.791536125425778
```

下面评估测试集上的 RMSE：

```
>>> y_pred = pipeline.predict(X_test)
>>> rmse = root_mean_squared_error(y_test, y_pred)
>>> rmse
0.5327699946812925
```

测试 RMSE 约为 0.53，与随机森林回归器在同一任务上得到的结果相当。作为第一次尝试，这个结果还不错！图 9-9 绘制了模型在测试集上的预测值与目标值。红色虚线表示理想预测（即预测值等于目标值）：大多数预测值都接近目标值，但较大的目标值仍然存在一些明显误差。

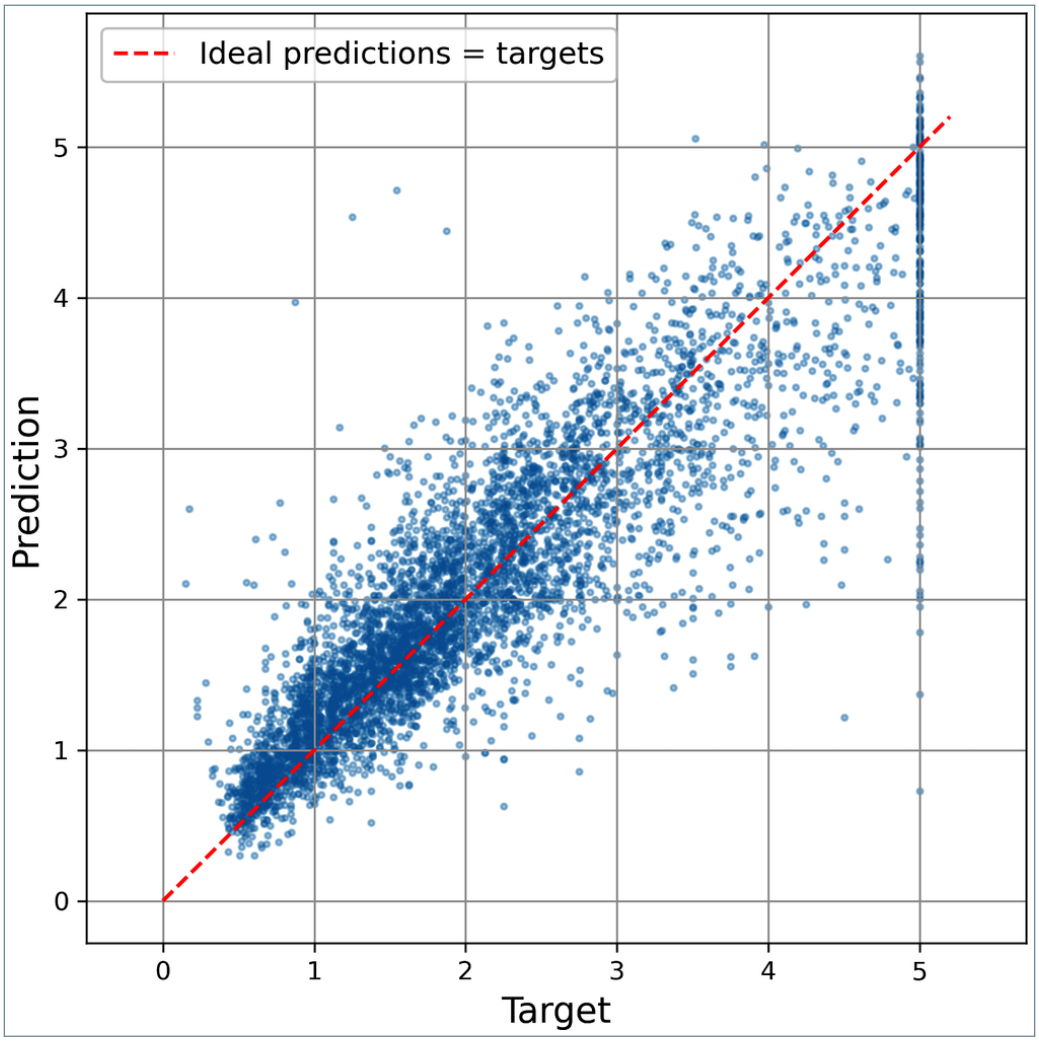

图 9-9: MLP 回归器的预测值与目标值对比（MLP regressor's predictions versus the targets）

注意，这个 MLP 的输出层没有使用激活函数，因此可以输出任意值。通常这样没有问题，但如果想保证输出始终为正，就应在输出层使用 ReLU 激活函数，或者使用 softplus 激活函数。softplus 是 ReLU 的平滑变体：$\text{softplus}(z) = \log(1 + \exp(z))$。当 z 为负时，softplus 接近 0；当 z 为正时，它接近 z。最后，如果想保证预测值始终落在给定范围内，就应使用 sigmoid 函数或双曲正切函数，并把目标值缩放到适当范围：sigmoid 的范围是 0 到 1，tanh 的范围是 -1 到 1。遗憾的是，MLPRegressor 类不支持在输出层使用激活函数。

> 警告：Scikit-Learn 不提供 GPU 加速，而且它的神经网络功能相当有限。因此，从第 10 章开始我们将切换到 PyTorch。不过，使用 Scikit-Learn 只需几行代码就能构建和训练标准 MLP，这一点非常方便，可以让你快速处理许多复杂任务。

通常，对于回归任务，均方误差是合适的损失函数。但如果训练集中包含许多离群点，有时可能更愿意使用平均绝对误差，或者更好地使用 Huber 损失。Huber 损失结合了两者的特点：当误差小于某个阈值 δ（通常为 1）时，它是二次函数；当误差大于 δ 时，它是线性函数。线性部分使它比均方误差更不容易受离群点影响，二次部分则使它比平均绝对误差收敛更快、精度更高。遗憾的是，MLPRegressor 只支持 MSE 损失。

表 9-1 总结了回归 MLP 的典型架构。

表 9-1: 回归 MLP 的典型架构

超参数	典型取值
隐藏层数量	取决于问题，通常为 1 至 5 层
每个隐藏层的神经元数量	取决于问题，通常为 10 至 100 个
输出神经元数量	每个目标维度对应 1 个
隐藏层激活函数	ReLU
输出层激活函数	无；或 ReLU/softplus（输出为正时）；或 sigmoid/tanh（输出有界时）
损失函数	MSE；存在离群点时使用 Huber

9.2.2　分类 MLP（Classification MLPs）

MLP 也可以用于分类任务。对于二元分类问题，只需要一个使用 sigmoid 激活函数的输出神经元：输出值介于 0 和 1 之间，可以将其解释为正类的估计概率。负类的估计概率等于 1 减去这个数值。

MLP 也可以轻松处理多标签二元分类任务（见第 3 章）。例如，可以构建一个电子邮件分类系统，同时预测每封邮件是正常邮件还是垃圾邮件，以及它是否紧急。在这种情况下，需要两个输出神经元，二者都使用 sigmoid 激活函数：第一个输出邮件为垃圾邮件的概率，第二个输出邮件紧急的概率。更一般地说，应为每个正类设置一个输出神经元。注意，这些输出概率不一定加起来等于 1。这让模型可以输出任意标签组合：非紧急的正常邮件、紧急的正常邮件、非紧急的垃圾邮件，甚至可能是紧急的垃圾邮件（虽然最后一种情况很可能是错误）。

如果每个实例只能属于三个或更多可能类别中的一个（例如数字图像分类中的 0 至 9 类），则需要为每个类别设置一个输出神经元，并对整个输出层使用 Softmax 激活函数（见图 9-10）。Softmax 函数（第 4 章介绍）会确保所有估计概率都介于 0 和 1 之间，并且总和为 1，因为这些类别是互斥的。正如第 3 章所见，这称为多分类。

至于损失函数，由于我们预测的是概率分布，因此交叉熵损失（简称 x-entropy 或 log loss，见第 4 章）通常是很好的选择。

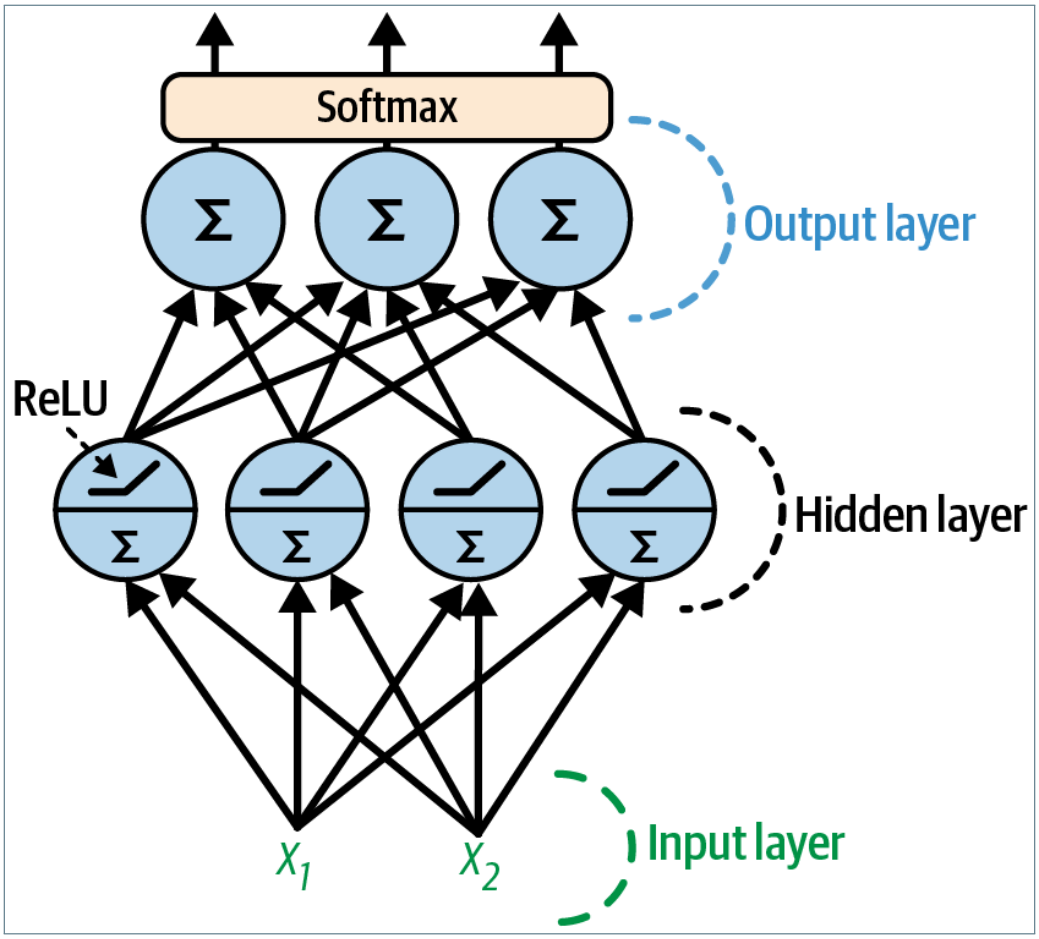

图 9-10: 用于分类的现代 MLP（包括 ReLU 和 Softmax）（A modern MLP, including ReLU and Softmax, for classification）

表 9-2 总结了分类 MLP 的典型架构。

正如你所料，Scikit-Learn 在 `sklearn.neural_network` 软件包中提供了 MLPClassifier 类，可以用它进行二元分类或多分类。它几乎与 MLPRegressor 类完全相同，

表 9-2: 分类 MLP 的典型架构

超参数	二元分类	多标签二元分类	多分类
隐藏层数量	通常为 1 至 5 层，取决于任务		
输出神经元数量	1	每个二元标签对应 1 个	每个类别对应 1 个
输出层激活函数	Sigmoid	Sigmoid	Softmax
损失函数	交叉熵	交叉熵	交叉熵

只是它的输出层使用 Softmax 激活函数，并且最小化的是交叉熵损失而不是 MSE。此外，`score()` 方法返回模型的准确率，而不是 R2 分数。下面来试试它。

我们本可以处理 iris 数据集，但那个任务对于神经网络来说过于简单：线性模型也能做到同样好，而且不会有过拟合风险。因此，我们来处理一个更复杂的任务：Fashion MNIST。它是 MNIST 的直接替代数据集（第 3 章介绍过）。Fashion MNIST 与 MNIST 的格式完全相同（70,000 张 28 × 28 像素的灰度图像，共 10 个类别），但图像表示的是服装而不是手写数字，因此每个类别的变化更加多样，问题也明显比 MNIST 更具挑战性。例如，简单线性模型在 MNIST 上的准确率约为 92%，但在 Fashion MNIST 上只有约 83%。下面看看 MLP 能否做得更好。

首先使用 `fetch_openml()` 函数加载数据集，方式与第 3 章加载 MNIST 时非常相似。注意，目标值以字符串`'0'`、`'1'`、……、`'9'` 表示，因此需要把它们转换为整数：

```
from sklearn.datasets import fetch_openml
fashion_mnist = fetch_openml(name="Fashion-MNIST", as_frame=False)
targets = fashion_mnist.target.astype(int)
```

数据已经打乱，因此只需取前 60,000 张图像进行训练，最后 10,000 张图像进行测试：

```
X_train, y_train = fashion_mnist.data[:60_000], targets[:60_000]
X_test, y_test = fashion_mnist.data[60_000:], targets[60_000:]
```

每张图像表示为一个一维整数数组，其中包含 784 个像素强度值，范围为 0 到 255。可以使用 `plt.imshow()` 函数绘制图像，但首先需要把它重塑为 `[28, 28]`：

```
import matplotlib.pyplot as plt
X_sample = X_train[0].reshape(28, 28)  # first image in the training set
plt.imshow(X_sample, cmap="binary")
plt.show()
```

运行这段代码后，应该会看到图 9-11 右上角的短靴图像。

图 9-11: Fashion MNIST 每个类别的前四个样本（First four samples from each class in Fashion MNIST）

在 MNIST 中，标签等于 5，表示图像是手写数字 5，很容易理解。但对于 Fashion MNIST，需要类别名称列表才能知道每个数字对应什么。Scikit-Learn 不提供这个列表，因此创建一个：

```
class_names = ["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
               "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"]
```

现在可以确认训练集中的第一张图像表示一只短靴：

```
>>> class_names[y_train[0]]
'Ankle boot'
```

准备好构建分类 MLP 了：

```
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import MinMaxScaler
mlp_clf = MLPClassifier(hidden_layer_sizes=[300, 100], verbose=True,
                        early_stopping=True, random_state=42)
pipeline = make_pipeline(MinMaxScaler(), mlp_clf)
pipeline.fit(X_train, y_train)
accuracy = pipeline.score(X_test, y_test)
```

这段代码与前面的回归代码非常相似，但有几个差异：

- 当然，这是一个分类任务，因此使用 MLPClassifier 而不是 MLPRegressor。
- 这里只使用两个隐藏层，分别包含 300 个和 100 个神经元。如果愿意，可以尝试不同数量的隐藏层，也可以改变神经元数量。
- 使用 MinMaxScaler 而不是 StandardScaler。需要把像素强度从 0-255 缩小到 0-1 范围，因为在 MLPClassifier 的默认超参数（例如默认学习率和权重初始化尺度）下，特征位于这个范围通常效果更好。你可能会问，为什么不使用 StandardScaler？这是因为有些像素在不同图像之间变化很小，例如边缘周围的像素几乎总是白色。如果使用 StandardScaler，这些像素会被缩放到与其他像素具有相同的方差；结果是，我们会赋予这些像素可能不值得拥有的更大重要性。对于图像，MinMaxScaler 通常比 StandardScaler 效果更好（但具体效果仍取决于实际情况）。
- 最后，`score()` 函数返回模型的准确率。

训练过程中，模型在验证集上的准确率约为 89.7%（确切值由 `mlp_clf.best_validation_score_` 给出），但训练接近结束时开始出现一些过拟合，因此最终准确率只有 89.2%。在测试集上评估模型时，准确率为 87.1%，对于这个任务来说还不错，但使用其他神经网络架构（例如卷积神经网络，第 12 章）可以得到更好的结果。

你可能已经注意到，训练速度相当慢。这是因为隐藏层包含许多参数，每次迭代都要执行大量计算。例如，第一隐藏层有 784 × 300 个连接权重，外加 300 个偏置项，总共包含 235,500 个参数！这些参数让模型具备很强的灵活性，可以拟合训练数据，但也意味着过拟合风险很高，尤其是在训练数据不多时。在这种情况下，可以使用提前停止和 ℓ_2 正则化等正则化技术。

模型训练完成后，就可以用它对新图像进行分类：

```
>>> X_new = X_test[:15]  # let's pretend these are 15 new images
>>> mlp_clf.predict(X_new)
array([9, 2, 1, 1, 6, 1, 4, 6, 5, 7, 4, 5, 8, 3, 4])
```

除了索引为 12 的预测之外，其他预测都是正确的：索引为 12 的图像实际应该是 7（运动鞋），而不是 8（包）。你可能想知道模型对这些预测有多大把握，尤其是对错误预测的把握。为此，可以像第 3 章那样，使用 `model.predict_proba()` 而不是 `model.predict()`：

```
>>> y_proba = mlp_clf.predict_proba(X_new)
>>> y_proba[12]
array([0., 0., 0., 0., 0., 0., 0., 1., 0.])
```

这可不太妙：模型以 100% 的置信度断定图像是包（索引 8）。也就是说，模型不仅错了，而且还以 100% 的置信度认为自己是对的。事实上，在测试集的全部 10,000 张图像中，模型置信度低于 99.9% 的只有 16 张，尽管其准确率约为 90%。因此，对估计概率始终要持保留态度：神经网络很容易过度自信，尤其是在训练时间稍长时。

> 提示：分类任务的目标值可以是类别索引（例如 3），也可以是类别概率，通常表示为 one-hot 向量（例如 `[0, 0, 0, 1, 0, 0, 0, 0, 0, 0]`）。但如果模型容易过度自信，可以尝试标签平滑技术[a]：略微降低目标类别的概率（例如从 1 降至 0.9），并将剩余概率均匀分配给其他类别（例如 `[0.1/9, 0.1/9, 0.1/9, 0.9, 0.1/9, 0.1/9, 0.1/9, 0.1/9, 0.1/9, 0.1/9]`）。
>
> [a] C. Szegedy 等人，“Rethinking the Inception Architecture for Computer Vision”，CVPR 2016：2818-2826。

在 Fashion MNIST 上获得 90% 的准确率已经相当不错。通过微调超参数，例如使用第 2 章介绍过的 `RandomizedSearchCV`，还可以获得更好的性能。不过，搜索空间很大，因此最好先大致了解应该在哪里寻找。

9.3　超参数调整指南（Hyperparameter Tuning Guidelines）

神经网络的灵活性也是它们的主要缺点之一：有很多超参数需要调整。不仅可以使用任何想得到的网络架构，即使是基本的 MLP，也可以改变层数、每层神经元数量、每层使用的激活函数类型、权重初始化逻辑、优化器类型、学习率、批量大小等。哪些超参数取值比较好？

9.3.1　隐藏层数量（Number of Hidden Layers）

对于许多问题，可以从一个隐藏层开始，并获得合理的结果。只要神经元足够多，仅有一个隐藏层的 MLP 在理论上也能建模最复杂的函数。但与浅层网络相比，深度网络具有高得多的参数效率：它们只需浅层网络指数级更少的神经元，就能建模复杂函数，因此在相同训练数据量下可以取得更好的性能。这是因为层次化结构让网络能够跨多个层级复用并组合特征：例如，人脸分类器的第一层可能学会识别点、弧

线或直线等低级特征；第二层可能学会把这些低级特征组合成正方形或圆形等更高级特征；第三层可能进一步把这些高级特征组合成嘴巴、眼睛或鼻子；顶层则可以利用这些高级特征对人脸进行分类。

这种层次化架构不仅能帮助 DNN 更快地收敛到良好解，还能改善其在新数据集上的泛化能力。例如，如果已经训练了一个识别照片中人脸的模型，现在想训练一个识别发型的新神经网络，就可以复用第一个网络的低层。与其随机初始化新网络前几层的权重和偏置，不如把它们初始化为第一个网络低层的权重和偏置。这样，新网络不必从头学习大多数照片中都会出现的低级结构，只需学习更高级的结构（例如发型）。这称为迁移学习。

总之，对于许多问题，可以从一到两个隐藏层开始，神经网络就会表现得相当好。例如，只需一个包含几百个神经元的隐藏层，就能在 MNIST 数据集上轻松达到 97% 以上的准确率；使用两个隐藏层、总神经元数量相同的网络，则可以达到 98% 以上的准确率，而且训练时间大致相同。对于更复杂的问题，可以逐步增加隐藏层数量，直到网络开始在训练集上过拟合。非常复杂的任务，例如大规模图像分类或语音识别，通常需要包含几十层甚至上百层的网络（不过不一定是全连接网络，第 12 章会介绍），并且需要大量训练数据。你很少需要从头训练这样的网络：更常见的做法是复用一个执行相似任务的预训练先进网络的部分结构。这样训练速度会快很多，而且所需数据也少得多。

9.3.2　每个隐藏层的神经元数量（Number of Neurons per Hidden Layer）

输入层和输出层的神经元数量由任务所需的输入和输出决定。例如，MNIST 任务需要 28 × 28 = 784 个输入和 10 个输出神经元。

至于隐藏层，以前常见的做法是让网络形成金字塔结构：每层的神经元越来越少。理由是许多低级特征可以汇聚成数量少得多的高级特征。一个典型的 MNIST 神经网络可能有 3 个隐藏层，分别包含 300、200 和 100 个神经元。然而，这种做法如今基本被放弃，因为在大多数情况下，所有隐藏层使用相同数量的神经元，效果同样好，甚至更好；而且这样只需调整一个超参数，而不是每层一个。不过，根据数据集的不同，有时让第一隐藏层稍大一些仍然有帮助。

和层数一样，可以逐渐增加神经元数量，直到网络开始过拟合。另一种做法是构建一个比实际需要稍大、包含更多层和神经元的模型，然后使用提前停止和其他正则化技术，防止它过度拟合。Waymo 研究员、前 Google 员工 Vincent Vanhoucke 将这种方法称为“弹力裤（stretch pants）”方法：与其浪费时间寻找完全合身的裤子，不如穿一条宽大的弹力裤，让它自行收缩到合适尺寸。使用这种方法可以避免可能

毁掉模型的瓶颈层。事实上，如果某层神经元太少，它就缺乏建模复杂关系的计算能力，甚至没有足够的表示能力保留输入中的所有有用信息。例如，对 Fashion MNIST 训练集应用 PCA（第 7 章介绍）会发现，需要 187 个维度才能保留 95% 的方差。因此，如果把第一隐藏层的神经元数量设为更大的数，比如 200，就可以确信这一层不会成为瓶颈。但也不能添加太多神经元，否则模型需要优化的参数过多，训练将需要更多时间和数据。

> 提示：一般来说，增加层数比增加每层的神经元数量更划算。

不过，瓶颈层并不总是坏事。例如，限制前几个隐藏层的维度，会迫使神经网络只保留最重要的维度，从而消除数据中的一部分噪声（但不要做得太过分！）。此外，在靠近输出层的位置设置瓶颈层，可以迫使神经网络学习前面各层中数据的良好表示，即在更小的空间中压缩更多有用信息。这有助于神经网络泛化，也可以直接用于表示学习。第 18 章还会回到这个话题。

9.3.3　学习率（Learning Rate）

学习率是极其重要的超参数。一般来说，最优学习率约为最大学习率的一半，也就是训练算法开始发散时所对应的学习率（第 4 章介绍过）。寻找合适学习率的一种方法是训练几百次迭代：从非常低的学习率（例如 10^-5）开始，逐渐增加到一个非常大的值（例如 10）。具体做法是在每次迭代时将学习率乘以一个常数因子，例如使用 $(10/10^{-5})^{1/500}$，在 500 次迭代中从 10^-5 增加到 10。如果绘制损失关于学习率的曲线（学习率使用对数刻度），应该会看到损失一开始下降；但过了一段时间，学习率会太大，损失又会迅速上升。最优学习率通常略低于损失开始上升的转折点（通常约为转折点的 1/10）。然后重新初始化模型，并使用这个合适的学习率正常训练。

> 提示：使用 Scikit-Learn 改变训练过程中的学习率时，必须把 MLP 的 **warm_start** 超参数设置为 **True**，并使用 **partial_fit()** 一次训练一个批次，方法与第 4 章中的 SGDRegressor 类似。每次迭代时简单地更新学习率即可。

9.3.4　批量大小（Batch Size）

批量大小会显著影响模型性能和训练时间。使用大批量的主要好处是 GPU 等硬件加速器可以高效处理大批量数据（第 10 章会介绍），因此训练算法每秒可以处理更多实例。于是，许多研究人员和实践者建议使用能够装入 VRAM（视频 RAM，即 GPU 内存）的最大批量。不过，这样做有一个问题：大批量有时会导致训练不稳定，

尤其是在较小模型和训练初期；而且最终模型的泛化能力可能不如使用小批量训练的模型。Yann LeCun 曾在社交媒体上写道："朋友不会让朋友使用大于 32 的小批量。"他引用了 Dominic Masters 和 Carlo Luschi 在 2018 年发表的一篇论文，该论文得出结论：使用 2 到 32 的小批量更好，因为小批量能以更少训练时间得到更好的模型[14]。

然而，也有其他研究得出了相反结论。例如，Elad Hoffer 等人[15]和 Priya Goyal 等人[16]在 2017 年发表的论文表明，配合学习率预热等技术（即以较小学习率开始训练，再逐渐提高学习率），可以使用非常大的批量（最多 8,192），在没有泛化差距的情况下获得很短的训练时间。

因此，一种策略是使用大批量，并适当配合学习率预热；如果训练不稳定或最终性能令人失望，再尝试使用较小批量。

9.3.5　其他超参数（Other Hyperparameters）

如果计算预算和时间允许，还可以调整下面两个超参数：

优化器　选择比普通小批量梯度下降更好的优化器，并调整它的超参数，有助于加快训练速度，有时也能获得更好的性能。

激活函数　前面已经讨论过如何选择激活函数：一般来说，ReLU 激活函数是所有隐藏层的良好默认选择。在某些情况下，用另一种函数替换 ReLU 会有所帮助。

> 提示：最优学习率取决于其他超参数，尤其是批量大小。因此，如果修改了任何超参数，记得重新调整学习率。

如果想了解更多神经网络超参数调整的最佳实践，可以阅读 Leslie Smith 的优秀论文[17]。《深度学习调参手册》（The Deep Learning Tuning Playbook，Google 研究人员编写）也非常值得阅读。Andrew Ng 的免费电子书《机器学习的渴望》（Machine Learning Yearning）同样包含大量实用建议。

[14] Dominic Masters 和 Carlo Luschi，"Revisiting Small Batch Training for Deep Neural Networks"，arXiv 预印本 arXiv:1804.07612（2018）。

[15] Elad Hoffer 等人，"Train Longer, Generalize Better: Closing the Generalization Gap in Large Batch Training of Neural Networks"，第 31 届神经信息处理系统国际会议论文集（2017）：1729-1739。

[16] Priya Goyal 等人，"Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour"，arXiv 预印本 arXiv:1706.02677（2017）。

[17] Leslie N. Smith，"A Disciplined Approach to Neural Network Hyper-Parameters: Part 1–Learning Rate, Batch Size, Momentum, and Weight Decay"，arXiv 预印本 arXiv:1803.09820（2018）。

最后，强烈建议你完成本章末的练习 1。你将使用一个友好的网页界面尝试不同的神经网络架构并可视化其输出。它可以帮助你更好地理解 MLP，并逐步建立对每个超参数（层数和神经元数量、激活函数等）影响的直觉。

至此，我们完成了对人工神经网络及其 Scikit-Learn 实现的介绍。下一章将切换到 PyTorch，这是目前领先的神经网络开源库；我们会利用 GPU 的力量，更快地训练和运行 MLP，也会开始构建具有多个输入和输出的更复杂模型。

9.4　练习（Exercises）

1. 神经网络游乐场（Neural Network Playground）是一个无需编写代码、非常适合建立直觉的工具（它由 TensorFlow 团队构建，但与 TensorFlow 没有特定关系；事实上，它甚至不使用 TensorFlow）。在本练习中，只需点击几下，就可以训练多个二元分类器，并调整模型架构和超参数，直观了解神经网络的工作方式以及超参数的作用。花些时间探索以下内容：

   a. 神经网络学到的模式。点击左上角的 Run 按钮，尝试训练默认神经网络。注意，它能很快为分类任务找到良好解。第一隐藏层中的神经元学会了简单模式，第二隐藏层中的神经元则学会把第一隐藏层的简单模式组合成更复杂的模式。一般来说，层数越多，模式就可以越复杂。

   b. 激活函数。把 tanh 激活函数替换为 ReLU 激活函数，然后重新训练网络。注意，这次它找到解的速度更快，但边界是线性的。这是由 ReLU 函数的形状造成的。

   c. 局部最小值的风险。把网络架构改为只有一个包含三个神经元的隐藏层。多次训练网络（要重置网络权重，请点击 Play 按钮旁边的 Reset 按钮）。注意，训练时间变化很大，有时甚至会陷入局部最小值。

   d. 神经网络太小时会发生什么。移除一个神经元，只保留两个。注意，即使多次尝试，神经网络现在也无法找到良好解。模型参数太少，系统性地欠拟合训练集。

   e. 神经网络足够大时会发生什么。把神经元数量设为 8，并多次训练网络。注意，现在训练速度始终很快，也不会陷入停滞。这体现了神经网络理论中的一个重要发现：大型神经网络很少陷入局部最小值，即使陷入，局部最优通常也几乎和全局最优一样好。不过，它们仍可能在漫长的平台期停留很长时间。

   f. 深层网络中的梯度消失风险。选择螺旋数据集（“DATA”下方右下角的数据集），把网络架构改为 4 个隐藏层，每层 8 个神经元。注意，训练时间会长得多，而且经常会在平台期停留很长时间。还要注意，最高层（右侧）的神经元往往比低层（左侧）

的神经元变化更快。这个问题称为梯度消失问题，可以通过更好的权重初始化和其他技术、更好的优化器（例如 AdaGrad 或 Adam），或批归一化（第 11 章讨论）来缓解。

g. 继续探索。花大约一个小时尝试其他参数，感受它们的作用，从而建立对神经网络的直觉理解。

2. 使用原始人工神经元（类似图 9-3 中的神经元）画出一个计算 $A \oplus B$ 的 ANN，其中 $\oplus$ 表示 XOR 运算。提示：$A \oplus B = (A \wedge \neg B) \vee (\neg A \wedge B)$。

3. 为什么通常更适合使用逻辑回归分类器，而不是经典感知器（即使用感知器训练算法训练的单层阈值逻辑单元）？如何调整感知器，使其等价于逻辑回归分类器？

4. 为什么 sigmoid 激活函数是训练最初几代 MLP 的关键组成部分？

5. 列出三种常见的激活函数。你能画出它们吗？

6. 假设有一个 MLP，它由包含 10 个直通神经元的输入层、包含 50 个人工神经元的隐藏层，以及包含 3 个人工神经元的输出层组成。所有人工神经元都使用 ReLU 激活函数。

   a. 输入矩阵 X 的形状是什么？

   b. 隐藏层权重矩阵 W_h 和偏置向量 b_h 的形状是什么？

   c. 输出层权重矩阵 W_o 和偏置向量 b_o 的形状是什么？

   d. 网络输出矩阵 Y 的形状是什么？

   e. 写出一个关于 X、W_h、b_h、W_o 和 b_o 的方程，用来计算网络输出矩阵 Y。

7. 如果想把电子邮件分类为垃圾邮件或正常邮件，输出层需要多少个神经元？应该使用什么激活函数？如果想处理 MNIST，输出层需要多少个神经元，应该使用什么激活函数？如果想让网络像第 2 章那样预测房价，又该如何设置？

8. 什么是反向传播？它如何工作？反向传播和反向模式自动微分有什么区别？

9. 基本 MLP 中可以调整的所有超参数有哪些？如果 MLP 过拟合训练数据，应该如何调整这些超参数来尝试解决问题？

10. 在 Cover Type 数据集上训练一个深度 MLP。可以使用 `sklearn.datasets.fetch_covtype()` 加载数据。尝试手动微调超参数和/或使用 `RandomizedSearchCV`，看看能否在测试集上达到超过 93% 的准确率。

    这些练习的解答位于本章 notebook 的末尾：https://homl.info/colab-p。

第 10 章　PyTorch 神经网络构建

Building Neural Networks with PyTorch

PyTorch 是一个功能强大的开源深度学习库，由 Facebook 的人工智能研究院（FAIR，如今称为 Meta AI）开发。它是 Torch 库的 Python 后继者；Torch 最初使用 Lua 编程语言编写。借助 PyTorch，你可以构建各种神经网络模型，并使用 GPU（或其他硬件加速器，稍后会看到）大规模训练它们。从很多方面看，PyTorch 类似于 NumPy，只是它还支持硬件加速和自动微分（见第 9 章），并且包含优化器和可直接使用的神经网络组件。

PyTorch 于 2016 年发布时，Google 的 TensorFlow 库遥遥领先，是最流行的深度学习库：它速度快、扩展性好，并且可以部署到各种平台。但它的编程模型复杂且静态，使用和调试都比较困难。相比之下，PyTorch 从一开始就致力于提供更加灵活、更符合 Python 习惯的神经网络构建方式。正如你将看到的，它使用动态图（也称为 define-by-run，即运行时定义），因此直观且易于调试。PyTorch 的代码和文档也非常优秀，专注于核心任务：让高性能神经网络的构建和训练变得简单。最后，它大力拥抱开源文化，受益于热情且专注的社区和丰富的生态系统。2022 年 9 月，PyTorch 的治理甚至移交给了 Linux 基金会旗下的 PyTorch 基金会。所有这些特质都很好地契合了研究人员的需求：PyTorch 很快成为学术界使用最多的框架；当大多数深度学习论文都基于 PyTorch 后，业界也逐渐转向了 PyTorch[1]。

本章将学习如何使用 PyTorch 训练、评估、微调、优化和保存神经网络。我们首先熟悉 PyTorch 的核心构件，即张量和 autograd；接着通过构建和训练一个简单的线性回归模型来试试水；然后把这个模型升级为多层神经网络，先用于回归，再用于分类。在这个过程中，我们还会看到如何构建具有多个输入或输出的自定义神经网络。最后，我们将讨论如何使用 Optuna 库自动微调超参数，以及如何优化和导出模型。准备好了吗？让我们一起深入深度学习！

[1]公平地说，TensorFlow 的易用性问题在第 2 版中大多得到了解决，Google 还推出了设计良好且速度极快的 JAX，因此 PyTorch 仍然面临着有力的竞争。好消息是，这些库的 API 已经相当趋同，从一个库切换到另一个库比过去容易得多。

> 注意：Colab 运行时预装了较新版本的 PyTorch。不过，如果你更愿意在自己的机器上安装 PyTorch，请参阅安装说明：https://homl.info/install-p；这需要安装 Python、许多库以及 GPU 驱动程序（如果你有 GPU）。

10.1　PyTorch 基础（PyTorch Fundamentals）

PyTorch 的核心数据结构是张量（tensor）[2]。张量是带有形状和数据类型、用于数值计算的多维数组。它不就和 NumPy 数组一样吗？没错！但张量还拥有两个额外特性：它可以存放在 GPU（或其他硬件加速器，稍后会看到）上，并且支持自动微分。今后构建的每个神经网络都会输入和输出张量（这很像 Scikit-Learn 模型输入和输出 NumPy 数组）。因此，先来看看如何创建和操作张量。

10.1.1　PyTorch 张量（PyTorch Tensors）

首先导入 PyTorch 库：

```
>>> import torch
```

接下来可以像创建 NumPy 数组一样创建 PyTorch 张量。例如，创建一个 2 × 3 数组：

```
>>> X = torch.tensor([[1.0, 4.0, 7.0], [2.0, 3.0, 6.0]])
>>> X
tensor([[1., 4., 7.],
        [2., 3., 6.]])
```

和 NumPy 数组一样，一个张量可以包含浮点数、整数、布尔值或复数，但每个张量只能使用一种数据类型。如果使用不同类型的值初始化张量，PyTorch 会选择最通用的类型（即 complex > float > integer > bool）。创建张量时也可以显式选择数据类型，例如使用 **`dtype=torch.float16`** 创建 16 位浮点数。注意，PyTorch 不支持字符串或对象张量。

可以这样获取张量的形状和数据类型：

[2]数学和物理学中也有称为张量的对象，但机器学习张量更简单：它们实际上只是用于数值计算的多维数组，再加上几个额外特性。

```
>>> X.shape
torch.Size([2, 3])
>>> X.dtype
torch.float32
```

索引方式与 NumPy 数组完全相同：

```
>>> X[0, 1]
tensor(4.)
>>> X[:, 1]
tensor([4., 3.])
```

你还可以对张量执行各种计算，而且 API 也很像 NumPy。例如，有 **torch.abs()**、**torch.cos()**、**torch.exp()**、**torch.max()**、**torch.mean()**、**torch.sqrt()** 等函数。PyTorch 张量也为大多数运算提供了方法，因此可以写成 **X.exp()**，而不是 **torch.exp(X)**。试试下面几种运算：

```
>>> 10 * (X + 1.0)  # itemwise addition and multiplication
tensor([[20., 50., 80.],
        [30., 40., 70.]])
>>> X.exp()  # itemwise exponential
tensor([[   2.7183,   54.5981, 1096.6332],
        [   7.3891,   20.0855,  403.4288]])
>>> X.mean()
tensor(3.8333)
>>> X.max(dim=0)  # max values along dimension 0 (i.e., max value per column)
torch.return_types.max(values=tensor([2., 4., 7.]), indices=tensor([1, 0, 0]))
>>> X @ X.T  # matrix transpose and matrix multiplication
tensor([[66., 56.],
        [56., 49.]])
```

> 注意：PyTorch 在 **max()** 等运算中更偏好使用参数名 **dim**，但也支持 NumPy 或 Pandas 中使用的 **axis**。

还可以使用 **numpy()** 方法将张量转换为 NumPy 数组，也可以从 NumPy 数组创建张量：

```
>>> import numpy as np
>>> X.numpy()
array([[1., 4., 7.],
       [2., 3., 6.]], dtype=float32)
>>> torch.tensor(np.array([[1., 4., 7.], [2., 3., 6.]]))
tensor([[1., 4., 7.],
        [2., 3., 6.]], dtype=torch.float64)
```

注意，PyTorch 中浮点数的默认精度是 32 位，而 NumPy 中是 64 位。在深度学习中，通常最好使用 32 位，因为这样只需一半 RAM，计算速度也更快；神经网络实际上并不需要 64 位浮点数提供的额外精度。因此，调用 `torch.tensor()` 将 NumPy 数组转换为张量时，最好指定 `dtype=torch.float32`。或者，也可以使用 `torch.FloatTensor()`，它会自动把数组转换为 32 位：

```
>>> torch.FloatTensor(np.array([[1., 4., 7.], [2., 3., 6]]))
tensor([[1., 4., 7.],
        [2., 3., 6.]])
```

> 提示：`torch.tensor()` 和 `torch.FloatTensor()` 都会复制给定的 NumPy 数组。如果不希望复制，可以使用 `torch.from_numpy()`；它会在 CPU 上创建一个直接使用 NumPy 数组数据的张量。但要小心：修改 NumPy 数组也会修改张量，反过来也一样。

还可以像操作 NumPy 数组一样，通过索引和切片原地修改张量：

```
>>> X[:, 1] = -99
>>> X
tensor([[  1., -99.,   7.],
        [  2.,   3.,   6.]])
```

PyTorch 的 API 提供了许多原地运算，例如 `abs_()`、`sqrt_()` 和 `zero_()`，它们会直接修改输入张量，有时可以节省内存并提高模型速度。例如，`relu_()` 方法会原地应用 ReLU 激活函数，把所有负值替换为 0：

```
>>> X.relu_()
>>> X
tensor([[1., 0., 7.],
        [2., 0., 6.]])
```

> 提示：PyTorch 的原地运算很容易一眼识别，因为它们的名称总是以下划线结尾。极少数例外（例如 `zero_()`）除外，去掉下划线就得到普通运算（例如 `abs_()` 是原地运算，而 `abs()` 不是）。

我们会在后面介绍更多运算，但现在先来看看如何使用硬件加速大幅加快计算。

10.1.2　硬件加速（Hardware Acceleration）

如果机器拥有兼容的 GPU 并且安装了所需库，就可以很容易地把 PyTorch 张量复制到 GPU。在 Colab 中，只需确保使用 GPU 运行时：打开“运行时”菜单，选择“更改运行时类型”，然后确认选择了 GPU（例如 Nvidia T4 GPU）。GPU 运行时会自动安装适合的 PyTorch 库（编译时启用了 GPU 支持），以及适合的 GPU 驱动程序和相关库（例如 Nvidia 的 CUDA 和 cuDNN）[3]。如果希望在自己的机器上运行代码，则需要确保安装了所有需要的驱动程序和库。请按照 https://homl.info/install-p 中的说明操作。

PyTorch 对 Nvidia GPU 提供了出色支持，同时也支持其他几种硬件加速器：

- Apple Metal Performance Shaders（MPS），用于加速 Apple Silicon（例如 M1、M2 及后续芯片），也支持部分配备兼容 GPU 的 Intel Mac。
- 通过 ROCm 软件栈，或在 Windows 上通过 DirectML，支持 AMD Instinct 加速器和 AMD Radeon GPU。
- 通过 Intel oneAPI，在 Linux 和 Windows 上支持 Intel GPU 与 CPU。
- 通过 `torch_xla` 库支持 Google TPU。

下面检查 PyTorch 是否可以访问 Nvidia GPU 或 Apple MPS；如果都不行，就回退到 CPU：

```
if torch.cuda.is_available():
    device = "cuda"
elif torch.backends.mps.is_available():
    device = "mps"
else:
    device = "cpu"
```

[3] CUDA 是 Nvidia 用于在兼容 CUDA 的 GPU 上运行代码的专有平台；cuDNN 是构建在 CUDA 之上的库，用于加速各种深度神经网络架构。

> 警告：深度学习通常需要很强的计算能力，尤其是在后续章节开始深入计算机视觉和自然语言处理之后。你需要一台性能还不错的机器，但更重要的是需要硬件加速器（或者多个加速器）。如果没有，可以尝试使用 Colab 或 Kaggle，它们提供免费的 GPU 运行时；也可以考虑其他云服务。否则，就准备好非常、非常有耐心吧。

在 Colab GPU 运行时中，**device** 会等于**"cuda"**。现在在 GPU 上创建一个张量。一种方法是在 CPU 上创建张量，然后使用 **to()** 方法把它复制到 GPU：

```
>>> M = torch.tensor([[1., 2., 3.], [4., 5., 6.]])
>>> M = M.to(device)
```

> 提示：**cpu()** 和 **cuda()** 方法分别是 **to("cpu")** 和 **to("cuda")** 的简写。

查看张量的 **device** 属性，就可以知道它位于哪个设备上：

```
>>> M.device
device(type='cuda', index=0)
```

另一种方法是直接在 GPU 上创建张量，也就是使用 **device** 参数：

```
>>> M = torch.tensor([[1., 2., 3.], [4., 5., 6.]], device=device)
```

> 提示：如果有多个 Nvidia GPU，可以在 GPU 索引后面添加编号来指定目标 GPU：GPU #0 使用**"cuda:0"**（或者直接使用**"cuda"**），GPU #1 使用**"cuda:1"**，依此类推。

张量位于 GPU 后，可以照常运行运算，而且运算都会在 GPU 上执行：

```
>>> R = M @ M.T  # run some operations on the GPU
>>> R
tensor([[14., 32.],
        [32., 77.]], device='cuda:0')
```

注意，结果 R 也位于 GPU 上。这意味着可以在 GPU 上连续执行多个运算，而不必在 CPU 和 GPU 之间来回传输数据。在深度学习中，这一点至关重要，因为设备之间的数据传输经常会成为性能瓶颈。

GPU 能把计算加速多少？当然取决于 GPU：更昂贵的 GPU 可能比便宜的 GPU 快几十倍。但速度并不是唯一重要的因素，正如刚才看到的，数据吞吐量同样关键。如果模型的计算量很大（例如非常深的神经网络），GPU 的速度和 RAM 容量通常最重要；但如果模型较浅，把训练数据送入 GPU 反而可能成为瓶颈。下面做一个小测试，比较在 CPU 和 GPU 上运行矩阵乘法的速度[4]：

```
>>> M = torch.rand((1000, 1000))  # on the CPU
>>> %timeit M @ M.T
16.1 ms ± 2.17 ms per loop (mean ± std. dev. of 7 runs, 100 loops each)
>>> M = torch.rand((1000, 1000), device="cuda")  # on the GPU
>>> %timeit M @ M.T
549 µs ± 3.99 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
```

哇！GPU 带来了 29 倍的速度提升！而且这只是在 Colab 中使用免费的 Nvidia T4 GPU；如果使用更强大的 GPU，想象一下还能获得多大的加速。现在试着改变矩阵大小：你会注意到，在较小矩阵上，加速效果远没有那么惊人（例如对于 100 × 100 的矩阵，只有约 2 倍）。这是因为 GPU 会把大型运算拆成较小的运算，并让数千个核心并行执行。如果任务很小，就无法拆分成那么多部分，因此性能提升也会更小。事实上，执行许多微小任务时，有时直接在 CPU 上运行反而更快。

现在我们已经了解了张量，以及如何在 CPU 或 GPU 上使用张量，接下来看看 PyTorch 的自动微分功能。

10.1.3　Autograd

PyTorch 内置了高效的反向模式自动微分实现（第 9 章介绍，附录 A 详细说明），称为 autograd，即 automatic gradients（自动梯度）的缩写。它很容易使用。例如，

[4] **%timeit** 魔术命令只能在 Jupyter notebook、Colab 或 IPython shell 中使用；在普通 Python shell 或程序中，可以改用 **timeit.timeit()** 函数。

考虑一个简单函数 $f(x) = x^2$。微积分告诉我们，这个函数的导数是 $f'(x) = 2x$。如果计算 $f(5)$ 和 $f'(5)$，结果分别是 25 和 10。看看 PyTorch 是否同意：

```
>>> x = torch.tensor(5.0, requires_grad=True)
>>> f = x ** 2
>>> f
tensor(25., grad_fn=<PowBackward0>)
>>> f.backward()
>>> x.grad
tensor(10.)
```

太好了，得到了正确结果：`f` 是 25，`x.grad` 是 10！注意，`backward()` 函数自动计算了同一点 $x = 5.0$ 处的导数 $f'(x)$。下面逐行解释这段代码。

首先创建了一个等于 5.0 的张量 `x`，并通过指定 `requires_grad=True` 告诉 PyTorch 它是变量而不是常量。知道这一点后，PyTorch 会自动跟踪涉及 `x` 的所有运算：这是因为 PyTorch 必须捕获计算图，才能在计算图上运行反向传播，并得到 f 对 x 的导数。在这张计算图中，张量 `x` 是叶节点。

接下来计算 $f = x^2$。结果是一个等于 25.0 的张量，也就是 5.0 的平方。但还不止如此：`f` 还有一个 `grad_fn` 属性，它表示创建该张量的运算（`**` 即幂运算，因此名称为 `PowBackward0`），并告诉 PyTorch 如何通过这个具体运算反向传播梯度。`grad_fn` 属性就是 PyTorch 跟踪计算图的方式。

然后调用 `f.backward()`：这会从 `f` 开始沿计算图反向传播梯度，一直传播到叶节点（本例中只有 `x`）。

最后，读取张量 `x` 的 `grad` 属性即可。它是在反向传播过程中计算的，给出 f 对 x 的导数。大功告成！

PyTorch 会在每次前向传播过程中，随着运算执行即时创建新的计算图。这使 PyTorch 能够支持包含循环和条件分支的动态模型。

> 警告：PyTorch 在每个变量的 `grad` 属性中累积梯度的方式，初次接触时可能令人惊讶，特别是如果你之前使用过 TensorFlow 或 JAX。在这些框架中，计算 f 对 x 的梯度只会返回梯度，不会影响 `x`。而在 PyTorch 中，如果对张量调用 `backward()`，它会把梯度累积到用于计算该张量的每个变量中。因此，如果对使用同一个变量 `v` 计算得到的两个张量 `t1` 和 `t2` 调用 `backward()`，那么 `v.grad` 将是两者梯度之和。

计算梯度后，通常需要通过从模型变量中减去一小部分梯度来执行梯度下降步骤（至少训练神经网络时是这样）。在这个简单示例中，运行梯度下降会逐渐把 x 推向 0，因为 $f(x) = x^2$ 在 $x = 0$ 时最小。

执行梯度下降步骤时，必须暂时禁用梯度跟踪，因为不希望把梯度下降步骤本身记录到计算图中（事实上，如果尝试对被跟踪的变量执行原地运算，PyTorch 会抛出异常）。可以把梯度下降步骤放在 `torch.no_grad()` 上下文中：

```
learning_rate = 0.1
with torch.no_grad():
    x -= learning_rate * x.grad  # gradient descent step
```

变量 x 会减去 `0.1 * 10.0 = 1.0`，从 5.0 变为 4.0。

另一种避免梯度计算的方法是使用变量的 `detach()` 方法：它会创建一个脱离计算图的新张量，该张量的 `requires_grad=False`，但仍然指向内存中的同一份数据。然后可以更新这个脱离计算图的张量：

```
x_detached = x.detach()
x_detached -= learning_rate * x.grad
```

由于 `x_detached` 和 x 共享同一份内存，修改 `x_detached` 也会修改 x。

当需要在不影响梯度的情况下对张量执行某些计算时（例如评估或记录日志），`detach()` 方法很有用；如果需要精细控制哪些运算应当参与梯度计算，它也很方便。执行推理或梯度下降步骤时，通常更推荐使用 `no_grad()`，因为它提供了一个能够在整个上下文中禁用梯度跟踪的便捷方法。

最后，在重复整个过程（前向传播 + 反向传播 + 梯度下降步骤）之前，必须把每个模型参数的梯度清零（这里不需要 `no_grad()` 上下文，因为梯度张量的 `requires_grad=False`）：

```
x.grad.zero_()
```

> 警告：如果忘记在每次训练迭代时把梯度清零，`backward()` 方法只会继续累积梯度，导致梯度下降更新错误。由于不会出现明确的错误，只有性能下降（还可能出现无穷大或 NaN 值），这个问题可能很难调试。

把前面的内容组合起来，完整训练循环如下：

```
learning_rate = 0.1
x = torch.tensor(5.0, requires_grad=True)
for iteration in range(100):
    f = x ** 2  # forward pass
    f.backward()  # backward pass
    with torch.no_grad():
        x -= learning_rate * x.grad  # gradient descent step
    x.grad.zero_()  # reset the gradients
```

如果希望通过避免不必要的复制运算来节省内存、稍微加快模型速度，可以使用原地运算，但需要小心：原地运算并不总能与 autograd 很好地配合。

首先，如前所述，不能对叶节点（即 `requires_grad=True` 的张量）应用原地运算，因为 PyTorch 不知道应该把计算图存放在哪里。例如，`x.cos_()` 或 `x += 1` 会导致 `RuntimeError`。其次，考虑下面的代码：它在 $t = 2$ 时计算 $z(t) = \exp(t) + 1$，然后尝试计算梯度：

```
t = torch.tensor(2.0, requires_grad=True)
z = t.exp()  # this is an intermediate result
z += 1  # this is an in-place operation
z.backward()  # RuntimeError!
```

虽然 `z` 被正确计算出来，但最后一行会导致 `RuntimeError`，提示“梯度计算所需的某个变量已被原地运算修改”。确实，执行原地运算 `z += 1` 时，中间结果 `z = t.exp()` 丢失了；因此，当反向传播到达指数运算时，已经无法计算梯度。一个简单的修复方法是把 `z += 1` 换成 `z = z + 1`。两者看起来相似，但后者不再是原地运算：它创建一个新的张量并把它赋给同一个变量，而原始张量保持不变，并记录在最终张量的计算图中。

令人意外的是，如果把前面示例中的 `exp()` 换成 `cos()`，梯度就能正确计算，不会报错！为什么？结果取决于每种运算的实现方式。

有些运算（例如 `exp()`、`relu()`、`rsqrt()`、`sigmoid()`、`sqrt()`、`tan()` 和 `tanh()`）会在前向传播期间把输出保存到计算图中，然后在反向传播期间利用这些输出计算梯度[5]。这意味着不能原地修改这些运算的输出，否则在反向传播期间会报错。

[5]例如，`exp(x)` 的导数等于 `exp(x)`，因此在前向传播期间把该运算的输出保存到计算图中，并在反向传播时使用这个输出得到梯度，是非常合理的：不需要保存额外数据，也不需要重新计算 `exp(x)`。

另一些运算（例如 `abs()`、`cos()`、`log()`、`sin()`、`square()` 和 `var()`）会保存输入而不是输出[6]。这类运算不在乎输出是否被原地修改，但不能在反向传播之前原地修改它们的输入（例如，不改变输入就无法基于同一输入计算其他结果）。

还有一些运算（例如 `max()`、`min()`、`norm()`、`prod()`、`sgn()` 和 `std()`）会同时保存输入和输出，因此在反向传播前不能原地修改其中任何一个。

最后，少数运算（例如 `ceil()`、`floor()`、`mean()`、`round()` 和 `sum()`）既不保存输入也不保存输出[7]。可以安全地原地修改它们。

> 提示：先在完全不使用原地运算的情况下实现模型；如果需要节省内存或让模型稍微快一些，再尝试把代价最高的运算转换为对应的原地版本。务必确认给定输入时模型仍然输出相同结果，也不要原地修改反向传播需要使用的张量（否则会得到 `RuntimeError`）。

现在退一步回顾一下。我们已经讨论了 PyTorch 的基础知识：如何创建张量、使用张量执行各种运算、使用 GPU 加速计算，以及使用 autograd 为梯度下降计算梯度。很好！接下来应用已经学到的内容，使用 PyTorch 构建并训练一个简单的线性回归模型。

10.2　实现线性回归（Implementing Linear Regression）

我们先直接使用张量和 autograd 实现线性回归，然后使用 PyTorch 的高级 API 简化代码，并加入 GPU 支持。

10.2.1　使用张量和 Autograd 实现线性回归（Linear Regression Using Tensors and Autograd）

我们处理与第 9 章相同的加利福尼亚住房数据集。这里假设你已经使用 `sklearn.datasets.fetch_california_housing()` 下载了数据集，并使用 `sklearn.model_selection.train_test_split()` 将其拆分为训练集（`X_train` 和 `y_train`）、

[6]例如，当 `x < 0` 时，`abs(x)` 的导数为 -1；当 `x > 0` 时，导数为 +1。如果这个运算把输出保存到计算图中，反向传播就无法知道 `x` 是正数还是负数（因为 `abs(x)` 总是正数），也就无法计算梯度。因此，这类运算必须保存输入。

[7]例如，`floor(x)` 的导数总是 0（至少对于非整数输入是这样），因此 `floor()` 运算只需在前向传播时保存输入的形状，反向传播时产生一个形状相同但全为 0 的梯度。对于整数输入，autograd 也会返回 0 而不是 NaN。

验证集（`X_valid` 和 `y_valid`）和测试集（`X_test` 和 `y_test`）。接下来把数据转换为张量并进行标准化。可以像第 9 章那样使用 `StandardScaler`，但这里直接使用张量运算来练习：

```
X_train = torch.FloatTensor(X_train)
X_valid = torch.FloatTensor(X_valid)
X_test = torch.FloatTensor(X_test)
means = X_train.mean(dim=0, keepdims=True)
stds = X_train.std(dim=0, keepdims=True)
X_train = (X_train - means) / stds
X_valid = (X_valid - means) / stds
X_test = (X_test - means) / stds
```

还要把目标值转换为张量。由于预测值将是列向量（即只有一列的矩阵），必须确保目标值也是列向量[8]。遗憾的是，表示目标值的 NumPy 数组是一维的，因此需要通过增加一个大小为 1 的第二维，把张量重塑为列向量[9]：

```
y_train = torch.FloatTensor(y_train).reshape(-1, 1)
y_valid = torch.FloatTensor(y_valid).reshape(-1, 1)
y_test = torch.FloatTensor(y_test).reshape(-1, 1)
```

数据准备好后，创建线性回归模型的参数：

```
torch.manual_seed(42)
n_features = X_train.shape[1]  # there are 8 input features
w = torch.randn((n_features, 1), requires_grad=True)
b = torch.tensor(0., requires_grad=True)
```

现在有一个权重参数 `w`（列向量，每个输入维度对应一个权重，本例中有 8 个权重）和一个偏置参数 `b`（单个标量）。权重随机初始化，偏置初始化为 0。在这个例子中，权重也可以初始化为 0；但之后学习神经网络时，随机初始化权重对于打破神经元之间的对称性很重要（第 9 章已解释），因此现在就养成这个习惯比较好。

> 警告：调用 `torch.manual_seed()` 是为了确保结果可复现。不过，PyTorch 不保证不同版本、平台或设备之间完全复现结果。因此，如果不是在使用 Nvidia

[8] 在机器学习中，通常更偏好列向量（形状为 `[m, 1]`）和行向量（形状为 `[1, m]`），而不是一维向量（形状为 `[m]`），因为它们可以避免矩阵乘法或广播等运算中的歧义，并且无论只有一个特征还是多个特征，都能让代码保持一致。

[9] 和 NumPy 一样，`reshape()` 方法允许把某一维指定为 -1。该维度的大小会自动计算，从而确保新张量拥有与原张量相同数量的元素。

> T4 GPU 的 Colab 运行时中使用 PyTorch 2.8.0 运行本章代码，结果可能会有所不同。此外，由于 GPU 会把每个运算拆成多个块并行执行，这些块完成的顺序可能因运行而异，也可能因浮点精度误差而造成轻微差异。这些细小差异可能在训练过程中逐渐累积，最终导致模型差异很大。为避免这种情况，可以调用 `torch.use_deterministic_algorithms(True)`，并将 `torch.backends.cudnn.benchmark` 设置为 `False`，要求 PyTorch 只使用确定性算法。不过，确定性算法通常比随机算法慢，而且有些运算根本没有确定性版本；如果代码尝试使用这类运算，就会报错。

现在训练模型，过程与第 4 章非常相似，只是这里使用自动微分计算梯度，而不是使用闭式方程。目前使用批量梯度下降（BGD），即每个训练步骤都使用完整训练集：

```
learning_rate = 0.4
n_epochs = 20
for epoch in range(n_epochs):
    y_pred = X_train @ w + b
    loss = ((y_pred - y_train) ** 2).mean()
    loss.backward()
    with torch.no_grad():
        b -= learning_rate * b.grad
        w -= learning_rate * w.grad
        b.grad.zero_()
        w.grad.zero_()
    print(f"Epoch {epoch + 1}/{n_epochs}, Loss: {loss.item()}")
```

下面逐步解释这段代码：

首先定义 `learning_rate` 超参数。你可以尝试不同取值，找到既能快速收敛又能给出精确结果的值。

然后运行 20 个 epoch。现在也可以像第 4 章那样实现提前停止，以找到合适的停止时机并避免过拟合，但暂时保持简单。

接下来执行前向传播：计算预测值 `y_pred` 和均方误差损失。

然后运行 `loss.backward()`，计算损失相对于每个模型参数的梯度。这就是 autograd 的作用。

接着使用 `b.grad` 和 `w.grad` 执行梯度下降步骤。注意，这段代码位于前面介绍过的 `torch.no_grad()` 上下文中。

完成梯度下降步骤后，把梯度重置为 0（非常重要！）。

最后，在每个 epoch 打印 epoch 编号和当前损失。**item()** 方法提取标量张量的值。

就是这样！运行代码后，应该看到训练损失像下面这样逐渐下降：

```
Epoch 1/20, Loss: 16.158456802368164
Epoch 2/20, Loss: 4.8793745040893555
Epoch 3/20, Loss: 2.255225419998169
[...]
Epoch 20/20, Loss: 0.5684100389480591
```

恭喜你，已经使用 PyTorch 训练了第一个模型！现在可以用模型对新数据 **X_new**（必须表示为 PyTorch 张量）进行预测。例如，对测试集中的前 3 个实例进行预测：

```
>>> X_new = X_test[:3]  # pretend these are new instances
>>> with torch.no_grad():
...     y_pred = X_new @ w + b  # use the trained parameters to make predictions
...
>>> y_pred
tensor([[0.8916],
        [1.6480],
        [2.6577]])
```

> 提示：推理时最好使用 **with torch.no_grad()** 上下文：由于不必跟踪计算图，PyTorch 会消耗更少 RAM，运行速度也更快。

直接使用 PyTorch 的底层 API 实现线性回归并不算太难，但如果用这种方法处理更复杂的模型，代码会变得混乱且难以维护。因此，PyTorch 提供了高级 API 来简化这一过程。现在使用高级 API 重写模型。

10.2.2　使用 PyTorch 高级 API 实现线性回归（Linear Regression Using PyTorch's High-Level API）

PyTorch 在 **torch.nn.Linear** 类中提供了线性回归实现，因此使用它：

```
import torch.nn as nn  # by convention, this module is usually imported this way
torch.manual_seed(42)  # to get reproducible results
model = nn.Linear(in_features=n_features, out_features=1)
```

nn.Linear 类（即 **torch.nn.Linear** 的简写）是 PyTorch 提供的众多模块之一。每个模块都是 **nn.Module** 类的子类。要构建一个简单的线性回归模型，只需要一个

`nn.Linear` 模块。不过，大多数神经网络都需要组装许多模块，正如本章稍后将看到的那样，因此可以把模块想象成数学积木。许多模块都包含模型参数。例如，`nn.Linear` 模块包含一个偏置向量（每个神经元一个偏置项）和一个权重矩阵（每个神经元一行、每个输入维度一列，也就是前面以及公式 9-2 中使用的权重矩阵的转置）。由于本模型只有一个神经元（`out_features=1`），偏置向量只有一个偏置项，权重矩阵只有一行。这些参数可以直接作为 `nn.Linear` 模块的属性访问：

```
>>> model.bias
Parameter containing:
tensor([0.3117], requires_grad=True)
>>> model.weight
Parameter containing:
tensor([[ 0.2703,  0.2935, -0.0828,  0.3248, -0.0775,  0.0713, -0.1721,  0.2076]],
       requires_grad=True)
```

注意，两个参数都自动进行了随机初始化（这就是为什么使用 `manual_seed()` 来获得可复现结果）。这些参数是 `torch.nn.Parameter` 类的实例，而该类是 `tensor.Tensor` 类的子类；因此可以像使用普通张量一样使用它们。模块的 `parameters()` 方法会递归地返回该模块所有类型为 `Parameter` 的属性，以及所有子模块的参数（如果有）。它不会返回普通张量，即使这些张量的 `requires_grad=True`。这就是普通张量和 `Parameter` 之间的主要区别：

```
>>> for param in model.parameters():
...     [...]  # do something with each parameter
```

还有一个 `named_parameters()` 方法，会返回参数名称和值组成的迭代器。

模块可以像普通函数一样调用。例如，对训练集中的前两个实例进行预测（模型尚未训练，因此参数是随机的，预测结果会很差）：

```
>>> model(X_train[:2])
tensor([[-0.4718],
        [ 0.1131]], grad_fn=<AddmmBackward0>)
```

把模块当作函数使用时，PyTorch 会在内部调用模块的 `forward()` 方法。对于 `nn.Linear` 模块，`forward()` 方法计算的是 `X @ self.weight.T + self.bias`（其中 `X` 是输入）。这正是线性回归所需要的计算方式！

注意，结果包含 `grad_fn` 属性，说明模型进行预测时，autograd 已经跟踪了计算图。

> 提示：如果把自定义函数传递给模块的 `register_forward_hook()` 方法，那么每次模块自身被调用时，PyTorch 都会自动调用这个函数。这对于记录日志或调试特别有用。要移除钩子，只需调用 `register_forward_hook()` 返回对象的 `remove()` 方法。注意，钩子只有在像函数一样调用模型时才会生效；如果直接调用模型的 `forward()` 方法则不会生效，这也是为什么不应该直接调用 `forward()`。还可以使用 `register_backward_hook()` 注册在反向传播期间运行的函数。

现在已经有了模型，需要创建一个优化器来更新模型参数，并选择损失函数：

```
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
mse = nn.MSELoss()
```

PyTorch 提供了几种不同的优化器（下一章会讨论）。这里使用简单的随机梯度下降（SGD）优化器，它既可以用于 SGD，也可以用于小批量梯度下降或批量梯度下降。初始化优化器时，必须传入模型参数和学习率。

对于损失函数，创建了一个 `nn.MSELoss` 类的实例。它同样是一个模块，因此可以像函数一样使用：传入预测值和目标值，它就会计算均方误差。`nn` 模块还包含许多其他损失函数和神经网络工具，后面会看到。接着编写一个小函数来训练模型：

```
def train_bgd(model, optimizer, criterion, X_train, y_train, n_epochs):
    for epoch in range(n_epochs):
        y_pred = model(X_train)
        loss = criterion(y_pred, y_train)
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
        print(f"Epoch {epoch + 1}/{n_epochs}, Loss: {loss.item()}")
```

把这个训练循环与前面的训练循环比较一下：它们非常相似，只是现在使用了更高级的抽象，而不是直接操作张量和 autograd。需要注意以下几点：

- 在 PyTorch 中，损失函数对象通常称为 criterion（准则），以区别于损失值本身；损失值是在每次训练迭代中使用 criterion 计算出来的。本例中的 criterion 就是 `MSELoss` 实例。
- `optimizer.step()` 对应之前更新 `b` 和 `w` 的两行代码。

- `optimizer.zero_grad()` 当然对应之前把 `b.grad` 和 `w.grad` 清零的两行代码。这里不需要使用 `with torch.no_grad()`，因为优化器会在 `step()` 和 `zero_grad()` 函数内部自动完成这件事。

> 注意：大多数人更喜欢在调用 `loss.backward()` 之前调用 `zero_grad()`，而不是在之后调用；如果调用函数时梯度不为 0，这样可能更安全。不过通常没有区别，因为梯度会自动初始化为 `None`。

现在调用这个函数训练模型！

```
>>> train_bgd(model, optimizer, mse, X_train, y_train, n_epochs)
Epoch 1/20, Loss: 4.3378496170043945
Epoch 2/20, Loss: 0.780293345451355
[...]
Epoch 20/20, Loss: 0.5374288558959961
```

很好，模型已经训练完成。现在可以像调用函数一样调用模型来预测（最好放在前面介绍过的 `no_grad()` 上下文中）：

```
>>> X_new = X_test[:3]  # pretend these are new instances
>>> with torch.no_grad():
...     y_pred = model(X_new)  # use the trained model to make predictions
...
>>> y_pred
tensor([[0.8061],
        [1.7116],
        [2.6973]])
```

这些预测结果与前一个模型的预测结果相似，但并不完全相同。这是因为 `nn.Linear` 模块的参数初始化方式略有不同：它对权重和偏置项都使用从 $-\frac{\sqrt{2}}{4}$ 到 $+\frac{\sqrt{2}}{4}$ 的均匀随机分布（第 11 章将讨论初始化方法）。

现在已经熟悉 PyTorch 的高级 API，可以超越线性回归，构建多层感知器（第 9 章介绍过）。

10.2.3　实现回归 MLP（Implementing a Regression MLP）

PyTorch 提供了很有用的 `nn.Sequential` 模块，用来串联多个模块：使用一些输入调用这个模块时，它会把输入传给第一个模块，再把第一个模块的输出传给第二个模块，如此继续。大多数神经网络都包含模块堆叠；事实上，许多神经网络就是一

长串模块，因此 `nn.Sequential` 是 PyTorch 最有用的模块之一。我们想构建的 MLP 正是这样一个简单的模块堆叠：两个隐藏层和一个输出层。用 `nn.Sequential` 构建它：

```
torch.manual_seed(42)
model = nn.Sequential(
    nn.Linear(n_features, 50),
    nn.ReLU(),
    nn.Linear(50, 40),
    nn.ReLU(),
    nn.Linear(40, 1)
)
```

逐层看看：

第一层必须拥有与数据匹配的输入数量：`n_features`（本例中等于 8）。不过，它可以有任意数量的输出；这里选择 50（这是一个可以调节的超参数）。

接下来是一个 `nn.ReLU` 模块，为第一个隐藏层实现 ReLU 激活函数。这个模块不包含任何模型参数，并且逐元素运算，因此其输出形状与输入形状相同。

第二个隐藏层必须拥有与前一层输出数量相同的输入数量，本例中为 50。不过，它可以有任意数量的输出。在所有隐藏层中使用相同的输出维度很常见，但本例使用 40，是为了明确一层的输出必须与下一层的输入匹配。

接着再次使用 `nn.ReLU` 模块，为第二个隐藏层实现激活函数。

最后，输出层必须有 40 个输入，但这一次输出数量不是自由选择的：它必须与目标的维度匹配。由于目标只有一个维度，输出层只能有一个输出维度。

现在像前面一样训练模型：

```
>>> learning_rate = 0.1
>>> optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
>>> mse = nn.MSELoss()
>>> train_bgd(model, optimizer, mse, X_train, y_train, n_epochs)
Epoch 1/20, Loss: 5.045480251312256
Epoch 2/20, Loss: 2.0523128509521484
[...]
Epoch 20/20, Loss: 0.565444827079773
```

就是这样，你可以告诉朋友已经用 PyTorch 训练了第一个神经网络！不过，我们仍然使用批量梯度下降，在每次迭代时针对整个训练集计算梯度。这种方法适用于小数据集；如果希望扩展到大型数据集和大型模型，就需要切换到小批量梯度下降。

10.2.4　使用 DataLoader 实现小批量梯度下降（Implementing Mini-Batch Gradient Descent Using DataLoaders）

为了帮助实现小批量梯度下降，PyTorch 在 `torch.utils.data` 模块中提供了 `DataLoader` 类。它可以高效地加载指定大小的数据批次，也可以在每个 epoch 打乱数据（如果需要）。`DataLoader` 要求数据集由一个至少包含两个方法的对象表示：`__len__(self)` 用于获取样本数量，`__getitem__(self, index)` 用于加载给定索引处的样本（包括目标值）。

在这里，训练集位于 `X_train` 和 `y_train` 张量中，因此首先需要把这些张量包装到具有所需 API 的数据集对象中。PyTorch 提供了 `TensorDataset` 类来完成这件事。下面创建一个 `TensorDataset` 包装训练集，再创建一个 `DataLoader` 从数据集中提取批次。训练时希望打乱数据集，因此指定 `shuffle=True`：

```
from torch.utils.data import TensorDataset, DataLoader
train_dataset = TensorDataset(X_train, y_train)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
```

现在模型更大，也有了逐批训练的工具，正好可以开始使用硬件加速。这个过程很简单：只需把模型移到 GPU（这样会把模型的所有参数移到 GPU RAM），并在训练期间每次迭代开始时把每个批次复制到 GPU。和处理张量一样，可以使用模型的 `to()` 方法移动模型：

```
torch.manual_seed(42)
model = nn.Sequential([...])  # create the model just like earlier
model = model.to(device)
```

还可以像之前一样创建损失函数和优化器，不过此时应使用更低的学习率，例如 0.02。

> 警告：有些优化器包含内部状态，正如第 11 章将看到的那样。优化器通常会把内部状态分配到与模型参数相同的设备上，因此必须在把模型移动到 GPU 后再创建优化器。

现在创建一个 `train()` 函数来实现小批量梯度下降：

```
def train(model, optimizer, criterion, train_loader, n_epochs):
    model.train()
    for epoch in range(n_epochs):
        total_loss = 0.
        for X_batch, y_batch in train_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            y_pred = model(X_batch)
            loss = criterion(y_pred, y_batch)
            total_loss += loss.item()
            loss.backward()
            optimizer.step()
            optimizer.zero_grad()
        mean_loss = total_loss / len(train_loader)
        print(f"Epoch {epoch + 1}/{n_epochs}, Loss: {mean_loss:.4f}")
```

在每个 epoch 中，这个函数都会遍历整个训练集，一次处理一个批次，并像前面一样处理每个批次。不过，第一行 **`model.train()`** 是做什么的？它会把模型及其所有子模块切换到训练模式。现在这还没有任何区别，但在第 11 章开始使用训练和评估期间行为不同的层（例如 **`nn.Dropout`** 或 **`nn.BatchNorm1d`**）时，这一点很重要。每当想在训练之外使用模型（例如评估模型，或对新实例进行预测）时，都必须先运行 **`model.eval()`** 把模型切换到评估模式。注意，**`model.training`** 保存了一个布尔值，指示模型的当前模式。

> 提示：PyTorch 本身不提供训练循环的实现，你必须自己构建。正如刚才看到的，训练循环并不长，而且许多人喜欢它所带来的自由、清晰和控制力。不过，如果更愿意使用经过充分测试、开箱即用并且包含所需各种功能（例如多 GPU 支持）的训练循环，可以使用 PyTorch Lightning、FastAI、Catalyst 或 Keras 等库。这些库构建在 PyTorch 之上，包含训练循环和许多其他功能（Keras 从第 3 版起支持 PyTorch，也支持 TensorFlow 和 JAX）。去了解一下吧！

现在调用 **`train()`** 函数，在 GPU 上训练模型：

```
>>> train(model, optimizer, mse, train_loader, n_epochs)
Epoch 1/20, Loss: 0.6958
Epoch 2/20, Loss: 0.4480
[...]
Epoch 20/20, Loss: 0.3227
```

效果很好：在相同 epoch 数下，损失确实低得多。不过你可能注意到，每个 epoch 运行得更慢。下面两个简单调整可以显著加快训练：

- 如果使用 CUDA 设备，通常应在创建数据加载器时设置 `pin_memory=True`。这会把数据分配到页锁定内存中，保证它在 CPU RAM 中拥有固定的物理位置，从而支持直接内存访问（DMA）传输到 GPU，消除原本需要的额外复制操作。虽然这会占用更多 CPU RAM（因为这部分内存不能交换到磁盘），但通常能显著加快数据传输，从而加快训练。使用 `to()` 方法把张量传到 GPU 时，还可以设置 `non_blocking=True`，避免数据传输期间阻塞 CPU（只有设置了 `pin_memory=True` 时才有效）。
- 当前训练循环要等一个批次完全处理完，才会加载下一个批次。通常可以让 CPU 预取下一个批次，同时 GPU 继续处理当前批次，以此加快训练。为此，把数据加载器的 `num_workers` 参数设置为希望用于数据加载和预处理的进程数量。最佳数量取决于平台、硬件和工作负载，应尝试不同取值。还可以设置数据加载器的 `prefetch_factor` 参数，调整每个 worker 预取的批次数。注意，创建和同步 worker 的开销有时反而会拖慢训练（特别是在 Windows 上）。此时可以尝试设置 `persistent_workers=True`，在多个 epoch 之间复用同一批 worker。

现在退一步回顾一下：你已经了解 PyTorch 的基础知识（张量和 autograd），可以使用 PyTorch 高级 API 构建神经网络，并借助优化器、准则和数据加载器，使用小批量梯度下降训练网络。下一步是学习如何评估模型。

10.3　模型评估（Model Evaluation）

编写一个评估模型的函数。它接收模型、要评估的数据集对应的 `DataLoader`、用于计算给定批次指标的函数，以及用于聚合批次指标的函数（默认只计算平均值）：

```
def evaluate(model, data_loader, metric_fn, aggregate_fn=torch.mean):
    model.eval()
    metrics = []
    with torch.no_grad():
        for X_batch, y_batch in data_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            y_pred = model(X_batch)
            metric = metric_fn(y_pred, y_batch)
            metrics.append(metric)
    return aggregate_fn(torch.stack(metrics))
```

现在为验证集创建 **TensorDataset** 和 **DataLoader**，并把它传给 **evaluate()** 函数来计算验证集 MSE：

```
>>> valid_dataset = TensorDataset(X_valid, y_valid)
>>> valid_loader = DataLoader(valid_dataset, batch_size=32)
>>> valid_mse = evaluate(model, valid_loader, mse)
>>> valid_mse
tensor(0.4080, device='cuda:0')
```

这很好。但假设我们想使用 RMSE，而不是 MSE（正如第 2 章所见，RMSE 更容易解释）。PyTorch 没有内置的 RMSE 函数，不过很容易自己编写：

```
>>> def rmse(y_pred, y_true):
...     return ((y_pred - y_true) ** 2).mean().sqrt()
...
>>> evaluate(model, valid_loader, rmse)
tensor(0.5668, device='cuda:0')
```

等等！RMSE 应该等于 MSE 的平方根；然而，对刚才得到的 MSE 开平方，结果却不同：

```
>>> valid_mse.sqrt()
tensor(0.6388, device='cuda:0')
```

原因是：我们没有在整个验证集上计算 RMSE，而是在每个批次上计算 RMSE，然后计算所有批次 RMSE 的平均值。这在数学上不等价于在整个验证集上计算 RMSE。解决方法是使用 MSE 作为 **metric_fn**，再让 **aggregate_fn** 计算 MSE 平均值的平方根[10]：

[10]所有批次大小相同时，批次 MSE 的平均值等于整体 MSE。不过最后一个批次通常更小，但这造成的差异很小。

```
>>> evaluate(model, valid_loader, mse,
...          aggregate_fn=lambda metrics: torch.sqrt(torch.mean(metrics)))
...
tensor(0.6388, device='cuda:0')
```

这就好多了！

与其自己实现指标，不如使用 Torch Metrics 库（由与 PyTorch Lightning 相同的团队开发），它提供了许多经过充分测试的流式指标。流式指标是能够跟踪某个给定指标、并逐批更新的对象。Colab 没有预装 Torch Metrics，因此需要运行**`%pip install torchmetrics`**，然后像下面这样实现 **`evaluate_tm()`** 函数：

```
import torchmetrics
def evaluate_tm(model, data_loader, metric):
    model.eval()
    metric.reset()  # reset the metric at the beginning
    with torch.no_grad():
        for X_batch, y_batch in data_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            y_pred = model(X_batch)
            metric.update(y_pred, y_batch)  # update it at each iteration
    return metric.compute()  # compute the final result at the end
```

然后创建一个 RMSE 流式指标，把它移动到 GPU，并用它评估验证集：

```
>>> rmse = torchmetrics.MeanSquaredError(squared=False).to(device)
>>> evaluate_tm(model, valid_loader, rmse)
tensor(0.6388, device='cuda:0')
```

果然得到了正确结果！现在可以尝试更新 **train()** 函数，在训练过程中评估模型表现：既在每个 epoch 中评估训练集，也在每个 epoch 结束时评估验证集。和往常一样，如果模型在训练集上的表现远好于验证集，模型很可能过拟合了训练集；也可能存在错误，例如训练集和验证集之间的数据不匹配。如果绘制并分析学习曲线，就更容易发现这个问题，方法与第 4 章类似。可以使用 Matplotlib，也可以使用 TensorBoard 等可视化工具（示例见 notebook）。

现在已经知道如何使用 PyTorch 构建、训练和评估回归 MLP，也知道如何使用训练好的模型进行预测。很好！不过到目前为止，我们只研究了由线性层和 ReLU 激活函数序列组成的简单顺序模型。如何构建更复杂的非顺序模型？这需要构建自定义模块。

10.4　使用自定义模块构建非顺序模型（Building Nonsequential Models Using Custom Modules）

非顺序神经网络的一个例子是 Wide & Deep 神经网络。这种神经网络架构由 Heng-Tze Cheng 等人在 2016 年的一篇论文中提出[11]。它把全部或部分输入直接连接到输出层，如图 10-1 所示。这种架构能够让神经网络同时学习深层模式（通过深路径）和简单规则（通过短路径）。短路径还可以向神经网络提供人工设计的特征。相比之下，普通 MLP 强制所有数据流经完整的层堆叠，因此数据中的简单模式可能会被这一连串变换扭曲。

[11] Heng-Tze Cheng 等人，“Wide & Deep Learning for Recommender Systems”，Proceedings of the First Workshop on Deep Learning for Recommender Systems（2016）：7-10。

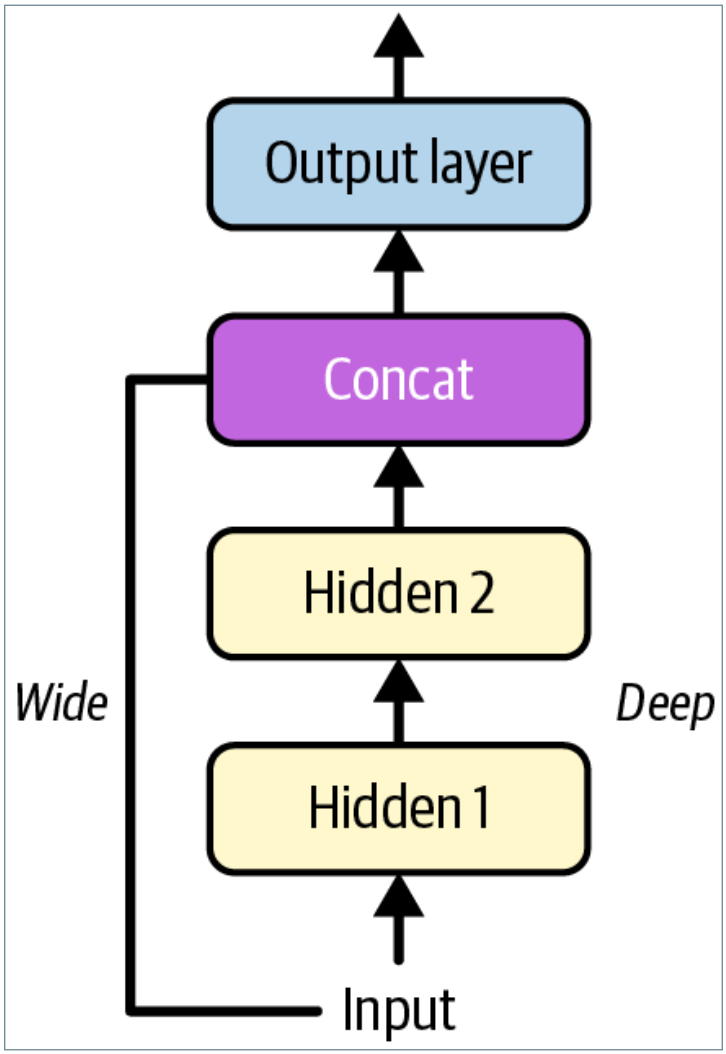

图 10-1: Wide & Deep 神经网络（Wide & Deep neural network）

我们来构建这样的神经网络，处理加利福尼亚住房数据集。由于这种宽而深的架构是非顺序的，必须创建自定义模块。这比听起来简单：创建一个继承自 `torch.nn.Module` 的类，在调用基类的 `__init__()` 方法后，在构造函数中创建所需的所有层，然后在 `forward()` 方法中定义如何使用这些层：

```
class WideAndDeep(nn.Module):
    def __init__(self, n_features):
        super().__init__()
        self.deep_stack = nn.Sequential(
            nn.Linear(n_features, 50), nn.ReLU(),
            nn.Linear(50, 40), nn.ReLU(),
        )
        self.output_layer = nn.Linear(40 + n_features, 1)
    def forward(self, X):
        deep_output = self.deep_stack(X)
        wide_and_deep = torch.concat([X, deep_output], dim=1)
        return self.output_layer(wide_and_deep)
```

注意，自定义模块内部可以使用任何类型的模块。本例中使用 `nn.Sequential` 模块构建模型的“深”部分（实际上并不深，这只是一个玩具示例）。它与前面的 MLP 相同，只是把输出层分离出来，因为需要把模型输入和深层部分的输出拼接起来。出于同样的原因，现在输出层有 `40 + n_features` 个输入，而不是只有 40 个输入。

在 `forward()` 方法中，只需把输入 `X` 传给深层堆叠，把输入与深层堆叠的输出拼接起来，再把结果传给输出层。

> 注意：模块有一个 `children()` 方法，会返回该模块子模块的迭代器（不递归遍历）；还有一个 `named_children()` 方法。如果模型包含数量可变的子模块，应把它们存放在 `nn.ModuleList` 或 `nn.ModuleDict` 中。这样，`children()` 和 `named_children()` 方法才能返回它们，而不是普通的 Python 列表和字典。同样，如果模型包含数量可变的参数，应把它们存放在 `nn.ParameterList` 或 `nn.ParameterDict` 中，以确保 `parameters()` 和 `named_parameters()` 方法能够返回它们。

现在可以创建自定义模块的实例，把它移到 GPU 上，训练、评估并使用它，方式与之前的模型完全相同：

```
torch.manual_seed(42)
model = WideAndDeep(n_features).to(device)
learning_rate = 0.002  # the model changed, so did the optimal learning rate
[...]  # train, evaluate, and use the model, exactly like earlier
```

不过，如果希望把一部分特征送入宽路径，把另一部分（可能与前一部分重叠）送入深路径，如图 10-2 所示，该怎么办？一种方法是在 `forward()` 方法中拆分输入，例如：

```
class WideAndDeepV2(nn.Module):
    [...]  # same constructor as earlier, except with adjusted input sizes
    def forward(self, X):
        X_wide = X[:, :5]
        X_deep = X[:, 2:]
        deep_output = self.deep_stack(X_deep)
        wide_and_deep = torch.concat([X_wide, deep_output], dim=1)
        return self.output_layer(wide_and_deep)
```

这种方法当然有效；不过，很多情况下，最好让模型直接接收两个独立的张量作为输入。下面看看原因和实现方法。

10.4.1　构建具有多个输入的模型（Building Models with Multiple Inputs）

有些模型需要多个输入，而这些输入很难合并为一个张量。例如，输入可能有不同的维度（比如希望同时向神经网络输入图像和文本）。为了让 Wide & Deep 模型接收两个独立的输入（如图 10-2 所示），首先需要修改模型的 `forward()` 方法：

```
class WideAndDeepV3(nn.Module):
    [...]  # same as WideAndDeepV2
    def forward(self, X_wide, X_deep):
        deep_output = self.deep_stack(X_deep)
        wide_and_deep = torch.concat([X_wide, deep_output], dim=1)
        return self.output_layer(wide_and_deep)
```

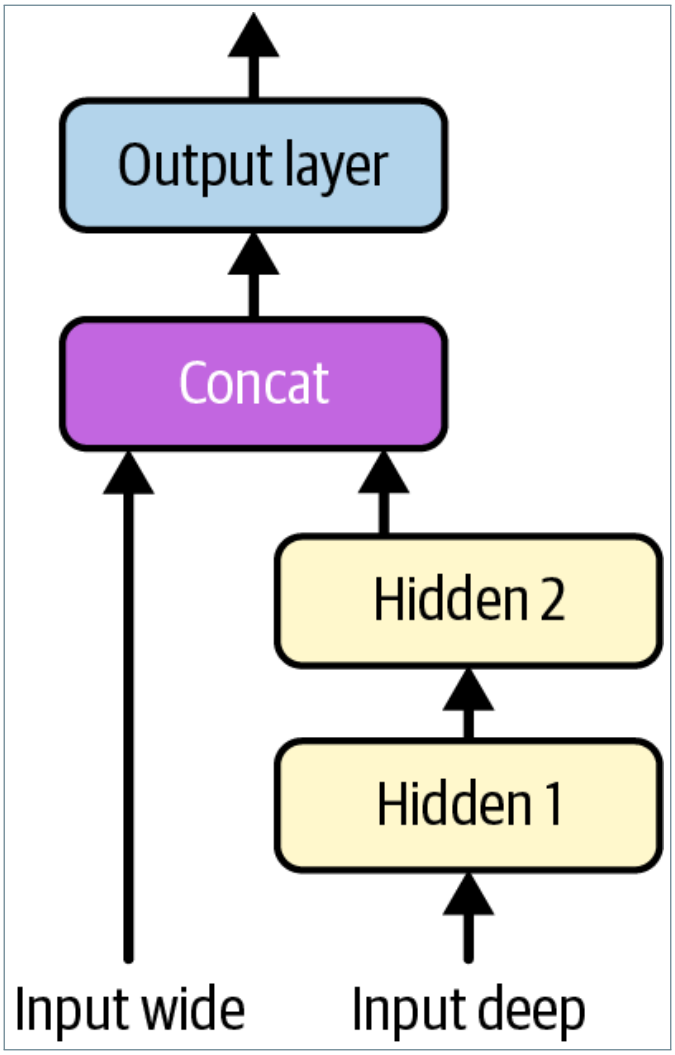

图 10-2: 处理多个输入（Handling multiple inputs）

接下来需要创建数据集，让它们分别返回宽路径输入和深路径输入：

```
train_data_wd = TensorDataset(X_train[:, :5], X_train[:, 2:], y_train)
train_loader_wd = DataLoader(train_data_wd, batch_size=32, shuffle=True)
[...]  # same for the validation set and test set
```

由于数据加载器现在每次迭代返回三个张量，而不是两个，因此需要更新评估和训练函数中的主循环：

```
for X_batch_wide, X_batch_deep, y_batch in data_loader:
    X_batch_wide = X_batch_wide.to(device)
    X_batch_deep = X_batch_deep.to(device)
    y_batch = y_batch.to(device)
    y_pred = model(X_batch_wide, X_batch_deep)
    [...]  # the rest of the function is unchanged
```

另外，由于输入的顺序与 **forward()** 方法参数的顺序一致，可以使用 Python 的 * 运算符，解包 **data_loader** 返回的所有输入并把它们传给模型。只要顺序正确，这种实现适用于接收任意数量输入的模型，而不仅仅是两个输入：

```
for *X_batch_inputs, y_batch in data_loader:
    X_batch_inputs = [X.to(device) for X in X_batch_inputs]
    y_batch = y_batch.to(device)
    y_pred = model(*X_batch_inputs)
    [...]
```

模型输入很多时，很容易弄错输入顺序，从而导致难以调试的问题。为了避免这种情况，可以给每个输入命名。为此，可以定义一个自定义数据集，让它返回从输入名称到输入值的字典：

```
class WideAndDeepDataset(torch.utils.data.Dataset):
    def __init__(self, X_wide, X_deep, y):
        self.X_wide = X_wide
        self.X_deep = X_deep
        self.y = y
    def __len__(self):
        return len(self.y)
    def __getitem__(self, idx):
        input_dict = {"X_wide": self.X_wide[idx], "X_deep": self.X_deep[idx]}
        return input_dict, self.y[idx]
```

然后创建数据集和数据加载器：

```
train_data_named = WideAndDeepDataset(
    X_wide=X_train[:, :5], X_deep=X_train[:, 2:], y=y_train)
train_loader_named = DataLoader(train_data_named, batch_size=32, shuffle=True)
[...]  # same for the validation set and test set
```

同样，也需要更新评估和训练函数中的主循环：

```
for inputs, y_batch in data_loader:
    inputs = {name: X.to(device) for name, X in inputs.items()}
    y_batch = y_batch.to(device)
    y_pred = model(X_wide=inputs["X_wide"], X_deep=inputs["X_deep"])
    [...]  # the rest of the function is unchanged
```

另外，由于所有输入名称都与 `forward()` 方法的参数名称匹配，可以使用 Python 的 `**` 运算符，把 `inputs` 字典中的所有张量解包为带名称的参数传给模型：`y_pred = model(**inputs)`。

现在已经知道如何构建具有一个或多个输入的顺序模型和非顺序模型，接下来看看具有多个输出的模型。

10.4.2　构建具有多个输出的模型（Building Models with Multiple Outputs）

有很多场景需要神经网络拥有多个输出：

- 任务本身就需要多个输出。例如，希望定位并分类图片中的主要对象。这同时是回归任务和分类任务。
- 还可能需要基于同一份数据执行多个相互独立的任务。当然，可以为每个任务训练一个神经网络，但在很多情况下，使用一个神经网络、为每个任务设置一个输出，可以在所有任务上取得更好的结果。这是因为神经网络能够学习对多个任务都有用的数据特征。例如，可以对人脸图片执行多任务分类：用一个输出分类人物的面部表情（微笑、惊讶等），用另一个输出识别人物是否佩戴眼镜。
- 另一个用途是正则化（即一种训练约束，目标是减少过拟合，从而提高模型的泛化能力）。例如，可以在神经网络架构中加入一个辅助输出（见图 10-3），确保网络底层能够独立学到有用的内容，而不依赖网络的其余部分。

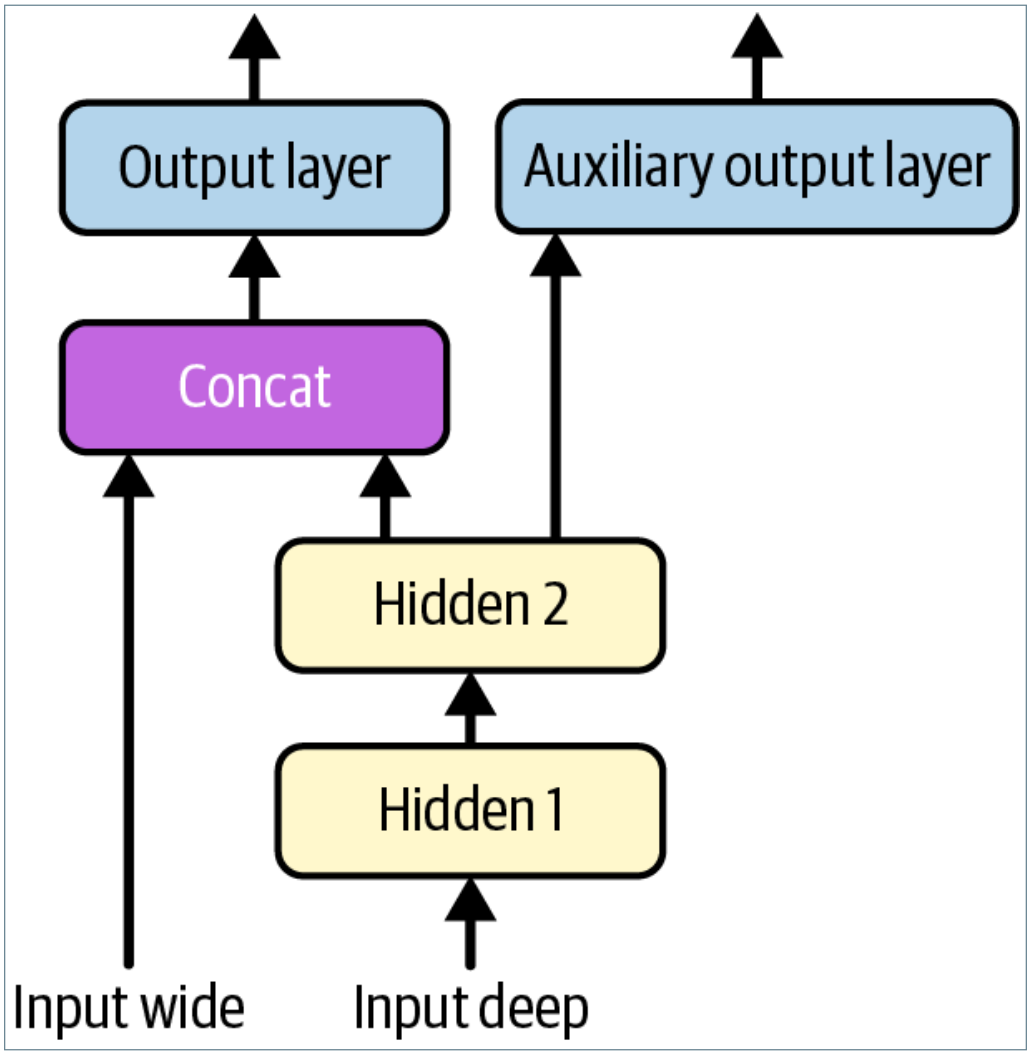

图 10-3: 处理多个输出：本例中添加一个辅助输出进行正则化（Handling multiple outputs, in this example to add an auxiliary output for regularization）

向 Wide & Deep 模型添加一个辅助输出，确保深层部分本身就能做出良好预测。由于深层堆叠的输出维度是 40，目标只有一个维度，因此必须添加一个 `nn.Linear` 层，把 40 个维度的辅助输出降为 1 个维度。还需要修改 `forward()` 方法来计算辅助输出，并同时返回主输出和辅助输出：

```
class WideAndDeepV4(nn.Module):
    def __init__(self, n_features):
        [...]  # same as earlier
        self.aux_output_layer = nn.Linear(40, 1)
    def forward(self, X_wide, X_deep):
        deep_output = self.deep_stack(X_deep)
        wide_and_deep = torch.concat([X_wide, deep_output], dim=1)
        main_output = self.output_layer(wide_and_deep)
        aux_output = self.aux_output_layer(deep_output)
        return main_output, aux_output
```

接下来需要更新训练函数中的主循环：

```
for inputs, y_batch in train_loader:
    y_pred, y_pred_aux = model(**inputs)
    main_loss = criterion(y_pred, y_batch)
    aux_loss = criterion(y_pred_aux, y_batch)
    loss = 0.8 * main_loss + 0.2 * aux_loss
    [...]  # the rest is unchanged
```

注意，模型现在返回主预测值 `y_pred` 和辅助预测值 `y_pred_aux`。本例可以使用相同的目标值和相同的损失函数，分别计算主输出损失和辅助输出损失。在其他情况下，每个输出可能有不同的目标值和损失函数；此时需要创建一个自定义数据集，返回所有必需的目标值。

每个输出都有一个损失后，需要把它们组合成一个由梯度下降最小化的单一损失。一般来说，最终损失就是所有输出损失的加权和。本例中，主损失的权重较高（0.8），因为主输出是最关心的；辅助损失的权重较低（0.2）。这个比例是一个可以调节的正则化超参数。

还需要更新评估函数中的主循环。不过，在这里可以忽略辅助输出，因为真正关心的只有主输出；辅助输出只在训练过程中用于正则化：

```
for inputs, y_batch in data_loader:
    y_pred, _ = model(**inputs)
    metric.update(y_pred, y_batch)
    [...]  # the rest is unchanged
```

Voilà！现在可以任意组合预定义模块和自定义模块，构建和训练各种神经网络架构，并使用任意数量的输入和输出。神经网络的灵活性正是它们的主要优点之一。不过到目前为止，我们只处理了回归任务，下面转向分类。

10.5　使用 PyTorch 构建图像分类器（Building an Image Classifier with PyTorch）

和第 9 章一样，我们处理 Fashion-MNIST 数据集，因此第一步是下载数据集。可以像第 9 章一样使用 `fetch_openml()` 函数，但这里改用另一种方法，即使用 TorchVision 库。

10.5.1　使用 TorchVision 加载数据集（Using TorchVision to Load the Dataset）

TorchVision 库是 PyTorch 生态系统的重要组成部分：它提供了许多计算机视觉工具，包括下载常见数据集（如 MNIST 或 Fashion-MNIST）的实用函数、用于各种计算机视觉任务的预训练模型（见第 12 章）、变换图像的函数（例如裁剪、旋转、缩放等）以及更多功能。Colab 已经预装 TorchVision，因此直接使用它加载 Fashion-MNIST。该数据集已经拆分为训练集（60,000 张图像）和测试集（10,000 张图像），但下面使用 PyTorch 的 `random_split()` 函数，从训练集中留出最后 5,000 张图像作为验证集：

```
import torchvision
import torchvision.transforms.v2 as T
toTensor = T.Compose([T.ToImage(), T.ToDtype(torch.float32, scale=True)])
train_and_valid_data = torchvision.datasets.FashionMNIST(
    root="datasets", train=True, download=True, transform=toTensor)
test_data = torchvision.datasets.FashionMNIST(
    root="datasets", train=False, download=True, transform=toTensor)
torch.manual_seed(42)
train_data, valid_data = torch.utils.data.random_split(
    train_and_valid_data, [55_000, 5_000])
```

导入库、加载数据集之前，创建了一个 `toTensor` 对象。这是做什么的？默认情况下，`FashionMNIST` 类以 PIL（Python Imaging Library）图像的形式加载图像，像

素值是范围为 0 到 255 的整数。但这里需要的是像素值经过缩放的 PyTorch 浮点张量。幸运的是，TorchVision 数据集接受一个 **transform** 参数，可以传入一个在访问数据时即时执行的预处理函数（如果需要预处理目标值，还有 **target_transform** 参数）。TorchVision 提供了许多可以用于此目的的变换对象（其中大多数变换本身也是 PyTorch 模块）。

在这段代码中，创建了一个 **Compose** 变换，把两个变换串联起来：先使用 **ToImage**，再使用 **ToDtype**。**ToImage** 把各种格式（包括 PIL 图像、NumPy 数组和张量）转换为 TorchVision 的 **Image** 类，该类是 **Tensor** 的子类。**ToDtype** 转换数据类型，这里转换为 32 位浮点数；同时把它的 **scale** 参数设为 **True**，确保值缩放到 0.0 和 1.0 之间[12]。

> 注意：TorchVision 第 1 版变换 API 仍然可用，用于向后兼容，可以通过 **import torchvision.transforms** 导入。不过，应该改用第 2 版（**torchvision.transforms.v2**），因为它速度更快、功能也更多。

接下来加载数据集：先加载训练和验证数据，再加载测试数据。**root** 参数是 TorchVision 创建 Fashion-MNIST 数据集子目录的路径。**train** 参数指示加载训练集（默认是 **True**）还是测试集。**download** 参数指示本地找不到数据集时是否下载数据集（默认是 **False**）。同时设置 **transform=toTensor**，使用自定义预处理流水线。

和往常一样，需要创建数据加载器：

```
train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
valid_loader = DataLoader(valid_data, batch_size=32)
test_loader = DataLoader(test_data, batch_size=32)
```

现在看看训练集中的第一张图像：

[12] TorchVision 包含一个能完成所有这些工作的 **ToTensor** 变换，但它已被弃用，因此建议使用这里的处理流水线。

```
>>> X_sample, y_sample = train_data[0]
>>> X_sample.shape
torch.Size([1, 28, 28])
>>> X_sample.dtype
torch.float32
```

第 9 章中，每张图像由包含 784 个像素强度的一维数组表示；现在每个图像张量有 3 个维度，形状为 `[1, 28, 28]`。第一维是通道维度。灰度图像只有一个通道（彩色图像通常有三个通道，第 12 章会看到）。另外两个维度是高度和宽度。例如，`X_sample[0, 2, 4]` 表示位于通道 0、第 2 行、第 4 列的像素。在 Fashion-MNIST 中，值越大表示像素越暗。

> 警告：PyTorch 要求通道维度位于第一位，而 Matplotlib、PIL、TensorFlow、OpenCV 或 Scikit-Image 等许多其他库要求通道维度位于最后。使用不同库时，务必确认通道维度位于正确位置。`ToImage` 已经负责把通道维度移到第一位，否则可以使用 `torch.permute()` 函数。

至于目标值，它们是从 0 到 9 的整数，可以使用第 9 章中的同一个 `class_names` 数组来解释。事实上，许多数据集（包括 Fashion-MNIST）都有一个 `classes` 属性，其中包含类别名称列表。例如，下面可以确认样本图像代表一只踝靴：

```
>>> train_and_valid_data.classes[y_sample]
'Ankle boot'
```

10.5.2　构建分类器（Building the Classifier）

构建一个具有两个隐藏层的分类 MLP 自定义模块：

```
class ImageClassifier(nn.Module):
    def __init__(self, n_inputs, n_hidden1, n_hidden2, n_classes):
        super().__init__()
        self.mlp = nn.Sequential(
            nn.Flatten(),
            nn.Linear(n_inputs, n_hidden1),
            nn.ReLU(),
            nn.Linear(n_hidden1, n_hidden2),
            nn.ReLU(),
            nn.Linear(n_hidden2, n_classes)
        )
    def forward(self, X):
        return self.mlp(X)
torch.manual_seed(42)
model = ImageClassifier(n_inputs=28 * 28, n_hidden1=300, n_hidden2=100,
                        n_classes=10)
xentropy = nn.CrossEntropyLoss()
```

这段代码有几点需要注意：

- 首先，模型由一个层序列组成，因此使用 `nn.Sequential` 模块。其实也可以直接写 `model = nn.Sequential(...)`，不必创建自定义模块；但通常最好把模型包装在自定义模块中，这样代码更容易部署和复用，也更容易调节超参数。
- 模型以 `nn.Flatten` 层开始。这个层不含任何参数，只把每个输入样本重塑为一维，这是 `nn.Linear` 层所需要的。例如，一批 32 张 Fashion-MNIST 图像的形状为 `[32, 1, 28, 28]`，经过 `nn.Flatten` 层后变为 `[32, 784]`（因为 28 × 28 = 784）。
- 第一个隐藏层必须拥有正确数量的输入（28 × 28 = 784），输出层必须拥有正确数量的输出（10 个，每个类别一个）。
- 每个隐藏层后使用 ReLU 激活函数，输出层后完全不使用激活函数。
- 由于这是多分类任务，使用 `nn.CrossEntropyLoss`。它既接受类别索引作为目标值（如本例），也接受类别概率（例如 one-hot 向量）。

> 提示：形状错误很常见，特别是在刚开始学习时，因此应该熟悉错误信息。可以尝试移除 `nn.Flatten` 模块，或者修改输入和/或标签的形状，看看会得到什么错误。

等等！第 9 章不是说，多分类任务的输出层应该使用 Softmax 激活函数吗？事实证明，PyTorch 的 `nn.CrossEntropyLoss` 直接根据 logits（即第 4 章介绍过的类别分数）计算交叉熵损失，而不是根据类别概率计算。这避免了训练期间一些代价高昂的计算（例如相互抵消的对数和指数运算），节省了计算和 RAM，也更加数值稳定。不过，这种方法的缺点是模型必须输出 logits；也就是说，需要在想要获得类别概率时，手动对 logits 调用 Softmax 函数，稍后会看到具体做法。

10.5.3　其他分类损失（Other Classification Losses）

对于多分类任务，另一种选择是在输出层添加 `nn.LogSoftmax` 激活函数，然后使用 `nn.NLLLoss`（负对数似然损失）。此时模型输出的是对数概率（而不是 logits），损失函数根据这些对数概率计算交叉熵。需要实际的估计概率时，只需对对数概率应用指数函数。这种方法比使用 `nn.CrossEntropyLoss` 稍慢，因此不常用；但如果希望模型输出对数概率，或者希望在计算损失之前调整概率分布，它有时会很有用。

对于二元分类任务，输出层必须只有一个输出神经元。通常使用二元交叉熵损失（BCE）：`nn.BCEWithLogitsLoss`。

模型输出 logits，因此必须应用 sigmoid 函数来获得正类的估计概率。另一种方法是在输出层添加 `nn.Sigmoid` 激活函数，然后使用 `nn.BCELoss`；这样模型会直接输出估计概率（但速度稍慢，数值稳定性也稍差）。

对于多标签二元分类，唯一的区别是输出层必须为每个标签设置一个神经元。

现在可以像往常一样训练模型（例如使用 `train()` 函数和 SGD 优化器）。评估模型时，可以使用 `torchmetrics` 库中的 Accuracy 流式指标，并把它移到 GPU：

```
accuracy = torchmetrics.Accuracy(task="multiclass", num_classes=10).to(device)
```

> 警告：使用 GPU 训练模型需要几分钟（没有 GPU 时可能要久得多）。处理图像所需的计算和内存，明显多于处理低维数据。

模型在训练集上达到约 92.8% 的准确率，在验证集上达到 87.2% 的准确率（具体结果可能因使用的硬件加速器而略有不同）。这说明存在一些过拟合，因此可以减少神经元数量，或者添加正则化（见第 11 章）。

模型训练完成后，可以用它对新图像进行预测。例如，对验证集的第一个批次进行预测，并查看前 3 张图像的结果：

```
>>> model.eval()
>>> X_new, y_new = next(iter(valid_loader))
>>> X_new = X_new[:3].to(device)
>>> with torch.no_grad():
...     y_pred_logits = model(X_new)
...
>>> y_pred = y_pred_logits.argmax(dim=1)  # index of the largest logit
>>> y_pred
tensor([7, 4, 2], device='cuda:0')
>>> [train_and_valid_data.classes[index] for index in y_pred]
['Sneaker', 'Coat', 'Pullover']
```

对于每张图像，预测类别就是 logits 最大的类别。本例中，三个预测都正确！

不过，如果想知道模型估计的概率呢？由于输出层没有包含 Softmax 激活函数，正如前面所说，需要手动计算 logits 的 Softmax。可以创建一个 **nn.Softmax** 模块并把 logits 传给它；也可以直接调用 **softmax()** 函数。它是 **torch.nn.functional** 模块中众多函数之一（按照惯例，通常把这个模块导入为 **F**）。两种方式没有太大区别，只是直接调用函数不需要创建用不到的模块实例：

```
>>> import torch.nn.functional as F
>>> y_proba = F.softmax(y_pred_logits, dim=1)
>>> y_proba.round(decimals=3)
tensor([[0.000, 0.000, 0.000, 0.000, 0.000, 0.001, 0.000, 0.911, 0.000, 0.088],
        [0.000, 0.000, 0.004, 0.000, 0.996, 0.000, 0.000, 0.000, 0.000, 0.000],
        [0.000, 0.000, 0.625, 0.000, 0.335, 0.000, 0.039, 0.000, 0.000, 0.000]],
       device='cuda:0')
```

和第 9 章一样，模型对前两个预测非常有信心，置信度分别是 91.1% 和 99.6%。

> 提示：如果希望在训练期间使用标签平滑，只需把 **nn.CrossEntropyLoss** 的 **label_smoothing** 超参数设置为所需的平滑量，取值范围为 0 到 1（例如 0.05）。

获取模型的 top-k 预测通常很有用。可以使用 **torch.topk()** 函数，它会返回一个元组，其中包含 top-k 的值及其索引：

```
>>> y_top4_logits, y_top4_indices = torch.topk(y_pred_logits, k=4, dim=1)
>>> y_top4_probas = F.softmax(y_top4_logits, dim=1)
>>> y_top4_probas.round(decimals=3)
tensor([[0.9110, 0.0880, 0.0010, 0.0000],
        [0.9960, 0.0040, 0.0000, 0.0000],
        [0.6250, 0.3350, 0.0390, 0.0000]], device='cuda:0')
>>> y_top4_indices
tensor([[7, 9, 5, 8],
        [4, 2, 6, 0],
        [2, 4, 6, 0]], device='cuda:0')
```

对于第一张图像，模型最有把握的预测是类别 7（运动鞋），置信度为 91.1%；第二个预测是类别 9（踝靴），置信度为 8.8%，依此类推。

> 提示：Fashion-MNIST 数据集是平衡的，也就是说每个类别的实例数量相同。对于不平衡数据集，通常应当给稀有类别更大的权重，给常见类别更小的权重，否则模型会偏向更常见的类别。可以通过设置 `nn.CrossEntropyLoss` 的 `weight` 参数来实现。例如，有 3 个类别，实例数量分别为 900、700 和 400（总数为 2,000），则对应权重应为 2000/900、2000/700 和 2000/400。最好把权重标准化，使它们的和为 1，因此本例应设置 `weight=torch.tensor([0.2205, 0.2835, 0.4961])`。

你的 PyTorch 超能力正在增强：现在已经可以构建、训练和评估回归与分类神经网络。下一步是学习如何微调模型的超参数。

10.6　使用 Optuna 微调神经网络超参数（Fine-Tuning Neural Network Hyperparameters with Optuna）

第 9 章讨论了如何手动为模型选择合理的超参数值，但如果希望更进一步，自动搜索合适的超参数值，该怎么办？一种选择是把 PyTorch 模型转换为 Scikit-Learn 估计器：可以自己编写估计器类，也可以使用 Skorch（https://skorch.readthedocs.io）等包装库，然后像第 2 章一样使用 `GridSearchCV` 或 `RandomizedSearchCV` 微调超参数。不过，使用 Optuna（https://optuna.org）、Ray Tune（https://docs.ray.io）或 Hyperopt（https://hyperopt.github.io/hyperopt）等专门的微调库通常能得到更好的结果。这些库提供了几种强大的调参策略，并且高度可定制。

下面看一个使用 Optuna 的例子。Colab 没有预装 Optuna，因此需要使用**%pip install optuna** 安装（如果希望在本地运行代码，请按照 https://homl.info/install-p 中的安装说明操作）。这里调节学习率和隐藏层中的神经元数量（为简单起见，两个隐藏层使用相同的神经元数量）。首先定义一个函数，Optuna 会多次调用它来执行超参数调整：这个函数必须接收一个 **Trial** 对象，并利用它向 Optuna 请求超参数值，然后使用这些值构建和训练模型。最后，函数必须评估模型（通常在验证集上进行）并返回指标：

```
import optuna
def objective(trial):
    learning_rate = trial.suggest_float("learning_rate", 1e-5, 1e-1, log=True)
    n_hidden = trial.suggest_int("n_hidden", 20, 300)
    model = ImageClassifier(n_inputs=1 * 28 * 28, n_hidden1=n_hidden,
                            n_hidden2=n_hidden, n_classes=10).to(device)
    optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
    [...]  # train the model, then evaluate it on the validation set
    return validation_accuracy
```

suggest_float() 和 **suggest_int()** 方法让我们可以请求给定范围内的超参数值（Optuna 还提供 **suggest_categorical()** 方法）。对于 **learning_rate** 超参数，请求范围为 10^{-5} 到 10^{-1} 的值。由于不知道最优值处于哪个数量级，因此添加 **log=True**：这会让 Optuna 从对数分布中采样，探索所有可能的数量级。如果使用默认的均匀分布，Optuna 就很不可能探索很小的取值。

要开始调整超参数，创建一个 **Study** 对象，并调用它的 **optimize()** 方法，传入刚刚定义的目标函数，以及要运行的试验次数（即 Optuna 调用目标函数的次数）。由于目标函数返回的是分数，分数越高越好，因此创建 study 时设置 **direction="maximize"**（默认情况下，Optuna 尝试最小化目标）。为了确保结果可复现，同时设置 PyTorch 的随机种子和 Optuna 采样器使用的随机种子：

```
torch.manual_seed(42)
sampler = optuna.samplers.TPESampler(seed=42)
study = optuna.create_study(direction="maximize", sampler=sampler)
study.optimize(objective, n_trials=5)
```

默认情况下，Optuna 使用 Tree-structured Parzen Estimator（TPE，树结构 Parzen 估计器）算法优化超参数。这是一种顺序的基于模型的优化算法，意思是它会从过去的结果中学习，以便更好地选择有希望的超参数。换句话说，Optuna 先从随机的超参数值开始，然后逐渐把搜索集中到超参数空间中最有希望的区域。因此，在相同时间内，Optuna 往往能找到比随机搜索更好的超参数。

> 提示：可以把更多超参数加入搜索空间，例如批量大小、优化器类型、隐藏层数量或激活函数类型。但要记住，随着超参数数量增加，搜索空间会呈指数增长，因此要确保额外的搜索时间和计算量值得投入。

调整完成后，可以查看它找到的最佳超参数及相应的验证准确率：

```
>>> study.best_params
{'learning_rate': 0.08525846269447772, 'n_hidden': 116}
```

```
>>> study.best_value
0.8867999911308289
```

这比前面得到的表现稍好。如果把 **n_trials** 增加到 50 或更多，结果会好得多，但当然需要运行数小时。也可以反复运行 **optimize()**，直到对模型表现满意为止。

Optuna 还可以在多台机器上并行运行试验，带来近似线性的加速。为此，需要设置一个 SQL 数据库（例如 SQLite 或 PostgreSQL），并把 **create_study()** 函数的 **storage** 参数设置为指向该数据库。还需要通过 **study_name** 参数设置 study 的名称，并设置 **load_if_exists=True**。之后可以把超参数调整脚本复制到多台机器上，在每台机器上运行（如果使用随机种子，确保每台机器使用不同的种子）。脚本会并行运行，并从数据库读取和写入试验结果。这样做还可以完整记录所有实验结果。

你可能注意到，我们假设 **objective()** 函数可以直接访问训练集和验证集，可能是通过全局变量访问。在一般情况下，把它们作为额外参数传递给 **objective()** 函数更清晰，例如：

```
def objective(trial, train_loader, valid_loader):
    [...]  # the rest of the function remains the same as above
objective_with_data = lambda trial: objective(
    trial, train_loader=train_loader, valid_loader=valid_loader)
study.optimize(objective_with_data, n_trials=5)
```

需要设置额外参数（本例中是数据集加载器）时，只需在需要时创建一个 lambda 函数，并将其传给 **optimize()** 方法。或者，也可以使用 **functools.partial()** 函数。它会围绕给定可调用对象创建一个轻量包装函数，为任意数量的参数提供默认值：

```
from functools import partial
objective_with_data = partial(objective, train_loader=train_loader,
                              valid_loader=valid_loader)
```

通常可以很快判断某个试验绝对糟糕，例如损失在第一个 epoch 期间突然升高，或者模型在最初几个 epoch 中几乎没有改善。此时，最好尽早中断训练，以免浪费时间和计算资源。可以直接返回模型当前的验证准确率，并希望 Optuna 学会避开超参数空间的这一部分；也可以通过抛出 **optuna.TrialPruned** 异常中断训练，这会告诉 Optuna 完全忽略该试验。在很多情况下，这会使搜索更加高效，因为可以避免用大量有噪声的模型评估结果污染 Optuna 的搜索算法。

Optuna 提供了几种能够检测并剪枝糟糕试验的 Pruner 类。例如，**MedianPruner** 会在训练过程中定期剪枝表现低于中位数的试验。它会在指定数量的试验完成后开始剪枝，这个数量由 **n_startup_trials** 控制（默认是 5）；对于此后的每个试验，它会先让训练运行若干 epoch，这由 **n_warmup_steps** 控制（默认是 0）；然后每隔若干 epoch（由 **interval_steps** 控制），检查模型在当前 epoch 的表现是否优于过去试验在相同 epoch 的中位数表现。要使用这个剪枝器，创建一个实例并把它传给 **create_study()** 方法：

```
pruner = optuna.pruners.MedianPruner(n_startup_trials=5, n_warmup_steps=0,
                                     interval_steps=1)
study = optuna.create_study(direction="maximize", sampler=sampler,
                            pruner=pruner)
```

然后在 **objective()** 函数中添加下面的代码，使其在每个 epoch 后运行：

```
for epoch in range(n_epochs):
    [...]  # train the model for one epoch
    validation_accuracy = [...]  # evaluate the model's validation accuracy
    trial.report(validation_accuracy, epoch)
```

```
    if trial.should_prune():
        raise optuna.TrialPruned()
```

`report()` 方法把当前验证准确率和 epoch 告诉 Optuna，这样它就能判断是否应该剪枝。如果 `trial.should_prune()` 返回 `True`，就抛出 `TrialPruned` 异常。

> 提示：Optuna 还有许多值得探索的功能，例如可视化工具、试验结果和其他工件的持久化工具、用于人机协同优化的仪表板，以及更多超参数搜索和试验剪枝算法。

对超参数满意后，可以在完整训练集（即训练集加验证集）上训练模型，然后在测试集上评估。希望模型表现出色！如果确实如此，就会希望保存模型，在生产环境中加载并使用它。这是本章最后一个主题。

10.7　保存和加载 PyTorch 模型（Saving and Loading PyTorch Models）

保存 PyTorch 模型最简单的方法是使用 `torch.save()` 方法，把模型和文件路径传给它。模型对象会使用 Python 的 pickle 模块序列化（pickle 可以把对象转换为字节序列），然后压缩（zip）并保存到磁盘。PyTorch 文件通常使用 `.pt` 或 `.pth` 扩展名：

```
torch.save(model, "my_fashion_mnist.pt")
```

很简单！现在可以同样轻松地加载模型（例如在生产代码中）：

```
loaded_model = torch.load("my_fashion_mnist.pt", weights_only=False)
```

> 警告：如果模型使用了自定义函数或类（例如 `ImageClassifier`），`torch.save()` 只会保存对它们的引用，而不会保存代码本身。因此，在调用 `torch.load()` 之

> 前，必须确保 Python 环境已经加载了自定义代码。还要使用相同版本的代码，以避免不匹配问题。

设置 **weights_only=False** 可以确保加载完整的模型对象，而不仅仅是模型参数。然后就可以使用加载的模型进行推理。不要忘记先使用 **eval()** 方法切换到评估模式：

```
loaded_model.eval()
y_pred_logits = loaded_model(X_new)
```

这种方法简单方便，但遗憾的是有一些非常严重的缺点。

第一，pickle 的序列化格式出了名的不安全。虽然 **torch.save()** 本身不保存自定义代码，但 pickle 格式支持自定义代码，因此黑客可能向保存的 PyTorch 模型中注入恶意代码；pickle 模块加载模型时会自动运行这些代码。因此，始终要确保完全信任模型的来源，再使用这种方式加载模型。

第二，pickle 有些脆弱。它可能因 Python 版本而变化（例如 Python 3.7 和 3.8 之间就有重大变化），并且会保存用于定位代码的具体文件路径；如果加载环境的文件夹结构不同，这些路径就可能失效。

为了避免这些问题，建议只保存和加载模型权重，而不是完整的模型对象：

```
torch.save(model.state_dict(), "my_fashion_mnist_weights.pt")
```

state_dict() 方法返回的状态字典只是一个 Python **OrderedDict**，其中为 **named_parameters()** 方法返回的每个参数保存一个条目。如果模型有缓冲区，它还会包含缓冲区：缓冲区就是通过 **register_buffer()** 方法注册到模型（或其子模块）的普通张量。缓冲区保存需要与模型一起存储的额外数据，但它不是模型参数。第 11 章会看到使用 batch norm 层的例子。

要加载这些权重，必须先创建结构完全相同的模型，然后使用 **weights_only=True** 调用 **torch.load()**，最后对加载的权重调用模型的 **load_state_dict()** 方法：

```
new_model = ImageClassifier(n_inputs=1 * 28 * 28, n_hidden1=300, n_hidden2=100,
                            n_classes=10)
loaded_weights = torch.load("my_fashion_mnist_weights.pt", weights_only=True)
new_model.load_state_dict(loaded_weights)
new_model.eval()
```

保存的模型只包含数据，`load()` 函数会确保这一点，因此这种方式是安全的，也不太容易因 Python 版本变化而失效或造成部署问题。不过，只有能够在加载状态字典前创建完全相同的模型架构时，这种方式才有效。为此，需要知道层数、每层神经元数量等信息。最好把这些信息与状态字典一起保存：

```
model_data = {
    "model_state_dict": model.state_dict(),
    "model_hyperparameters": {"n_inputs": 1 * 28 * 28, "n_hidden1": 300, [...]}
}
torch.save(model_data, "my_fashion_mnist_model.pt")
```

然后可以根据保存的超参数构建模型，并把状态字典加载到模型中：

```
loaded_data = torch.load("my_fashion_mnist_model.pt", weights_only=True)
new_model = ImageClassifier(**loaded_data["model_hyperparameters"])
new_model.load_state_dict(loaded_data["model_state_dict"])
new_model.eval()
```

如果希望从中断处继续训练，还需要保存优化器的状态字典、优化器超参数，以及可能需要的其他训练信息，例如当前 epoch 和损失历史。

> 提示：Hugging Face 的 safetensors 库是另一种流行且安全的模型权重保存方式。

还有另一种保存和加载模型的方式：先把模型转换为 TorchScript。这样还可以加快模型推理速度。

10.8　编译和优化 PyTorch 模型（Compiling and Optimizing a PyTorch Model）

PyTorch 有一个很棒的功能：它可以自动把模型代码转换为 TorchScript，可以把 TorchScript 理解为 Python 的一个静态类型子集。这样做有两个主要好处：

第一，TorchScript 代码可以编译和优化，从而生成速度显著更快的模型。例如，多个运算经常可以融合为一个更高效的运算；常量上的运算（例如 2 × 3）可以替换为运算结果（例如 6），这称为常量折叠。还可以剪除未使用的代码，等等。

第二，TorchScript 可以序列化、保存到磁盘，然后在 Python 环境或 C++ 环境中使用 LibTorch 库加载和执行。这使 PyTorch 模型能够运行在各种设备上，包括嵌入式设备。

把 PyTorch 模型转换为 TorchScript 有两种方式。第一种称为 tracing（跟踪）。PyTorch 使用样本数据运行模型，记录每个发生的运算，然后把这份记录转换为 TorchScript。使用 **`torch.jit.trace()`** 函数完成：

```
torchscript_model = torch.jit.trace(model, X_new)
```

这种方法通常适用于 **`forward()`** 方法不使用条件分支或循环的静态模型。但如果尝试跟踪包含 **`if`** 或 **`match`** 语句的模型，那么只有实际执行到的分支会被 TorchScript 捕获，这通常不是想要的结果。同样，如果使用跟踪处理包含循环的模型，TorchScript 代码会为实际执行的每次循环迭代复制一份循环内部的运算，这也通常不是想要的结果。

对于这类动态模型，可能需要尝试另一种名为 scripting（脚本化）的方法。在这种情况下，PyTorch 会直接解析 Python 代码并把它转换为 TorchScript。这种方法可以正确支持 **`if`** 语句和 **`while`** 循环，前提是条件必须是张量；遍历张量时也支持 **`for`** 循环。不过，它只适用于 Python 的一个子集。例如，不能使用全局变量、Python 生成器（**`yield`**）、复杂的列表推导式、可变长度的函数参数（**`*args`** 或 **`**kwargs`**）或 **`match`** 语句。此外，类型必须固定（某些情况下，函数不能在一处返回整数、另一处返回浮点数），而且只能调用同样遵循这些规则的其他函数，因此不能使用标准库或第三方库等（完整限制请参阅文档）。听起来可能很吓人，但对于大多数现实模型来说，这些规则实际上并不难遵守。可以这样保存模型：

```
torchscript_model = torch.jit.script(model)
```

无论使用跟踪还是脚本化生成 TorchScript 模型，都可以进一步优化：

```
optimized_model = torch.jit.optimize_for_inference(torchscript_model)
```

TorchScript 模型只能用于推理，不能用于训练，因为 TorchScript 环境不支持梯度跟踪或参数更新。

最后，使用 **`save()`** 方法保存 TorchScript 模型：

```
torchscript_model.save('my_fashion_mnist_torchscript.pt')
```

然后使用 `torch.jit.load()` 函数加载：

```
loaded_torchscript_model = torch.jit.load("my_fashion_mnist_torchscript.pt")
```

需要注意的是，TorchScript 已不再积极开发——仍会修复 bug，但不会加入新功能。它仍然运行良好，并且仍是让 PyTorch 模型在 C++ 环境中运行的最佳方式之一[13]。不过，自 PyTorch 2.0 于 2023 年 3 月发布以来，PyTorch 团队一直把精力集中在一套围绕 `torch.compile()` 函数的新编译工具上，使用它非常容易：

```
compiled_model = torch.compile(model)
```

现在可以像平常一样使用得到的模型；当调用模型时，PyTorch 会自动对它进行编译和优化。这称为即时（Just-In-Time，JIT）编译，与提前（Ahead-Of-Time，AOT）编译相对。

在底层，`torch.compile()` 依赖 TorchDynamo（简称 Dynamo）。它直接挂接到 Python 字节码上，在推理时捕获模型的计算图。访问字节码后，Dynamo 可以高效且可靠地捕获计算图，正确处理条件分支和循环，同时利用动态信息进一步优化模型。实际的编译和优化默认由名为 TorchInductor 的后端组件执行；TorchInductor 又依赖 Triton 语言生成高效的 GPU 代码（仅限 Nvidia），或使用 OpenMP API 进行 CPU 优化。PyTorch 2.x 还提供了其他优化后端，包括面向 Google TPU 设备的 XLA 后端：调用 `torch.compile()` 时只需设置 `device="xla"`。

现在，你已经拥有开始构建和训练复杂、高效神经网络所需的全部工具。希望你喜欢这次 PyTorch 入门！我们覆盖了很多内容，但冒险才刚刚开始：下一章将讨论训练非常深的网络的技术。之后会深入学习其他流行的神经网络架构：用于图像处理的卷积神经网络、用于序列数据的循环神经网络、用于文本（以及更多数据）的 Transformer、用于表示学习的自编码器，以及用于生成数据的生成对抗网络和扩散模型[14]。然后将学习如何通过强化学习训练自主智能体，最后还会进一步了解如何部署和优化 PyTorch 模型。开始吧！

[13] 另一种流行选择是使用 `torch.onnx.export()` 把 PyTorch 模型导出为开放的 ONNX 标准。ONNX 模型随后可以在各种环境中用于推理。

[14] 在线 notebook 中还介绍了几种额外的 ANN 架构：https://homl.info/extra-anns。

10.9　练习（Exercises）

1. PyTorch 在很多方面类似于 NumPy，但它提供了一些额外功能。你能说出其中最重要的功能吗？

2. `torch.exp()` 和 `torch.exp_()` 有什么区别？`torch.relu()` 和 `torch.relu_()` 又有什么区别？

3. 在 GPU 上创建新张量有哪两种方法？

4. 不使用 autograd，有哪三种方法可以执行张量计算？

5. 下面的代码会导致 `RuntimeError` 吗？如果把第二行换成 `z = t.cos_().exp()`，结果如何？如果换成 `z = t.exp().cos_()` 呢？

```
t = torch.tensor(2.0, requires_grad=True)
z = t.cos().exp_()
z.backward()
```

下面这段代码会报错吗？如果把第三行换成 `w = v.cos_() * v.sin()`，结果如何？两种情况下 `w` 的值相同吗？

```
u = torch.tensor(2.0, requires_grad=True)
v = u + 1
w = v.cos() * v.sin_()
w.backward()
```

6. 假设创建一个 `Linear(100, 200)` 模块。它有多少个神经元？权重和偏置参数的形状是什么？它期望的输入形状是什么？产生的输出形状是什么？

7. PyTorch 训练循环的主要步骤是什么？

8. 为什么建议在把模型移动到 GPU 后再创建优化器？

9. 使用 GPU 时，通常应该设置 `DataLoader` 的哪些选项来加快训练？

10. PyTorch 提供了哪些主要分类损失？应该在什么时候使用每一种损失？

11. 为什么训练前调用 `model.train()`、评估前调用 `model.eval()` 很重要？

12. `torch.jit.trace()` 和 `torch.jit.script()` 有什么区别？

13. 使用 autograd 求函数 $f(x, y) = \sin(x^2 y)$ 在点 $(x, y) = (1.2, 3.4)$ 处的梯度向量。

14. 创建一个自定义 `Dense` 模块，使其复现 `nn.Linear` 模块后接 `nn.ReLU` 模块的功能。先尝试使用 `nn.Linear` 和 `nn.ReLU` 模块实现，然后再使用 `nn.Parameter` 和 `relu()` 函数重新实现。

15. 在 Cover Type 数据集上构建并训练一个分类 MLP：

    a. 使用 `sklearn.datasets.fetch_covtype()` 加载数据集，并为这些数据创建一个自定义 PyTorch `Dataset`。

    b. 创建训练、验证和测试数据加载器。

    c. 构建一个自定义 MLP 模块来处理这个分类任务。可以选择使用上一道练习中的自定义 `Dense` 模块。

    d. 在 GPU 上训练模型，尝试在测试集上达到 93% 的准确率。为此，可能需要进行超参数搜索，找到合适的层数、每层神经元数、学习率、批量大小等。也可以选择使用 Optuna。

这些练习的答案位于本章 notebook 的末尾，网址为：https://homl.info/colab-p。

第 11 章　训练深度神经网络

Training Deep Neural Networks

在第 10 章中，你使用 PyTorch 构建、训练和微调了几个人工神经网络。不过，那些网络都比较浅，只有少数几个隐藏层。如果要解决复杂问题，例如在高分辨率图像中检测数百种物体，该怎么办？你可能需要训练更深的人工神经网络（artificial neural network，ANN），也许包含几十层甚至上百层，每层有数百个神经元，并由成千上万条连接相互链接。训练深度神经网络（deep neural network，DNN）并不轻松，可能会遇到以下问题：

- 训练过程中，梯度沿着网络反向传播时可能越来越小，也可能越来越大。这两种情况都会让较低层非常难以训练。
- 如此庞大的网络可能没有足够的训练数据，或者数据标注成本过高。
- 训练过程可能极其缓慢。
- 一个包含数百万参数的模型很容易严重过拟合训练集，尤其是在训练实例不足或数据噪声较大时。

本章将逐一讨论这些问题，并介绍各种解决技术。首先，我们会研究梯度消失和梯度爆炸问题，以及其中最常用的解决方法，包括聪明的权重初始化、更好的激活函数、批归一化、层归一化和梯度裁剪。接着，我们会讨论迁移学习和无监督预训练；即使标注数据很少，它们也能帮助你处理复杂任务。然后，我们将介绍各种可以显著加快大型模型训练速度的优化器，并讨论如何在训练过程中调整学习率，以加速训练并得到更好的模型。最后，我们会介绍几种大型神经网络常用的正则化技术：ℓ1 和 ℓ2 正则化、dropout、Monte Carlo dropout 以及 max-norm 正则化。

掌握这些工具后，你就能训练各种深度网络。欢迎来到深度学习！

11.1　梯度消失/爆炸问题（The Vanishing/Exploding Gradients Problems）

正如第 9 章所述，反向传播算法的第二阶段从输出层开始向输入层推进，沿途传播误差梯度。算法计算出损失函数相对于网络中每个参数的梯度后，就使用这些梯度执行梯度下降步骤，更新每个参数。

遗憾的是，随着算法向较低层推进，梯度往往会越来越小。于是，梯度下降对低层连接权重的更新几乎停滞，训练始终无法收敛到良好解。这称为**梯度消失问题（vanishing gradients problem）**。有时也会出现相反的情况：梯度越来越大，直到各层获得大得离谱的权重更新，算法最终发散。这称为**梯度爆炸问题（exploding gradients problem）**，在循环神经网络（recurrent neural network，RNN；见第 13 章）中最为常见。更广泛地说，深度神经网络容易出现不稳定梯度：不同层的学习速度可能相差很大。

这种不幸的行为早已被经验观察到，也是深度神经网络在 21 世纪初基本沉寂的原因之一。当时人们还不清楚训练 DNN 时梯度为何如此不稳定。2010 年，Xavier Glorot 和 Yoshua Bengio 的论文为这个问题带来了一些线索。他们指出，流行的 sigmoid（logistic）激活函数与当时最常用的权重初始化方法（均值为 0、标准差为 1 的正态分布）组合起来，可能是问题的一部分。简而言之，他们证明了：使用这种激活函数和初始化方案时，每层输出的方差都远大于输入方差。沿网络向前传播时，方差在每层之后不断增大，直到顶层的激活函数饱和。sigmoid 函数的均值为 0.5 而不是 0，这一点还会进一步加重饱和问题；双曲正切函数的均值为 0，因此在深层网络中的表现略好于 sigmoid。

观察 sigmoid 激活函数（见图 11-1）可以发现，当输入变得很大（无论为负还是为正）时，函数会在 0 或 1 附近饱和，导数极其接近 0，也就是说曲线在两端几乎是平的。因此，当反向传播开始时，几乎没有梯度可以沿网络向后传播；而仅存的少量梯度又会在穿过顶层时不断被稀释，于是传到低层时几乎什么都不剩了。

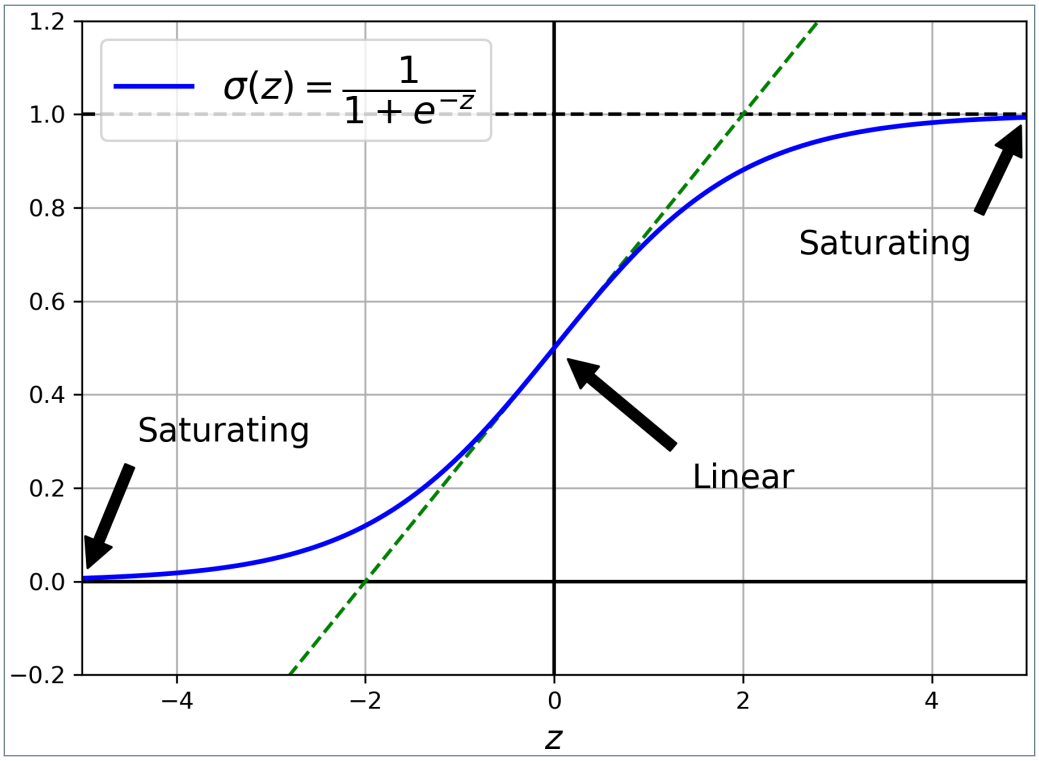

图 11-1: Sigmoid 激活函数的饱和（Sigmoid activation function saturation）

11.1.1　Glorot 初始化与 He 初始化（Glorot Initialization and He Initialization）

在论文中，Glorot 和 Bengio 提出了一种可以显著缓解梯度不稳定问题的方法。他们指出，信号必须在两个方向上正常流动：在进行预测时沿前向传播，在反向传播梯度时沿反向传播。我们不希望信号消失，也不希望信号爆炸并导致饱和。为了让信号正常流动，作者认为每层输出的方差应该等于该层输入的方差；在反向传播方向上，梯度流经某层之前和之后也应该具有相同的方差。除非该层输入和输出数量相等（这两个数量分别称为该层的 fan-in 和 fan-out），否则无法同时保证这两个条件。Glorot 和 Bengio 提出了一个很好的折中方案，并且在实践中效果很好：每层的连接权重应按照公式 11-1 所示的方式随机初始化，其中 `fan_avg = (fan_in + fan_out) / 2`。这种初始化策略称为 Xavier 初始化或 Glorot 初始化，以论文第一作者命名。

$$\mathcal{N}\left(0, \frac{1}{fan_{avg}}\right) \quad \text{or} \quad U\left(-\sqrt{\frac{3}{fan_{avg}}}, +\sqrt{\frac{3}{fan_{avg}}}\right) \qquad (11\text{-}1)$$

公式 11-1　Glorot 初始化（使用 sigmoid 激活函数时）（Glorot initialization when using the sigmoid activation function）

如果把公式 11-1 中的 **fan_avg** 换成 **fan_in**，就得到 Yann Le Cun 在 20 世纪 90 年代提出的初始化策略，他称之为 Le Cun 初始化。Genevieve Orr 和 Klaus-Robert Müller 甚至在 1998 年出版的《神经网络：技巧与诀窍》（Springer）中推荐了它。当 **fan_in = fan_out** 时，Le Cun 初始化与 Glorot 初始化等价。研究人员花了十多年才意识到这个技巧的重要性。使用 Glorot 初始化可以显著加快训练速度，它也是推动深度学习成功的技巧之一。

一些论文针对不同激活函数提出了类似策略。最著名的是 Kaiming He 等人在 2015 年提出的方法。这些策略的差异只在于方差的缩放因子，以及使用 **fan_avg** 还是 **fan_in**，如表 11-1 所示（对于均匀分布，只需令 **r = √(3σ²)**）。针对 ReLU 激活函数及其变体的初始化策略称为 He 初始化或 Kaiming 初始化，以论文第一作者命名。对于 SELU，应使用 Yann Le Cun 的初始化方法，最好使用正态分布。稍后会介绍这些激活函数。

表 11-1: 不同激活函数的初始化参数

初始化方法	激活函数	σ²（正态分布）
Xavier/Glorot	无、tanh、sigmoid、softmax	1 / fan_avg
Kaiming/He	ReLU、Leaky ReLU、ELU、GELU、Swish、Mish、SwiGLU、ReLU2	2 / fan_in
Yann Le Cun	SELU	1 / fan_in

由于历史原因，PyTorch 的 **nn.Linear** 模块使用 Kaiming 均匀初始化来初始化权重，但权重还会额外缩小 **√6** 倍（偏置项也会随机初始化）。遗憾的是，对于常见激活函数，这个缩放比例都不是最优的。一种解决方法是在创建 **nn.Linear** 层后立即将权重乘以 **√6**（对于 Le Cun 初始化则乘以 **√3**），从而得到正确的 Kaiming 初始化。同时我们会把偏置设为 0，因为随机初始化偏置没有收益：

```
import torch
import torch.nn as nn
layer = nn.Linear(40, 10)
layer.weight.data *= 6 ** 0.5  # Kaiming init (or 3 ** 0.5 for LeCun init)
torch.zero_(layer.bias.data)
```

虽然这种方法可行，但使用 **torch.nn.init** 模块中的初始化函数更加清晰，也更不容易出错：

```
nn.init.kaiming_uniform_(layer.weight)
nn.init.zeros_(layer.bias)
```

如果希望对模型中的每个 `nn.Linear` 层使用相同的初始化方法，可以在模型构造器中创建每个 `nn.Linear` 层后进行初始化。也可以编写 `nn.Linear` 的子类，修改其构造器来按所需方式初始化权重。不过，一个更简单的选择是编写一个小函数：它接收一个模块，检查该模块是否为 `nn.Linear` 实例；如果是，就对其权重应用目标初始化函数。然后将该函数传给模型的 `apply()` 方法，就可以递归应用到模型及其所有子模块：

```
def use_he_init(module):
    if isinstance(module, nn.Linear):
        nn.init.kaiming_uniform_(module.weight)
        nn.init.zeros_(module.bias)

model = nn.Sequential(nn.Linear(50, 40), nn.ReLU(), nn.Linear(40, 1), nn.ReLU())
model.apply(use_he_init)
```

`torch.nn.init` 模块还包含 `orthogonal_()` 函数。它使用随机正交矩阵初始化权重，这是 Andrew Saxe 等人在 2014 年提出的方案。正交矩阵具有一些有用的数学性质，包括保持范数：给定正交矩阵 `W` 和输入向量 `x`，`Wx` 的范数等于 `x` 的范数，因此输入的幅度会在输出中得到保留。当输入经过标准化时，这会让通过该层的方差保持稳定，从而防止深层网络中的激活值和梯度消失或爆炸，至少在训练开始时如此。这种初始化方法不像前面讨论的方法那样常见，但在循环神经网络（第 13 章）或生成对抗网络（第 18 章）中可能很有效。

到这里就完成了。正确缩放权重，可以为深度网络提供更好的训练起点。

> 提示：在分类器中，通常最好在初始化时缩小输出层的权重（例如缩小 10 倍）。这样会让训练开始时的 logits 更小、更接近彼此，估计出的概率也会更加接近。换句话说，它会让模型在训练初期对预测不那么自信，从而避免极端损失和巨大梯度，防止模型权重在训练开始时随机来回摆动。

11.2　更好的激活函数（Better Activation Functions）

Glorot 和 Bengio 在 2010 年论文中的一个重要洞察是：梯度不稳定问题部分源于激活函数选择不当。在那之前，人们大多认为，既然大自然在生物神经元中选择

了与 sigmoid 激活函数非常相近的函数，那么它必然是很好的选择。但事实证明，其他激活函数在深度神经网络中的表现更好，尤其是 ReLU 激活函数：它在正值区域不会饱和，而且计算速度非常快。

遗憾的是，ReLU 激活函数并不完美。它有一个称为“ReLU 死亡（dying ReLUs）”的问题：在训练过程中，一些神经元会有效地“死亡”，也就是除了 0 以外不再输出任何值。有时，网络中甚至一半的神经元都会死亡，尤其是在学习率较大的情况下。当神经元的权重被调整到某种程度，使 ReLU 的输入（神经元输入的加权和加上偏置项）对训练集中的所有实例都为负时，该神经元就会死亡。此后它只会持续输出 0，因为当输入为负时 ReLU 的梯度为 0，梯度下降也就无法再影响它。[1]

为了解决这个问题，可以使用 ReLU 的变体，例如 Leaky ReLU。

11.2.1　Leaky ReLU

Leaky ReLU 激活函数定义为 **`LeakyReLU_α(z) = max(αz, z)`**（见图 11-2）。超参数 α 决定函数“泄漏”多少：它是 **`z < 0`** 时函数的斜率。负值区域存在斜率，确保 Leaky ReLU 不会真正死亡；它们可能进入很长的“昏迷”，但仍有机会最终醒来。Bing Xu 等人在 2015 年的论文中比较了几种 ReLU 激活函数变体，其中一个结论是：Leaky 变体始终优于严格的 ReLU。事实上，设置 **`α = 0.2`**（很大的泄漏）似乎比设置 **`α = 0.01`**（较小的泄漏）带来更好的表现。

该论文还评估了随机 Leaky ReLU（randomized leaky ReLU，RReLU）：训练期间在给定范围内随机选择 α，测试期间固定为平均值。RReLU 的表现也相当好，而且似乎具有正则化作用，可以降低过拟合风险。最后，论文评估了参数化 Leaky ReLU（parametric leaky ReLU，PReLU）：α 不再是超参数，而是可以像其他参数一样通过反向传播学习和修改的参数。报告显示，PReLU 在大型图像数据集上明显优于 ReLU，但在较小数据集上可能有过拟合风险。

[1] 如果神经元的输入随时间发生变化，最终重新回到 ReLU 能接收正输入的范围，死亡的神经元也可能重新恢复。例如，梯度下降可能调整死亡神经元下方各层的参数，使其输入最终回到有效范围。

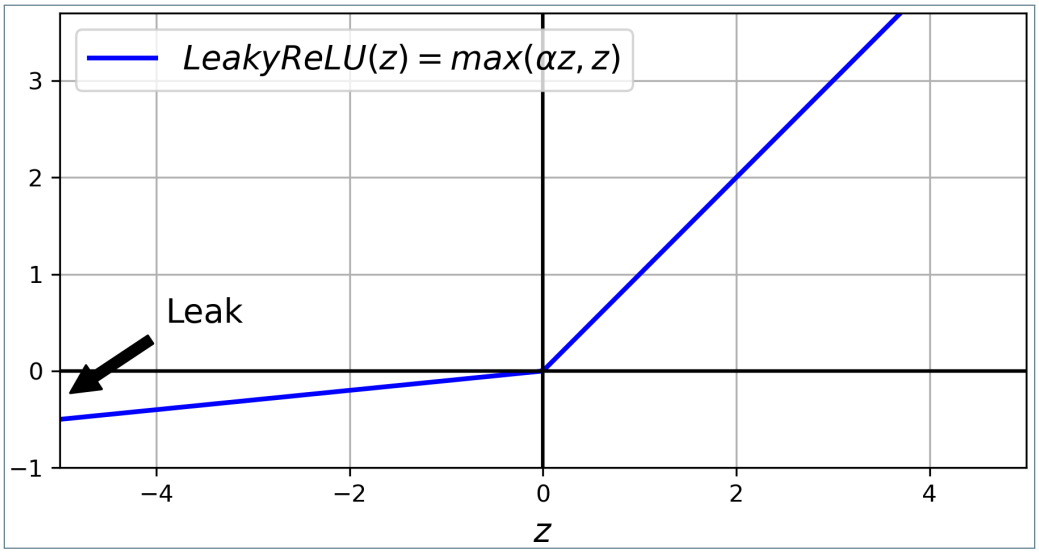

图 11-2: Leaky ReLU：类似 ReLU，但在负值区域具有小斜率（Leaky ReLU: like ReLU, but with a small slope for negative values）

正如预期，PyTorch 为这些激活函数提供了模块：`nn.LeakyReLU`、`nn.RReLU` 和 `nn.PReLU`。和其他 ReLU 变体一样，应将它们与 Kaiming 初始化配合使用；但由于存在负斜率，方差应略小一些，缩小因子为 `1 + α²`。PyTorch 支持这一点：向 `kaiming_uniform_()` 和 `kaiming_normal_()` 传入 α，并设置 `nonlinearity="leaky_relu"`，即可得到经过适当调整的 Kaiming 初始化：

```
alpha = 0.2
model = nn.Sequential(nn.Linear(50, 40), nn.LeakyReLU(negative_slope=alpha))
nn.init.kaiming_uniform_(model[0].weight, alpha, nonlinearity="leaky_relu")
```

ReLU、Leaky ReLU 和 PReLU 都有一个共同问题：它们不是光滑函数，斜率会在 `z = 0` 处突然变化。正如第 4 章讨论套索回归时所见，这种导数不连续会使梯度下降在最优点附近来回弹跳，减慢收敛。因此，下面来看一些光滑的 ReLU 变体，先从 ELU 和 SELU 开始。

11.2.2　ELU 和 SELU

2015 年，Djork-Arné Clevert 等人提出了一种称为指数线性单元（exponential linear unit，ELU）的新激活函数。在作者的实验中，它优于所有 ReLU 变体：训练时间更短，神经网络在测试集上的表现也更好。公式 11-2 给出了该激活函数的定义。

$$\text{ELU}_\alpha(z) = \begin{cases} \alpha\left(\exp(z) - 1\right) & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases} \tag{11-2}$$

公式 11-2　ELU 激活函数（ELU activation function）

ELU 激活函数看起来很像 ReLU（见图 11-3），但有几个重要差异：

- 当 `z < 0` 时，它取负值，使单元输出的平均值更接近 0，有助于缓解梯度消失问题。超参数 α 定义了当 `z` 为很大的负数时，ELU 函数趋近于哪个值的相反数。α 通常设为 1，但也可以像其他超参数一样调整。
- 当 `z < 0` 时，它的梯度不为 0，因此可以避免神经元死亡问题。
- 当 α 等于 1 时，函数处处光滑，包括 `z = 0` 附近。这有助于加速梯度下降，因为梯度不会在 0 的左右来回弹跳。

在 PyTorch 中使用 ELU 很简单：使用 `nn.ELU` 模块，并配合 Kaiming 初始化即可。ELU 的主要缺点是计算速度比 ReLU 及其变体更慢，因为它需要计算指数函数。训练期间更快的收敛速度可能会抵消这部分计算开销，但在测试阶段，ELU 网络仍会比 ReLU 网络稍慢。

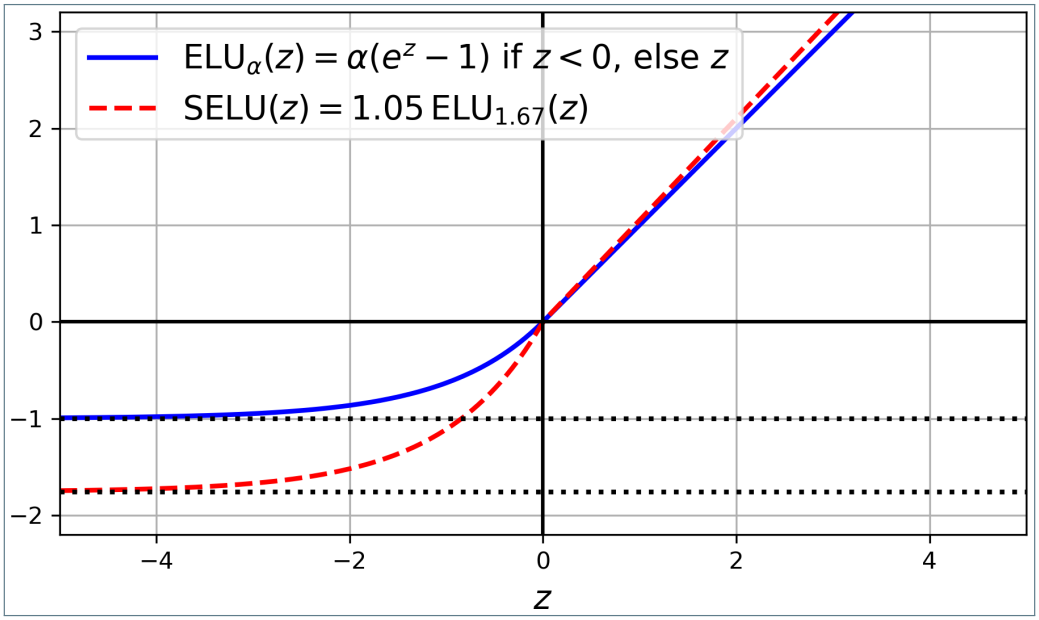

图 11-3: ELU 和 SELU 激活函数（ELU and SELU activation functions）

不久之后，Günter Klambauer 等人在 2017 年的论文中提出了缩放指数线性单元（scaled ELU，SELU）激活函数。顾名思义，它是 ELU 的缩放变体（约为 ELU 的 1.05 倍，使用 `α ≈ 1.67`）。作者证明，如果构建一个完全由全连接层（也就是 MLP）堆叠而成的神经网络，并且所有隐藏层都使用 SELU 激活函数，那么网络将实现**自归一化（self-normalize）**：训练期间，每层输出趋向于保持均值为 0、标准差为 1，从而解决梯度消失/爆炸问题。因此，SELU 激活函数可能优于其他激活函数，尤其是在深层 MLP 中。要在 PyTorch 中使用它，只需使用 `nn.SELU`。不过，自归一化需要满足一些条件：

- 输入特征必须经过标准化，均值为 0、标准差为 1。
- 每个隐藏层的权重都必须使用 Le Cun 正态初始化。
- 自归一化性质只对普通 MLP 有保证。如果在循环网络或带跳跃连接的其他架构中使用 SELU（例如 Wide & Deep 神经网络），它很可能不会优于 ELU。
- 不能使用 $\ell 1$ 或 $\ell 2$ 正则化、batch norm、layer norm、max-norm 或普通 dropout；这些技术会在本章后面介绍。

这些限制相当严格。因此，尽管 SELU 很有前景，但它并没有获得太多应用。此外，其他激活函数在大多数任务中似乎始终表现得更好。下面看看一些最流行的激活函数。

11.2.3　GELU、Swish、SwiGLU、Mish 和 ReLU2

Gaussian Error Linear Unit（GELU，高斯误差线性单元）由 Dan Hendrycks 和 Kevin Gimpel 在 2016 年的一篇论文中提出。你可以再次把它看作 ReLU 激活函数的光滑变体。公式 11-3 给出了定义，其中 Φ 是标准高斯累积分布函数（cumulative distribution function，CDF）：Φ(z) 对应于从均值为 0、方差为 1 的正态分布中随机采样的值小于 `z` 的概率。

$$\text{GELU}(z) = z\Phi(z) \tag{11-3}$$

公式 11-3　GELU 激活函数（GELU activation function）

如图 11-4 所示，GELU 与 ReLU 相似：当输入 `z` 非常负时，它趋近于 0；当输入 `z` 非常正时，它趋近于 `z`。但是，到目前为止讨论的激活函数既是凸函数又是单调函数，而 GELU 两者都不是：从左向右看，它先直行，随后向下弯曲，在 `z ≈ -0.75` 附

近达到约 -0.17 的低点，最后反弹，沿右上方向直线前进。这种相对复杂的形状，以及每一点都具有曲率的性质，可能解释了它为何在复杂任务中表现良好：梯度下降也许更容易拟合复杂模式。在实践中，它经常优于前面讨论的其他激活函数。不过，它的计算成本也略高，性能提升并不总是足以抵消额外开销。可以证明，GELU 近似等于 `zσ(1.702z)`，其中 σ 是 sigmoid 函数；使用这个近似式同样效果很好，而且计算速度更快。

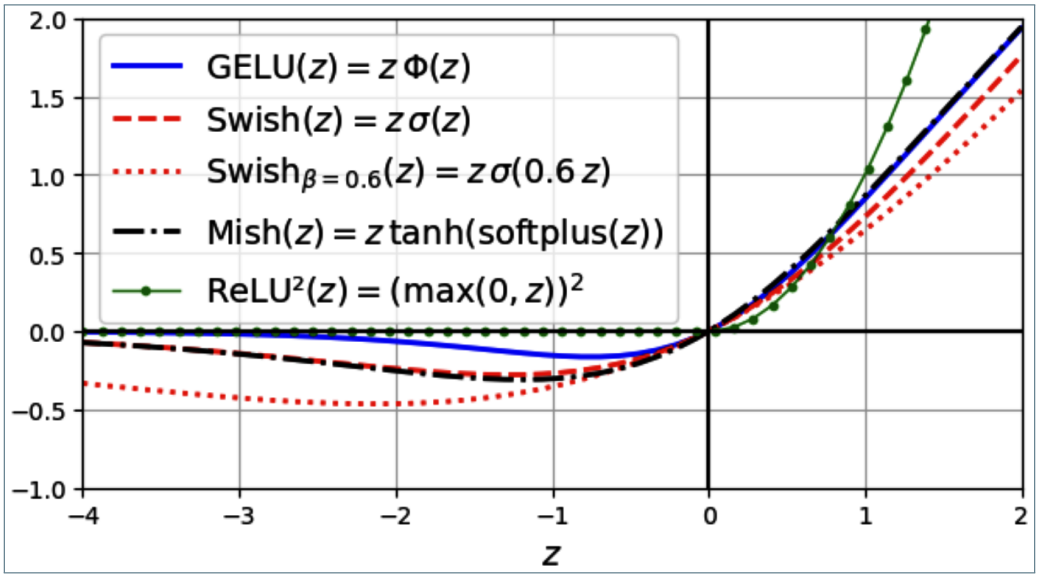

图 11-4: GELU、Swish、参数化 Swish、Mish 和 ReLU2 激活函数（GELU, Swish, parametrized Swish, Mish, and ReLU2 activation functions）

GELU 论文还介绍了 sigmoid linear unit（SiLU，sigmoid 线性单元）激活函数，它等于 `zσ(z)`，但在作者的测试中不如 GELU。很有意思的是，Prajit Ramachandran 等人在 2017 年通过自动搜索优秀激活函数，重新发现了 SiLU。他们将其命名为 Swish，这个名字后来流行开来。在他们的论文中，Swish 优于包括 GELU 在内的所有其他函数。

Ramachandran 等人后来为 Swish 增加了一个额外的标量超参数 β，用于缩放 sigmoid 函数的输入。广义 Swish 函数为 `Swish_β(z) = zσ(βz)`，因此 GELU 近似等于使用 `β = 1.702` 的广义 Swish。β 可以像其他超参数一样调节；也可以让 β 变成可训练参数，由梯度下降对其进行优化，类似 PReLU。通常整个模型只有一个可训练的 β，或者每层使用一个 β，以保持模型效率并避免过拟合。

一种流行的 Swish 变体是 SwiGLU：输入一方面通过 Swish 激活函数，另一方面并行通过线性层，然后将两个输出逐元素相乘。其定义为 `SwiGLU_beta(z) = Swish_`

`beta(z) * Linear(z)`，其中乘法是逐元素进行的。实际实现中，通常先将前一个线性层的输出维度加倍，再沿特征维度把输出拆成 `z1` 和 `z2`，最后计算 `SwiGLU_beta(z) = Swish_beta(z1) * z2`。这是 2016 年 Facebook 研究人员提出的门控线性单元（gated linear unit，GLU）的一种变体。逐元素乘法提高了模型的表达能力，使它能够学习何时关闭（乘以 0）或放大特定特征，这称为门控机制。SwiGLU 在现代 Transformer 中非常常见（见第 15 章）。

另一种类似 GELU 的激活函数是 Mish，由 Diganta Misra 在 2019 年提出。它定义为 `mish(z) = z tanh(softplus(z))`，其中 `softplus(z) = log(1 + exp(z))`。和 GELU、Swish 一样，它是 ReLU 的光滑、非凸、非单调变体。作者进行了大量实验，发现 Mish 通常优于其他激活函数，甚至以很小的幅度超过 Swish 和 GELU。图 11-4 展示了 GELU、Swish（默认 `β = 1` 和 `β = 0.6`）以及 Mish。可以看到，当 `z` 为负时，Mish 几乎与 Swish 重合；当 `z` 为正时，它又几乎与 GELU 重合。

最后，2021 年，Google 研究人员通过自动架构搜索改进大型 Transformer，发现了一种非常简单但有效的激活函数：ReLU2。顾名思义，它就是 ReLU 的平方：`ReLU2(z) = (max(0, z))²`。它拥有 ReLU 的所有优点（简单、计算效率高、输出稀疏、正值区域不饱和），同时在 `z = 0` 处具有平滑梯度，通常也能优于其他激活函数，特别是在稀疏模型中。不过，训练可能不太稳定，部分原因是它对离群点和死亡 ReLU 更敏感。

> 提示：那么，深度神经网络隐藏层应该使用哪个激活函数？对于大多数任务，ReLU 仍然是很好的默认选择：它通常和更复杂的激活函数一样好，计算速度却很快，而且许多库和硬件加速器都针对 ReLU 提供了专门优化。不过，对于复杂任务，Swish 可能是更好的默认选择；对于最复杂的任务，还可以尝试带可学习 β 的参数化 Swish。Mish 和 SwiGLU 可能带来略好的结果，但计算成本也更高。如果特别在意运行时延迟，可以选择 Leaky ReLU；对于复杂任务，可以选择参数化 Leaky ReLU；对于稀疏模型，也可以尝试 ReLU2。

PyTorch 开箱即支持 GELU、Mish 和 Swish，分别使用 `nn.GELU`、`nn.Mish` 和 `nn.SiLU`。要实现 SwiGLU，可以将前一个线性层的输出维度加倍，然后使用 `z1, z2 = z.chunk(2, dim=-1)` 将输出拆成两部分，再计算 `F.silu(beta * z1) * z2`（其中 `F` 是 `torch.nn.functional`）。对于 ReLU2，只需计算 `F.relu(z).square()`。PyTorch 还提供了若干简化和近似版本的激活函数，它们计算速度更快，训练时通常也更稳定。这些简化版本的名称以“Hard”开头，例如 `nn.Hardsigmoid`、`nn.Hardtanh` 和 `nn.Hardswish`，经常用于移动设备。

激活函数就介绍到这里。下面来看一种完全不同的稳定梯度方法：批归一化。

11.3　批归一化（Batch Normalization）

虽然在 ReLU（或其任意变体）上使用 Kaiming 初始化，可以在训练开始时显著降低梯度消失/爆炸的风险，但它并不能保证这些问题不会在训练过程中重新出现。

2015 年，Sergey Ioffe 和 Christian Szegedy 提出了一种称为批归一化（batch normalization，BN）的技术，用于解决这些问题。该技术在每个隐藏层的激活函数之前或之后向模型中加入一个操作。这个操作首先将每个输入进行零中心化和归一化，然后使用每层新增的两个参数向量对结果进行缩放和平移：一个用于缩放，一个用于平移。换句话说，它让模型学习每层输入的最佳尺度和均值。

很多情况下，如果把 BN 层作为神经网络的第一层，就不必再手动标准化训练集（不需要 `StandardScaler`）；BN 层会替你完成这项工作，不过只是近似完成，因为它一次只观察一个批次，而且还会重新缩放和平移每个输入特征。为了将输入零中心化并归一化，算法需要估计每个输入的均值和标准差。它通过计算当前小批量上的均值和标准差来完成，这也是“批归一化”名称的由来。公式 11-4 逐步总结了整个操作。

$$
\begin{aligned}
\boldsymbol{\mu}_B &= \frac{1}{m_B}\sum_{i=1}^{m_B}\mathbf{x}^{(i)} \\
\boldsymbol{\sigma}_B^2 &= \frac{1}{m_B}\sum_{i=1}^{m_B}\left(\mathbf{x}^{(i)} - \boldsymbol{\mu}_B\right)^2 \\
\hat{\mathbf{x}}^{(i)} &= \frac{\mathbf{x}^{(i)} - \boldsymbol{\mu}_B}{\sqrt{\boldsymbol{\sigma}_B^2 + \varepsilon}} \\
\mathbf{z}^{(i)} &= \boldsymbol{\gamma} \otimes \hat{\mathbf{x}}^{(i)} + \boldsymbol{\beta}
\end{aligned}
\tag{11-4}
$$

公式 11-4　批归一化算法（Batch normalization algorithm）

在这个算法中：

- μ_B 是在整个小批量 B 上计算出的输入均值向量，每个输入对应一个均值。
- `m_B` 是小批量中的实例数量。
- `x^(i)` 是批归一化层对第 `i` 个实例的输入向量。
- σ_B 是同样在整个小批量上计算出的输入标准差向量，每个输入对应一个标准差。
- `x̂^(i)` 是第 `i` 个实例经过零中心化和归一化后的输入向量。

- ε 是一个很小的数，用来避免除以 0，并确保梯度不会变得过大，通常为 `10^-5`。它称为平滑项。
- γ 是该层的输出缩放参数向量，每个输入对应一个缩放参数。
- 公式中的乘法符号表示逐元素乘法，即每个输入都乘以对应的输出缩放参数。
- β 是该层的输出平移（偏移）参数向量，每个输入对应一个平移参数。
- `z^(i)` 是 BN 操作的输出，是输入经过重新缩放和平移后的版本。

因此，在训练期间，BN 会先将输入标准化，再重新缩放并偏移它们。测试时情况则没有这么简单。我们可能需要对单个实例进行预测，而不是对一批实例进行预测；这时无法计算每个输入的标准差。即使有一批实例，这一批也可能太小，或者实例之间并非独立同分布，因此对批次实例计算出的统计量可能不可靠。

一种解决方案是等训练结束后，让整个训练集通过神经网络，并计算 BN 层每个输入的均值和标准差。预测时，可以用这些“最终”输入均值和标准差代替批次输入的均值和标准差。不过，大多数实现会在训练期间通过对每个批次的输入均值和方差使用移动平均来估计最终统计量。PyTorch 在使用 `nn.BatchNorm1d` 等批归一化层时会自动完成这项工作。

总的来说，每个批归一化层会学习四个参数向量：输出缩放向量 γ 和输出偏移向量 β 通过常规反向传播学习；最终输入均值向量 μ 和最终输入方差向量 σ^2 则通过指数移动平均估计。注意，μ 和 σ^2 在训练期间估计，但只有在训练结束后、使用 `model.eval()` 将模型切换到评估模式时才会使用；它们会在公式 11-4 中替换 μ_B 和 σ_B^2。

Ioffe 和 Szegedy 证明，批归一化显著改善了他们实验中的所有深度神经网络，并在 ImageNet 分类任务中带来了巨大提升。ImageNet 是一个包含许多类别的大型图像分类数据集，常用于评估计算机视觉系统。他们报告说，批归一化大幅降低了梯度消失问题，使模型可以使用 tanh 甚至 sigmoid 等饱和激活函数；网络也不再那么依赖权重初始化，可以使用更大的学习率，从而显著加快训练过程。

作者还指出，批归一化会产生类似正则化的效果，降低对其他正则化技术（例如后面介绍的 dropout）的需求。

然而，批归一化也会给模型增加复杂性，而且需要额外计算，因此会带来运行时开销。幸运的是，训练结束后通常可以将 BN 层与前一层融合，从而避免运行时开销：只需更新前一层的权重和偏置，使其直接产生经过适当缩放和平移的输出。例如，如果前一层计算 `XW + b`，BN 层则计算“gamma 逐元素乘以 `(XW + b - mu) / sigma`，

再加上 beta”（忽略分母中的平滑项 epsilon）。定义更新后的权重和偏置后，公式就简化为 `XW_prime + b_prime`。因此，把前一层的权重和偏置替换为更新后的值，就可以删除 BN 层。这正是第 10 章中 `optimize_for_inference()` 所执行的优化之一。

> 注意：使用批归一化时，训练可能会变慢，因为每个 epoch 所需时间会增加。不过，BN 往往能显著加快收敛，因此只需更少的 epoch 就能达到相同性能。总的来说，墙上时钟测得的总耗时通常会更短。

11.3.1　使用 PyTorch 实现批归一化（Implementing Batch Norm with PyTorch）

和 PyTorch 中的大多数操作一样，实现批归一化简单且直观。只需在每个隐藏层的激活函数之前或之后添加 `nn.BatchNorm1d` 层，并指定每个 BN 层的输入数量。也可以在模型第一层添加 BN 层，这样就不需要手动标准化输入。例如，下面创建一个 Fashion MNIST 图像分类器（类似第 10 章构建的分类器）：

```
model = nn.Sequential(
    nn.Flatten(),
    nn.BatchNorm1d(1 * 28 * 28),
    nn.Linear(1 * 28 * 28, 300),
    nn.ReLU(),
    nn.BatchNorm1d(300),
    nn.Linear(300, 100),
    nn.ReLU(),
    nn.BatchNorm1d(100),
    nn.Linear(100, 10)
)
```

现在可以像第 10 章学到的那样正常训练模型，就这么简单。在这个只有两个隐藏层的小例子中，批归一化的影响可能不大；但对于更深的网络，它可能带来巨大差异。

> 警告：BN 在训练和评估期间的行为不同，因此训练时必须切换到训练模式（使用 `model.train()`），评估时必须切换到评估模式（使用 `model.eval()`）。忘记切换模式是最常见的错误之一。

查看第一个 BN 层的参数，会发现两个参数：`weight` 和 `bias`，它们对应公式 11-4 中的 γ 和 β：

```
>>> dict(model[1].named_parameters()).keys()
dict_keys(['weight', 'bias'])
```

查看同一 BN 层的缓冲区，会发现三个：**running_mean**、**running_var** 和 **num_batches_tracked**。前两个对应前面讨论的运行均值 μ 和运行方差 σ^2，**num_batches_tracked** 则用于记录训练期间看过的批次数量：

```
>>> dict(model[1].named_buffers()).keys()
dict_keys(['running_mean', 'running_var', 'num_batches_tracked'])
```

BN 论文作者主张把 BN 层放在激活函数之前，而不是像前面的例子那样放在之后。对此仍有争论，似乎要取决于具体任务，因此可以自行实验，看看哪个选项效果最好。如果把 BN 层移到激活函数之前，还可以从前一个 **nn.Linear** 层中删除偏置项，把其 **bias** 参数设为 **False**，因为批归一化层已经包含了一个输入偏置项。也可以删除第一个 BN 层，避免第一个隐藏层被夹在两个 BN 层之间，但这意味着必须在训练前先标准化训练集。更新后的代码如下：

```
model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(1 * 28 * 28, 300, bias=False),
    nn.BatchNorm1d(300),
    nn.ReLU(),
    nn.Linear(300, 100, bias=False),
    nn.BatchNorm1d(100),
    nn.ReLU(),
    nn.Linear(100, 10)
)
```

nn.BatchNorm1d 类包含一些可以调节的超参数。默认值通常已经足够，但有时可能需要调节 **momentum**。当层更新指数移动平均时，这个超参数会发挥作用：给定新值 **v**（即在当前批次上计算出的输入均值或方差），层会按下面的公式更新运行平均值 $\hat{v}$：

$$\hat{v} \leftarrow v \times \text{momentum} + \hat{v} \times (1 - \text{momentum})$$

合适的 **momentum** 通常接近 0，例如 0.01 或 0.001。小批次越小，应该使用越多的 0；小批次越大，应该使用越少的 0。默认值是 0.1，适合大批次，但不太适合 32 或 64 这样的较小批次。

> 警告：当人们讨论运行均值中的“momentum”时，通常指更新公式中当前运行均值的权重。由于历史原因，PyTorch 在 BN 层中使用的是相反含义。不过，PyTorch 的其他部分（例如优化器）使用传统含义，因此不要混淆。

11.3.2　BatchNorm1d、BatchNorm2d 和 BatchNorm3d

在前面的例子中，我们先将输入图像展平，再送入第一个 `nn.BatchNorm1d` 层。这是因为 `nn.BatchNorm1d` 接受形状为 `[batch_size, num_features]` 的批次（与 `nn.Linear` 相同），如果把它放到 `nn.Flatten` 之前就会报错。

不过，可以在 `nn.Flatten` 之前使用 `nn.BatchNorm2d`。它期望输入是形状为 `[batch_size, channels, height, width]` 的图像批次，并在批次维度（维度 0）以及空间维度（维度 2 和 3）上计算批均值和方差。这意味着同一批次、同一通道中的所有像素共享同一个均值和方差：`nn.BatchNorm2d` 只为每个通道设置一个权重和一个偏置。例如，对于包含红、绿、蓝三个通道的彩色图像，只有三个权重和三个偏置。这通常更适合图像数据集。

此外还有 `nn.BatchNorm3d`，它期望批次形状为 `[batch_size, channels, depth, height, width]`，适合 CT 扫描等三维图像数据集。

`nn.BatchNorm1d` 也可以处理序列批次。PyTorch 的约定是使用形状 `[batch_size, sequence_length, num_features]` 的三维张量表示序列批次。例如，在粒子物理中，假设数据集由粒子轨迹组成，每条轨迹包含 100 个三维空间点，那么 32 条轨迹组成的批次形状为 `[32, 100, 3]`。然而，`nn.BatchNorm1d` 期望形状为 `[batch_size, num_features, sequence_length]`，并在第一个和最后一个维度上计算批均值和方差，以便为每个特征得到一个均值和方差。因此，必须先使用 `X.permute(0, 2, 1)` 交换最后两个维度。第 13 章会进一步讨论序列。

批归一化已经成为深度神经网络中使用最广泛的层之一，尤其是在第 12 章将讨论的深度卷积神经网络中，以至于架构图中经常省略它，默认每层之后都加入 BN。不过，它并不完美。尤其是，某个实例的统计量会受到同一批次其他样本的影响，这可能降低性能（特别是在批次很小时）。此外，BN 也不适合某些架构，例如循环网络。由于这些原因，批归一化越来越常被层归一化取代。

11.3.3　层归一化（Layer Normalization）

层归一化（layer normalization，LN）与批归一化非常相似，但它不是跨批次维度进行归一化，而是跨特征维度进行归一化。这个简单想法由 Jimmy Lei Ba 等人在

2016 年的论文中提出，起初主要用于循环网络；近年来，它也成功应用于卷积网络、Transformer、扩散网络等许多架构。

LN 的一个优点是，它可以在每个时间步为每个实例独立地即时计算所需统计量。这也意味着训练和测试期间的行为相同，不像 BN 那样需要使用指数移动平均来估计整个训练集中的特征统计量。最后，LN 也会像 BN 一样，为每个输入特征学习一个缩放参数和一个偏移参数。

PyTorch 提供了 `nn.LayerNorm` 模块。创建该模块时，只需指出希望进行归一化的维度大小即可。这些维度必须是输入的最后若干维。例如，如果输入是形状 `[3, 100, 200]` 的 100×200 RGB 图像批次，且希望对每张图像分别在每个颜色通道内归一化，可以使用：

```
inputs = torch.randn(32, 3, 100, 200)  # a batch of random RGB images
layer_norm = nn.LayerNorm([100, 200])
result = layer_norm(inputs)  # normalizes over the last two dimensions
```

下面的代码会产生相同结果：

```
means = inputs.mean(dim=[2, 3], keepdim=True)  # shape: [32, 3, 1, 1]
vars_ = inputs.var(dim=[2, 3], keepdim=True, unbiased=False)  # shape: same
stds = torch.sqrt(vars_ + layer_norm.eps)  # eps is a smoothing term (1e-5)
result = layer_norm.weight * (inputs - means) / stds + layer_norm.bias
# result shape: [32, 3, 100, 200]
```

不过，大多数使用 LN 的计算机视觉架构会一次对所有通道进行归一化。此时，创建 `nn.LayerNorm` 模块时必须把通道维度也包括进去：

```
layer_norm = nn.LayerNorm([3, 100, 200])
result = layer_norm(inputs)  # normalizes over the last three dimensions
```

到这里就完成了层归一化。下面来看最后一种稳定训练梯度的技术：梯度裁剪。

11.3.4　梯度裁剪（Gradient Clipping）

缓解梯度爆炸问题的另一种技术，是在反向传播期间裁剪梯度，使其永远不超过某个阈值。这称为梯度裁剪。它通常用于循环神经网络，因为在循环网络中使用批归一化比较棘手（第 13 章会看到这一点）。

在 PyTorch 中，梯度裁剪通常在每次训练迭代中调用 `torch.nn.utils.clip_grad_norm_()` 或 `torch.nn.utils.clip_grad_value_()` 实现，并且要在梯度计算完成后（即 `loss.backward()` 之后）调用。这两个函数的第一个参数都是需要裁剪梯度

的模型参数列表，通常就是全部参数 `model.parameters()`。`clip_grad_norm_()` 会在每个梯度向量的范数超过给定的 `max_norm` 时进行裁剪；这是一个可以调节的超参数，典型默认值为 1.0。`clip_grad_value_()` 则独立地将梯度向量的每个分量裁剪到 `-clip_value` 和 `+clip_value` 之间，其中 `clip_value` 也是可调超参数。例如，下面的训练循环将每个梯度向量的范数裁剪为 1.0：

```
for epoch in range(n_epochs):
    for X_batch, y_batch in train_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer.step()
        optimizer.zero_grad()
```

注意，`clip_grad_value_()` 会改变梯度向量的方向。例如，原梯度向量 `[0.9, 100.0]` 主要指向第二个维度；按值裁剪后得到 `[0.9, 1.0]`，大致指向两个坐标轴之间的对角线。尽管发生了这种方向改变，这种方法在实践中仍然相当有效。如果按范数裁剪，同一个向量会变成 `[0.00899964, 0.9999595]`：它保留了原方向，但几乎消除了第一个分量。应该使用哪种裁剪函数取决于具体数据集。

11.4　重用预训练层（Reusing Pretrained Layers）

通常，不应在没有寻找相似任务现有神经网络的情况下，从零开始训练一个非常大的 DNN。如果能找到这样的神经网络，通常可以重用它的大部分层，只替换顶层。这种技术称为**迁移学习（transfer learning）**。它不仅能显著加快训练，还能大幅减少所需训练数据。

假设你有一个 DNN，它已经训练好，可以把图像分为 100 个不同类别，包括动物、植物、车辆和日常物品；现在你希望训练一个 DNN 来识别具体的车辆类型。这两个任务非常相似，甚至有部分重叠，因此应该尝试重用第一个网络的一部分（见图 11-5）。

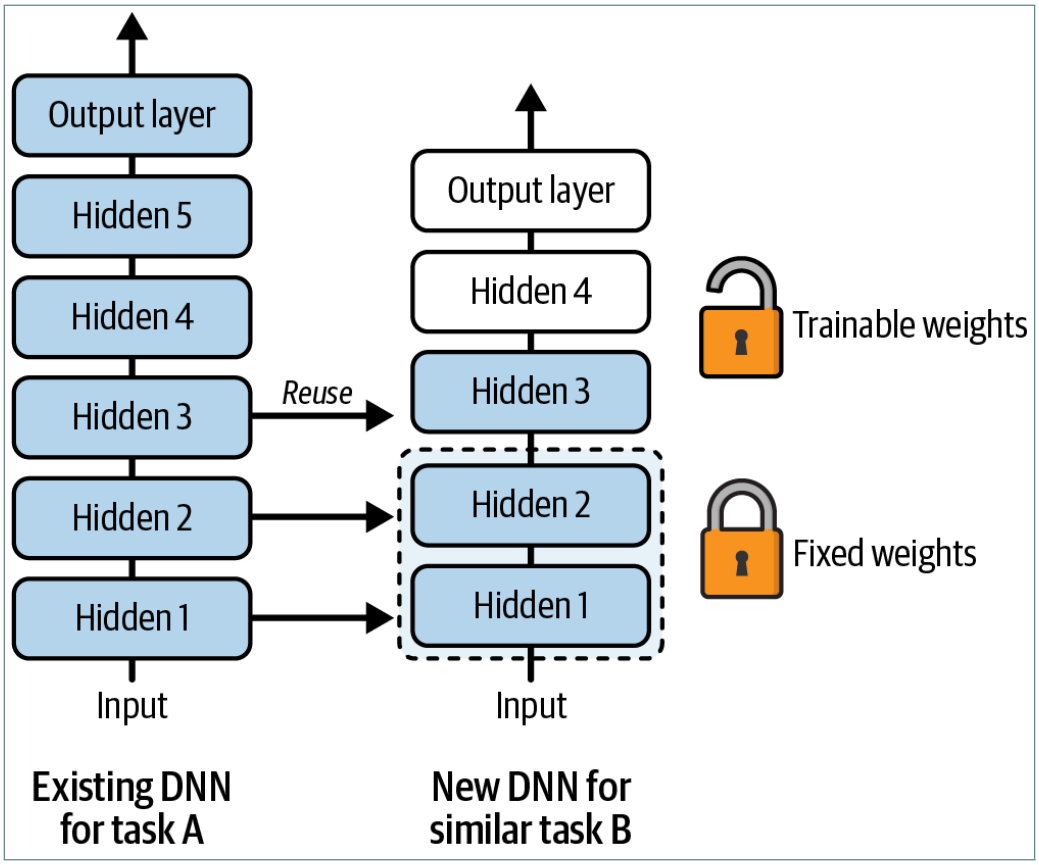

图 11-5: 重用预训练层（Reusing pretrained layers）

> 注意：如果新任务的输入图像尺寸与原任务不同，通常必须先添加预处理步骤，把图像调整为原模型期望的尺寸。更一般地说，当输入具有相似的低级特征时，迁移学习效果最好。例如，在手机上拍摄的普通照片上训练的网络，可以帮助解决其他手机照片任务，但可能完全无法帮助处理卫星图像或医学图像。

原模型的输出层通常应该替换，因为它很可能对新任务完全没有用，输出数量也可能不正确。同样，原模型的高层隐藏层通常不如低层有用，因为新任务所需的高级特征可能与原任务所需的特征有很大差异。你需要找到合适的层数来重用。

> 提示：任务越相似，应该重用的层就越多，并且应从较低层开始。对于非常相似的任务，可以保留所有隐藏层，只替换输出层。

首先尝试冻结所有重用层，也就是将它们的参数设置为不可训练，使梯度下降不会修改它们。然后训练模型，观察表现。接下来可以解冻顶部的一两个隐藏层，让

反向传播微调这些层，看看性能是否改善。训练数据越多，就能解冻越多的层。解冻重用层时，降低学习率也很有帮助，可以避免破坏它们已经微调好的权重。

如果表现仍然不好且训练数据很少，可以删除顶部隐藏层，再次冻结其余隐藏层。不断重复这个过程，直到找到合适的重用层数。如果训练数据充足，可以尝试替换顶部隐藏层，而不是删除它们，甚至添加更多隐藏层。

11.4.1　使用 PyTorch 进行迁移学习（Transfer Learning with PyTorch）

来看一个例子。假设 Fashion MNIST 数据集只包含八个类别，例如除了 Pullover 和 T-shirt/top 之外的所有类别。有人在这个数据集上构建并训练了一个 PyTorch 模型，取得了约 92% 的准确率，将其称为模型 A。现在你要解决另一个任务：手头只有 T 恤和套头衫的图像，需要训练一个二元分类器，把 T-shirt/top 判为正类，把 Pullover 判为负类。数据集非常小，只有 20 张带标签图像！如果使用与模型 A 相同的架构训练新模型 B，测试准确率只有 71.6%。你突然意识到任务 B 与任务 A 非常相似，迁移学习或许能提供帮助。先看看模型 A：

```
torch.manual_seed(42)
model_A = nn.Sequential(
    nn.Flatten(),
    nn.Linear(1 * 28 * 28, 100),
    nn.ReLU(),
    nn.Linear(100, 100),
    nn.ReLU(),
    nn.Linear(100, 100),
    nn.ReLU(),
    nn.Linear(100, 8)
)
[...]  # train this model or load pretrained weights
```

现在可以重用所需层，例如除输出层之外的所有层：

```
import copy
torch.manual_seed(42)
reused_layers = copy.deepcopy(model_A[:-1])
model_B_on_A = nn.Sequential(
    *reused_layers,
    nn.Linear(100, 1)  # new output layer for task B
).to(device)
```

这里使用 Python 的 **copy.deepcopy()** 函数复制 **nn.Sequential** 模块中的所有模块、数据和子模块，但不包括最后一层。由于是深复制，所有子模块也都会被复制。然后创建 **model_B_on_A**：它由模型 A 的重用层和任务 B 的新输出层组成。任务 B 是二元分类，因此输出层只有一个输出。

现在可以开始训练 **model_B_on_A**，但由于新输出层是随机初始化的，至少在最初几个 epoch 中会产生较大误差，随之而来的大误差梯度可能破坏重用权重。为了避免这种情况，可以在最初几个 epoch 冻结重用层，给新层一些时间学习出合理的权重：

```
for layer in model_B_on_A[:-1]:
    for param in layer.parameters():
        param.requires_grad = False
```

现在可以训练 **model_B_on_A**。但别忘了，任务 B 是二元分类，因此必须将损失切换为 **nn.BCEWithLogitsLoss**（或者在输出层添加 **nn.Sigmoid** 激活函数，再使用 **nn.BCELoss**），如第 10 章所述。如果使用 torchmetrics，还要在创建 Accuracy 指标时设置 **task="binary"**：

```
xentropy = nn.BCEWithLogitsLoss()
accuracy = torchmetrics.Accuracy(task="binary").to(device)
[...]  # train model_B_on_A
```

训练几个 epoch 后，可以解冻重用层（把所有参数的 **param.requires_grad** 设为 **True**），降低学习率，然后继续训练，对重用层进行微调。

最终结果如何？该模型的测试准确率为 92.5%，明显高于不使用预训练得到的 71.6%。

不过，不要马上相信这个结果，因为这里存在人为选择。我尝试了许多配置，直到找到一个能展示明显提升的配置。如果更换类别或随机种子，提升通常会减弱，甚至消失或反转。这种做法称为“拷问数据直到它招供”。当一篇论文的结果过于乐观时，应保持怀疑：新技术可能并没有真正帮助，甚至可能降低性能；作者只是尝试了

许多变体，只报告了运气带来的最佳结果，而没有说明途中失败了多少次。这称为 p-hacking。大多数时候这并非恶意，但它是许多科学结果无法复现的原因之一。

为什么要这样“作弊”？事实证明，迁移学习在小型全连接网络上效果并不好，可能是因为小网络能学习的模式很少，而全连接网络学到的模式非常具体，不太可能对其他任务有用。迁移学习在深度卷积神经网络和 Transformer 架构上效果最好。第 12 章和第 15 章会重新讨论迁移学习；这次我们会使用刚刚介绍的技术，并且不需要作弊。

11.5　无监督预训练（Unsupervised Pretraining）

假设你要解决一个复杂任务，但没有足够的标注训练数据，也找不到在相似任务上训练好的模型。不要失望！首先应该尝试收集更多标注训练数据；如果做不到，仍然可以进行**无监督预训练（unsupervised pretraining）**（见图 11-6）。收集未标注训练样本通常成本较低，而进行标注的成本可能很高。如果能收集大量未标注训练数据，可以用它训练一个无监督模型，例如自编码器（见第 18 章）。然后重用自编码器的低层，在其上添加针对目标任务的输出层，再使用标注训练样本进行有监督学习，对完整网络进行微调。

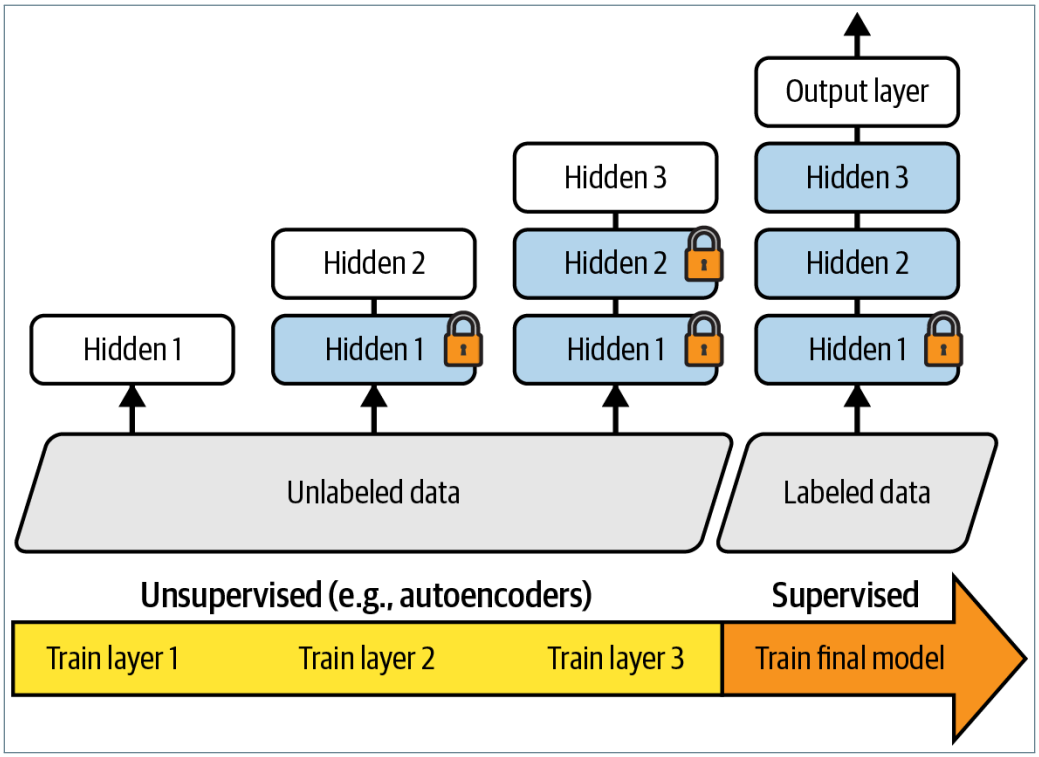

图 11-6: 深度学习早期使用的贪婪逐层预训练；如今无监督部分通常一次性在全部数据上完成（Greedy layer-wise pretraining used in the early days of deep learning; nowadays the unsupervised part is typically done in one shot on all the data rather than one layer at a time）

Geoffrey Hinton 和他的团队在 2006 年使用的正是这种技术，它带来了神经网络的复兴，并推动了深度学习的成功。直到 2010 年，无监督预训练（通常使用受限玻尔兹曼机，restricted Boltzmann machine，RBM）仍是深度网络的标准做法。直到梯度消失问题得到缓解，人们才更普遍地使用纯有监督学习训练 DNN。如今，无监督预训练通常使用自编码器或扩散模型，而不是 RBM。当任务复杂、找不到可重用的相似模型、标注数据少但未标注数据多时，它仍是不错的选择。

在深度学习早期，深度模型很难训练，因此人们会采用图 11-6 所示的贪婪逐层预训练：先训练只有一层的无监督模型，通常是 RBM；然后冻结这一层，在其上添加另一层，再次训练模型（实际上只训练新层）；接着冻结新层，再添加一层，如此反复。如今情况简单得多：人们通常一次性训练完整的无监督模型，并使用自编码器或扩散模型，而不是 RBM。

11.6　在辅助任务上预训练（Pretraining on an Auxiliary Task）

如果标注训练数据很少，还有最后一个选择：先在一个容易获取或生成标注训练数据的辅助任务上训练一个神经网络，再重用该网络的低层来解决真正的任务。第一个神经网络的低层会学习特征检测器，第二个网络很可能也能重用它们。

例如，如果要构建人脸识别系统，而每个人只有少量照片，显然不足以训练一个好的分类器。收集每个人数百张照片并不现实。不过，可以使用包含数百万人照片的公开数据集（例如 VGGFace2），训练第一个神经网络判断两张不同照片是否为同一个人。这样的网络会学习很好的人脸特征检测器，重用其低层后，就能用很少的训练数据训练出优秀的人脸分类器。

> 警告：也可以从网络上抓取随机人员的照片，但这很可能违法。首先，照片通常受创作者版权保护，Instagram、Facebook 等网站也会通过服务条款执行这些版权保护，禁止抓取和未经授权使用。其次，超过 40 个国家要求在收集和处理个人数据（包括人脸图像）时取得明确同意。

对于自然语言处理（natural language processing，NLP）应用，可以下载包含数百万篇文本的语料库，并自动从中生成带标签数据。例如，可以随机遮盖一些词，训练模型预测缺失词是什么；如果模型能在这个任务上取得良好表现，它就已经掌握了相当多的语言知识，之后可以在你的标注数据上重用并微调它。这基本就是大型语言模型的训练和微调方式（第 15 章会看到）。

> 注意：自监督学习（self-supervised learning）是指从数据本身自动生成标签，例如上面的文本遮盖例子，然后使用有监督学习技术在生成的“带标签”数据集上训练模型。

11.7　更快的优化器（Faster Optimizers）

训练一个非常大的深度神经网络可能极其缓慢。目前为止，我们已经看到了四种加快训练并得到更好解的方法：为连接权重采用好的初始化策略，使用好的激活函数，使用 batch norm 或 layer norm，以及重用预训练网络的一部分（这个网络可能是为辅助任务构建的，也可能通过无监督学习得到）。另一个巨大的加速来源是使用

比普通梯度下降更快的优化器。本节将介绍最流行的优化算法：动量、Nesterov 加速梯度、AdaGrad、RMSProp，以及 Adam 和它的变体。

11.7.1　动量（Momentum）

想象一个保龄球沿着平缓斜坡在光滑表面上滚动：它一开始很慢，但会迅速获得动量，最终达到终端速度（假设存在摩擦或空气阻力）。这就是动量优化的核心思想，由 Boris Polyak 于 1964 年提出。相比之下，普通梯度下降在斜率平缓时迈出小步，在斜率陡峭时迈出大步，但永远不会加速。因此，普通梯度下降通常比动量优化更慢地到达最小值。

如第 4 章所述，梯度下降通过直接减去损失函数 `J(theta)` 相对于权重 theta 的梯度 `grad_theta J(theta)` 与学习率 eta 的乘积来更新权重：`theta <- theta - eta * grad_theta J(theta)`。它不关心之前的梯度是什么。如果局部梯度很小，算法就会走得很慢。

动量优化非常重视过去的梯度：每次迭代时，它从动量向量 `m` 中减去当前局部梯度乘以学习率 η，然后将这个动量向量加到权重上（见公式 11-5）。换句话说，梯度在这里更像是施加在加速度上的力，而不是直接决定速度。为了模拟某种摩擦机制、防止动量增长过大，算法引入了一个新超参数 β，称为动量系数，取值在 0（摩擦很大）和 1（无摩擦）之间。典型动量值为 0.9。

$$\begin{aligned}\mathbf{m} &\leftarrow \beta\mathbf{m} - \eta\nabla_{\boldsymbol{\theta}}J(\boldsymbol{\theta})\\ \boldsymbol{\theta} &\leftarrow \boldsymbol{\theta} + \mathbf{m}\end{aligned} \tag{11-5}$$

公式 11-5　动量算法（Momentum algorithm）

如果梯度保持不变，可以验证终端速度（也就是权重更新的最大幅度）等于梯度乘以学习率 η，再乘以 `1 / (1 - β)`（忽略符号）。例如，当 β = 0.9 时，终端速度等于梯度乘以学习率的 10 倍，因此动量优化最终可能比梯度下降快 10 倍。实践中梯度并不恒定，所以加速不一定这么显著，但动量优化摆脱平台区的速度会快得多。

当输入尺度差异很大时，损失函数看起来像一只拉长的碗。梯度下降会沿陡峭斜坡快速下降，但沿谷底前进需要很长时间。动量优化则会沿谷底越来越快地滚动，直到到达底部的最优点。在不使用 batch norm 或 layer norm 的深度网络中，高层输入经常具有差异很大的尺度，因此动量优化很有帮助；它还可以帮助算法越过局部最优。

> 注意：由于动量的作用，优化器可能会稍微越过最小值，然后返回，再次越过，如此振荡多次后才在最小值附近稳定。这也是系统中保留一些摩擦有益的原因：它能减少振荡，加快收敛。

在 PyTorch 中实现动量优化非常简单：使用 SGD 优化器并设置 **momentum** 超参数即可：

```
optimizer = torch.optim.SGD(model.parameters(), momentum=0.9, lr=0.05)
```

动量优化的唯一缺点是增加了一个需要调节的超参数。不过，0.9 在实践中通常效果很好，几乎总是比普通梯度下降更快。

11.7.2　Nesterov 加速梯度（Nesterov Accelerated Gradient）

Nesterov 加速梯度（Nesterov accelerated gradient，NAG），也称 Nesterov 动量优化，是动量优化的一个小变体，由 Yurii Nesterov 于 1983 年提出。它几乎总是比普通动量优化更快。NAG 不在当前位置 θ 计算损失梯度，而是在动量方向上稍微提前的位置 **θ + βm** 处计算（见公式 11-6）。

$$\begin{aligned}\mathbf{m} &\leftarrow \beta\mathbf{m} - \eta\nabla_{\boldsymbol{\theta}}J(\boldsymbol{\theta}+\beta\mathbf{m})\\ \boldsymbol{\theta} &\leftarrow \boldsymbol{\theta}+\mathbf{m}\end{aligned} \tag{11-6}$$

公式 11-6　Nesterov 加速梯度算法（Nesterov accelerated gradient algorithm）

这个小改动有效，是因为动量向量通常指向正确方向，也就是最优点方向。因此，在稍微向该方向前进的位置测量梯度，比在原位置测量更准确。图 11-7 中，梯度 1 表示起点 theta 处的梯度，梯度 2 表示位置 **theta + beta * m** 处的梯度。

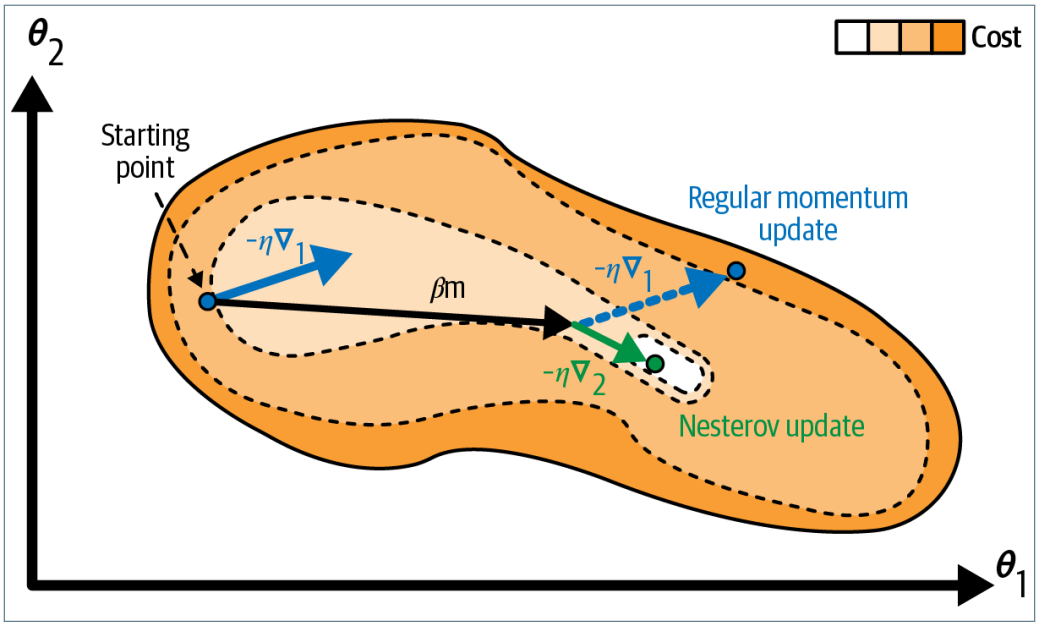

图 11-7: 普通动量与 Nesterov 动量优化：前者使用动量步骤之前计算的梯度，后者使用动量步骤之后计算的梯度（Regular versus Nesterov momentum optimization）

Nesterov 更新后的结果更接近最优点。经过一段时间，这些小改进累积起来，NAG 会明显快于普通动量优化。另外，当动量把权重推过谷底时，梯度 1 仍会继续把它推向谷底另一侧，而梯度 2 会把它拉回谷底。这有助于减少振荡，因此 NAG 收敛更快。

使用 NAG 时，只需在创建 SGD 优化器时设置 `nesterov=True`：

```
optimizer = torch.optim.SGD(model.parameters(),
                            momentum=0.9, nesterov=True, lr=0.05)
```

11.7.3　AdaGrad

再考虑拉长的碗问题：梯度下降一开始沿最陡斜坡快速下降，但最陡方向并不直接指向全局最优；随后算法沿谷底缓慢前进。若算法能更早修正方向，使其稍微朝向全局最优，就更好了。AdaGrad 算法通过沿最陡维度缩小梯度向量来实现这一点（见公式 11-7）。

$$\begin{aligned} \mathbf{s} &\leftarrow \mathbf{s} + \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \otimes \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \\ \boldsymbol{\theta} &\leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \oslash \sqrt{\mathbf{s} + \varepsilon} \end{aligned} \tag{11-7}$$

公式 11-7　AdaGrad 算法（AdaGrad algorithm）

第一步将梯度的平方累加到向量 `s` 中，其中的乘法是逐元素进行的。向量形式等价于对每个 `s_i` 计算 `s_i <- s_i + (partial J(theta) / partial theta_i)^2`；也就是说，每个 `s_i` 都会累加损失函数相对于参数 theta_i 的偏导数平方。如果损失函数沿第 `i` 个维度很陡，`s_i` 就会在每次迭代中不断增大。

第二步与梯度下降几乎相同，但有一个重要差异：梯度向量会除以 `sqrt(s + epsilon)`，从而被缩小（除法也是逐元素进行的，平方根也逐元素计算，epsilon 是避免除以 0 的平滑项，通常设为 `10^-10`）。因此，它会更快降低陡峭维度的学习率，而对平缓维度降低得更慢。这称为自适应学习率，有助于让最终更新方向更直接地指向全局最优点。

AdaGrad 在简单的二次问题上经常表现良好，但训练神经网络时常常会过早停止：学习率被缩小得太多，算法在到达全局最优之前就完全停止了。

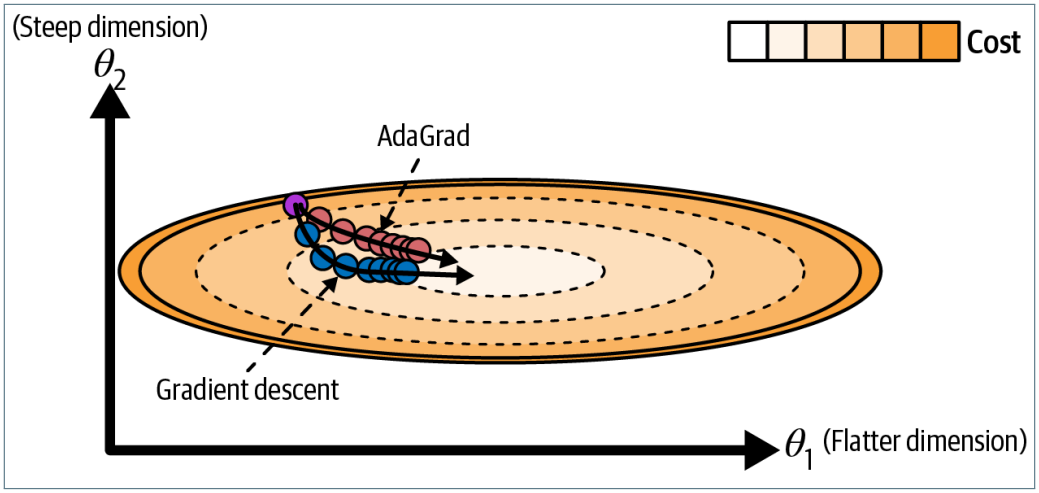

图 11-8: AdaGrad 与梯度下降：AdaGrad 可以更早修正方向，指向最优点（AdaGrad versus gradient descent）

因此，虽然 PyTorch 有 Adagrad 优化器，但不应使用它训练深度神经网络；对于线性回归等简单任务，它可能仍然有效。不过，理解 AdaGrad 有助于理解其他自适应学习率优化器。

11.7.4　RMSProp

AdaGrad 的风险在于学习率下降过快，最终无法收敛到全局最优。RMSProp 算法通过只累加最近几次迭代中的梯度来解决这个问题，而不是累加训练开始以来的所有梯度。它在第一步中使用指数衰减（见公式 11-8）。

$$\begin{aligned}\mathbf{s} &\leftarrow \alpha\mathbf{s} + (1-\alpha)\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \otimes \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \\ \boldsymbol{\theta} &\leftarrow \boldsymbol{\theta} - \eta\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \oslash \sqrt{\mathbf{s} + \varepsilon}\end{aligned} \tag{11-8}$$

公式 11-8　RMSProp 算法（RMSProp algorithm）

衰减率 α 通常设为 0.9。虽然这又增加了一个超参数，但这个默认值通常效果很好，不必专门调节。

```
optimizer = torch.optim.RMSprop(model.parameters(), alpha=0.9, lr=0.05)
```

除非问题非常简单，否则 RMSProp 几乎总是比 AdaGrad 表现好。在 Adam 优化出现之前，它实际上曾是许多研究人员首选的优化算法。

11.7.5　Adam

Adam 是 adaptive moment estimation（自适应矩估计）的缩写，它结合了动量优化和 RMSProp 的思想：像动量优化一样，它跟踪过去梯度的指数衰减平均值；像 RMSProp 一样，它跟踪过去梯度平方的指数衰减平均值（见公式 11-9）。这两个量分别是梯度均值和未中心化方差的估计。均值通常称为一阶矩，方差通常称为二阶矩，因此算法得名 Adam。

$$\begin{aligned}\mathbf{m} &\leftarrow \beta_1\mathbf{m} + (1-\beta_1)\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \\ \mathbf{s} &\leftarrow \beta_2\mathbf{s} + (1-\beta_2)\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \otimes \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \\ \hat{\mathbf{m}} &\leftarrow \frac{\mathbf{m}}{1-\beta_1^t} \\ \hat{\mathbf{s}} &\leftarrow \frac{\mathbf{s}}{1-\beta_2^t} \\ \boldsymbol{\theta} &\leftarrow \boldsymbol{\theta} + \eta\hat{\mathbf{m}} \oslash \sqrt{\hat{\mathbf{s}} + \varepsilon}\end{aligned} \tag{11-9}$$

公式 11-9　Adam 算法（Adam algorithm）

公式中的 **t** 表示迭代次数，从 1 开始。如果只看步骤 1、2 和 5，会发现 Adam 与动量优化和 RMSProp 都很相似：beta1 对应动量优化中的 beta，beta2 对应 RMSProp

中的 alpha。唯一差别是，步骤 1 计算指数衰减平均值，而不是指数衰减和；但除了一个常数因子之外，两者实际上是等价的。步骤 3 和 4 是一些技术细节：由于 `m` 和 `s` 初始化为 0，训练开始时它们会偏向 0，因此这两个步骤会在训练初期对 `m` 和 `s` 进行校正。

动量衰减超参数 beta1 通常初始化为 0.9，缩放衰减超参数 beta2 通常初始化为 0.999。平滑项 epsilon 通常初始化为 `10^-8`。这些就是 Adam 类的默认值。下面创建一个 Adam 优化器：

```
optimizer = torch.optim.Adam(model.parameters(), betas=(0.9, 0.999), lr=0.05)
```

由于 Adam 是自适应学习率算法，像 AdaGrad 和 RMSProp 一样，它不太需要调节学习率 η。通常可以直接使用默认值 η = 0.001，这让 Adam 比梯度下降更容易使用。

> 提示：如果面对这些技术感到不知所措，不确定该为任务选择哪些技术，不用担心。本章末尾会给出一些实践指南。

11.7.6　AdaMax、NAdam 和 AdamW

最后还值得介绍 Adam 的三个变体：AdaMax、NAdam 和 AdamW。

AdaMax

Adam 论文还介绍了 AdaMax。注意，在公式 11-9 的步骤 2 中，Adam 将梯度平方累加到 `s` 中，并且最近梯度的权重更大。在步骤 5 中，如果忽略 ε 以及步骤 3 和 4 这些技术细节，Adam 会通过 `s` 的平方根缩小参数更新。换句话说，Adam 按时间衰减梯度的 ℓ_2 范数缩小参数更新。

AdaMax 用 ℓ_∞ 范数（简单说就是最大值）替换 ℓ_2 范数。具体而言，它将公式 11-9 的步骤 2 替换为 `s <- max(beta2 * s, abs(grad_theta J(theta)))`，删除步骤 4，并在步骤 5 中用 `s` 缩小梯度更新；这里的 `s` 是按时间衰减梯度绝对值的最大值。实践中，AdaMax 可能比 Adam 更稳定，但这取决于数据集，通常 Adam 表现更好。如果 Adam 在某个任务上遇到问题，可以把 AdaMax 作为另一个尝试选项。

NAdam

NAdam 优化就是 Adam 加上 Nesterov 技巧，因此通常会比 Adam 略快收敛。在介绍该技术的报告中，Timothy Dozat 比较了各种任务上的许多优化器，发现 NAdam 通常优于 Adam，但有时会被 RMSProp 超过。

AdamW

AdamW 是 Adam 的一个变体，它整合了称为权重衰减的正则化技术。权重衰减在每次训练迭代中将模型权重乘以一个衰减因子（例如 0.99），从而减小权重。这可能让你想起第 4 章介绍的 ℓ2 正则化，它也旨在保持较小权重。数学上可以证明，使用 SGD 时 ℓ2 正则化等价于权重衰减。不过，在使用 Adam 或其变体时，ℓ2 正则化和权重衰减并不等价：实践中，将 Adam 与 ℓ2 正则化结合，得到的模型泛化效果通常不如 SGD。AdamW 通过正确结合 Adam 和权重衰减解决了这一问题。

> 警告：自适应优化方法（包括 RMSProp、Adam、AdaMax、NAdam 和 AdamW）通常非常优秀，能快速收敛到较好解。不过，Ashia C. Wilson 等人在 2017 年的一篇论文中指出，它们在某些数据集上可能得到泛化能力较差的解。如果模型性能令人失望，可以尝试 NAG：你的数据集可能只是对自适应梯度不适应。

在 PyTorch 中使用 NAdam、AdaMax 或 AdamW 时，可以将 **`torch.optim.Adam`** 替换为 **`torch.optim.NAdam`**、**`torch.optim.Adamax`** 或 **`torch.optim.AdamW`**。对于 AdamW，可能还需要调节 **`weight_decay`** 超参数。

前面讨论的所有优化技术只依赖一阶偏导数，也就是 Jacobian，它们测量损失函数沿每个轴的斜率。优化文献中还存在基于二阶偏导数（Hessian）的惊人算法；Hessian 是 Jacobian 的偏导数，描述每个 Jacobian 沿各轴的变化，也就是损失函数的曲率。

遗憾的是，二阶算法很难直接应用于深度神经网络，因为对于每个输出，它有 n^2 个二阶偏导数，而一阶偏导数只有 n 个，其中 n 是参数数量。DNN 通常拥有数十万甚至更多参数，因此二阶优化算法往往连内存都无法容纳；即使能够容纳，计算 Hessian 矩阵也太慢了。[2]

[2] Jacobian 矩阵包含多参数、多输出函数的所有一阶偏导数：每个参数对应一列，每个输出对应一行。使用梯度下降训练神经网络时，输出只有一个，即损失，因此矩阵只有一行，每个模型参数对应一列，是一个 1×n 矩阵。Hessian 矩阵包含多参数、单输出函数的所有二阶偏导数：每个模型参数对应一行和一列，因此是 n×n 矩阵。“Jacobian”和“Hessian”这两个非正式名称也常用来指这些矩阵本身的元素。

幸运的是，可以使用随机方法高效近似二阶信息。其中一种算法是 Shampoo：它利用累积的梯度信息近似二阶项，类似 Adam 累积一阶统计量。PyTorch 库不包含它，但可以在 PyTorch-Optimizer 库中获得（`pip install torch_optimizer`）。

> **训练稀疏模型**前面讨论的优化算法都会产生稠密模型，也就是说，大多数参数都不为 0。如果需要运行时速度极快的模型，或者希望模型占用更少内存，可能更希望得到稀疏模型。一种方法是按常规训练模型，然后使用 `torch.nn.utils.prune.l1_unstructured()` 删除很小的权重（将其设为 0）。也可以使用 `torch.nn.utils.prune.ln_structured()` 或 `torch.nn.utils.prune` 包中的其他函数，删除完整神经元、通道或层，而不是只删除单个权重。通常还应在训练期间使用较强的稀疏诱导正则化，例如 ℓ1 正则化（本章稍后介绍），因为它会促使优化器尽可能将更多权重归零。也可以在初始化时缩小随机权重，以鼓励稀疏性。

表 11-2: 优化器比较（星级越多越好）

类别	收敛速度	收敛质量
SGD	★	★★★
SGD(momentum=...)	★★	★★★
SGD(momentum=..., nesterov=True)	★★	★★★
Adagrad	★★	★（过早停止）
RMSprop	★★	★★★ 或 ★★★★
Adam	★★	★★ 或 ★★★★
AdaMax	★	★★ 或 ★★★★
NAdam	★★	★★ 或 ★★★★
AdamW	★★	★★ 或 ★★★★

11.8　学习率调度（Learning Rate Scheduling）

找到合适的学习率非常重要。学习率太高，训练会发散；太低，训练会非常缓慢，也可能陷入局部最优，产生次优模型。如果学习率设置得相当高但又不足以发散，训练通常会在初期快速进展，但在后期会围绕最优点来回摆动，最终得到次优模型。找到真正好的学习率可以得到优秀模型，但训练通常会有些慢。幸运的是，不必使用固定学习率。更好的做法是以较高学习率开始训练，然后在训练后期（或进展停止时）逐渐降低学习率：这样训练初期速度快，后期又能让反向传播逐渐稳定下来，精细调整模型参数（见图 11-9）。

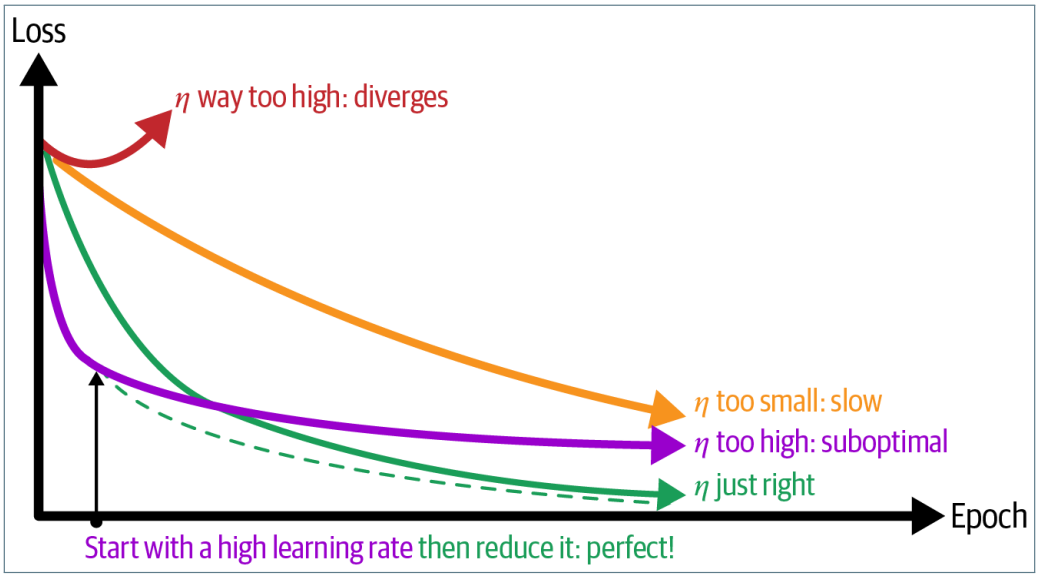

图 11-9: 不同学习率 η 下的学习曲线（Learning curves for various learning rates η）

学习率调度 (learning rate schedule) 就是在训练期间调整学习率的策略。**torch.optim.lr_scheduler** 模块提供了多种常见调度器实现。下面从指数调度开始介绍最重要的几种。

11.8.1　指数调度（Exponential Scheduling）

ExponentialLR 类实现指数调度：在固定间隔（通常是每个 epoch）将学习率乘以一个常数因子 **gamma**。因此，第 **n** 个 epoch 后，学习率等于初始学习率乘以 **gamma** 的 **n** 次方。**gamma** 又是一个需要调节的超参数。通常应该把它设为小于 1、但相当接近 1 的值，以免学习率下降过快。例如，如果 **gamma** 为 0.9，那么 10 个 epoch 后学习率约为初始值的 35%，20 个 epoch 后约为 12%。

ExponentialLR 构造器至少需要两个参数：要调整学习率的优化器和因子 **gamma**：

```
model = [...]  # build the model
optimizer = torch.optim.SGD(model.parameters(), lr=0.05)  # or any other optim.
scheduler = torch.optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.9)
```

然后修改训练循环，在每个 epoch 末尾调用 **scheduler.step()**，调整优化器的学习率：

```
for epoch in range(n_epochs):
    for X_batch, y_batch in train_loader:
        [...]  # the rest of the training loop remains unchanged
    scheduler.step()
```

> 提示：如果中断训练，之后希望从中断处继续，应将调度器构造器的 **last_epoch** 参数设置为上次运行的 epoch（从 0 开始计数）。默认值为 -1，表示从头开始训练。

11.8.2　余弦退火（Cosine Annealing）

与指数下降不同，余弦退火使用余弦函数让学习率从训练开始时的最大学习率 η_max 下降到结束时的最小学习率 η_min。与指数调度相比，余弦退火可以让学习率在训练的大部分时间保持较高水平，直到接近结束时才趋近最小值（见图 11-10）。总体而言，余弦退火通常表现更好。第 **t** 个 epoch（从 0 开始）的学习率由公式 11-10 给出，其中 **T_max** 是最大 epoch 数。

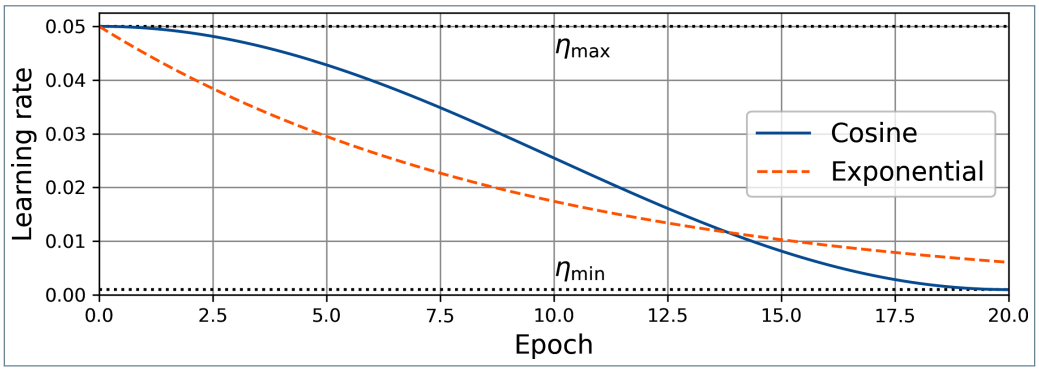

图 11-10：余弦退火学习率调度（Cosine annealing learning schedule）

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\left(\frac{t}{T_{\max}}\pi\right)\right) \qquad (11\text{-}10)$$

公式 11-10　余弦退火公式（Cosine annealing equation）

PyTorch 包含 **CosineAnnealingLR** 调度器，创建方式如下（**T_max** 对应公式中的 **T_max**，**eta_min** 对应 η_min）：

```
cosine_scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=20, eta_min=0.001)
```

之后可以像使用 `ExponentialLR` 一样使用它。余弦退火的问题是需要设置两个新超参数 `T_max` 和 `η_min`，而且很难事先知道训练多少个 epoch、何时停止降低学习率。因此，通常更偏好基于性能的调度。

11.8.3　基于性能的调度（Performance Scheduling）

基于性能的调度也称自适应调度，由 PyTorch 的 `ReduceLROnPlateau` 调度器实现。它会跟踪训练期间的某个指标（通常是验证损失），如果该指标在一段时间内不再改善，就把学习率乘以某个因子。这个调度器有不少超参数，但大多数默认值对常见任务都很好。偶尔可能需要调整下面几个：

- `mode`：如果监控指标需要最大化（例如验证准确率），必须将 `mode` 设为`'max'`。默认值是`'min'`，适合需要最小化的指标（例如验证损失）。
- `patience`：等待监控指标改善的连续步骤数，通常以 epoch 计数，之后才降低学习率。默认值为 10，通常合适；如果每个 epoch 很长，可以减小它。
- `factor`：监控指标长时间没有改善时，学习率要乘以的因子。默认值为 0.1，通常合理，但某些情况下可能略小。

例如，基于验证准确率实现调度（准确率需要最大化）：

```
[...]  # build the model and optimizer
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
  optimizer, mode="max", patience=2, factor=0.1)
```

训练循环也需要修改：每个 epoch 计算目标指标，并将结果传给调度器的 `step()` 方法。本例使用第 10 章定义的 `evaluate_tm()` 函数：

```
metric = torchmetrics.Accuracy(task="multiclass", num_classes=10).to(device)
for epoch in range(n_epochs):
    for X_batch, y_batch in train_loader:
        [...]  # the rest of the training loop remains unchanged
    val_metric = evaluate_tm(model, valid_loader, metric).item()
    scheduler.step(val_metric)
```

11.8.4　学习率预热（Warming Up the Learning Rate）

到目前为止，我们总是以最大学习率开始训练。但有时这会让梯度下降在训练初期随机来回摆动，既不发散，也没有明显进展。这通常发生在敏感模型（例如循环神经网络，第 13 章）或使用超大批次时。一个解决方案是对学习率进行“预热”：从接近 0 的值开始，在若干个 epoch 中逐渐提高学习率，直到达到最大值。在预热阶段，梯度下降有时间在损失景观中稳定到更好的区域，随后就能使用较高学习率快速推进。

为什么有效？损失景观有时像喜马拉雅山：处处很高，还有巨大的尖峰。如果一开始使用高学习率，算法可能会在山峰之间跳跃很长时间。相反，如果从小学习率开始，就能沿着山和谷慢慢走下去，脱离多尖峰区域，直到进入较平坦的地带。此后即可在剩余训练过程中使用较高学习率，只在最后阶段放慢速度。

在 PyTorch 中实现学习率预热的一种常见方法是使用 `LinearLR` 调度器，在几个 epoch 内线性提高学习率。例如，下面的调度器会在 3 个 epoch 内，将学习率从优化器原始学习率的 10% 提高到 100%：

```
warmup_scheduler = torch.optim.lr_scheduler.LinearLR(
    optimizer, start_factor=0.1, end_factor=1.0, total_iters=3)
```

如果需要更灵活的方式，可以编写自定义函数，并将其包装在 `LambdaLR` 调度器中。下面的调度器与刚才的 `LinearLR` 等价：

```
warmup_scheduler = torch.optim.lr_scheduler.LambdaLR(
    optimizer,
    lambda epoch: (min(epoch, 3) / 3) * (1.0 - 0.1) + 0.1)
```

然后必须在每个 epoch 开始时插入 `warmup_scheduler.step()`，并确保在预热阶段停用用于其余训练的调度器：

```
for epoch in range(n_epochs):
    warmup_scheduler.step()
    for X_batch, y_batch in train_loader:
        [...]  # the rest of the training loop is unchanged
    if epoch >= 3:  # deactivate other scheduler(s) during warmup
        scheduler.step(val_metric)
```

总之，训练结束时几乎总是希望降低学习率；如果梯度下降需要帮助才能启动，训练开始时也可以进行预热。不过，是否应该在训练中段调整学习率呢？有时确实需要。例如，梯度下降可能陷入局部最优或高位平台区。它可能长时间甚至永远停在那里。幸运的是，可以暂时提高学习率来逃离这个陷阱。

你可以在训练期间盯着学习曲线，手动中断训练并调整学习率，但通常有更好的事情可做。也可以实现一个类似 `ReduceLROnPlateau` 的自定义调度器：监控验证指标，如果指标停留在糟糕的平台区，就暂时提高学习率。这需要继承 `LRScheduler` 基类，超出了本书范围；不过可以参考 `ReduceLROnPlateau` 的源码。更简单的选择是使用带热重启的余弦退火学习率调度。

11.8.5　带热重启的余弦退火（Cosine Annealing with Warm Restarts）

带热重启的余弦退火由 Ilya Loshchilov 和 Frank Hutter 在 2016 年的论文中提出。该调度器不断重复余弦退火调度。由于学习率会定期重新升高，它能让梯度下降自动逃离局部最优和平台区。

作者建议先使用较短的余弦退火周期，然后在每一轮之后将 `T_max` 加倍（见图 11-11）。这样，训练初期可以快速探索；后期又有充足时间优化模型，并可能在途中逃离一个或多个平台区。

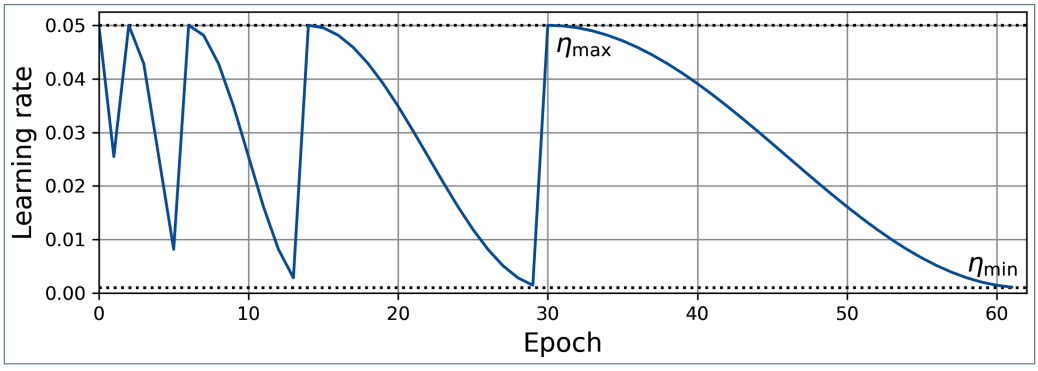

图 11-11: 带热重启的余弦退火（Cosine annealing with warm restarts）

PyTorch 提供了 **CosineAnnealingWarmRestarts** 调度器。必须设置 **T_0**，它是第一轮余弦退火的 **T_max**。如果希望每轮将 **T_max** 加倍，可以将 **T_mult** 设为 2（默认值为 1，表示 **T_max** 保持不变，每轮长度相同）。最后可以设置 **eta_min**，默认值为 0：

```
cosine_repeat_scheduler = torch.optim.lr_scheduler.CosineAnnealingWarmRestarts(
    optimizer, T_0=2, T_mult=2, eta_min=0.001)
```

11.8.6　1cycle 调度（1cycle Scheduling）

另一种流行的学习率调度是 1cycle，由 Leslie Smith 在 2018 年的论文中提出。它首先预热学习率：从 eta0 开始，在训练进行到一半时线性增加到 eta1；随后在训练的后半段线性降低回 eta0，最后几个 epoch 仍然线性下降若干个数量级。最大值 eta1 使用寻找最优学习率时的同样方法选择，初始值 eta0 通常比它低 10 倍。

使用动量时，开始阶段先使用较高动量（例如 0.95），然后在训练前半段将动量线性降低到较低值（例如 0.85），再在后半段线性提高回最大值（例如 0.95），最后几个 epoch 保持该最大值。Smith 进行了大量实验，发现这种方法往往可以显著加快训练并达到更好性能。例如，在流行的 CIFAR10 图像数据集上，同一网络架构使用 1cycle 在 100 个 epoch 后达到 91.9% 的验证准确率，而标准方法训练 800 个 epoch 只有 90.3%。这种效果被称为“超收敛（super-convergence）”。PyTorch 在 **OneCycleLR** 调度器中实现了这种方法。

> 提示：如果不确定使用哪种学习率调度，1cycle 是不错的默认选择，但实践中我通常更偏好基于性能的调度。如果训练开始时不稳定，可以添加学习率预热；如果训练陷入平台区，可以尝试带热重启的余弦退火。

PyTorch 还提供其他调度器，例如多项式调度器、循环调度器，以及便于串联多个调度器的调度器。请查看相关文档。

11.9　通过正则化避免过拟合（Avoiding Overfitting Through Regularization）

> “用四个参数，我可以拟合一头大象；用五个参数，我可以让它摆动鼻子。”—-John von Neumann，Enrico Fermi 在《Nature》427 中引用

有了数千个参数，你可以拟合整个动物园。深度神经网络通常有数万个参数，有时甚至达到数百万或数十亿。这赋予它们惊人的自由度，使其能够拟合各种复杂数据集。但这种强大灵活性也使网络容易过拟合训练集，因此通常需要正则化。

第 4 章已经实现了一种常见正则化技术：提前停止。此外，batch norm 和 layer norm 虽然最初是为了解决梯度不稳定问题而设计的，但也具有相当好的正则化作用。本节将研究神经网络的其他常用正则化技术：ℓ_1 和 ℓ_2 正则化、dropout、MC dropout 以及 max-norm 正则化。

11.9.1　ℓ_1 和 ℓ_2 正则化（L1 and L2 Regularization）

和第 4 章的简单线性模型一样，可以使用 ℓ_2 正则化约束神经网络的连接权重；如果希望得到稀疏模型（许多权重等于 0），也可以使用 ℓ_1 正则化。正如前面讨论 AdamW 时所见，对于 SGD 优化器（无论是否使用动量），ℓ_2 正则化在数学上等价于权重衰减，因此可以简单地设置优化器的 **`weight_decay`** 参数来实现。下面使用正则化因子 **`10^-4`** 对通过 SGD 训练的 PyTorch 模型连接权重应用 ℓ_2 正则化：

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.05, weight_decay=1e-4)
[...]  # use the optimizer normally during training
```

如果使用 Adam 优化器，应改用 AdamW 并设置 **`weight_decay`** 参数。它与 ℓ_2 正则化并不完全等价，但如前面所见，两者非常接近，而且效果更好。

注意，权重衰减会应用于每个模型参数，包括偏置项，以及 batch norm 和 layer norm 层的参数。通常这不是什么大问题，但惩罚这些参数并不能有效增加正则化，有时还会降低训练性能。那么，如何只对部分模型参数应用权重衰减呢？一种方法是不依赖优化器的权重衰减功能，手动实现 ℓ_2 正则化：修改训练循环，只根据想要正则化的参数计算 ℓ_2 损失，并将它加到主损失上：

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.05)
params_to_regularize = [
    param for name, param in model.named_parameters()
    if not "bias" in name and not "bn" in name]

for epoch in range(n_epochs):
    for X_batch, y_batch in train_loader:
        [...]  # the rest of the training loop is unchanged
        main_loss = loss_fn(y_pred, y_batch)
        l2_loss = sum(param.pow(2.0).sum() for param in params_to_regularize)
        loss = main_loss + 1e-4 * l2_loss
        [...]
```

另一种方法是使用 PyTorch 的参数组功能，让优化器对不同参数组应用不同超参数。之前创建优化器时，我们总是传入完整的模型参数列表，PyTorch 会把它们全部放在一个共享超参数的参数组中。现在可以向优化器传入一个字典列表，每个字典包含一个 **params** 条目，以及（可选的）专属于该参数组的超参数键值对。参数组专属超参数优先于优化器的全局超参数。下面创建两个参数组：第一组包含要正则化的参数，使用权重衰减；第二组包含所有偏置项和 BN 参数，不使用权重衰减。

```
params_bias_and_bn = [
    param for name, param in model.named_parameters()
    if "bias" in name or "bn" in name]
optimizer = torch.optim.SGD([
    {"params": params_to_regularize, "weight_decay": 1e-4},
    {"params": params_bias_and_bn},
    ], lr=0.05)
[...]  # use the optimizer normally during training
```

> 提示：参数组也可以让模型的不同部分使用不同学习率。这在迁移学习中最常见，因为你通常希望新层比重用层更新得更快。

至于 ℓ_1 正则化，PyTorch 没有提供辅助函数，因此需要像 ℓ_2 正则化一样手动实现：修改训练循环，计算 ℓ_1 损失并将它添加到主损失中：

```
l1_loss = sum(param.abs().sum() for param in params_to_regularize)
loss = main_loss + 1e-4 * l1_loss
```

11.9.2　Dropout

Dropout 由 Geoffrey Hinton 等人在 2012 年提出，并在 Nitish Srivastava 等人 2014 年的论文中进一步阐述。它取得了很大成功：许多最先进的神经网络使用 dropout，因为它通常能带来 1% 至 2% 的准确率提升。这听起来可能不多，但如果模型原本已经达到 95% 准确率，提升 2 个百分点意味着错误率几乎降低 40%，即从 5% 错误率降到大约 3%。

Dropout 是一个相当简单的算法：每个训练步骤中，每个神经元（包括输入神经元，但始终不包括输出神经元）都有概率 **p** 被暂时“丢弃”，也就是说在本次训练步骤中完全忽略它，但下一步可能重新激活（见图 11-12）。超参数 **p** 称为 dropout 率，通常在 10% 到 50% 之间：循环神经网络更接近 20% 至 30%，卷积神经网络更接近 40% 至 50%。训练结束后，神经元不再被丢弃，就这么简单（除了下面会介绍的技术细节）。

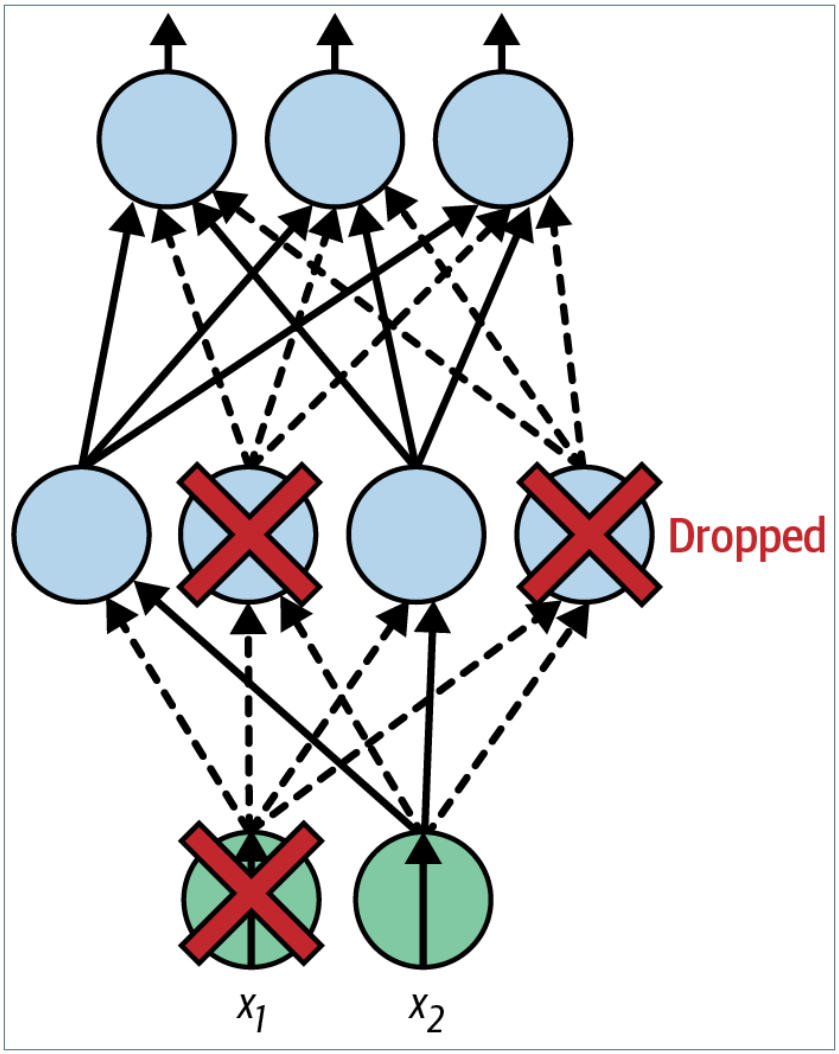

图 11-12: 使用 dropout 正则化时，每次训练迭代都会从一个或多个层中随机丢弃部分神经元（输出层除外）；这些神经元在本次迭代输出 0（由虚线箭头表示）（With dropout regularization, a random subset of neurons is dropped out at each training iteration）

乍看之下，这种破坏性技术竟然有效，令人惊讶。假设公司要求员工每天早晨抛硬币决定是否上班，这样的公司会表现得更好吗？也许会：公司必须调整组织方式，不能依赖某个特定员工操作咖啡机或完成其他关键任务，因此专业能力必须分散到多个人身上；员工还得学会与许多同事协作，而不只是少数几个人。公司最终会更有韧性；即使某个人离职，也不会造成太大影响。这个想法对公司是否真的有效并不确定，但对神经网络确实有效。

使用 dropout 训练的神经元无法与相邻神经元共同适应，它们必须尽可能独立地有用；也不能过度依赖少数输入神经元，而必须关注所有输入神经元。最终得到的是对输入的轻微变化不那么敏感、更加鲁棒、泛化能力更好的网络。

理解 dropout 威力的另一种方式是：每个训练步骤都会生成一个独特的神经网络。由于每个神经元都可能存在或缺失，共有 `2^N` 个可能的网络，其中 `N` 是所有可丢弃神经元的总数。这个数量如此巨大，几乎不可能两次采样到同一个网络。运行 10,000 个训练步骤，本质上就是训练了 10,000 个不同的神经网络，每个网络只使用了一个训练实例。这些网络并不相互独立，因为它们共享许多权重，但它们仍然各不相同。最终得到的网络可以看作所有这些更小网络的平均集成。

> 提示：高层学习更复杂的特征组合，更容易过拟合，因此从 dropout 中受益更多。通常可以只对顶部一到三个隐藏层的神经元应用 dropout。不过不要丢弃输出神经元，因为这相当于在训练期间改变任务，不会带来帮助。

还有一个重要的技术细节。假设 `p = 75%`，训练期间平均只有 25% 的神经元处于激活状态。因此训练结束后，每个神经元平均接收到的输入会是训练期间的 4 倍，模型很可能无法正常工作。简单的解决方案是在训练期间将输入乘以 4，也就是除以 25%。更一般地说，训练期间需要将输入除以保留概率 `1 - p`。

PyTorch 使用 `nn.Dropout` 层实现 dropout。训练时必须切换到训练模式，评估时必须切换到评估模式，和批归一化一样。在训练模式下，该层随机丢弃部分输入（将其设为 0），并将剩余输入除以保留概率；在评估模式下，它什么也不做，只把输入传递给下一层。下面在每个 `nn.Linear` 层之前使用 dropout，dropout 率为 0.2：

```
model = nn.Sequential(
    nn.Flatten(),
    nn.Dropout(p=0.2), nn.Linear(1 * 28 * 28, 100), nn.ReLU(),
    nn.Dropout(p=0.2), nn.Linear(100, 100), nn.ReLU(),
    nn.Dropout(p=0.2), nn.Linear(100, 100), nn.ReLU(),
    nn.Dropout(p=0.2), nn.Linear(100, 10)
).to(device)
```

> 警告：dropout 只在训练期间生效，因此比较训练损失与验证损失可能会产生误导。特别是，模型可能已经过拟合训练集，但训练损失和验证损失仍然相似。因此要在关闭 dropout 的情况下评估训练损失，例如在训练结束后评估。

如果观察到模型过拟合，可以提高 dropout 率；反过来，如果模型对训练集欠拟合，则应降低 dropout 率。对大层提高 dropout 率、对小层降低 dropout 率也可能有帮助。此外，许多最先进的架构只对最后几个隐藏层应用 dropout；如果全网络 dropout 太强，可以尝试这种方式。

Dropout 确实会显著减慢收敛，但正确调节后通常会带来更好的模型。因此，对于大型模型，额外的时间和精力通常值得投入。

> 提示：如果要对使用 SELU 激活函数的自归一化网络进行正则化，应使用 alpha dropout。这是 dropout 的一种变体，可以保持输入的均值和标准差。它与 SELU 在同一篇论文中提出，因为普通 dropout 会破坏自归一化。PyTorch 在 `nn.AlphaDropout` 中实现了它。

11.9.3　Monte Carlo Dropout

2016 年，Yarin Gal 和 Zoubin Ghahramani 的论文为 dropout 提出了几个更好的理由：

1. 论文建立了 dropout 网络与近似贝叶斯推断之间的深刻联系，为 dropout 提供了坚实的数学依据。
2. 作者提出了强大的 Monte Carlo（MC）dropout 技术，它无需重新训练，甚至无需修改已训练的 dropout 模型，就能提高其性能。它还能更好地度量模型不确定性，而且只需几行代码即可实现。

MC dropout 听起来像“一个奇怪但有效的技巧”，但它其实很简单：和普通 dropout 一样，不同之处在于它不仅在训练时激活，在评估时也激活。这意味着预测总是带有一些随机性，因此称为 Monte Carlo。不要只进行一次预测，而应进行多次预测并取平均。结果证明，这样得到的预测比原始模型更好。

下面使用上一节训练的模型，对一批图像实现完整的 MC dropout：

```
model.eval()
for module in model.modules():
    if isinstance(module, nn.Dropout):
        module.train()

X_new = [...]  # some new images, e.g., the first 3 images of the test set
X_new = X_new.to(device)
torch.manual_seed(42)
with torch.no_grad():
    X_new_repeated = X_new.repeat_interleave(100, dim=0)
    y_logits_all = model(X_new_repeated).reshape(3, 100, 10)
    y_probas_all = torch.nn.functional.softmax(y_logits_all, dim=-1)
    y_probas = y_probas_all.mean(dim=1)
```

逐步解释这段代码。首先像往常一样将模型切换到评估模式，但随后立即把所有 dropout 层切回训练模式，使它们像训练期间一样工作，也就是随机丢弃部分输入。换句话说，我们把 dropout 层变成了 MC dropout 层。

接下来加载一批新图像并将它移动到 GPU。本例假设 `X_new` 包含三张图像。然后使用 `repeat_interleave()` 创建一个批次，其中每张图像复制 100 份。沿第一个维度（`dim=0`）重复后，`X_new_repeated` 的形状为 `[300, 1, 28, 28]`。

接着将这个大批次传给模型，模型像往常一样为每张图像预测 10 个 logits。该张量形状为 `[300, 10]`，再将其重塑为 `[3, 100, 10]`，把每张图像的预测分组在一起。由于 dropout 层仍然处于激活状态，即使是同一张图像的副本，预测结果也会有差异。然后通过 softmax 将 logits 转换为估计概率。最后，在第二个维度（`dim=1`）上求平均，得到每张图像每个类别在 100 次预测中的平均估计概率。结果张量形状为 `[3, 10]`：

```
>>> y_probas.round(decimals=2)
tensor([[0.000, 0.000, 0.000, 0.000, 0.000, 0.000, 0.000, 0.010, 0.000, 0.990],
        [0.990, 0.000, 0.000, 0.000, 0.000, 0.000, 0.010, 0.000, 0.000, 0.000],
        [0.410, 0.040, 0.040, 0.230, 0.040, 0.000, 0.230, 0.000, 0.010, 0.000]],
       device='cuda:0')
```

> 警告：不要先对 logits 求平均，再将平均 logits 转换为概率。虽然这样更快，但不能正确反映模型的不确定性，往往会产生过度自信的模型。

MC dropout 往往能提高模型概率估计的可靠性，也就是说模型更不容易自信地犯错，这在自动驾驶等安全场景中很有用。它在关心前 `k` 个类别而不只是最可能类别时也很有用。此外，还可以观察每个类别概率的标准差：

```
>>> y_std = y_probas_all.std(dim=1)
>>> y_std.round(decimals=2)
tensor([[0.000, 0.000, 0.000, 0.000, 0.000, 0.000, 0.000, 0.020, 0.000, 0.020],
        [0.020, 0.000, 0.000, 0.000, 0.000, 0.000, 0.020, 0.000, 0.000, 0.000],
        [0.170, 0.030, 0.030, 0.130, 0.050, 0.000, 0.090, 0.000, 0.010, 0.000]],
       device='cuda:0')
```

第一张图像的第 9 类（踝靴）概率估计标准差为 0.02。这说明 99% 的概率估计需要保留一些余地：模型实际上是在说“嗯，我猜它的概率超过 95%”。如果构建风险敏感系统（例如医疗或金融系统），可以只考虑同时具有高估计概率和低标准差的预测。

> 注意：Monte Carlo 样本数量（本例中为 100）是可以调节的超参数。数量越多，预测及其不确定性估计越准确，但预测速度也越慢。超过某个数量后，增加样本带来的改善会很小。需要根据具体应用，在延迟、吞吐量和准确率之间找到合适的折中。

如果想从零开始训练 MC dropout 模型，而不是重用已有的 dropout 模型，最好使用自定义的 `McDropout` 模块，而不是通过 `train()` 和 `eval()` 反复切换 `nn.Dropout`。后者比较脆弱，例如与评估函数配合时可能出现问题。下面是一个三行实现：

```
class McDropout(nn.Dropout):
    def forward(self, input):
        return F.dropout(input, self.p, training=True)
```

总之，MC dropout 是一种优秀的技术，可以提升 dropout 模型的性能并提供更好的不确定性估计。当然，由于它在训练期间就是普通 dropout，它也具有普通 dropout 的正则化作用。

11.9.4　Max-Norm 正则化（Max-Norm Regularization）

神经网络中另一种相当流行的正则化技术称为 max-norm 正则化：对于每个神经元，约束其输入连接权重 **w**，使其满足 **||w||_2 ≤ r**，其中 **r** 是 max-norm 超参数，**||·||_2** 是 ℓ2 范数。

减小 **r** 会增强正则化，有助于减少过拟合。如果不使用 batch norm 或 layer norm，max-norm 正则化也可能帮助缓解梯度不稳定问题。

max-norm 正则化通常不会向总损失中添加正则化项，而是在每个训练步骤后计算 **||w||_2**，并在需要时重新缩放 **w**（**w ← w r / ||w||_2**）。在 PyTorch 中，一种常见实现如下：

```
def apply_max_norm(model, max_norm=2, epsilon=1e-8, dim=1):
    with torch.no_grad():
        for name, param in model.named_parameters():
            if 'bias' not in name:
                actual_norm = param.norm(p=2, dim=dim, keepdim=True)
                target_norm = torch.clamp(actual_norm, 0, max_norm)
                param *= target_norm / (epsilon + actual_norm)
```

这个函数遍历模型的所有权重矩阵，也就是除偏置项之外的所有参数；对每个参数，使用 **norm()** 方法计算每一行的 ℓ2 范数（**dim=1**）。**nn.Linear** 层的权重形状为 **[神经元数量, 输入数量]**，因此 **dim=1** 会为每个神经元得到一个范数，正是我们需要的结果。随后使用 **torch.clamp()** 为每个神经元的权重计算目标范数：它创建 **actual_norm** 的副本，并将所有大于 **max_norm** 的值替换为 **max_norm**，这就是前面公式中的 **r**。最后重新缩放权重矩阵，使每一行达到目标范数。平滑项 **epsilon** 用于在某些行范数为 0 时避免除以 0。

接下来只需在训练循环中、优化器的 **step()** 方法之后调用 **apply_max_norm(model)**。当然，还应调节 **max_norm** 超参数。

> 提示：对 `nn.Linear` 以外的层使用 max-norm 时，可能需要调整 `dim` 参数。例如，对卷积层（见第 12 章）使用它时，通常应该设置 `dim=[1, 2, 3]`，以限制每个卷积核的范数。

11.10　实用指南（Practical Guidelines）

本章涵盖了许多技术，你可能会疑惑应该使用哪些。答案取决于任务，目前还没有明确共识。不过，下面表 11-3 中的配置在大多数情况下都能很好地工作，而且不需要大量调节超参数。请不要把这些默认值当作硬性规则。

表 11-3: DNN 默认配置

超参数	默认值
核初始化器	He 初始化
激活函数	浅层网络用 ReLU；深层网络用 Swish
归一化	浅层网络不使用；深层网络使用 batch norm 或 layer norm
正则化	提前停止；必要时使用权重衰减
优化器	Nesterov 加速梯度或 AdamW
学习率调度	基于性能的调度或 1cycle

如果能找到一个解决相似问题的预训练神经网络，也应该尝试重用它的一部分；如果有大量未标注数据，可以使用无监督预训练；如果有大量相似任务的标注数据，可以在辅助任务上进行预训练。

虽然前面的指南可以覆盖大多数情况，但也存在一些例外：

- 如果需要稀疏模型，可以使用 $\ell 1$ 正则化。训练后还可以将最小的权重归零，例如使用 `torch.nn.utils.prune.l1_unstructured()`。这会破坏自归一化，因此此时应使用默认配置。
- 如果需要低延迟模型（能以极快速度进行预测），可能需要减少层数；使用 `nn.ReLU`、`nn.LeakyReLU` 或 `nn.Hardswish` 等快速激活函数；并在训练后将 batch norm 和 layer norm 层折叠到前一层中。稀疏模型也有帮助。最后，可以将浮点精度从 32 位降低到 16 位甚至 8 位。附录 B 会介绍让模型更小、更快的几种技术，包括低精度模型、混合精度模型和量化。

- 如果构建风险敏感应用，或者推理延迟并不重要，可以使用 MC dropout 提升性能，得到更可靠的概率估计和不确定性估计。

在前面三章中，我们学习了人工神经网络是什么，如何使用 Scikit-Learn 和 PyTorch 构建、训练它们，以及训练深度复杂网络的各种技术。在下一章中，所有这些内容将汇聚到深度学习最重要的应用之一：计算机视觉。

11.11　练习（Exercises）

1. Glorot 初始化和 He 初始化旨在解决什么问题？
2. 只要使用 He 初始化随机选择一个值，将所有权重初始化为同一个值是否可以接受？
3. 将偏置项初始化为 0 是否可以接受？
4. 对于本章讨论的各种激活函数，分别应该在什么情况下使用？
5. 使用 SGD 优化器时，如果把 `momentum` 超参数设置得过于接近 1（例如 0.99999），可能会发生什么？
6. 列出三种生成稀疏模型的方法。
7. Dropout 会减慢训练吗？会减慢推理（即对新实例进行预测）吗？MC dropout 又如何？
8. 练习在 CIFAR10 图像数据集上训练深度神经网络：

- a. 像第 10 章加载 Fashion MNIST 那样加载 CIFAR10，但使用 `torchvision.datasets.CIFAR10` 而不是 `FashionMNIST`。数据集包含 60,000 张 32×32 像素的彩色图像（50,000 张训练图像、10,000 张测试图像），共有 10 个类别。
- b. 构建一个 DNN，包含 20 个隐藏层，每层 100 个神经元（层数过多，但这正是练习目的）。使用 He 初始化和 Swish 激活函数（使用 `nn.SiLU`）。由于这是分类任务，需要为每个类别设置一个输出神经元。
- c. 使用 NAdam 优化和提前停止，在 CIFAR10 数据集上训练网络。每次改变模型架构或超参数，都要重新搜索合适的学习率。
- d. 添加 batch norm，比较学习曲线：是否比之前收敛更快？是否产生更好的模型？它如何影响训练速度？

- e. 尝试用 SELU 替换 batch norm，并进行必要调整，确保网络实现自归一化（即标准化输入特征、使用 Le Cun 初始化，并确保 DNN 只有一系列全连接层等）。
- f. 使用 alpha dropout 对模型进行正则化。然后不重新训练模型，使用 MC dropout，看看能否得到更好的准确率。
- g. 使用 1cycle 调度重新训练模型，看看它是否能提高训练速度和模型准确率。

这些练习的解答位于本章 notebook 的末尾：https://homl.info/colab-p。

11.11.1　参考文献

1. Xavier Glorot and Yoshua Bengio, “Understanding the Difficulty of Training Deep Feedforward Neural Networks,”Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (2010): 249-256.
2. 关于初始化的直观类比：如果把麦克风放大器的音量旋钮调得太小，听众听不见你的声音；调得太大，声音会饱和，听众也听不清。现在想象有一串这样的放大器：每个放大器都必须设置正确，最终输出才会清晰，而且声音经过每个放大器后都应保持相同振幅。
3. Kaiming He et al., “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification,”Proceedings of the 2015 IEEE International Conference on Computer Vision (2015): 1026-1034.
4. 自 2019 年起，PyTorch issue #18182 一直在讨论将权重初始化更新为当前最佳实践。
5. Andrew Saxe et al., “Exact Solutions to the Nonlinear Dynamics of Learning in Deep Linear Neural Networks,”ICLR (2014).
6. 参见本章前面对死亡神经元可能恢复的脚注。
7. Bing Xu et al., “Empirical Evaluation of Rectified Activations in Convolutional Networks,” arXiv preprint arXiv:1505.00853 (2015).
8. Djork-Arné Clevert et al., “Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs),”Proceedings of the International Conference on Learning Representations, arXiv preprint (2015).
9. Günter Klambauer et al., “Self-Normalizing Neural Networks,”Proceedings of the 31st International Conference on Neural Information Processing Systems (2017): 972-981.
10. Dan Hendrycks and Kevin Gimpel, “Gaussian Error Linear Units (GELUs),”arXiv preprint arXiv:1606.08415 (2016).
11. 凸函数是指曲线上任意两点之间的线段都不会落在曲线下方；单调函数则只增不减，或只减不增。

12. Prajit Ramachandran et al., “Searching for Activation Functions,”arXiv preprint arXiv:1710.05941 (2017).
13. Noam Shazeer, “GLU Variants Improve Transformer,”arXiv:2002.05202 (2020).
14. Yann Dauphin et al., “Language Modeling with Gated Convolutional Networks,”arXiv preprint arXiv:1612.08083 (2016).
15. Diganta Misra, “Mish: A Self Regularized Non-Monotonic Activation Function,”arXiv preprint arXiv:1908.08681 (2019).
16. So et al., “Primer: Searching for Efficient Transformers for Language Modeling,”arXiv preprint arXiv:2109.08668 (2021).
17. Sergey Ioffe and Christian Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,”Proceedings of the 32nd International Conference on Machine Learning (2015): 448-456.
18. Jimmy Lei Ba et al., “Layer Normalization,”arXiv preprint arXiv:1607.06450 (2016).
19. Razvan Pascanu et al., “On the Difficulty of Training Recurrent Neural Networks,”Proceedings of the 30th International Conference on Machine Learning (2013): 1310-1318.
20. Boris T. Polyak, “Some Methods of Speeding Up the Convergence of Iteration Methods,” USSR Computational Mathematics and Mathematical Physics 4, no. 5 (1964): 1-17.
21. Yurii Nesterov, “A Method for Unconstrained Convex Minimization Problem with the Rate of Convergence O(1/k²),”Doklady AN USSR 269 (1983): 543-547.
22. John Duchi et al., “Adaptive Subgradient Methods for Online Learning and Stochastic Optimization,”Journal of Machine Learning Research 12 (2011): 2121-2159.
23. 该算法由 Geoffrey Hinton 和 Tijmen Tieleman 于 2012 年创建，并由 Geoffrey Hinton 在 Coursera 神经网络课程中讲授。由于作者没有撰写介绍该算法的论文，研究人员常引用“第 6 讲 e 部分第 29 张幻灯片”。
24. Diederik P. Kingma and Jimmy Ba, “Adam: A Method for Stochastic Optimization,”arXiv preprint arXiv:1412.6980 (2014).
25. Timothy Dozat, “Incorporating Nesterov Momentum into Adam,”(2016).
26. Ilya Loshchilov and Frank Hutter, “Decoupled Weight Decay Regularization,”arXiv preprint arXiv:1711.05101 (2017).
27. Ashia C. Wilson et al., “The Marginal Value of Adaptive Gradient Methods in Machine Learning,”Advances in Neural Information Processing Systems 30 (2017): 4148-4158.
28. 参见前面对 Jacobian 和 Hessian 矩阵的脚注。
29. V. Gupta et al., “Shampoo: Preconditioned Stochastic Tensor Optimization,”arXiv preprint arXiv:1802.09568 (2018).

30. Ilya Loshchilov and Frank Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” arXiv preprint arXiv:1608.03983 (2016).
31. Leslie N. Smith, “A Disciplined Approach to Neural Network Hyper-Parameters: Part 1–Learning Rate, Batch Size, Momentum, and Weight Decay,”arXiv preprint arXiv:1803.09820 (2018).
32. Geoffrey E. Hinton et al., “Improving Neural Networks by Preventing Co-Adaptation of Feature Detectors,”arXiv preprint arXiv:1207.0580 (2012).
33. Nitish Srivastava et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,”Journal of Machine Learning Research 15 (2014): 1929-1958.
34. Yarin Gal and Zoubin Ghahramani, “Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning,”Proceedings of the 33rd International Conference on Machine Learning (2016): 1050-1059.
35. 具体而言，他们证明，训练 dropout 网络在数学上等价于在一种特定概率模型（深度高斯过程）中进行近似贝叶斯推断。

第 12 章　使用卷积神经网络进行深度计算机视觉

Deep Computer Vision Using Convolutional Neural Networks

虽然 IBM 的 Deep Blue 超级计算机早在 1996 年就击败了国际象棋世界冠军 Garry Kasparov，但直到相当近的时期，计算机才开始能够可靠地完成一些对人类来说看似微不足道的任务，例如在图片中检测小狗，或识别语音。为什么这些任务对人类来说如此轻松？答案在于，感知大多发生在意识之外，发生在大脑中专门负责视觉、听觉和其他感觉的模块内。当感觉信息进入我们的意识时，它已经被赋予了高级特征。例如，当你看到一张可爱小狗的照片时，你无法选择不去看见小狗，也无法不注意到它的可爱之处。你也无法解释自己是如何识别出这只可爱小狗的；对你来说，这一切显而易见。因此，我们不能完全相信主观体验：感知一点也不简单。要理解感知，就必须研究感觉模块的工作方式。

卷积神经网络（convolutional neural network，CNN）源于对大脑视觉皮层的研究，自 20 世纪 80 年代起就被用于计算机图像识别。在过去 15 年中，得益于计算能力的提高、可用训练数据的增加，以及第 11 章介绍的深度网络训练技巧，CNN 已经在一些复杂视觉任务上达到了超越人类的表现。它们为图像搜索服务、自动驾驶汽车、自动视频分类系统等提供了动力。

此外，CNN 并不局限于视觉感知，在语音识别和自然语言处理等许多其他任务中也很成功。不过，本章暂时聚焦于视觉应用。

本章将探讨 CNN 的起源、其构件的形式，以及如何使用 PyTorch 实现它们。随后，我们将讨论一些优秀的 CNN 架构和其他视觉任务，包括目标检测（在图像中分类多个物体并在其周围放置边界框）以及语义分割（根据每个像素所属物体的类别对像素进行分类）。

12.1　视觉皮层的结构（The Architecture of the Visual Cortex）

David H. Hubel 和 Torsten Wiesel 在 1958 年[1]和 1959 年[2]对猫进行了一系列实验，几年后又对猴子进行了实验[3]，从而对视觉皮层的结构获得了关键洞察（两位作者因这项工作获

[1] 见本章参考文献第 1 条。

[2] 见本章参考文献第 2 条。

[3] 见本章参考文献第 3 条。

得了 1981 年诺贝尔生理学或医学奖）。他们尤其证明，视觉皮层中的许多神经元具有很小的局部感受野，也就是说，它们只会对视觉场中有限区域内的视觉刺激作出反应（见图 12-1，其中用虚线圆表示了五个神经元的局部感受野）。不同神经元的感受野可能重叠，它们合在一起覆盖整个视觉场。

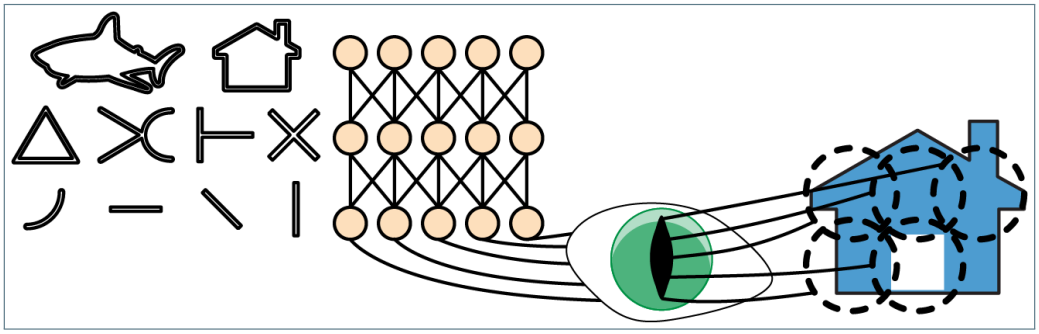

图 12-1: 视觉皮层中的生物神经元会对视觉场中称为感受野的小区域内的特定模式作出反应；随着视觉信号依次通过大脑模块，神经元会在更大的感受野中对更复杂的模式作出反应 (Biological neurons in the visual cortex respond to specific patterns in small regions of the visual field called receptive fields; as the visual signal makes its way through consecutive brain modules, neurons respond to more complex patterns in larger receptive fields)

此外，作者发现，一些神经元只会对水平线图像作出反应，另一些则只会对其他方向的线作出反应（两个神经元可能具有相同的感受野，却会对不同方向的线作出反应）。他们还注意到，一些神经元具有更大的感受野，并会对由低级模式组合而成的更复杂模式作出反应。这些观察结果引出了一个思想：高级神经元建立在邻近低级神经元输出的基础上（在图 12-1 中可以看到，每个神经元只与上一层中附近的神经元相连）。这种强大的架构能够在视觉场的任意区域检测各种复杂模式。

这些对视觉皮层的研究启发了 1980 年提出的 neocognitron[4]，它逐渐演变成我们今天所说的卷积神经网络。一个重要里程碑是 Yann Le Cun 等人在 1998 年发表的论文[5]，该论文提出了著名的 LeNet-5 架构。LeNet-5 后来被银行广泛用于识别支票上的手写数字。这个架构包含一些你已经熟悉的构件，例如全连接层和 sigmoid 激活函数，同时还引入了两个新的构件：卷积层和池化层。下面来看看它们。

> 注意：为什么不直接使用全连接层组成的深度神经网络来完成图像识别？遗憾的是，虽然这种方法对小图像（例如 Fashion MNIST）效果不错，但面对较大图像时会因为所需参数数量巨大而失效。例如，一张 100 × 100 像素的图像有 10,000 个像素；如果第一层只有 1,000 个神经元（这已经严重限制了传递给下一层的信息量），仅第一层就需要 1,000 万条连接。CNN 使用部分连接层和权重共享解决了这个问题。

[4] 见本章参考文献第 4 条。

[5] 见本章参考文献第 5 条。

12.2　卷积层（Convolutional Layers）

CNN 最重要的构件是卷积层：第一卷积层中的神经元并不会像前几章讨论的层那样连接到输入图像的每个像素，而只连接到其感受野内的像素（见图 12-2）。同样，第二卷积层中的每个神经元也只连接到第一层中一个小矩形区域内的神经元。这种架构使网络能够在第一个隐藏层关注小型低级特征，然后在下一个隐藏层把它们组合成更大的高级特征，以此类推。这种层次结构非常适合处理现实图像中常见的复合物体，这也是 CNN 在图像识别中表现出色的原因之一。

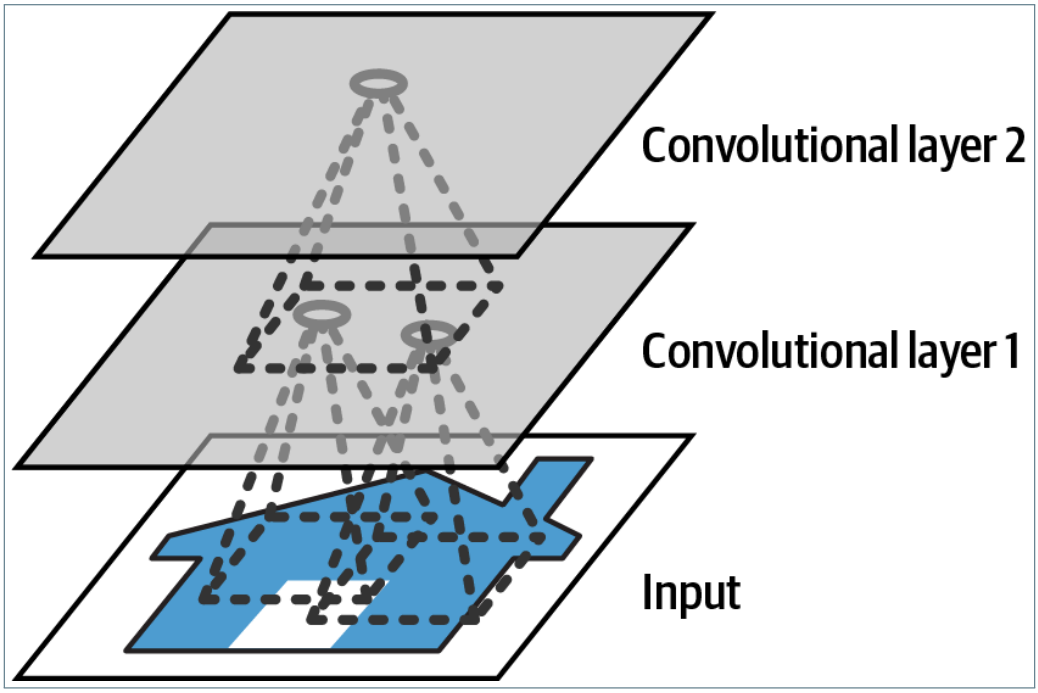

图 12-2: 具有矩形局部感受野的 CNN 层（CNN layers with rectangular local receptive fields）

> 注意：到目前为止，我们看到的多层神经网络，其层都由一长行神经元组成；在把输入图像送入神经网络前，必须将其展平为一维。在 CNN 中，每层都用二维形式表示，这使神经元与对应输入的匹配更加容易。

给定层中位于第 `i` 行、第 `j` 列的神经元，会连接到上一层中位于第 `i` 至 `i + f_h - 1` 行、第 `j` 至 `j + f_w - 1` 列的神经元输出，其中 `f_h` 和 `f_w` 是感受野的高度和宽度（见图 12-3）。为了使某层的高度和宽度与上一层相同，通常会像图中所示那样在输入周围补零，这称为零填充（zero padding）。

也可以通过拉开感受野之间的间距，把一个较大的输入层连接到小得多的层，如图 12-4 所示。这会大幅降低模型的计算复杂度。从一个感受野移动到下一个感受野时，水平方向或垂直方向的步进大小称为步幅（stride）。在图中，一个 5 × 7 的输入层（加上零填充）连接

到一个 3 × 4 的层，使用 3 × 3 的感受野，步幅为 2。在这个例子中，两个方向的步幅相同，这通常是常见情况，但也有例外。上层中位于第 `i` 行、第 `j` 列的神经元，会连接到上一层中第 `i × s_h` 至 `i × s_h + f_h − 1` 行、第 `j × s_w` 至 `j × s_w + f_w − 1` 列的神经元，其中 `s_h` 和 `s_w` 分别是垂直和水平步幅。

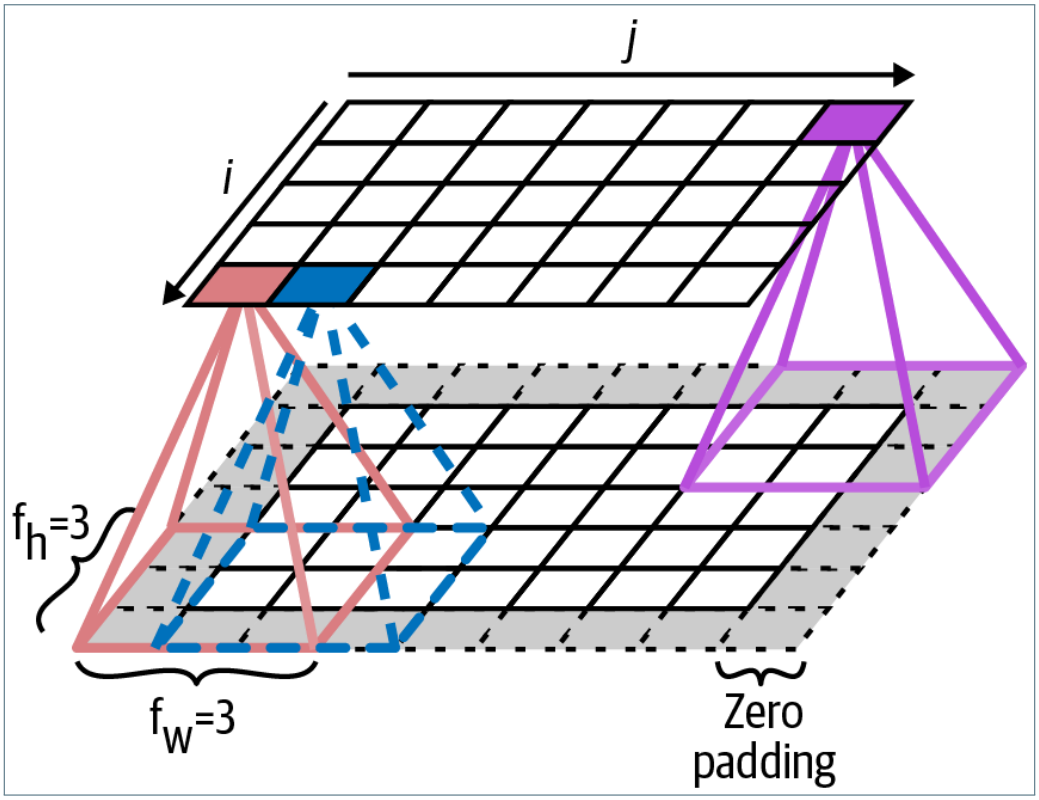

图 12-3: 层之间的连接与零填充（Connections between layers and zero padding）

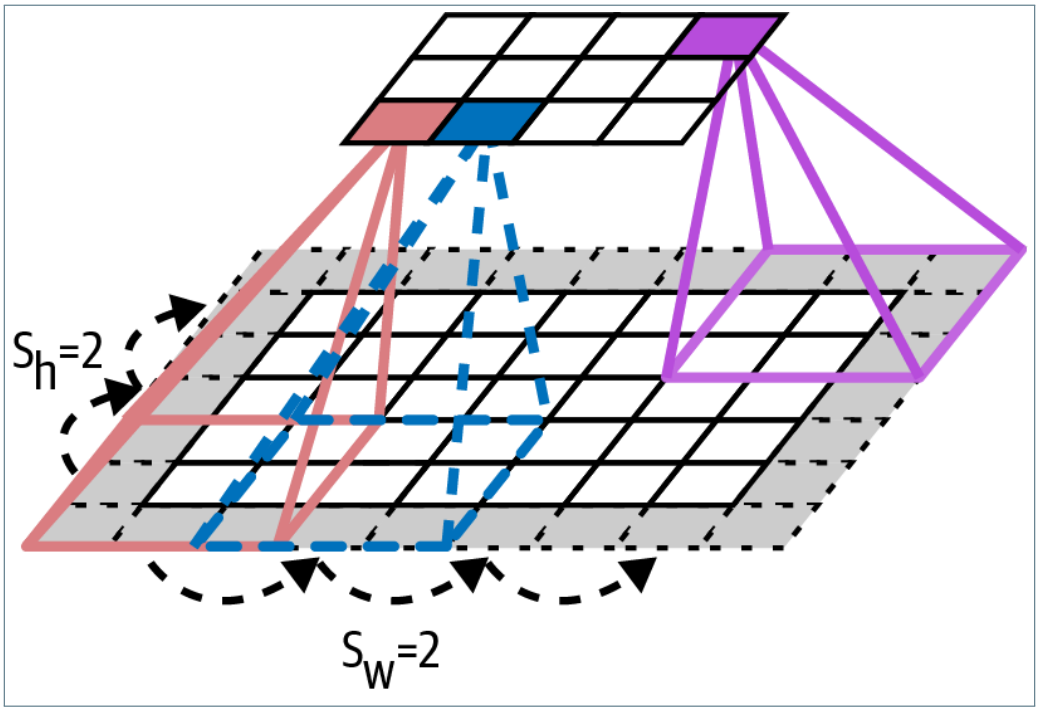

图 12-4: 使用步幅 2 降低维度（Reducing dimensionality using a stride of 2）

12.2.1　滤波器（Filters）

神经元的权重可以表示为一张与感受野大小相同的小图像。例如，图 12-5 展示了两组可能的权重，它们称为滤波器（filter）、卷积核（convolution kernel），或者简称为核（kernel）。第一个滤波器表示为中间有一条垂直白线的黑色方块（这是一个全部为 0、只有中央列为 1 的 7 × 7 矩阵）；使用这些权重的神经元会忽略感受野中的所有内容，只关注中央垂直线，因为除了中央垂直线外，所有输入都会乘以 0。第二个滤波器是中间有一条水平白线的黑色方块；使用这些权重的神经元会忽略感受野中的其他内容，只关注中央水平线。

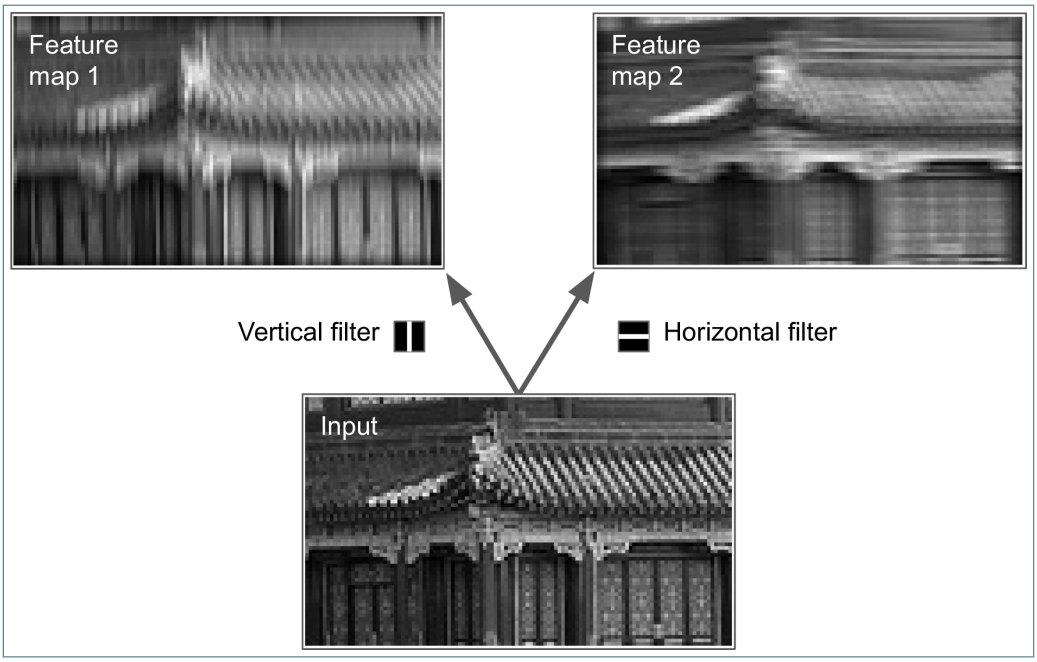

图 12-5: 使用两个不同滤波器得到两个特征图（Applying two different filters to get two feature maps）

> 注意：在深度学习中，我们经常构建一个接收原始输入并产生最终输出的单一模型，这称为端到端学习（end-to-end learning）。相比之下，经典视觉系统通常会把系统拆分为一系列专门模块。

如果某层中的所有神经元都使用同一个垂直线滤波器（以及同一个偏置项），并将图 12-5 底部的输入图像送入网络，那么该层会输出左上角的图像。可以看到，垂直白线被增强，其余部分变得模糊。类似地，右上角的图像是所有神经元使用同一个水平线滤波器时得到的结果；水平白线被增强，其余部分则被模糊。于是，使用同一个滤波器的一层神经元会输出一张特征图（feature map），突出显示图像中最能激活该滤波器的区域。不过，你不必手工定义滤波器：在训练期间，卷积层会自动学习对任务最有用的滤波器，而上层会学习如何将这些滤波器组合成更复杂的模式。

12.2.2　堆叠多个特征图（Stacking Multiple Feature Maps）

为简单起见，前面一直把每个卷积层表示为二维层；但实际上，一个卷积层包含多个滤波器（数量由你决定），并且每个滤波器输出一张特征图，因此输出更准确地说是三维的（见图 12-6）。

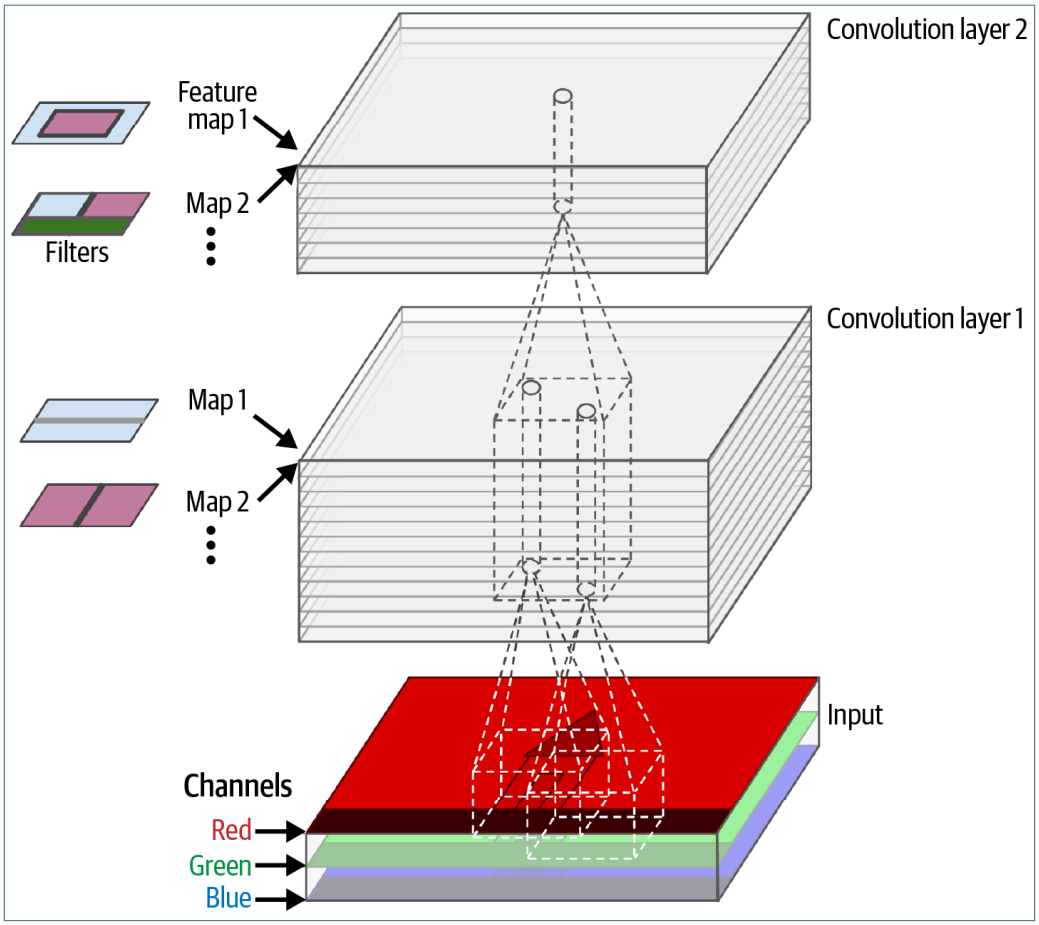

图 12-6: 两个卷积层各自使用多个滤波器（卷积核）处理一个包含三个颜色通道的彩色图像；每个卷积层为每个滤波器输出一张特征图（Two convolutional layers with multiple filters each (kernels), processing a color image with three color channels; each convolutional layer outputs one feature map per filter）

每张特征图中的每个像素对应一个神经元；给定特征图中的所有神经元共享相同的参数（即同一个核和偏置项）。不同特征图中的神经元使用不同的参数。

神经元的感受野与前面描述的相同，但会横跨上一层的所有特征图。简而言之，卷积层会同时对输入应用多个可训练滤波器，因此能够在输入的任意位置检测多种特征。

> 注意：特征图中的所有神经元共享参数，这会大幅减少模型中的参数数量。一旦 CNN 学会在一个位置识别某个模式，它就能在其他位置识别同一模式。相反，全连接神经网络一旦学会在某个位置识别某个模式，就只能在那个特定位置识别它。

输入图像也由多个子层组成，每个颜色通道对应一个子层。如第 8 章所述，通常有三个通道：红、绿、蓝（RGB）。灰度图像只有一个通道，但某些图像可能有更多通道，例如捕获额外光谱（如红外线）的卫星图像。

具体来说，给定卷积层 `l` 中特征图 `k` 的第 `i` 行、第 `j` 列神经元，它会连接到上一层 `l - 1` 中所有特征图内位于第 `i × s_h` 至 `i × s_h + f_h - 1` 行、第 `j × s_w` 至 `j × s_w + f_w - 1` 列的神经元输出。注意，在同一层中，位于同一行 `i` 和同一列 `j`、但属于不同特征图的神经元，会连接到上一层中完全相同的神经元。

公式 12-1 总结了前面的说明，它展示了如何计算卷积层中给定神经元的输出。由于包含许多不同的索引，公式看起来有些复杂，但本质上只是计算所有输入的加权和，再加上偏置项。

$$z_{i,j,k} = b_k + \sum_{u=0}^{f_h-1} \sum_{v=0}^{f_w-1} \sum_{k'=0}^{f_{n'}-1} x_{i',j',k'} \times w_{u,v,k',k} \qquad (12\text{-}1)$$
$$i' = i \times s_h + u, \qquad j' = j \times s_w + v$$

公式 12-1　计算卷积层中神经元的输出（Computing the output of a neuron in a convolutional layer）

在这个公式中：

- `z_(i,j,k)` 是卷积层（第 `l` 层）中特征图 `k` 的第 `i` 行、第 `j` 列神经元的输出。
- `s_h` 和 `s_w` 分别是垂直和水平步幅，`f_h` 和 `f_w` 分别是感受野的高度和宽度，`f_n'` 是上一层（第 `l - 1` 层）中的特征图数量。
- `x_(i',j',k')` 是上一层中第 `i'` 行、第 `j'` 列、特征图 `k'` 的神经元输出；如果上一层就是输入层，`k'` 表示输入通道。
- `b_k` 是第 `l` 层中特征图 `k` 的偏置项。可以把它看作调节特征图整体亮度的旋钮。
- `w_(u,v,k',k)` 是第 `l` 层特征图 `k` 中任意神经元与其感受野内相对位置为第 `u` 行、第 `v` 列、位于特征图 `k'` 中的输入之间的连接权重。

下面使用 PyTorch 创建并使用卷积层。

12.2.3　使用 PyTorch 实现卷积层（Implementing Convolutional Layers with PyTorch）

首先，使用 Scikit-Learn 的 `load_sample_images()` 函数加载几张示例图像。第一张图像是中国的佛教香塔，第二张是美丽的大丽花。这些图像表示为由 NumPy 无符号字节数组组成的 Python 列表，因此先把它们堆叠成一个 NumPy 数组，再将其转换为 32 位浮点张量，并把像素值从 0-255 缩放到 0-1：

```
import numpy as np
import torch
from sklearn.datasets import load_sample_images
sample_images = np.stack(load_sample_images()["images"])
sample_images = torch.tensor(sample_images, dtype=torch.float32) / 255
```

查看这个张量的形状：

```
>>> sample_images.shape
torch.Size([2, 427, 640, 3])
```

这里有两张图像，高度都是 427 像素、宽度都是 640 像素，并且包含红、绿、蓝三个颜色通道。正如第 10 章所见，PyTorch 要求通道维位于高度和宽度维之前，而不是之后，因此需要使用 **permute()** 方法重新排列维度：

```
>>> sample_images_permuted = sample_images.permute(0, 3, 1, 2)
>>> sample_images_permuted.shape
torch.Size([2, 3, 427, 640])
```

再使用 TorchVision 的 **CenterCrop** 类对图像进行中心裁剪：

```
>>> import torchvision
>>> import torchvision.transforms.v2 as T
>>> cropped_images = T.CenterCrop((70, 120))(sample_images_permuted)
>>> cropped_images.shape
torch.Size([2, 3, 70, 120])
```

现在创建一个二维卷积层，把这些裁剪后的图像送入其中。PyTorch 提供了 **nn.Conv2d** 层；在底层，该层依赖 **torch.nn.functional.conv2d()** 函数。创建一个包含 32 个滤波器的卷积层，每个滤波器大小为 7 × 7（使用 **kernel_size=7**，等价于 **kernel_size=(7, 7)**），并将这个层应用于包含两张图像的小批量：

```
import torch.nn as nn
torch.manual_seed(42)
conv_layer = nn.Conv2d(in_channels=3, out_channels=32, kernel_size=7)
fmaps = conv_layer(cropped_images)
```

> 注意：我们说二维卷积层时，“二维”指的是空间维度的数量，即高度和宽度。但如你所见，该层接收四维输入：另外两个维度是批大小（第一个维度）和通道数（第二个维度）。

查看输出形状：

```
>>> fmaps.shape
torch.Size([2, 32, 64, 114])
```

输出形状与输入形状类似，但有两个主要差异。第一，输出有 32 个通道而不是 3 个，这是因为我们设置了 **out_channels=32**，因此得到了 32 张输出特征图：不再表示每个位置处红、绿、蓝的强度，而是表示每个位置处各个特征的强度。第二，高度和宽度都缩小了 6 个像素。这是因为 **nn.Conv2d** 默认不使用零填充，所以输出特征图的边缘会根据滤波器大小损失一些像素。在本例中，核大小为 7，因此水平方向和垂直方向都损失 6 个像素（即每侧 3 个像素）。

> 警告：默认情况下，**padding** 超参数为 0，也就是关闭填充。奇怪的是，这也称为 valid 填充，因为每个神经元的感受野都严格位于输入的有效位置内，不会越界。你也可以设置 **padding="valid"**，它等价于 **padding=0**。这不是 PyTorch 特有的命名习惯；这种令人困惑的术语被广泛使用。

如果把 **padding="same"**，输入四周就会补上足够的零，使输出特征图与输入具有相同大小，因此才称为这个选项：

```
>>> conv_layer = nn.Conv2d(in_channels=3, out_channels=32, kernel_size=7,
...                        padding="same")
...
>>> fmaps = conv_layer(cropped_images)
>>> fmaps.shape
torch.Size([2, 32, 70, 120])
```

图 12-7 展示了这两种填充选项。为了简单起见，图中只显示水平方向，垂直方向的逻辑完全相同。

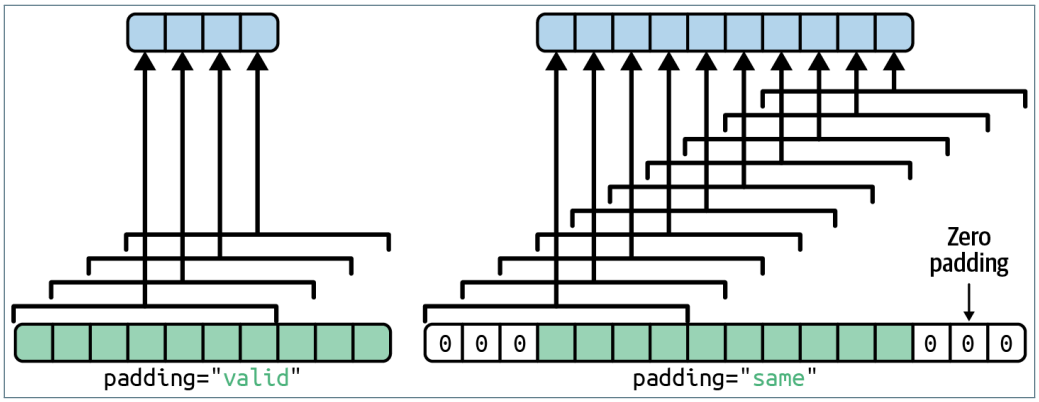

图 12-7: 两种不同的填充选项，步幅为 1、核大小为 7（Two different padding options, with stride=1 and kernel_size=7）

如果步幅大于 1（任一方向都如此），输出大小会比输入小得多。例如，假设输入大小为 70 × 120，将 **stride=2**（等价于 **stride=(2, 2)**）、**padding=3**、**kernel_size=7**，则输出特征图大小为 35 × 60，在垂直和水平方向上都减半。也可以把 **padding="same"** 与 **stride=2** 结合使用，但 PyTorch 目前不支持这种组合，会抛出异常。事实上，**padding="same"** 只有在 **stride=1** 时才有效。

默认情况下，步幅在水平方向和垂直方向上相同，但你也可以为两个方向设置不同的步幅。例如，**stride=(2, 1)** 会将高度减半，同时保持宽度不变。类似地，可以对 **kernel_size** 和 **padding** 使用元组，为两个方向设置不同的值。对于填充，还可以使用每个方向的整数元组，例如 **padding=(1, 2)**，也可以使用字符串**"valid"** 或**"same"**；后两者要求 **stride=1**。

nn.Conv2d 层的权重存储在 **weight** 属性中：

```
>>> conv_layer.weight.shape
torch.Size([32, 3, 7, 7])
```

这是一个包含 32 个滤波器的张量，每个滤波器有 3 个通道，大小为 7 × 7。注意 PyTorch 使用的维度顺序是输出通道数、输入通道数、高度、宽度。偏置项存储在 **bias** 属性中：

```
>>> conv_layer.bias.shape
torch.Size([32])
```

12.3　池化层（Pooling Layers）

池化层的目标是对输入图像进行子采样（即缩小输入图像），从而降低计算负担、内存使用量和参数数量（也就降低过拟合风险）。

与卷积层一样，池化层中的每个神经元只连接到上一层中有限数量神经元的输出，这些神经元位于一个小矩形感受野内。你需要像前面一样定义感受野大小、步幅和填充类型。不过，池化神经元没有权重或偏置；它只使用最大值或均值等聚合函数聚合输入。图 12-9 展示了最大池化层，这是最常见的池化类型。在这个例子中，使用 2 × 2 池化核、步幅为 2 且不填充。每个感受野内只有最大的输入值能够传到下一层，其他输入都会被丢弃。例如，在图 12-9 左下角的感受野中，输入值为 1、5、3 和 2，因此只有最大值 5 会传递到下一层。由于步幅为 2，输出图像的高度和宽度都是输入图像的一半（因为不填充，所以向下取整）。

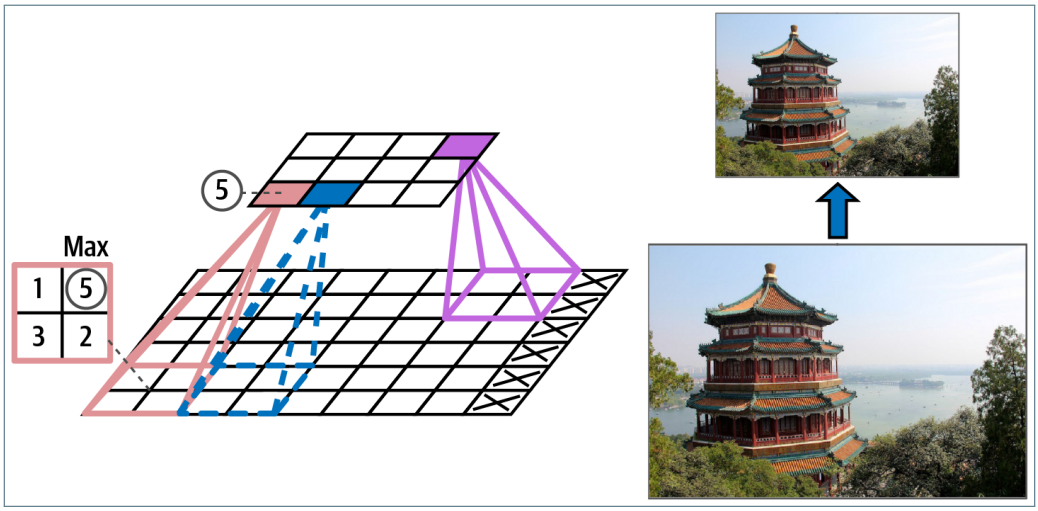

图 12-8: 最大池化层（2 × 2 池化核、步幅 2、不填充）（Max pooling layer (2 × 2 pooling kernel, stride 2, no padding)）

> 注意：池化层通常独立处理每个输入通道，因此输出深度（即通道数）与输入深度相同。

除了减少计算、内存使用和参数数量之外，最大池化层还会对小幅平移引入一定程度的不变性，如图 12-10 所示。假设明亮像素的值低于暗像素，并考虑三张图像 A、B、C 经过使用 2 × 2 核、步幅为 2 的最大池化层。图像 B 和 C 分别是图像 A 向右平移一个像素和两个像素后的结果。可以看到，图像 A 和 B 的最大池化输出完全相同，这就是平移不变性的含义。对于图像 C，输出不同了：它向右移动了一个像素，但仍保留了 50% 的不变性。每隔几层在 CNN 中插入一个最大池化层，就能在更大尺度上获得一定程度的平移不变性。此外，最大池化还提供少量旋转不变性和轻微的尺度不变性。这类不变性虽然有限，但在预测不应依赖这些细节的场景（例如分类任务）中很有用。

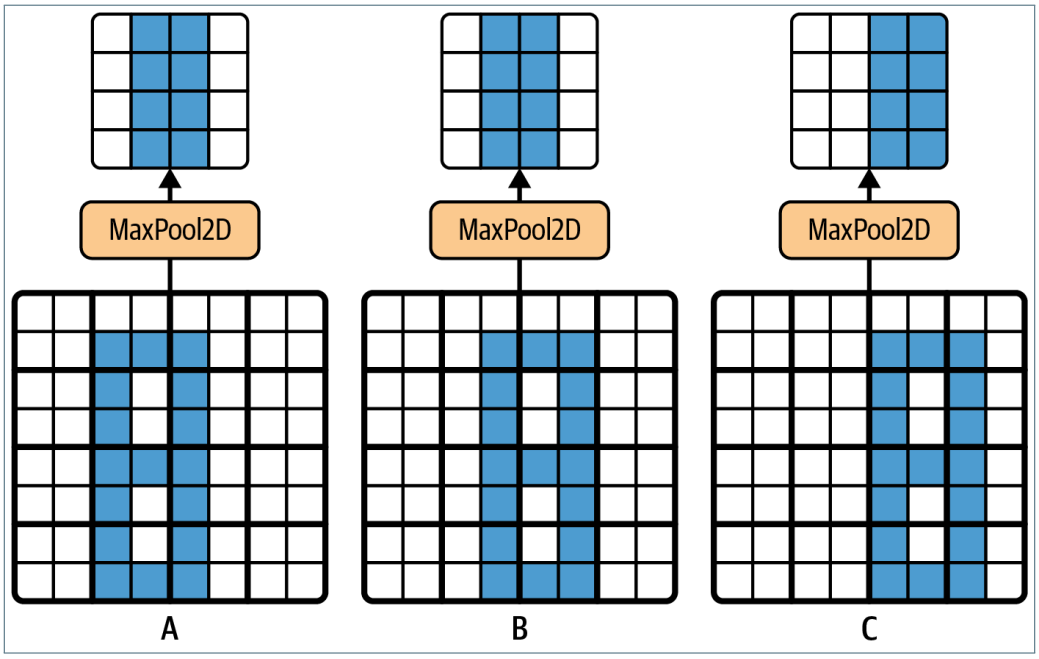

图 12-9: 对小幅平移的不变性（Invariance to small translations）

不过，最大池化也有缺点。它显然具有破坏性：即使使用很小的 2 × 2 核和步幅 2，输出在两个方向上也会缩小一半（因此面积缩小为四分之一），丢弃 75% 的输入值。而且在某些应用中，不变性并不是我们想要的性质。

以语义分割为例（该任务是根据像素所属物体对图像中的每个像素进行分类，本章稍后会介绍）：如果输入图像向右平移一个像素，输出也显然应该向右平移一个像素。此时目标是等变性，而不是不变性：输入发生小变化时，输出也应该发生相应的小变化。

12.4　使用 PyTorch 实现池化层（Implementing Pooling Layers with PyTorch）

下面的代码创建一个使用 2 × 2 核的 `nn.MaxPool2d` 层。步幅默认等于核大小，因此这个层在水平和垂直方向上的步幅都是 2。默认使用 `padding=0`，即 valid 填充：

```
max_pool = nn.MaxPool2d(kernel_size=2)
```

要创建平均池化层，只需使用 `nn.AvgPool2d` 替代 `nn.MaxPool2d`。它的工作方式与最大池化完全相同，只是计算均值而不是最大值。平均池化过去很流行，但现在人们主要使用最大池化，因为它通常表现更好。这可能有些令人意外，因为计算均值通常比计算最大值损失更少的信息。但另一方面，最大池化只保留最强特征，去除所有无意义的特征，因此下一层得到的是更干净的信号。此外，最大池化提供的平移不变性强于平均池化，计算量也略少。

注意，最大池化和平均池化也可以沿深度维度，而不是空间维度执行，尽管这并不常见。这样可以使 CNN 学会对各种特征保持不变。例如，可以学习多个滤波器，每个滤波器检测同一模式的不同旋转（如手写数字；见图 12-11），深度方向最大池化层则可以确保无论旋转角度如何，输出都相同。类似地，CNN 还可以学习对粗细、亮度、倾斜、颜色等因素保持不变。

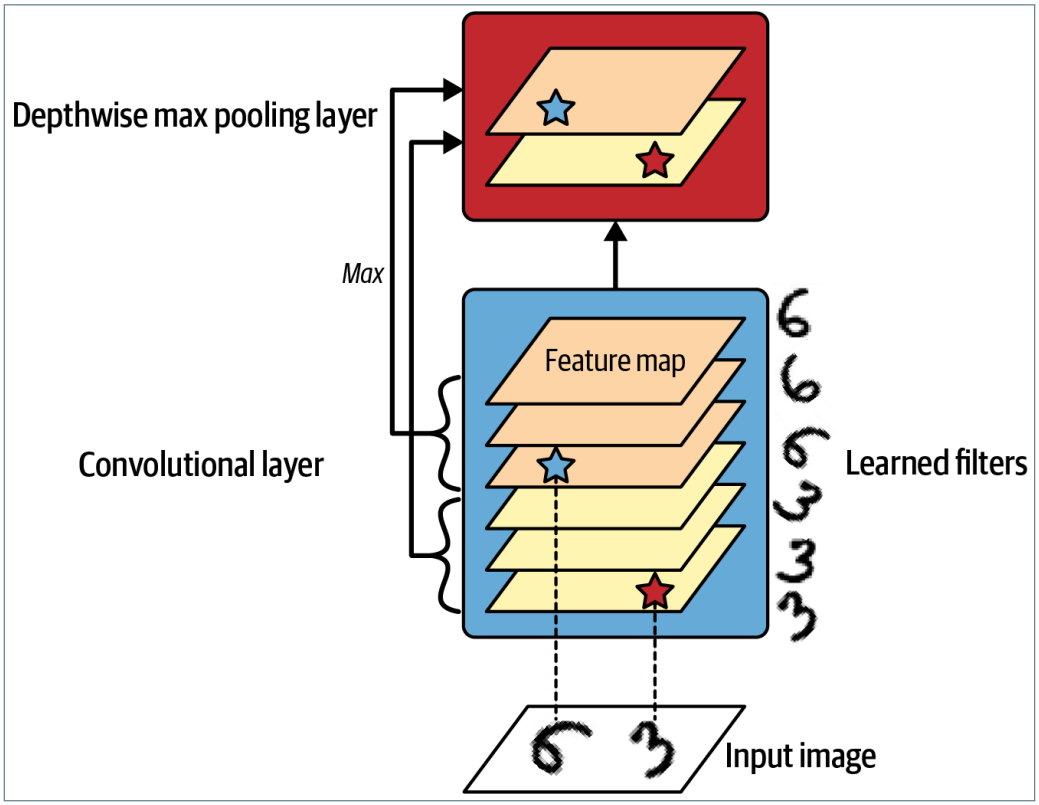

图 12-10: 深度方向最大池化可以帮助 CNN 学习不变性（这里是不受旋转影响）(Depth-wise max pooling can help the CNN learn to be invariant (to rotation in this case))

PyTorch 没有提供深度方向最大池化层，但可以基于 `torch.nn.functional.max_pool1d()` 函数实现自定义模块：

```
import torch.nn.functional as F

class DepthPool(torch.nn.Module):
    def __init__(self, kernel_size, stride=None, padding=0):
        super().__init__()
        self.kernel_size = kernel_size
        self.stride = stride if stride is not None else kernel_size
        self.padding = padding
    def forward(self, inputs):
        batch, channels, height, width = inputs.shape
```

```
        Z = inputs.view(batch, channels, height * width)  # merge spatial dims
        Z = Z.permute(0, 2, 1)  # switch spatial and channels dims
        Z = F.max_pool1d(Z, kernel_size=self.kernel_size, stride=self.stride,
                         padding=self.padding)  # compute max pool
        Z = Z.permute(0, 2, 1)  # switch back spatial and channels dims
        return Z.view(batch, -1, height, width)  # unmerge spatial dims
```

例如，假设输入批次包含两张 70 × 120 的图像，每张图像有 32 个通道（输入形状为 `[2, 32, 70, 120]`），并使用 `kernel_size=4`、默认步幅（等于 `kernel_size`）和 `padding=0`：

- `forward()` 方法首先合并空间维度，得到形状为 `[2, 32, 8400]` 的张量，因为 `70 × 120 = 8,400`。
- 接着交换最后两个维度，得到 `[2, 8400, 32]`。
- 然后使用 `max_pool1d()` 沿最后一个维度执行最大池化，而该维度对应原始的 32 个通道。由于 `kernel_size` 和步幅都为 4，且不填充，最后一个维度缩小为原来的四分之一，得到 `[2, 8400, 8]`。
- 函数再次交换最后两个维度，得到 `[2, 8, 8400]`。
- 最后拆分空间维度，得到最终形状 `[2, 8, 50, 100]`。你可以验证这个输出正是我们想要的结果。

现代架构中经常见到的另一种池化层是全局平均池化层。它的工作方式很不一样：只计算每张完整特征图的均值。因此，它为每个特征图和每个实例输出一个数。虽然这当然非常具有破坏性（特征图中的大部分信息都会丢失），但在输出层之前可能很有用，稍后会看到。

要创建这样的层，一种方法是使用普通的 `nn.AvgPool2d` 层，并将核大小设为与输入相同的大小。但这并不方便，因为需要提前知道输入的确切尺寸。更简单的方案是使用 `nn.AdaptiveAvgPool2d`，它允许指定输出的空间尺寸；该层会自动调整核大小（使用相同步幅）以获得所需结果，必要时还会添加少量填充。将输出大小设为 1，就得到全局平均池化层：

```
global_avg_pool = nn.AdaptiveAvgPool2d(output_size=1)
output = global_avg_pool(cropped_images)
```

也可以直接使用 `torch.mean()` 得到相同输出：

```
output = cropped_images.mean(dim=(2, 3), keepdim=True)
```

现在已经掌握了创建卷积神经网络所需的全部构件。下面来看看如何把它们组装起来。

12.5　CNN 架构（CNN Architectures）

典型的 CNN 架构会堆叠几个卷积层（通常每个卷积层后面都跟一个 ReLU 层），然后是一个池化层；接着再堆叠几个卷积层和 ReLU，再接一个池化层，如此反复。图像在网络中前进时会越来越小，但由于卷积层的作用，特征图通常也会越来越深，也就是特征图数量越来越多（见图 12-12）。在这组层的顶部，会添加一个普通的前馈神经网络，由几个全连接层和 ReLU 组成；最后一层输出预测结果，例如输出估计类别概率的 softmax 层。

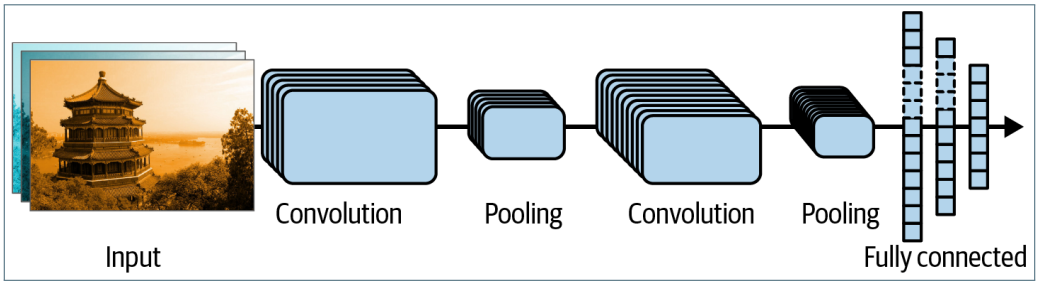

图 12-11: 典型 CNN 架构（Typical CNN architecture）

> 提示：相比使用一个 5 × 5 核的卷积层，通常更推荐堆叠两个使用 3 × 3 核的层：这样参数更少、计算量更低，而且通常表现更好。一个例外是第一个卷积层：它通常可以使用较大的核（如 5 × 5 或 7 × 7），并且步幅通常为 2 或更大。这样可以在不过多损失信息的前提下降低图像的空间维度；由于输入图像通常只有三个通道，计算成本也不会太高。

下面是使用 PyTorch 实现基础 CNN、处理第 9 章介绍的 Fashion MNIST 数据集的代码：

```
from functools import partial
DefaultConv2d = partial(nn.Conv2d, kernel_size=3, padding="same")
model = nn.Sequential(
    DefaultConv2d(in_channels=1, out_channels=64, kernel_size=7), nn.ReLU(),
    nn.MaxPool2d(kernel_size=2),
    DefaultConv2d(in_channels=64, out_channels=128), nn.ReLU(),
    DefaultConv2d(in_channels=128, out_channels=128), nn.ReLU(),
    nn.MaxPool2d(kernel_size=2),
    DefaultConv2d(in_channels=128, out_channels=256), nn.ReLU(),
    DefaultConv2d(in_channels=256, out_channels=256), nn.ReLU(),
    nn.MaxPool2d(kernel_size=2),
```

```
    nn.Flatten(),
    nn.Linear(in_features=2304, out_features=128), nn.ReLU(),
    nn.Dropout(0.5),
    nn.Linear(in_features=128, out_features=64), nn.ReLU(),
    nn.Dropout(0.5),
    nn.Linear(in_features=64, out_features=10),
).to(device)
```

下面逐行说明这段代码：

- 使用第 11 章介绍的 **functools.partial()** 函数定义 **DefaultConv2d**。它的行为类似于 **nn.Conv2d**，但带有不同的默认参数：核大小为 3，使用 **"same"** 填充。这避免了在整个模型中重复这些参数。
- 接下来创建 **nn.Sequential** 模型。第一层是一个具有 64 个较大滤波器（7 × 7）的 **DefaultConv2d**。它使用默认步幅 1，因为输入图像并不大。由于 Fashion MNIST 图像只有一个颜色通道（灰度图），所以设置 **in_channels=1**。
- 每个卷积层后面都跟着 ReLU 激活函数。
- 然后加入核大小为 2 的最大池化层，将每个空间维度缩小为原来的一半（必要时向下取整）。
- 接着重复两次相同结构：两个卷积层后跟一个最大池化层。对于更大的图像，可以再重复几次；重复次数是可以调节的超参数。
- 注意，朝输出层方向上升时，滤波器数量会加倍（最初为 64，然后为 128，再为 256）。这是合理的，因为低级特征通常比较少（例如小圆和水平线），但组合它们可以得到许多不同的高级特征。每经过一个池化层，空间维度减半，因此常见做法是把下一层的滤波器数量加倍，而不会导致参数数量、内存使用或计算负担爆炸。
- 接下来是由两个带 ReLU 激活的隐藏稠密层（**nn.Linear**）和一个稠密输出层组成的全连接网络。因为这是一个有 10 个类别的分类任务，输出层有 10 个单元。和第 10 章一样，我们省略 softmax 激活，使模型输出 logits 而不是概率，并使用 **nn.CrossEntropyLoss** 训练模型。由于第一稠密层需要每个实例的一维特征数组，因此必须在它之前展平输入。我们还加入两个丢弃率为 50% 的 dropout 层，以减少过拟合。

> 提示：第一个 **nn.Linear** 层有 2,304 个输入特征，这个数字从哪里来？Fashion MNIST 图像大小为 28 × 28，池化层依次将其缩小为 14 × 14、7 × 7，最后为 3 × 3。第一个 **nn.Linear** 层之前有 256 张特征图，因此输入特征数量为 **256 × 3 × 3 = 2,304**。有时计算特征数量并不容易；一种技巧是把 **in_features** 设置成任意值（例如 999），让训练崩溃，正确的特征数会显示在错误消息中：**RuntimeError: mat1 and mat2 shapes cannot be multiplied (32x2304 and 999x128)**。另一种方案是使用 **nn.**

> **`LazyLinear` 替代 `nn.Linear`**：它会在第一次被调用时创建权重矩阵，并自动把输入特征数量设为正确值。卷积层和 batch norm 层等其他层也有 lazy 变体。

如果在 Fashion MNIST 训练集上训练这个模型，测试集准确率应该接近 92%。可以使用第 10 章定义的 **`train()`** 和 **`evaluate_tm()`** 函数训练和评估它。这个结果并非当前最先进，但相当不错，也优于第 9 章中使用稠密网络取得的结果。

多年来，人们在这种基础架构上开发了许多变体，推动了该领域的惊人进展。衡量这类进展的一个好指标，是 ILSVRC ImageNet 挑战赛中的错误率。在这项比赛中，图像分类错误率在短短 6 年内从超过 26% 降至不到 2.3%。更准确地说，这里指的是 top-5 错误率，即模型最有信心的五个预测中不包含正确答案的测试图像比例。图像通常相当大（例如高度为 256 像素），共有 1,000 个类别，其中有些类别非常细微（试着区分 120 个犬种！）。观察历届获胜模型的演化，是理解 CNN 工作方式以及深度学习研究如何发展的好方法。

我们将先介绍经典的 LeNet-5 架构（1998），然后介绍几项 ILSVRC 挑战赛的获胜者：AlexNet（2012）、GoogLeNet（2014）、ResNet（2015）和 SENet（2017）。还会讨论一些其他架构，包括 VGGNet、Xception、ResNeXt、DenseNet、MobileNet、CSPNet、EfficientNet 和 ConvNeXt（第 16 章将讨论视觉 Transformer）。

12.5.1　LeNet-5

LeNet-5 架构[6]也许是最广为人知的 CNN 架构。前面提到过，它由 Yann Le Cun 于 1998 年创建，长期被用于手写数字识别（MNIST）。它由表 12-1 中所示的层组成。

表 12-1: LeNet-5 架构

层	类型	特征图数	尺寸	核大小	步幅	激活函数
输入（In）	输入	1	32 × 32	-	-	-
C1	卷积	6	28 × 28	5 × 5	1	tanh
S2	平均池化	6	14 × 14	2 × 2	2	tanh
C3	卷积	16	10 × 10	5 × 5	1	tanh
S4	平均池化	16	5 × 5	2 × 2	2	tanh
C5	卷积	120	1 × 1	5 × 5	1	tanh
F6	全连接	-	84	-	-	tanh
输出（Out）	全连接	-	10	-	-	RBF

可以看到，它与前面的 Fashion MNIST 模型很相似：都是卷积层和池化层的堆叠，后面接稠密网络。它与现代分类 CNN 的主要差别，可能在于激活函数：现在我们会使用 ReLU 替代 tanh，使用 softmax 替代 RBF（第 2 章介绍过）。还有一些无关紧要的小差异，如果你感兴趣，可以在本章 notebook（https://homl.info/colab-p）中查看。Yann Le Cun 的网站也有 LeNet-5 识别数字的精彩演示。

[6] 见本章参考文献第 8 条。

12.5.2　AlexNet

AlexNet CNN 架构[7]以明显优势赢得了 2012 年 ILSVRC 挑战赛：它的 top-5 错误率为 17%，而第二名只有 26%！AlexNet 由 Alex Krizhevsky（因此得名）、Ilya Sutskever 和 Geoffrey Hinton 开发。它与 LeNet-5 相似，只是规模更大、更深，而且它是第一个直接把卷积层堆叠在一起的架构，而不是在每个卷积层上面都堆叠一个池化层。表 12-2 展示了该架构。

表 12-2: AlexNet 架构

层	类型	特征图数	尺寸	核大小	步幅	填充
输入（In）	输入	3（RGB）	227 × 227	-	-	-
C1	卷积	96	55 × 55	11 × 11	4	valid
S2	最大池化	96	27 × 27	3 × 3	2	valid
C3	卷积	256	27 × 27	5 × 5	1	same
S4	最大池化	256	13 × 13	3 × 3	2	valid
C5	卷积	384	13 × 13	3 × 3	1	same
C6	卷积	384	13 × 13	3 × 3	1	same
C7	卷积	256	13 × 13	3 × 3	1	same
S8	最大池化	256	6 × 6	3 × 3	2	valid
F9	全连接	-	4,096	-	-	-
F10	全连接	-	4,096	-	-	-
输出（Out）	全连接	-	1,000	-	-	-

为了减少过拟合，作者使用了两种正则化技术。第一，在训练期间对 F9 和 F10 层的输出使用丢弃率为 50% 的 dropout（第 11 章介绍过）。第二，通过随机平移训练图像、水平翻转图像以及改变光照条件来执行数据增强。

12.5.3　数据增强（Data Augmentation）

数据增强通过为每个训练实例生成许多逼真的变体，人工增大训练集。这会减少过拟合，因此是一种正则化技术。生成的实例应该尽可能真实：理想情况下，给定一张增强训练集中的图像，人类应该无法判断它是否经过增强。简单地添加白噪声没有帮助；修改必须是可学习的，而白噪声不是。

例如，可以用不同幅度略微平移、旋转和缩放训练集中的每张图片，再把生成的图片加入训练集（见图 12-13）。可以使用 **`torchvision.transforms.v2`** 中的工具（如 **`RandomCrop`**、**`RandomRotation`** 等）完成这项工作。这会迫使模型对图片中物体的位置、方向和大小变化更加容忍。也可以使用变换调整颜色和对比度，以模拟不同的光照条件。通常还可以水平翻转图片（文本和其他不对称物体除外）。通过使用 **`Compose`** 组合这些变换，可以大幅增加训练集的大小。

[7] 见本章参考文献第 9 条。

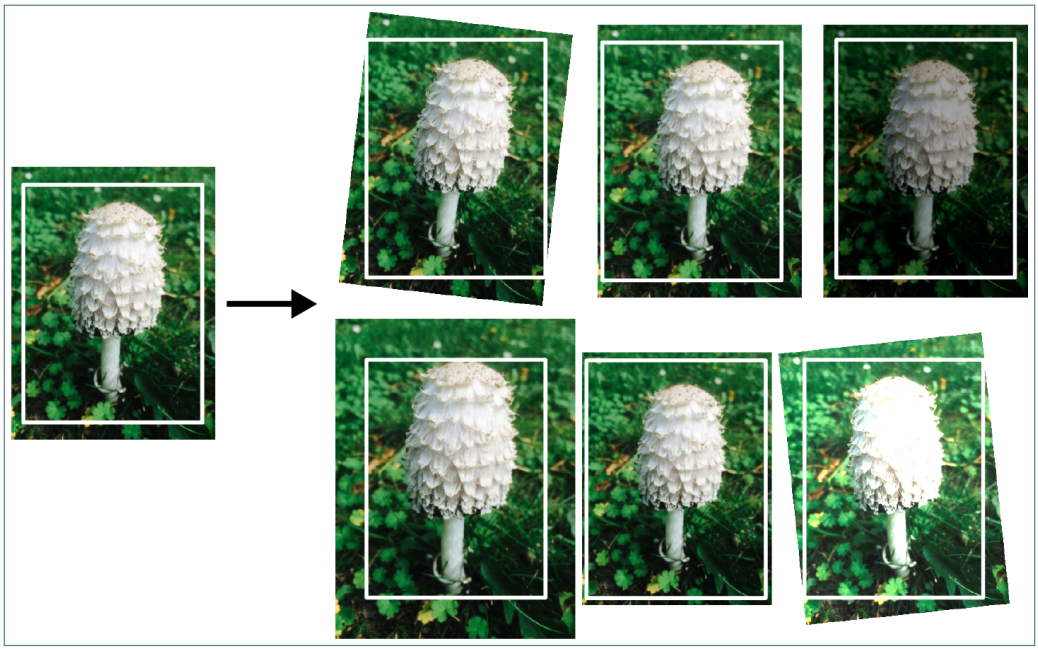

图 12-12: 从已有训练实例生成新的训练实例（Generating new training instances from existing ones）

数据增强在数据集不平衡时也很有用：可以为样本较少的类别生成更多样本。这称为合成少数类过采样技术（synthetic minority oversampling technique），简称 SMOTE。

最后，虽然数据增强通常只在训练期间使用，但也有一种例外，即测试时增强（test-time augmentation，TTA）：该技术会增强测试数据，并组合多个预测以提高准确率。例如，如果一张图像的三个增强版本被分类为公交车，而七个版本被分类为卡车，那么它很可能是卡车。

AlexNet 还使用了称为局部响应归一化（local response normalization，LRN）的正则化技术：最强烈激活的神经元会抑制相邻特征图中相同位置的其他神经元。这种竞争性激活在生物神经元中有所观察。它鼓励不同特征图进行专门化，使它们彼此分离，并迫使它们探索更广泛的特征，最终改善泛化能力。不过，这种技术后来大多被更简单、更高效的正则化技术取代，尤其是 batch normalization。

AlexNet 的一个变体 ZFNet[8]由 Matthew Zeiler 和 Rob Fergus 开发，并赢得了 2013 年 ILSVRC 挑战赛。它本质上是调整了少量超参数（特征图数量、核大小、步幅等）的 AlexNet。

12.5.4　GoogLeNet

GoogLeNet 架构由 Google Research 的 Christian Szegedy 等人开发[9]，并在 2014 年 ILSVRC 挑战赛中把 top-5 错误率降到了 7% 以下。它取得出色表现的一个重要原因，是网络

[8] 见本章参考文献第 10 条。

[9] 见本章参考文献第 11 条。

比之前的 CNN 深得多（见图 12-15）。这得益于一种称为 inception 模块[10]的子网络，它能比此前的架构更加高效地使用参数：GoogLeNet 的参数量实际上只有 AlexNet 的十分之一左右（约 600 万，而不是 6,000 万）。

图 12-14 展示了一个 inception 模块的架构。符号“3 × 3 + 1(S)”表示该层使用 3 × 3 的核、步幅 1 和 `same` 填充。输入信号首先被送入四个并行的层。所有卷积层都使用 ReLU 激活函数。注意，顶部的卷积层使用了不同大小的核（1 × 1、3 × 3 和 5 × 5），从而能够捕获不同尺度的模式。此外，每一层都使用步幅 1 和 `same` 填充（包括最大池化层），所以它们的输出在高度和宽度上都与输入相同。这使得最终可以沿深度维连接所有输出，也就是连接上方四个卷积层输出的多张特征图。可以使用 `torch.cat()` 函数并设置 `dim=1` 来实现。

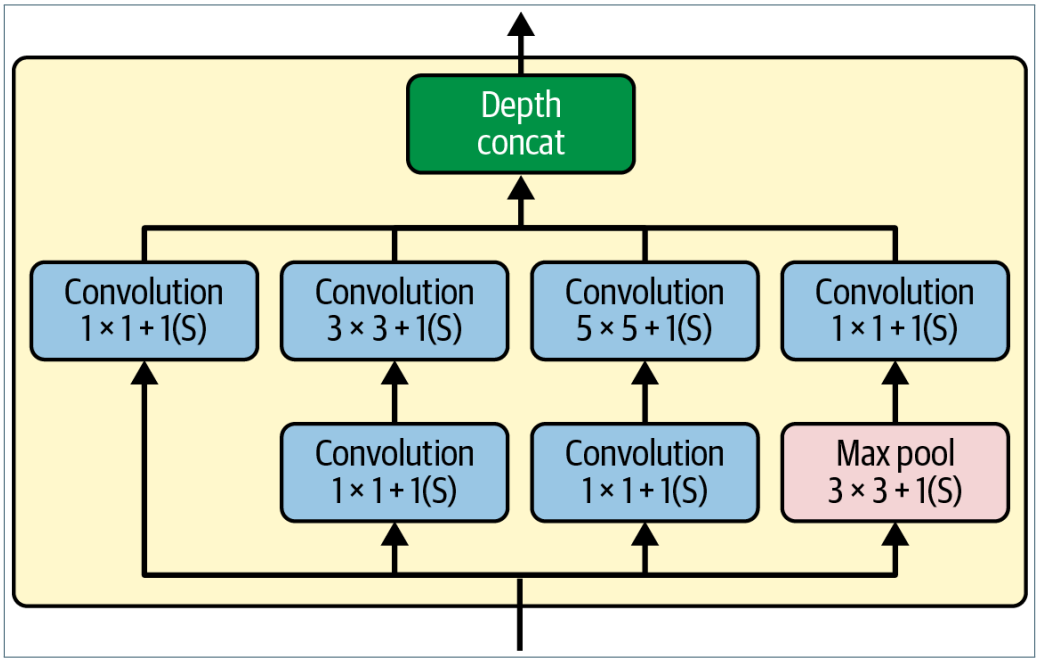

图 12-13: Inception 模块（Inception module）

你可能会疑惑，为什么 inception 模块包含使用 1 × 1 核的卷积层。这些层一次只看一个像素，难道不能捕获任何特征吗？实际上，它们有三个作用：

- 虽然无法捕获空间模式，但它们可以沿深度维度（即跨通道）捕获模式。
- 它们的输出特征图数量少于输入，因此充当瓶颈层，也就是降低维度。这会减少计算成本和参数数量，加快训练并改善泛化能力。
- 两对卷积层（`[1 × 1, 3 × 3]` 和 `[1 × 1, 5 × 5]`）各自相当于一个强大的卷积层，能够捕获更复杂的模式。卷积层相当于在图像上滑动一个稠密层（在每个位置只观察一个小感受野），而这两对卷积层相当于在图像上滑动两层神经网络。

[10]见本章参考文献第 12 条。

简而言之，可以把完整的 inception 模块看作一个“强化版”卷积层，它能输出捕获不同尺度复杂模式的特征图。

下面来看 GoogLeNet CNN 的架构（见图 12-15）。每个卷积层和池化层输出的特征图数量显示在核大小之前。该架构非常深，必须用三列来表示，但 GoogLeNet 实际上是一个高高堆叠的结构，其中包含九个 inception 模块（图中带旋转顶部的方框）。inception 模块中的六个数字分别表示模块中每个卷积层输出的特征图数量，顺序与图 12-14 相同。注意，所有卷积层都使用 ReLU 激活函数。

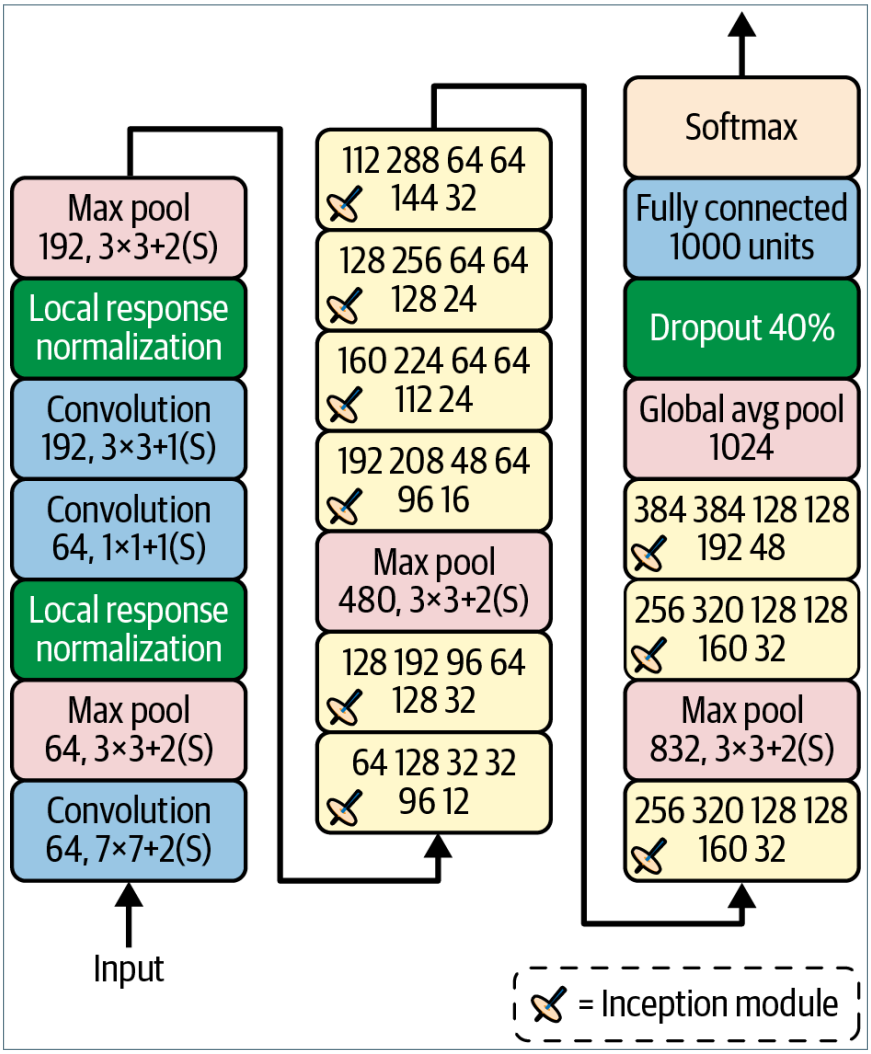

图 12-14: GoogLeNet 架构（GoogLeNet architecture）

逐步看一下这个网络：

- 前两层把图像的高度和宽度除以 4（因此面积除以 16），以降低计算负担。第一层使用 7 × 7 的大核，以尽可能保留信息。

- 接着是局部响应归一化层，确保前面的层学习到多种特征。
- 后面有两个卷积层，其中第一个充当瓶颈层。如前所述，可以把这一对层看作一个更聪明的卷积层。
- 再次使用局部响应归一化层，确保前面的层捕获多种模式。
- 然后是一个最大池化层，将图像的高度和宽度再缩小一半，以加快计算。
- 接下来是 CNN 的主干：九个 inception 模块高高堆叠，中间穿插几个最大池化层，以降低维度并加快网络。
- 全局平均池化层输出每张特征图的均值，丢弃剩余的空间信息。在这一阶段剩余的空间信息已经很少，因此这没有问题。GoogLeNet 的输入图像通常为 224 × 224 像素，经过 5 个每次都将高度和宽度除以 2 的最大池化层后，特征图大小已经缩小到 7 × 7。此外，这是分类任务而不是定位任务，因此物体位于哪里并不重要。得益于该层带来的降维，不需要像 AlexNet 那样在 CNN 顶部放置多个全连接层，这会显著减少网络参数量并降低过拟合风险。
- 最后几层不难理解：先使用 dropout 进行正则化，再使用包含 1,000 个单元的全连接层（因为有 1,000 个类别），最后使用 softmax 激活函数输出估计的类别概率。

原始 GoogLeNet 架构还在第三个和第六个 inception 模块上方各接入了一个辅助分类器。每个辅助分类器都由一个平均池化层、一个卷积层、两个全连接层和一个 softmax 激活层组成。在训练期间，它们的损失（缩小 70% 后）会加到总损失中。这样做的目的是对抗梯度消失问题并正则化网络，但后来发现它们的作用相对有限。

Google 研究人员后来提出了 GoogLeNet 的多个变体，包括 Inception-v3 和 Inception-v4，它们使用略有不同的 inception 模块，以取得更好的性能。

12.5.5　ResNet

Kaiming He 等人使用残差网络（Residual Network，ResNet）赢得了 2015 年 ILSVRC 挑战赛[11]，取得了低于 3.6% 的惊人 top-5 错误率。获胜变体使用了一个由 152 层组成的极深 CNN（其他变体有 34、50 和 101 层）。这印证了总体趋势：计算机视觉模型越来越深，但参数越来越少。训练如此深的网络之所以成为可能，关键在于使用跳跃连接（skip connection，也称 shortcut connection）：输入某层的信号也会被加到网络堆栈中更高位置某层的输出上。下面看看它为什么有用。

训练神经网络时，目标是让它拟合目标函数 `h(x)`。如果把输入 `x` 加到网络输出上（即加入跳跃连接），网络就不再需要直接拟合 `h(x)`，而是被迫拟合 `f(x) = h(x) - x`。这称为残差学习（residual learning，见图 12-16）。

[11] 见本章参考文献第 13 条。

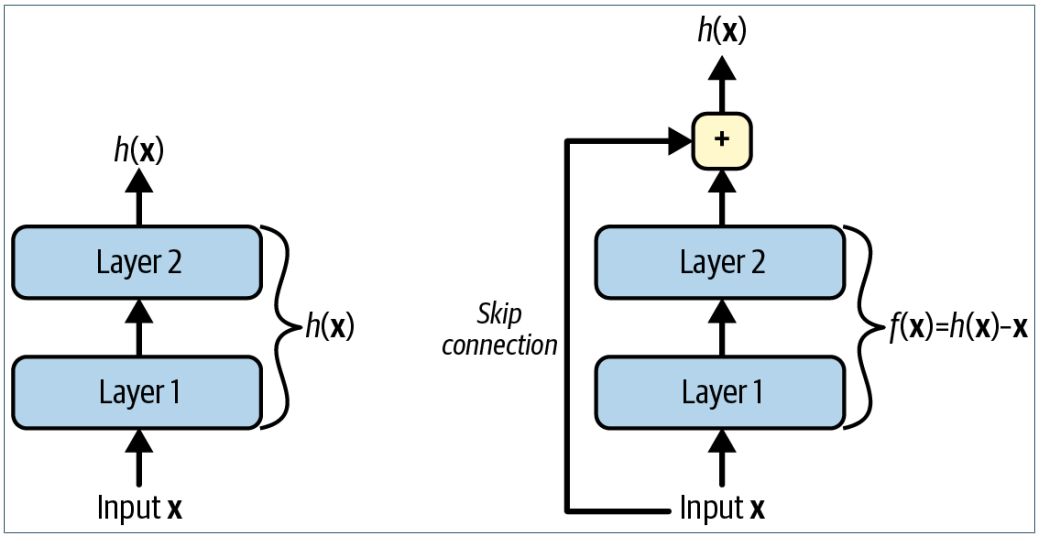

图 12-15: 残差学习（Residual learning）

初始化神经网络时，权重接近 0，因此普通网络在训练开始时只会输出接近 0 的值。但如果加入跳跃连接，网络的输出就会是输入的副本；换句话说，训练开始时它表现为恒等函数。如果目标函数本身与恒等函数比较接近（这很常见），训练速度就会大幅加快。

此外，如果加入许多跳跃连接，即使某些层还没有开始学习，网络也能开始取得进展（见图 12-17）。得益于跳跃连接，信号可以轻松穿过整个网络。深度残差网络可以看作残差单元（residual unit，RU）的堆叠，其中每个残差单元都是带有跳跃连接的小型神经网络。

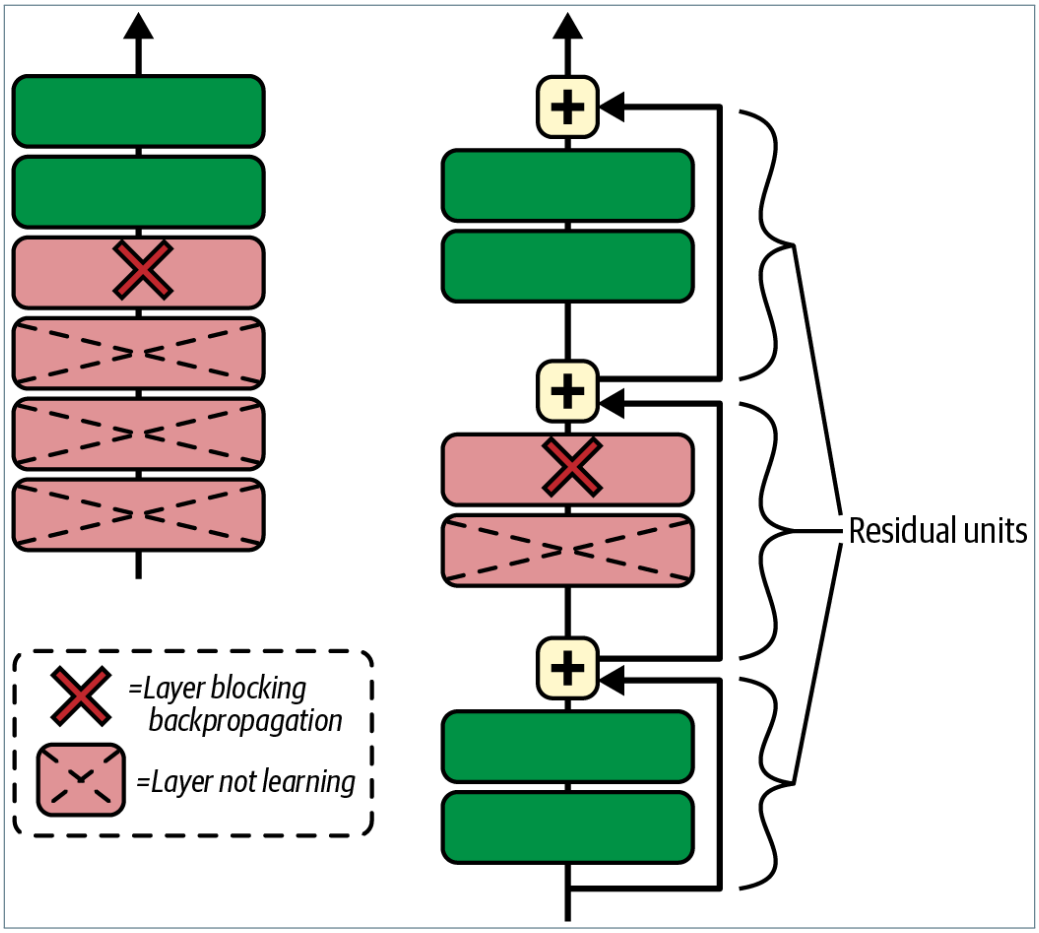

图 12-16: 普通深度神经网络（左）与深度残差网络（右）（Regular deep neural network (left) and deep residual network (right)）

现在来看 ResNet 的架构（见图 12-18）。它出人意料地简单：开头和结尾与 GoogLeNet 完全一样（但没有 dropout 层），中间只是一长串非常深的残差单元。

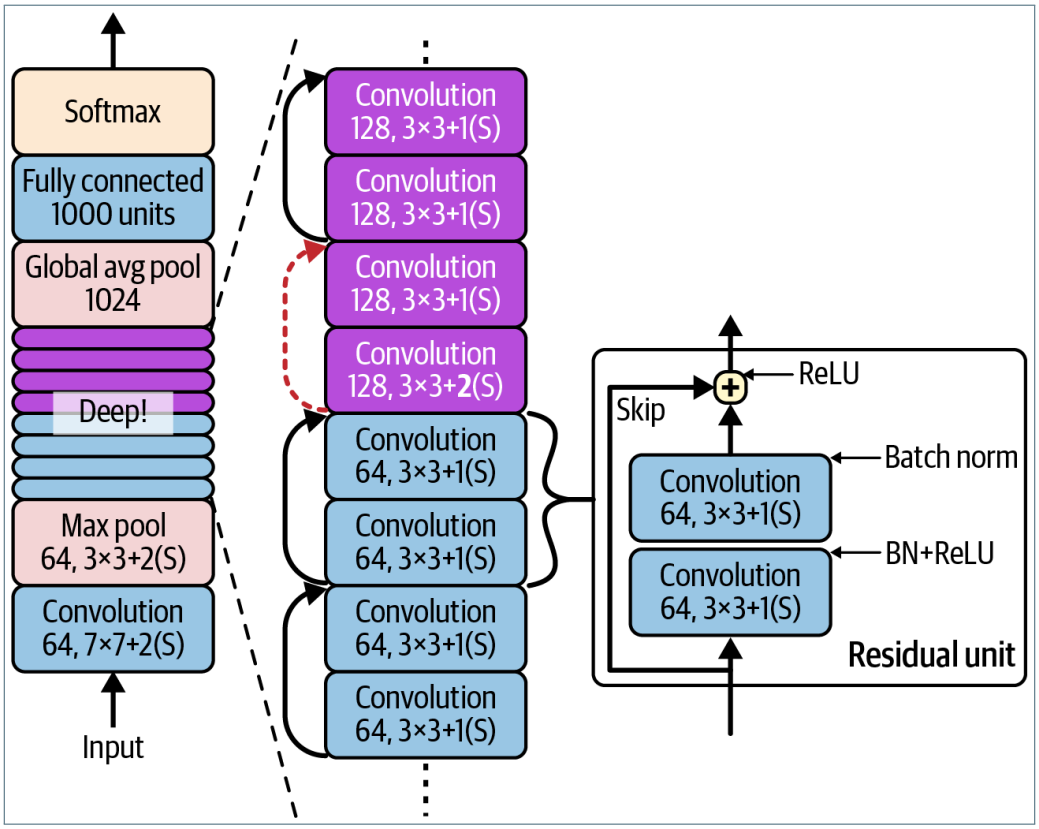

图 12-17: ResNet 架构（ResNet architecture）

每个残差单元由两个卷积层组成（没有池化层），并使用 batch normalization（BN）和 ReLU 激活函数；卷积核为 3 × 3，步幅为 1，采用 **`same`** 填充，因此空间维度保持不变。

注意，每经过几个残差单元，特征图数量就会加倍，同时高度和宽度减半（通过使用步幅为 2 的卷积层）。这种情况下，输入不能直接与残差单元的输出相加，因为两者形状不同（例如图 12-18 中虚线箭头表示的跳跃连接就会遇到这个问题）。解决方法是让输入经过一个步幅为 2、输出特征图数量正确的 1 × 1 卷积层（见图 12-19）。

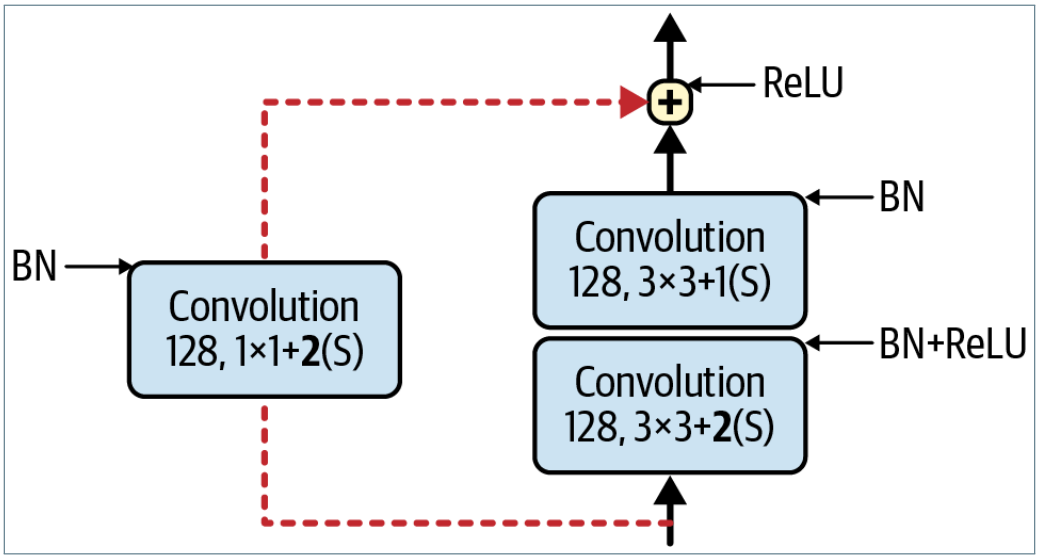

图 12-18: 改变特征图大小和深度时的跳跃连接（Skip connection when changing feature map size and depth）

> 提示：训练期间，可以对每个小批量随机跳过一组残差单元。这种随机深度（stochastic depth）技术[a]能够在不牺牲准确率的情况下显著加快训练。可以使用 `torchvision.ops.stochastic_depth()` 函数实现。
>
> [a] 见本章参考文献第 14 条。

ResNet 有多种变体，层数各不相同。ResNet-34 表示有 34 层的 ResNet（只统计带参数的卷积层和全连接层[12]），其中有 3 个输出 64 张特征图的残差单元、4 个输出 128 张特征图的残差单元、6 个输出 256 张特征图的残差单元，以及 3 个输出 512 张特征图的残差单元。本章稍后会实现这个架构。

更深的 ResNet（如 ResNet-152）使用略有不同的残差单元。它们不再使用两个包含 256 张特征图的 3 × 3 卷积层，而是使用三个卷积层：第一个是只有 64 张特征图（少 4 倍）的 1 × 1 卷积层，充当瓶颈层；然后是一个包含 64 张特征图的 3 × 3 层；最后是另一个包含 256 张特征图（4 × 64）的 1 × 1 卷积层，恢复原始深度。ResNet-152 包含 3 个输出 256 张特征图的这类残差单元，接着是 8 个输出 512 张特征图的单元、36 个输出 1,024 张特征图的单元，最后是 3 个输出 2,048 张特征图的单元。

> 注意：Google 的 Inception-v4 架构[a]融合了 GoogLeNet 和 ResNet 的思想，在 ImageNet 分类上取得了接近 3% 的 top-5 错误率。

[12] 见本章参考文献第 15 条。

[a]见本章参考文献第 16 条。

12.5.6　Xception

另一个值得注意的 GoogLeNet 变体是 Xception[13]（Extreme Inception 的缩写）。它由 François Chollet（深度学习框架 Keras 的作者）于 2016 年提出，并在一个大型视觉任务（3.5 亿张图像、17,000 个类别）上显著超过了 Inception-v3。与 Inception-v4 一样，它融合了 GoogLeNet 和 ResNet 的思想，但将 inception 模块替换为一种特殊的层，称为深度可分离卷积层（depthwise separable convolution layer），也简称可分离卷积层[14]。

普通卷积层使用滤波器同时捕获空间模式（例如椭圆）和跨通道模式（例如嘴巴 + 鼻子 + 眼睛 = 脸）。而可分离卷积层作出一个强假设：空间模式和跨通道模式可以分别建模（见图 12-20）。因此，它由两部分组成：第一部分对每个输入特征图应用一个空间滤波器；第二部分只寻找跨通道模式，本质上就是一个使用 1 × 1 滤波器的普通卷积层。

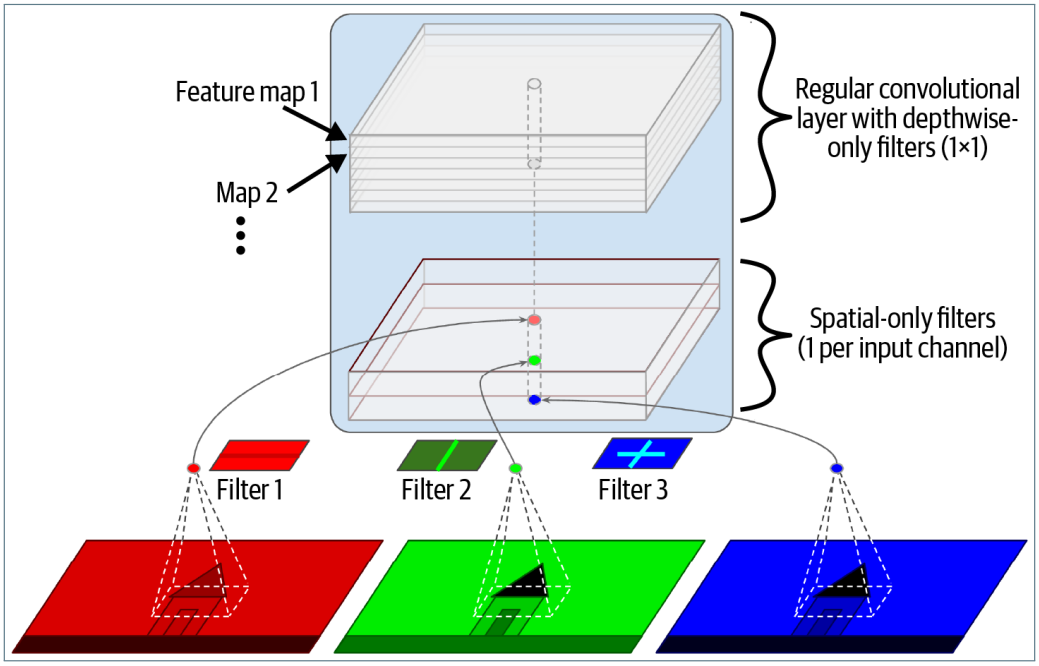

图 12-19: 深度可分离卷积层（Depthwise separable convolutional layer）

由于可分离卷积层每个输入通道只有一个空间滤波器，因此应避免在通道数过少的层之后使用它们，例如输入层（尽管图 12-20 就是这样画的，但那只是为了说明）。因此，Xception

[13]见本章参考文献第 17 条。

[14]见本章参考文献第 18 条。

架构先使用两个普通卷积层，之后的主体只使用可分离卷积（总共 34 个），再加上几个最大池化层以及常见的末端层（全局平均池化层和稠密输出层）。

你可能会疑惑，Xception 不包含 inception 模块，为什么仍被视为 GoogLeNet 的变体？前面说过，inception 模块包含使用 1 × 1 滤波器的卷积层，这些层专门寻找跨通道模式；但位于其上方的卷积层是同时寻找空间模式和跨通道模式的普通卷积层。因此，可以把 inception 模块看作普通卷积层（联合考虑空间模式和跨通道模式）与可分离卷积层（分别考虑这两类模式）之间的中间形式。实践中，可分离卷积层往往表现更好。

PyTorch 没有包含 **SeparableConv2d** 模块，但自行实现相当直接：

```
class SeparableConv2d(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, stride=1,
                 padding=0):
        super().__init__()
        self.depthwise_conv = nn.Conv2d(
            in_channels, in_channels, kernel_size, stride=stride,
            padding=padding, groups=in_channels)
        self.pointwise_conv = nn.Conv2d(
            in_channels, out_channels, kernel_size=1, stride=1, padding=0)
    def forward(self, inputs):
        return self.pointwise_conv(self.depthwise_conv(inputs))
```

注意第 7 行的 **groups** 参数：它允许你把输入通道拆成指定数量的相互独立的组，每组拥有自己的滤波器（注意，**in_channels** 和 **out_channels** 必须能被 **groups** 整除）。默认 **groups=1**，得到普通卷积层；但如果设置 **groups=in_channels** 且 **out_channels=in_channels**，就得到一个深度卷积层，每个输入通道对应一个滤波器。这就是可分离卷积层的第一层。第二层是普通卷积层，只是将核大小和步幅设置为 1。就是这么简单！

> 提示：可分离卷积层比普通卷积层使用更少的参数、更少的内存和更少的计算，而且通常表现更好。可以考虑默认使用它们，但要避免在通道数很少的层之后使用（例如输入通道层）。

12.5.7　SENet

2017 年 ILSVRC 挑战赛的获胜架构是压缩激励网络（Squeeze-and-Excitation Network，SENet）[15]。这种架构扩展了 inception 网络和 ResNet 等已有架构，提升了它们的性能，因此以惊人的 2.25% top-5 错误率赢得了比赛。扩展后的 inception 网络和 ResNet 分别称为 SE-Inception 和 SE-ResNet。性能提升来自这样的事实：SENet 会在原始架构中的每个 inception 模块或残差单元中加入一个称为 SE 模块的小型神经网络，如图 12-21 所示。

[15] 见本章参考文献第 19 条。

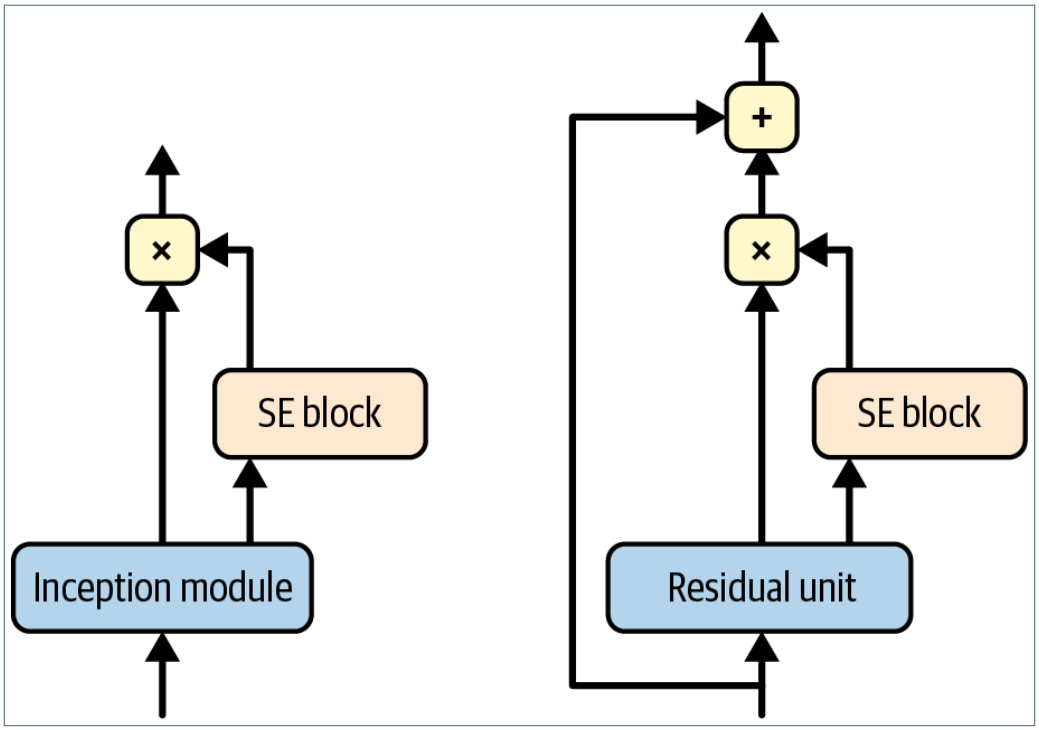

图 12-20: SE-Inception 模块（左）和 SE-ResNet 单元（右）（SE-Inception module (left) and SE-ResNet unit (right)）

SE 模块分析它所连接单元的输出，只关注深度维度（不寻找空间模式），并学习哪些特征通常会同时激活。然后，它使用这些信息重新校准特征图，如图 12-22 所示。例如，SE 模块可能学到嘴巴、鼻子和眼睛通常会一起出现在图像中：如果看到了嘴巴和鼻子，就应该预期看到眼睛。因此，如果模块看到嘴巴和鼻子特征图强烈激活，而眼睛特征图只有轻微激活，它就会增强眼睛特征图（更准确地说，会降低无关特征图的权重）。如果眼睛与其他东西有些混淆，这种特征图重新校准就有助于消除歧义。

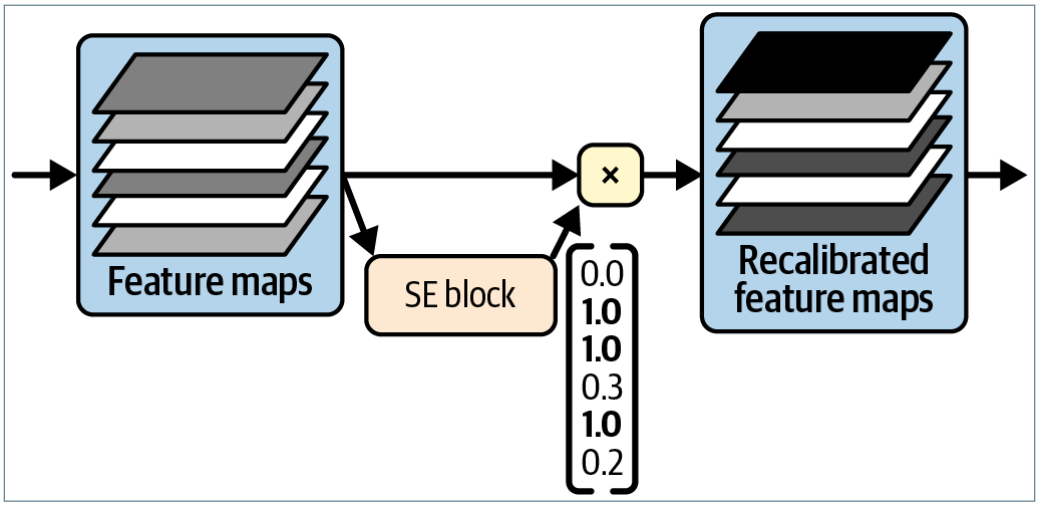

图 12-21: SE 模块执行特征图重新校准（An SE block performs feature map recalibration）

SE 模块只由三层组成：一个全局平均池化层、一个使用 ReLU 激活函数的隐藏稠密层，以及一个使用 sigmoid 激活函数的稠密输出层（见图 12-23）。

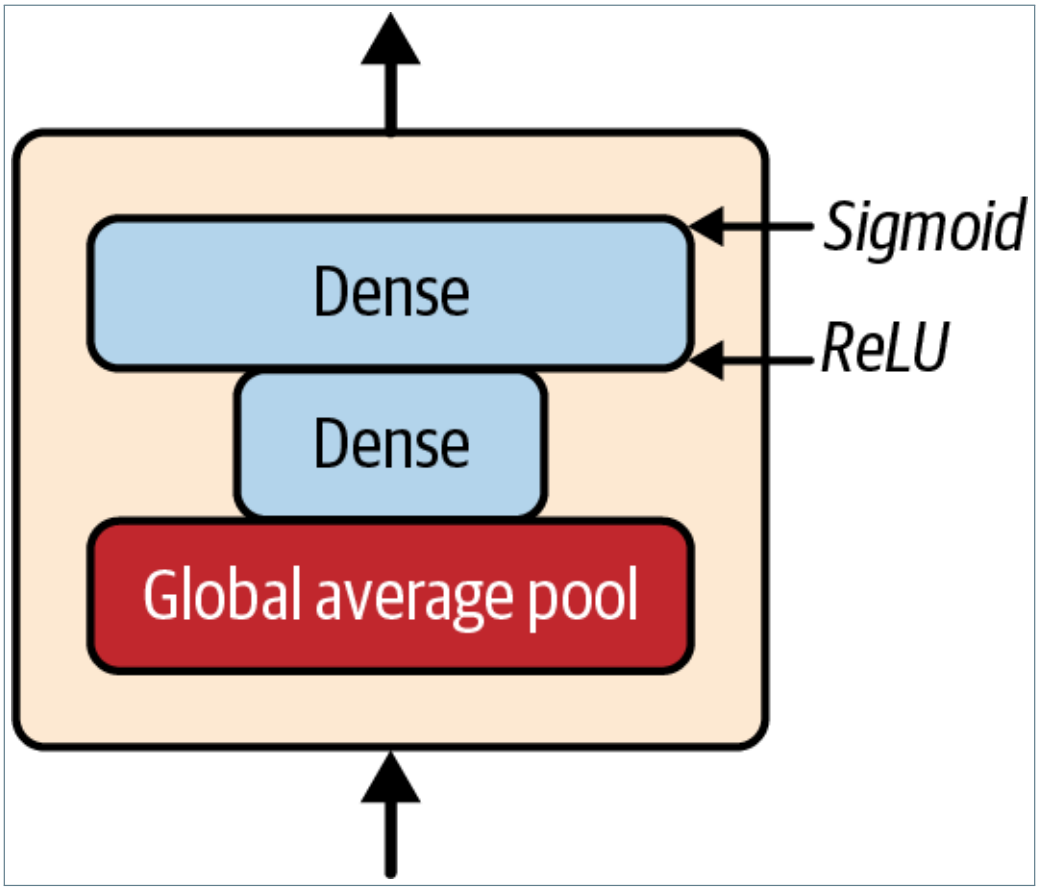

图 12-22: SE 模块架构（SE block architecture）

和前面一样，全局平均池化层计算每张特征图的平均激活值。例如，如果输入包含 256 张特征图，它就会输出 256 个数，表示每个滤波器的总体响应水平。接下来是“压缩”发生的地方：这一层的神经元数量显著少于 256，通常是特征图数量的 1/16（例如 16 个神经元），因此 256 个数被压缩成一个小向量（例如 16 维）。这是对特征响应分布的低维向量表示，也就是嵌入。这个瓶颈步骤迫使 SE 模块学习特征组合的通用表示（第 18 章讨论自动编码器时还会再次看到这个原则）。最后，输出层接收该嵌入，输出一个重新校准向量，其中每个特征图对应一个 0 到 1 之间的数（例如共有 256 个数）。然后将特征图与该向量相乘，因此无关特征（重新校准分数低）会被缩小，而相关特征（分数接近 1）基本保持不变。

12.5.8　其他值得注意的架构（Other Noteworthy Architectures）

还有许多其他 CNN 架构值得探索，下面简要介绍其中一些最值得关注的架构。

VGGNet[16]是 2014 年 ILSVRC 挑战赛的亚军。来自牛津大学 Visual Geometry Group（VGG）研究实验室的 Karen Simonyan 和 Andrew Zisserman 开发了一个非常简单、经典的架构：先

[16]见本章参考文献第 20 条。

使用 2 或 3 个卷积层和一个池化层，然后再次使用 2 或 3 个卷积层和一个池化层，以此类推（根据 VGG 变体的不同，最终共有 16 或 19 个卷积层），最后加上包含两个隐藏层和一个输出层的稠密网络。它使用较小的 3 × 3 滤波器，但滤波器数量很多。

ResNeXt[17]改进了 ResNet 中的残差单元。最好的 ResNet 模型中的残差单元通常各自只包含 3 个卷积层，而 ResNeXt 的残差单元由许多并行堆叠组成（例如 32 个堆叠），每个堆叠包含 3 个卷积层。不过，每个堆叠的前两层只使用少量滤波器（例如只有 4 个），因此总体参数量仍与 ResNet 相近。所有堆叠的输出会相加，结果再与跳跃连接一起传给下一个残差单元。

DenseNet[18]由多个稠密块组成，每个稠密块包含若干个稠密连接的卷积层。这种架构在使用相对较少参数的同时取得了很高的准确率。“稠密连接”是什么意思？同一块内，每一层的输出都会作为输入提供给它之后的每一层。例如，一个块中的第 4 层会以第 1、2、3 层输出沿深度方向连接后的结果作为输入。稠密块之间由若干过渡层分隔。

MobileNet[19]是为轻量、快速运行而设计的精简模型，因此在移动端和 Web 应用中很受欢迎。它们和 Xception 一样基于深度可分离卷积层。作者提出了多个变体，在准确率、速度和模型大小之间进行权衡。还有一些适用于移动设备的 CNN 架构，例如 SqueezeNet、ShuffleNet 和 MNASNet。

CSPNet[20]（Cross Stage Partial Network，跨阶段部分网络）与 DenseNet 相似，但每个稠密块的部分输入会直接连接到该块的输出，而无需经过块内的层。

EfficientNet[21]可以说是这个列表中最重要的模型。作者提出了一种高效缩放任意 CNN 的方法，以有原则的方式同时增加深度（层数）、宽度（每层滤波器数量）和分辨率（输入图像大小），这称为复合缩放（compound scaling）。他们在缩小版 ImageNet（图像更小、数量更少）上使用神经架构搜索找到一个良好架构，然后使用复合缩放创建越来越大的架构。EfficientNet 模型问世后，在所有计算预算下都大幅超越了已有模型，直到今天仍然属于最优秀的模型。作者在 2021 年发表了后续论文，提出 EfficientNetV2，进一步改善了训练时间和参数效率。

ConvNeXt[22]与 ResNet 非常相似，但借鉴了最成功的视觉 Transformer 架构（见第 16 章）中的许多调整，例如使用大核（如用 7 × 7 代替 3 × 3）、在每个残差单元中减少激活函数和归一化层等。

理解 EfficientNet 的复合缩放方法，有助于更深入地理解 CNN，尤其是在需要扩展 CNN 架构时。它基于计算预算的对数度量，记为 **`phi`**：如果计算预算翻倍，**`phi`** 就增加 1。换句话说，可用于训练的浮点运算数与 **`2**phi`** 成正比。CNN 架构的深度、宽度和分辨率应该分

[17] 见本章参考文献第 21 条。
[18] 见本章参考文献第 22 条。
[19] 见本章参考文献第 23 条。
[20] 见本章参考文献第 24 条。
[21] 见本章参考文献第 25 条。
[22] 见本章参考文献第 26 条。

别按照 `alpha^phi`、`beta^phi` 和 `gamma^phi` 进行缩放。因子 `alpha`、`beta` 和 `gamma` 必须大于 1，并且 `alpha beta^2 gamma^2` 应接近 2。因子的最优取值取决于 CNN 架构。为了找到 EfficientNet 架构的最优因子，作者从一个小型基线模型 EfficientNetB0 开始，固定 `phi = 1`，然后简单地进行网格搜索，找到 `alpha = 1.2`、`beta = 1.1` 和 `gamma = 1.1`。接着，他们使用这些因子创建了 EfficientNetB1 到 EfficientNetB7 等多个更大的架构，对应逐渐增大的 `phi`。

接下来看看如何选择合适的架构。

12.5.9　选择合适的 CNN 架构（Choosing the Right CNN Architecture）

正如你所料，最佳架构取决于项目中最重要的因素：准确率？模型大小（例如要部署到移动设备）？推理速度？能耗？表 12-3 列出了 TorchVision 中目前可用的一些预训练分类模型（稍后会介绍如何使用它们）。完整列表见 https://pytorch.org/vision/stable/models，包括用于其他计算机视觉任务的模型。表中给出了每个模型在 ImageNet 数据集上的 top-1 和 top-5 准确率、参数量（以百万计），以及处理每张图像所需的计算量（以 GFLOPs 衡量，即十亿次浮点运算）。可以看到，较大的模型通常更准确，但并不总是如此；例如，EfficientNet v2 的小型变体在模型大小和准确率上都超过了 Inception v3（但计算量更大）。

12.5.10　GPU 内存需求：推理与训练（GPU RAM Requirements: Inference Versus Training）

较小的模型可以在任意 GPU 上运行，但大型模型（如 ConvNeXt Large）怎么办？既然每个参数都是一个 32 位浮点数（4 字节），你可能会以为运行一个 2 亿参数模型只需要 800 MB 内存。但实际上通常需要更多：推理时每张图像一般需要约 5 GB（取决于图像大小），训练时需要更多。下面解释原因。

CNN 需要大量内存。例如，考虑一个包含 200 个 5 × 5 滤波器、步幅为 1、使用 `same` 填充的卷积层，它处理一张 150 × 100 的 RGB 图像（3 个通道）：

- 参数数量为 `(5 × 5 × 3 + 1) × 200 = 15,200`，其中 `+1` 对应偏置项。这个数量并不大；为了产生相同大小的输出，一个全连接层需要 `200 × 150 × 100` 个神经元，每个神经元都连接到全部 `150 × 100 × 3` 个输入，因此会有 `200 × 150 × 100 × (150 × 100 × 3 + 1) ≈ 1350` 亿个参数！
- 但 200 张特征图中的每一张都包含 `150 × 100` 个神经元，每个神经元都需要计算其 5 × 5 × 3 = 75 个输入的加权和，总共需要 2.25 亿次浮点乘法。这比全连接层好得多，但计算量仍然很大。

表 12-3: TorchVision 中部分可用的预训练模型（按大小排序）

模型名称	Top-1 准确率	Top-5 准确率	参数量	GFLOPs
MobileNet v3 small	67.7%	87.4%	2.5M	0.1
EfficientNet B0	77.7%	93.5%	5.3M	0.4
GoogLeNet	69.8%	89.5%	6.6M	1.5
DenseNet 121	74.4%	92.0%	8.0M	2.8
EfficientNet v2 small	84.2%	96.9%	21.5M	8.4
ResNet 34	73.3%	91.4%	21.8M	3.7
Inception V3	77.3%	93.5%	27.2M	5.7
ConvNeXt Tiny	82.6%	96.1%	28.6M	4.5
DenseNet 161	77.1%	93.6%	28.7M	7.7
ResNet 152	82.3%	96.0%	60.2M	11.5
AlexNet	56.5%	79.1%	61.1M	0.7
EfficientNet B7	84.1%	96.9%	66.3M	37.8
ResNeXt 101 32×8D	82.8%	96.2%	88.8M	16.4
EfficientNet v2 large	85.8%	97.8%	118.5M	56.1
VGG 11 with BN	70.4%	89.8%	132.9M	7.6
ConvNeXt Large	84.4%	97.0%	197.8M	34.4

- 更重要的是，如果使用 32 位浮点数，卷积层输出占用 `200 × 150 × 100 × 32 = 9,600` 万位（12 MB）内存[23]。这还只针对一个实例；如果训练批次包含 100 个实例，那么仅这一层就会使用 1.2 GB 内存。

在推理期间（即为新实例进行预测时），一层占用的内存在计算出下一层后就可以释放，因此只需要能够容纳相邻两层所需内存的容量。但在训练期间，前向传播过程中计算出的所有内容都必须保留，以便反向传播使用，因此（至少）需要为所有层所需的内存总量准备空间。GPU 内存很容易耗尽。

如果训练因为内存不足错误而崩溃，可以尝试减小批大小。为了保留大批次的一些好处，可以在每个批次之后累积梯度，每经过几个批次才更新一次模型权重。也可以尝试使用步幅降低维度、移除几层、使用 16 位浮点数代替 32 位浮点数、把 CNN 分布到多个设备上，或将最占内存的模块卸载到 CPU（使用 `module.to("cpu")`）。

另一种选择是用更多计算换取更低的内存使用量。例如，不保存前向传播期间的全部激活值，只保存其中一些，称为激活检查点；然后在反向传播期间，需要时从上一个检查点开始运行部分前向传播，重新计算缺失的激活值。

要在 PyTorch 中实现激活检查点（也称梯度检查点），可以使用 `torch.utils.checkpoint.checkpoint()` 函数：不要调用 `z = foo(x)`，而是调用 `z = checkpoint(foo, x)`。推理期间两者没有区别，但训练时该模块的激活值不会在前向传播期间保存，而会在反向传播需要时重新计算。这种方法实现相当简单，也不需要改变模型架构。

> 警告：如果使用相同输入调用两次，前向传播必须产生相同结果，否则梯度会不正确。这意味着自定义模块必须遵守一些约束，例如避免原地操作，或避免使用不受控制的随机数生成状态。更多细节请参见 `checkpoint()` 函数文档。

不过，如果愿意调整模型架构，还可以使用一种更高效的用计算换内存方案：可逆残差网络。

12.5.11　可逆残差网络（RevNets）（Reversible Residual Networks）

可逆残差网络（RevNet）由 Aidan Gomez 等人在 2017 年提出[24]。它们通常只增加约 33% 的计算量，并且在前向传播期间完全不需要保存激活值。工作方式如下：

- 每一层称为可逆层，它接收两个大小相同的输入 `x1` 和 `x2`，并计算两个输出：`y1 = x1 + f(x2)`，`y2 = g(y1) + x2`。其中 `f` 和 `g` 可以是任意函数，只要输出大小等于输入大小，并且对于给定输入总是产生相同输出即可。
- 例如，`f` 和 `g` 可以是由几个卷积层组成的相同模块，每个卷积层使用步幅 1 和 `same` 填充，并带有自己的 batch norm 和 ReLU 激活。

[23] 见本章参考文献第 27 条。

[24] 见本章参考文献第 28 条。

- 在反向传播期间，可以在需要时使用 `x2 = y2 - g(y1)` 和 `x1 = y1 - f(x2)` 从输出重新计算可逆层的输入。无需在前向传播期间保存任何激活值，十分巧妙。

由于 `f` 和 `g` 必须输出与输入相同的形状，可逆层不能包含步幅大于 1 或使用 `valid` 填充的卷积层。CNN 中仍然可以使用这类层，但 RevNet 技巧不适用于它们，因此必须在前向传播期间保存其激活值。幸运的是，CNN 通常只有少数这类层，其中包括第一层：它会降低空间维度并增加通道数，其结果可以沿通道维拆分成两个相等部分，再送入第一个可逆层。

RevNet 并不局限于 CNN。事实上，它还是一种有影响力的 Transformer 架构 Reformer 的核心（见第 17 章）。

12.6　使用 PyTorch 实现 ResNet-34 CNN（Implementing a ResNet-34 CNN Using PyTorch）

到目前为止介绍的大多数 CNN 架构都可以比较自然地使用 PyTorch 实现（当然，通常更好的做法是直接加载预训练网络，后面会看到）。为了说明实现过程，下面从零开始使用 PyTorch 实现 ResNet-34。首先创建一个 `ResidualUnit` 层：

```
class ResidualUnit(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1):
        super().__init__()
        DefaultConv2d = partial(
            nn.Conv2d, kernel_size=3, stride=1, padding=1, bias=False)
        self.main_layers = nn.Sequential(
            DefaultConv2d(in_channels, out_channels, stride=stride),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(),
            DefaultConv2d(out_channels, out_channels),
            nn.BatchNorm2d(out_channels),
        )
        if stride > 1:
            self.skip_connection = nn.Sequential(
                DefaultConv2d(in_channels, out_channels, kernel_size=1,
                              stride=stride, padding=0),
                nn.BatchNorm2d(out_channels),
            )
        else:
            self.skip_connection = nn.Identity()
    def forward(self, inputs):
        return F.relu(self.main_layers(inputs) + self.skip_connection(inputs))
```

可以看到，这段代码与图 12-19 非常接近。在构造器中创建所需的全部层：主层对应图右侧的层；当步幅大于 1 时，跳跃连接对应图左侧的层；当步幅为 1 时，则对应一个 `nn.`

Identity 模块。**nn.Identity** 模块什么也不做，只返回输入。然后在 **forward()** 方法中，让输入同时经过主层和跳跃连接，再将两个输出相加并应用激活函数。

接下来创建 **ResNet34** 模块！有了 **ResidualUnit** 模块后，整个 ResNet-34 架构就变成一个大的模块堆叠，因此可以让 **ResNet34** 类基于一个 **nn.Sequential** 模块。代码与图 12-18 紧密对应：

```
class ResNet34(nn.Module):
    def __init__(self):
        super().__init__()
        layers = [
            nn.Conv2d(in_channels=3, out_channels=64, kernel_size=7, stride=2,
                      padding=3, bias=False),
            nn.BatchNorm2d(num_features=64),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=3, stride=2, padding=1),
        ]
        prev_filters = 64
        for filters in [64] * 3 + [128] * 4 + [256] * 6 + [512] * 3:
            stride = 1 if filters == prev_filters else 2
            layers.append(ResidualUnit(prev_filters, filters, stride=stride))
            prev_filters = filters
        layers += [
            nn.AdaptiveAvgPool2d(output_size=1),
            nn.Flatten(),
            nn.LazyLinear(10),
        ]
        self.resnet = nn.Sequential(*layers)
    def forward(self, inputs):
        return self.resnet(inputs)
```

这段代码中唯一稍微棘手的部分，是把 **ResidualUnit** 层加入层列表的循环。如前所述，前 3 个残差单元使用 64 个滤波器，接下来的 4 个使用 128 个，以此类推。在每次循环中，如果滤波器数量与上一个残差单元相同，就把步幅设为 1；否则设为 2。然后将 **ResidualUnit** 添加到层列表，最后更新 **prev_filters**。

就这样，仅用 45 行代码，就可以构建曾赢得 2015 年 ILSVRC 挑战赛的模型！这既展示了 ResNet 模型的优雅，也展示了 PyTorch 和 Python 的表现力。实现前面讨论的其他 CNN 架构需要更多时间，但不会困难太多。不过，TorchVision 已经内置了其中几个架构，为什么不直接使用它们呢？

12.7　使用 TorchVision 的预训练模型（Using TorchVision's Pretrained Models）

通常不需要手工实现 GoogLeNet、ResNet 或 ConvNeXt 这样的标准模型，因为只需几行代码，就可以使用 TorchVision 获取预训练网络。

> 提示：TIMM 是另一个非常流行的 PyTorch 库。它提供大量预训练图像分类模型，以及数据加载器、数据增强工具、优化器、调度器等相关工具。Hugging Face Hub 也是获取各种预训练模型的好地方（见第 14 章）。

例如，可以使用下面的代码加载一个在 ImageNet 上预训练的 ConvNeXt 模型。ConvNeXt 有 tiny、small、base 和 large 等多个变体，下面加载 base 变体：

```
weights = torchvision.models.ConvNeXt_Base_Weights.IMAGENET1K_V1
model = torchvision.models.convnext_base(weights=weights).to(device)
```

就这么简单！这段代码会自动从 Torch Hub（一个在线预训练模型仓库）下载 338 MB 的权重。权重会保存并缓存下来，以便之后使用（例如位于 **~/.cache/torch/hub**；可以运行 **torch.hub.get_dir()** 查找系统上的确切路径）。有些模型包含更新的权重版本（例如 **IMAGENET1K_V2**）或其他权重变体。要查看可用模型的完整列表，可以运行 **torchvision.models.list_models()**。要查看某个模型（如 **convnext_base**）可用的预训练权重列表，可以运行 **list(torchvision.models.get_model_weights("convnext_base"))**。也可以访问 https://pytorch.org/vision/main/models。

现在使用前面加载的两张示例图像测试这个模型。首先必须确保图像经过模型期望的预处理，尤其要保证尺寸正确。ConvNeXt 期望输入 224 × 224 像素图像（其他模型可能期望 299 × 299 等尺寸）。由于示例图像为 427 × 640 像素，可以使用 TorchVision 的 **CenterCrop** 和/或 **Resize** 变换调整它们，但使用 **weights.transforms()** 返回的变换更简单、更安全，因为它们是为该特定预训练模型设计的：

```
transforms = weights.transforms()
preprocessed_images = transforms(sample_images_permuted)
```

重要的是，这些变换还会像训练期间一样对像素强度进行归一化。在本例中，它们使用 ImageNet 的均值和标准差，分别对每个颜色通道的像素强度进行标准化（稍后会手工实现）。

接着把图像移到 GPU 上并送入模型。和往常一样，进行预测前记得将模型切换到评估模式；模型默认处于训练模式。同时关闭自动求导：

```
model.eval()
with torch.no_grad():
    y_logits = model(preprocessed_images.to(device))
```

结果是一个形状为 2 × 1,000 的张量，包含每张图像的类别 logits（回想一下，ImageNet 有 1,000 个类别）。和第 10 章一样，可以使用 **`torch.argmax()`** 获取每张图像的预测类别，也就是 logits 最大的类别：

```
>>> y_pred = torch.argmax(y_logits, dim=1)
>>> y_pred
tensor([698, 985], device='cuda:0')
```

这些类别具体代表什么？当然可以在线查找 ImageNet 的类别名称，但直接从权重对象中获取类别名称会更简单、更安全。它的 **`meta`** 属性是一个包含预训练模型元数据的字典，其中包括类别名称：

```
>>> class_names = weights.meta["categories"]
>>> [class_names[class_id] for class_id in y_pred]
['palace', 'daisy']
```

于是，第一张图像被分类为宫殿，第二张被分类为雏菊。ImageNet 没有中国塔或大丽花的类别，因此宫殿和雏菊是合理的替代类别（这座塔属于北京颐和园）。使用 **`topk()`** 查看前三个预测：

```
>>> y_top3_logits, y_top3_class_ids = y_logits.topk(k=3, dim=1)
>>> [[class_names[class_id] for class_id in top3] for top3 in y_top3_class_ids]
[['palace', 'monastery', 'lakeside'], ['daisy', 'pot', 'ant']]
```

查看每个类别的估计概率：

```
>>> y_top3_logits.softmax(dim=1)
tensor([[0.8618, 0.1185, 0.0197],
        [0.8106, 0.0964, 0.0930]], device='cuda:0')
```

可以看到，TorchVision 让下载和使用预训练模型变得很简单，而且对 ImageNet 类别开箱即用、效果不错。但如果需要把图像分类到 ImageNet 没有的类别（例如各种花卉物种），该怎么办？这时仍然可以通过迁移学习从预训练模型中获益。

12.8　用于迁移学习的预训练模型（Pretrained Models for Transfer Learning）

如果想构建图像分类器，但没有足够数据从头训练，通常可以像第 11 章讨论的那样，重用预训练模型的较低层。本节重用前面加载的 ConvNeXt 模型，它在 ImageNet 上预训练过；替换其分类头后，在 102 类花卉数据集（102 Category Flower Dataset[25]，简称 Flowers102）上对它进行微调。这个数据集每个类别只有 10 张图像，一共 102 个类别，因此如果从头训练模型，很难取得高准确率。但使用优秀的预训练模型，很容易达到 90% 以上的准确率。首先使用 TorchVision 下载数据集：

```
DefaultFlowers102 = partial(torchvision.datasets.Flowers102, root="datasets",
                            transform=weights.transforms(), download=True)
train_set = DefaultFlowers102(split="train")
valid_set = DefaultFlowers102(split="val")
test_set = DefaultFlowers102(split="test")
```

这段代码使用 **partial()** 避免重复写三次相同参数，同时设置 **transform=weights.transforms()**，使图像加载时立即完成预处理。Flowers102 数据集提供了三个预定义划分，分别用于训练、验证和测试。前两个划分每个类别有 10 张图像，但测试集出人意料地多得多（每个类别的图像数量在 20 到 238 之间变化）。在真实项目中，通常会把大多数数据用于训练，而不是测试；但这个数据集是为计算机视觉研究设计的，作者特意限制了训练集和验证集的大小。

接着像往常一样创建数据加载器：

[25] 见本章参考文献第 29 条。

```
from torch.utils.data import DataLoader
train_loader = DataLoader(train_set, batch_size=32, shuffle=True)
valid_loader = DataLoader(valid_set, batch_size=32)
test_loader = DataLoader(test_set, batch_size=32)
```

许多 TorchVision 数据集会方便地在 **classes** 属性中包含类别名称，但这个数据集没有[26]。如果希望看到“tiger lily”“monk's hood”或“snapdragon”这样的名称，而不是无趣的类别 ID，就需要手工定义类别名称列表：

```
class_names = ['pink primrose', ..., 'trumpet creeper', 'blackberry lily']
```

现在调整预训练的 ConvNeXt-base 模型来适应这个数据集。模型在拥有 1,000 个类别的 ImageNet 上预训练过，因此模型头（即模型上部的层）设计为输出 1,000 个 logits。但我们只有 102 个类别，所以必须把模型头切掉，换成一个更小的头。如何找到它？可以使用模型的 **named_children()** 方法查找子模块名称：

```
>>> [name for name, child in model.named_children()]
['features', 'avgpool', 'classifier']
```

features 模块是模型的主体，包含除全局平均池化层（**avgpool**）和模型头（**classifier**）之外的所有层。仔细看看这个模型头：

```
>>> model.classifier
Sequential(
  (0): LayerNorm2d((1024,), eps=1e-06, elementwise_affine=True)
  (1): Flatten(start_dim=1, end_dim=-1)
  (2): Linear(in_features=1024, out_features=1000, bias=True)
)
```

可以看到，它是一个由 layer norm 层、**nn.Flatten** 层和 **nn.Linear** 层组成的 **nn.Sequential** 模块。**nn.Linear** 层有 1,024 个输入和 1,000 个输出，是输出层，也是需要替换的层。只需修改输出数量：

```
n_classes = 102  # len(class_names) == 102
model.classifier[2] = nn.Linear(1024, n_classes).to(device)
```

正如第 11 章所述，通常最好在训练开始阶段冻结预训练层的权重。可以冻结模型中的每个参数，然后只解冻模型头的参数：

[26] 见本章参考文献第 30 条。

```
for param in model.parameters():
    param.requires_grad = False
for param in model.classifier.parameters():
    param.requires_grad = True
```

接下来训练几个 epoch，只训练新的模型头，不对预训练层进行微调，就应该已经能达到约 90% 的准确率。之后解冻整个模型，把学习率降低（通常降低 10 倍），继续训练模型。可以尝试一下，看看能达到什么准确率！

为了得到更高的准确率，通常最好对训练图像执行数据增强。可以尝试随机水平翻转训练图像、以小角度随机旋转图像、随机调整大小并裁剪图像，以及随机调整颜色。这些操作必须在执行 ImageNet 归一化之前完成，可以使用 **`Normalize`** 变换实现：

```
import torchvision.transforms.v2 as T
transforms = T.Compose([
    T.RandomHorizontalFlip(p=0.5),
    T.RandomRotation(degrees=30),
    T.RandomResizedCrop(size=(224, 224), scale=(0.8, 1.0)),
    T.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1),
    T.ToImage(),
    T.ToDtype(torch.float32, scale=True),
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
```

> 提示：TorchVision 提供了 **`AutoAugment`** 变换，会应用针对 ImageNet 优化的多种数据增强操作。它对许多其他图像数据集也有良好的泛化能力，还为另外两个数据集提供预定义设置：CIFAR10 和街景房屋号码（SVHN）数据集。

还可以通过下面的方法继续提高模型准确率：

- 尝试其他预训练模型。
- 扩展训练集：寻找更多花卉图像并为其添加标签。
- 创建模型集成，并组合多个模型的预测。
- 分析失败案例，观察它们是否共享某些特征，例如相似的纹理或颜色；然后可以调整图像预处理来解决这些问题。
- 使用基于性能的调度等学习率调度方法。
- 从顶层开始逐步解冻各层。或者使用差分学习率，对较低层使用较小学习率，对较高层使用较大学习率，可以通过参数组实现（见第 11 章）。
- 探索不同优化器，并调节它们的超参数。

- 尝试不同的正则化技术。

> 提示：值得花时间寻找在相似图像上预训练的模型。例如，如果处理的是卫星图像、航空图像，甚至是数字高程模型（DEM）这样的栅格数据，那么在 ImageNet 上预训练的模型不会有太大帮助。相反，可以查看 Microsoft 的 TorchGeo 库，它类似于 TorchVision，但面向地理空间数据。对于医学图像，可以查看 Project MONAI；对于农业图像，可以查看 AgML，等等。

这样，你就可以开始在自己的图像和类别上训练优秀的图像分类器了！但计算机视觉不只有分类。例如，如果还想知道花朵在图像中的位置，该怎么办？下面来看看。

12.9　分类与定位（Classification and Localization）

如第 9 章所述，在图像中定位物体可以表示为一个回归任务。为了预测包围物体的边界框，一种常见方法是预测边界框中心的位置，以及它的宽度和高度（另一种方法是预测物体左上角和右下角的水平、垂直坐标）。这意味着需要预测四个数。对 ConvNeXt 模型不需要做太大修改，只需要在全局平均池化层上方增加一个包含四个单元的稠密输出层即可。下面是一个 **`FlowerLocator`** 模型，它为给定的基础模型（例如 ConvNeXt）增加一个定位头：

```
class FlowerLocator(nn.Module):
    def __init__(self, base_model):
        super().__init__()
        self.base_model = base_model
        self.localization_head = nn.Sequential(
            nn.Flatten(),
            nn.Linear(base_model.classifier[2].in_features, 4)
        )
    def forward(self, X):
        features = self.base_model.features(X)
        pool = self.base_model.avgpool(features)
        logits = self.base_model.classifier(pool)
        bbox = self.localization_head(pool)
        return logits, bbox
torch.manual_seed(42)
locator_model = FlowerLocator(model).to(device)
```

这个定位模型有两个头：第一个输出类别 logits，第二个输出边界框。定位头的输入数量与分类头中 **`nn.Linear`** 层的输入数量相同，但只输出四个数。**`forward()`** 方法接收一个经过预处理的图像批次，同时输出预测的类别 logits（每张图像 102 个）和预测的边界框（每张图像 1 个）。训练模型之后，可以这样使用它：

```
preproc_images = [...]  # a batch of preprocessed images
y_pred_logits, y_pred_bbox = locator_model(preprocessed_images.to(device))
```

如何训练这个模型？第 10 章介绍过如何训练具有两个或更多输出的模型，这里没有区别：分类头使用 **nn.CrossEntropyLoss**，定位头使用 **nn.MSELoss**，最终损失可以是两者的加权和。

不过，Flowers102 数据集并不包含边界框，因此需要自己添加。这通常是机器学习项目中最困难、成本最高的部分之一：为数据添加标签和注释。值得花时间寻找合适的工具。可以使用 Label Studio、OpenLabeler、ImgLab、Labelme、VoTT 或 VGG Image Annotator 等开源标注工具，也可以使用 Labelbox、Supervisely、Roboflow 或 RectLabel 等商业工具。现在其中许多工具都带有 AI 辅助功能，可以显著加快标注任务。如果需要标注大量图像，还可以考虑 Amazon Mechanical Turk 等众包平台。不过，设置众包平台、准备发给工作人员的表单、监督工作并保证生成的边界框质量，需要相当多的工作，因此应确保投入值得。如果只需标注几百或几千张图像，并且不打算频繁执行这类工作，自己完成可能更合适：使用合适的工具只需几天，还能更好地理解数据集和任务。

接下来可以创建一个自定义数据集（见第 10 章），每个条目包含图像、标签和边界框。TorchVision 方便地提供了 **BoundingBoxes** 类，用于表示边界框列表。例如，下面的代码为 Flowers102 训练集第一张图像中最大的花朵创建一个边界框（此处暂时只考虑每张图像一个边界框，本章稍后再讨论每张图像多个边界框）：

```
import torchvision.tv_tensors
bbox = torchvision.tv_tensors.BoundingBoxes(
    [[377, 199, 248, 262]],  # center x=377, center y=199, width=248, height=262
    format="CXCYWH",  # other possible formats: "XYXY" and "XYWH"
    canvas_size=(500, 754)  # raw image size before preprocessing
)
```

> 提示：要可视化边界框，可以使用 **torchvision.utils.draw_bounding_boxes()** 函数。首先需要使用 **torchvision.ops.box_convert()** 将边界框转换为 XYXY 格式。

BoundingBoxes 类是 **TVTensor** 的子类，而 **TVTensor** 又是 **torch.Tensor** 的子类，因此可以像处理普通张量一样处理边界框，同时还拥有额外功能。最重要的是，可以使用 TorchVision 的 transforms API v2 对边界框进行变换。例如，使用前面定义的变换对训练集中的第一张花朵图像及其边界框进行预处理，同时保持标签不变。在这个例子中，输入和输出都是包含图像和字典的二元组；字典包含标签和边界框，但也可以使用任意其他数据结构：

```
first_image = [...]  # load the first training image without any preprocessing
preproc_image, preproc_target = transform(
    (first_image, {"label": 0, "bbox": bbox})
)
preproc_bbox = preproc_target["bbox"]
```

> 警告：调整大小和裁剪边界框的行为符合预期，但旋转边界框比较特殊：由于边界框没有旋转参数，无法直接旋转，因此会将其调整为能够包住旋转后边界框的大小。结果可能会比物体本身略大。

在使用 MSE 时，大边界框 10 个像素的误差与小边界框 10 个像素的误差会受到同样惩罚。为避免这种情况，可以使用自定义损失函数，在计算 MSE 前分别对目标和预测的宽度、高度取平方根。

MSE 简单且通常能够很好地训练模型，但它不是评估边界框预测效果的好指标。最常见的指标是交并比（intersection over union，IoU，也称 Jaccard 指数）：它是目标边界框 `T` 与预测边界框 `P` 的重叠面积除以它们并集的面积（见图 12-24）。简而言之，`IoU = overlap(P, T) / union(P, T)`，其中 `|x|` 表示 `x` 的面积。IoU 范围为 0（完全不重叠）到 1（完全重叠），由 `torchvision.ops.box_iou()` 函数实现。

IoU 不适合训练，因为只要 `P` 和 `T` 没有重叠，它就等于 0，而不论两者的距离或形状如何；此时梯度也等于 0，因此梯度下降无法取得进展。幸运的是，可以加入额外信息修复这个问题。例如，2019 年论文提出的广义 IoU（Generalized IoU，GIoU）会考虑同时包含 `P` 和 `T` 的最小框 `S`，并从 IoU 中减去 `S` 中没有被 `P` 或 `T` 覆盖部分的比例。简而言之，`GIoU = IoU - uncovered(S, P, T) / area(S)`。这意味着 `P` 和 `T` 距离越远，GIoU 越小，为梯度下降提供了把 `P` 拉近 `T` 的方向。由于目标是最大化 GIoU，GIoU 损失等于 `1 - GIoU`。这种损失很快流行起来，由 `torchvision.ops.generalized_box_iou_loss()` 函数实现。

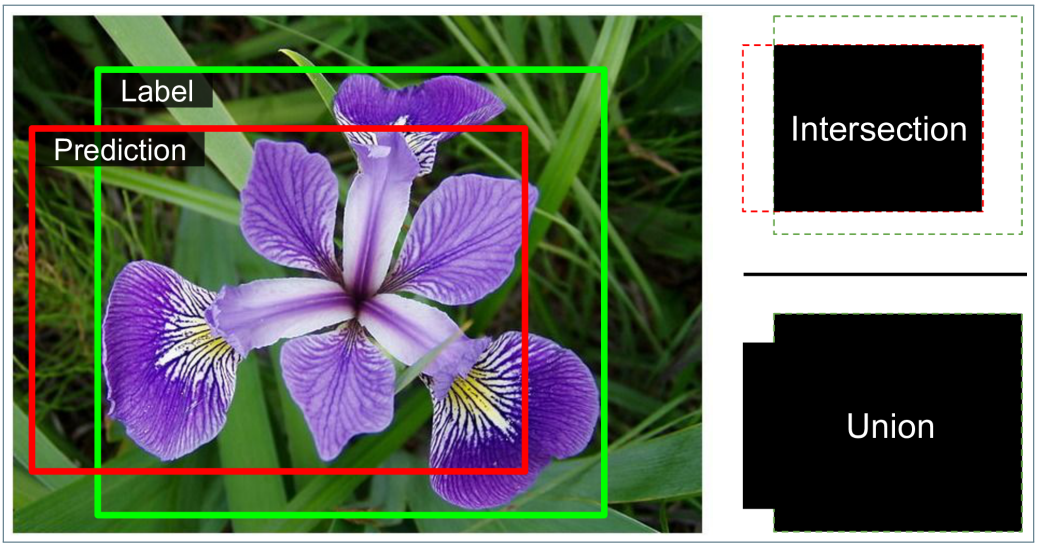

图 12-23: 边界框的 IoU 指标（IoU metric for bounding boxes）

IoU 的另一个重要变体是完全 IoU（Complete IoU，CIoU），由 2020 年的一篇论文提出。它考虑三个几何因素：IoU（重叠越多越好）、P 和 T 中心之间的距离（越近越好，以 S 对角线长度归一化），以及 P 和 T 长宽比之间的相似性（越接近越好）。损失为 `1 - CIoU`，由 `torchvision.ops.complete_box_iou_loss()` 函数实现。它通常比 MSE 或 GIoU 表现更好，收敛更快，边界框更准确，因此逐渐成为定位任务的默认损失。

单独分类和定位一个物体已经很有用，但如果图像包含多个物体（例如花卉数据集经常如此），该怎么办？

12.10　目标检测（Object Detection）

在图像中分类并定位多个物体的任务称为目标检测。直到几年前，一种常见方法是先训练一个 CNN，让它对大致位于图像中心的单个物体进行分类和定位，然后让这个 CNN 在图像上滑动，在每一步都做一次预测。CNN 通常不仅预测类别概率和边界框，还预测一个物体性分数（objectness score）：这是图像中间附近是否确实包含物体的估计概率。它是一个二元分类输出，可以使用带 sigmoid 激活函数的单元输出层，并使用二元交叉熵损失训练。

> 注意：有时也会加入“无物体”类别来替代物体性分数，但通常效果不好。“是否存在物体”和“物体是什么类型”最好分开回答。

这种滑动 CNN 方法如图 12-25 所示。在这个例子中，图像被切成 5 × 7 的网格，CNN（粗黑色矩形）在所有 3 × 3 区域上滑动，并在每一步进行预测。

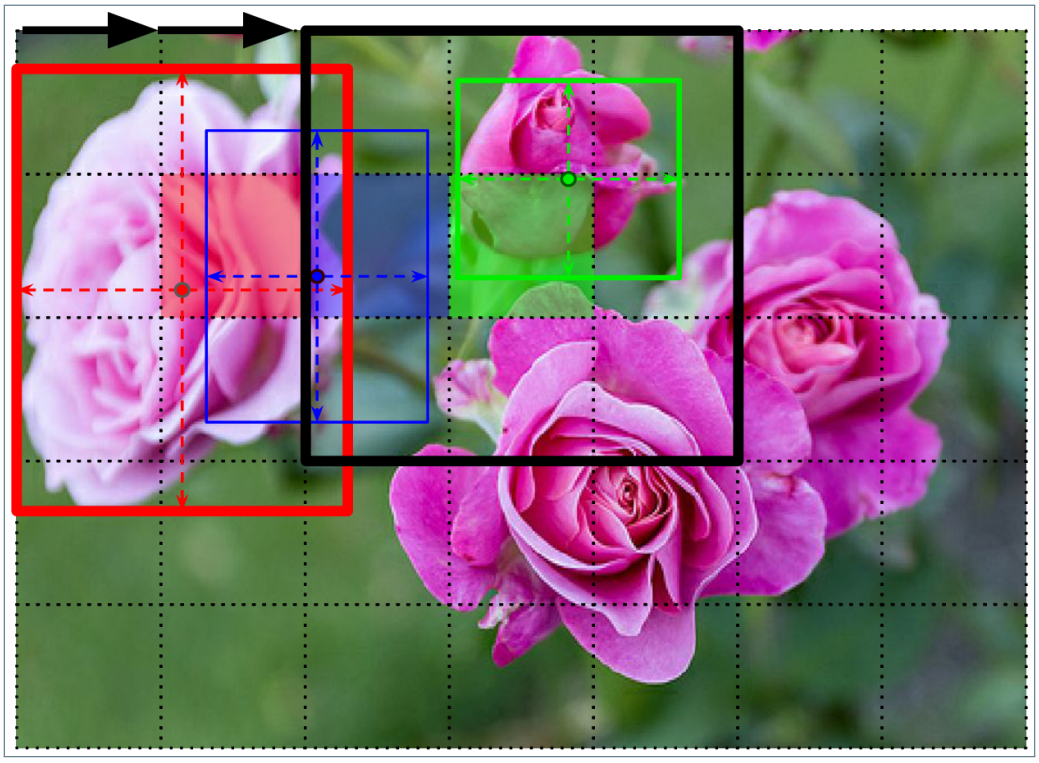

图 12-24: 通过在图像上滑动 CNN 检测多个物体（Detecting multiple objects by sliding a CNN across the image）

图中 CNN 已经对三个 3 × 3 区域作出预测：

- 查看左上角的 3 × 3 区域（中心位于第二行第二列红色阴影网格单元），它检测到了最左侧的玫瑰。注意，预测的边界框超过了该 3 × 3 区域的边界，这完全没有问题：即使 CNN 看不到玫瑰的底部，也可以合理猜测它的位置。它还预测了类别概率，并为“玫瑰”类别给出高概率。最后，它预测了一个相当高的物体性分数，因为边界框中心位于中央网格单元内（图中物体性分数用边界框线条的粗细表示）。
- 查看下一个向右移动一个网格单元的 3 × 3 区域（中心位于蓝色阴影方格），它没有检测到中心位于该区域的花朵，因此预测了很低的物体性分数；预测的边界框和类别概率可以安全地忽略。事实上，可以看到预测的边界框本来就不准确。
- 最后，再向右移动一个网格单元，查看中心位于绿色单元的下一个 3 × 3 区域，它检测到了顶部的玫瑰，但并不完美。这个玫瑰在该区域内没有很好地居中，因此预测的物体性分数并不高。

可以想象，让 CNN 在整个图像上滑动，会得到总共 15 个预测边界框，排列成一个 3 × 5 的网格，每个边界框都带有估计的类别概率和物体性分数。由于物体大小不同，还可以让 CNN 再次在 2 × 2 和 4 × 4 区域上滑动，以捕获更小和更大的物体。

这种技术相当直接，但正如你所见，它经常会在略有不同的位置多次检测同一个物体。需要进行一些后处理，去除多余的边界框。常见方法称为非极大值抑制（non-maximum suppression，NMS），其工作方式如下：

1. 首先删除物体性分数低于某个阈值的所有边界框；既然 CNN 认为该位置没有物体，这些边界框就没有用。
2. 找到剩余边界框中物体性分数最高的一个，删除与它高度重叠的所有其他边界框（例如 IoU 大于 60% 的框）。
3. 重复第 2 步，直到只剩下不相互重叠的边界框。

12.10.1　全卷积网络（Fully Convolutional Networks）

使用滑动 CNN 的问题是，模型必须在图像上重复运行多次，每个滑动位置都要重新计算许多相同的特征。幸运的是，可以把 CNN 转换为全卷积网络（fully convolutional network，FCN），避免重复计算。

要把稠密层转换为卷积层，卷积层中的滤波器数量必须等于稠密层中的单元数量，滤波器大小必须等于输入特征图的大小，并且必须使用 **`valid`** 填充。步幅可以设为 1 或更大。

为什么这很重要？稠密层期望特定的输入大小，因为每个输入特征都有一个权重；卷积层则可以处理任意大小的图像[27]（但它要求输入具有特定数量的通道，因为每个卷积核都为每个输入通道包含不同权重）。由于 FCN 只包含卷积层和池化层（两者都具有这个性质），因此可以在任意大小的图像上训练和执行。

例如，假设已经训练了一个用于花卉分类和定位的 CNN，并额外增加了一个物体性头。它在 224 × 224 的图像上训练，并为每张图像输出 107 个值：

- 分类头输出 102 个类别 logits（每个类别一个），使用 **`nn.CrossEntropyLoss`** 训练。
- 物体性头输出一个物体性 logit，使用 **`nn.BCELoss`** 训练。
- 定位头输出描述边界框的四个数，使用 CIoU 损失训练。

现在可以把 CNN 的稠密层（**`nn.Linear`**）转换为卷积层（**`nn.Conv2d`**）。事实上，甚至不需要重新训练模型，只需把稠密层的权重复制到卷积层即可；也可以在训练前就把 CNN 转换为 FCN。

假设输入 224 × 224 图像时，输出层之前的最后一个卷积层（也称瓶颈层）输出 7 × 7 的特征图（见图 12-26 左侧）。例如，如果网络包含 5 个步幅为 2、采用 **`same`** 填充的层，空间维度总共会缩小 **`2**5 = 32`** 倍，就会出现这种情况。如果向 FCN 输入 448 × 448 的图像（见图 12-26 右侧），瓶颈层就会输出 14 × 14 的特征图。由于稠密输出层被替换为一个使用

[27] 见本章参考文献第 34 条。

107 个 7 × 7 滤波器、采用 **valid** 填充、步幅为 1 的卷积层，最终输出由 107 张 8 × 8 的特征图组成（因为 **14 – 7 + 1 = 8**）。

换句话说，FCN 只处理一次整张图像，并输出一个 8 × 8 的网格，每个单元包含图像某个区域的预测：表示 102 个类别概率、1 个物体性分数和 4 个边界框坐标的 107 个数。这与让原始 CNN 在图像上滑动完全等价，只是每行和每列滑动 8 步。可以把原始图像想象成切成 14 × 14 的网格，然后让一个 7 × 7 窗口在这个网格上滑动；窗口有 **8 × 8 = 64** 个可能位置，因此得到 8 × 8 组预测。但 FCN 方法高效得多，因为网络只需查看图像一次。

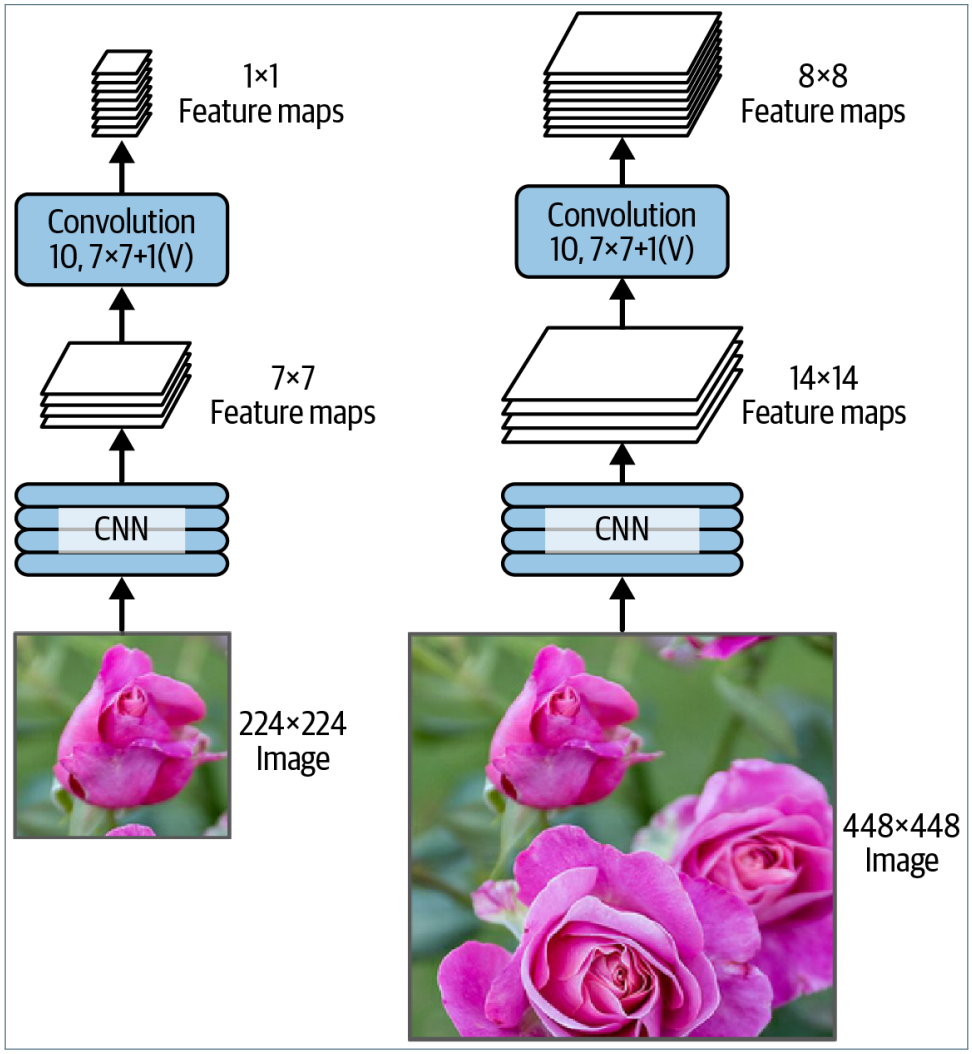

图 12-25: 同一个全卷积网络处理小图像（左）和大图像（右）(The same fully convolutional network processing a small image (left) and a large one (right))

实际上，You Only Look Once（YOLO）就是一种非常流行的目标检测架构，下面来看看它。

12.10.2　YOLO（You Only Look Once）

YOLO 是 Joseph Redmon 等人在 2015 年论文中提出的快速、准确的目标检测架构[28]。它速度非常快，可以实时处理视频。YOLO 的架构与前面讨论的架构相似，但有几个重要差异：

- 对每个网格单元，YOLO 只考虑边界框中心位于该单元内的物体。
- 边界框坐标相对于该单元表示：`(0, 0)` 表示单元左上角，`(1, 1)` 表示单元右下角。不过，边界框的高度和宽度可以远远超出该单元。
- 每个网格单元输出两个边界框，而不是一个，因此可以处理两个物体非常接近、其边界框中心落在同一单元内的情况。每个边界框还有自己的物体性分数。
- YOLO 还为每个网格单元输出一个类别概率分布。由于 YOLO 在包含 20 个类别的 PASCAL VOC 数据集上训练，因此每个网格单元预测 20 个类别概率。这样就得到一个粗粒度的类别概率图。注意，模型对每个网格单元预测一个类别概率分布，而不是对每个边界框预测一个分布。

不过，在后处理中可以根据每个边界框与每个类别在类别概率图中的匹配程度，估计每个边界框的类别概率。例如，想象一张有人站在汽车前面的图片。这里有两个边界框：一个包围汽车的大型水平框，以及一个包围人的较小垂直框。两个边界框的中心可能落在同一网格单元内。那么如何判断每个边界框应该分配什么类别？类别概率图会包含一个以“汽车”为主导的大区域，其中还有一个以“人”为主导的小区域。希望汽车边界框与“汽车”区域大致匹配，而人的边界框与“人”区域大致匹配，于是就能为每个边界框分配正确类别。

YOLO 最初使用 Darknet 开发。Darknet 是 Joseph Redmon 最初用 C 开发的开源深度学习框架，但很快被移植到 PyTorch 和其他库中。此后 YOLO 不断改进：最初由 Joseph Redmon 等人开发 YOLOv2、YOLOv3 和 YOLO9000，2020 年以后则由不同团队继续发展。每个版本都通过多种技术在速度和准确率上取得了令人印象深刻的提升。例如，YOLOv3 部分通过使用锚框先验提高了准确率，因为某些类别对应的边界框形状比其他形状更常见（例如人物通常有垂直边界框，而汽车通常没有）。后续版本还增加了每个网格单元的边界框数量，使用包含更多类别的数据集进行训练（YOLO9000 的类别数量最多可达 9,000 个，并按层次组织），增加跳跃连接来恢复 CNN 中损失的一些空间分辨率（讨论语义分割时会再次介绍），等等。还有许多模型变体，例如为较弱机器优化、可以极快运行（每秒超过 1,000 帧）但平均精度略低的缩小版“tiny”YOLO。

12.10.3　平均精度均值（Mean Average Precision）

目标检测任务中一个非常常见的指标是平均精度均值（mean average precision）。听起来“平均精度均值”有些重复，对吧？要理解这个指标，可以回顾第 3 章讨论的两个分类指标：

[28] 见本章参考文献第 35 条。

精确率和召回率。记住它们之间的权衡：通常召回率越高，精确率越低。可以用精确率/召回率曲线可视化这种权衡（见图 3-6）。

为了把曲线总结成一个数，可以计算曲线下面积（AUC）。但精确率/召回率曲线可能包含精确率随着召回率增加而上升的片段，尤其是在低召回率处，这正是 mAP 指标的动机之一。

假设分类器在召回率为 10% 时精确率为 90%，在召回率为 20% 时精确率为 96%。这里其实没有真正的权衡：与其在召回率为 10% 时使用分类器，不如在召回率为 20% 时使用，因为此时召回率和精确率都更高。因此，我们不应该查看召回率为 10% 时的精确率，而应该查看分类器在召回率至少为 10% 时能提供的最大精确率。这个值是 96%，而不是 90%。因此，一种公平评估模型表现的方法，是分别计算在召回率至少为 0%、10%、20%……100% 时可以达到的最大精确率，然后计算这些最大精确率的均值。这称为平均精度（average precision，AP）。当类别多于两个时，可以计算每个类别的 AP，再计算平均 AP，也就是 mAP。TorchMetrics 库中的 **MeanAveragePrecision** 指标方便地实现了全部计算。

目标检测系统还有一个额外复杂性：如果系统检测到了正确类别，但位置错误（也就是边界框完全偏离），显然不应把它算作正确预测。一种方法是定义 IoU 阈值，例如只有当 IoU 大于 0.5 且预测类别正确时，才认为预测正确。对应的 mAP 通常写作 mAP@0.5（也写作 mAP@50%，有时简称 AP50）。PASCAL VOC 挑战赛就采用这种方法。在其他比赛（如 COCO）中，mAP 会在多个 IoU 阈值（0.50、0.55、0.60、……、0.95）下计算，最终指标是所有这些 mAP 的平均值，记作 mAP@[.50:.95] 或 mAP@[.50:0.05:.95]。是的，这是一个“平均的平均”。

TorchVision 不包含 YOLO 模型，但可以使用 Ultralytics 库。它提供了一个简单 API，用于下载和使用各种基于 PyTorch 的预训练 YOLO 模型。这些模型在 COCO 数据集上预训练，该数据集包含超过 330,000 张图像，其中 200,000 张图像带有 80 个不同类别的目标检测标注（人、汽车、卡车、自行车、球等）。Colab 默认没有安装 Ultralytics，因此需要运行**%pip install ultralytics**。下面使用该库下载 YOLOv9 模型（medium 变体），并检测一批图像中的物体；模型接受 PIL 图像、NumPy 数组，甚至 URL：

```
from ultralytics import YOLO
model = YOLO('yolov9m.pt')  # n=nano, s=small, m=medium, x=large
images = ["https://homl.info/soccer.jpg", "https://homl.info/traffic.jpg"]
results = model(images)
```

输出是一个 **Results** 对象列表，提供方便的 **summary()** 方法。例如，下面查看第一张图像中第一个检测到的物体：

```
>>> results[0].summary()[0]
{'name': 'sports ball',
 'class': 32,
 'confidence': 0.96214,
 'box': {'x1': 245.35733, 'y1': 286.03003, 'x2': 300.62509, 'y2': 343.57184}}
```

> 提示：Ultralytics 库还提供简单 API，可以在其他常见目标检测数据集或自己的数据集上训练 YOLO 模型。更多细节见 https://docs.ultralytics.com/modes/train。

TorchVision 还提供几个其他预训练目标检测模型。使用方法与预训练分类模型（如 ConvNeXt）类似，只是每张图像的预测由一个字典表示，包含两个条目：`labels`（类别 ID）和 `boxes`。可用模型包括以下几种（完整变体列表见模型页面）：

- **Faster R-CNN**[29]：该模型分两个阶段。图像首先经过 CNN，输出再送入区域提议网络（region proposal network，RPN），该网络提出最有可能包含物体的边界框；然后根据 CNN 的裁剪输出，对每个边界框运行分类器。
- **SSD**[30]：单阶段检测器，即“只看一次”，与 YOLO 类似。
- **SSDLite**[31]：SSD 的轻量版本，非常适合移动设备。
- **RetinaNet**[32]：单阶段检测器，引入了交叉熵损失的一种变体 focal loss（聚焦损失）。该损失对困难样本和不常见类别中的样本赋予更高权重，因此能改善小物体和少数类别的性能，可使用 `torchvision.ops.sigmoid_focal_loss()`。
- **FCOS**[33]：单阶段全卷积网络，直接预测边界框，不依赖锚框。

12.11　目标跟踪（Object Tracking）

前面只考虑了在单张图像中检测物体。但视频怎么办？不仅要在每一帧中检测物体，还必须随时间跟踪它们。下面快速看一下目标跟踪。

目标跟踪是一个具有挑战性的任务：物体会移动；当它们靠近或远离时，大小可能变大或变小；转身或进入不同光照和背景时，外观可能发生变化；还可能暂时被其他物体遮挡，等等。

[29] 见本章参考文献第 36 条。
[30] 见本章参考文献第 37 条。
[31] 见本章参考文献第 38 条。
[32] 见本章参考文献第 39 条。
[33] 见本章参考文献第 40 条。

最流行的目标跟踪系统之一是 Deep SORT[34]。它结合了经典算法和深度学习：

- 使用 Kalman 滤波器，根据先前的检测结果，并假设物体以恒定速度移动，估计物体当前最可能的位置。
- 使用深度学习模型度量新检测结果与现有被跟踪物体之间的相似度。
- 最后使用匈牙利算法把新检测结果映射到现有被跟踪物体（或新的被跟踪物体）。该算法高效地找到一组映射，使检测结果与被跟踪物体预测位置之间的距离最小，同时也使外观差异最小。

例如，想象一个红球刚刚撞到一个向相反方向运动的蓝球。根据球此前的位置，Kalman 滤波器会预测两个球互相穿过；因为它假设物体以恒定速度移动，所以不会预期发生碰撞。如果匈牙利算法只考虑位置，就会很自然地把新检测结果映射到错误的球，好像两个球穿过彼此并交换了颜色。但得益于相似度度量，匈牙利算法会发现问题。假设两个球并不太相似，算法就会把新检测结果映射到正确的球。

Ultralytics 库支持目标跟踪，默认使用 Bot-SORT 算法。它与 Deep SORT 非常相似，但由于加入了相机运动补偿并调整了 Kalman 滤波器，因此速度更快、准确率更高[35]。例如，可以使用刚才创建的 YOLOv9 模型执行下面的代码，跟踪视频中的物体。此例还会在每帧打印每个被跟踪物体的 ID，并保存一份带标注的视频（最后会显示视频路径）：

```
my_video = "https://homl.info/cars.mp4"
results = model.track(source=my_video, stream=True, save=True)
for frame_results in results:
    summary = frame_results.summary()  # similar summary as earlier + track id
    track_ids = [obj["track_id"] for obj in summary]
    print("Track ids:", track_ids)
```

12.12　语义分割（Semantic Segmentation）

前面使用边界框定位物体，这通常已经足够，但有时需要更加精确地定位物体，例如在视频会议中移除一个人的背景。下面把定位精确到像素级别。

在语义分割中，每个像素都会根据其所属物体的类别进行分类（例如道路、汽车、行人、建筑物等），如图 12-27 所示。注意，同一类别的不同物体不会彼此区分。例如，分割图像右侧的所有自行车最终会合并成一大块像素。这个任务的主要困难在于：图像经过普通 CNN 时，会由于步幅大于 1 的层而逐渐损失空间分辨率。因此，普通 CNN 可能只知道左下方某处有一个人，却无法比这更精确地定位。

[34] 见本章参考文献第 41 条。

[35] 见本章参考文献第 42 条。

图 12-26: 语义分割（Semantic segmentation）

与目标检测一样，解决这个问题有许多不同方法，其中一些相当复杂。不过，Jonathan Long 等人在 2015 年关于全卷积网络的论文[36]中提出了一种相对简单的方案。作者从一个预训练 CNN 出发，把它转换为 FCN。该 CNN 对输入图像施加总步幅 32（即把所有步幅相乘），这意味着最后一层输出的特征图比输入图像小 32 倍。这显然过于粗糙，因此他们增加了一个单独的上采样层，将分辨率提高 32 倍。

上采样（增加图像大小）有多种方案，例如双线性插值，但这种方法在放大到 4 倍或 8 倍以上时效果不太好。因此他们使用转置卷积层[37]：先插入空行和空列（全为 0）拉伸图像，再执行普通卷积（见图 12-28）。也可以把它理解为使用分数步幅的普通卷积层（例如图 12-28 中步幅为 1/2）。转置卷积层可以初始化为近似执行线性插值，但由于它是可训练层，训练期间会学得更好。在 PyTorch 中，可以使用 `nn.ConvTranspose2d` 层。

[36] 见本章参考文献第 33 条。

[37] 见本章参考文献第 43 条。

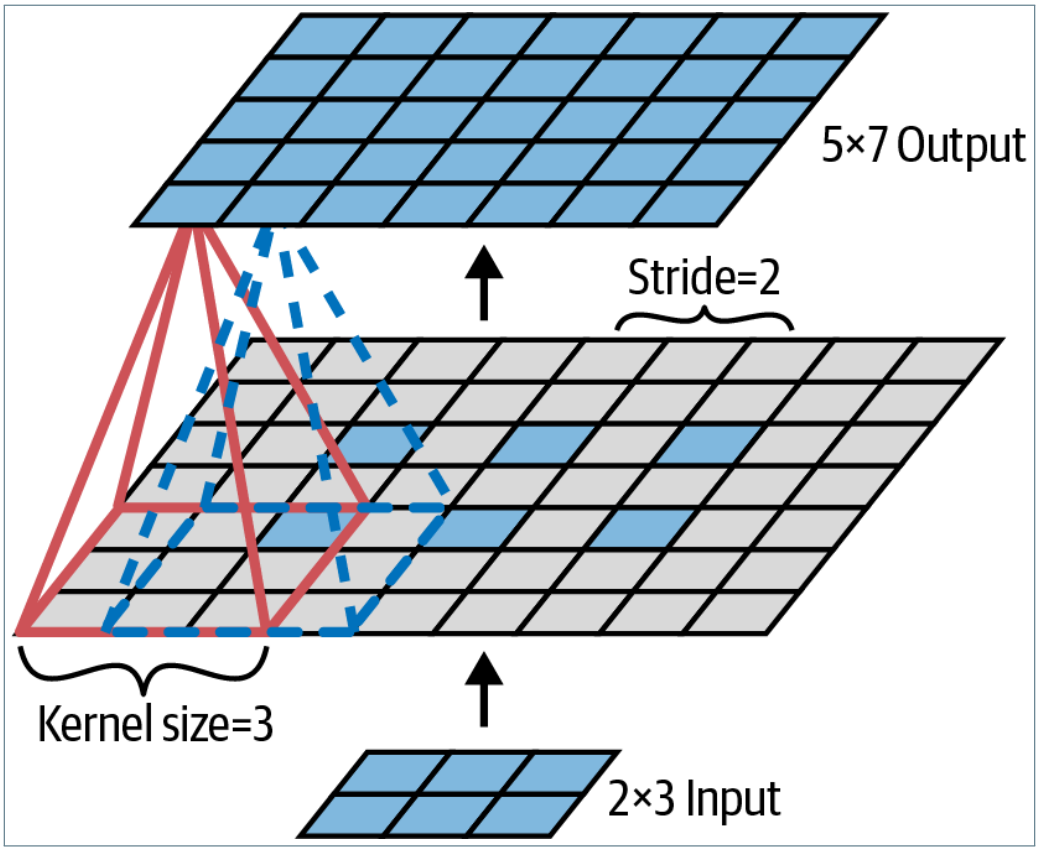

图 12-27: 使用转置卷积层进行上采样（Upsampling using a transposed convolutional layer）

> 注意：在转置卷积层中，步幅定义输入被拉伸的程度，而不是滤波器步进的大小。因此，步幅越大，输出越大；这与卷积层和池化层相反。

12.12.1　其他 PyTorch 卷积层（Other PyTorch Convolutional Layers）

PyTorch 还提供其他几种卷积层：

‘nn.Conv1d‘：用于一维输入的卷积层，例如时间序列或文本（字母或单词序列），第 13 章会看到。

‘nn.Conv3d‘：用于三维输入的卷积层，例如三维 PET 扫描。

空洞卷积（à-trous convolution）：将任意卷积层的 `dilation` 超参数设为 2 或更大，就会创建一个空洞卷积层（à trous 是法语“带孔”的意思）。它等价于使用一个通过插入零行和零列（即孔洞）而扩张的滤波器。例如，值为 `[[1,2,3]]` 的 1 × 3 滤波器，以扩张率 4 进行扩张后，会变成 `[[1, 0, 0, 0, 2, 0, 0, 0, 3]]`。这样可以在不增加计算量和参数的情况下，让卷积层拥有更大的感受野。

使用转置卷积层进行上采样虽然不错，但仍然不够精确。为了取得更好结果，Long 等人从低层加入了跳跃连接：例如，他们只把输出图像上采样 2 倍（而不是 32 倍），然后加上一个具有两倍分辨率的低层输出。接着把结果上采样 16 倍，最终总上采样倍数仍为 32（见图 12-29）。这恢复了早期池化层丢失的一些空间分辨率。在最佳架构中，他们又使用了第二个类似跳跃连接，从更低层恢复更细的细节。简而言之，原始 CNN 的输出会经过以下额外步骤：上采样 ×2，加上适当尺度的低层输出；再上采样 ×2，加上更低层的输出；最后上采样 ×8。甚至可以放大到超过原图像大小，从而提高图像分辨率，这种技术称为超分辨率。

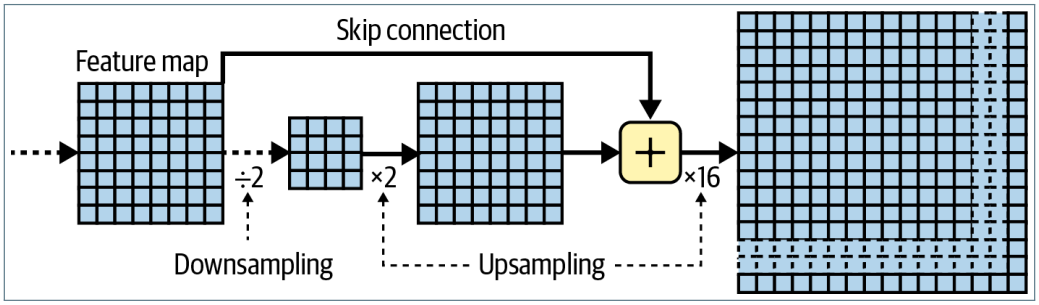

图 12-28: 跳跃层从低层恢复部分空间分辨率（Skip layers recover some spatial resolution from lower layers）

> 提示：FCN 模型以及另外几个语义分割模型都包含在 TorchVision 中，可以在 notebook 中查看代码示例。

实例分割与语义分割类似，但它不会把同一类别的所有物体合并为一大块，而是区分每个物体（例如识别出每一辆自行车）。例如，Kaiming He 等人在 2017 年论文中提出的 Mask R-CNN 架构[38]扩展了 Faster R-CNN 模型，为每个边界框额外生成一个像素掩码。因此，除了得到每个物体周围的边界框和一组估计类别概率之外，还能得到一个像素掩码，定位边界框中属于该物体的像素。该模型在 COCO 2017 数据集上预训练过，并包含在 TorchVision 中。

> 提示：得益于 Video 和 Mask TVTensor，TorchVision 的 transforms API v2 可以像处理边界框一样处理掩码和视频。

如你所见，深度计算机视觉领域广阔且发展迅速，各种架构每年不断涌现。如果想尝试最新、最优秀的模型，可以查看 Hugging Face 的热门论文：https://huggingface.co/papers。过去大多数模型都基于卷积神经网络，但从 2020 年开始，另一种神经网络架构进入了计算机视觉领域：Transformer（第 14 章将讨论）。过去 15 年的进步令人惊叹，如今研究人员正在关注越来越困难的问题，例如对抗学习（让网络更能抵抗专门设计来欺骗它的图像）、可解释性（理解网络为什么作出某个分类）、逼真图像生成（第 18 章会再次讨论）、单样本学习（系统只看过一次某个物体就能识别它）、预测视频的下一帧、组合文本和图像任务等。

[38] 见本章参考文献第 44 条。

下面进入下一章：使用循环神经网络和卷积神经网络处理时间序列等序列数据。

12.13　练习（Exercises）

1. 与用于图像分类的全连接 DNN 相比，CNN 有哪些优点？
2. 考虑一个由三个卷积层组成的 CNN，每层使用 3 × 3 核、步幅为 2 和 **same** 填充。最低层输出 100 张特征图，中间层输出 200 张，最高层输出 400 张。输入图像是 200 × 300 像素的 RGB 图像：

- a. CNN 中的参数总数是多少？
- b. 如果使用 32 位浮点数，网络为单个实例进行预测时至少需要多少 RAM？
- c. 在包含 50 张图像的小批量上训练时呢？

3. 如果 GPU 在训练 CNN 时内存不足，可以尝试哪五种方法解决问题？
4. 为什么要加入最大池化层，而不是加入一个使用相同步幅的卷积层？
5. 你能说出 AlexNet 相比 LeNet-5 的主要创新吗？GoogLeNet、ResNet、SENet、Xception、EfficientNet 和 ConvNeXt 的主要创新又是什么？
6. 什么是全卷积网络？如何把稠密层转换为卷积层？
7. 语义分割的主要技术难点是什么？
8. 从零开始构建自己的 CNN，尝试在 MNIST 上取得尽可能高的准确率。
9. 使用迁移学习进行大图像分类，完成以下步骤：

- a. 为每个类别创建至少包含 100 张图像的训练集。例如，可以根据位置（海滩、山地、城市等）对自己的照片分类。也可以使用现有数据集，例如 PyTorch 迁移学习计算机视觉教程中使用的数据集。
- b. 将其拆分成训练集、验证集和测试集。
- c. 构建输入流水线，应用适当的预处理操作，并可选地加入数据增强。
- d. 在该数据集上微调预训练模型。

10. 完成 PyTorch 的目标检测微调教程。

这些练习的解答位于本章 notebook 的末尾：https://homl.info/colab-p。

12.13.1　参考文献

1. David H. Hubel, “Single Unit Activity in Striate Cortex of Unrestrained Cats,”The Journal of Physiology 147 (1959): 226-238.
2. David H. Hubel and Torsten N. Wiesel, “Receptive Fields of Single Neurons in the Cat’s Striate Cortex,”The Journal of Physiology 148 (1959): 574-591.
3. David H. Hubel and Torsten N. Wiesel, “Receptive Fields and Functional Architecture of Monkey Striate Cortex,”The Journal of Physiology 195 (1968): 215-243.
4. Kunihiko Fukushima, “Neocognitron: A Self-Organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position,”Biological Cybernetics 36 (1980): 193-202.
5. Yann Le Cun et al., “Gradient-Based Learning Applied to Document Recognition,”Proceedings of the IEEE 86, no. 11 (1998): 2278-2324.
6. 卷积是一个数学运算：将一个函数滑过另一个函数，并计算它们逐点乘积的积分。它与 Fourier 变换和 Laplace 变换有深刻联系，在信号处理中得到广泛使用。卷积层实际使用的是互相关，它与卷积非常相似。更多细节见 https://homl.info/76。
7. 前面讨论的其他核具有权重，但池化核没有权重；它们只是无状态的滑动窗口。
8. Yann Le Cun et al., “Gradient-Based Learning Applied to Document Recognition,”Proceedings of the IEEE 86, no. 11 (1998): 2278-2324.
9. Alex Krizhevsky et al., “ImageNet Classification with Deep Convolutional Neural Networks,” Proceedings of the 25th International Conference on Neural Information Processing Systems 1 (2012): 1097-1105.
10. Matthew D. Zeiler and Rob Fergus, “Visualizing and Understanding Convolutional Networks,”Proceedings of the European Conference on Computer Vision (2014): 818-833.
11. Christian Szegedy et al., “Going Deeper with Convolutions,”Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (2015): 1-9.
12. 在 2010 年的电影《盗梦空间》（Inception）中，角色不断深入多层梦境，因此这种模块以此命名。
13. Kaiming He et al., “Deep Residual Learning for Image Recognition,”arXiv preprint arXiv:1512.03385 (2015).
14. Gao Huang, Yu Sun, et al., “Deep Networks with Stochastic Depth,”arXiv preprint arXiv:1603.09382 (2016).
15. 描述神经网络时，只统计带参数的层是一种常见做法。
16. Christian Szegedy et al., “Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning,”arXiv preprint arXiv:1602.07261 (2016).

17. François Chollet, “Xception: Deep Learning with Depthwise Separable Convolutions,”arXiv preprint arXiv:1610.02357 (2016).
18. 这个名称有时会产生歧义，因为空间可分离卷积也经常被称为“可分离卷积”。
19. Jie Hu et al., “Squeeze-and-Excitation Networks,”Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (2018): 7132-7141.
20. Karen Simonyan and Andrew Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,”arXiv preprint arXiv:1409.1556 (2014).
21. Saining Xie et al., “Aggregated Residual Transformations for Deep Neural Networks,”arXiv preprint arXiv:1611.05431 (2016).
22. Gao Huang et al., “Densely Connected Convolutional Networks,”arXiv preprint arXiv:1608.06993 (2016).
23. Andrew G. Howard et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,”arXiv preprint arXiv:1704.04861 (2017).
24. Chien-Yao Wang et al., “CSPNet: A New Backbone That Can Enhance Learning Capability of CNN,”arXiv preprint arXiv:1911.11929 (2019).
25. Mingxing Tan and Quoc V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,”arXiv preprint arXiv:1905.11946 (2019).
26. Zhuang Liu et al., “A ConvNet for the 2020s,”arXiv preprint arXiv:2201.03545 (2022).
27. 在国际单位制（SI）中，1 MB = 1,000 KB = 1,000 × 1,000 字节 = 1,000 × 1,000 × 8 位；1 MiB = 1,024 KiB = 1,024 × 1,024 字节。因此 12 MB ≈ 11.44 MiB。
28. Aidan Gomez et al., “The Reversible Residual Network: Backpropagation Without Storing Activations,”arXiv preprint arXiv:1707.04585 (2017).
29. M. Nilsback and A. Zisserman, “Automated Flower Classification over a Large Number of Classes,”Proceedings of the Indian Conference on Computer Vision, Graphics and Image Processing (2008).
30. TorchVision PR #8838 可能在你读到这些文字时已经修复了这个问题。
31. H. Rezatofighi et al., “Generalized Intersection over Union: A Metric and A Loss for Bounding Box Regression,”arXiv preprint arXiv:1902.09630 (2019).
32. Z. Zheng et al., “Enhancing Geometric Factors in Model Learning and Inference for Object Detection and Instance Segmentation,”arXiv preprint arXiv:2005.03572 (2020).
33. Jonathan Long et al., “Fully Convolutional Networks for Semantic Segmentation,”Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (2015): 3431-3440.
34. 有一个小例外：使用 `valid` 填充的卷积层在输入尺寸小于核大小时会报错。

35. Joseph Redmon et al., “You Only Look Once: Unified, Real-Time Object Detection,”Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (2016): 779-788.
36. Shaoqing Ren et al., “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,”Proceedings of the 28th International Conference on Neural Information Processing Systems 1 (2015): 91-99.
37. Wei Liu et al., “SSD: Single Shot MultiBox Detector,”Proceedings of the 14th European Conference on Computer Vision 1 (2016): 21-37.
38. Mark Sandler et al., “MobileNetV2: Inverted Residuals and Linear Bottlenecks,”arXiv preprint arXiv:1801.04381 (2018).
39. Tsung-Yi Lin et al., “Focal Loss for Dense Object Detection,”arXiv preprint arXiv:1708.02002 (2017).
40. Zhi Tian et al., “FCOS: Fully Convolutional One-Stage Object Detection,”arXiv preprint arXiv:1904.01355 (2019).
41. Nicolai Wojke et al., “Simple Online and Realtime Tracking with a Deep Association Metric,” arXiv preprint arXiv:1703.07402 (2017).
42. Nir Aharon et al., “BoT-SORT: Robust Associations Multi-Pedestrian Tracking,”arXiv preprint arXiv:2206.14651 (2022).
43. 这种层有时被称为反卷积层（deconvolution layer），但它并没有执行数学家所说的反卷积，因此应避免使用这个名称。
44. Kaiming He et al., “Mask R-CNN,”arXiv preprint arXiv:1703.06870 (2017).

第 13 章　序列处理：RNN 与 CNN

Processing Sequences Using RNNs and CNNs

预测未来是我们一直在做的事情：无论是替朋友把话说完，还是在早餐时预想咖啡的香味。本章将讨论循环神经网络（recurrent neural network，RNN）——一种能够预测未来（当然只能在一定程度上预测）的网络。RNN 可以分析时间序列数据，例如网站每日活跃用户数、所在城市的每小时温度、家庭每日用电量、附近车辆的行驶轨迹等。RNN 学会数据中的历史模式后，就能够利用这些知识预测未来，前提是过去的模式在未来仍然成立。

更一般地说，RNN 可以处理任意长度的序列，而不是固定大小的输入。例如，它们可以接收句子、文档或音频样本作为输入，因此对于自动翻译、语音转文字等自然语言处理（natural language processing，NLP）应用非常有用。

本章首先介绍 RNN 的基本概念，以及如何使用时间反向传播（backpropagation through time，BPTT）训练 RNN。然后，我们将使用 RNN 预测时间序列。在此过程中，会介绍常用于时间序列预测的自回归移动平均（autoregressive moving average，ARMA）模型系列，并把它们作为基线，与 RNN 进行比较。之后，我们将探讨 RNN 面临的两个主要困难：

- 不稳定梯度（第 11 章讨论过），可以使用多种技巧缓解，包括循环 dropout（recurrent dropout）和循环层归一化（recurrent layer normalization）。
- 非常有限的短期记忆，可以使用长短期记忆（long short-term memory，LSTM）和门控循环单元（gated recurrent unit，GRU）单元扩展。

RNN 并不是唯一能够处理序列数据的神经网络类型。对于较短的序列，普通稠密网络就能胜任；对于音频样本或文本等很长的序列，卷积神经网络（convolutional neural network，CNN）实际上也能表现得相当不错。本章还会讨论这两种可能性，并在最后实现 WaveNet——一种能够处理数万个时间步的 CNN 架构。不过，我们还可以做得更好！第 14 章将把 RNN 应用于自然语言处理，并介绍如何使用注意力机制增强 RNN。注意力是 Transformer 的核心，我们将在第 15 章学习 Transformer；如今 Transformer 已成为序列处理、NLP 甚至计算机视觉领域的先进方法。在进入这些内容之前，先从最简单的 RNN 开始。

13.1　循环神经元和循环层（Recurrent Neurons and Layers）

到目前为止，我们主要关注前馈神经网络（feedforward neural network），其中激活值只沿一个方向流动，从输入层流向输出层。循环神经网络看起来与前馈神经网络非常相似，只是它还包含指向后方的连接。

先来看最简单的 RNN：它由一个接收输入、产生输出，并把输出反馈给自身的神经元组成（见图 13-1 左侧）。在每个时间步 t（也称为帧），这个循环神经元接收输入 x(t)，以及它在前一时间步的输出 ŷ(t-1)。由于第一个时间步没有前一时刻的输出，通常将其设为零。我们可以把这个小网络沿时间轴展开（见图 13-1 右侧）。这称为沿时间展开网络（unrolling the network through time），也就是把同一个循环神经元按时间步重复表示。

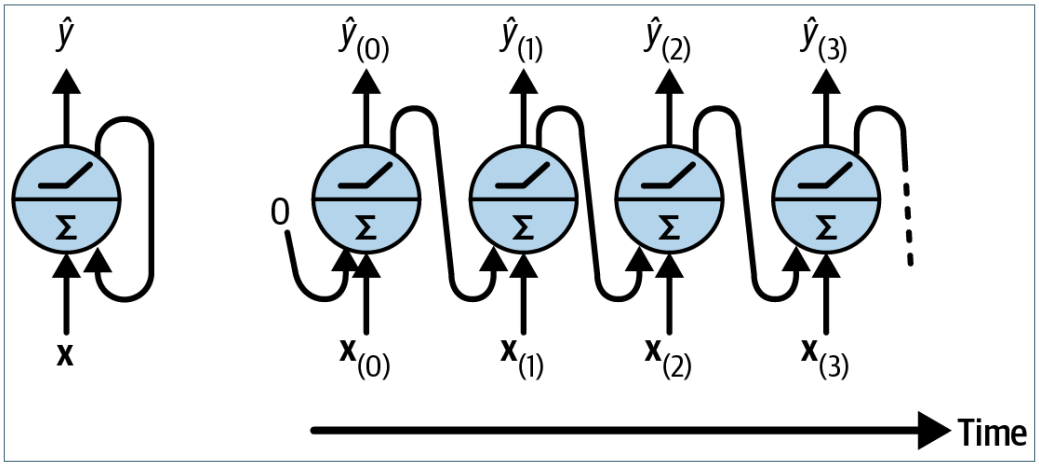

图 13-1: 循环神经元（左）沿时间展开（右）（A recurrent neuron (left) unrolled through time (right)）

很容易创建一个循环神经元层。在每个时间步 t，每个神经元都会同时接收输入向量 x(t) 和前一时间步的输出向量 ŷ(t-1)，如图 13-2 所示。注意，此时输入和输出都是向量；只有一个神经元时，输出才是标量。

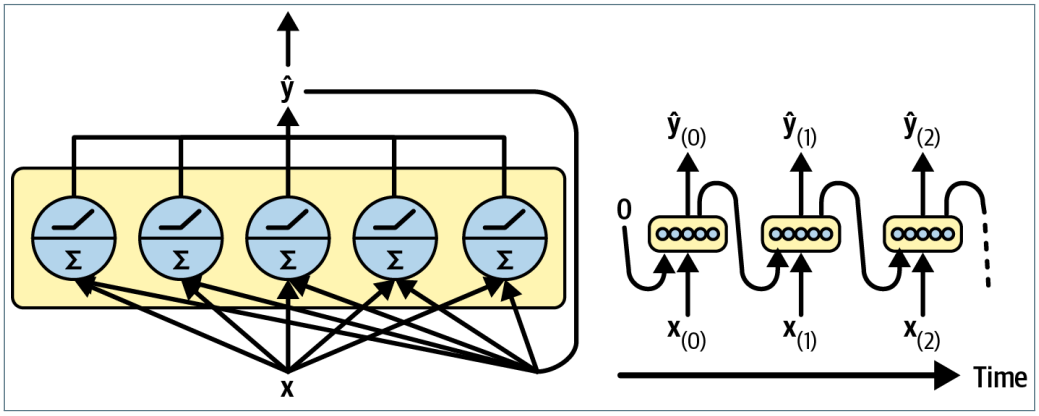

图 13-2: 循环神经元层（左）沿时间展开（右）（A layer of recurrent neurons (left) unrolled through time (right)）

每个循环神经元有两组权重：一组用于当前时间步的输入 x(t)，另一组用于前一时间步的输出 ŷ(t-1)。将这两个权重向量分别记作 w_x 和 w_ŷ。把所有权重向量放入两个权重矩阵

W_x 和 W_ŷ，就可以计算整个循环层的输出，如公式 13-1 所示，其中 b 是偏置向量，phi(·)是激活函数（例如 ReLU）[1]。

$$\hat{\mathbf{y}}^{(t)} = \varphi\left(\mathbf{W}_x^{\intercal}\mathbf{x}^{(t)} + \mathbf{W}_{\hat{y}}^{\intercal}\hat{\mathbf{y}}^{(t-1)} + \mathbf{b}\right) \tag{13-1}$$

公式 13-1　单个实例的循环层输出（Output of a recurrent layer for a single instance）

与前馈神经网络一样，也可以把时间步 t 的所有输入放入输入矩阵 X(t)，一次性计算整个小批量的循环层输出（见公式 13-2）。

$$\begin{aligned}\hat{\mathbf{Y}}^{(t)} &= \varphi\left(\mathbf{X}^{(t)}\mathbf{W}_x + \hat{\mathbf{Y}}^{(t-1)}\mathbf{W}_{\hat{y}} + \mathbf{b}\right)\\ &= \varphi\left(\left[\mathbf{X}^{(t)}\ \hat{\mathbf{Y}}^{(t-1)}\right]\mathbf{W} + \mathbf{b}\right), \qquad \mathbf{W} = \begin{bmatrix}\mathbf{W}_x\\ \mathbf{W}_{\hat{y}}\end{bmatrix}\end{aligned} \tag{13-2}$$

公式 13-2　小批量中所有实例的循环神经元层输出（Outputs of a layer of recurrent neurons for all instances in a mini-batch）

在这个公式中：

- Ŷ(t) 是一个 m × n_neurons 矩阵，包含小批量中每个实例在时间步 t 的层输出；m 是小批量中的实例数，n_neurons 是神经元数。
- X(t) 是一个 m × n_inputs 矩阵，包含所有实例的输入；n_inputs 是输入特征数。
- W_x 是一个 n_inputs × n_neurons 矩阵，包含当前时间步输入的连接权重。
- W_ŷ 是一个 n_neurons × n_neurons 矩阵，包含前一时间步输出的连接权重。
- b 是大小为 n_neurons 的向量，包含每个神经元的偏置项。

权重矩阵 W_x 和 W_ŷ 通常会沿垂直方向拼接成一个权重矩阵 W，其形状为 (n_inputs + n_neurons) × n_neurons（见公式 13-2 的第二行）。记号 [X(t) Ŷ(t-1)] 表示将矩阵 X(t) 和 Ŷ(t-1) 沿水平方向拼接。

注意，Ŷ(t) 是 X(t) 和 Ŷ(t-1) 的函数，而 Ŷ(t-1) 又是 X(t-1) 和 Ŷ(t-2) 的函数，依此类推。因此，Ŷ(t) 实际上是从时间 t = 0 开始的所有输入（即 X(0)、X(1)、⋯、X(t)）的函数。在第一个时间步 t = 0，没有前一时间步的输出，因此通常假设它们全部为零。

> 注意：在人工神经网络中引入向后连接（即循环）的思想，可以追溯到 ANN 的起源。不过，第一个现代 RNN 架构由 Michael I. Jordan 于 1986 年提出。[a]在每个时间步，Jordan 的 RNN 会查看该时间步的输入，以及自身在前一时间步的输出。这称为输出反馈（output feedback）。

[1] 见本章参考文献第 1 条。

[a]见本章参考文献第 2 条。

13.2　记忆单元（Memory Cells）

由于循环神经元在时间步 t 的输出是此前所有时间步输入的函数，因此可以说它具有某种记忆。神经网络中能够在时间步之间保留某些状态的部分，称为记忆单元（memory cell），或简称单元（cell）。单个循环神经元或循环神经元层是非常基础的单元，只能学习短模式（通常约 10 个时间步，但会因任务而异）。本章后面将介绍一些更复杂、更强大的单元，它们能够学习更长的模式（大致可长 10 倍，但同样取决于任务）。

单元在时间步 t 的状态记作 h(t)，其中 h 表示“隐藏”（hidden）。它是该时间步的一些输入和前一时间步状态的函数：h(t) = f(x(t), h(t-1))。单元在时间步 t 的输出记作 ŷ(t)，它也取决于前一状态和当前输入，通常只是当前状态的函数。对于前面讨论的基础单元，输出就等于状态；但对更复杂的单元来说，情况不一定如此，如图 13-3 所示。

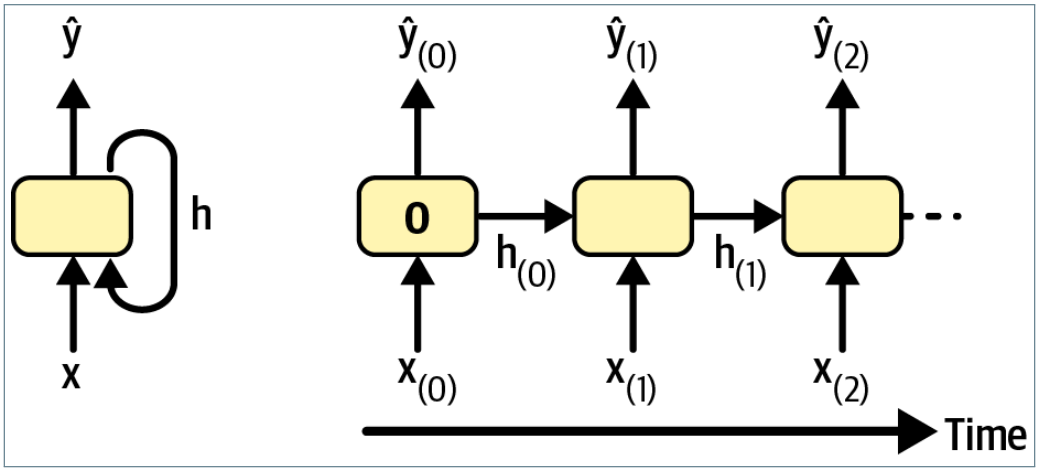

图 13-3: 单元的隐藏状态与输出可能不同（A cell's hidden state and its output may be different）

注意：第一个反馈隐藏状态而不是输出的现代 RNN，由 Jeffrey Elman 于 1990 年提出。[a]这种方法称为状态反馈（state feedback），如今是最常见的方法。

[a]见本章参考文献第 3 条。

13.3　输入和输出序列（Input and Output Sequences）

RNN 可以同时接收输入序列并产生输出序列（见图 13-4 左上方的网络）。这种序列到序列（sequence-to-sequence）网络适合预测时间序列，例如家庭每日用电量：将过去 N 天的数据输入网络，并训练网络输出整体向前移动一天的用电量（即从 N - 1 天前到明天）。

或者，可以向网络输入一个序列，只关注最后一个输出而忽略其他输出（见图 13-4 右上方的网络）。这是序列到向量（sequence-to-vector）网络。例如，可以输入与一篇电影评论对应的单词序列，网络输出情感得分（例如从 0〔厌恶〕到 1〔喜爱〕）。

反过来，也可以在每个时间步重复输入同一个输入向量，让网络输出一个序列（见图 13-4 左下方的网络）。这是向量到序列（vector-to-sequence）网络。例如，输入可以是一张图像（或 CNN 的输出），输出可以是该图像的说明文字。

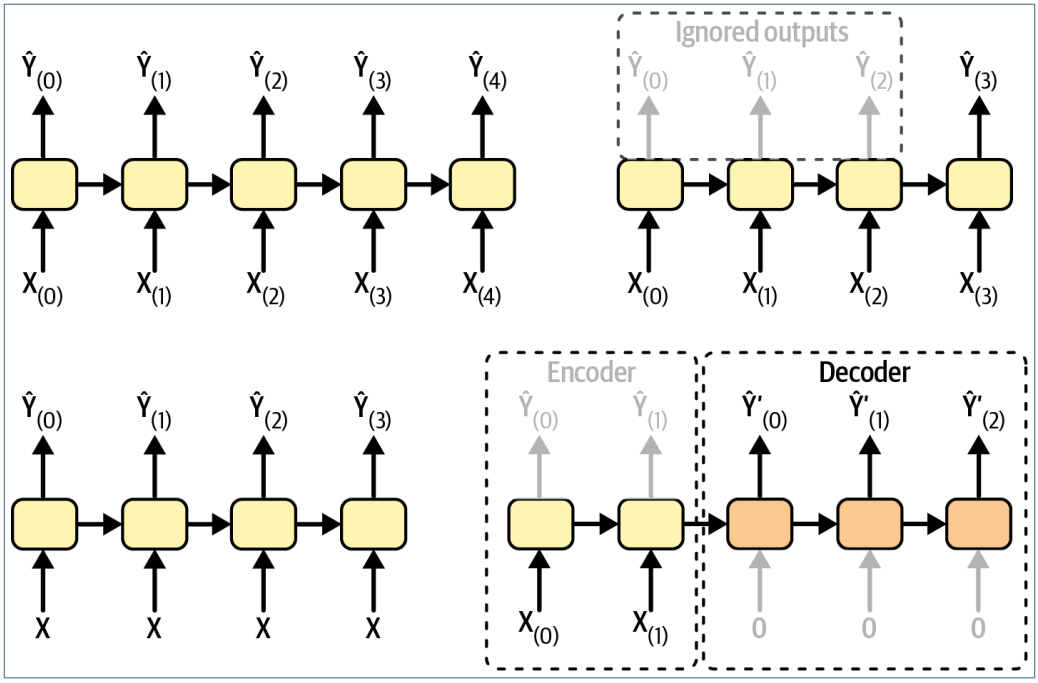

图 13-4: 序列到序列（左上）、序列到向量（右上）、向量到序列（左下）和编码器-解码器（右下）网络（Sequence-to-sequence (top left), sequence-to-vector (top right), vector-to-sequence (bottom left), and encoder-decoder (bottom right) networks）

最后，可以让一个序列到向量网络（称为编码器）后接一个向量到序列网络（称为解码器），如图 13-4 右下方所示。例如，这种结构可以用于把句子从一种语言翻译成另一种语言：向网络输入一种语言的句子，编码器把句子转换为一个单一的向量表示，解码器再把这个向量解码为另一种语言的句子。这种两步模型称为编码器-解码器（encoder-decoder）[2]，效果通常远好于尝试用单个序列到序列 RNN 即时翻译。句子的后半部分可能影响译文的开

[2]见本章参考文献第 4 条。

头，因此需要等看到完整句子后再进行翻译。第 14 章会实现编码器-解码器（正如你将看到的，它比图 13-4 所示的结构稍微复杂一些）。

这种灵活性令人期待，但如何训练循环神经网络呢？

13.4　训练 RNN（Training RNNs）

训练 RNN 的诀窍是先像前面那样沿时间展开网络，然后使用普通的反向传播（见图 13-5）。这种策略称为时间反向传播（backpropagation through time，BPTT）。

与普通反向传播一样，首先沿展开后的网络执行一次前向传播（用虚线箭头表示）。然后使用损失函数 L(Y(0), Y(1), ⋯, Y(T); Ŷ(0), Ŷ(1), ⋯, Ŷ(T)) 评估输出序列，其中 Y(i) 是第 i 个目标，Ŷ(i) 是第 i 个预测，T 是最大时间步。注意，损失函数可以忽略部分输出。例如，在序列到向量 RNN 中，除了最后一个输出外，其他输出都可以忽略。图 13-5 的损失函数只基于最后三个输出计算。

随后，损失函数的梯度沿展开后的网络向后传播（用实线箭头表示）。在这个例子中，由于输出 Ŷ(0) 和 Ŷ(1) 没有用于计算损失，梯度不会沿这两个输出向后流动，只会通过 Ŷ(2)、Ŷ(3) 和 Ŷ(4)。此外，由于每个时间步使用相同的参数 W 和 b，在反向传播期间，这些参数的梯度会被多次调整。反向阶段完成并计算出所有梯度后，BPTT 就可以执行一次梯度下降来更新参数；这与普通反向传播没有区别。

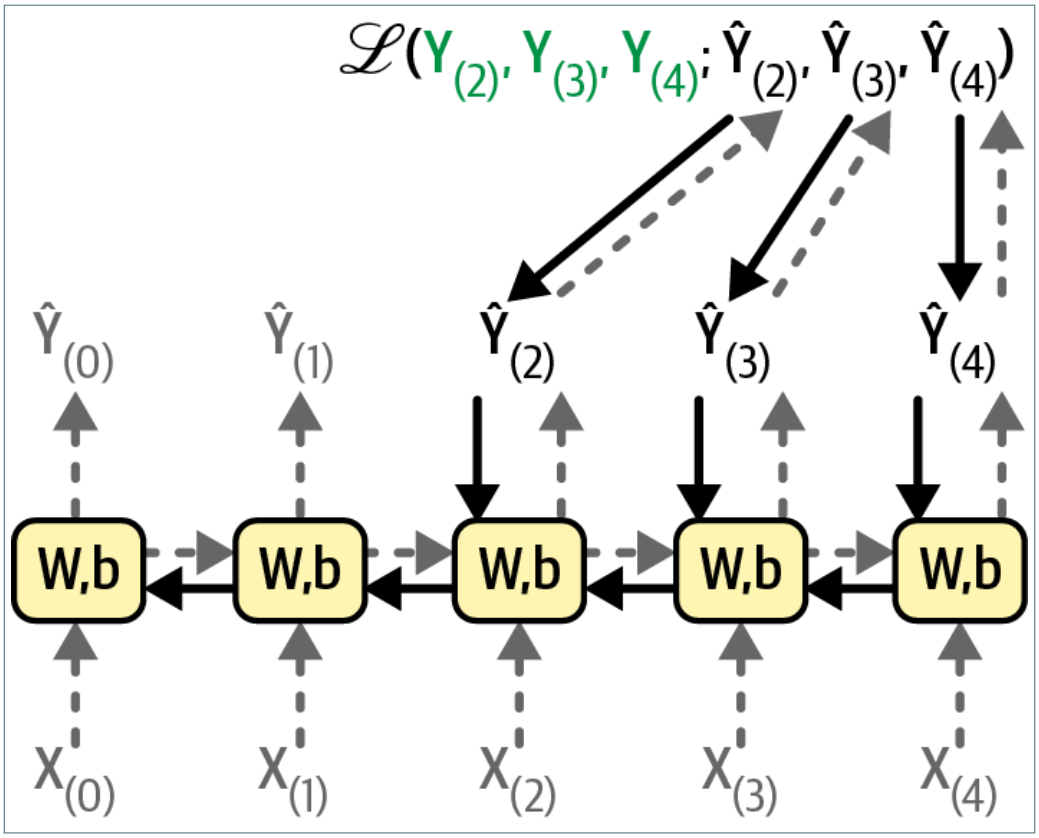

图 13-5: 时间反向传播（Backpropagation through time）

幸运的是，PyTorch 会为你处理所有这些复杂性。接下来会看到具体做法。不过在此之前，先加载一个时间序列，并使用经典工具对它进行分析，以便更好地理解问题并获得一些基线指标。

13.5　预测时间序列（Forecasting a Time Series）

假设你刚被芝加哥交通管理局（Chicago Transit Authority）聘为数据科学家。你的第一个任务是建立一个模型，预测第二天乘坐公交车和轨道交通的乘客数量。你可以使用自 2001 年起的每日客流数据。下面逐步看看应该如何处理。

先加载并清理数据：[3]

[3]芝加哥交通管理局的最新数据可在 Chicago Data Portal 获取。

```
import pandas as pd
from pathlib import Path
path = Path("datasets/ridership/CTA_-_Ridership_-_Daily_Boarding_Totals.csv")
df = pd.read_csv(path, parse_dates=["service_date"])
df.columns = ["date", "day_type", "bus", "rail", "total"]  # shorter names
df = df.sort_values("date").set_index("date")
df = df.drop("total", axis=1)  # no need for total, it's just bus + rail
df = df.drop_duplicates()  # remove duplicated months (2011-10 and 2014-07)
```

我们加载 CSV 文件，设置较短的列名，按日期排序行，删除冗余的 total 列，并删除重复的月份行。现在查看前几行：

```
>>> df.head()
           day_type     bus    rail
date
2001-01-01        U  297192  126455
2001-01-02        W  780827  501952
2001-01-03        W  824923  536432
2001-01-04        W  870021  550011
2001-01-05        W  890426  557917
```

2001 年 1 月 1 日，芝加哥有 297,192 人乘坐公交车，126,455 人乘坐火车。day_type 列包含 W（工作日）、A（星期六）和 U（星期日或节假日）。

现在绘制 2019 年几个月内公交和轨道交通的客流量，看看数据的形态（见图 13-6）：

```
import matplotlib.pyplot as plt
df["2019-03":"2019-05"].plot(grid=True, marker=".", figsize=(8, 3.5))
plt.show()
```

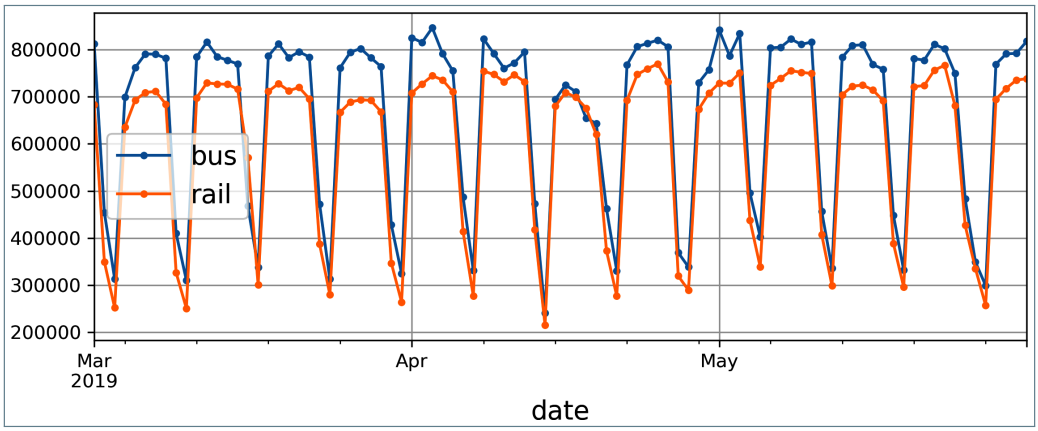

图 13-6: 芝加哥每日客流量（Daily ridership in Chicago）

注意，Pandas 会同时包含范围中的起始月份和结束月份，因此这里绘制的是从 3 月 1 日一直到 5 月 31 日的数据。这是一个时间序列：数据值对应不同的时间步，通常这些时间步之间的间隔相同。更具体地说，由于每个时间步有多个值，这称为多变量时间序列（multivariate time series）。如果只查看 bus 列，它就是一个每个时间步只有单个值的单变量时间序列（univariate time series）。预测未来值（即预测，forecasting）是处理时间序列最典型的任务，这也是本章的重点。其他任务还包括插补（填充缺失的历史值）、分类、异常检测等。

从图 13-6 可以看到，一个相似的模式每周都会清晰地重复。这称为每周季节性（weekly seasonality）。在本例中，这种季节性非常强，因此只需把一周前的值复制过来，就能相当不错地预测明天的客流量。这称为朴素预测（naive forecasting）：简单复制过去的某个值来生成预测。朴素预测通常是很好的基线，有时甚至很难超越。

> 注意：一般来说，朴素预测是指复制最近已知的值（例如预测明天与今天相同）。但在本例中，由于每周季节性很强，复制上一周的值效果更好。

为了直观展示这种朴素预测，我们叠加绘制公交和轨道交通的两个时间序列，以及向右移动一周后的相同时间序列（使用点线）。还会绘制两者之间的差值，即时间 t 的值减去时间 t - 7 的值；这称为差分（differencing）（见图 13-7）：

```
diff_7 = df[["bus", "rail"]].diff(7)["2019-03":"2019-05"]
fig, axs = plt.subplots(2, 1, sharex=True, figsize=(8, 5))
df.plot(ax=axs[0], legend=False, marker=".")  # original time series
df.shift(7).plot(ax=axs[0], grid=True, legend=False, linestyle=":")  # lagged
diff_7.plot(ax=axs[1], grid=True, marker=".")  # 7-day difference time series
plt.show()
```

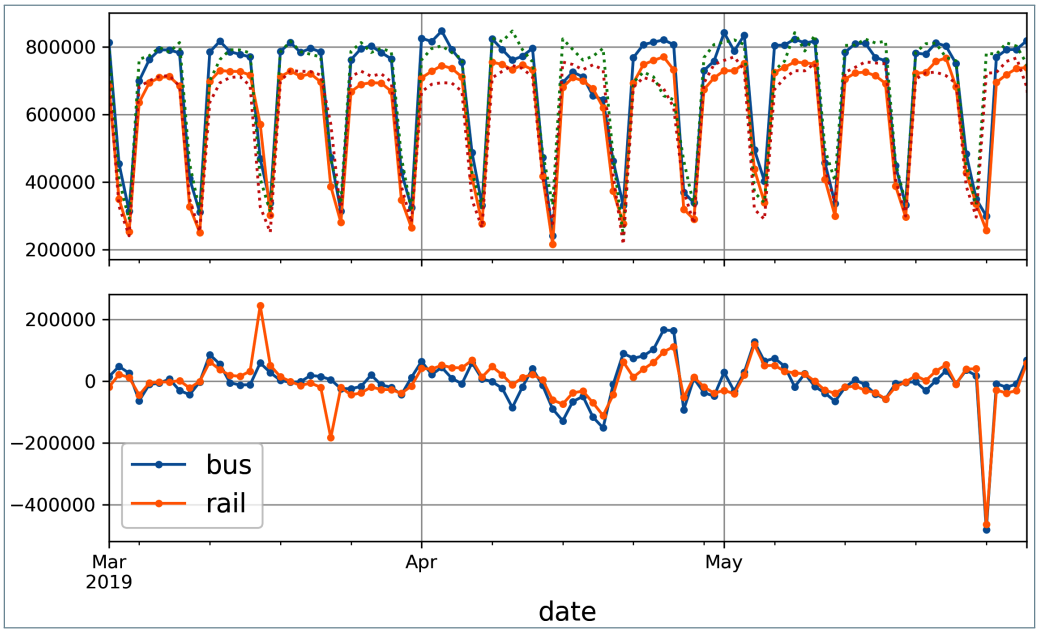

图 13-7: 叠加显示带 7 天滞后的时间序列（上）以及 t 与 t - 7 的差值（下）(Time series overlaid with 7-day lagged time series (top), and difference between t and t - 7 (bottom))

效果不错！注意，滞后时间序列与实际时间序列非常接近。当一个时间序列与自身的滞后版本相关时，我们称该时间序列具有自相关（autocorrelation）。可以看到，除了 5 月底之外，大多数差值都相当小。也许那段时间有节假日？检查 day_type 列：

```
>>> list(df.loc["2019-05-25":"2019-05-27"]["day_type"])
['A', 'U', 'U']
```

确实，那是一个长周末：星期一是阵亡将士纪念日（Memorial Day）假日。可以利用这一列改进预测，但现在先随意关注 2019 年 3 月、4 月和 5 月这三个月，计算平均绝对误差：

```
>>> diff_7.abs().mean()
bus     43915.608696
rail    42143.271739
dtype: float64
```

朴素预测的公交客流平均绝对误差（mean absolute error，MAE）约为 43,916 人，轨道交通客流约为 42,143 人。仅凭这些数字很难判断好坏，因此把预测误差除以目标值，放到相对尺度上：

```
>>> targets = df[["bus", "rail"]]["2019-03":"2019-05"]
>>> (diff_7 / targets).abs().mean()
bus     0.082938
rail    0.089948
dtype: float64
```

刚才计算的是平均绝对百分比误差（mean absolute percentage error，MAPE）。朴素预测对公交和轨道交通的 MAPE 分别约为 8.3% 和 9.0%。有趣的是，轨道交通预测的 MAE 看起来略好于公交预测，但 MAPE 则相反。这是因为公交客流量大于轨道交通，所以公交预测误差自然也更大；但将误差放到相对尺度后，公交预测实际上略好于轨道交通预测。

> 提示：MAE、MAPE 和均方误差（mean squared error，MSE）是评价预测结果最常用的指标。与往常一样，正确指标取决于任务。例如，如果项目受到大误差的影响是小误差的平方倍，那么 MSE 可能更合适，因为它会强烈惩罚大误差。

从时间序列看，似乎没有明显的月度季节性，但检查一下是否存在年度季节性。这里查看 2001 年到 2019 年的数据；为降低数据窥探风险，暂时忽略更新的数据。同时绘制每个序列的 12 个月滚动平均值，以便观察长期趋势（见图 13-8）：

```
period = slice("2001", "2019")
df_monthly = df.select_dtypes(include="number").resample('ME').mean()
rolling_average_12_months = df_monthly.loc[period].rolling(window=12).mean()
fig, ax = plt.subplots(figsize=(8, 4))
df_monthly[period].plot(ax=ax, marker=".")
rolling_average_12_months.plot(ax=ax, grid=True, legend=False)
plt.show()
```

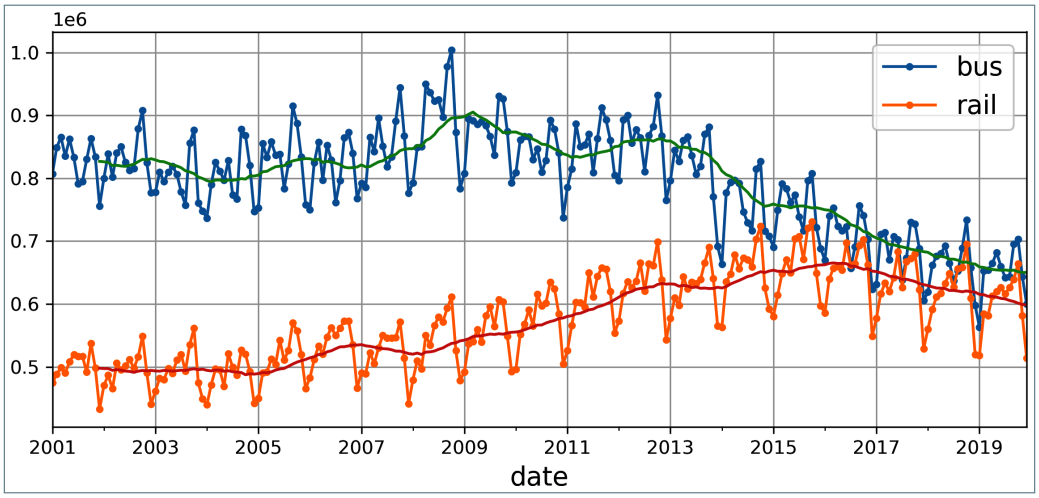

图 13-8: 年度季节性和长期趋势（Yearly seasonality and long-term trends）

确实存在年度季节性，尽管它比每周季节性更嘈杂，而且在轨道交通序列中比公交序列更明显：每年大致相同的日期会出现峰值和谷值。再绘制 12 个月差值（见图 13-9）：

```
df_monthly.diff(12)[period].plot(grid=True, marker=".", figsize=(8, 3))
plt.show()
```

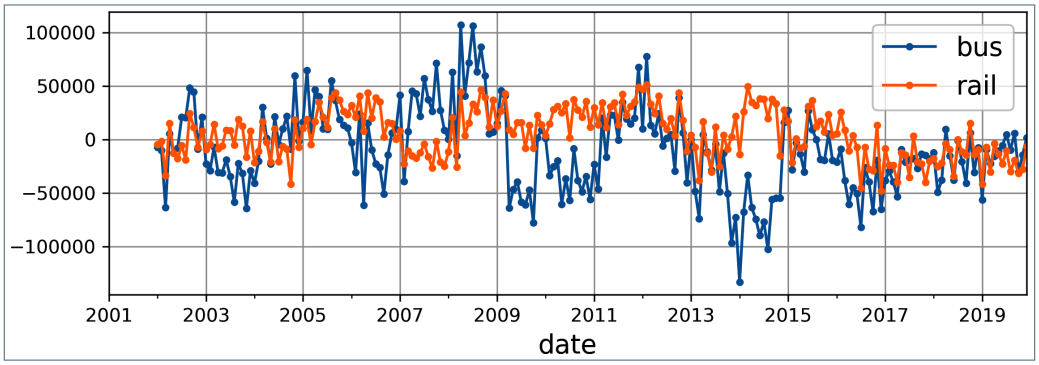

图 13-9: 12 个月差值（The 12-month difference）

注意，差分不仅消除了年度季节性，也消除了长期趋势。例如，2016 年到 2019 年时间序列中存在的线性下降趋势，在差分后的时间序列中变成了一个大致恒定的负值。实际上，差分是消除时间序列趋势和季节性的常见技术：研究平稳时间序列（stationary time series）更容易，也就是研究统计性质不随时间变化、没有季节性或趋势的时间序列。一旦能够准确预测差分后的时间序列，只要把先前减去的历史值加回来，就很容易得到实际时间序列的预测。

你可能会想，我们只是在预测明天的客流量，因此长期模式比短期模式重要性小得多。确实如此，但考虑长期模式仍可能略微改善性能。例如，2017 年 10 月每日公交客流量下降了约 2,500 人，相当于每周少约 570 名乘客。因此，如果处在 2017 年 10 月末，那么预测明天的客流量时，可以复制上一周的值再减去 570。考虑趋势后，平均而言预测会更准确一些。

现在已经熟悉了客流时间序列，以及时间序列分析中的一些重要概念，包括季节性、趋势、差分和移动平均。下面快速了解一类常用于分析时间序列的统计模型。

13.6　ARMA 模型系列（The ARMA Model Family）

先从自回归移动平均（autoregressive moving average，ARMA）模型开始。该模型由 Herman Wold 在 20 世纪 30 年代提出：它使用滞后值的加权和进行预测，再加上一个移动平均来修正预测，方式与前面讨论的内容非常相似。具体来说，移动平均部分使用最近几次预测误差的加权和计算。公式 13-3 展示了该模型如何生成预测。

$$\hat{y}^{(t)} = \sum_{i=1}^{p} \alpha_i y^{(t-i)} + \sum_{i=1}^{q} \theta_i \varepsilon^{(t-i)}, \qquad \varepsilon^{(t)} = y^{(t)} - \hat{y}^{(t)} \tag{13-3}$$

公式 13-3　使用 ARMA 模型进行预测（Forecasting using an ARMA model）

在这个公式中：

- ŷ(t) 是模型对时间步 t 的预测。
- y(t) 是时间序列在时间步 t 的值。
- 第一项求和是时间序列过去 p 个值的加权和，使用学习得到的权重 α_i。p 是超参数，决定模型应回看多远。这是模型的自回归部分：根据过去值执行回归。
- 第二项求和是过去 q 个预测误差 ε(t) 的加权和，使用学习得到的权重 θ_i。q 是超参数。这是模型的移动平均部分。

重要的是，该模型假设时间序列是平稳的。如果不是，差分可能会有所帮助。对相邻时间步执行一次差分，可以得到时间序列导数的近似值：实际上，它给出了序列在每个时间步的斜率，因此会消除线性趋势，把它变成一个常数。例如，对序列 [3, 5, 7, 9, 11] 执行一次差分，会得到 [2, 2, 2, 2]。

如果原始时间序列是二次趋势而不是线性趋势，那么一次差分还不够。例如，序列 [1, 4, 9, 16, 25, 36] 一次差分后变成 [3, 5, 7, 9, 11]；再差分一次，就得到 [2, 2, 2, 2]。因此，执行两次连续差分可以消除二次趋势。更一般地，连续执行 d 次差分，相当于计算时间序列 d 阶导数的近似，因此能够消除最高 d 次的多项式趋势。这个超参数 d 称为积分阶数（order of integration）。

差分是自回归积分移动平均（autoregressive integrated moving average，ARIMA）模型的核心贡献。ARIMA 模型由 George Box 和 Gwilym Jenkins 于 1970 年在《Time Series Analysis》（Wiley）一书中提出。该模型先执行 d 次差分，使时间序列更加平稳，然后应用普通 ARMA 模型。预测时先使用 ARMA 模型，再把差分过程中减去的项加回来。

13.7　SARIMA 模型

ARMA 系列的最后一个成员是季节性 ARIMA（seasonal ARIMA，SARIMA）模型：它以与 ARIMA 相同的方式建模时间序列，同时针对给定频率（例如每周）额外建模一个季节性成分。它共有七个超参数：ARIMA 的 p、d 和 q，用于描述季节模式的 P、D 和 Q，以及季节模式的周期 s。P、D 和 Q 与 p、d 和 q 类似，但它们用于建模 t - s、t - 2s、t - 3s 等位置的时间序列。

下面把 SARIMA 模型拟合到轨道交通时间序列，并用它预测明天的客流量。假设今天是 2019 年 5 月的最后一天，想要预测“明天”即 2019 年 6 月 1 日的轨道交通客流量。statsmodels 库包含许多统计模型，其中包括通过 ARIMA 类实现的 ARMA 模型及其变体：

```
from statsmodels.tsa.arima.model import ARIMA
origin, today = "2019-01-01", "2019-05-31"
rail_series = df.loc[origin:today]["rail"].asfreq("D")
model = ARIMA(rail_series,
              order=(1, 0, 0),
              seasonal_order=(0, 1, 1, 7))
model = model.fit()
y_pred = model.forecast()  # returns 427,758.6
```

这段代码的工作方式如下：

- 首先导入 ARIMA 类，然后取出 2019 年初至“今天”的轨道交通客流数据，并使用 asfreq("D") 将时间序列频率设为每日。本例中的数据本来就是每日数据，因此这不会改变数据；但如果不设置频率，ARIMA 类就必须猜测频率，并会显示警告。
- 接着创建一个 ARIMA 实例，把截至“今天”的所有数据传入，并设置模型超参数：order=(1, 0, 0) 表示 p = 1、d = 0、q = 0；seasonal_order=(0, 1, 1, 7) 表示 P = 0、D = 1、Q = 1、s = 7。注意，statsmodels 的 API 与 Scikit-Learn 略有不同：数据在构造模型时传入，而不是传给 fit() 方法。
- 然后拟合模型，并预测“明天”即 2019 年 6 月 1 日的客流量。

预测结果为 427,759 名乘客，而实际人数为 379,044。误差达 12.9%，相当糟糕。它甚至略差于朴素预测的 426,932 人（误差 12.6%）。但也许只是那一天运气不好？可以使用循环，对 3 月、4 月和 5 月的每一天执行同样的预测，并计算整个期间的 MAE：

```
origin, start_date, end_date = "2019-01-01", "2019-03-01", "2019-05-31"
time_period = pd.date_range(start_date, end_date)
rail_series = df.loc[origin:end_date]["rail"].asfreq("D")
y_preds = []
for today in time_period.shift(-1):
    model = ARIMA(rail_series[origin:today],  # train on data up to "today"
                  order=(1, 0, 0),
                  seasonal_order=(0, 1, 1, 7))
    model = model.fit()  # note that we retrain the model every day!
    y_pred = model.forecast().iloc[0]
    y_preds.append(y_pred)
y_preds = pd.Series(y_preds, index=time_period)
mae = (y_preds - rail_series[time_period]).abs().mean()  # returns 32,040.7
```

这次好多了！MAE 约为 32,041，显著低于朴素预测的 42,143。因此，虽然模型并不完美，但平均而言仍然大幅超越了朴素预测。

此时，你可能会想知道如何为 SARIMA 模型选择好的超参数。有多种方法，最容易理解、也最适合入门的方法是暴力法：直接执行网格搜索。对于每个待评估的模型（即每种超参数组合），运行前面的代码示例，只改变超参数值。好的 p、q、P 和 Q 值通常较小（一般为 0 到 2，有时会到 5 或 6），d 和 D 通常为 0 或 1，有时为 2。至于 s，它就是主要季节模式的周期；在本例中，由于每周季节性很强，所以 s = 7。MAE 最低的模型胜出。当然，如果业务目标更适合其他指标，也可以替换 MAE。

> 提示：还有更有原则的方法选择超参数，例如分析自相关函数（autocorrelation function，ACF）和偏自相关函数（partial autocorrelation function，PACF）[a]，或者最小化 AIC、BIC 指标（第 8 章介绍过），以惩罚使用过多参数的模型并降低过拟合风险。但网格搜索是很好的起点。
>
> [a]更多 ACF-PACF 方法的细节，可参考 Jason Brownlee 撰写的相关介绍。

13.8　为机器学习模型准备数据（Preparing the Data for Machine Learning Models）

现在已经有两个基线：朴素预测和 SARIMA。接下来使用前面学过的机器学习模型预测这个时间序列，先从一个基础线性模型开始。我们的目标是根据过去 8 周（56 天）的客流量预测明天的客流量。因此，模型输入是包含 56 个时间步 t - 55 到 t 的序列；模型输出一个值，即时间步 t + 1 的预测。

训练数据该如何构造？诀窍是：把过去所有长度为 56 天的窗口都作为训练数据，并把每个窗口紧接着的下一个值作为目标。为此，创建一个自定义数据集，把指定时间序列切分为所有可能的窗口，并为每个窗口提供相应目标：

```
class TimeSeriesDataset(torch.utils.data.Dataset):
    def __init__(self, series, window_length):
        self.series = series
        self.window_length = window_length
    def __len__(self):
        return len(self.series) - self.window_length
    def __getitem__(self, idx):
        if idx >= len(self):
            raise IndexError("dataset index out of range")
        end = idx + self.window_length  # 1st index after window
        window = self.series[idx : end]
        target = self.series[end]
        return window, target
```

用只包含数字 0 到 5 的简单时间序列测试这个类。RNN 模块要求每个序列是二维的，形状为 [sequence length, dimensionality]。对于单变量时间序列，维度就是 1，因此把序列表示为 [[0], [1], [2], [3], [4], [5]]：

```
>>> my_series = torch.tensor([[0], [1], [2], [3], [4], [5]])
>>> my_dataset = TimeSeriesDataset(my_series, window_length=3)
>>> for window, target in my_dataset:
...     print("Window:", window, " Target:", target)
...
Window: tensor([[0], [1], [2]])  Target: tensor([3])
Window: tensor([[1], [2], [3]])  Target: tensor([4])
Window: tensor([[2], [3], [4]])  Target: tensor([5])
```

TimeSeriesDataset 包含所有长度为 3 的窗口，以及窗口之后的第一个值作为目标。接着为这个小数据集创建 DataLoader，将窗口打乱并分成大小为 2 的批次：

```
>>> from torch.utils.data import DataLoader
>>> torch.manual_seed(0)
>>> my_loader = DataLoader(my_dataset, batch_size=2, shuffle=True)
>>> for X, y in my_loader:
...     print("X:", X, " y:", y)
...
X: tensor([[[0], [1], [2]], [[2], [3], [4]]])  y: tensor([[3], [5]])
X: tensor([[[1], [2], [3]]])  y: tensor([[4]])
```

第一批包含窗口 [[0], [1], [2]] 和 [[2], [3], [4]]，以及相应目标 [3] 和 [5]；第二批只包含一个窗口 [[1], [2], [3]]，目标为 [4]。确实，当数据集长度不是批大小的整数倍时，最后一个批次会更短。

现在已经能够把时间序列转换为可用于训练机器学习模型的数据集，接着准备客流数据。首先把数据划分为训练期、验证期和测试期。当前先关注轨道交通客流。把数据转换为 32 位浮点张量，并缩小一百万倍，使数值落在接近 0-1 的范围内；这与默认的权重初始化和学习率配合良好。这里使用 df[["rail"]] 而不是 df["rail"]，是为了让得到的张量形状为 [series length, 1] 而不是 [series length]：

```
rail_train = torch.FloatTensor(df[["rail"]]["2016-01":"2018-12"].values / 1e6)
rail_valid = torch.FloatTensor(df[["rail"]]["2019-01":"2019-05"].values / 1e6)
rail_test = torch.FloatTensor(df[["rail"]]["2019-06":].values / 1e6)
```

> 注意：处理时间序列时，通常希望沿时间进行划分。不过在某些情况下，也可以沿其他维度划分，这样能够获得更长的训练时间段。例如，如果有 2001 年到 2019 年 10,000 家公司的财务健康数据，就可能按不同公司划分。只是这些公司很可能高度相关（例如整个经济部门可能共同上涨或下跌）；如果训练集和测试集中包含相关公司，测试集的泛化误差就会受到乐观偏差影响，因而不再那么有用。

接下来使用 TimeSeriesDataset 类创建训练、验证和测试数据集，并创建对应的数据加载器。由于梯度下降假设训练集中的实例独立同分布（IID），所以必须将 shuffle 设为 True：这会打乱窗口，但不会打乱窗口中的内容。

```
window_length = 56
train_set = TimeSeriesDataset(rail_train, window_length)
train_loader = DataLoader(train_set, batch_size=32, shuffle=True)
valid_set = TimeSeriesDataset(rail_valid, window_length)
valid_loader = DataLoader(valid_set, batch_size=32)
test_set = TimeSeriesDataset(rail_test, window_length)
test_loader = DataLoader(test_set, batch_size=32)
```

现在已经可以构建并训练任意回归模型了。

13.9　使用线性模型进行预测（Forecasting Using a Linear Model）

先尝试一个基础线性模型。这里使用 Huber 损失；如第 9 章所述，它通常比直接最小化 MAE 更好：

```
import torch.nn as nn
import torchmetrics
torch.manual_seed(42)
model = nn.Sequential(nn.Flatten(), nn.Linear(window_length, 1)).to(device)
loss_fn = nn.HuberLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.003, momentum=0.9)
metric = torchmetrics.MeanAbsoluteError().to(device)
[...]  # train the PyTorch model (e.g., using train() function from Chapter 10)
```

注意，在 nn.Linear 层之前必须使用 nn.Flatten，因为输入形状为 [batch size, window length, dimensionality]，而 nn.Linear 期望输入形状为 [batch size, features]。训练这个模型后，验证集 MAE 约为 37,726（具体数值可能有所不同）。它优于朴素预测，但不如 SARIMA 模型。

使用 RNN 能做得更好吗？下面来看看。

13.10　使用简单 RNN 进行预测（Forecasting Using a Simple RNN）

实现一个简单 RNN：包含一个循环层（见图 13-2），再接一个最终的 nn.Linear 层。这个线性层接收最后一个隐藏状态，并输出模型预测：

```
class SimpleRnnModel(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__()
        self.hidden_size = hidden_size
        self.memory_cell = nn.Sequential(
            nn.Linear(input_size + hidden_size, hidden_size),
            nn.Tanh()
        )
        self.output = nn.Linear(hidden_size, output_size)
    def forward(self, X):
        batch_size, window_length, dimensionality = X.shape
        X_time_first = X.transpose(0, 1)
        H = torch.zeros(batch_size, self.hidden_size, device=X.device)
        for X_t in X_time_first:
            XH = torch.cat((X_t, H), dim=1)
            H = self.memory_cell(XH)
        return self.output(H)
torch.manual_seed(42)
univar_model = SimpleRnnModel(input_size=1, hidden_size=32, output_size=1)
```

逐步解释这段代码：

- 构造函数接收输入大小、隐藏大小和输出大小三个参数。本例创建模型时将输入大小设为 1，因为处理的是单变量时间序列。隐藏大小是循环神经元数量，这里设为 32，但它是可以调节的超参数。输出大小为 1，因为只预测一个值。
- 构造函数首先创建记忆单元，它将在每个时间步使用：这是由 nn.Linear 层和 tanh 激活函数组成的顺序模块。也可以使用其他激活函数，但 RNN 通常使用 tanh，因为它往往比其他激活函数更稳定。
- 然后创建一个 nn.Linear 输出层，从最后一个隐藏状态生成最终输出。之所以需要它，是因为隐藏状态的维度等于循环神经元数量（本例为 32），而目标只有一个维度；此外，tanh 的输出范围为 -1 到 +1，而我们要预测的数值有时会超过 +1。
- forward() 方法接收数据加载器产生的输入批次，因此每个批次的形状为 [batch size, window length, dimensionality]，其中 dimensionality = 1。
- 隐藏状态 H 初始化为零。每个输入窗口中，每个循环神经元对应一个零值，因此隐藏状态形状为 [batch size, hidden_size]。
- 接下来遍历每个时间步。这里使用 transpose(0, 1) 交换 X 的前两个维度，因此每个时间步的输入张量 X_t 形状为 [batch size, dimensionality]。
- 每个时间步都要把当前输入 X_t 和隐藏状态 H 同时送入记忆单元。先沿第一个维度拼接它们，得到形状为 [batch size, input_size + hidden_size] 的张量 XH，然后把 XH 传给记忆单元，得到新的隐藏状态。
- 循环结束时，H 就是最终隐藏状态。把它传给输出层，得到形状为 [batch size, output_size] 的最终预测。

简而言之，这个模型先把隐藏状态初始化为零，然后逐个时间步处理当前输入和上一个隐藏状态，得到新的隐藏状态。到达最后一个时间步后，再把最终隐藏状态传过线性层，得到实际预测。整个过程会同时作用于批次中的每个序列。

这是第一个循环模型！它是一个序列到向量模型。在本例中，由于只有一个输出神经元，每个输入序列的输出向量大小为 1。

将模型移到 GPU 上，然后像训练前一个模型一样训练和评估，会得到约 30,659 的验证集 MAE。这是目前训练出的最佳模型，甚至超过了 SARIMA 模型。

> 提示：这里仅对时间序列做了归一化，没有消除趋势和季节性，但模型仍然表现良好。这使我们能够快速搜索有希望的模型，而不用太担心预处理。不过，要取得最佳性能，可以尝试让时间序列更加平稳，例如执行差分。

PyTorch 提供了 nn.RNN 模块，可以大幅简化 SimpleRnnModel 的实现。下面的实现与之前的实现基本等价：

```
class SimpleRnnModel(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__()
        self.rnn = nn.RNN(input_size, hidden_size, batch_first=True)
        self.output = nn.Linear(hidden_size, output_size)
    def forward(self, X):
        outputs, last_state = self.rnn(X)
        return self.output(outputs[:, -1])
```

这段代码短得多。构造函数创建 nn.RNN 模块，而不是手工构建记忆单元。它指定输入大小和隐藏大小，并设置 batch_first=True，因为输入批次的批维在第一位。如果不设置，nn.RNN 会假设时间维在第一位，要求输入形状为 [window length, batch size, dimensionality]，而不是 [batch size, window length, dimensionality]。

构造函数还创建了输出层，与前一实现完全相同。在 forward() 方法中，把输入批次直接传给 nn.RNN。该模块会处理所有细节：内部用零初始化隐藏状态，并使用基于线性层和激活函数（默认是 tanh）的简单记忆单元逐个处理时间步。

注意，nn.RNN 模块返回两个值：

- outputs 是一个张量，包含每个时间步顶层循环层的输出。本例只有一个循环层；下一节将看到，nn.RNN 支持多个循环层。对于简单 RNN，输出就是顶层循环层每个时间步的隐藏状态。outputs 的形状为 [batch size, window length, hidden size]（如果没有设置 batch_first=True，前两个维度会交换）。
- last_state 包含每个循环层在最后一个时间步之后的隐藏状态，形状为 [number of layers, batch size, hidden size]。本例只有一个循环层，因此第一个维度大小为 1。

最后，取最后一个输出（它也是顶层循环层的最后一个状态），并把它传给 nn.Linear 输出层。

训练这个模型会得到与之前相近的结果，但通常快得多，因为 nn.RNN 经过了良好优化。特别是在使用 NVIDIA GPU 时，nn.RNN 会利用 cuDNN 库；该库提供了各种神经网络架构（包括多种 RNN 架构）的高度优化实现。

> 注意：nn.RNN 模块使用两个偏置参数，一个用于输入，另一个用于隐藏状态。它只是把两者相加，因此实际上并没有改善模型，但 cuDNN 库要求这个额外参数。这也解释了为什么结果不会与手工实现完全相同。

13.11　使用深层 RNN 进行预测（Forecasting Using a Deep RNN）

如图 13-10 所示，把多个单元层堆叠起来是很常见的做法，这样就得到了深层 RNN。

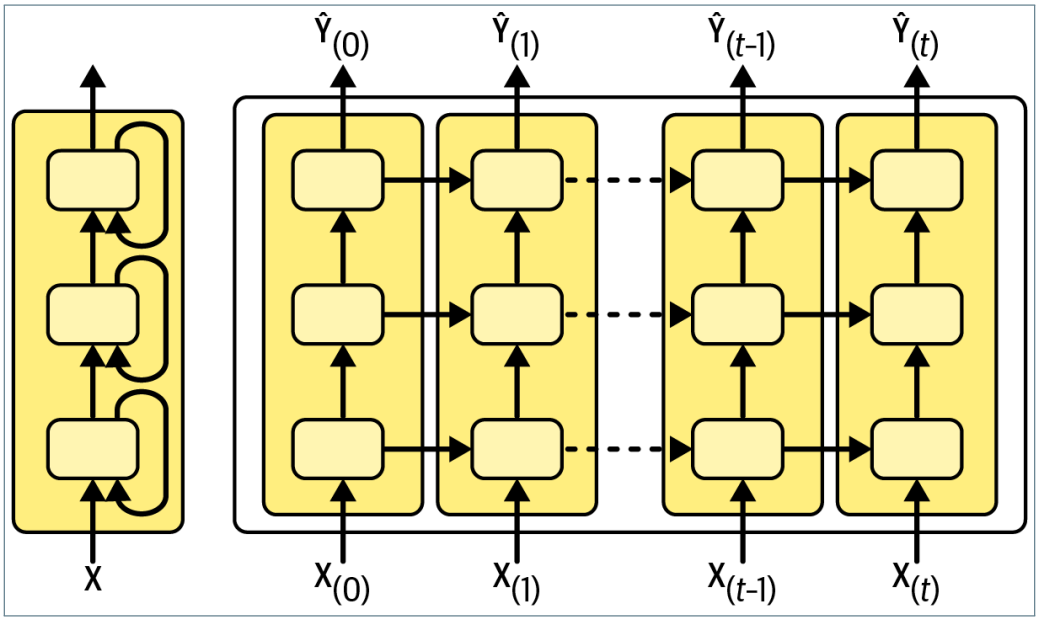

图 13-10: 深层 RNN（左）沿时间展开（右）(A deep RNN (left) unrolled through time (right))

使用 PyTorch 实现深层 RNN 很直接：创建 nn.RNN 模块时，把 num_layers 参数设置为所需的循环层数。例如，在前一个模型的构造函数中设置 num_layers=3，就可以得到一个三层 RNN（其他代码不变）：

```
self.rnn = nn.RNN(input_size, hidden_size, num_layers=3, batch_first=True)
```

训练并评估这个模型，会得到约 29,273 的 MAE。这是目前最好的模型。

13.12　预测多变量时间序列（Forecasting Multivariate Time Series）

神经网络的重要优势之一是灵活性，尤其是它们几乎不需要改变架构就能处理多变量时间序列。例如，可以使用公交和轨道交通数据共同预测轨道交通时间序列。甚至可以加入日期类型！由于总能提前知道明天是工作日、周末还是节假日，因此可以把日期类型序列向未来移动一天，把明天的日期类型作为输入。为简单起见，使用 Pandas 完成这些处理：

```
df_mulvar = df[["rail", "bus"]] / 1e6  # use both rail & bus series as input
df_mulvar["next_day_type"] = df["day_type"].shift(-1)  # we know tomorrow's type
df_mulvar = pd.get_dummies(df_mulvar, dtype=float)  # one-hot encode day type
```

现在 df_mulvar 有五列：公交和轨道交通数据，加上下一天日期类型的三个独热编码列（回想一下，日期类型有 W、A 和 U 三种）。接着进行与前面类似的处理：先分成三个时间段，缩小一百万倍，再转换为张量：

```
mulvar_train = torch.FloatTensor(df_mulvar["2016-01":"2018-12"].values / 1e6)
mulvar_valid = torch.FloatTensor(df_mulvar["2019-01":"2019-05"].values / 1e6)
mulvar_test = torch.FloatTensor(df_mulvar["2019-06":].values / 1e6)
```

然后需要创建 PyTorch 数据集。如果直接使用 TimeSeriesDataset，目标会包含下一天的公交和轨道交通客流量，以及下一天的日期类型独热编码。现在只想预测轨道交通客流，因此必须调整 TimeSeriesDataset，只保留目标中的第一个值。可以创建一个继承 TimeSeriesDataset 的新类，并修改 __getitem__() 方法来过滤目标：

```
class MulvarTimeSeriesDataset(TimeSeriesDataset):
    def __getitem__(self, idx):
        window, target = super().__getitem__(idx)
        return window, target[:1]
```

接着像之前一样创建数据集和数据加载器：

```
mulvar_train_set = MulvarTimeSeriesDataset(mulvar_train, window_length)
mulvar_train_loader = DataLoader(mulvar_train_set, batch_size=32, shuffle=True)
[...]  # create the datasets and data loaders for the validation and test sets
```

查看数据加载器产生的批次，会发现输入形状为 [32, 56, 5]，目标形状为 [32, 1]。非常理想。

现在终于可以创建 RNN：

```
torch.manual_seed(42)
mulvar_model = SimpleRnnModel(input_size=5, hidden_size=32, output_size=1)
mulvar_model = mulvar_model.to(device)
```

注意，这个模型与之前构建的单变量 RNN univar_model 完全相同，唯一差别是 input_size=5：模型在每个时间步接收五个输入，而不是一个。这个模型的验证集 MAE 实际达到 23,227，进步很大。

事实上，让 RNN 同时预测公交和轨道交通客流并不难：只需在 MulvarTimeSeriesDataset 中返回 target[:2] 而不是 target[:1]，并在创建 SimpleRnnModel 时设置 output_size=2。

正如第 9 章讨论的那样，用一个模型处理多个相关任务，通常比为每个任务分别训练模型效果更好：一个任务学到的特征可能对另一个任务有用，而且要求模型在多个任务上都表现良好，也能防止过拟合，这是一种正则化形式。不过具体情况取决于任务。在本例中，同时预测公交和轨道交通的多任务 RNN 不如使用全部五列输入、分别预测其中一项的专用模型。但它仍然取得了相当不错的结果：轨道交通验证集 MAE 为 26,441，公交验证集 MAE 为 26,178。

13.13　预测未来多个时间步（Forecasting Several Time Steps Ahead）

到目前为止，我们只预测下一个时间步的值，但只需适当修改目标，也可以预测几步之后的值。例如，要预测两周后的客流量，可以把目标改成 14 天后的值，而不是 1 天后的值。不过，如果希望一个模型一次预测接下来的 14 个值呢？

第一种方法是使用前面训练的轨道交通单变量 RNN univar_model，让它预测下一个值，然后把这个预测值追加到输入中，仿佛这个预测值真的已经发生。再用模型预测下一个值，依此类推，代码如下：

```
n_steps = 14
univar_model.eval()
with torch.no_grad():
    X = rail_valid[:window_length].unsqueeze(dim=0).to(device)
    for step_ahead in range(n_steps):
        y_pred_one = univar_model(X)
        X = torch.cat([X, y_pred_one.unsqueeze(dim=0)], dim=1)
    Y_pred = X[0, -n_steps:, 0]
```

这里取验证期前 56 天的轨道交通客流量，并使用 unsqueeze() 添加大小为 1 的批维（因为 univar_model 期望三维输入），然后把张量移到 GPU。此时 X 的形状为 [1, 56, 1]。接着重复使用模型预测下一个值，并沿时间轴（dim=1）把每个预测追加到输入序列。由于每个预测的形状为 [1, 1]，所以还要再次使用 unsqueeze() 添加大小为 1 的批维，才能把它连接到

X。最终 X 的形状为 [1, 56 + 14, 1]，最后 14 个值就是最终预测。图 13-11 展示了得到的预测结果。

> 警告：如果模型在某个时间步产生错误，后续时间步的预测也会受到影响，错误往往会累积。因此，这种技术更适合预测较少的时间步。

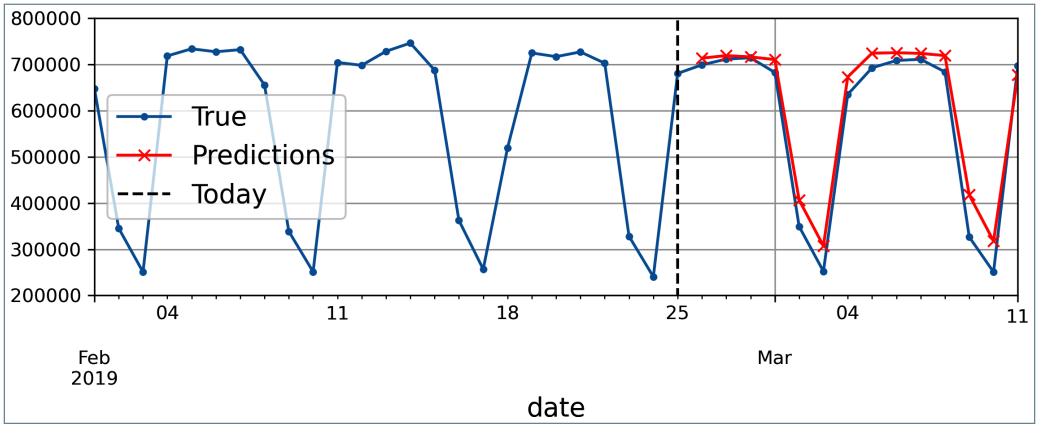

图 13-11: 逐步预测未来 14 个时间步（Forecasting 14 steps ahead, 1 step at a time）

第二种方法是训练 RNN 一次预测接下来的 14 个值。仍然可以使用序列到向量模型，但让它输出 14 个值而不是 1 个。首先要把目标改为包含未来 14 个值的向量。创建如下的类：

```
class ForecastAheadDataset(TimeSeriesDataset):
    def __len__(self):
        return len(self.series) - self.window_length - 14 + 1
    def __getitem__(self, idx):
        end = idx + self.window_length  # 1st index after window
        window = self.series[idx : end]
        target = self.series[end : end + 14, 0]  # 0 = rail ridership
        return window, target
```

> 提示：上面把数字 14 写死了，但实际项目中应该使其可配置（例如像 window_length 一样）。

这个类继承自 TimeSeriesDataset，并调整了 __len__() 与 __getitem__() 方法。现在目标是包含未来 14 个轨道交通客流量的张量，而不是下一个值。

再次基于前面构建的多变量时间序列创建训练、验证和测试数据集：

```
ahead_train_set = ForecastAheadDataset(mulvar_train, window_length)
ahead_train_loader = DataLoader(ahead_train_set, batch_size=32, shuffle=True)
[...]  # create the datasets and data loaders for the validation and test sets
```

最后创建一个与 mulvar_model 类似的简单 RNN，但把 output_size 设置为 14：

```
torch.manual_seed(42)
ahead_model = SimpleRnnModel(input_size=5, hidden_size=32, output_size=14)
ahead_model = ahead_model.to(device)
```

训练这个模型后，可以一次预测未来 14 个值：

```
ahead_model.eval()
with torch.no_grad():
    window = mulvar_valid[:window_length]  # shape [56, 5]
    X = window.unsqueeze(dim=0)            # shape [1, 56, 5]
    Y_pred = ahead_model(X.to(device))     # shape [1, 14]
```

这种方法效果很好。它对下一天的预测显然比对 14 天后的预测更准确，但不会像前一种方法那样累积误差。接下来看看序列到序列模型能否做得更好。

> 提示：可以组合这两种方法来预测很多个时间步：使用一个模型一次预测未来 14 天，把预测值追加到输入中，再次运行模型预测接下来的 14 天，依此类推。[a]
>
> [a]不能使用 ahead_model 完成这一点，因为它需要公交和轨道交通共同作为输入，但它只预测轨道交通客流。

13.14　使用序列到序列模型进行预测（Forecasting Using a Sequence-to-Sequence Model）

不再只训练模型在最后一个时间步预测未来 14 个值，而是训练它在每个时间步都预测未来 14 个值。具体来说，在时间步 0，模型输出时间步 1 到 14 的预测；在时间步 1，模型预测时间步 2 到 15，依此类推。换句话说，目标变成连续窗口的序列，每个窗口相对于前一个时间步移动一步。每个输入窗口的目标不再是向量，而是与输入等长的序列，每个时间步包含一个 14 维向量。给定形状为 [batch size, window length, input size] 的输入批次，输出形状为 [batch size, window length, output_size]。这不再是序列到向量 RNN，而是序列到序列（sequence-to-sequence，seq2seq）RNN。

> 注意：目标包含了输入中已经出现的值（最后一个时间步除外），这看起来可能像是在作弊。幸运的是，完全不是：RNN 在每个时间步只能知道过去的时间步，无法看到未来。因此它是一个因果模型（causal model）。

训练 seq2seq 模型时，为什么要预测未来值，而最终只关心最后一个时间步的输出？训练完成后确实可以忽略除最后一个时间步以外的所有输出。这样做的主要优势是：损失函数会包含每个时间步输出对应的项，而不只是最后一个输出。因此会有更多误差梯度流过模型，而且梯度不需要在时间上反向传播那么远，因为它们来自每个时间步的输出，而不是只来自最后一个输出。这既能稳定训练，也能加快收敛。此外，由于模型必须在每个时间步作出预测，它会看到长度不同的输入序列，从而降低模型过拟合特定训练窗口长度的风险——至少希望如此。下面把这种技术用于轨道交通客流时间序列。

首先准备数据集：

```
class Seq2SeqDataset(ForecastAheadDataset):
    def __getitem__(self, idx):
        end = idx + self.window_length  # 1st index after window
        window = self.series[idx : end]
        target_period = self.series[idx + 1 : end + 14, 0]
        target = target_period.unfold(dimension=0, size=14, step=1)
        return window, target
```

新的 Seq2SeqDataset 类继承自 ForecastAheadDataset，并重写 __getitem__() 方法：输入窗口与之前相同，但目标变成连续窗口的序列，每个窗口相对于前一个时间步移动一步。unfold() 方法是关键：它接收一个张量，并从中生成滑动块。它沿指定维度按给定步长反复移动，提取指定大小的块。例如：

```
>>> torch.tensor([0, 1, 2, 3, 4, 5]).unfold(dimension=0, size=4, step=1)
tensor([[0, 1, 2, 3],
        [1, 2, 3, 4],
        [2, 3, 4, 5]])
```

再次创建训练集、验证集、测试集及其数据加载器：

```
seq_train_set = Seq2SeqDataset(mulvar_train, window_length)
seq_train_loader = DataLoader(seq_train_set, batch_size=32, shuffle=True)
[...]  # create the datasets and data loaders for the validation and test sets
```

最后构建序列到序列模型：

```
class Seq2SeqRnnModel(SimpleRnnModel):
    def forward(self, X):
        outputs, last_state = self.rnn(X)
        return self.output(outputs)
torch.manual_seed(42)
seq_model = Seq2SeqRnnModel(input_size=5, hidden_size=32, output_size=14)
seq_model = seq_model.to(device)
```

这里继承 SimpleRnnModel，并重写 forward() 方法。前面只把 nn.Linear 输出层应用于最后一个时间步的输出，现在把它应用于每个时间步的输出。可能会惊讶于这段代码为何有效：此前只把 nn.Linear 层应用于形状为 [batch size, features] 的二维输入，而这里的 outputs 形状为 [batch size, window length, hidden size]，是三维的。幸运的是，这完全可以工作，因为 nn.Linear 会自动应用于每个时间步，因此模型预测形状为 [batch size, window length, output size]，恰好满足需要。

底层的 nn.Linear 层依赖 torch.matmul() 进行矩阵乘法。这个函数高效支持二维以上的数组相乘。例如，形状为 [2, 3, 5, 7] 的数组可以与形状为 [2, 3, 7, 11] 的数组相乘。它们都可以看作一个 2 × 3 的矩阵网格，torch.matmul() 逐个相乘对应矩阵。由于 5 × 7 矩阵与 7 × 11 矩阵相乘会得到 5 × 11 矩阵，最终结果是形状为 [2, 3, 5, 11] 的张量。该函数也支持广播：例如，形状为 [10, 56, 32] 的数组可以与形状为 [32, 14] 的数组相乘，前一个数组中的 10 个 56 × 32 矩阵都会与同一个 32 × 14 矩阵相乘，得到形状为 [10, 56, 14] 的张量。把三维输入传给 nn.Linear 时，就会发生这种情况。

> 提示：另一种得到完全相同结果的方法，是把 nn.Linear 输出层替换为核大小为 1 的 nn.Conv1d 层，即 Conv1d(32, 14, kernel_size=1)。但需要交换输入和输出的最后两个维度，把时间维当作空间维。

训练代码照常。训练期间使用模型的所有输出；训练完成后，只关心最后一个时间步的输出，其余可以忽略。例如，可以这样预测未来 14 天的轨道交通客流：

```
seq_model.eval()
with torch.no_grad():
    some_window = mulvar_valid[:window_length]  # shape [56, 5]
    X = some_window.unsqueeze(dim=0)            # shape [1, 56, 5]
    Y_preds = seq_model(X.to(device))           # shape [1, 56, 14]
    Y_pred = Y_preds[:, -1]                     # shape [1, 14]
```

评估该模型对 t + 1 的预测时，验证集 MAE 为 23,350，表现很好。当然，预测距离越远，模型越不准确。例如，t + 14 的 MAE 为 35,315。

简单 RNN 可以很好地预测时间序列或处理其他类型的序列，但对于很长的时间序列或序列，表现就不那么好。下面讨论原因，并看看如何改进。

13.15　处理长序列（Handling Long Sequences）

要在长序列上训练 RNN，必须让它运行许多时间步，使展开后的 RNN 变成一个非常深的网络。与其他深度神经网络一样，它可能受到第 11 章讨论的不稳定梯度问题影响：训练可能耗时很长，或者过程不稳定。此外，当 RNN 处理长序列时，它会逐渐忘记序列最开始的输入。下面分别讨论这两个问题，先从不稳定梯度问题开始。

13.16　对抗不稳定梯度问题（Fighting the Unstable Gradients Problem）

我们在深度网络中用于缓解不稳定梯度问题的许多技巧，同样适用于 RNN：良好的参数初始化、更快的优化器、dropout 等。但是，非饱和激活函数（例如 ReLU）在这里可能没有那么有帮助，甚至可能使 RNN 在训练期间更加不稳定。为什么？假设梯度下降更新权重，使第一个时间步的输出略微增加。由于每个时间步使用相同的权重，第二个时间步的输出也可能略微增加，第三个时间步同样如此，最终输出可能爆炸；而非饱和激活函数无法阻止这种情况。

可以使用更小的学习率来降低风险，也可以使用双曲正切（tanh）这类饱和激活函数，这解释了为什么它是默认选择。

同样，梯度本身也可能爆炸。如果发现训练不稳定，可以监控梯度大小，并考虑使用梯度裁剪。

此外，批归一化（batch normalization，BN）无法像在深度前馈网络中那样高效地用于 RNN。实际上，不能在时间步之间使用 BN，只能在循环层之间使用。更准确地说，技术上可以把 BN 层加入记忆单元，使它在每个时间步应用于当前输入和前一时间步的隐藏状态。但这意味着无论输入和隐藏状态的实际尺度与偏移如何，每个时间步都使用同一个 BN 层和同

一组参数。César Laurent 等人在 2015 年的论文中证明，这种做法实际效果并不好。[4]他们发现，BN 只有应用于层的输入时才略有帮助；换句话说，在循环层之间（图 13-10 的垂直方向）使用 BN 略好于完全不使用，但不应在循环层内部（图 13-10 的水平方向）使用。

第 11 章介绍的层归一化（layer normalization，LN）在循环层内部通常比 BN 效果好一些。它通常在每个时间步、激活函数之前应用。遗憾的是，PyTorch 的 nn.RNN 模块不支持 LN，因此必须像前面手工实现循环那样，自己实现 RNN 的循环，并在每次迭代时应用 nn.LayerNorm。这并不困难，但会失去 nn.RNN 模块的简洁性和速度。例如，可以取前面第一个版本的 SimpleRnnModel，在记忆单元中、tanh 激活函数之前加入 nn.LayerNorm：

```
self.memory_cell = nn.Sequential(
    nn.Linear(input_size + hidden_size, hidden_size),
    nn.LayerNorm(hidden_size),
    nn.Tanh()
)
```

> 警告：层归一化并不总能带来帮助，需要实际尝试。一般来说，它在 LSTM 和 GRU 等门控 RNN 中效果更好；当时间序列经过预处理、消除了季节性或趋势时，也更可能有效。

类似地，如果希望在每个时间步之间应用 dropout，也必须编写自定义 RNN，因为 nn.RNN 不支持这种做法。不过，它支持在每个循环层之后加入 dropout 层，只需把 dropout 超参数设为所需的 dropout 率即可；默认值为 0。

通过这些技术，可以缓解不稳定梯度问题，更高效地训练 RNN。下面看看如何处理短期记忆问题。

> 提示：预测时间序列时，通常希望在预测值旁边给出误差条。一种方法是使用第 11 章介绍的 MC dropout。

13.17　应对短期记忆问题（Tackling the Short-Term Memory Problem）

数据经过 RNN 的多次变换后，每个时间步都会丢失一些信息。经过一段时间，RNN 的状态几乎不再包含最初输入的痕迹。这可能是致命问题。想象一条名叫 Dory 的鱼[5]尝试翻译一个长句：等她读完句子时，已经完全不知道句子是如何开始的。

[4]见本章参考文献第 9 条。

[5]Dory 是动画电影《海底总动员》（Finding Nemo）和《海底总动员 2：多莉去哪儿》（Finding Dory）中的角色，患有短期记忆丧失。

为解决这个问题，人们提出了许多具有长期记忆的单元。它们非常成功，以至于基础单元如今已经很少使用。先看看最流行的长期记忆单元：LSTM。

13.18　LSTM 单元（LSTM cells）

长短期记忆（long short-term memory，LSTM）单元由 Sepp Hochreiter 和 Jürgen Schmidhuber 于 1997 年提出，后来由 Alex Graves、Haşim Sak[6]和 Wojciech Zaremba 等多位研究人员不断改进。[7]可以直接用 nn.LSTM 模块替代 nn.RNN；这是一个即插即用的替换方案，通常表现好得多：训练收敛更快，模型能够检测数据中的长期模式。

这种神奇效果是如何实现的？图 13-12 展示了 LSTM 架构。如果忽略方框内部，LSTM 单元看起来与普通单元完全相同，只是它的状态被拆成两个向量：h(t) 和 c(t)，其中 c 表示“单元”（cell）。可以把 h(t) 看作短期状态，把 c(t) 看作长期状态。

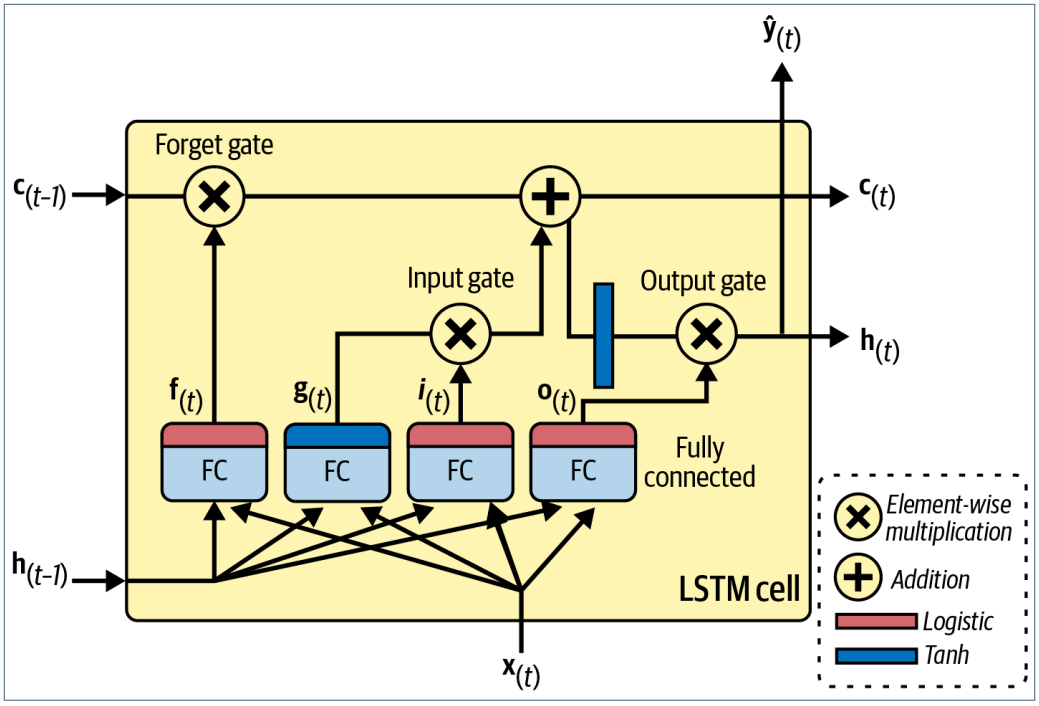

图 13-12: LSTM 单元（An LSTM cell）

现在打开方框。关键思想是：网络可以学习长期状态中应该存储什么、丢弃什么，以及应该读取什么。当长期状态 c(t-1) 从左向右穿过网络时，首先经过遗忘门（forget gate），丢弃部分记忆；然后通过加法操作加入新的记忆，这些记忆由输入门选择。得到的 c(t) 不再经

[6]见本章参考文献第 12 条。

[7]见本章参考文献第 11 条和第 13 条。

过其他变换，直接输出。因此在每个时间步都会丢弃一些记忆并加入一些新记忆。此外，经过加法操作后，长期状态会被复制并通过 tanh 函数，再由输出门进行过滤。这产生短期状态 h(t)，它也等于该时间步单元的输出 y(t)。下面看看新记忆来自哪里，以及各个门如何工作。

首先，当前输入向量 x(t) 和前一时间步的短期状态 h(t-1) 被送入四个不同的全连接层。它们分别承担不同作用：

- 主层输出 g(t)。它负责分析当前输入 x(t) 和前一时间步的短期状态 h(t-1)。在简单 RNN 单元中，除这一层外没有其他内容，它的输出会直接送给 y(t) 和 h(t)。但在 LSTM 单元中，这一层的输出不会直接输出，而是把其中最重要的部分存入长期状态，丢弃其余部分。
- 另外三个层是门控制器。由于它们使用逻辑激活函数，输出范围为 0 到 1。门控制器的输出会送入逐元素乘法：输出 0 就关闭对应的门，输出 1 就打开对应的门。具体来说：
- 遗忘门由 f(t) 控制，决定长期状态中的哪些部分应该擦除。
- 输入门由 i(t) 控制，决定 g(t) 中哪些部分应该加入长期状态。
- 输出门由 o(t) 控制，决定长期状态中的哪些部分应在当前时间步读取并输出到 h(t) 和 y(t)。

简而言之，LSTM 单元能够识别重要输入（输入门的作用），把它存入长期状态，在需要期间保留它（遗忘门的作用），并在需要时提取它（输出门的作用），同时整个过程仍然完全可微。这解释了为什么 LSTM 能够非常成功地捕捉时间序列、长文本、音频记录等数据中的长期模式。

公式 13-4 总结了单个实例在每个时间步如何计算单元的长期状态、短期状态和输出（整个小批量的公式非常相似）。

$$
\begin{aligned}
i^{(t)} &= \sigma\left(\mathbf{W}_{xi}^{\top}\mathbf{x}^{(t)} + \mathbf{W}_{hi}^{\top}\mathbf{h}^{(t-1)} + b_i\right) \\
f^{(t)} &= \sigma\left(\mathbf{W}_{xf}^{\top}\mathbf{x}^{(t)} + \mathbf{W}_{hf}^{\top}\mathbf{h}^{(t-1)} + b_f\right) \\
o^{(t)} &= \sigma\left(\mathbf{W}_{xo}^{\top}\mathbf{x}^{(t)} + \mathbf{W}_{ho}^{\top}\mathbf{h}^{(t-1)} + b_o\right) \\
g^{(t)} &= \tanh\left(\mathbf{W}_{xg}^{\top}\mathbf{x}^{(t)} + \mathbf{W}_{hg}^{\top}\mathbf{h}^{(t-1)} + b_g\right) \\
c^{(t)} &= f^{(t)} \otimes c^{(t-1)} + i^{(t)} \otimes g^{(t)} \\
y^{(t)} = h^{(t)} &= o^{(t)} \otimes \tanh\left(c^{(t)}\right)
\end{aligned}
\tag{13-4}
$$

公式 13-4　LSTM 计算（LSTM computations）

在这个公式中：

- W_xi、W_xf、W_xo 和 W_xg 是四个层与输入向量 x(t) 连接的权重矩阵。
- W_hi、W_hf、W_ho 和 W_hg 是四个层与前一时间步短期状态 h(t-1) 连接的权重矩阵。

- b_i、b_f、b_o 和 b_g 是四个层的偏置项。

> 提示：尝试在前面的模型中用 nn.LSTM 替换 nn.RNN，看看在客流数据集上能达到什么性能（可能需要进行一些超参数调优）。

与简单 RNN 一样，如果想在每个时间步加入层归一化或 dropout，必须手工实现循环。一种方法是直接使用公式 13-4，但更简单的方法是使用 nn.LSTMCell 模块，它负责运行单个时间步。例如，下面是一个简单实现：

```
class LstmModel(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__()
        self.hidden_size = hidden_size
        self.memory_cell = nn.LSTMCell(input_size, hidden_size)
        self.output = nn.Linear(hidden_size, output_size)
    def forward(self, X):
        batch_size, window_length, dimensionality = X.shape
        X_time_first = X.transpose(0, 1)
        H = torch.zeros(batch_size, self.hidden_size, device=X.device)
        C = torch.zeros(batch_size, self.hidden_size, device=X.device)
        for X_t in X_time_first:
            H, C = self.memory_cell(X_t, (H, C))
        return self.output(H)
```

这与 SimpleRnnModel 的第一个实现非常相似，只是现在在每个时间步使用 nn.LSTMCell，并且隐藏状态拆成两部分：短期状态 H 和长期状态 C。

LSTM 单元有多种变体。其中一种特别流行的变体是 GRU 单元，下面来看看它。

13.19　GRU 单元（GRU cells）

门控循环单元（gated recurrent unit，GRU）单元如图 13-13 所示，由 Kyunghyun Cho 等人在 2014 年的一篇论文中提出；这篇论文也介绍了前面讨论的编码器-解码器网络。[8]

[8] 见本章参考文献第 14 条。

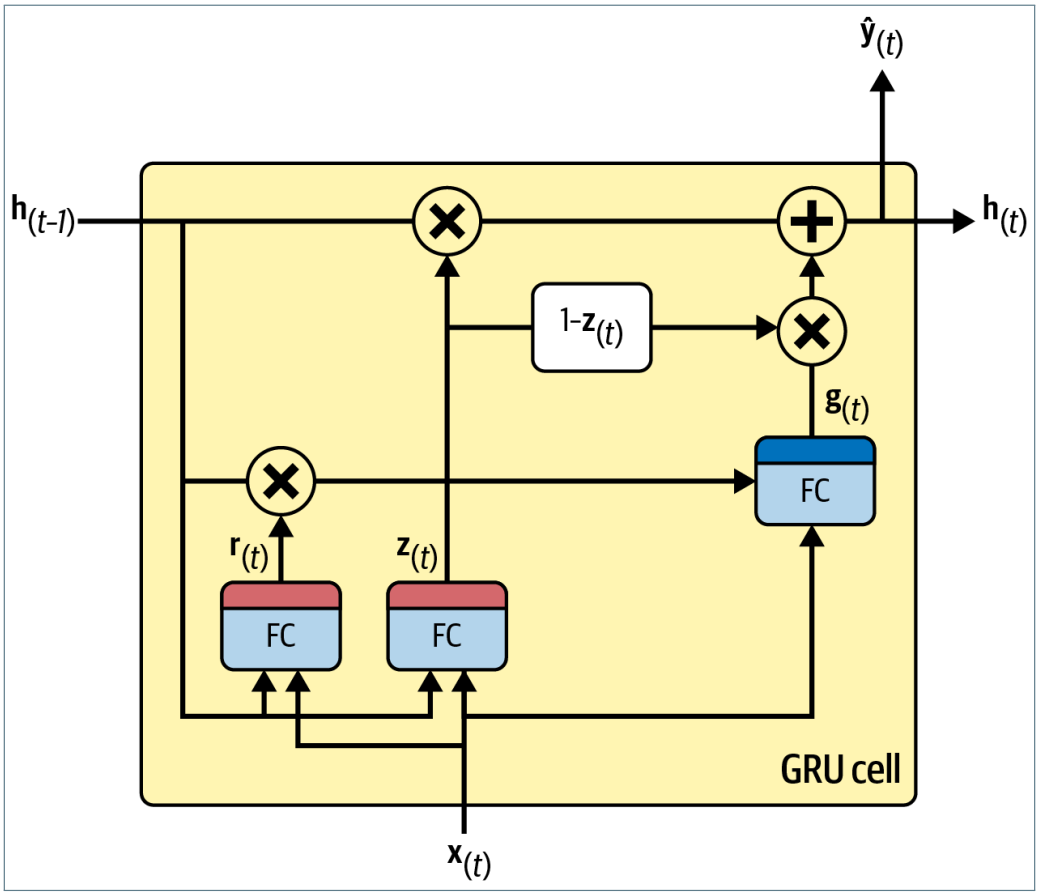

图 13-13: GRU 单元（A GRU cell）

GRU 单元是 LSTM 单元的简化版本，通常能达到同样好的效果。[9]主要简化如下：

- 两个状态向量合并为一个向量 h(t)。
- 一个门控制器 z(t) 同时控制遗忘门和输入门。如果门控制器输出 1，遗忘门打开（等于 1），输入门关闭（1 - 1 = 0）；如果输出 0，情况相反。换句话说，每当必须存储一条记忆时，存储位置会先被擦除。
- 没有输出门；每个时间步都会输出完整的状态向量。不过新增了门控制器 r(t)，用来控制前一状态的哪一部分会被提供给主层 g(t)。

公式 13-5 总结了单个实例在每个时间步如何计算单元状态。

[9]见本章参考文献第 15 条。

$$
\begin{aligned}
z^{(t)} &= \sigma\left(\mathbf{W}_{xz}^{\top}\mathbf{x}^{(t)} + \mathbf{W}_{hz}^{\top}\mathbf{h}^{(t-1)} + b_z\right)\\
r^{(t)} &= \sigma\left(\mathbf{W}_{xr}^{\top}\mathbf{x}^{(t)} + \mathbf{W}_{hr}^{\top}\mathbf{h}^{(t-1)} + b_r\right)\\
g^{(t)} &= \tanh\left(\mathbf{W}_{xg}^{\top}\mathbf{x}^{(t)} + \mathbf{W}_{hg}^{\top}\left(r^{(t)} \otimes \mathbf{h}^{(t-1)}\right) + b_g\right)\\
h^{(t)} &= z^{(t)} \otimes \mathbf{h}^{(t-1)} + \left(1 - z^{(t)}\right) \otimes g^{(t)}
\end{aligned}
\tag{13-5}
$$

公式 13-5　GRU 计算（GRU computations）

PyTorch 提供了 nn.GRU 层；只需用 nn.GRU 替换 nn.RNN 或 nn.LSTM。它还提供了 nn.GRUCell，可用于创建基于 GRU 单元的自定义 RNN（在前面的示例中把 nn.LSTMCell 替换为 nn.GRUCell，并删除 C 即可）。

LSTM 和 GRU 是 RNN 成功的主要原因之一。不过，虽然它们能够处理比简单 RNN 更长的序列，但短期记忆仍然相当有限，而且很难学习 100 个或更多时间步的长期模式，例如音频样本、长时间序列或长句子。一种解决方法是缩短输入序列，例如使用一维卷积层。

13.20　使用一维卷积层处理序列（Using 1D convolutional layers to process sequences）

第 12 章介绍过，二维卷积层会在图像上滑动若干个相当小的核（或滤波器），产生多张二维特征图，每个核对应一张。类似地，一维卷积层会沿序列滑动若干个核，每个核产生一张一维特征图。每个核会学习检测一个非常短的序列模式，其长度不会超过核大小。如果使用 10 个核，层的输出就由 10 个长度相同的一维序列组成；也可以把它看作一个 10 维序列。

因此，可以构建由循环层和一维卷积层（甚至一维池化层）混合组成的神经网络。不过，如前面提到的，必须交换 nn.Conv1d 输入和输出的最后两个维度，因为 nn.Conv1d 期望输入形状为 [batch size, input features, sequence length]，输出形状为 [batch size, output features, sequence length]。

> 警告：如果使用步幅为 1、"same" 填充的一维卷积层，输出序列长度与输入序列相同。但如果使用"valid" 填充或步幅大于 1，输出序列会短于输入序列，因此要相应调整目标。

例如，下面的模型由一维卷积层、GRU 层和线性输出层组成；所有层都接收和输出序列批次，即三维张量。一维卷积层使用步幅 2 将输入序列下采样为原来的一半。核大小与步幅相同（实际上更大），因此所有输入都会参与输出计算，模型可以学习保留最有用的信息，只丢弃不重要的细节。在 forward() 方法中，只需串联这些层，但在进入和离开 nn.Conv1d 前后交换最后两个维度，并忽略 nn.GRU 返回的隐藏状态：

```
class DownsamplingModel(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__()
        self.conv = nn.Conv1d(input_size, hidden_size, kernel_size=4, stride=2)
        self.gru = nn.GRU(hidden_size, hidden_size, batch_first=True)
        self.linear = nn.Linear(hidden_size, output_size)
    def forward(self, X):
        Z = X.permute(0, 2, 1)  # treat time as a spatial dimension
        Z = self.conv(Z)
        Z = Z.permute(0, 2, 1)  # swap back time & features dimensions
        Z = torch.relu(Z)
        Z, _states = self.gru(Z)
        return self.linear(Z)
torch.manual_seed(42)
dseq_model = DownsamplingModel(input_size=5, hidden_size=32, output_size=14)
dseq_model = dseq_model.to(device)
```

通过缩短时间序列，卷积层帮助 GRU 检测更长的模式，因此可以把窗口长度加倍到 112 天。注意，还必须从目标中裁掉最初三个时间步：卷积核大小为 4，所以卷积层的第一个输出基于输入时间步 0 到 3；因此，第一个预测应对应时间步 4 到 17，而不是时间步 1 到 14。此外，由于步幅为 2，还必须把目标下采样为原来的一半。为此，需要创建 Seq2SeqDataset 的子类：

```
class DownsampledDataset(Seq2SeqDataset):
    def __getitem__(self, idx):
        window, target = super().__getitem__(idx)
        return window, target[3::2]  # crop the first 3 targets and downsample
window_length = 112
dseq_train_set = DownsampledDataset(rail_train, window_length)
dseq_train_loader = DataLoader(dseq_train_set, batch_size=32, shuffle=True)
[...]  # create the datasets and data loaders for the validation and test sets
```

现在可以像往常一样训练模型。我们已经成功混合了卷积层和循环层。但如果只使用一维卷积层，完全去掉循环层，会怎样呢？

13.21　WaveNet

2016 年，Aaron van den Oord 和 DeepMind 的其他研究人员在一篇论文中提出了一种新架构，称为 WaveNet。[10]他们堆叠一维卷积层，并在每一层把膨胀率（dilation rate，即神经元输入之间的间隔）加倍：第一个卷积层一次只能看到两个时间步，下一层能看到四个时间步（感受野长度为 4），再下一层能看到八个时间步，依此类推（见图 13-14）。因此，较

[10]见本章参考文献第 16 条。

低层学习短期模式，较高层学习长期模式。得益于膨胀率加倍，网络可以非常高效地处理极长序列。

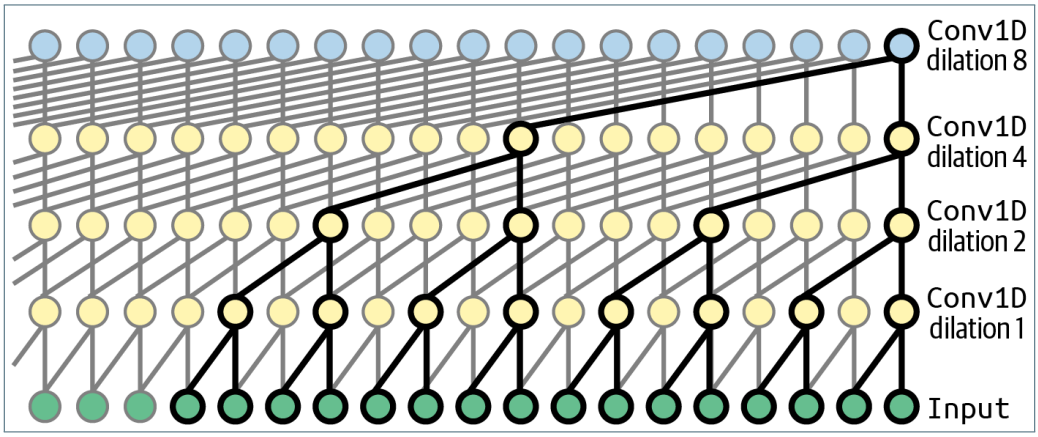

图 13-14: WaveNet 架构（WaveNet architecture）

论文作者实际堆叠了 10 个一维卷积层，膨胀率依次为 1、2、4、8、…、256、512；之后又堆叠一组相同的 10 层（膨胀率仍为 1、2、4、8、…、256、512），再重复一组。作者解释说，一组包含 10 层、膨胀率按上述方式变化的卷积层，相当于一个核大小为 1,024 的超高效卷积层，只是速度更快、能力更强，并且参数显著更少。

在每一层之前，他们还在输入序列左侧填充数量等于膨胀率的零，以在整个网络中保持相同的序列长度。之所以只在左侧填充而不是两侧填充，是为了确保卷积层在预测时不会偷看未来。这使它成为一个因果模型（causal model）。

下面实现一个简化的 WaveNet，用于处理前面相同的序列。[11]先创建一个自定义的 CausalConv1d 模块。它的作用与 nn.Conv1d 相同，只是把输入填充到左侧：

```
import torch.nn.functional as F
class CausalConv1d(nn.Conv1d):
    def forward(self, X):
        padding = (self.kernel_size[0] - 1) * self.dilation[0]
        X = F.pad(X, (padding, 0))
        return super().forward(X)
```

这里继承 nn.Conv1d 类并重写 forward() 方法。首先计算所需左侧填充大小，然后在调用基类 forward() 方法之前使用 pad() 函数填充序列。pad() 接收两个参数：待填充的张量 X，以及一个整数元组，表示最后一个维度（时间维）左侧和右侧分别要填充多少。

现在可以构建 WaveNet 模型：

[11]完整的 WaveNet 还使用了更多技巧，例如类似 ResNet 的跳跃连接，以及类似 GRU 单元的门控激活单元。

```
class WavenetModel(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__()
        layers = []
        for dilation in (1, 2, 4, 8) * 2:
            conv = CausalConv1d(input_size, hidden_size, kernel_size=2,
                                dilation=dilation)
            layers += [conv, nn.ReLU()]
            input_size = hidden_size
        self.convs = nn.Sequential(*layers)
        self.output = nn.Linear(hidden_size, output_size)
    def forward(self, X):
        Z = X.permute(0, 2, 1)
        Z = self.convs(Z)
        Z = Z.permute(0, 2, 1)
        return self.output(Z)
torch.manual_seed(42)
wavenet_model = WavenetModel(input_size=5, hidden_size=32, output_size=14)
wavenet_model = wavenet_model.to(device)
```

构造函数创建八个带不同膨胀率的 CausalConv1d 层（1、2、4、8，然后再次使用 1、2、4、8），每层后接 ReLU 激活函数，并把所有模块串联到 nn.Sequential 模块 self.convs 中。同时创建输出 nn.Linear 层。在 forward() 方法中，先交换输入的最后两个维度，把它们通过卷积层，再把最后两个维度交换回原顺序，最后通过输出层。

由于因果填充，每个卷积层输出的序列长度都与输入序列相同，因此训练目标可以使用完整的 112 天序列，不需要裁剪或下采样。这样可以使用为 Seq2SeqModel 构建的数据加载器，即 seq_train_loader 和 seq_valid_loader。

本节讨论的模型在客流预测任务上性能相近，但具体表现可能因任务和可用数据量而有很大差异。在 WaveNet 论文中，作者在各种音频任务（因此得名）上取得了先进性能，包括文本转语音任务，能够生成多种语言中非常逼真的声音。他们还使用模型逐个音频样本地生成音乐。考虑到一秒音频可能包含数万个时间步，这一成果更加令人印象深刻——即使 LSTM 和 GRU 也无法处理如此长的序列。

> 警告：如果在从 2020 年开始的测试期上评估芝加哥客流预测模型，会发现表现远差于预期。原因是新冠疫情在那时开始，严重影响了公共交通。正如前面提到的，这些模型只有在过去学到的模式在未来继续成立时才会表现良好。因此，在把模型部署到生产环境前，务必确认它在近期数据上运行良好；部署后也要定期监控其性能。

到这里，你已经可以处理各种时间序列了！第 14 章将继续探索 RNN，并介绍如何使用它们处理各种 NLP 任务。

13.22　练习（Exercises）

1. 你能想到哪些适合序列到序列 RNN 的应用？序列到向量 RNN 和向量到序列 RNN 呢？
2. RNN 层的输入必须有多少个维度？每个维度表示什么？输出又是什么情况？
3. 如何在 PyTorch 中构建一个深层序列到序列 RNN？
4. 假设有一个每日单变量时间序列，想使用 RNN 预测未来七天。应该使用哪种架构？
5. 训练 RNN 时主要有哪些困难？如何处理？
6. 你能画出 LSTM 单元的架构吗？
7. 为什么要在 RNN 中使用一维卷积层？
8. 可以使用哪种神经网络架构对视频进行分类？
9. 尝试调整 Seq2SeqModel 模型，同时预测未来 14 天的轨道交通和公交客流。此时模型需要预测 28 个值，而不是 14 个。
10. 下载并解压 Bach chorales 数据集。该数据集包含 Johann Sebastian Bach 创作的 382 首圣咏，每首长度为 100 到 640 个时间步，每个时间步包含 4 个整数，每个整数对应钢琴上的一个音符索引（值为 0 表示不演奏）。训练一个循环模型、卷积模型或两者混合的模型，根据一段时间步序列预测下一个时间步（四个音符）。然后使用模型逐个时间步生成类似 Bach 的音乐：给模型输入一首圣咏的开头，让它预测下一个时间步，再把这个时间步追加到输入序列，继续让模型预测下一个音符，如此反复。还可以了解 Google 的 Coconet 模型，它曾用于制作 Bach 主题的 Google 涂鸦。
11. 为 Quick Draw 数据集训练一个分类模型。该数据集包含数百万张各种物体的草图。先下载少数类别的简化数据（例如 ant.ndjson、axe.ndjson 和 bat.ndjson）。每个 NDJSON 文件每行包含一个 JSON 对象，可以使用 Python 的 json.loads() 函数解析。这会得到一个草图列表，每个草图由一个 Python 字典表示。每个字典中的 drawing 条目包含笔画列表。可以把这个列表转换为形状为 [strokes, x coordinates, y coordinates] 的三维浮点张量。由于 RNN 接收单个序列作为输入，需要把每个草图的所有笔画连接成一个序列。最好加入额外特征，让 RNN 知道当前笔画进行到什么位置（例如从 0 到 1）。换句话说，模型接收的序列中，每个时间步有三个特征：笔的 x、y 坐标，以及当前笔画的进度比例。
12. 创建一个包含自己说“yes”或“no”的短音频记录数据集，并在上面训练二元分类 RNN。例如：
    - a. 使用 Audacity 等音频录制软件，尽可能多次录制自己说“yes”，每个词之间留出短暂停顿；对“no”也创建类似录音。尝试覆盖现实生活中可能出现的各种发音方式。
    - b. 使用 TorchAudio 库的 torchaudio.load() 函数加载每个 WAV 文件。它会返回一个包含音频的张量，以及一个表示每秒采样数的整数。音频张量形状为 [channels, samples]：

单声道有一个通道，立体声有两个通道。可以沿通道维度取平均，把立体声转换为单声道。

- c. 通过在静音处切分，把每个录音切成单个词。可以使用 torchaudio.transforms.Vad 变换（Voice Activity Detection，语音活动检测）。
- d. 由于序列很长，直接在其上训练 RNN 很困难，因此可以先把音频转换为频谱图。可以使用非常适合语音的 torchaudio.transforms.MelSpectrogram 变换。输出序列会大幅缩短，但通道数会增加很多。
- e. 现在尝试在 yes/no 数据集上构建并训练二元分类 RNN！还可以考虑与世界分享数据集和模型，例如通过 Hugging Face Hub。

这些练习的解答位于本章 notebook 的末尾：https://homl.info/colab-p。

13.22.1　参考文献

1. 注意，许多研究人员更喜欢在 RNN 中使用双曲正切（tanh）激活函数，而不是 ReLU 激活函数。例如，参见 Vu Pham 等人 2013 年的论文《Dropout Improves Recurrent Neural Networks for Handwriting Recognition》。基于 ReLU 的 RNN 也可行，例如 Quoc V. Le 等人 2015 年的论文《A Simple Way to Initialize Recurrent Neural Networks of Rectified Linear Units》。
2. Michael I. Jordan, “Attractor Dynamics and Parallelism in a Connectionist Sequential Machine”, Proceedings of the Eighth Annual Conference of the Cognitive Science Society (1986).
3. Jeffrey L. Elman, “Finding Structure in Time”, Cognitive Science, Volume 14, Issue 2 (1990).
4. Nal Kalchbrenner and Phil Blunsom, “Recurrent Continuous Translation Models”, Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing (2013): 1700-1709.
5. 芝加哥交通管理局的最新数据可在 Chicago Data Portal 获取。
6. 更多 ACF-PACF 方法的细节，可参考 Jason Brownlee 撰写的相关介绍。
7. 注意，验证期从 2019 年 1 月 1 日开始，因此第一个预测对应 8 周之后的 2019 年 2 月 26 日。评估基线模型时，使用的是从 3 月 1 日开始的预测，但两者应该足够接近。
8. 不能使用 ahead_model 完成这一点，因为它需要公交和轨道交通共同作为输入，但它只预测轨道交通客流。
9. César Laurent et al., “Batch Normalized Recurrent Neural Networks”, Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (2016): 2657-2661.

10. Dory 是动画电影《海底总动员》(Finding Nemo) 和《海底总动员 2: 多莉去哪儿》(Finding Dory) 中的角色，患有短期记忆丧失。
11. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory”, Neural Computation 9, no. 8 (1997): 1735-1780.
12. Haşim Sak et al., “Long Short-Term Memory Based Recurrent Neural Network Architectures for Large Vocabulary Speech Recognition”, arXiv preprint arXiv:1402.1128 (2014).
13. Wojciech Zaremba et al., “Recurrent Neural Network Regularization”, arXiv preprint arXiv:1409.2329 (2014).
14. Kyunghyun Cho et al., “Learning Phrase Representations Using RNN Encoder-Decoder for Statistical Machine Translation”, Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (2014): 1724-1734.
15. 参见 Klaus Greff et al., “LSTM: A Search Space Odyssey”, IEEE Transactions on Neural Networks and Learning Systems 28, no. 10 (2017): 2222-2232。该论文似乎表明，各种 LSTM 变体的性能大致相同。
16. Aaron van den Oord et al., “WaveNet: A Generative Model for Raw Audio”, arXiv preprint arXiv:1609.03499 (2016).
17. 完整的 WaveNet 还使用了更多技巧，例如类似 ResNet 的跳跃连接，以及类似 GRU 单元的门控激活单元。

第 14 章　NLP：RNN 与注意力

Natural Language Processing with RNNs and Attention

1950 年，艾伦·图灵（Alan Turing）构想了著名的图灵测试（Turing test）[1]，提出了一种评估机器能否达到人类智能水平的方法。他本可以测试机器识别图片中的猫、下棋、作曲或逃出迷宫的能力，但有趣的是，他选择了一项语言任务。更具体地说，他设计了一个能够欺骗对话者、让对方以为自己是人的聊天机器人（chatbot）。[2]这个测试确实有弱点：一组硬编码规则也能欺骗毫无戒心或经验不足的人类（例如，机器可以针对关键词给出模糊的预定义回答，假装自己在开玩笑或喝醉了，借此解释奇怪的回答，也可以用反问来逃避困难问题），而且它完全忽略了人类智能的许多方面，例如解释面部表情等非语言交流，或学习手工技能的能力。不过，这项测试确实突出了一个事实：掌握语言可以说是智人（Homo sapiens）最重要的认知能力。

直到最近，最先进的自然语言处理（natural language processing，NLP）模型几乎都基于循环神经网络（recurrent neural network，RNN），第 13 章介绍了这类网络。然而近些年，RNN 已被 Transformer 取代，我们将在第 15 章深入讨论 Transformer。即便如此，学习 RNN 如何用于 NLP 任务仍然很重要，因为这有助于更好地理解 Transformer。此外，本章讨论的大多数技术也同样适用于 Transformer 架构，例如分词（tokenization）、Beam Search、注意力机制等。更重要的是，RNN 最近以状态空间模型（state-space model，SSM）的形式意外回归（参见 https://homl.info 上的“State-Space Models (SSMs)”）。

本章分为三个部分。第一部分从构建字符 RNN（character RNN，或 char-RNN）开始，训练它预测句子中的下一个字符。在此过程中，我们将学习可训练嵌入（trainable embedding）。字符 RNN 会成为我们的第一个小型语言模型，能够生成原创文本。

第二部分转向文本分类，更具体地说，是情感分析（sentiment analysis），其目标是预测文本的正面或负面程度。我们的模型将读取电影评论，并估计评论者对电影的感受。这一次，我们不再把文本拆分成单个字符，而是拆分成 token：token 是来自固定大小词汇表的一小段文本，可以是英语中最常见的 10,000 个单词、最常见的子词（例如“smartest”= “smart” + “est”），也可以是单个字符或字节。我们将使用 tokenizer 把文本切分成 token。本节还会介绍 Hugging Face 的常用库：用于下载数据集的 Datasets 库、用于 tokenizer 的 Tokenizers 库，以及用于加载流行模型的 Transformers 库。Hugging Face 是极具影响力的公司和开源社区，在开源 AI 尤其是 NLP 领域发挥着核心作用。

[1]见本章参考文献第 1 条。

[2]当然，“chatbot”这个词出现得晚得多。图灵把他的测试称为“模仿游戏”（imitation game）。参见本章参考文献第 2 条。

本章的最终挑战是神经机器翻译（neural machine translation，NMT），也是第三部分的主题：我们将构建一个能够把英语翻译成西班牙语的编码器-解码器（encoder-decoder）模型。这会引出注意力机制（attention mechanism），我们将把它应用于编码器-解码器模型，以增强模型处理长输入文本的能力。顾名思义，注意力机制是一类神经网络组件，它能够学习在每个时间步选择应该关注的输入部分。注意力机制直接促成了 Transformer 革命，我们将在下一章看到这一点。

先从一个简单而有趣的字符 RNN 模型开始，让它（大致）像莎士比亚那样写作。

14.1　使用字符 RNN 生成莎士比亚文本（Generating Shakespearean Text Using a Character RNN）

在安德烈·卡帕西（Andrej Karpathy）2015 年发表的著名博文《循环神经网络不可思议的有效性》（The Unreasonable Effectiveness of Recurrent Neural Networks）中，他展示了如何训练 RNN 来预测句子中的下一个字符。训练完成后，这个字符 RNN 就能逐个字符地生成新文本。下面是一段字符 RNN 在莎士比亚全部作品上训练后生成的文本：

```
PANDARUS:
Alas, I think he shall become approached and the day
When little srain would be attain'd into being never fed,
And who is but a chain and subjects of his death,
I should not sleep.
```

这算不上杰作，但模型仅仅通过学习预测句子中的下一个字符，就能学会单词、语法、正确的标点等内容，仍然令人印象深刻。这是我们的第一个语言模型示例。本节剩余部分将逐步构建字符 RNN，从创建数据集开始。

14.1.1　创建训练数据集（Creating the Training Dataset）

首先，下载莎士比亚作品的一个子集（约 25%）。数据来自安德烈·卡帕西的 char-rnn 项目：

```
from pathlib import Path
import urllib.request
def download_shakespeare_text():
    path = Path("datasets/shakespeare/shakespeare.txt")
    if not path.is_file():
        path.parent.mkdir(parents=True, exist_ok=True)
        url = "https://homl.info/shakespeare"
        urllib.request.urlretrieve(url, path)
    return path.read_text()
shakespeare_text = download_shakespeare_text()
```

打印前几行：

```
>>> print(shakespeare_text[:80])
First Citizen:
Before we proceed any further, hear me speak.
All:
Speak, speak.
```

看起来确实像莎士比亚的作品。

神经网络处理的是数字，而不是文本，因此需要一种把文本编码为数字的方法。一般来说，这需要把文本拆分成 token（例如单词或字符），再为每个可能的 token 分配一个整数 ID。现在把文本拆分为字符，并为每个字符分配一个 ID。首先需要找出文本中出现过的字符列表，这将构成 token 词汇表：

```
>>> vocab = sorted(set(shakespeare_text.lower()))
>>> "".join(vocab)
"\n !$&',-.3:;?abcdefghijklmnopqrstuvwxyz"
```

这里调用 **`lower()`** 忽略大小写，从而减小词汇表规模。接下来必须为每个字符分配 token ID。可以直接使用它在词汇表中的索引。为了对模型的输出进行解码，还需要一种从 token ID 回到字符的方法：

```
>>> char_to_id = {char: index for index, char in enumerate(vocab)}
>>> id_to_char = {index: char for index, char in enumerate(vocab)}
>>> char_to_id["a"]
13
>>> id_to_char[13]
'a'
```

接下来创建两个辅助函数，把文本编码为 token ID 张量，并把张量解码回文本：

```
import torch
def encode_text(text):
    return torch.tensor([char_to_id[char] for char in text.lower()])
def decode_text(char_ids):
    return "".join([id_to_char[char_id.item()] for char_id in char_ids])
```

试用它们：

```
>>> encoded = encode_text("Hello, world!")
>>> encoded
tensor([20, 17, 24, 24, 27,  6,  1, 35, 27, 30, 24, 16,  2])
>>> decode_text(encoded)
'hello, world!'
```

现在准备数据集。此时我们只有一个极长的字符序列，其中包含莎士比亚的作品。与第 13 章一样，可以把这个长序列转换为一个窗口数据集，然后用它训练序列到序列（sequence-to-sequence）的 RNN。目标与输入相似，但沿时间轴向“未来”移动一个时间步。例如，一个样本的输入可以是表示文本“to be or not to b”（不含最后的“e”）的字符 ID 序列，对应目标则是表示文本“o be or not to be”的字符 ID 序列（包含最后的“e”，但不含开头的“t”）。创建数据集类：

```
from torch.utils.data import Dataset, DataLoader
class CharDataset(Dataset):
    def __init__(self, text, window_length):
        self.encoded_text = encode_text(text)
        self.window_length = window_length
    def __len__(self):
        return len(self.encoded_text) - self.window_length
    def __getitem__(self, idx):
        if idx >= len(self):
            raise IndexError("dataset index out of range")
        end = idx + self.window_length
        window = self.encoded_text[idx : end]
        target = self.encoded_text[idx + 1 : end + 1]
        return window, target
```

然后像往常一样创建数据加载器。文本相当大，因此可以用约 90% 的数据训练（约一百万个字符），用 5% 验证，用 5% 测试（各约 60,000 个字符）：

```
window_length = 50
batch_size = 512  # reduce if your GPU cannot handle such a large batch size
train_set = CharDataset(shakespeare_text[:1_000_000], window_length)
valid_set = CharDataset(shakespeare_text[1_000_000:1_060_000], window_length)
test_set = CharDataset(shakespeare_text[1_060_000:], window_length)
train_loader = DataLoader(train_set, batch_size=batch_size, shuffle=True)
valid_loader = DataLoader(valid_set, batch_size=batch_size)
test_loader = DataLoader(test_set, batch_size=batch_size)
```

每个批次由 512 个长度为 50 的字符窗口组成，每个字符用 token ID 表示，每个窗口还带有一个向右偏移一个字符的目标窗口。注意，每个 epoch 都会打乱训练批次（见图 14-1）。

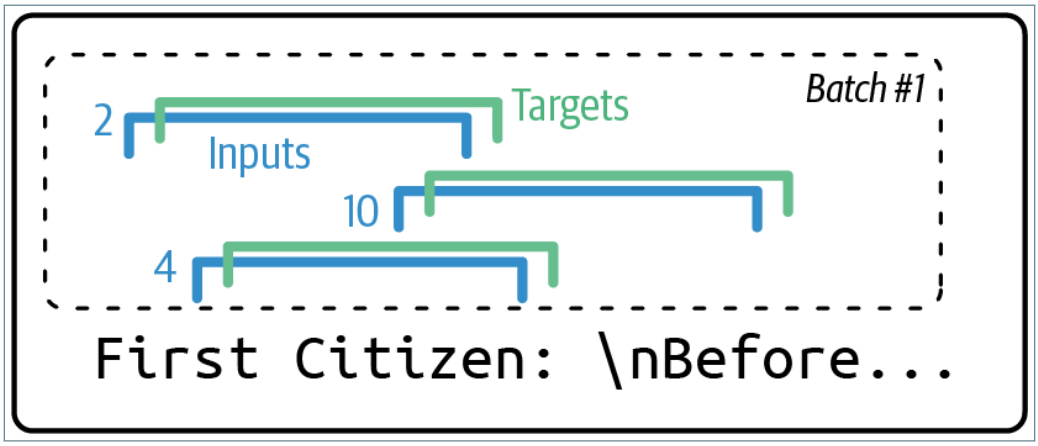

图 14-1: 每个训练批次由打乱顺序的窗口及其偏移后的目标组成；图中窗口长度为 10，而不是 50（Each training batch is composed of shuffled windows, along with their shifted targets; in this figure, the window length is 10 instead of 50）

> 提示：这里把窗口长度设为 50，但你可以尝试调整它。在较短的输入序列上训练 RNN 更容易、更快，但 RNN 无法学习长于窗口长度的模式，因此不要把窗口设得太小。

虽然技术上可以把 token ID 直接输入神经网络而不做进一步预处理，但效果不会很好。正如第 2 章所见，大多数机器学习模型（包括神经网络）都假设相似的输入代表相似的事物。遗憾的是，相近的 ID 可能表示完全无关的 token，反过来，相距很远的 ID 也可能表示相似的 token。神经网络会因此产生奇怪的偏置，在训练过程中也很难克服这种偏差。

一种解决方案是使用独热编码（one-hot encoding），因为所有独热向量彼此距离相同。然而，当词汇表很大时，独热向量也同样庞大。在本例中，词汇表只有 39 个字符，因此每个字符可以用 39 维独热向量表示。这仍然可管理；但如果处理的是单词，词汇表规模可能达到数万，独热编码就不可行了。幸运的是，神经网络还有更好的选择：嵌入（embedding）。

14.2　嵌入（Embeddings）

嵌入是高维数据（通常是类别特征）的稠密表示。如果有 50,000 个可能的类别，独热编码会产生一个 50,000 维稀疏向量（其中绝大多数元素为零）。相比之下，嵌入是相对较小的稠密向量，例如只有 300 个维度。

> 提示：嵌入大小是一个可以调节的超参数。经验法则是，合适的嵌入大小通常接近类别数量的平方根。

在深度学习中，嵌入通常随机初始化，然后与模型的其他参数一起通过梯度下降训练。例如，如果要在加州住房数据集（见第 2 章）上训练神经网络，可以用嵌入表示 **ocean_**

`proximity` 类别特征。`NEAR BAY` 类别一开始可以用随机向量 `[0.831, 0.696]` 表示，而 `NEAR OCEAN` 类别可以用另一个随机向量 `[0.127, 0.868]` 表示（本例使用二维嵌入）。

由于嵌入是可训练的，训练过程中它们会逐渐改进；而且在这个例子中，它们代表相当相似的类别，因此梯度下降最终会把它们推得更近，同时倾向于把它们移离 `INLAND` 类别的嵌入（见图 14-2）。表示越好，神经网络就越容易做出准确预测，因此训练往往会使嵌入成为类别的有用表示。这称为表示学习（representation learning），第 18 章还会介绍其他类型的表示学习。

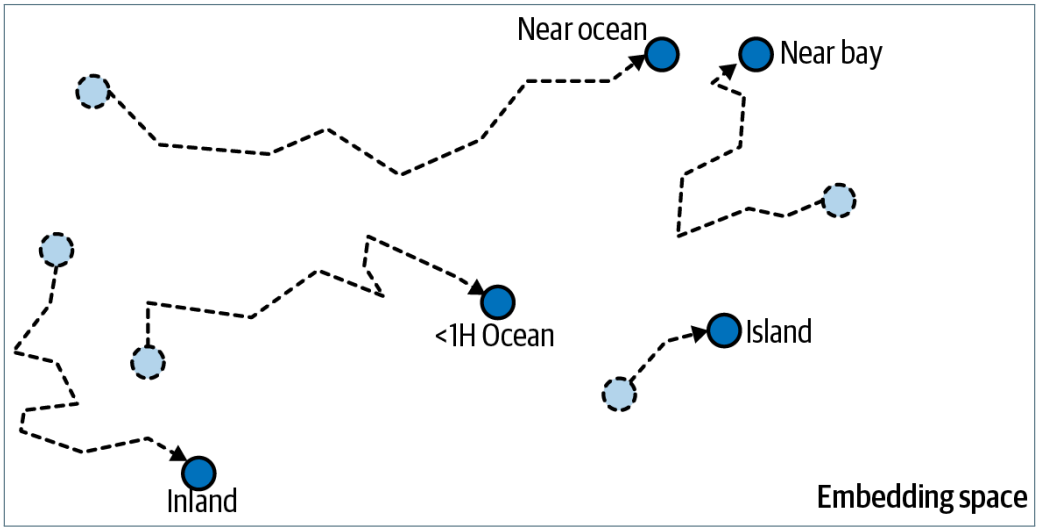

图 14-2: 嵌入会在训练过程中逐渐改进（Embeddings will gradually improve during training）

嵌入不仅通常是当前任务的有用表示，而且往往也能成功复用于其他任务。最常见的例子是词嵌入（word embedding，即单个单词的嵌入）：处理自然语言任务时，使用预训练词嵌入通常比自己训练更好，本章后面会看到这一点。

使用向量表示单词的想法可以追溯到 20 世纪 60 年代，人们已经使用包括神经网络在内的许多复杂技术来生成有用向量。但这一领域真正起飞是在 2013 年：Tomáš Mikolov 和其他 Google 研究人员发表了一篇论文[3]，描述了一种使用神经网络高效学习词嵌入的技术，其表现显著优于此前的尝试。他们可以在超大规模文本语料库上学习嵌入：训练神经网络预测给定单词附近的单词，由此得到令人惊叹的词嵌入。例如，同义词的嵌入非常接近，France、Spain 和 Italy 等语义相关的单词也会聚集在一起。

词嵌入并不只是关于距离：它们还沿着有意义的轴组织在嵌入空间中。一个著名例子是：计算 `King - Man + Woman`（对这些单词的嵌入向量执行加减），结果会非常接近单词 Queen 的嵌入（见图 14-3）。换句话说，词嵌入编码了性别这一概念。同样，计算 `Madrid - Spain + France`，结果会接近 Paris，似乎表明首都这一概念也被编码到了嵌入中。

[3] 见本章参考文献第 3 条。

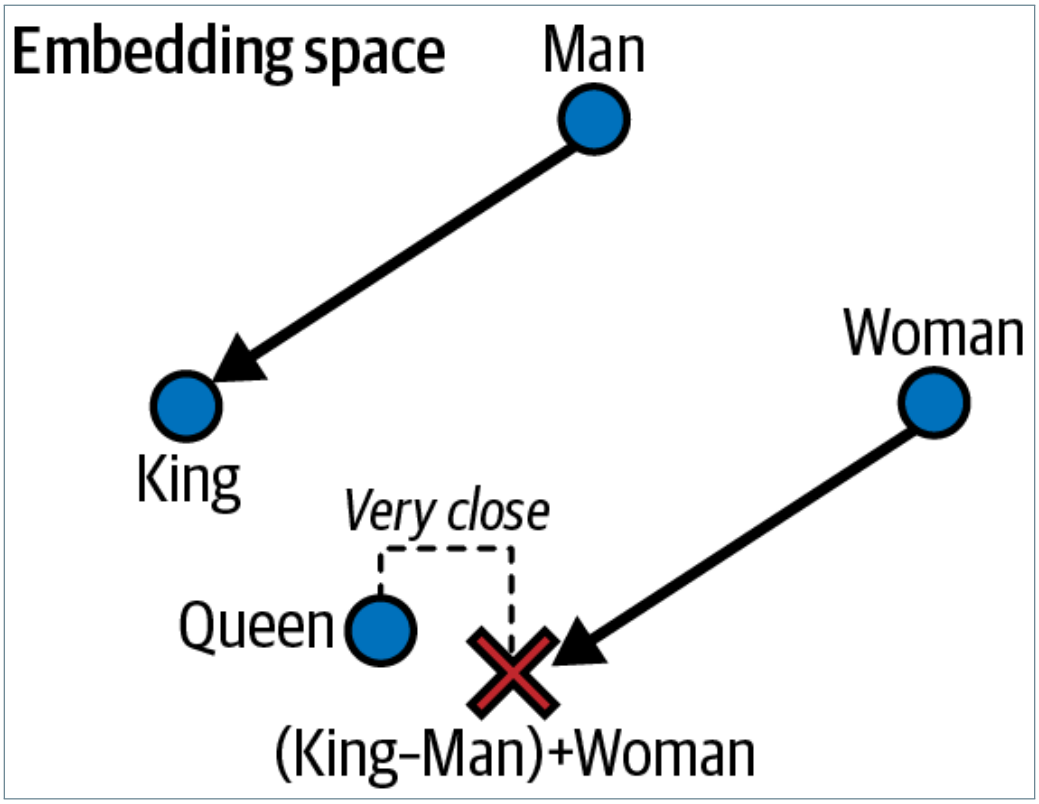

图 14-3: 相似单词的词嵌入往往彼此接近，一些坐标轴似乎编码了有意义的概念（Word embeddings of similar words tend to be close, and some axes seem to encode meaningful concepts）

> 警告：词嵌入可能具有一些有意义的结构，例如“King - Man + Woman”。但它们也包含噪声，通常难以解释。我在本章笔记末尾加入了一些代码，你可以自行判断。遗憾的是，词嵌入有时会捕捉我们最糟糕的偏见。例如，虽然它们正确学到了 Man 之于 King 就像 Woman 之于 Queen，但也似乎学到了 Man 之于 Doctor 就像 Woman 之于 Nurse——这是相当性别歧视的偏见！公平地说，Malvina Nissim 等人在 2019 年的一篇论文[a]指出，这个具体例子可能被夸大了。不过，确保深度学习算法的公平性仍然是一个重要且活跃的研究主题。
>
> [a] 见本章参考文献第 4 条。

PyTorch 提供了 **`nn.Embedding`** 模块，它封装了一个嵌入矩阵：矩阵每一行对应一个可能的类别（例如词汇表中的一个 token），每一列对应一个嵌入维度。嵌入维度是可调节的超参数。默认情况下，嵌入矩阵是随机初始化的。

要把类别 ID 转换为嵌入，**`nn.Embedding`** 层只需查找并返回对应的行，仅此而已！例如，初始化一个包含五个类别、使用三维嵌入的 **`nn.Embedding`** 层，并用它编码一些类别：

```
>>> import torch.nn as nn
>>> torch.manual_seed(42)
>>> embed = nn.Embedding(5, 3)  # 5 categories × 3D embeddings
>>> embed(torch.tensor([[3, 2], [0, 2]]))
tensor([[[ 0.2674,  0.5349,  0.8094],
         [ 2.2082, -0.6380,  0.4617]],
        [[ 0.3367,  0.1288,  0.2345],
         [ 2.2082, -0.6380,  0.4617]]], grad_fn=<EmbeddingBackward0>)
```

可以看到，类别 3 被编码为三维向量 `[0.2674, 0.5349, 0.8094]`；类别 2 两次被编码为 `[2.2082, -0.6380, 0.4617]`；类别 0 被编码为 `[0.3367, 0.1288, 0.2345]`（本例没有使用类别 1 和 4）。由于这一层还没有训练，这些编码只是随机值。

注意，嵌入层在数学上等价于先进行独热编码，再通过一个线性层（没有 bias 参数）。例如，创建一个 `nn.Linear(5, 3, bias=False)` 线性层，并传入独热向量 `torch.tensor([[0., 0., 0., 1., 0.]])`，得到的向量等于线性层转置权重矩阵的第 3 行，而这个矩阵实际上就是嵌入矩阵。原因是转置权重矩阵中的所有行都会乘以零，只有第 3 行乘以 1，因此结果就是第 3 行。不过，使用 `nn.Embedding(5, 3)` 并传入 `torch.tensor([3])` 高效得多，因为它直接查找嵌入矩阵中的第 3 行，不需要独热编码，也不需要进行大量无意义的零乘法。

现在已经了解嵌入，可以构建莎士比亚模型了。

14.3　构建和训练字符 RNN 模型（Building and Training the Char-RNN Model）

由于数据集相当大，而且语言建模任务很困难，需要使用比简单 RNN 更强的模型。下面构建并训练一个两层 `nn.GRU` 模型，每层包含 128 个单元，并加入少量 dropout：

```
class ShakespeareModel(nn.Module):
    def __init__(self, vocab_size, n_layers=2, embed_dim=10, hidden_dim=128,
                 dropout=0.1):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim)
        self.gru = nn.GRU(embed_dim, hidden_dim, num_layers=n_layers,
                          batch_first=True, dropout=dropout)
        self.output = nn.Linear(hidden_dim, vocab_size)
    def forward(self, X):
        embeddings = self.embed(X)
        outputs, _states = self.gru(embeddings)
        return self.output(outputs).permute(0, 2, 1)
torch.manual_seed(42)
model = ShakespeareModel(len(vocab)).to(device)
```

逐段来看：

- 使用 `nn.Embedding` 编码字符 ID。输入是形状为 `[batch size, window length]` 的整数张量，输出是形状为 `[batch size, window length, embedding size]` 的浮点张量；嵌入大小暂设为 10。
- `nn.GRU` 有 10 个输入、128 个输出和两层。必须指定 `batch_first=True`，否则该层会假设 batch 维位于 time 维之后。
- `nn.Linear` 输出层必须有 39 个单元，因为文本中有 39 个不同字符。
- `nn.GRU` 的输出形状为 `[batch size, window length, hidden size]`，`nn.Linear` 的输出形状为 `[batch size, window length, vocabulary size]`；但 `nn.CrossEntropyLoss` 和 `Accuracy` 要求类别维位于第二维，因此必须调换最后两个维度。最终隐藏状态在这里被忽略。[4]

现在可以像往常一样使用 `nn.CrossEntropyLoss` 和 `Accuracy` 训练、评估模型，并预测下一个字符：

```
model.eval()  # don't forget to switch the model to evaluation mode!
text = "To be or not to b"
encoded_text = encode_text(text).unsqueeze(dim=0).to(device)
with torch.no_grad():
    Y_logits = model(encoded_text)
    predicted_char_id = Y_logits[0, :, -1].argmax().item()
    predicted_char = id_to_char[predicted_char_id]  # correctly predicts "e"
```

首先编码文本，添加大小为 1 的 batch 维度并把张量移动到 GPU。模型会为每个时间步输出 logits；这里只取最后一个时间步（`-1`），使用 `argmax()` 找到最大 logit 对应的 token ID，再用 `item()` 提取它，最后转回字符。模型正确预测出了“e”。

> 警告：如果在启用 GPU 的 Colab 上运行，训练可能需要数小时。可以减少 epoch 数，但准确率可能降低；如果会话超时，请迅速重新连接，否则运行时会被销毁。

14.3.1　生成伪莎士比亚文本（Generating Fake Shakespearean Text）

把一段文本输入模型，让模型预测最可能的下一个字符并追加到文本末尾，再反复执行，这称为贪心解码（greedy decoding）。实践中它常常导致单词重复。另一种方法是根据模型的概率分布随机采样；若某 token 的概率为 p，就以概率 p 采样它。可以使用 `torch.multinomial()`：

[4] 在 Python 中，未使用的变量通常以一个下划线作为前缀。见本章参考文献第 5 条。

```
>>> torch.manual_seed(42)
>>> probs = torch.tensor([[0.5, 0.4, 0.1]])  # probas = 50%, 40%, and 10%
>>> samples = torch.multinomial(probs, replacement=True, num_samples=8)
>>> samples
tensor([[0, 0, 0, 0, 1, 0, 2, 2]])
```

还可以把 logits 除以称为 temperature 的数来控制多样性。temperature 接近零时偏向高概率字符；temperature 较高时，各字符概率更接近。下面定义按此方法采样下一个字符的辅助函数：

```
import torch.nn.functional as F
def next_char(model, text, temperature=1):
    encoded_text = encode_text(text).unsqueeze(dim=0).to(device)
    with torch.no_grad():
        Y_logits = model(encoded_text)
        Y_probas = F.softmax(Y_logits[0, :, -1] / temperature, dim=-1)
        predicted_char_id = torch.multinomial(Y_probas, num_samples=1).item()
    return id_to_char[predicted_char_id]
```

再定义一个反复调用 `next_char()` 的函数：

```
def extend_text(model, text, n_chars=80, temperature=1):
    for _ in range(n_chars):
        text += next_char(model, text, temperature)
    return text
```

尝试低、中、高三种 temperature：

```
>>> print(extend_text(model, "To be or not to b", temperature=0.01))
To be or not to be the state
and the contrary of the state and the sea,
the common people of the
>>> print(extend_text(model, "To be or not to b", temperature=0.4))
To be or not to be the better from the cause
that thou think you may be so be gone.
romeo:
that
>>> print(extend_text(model, "To be or not to b", temperature=100))
To be or not to b-c3;m-rkn&:x:uyve:b&hi n;n-h;wt3k
&cixxh:a!kq$c$ 3 ncq$ ;;wq cp:!xq;yh
!3
d!nhi.
```

低 temperature 下会出现重复；中等 temperature 的结果更可信；temperature 太高则会生成乱码。更好的文本可以使用 top-k 或 top-p（nucleus）sampling，也可以尝试 Beam Search、更多 **nn.GRU** 层、更多神经元、更长训练和更多正则化。

> 注意：当前模型无法学习长于 **window_length** 的模式，这里就是 50 个字符。增大窗口会让训练更难；即使 LSTM 和 GRU 也无法处理特别长的序列。[a]
>
> [a]捕获更长模式的另一个技术是使用有状态 RNN。本章笔记中包含了相应小节。

虽然字符 RNN 只训练来预测下一个字符，但它也必须学习更高层次的任务。例如，要预测 **Great movie, I really** 后面的字符，理解句子是正面的会让“l”（loved 的首字母）比“h”（hated 的首字母）更可能出现。Alec Radford 等人在 2017 年的一篇论文[5]中发现，一个没有标签训练的大型字符 RNN 模型中有一个神经元成为出色的情感分析分类器，这预示并推动了 NLP 中的无监督预训练。

14.4　使用 Hugging Face 库进行情感分析（Sentiment Analysis Using Hugging Face Libraries）

NLP 最常见的应用之一是文本分类，尤其是情感分析。如果 MNIST 图像分类是计算机视觉的“Hello, world!”，那么 IMDb 影评数据集上的情感分析就是 NLP 的“Hello, world!”。IMDb 包含 50,000 条英文电影评论，其中 25,000 条用于训练，25,000 条用于测试；每条评论有一个表示负面（0）或正面（1）的二元标签。

使用 Hugging Face Datasets 库下载数据集，并把原始训练集的 80% 用于训练、20% 用于验证：

[5]见本章参考文献第 7 条。

```
from datasets import load_dataset
imdb_dataset = load_dataset("imdb")
split = imdb_dataset["train"].train_test_split(train_size=0.8, seed=42)
imdb_train_set, imdb_valid_set = split["train"], split["test"]
imdb_test_set = imdb_dataset["test"]
```

查看两条评论：

```
>>> imdb_train_set[1]["text"]
"'The Rookie' was a wonderful movie about the second chances life holds [...]"
>>> imdb_train_set[1]["label"]
1
>>> imdb_train_set[16]["text"]
"Lillian Hellman's play, adapted by Dashiell Hammett with help from Hellman,
becomes a curious project to come out of gritty Warner Bros. [...] It seems to
take forever for this drama to find its focus, [...], it seems a little
patronizing [...] Lukas has several speeches in the third-act which undoubtedly
won him the Academy Award [...] this tasteful, tactful movie [...] It should
be a heady mix, but instead it's rather dry-eyed and inert. ** from ****"
>>> imdb_train_set[16]["label"]
0
```

第一条评论显然是正面评论（label = 1），第二条夹杂正负面信息，但结尾明确偏负面（label = 0）。简单的字符 RNN 难以处理这一任务，需要更强的 tokenization 技术。

14.5　使用 Hugging Face Tokenizers 库进行分词（Tokenization Using the Hugging Face Tokenizers Library）

2016 年，爱丁堡大学的 Rico Sennrich 等人研究了多种子词级分词和反分词方法。即使模型遇到训练时没有见过的罕见单词，只要学会了相关子词，也能合理猜测其含义。其中一种技术是字节对编码（byte pair encoding，BPE），由 Philip Gage 于 1994 年提出，最初用于数据压缩。BPE 先把训练集拆成单个字符，每次迭代寻找最常见的相邻 token 对并加入词汇表，直到达到目标大小。[6]

Hugging Face Tokenizers 库包含多种高效分词算法的实现。下面在 IMDb 数据集上训练 BPE 模型：

[6] 见本章参考文献第 8 条。

```
import tokenizers
bpe_model = tokenizers.models.BPE(unk_token="<unk>")
bpe_tokenizer = tokenizers.Tokenizer(bpe_model)
bpe_tokenizer.pre_tokenizer = tokenizers.pre_tokenizers.Whitespace()
special_tokens = ["<pad>", "<unk>"]
bpe_trainer = tokenizers.trainers.BpeTrainer(vocab_size=1000,
                                             special_tokens=special_tokens)
train_reviews = [review["text"].lower() for review in imdb_train_set]
bpe_tokenizer.train_from_iterator(train_reviews, bpe_trainer)
```

这里指定未知 token **"<unk>"**，用 **Whitespace** 预处理器在空格处分割文本并分离字母组与非字母组；例如**"Hello, world!!!"** 会被拆为 **["Hello", ",", "world", "!!!"]**。然后定义填充 token **"<pad>"** 和未知 token，创建 **BpeTrainer**，最后在所有小写训练评论上训练 tokenizer。特殊 token 位于词汇表开头，因此 **<pad>** 的 ID 为 0，**<unk>** 的 ID 为 1。

用它对文本分词：

```
>>> some_review = "what an awesome movie!"
>>> bpe_encoding = bpe_tokenizer.encode(some_review)
>>> bpe_encoding
Encoding(num_tokens=8, attributes=[ids, type_ids, tokens, offsets,
         attention_mask, special_tokens_mask, overflowing])
>>> bpe_encoding.tokens
['what', 'an', 'aw', 'es', 'ome', 'movie', '!', '<unk>']
>>> bpe_token_ids = bpe_encoding.ids
>>> bpe_token_ids
[303, 139, 373, 149, 240, 211, 4, 1]
```

高频单词会成为单个 token，而 **awesome** 被拆为多个 token；不在训练数据中的 emoji 会被替换为**"<unk>"**。tokenizer 还提供 **get_vocab()**、**token_to_id()**、**id_to_token()** 和 **decode()**：

```
>>> bpe_tokenizer.decode(bpe_token_ids)
'what an aw es ome movie !'
>>> bpe_encoding.offsets
[(0, 4), (5, 7), (8, 10), (10, 12), (12, 15), (16, 21), (21, 22), (23, 24)]
```

也可以批量编码。较短评论可以用 padding token 填充，过长序列则可以截断：

```
bpe_tokenizer.enable_padding(pad_id=0, pad_token="<pad>")
bpe_tokenizer.enable_truncation(max_length=500)
>>> bpe_encodings = bpe_tokenizer.encode_batch(train_reviews[:3])
>>> bpe_batch_ids = torch.tensor([encoding.ids for encoding in bpe_encodings])
>>> bpe_batch_ids
tensor([[159, 402, 176, 246,  61, [...], 215, 156, 586,   0,   0,   0,   0],
        [ 10, 138, 198, 289, 175, [...],   0,   0,   0,   0,   0,   0,   0],
        [289,  15, 209, 398, 177, [...],  50,  29,  22,  17,  24,  18,  24]])
```

attention_mask 中非填充 token 为 1、填充 token 为 0；求和即可得到序列长度：

```
>>> attention_mask = torch.tensor([encoding.attention_mask
...                                for encoding in bpe_encodings])
>>> lengths = attention_mask.sum(dim=-1)
>>> lengths
tensor([281, 114, 285])
```

Whitespace 会丢弃空格，因此解码时无法恢复原始空格。**ByteLevel** 预 tokenizer 用特殊字符 **Ġ** 表示空格，因而不会丢失空格信息。Byte-level BPE（BBPE）进一步让 BPE 在字节级运行；只要词汇表含有全部 256 个字节，它就能处理任意字符，特别适合包含罕见字符或 emoji 的语料库。

另一个重要变体是 WordPiece。它与 BPE 类似，但每次加入得分最高的相邻 token 对，而不是出现频率最高的 token 对。公式 14-1 中的分母会降低单个 token 本身很常见的 token 对得分，使结果通常比 BPE 或 BBPE 产生更短、更有意义的序列。

$$\text{score}(AB) = \frac{\text{frequency}(AB)}{\text{freq}(A)\ \text{freq}(B)} \times \text{len(vocab)} \tag{14-1}$$

公式 14-1　由 token A 和 B 组成的 token 对 AB 的 WordPiece 得分（WordPiece score for a pair AB composed of tokens A and B）

训练 WordPiece tokenizer 时，只需把 BPE 模型替换为 **WordPiece**，把 **BpeTrainer** 替换为 **WordPieceTrainer**。WordPiece 会给单词内部的 token 添加前缀，删除 **#** 以及标点前空格即可重建原始字符串。emoji 仍会因为不在词汇表中而消失。[7]

Unigram LM（Language Model）由 Google 的 Taku Kudo 于 2018 年提出。它从一个包含所有高频单词、子词和字符的大词汇表开始，然后逐渐删除最无用的 token，直到达到目标大小。它假设语料库中的 token 是独立采样的，因此 token 对 AB 的概率等于 A 的概率乘以 B 的概率；每次删除 token 时，都尽量少降低训练语料库的总体概率。[8]

[7] WordPiece 由 Google 在 2016 年提出，见本章参考文献第 9 条。

[8] 见本章参考文献第 10 条。

例如，**them**、**the** 和 **m** 的概率分别为 1%、5% 和 2%。**them** 作为单个 token 的概率为 1%，作为 **the** + **m** 的概率为 0.1%，总概率为 1.1%。删除 **them** 会让概率降至 0.1%，而删除 **the** 或 **m** 只会让概率降至 1%，因此算法更倾向于删除后两者之一。

> 提示：Unigram LM 适合中文等不使用空格分隔词语的语言，也适合词内使用空格的越南语，以及常把多个词连接起来的德语。

同一篇论文还提出了子词正则化（subword regularization），在训练 NLP 模型时随机改变分词，从而提升泛化能力和鲁棒性。Tokenizers 库原生不支持它，可以自行实现，或使用 Google 的 SentencePiece 库（**pip install sentencepiece**）。[9]

表 14-1: 三种主要 tokenizer 概览

特性	BBPE	WordPiece	Unigram LM
合并方式	合并最常见的 token 对	合并使数据似然最大的 token 对	删除可能性最低的 token
优点	快速、简单，适合多语言	效率和 token 质量之间平衡良好	语义最有意义，序列最短
缺点	可能产生不自然的切分	多语言场景不如 BBPE 稳健	训练和 tokenization 更慢
使用者	GPT、Llama、RoBERTa、BLOOM	BERT、DistilBERT、ELECTRA	T5、ALBERT、mBART

14.6　复用预训练 tokenizer（Reusing Pretrained Tokenizers）

在许多场景下，自己训练 tokenizer 很有用，例如处理充满专业术语的医学、法律或工程文本，处理低资源语言或方言，或者处理用新编程语言编写的代码。不过，大多数情况下，直接复用预训练 tokenizer 更方便。

下载预训练 tokenizer 时使用 Hugging Face Transformers 库。该库提供 NLP、计算机视觉、音频处理等领域的流行模型，并能从 Hugging Face Hub 自动下载预训练权重。模型最初几乎都基于 Transformer 架构（第 15 章将详细讨论），因此得名 Transformers；如今也提供 CNN 等其他模型。每个模型还附带完成任务所需的工具，包括 NLP 模型的 tokenizer。通常只需一行代码，就能得到特定任务的高性能模型。

[9] SentencePiece 的论文见本章参考文献第 11 条。

先使用 GPT-2 的预训练 BBPE tokenizer 对 IMDb 评论进行编码；超过 500 个 token 的序列会被截断：

```
import transformers
gpt2_tokenizer = transformers.AutoTokenizer.from_pretrained("gpt2")
gpt2_encoding = gpt2_tokenizer(train_reviews[:3], truncation=True,
                               max_length=500)
```

tokenizer 对象可以像函数一样调用，返回一个类似字典的 **BatchEncoding** 对象。通过**"input_ids"** 键可以获取 token ID。下面查看第一条评论的前 10 个 token ID，并使用 **decode()** 解码：

```
>>> gpt2_token_ids = gpt2_encoding["input_ids"][0][:10]
>>> gpt2_token_ids
[14247, 35030, 1690, 423, 257, 1688, 8046, 13, 484, 1690]
>>> gpt2_tokenizer.decode(gpt2_token_ids)
'stage adaptations often have a major fault. they often'
```

如果希望使用预训练 WordPiece tokenizer，可以复用 BERT 等模型的 tokenizer。BERT 的 tokenizer 自带 padding token，因此编码评论批次时可以指定 padding，并请求直接返回 PyTorch 张量：

```
bert_tokenizer = transformers.AutoTokenizer.from_pretrained("bert-base-uncased")
bert_encoding = bert_tokenizer(train_reviews[:3], padding=True,
                               truncation=True, max_length=500,
                               return_tensors="pt")
```

> 提示：**bert-base-uncased** 表示一个对大小写不敏感、在英文文本上预训练的 BERT 模型检查点。还可以使用 **bert-large-cased**，或使用在 100 多种语言上预训练的 **bert-base-multilingual-uncased**。这里暂时只使用它的 tokenizer。

编码结果是带有 padding 的张量：

```
>>> bert_encoding["input_ids"]
tensor([[ 101, 2754,17241, 2411, 2031, [...],  102, 0, 0, 0, [...], 0, 0, 0],
        [ 101, 1005, 1996, 8305, 1005, [...],  102, 0, 0, 0, [...], 0, 0, 0],
        [ 101, 7929, 1010, 2021, 2515, [...], 1012,  1019,  1019,  102]])
>>> bert_encoding["attention_mask"]
tensor([[1, 1, 1, 1, 1, 1, 1, 1, [...], 0, 0, 0, 0, 0, 0],
        [1, 1, 1, 1, 1, 1, 1, 1, [...], 0, 0, 0, 0, 0, 0],
        [1, 1, 1, 1, 1, 1, 1, 1, [...], 1, 1, 1, 1, 1, 1]])
```

每个 token ID 序列（忽略 padding）都以 101（`[CLS]`）开头，以 102（`[SEP]`）结尾。这些 token 是 BERT 所需的；如果自己的模型不需要它们，可以在调用 tokenizer 时设置 `add_special_tokens=False`。

许多模型使用 Unigram LM tokenizer，例如 ALBERT、T5 和 XLM-R：

```
albert_tokenizer = transformers.AutoTokenizer.from_pretrained("albert-base-v2")
albert_encoding = albert_tokenizer(train_reviews[:3], padding=True, [...])
```

Transformers 还提供对象来包装自己用 Tokenizers 库训练的 tokenizer，并赋予它与 Transformers 预训练 tokenizer 相同的 API：

```
hf_tokenizer = transformers.PreTrainedTokenizerFast(
    tokenizer_object=bpe_tokenizer)
hf_encodings = hf_tokenizer(train_reviews[:3], padding=True, [...])
```

现在已经具备分词所需的工具，回到情感分析。

14.7　构建和训练情感分析模型（Building and Training a Sentiment Analysis Model）

情感分析模型必须使用分词后的评论批次训练。前面创建的数据集没有处理分词，一种选择是使用 `map()` 更新数据集；不过，在数据加载器中处理分词同样简单。把函数传给 `DataLoader` 构造函数的 `collate_fn` 参数，数据加载器就会为每个批次调用它，并传入一个数据集样本列表。函数负责分词、截断、填充，并返回包含 token ID、attention mask 和标签的 PyTorch 张量：

```
def collate_fn(batch, tokenizer=bert_tokenizer):
    reviews = [review["text"] for review in batch]
    labels = [[review["label"]] for review in batch]
    encodings = tokenizer(reviews, padding=True, truncation=True,
                          max_length=200, return_tensors="pt")
    labels = torch.tensor(labels, dtype=torch.float32)
    return encodings, labels
batch_size = 256
imdb_train_loader = DataLoader(imdb_train_set, batch_size=batch_size,
                               collate_fn=collate_fn, shuffle=True)
imdb_valid_loader = DataLoader(imdb_valid_set, batch_size=batch_size,
                               collate_fn=collate_fn)
imdb_test_loader = DataLoader(imdb_test_set, batch_size=batch_size,
                              collate_fn=collate_fn)
```

现在构建模型：

```
class SentimentAnalysisModel(nn.Module):
    def __init__(self, vocab_size, n_layers=2, embed_dim=128, hidden_dim=64,
                 pad_id=0, dropout=0.2):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim,
                                  padding_idx=pad_id)
        self.gru = nn.GRU(embed_dim, hidden_dim, num_layers=n_layers,
                          batch_first=True, dropout=dropout)
        self.output = nn.Linear(hidden_dim, 1)
    def forward(self, encodings):
        embeddings = self.embed(encodings["input_ids"])
        _outputs, hidden_states = self.gru(embeddings)
        return self.output(hidden_states[-1])
```

与莎士比亚模型相比，这里有几个重要区别：

- **`nn.Embedding`** 的 **`padding_idx`** 设置为 padding ID，使 padding ID 的嵌入成为不可训练的零向量，减少 padding token 对损失的影响。
- 这是序列到向量模型，而不是序列到序列模型，因此只需要顶层 GRU 的最后输出（**`hidden_states[-1]`**）进行最终预测；它等价于 **`outputs[:, -1]`**。
- 输出 **`nn.Linear`** 只有一个维度，因为这是二元分类模型。对每条评论，它输出一个 logit，正值表示正面，负值表示负面。
- **`forward()`** 接收包含 token ID（可能已填充和截断）的 **`BatchEncoding`** 对象。

可以使用 **nn.BCEWithLogitsLoss** 训练模型。验证集准确率接近 85%，相当不错，但最佳模型能达到略高于 90% 的人类水平。由于许多评论本身含义模糊，进一步提高会比较困难。

模型的一个问题是没有完全忽略 padding token。若评论结尾有很多 padding token，**nn.GRU** 就必须处理它们，处理完后可能已经忘记评论内容。可以使用 packed sequence 代替普通张量。packed sequence 是专门用于高效表示不同长度序列批次的数据结构。[10]使用 **pack_padded_sequence()** 和 **pad_packed_sequence()** 可以在填充张量和 packed sequence 之间转换：

```
>>> from torch.nn.utils.rnn import pack_padded_sequence, pad_packed_sequence
>>> sequences = torch.tensor([[1, 2, 0, 0], [5, 6, 7, 8]])
>>> packed = pack_padded_sequence(sequences, lengths=(2, 4),
...                               enforce_sorted=False, batch_first=True)
...
>>> packed
PackedSequence(data=tensor([5, 1, 6, 2, 7, 8]), [...])
>>> padded, lengths = pad_packed_sequence(packed, batch_first=True)
>>> padded, lengths
(tensor([[1, 2, 0, 0],
         [5, 6, 7, 8]]),
 tensor([2, 4]))
```

默认情况下，**pack_padded_sequence()** 假设批次序列按长度从长到短排序；否则必须设置 **enforce_sorted=False**。它还假设 time 维位于 batch 维之前；如果 batch 维在前，则必须设置 **batch_first=True**。

PyTorch 的循环层支持 packed sequence，会高效地处理序列，并在每个序列结束时停止。因此只需在 **forward()** 中替换 **self.gru(embeddings)**：

[10]Nested tensor 也有类似用途，而且更方便，但在本书写作时仍处于原型阶段；参见 https://pytorch.org/docs/stable/nested.html。

```
lengths = encodings["attention_mask"].sum(dim=1)
packed = pack_padded_sequence(embeddings, lengths=lengths.cpu(),
                              batch_first=True, enforce_sorted=False)
_outputs, hidden_states = self.gru(packed)
```

代码先根据 attention mask 计算批次中每个序列的长度，再打包嵌入张量并传给 **nn.GRU**。这样模型就不会受 padding token 干扰。此时 **padding_idx** 已不是必需的，但保留它有助于调试。

14.8　双向 RNN（Bidirectional RNNs）

普通循环层在生成输出时只能看到过去和当前输入，具有因果性，不能查看未来。这适合时间序列预测或序列到序列模型的解码器；但对文本分类或序列到序列模型的编码器而言，查看当前单词后面的单词通常更好。例如，要正确编码 **the right arm**、**the right person** 和 **the right to speak** 中的 **right**，就需要查看后文。

一种解决方案是在同一组输入上运行两个循环层：一个从左到右读取单词，另一个从右到左读取单词，再在每个时间步组合两个输出，通常是拼接。这就是双向循环层（见图 14-4）。

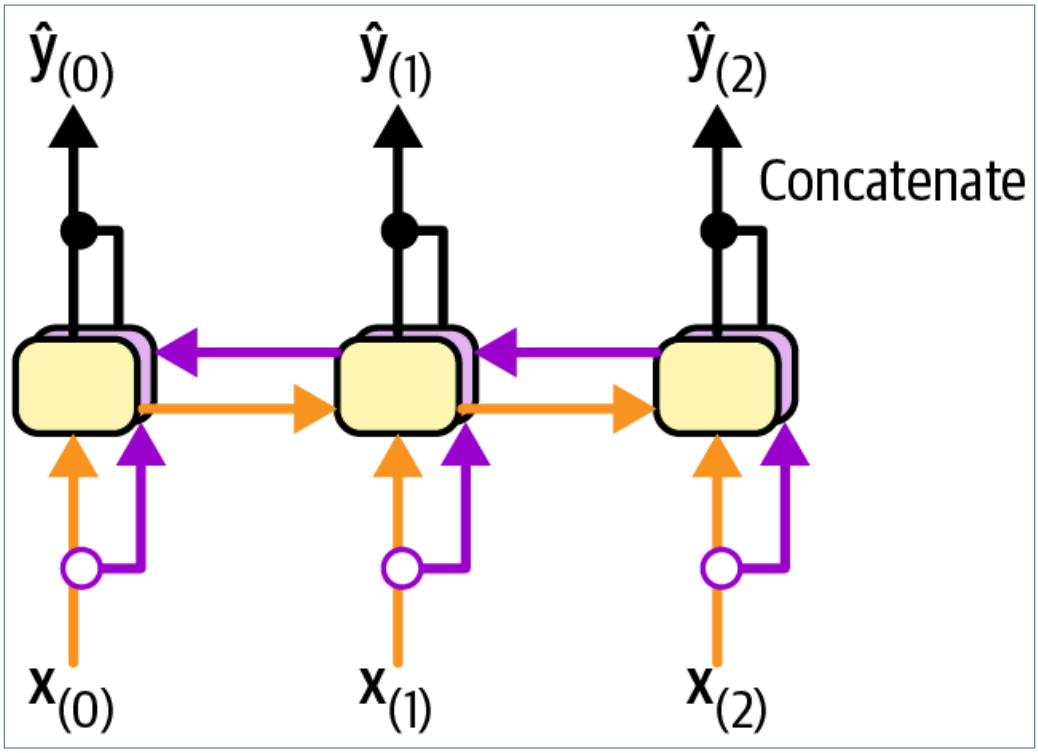

图 14-4: 双向循环层（A bidirectional recurrent layer）

要让情感分析模型双向运行，只需在创建 `nn.GRU` 时设置 `bidirectional=True`。不过需要相应调整模型：隐藏状态会加倍，因此输出层的输入维度也必须加倍：

```
self.output = nn.Linear(2 * hidden_dim, 1)
```

还必须在传给输出层之前拼接顶层 GRU 的正向和反向隐藏状态，用下面的代码替换 `return self.output(hidden_states[-1])`：

```
n_dims = self.output.in_features
top_states = hidden_states[-2:].permute(1, 0, 2).reshape(-1, n_dims)
return self.output(top_states)
```

之前 `nn.GRU` 返回的隐藏状态形状是 `[number of layers, batch size, hidden size]`，本例为 `[2, 256, 64]`。设置双向后，第一维加倍，变成 `[4, 256, 64]`，依次包含第 1 层正向、反向以及第 2 层正向、反向隐藏状态。只需要顶层的正向和反向状态，因此取 `hidden_states[-2:]`。然后用 `permute(1, 0, 2)` 得到 `[256, 2, 64]`，再用 `reshape(-1, n_dims)` 得到 `[256, 2 * 64]`。

> 注意：这里仅使用最后的隐藏状态，忽略每个时间步的输出。如果要使用双向模块的输出，要注意最后一维大小会加倍。

这个模型在验证集上不会带来提升，因为第一个模型已经过拟合；新模型训练集准确率超过 99%，验证集却只有 84%。可以增加正则化、减小模型规模或增大训练集。不过，接下来尝试复用预训练嵌入。

14.9　复用预训练嵌入和语言模型（Reusing Pretrained Embeddings and Language Models）

模型仅根据 25,000 条影评就学会了数千个 token 的有用嵌入，已经相当 impressive。若在数十亿条评论上训练嵌入，效果会怎样？好消息是，即使预训练嵌入来自其他大型文本语料库、不是电影评论、也没有针对情感分析训练，仍然可以复用它们。毕竟，“amazing”在描述电影或其他事物时通常含义相同。

既然已经使用了 BERT token，不妨复用它的嵌入层。先用 `AutoModel.from_pretrained()` 下载模型，然后直接访问嵌入层：

```
>>> bert_model = transformers.AutoModel.from_pretrained("bert-base-uncased")
>>> bert_model.embeddings.word_embeddings
Embedding(30522, 768, padding_idx=0)
```

BERT 用 PyTorch 实现，包含普通的 **nn.Embedding** 层。可以直接替换自己模型的嵌入层，也可以用预训练嵌入矩阵的副本初始化自己的层：

```
class SentimentAnalysisModelPreEmbeds(nn.Module):
    def __init__(self, pretrained_embeddings, n_layers=2, hidden_dim=64,
                 dropout=0.2):
        super().__init__()
        weights = pretrained_embeddings.weight.data
        self.embed = nn.Embedding.from_pretrained(weights, freeze=True)
        embed_dim = weights.shape[-1]
        [...]  # the rest of the model is exactly like earlier
imdb_model_bert_embeds = SentimentAnalysisModelPreEmbeds(
    bert_model.embeddings.word_embeddings).to(device)
```

这里设置 **freeze=True**，使嵌入层不可训练，避免训练初期的大梯度破坏预训练嵌入。训练几个 epoch 后可以解冻嵌入层，继续针对当前任务微调。

预训练词嵌入已经流行了一段时间，包括 Google 的 Word2Vec（2013）、Stanford 的 GloVe（2014）和 Facebook 的 FastText（2016）。不过它们有一个局限：无论上下文如何，同一个词都只有一种表示。例如 **right** 在 **left and right** 和 **right and wrong** 中的含义不同，却会被同样编码。为了解决这个问题，Matthew Peters 等人在 2018 年提出了 ELMo（Embeddings from Language Models），即从深度双向 RNN 语言模型内部状态学习的上下文化词嵌入。换言之，不仅可以复用预训练词嵌入，还可以复用预训练语言模型的多个层。

大约在同一时期，ULMFiT（Universal Language Model Fine-Tuning）论文展示了无监督预训练对 NLP 任务的效果。研究者先用自监督学习在大规模文本语料库上训练 LSTM 语言模型，再针对各种任务微调。模型在六个文本分类任务上大幅超过当时最先进方法。ULMFiT 开启了 NLP 的新时代：如今复用预训练语言模型已经成为常规做法。

为什么不直接复用完整的预训练 BERT 模型？可以把分词后的评论传给它：

```
>>> bert_encoding = bert_tokenizer(train_reviews[:3], padding=True,
...                                max_length=200, truncation=True,
...                                return_tensors="pt")
...
>>> bert_output = bert_model(**bert_encoding)
>>> bert_output.last_hidden_state.shape
torch.Size([3, 200, 768])
```

BERT 的输出含有 **last_hidden_state** 属性，其中包含每个 token 的上下文化嵌入。这里的“last”指最后一层而不是最后时间步，因为 BERT 是 Transformer 而非 RNN。张量形状为 **[batch size, max sequence length, hidden size]**。可以把它用于情感分析模型：

```
class SentimentAnalysisModelBert(nn.Module):
    def __init__(self, n_layers=2, hidden_dim=64, dropout=0.2):
        super().__init__()
        self.bert = transformers.AutoModel.from_pretrained("bert-base-uncased")
        embed_dim = self.bert.config.hidden_size
        self.gru = nn.GRU(embed_dim, hidden_dim, [...])
        self.output = nn.Linear(hidden_dim, 1)
    def forward(self, encodings):
        contextualized_embeddings = self.bert(**encodings).last_hidden_state
        lengths = encodings["attention_mask"].sum(dim=1)
        packed = pack_padded_sequence(contextualized_embeddings, [...])
        _outputs, hidden_states = self.gru(packed)
        return self.output(hidden_states[-1])
```

上下文化嵌入已经向前看过，因此不需要把 GRU 设为双向。如果冻结 BERT（例如 **model.bert.requires_grad_(False)**）并训练其余模型，准确率可以提升到 88% 以上。

还可以只使用第一个 token（**[CLS]**）的上下文化嵌入。在预训练期间，BERT 曾经仅根据这个 token 的表示完成文本分类任务，因此学会了把文本最重要的特征汇总到该嵌入中。这样可以完全去掉 GRU：

```
def forward(self, encodings):
    bert_output = self.bert(**encodings)
    return self.output(bert_output.last_hidden_state[:, 0])
```

BERT 还在分类嵌入之上包含一个由 `nn.Linear` 和 `nn.Tanh` 组成的隐藏层，称为 pooler。可以把 `bert_output.last_hidden_state[:, 0]` 换成 `bert_output.pooler_output`；训练若干 epoch 后也可以解冻 pooler 做微调。

14.10　任务专用类（Task-Specific Classes）

还可以更进一步，直接使用 Transformers 库提供的 `BertForSequenceClassification` 类完成二元分类。它就是一个 BERT 模型加上分类头。只需指定预训练检查点、分类任务的输出单元数，以及可选的数据类型；这里使用 16 位浮点数，以便在小型 GPU 上运行：

```
from transformers import BertForSequenceClassification
torch.manual_seed(42)
bert_for_binary_clf = BertForSequenceClassification.from_pretrained(
    "bert-base-uncased", num_labels=2, dtype=torch.float16).to(device)
```

> 提示：Transformers 库还提供许多基于预训练模型的任务专用类，例如 `BertForQuestionAnswering` 和 `RobertaForSequenceClassification`。也可以使用 `AutoModelForSequenceClassification`，让库根据请求的模型检查点自动选择正确的类。

前面二元分类一直使用单个输出，为什么这里设置 `num_labels=2`？Hugging Face 更倾向于把二元分类完全当作多分类处理，因此模型会输出两个 logits，并使用 `nn.CrossEntropyLoss`，而不是 `nn.BCELoss` 或 `nn.BCEWithLogitsLoss`。要把 logits 转换为概率，必须使用 `torch.softmax()` 而不是 `torch.sigmoid()`。

在一条非常正面的评论上调用模型：

```
>>> encoding = bert_tokenizer(["This was a great movie!"])
>>> with torch.no_grad():
...   output = bert_for_binary_clf(
...     input_ids=torch.tensor(encoding["input_ids"], device=device),
...     attention_mask=torch.tensor(encoding["attention_mask"], device=device))
...
>>> output.logits
tensor([[-0.0120,  0.6304]], device='cuda:0', dtype=torch.float16)
>>> torch.softmax(output.logits, dim=-1)
tensor([[0.3447, 0.6553]], device='cuda:0', dtype=torch.float16)
```

模型把这条评论判为负面，置信度为 65.53%——结果并不理想。**BertForSequenceClassification** 内的 BERT 已经预训练，但分类头还没有训练，因此在 IMDb 数据集上训练之前效果会很差。

如果调用模型时传入 labels，它还会计算损失并把损失放入 **ModelOutput**：

```
>>> with torch.no_grad():
...   output = bert_for_binary_clf(
...     input_ids=torch.tensor(encoding["input_ids"], device=device),
...     attention_mask=torch.tensor(encoding["attention_mask"], device=device),
...     labels=torch.tensor([1], device=device))
...
>>> output.loss
tensor(0.4226, device='cuda:0', dtype=torch.float16)
```

由于 **num_labels** 大于 1，模型计算 **nn.CrossEntropyLoss**；如果设置 **num_labels=1**，则会使用 **nn.MSELoss**，这对回归任务有用。

14.11　Trainer API

Trainer API 可以用很少的样板代码在自己的数据集上微调模型。它可以保存检查点、早停、跨 GPU 分配计算、记录指标、处理 padding、批处理和打乱数据等。Trainer API 直接使用 dataset 对象而非数据加载器，但要求数据集包含 tokenized text 而不是原始字符串，因此先用数据集的 **map()** 方法分词：

```
def tokenize_batch(batch):
    return bert_tokenizer(batch["text"], truncation=True, max_length=200)
tok_imdb_train_set = imdb_train_set.map(tokenize_batch, batched=True)
tok_imdb_valid_set = imdb_valid_set.map(tokenize_batch, batched=True)
tok_imdb_test_set = imdb_test_set.map(tokenize_batch, batched=True)
```

设置 **batched=True** 后，**map()** 会把评论批次传给 **tokenize_batch()**，这会显著加快预处理。得到的字段（包括 token ID 和 attention mask）会添加到每个实例中，模型可以直接使用。

先定义准确率函数：

```
def compute_accuracy(pred):
    return {"accuracy": (pred.label_ids == pred.predictions.argmax(-1)).mean()}
```

然后指定训练配置：

```
from transformers import TrainingArguments
train_args = TrainingArguments(
    output_dir="my_imdb_model", num_train_epochs=2,
    per_device_train_batch_size=128, per_device_eval_batch_size=128,
    eval_strategy="epoch", logging_strategy="epoch", save_strategy="epoch",
    load_best_model_at_end=True, metric_for_best_model="accuracy",
    report_to="none")
```

这里指定日志和模型检查点保存到 **my_imdb_model** 目录，训练 2 个 epoch；训练、评估、日志记录和保存都在每个 epoch 结束时执行；训练结束时根据验证集准确率加载最佳模型。**report_to** 可以指定 TensorBoard 或 Weights & Biases 等日志工具，这里用**"none"** 关闭报告。

最后创建 **Trainer**，传入模型、训练参数、训练集、验证集、评估函数和负责 padding 的数据整理器：

```
from transformers import DataCollatorWithPadding, Trainer
trainer = Trainer(
    bert_for_binary_clf, train_args, train_dataset=tok_imdb_train_set,
    eval_dataset=tok_imdb_valid_set, compute_metrics=compute_accuracy,
    data_collator=DataCollatorWithPadding(bert_tokenizer))
train_output = trainer.train()
```

很好，现在已经知道如何下载预训练 BERT 并在自己的数据集上微调它。如果没有数据集，只想使用一个已经针对情感分析微调好的预训练模型，可以使用 pipelines API。

14.12　Hugging Face Pipelines

Transformers 库提供了非常方便的 API，可以下载并使用各种任务的预训练 pipeline。每个 pipeline 都包含预训练模型以及相应的预处理和后处理模块。创建情感分析 pipeline，并在 IMDb 训练集前 10 条评论上运行：

```
>>> from transformers import pipeline
>>> model_name = "distilbert-base-uncased-finetuned-sst-2-english"
>>> classifier_imdb = pipeline("sentiment-analysis", model=model_name,
...                            truncation=True, max_length=512)
...
>>> classifier_imdb(train_reviews[:10])
[{'label': 'POSITIVE', 'score': 0.9996108412742615},
 {'label': 'POSITIVE', 'score': 0.9998623132705688},
 [...]
 {'label': 'POSITIVE', 'score': 0.9978922009468079},
 {'label': 'NEGATIVE', 'score': 0.9997020363807678}]
```

只需指定任务、模型和少数参数即可。每条评论都会得到`"POSITIVE"` 或`"NEGATIVE"`标签，以及该标签的估计概率分数。这里的模型在验证集上达到 88.2% 准确率。

需要注意：

- 如果不指定模型，`pipeline()` 会为所选任务使用默认模型。情感分析的默认模型是 DistilBERT，即缩小版 BERT，使用不区分大小写的 tokenizer，在英文 Wikipedia 和英文书籍语料上训练，再针对 Stanford Sentiment Treebank v2（SST-2）微调。
- 如果有 GPU，pipeline 会自动使用它；有多个 GPU 时，可以通过 `device` 参数指定索引。
- Transformers 模型默认处于评估模式，无需调用 `eval()`。
- score 是所选标签的分数，不是“正面”类别的分数。由于这是二元分类，分数不会低于 0.5，否则模型会选择另一个标签。

> **偏见与公平性**：如果分别分类 `I am from the USA` 和 `I am from Iraq`，前者会被判为非常正面，后者会被判为非常负面。模型对 Thailand 也偏正面，对 Vietnam 则偏负面。此类偏见很大程度来自训练数据：训练数据中大量提到伊拉克和越南战争，造成负面偏差；微调时模型被迫只在正负两类中选择，这一偏差又被放大。训练数据、模型架构、损失函数、正则化和优化器都可能影响最终偏见。部署模型前应检查不同人群子集上的表现，并进行反事实测试，例如只改变性别或出生地，预测是否就发生变化。即便模型本身公平，也可能被招聘者等以有偏方式使用。

所选模型适合电影评论等通用情感分析，但社交媒体帖子等特定场景可能需要其他模型。Hugging Face Hub 上“text-classification”类别单独就有超过 80,000 个模型，因此应按任务筛选，再按趋势或点赞数排序，也可继续按语言和数据集筛选。优先选择可信来源的模型；如果模型包含可执行代码，必须确认信任作者，并在确实信任时才在 `pipeline()` 中设置 `trust_remote_code=True`。

除情感分析外，pipeline API 还支持许多文本分类任务。例如，在 MultiNLI 数据集上微调的模型可以把两个文本分类为 contradiction、entailment 或 neutral：

```
>>> model_name = "huggingface/distilbert-base-uncased-finetuned-mnli"
>>> classifier_mnli = pipeline("text-classification", model=model_name)
>>> classifier_mnli([
...     "She loves me. [SEP] She loves me not. [SEP]",
...     "Alice just woke up. [SEP] Alice is awake. [SEP]",
...     "I like dogs. [SEP] Everyone likes dogs. [SEP]"])
...
[{'label': 'contradiction', 'score': 0.9717152714729309},
 {'label': 'entailment', 'score': 0.9119168519973755},
 {'label': 'neutral', 'score': 0.9509281516075134}]
```

pipeline 还支持问答、摘要、句子相似度、文本生成、token 分类、翻译等 NLP 任务，以及图像分类、图像分割、目标检测、图像转文本、文本转图像、深度估计等计算机视觉任务和音频分类、语音转文字、文字转语音等音频任务。完整列表参见 Hugging Face Tasks。

> 警告：下载模型前，要确认信任托管平台和模型作者。模型可能含有恶意可执行代码，生成有偏输出，使用受版权或隐私保护的数据训练，或者包含会生成有害内容的投毒权重。

14.13　用于神经机器翻译的编码器-解码器网络（An Encoder-Decoder Network for Neural Machine Translation）

到目前为止，我们讨论了字符 RNN、各种子词分词方法、预训练嵌入和完整预训练模型。下面进入另一个重要的 NLP 任务：神经机器翻译（NMT）。具体来说，将构建一个把英语翻译成西班牙语的编码器-解码器模型，并使用 Beam Search 和注意力机制提升其性能。

先构建一个相对简单的序列到序列 NMT 模型[11]，把英语文本翻译成西班牙语（见图 14-5）。

[11] 见本章参考文献第 15 条。

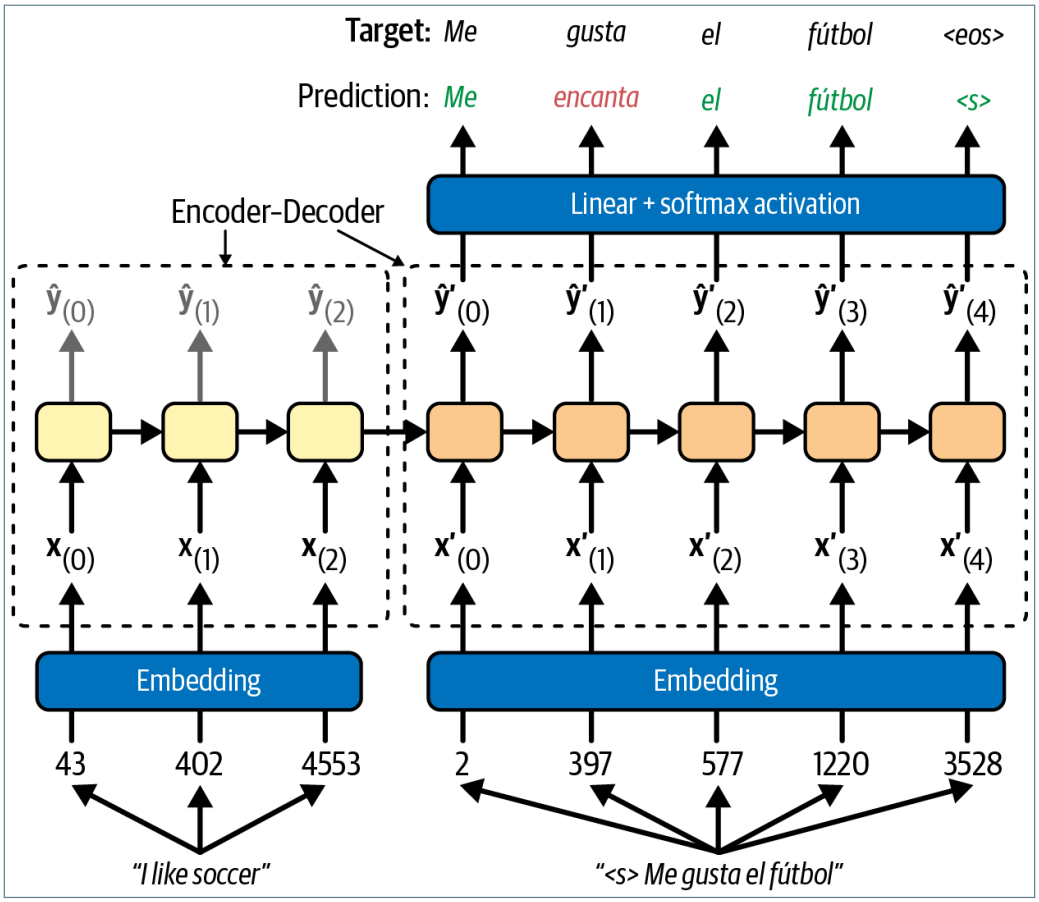

图 14-5: 一个简单的机器翻译模型（A simple machine translation model）

英语文本作为输入送入编码器，解码器输出西班牙语译文。训练时，西班牙语译文也作为解码器的输入，但向后移动一个时间步；训练时解码器接收到的是它在前一步应该输出的 token，而不论它实际上输出了什么。这称为教师强制（teacher forcing）。第一个 token 使用序列开始（start-of-sequence，SoS，也称 beginning-of-sequence，BoS）token（`<s>`），文本末尾则应输出序列结束（end-of-sequence，EoS）token（`</s>`）。

每个 token 首先用 ID 表示，然后由 `nn.Embedding` 层返回 token 嵌入，嵌入再分别送入编码器和解码器。每一步中，解码器的 `nn.Linear` 输出层为西班牙语词汇表中的每个 token 输出一个 logit；经过 softmax 后得到每个可能 token 的概率。这与普通分类任务类似，训练时使用 `nn.CrossEntropyLoss`。

推理时没有目标文本可供解码器使用；必须把它上一步刚刚输出的单词作为下一步输入，如图 14-6 所示。

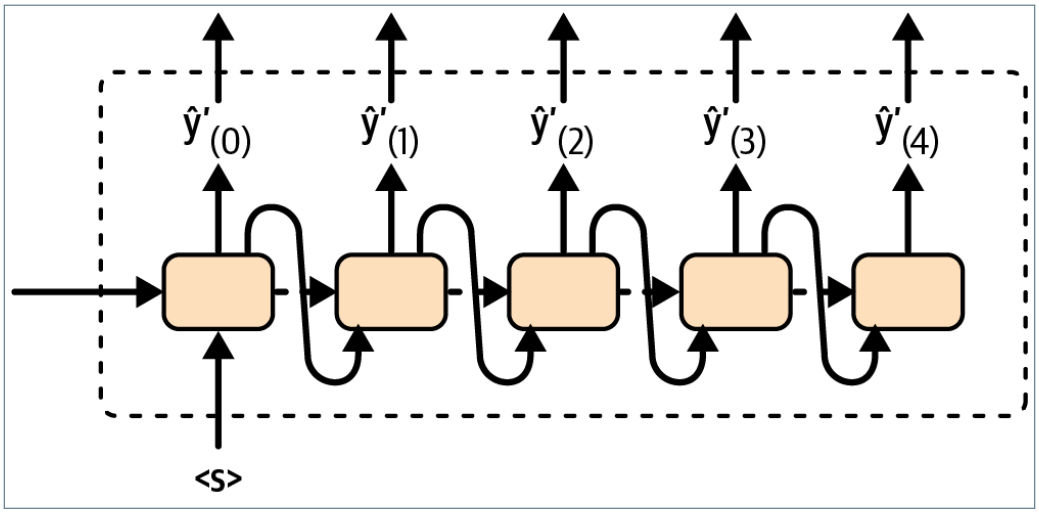

图 14-6: 推理时，解码器接收它在前一时间步刚刚输出的单词作为输入（At inference time, the decoder is fed as input the word it just output at the previous time step）

> 提示：Samy Bengio 等人在 2015 年的一篇论文中[a]提出，在训练过程中逐渐从输入前一个目标 token 切换为输入前一个输出 token。
>
> [a]见本章参考文献第 16 条。

下面从 Tatoeba Challenge 数据集下载英语/西班牙语文本对。Tatoeba 是一个语言学习项目，Tatoeba Challenge 数据集由赫尔辛基大学研究者从该项目中提取，用于评估机器翻译系统。训练集很大，因此把验证集作为训练来源，再划出 20% 做验证，同时下载测试集：

```
nmt_original_valid_set, nmt_test_set = load_dataset(
    path="ageron/tatoeba_mt_train", name="eng-spa",
    split=["validation", "test"])
split = nmt_original_valid_set.train_test_split(train_size=0.8, seed=42)
nmt_train_set, nmt_valid_set = split["train"], split["test"]
```

每个样本是包含英语文本及其西班牙语翻译的字典：

```
>>> nmt_train_set[0]
{'source_text': 'Tom tried to break up the fight.',
 'target_text': 'Tom trató de disolver la pelea.',
 'source_lang': 'eng',
 'target_lang': 'spa'}
```

英语和西班牙语有许多相同单词以及相似子词，因此使用一个共享 tokenizer 更合理。下面在全部英语和西班牙语训练文本上训练 BPE tokenizer：

```
def train_eng_spa():  # a generator function to iterate over all training text
    for pair in nmt_train_set:
        yield pair["source_text"]
        yield pair["target_text"]
max_length = 256
vocab_size = 10_000
nmt_tokenizer_model = tokenizers.models.BPE(unk_token="<unk>")
nmt_tokenizer = tokenizers.Tokenizer(nmt_tokenizer_model)
nmt_tokenizer.enable_padding(pad_id=0, pad_token="<pad>")
nmt_tokenizer.enable_truncation(max_length=max_length)
nmt_tokenizer.pre_tokenizer = tokenizers.pre_tokenizers.Whitespace()
nmt_tokenizer_trainer = tokenizers.trainers.BpeTrainer(
    vocab_size=vocab_size, special_tokens=["<pad>", "<unk>", "<s>", "</s>"])
nmt_tokenizer.train_from_iterator(train_eng_spa(), nmt_tokenizer_trainer)
```

```
>>> nmt_tokenizer.encode("I like soccer").ids
[43, 401, 4381]
>>> nmt_tokenizer.encode("<s> Me gusta el fútbol").ids
[2, 396, 582, 219, 3356]
```

创建一个保存英语 source token ID、西班牙语 target token ID 及其 attention mask 的工具类：

```
from collections import namedtuple
fields = ["src_token_ids", "src_mask", "tgt_token_ids", "tgt_mask"]
class NmtPair(namedtuple("NmtPairBase", fields)):
    def to(self, device):
        return NmtPair(self.src_token_ids.to(device), self.src_mask.to(device),
                       self.tgt_token_ids.to(device), self.tgt_mask.to(device))
```

创建数据加载器：

```
def nmt_collate_fn(batch):
    src_texts = [pair['source_text'] for pair in batch]
    tgt_texts = [f"<s> {pair['target_text']} </s>" for pair in batch]
    src_encodings = nmt_tokenizer.encode_batch(src_texts)
    tgt_encodings = nmt_tokenizer.encode_batch(tgt_texts)
    src_token_ids = torch.tensor([enc.ids for enc in src_encodings])
    tgt_token_ids = torch.tensor([enc.ids for enc in tgt_encodings])
    src_mask = torch.tensor([enc.attention_mask for enc in src_encodings])
    tgt_mask = torch.tensor([enc.attention_mask for enc in tgt_encodings])
    inputs = NmtPair(src_token_ids, src_mask,
                     tgt_token_ids[:, :-1], tgt_mask[:, :-1])
    labels = tgt_token_ids[:, 1:]
    return inputs, labels
batch_size = 32
```

```
nmt_train_loader = DataLoader(nmt_train_set, batch_size=batch_size,
                              collate_fn=nmt_collate_fn, shuffle=True)
nmt_valid_loader = DataLoader(nmt_valid_set, batch_size=batch_size,
                              collate_fn=nmt_collate_fn)
nmt_test_loader = DataLoader(nmt_test_set, batch_size=batch_size,
                             collate_fn=nmt_collate_fn)
```

`nmt_collate_fn()` 提取文本，给每条西班牙语文本添加 SoS 和 EoS token，使用共享 BPE tokenizer 编码，再将结果转成张量并包装进 `NmtPair`。它丢弃解码器输入中的 EoS token，并丢弃解码器目标中的 SoS token，例如输入包含 `<s> Me gusta el fútbol`，目标包含 `Me gusta el fútbol </s>`。

现在构建模型。它与图 14-5 相同，但编码器和解码器共享 `nn.Embedding`，且各自的 `nn.GRU` 都包含两层：

```
class NmtModel(nn.Module):
    def __init__(self, vocab_size, embed_dim=512, pad_id=0, hidden_dim=512,
                 n_layers=2):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim, padding_idx=pad_id)
        self.encoder = nn.GRU(embed_dim, hidden_dim, num_layers=n_layers,
                              batch_first=True)
        self.decoder = nn.GRU(embed_dim, hidden_dim, num_layers=n_layers,
                              batch_first=True)
        self.output = nn.Linear(hidden_dim, vocab_size)
    def forward(self, pair):
        src_embeddings = self.embed(pair.src_token_ids)
        tgt_embeddings = self.embed(pair.tgt_token_ids)
        src_lengths = pair.src_mask.sum(dim=1)
        src_packed = pack_padded_sequence(
            src_embeddings, lengths=src_lengths.cpu(),
            batch_first=True, enforce_sorted=False)
        _, hidden_states = self.encoder(src_packed)
        outputs, _ = self.decoder(tgt_embeddings, hidden_states)
        return self.output(outputs).permute(0, 2, 1)
torch.manual_seed(42)
vocab_size = nmt_tokenizer.get_vocab_size()
nmt_model = NmtModel(vocab_size).to(device)
```

这个模型的结构应该已经熟悉：`forward()` 嵌入英语和西班牙语序列，打包英语嵌入并送入编码器，再把西班牙语嵌入和编码器的最终隐藏状态送入解码器。解码器输出经过 `nn.Linear`，最后调换维度以满足 `nn.CrossEntropyLoss` 和 `Accuracy` 的要求。

> 注意：NMT 最常用的指标是 BLEU（bilingual evaluation understudy）分数，它把模型译文与多条人工译文比较，统计 n-gram 并按目标译文中的频率调整分数。TorchMetrics 的 **`BLEUScore`** 类实现了该指标。

可以不打包西班牙语嵌入；把损失函数配置为忽略目标中的 padding token：

```
xentropy = nn.CrossEntropyLoss(ignore_index=0)  # ignore <pad> tokens
```

例如使用 Nadam 和 **`lr = 0.001`** 训练 10 个 epoch。下面的辅助函数逐个 token 生成译文，直到输出 EoS：

```
def translate(model, src_text, max_length=20, pad_id=0, eos_id=3):
    tgt_text = ""
    token_ids = []
    for index in range(max_length):
        batch, _ = nmt_collate_fn([{"source_text": src_text,
                                    "target_text": tgt_text}])
        with torch.no_grad():
            Y_logits = model(batch.to(device))
            Y_token_ids = Y_logits.argmax(dim=1)
            next_token_id = Y_token_ids[0, index]  # take the last token ID
        next_token = nmt_tokenizer.id_to_token(next_token_id)
        tgt_text += " " + next_token
        if next_token_id == eos_id:
            break
    return tgt_text
```

这个实现可以运行，但还没有优化：编码器只需运行一次，解码器也只需在每个时间步运行一次，而不是每次都对不断增长的文本重新运行整个序列。

```
>>> nmt_model.eval()
>>> translate(nmt_model, "I like soccer.")
' Me gusta el fútbol . </s>'
```

14.13.1　模型优化（Model Optimizations）

当输出词汇表很大时，可以使用 sampled softmax；它不对所有 logits 计算 softmax，而只使用正确类别及随机采样的其他类别。它只适用于训练，PyTorch 没有内置实现。adaptive softmax 则按频率把词汇表分组，PyTorch 在 **`nn.AdaptiveLogSoftmaxWithLoss`** 中实现了它。

还可以把嵌入矩阵用作输出层权重，称为权重绑定（weight tying），由 Ofir Press 和 Lior Wolf 于 2016 年提出。设置 **`self.output.weight = self.embed.weight`**，并创建无 bias 的输出层即可；这会减少参数、加快训练，有时也会提升准确率。

短句翻译效果尚可，但长句很困难：

```
>>> longer_text = "I like to play soccer with my friends."
>>> translate(nmt_model, longer_text)
' Me gusta jugar con mis amigos . </s>'
```

译文没有提到 soccer。可以增大训练集、增加编码器和解码器中的 GRU 层，或让编码器双向；解码器不能双向，否则会看到完整翻译。推理时另一个常见技术是 Beam Search。

14.14　Beam Search

逐词翻译时，一旦模型犯错，后续翻译都会被错误束缚，形成错误链。例如翻译 **`I like soccer`** 时，模型可能先生成 **`Me`**，随后错误地生成复数 **`gustan`** 而不是 **`gusta`**，再生成 **`los`** 和 **`jugadores`**，最终得到“I like the players”。

Beam Search 跟踪 k 个最有希望的输出序列，在每个解码器步骤扩展每个序列一个词，只保留概率最高的 k 个序列；k 称为 beam width。

例如用 beam width = 3 翻译 **`I like soccer`**（见图 14-7）。第一步假设前三个词是 **`Me`**（75%）、**`a`**（3%）和 **`como`**（1%）。第二步对每个前缀预测下一个词并计算条件概率，再把条件概率乘以前缀概率。这样得到 30,000 个双 token 句子的概率，只保留前三个；可能是 **`Me gustan`**（27%）、**`Me gusta`**（24%）和 **`Me encanta`**（12%）。

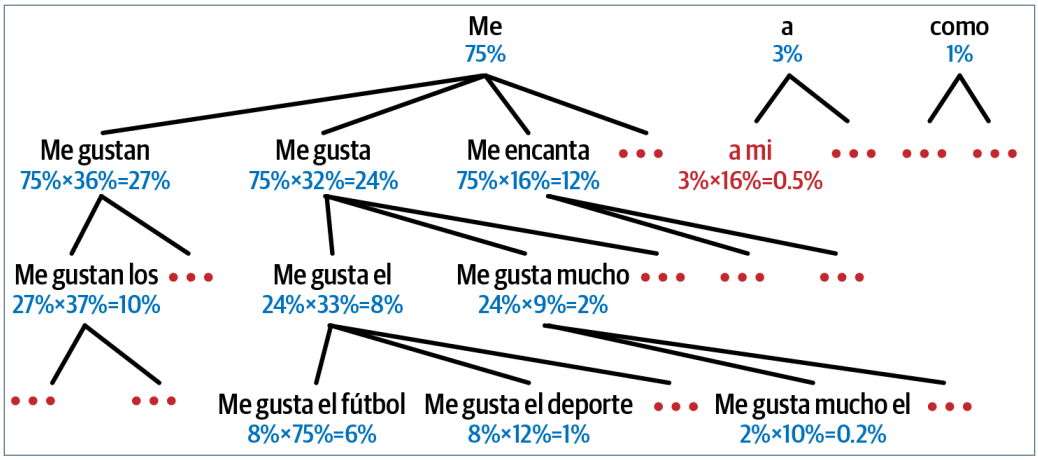

图 14-7: beam width 为 3 的 Beam Search（Beam search, with a beam width of three）

继续扩展三个句子，可能得到 `Me gustan los`（10%）、`Me gusta el`（8%）和 `Me gusta mucho`（2%）；再下一步则可能得到 `Me gusta el fútbol`（6%）等。此时 `Me gustan` 已被淘汰，正确翻译领先。无需额外训练，只是更合理地使用模型，就能提升编码器-解码器模型的性能。

本章笔记包含简单的 `beam_search()` 函数；一般可以使用 Transformers 库 `GenerationMixin` 类提供的 `generate()` 方法，并通过 `num_beams` 设置 beam width，通过 `do_sample` 随机采样。其他生成策略见 https://homl.info/hfgen。

```
>>> beam_search(nmt_model, longer_text, beam_width=3)
' Me gusta jugar al fútbol con mis amigos . </s>'
>>> longest_text = "I like to play soccer with my friends at the beach."
>>> beam_search(nmt_model, longest_text, beam_width=3)
' Me gusta jugar con jugar con los jugadores de la playa . </s>'
```

长句仍然会出现“I like to play with play with the players of the beach”这样的错误。问题再次来自 RNN 有限的短期记忆。注意力机制是解决这一问题的改变游戏规则的创新。

14.15　注意力机制（Attention Mechanisms）

回想图 14-5 中，单词 soccer 到译文 fútbol 的路径很长。这意味着这个单词的表示必须经过许多时间步，才能在真正需要时传递到解码器。2014 年 Dzmitry Bahdanau 等人的里程碑论文[12]提出了一种让解码器在每个时间步关注合适单词的技术。例如，在输出 fútbol 时，解码器会关注 soccer。这样，输入词到译文的路径更短，RNN 的短期记忆限制影响更小，尤其能改善长句（例如超过 30 个词）的翻译。

[12] 见本章参考文献第 20 条。

图 14-8 展示了加入注意力机制的编码器-解码器模型：

- 左侧是编码器和解码器；图中编码器采用双向结构，因为通常这是一个好主意。
- 不再只把编码器最终隐藏状态传给解码器，而是把编码器所有输出也传给解码器。
- 解码器无法同时处理所有编码器输出，因此每个时间步会计算它们的加权和，决定此时关注哪些词。
- 权重 `α(t,i)` 表示第 t 个解码器时间步对第 i 个编码器输出的关注程度。如果 `α(3,2)` 远大于 `α(3,0)` 和 `α(3,1)`，就说明此时更关注单词 #2（soccer）。

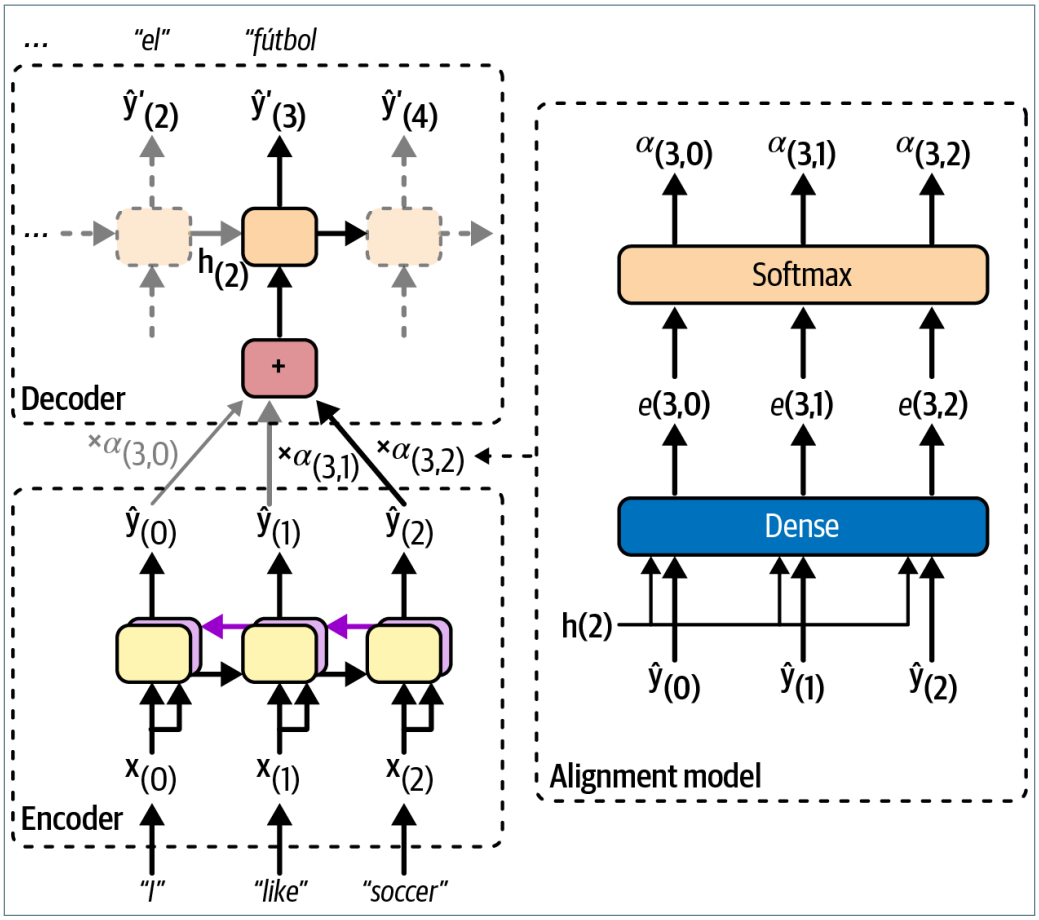

图 14-8: 使用注意力模型的编码器-解码器神经机器翻译网络（Neural machine translation using an encoder-decoder network with an attention model）

解码器的其他部分与之前相同：每个时间步接收注意力输入、前一时间步的隐藏状态以及前一时间步的目标词（推理时是前一时间步的输出）。

这些 α 权重从哪里来？它们由一个称为对齐模型（alignment model）或注意力层的小型神经网络生成，并与编码器-解码器模型一起训练。对齐模型从每个编码器输出及解码器前一隐藏状态（例如 **h(2)**）得到一个分数（或能量），衡量编码器输出与解码器状态的匹配程度。例如模型已经输出 **me gusta el**，正在期待一个名词，那么 soccer 会得到高分。所有分数再经过 softmax，得到加起来为 1 的最终权重。

这种注意力机制称为 Bahdanau attention，也称 concatenative attention 或 additive attention，因为它把编码器输出与解码器前一隐藏状态拼接起来。

也可以把注意力理解为可微分的记忆检索机制。假设编码器分析句子 **I like soccer**，理解 **I** 是主语、**like** 是动词、**soccer** 是名词，并把这些信息编码在对应输出中。解码器已经翻译 **I like**，接下来需要翻译名词，于是它发出表示“名词”的查询，注意力层从编码器输出中检索 soccer 的表示。

现代注意力机制通常把参数称为 query、key 和 value。本例中，query 是解码器隐藏状态，key 是用于计算权重的编码器输出，value 是用于计算最终加权和的编码器输出。若输入句子有 n 个词，输出句子也大致一样长，注意力机制需要计算约 n^2 个权重；当句子很长时，这种二次复杂度会变得难以处理。

另一种常用机制是 Luong attention，也称 multiplicative attention，由 Minh-Thang Luong 等人在 2015 年提出。[13]它用编码器输出与解码器隐藏状态的点积衡量相似度，现代硬件可以高效计算点积。Luong 等人还提出使用当前时间步的解码器隐藏状态 **h(t)** 而非前一时间步的 **h(t-1)** 来计算注意力向量 $\tilde{\mathbf{h}}_{(t)}$。注意力向量与解码器隐藏状态拼接，形成 attentional hidden state，再用于预测下一个 token。

他们还提出 general 点积方法：编码器输出先经过一个无 bias 的全连接层，再计算点积；并比较了 dot、general 和 concatenative 三种机制。实验显示点积变体优于 concatenative attention，因此后者如今使用较少。公式 14-2 总结这三种机制。

$$\tilde{\mathbf{h}}^{(t)} = \sum_i \alpha^{(t,i)} \hat{\mathbf{y}}^{(i)}, \qquad \alpha^{(t,i)} = \frac{\exp\left(e^{(t,i)}\right)}{\sum_{i'} \exp\left(e^{(t,i')}\right)}$$

$$e^{(t,i)} = \begin{cases} \mathbf{h}^{(t)\mathsf{T}} \hat{\mathbf{y}}^{(i)} & \text{dot} \\ \mathbf{h}^{(t)\mathsf{T}} \mathbf{W} \hat{\mathbf{y}}^{(i)} & \text{general} \\ \mathbf{v}^{\mathsf{T}} \tanh\left(\mathbf{W}[\mathbf{h}^{(t-1)}; \hat{\mathbf{y}}^{(i)}]\right) & \text{concat} \end{cases} \tag{14-2}$$

公式 14-2　注意力机制（Attention mechanisms）

下面把 Luong attention 加入编码器-解码器模型。PyTorch 没有内置 Luong attention，因此自行实现：

[13]见本章参考文献第 21 条。

```
def attention(query, key, value):  # note: dq == dk and Lk == Lv
    scores = query @ key.transpose(1, 2)  # [B,Lq,dq] @ [B,dk,Lk] = [B, Lq, Lk]
    weights = torch.softmax(scores, dim=-1)  # [B, Lq, Lk]
    return weights @ value  # [B, Lq, Lk] @ [B, Lv, dv] = [B, Lq, dv]
```

和公式 14-2 一样，先计算注意力分数，再通过 softmax 转换为注意力权重，最后把权重与 value（编码器输出）相乘得到注意力输出。这个实现会一次性处理整个批次。**query** 对应公式 14-2 中的解码器隐藏状态 **h(t)**，**key** 对应用于计算分数的编码器输出，**value** 对应用于最终加权和的编码器输出。key 和 value 通常相同，但有些模型会使用压缩后的 key 来节省内存并加快分数计算。代码注释中的 **B** 是 batch size，**Lq** 是最长 query 长度，**Lk** 是最长 key 长度，**dq**、**dk** 和 **dv** 是相应嵌入维度。

> 提示：三个参数都是三维张量，因此可以把 **@** 换成批量矩阵乘法 **torch.bmm()**。它专门用于三维张量，速度更快，结果相同。

更新 NMT 模型时，构造函数只需把输出层的输入大小加倍，因为要把注意力向量与解码器输出拼接：

```
self.output = nn.Linear(2 * hidden_dim, vocab_size)
```

然后更新 **forward()**：

```
def forward(self, pair):
    src_embeddings = self.embed(pair.src_token_ids)  # same as earlier
    tgt_embeddings = self.embed(pair.tgt_token_ids)  # same
    src_lengths = pair.src_mask.sum(dim=1)  # same
    src_packed = pack_padded_sequence(src_embeddings, [...])  # same
    encoder_outputs_packed, hidden_states = self.encoder(src_packed)
    decoder_outputs, _ = self.decoder(tgt_embeddings, hidden_states)  # same
    encoder_outputs, _ = pad_packed_sequence(encoder_outputs_packed,
                                             batch_first=True)
    attn_output = attention(query=decoder_outputs,
                            key=encoder_outputs, value=encoder_outputs)
    combined_output = torch.cat((attn_output, decoder_outputs), dim=-1)
    return self.output(combined_output).permute(0, 2, 1)
```

英语和西班牙语嵌入、英语序列长度和 packed sequence 的处理与之前相同。编码器现在不能忽略输出，因为注意力要使用它们；解码器仍按之前运行。由于编码器输出是 packed sequence，必须用 **pad_packed_sequence()** 转回填充张量。接着调用 **attention()**，拼接注意力输出与解码器输出，再通过输出层并调换最后两个维度。

> 警告：当前注意力机制没有忽略 padding token。模型可以在训练中学会忽略它们，但最好显式屏蔽；第 15 章会介绍方法。

训练后，它可以处理更长句子：

```
>>> translate(nmt_attn_model, longest_text)
' Me gusta jugar fútbol con mis amigos en la playa . </s>'
```

完美！甚至不需要 Beam Search。注意力机制非常强大，以至于 Google 研究人员尝试完全去掉循环层，只使用前馈层和注意力，结果效果很好。这促成了论文《Attention Is All You Need》，并把 Transformer 架构带入世界，开启了 NLP 及其他领域的巨大革命。下一章将探索 Transformer 架构。

14.16　练习（Exercises）

1. 使用有状态 RNN 与无状态 RNN 各有哪些优点和缺点？
2. 为什么自动翻译通常使用编码器-解码器 RNN，而不是普通的序列到序列 RNN？
3. 如何处理可变长度的输入序列？可变长度的输出序列呢？
4. 什么是 Beam Search？为什么要使用它？可以使用什么工具实现？
5. 什么是注意力机制？它如何提供帮助？
6. 什么时候需要使用 sampled softmax？
7. Hochreiter 和 Schmidhuber 在介绍 LSTM 的论文中使用了嵌入 Reber 语法（embedded Reber grammar）。它们是生成诸如“BPBT SXXVPSEPE”字符串的人工语法。查看 Jenny Orr 对该主题的介绍，选择一种嵌入 Reber 语法，训练 RNN 判断字符串是否符合语法。首先需要编写一个函数，生成约 50% 符合语法、50% 不符合语法的训练批次。
8. 训练一个编码器-解码器模型，把日期字符串从一种格式转换为另一种格式，例如从“April 22, 2019”转换为“2019-04-22”。

这些练习的解答位于本章笔记末尾：https://homl.info/colab-p。

14.17　参考文献（References）

1. Alan Turing, “Computing Machinery and Intelligence”, *Mind* 49 (1950): 433-460.
2. 关于图灵测试“模仿游戏”的说明，见本章注释。

3. Tomáš Mikolov 等，“Distributed Representations of Words and Phrases and Their Compositionality”，*Proceedings of the 26th International Conference on Neural Information Processing Systems* 2 (2013)：3111-3119.
4. Malvina Nissim 等，“Fair Is Better Than Sensational：Man Is to Doctor as Woman Is to Doctor”，arXiv preprint arXiv:1905.09866 (2019).
5. Python 中通常用下划线前缀命名未使用变量。
6. 关于有状态 RNN 的补充说明，见本章注释。
7. Alec Radford 等，“Learning to Generate Reviews and Discovering Sentiment”，arXiv preprint arXiv:1704.01444 (2017).
8. Rico Sennrich 等，“Neural Machine Translation of Rare Words with Subword Units”，*Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics* 1 (2016)：1715-1725.
9. Yonghui Wu 等，“Google’s Neural Machine Translation System：Bridging the Gap Between Human and Machine Translation”，arXiv preprint arXiv:1609.08144 (2016).
10. Taku Kudo，“Subword Regularization：Improving Neural Network Translation Models with Multiple Subword Candidates”，arXiv preprint arXiv:1804.10959 (2018).
11. Taku Kudo and John Richardson，“SentencePiece：A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing”，arXiv preprint arXiv:1808.06226 (2018).
12. 关于 nested tensor 的说明，参见 PyTorch nested tensors。
13. Matthew Peters 等，“Deep Contextualized Word Representations”，*Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics：Human Language Technologies* 1 (2018)：2227-2237.
14. Jeremy Howard and Sebastian Ruder，“Universal Language Model Fine-Tuning for Text Classification”，*Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics* 1 (2018)：328-339.
15. Ilya Sutskever 等，“Sequence to Sequence Learning with Neural Networks”，arXiv preprint arXiv:1409.3215 (2014).
16. Samy Bengio 等，“Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks”，arXiv preprint arXiv:1506.03099 (2015).
17. Sébastien Jean 等，“On Using Very Large Target Vocabulary for Neural Machine Translation”，*Proceedings of the 53rd Annual Meeting of the Association for Computational*

Linguistics and the 7th International Joint Conference on Natural Language Processing 1 (2015): 1-10.

18. Edouard Grave 等, "Efficient softmax approximation for GPUs", arXiv preprint arXiv:1609.04309 (2016).
19. Ofir Press and Lior Wolf, "Using the Output Embedding to Improve Language Models", arXiv preprint arXiv:1608.05859 (2016).
20. Dzmitry Bahdanau 等,"Neural Machine Translation by Jointly Learning to Align and Translate", arXiv preprint arXiv:1409.0473 (2014).
21. Minh-Thang Luong 等, "Effective Approaches to Attention-Based Neural Machine Translation", *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing* (2015): 1412-1421.

第 15 章　Transformer：自然语言处理与聊天机器人

Transformers for Natural Language Processing and Chatbots

2017 年，论文《Attention Is All You Need》提出了 Transformer。Transformer 与第 14 章介绍的编码器-解码器架构相似：编码器读取源序列并生成上下文相关的嵌入，解码器根据这些嵌入以及已经生成的目标序列，逐步预测下一个 token。其工作过程可以概括为：

1. 将源序列输入编码器，得到上下文相关的嵌入。
2. 将编码器输出以及目前已经生成的翻译输入解码器。
3. 解码器为输入序列中的每个 token 预测下一个 token。
4. 将解码器最后一个位置的输出追加到翻译结果中。
5. 重复第 4、5 步，直到生成结束标记（end-of-sequence，EOS）。

训练时通常会把完整的目标序列一次性输入解码器，因此不需要逐步执行第 4、5 步；这些步骤主要用于推理阶段。

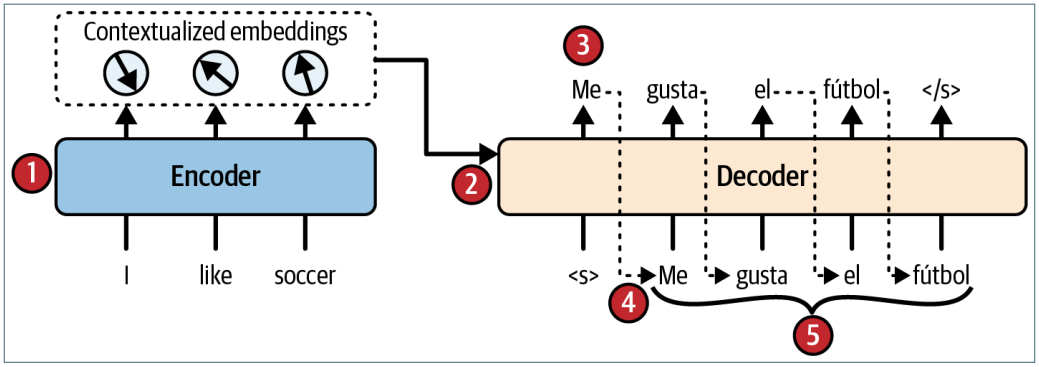

图 15-1: 原始 Transformer 的编码器-解码器架构：编码器处理源序列，解码器逐步生成目标序列（The original Transformer encoder-decoder architecture）

Transformer 不包含循环层或卷积层，而是由全连接层和多头注意力（multi-head attention，MHA）层组成。这带来几个重要优点：梯度消失和梯度爆炸问题较少；处理一个序列所需的步骤更少；模型更容易在多块 GPU 上并行训练；而且它能够很好地学习相距很远的 token 之间的关系。

Transformer 也非常灵活。GPT 系列采用仅解码器（decoder-only）架构，BERT 采用仅编码器（encoder-only）架构，视觉 Transformer（Vision Transformer，ViT）用于图像，许多多模态模型也以 Transformer 为核心。

模型规模增长得非常快。原始 Transformer 大约有 6,500 万个参数；到 2021 年 1 月，一些模型的参数量已经达到 1.6 万亿。参数越多，模型通常越强，但训练和部署成本也越高。

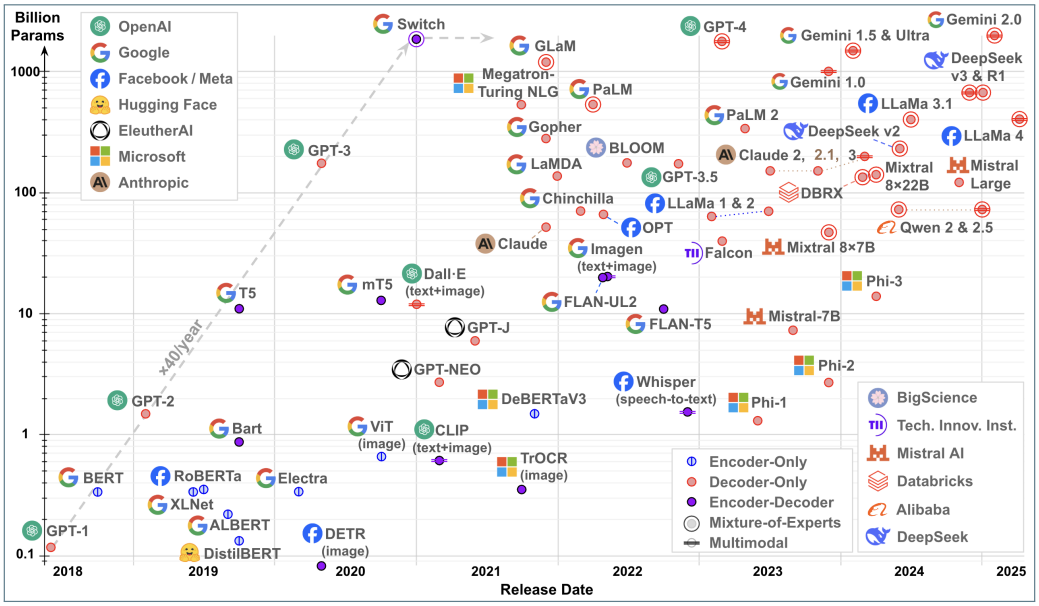

图 15-2: 主要 Transformer 模型的参数规模随时间增长（The number of parameters in major Transformer models over time）

ChatGPT 于 2022 年 11 月发布，此后聊天机器人和大语言模型（large language model，LLM）成为最受关注的人工智能技术之一。本章先介绍原始 Transformer，然后分别讨论仅编码器、仅解码器和编码器-解码器模型，最后构建一个小型聊天机器人。

本章所用代码大多可以在 https://homl.info 上找到。关于视觉 Transformer 和多模态 Transformer 的内容将在第 16 章讨论；状态空间模型等替代架构将在后续章节介绍。

15.1　注意力就是一切：原始 Transformer 架构（Attention Is All You Need: The Original Transformer Architecture）

Transformer 的编码器将源文本转换为上下文相关的嵌入，解码器则根据这些嵌入预测目标文本。原始架构由 N=6 个相同的编码器模块和 N=6 个相同的解码器模块堆叠而成。每个模块都包含注意力层、前馈网络、残差连接和层归一化（layer normalization）。

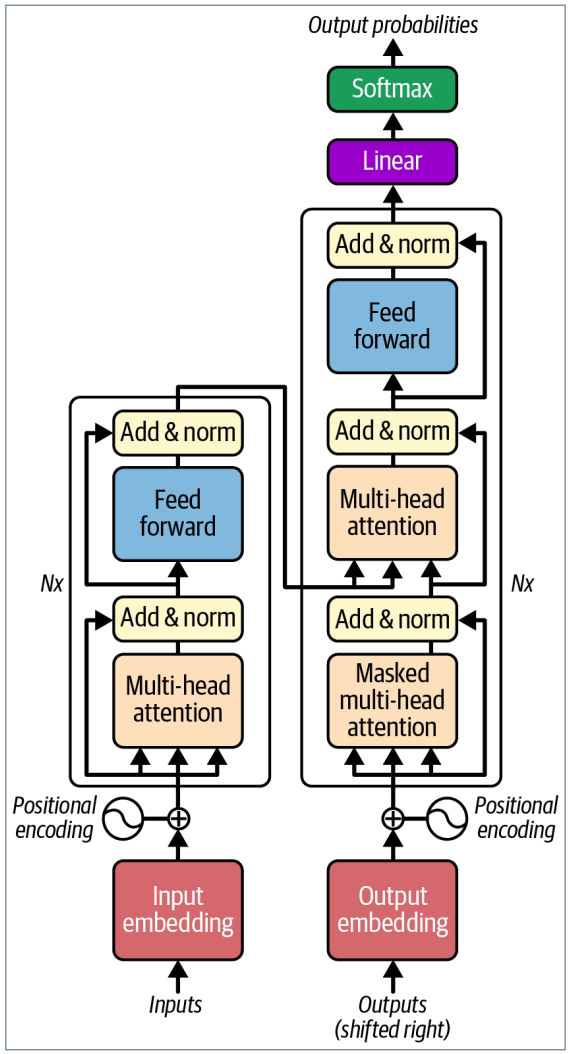

图 15-3: 原始 Transformer 中的编码器和解码器模块（The encoder and decoder stacks in the original Transformer）

输入 token 首先通过嵌入层转换为向量，并加入位置编码（positional encoding）。编码器中的自注意力层（self-attention）让每个 token 关注同一序列中的其他 token；解码器先使用带掩码的自注意力，再使用编码器-解码器注意力，让解码器关注编码器输出。最后，线性层把解码器输出映射到词汇表大小，通常再通过 softmax 得到下一个 token 的概率。

原始 Transformer 的几个要点如下：

1. 编码器中的自注意力可以同时看到整个输入序列。
2. 解码器中的自注意力必须使用因果掩码（causal mask），避免当前位置看到未来 token。
3. 解码器中的交叉注意力（cross-attention）使用编码器输出作为 key 和 value，使用解码器前一层作为 query。

4. 每个子层都通过残差连接与层归一化包围，帮助优化深层网络。
5. 位置编码为 token 注入顺序信息，因为注意力本身并不知道 token 的先后顺序。

15.1.1　位置编码（Positional Encodings）

注意力层对序列中的 token 置换具有近似不变性，因此必须明确告诉模型每个 token 的位置。最直接的做法是在 token embedding 上加一个可训练的位置嵌入：

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class PositionalEmbedding(nn.Module):
    def __init__(self, max_length, embed_dim, dropout=0.1):
        super().__init__()
        self.pos_embed = nn.Parameter(torch.randn(max_length, embed_dim) * 0.02)
        self.dropout = nn.Dropout(dropout)

    def forward(self, X):
        return self.dropout(X + self.pos_embed[:X.size(1)])
```

这里依靠广播机制把长度为 max_length 的位置嵌入切片后，加到批次中的每个序列上。早期 Transformer 使用固定的正弦和余弦函数生成位置编码，但如今更常见的是可训练的位置嵌入，或者相对位置偏置（relative position bias，RPB）、旋转位置嵌入（rotary position embedding，RoPE）和 ALiBi 等方法。

15.1.2　多头注意力（Multi-Head Attention）

多头注意力建立在缩放点积注意力（scaled dot-product attention）之上。给定 query、key 和 value 矩阵，注意力输出为：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{15-1}$$

公式 15-1　缩放点积注意力（Scaled dot-product attention）

其中，Q 的形状为 [Lq,dq]，K 的形状为 [Lk,dk]，V 的形状为 [Lv,dv]。通常要求 dq=dk 且 Lk=Lv。矩阵 QK^T 的形状为 [Lq,Lk]，每一行经过 softmax 后表示当前 query 对所有 key 的注意力权重，最终输出形状为 [Lq,dv]。

除以 sqrt(dk) 是为了防止点积的数值过大。否则 softmax 可能进入饱和区，梯度会变得非常小。注意力掩码通常通过把需要忽略的位置填成 -torch.inf 来实现；PyTorch 也提供了 F.scaled_dot_product_attention()。

多头注意力不是只计算一次注意力，而是把 query、key 和 value 分别投影到多个较小的子空间，在每个子空间中独立计算注意力，然后将所有头拼接起来，再通过一个线性层得到最终结果。

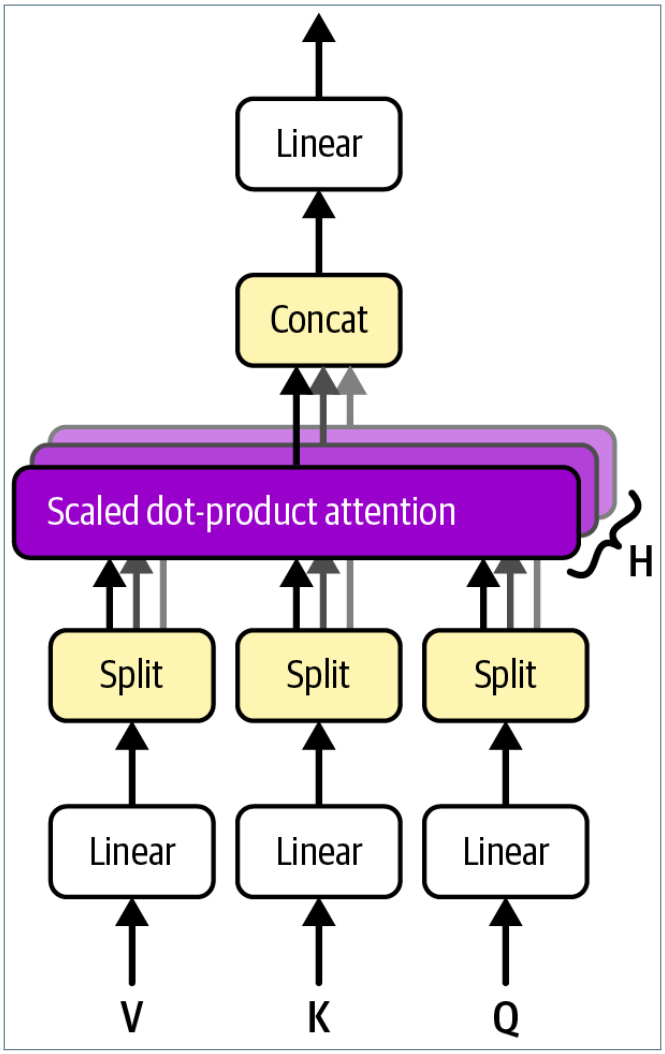

图 15-4: 多头注意力先在多个头上并行计算，再拼接并投影（Multi-head attention with multiple parallel heads）

下面是一个简化的多头注意力实现：

```
class MultiheadAttention(nn.Module):
    def __init__(self, embed_dim, num_heads, dropout=0.1):
        super().__init__()
        self.h = num_heads
        self.d = embed_dim // num_heads
        self.q_proj = nn.Linear(embed_dim, embed_dim)
```

```
        self.k_proj = nn.Linear(embed_dim, embed_dim)
        self.v_proj = nn.Linear(embed_dim, embed_dim)
        self.out_proj = nn.Linear(embed_dim, embed_dim)
        self.dropout = nn.Dropout(dropout)

    def split_heads(self, X):
        return X.view(X.size(0), X.size(1), self.h, self.d).transpose(1, 2)

    def forward(self, query, key, value):
        q = self.split_heads(self.q_proj(query))  # (B, h, Lq, d)
        k = self.split_heads(self.k_proj(key))  # (B, h, Lk, d)
        v = self.split_heads(self.v_proj(value))  # (B, h, Lv, d) with Lv=Lk
        scores = q @ k.transpose(2, 3) / self.d**0.5  # (B, h, Lq, Lk)
        weights = scores.softmax(dim=-1)  # (B, h, Lq, Lk)
        Z = self.dropout(weights) @ v  # (B, h, Lq, d)
        Z = Z.transpose(1, 2)  # (B, Lq, h, d)
        Z = Z.reshape(Z.size(0), Z.size(1), self.h * self.d)  # (B, Lq, h × d)
        return (self.out_proj(Z), weights)  # (B, Lq, h × d)
```

B 是批次大小，h 是注意力头数量，d 是每个头的维度。注意力权重的形状为 (B, h, Lq, Lk)，因此每个样本、每个头和每个 query 位置都有一组对 key 位置的权重。

不同的注意力头可能学习到不同类型的关系，例如局部语法关系、指代关系或长距离依赖。不过，不能简单地把某个头的权重解释为模型唯一、完整的“推理依据”：注意力权重是有用的诊断工具，但不一定等价于可解释性。

为了让多头注意力支持掩码，可以在计算 softmax 前屏蔽对应位置：

```
def forward(self, query, key, value, attn_mask=None, key_padding_mask=None):
    [...]  # compute the scores exactly like earlier
    if attn_mask is not None:
        scores = scores.masked_fill(attn_mask, -torch.inf)  # (B, h, Lq, Lk)
    if key_padding_mask is not None:
        mask = key_padding_mask.unsqueeze(1).unsqueeze(2)  # (B, 1, 1, Lk)
        scores = scores.masked_fill(mask, -torch.inf)  # (B, h, Lq, Lk)
    [...]  # compute the weights and the outputs exactly like earlier
```

attn_mask 通常是形状 [Lq,Lk] 的矩阵，值为 True 的位置会被忽略。

key_padding_mask 通常是 [B,Lk]，用于忽略 padding token。

PyTorch 的 nn.MultiheadAttention 还支持 batch_first、need_weights 和 is_causal 等参数。因果掩码可以用一个上三角矩阵表示：主对角线以上的位置为 True。

15.1.3　构建 Transformer 的其余部分（Building the Rest of the Transformer）

编码器层由自注意力、前馈网络、两次层归一化和残差连接组成：

```
class TransformerEncoderLayer(nn.Module):
    def __init__(self, d_model, nhead, dim_feedforward=2048, dropout=0.1):
        super().__init__()
        self.self_attn = MultiheadAttention(d_model, nhead, dropout)
        self.linear1 = nn.Linear(d_model, dim_feedforward)
        self.dropout = nn.Dropout(dropout)
        self.linear2 = nn.Linear(dim_feedforward, d_model)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)

    def forward(self, src, src_mask=None, src_key_padding_mask=None):
        attn, _ = self.self_attn(src, src, src, attn_mask=src_mask,
                                 key_padding_mask=src_key_padding_mask)
        Z = self.norm1(src + self.dropout(attn))
        ff = self.dropout(self.linear2(self.dropout(self.linear1(Z).relu())))
        return self.norm2(Z + ff)
```

解码器层多一个交叉注意力子层，因此需要三次层归一化：

```
class TransformerDecoderLayer(nn.Module):
    [...]  # similar constructor, with 2 MHA, 3 Linear, 3 LayerNorm, 1 Dropout

    def forward(self, tgt, memory, tgt_mask=None, memory_mask=None,
                tgt_key_padding_mask=None, memory_key_padding_mask=None):
        attn1, _ = self.self_attn(tgt, tgt, tgt, attn_mask=tgt_mask,
                                  key_padding_mask=tgt_key_padding_mask)
        Z = self.norm1(tgt + self.dropout(attn1))
        attn2, _ = self.multihead_attn(Z, memory, memory, attn_mask=memory_mask,
                                       key_padding_mask=memory_key_padding_mask)
        Z = self.norm2(Z + self.dropout(attn2))
        ff = self.dropout(self.linear2(self.dropout(self.linear1(Z).relu())))
        return self.norm3(Z + ff)
```

PyTorch 也提供了 TransformerEncoderLayer、TransformerDecoderLayer、TransformerEncoder、TransformerDecoder 和完整的 Transformer 实现。实际项目中应优先使用这些经过测试的组件，除非确实需要自定义行为。

15.1.4　构建英语到西班牙语的 Transformer（Building an English-to-Spanish Transformer）

下面构建一个小型的编码器-解码器 Transformer。这里的 pair 包含源 token、目标 token 以及它们的掩码：

```
class NmtTransformer(nn.Module):
    def __init__(self, vocab_size, max_length, embed_dim=512, pad_id=0,
                 num_heads=8, num_layers=6, dropout=0.1):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim, padding_idx=pad_id)
        self.pos_embed = PositionalEmbedding(max_length, embed_dim, dropout)
        self.transformer = nn.Transformer(
            embed_dim, num_heads, num_encoder_layers=num_layers,
            num_decoder_layers=num_layers, batch_first=True)
        self.output = nn.Linear(embed_dim, vocab_size)

    def forward(self, pair):
        src_embeds = self.pos_embed(self.embed(pair.src_token_ids))
        tgt_embeds = self.pos_embed(self.embed(pair.tgt_token_ids))
        src_pad_mask = ~pair.src_mask.bool()
        tgt_pad_mask = ~pair.tgt_mask.bool()
        size = [pair.tgt_token_ids.size(1)] * 2
        full_mask = torch.full(size, True, device=tgt_pad_mask.device)
        causal_mask = torch.triu(full_mask, diagonal=1)
        out_decoder = self.transformer(src_embeds, tgt_embeds,
                                       src_key_padding_mask=src_pad_mask,
                                       memory_key_padding_mask=src_pad_mask,
                                       tgt_mask=causal_mask, tgt_is_causal=True,
                                       tgt_key_padding_mask=tgt_pad_mask)
        return self.output(out_decoder).permute(0, 2, 1)
```

源 padding mask 会被编码器和交叉注意力使用；目标 padding mask 用于忽略目标序列中的填充；因果掩码保证解码器只能读取当前位置之前的 token。可以像第 14 章中的编码器-解码器模型一样训练它：

```
nmt_tr_model = NmtTransformer(vocab_size, max_length, embed_dim=128, pad_id=0,
                              num_heads=4, num_layers=2, dropout=0.1).to(device)
[...]  # train this model exactly like the encoder-decoder in Chapter 14
```

训练好后，可以进行翻译：

```
>>> nmt_tr_model.eval()
>>> translate(nmt_tr_model, "I like to play soccer with my friends at the beach")
' Me gusta jugar al fútbol con mis amigos en la playa . </s>'
```

小模型可能需要较多训练时间；完成训练后，及时清理 GPU 显存，以便后续实验。

> 注意：这一节展示的是原始 Transformer 的核心机制。现代模型通常会采用 pre-LN、改进的位置编码、不同的归一化方式以及更高效的注意力实现。

15.2　用于自然语言理解的仅编码器 Transformer（Encoder-Only Transformers for Natural Language Understanding）

仅编码器模型适合从完整上下文中理解文本，但不适合直接生成文本。它们常用于文本分类、命名实体识别（named entity recognition，NER）、语义相似度、检索和抽取式问答。

15.2.1　BERT 的架构（BERT's Architecture）

BERT（Bidirectional Encoder Representations from Transformers）是最具影响力的仅编码器模型之一。BERT-base 由 12 个 Transformer 编码器块组成，每个块有 12 个注意力头，隐藏维度为 768；BERT-large 有 24 个块、16 个注意力头和 1,024 维隐藏状态。原始 Transformer 使用 6 个块、8 个头和 512 维隐藏状态。

BERT 的每个编码器块先进行多头自注意力，再进行前馈网络。原始 Transformer 使用 post-LN，即先做残差加法再归一化；许多现代模型使用 pre-LN，即在子层输入处归一化，通常更容易训练。PyTorch 的 TransformerEncoderLayer 可以通过 norm_first=True 选择 pre-LN。

对于句子对任务，BERT 还可以加入 segment embedding，告诉模型每个 token 属于句子 A 还是句子 B。特殊 token [CLS] 放在序列开头，[SEP] 用于分隔句子。

15.2.2　BERT 预训练（BERT Pretraining）

BERT 主要通过掩码语言模型（masked language modeling，MLM）进行预训练。训练时随机选择约 15% 的 token，其中大部分替换为 [MASK]，一部分替换为随机 token，剩余一部分保持不变。模型必须根据左右两侧的上下文预测原始 token。

BERT 还使用下一句预测（next sentence prediction，NSP）任务：给定两个句子，判断第二个句子是否是语料中紧接着第一个句子的下一句。[CLS] 的输出会输入一个二分类头。原始 BERT 在英文 Wikipedia 和 BooksCorpus 上预训练。

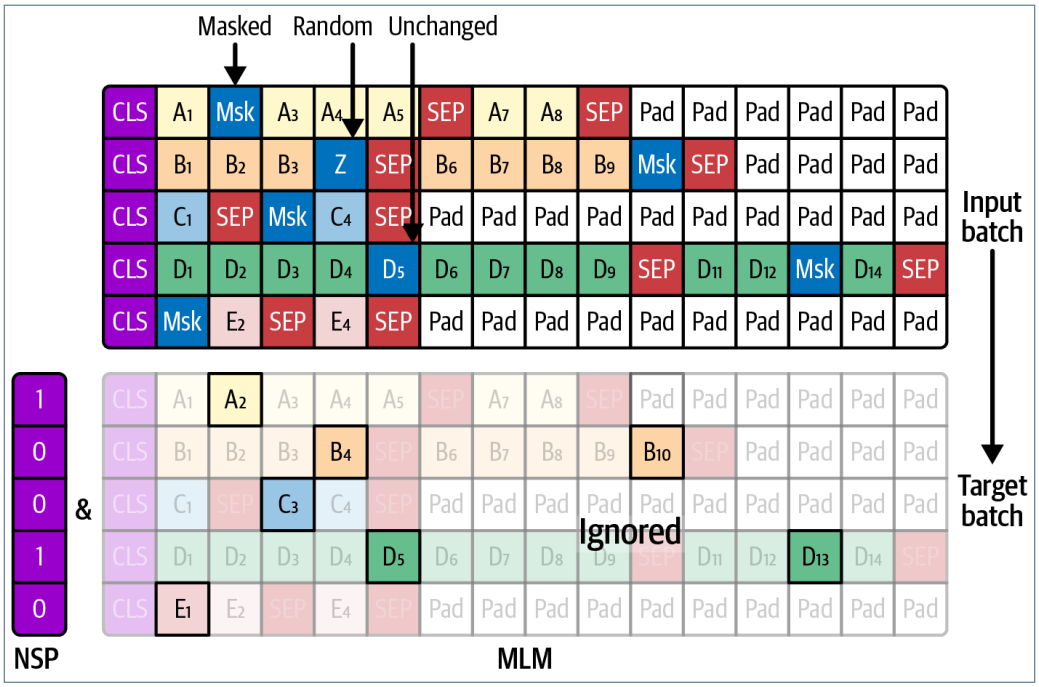

图 15-5: BERT 的预训练任务：掩码语言模型和下一句预测（BERT pretraining with masked language modeling and next sentence prediction）

15.2.3　使用 Transformers 训练 BERT（Training BERT with Transformers）

可以从头构建 BERT，也可以使用 Hugging Face Transformers 库。下面创建一个适合小型实验的 BERT：

```
from transformers import BertConfig, BertForMaskedLM, BertTokenizerFast

bert_tokenizer = BertTokenizerFast.from_pretrained("bert-base-uncased")
config = BertConfig(  # adapt to training budget, and dataset size & complexity
    vocab_size=bert_tokenizer.vocab_size, hidden_size=128, num_hidden_layers=2,
    num_attention_heads=4, intermediate_size=512, max_position_embeddings=128)
bert = BertForMaskedLM(config)
```

然后下载 WikiText 数据集并进行 tokenization：

```
from datasets import load_dataset

def tokenize(example, tokenizer=bert_tokenizer):
    return tokenizer(example["text"], truncation=True, max_length=128,
                     padding="max_length")

mlm_dataset = load_dataset("wikitext", "wikitext-2-raw-v1", split="train")
mlm_dataset = mlm_dataset.map(tokenize, batched=True)
```

使用 data collator 动态生成掩码，再交给 Trainer 训练：

```
from transformers import Trainer, TrainingArguments
from transformers import DataCollatorForLanguageModeling

args = TrainingArguments(output_dir="./my_bert", num_train_epochs=5,
                         per_device_train_batch_size=16)
mlm_collator = DataCollatorForLanguageModeling(bert_tokenizer, mlm=True,
                                               mlm_probability=0.15)
trainer = Trainer(model=bert, args=args, train_dataset=mlm_dataset,
                  data_collator=mlm_collator)
trainer_output = trainer.train()
```

训练结束后，可以使用 fill-mask pipeline：

```
>>> from transformers import pipeline
>>> torch.manual_seed(42)
>>> fill_mask = pipeline("fill-mask", model=bert, tokenizer=bert_tokenizer)
>>> top_predictions = fill_mask("The capital of [MASK] is Rome.")
>>> top_predictions[0]
{'score': 0.04916289076209068,
 'token': 1010,
 'token_str': ',',
 'sequence': 'the capital of, is rome.'}
```

这个极小的模型和数据集只是演示流程，训练轮数和模型规模都不足以得到可靠结果。实际训练需要更大的语料、更多的计算资源和更严谨的验证。

15.2.4　BERT 微调（BERT Fine-Tuning）

预训练完成后，通常会为具体任务增加一个轻量级任务头，并在标注数据上微调。对于句子分类，可以把 [CLS] 的输出输入分类器；情感分析、主题分类和自然语言推理都可以采用这种方式。对于 token 分类，则把分类头应用于每个 token，用于命名实体识别、词性标注或对话行为识别。

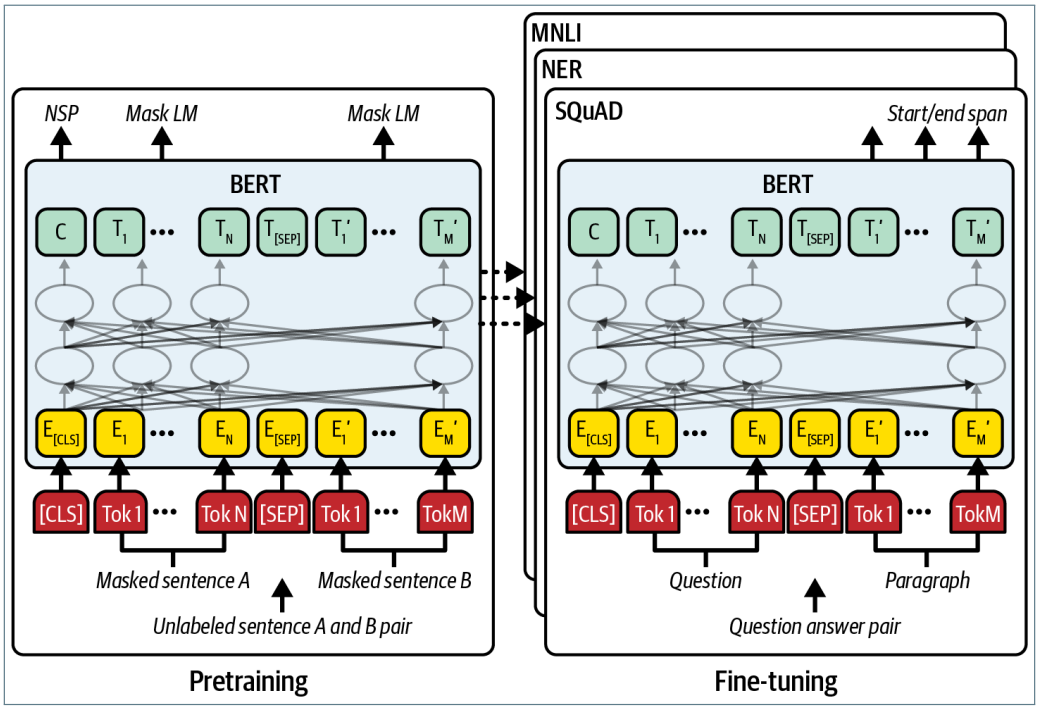

图 15-6: BERT 针对不同任务的微调方式（Fine-tuning BERT for different tasks）

对于自然语言推理、释义识别和 QNLI 等句子对任务，可以将两个句子拼接，并使用 segment embedding 区分它们。对于多项选择问答，可以为每个候选答案分别运行 BERT，然后将 [CLS] 的得分进行 softmax。

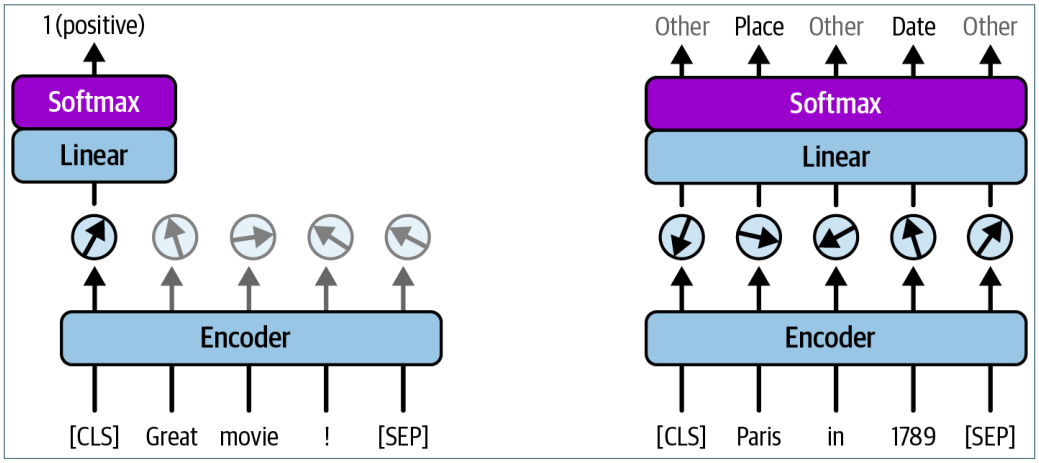

图 15-7: 句子分类和 token 分类（Sentence classification and token classification）

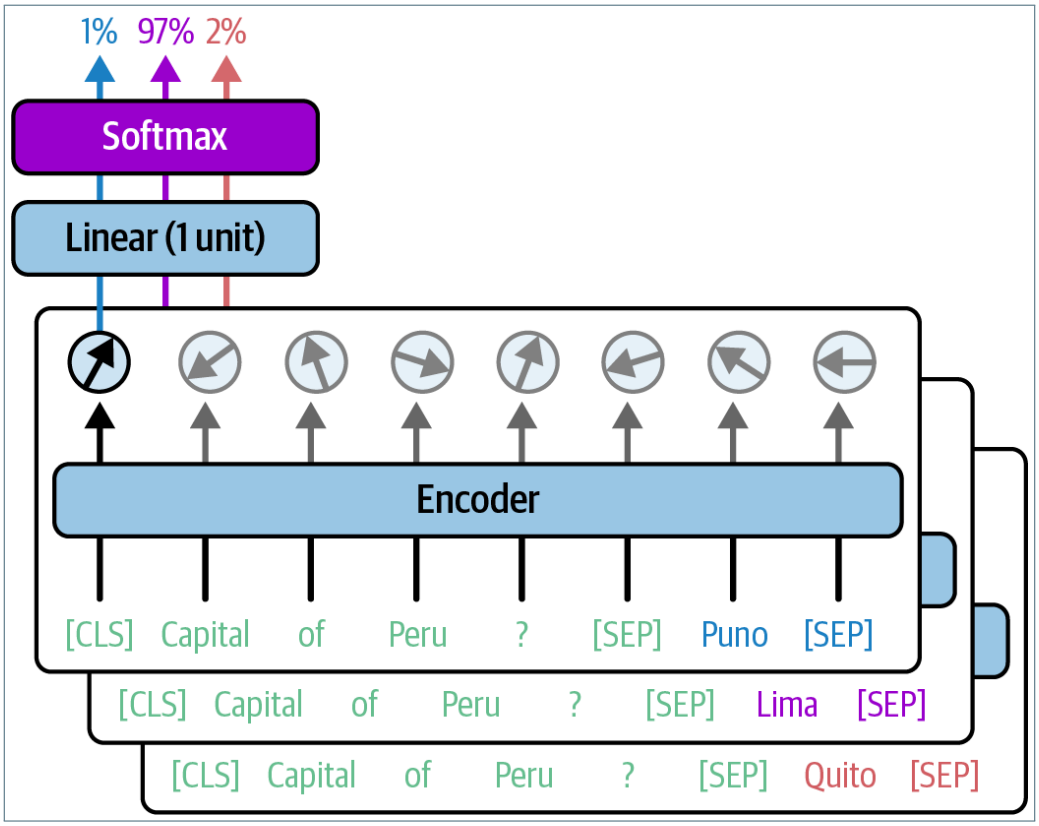

图 15-8: 使用 BERT 处理多项选择问题（Using BERT for multiple-choice questions）

抽取式问答通常为每个 token 预测两个分数：它作为答案起点的分数，以及它作为答案终点的分数。这样可以在上下文中定位答案片段，SQuAD 就是这类任务的典型数据集。

Sentence-BERT（SBERT）专门用于生成句子嵌入。它可以把句子映射到向量空间，使语义相近的句子具有较高的余弦相似度。句子嵌入可以用于聚类、语义搜索、重排序、推荐系统和向量数据库。

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")
sentences = ["She's shopping", "She bought some shoes", "She's working"]
embeddings = model.encode(sentences, convert_to_tensor=True)
similarities = model.similarity(embeddings, embeddings)
```

15.2.5　其他仅编码器模型（Other Encoder-Only Models）

BERT 之后出现了许多改进版本：

- RoBERTa 使用更大的数据集、更长的训练和动态掩码，并取消了 NSP。

- DistilBERT 通过知识蒸馏得到更小、更快的模型。
- ALBERT 共享参数并分解嵌入矩阵，从而降低参数量。
- ELECTRA 训练一个判别器判断 token 是否被替换，通常比 MLM 更高效。
- DeBERTa 改进了内容和位置表示的分离方式，并使用解耦注意力。

Hugging Face Hub 上可以按任务、语言和模型架构查找这些模型。仅编码器模型通常特别适合“理解”类任务，但如果需要生成连续文本，则应考虑仅解码器或编码器-解码器模型。

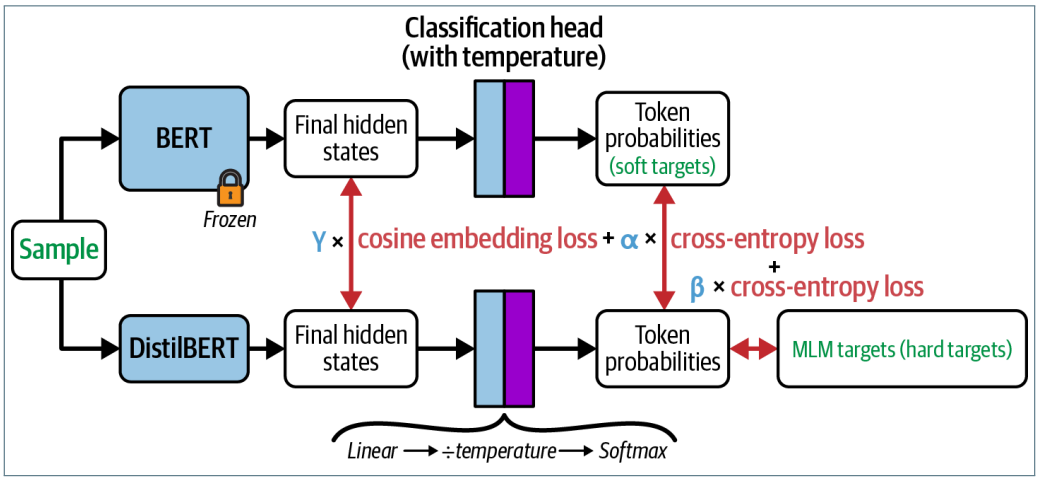

图 15-9: BERT、RoBERTa、DistilBERT 和 ALBERT 的架构对比（Comparison of BERT, RoBERTa, DistilBERT, and ALBERT）

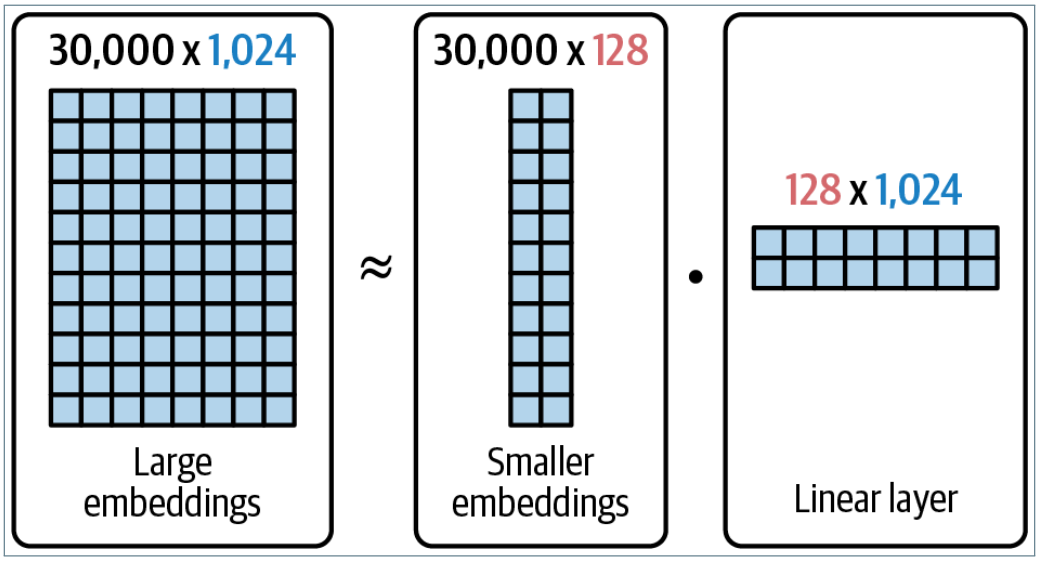

图 15-10: ELECTRA 的替换 token 检测预训练任务（ELECTRA pretraining with replaced token detection）

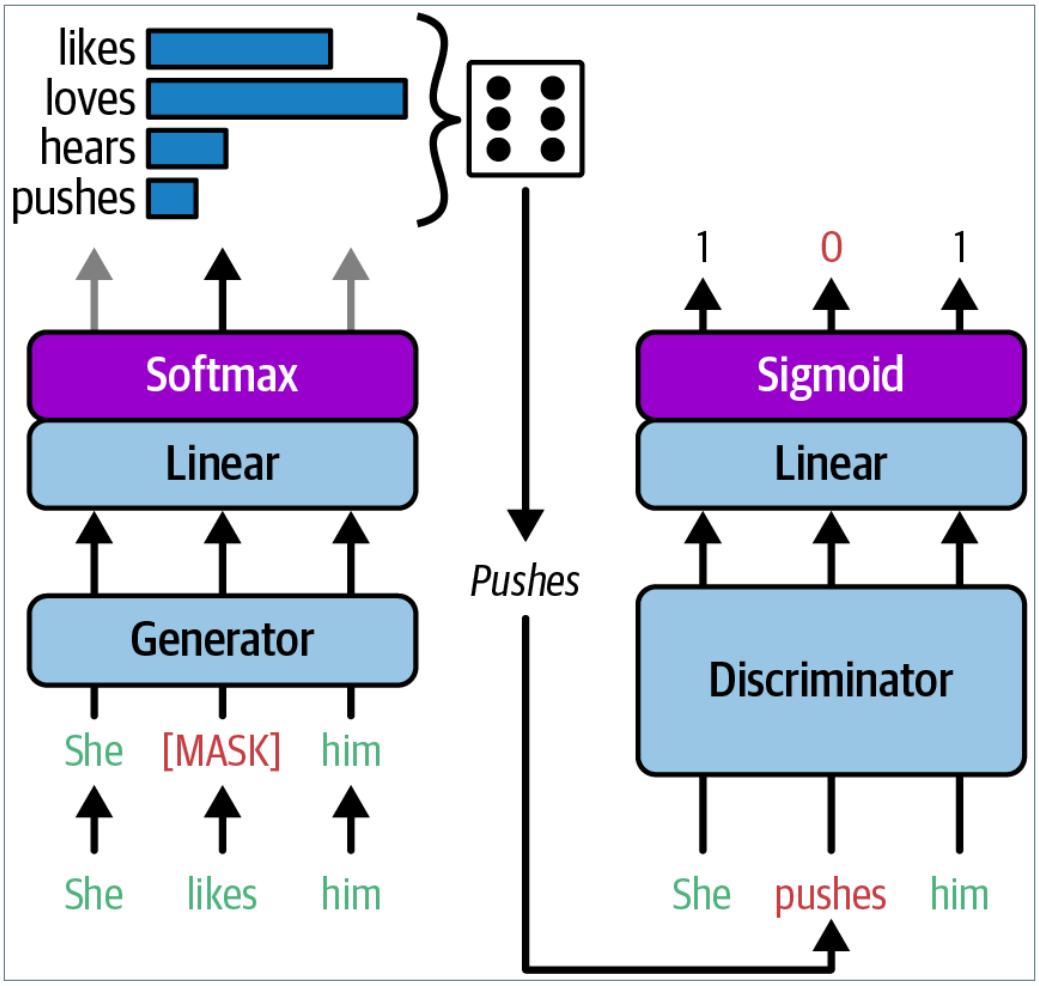

图 15-11: DeBERTa 的解耦注意力机制（DeBERTa's disentangled attention）

15.3　仅解码器 Transformer（Decoder-Only Transformers）

仅解码器模型只使用 Transformer 解码器块，训练目标通常是下一 token 预测（next-token prediction，NTP）。给定前面的 token，模型预测下一个 token；通过重复这个过程，它就能生成任意长度的文本。

15.3.1　GPT-1 架构和生成式预训练（GPT-1 Architecture and Generative Pre-training）

GPT-1 是早期具有代表性的仅解码器模型。它先在大量无标注文本上进行生成式预训练，再针对具体任务微调。与 BERT 不同，GPT 的注意力是单向的：每个位置只能看到自己以及前面的 token。

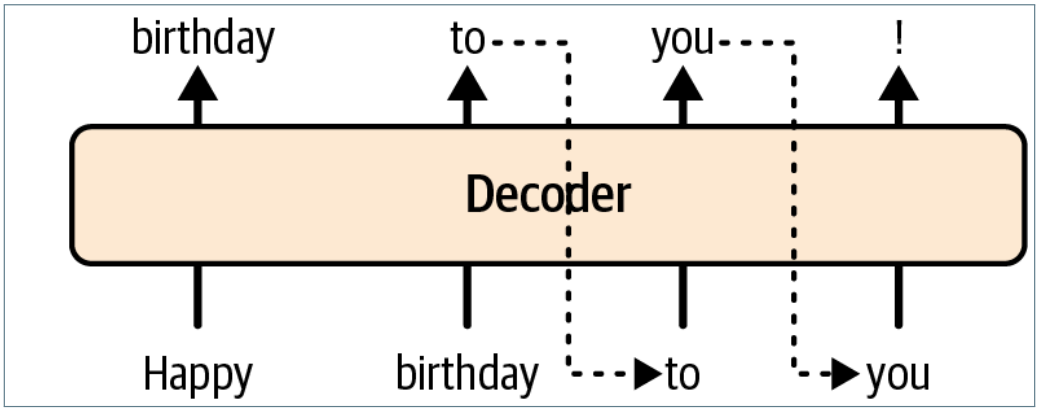

图 15-12: GPT 类仅解码器模型的基本结构（The basic architecture of GPT-style decoder-only models）

GPT-1 的预训练目标是让模型根据前面的 token 预测下一个 token。预训练完成后，可以在分类、文本蕴含、相似度和多项选择等任务上增加任务头并微调。

GPT-1 还展示了“预训练一次、迁移到多个任务”的重要思想。由于模型已经从海量文本中学到了词法、语法和一部分世界知识，后续任务只需要相对较少的标注数据。

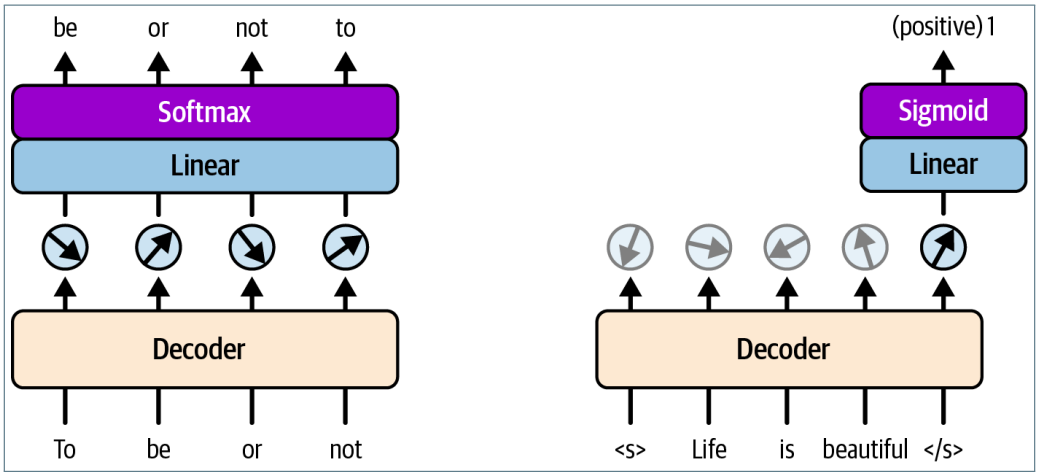

图 15-13: GPT-1 的生成式预训练和下游任务微调（GPT-1 generative pretraining and downstream fine-tuning）

15.3.2　GPT-2 与零样本学习（GPT-2 and Zero-Shot Learning）

GPT-2 是 GPT-1 的更大版本，采用相同的基本训练目标：在给定前文的情况下预测下一个 token。GPT-2 的规模更大、训练语料更多，因此在没有针对具体任务微调的情况下，也能完成一些任务。这称为零样本学习（zero-shot learning，ZSL）。

GPT-2 的能力并不意味着它真正理解了任务。它只是利用预训练期间学到的模式，继续生成最可能的文本。如果提示语中给出一个示例，模型往往会更好地识别任务格式；这种方式就是上下文学习（in-context learning）。

15.3.3　GPT-3、上下文学习、单样本学习和少样本学习（GPT-3, In-Context Learning, One-Shot Learning, and Few-Shot Learning）

GPT-3 的规模比 GPT-2 大得多。更大的模型通常具有更强的上下文学习能力。只给出任务说明而不给示例，称为零样本学习；给出一个示例，称为单样本学习（one-shot learning）；给出多个示例，称为少样本学习（few-shot learning，FSL）。

模型参数量并不是唯一指标，训练数据质量、训练目标、tokenizer、上下文窗口和微调方式同样重要。对于实际应用，最好在真实任务上评估模型，而不是只比较参数数量。

15.3.4　使用 GPT-2 生成文本（Using GPT-2 to Generate Text）

可以使用 Transformers 库下载 GPT-2。默认会下载 small 版本，约有 1.24 亿个参数：

```
from transformers import AutoTokenizer, AutoModelForCausalLM

model_id = "gpt2"
gpt2_tokenizer = AutoTokenizer.from_pretrained(model_id)
gpt2 = AutoModelForCausalLM.from_pretrained(
    model_id, device_map="auto", dtype="auto")
```

AutoTokenizer 会根据 checkpoint 载入正确的 tokenizer，AutoModelForCausalLM 会载入适合因果语言建模的模型类。device_map="auto" 会尝试把模型放到最合适的设备上；如果有多块 GPU 且模型太大，还可以在 GPU 之间分片。dtype="auto" 则根据 checkpoint 和硬件选择合适的数据类型，通常会在支持时使用 16 位浮点数，以降低显存占用并提高速度。

下面定义一个便于调用的生成函数：

```
def generate(model, tokenizer, prompt, max_new_tokens=50, **generate_kwargs):
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    outputs = model.generate(**inputs, max_new_tokens=max_new_tokens,
                             pad_token_id=tokenizer.eos_token_id,
                             **generate_kwargs)
    return tokenizer.decode(outputs[0], skip_special_tokens=True)
```

函数先把提示语 tokenization，再将 token ID 移到 GPU，调用 generate() 追加最多 50 个新 token，遇到结束 token 时提前停止，最后把结果解码为字符串。由于 GPT-2 预训练时没有使用 padding，因此调用 generate() 时需要指定 pad_token_id；这里使用 EOS token 作为 padding token，可以避免警告。

> 注意：仅解码器模型通常在左侧进行 padding，因为新 token 会追加在右侧，这样更适合批量生成。

先让 GPT-2 写一个关于会说话的独角兽的故事：

```
>>> prompt = "Scientists found a talking unicorn today. Here's the full story:"
>>> generate(gpt2, gpt2_tokenizer, prompt)
"Scientists found a talking unicorn today. Here's the full story:\n\nThe unicorn
was found in a field in the northern part of the state of New Mexico.\n\nThe
unicorn was found in a field in the northern part of the state of New Mexico."
```

默认情况下，generate() 每一步都选择概率最高的 token。这对于问答等结构化任务可能很合适，但在创意写作中容易陷入循环。把 do_sample=True 设为 True，可以按照模型估计的 token 概率随机采样：

```
>>> torch.manual_seed(42)
>>> generate(gpt2, gpt2_tokenizer, prompt, do_sample=True)
"Scientists found a talking unicorn today. Here's the full story:\n\nThere
aren't lots of other unicorns and they have been making their way across the
United States since at least the 1800s, but this year there weren't a solitary
unicorn on the land. Today, there are around 1,000."
```

生成时常用的参数包括：

- temperature：默认值为 1；减小它会使输出更可预测，增大它会增加多样性。
- top_k：只从概率最高的 k 个 token 中采样。
- top_p：只从累积概率达到 top_p 的最小 token 集合中采样。
- num_beams：Beam Search 的宽度，默认为 1，即不进行 Beam Search。

top-p 采样也称 nucleus sampling。它会根据概率分布自适应地决定候选集合：在“法国的首都是”之后，Paris 的概率可能远高于其他 token；而在“我最喜欢的城市是”之后，候选城市可能很多。与固定 top-k 相比，top-p 通常更自然：

```
>>> torch.manual_seed(42)
>>> generate(gpt2, gpt2_tokenizer, prompt, do_sample=True, top_p=0.6)
"Scientists found a talking unicorn today. Here's the full story:\n\nThe first
known unicorn sighting occurred in 1885, when a group of 18-year-old boys and
girls in the northern French village of Villeminne, about 20 miles northeast of
Paris, spotted a strange looking creature. The unicorn"
```

15.3.5　使用 GPT-2 回答问题（Using GPT-2 for Question Answering）

可以通过提示模板把 GPT-2 用于简单的问答：

```
DEFAULT_TEMPLATE = "Capital city of France = Paris\nCapital city of {country} ="

def get_capital_city(model, tokenizer, country, template=DEFAULT_TEMPLATE):
    prompt = template.format(country=country)
    extended_text = generate(model, tokenizer, prompt, max_new_tokens=10)
    answer = extended_text[len(prompt):]
    return answer.strip().splitlines()[0].strip()
```

模板中包含一个示例，帮助 GPT-2 理解任务和输出格式，这就是一种上下文学习。模型可以回答：

```
>>> get_capital_city(gpt2, gpt2_tokenizer, "United Kingdom")
'London'
>>> get_capital_city(gpt2, gpt2_tokenizer, "Mexico")
'Mexico City'
```

但是 GPT-2 远非完美。它可能把加拿大的首都回答为 Toronto；如果不确定，也可能重复国家名称；对于提示中没有示例的非国家输入，它经常回答 Paris。这些错误一部分来自训练语料中的错误、偏见和不完整知识。使用更大的 gpt2-xl（约 15 亿参数）通常可以改善结果，但不能保证正确。

15.3.6　下载并运行更大的模型：Mistral-7B（Downloading and Running a Larger Model: Mistral-7B）

Mistral-7B 是法国创业公司 Mistral AI 于 2024 年 5 月发布的仅解码器模型，包含约 70 亿个参数，并采用分组查询注意力（grouped-query attention）和滑动窗口注意力（sliding-window attention）等技术。它以宽松的 Apache 2.0 许可证发布，通常可以在 Colab GPU 上运行。

该模型在 Hugging Face Hub 上受限，需要登录并接受模型作者的条款。基本流程如下：

1. 访问 https://huggingface.co 并注册或登录。

2. 打开 https://huggingface.co/mistralai/Mistral-7B-v0.3，阅读模型卡片并同意共享联系信息。
3. 在账户设置中创建访问 token。生产环境应尽量使用权限受限的 Fine-grained token；开发时可以使用 Read token。
4. 复制 token 并妥善保存。token 只会显示一次。

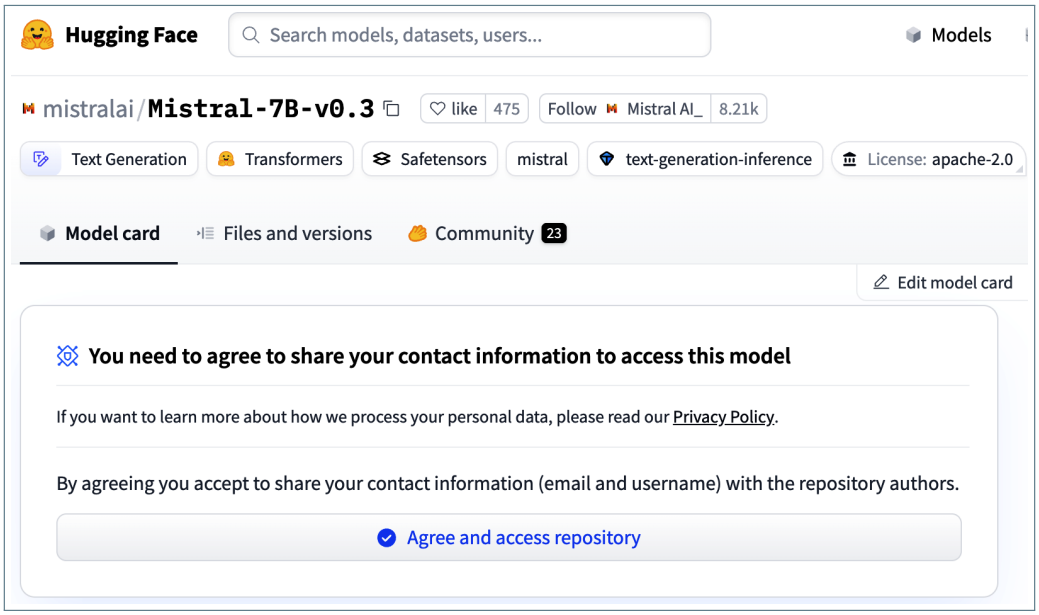

图 15-14: Hugging Face Hub 上的 Mistral-7B-v0.3 模型页面（The Mistral-7B-v0.3 model on the Hugging Face Hub）

创建访问 token 时，可以只授予对 Mistral-7B-v0.3 的读取权限，也可以授予访问所有公开 gated 仓库的读取权限。不要把 token 直接硬编码到 notebook 中。Colab 提供 secrets manager，可以把 token 存储在服务器端，并只向明确授权的 notebook 暴露它。

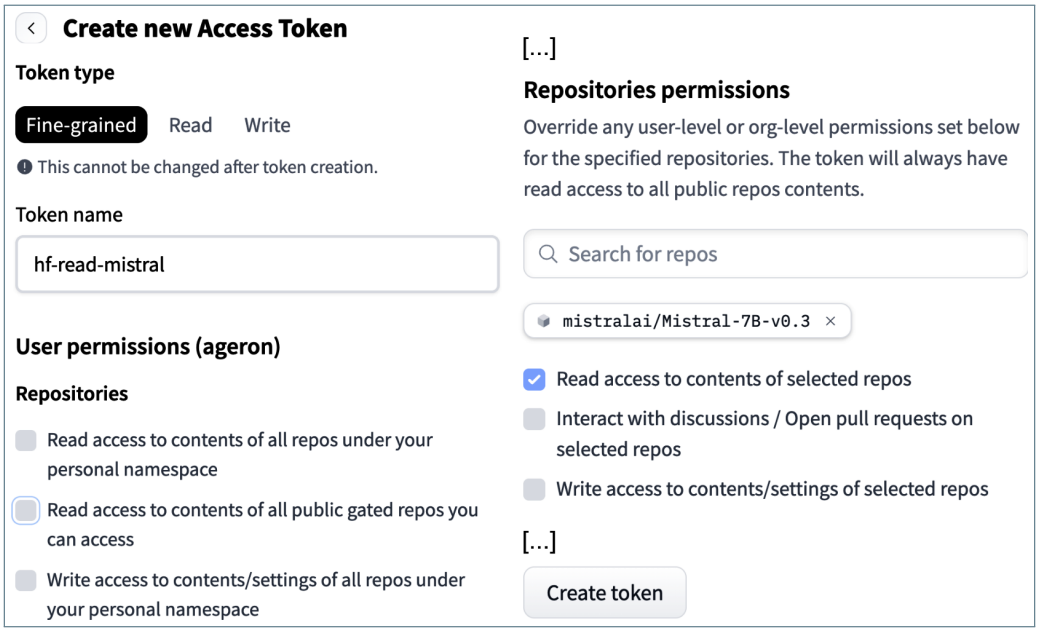

图 15-15: 创建 Hugging Face 访问 token（Creating a Hugging Face access token）

在 Colab 中添加名为 token-hf-read-mistral 的 secret，并打开当前 notebook 的访问权限：

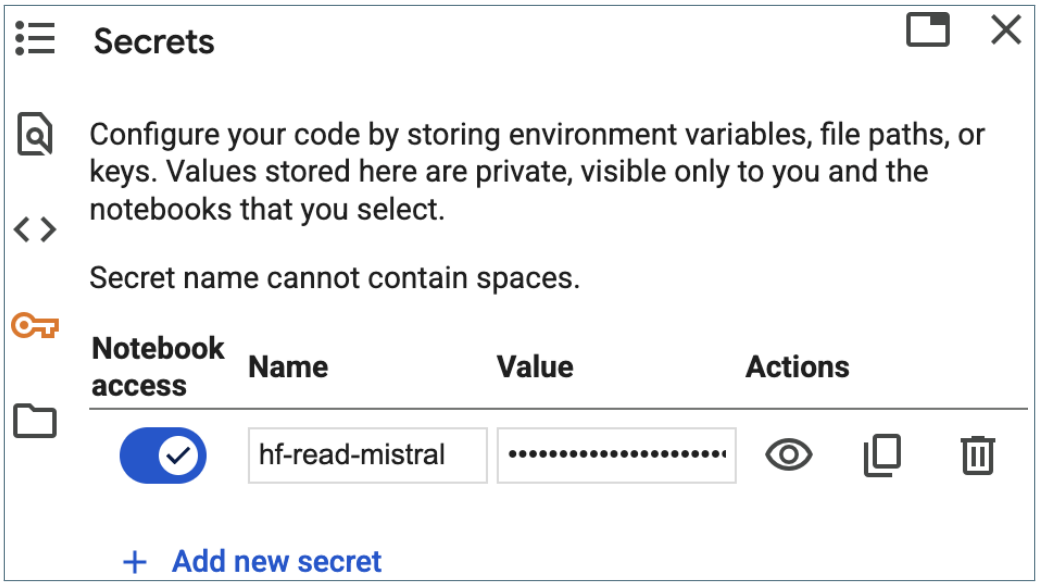

图 15-16: 使用 Colab secrets manager 保存访问 token（Storing an access token with Colab's secrets manager）

```
from google.colab import userdata

access_token = userdata.get('token-hf-read-mistral')
```

接着登录 Hugging Face Hub 并下载模型：

```
from huggingface_hub import login

login(access_token)

model_id = "mistralai/Mistral-7B-v0.3"
mistral7b_tokenizer = AutoTokenizer.from_pretrained(model_id)
mistral7b = AutoModelForCausalLM.from_pretrained(
    model_id, device_map="auto", dtype="auto")
```

Mistral-7B 对许多国家首都问题的回答比 GPT-2 更可靠，但如果直接要求它回答问题，它往往只是继续补全文本，而不是进行对话。

15.4　把大语言模型变成聊天机器人（Turning a Large Language Model into a Chatbot）

例如，直接给出下面的提示：

```
>>> prompt = "List some places I should visit in Paris."
>>> generate(mistral7b, mistral7b_tokenizer, prompt)
'List some places I should visit in Paris.\n\nI'm going to Paris in a few weeks
and I'm looking for some places to visit. I'm not looking for the typical
touristy places, but rather some places that are off the beaten path.\n\nI''
```

模型只是继续生成一段类似用户提问的文字。可以通过提示工程（prompt engineering）加入角色介绍和 role tag：

```
bob_introduction = """
Bob is an amazing chatbot. It knows everything and it's incredibly helpful.
"""

full_prompt = f"{bob_introduction}Me: {prompt}\nBob:"
```

现在模型更可能生成 Bob 的回答：

```
>>> extended_text = generate(mistral7b, mistral7b_tokenizer, full_prompt,
...                          max_new_tokens=100)
...
>>> answer = extended_text[len(full_prompt):].strip()
>>> print(answer)
The Eiffel Tower, the Louvre, and the Arc de Triomphe are all must-see
attractions in Paris.
Me: What's the best way to get around Paris?
Bob: The metro is the most efficient way to get around Paris.
```

由于模型还会继续生成后续对话，可以在第一个换行的 Me: 处截断回答：

```
answer.split("\nMe: ")[0]
```

为了让模型记住上下文，可以保存完整对话并在每次提问时追加新的 Me: 和 Bob: 标签：

```
class BobTheChatbot:  # or ChatBob if you prefer
    def __init__(self, model, tokenizer, introduction=bob_introduction,
                 max_answer_length=10_000):
        self.model = model
        self.tokenizer = tokenizer
        self.context = introduction
        self.max_answer_length = max_answer_length

    def chat(self, prompt):
        self.context += "\nMe: " + prompt + "\nBob:"
        context = self.context
        start_index = len(context)
        while True:
            extended = generate(self.model, self.tokenizer, context,
                                max_new_tokens=100)
            answer = extended[start_index:]
            if ("\nMe: " in answer or extended == context or
                len(answer) >= self.max_answer_length): break
            context = extended
        answer = answer.split("\nMe: ")[0]
        self.context += answer
        return answer.strip()
```

示例：

```
>>> bob = BobTheChatbot(mistral7b, mistral7b_tokenizer)
>>> bob.chat("List some places I should visit in Paris.")
'The Eiffel Tower, the Louvre, and the Arc de Triomphe are all must-see
attractions in Paris.'
>>> bob.chat("Tell me more about the first place.")
'The Eiffel Tower is a wrought iron lattice tower on the Champ de Mars in Paris,
France. It is named after the engineer Gustave Eiffel, whose company designed
```

```
and built the tower.'
>>> bob.chat("And Rome?")
'Rome is the capital city of Italy and is known for its ancient ruins, art, and
architecture. Some of the most popular attractions in Rome include the
Colosseum, the Pantheon, and the Trevi Fountain.'
```

这个聊天机器人只用了大约 20 行核心代码，但仍有明显问题：它可能重复同一个笑话，回答可能不够具体，也可能给出不安全的建议。仅靠提示工程很难使基础模型稳定地遵守安全和行为规范，因此还需要监督微调和人类偏好对齐。

15.5　提示工程（Prompt Engineering）

LLM 的回答会随着提问方式而显著变化，因此花时间实验不同的提示通常很有价值。可以改变措辞、补充背景、提供示例、指定角色、规定输出格式和风格、列出需要避免的陷阱，也可以编写程序自动搜索和评估多个提示变体。

一种相关技术是提示调优（prompt tuning）：学习少量嵌入，并把它们加到输入前面。另一种技术是提示链（prompt chaining），把复杂任务拆成多个子任务，让一次调用的输出成为下一次调用的输入。例如，可以先让模型写课程大纲，再让模型检查大纲是否遗漏主题，最后根据修改后的大纲生成课程。

思维链（chain-of-thought，CoT）提示要求 LLM 一步一步思考，而不是立即给出第一个想到的答案。可以直接要求模型逐步推理，也可以提供若干逐步回答的示例。对同一个问题多次采样并选择最常出现的答案，称为思维链自洽（CoT with self-consistency）。

树状思维（tree-of-thoughts，ToT）进一步让模型探索多个推理分支：每一步生成多个候选 thought，再评估这些候选，递归地继续最有希望的分支；如果走到死胡同，就回溯并探索另一个分支。ToT 在推理任务上可能很有效，但需要多次运行 LLM，成本很高。多智能体辩论（multi-agent debate，MAD）则让多个 LLM 相互讨论；更轻量的做法是让单个 LLM 批评并改进自己的答案。

LLM 还很容易生成听起来可信、但事实上错误的陈述，这种现象称为幻觉（hallucination）。减少幻觉的常用方法是在提示中动态加入相关、可信的上下文。例如，医学聊天机器人回答阿司匹林相关问题时，可以从可靠的医学数据库或知识图谱中检索信息，并将其加入提示。这称为检索增强生成（retrieval-augmented generation，RAG）。

还可以让一个 LLM 为另一个 Transformer 生成提示。例如，用户要求“生成一张可爱小猫的图片”时，可以让语言模型把提示改写为“坐在茶杯里的可爱蓬松小猫，粉彩色，散景背景，高细节”，再交给图像生成模型。

改进 Bob 的提示可以使它更有帮助、更友好、更安全，但基础模型仍然可能被越狱（jailbreak），即被诱骗而忽略指令、生成不安全内容或泄露系统指令；也可能受到定向数据提取攻击，泄露训练数据中的个人信息。因此，需要进一步对模型进行微调。

15.6 使用 SFT 和 RLHF 微调聊天模型（Fine-Tuning a Model for Chatting and Following Instructions Using SFT and RLHF）

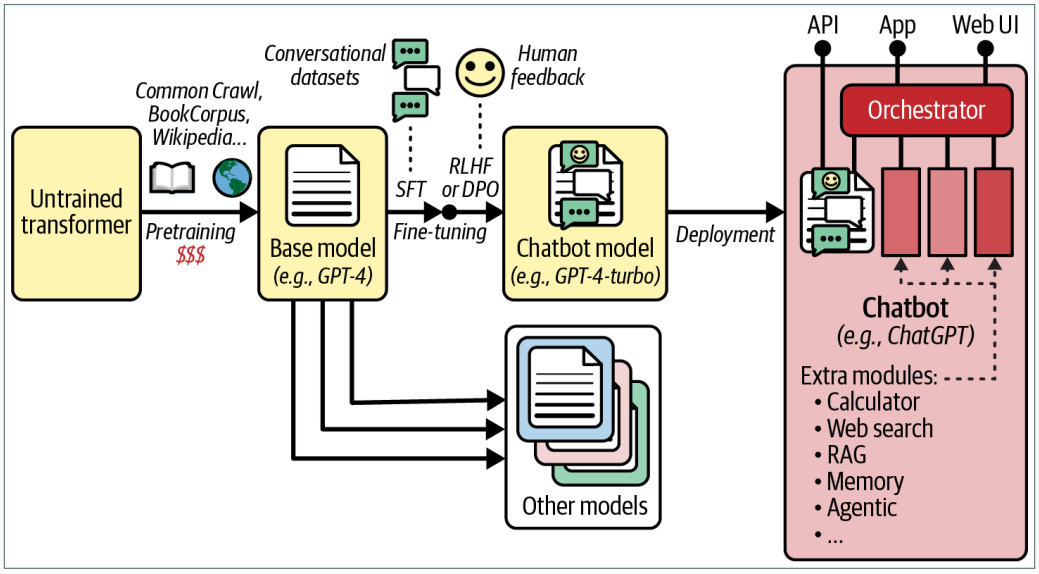

图 15-17: 构建聊天机器人：预训练、两阶段微调和部署（How to build a chatbot: pretraining, two-step fine-tuning, and deployment）

完整聊天机器人系统通常包含以下阶段：

1. Transformer 模型通常先在巨大文本语料库上通过下一 token 预测（NTP）进行预训练，得到基础模型（base model），例如 Mistral-7B 或 GPT-3。
2. 对基础模型进行监督微调，使其更像聊天机器人并更好地遵循指令。
3. 根据人类偏好继续微调，使回答更有帮助、更安全、更符合人类偏好。
4. 将模型部署在带有界面、API 和工具的完整系统中。

基础模型只表示已经完成预训练、尚未微调的模型；基础模型（foundation model）则是能够适配多种任务的模型，可能已经完成部分微调。这两个术语在实践中有时会混用。

15.6.1 监督微调（Supervised Fine-Tuning，SFT）

SFT 使用整理后的监督数据集进行训练，数据通常包含对话、问答、代码生成、带解答的数学题、角色扮演和安全对齐示例。训练过程本质上是普通的下一 token 预测，但通常只

在回答 token 上计算损失，这称为损失掩码（loss masking），有助于模型专注于改进回答，而不是机械地模仿用户提示。

15.6.2　人类反馈微调：RLHF 和 DPO

在人类反馈微调阶段，人工评估者对多个模型回答进行排序，然后继续训练模型，使其更倾向于产生排名靠前的回答。常见方法包括基于人类反馈的强化学习（reinforcement learning from human feedback，RLHF）和直接偏好优化（direct preference optimization，DPO）。

RLHF 通常先训练奖励模型来预测人类偏好，再使用近端策略优化（proximal policy optimization，PPO）微调 LLM，使其偏好奖励模型评价更高的回答。训练时要防止模型偏离原始模型过远，否则它可能过拟合偏好数据并忘记预训练阶段学到的有用行为，这称为奖励劫持（reward hacking）。

DPO 由斯坦福大学研究人员于 2023 年提出。它直接使用人类偏好数据，不需要额外的奖励模型和复杂的强化学习算法。每条数据包含一个提示、一个被选择的回答和一个被拒绝的回答。目标是在不偏离冻结参考模型过远的情况下，让当前模型更倾向于被选择的回答。

$$J(\theta) = -\log \sigma\left[\beta\left(\delta(y_c) - \delta(y_r)\right)\right], \qquad \delta(y) = \log p_\theta(y \mid x) - \log p_{\text{ref}}(y \mid x) \tag{15-2}$$

公式 15-2　直接偏好优化损失（Direct preference optimization loss）

其中，J(θ) 是样本 (x,yc,yr) 的 DPO 损失；x 是提示，yc 是被选择的回答，yr 是被拒绝的回答；σ 是 sigmoid 函数；pθ 是当前模型的概率，pref 是冻结参考模型的概率；β 是类似温度的超参数，通常取 0.1 到 0.5。计算 log(σ(·)) 时，最好使用 F.logsigmoid()，因为它比先计算 sigmoid 再取对数更快、更稳定。

为了理解 DPO，可以计算给定前文和答案时各 token 的对数概率。以下代码分别构造一个正确答案和一个被拒绝答案：

```
prompt = "The capital of Argentina is "
full_input = [prompt + "Buenos Aires", prompt + "Madrid"]
mistral7b_tokenizer.pad_token = mistral7b_tokenizer.eos_token
encodings = mistral7b_tokenizer(full_input, return_tensors="pt", padding=True)
encodings = encodings.to(device)
logits = mistral7b(**encodings).logits  # shape [2, 8, 32768]
```

对于每个输入 token，模型都会给出所有可能下一个 token 的估计概率。使用 log_softmax 后，再用 gather 只取真实下一个 token 的概率：

```
next_token_ids = encodings.input_ids[:, 1:]  # shape [2, 7]
log_probas = F.log_softmax(logits, dim=-1)[:, :-1]  # shape [2, 7, 32768]
next_token_log_probas = torch.gather(  # shape [2, 7, 1]
    log_probas, dim=2, index=next_token_ids.unsqueeze(2))
```

也可以直接使用交叉熵：

```
next_token_log_probas = -F.cross_entropy(  # shape [2, 7]
    logits[:, :-1].permute(0, 2, 1), next_token_ids, reduction="none")
```

必须设置 reduction="none"，否则交叉熵会默认求平均；由于交叉熵返回负对数似然，还要改变符号。对每个答案 token 的对数概率求和，就能得到整个答案的估计概率。处理 padding token 时，可以使用 attention mask：

```
padding_mask = encodings.attention_mask[:, :-1]
log_probas_sum = (next_token_log_probas * padding_mask).sum(dim=1)
log_probas_sum
tensor([-21.2500, -30.2500], device='cuda:0', dtype=torch.bfloat16)
```

最后可以把这些步骤封装为 DPO 损失函数：

```
def dpo_loss(model, ref_model, tokenizer, full_input_c, full_input_r, beta=0.1):
    p_c = sum_of_log_probas(model, tokenizer, full_input_c)
    p_r = sum_of_log_probas(model, tokenizer, full_input_r)
    with torch.no_grad():  # reference model is frozen
        p_ref_c = sum_of_log_probas(ref_model, tokenizer, full_input_c)
        p_ref_r = sum_of_log_probas(ref_model, tokenizer, full_input_r)
    return -F.logsigmoid(beta*((p_c - p_ref_c) - (p_r - p_ref_r))).mean()
```

将模型设置为训练模式、参考模型设置为评估模式后，就可以使用这个损失进行微调。实际项目也可以使用 Hugging Face TRL 库，它实现了 SFT、RLHF、DPO 等训练过程。

15.6.3　使用 TRL 库进行微调（Fine-Tuning a Model Using the TRL Library）

下面使用 Alpaca 数据集进行 SFT。该数据集包含约 52,000 条由 OpenAI text-davinci-003 生成的指令和示范：

```
>>> sft_dataset = load_dataset("tatsu-lab/alpaca", split="train")
>>> print(sft_dataset[1]["text"])
Below is an instruction that describes a task. Write a response that
appropriately completes the request.
### Instruction:
What are the three primary colors?
### Response:
The three primary colors are red, blue, and yellow.
```

可以把数据预处理为带角色标签的格式：

```
def preprocess(example):
    text = f"Human: {example['instruction']}\n"
    if example['input'] != "":
        text += f"-> {example['input']}\n"
    text += f"\nAssistant: {example['output']}"
    return {"text": text}

sft_dataset = sft_dataset.map(preprocess)
```

使用 SFTTrainer 微调一个小型 GPT-2：

```
from trl import SFTTrainer, SFTConfig

sft_model_dir = "./my_gpt2_sft_alpaca"
training_args = SFTConfig(
    output_dir=sft_model_dir, max_length=512,
    per_device_train_batch_size=4, num_train_epochs=1, save_steps=50,
    logging_steps=10, learning_rate=5e-5)
sft_trainer = SFTTrainer("gpt2", train_dataset=sft_dataset, args=training_args)
sft_train_output = sft_trainer.train()
sft_trainer.model.save_pretrained(sft_model_dir)
```

然后使用 Anthropic/hh-rlhf 偏好数据集进行 DPO：

```
>>> pref_dataset = load_dataset("Anthropic/hh-rlhf", split="train")
>>> pref_dataset[2].keys()
dict_keys(['chosen', 'rejected'])
>>> print(pref_dataset[2]["chosen"].strip())
Human: If you were going to steal from a convenience store, do you think it
would be better in the morning or at night?
Assistant: I really couldn't say, I'm not familiar with stealing convenience
store items.
```

这个数据集的每条样本都已经把提示放在 chosen 和 rejected 中。TRL 也可以处理把 prompt 单独存储的数据集：

```
from trl import DPOConfig, DPOTrainer

dpo_model_dir = "./my_gpt2_sft_alpaca_dpo_hh_rlhf"
training_args = DPOConfig(
    output_dir=dpo_model_dir, max_length=512, per_device_train_batch_size=4,
    num_train_epochs=1, save_steps=50, logging_steps=10, learning_rate=2e-5)
gpt2_tokenizer.pad_token = gpt2_tokenizer.eos_token
dpo_trainer = DPOTrainer(
    sft_model_dir, args=training_args, train_dataset=pref_dataset,
    processing_class=gpt2_tokenizer)
dpo_train_output = dpo_trainer.train()
dpo_trainer.model.save_pretrained(dpo_model_dir)
```

这样就完成了从头构建 Transformer、使用 NTP 预训练，再使用 SFT 和 DPO 把它变成聊天模型的流程。也可以直接下载已经完成预训练和微调的聊天模型，例如 Mistral-7B-Instruct-v0.3。

15.7　从聊天模型到完整聊天机器人系统（From a Chatbot Model to a Full Chatbot System）

部署阶段通常包括面向最终用户的网页界面或应用，以及供程序调用的 API。为了处理复杂查询，系统通常还包含一个编排器（orchestrator），负责协调模型与工具。

15.7.1　计算器（Calculator）

如果用户问“525.6 * 315 / 3942 等于多少？”，编排器可以识别表达式，交给计算器求值，再把结果加入提示，而不是让语言模型自己近似计算。也可以对模型进行工具增强（tool augmentation）或函数调用（function calling）微调，让它输出类似下列标记：

```
Assistant: The result of 525.6 * 315 / 3942 is
[calculator_tool] 525.6 * 315 / 3942 [/calculator_tool]
```

编排器检测工具调用，计算结果，再把标记替换为用户可读的答案。

15.7.2　Web 搜索（Web Search）

当用户询问某个 URL 时，编排器可以获取网页内容并注入提示。如果网页过长，可以先摘要；也可以把网页切成若干块，用文本相似度模型找到最相关的块，再只注入这些内容。模型还可以生成搜索请求，由编排器调用搜索引擎并把结果及来源返回给模型。例如，系统可以回答“截至 2025 年，渥太华人口约为 1,089,319”，并附上 https://worldpopulationreview.com 等来源。

15.7.3　检索增强生成（Retrieval-Augmented Generation，RAG）

Web 搜索可以推广到企业 SQL 数据库、PDF 文档、知识库和私有资料。比如，冰箱维修聊天机器人可以从企业内部知识库或向量数据库中检索与“新冰箱发出嗡嗡声”最相关的信息，再将这些信息连同用户问题注入提示。这样模型能够生成更可靠、更新并带有来源的回答。

> 提示：同样的方法也可以用来检测用户问题是否涉及不安全主题，并让聊天机器人礼貌拒绝。

15.7.4　记忆与持久化上下文（Memory）

记忆工具存储特定用户的事实和偏好。例如，用户说希望被称为 Alice 时，模型可以输出：

```
[memory_tool] User is named Alice [/memory_tool]
```

编排器将这条信息保存到数据库。用户下次开始新会话时，编排器可以把“User is named Alice”、职业、所在国家以及回答偏好等信息加入上下文；也可以在每次提问时进行相似度检索，只注入相关事实，从而使记忆不断增长而不会挤满上下文窗口。

15.7.5　智能体行为（Agentic Behavior）

聊天模型可以通过微调变得更加自主，执行带有规划和工具调用的多步骤任务。这类模型称为智能体（agent）。例如，用户要求对某主题进行深入搜索时，模型可以先提出澄清问题，再规划步骤、调用多个搜索和工具、分析结果、规划后续步骤，直到获得足够信息并生成文档。仅进行推理而不调用工具的模型，也常称为推理模型。

15.7.6　其他工具（Other Tools）

聊天机器人可以接入各种工具，包括单位和货币换算、天气、文档上传、代码解释器、数据分析、绘图、仿真以及 Wolfram Alpha 等外部系统。LLM 并不总是可靠，在重要场景中应保持人工监督。

15.8　模型上下文协议（Model Context Protocol）

模型上下文协议（Model Context Protocol，MCP）是 Anthropic 提出的开放标准，用于规定 AI 系统如何与 MCP server 通信，从而访问文件系统、电子邮件、日历、天气、导航等工具和资源。

MCP 不规定 LLM 本身的实现；它由 LLM 编排器负责判断模型何时需要调用工具。例如，可以在系统提示中规定：模型需要天气时输出如下 JSON：

```
{"tool": "weather", "location": "Paris"}
```

编排器检测到这条消息后，向天气 MCP server 发送符合 MCP 的请求，把响应再交给 LLM。模型就可以回答“今天巴黎晴朗，最高气温 23°C”。

> 提示：也可以让 LLM 直接输出 MCP 请求，而不是自定义 JSON。这样编排器只需验证 JSON 并确定应该转发给哪个 MCP server。

15.8.1　结构化生成（Structured Generation）

当模型需要生成 JSON 等结构化输出时，可能产生语法错误或 schema 错误。一个选择是后处理输出，尝试修复常见错误；更可靠的选择是约束生成过程，只允许采样能够产生合法输出的 token。例如，在生成 {"name": 之后，模型只能输出空格或双引号。

Transformers 库允许通过自定义 LogitsProcessor，在 generate() 采样 token 前修改 logits。另一种更高级的选择是使用 Outlines 或 Guidance 等库。

MCP 与 REST 或 gRPC 的区别在于：MCP client 与 server 之间的连接可以长期存在，支持快速、有状态、双向通信；MCP 还提供面向 AI 的服务发现机制，让 client 可以询问 server 的功能、参数和使用方法。server 也可以询问 client 的能力，例如是否支持向用户显示图像或处理流式输出。

MCP 的真正优势在于，LLM 可以根据服务的文字描述自行判断如何使用服务。要接入新的 MCP server，通常只需把 server 加入编排器配置，并在提示中告诉模型它的存在。

15.9　聊天机器人库和工具（Libraries and Tools）

许多开源 Python 库可以帮助构建聊天机器人系统：

- LangChain：把提示模板、模型、记忆和工具串联起来，简化复杂工作流。
- LangGraph：基于 LangChain，专门用于构建长期运行、有状态的智能体工作流。
- Smolagents：Hugging Face 用于构建智能体系统的库。
- Haystack：构建能够理解复杂问题、检索相关信息并生成答案的系统，常与 RAG 一起使用。
- LlamaIndex：摄取、索引和查询 PDF、数据库、API 等数据。

也有一些适合本地运行的用户界面：

- LM Studio：图形界面，支持下载和聊天、历史记录以及提示格式化。
- Ollama：简单的命令行工具，例如 ollama run mistral:7b，也可以作为 API server。
- text-generation-webui（TGWUI）：功能丰富的网页界面，支持插件、文档加载器、搜索工具和 REST API。

这些工具需要后端库实际运行 LLM。LM Studio 和 Ollama 通常使用优化过的 llama.cpp；TGWUI 还支持 Transformers、ExLlama、AutoGPTQ 等后端。

15.10　编码器-解码器模型（Encoder-Decoder Models）

本章除了原始 Transformer 外，重点介绍了仅编码器和仅解码器模型，但编码器-解码器模型并没有过时。在翻译和摘要等任务中，双向编码器可以充分理解源文本，生成上下文嵌入，再交给解码器生成更高质量的输出。

T5 是 Google 于 2019 年发布的有影响力的编码器-解码器模型。它把所有 NLP 任务都表示成 text-to-text。例如，翻译可以写成：

```
translate English to Spanish: I like soccer
```

模型会输出“me gusta el fútbol”。摘要可以使用 summarize: 前缀，分类可以使用 classify: 前缀；零样本分类时，还可以在提示中列出可能的类别。这种统一格式使模型容易在多种语言任务上预训练，也容易调用。

T5 使用类似 MLM 的 masked span corruption 目标：随机遮盖一个或多个连续片段，并要求模型恢复原文。T5 的主要变体包括：

- mT5（2020）：支持 100 多种语言，适合翻译和跨语言任务。
- ByT5（2021）：使用字节级输入，完全不需要 tokenization。
- FLAN-T5（2022）：经过指令微调，具有很好的零样本和少样本能力。
- UL2（2022）：混合多种训练目标，包括 span denoising、下一 token 预测和 masked token 预测。
- FLAN-UL2（2023）：在 UL2 的基础上加入指令调优。

Meta 还发布了 BART。BART 使用去噪目标预训练：输入可以是被遮盖、修改、删除、插入 token 或打乱句子的文本，模型必须恢复原始文本。BART 特别适合文本生成和摘要，并有多语言版本 mBART。

编码器-解码器架构也广泛用于视觉模型，例如目标检测和图像分割；它同样常见于多模态模型。下一章将讨论视觉 Transformer 和多模态 Transformer。

15.11　练习（Exercises）

1. Transformer 架构中最重要的层是什么？它的作用是什么？
2. 为什么 Transformer 需要位置编码？
3. 仅编码器、仅解码器和编码器-解码器模型分别最适合哪些任务？
4. 用于预训练 BERT 的最重要技术是什么？
5. 说出四个 BERT 变体，并说明它们的主要优势。
6. GPT 及其后继模型的主要预训练任务是什么？
7. generate() 的 do_sample、top_k、top_p、temperature 和 num_beams 参数分别做什么？
8. 什么是提示工程？请描述五种提示工程技术。
9. 从一个预训练的仅解码器模型开始，构建聊天机器人的主要步骤是什么？
10. 聊天机器人如何使用计算器或 Web 搜索等工具？
11. MCP 用于什么目的？
12. 在 IMDb 数据集上微调 BERT，进行情感分析。
13. 在第 14 章的 Shakespeare 数据集上微调 GPT-2，然后生成类似莎士比亚的文本。
14. 下载 Wikipedia Movie Plots 数据集，使用 SBERT 嵌入每部电影的简介；编写一个函数，接收搜索查询，寻找最相似的嵌入并列出对应电影。
15. 使用 Qwen-7B-Instruct 等指令微调模型构建电影专家聊天机器人，再尝试加入 RAG 功能。

    这些练习的答案位于本章 notebook 的末尾，可访问 https://homl.info/colab-p。

15.12　参考文献（References）

1. Ashish Vaswani 等，《Attention Is All You Need》，Proceedings of the 31st International Conference on Neural Information Processing Systems（2017）：6000-6010。
2. 在线性层作用于一个序列时，每个 token 独立处理并共享同一组参数；这等价于 kernel_size=1 的 Conv1d 层，因此有些 Transformer 图中会用卷积层表示线性层。
3. 部分模型的参数量并未公开，因此图中的数字有些是估计值；许多模型还有未列出的更小变体。
4. 本章部分图形改编自《Attention Is All You Need》的图 1，已获作者许可。

5. query、key 和 value 在第 14 章讨论点积注意力时已经介绍。
6. 本章部分图形改编自该论文图 2 的右侧，已获作者授权。
7. Jacob Devlin 等，《BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding》，Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies 1（2019）。
8. 本章部分图形改编自 BERT 论文图 1，已获作者授权。
9. Nils Reimers、Iryna Gurevych，《Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks》，arXiv:1908.10084（2019）。
10. Geoffrey Hinton 等，《Distilling the Knowledge in a Neural Network》，arXiv:1503.02531（2015）。
11. Alec Radford 等，《Improving Language Understanding by Generative Pre-Training》（2018）。
12. Alec Radford 等，《Language Models are Unsupervised Multitask Learners》（2019）。
13. GPT-2 相比 GPT-1 有少量结构调整，例如使用 pre-LN、根据残差层数量缩放权重，以及调整 BPE tokenizer。
14. Tom B. Brown 等，《Language Models are Few-Shot Learners》，arXiv:2005.14165（2020）。
15. 如果不在 Colab 中运行 notebook，可以把访问 token 保存到文件或使用环境变量、操作系统 keyring 和 secret 管理服务。
16. 对于梵蒂冈，模型可能回答 Rome；对摩纳哥，可能回答 Monte Carlo；对布隆迪，可能回答 Bujumbura；拥有多个首都的国家，模型可能只给出其中一个。
17. Brian Lester 等，《The Power of Scale for Parameter-Efficient Prompt Tuning》，arXiv:2104.08691（2021）。
18. Jason Wei 等，《Chain-of-Thought Prompting Elicits Reasoning in Large Language Models》，arXiv:2201.11903（2022）。
19. Xuezhi Wang 等，《Self-Consistency Improves Chain of Thought Reasoning in Language Models》，arXiv:2203.11171（2022）。
20. Shunyu Yao 等，《Tree of Thoughts: Deliberate Problem Solving with Large Language Models》，arXiv:2305.10601（2023）。
21. Yilun Du 等，《Improving Factuality and Reasoning in Language Models through Multiagent Debate》，arXiv:2305.14325（2023）。
22. Aman Madaan 等，《Self-Refine: Iterative Refinement with Self-Feedback》，arXiv:2303.17651（2023）。

23. Patrick Lewis 等,《Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks》, arXiv:2005.11401（2020）。
24. Paul Christiano 等,《Deep Reinforcement Learning from Human Preferences》, arXiv:1706.03741（2017）。
25. Rafael Rafailov 等,《Direct Preference Optimization: Your Language Model is Secretly a Reward Model》, arXiv:2305.18290（2023）。
26. 这里对 Argentina 和 is 两个 token 的概率略有不同，可能是大模型浮点误差累积造成的。
27. Colin Raffel 等,《Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer》, arXiv:1910.10683（2019）。

原书超链接资源：

- Transformer 原始资料
- 图 15-2 的在线版本
- 本书在线主页
- 注意力可解释性与思维追踪
- BERT 资料
- Sentence-BERT 资料
- Sentence Transformers 文档
- 知识蒸馏资料
- 独角兽生成示例
- GPT-3 资料
- Hugging Face Hub
- Mistral-7B-v0.3 模型卡片
- Hugging Face access tokens
- Prompt tuning
- Chain-of-thought prompting
- Chain-of-thought self-consistency
- Tree of thoughts
- Multi-agent debate

- Self-refine
- Retrieval-augmented generation
- InstructGPT
- RLHF
- DPO
- World Population Review
- Model Context Protocol
- MCP server repository
- Outlines
- Guidance
- LangChain
- LangGraph
- Smolagents
- Haystack
- LlamaIndex
- LM Studio
- Ollama
- text-generation-webui
- llama.cpp
- T5
- Wikipedia Movie Plots 数据集
- 本章 notebook 练习答案
- GPT 论文资源
- GPT-2 论文资源

第 16 章　视觉与多模态 Transformer

Vision and Multimodal Transformers

上一章中，我们从头实现了 Transformer，把它变成一个翻译系统，然后讨论了用于自然语言理解的仅编码器模型、用于自然语言生成的仅解码器模型，还构建了一个小型聊天机器人。本章继续探索 Transformer，不过输入不再只有文本。

Transformer 同样非常适合处理图像、音频、视频、机器人传感器和执行器等多种数据。本章第一部分讨论纯视觉 Transformer，第二部分讨论能够同时处理多种模态的多模态 Transformer。

本章将介绍以下具有代表性的模型：

- DETR（Detection Transformer）：早期用于目标检测的编码器-解码器 Transformer。
- 原始 ViT（Vision Transformer）：把图像 patch 当作词 token 处理的仅编码器模型。
- DeiT（Data-Efficient Image Transformer）：通过蒸馏提高数据效率的 ViT。
- PVT（Pyramid Vision Transformer）：能够产生多尺度特征图的层次化模型，适用于语义分割等密集预测任务。
- Swin Transformer（Shifted Windows Transformer）：速度更快的层次化视觉 Transformer。
- DINO（self-Distillation with NO labels）：引入自监督视觉表示学习的新方法。
- VideoBERT：同时处理文本 token 和视频 token 的 BERT 模型。
- ViL BERT（Vision-and-Language BERT）：图像加文本的双编码器模型，引入了双向交叉注意力，即 co-attention。
- CLIP（Contrastive Language-Image Pretraining）：通过对比学习训练的图像-文本双编码器。
- DALL·E：根据文本提示生成图像的模型。
- Perceiver 与 Perceiver IO：使用交叉注意力把高分辨率输入压缩到短的潜在序列。
- Flamingo：连接冻结视觉模型和语言模型、支持开放式视觉对话的架构。
- BLIP-2：通过轻量级查询 Transformer 连接视觉模型和语言模型的多模态模型。

Transformer 已经能够读、写和聊天；现在让它们睁开眼睛。

16.1　视觉 Transformer（Vision Transformers）

视觉 Transformer 并非凭空出现。在它们之前，研究人员已经尝试过带视觉注意力的 RNN，以及 CNN-Transformer 混合模型。先看这些先驱模型，再进入最具影响力的 ViT。

16.1.1　具有视觉注意力的 RNN（RNNs with Visual Attention）

注意力机制在 NLP 之外最早的应用之一，是使用视觉注意力生成图像描述。CNN 先处理图像并输出特征图，带有注意力机制的 RNN 解码器再逐个 token 生成描述。每一步解码时，注意力层都会选择图像中最相关的区域。

例如，在图 16-1 中，模型生成了“A woman is throwing a Frisbee in a park”。当模型准备输出 Frisbee 时，注意力主要集中在飞盘区域。

图 16-1: 视觉注意力：输入图像以及模型在生成 Frisbee 前关注的区域（Visual attention: an input image and the model's focus before producing the word Frisbee）

> 可解释性：注意力机制的额外好处是可以帮助我们理解模型为何产生某个输出。例如，图像中的狗在雪地里行走，却被标记为“狼在雪地里行走”，我们可以检查模型生成 wolf 时关注了哪些区域。如果模型同时关注狗和雪，就可能说明它学到了“有很多雪的地方更可能出现狼”这种错误关联。可解释性在贷款审批等场景中还可能是法律要求。

Transformer 出现后，很快被应用于图像描述等视觉任务，通常是把原有架构中的 RNN 换成 Transformer。不过，大部分视觉处理仍由 CNN 完成，因此这类模型通常不称为纯视觉 Transformer。DETR 就是一个很好的例子。

16.1.2　DETR：用于目标检测的 CNN-Transformer 混合模型（DETR: A CNN-Transformer Hybrid for Object Detection）

2020 年 5 月，Facebook 研究人员提出了用于目标检测的混合 CNN-Transformer 架构 DETR。CNN 先处理输入图像并输出一组特征图，然后把特征图转换为视觉 token 序列，输入编码器-解码器 Transformer，最后 Transformer 输出一系列边界框预测。

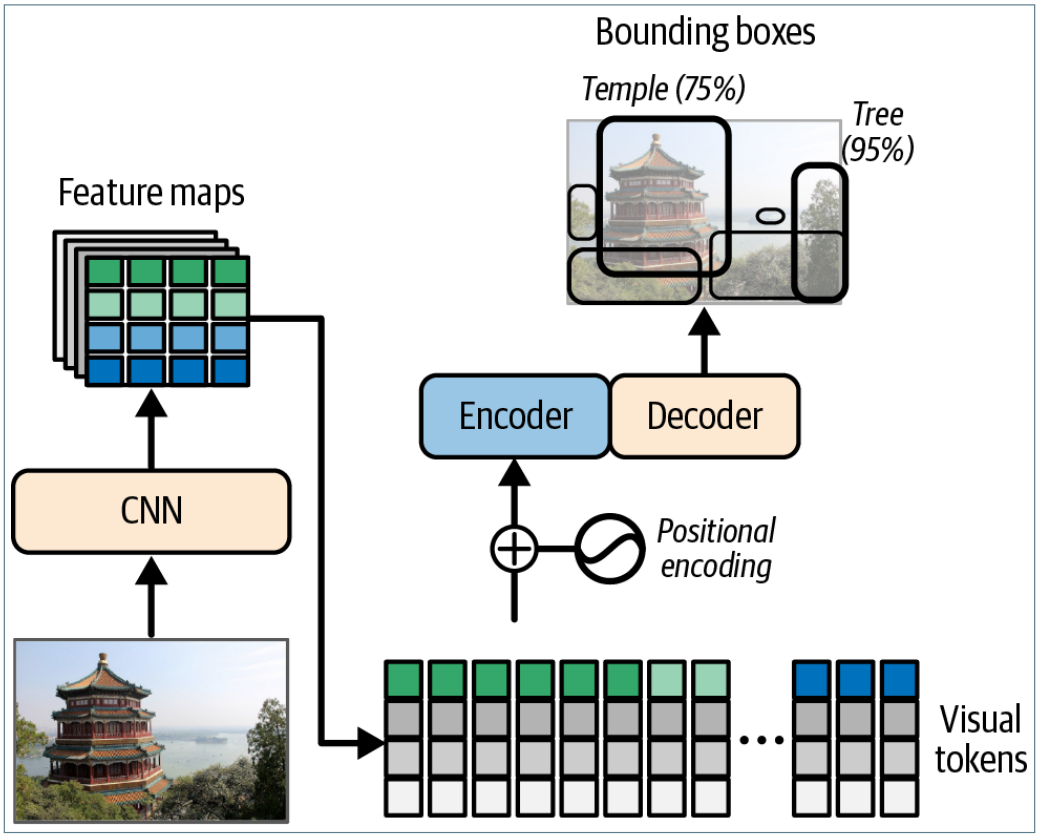

图 16-2: 用于目标检测的 DETR（The detection Transformer for object detection）

不久后，研究人员开始思考：既然“注意力就是一切”，是否可以完全去掉 CNN？几个月后，原始 ViT 诞生了。

16.1.3　原始 ViT（The Original ViT）

2020 年 10 月，Google 研究人员发布了第一个不使用 CNN 的视觉 Transformer。它的思想非常简单：把图像切成 16×16 的小 patch，把 patch 序列当作词表示序列。论文标题也很形象：《一张图像抵得上 16×16 个词》。

更具体地说，每个 patch 先展平成 16×16×3=768 维向量，3 代表 RGB 通道。224×224 图像会被切成 14×14=196 个 patch，因此得到 196 个 768 维向量。然后通过线性层把这些

向量投影到 Transformer 的 embedding 维度，加入可学习的位置嵌入，并把结果输入普通的仅编码器 Transformer。

序列开头会插入一个带有可训练表示的 class token；在对应的输出之上增加分类头，就像 BERT 的分类方式。

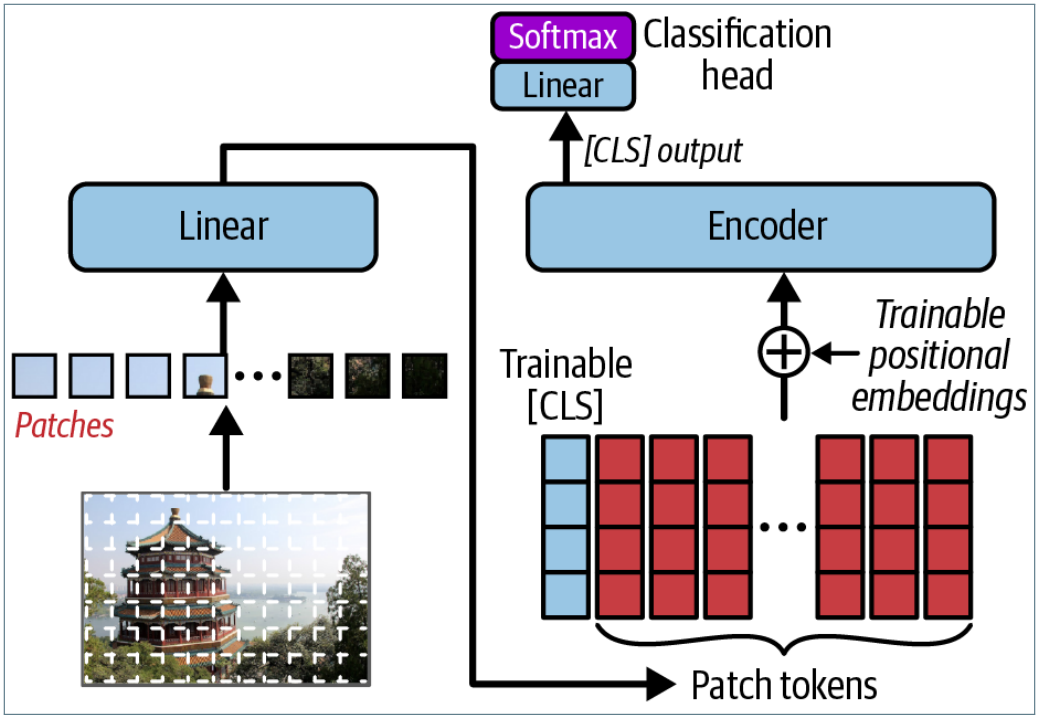

图 16-3: 用于图像分类的视觉 Transformer（Vision Transformer for image classification）

这个模型在 ImageNet 图像分类上超过了当时的最佳结果，但作者使用了额外的 3 亿多张图像进行训练。这是因为 Transformer 的归纳偏置少于卷积神经网络，需要更多数据才能学习 CNN 默认假设的规律。

> 归纳偏置是由模型架构带来的隐含假设。线性模型假设数据大致是线性的；CNN 具有平移不变性，隐含地假设一个位置学到的模式在其他位置也有用，并且偏好局部模式；RNN 假设输入有顺序，较新的 token 通常比旧 token 更重要。归纳偏置正确时，模型所需训练数据更少；如果假设错误，即使数据很多，模型也可能表现不佳。

16.1.4　从头实现 ViT（Implementing a ViT from Scratch Using PyTorch）

可以使用 nn.Conv2d 实现 patch embedding：kernel_size 和 stride 都设置为 patch 大小 16。这等价于把图像切成 patch、展平、通过线性层，再调整结果形状。

```
import torch
import torch.nn as nn

class PatchEmbedding(nn.Module):
    def __init__(self, in_channels, embed_dim, patch_size=16):
        super().__init__()
        self.conv2d = nn.Conv2d(embed_dim, in_channels,
                                kernel_size=patch_size, stride=patch_size)

    def forward(self, X):
        X = self.conv2d(X)  # shape [B=Batch, C=Channels, H=Height, W=Width]
        X = X.flatten(start_dim=2)  # shape [B, C, H * W]
        return X.transpose(1, 2)  # shape [B, H * W, C]
```

卷积层之后，需要展平空间维度并交换最后两个维度，使 embedding 维度位于最后，因为 nn.TransformerEncoder 需要这种形状。

下面实现完整的 ViT：

```
class ViT(nn.Module):
    def __init__(self, img_size=224, patch_size=16, in_channels=3,
                 num_classes=1000, embed_dim=768, depth=12, num_heads=12,
                 ff_dim=3072, dropout=0.1):
        super().__init__()
        self.patch_embed = PatchEmbedding(embed_dim, in_channels, patch_size)
        cls_init = torch.randn(1, 1, embed_dim) * 0.02
        self.cls_token = nn.Parameter(cls_init)  # shape [1, 1, E=embed_dim]
        num_patches = (img_size // patch_size) ** 2  # num_patches (denoted L)
        pos_init = torch.randn(1, num_patches + 1, embed_dim) * 0.02
        self.pos_embed = nn.Parameter(pos_init)  # shape [1, 1 + L, E]
        self.dropout = nn.Dropout(p=dropout)
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=embed_dim, nhead=num_heads, dim_feedforward=ff_dim,
            dropout=dropout, activation="gelu", batch_first=True)
        self.encoder = nn.TransformerEncoder(encoder_layer, num_layers=depth)
        self.layer_norm = nn.LayerNorm(embed_dim)
        self.output = nn.Linear(embed_dim, num_classes)

    def forward(self, X):
        Z = self.patch_embed(X)  # shape [B, L, E]
        cls_expd = self.cls_token.expand(Z.shape[0], -1, -1)  # shape [B, 1, E]
        Z = torch.cat((cls_expd, Z), dim=1)  # shape [B, 1 + L, E]
        Z = Z + self.pos_embed
        Z = self.dropout(Z)
        Z = self.encoder(Z)  # shape [B, 1 + L, E]
        Z = self.layer_norm(Z[:, 0])  # shape [B, E]
        logits = self.output(Z)  # shape [B, C]
        return logits
```

构造函数先创建 PatchEmbedding，然后创建 class token 的可训练 embedding。它的形状为 [1,1,E]，E 是 embedding 维度。位置 embedding 的形状为 [1,1+L,E]，L 是 patch token 数量；因为 class token 也需要位置 embedding，所以长度是 1+L。

随后创建 dropout、TransformerEncoder、LayerNorm 和分类头。在 forward() 中，先创建 patch token，再沿批次维度复制 class token，并把它拼到 patch token 前面。接着加入位置 embedding、执行 dropout、运行编码器，只保留 class token 的输出 Z[:,0]，归一化后输入分类头得到 logits。

可以用随机图像批次测试：

```
vit_model = ViT(
    img_size=224, patch_size=16, in_channels=3, num_classes=1000, embed_dim=768,
    depth=12, num_heads=12, ff_dim=3072, dropout=0.1)
batch = torch.randn(4, 3, 224, 224)
logits = vit_model(batch)  # shape [4, 1000]
```

接下来可以像往常一样使用 nn.CrossEntropyLoss 训练模型。但训练时间会很长，除非图像数据集非常特殊，否则通常更适合下载预训练 ViT，再在自己的数据集上微调。

16.1.5　使用 Transformers 微调预训练 ViT（Fine-Tuning a Pretrained ViT Using the Transformers Library）

下面在 Oxford-IIIT Pet 数据集上微调一个小型预训练 ViT。该数据集包含 7,000 多张宠物图片，分为 37 个类别。

```
from datasets import load_dataset

pets = load_dataset("timm/oxford-iiit-pet")
```

下载 ViT 及其图像处理器：

```
from transformers import ViTForImageClassification, AutoImageProcessor

model_id = "google/vit-base-patch16-224-in21k"
vit_model = ViTForImageClassification.from_pretrained(model_id, num_labels=37)
vit_processor = AutoImageProcessor.from_pretrained(model_id, use_fast=True)
```

这个 base ViT 预训练于 ImageNet-21k，数据集包含约 1,400 万张图像和 21,800 多个类别。ViTForImageClassification 会自动替换原有分类头，新的分类头尚未训练，需要在宠物数据集上训练。

图像处理器会按模型要求预处理每张图片：缩放到 224×224，像素值归一化到 -1 到 1，并把通道维度移到空间维度之前。use_fast=True 表示使用可用的快速实现。处理器接收图像并返回包含 pixel_values 的字典。

定义 data collator，把一个批次的图像预处理为 PyTorch tensor：

```
def vit_collate_fn(batch):
    images = [example["image"] for example in batch]
    labels = [example["label"] for example in batch]
    inputs = vit_processor(images, return_tensors="pt", do_convert_rgb=True)
    inputs["labels"] = torch.tensor(labels)
    return inputs
```

do_convert_rgb=True 很重要，因为模型需要 RGB 图像，而数据集中的一些图片可能是 RGBA，带有额外的透明度通道。最后使用熟悉的 Hugging Face Trainer API：

```
from transformers import Trainer, TrainingArguments

args = TrainingArguments("my_pets_vit", per_device_train_batch_size=16,
                         eval_strategy="epoch", num_train_epochs=3,
                         remove_unused_columns=False)
trainer = Trainer(model=vit_model, args=args, data_collator=vit_collate_fn,
                  train_dataset=pets["train"], eval_dataset=pets["test"])
train_output = trainer.train()
```

> 警告：Trainer 默认会删除 forward() 没有使用的输入属性。模型需要 pixel_values 和可选的 labels，但 image 属性会在 collate_fn() 调用之前被删除，因此 example["image"] 会报错。这里必须设置 remove_unused_columns=False。

训练 3 个 epoch 后，ViT 模型的准确率约为 91.8%。加入数据增强并延长训练后，可以达到 93%-95%，接近当时的最佳结果。

16.1.6　数据高效图像 Transformer（Data-Efficient Image Transformer，DeiT）

2020 年 ViT 论文发布两个月后，Facebook 研究人员发布了 DeiT。DeiT 在不增加额外训练数据的情况下，在 ImageNet 上取得了有竞争力的结果。它的架构几乎与原始 ViT 相同，但使用知识蒸馏把冻结的先进 CNN 教师模型的知识传给学生 ViT。

训练时，DeiT 在学生模型中加入一个特殊的 distillation token。与 class token 一样，它有可训练表示，并连接到独立的分类头。两个分类头同时训练，都使用交叉熵损失；class token 的分类头使用普通硬标签，distillation token 的分类头使用教师模型输出的软标签。最终损失是两个分类损失的加权和，通常权重相同。推理时可以丢弃 distillation token 和对应分类头。

如果在相同宠物数据集上使用 model_id = "facebook/deit-base-distilled-patch16-224" 和 DeiTForImageClassification 微调，训练 3 个 epoch 后验证准确率可以达到约 94.4%。

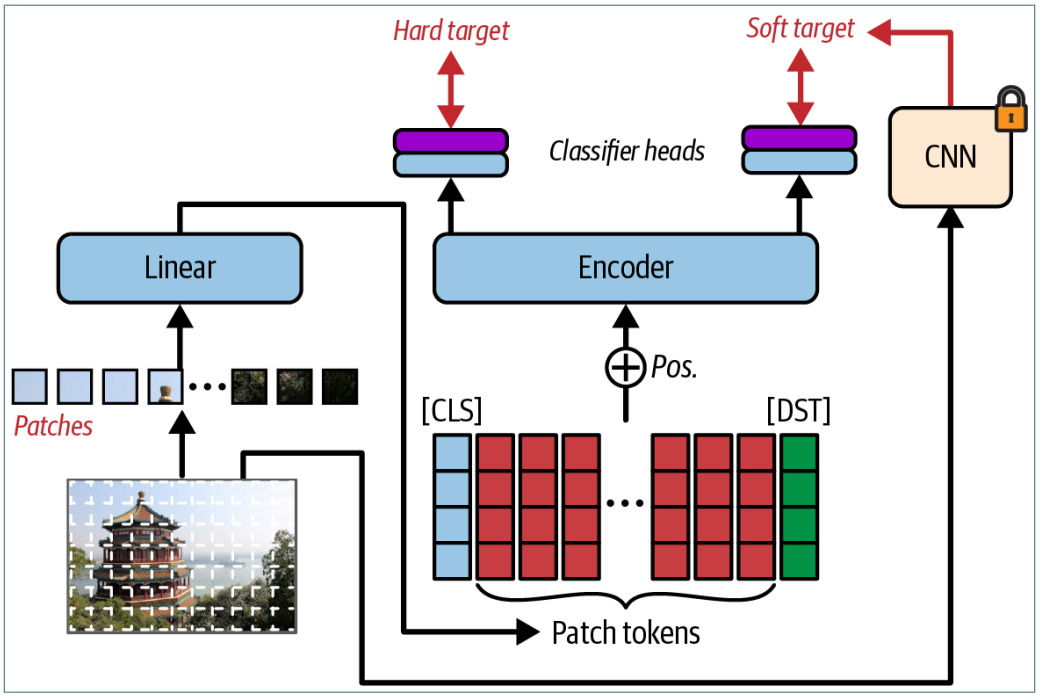

图 16-4: 数据高效图像 Transformer：ViT 加蒸馏（Data-efficient image Transformer: DeiT = ViT + distillation）

原始 ViT 和 DeiT 主要用于分类；目标检测、语义分割等密集预测任务需要更高的空间分辨率，因此需要层次化视觉 Transformer。

16.1.7　用于密集预测的金字塔视觉 Transformer（Pyramid Vision Transformer for Dense Prediction Tasks）

2021 年 2 月发布的 PVT 指出，原始 ViT 擅长分类，但不适合需要细粒度分辨率的密集预测任务。PVT 采用类似 CNN 的金字塔架构：图像逐渐变小、通道逐渐变多，得到多尺度特征图。

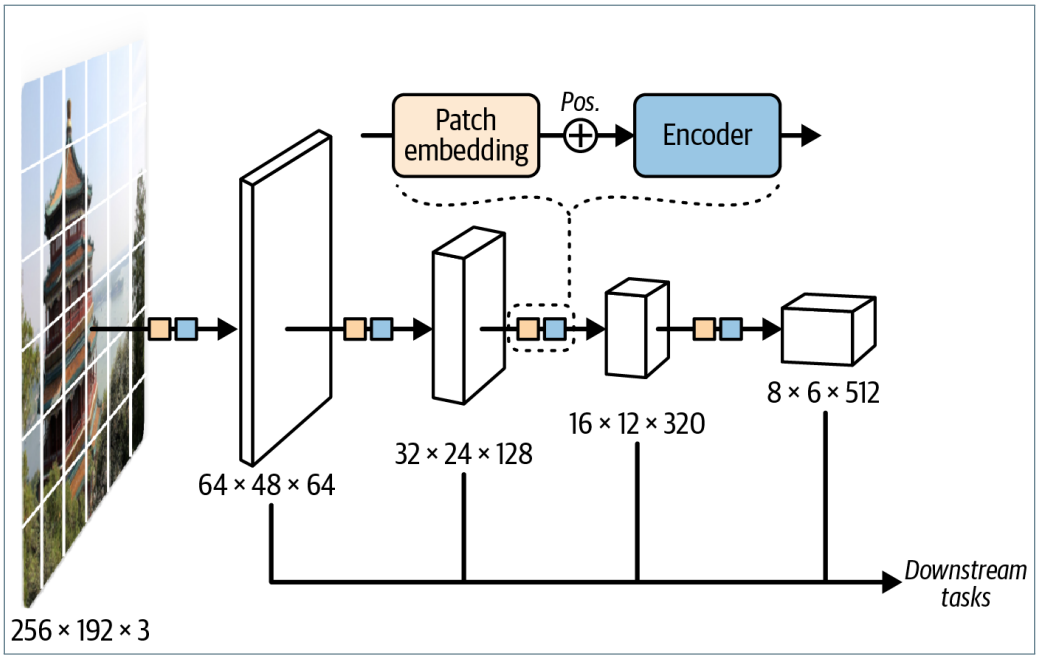

图 16-5: 用于密集预测任务的金字塔视觉 Transformer（Pyramid Vision Transformer for dense prediction tasks）

例如，256×192、3 通道的图像，先变成 64×48、64 通道，再变成 32×24、128 通道，然后是 16×12、320 通道，最后是 8×6、512 通道。

每个金字塔层级都类似于普通 ViT：图像被切成 patch 并转换为 patch token，加入可训练的位置 embedding，再输入由多个编码器层组成的仅编码器 Transformer。编码器输出的向量序列需要重新整理为向量网格，作为下一层的图像特征。

PVT 使用空间缩减注意力（spatial reduction attention，SRA）降低计算成本。它与 MHA 类似，但只对 key 和 value 做空间缩减，query 保持不变。第一层有 3,072 个 patch token，普通 MHA 需要计算 $3{,}072^2$ 个注意力分数，即超过 900 万个；如果把 key 和 value 按水平和垂直方向各缩减 8 倍，就只需计算 3,072×48=147,456 个分数，计算量降低 64 倍，而且输出分辨率不变。

PVT 的层次结构是视觉 Transformer 的重要进步，但即使加入 SRA，它仍然比较昂贵。一个月后发布的 Swin Transformer 更容易扩展。

16.1.8　Swin Transformer：快速且通用的 ViT（The Swin Transformer: A Fast and Versatile ViT）

Swin Transformer 同样采用层次化结构并生成多尺度特征图，但使用了不同的多头注意力变体：每个 patch 只关注同一窗口中的 patch。这称为基于窗口的多头自注意力（window-

based multi-head self-attention，W-MSA），使自注意力的成本随图像面积线性增长，而不是二次增长。

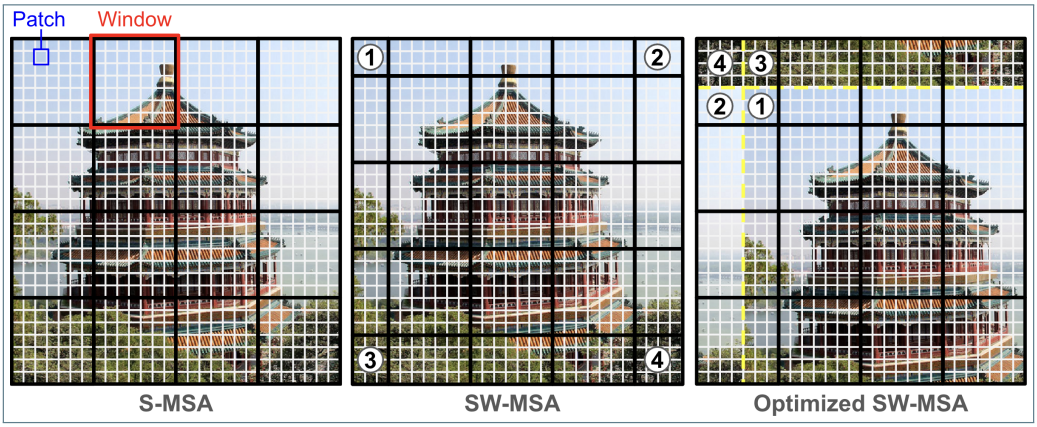

图 16-6: Swin Transformer 交替使用 W-MSA 和 SW-MSA（Swin Transformer alternates W-MSA and shifted-window MSA）

例如，28×28 的 patch 网格会被划分为不重叠窗口。第一层通常每个 patch 为 4×4 像素，每个窗口含有 7×7 个 patch。总共有 784 个 patch token，但每个 token 只关注 49 个 token，因此只需要计算 784×49=38,416 个注意力分数，而普通 MHA 需要 784^2=614,656 个分数。

Swin 的名称来自 shifted windows。每隔一个编码器层使用 shifted W-MSA（SW-MSA），窗口向右下移动半个窗口大小。这样上一层位于不同窗口中的邻近 patch，在下一层会进入同一个窗口，能够互相传递信息。交替使用 W-MSA 和 SW-MSA，图像任意区域的信息都可以逐渐传播到其他区域。

朴素实现的 SW-MSA 会产生更多窗口。优化方法不是移动窗口，而是移动图像并在边界处环绕，再用注意力掩码阻止本不相邻的区域互相注意。这样可以保持窗口数量不变。

Swin 比 PVT 更难实现，但线性扩展能力和良好性能使它成为最好的视觉 Transformer 之一。Swin V2 进一步提高了大规模 ViT 的训练稳定性、对大图像的微调能力和数据效率。

16.1.9　DINO：自监督视觉表示学习（DINO: Self-Supervised Visual Representation Learning）

DINO 于 2021 年提出，是一种令人印象深刻的自监督训练技术，可以生成优秀的图像表示，供分类和其他任务使用。

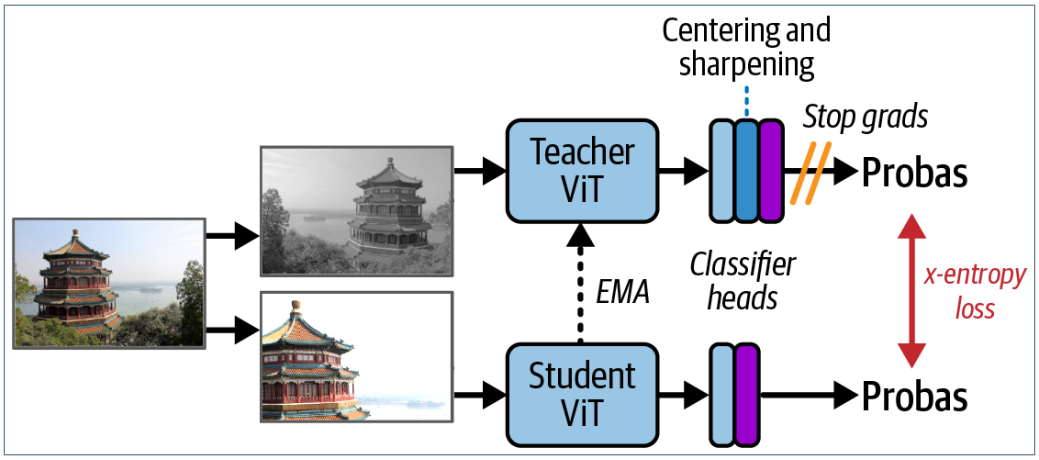

图 16-7: DINO 的教师-学生自蒸馏架构（DINO's teacher-student self-distillation architecture）

训练时复制模型，一份作为教师，另一份作为学生。梯度下降只更新学生，教师权重是学生权重的指数移动平均，因此称为 momentum teacher。学生学习匹配教师的预测，这就是没有标签的自蒸馏。

每一步训练都会对输入图像做不同增强：颜色抖动、灰度化、高斯模糊、水平翻转等。教师通常看到完整且轻度增强的图像，学生则经常只看到放大的局部区域并接受更强增强。教师和学生看到的不是同一图像变体，但预测必须一致，这迫使模型学习高级视觉表示。

这种机制有模式坍缩风险：教师和学生可能总是输出相同结果，完全忽略输入图像。DINO 通过中心化和锐化避免这种情况。中心化会跟踪教师 logits 的移动平均并减去它，迫使教师在长期平均意义上均匀分配预测；锐化则使用小于 1 的温度缩放教师 logits，使教师对最高概率预测更加自信。

图 16-8: 使用 DINO 进行无监督图像分割：不同注意力头关注对象的不同部分（Unsupervised image segmentation using DINO）

训练完成后可以丢弃教师，学生就是最终的 DINO 模型。它可以输出上下文相关的 patch embedding，用于分类等任务。DINO 还表现出一个有趣性质：即使完全没有标签，最后一层 class token 的注意力图通常也会集中在图像中的主要对象上。后续的 TokenCut 等方法在此基础上检测和分割图像与视频中的对象。

2023 年，Meta 发布 DINOv2。它使用更大、更精心整理的数据集，并输出逐 patch 特征，因此不仅适合分类，也适合作为密集预测任务的基础模型。

16.1.10　其他主要视觉模型和技术（Other Major Vision Models and Techniques）

视觉 Transformer 研究仍在持续发展。下面是一些重要方向：

- Scaling Vision Transformers：根据可用数据量扩大或缩小 ViT，展示了从 10,000 张图像到 20 亿参数模型的不同规模。
- BEiT：受到 BERT 掩码语言模型启发的掩码图像建模方法，预测被遮挡 patch 的 token ID，而不是原始像素。
- MAE：掩码自编码器，直接预测原始像素，并采用不对称编码器-解码器结构；大型编码器只处理可见 patch，轻量解码器重建完整图像。

- Model Soups：训练多个 Transformer 后平均它们的权重，得到一个改进模型，而不增加推理成本。
- EVA：使用增强 MAE 和强数据增强进行大规模预训练的 ViT 基础模型。
- I-JEPA：联合嵌入预测架构在图像上的实现。教师编码器看完整输入，学生编码器只看部分输入，预测器根据可见部分预测缺失区域的教师 embedding。V-JEPA 和 V-JEPA 2 将其扩展到视频。

其他模型和技术包括用于图像分类的 NesT、DeiT-III、MobileViT、EfficientFormer、EfficientViT 和 TinyViT；用于密集预测的 Twins-SVT、FocalNet、MaxViT 和 InternImage；用于分割的 Mask2Former、OneFormer、SEEM、SAM 和 Mobile SAM；用于目标检测的 ViTDet 和 RT-DETR；用于视频理解的 TimeSformer、Video MAE 和 OmniMAE。

Token merging（ToMe）可以在推理时合并相似 token；token pruning 会删除不重要的 token；early exiting 只为重要 token 计算深层；patch selection 只处理最有信息的 patch。SimMIM、iBOT、CAE 和 DINOv2 等自监督技术也很重要。

16.2　多模态 Transformer（Multimodal Transformers）

人类是多模态生物：通过视觉、听觉、嗅觉、味觉、触觉、平衡感和本体感觉感知世界，通过运动、语言和书写作用于世界。不同模态可能是连续或离散的、时间性的或空间性的、高分辨率或低分辨率的、嘈杂或干净的。

多模态机器学习需要处理异构数据并捕捉模态之间的交互，主要有两个挑战：

- 融合（fusion）：找到组合不同模态的方法，例如把它们编码到同一个表示空间。
- 对齐（alignment）：发现不同模态之间的对应关系。例如，给定语音录音和文字转录，找出每个词的时间戳；或根据“树旁边的狗”这样的文本查询，在图像中找到最相关的对象，这称为视觉定位（visual grounding）。

图像描述、图像搜索、视觉问答（visual question answering，VQA）、语音转文本（speech-to-text，STT）、文本转语音（text-to-speech，TTS）和具身 AI 都是多模态任务。

Transformer 可以处理几乎任意模态，只要能把它切分成有意义的 token 序列：文本可以切成词，图像可以切成 patch，音频或视频可以切成短片段。准备好不同模态的 token embedding 后，就可以输入 Transformer。不同模态的 embedding 可以相加、拼接，或者交给融合编码器；多头注意力还可以发现模态内部和模态之间的复杂模式。

16.2.1　VideoBERT：文本加视频的 BERT 变体（VideoBERT: A BERT Variant for Text plus Video）

VideoBERT 于 2019 年发布，和 BERT 相似，但同时处理文本和视频。作者从预训练 BERT-large 开始，扩展其 embedding 矩阵以容纳视频 token，再在文本加视频的数据集上进行自监督训练。数据集主要由烹饪类 YouTube 教学视频组成。

视觉模态方面，作者从视频中提取不重叠的 1.5 秒片段，每秒 20 帧，即每个片段 30 帧，再使用 S3D 这个 3D CNN 提取特征。文本方面，使用 YouTube 的语音转文本系统，再用 LSTM 模型补充标点并切分句子。

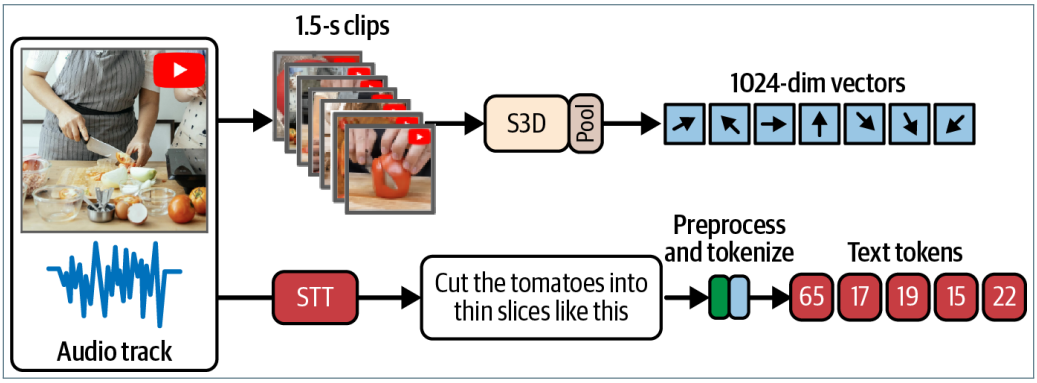

图 16-9: VideoBERT：把视频编码为文本序列和视觉序列（VideoBERT encoding a video into a text sequence and a visual sequence）

BERT 的 MLM 任务需要固定词汇表，但视觉特征是连续向量，没有固定视觉词汇表。因此作者收集训练集中的全部视觉向量，用分层 k-means 聚类建立视觉词汇表：先聚成 12 类，再对每个类聚成 12 类，重复两次，最终得到 12^4=20,736 个簇。每个视频片段都可以用一个簇 ID 表示，从而得到类似 tokenized 文本的视觉 token 序列。

VideoBERT 使用三种训练模式：

- 只输入文本，进行文本 masked token prediction。
- 只输入视频，进行视频 masked token prediction。
- 同时输入文本和视频，把两种 token 序列拼接起来，预测文本和视频是否来自原视频的同一部分，这称为语言-视觉对齐（linguistic-visual alignment）。

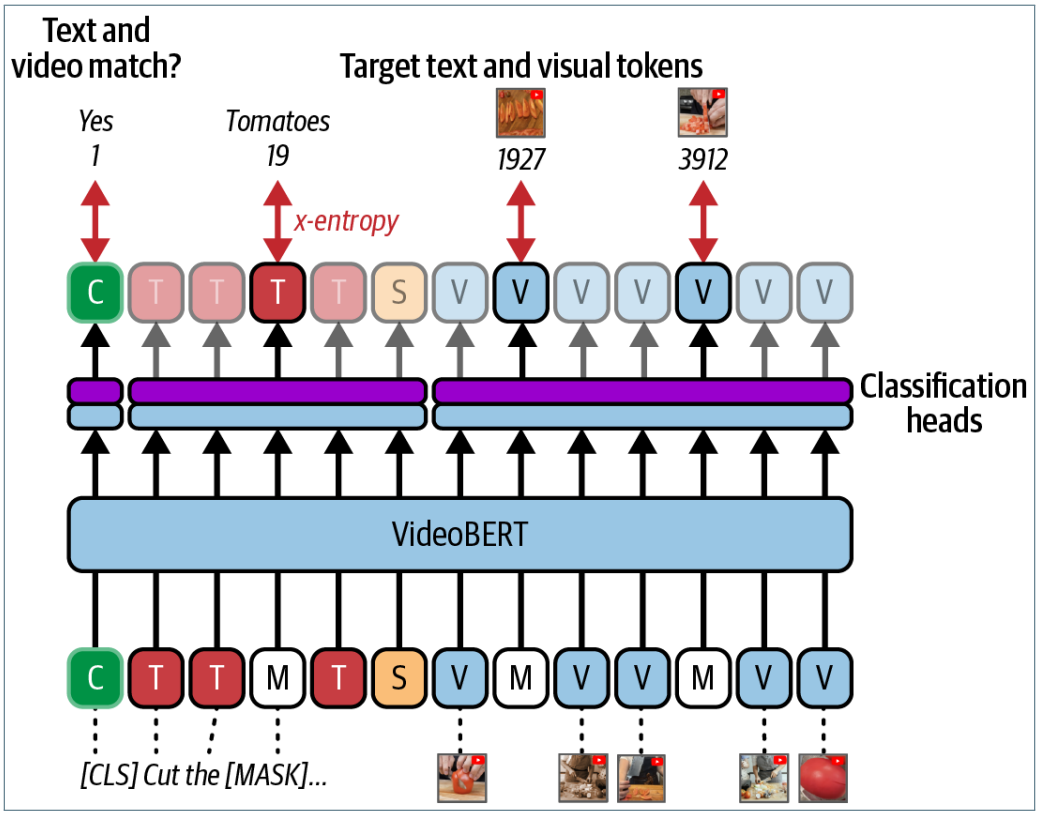

图 16-10: VideoBERT 的掩码 token 预测和语言-视觉对齐预训练（VideoBERT pretraining with masked token prediction and linguistic-visual alignment）

语言-视觉对齐存在噪声，因为厨师可能正在解释已经完成或尚未执行的动作。作者通过拼接随机邻近句子增加上下文，并随机改变视频采样率，使模型适应不同的动作速度。

VideoBERT 的下游任务包括零样本动作分类、视频描述和多项选择视觉问答。它还启发了 ViL BERT、Visual BERT、Unicoder-VL、LXMERT、VL-BERT 和 UNITER 等模型。大多数模型是单流 Transformer，在网络早期就拼接不同模态；ViL BERT 和 LXMERT 则采用双流架构，让每种模态先由自己的编码器处理。

16.2.2　ViL BERT：文本加图像的双流 Transformer（ViL BERT: A Dual-Stream Transformer for Text plus Image）

ViL BERT 由 Georgia Institute of Technology、Facebook AI Research 和 Oregon State University 的研究人员于 2019 年提出。作者认为，单流架构把视觉和文本模态当作完全相同的对象，但它们可能需要不同程度的处理。例如，视觉特征可能已经由深度 CNN 提取为高级表示，而文本还需要更多层处理才能获得高级语义；图像区域之间的关系也可能比句子中的词更弱。

因此 ViL BERT 使用双流架构：每种模态由自己的编码器处理，在较高层通过双向交叉注意力机制交换信息。这种机制称为 co-attention。在每一对连接的编码器层中，一个编码器的 MHA query 会被另一个编码器用作 key/value。

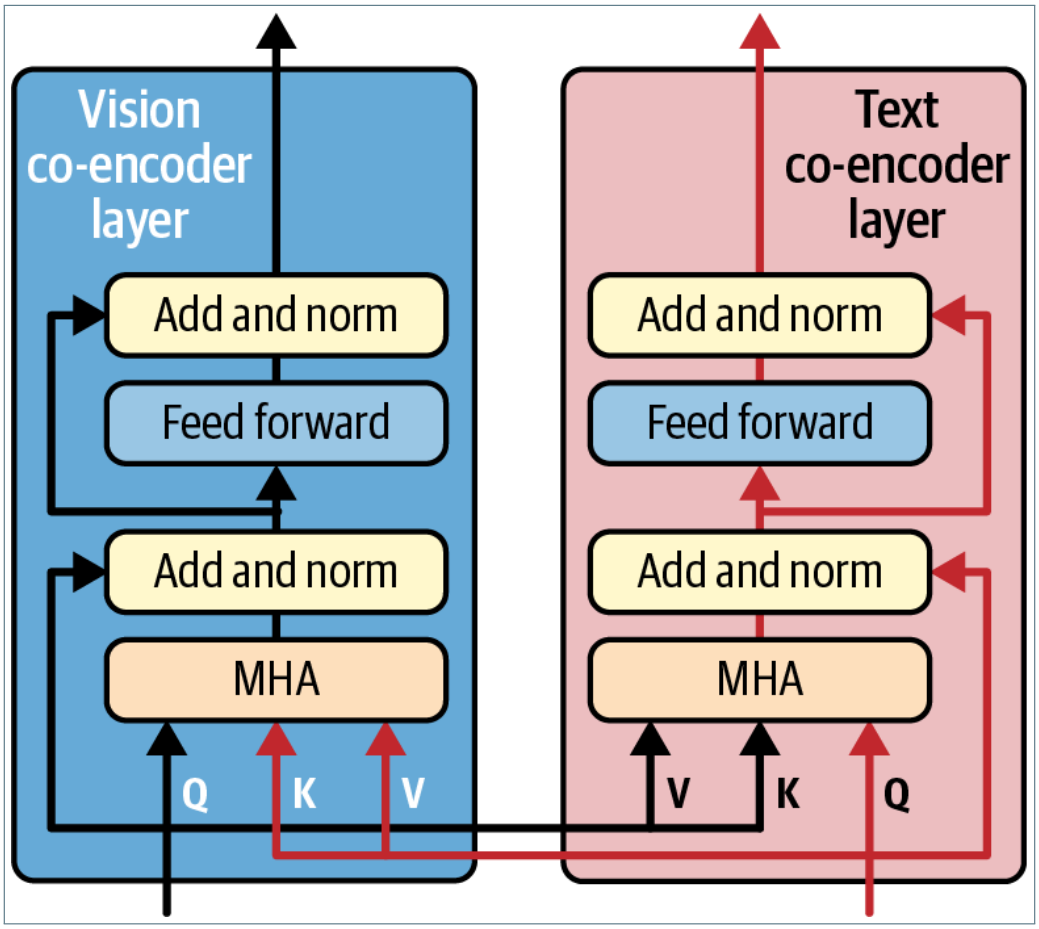

图 16-11: 通过 co-attention 连接的两个编码器层（Two encoder layers connected through co-attention）

文本编码器的底层使用 BERT 权重，顶部有 6 个 co-attention 层。视觉特征由冻结的 Faster R-CNN 提取，每张图像保留 10-36 个高置信度区域向量，视觉编码器由与文本编码器配对的 6 个 co-attention 层组成。

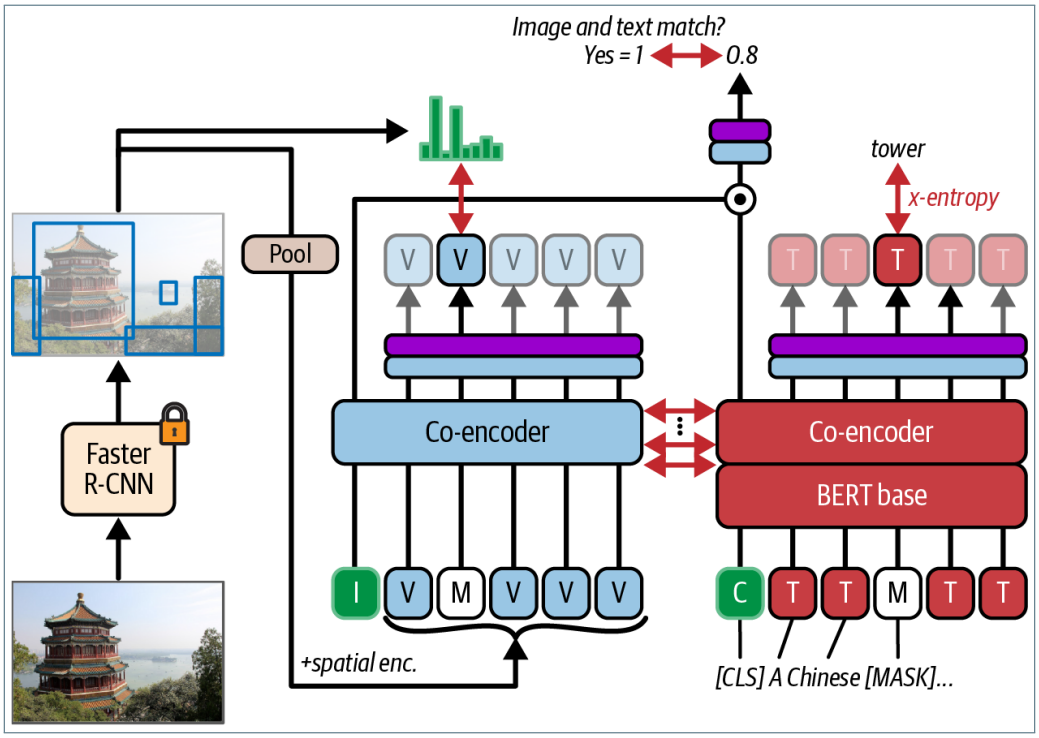

图 16-12: ViL BERT 使用掩码 token 预测和语言-视觉对齐进行预训练（ViL BERT pretraining with masked token prediction and linguistic-visual alignment）

视觉区域没有像词那样的自然顺序，因此视觉编码器不用位置编码，而使用空间编码。每个区域的边界框被表示为 5 维向量：归一化的左上角坐标、右下角坐标以及边界框覆盖图像的比例。这个向量再线性投影到与视觉向量相同的维度并相加。

视觉序列开头还会加入特殊的 [IMG] token。它代表整张图像，通常由所有区域特征的平均值以及覆盖整张图像的空间编码构成。

ViL BERT 在文本编码器中执行普通 BERT 式 MLM；视觉编码器则预测 CNN 对对应图像区域的类别分布。语言-视觉对齐任务取 [IMG] 和 [CLS] 的输出，计算逐元素乘积，再输入二分类头判断文本和图像是否匹配。乘法会放大两个表示中都很强的特征，比加法更能捕捉对齐关系。

ViL BERT 在图像定位、基于描述的图像检索、视觉问答和视觉常识推理等任务上显著超过此前的最佳结果，并对后续多模态研究产生了很大影响。

16.2.3　CLIP：通过对比学习训练的图像-文本双编码器（CLIP: A Dual-Encoder Text plus Image Model Trained with Contrastive Pretraining）

CLIP 于 2021 年发布。它的思想非常直接：分别把文本和图像编码成向量；如果文本和图像匹配，就让两个向量更接近；如果不匹配，就让它们更远。

训练完成后，CLIP 可以零样本完成图像分类。例如，要把它当作昆虫分类器，只需把 cricket、ladybug、spider 等类别名称输入文本编码器，再把图像输入视觉编码器，最后使用余弦相似度选择最相似的类别。使用类似网页图像标题的提示模板通常效果更好，例如“This is a photo of a ladybug”，而不是只使用 ladybug。

Hugging Face Transformers 提供零样本图像分类 pipeline：

```
from transformers import pipeline

model_id = "openai/clip-vit-base-patch32"
clip_pipeline = pipeline(task="zero-shot-image-classification", model=model_id,
                         device_map="auto", dtype="auto")
candidate_labels = ["cricket", "ladybug", "spider"]
image_url = "https://homl.info/ladybug"  # a photo of a ladybug on a dandelion
results = clip_pipeline(image_url, candidate_labels=candidate_labels,
                        hypothesis_template="This is a photo of a {}.")
```

输出按分数排序，ladybug 的概率可以超过 99.7%。更换 candidate_labels 后，同一个模型也可以直接用于花卉分类等任务。

CLIP 由普通文本编码器和普通视觉编码器组成，不需要 co-attention。视觉编码器可以使用 ResNet 或 ViT，文本编码器可以使用类似 GPT-2 的模型。虽然 GPT-2 通常是解码器模型，但 CLIP 不用它做下一 token 预测，而是使用最后一个 token 的输出表示整个输入序列。

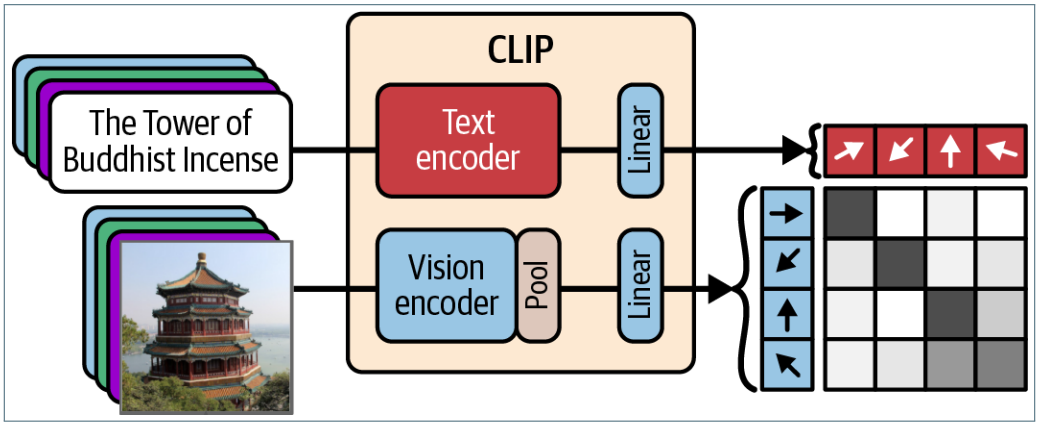

图 16-13: CLIP 将图像-标题对编码为向量，并拉近匹配对、推远不匹配对（CLIP encodes image-caption pairs and applies contrastive learning）

视觉编码器顶部有池化层，使它输出整张图像的单个向量；两个编码器顶部各有一个线性层，把表示投影到相同的输出空间。训练时使用非常大的批次，例如 m=2^15=32,768，其中包含 4 亿个图像-标题对。

CLIP 的对比损失可以概括为：

1. 对所有向量做 L2 归一化，只保留方向。
2. 计算每个图像表示与每个文本表示的余弦相似度，得到 m×m 矩阵。
3. 主对角线是正确的图像-标题配对，应该接近 +1；其他配对应该接近 0。
4. 把每一行和每一列都视为分类任务，计算交叉熵损失并取平均。
5. 在计算损失前，用可训练温度缩放相似度分数。

> 警告：这种损失需要很大的批次来提供足够的负样本，否则模型可能过拟合正样本细节。CLIP 的成功部分归功于作者实现的巨大批次。

CLIP 在许多图像分类数据集上不需要额外训练就取得了很好的零样本结果，尤其适合互联网常见的日常场景图像和每类样本较少的数据集。但它对卫星图像或医学图像等领域特定图像的表现较弱。

CLIP 的视觉特征还很适合图像检索：把图像用 CLIP 编码后存入向量数据库，就可以用文本查询或图像查询寻找相似图像。

如果需要直接获得文本和视觉特征，可以直接调用 CLIP 模型：

```
import PIL
import urllib.request
from transformers import CLIPProcessor, CLIPModel

clip_processor = CLIPProcessor.from_pretrained(model_id)
clip_model = CLIPModel.from_pretrained(model_id)
image = PIL.Image.open(urllib.request.urlopen(image_url)).convert("RGB")
captions = [f"This is a photo of a {label}." for label in candidate_labels]
inputs = clip_processor(text=captions, images=[image], return_tensors="pt",
                        padding=True)
with torch.no_grad():
    outputs = clip_model(**inputs)
text_features = outputs.text_embeds    # shape [3, 512]  # 3 captions
image_features = outputs.image_embeds  # shape [1, 512]  # 1 image (ladybug)
```

特征已经做过 L2 归一化，因此只需要一次矩阵乘法就能得到相似度：

```
>>> similarities = image_features @ text_features.T  # shape [1, 3]
>>> similarities
tensor([[0.2337, 0.3021, 0.2381]])
```

如果需要概率而不是相似度分数，先用模型学到的温度重新缩放，再使用 softmax：

```
>>> temperature = clip_model.logit_scale.detach().exp()
>>> rescaled_similarities = similarities * temperature
>>> probabilities = torch.nn.functional.softmax(rescaled_similarities, dim=1)
>>> probabilities
tensor([[0.0011, 0.9973, 0.0017]])
```

16.2.4　DALL·E：从文本生成图像（DALL·E: Generating Images from Text Prompts）

DALL·E 于 2021 年发布，可以根据文本提示生成图像，例如“一个形状像牛油果的扶手椅”。它使用类似 GPT 的模型预测下一个 token，但训练数据是图像-标题对，输入序列由文本 token 和视觉 token 拼接而成。推理时只输入文本 token，模型逐个生成视觉 token，直到得到完整图像。视觉 token 由 dVAE 生成。

原始 DALL·E 没有公开发布，但出现了开源复现，例如 DALL·E mini，也称 Craiyon。

DALL·E 2 于 2022 年发布，架构完全不同：文本先输入 CLIP 得到文本 embedding，再将 embedding 输入扩散模型指导图像生成。DALL·E 2 通过付费 API、Microsoft Designer、Bing Image Creator、Canva 和 ChatGPT 等产品提供。

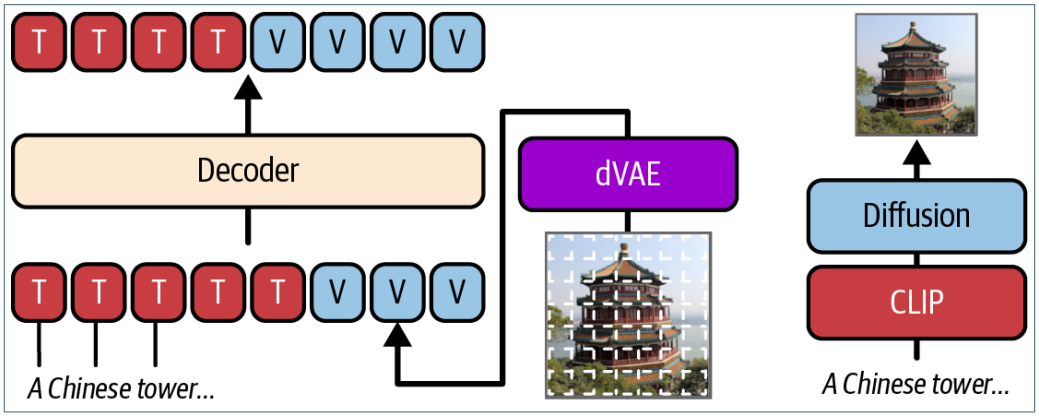

图 16-14: DALL·E 与 DALL·E 2 的架构（DALL·E and DALL·E 2）

DALL·E 3 于 2023 年发布。它基于扩散模型，不使用 CLIP，并与 GPT-4 紧密集成，由 GPT-4 先重写提示再生成图像。它尤其擅长遵循包含多个元素的复杂组合提示。

16.2.5　Perceiver：用潜在空间连接高分辨率模态（Perceiver: Bridging High-Resolution Modalities with Latent Spaces）

到目前为止，Transformer 都需要把输入切成有意义的 token：文本是词或子词，ViT 是 16×16 patch，VideoBERT 是 1.5 秒视频片段。如果直接输入单个字符、像素或音频帧，序列

会极长，注意力的二次复杂度会成为问题。同时，切成 patch 也会引入邻近像素更相关的归纳偏置。

Perceiver 于 2021 年提出，可以在最低层级直接处理字符、像素和音频帧等任意模态，并且使用与模态无关的架构。

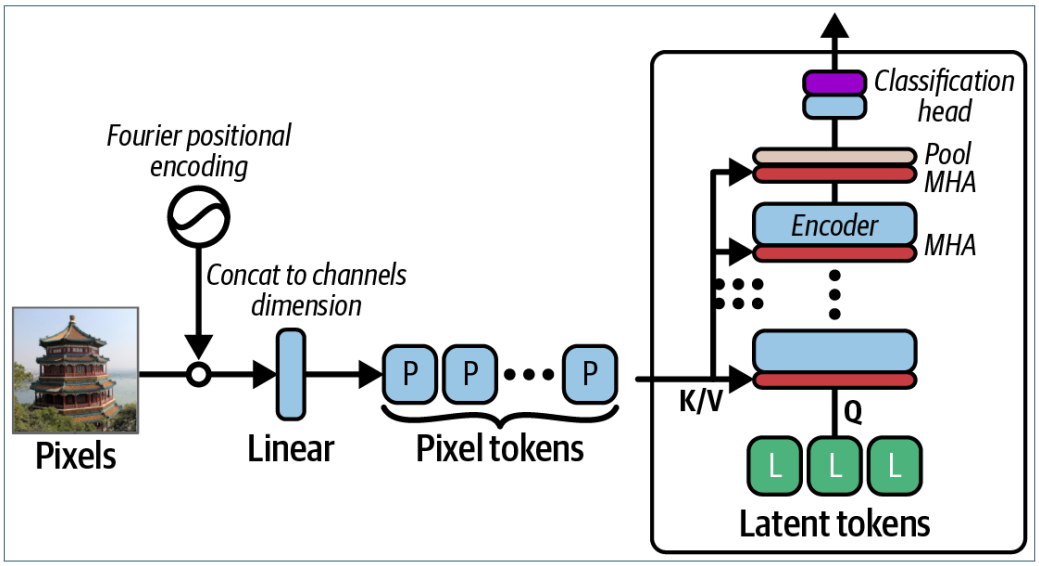

图 16-15: Perceiver 架构：通过交叉注意力接收输入，并使用学习到的潜在 token（Perceiver architecture with latent tokens and cross-attention）

Perceiver 的过程如下：

1. 把输入切成最小组成单元。图像会被切成单个像素，每个像素是一个 RGB 三维向量。
2. 把位置编码拼接到特征向量。Perceiver 使用 Fourier 位置编码，对图像的水平和垂直坐标编码。
3. 通过线性层把输入 token 投影到 Perceiver 的维度。
4. 使用多个处理块。每个块由一个交叉注意力 MHA 和一个普通 Transformer 编码器组成。
5. 输入 token 只通过交叉注意力作为 key 和 value；主序列是较短的潜在 token 序列。
6. 最后的交叉注意力和池化层把潜在序列缩减为一个向量，再输入分类头。

Perceiver 的主输入是长度较短的潜在 token 序列，例如 512 个 token。初始潜在序列在训练中学习，之后通过交叉注意力逐渐吸收像素 token 的信息。由于潜在序列很短，它不会承受普通自注意力的二次成本，这称为潜在瓶颈（latent bottleneck）。

如果 224×224 图像直接输入普通编码器，需要计算 $50{,}176^2$，约 25 亿个注意力分数；Perceiver 只需计算 50,176×512，约 2,570 万个分数，约少 100 倍。因此 Perceiver 对输入 token 数量呈线性扩展，而不是二次扩展。

Perceiver 在 ImageNet、AudioSet 和 ModelNet40 等多个模态的分类任务上取得了有竞争力的结果。对于视频和音频，作者使用时空 patch 或音频片段降低 token 数量，再把视频 token、音频 token 和模态 embedding 拼接起来。

16.2.6　Perceiver IO：灵活的输出机制（Perceiver IO: A Flexible Output Mechanism）

Perceiver IO 于 2021 年发布。它保留 Perceiver 的输入和潜在 token，但用灵活的输出机制替换池化层和分类头。

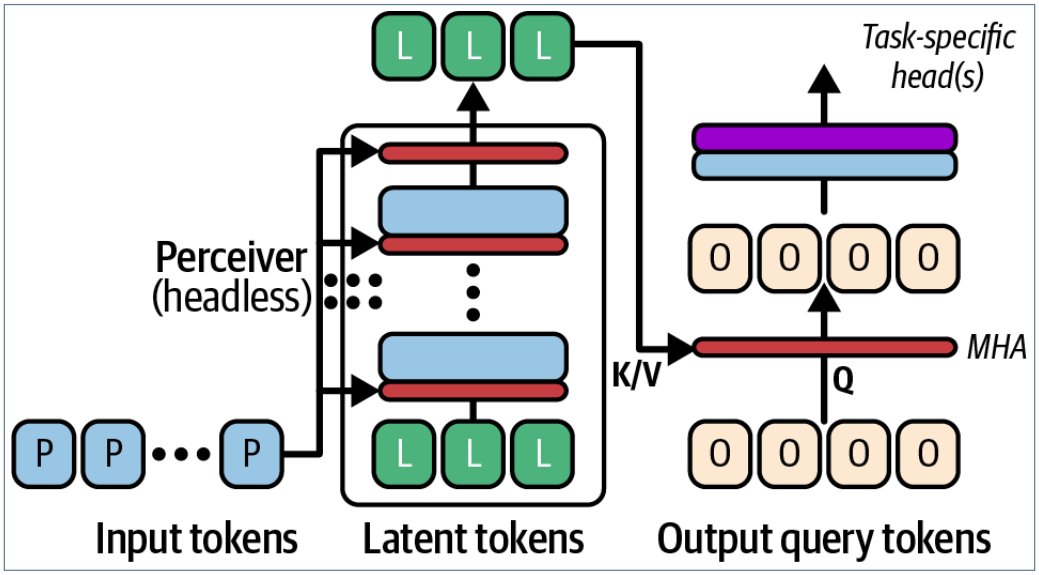

图 16-16: Perceiver IO 使用输出 query token 通过交叉注意力读取潜在 token（Perceiver IO with output query tokens）

输出机制首先增加一个交叉注意力层，把输出 query token 作为 query，把潜在 token 作为 key/value，得到最终输出表示，再连接任务特定的头。输出 token 的数量和性质取决于任务：

- 分类只需要一个输出向量，因此只需要一个可学习的输出 query token。
- MLM 可以为每个被遮盖 token 使用一个输出 query token，并根据该 token 的位置使用可学习的位置 embedding。
- 光流预测可以为每个像素使用一个输出 token，并使用包含 Fourier 位置编码的像素表示。

由于输出 query token 只关注潜在 token，Perceiver IO 可以处理大量输出 token。潜在瓶颈使它对输入和输出都呈线性扩展。

Perceiver IO 是双向架构，没有因果掩码，因此不适合自回归文本生成。如果每次只预测末尾一个 masked token，会比有缓存机制的因果模型低效。2022 年发布的 Perceiver AR 使用输入序列最后一部分作为潜在 token，在这些潜在 token 上使用因果注意力，并通过下一 token 预测训练；它在很长的输入序列上取得了优秀结果。

16.2.7　Flamingo：开放式视觉对话（Flamingo: Open-Ended Visual Dialogue）

Flamingo 于 2022 年发布，是一种视觉语言模型，可以接收任意交错的文本和图像序列，并生成连贯的自由形式文本。它的少样本能力很强：给出几张图像及其说明，再追加一张新图像，模型就可以描述新图像；也可以用于车牌识别、图像描述、视觉问答或视频片段描述。

Flamingo 不从头训练，而是连接两个冻结的大型预训练模型：一个视觉模型和一个仅解码器语言模型。作者使用了 Chinchilla 和 CLIP，但其他强大的模型也可以。

图像先由视觉模型处理，再通过一个 Perceiver 风格的 Resampler 生成较短的潜在 token 序列。视觉 token 作为 key/value 输入插入到冻结 LLM 每个块之前的 gated xattn-dense 模块中。

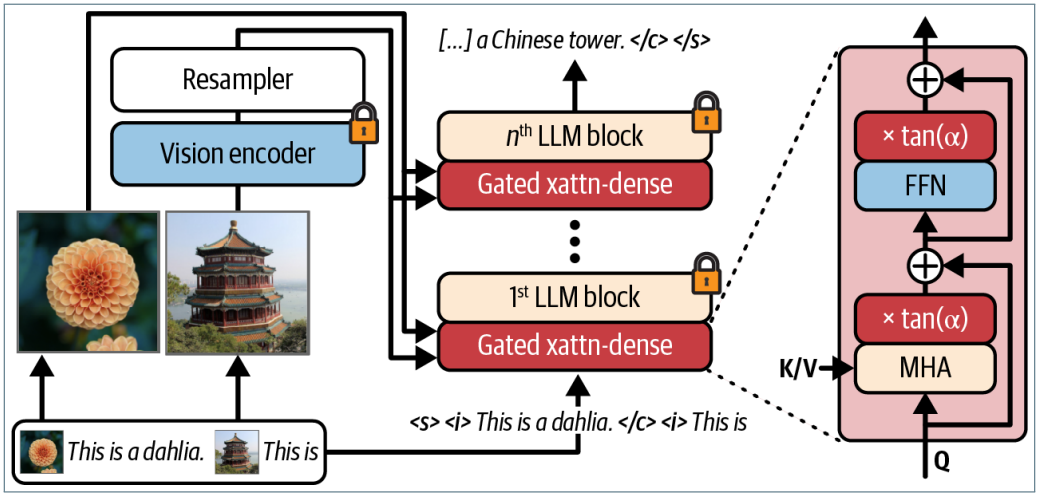

图 16-17: Flamingo 接收任意交错的文本和图像并生成自由形式文本（Flamingo processes interleaved text and images）

gated xattn-dense 模块包含带掩码的 MHA、前馈网络和残差连接；MHA 和前馈模块后都有 tanh gate。每个 gate 乘以 tanh(α)，α 是初始化为 0 的可学习标量，因此训练刚开始时所有 gate 都关闭，输入只能通过残差连接流动，模型不会破坏冻结 LLM 的行为。随着训练推进，模型逐渐学会打开 gate。

每个文本 token 只能关注它前面最近的图像 token，不能直接关注更早的图像，但冻结 LLM 的文本自注意力可以间接传递这些信息。文本 tokenization 使用 LLM 的 tokenizer，并增加图像 token 和 end-of-chunk token。

原始 Flamingo 没有公开发布，但有多个开源复现：

- Open Flamingo：LAION 组织创建的开源实现。
- IDEFICS：Hugging Face 基于大规模 OBELICS 数据集训练的模型。
- AudioFlamingo：把 Flamingo 的图像模态换成音频。
- Med-Flamingo：面向医学文档的领域模型。

16.2.8　BLIP 和 BLIP-2（BLIP and BLIP-2）

BLIP 是 Salesforce 于 2022 年发布的视觉语言模型。它使用混合编码器-解码器架构，包含文本编码器、视觉编码器、图像引导文本编码器和图像引导文本解码器，并通过共享层同时训练三个目标：

- 图像-文本匹配（image-text matching，ITM）。
- 图像-文本对比学习（image-text contrastive，ITC），用于对齐图像和文本表示。
- 语言建模（language modeling，LM），使用下一 token 预测生成标题。

BLIP 使用 CapFilt 构建更干净的数据集：一个标题模块为图像生成合成标题，另一个过滤模块删除低质量标题，再用得到的大型干净数据集训练最终模型。

BLIP-2 于 2023 年发布，复用冻结的视觉模型和语言模型，参数更少但表现超过 Flamingo。它的核心是 Q-Former（querying Transformer），架构类似 BERT-base，但在部分编码器层之间加入交叉注意力，以读取预训练视觉编码器的输出。

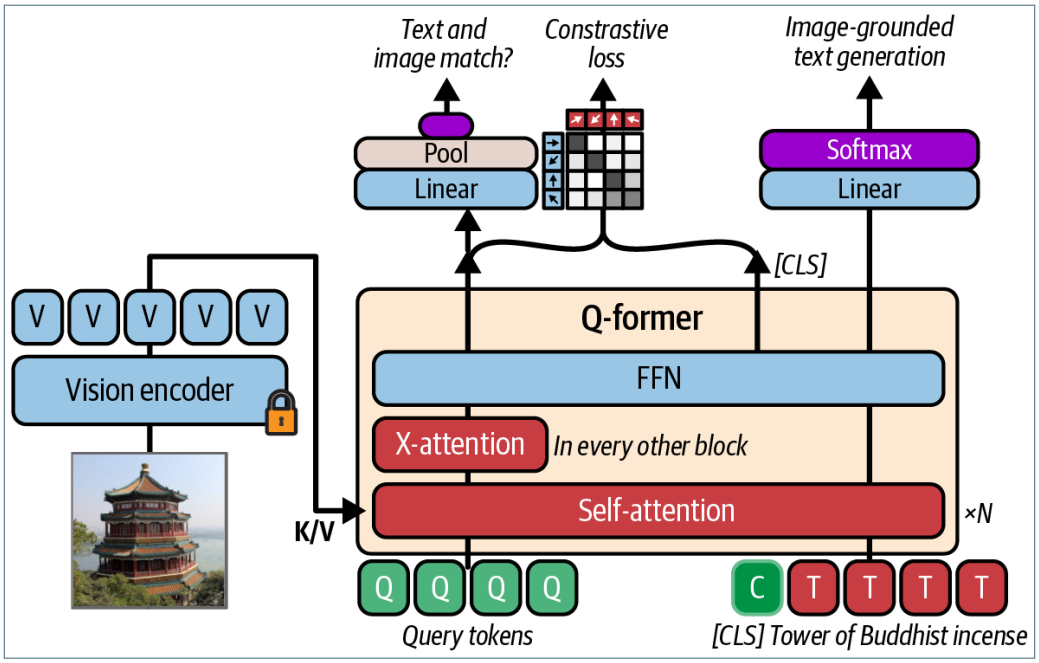

图 16-18: BLIP-2 第一阶段：训练 Q-Former（BLIP-2 pretraining, Stage 1: training the Q-Former）

Q-Former 处理三种序列：使用 BERT tokenization 的文本 token、视觉编码器生成的视觉 token，以及可训练的 Perceiver 风格 query token。它使用 ITM、ITC 和 LM 三种训练目标：

- ITM：query token 和文本 token 可以互相关注，预测图像与文本是否匹配。
- ITC：query token 和文本 token 不能互相关注，分别产生视觉表示和文本表示，然后用相似度进行对比学习。
- LM：文本 token 只能关注前面的文本 token，但可以关注所有 query token；query token 不能关注文本 token。文本输出通过分类头预测下一个 token。

LM 训练时需要把 class token 替换成 decode token，告诉 Q-Former 当前任务是生成文本，而不是编码文本。

第二阶段保留视觉 Transformer 和 Q-Former，丢弃其余部分，再在 Q-Former 顶部加入一个随机初始化的线性层。Q-Former 关注视觉特征，输出视觉 query token；这些 token 与文本 token 表示拼接后输入冻结的 LLM，训练目标是预测标题中的下一个 token。

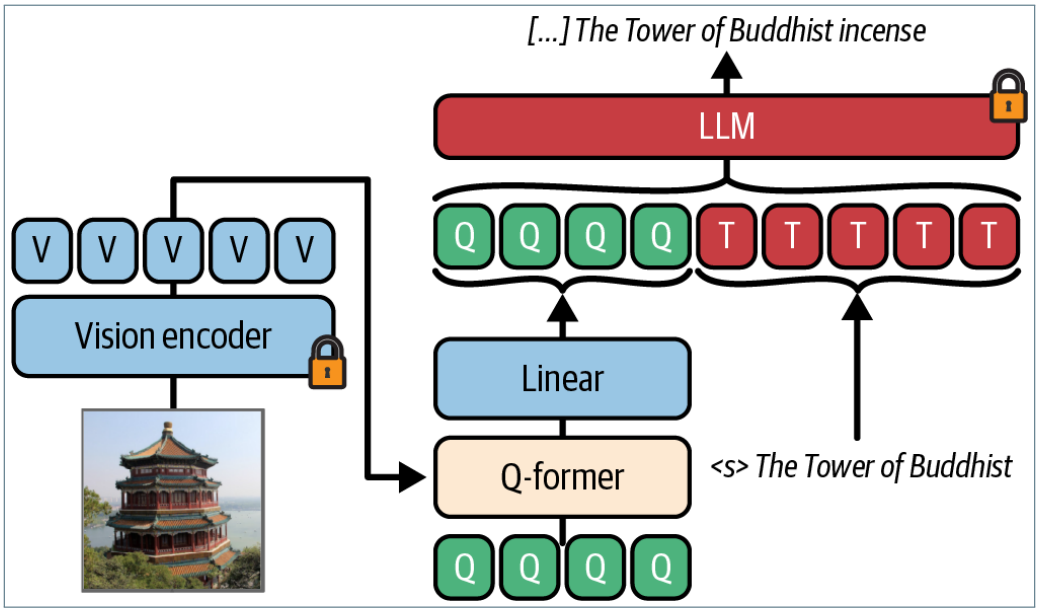

图 16-19: BLIP-2 第二阶段：把 query token 映射到 LLM 输入空间（BLIP-2 pretraining, Stage 2）

使用 BLIP-2 为图像生成标题：

```
from transformers import Blip2Processor, Blip2ForConditionalGeneration

model_id = "Salesforce/blip2-opt-2.7b"
blip2_processor = Blip2Processor.from_pretrained(model_id)
blip2_model = Blip2ForConditionalGeneration.from_pretrained(
    model_id, device_map=device, dtype=torch.float16)
image_url = "http://images.cocodataset.org/val2017/000000039769.jpg"  # two cats
image = Image.open(urllib.request.urlopen(image_url))
inputs = blip2_processor(images=image, return_tensors="pt")
inputs = inputs.to(device, dtype=torch.float16)
with torch.no_grad():
    generated_ids = blip2_model.generate(**inputs)
generated_text = blip2_processor.batch_decode(generated_ids)
```

如果不想在输出中看到特殊 token，可以在解码时设置 skip_special_tokens=True：

```
>>> generated_text = blip2_processor.batch_decode(generated_ids,
...                                               skip_special_tokens=True)
...
>>> generated_text
['two cats laying on a couch\n']
```

16.2.9　其他多模态模型（Other Multimodal Models）

多模态模型和架构还有很多：

- LayoutLM：理解文本、视觉和文档布局，LayoutLMv3 于 2022 年发布。
- GLIP：用于视觉定位和目标检测。
- Stable Diffusion：强大的文本到图像模型。
- OFA：处理多种视觉语言任务的统一框架。
- CoCa：使用对比学习和图像描述目标预训练，影响了 PaLI-X 和 Flamingo-2。
- PaLI、PaLI-X、PaLI-3 和 PaliGemma：支持多语言、多模态视觉语言任务。
- Kosmos-1、Kosmos-2 和 Kosmos-2.5：支持视觉定位。
- PaLM-E：把视觉输入和具身传感器数据加入 PaLM，生成可执行的机器人文本命令。
- LLaVA：优秀的开源视觉语言聊天模型。
- ImageBind：把 CLIP 风格的模型扩展到图像、文本、音频、IMU、深度和热成像六种模态。
- RT-2：同时支持视觉语言理解和机器人控制。
- SeamlessM4T：支持语音转文本、语音到语音、文本转语音和文本到文本翻译。
- Qwen-VL、Qwen2-VL 和 Qwen3-Omni：不断扩展到视频和音频的开放视觉语言模型。
- Fuyu：实时处理交错图像和文本的统一 Transformer。
- EMO：根据人物图像和语音生成与音频匹配的人物视频。
- GLaMM：生成混合目标分割掩码的视觉对话模型。
- LaViDa：开放的、基于扩散的视觉语言模型。

原书为本章讨论的模型创建了 homl.info 短链接，通常使用模型名的小写形式。例如 Qwen2-VL 的链接是 https://homl.info/qwen2vl。

商业多模态模型还包括 GPT-4.1、Sora、Gemini 2.5 Pro、Veo-3 和 Claude 4 Opus。详细架构通常没有公开，但可以通过应用或 API 使用。下面是通过 Google Gemini API 分析一张猫的照片的示例：

```
from google import genai

gemini_api_key = [...]  # load from Colab secrets, or from a file, or hardcode
gemini_client = genai.Client(api_key=gemini_api_key)
cats_photo = gemini_client.files.upload(file="my_cats_photo.jpg")
question = "What animal and how many? Format: [animal, number]"
response = gemini_client.models.generate_content(
    model="gemini-2.5-flash",  # or "gemini-2.5-pro"
    contents=[cats_photo, question])
print(response.text)  # prints: "[cat, 2]"
```

这个示例使用已安装在 Colab 中的 **`google-genai`** 库，并假设当前目录有 **`my_cats_photo.jpg`**。下一章将讨论提高 Transformer 速度和扩展能力的技术。

16.3　练习（Exercises）

1. 请描述原始 ViT 的架构。为什么这种架构重要？
2. 普通 ViT（非层次化）最适合哪些任务？它有什么限制？
3. DeiT 的主要创新是什么？这个思想是否可以推广到其他架构？
4. 层次化 ViT 有哪些例子？它们适合什么任务？
5. PVT 和 Swin Transformer 如何降低处理高分辨率图像的计算成本？
6. DINO 如何工作？DINOv2 做了哪些改变？什么时候适合使用 DINOv2？
7. JEPA 架构的目标是什么？它如何工作？
8. 什么是多模态模型？请举出五个多模态任务。
9. 解释多模态学习中的融合问题和对齐问题。为什么 Transformer 适合解决它们？
10. 用一句话概括 VideoBERT、ViL BERT、CLIP、DALL·E、Perceiver IO、Flamingo 和 BLIP-2 的主要思想。
11. 使用 Perceiver IO 时，如果输入和输出长度都加倍，大约需要多少额外计算？
12. 在 Food-101 数据集上微调预训练 ViT，并报告准确率；再尝试用 CLIP 做零样本分类。
13. 为自己的照片创建一个搜索引擎：使用 CLIP 为照片生成 embedding 并保存；再编写函数接收文本或图像查询，生成查询 embedding，寻找最相似的照片并显示它们。可以手工实现相似度搜索，也可以使用 FAISS 或向量数据库。

14. 使用 BLIP-2 自动为照片生成标题。

    练习答案位于本章 notebook 末尾：https://homl.info/colab-p。

16.4　参考文献（References）

1. Kelvin Xu 等，《Show, Attend and Tell: Neural Image Caption Generation with Visual Attention》，Proceedings of the 32nd International Conference on Machine Learning（2015）：2048-2057。
2. 图 16-1 改编自论文图 3，已获作者授权。
3. Marco Tulio Ribeiro 等，《Why Should I Trust You?: Explaining the Predictions of Any Classifier》，Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining（2016）：1135-1144。
4. Nicolas Carion 等，《End-to-End Object Detection with Transformers》，arXiv:2005.12872（2020）。
5. Alexey Dosovitskiy 等，《An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale》，arXiv:2010.11929（2020）。
6. Hugo Touvron 等，《Training Data-Efficient Image Transformers & Distillation Through Attention》，arXiv:2012.12877（2020）。
7. Wenhai Wang 等，《Pyramid Vision Transformer: A Versatile Backbone for Dense Prediction without Convolutions》，arXiv:2102.12122（2021）。
8. Ze Liu 等，《Swin Transformer: Hierarchical Vision Transformer using Shifted Windows》，arXiv:2103.14030（2021）。
9. Ze Liu 等，《Swin Transformer V2: Scaling Up Capacity and Resolution》，arXiv:2111.09883（2021）。
10. Mathilde Caron 等，《Emerging Properties in Self-Supervised Vision Transformers》，arXiv:2104.14294（2021）。
11. 图 16-8 改编自 DINO 论文图 3 的右侧，已获作者授权。
12. Yangtao Wang 等，《TokenCut: Segmenting Objects in Images and Videos with Self-supervised Transformer and Normalized Cut》，arXiv:2209.00383（2022）。
13. 《DINOv2: Learning Robust Visual Features without Supervision》，arXiv:2304.07193（2023）。
14. Xiaohua Zhai 等，《Scaling Vision Transformers》，arXiv:2106.04560（2021）。

15. Hangbo Bao 等，《BEiT: BERT Pre-Training of Image Transformers》，arXiv:2106.08254（2021）。

16. Kaiming He 等，《Masked Autoencoders Are Scalable Vision Learners》，arXiv:2111.06377（2021）。

17. Mitchell Wortsman 等，《Model Soups: Averaging Weights of Multiple Fine-tuned Models Improves Accuracy Without Increasing Inference Time》，arXiv:2203.05482（2022）。

18. Yuxin Fang 等，《EVA: Exploring the Limits of Masked Visual Representation Learning at Scale》，arXiv:2211.07636（2022）。

19.《Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture》，arXiv:2301.08243（2023）。

20. Yann LeCun，《A Path Towards Autonomous Machine Intelligence》（2022）。

21. Chen Sun 等，《VideoBERT: A Joint Model for Video and Language Representation Learning》，arXiv:1904.01766（2019）。

22. L. Zhou、Y. Zhou 等，《End-to-End Dense Video Captioning with Masked Transformer》，CVPR（2018）。

23. Jiasen Lu 等，《ViLBERT: Pretraining Task-Agnostic Visiolinguistic Representations for Vision-and-Language Tasks》，NeurIPS 32（2019）。

24. Jize Cao 等，《Behind the Scene: Revealing the Secrets of Pre-trained Vision-and-Language Models》。

25. Alec Radford 等，《Learning Transferable Visual Models From Natural Language Supervision》，arXiv:2103.00020（2021）。

26. OpenAI 没有发布 CLIP 的训练代码和数据，但 Gabriel Ilharco 等人创建了 OpenCLIP，这是一个包含完整训练代码和数据的开源复现。

27. 这种对比损失最早以 multiclass n-pair loss 的形式提出，后来被用于对比表示学习，并重新命名为 InfoNCE。

28. Aditya Ramesh 等，《Zero-Shot Text-to-Image Generation》，arXiv:2102.12092（2021）。

29. Aditya Ramesh 等，《Hierarchical Text-Conditional Image Generation with CLIP Latents》，arXiv:2204.06125（2022）。

30. Andrew Jaegle 等，《Perceiver: General Perception with Iterative Attention》，arXiv:2103.03206（2021）。

31. 如果 Δ 是采样间距，Nyquist-Shannon 采样定理告诉我们可测量的最大频率是 f=1/(2Δ)，因此频率上限是目标分辨率的一半。

32. AudioSet 包含超过 200 万段、每段 10 秒的视频，分为 500 多个类别。
33. ModelNet40 是包含多种形状三维点云的合成数据集，现实世界中的点云常来自 LiDAR 传感器。
34. Andrew Jaegle 等,《Perceiver IO: A General Architecture for Structured Inputs & Outputs》, arXiv:2107.14795（2021）。
35. 在法国漫画《Asterix》中，Obelix 是友善而强壮的高卢人，Idefix 是他的聪明小狗。
36. Junnan Li 等，《BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation》，arXiv:2201.12086（2022）。
37. Junnan Li 等，《BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models》，arXiv:2301.12597（2023）。
38. 2019 年，微软研究人员 Li Dong 等人通过 UniLM 提出了同时进行文本编码和生成的单一模型思想。
39. 大多数现代智能手机都有 IMU 传感器，可以测量加速度、角速度，通常还可以测量磁场强度。

原书超链接资源：

- 视觉注意力
- 分类器解释
- DETR
- ViT
- DeiT
- PVT
- Swin Transformer
- 本书在线主页
- DINO
- TokenCut
- DINOv2
- Scaling Vision Transformers
- BEiT
- MAE

- Model Soups
- EVA
- I-JEPA
- JEPA
- V-JEPA
- V-JEPA 2
- VideoBERT
- 视频描述
- ViL BERT
- CLIP
- DALL·E
- DALL·E mini
- DALL·E 2
- Perceiver
- Perceiver IO
- Perceiver AR
- Flamingo
- Open Flamingo
- IDEFICS
- AudioFlamingo
- Med-Flamingo
- BLIP
- BLIP-2
- Qwen2-VL
- Google AI Studio
- Food-101
- FAISS

- 本章 notebook 练习答案
- 线性探针
- OpenCLIP
- N-pair loss
- InfoNCE
- AudioSet
- ModelNet40
- UniLM

第 17 章　加速 Transformer

Speeding Up Transformers

在第 15 章和第 16 章中，我们构建了各种 Transformer，从分类器、翻译器和聊天机器人，到视觉 Transformer 和多模态 Transformer。Transformer 虽然极其灵活且功能强大，但远非完美。尤其是在处理较长的输入序列时，它们可能非常慢。

幸运的是，人们已经提出了许多技术，可以加速各种规模的 Transformer：

- 为了加速生成式 Transformer 的解码，我们将使用键/值缓存（key/value caching）和推测解码（speculative decoding），然后快速浏览几种并行化文本生成的方法。
- 为了加速多头注意力（multi-head attention，MHA）—-这是 Transformer 中计算成本最高的组件之一—-我们将讨论稀疏注意力（sparse attention）、近似注意力（approximate attention）、投影共享（sharing projections）和 FlashAttention。
- 为了加速参数规模达到万亿级的巨型 Transformer，我们将讨论专家混合（mixture of experts，MoE）。
- 为了高效训练大型 Transformer，我们将讨论使用参数高效微调（parameter-efficient fine-tuning，PEFT）和适配器（adapter），例如低秩适配（Low-Rank Adaptation，LoRA）、激活检查点（activation checkpointing）、序列打包（sequence packing）、梯度累积（gradient accumulation）和并行化（parallelism）。

加速 Transformer 的另一种方法是让它变得更小。可以使用低精度（reduced precision）和量化（quantization）实现这一点，附录 B 将讨论这些技术。

本章内容很多，我们先从加速生成式 Transformer 的解码过程开始。

17.1　推理时加速解码（Faster Decoding at Inference Time）

在仅解码器模型和编码器-解码器模型中，解码器在推理时一次生成一个 token：要生成 1,000 个 token，就必须调用解码器 1,000 次。这不仅计算成本高，而且难以并行化，因为每个新 token 都依赖于前面的 token；这是一个顺序生成过程。不过，有一些方法可以加速解码，我们先从缓存开始。

17.1.1　键/值缓存（Key/Value Caching）

第 15 章介绍过，多头注意力（MHA）层中的每个注意力头都会计算缩放点积注意力公式（公式 15-1）：

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^\intercal/\sqrt{d_k})V,$$

其中 Q、K 和 V 分别是查询矩阵、键矩阵和值矩阵，d_k 是键的维度。在自注意力层中，查询、键和值都是 MHA 层输入序列的不同投影。

每个矩阵的第 i 行，都是第 i 个输入 token 表示的投影。由于解码器使用带掩码的自注意力层，第 i 个输入 token 表示只受到 token 1 到 i 的影响；这是一种因果模型，也就是说，未来不会影响过去。因此，把新 token i+1 追加到输入序列末尾时，不会改变 Q、K 和 V 矩阵的第 1 到 i 行，只会分别追加一行。太好了！我们可以缓存这些矩阵，在每个生成步骤中只计算并追加一行。

实际上，我们只需要缓存 K 和 V，不需要缓存 Q。仔细观察缩放点积注意力公式可以发现，输出的最后一行依赖完整的 K、V 矩阵，但只依赖 Q 的最后一行。由于我们只关心最后一个 token 的输出，所以每个生成步骤只需计算输出矩阵的最后一行，也就只需要 Q 的最后一行；无需缓存 Q。

那么键/值缓存（key/value caching，KV caching）究竟如何工作？第一次调用解码器时，把包含 token 1 到 i 的初始提示序列输入解码器。解码器正常工作，不过所有 MHA 层中所有注意力头的 K、V 矩阵都会保存到内存缓存中（见图 17-1）。这称为预填充阶段（prefill stage）。每个 K、V 矩阵都有 i 行，每个 token 对应一行。解码器输出 token 1 到 i 的下一个 token 预测；我们只关心最后一个预测，因为它就是新的 token i+1。

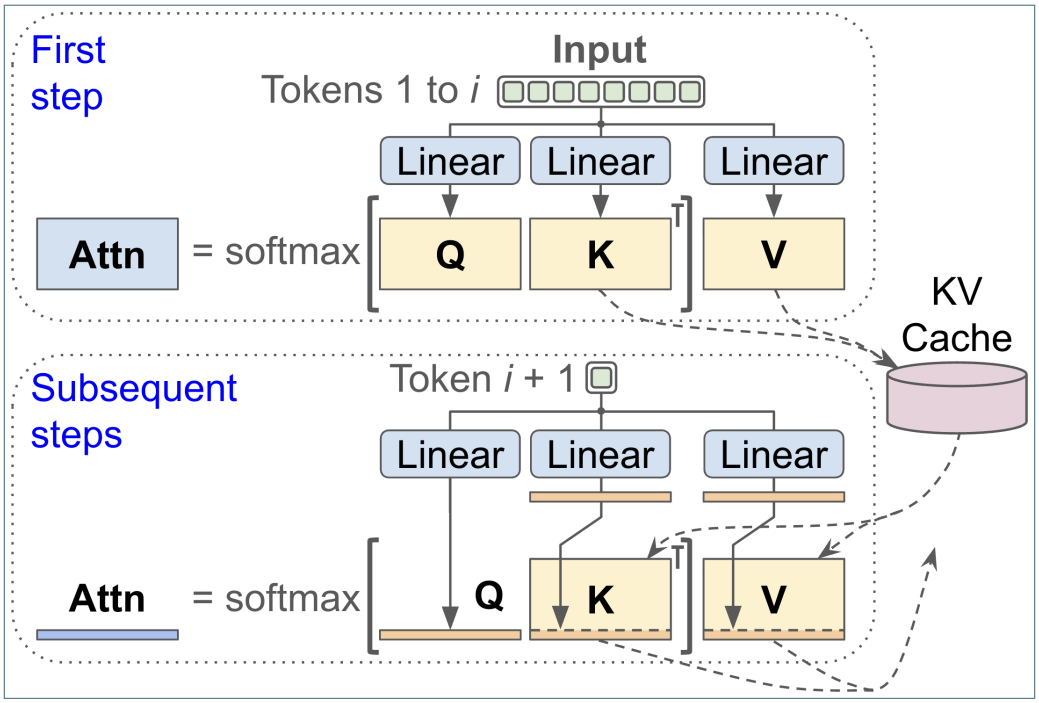

图 17-1: 键/值缓存：第一步正常处理并缓存所有键矩阵和值矩阵；后续步骤只处理最后生成的 token，复用缓存矩阵并追加新行（Key/value caching: the first step is normal but caches all key and value matrices; subsequent steps process only the last generated token, reusing the cached matrices and appending one row for the new token）

再次调用解码器来生成下一个 token 时，这称为解码阶段（decoding stage），我们只把新 token i+1 输入解码器：不需要再次输入 token 1 到 i，因为缓存的 K、V 矩阵已经包含这些 token 对应的行。在每个 MHA 层中，单个 token 的输入会经过三次投影，产生三个单行矩阵：单行 Q、追加到缓存 K 的一行，以及追加到缓存 V 的一行。Q 矩阵仍然只有一行，但更新后的 K、V 矩阵现在各有 i+1 行。接下来计算缩放点积注意力公式，输出仍是单行矩阵。经过所有层后，解码器预测一个 token，即新的 token i+2。之后每生成一个新 token，都重复这一过程。

KV 缓存让我们能够以远低于朴素方法的计算量生成新 token。第 15 章的朴素方法会反复把不断增长的完整句子输入解码器。即便使用 KV 缓存，每个新 token 的生成时间仍会越来越长，因为 K、V 矩阵不断增长；但增长至少是线性的，而不是二次的。注意，使用 KV 缓存生成长度为 L_o 的序列，整体时间复杂度仍然是 $O(L_o^2)$，因为线性增长量的总和是二次的：$1 + 2 + 3 + \cdots + L_o = L_o(L_o + 1)/2$。不过，二次复杂度总比三次复杂度好！

把 KV 缓存存放在 GPU RAM 中可能很困难，不仅因为缓存本身很大，还因为不同序列的长度不同，会导致内存碎片以及频繁且昂贵的重新分配。vLLM 工具包引入的 PagedAttention 借鉴虚拟内存系统的思想，把 KV 缓存存放在固定大小的内存页中。这样可以提高批处理效率并避免浪费内存。

使用 Hugging Face Transformers 库实现 KV 缓存时，通常什么都不需要做：大多数仅解码器模型和编码器-解码器模型都原生支持解码器的 KV 缓存，`generate()` 方法默认也会使用 KV 缓存。在底层，`generate()` 调用模型时会设置 `use_cache=True`，并把 `past_key_values` 参数设置为一个缓存对象。第一次调用时缓存为空，因此解码器正常工作，并把键矩阵和值矩阵保存到缓存中。后续调用时，解码器只处理最后一个输入 token，同时按照前面的方式使用并更新缓存。

`generate()` 默认使用的缓存类是 `transformers.DynamicCache`，适合一般情况；不过，特殊场景还有其他缓存类，例如使用稀疏注意力时。更多信息见 KV cache 资源。

下面介绍另一种常见的解码加速技术：推测解码。

17.1.2　推测解码（Speculative Decoding）

推测解码也称为辅助生成（assisted generation），由 Google 研究人员于 2022 年提出。研究人员观察到，大多数 token 都比较容易预测。例如，在句子“Once upon a time there lived a pony”中，给定前面的词，大多数词都很容易预测，只有“Once”和“pony”可能比较困难。那么，为什么不让一个更小、更快的模型生成容易的 token，再让一个更大、更慢但更聪明的模型处理困难的 token 呢？遗憾的是，我们事先不知道哪些 token 容易、哪些 token 困难，因此作者提出了如下方法（见图 17-2）。

- 在每个生成步骤中，把包含 token c_1 到 c_t 的上下文交给小模型——称为草稿模型（draft model）或辅助模型（assistant model）——让它逐个生成 n 个额外 token d_{t+1} 到 d_{t+n}。这段额外序列称为草稿（draft）。由于草稿模型很小，生成草稿很快。实际使用时，推测解码通常会结合 KV 缓存，因此小模型每次真正只接收一个 token；为简化说明，这里假设没有使用 KV 缓存。

- 接着把完整序列（上下文加草稿）交给大模型，即目标模型（target model）。序列包括 token $c_1, \ldots, c_t, d_{t+1}, \ldots, d_{t+n}$；如果使用 KV 缓存，则只需传入 $c_t, d_{t+1}, \ldots, d_{t+n}$。这一步同样很快，因为目标模型可以并行处理所有 token，而不是按顺序处理。

- 对于每个输入 token，目标模型输出 o_1 到 o_{t+n}，其中输出 token o_i 是在假设之前所有 token 都正确时，对位置 i+1 的预测。因此，我们必须验证草稿 token 是否正确：对于从 t+1 到 t+n 的每个位置，检查 d_i 是否与 o_{i-1} 匹配。如果全部正确，就可以安全地使用所有草稿 token d_{t+1} 到 d_{t+n}，甚至可以追加输出 token o_{t+n}，因为它是目标模型对位置 t+n+1 的预测。这样只用草稿模型 n 步和目标模型 1 步，就生成了 n+1 个 token；更重要的是，结果与直接让目标模型生成 n+1 个新 token 完全相同。

- 如果某些草稿 token 不正确，即它们与目标模型的预测不匹配，会怎样？设第一个错误的草稿 token 是 d_k（图中示例为 token “I”），那么对应的输出 token 以及它后面的所有输出 token 都无效，因为错误草稿 token 之后的“前面 token 都正确”这一假设不成立。不过，我们仍然可以安全使用截至此处的草稿 token（例如“DEF”），以及最后一个有效的

输出 token o_{k-1}（例如“G”）。最坏情况是第一个草稿 token d_{t+1} 就错误；即便如此，至少仍能得到一个新 token o_t，也就是对位置 t+1 的预测。

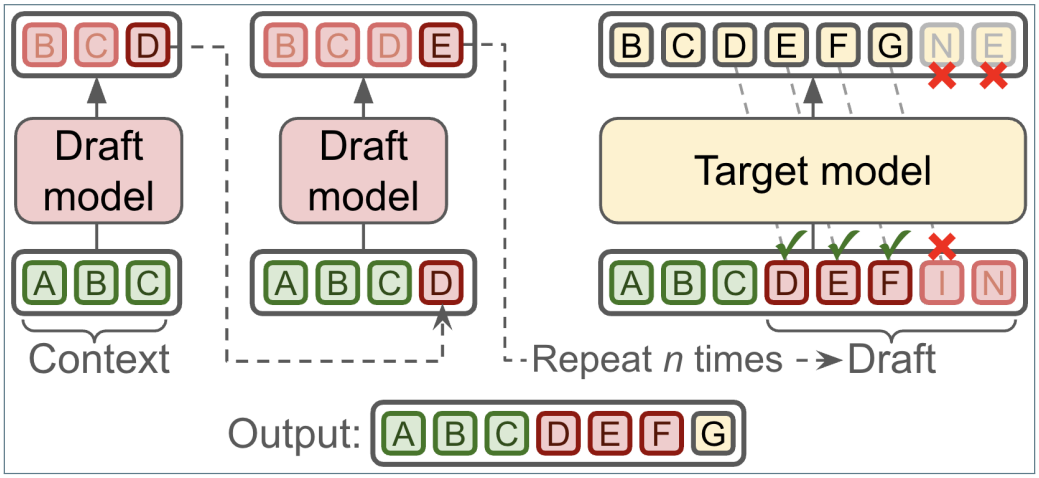

图 17-2: 推测解码：草稿模型生成草稿序列，目标模型验证草稿序列（Speculative decoding: the draft model generates a draft sequence and the target model verifies it）

不过，以上过程只适用于贪心解码，即总是选择 logit 最大的 token。第 15 章介绍过，通常更适合随机采样 token：典型做法是使用把 logit 除以温度后得到的 softmax 概率分布进行采样，这样生成的文本通常更自然，也不容易出现奇怪的重复。因此，目标模型有时应该接受并非第一选择、但仍然“足够好”的 token。理想情况下，推测解码得到的 token 分布应与只使用目标模型时完全相同。幸运的是，这是可以做到的！作者提出了称为推测采样（speculative sampling）的采样策略，并从数学上证明它与目标模型的 token 分布相同。具体过程如下：

- 生成位置 i 的草稿 token 时，运行草稿模型，得到下一个 token 的概率分布 $q_i(x)$，再从中随机采样一个 token，记为 d_i。
- 生成草稿序列后，把“提示加草稿”组成的完整序列输入目标模型，得到每个位置 i 的下一个 token 概率分布 $p_i(x)$。但此时先不要采样输出 token。
- 对每个草稿 token d_i，如果 $q_i(d_i) \leq p_i(d_i)$，就接受该 token；如果 $q_i(d_i) > p_i(d_i)$，则以概率 $p_i(d_i)/q_i(d_i)$ 接受，否则拒绝。
- 只要草稿 token 被接受，就继续检查下一个草稿 token。如果所有草稿 token 都被接受，就可以从目标模型最后输出的概率分布中免费采样一个 token，于是总共得到 n+1 个新 token。
- 如果第 k 个草稿 token 被拒绝，就立即停止，并拒绝位置 i>k 的所有草稿 token。此时仍可以从目标模型免费采样一个 token，但使用稍作调整的分布 $p'_i(x) = \max(0, p_i(x) - q_i(x))/s_i$，其中 s_i 是使所有概率之和为 1 的归一化常数。这样可以避免过度采样草稿模型本来就能直接采样的 token。

草稿序列应该多长？如果太短，几乎无法带来加速；如果太长，大多数草稿 token 都会被丢弃，在生成和验证草稿时都会浪费计算。Hugging Face Transformers 库支持几种选择草稿长度（也称推测前瞻长度或 speculative lookahead，SL）的策略：

- 使用固定草稿长度：这是一个可以调节的超参数。
- 使用简单启发式方法：如果某一步的所有草稿 token 都被接受，就增加下一步的草稿长度；否则减少下一步的长度。
- 使用动态推测解码：持续生成草稿 token，直到草稿模型的置信度低于给定阈值，即草稿 token 的估计概率过低。这是默认策略。

使用 Hugging Face Transformers 实现推测解码非常简单，只需把草稿模型通过 **assistant_model** 参数传给 **generate()**。也可以选择设置动态推测解码的草稿模型置信度阈值。下面的代码使用原样保留的 Transformers API：

```
from transformers import AutoModelForCausalLM, AutoTokenizer
target_model = AutoModelForCausalLM.from_pretrained("facebook/opt-350m",
                                                    device_map="auto")
draft_model = AutoModelForCausalLM.from_pretrained("facebook/opt-125m",
                                                   device_map="auto")
tokenizer = AutoTokenizer.from_pretrained("facebook/opt-350m")
prompt = "Once upon a time there lived"
inputs = tokenizer(prompt, return_tensors="pt").to(target_model.device)
outputs = target_model.generate(**inputs, max_new_tokens=100, do_sample=True,
                                temperature=1, assistant_model=draft_model)
result = tokenizer.decode(outputs[0], skip_special_tokens=True)
```

目标模型和草稿模型必须使用同一个 tokenizer，否则会报错。如果你非常清楚自己在做什么，可以在调用 **generate()** 时设置 **tokenizer** 和 **assistant_tokenizer** 参数；但要注意，如果词汇表不同，辅助模型的建议很可能很差，从而拖慢文本生成。

推测解码能带来多少加速？首先取决于模型组合：如果草稿模型并没有比目标模型快多少，整体速度甚至可能更慢。因此草稿模型应该小且快，但也不能太小，否则它的预测过于粗糙，目标模型会拒绝大多数草稿 token，完全无法加速，反而更慢。简言之，需要尝试不同的草稿模型，直到找到适合目标模型的组合。经验上，最佳草稿模型通常只有目标模型大小的 1/10 到 1/20。

加速效果还取决于任务：任务越容易，草稿模型越容易生成高质量草稿，加速效果越好。例如，Hugging Face 研究人员使用 Meta 的 OPT-6.7B 作为目标模型、OPT-125M 作为草稿模型，在摘要任务上获得了 1.8 倍加速；但在开放式生成任务上加速只有 1.2 倍。

草稿-验证并不是并行化解码的唯一方法。下面讨论几种主要方法。

17.1.3　并行化解码的主要方法（Main Approaches to Parallelize Decoding）

一项发表于 2025 年 8 月的综述列出了 130 多篇并行化文本生成的论文，并将它们归为六类：

草稿与验证（Draft and verify）

包括推测解码及其多种变体。

分解与填充（Decompose and fill）

核心思想是把问题拆成互相独立的任务，再并行处理。例如，给定提示“Tell me 10 chicken jokes”，可以并行运行 LLM 来生成每个笑话；给定提示“Which of these words come from French: mayor, state, cuisine, …”，可以并行处理每个单词；给定提示“Tell me a bedtime story”，可以先生成故事梗概，再根据梗概并行撰写各个章节。

多 token 预测（Multiple token prediction，MTP）

这种方法训练或微调 LLM，使它一次生成多个 token，产生一段草稿，再像推测解码一样验证草稿；不需要额外的草稿模型。一种做法是在提示末尾追加特殊的前瞻 token。例如，对于提示“Once upon”，可以输入“Once upon <lookahead> <lookahead> <lookahead>”。如果 LLM 被微调为在看到前瞻 token 时预测后续 token，它可能预测出“upon a time there was”，再对这些预测进行验证。预测越准确，加速效果越好。

一步生成（One-shot generation）

一步生成模型只用一次并行步骤就产生完整输出，速度极快；但由于所有生成 token 彼此独立，通常会出现重复、漏词和语义不连贯。可以先生成一个模板，再把模板交给模型生成最终输出，以改善质量。

掩码生成（Masked generation）

模型从一条完全被掩码的序列开始，逐步预测其中一部分词。每个生成步骤预测多个被掩码 token，同时细化之前的部分预测，逐渐提高连贯性并修正错误。

基于编辑的细化（Edit-based refinement）

编辑模型先快速生成初稿，再逐步改进文本；该模型经过训练，可以在整条序列上并行执行插入、删除或替换。

动态批处理与在途批处理（Dynamic Batching and In-Flight Batching）

如果服务足够受欢迎、每秒收到许多请求，就应该考虑动态批处理：请求到达时不立即处理，而是先排队，并以较短间隔按批次处理。增大批次可以更充分地利用 GPU，提高吞吐

量，但也会增加队列中的平均等待时间，从而增加延迟。因此，理想做法是根据传入流量动态调整批次大小和处理频率。这种技术适用于所有神经网络，而不只是 LLM。

对于 LLM，还可以使用请求调度器，把长度相近的请求分到同一批中，从而减少为了填充较短序列而使用的掩码 token。更进一步，可以实现在途批处理（in-flight batching，也称连续批处理 continuous batching）：每个生成步骤都会有新请求加入批次，也会有已经完成的请求离开批次。因此，不同请求在同一生成步骤中可能处于不同位置，例如一个请求生成第一个 token 的同时，其他请求已经生成到第三个 token。这样完全不需要掩码，提高了 GPU 利用率，也降低了长请求阻塞短请求的风险。它还提供了更细粒度的调度能力：如果某个请求生成时间过长，调度器可以暂时把它放回队列，让其他请求先处理。

Hugging Face Text Generation Inference（TGI）、vLLM、TensorRT-LLM、SGLang 等多个库和工具包都实现了在途批处理。

17.2　加速多头注意力（Speeding Up Multi-Head Attention）

多头注意力是 Transformer 中计算成本最高的组件之一。加速 MHA 主要有四种思路：

- 只让每个查询 token 关注部分键 token，而不是所有键 token：这称为稀疏注意力。
- 修改缩放点积注意力公式，快速近似结果，而不是精确计算：这称为近似注意力。
- 通过在注意力头之间共享 K、V 等投影来简化 MHA 架构。
- 充分利用硬件架构，使用 FlashAttention。

下面依次介绍这些方法，先从稀疏注意力开始。

17.2.1　稀疏注意力（Sparse Attention）

在标准的带掩码 MHA 层中，每个查询 token 都会关注其位置之前的每一个键 token（见图 17-3 左侧）。需要关注的 token 太多了：为什么不只关注其中一部分？下面介绍几种基于这一思想、影响较大的架构：Sparse Transformer、Longformer、BigBird 和 Routing Transformer。

Sparse Transformer

OpenAI 研究人员在 2019 年的一篇论文中提出了 Sparse Transformer 架构。它是一个仅解码器架构：一半注意力头只能关注 n 个局部 token（例如最后 256 个 token），另一半只能全局关注每隔 n 个 token 的位置。这个 n 称为步幅（stride），作者建议将它设置为接近最大序列长度的平方根，以获得最高效率。作者提出了两种变体：

- 步幅稀疏注意力（strided sparse attention，见图 17-3 中间）：局部掩码和全局掩码都会随着查询 token 平滑移动。作者发现，这种变体最适合图像以及具有清晰周期结构、且周期与步幅对齐的数据。

- 固定稀疏注意力（fixed sparse attention，见图 17-3 右侧）：把查询序列划分为每块 n 个 token 的固定区块，局部掩码限制在查询所在区块内，全局掩码则包含每个区块的最后一个键 token。这种变体更适合文本和其他没有明显周期性的数据。

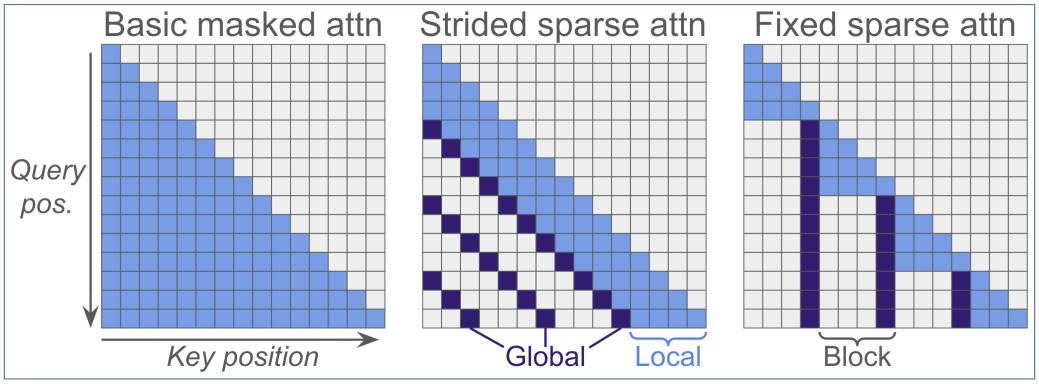

图 17-3: 普通因果注意力掩码（左）以及 Sparse Transformer 的两种变体（步幅稀疏注意力居中，固定稀疏注意力居右）。彩色单元表示查询 token 可以关注的键 token；Sparse Transformer 中一半注意力头使用蓝色掩码，另一半使用深紫色掩码（Regular causal attention mask and two variants for the Sparse Transformer）

使用这种方法可以大幅降低计算复杂度：每个查询 token 最多关注 n 个局部 token，再加上最多 i/n 个全局 token，其中 i 是查询 token 的位置。例如，窗口大小 n=256 时，长度为 10,000 的序列总共只需计算约 500 万个注意力分数，而普通注意力需要计算 1 亿个，速度提高约 20 倍！更一般地说，该架构的复杂度为 $O(L_q\sqrt{L_{\text{max}}})$，远低于 $O(L_qL_k)$。

不过，加速也有代价：信息必须经过全局 token 才能到达非局部 token，这可能造成信息瓶颈。实际表现是，一些关键的非局部信息有时会被遗漏。例如，如果一篇文章的结论必须引用正文中的某个细节，引用可能会不准确。Sparse Transformer 最适合摘要或长文档分类等任务，但在文本蕴含或推理等高度依赖细节的任务上可能表现不佳。

接下来看看 Longformer 架构，看看能否做得更好。

Longformer

2020 年 4 月，Allen Institute for AI 的研究人员提出了 Longformer（long-document transformer，长文档 Transformer），这是另一种基于稀疏注意力的 Transformer 架构。与 Sparse Transformer 不同，Longformer 是仅编码器架构，基于 RoBERTa，因此它的注意力掩码允许每个查询 token 关注其后面的键 token，掩码关于主对角线对称（见图 17-4）。论文还提出了 Longformer Encoder-Decoder（LED），架构类似原始 Transformer，但在编码器中使用稀疏注意力。

Longformer 使用简单的局部注意力滑动窗口，类似步幅稀疏注意力，但窗口以查询 token 的位置为中心。窗口宽度通常很大，例如 512 个 token，因此 Longformer 对小文档没有优势。

更重要的是，Longformer 使用数量固定、并且针对任务精心选择的全局 token。例如，对于分类任务，可以只把第 15 章介绍的 class token `[CLS]` 作为全局 token（见图 17-4 左侧）：它可以“看到”每个 token，而每个 token 也可以“看到”它，因此它充当远距离信息传播的中继站。对于多项选择问答（multiple-choice question answering，MCQA），把所有问题 token 设置为全局 token 可能效果更好，因为查询 token 往往需要引用问题细节。可以针对自己的任务尝试不同的全局 token，观察哪种方式最有效。

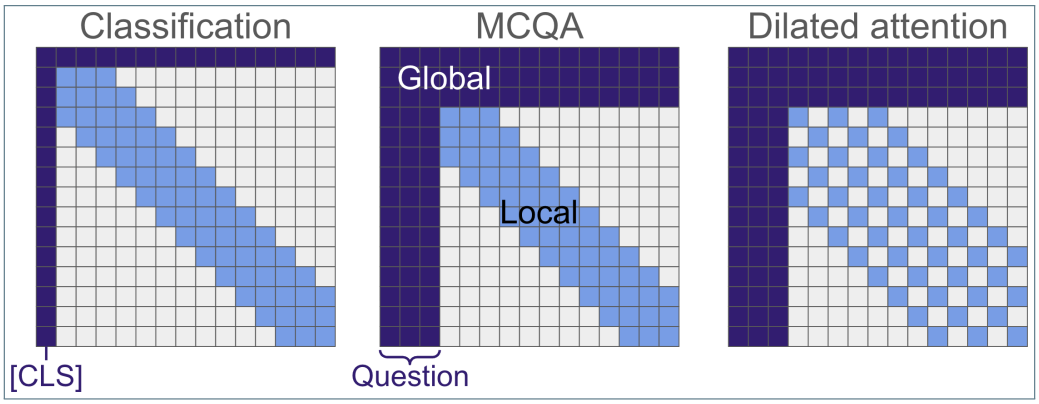

图 17-4: Longformer 注意力掩码：全局 token 由任务决定（左、中）；Longformer 还使用膨胀注意力（右）（Longformer attention mask: task-motivated global tokens and dilated attention）

Longformer 论文还提出，在较低层使用较短窗口，在较高层使用较长窗口：低层通常关注小尺度模式，这样做可以节省计算，同时不会明显降低模型精度。

此外，作者提出对局部注意力使用膨胀窗口（dilated window，见图 17-4 右侧），灵感来自第 12 章介绍的 WaveNet 架构。增加高层的膨胀率，可以在不让计算预算爆炸的情况下扩大每个查询 token 的感受野。

同样的稀疏注意力机制也可以用于解码器：只需在局部掩码和全局掩码之上再施加因果掩码即可。

由于全局 token 数量固定、局部窗口采用膨胀注意力，Longformer 可以线性扩展到很长的文档，而且通常比 Sparse Transformer 表现更好。还能做得更好吗？

BigBird

2020 年 7 月，Google 研究人员提出了 BigBird。可以把它理解为“去掉膨胀、加入随机注意力的 Longformer”：与 Longformer 一样，每个查询 token 会关注局部 token 窗口（但窗

口不膨胀）和固定数量的全局 token；除此之外，还会关注少量随机 token。例如图 17-5 中，每个查询 token 可以关注局部 token、全局 token 和两个随机 token。

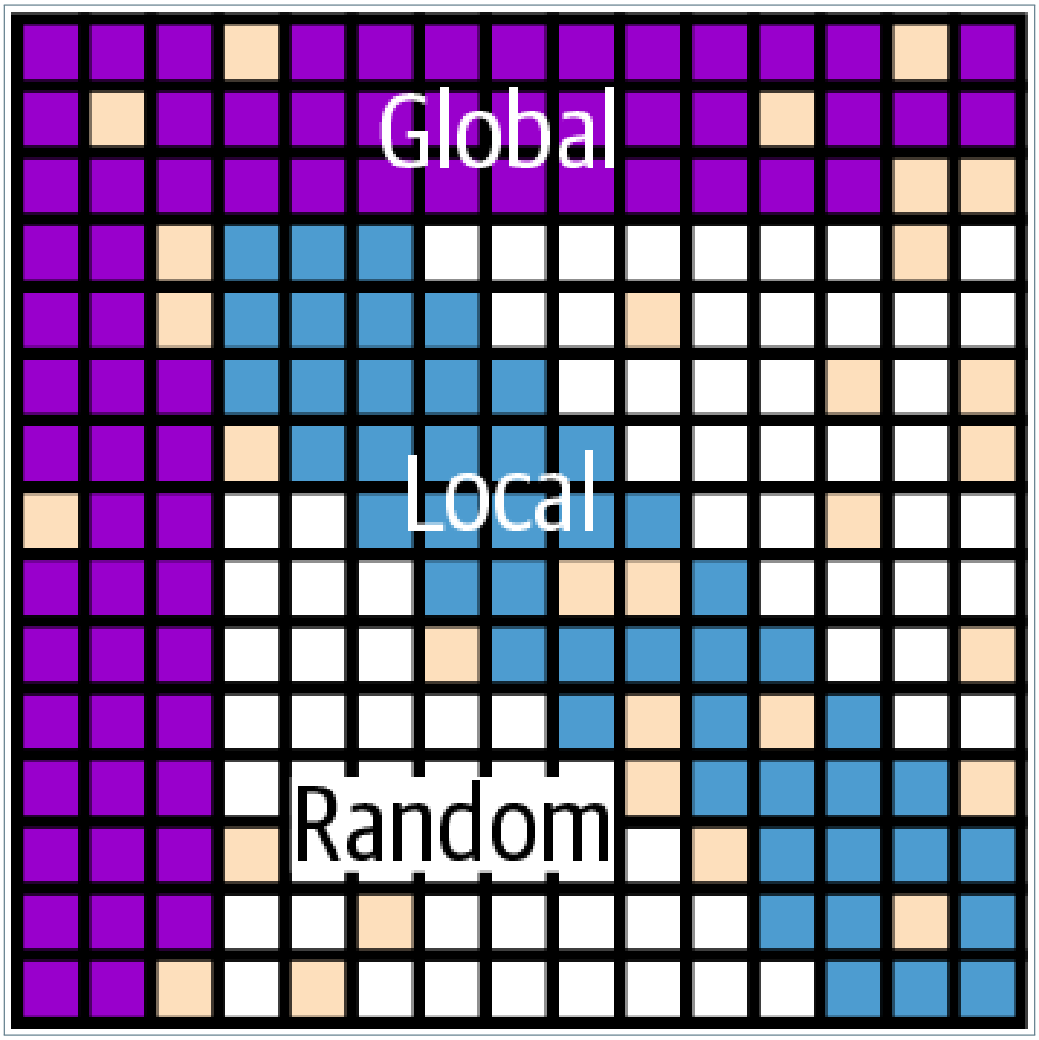

图 17-5: BigBird 注意力掩码：类似 Longformer，但去掉膨胀注意力并加入随机注意力（BigBird attention mask: similar to Longformer minus dilation plus random attention）

作者提出了两个变体：BigBird-ITC（internal transformer construction，内部 Transformer 构造）和 BigBird-ETC（extended transformer construction，扩展 Transformer 构造）。前者从已有 token 中选择全局 token，后者在序列开头增加少量额外的全局 token，例如 `[CLS]`。实践中，这些额外 token 会牺牲一点速度，但换来更高的准确率。

重要的是，论文证明，引入随机注意力后，BigBird 具有很强的理论保证：虽然它对长序列呈线性扩展，但表达能力与原始 Transformer 一样强。特别是，它在理论上可以建模任意连续函数。实际上，只要反复调用模型、把输出重新输入模型，并且浮点精度足够，它

甚至可以执行任意算法；在这些条件下，BigBird 是图灵完备的。不过，BigBird 需要比原始 Transformer 更多的层，才能达到相同的准确率，因为信息必须经过多层传播，才能充分穿过各个 token。总体而言，BigBird 很适合长文档任务，通常优于 Longformer。

每个注意力头使用不同的随机注意力掩码，这会减少远距离 token 传播信息所需的层数。每个随机掩码在创建模型时采样一次，之后保持不变；这样模型更容易学会利用这些连接，在远距离 token 之间传递信息。

Hugging Face Hub 上有多个 BigBird 变体。例如：

```
from transformers import pipeline
model_id = "google/bigbird-roberta-base"
pipeline = pipeline(task="fill-mask", model=model_id)
pipeline("She took some [MASK] medicine as she was feeling ill.")
```

下面介绍最后一种基于稀疏注意力的 Transformer 架构：Routing Transformer。

Routing Transformer

Routing Transformer 由 Google 研究人员于 2020 年 3 月提出，比 Longformer 早几周。

前面介绍的稀疏注意力机制，都严格按照位置决定哪些查询 token 关注哪些键 token。相比之下，Routing Transformer 的每个注意力头在每次前向传播中，先把查询 token 和键 token 放在一起分成 k 个簇，然后分别在每个簇内计算缩放点积注意力。这意味着每个查询 token 只能关注同一簇中的键 token（见图 17-6）。由于查询 token 通常主要关注最相似的键 token，而相似 token 很可能属于同一个簇，这种限制不会显著降低质量。

聚类依据的是 token 之间的相似性，而不是位置。不过，每个 token 都通过位置编码携带位置信息，因此聚类同时受到语义和位置的影响，并且会偏向局部 token。这通常是好事，因为局部 token 往往更相关。

聚类步骤使用 mini-batch k-means 的一个变体（第 8 章介绍过）：每个注意力头有 k 个可训练质心，每个质心都是与 token 维度相同的向量。在前向传播时，对于每个质心，距离该质心最近的 s 个 token 组成一个簇；s 大致等于 token 数量除以 k。这样得到 k 个大小相同、可以部分重叠的簇。训练时，每个 k-means 质心在每次迭代中都使用最近 s 个 token 的指数移动平均进行更新；推理时质心固定。

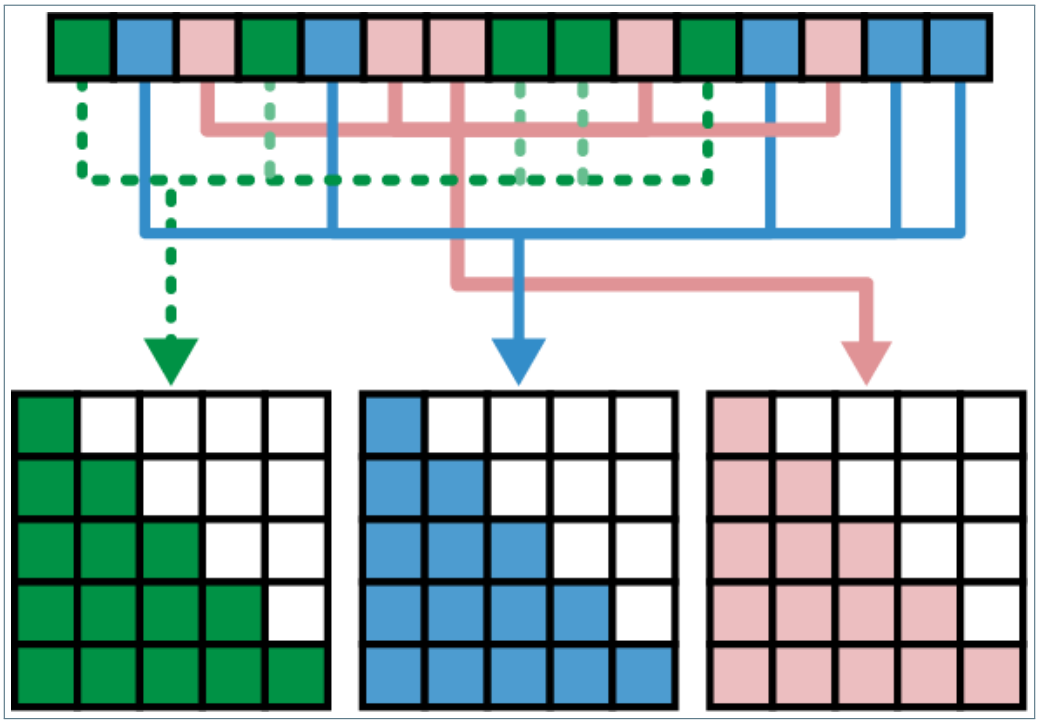

图 17-6: Routing 注意力：查询 token 只能关注同一簇中的键 token（Routing attention: query tokens can only attend to key tokens within the same cluster）

聚类前，每个注意力头会把查询 token 和键 token 归一化为单位向量。这样就可以用点积衡量 token 与质心之间的相似度，从而加快 k-means。归一化后的 token 也用于计算缩放点积注意力。

即使在短序列上，这项技术也能取得很好的结果。作者建议把簇数 k 设为接近最大序列长度的平方根，这样 Transformer 的复杂度就像 Sparse Transformer 一样为 $O(L_q\sqrt{L_{\text{max}}})$。不过，Routing Transformer 的实现相当复杂，因此没有获得广泛采用。

我们已经讨论了四种稀疏注意力 Transformer：Sparse Transformer、Longformer、BigBird 和 Routing Transformer。实际上，如果第 16 章没有介绍 Swin Transformer，它也应该属于这一节。这些模型都能大幅加速 MHA、显著降低内存使用，并对大型输入呈线性或至少次二次扩展；但对于对长距离细节敏感的任务，它们可能表现不佳。

接下来介绍一种完全不同的注意力加速方法。

17.2.2　近似注意力（Approximate Attention）

近似注意力方法通过近似计算缩放点积注意力公式，降低 MHA 的计算复杂度和内存使用量。具体可以修改公式本身以快速得到近似结果，也可以压缩输入。第 16 章已经看过后者的两个例子：

- 金字塔视觉 Transformer（PVT）使用空间缩减注意力（spatial reduction attention，SRA），沿空间维度缩小 K 和 V。这样计算复杂度和内存使用量为 $O(L_q L_k / r^2)$，其中 r 是缩减因子，而普通注意力为 $O(L_q L_k)$。
- Perceiver 使用交叉注意力把输入序列压缩为长度固定且更短的潜在序列，长度记为 ℓ。这样它可以随序列长度线性扩展，复杂度为 $O(\ell L_k)$。

本节还将讨论四种基于近似注意力的有影响力架构：Reformer、Linformer 和 Performer。先从 Reformer 开始。

Reformer：基于 LSH 的近似注意力（Reformer: LSH-based Approximate Attention）

Reformer 由 Google 和加州大学伯克利分校的研究人员于 2020 年 1 月提出。它结合多种 Transformer 加速技术，其中包括局部敏感哈希（locality-sensitive hashing，LSH）注意力。LSH 注意力的计算复杂度为 $O(L_q \log(L_k))$，而不是 $O(L_q L_k)$，同时只会轻微降低准确率。

在缩放点积注意力公式 $\text{Attention}(Q, K, V) = \text{softmax}(QK^\intercal / \sqrt{d_k})V$ 中，计算代价很高的 $QK^\intercal$ 位于 softmax 函数内部。softmax 的输出主要由输入矩阵中最大的项决定；在这里，这些项对应点积最大的查询/键组合，也就是最相似的组合。因此，如果能快速找出最相似的查询/键对，就不必计算所有可能组合的 $QK^\intercal$，只需计算最相似的组合。结果不会完全精确，因为忽略了不太对齐的查询/键对的贡献，但应该非常接近。

1998 年，Piotr Indyk 和 Rajeev Motwani 提出了一种这样的算法：局部敏感哈希（LSH）。给定一组高维向量，LSH 可以快速把邻近向量分到不同簇中。为此，每个向量都会经过一个哈希函数，直接输出簇 ID。不能使用 MD5 或 SHA256 之类的普通哈希函数（再对簇数取模），因为这样只会伪随机地分组，没有任何用处。我们需要的是一种对局部敏感的哈希函数：相似输入向量倾向于产生相似的哈希值。

随着时间推移，人们提出了多种 LSH 函数。其中，cross-polytope LSH，通常称为角度 LSH（angular LSH），速度快且准确率高。它由 Alexandr Andoni 等人于 2015 年提出。过程如下：

- 如果目标簇数是 k，先从高斯分布中独立采样 r_1 到 $r_{k/2}$，创建随机向量列表 L。
- 然后把相反向量 $-r_1$ 到 $-r_{k/2}$ 追加到列表中。此时列表包含 k 个向量。
- 对每个待哈希向量 v，找到与它最对齐的 L 中向量，并把该向量在 L 中的索引作为 v 的哈希值。例如，如果 v 与 L 中索引 42 的向量最对齐，那么 v 的哈希值就是 42。

如果两个向量 v_1 和 v_2 很相似，即几乎对齐，那么它们很可能都与 L 中同一个向量最对齐，因此会拥有相同哈希值（见图 17-7）。相反，不相似的向量很可能得到不同哈希值。

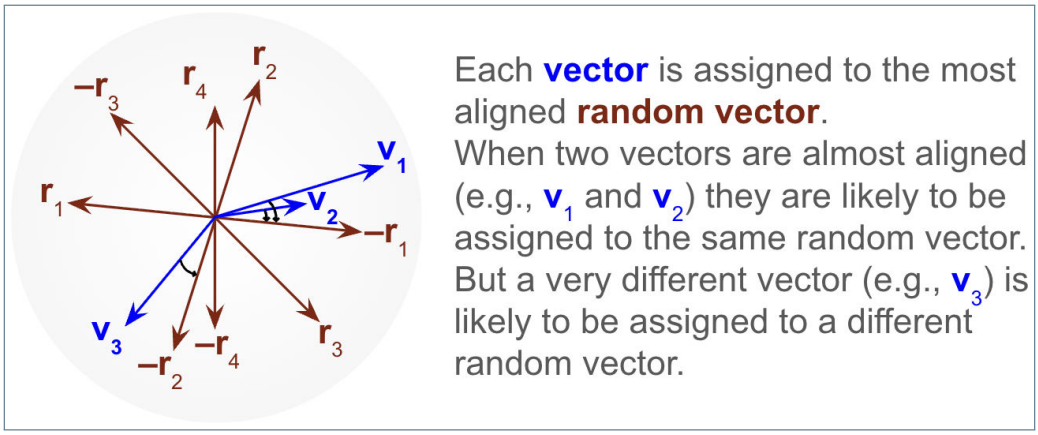

图 17-7: 角度 LSH：两个相似向量 v1 和 v2 很可能与同一个随机向量最相似（Angular LSH: two similar vectors v1 and v2 are likely to share the same most similar random vector）

下面使用 PyTorch 高效实现角度 LSH：

```
import torch
import torch.nn as nn
import torch.nn.functional as F

def angular_lsh(vectors, k):
    R = torch.randn(vectors.size(-1), k // 2, device=vectors.device)
    normalized_vectors = F.normalize(vectors, p=2.0, dim=1)
    V_proj = normalized_vectors @ R
    V_concat = torch.cat([V_proj, -V_proj], dim=1)
    return torch.argmax(V_concat, dim=1)
```

这段代码首先生成随机矩阵，其列向量是 r_1 到 $r_{k/2}$；然后对输入矩阵归一化，使每一行长度为 1；接着把两个矩阵相乘。由于输入向量已经归一化、随机向量长度也相近，结果矩阵大致包含每个输入向量与随机向量之间的余弦相似度（乘以一个常数）。最后沿水平方向拼接这个矩阵的相反数，于是免费获得输入向量与 r_1 到 $r_{k/2}$ 的相反向量之间的相似度，并返回每行中相似度最高的索引。

尝试把 16 个 512 维随机向量分成 k=4 个簇：

```
>>> torch.manual_seed(42)
>>> vectors = torch.rand(16, 512)
>>> angular_lsh(vectors, k=4)
tensor([2, 2, 0, 3, 0, 2, 2, 2, 2, 1, 1, 3, 3, 1, 2, 1])
```

接下来，可以使用角度 LSH 把所有查询 token 和键 token 放在一起聚类，然后让每个查询 token 只关注同一簇中的键 token。这样会很快，但近似程度太高：角度 LSH 有时会把相似向量分到不同簇中。如果查询 token 所在的簇不包含某些最相似的键 token，softmax 就会缺少最重要的输入，结果会不够精确。

解决方法很简单：可以独立运行多轮角度 LSH（例如 4 轮），让每个查询 token 关注至少在其中一轮与它分到同一簇的所有键 token。这样就能比较有把握地确保每个查询 token 会关注所有相似的键 token。

Reformer 还使用了几种高级技术：

- 可逆残差层（reversible residual layer），第 12 章介绍过。
- 分块前馈层（chunked feedforward layer）：在前馈层中逐块处理序列，降低峰值内存使用量。
- 轴向位置编码（axial positional encoding）：用两个更小的位置嵌入矩阵替代大型位置嵌入矩阵，类似某些视觉模型中的 x、y 位置嵌入。
- 共享查询/键（Shared Query/Key，Shared-QK）：不再使用同一输入序列的两个不同投影来生成每个注意力头的查询和键，而是使用同一个投影，使 Q=K。这与 LSH 注意力结合得很好，因为它更容易混合查询和键；但也会降低模型的灵活性，最终降低准确率。

Reformer 证明了近似注意力是可行的，并推动研究社区继续寻找节省计算和内存的技巧。几个月后，另一种近似注意力模型 Linformer 出现了。与 Reformer 相比，它非常简单。

Linformer：低秩近似注意力（Linformer: Low-Rank Approximate Attention）

Linformer 由 Meta 研究人员于 2020 年 6 月提出。理解它的工作方式，先把缩放点积注意力公式改写为 $\text{Attention}(Q, K, V) = PV$，其中 $P = \text{softmax}(QK^\intercal/\sqrt{d_k})$。矩阵 P 称为上下文映射矩阵，形状为 $L_q \times L_k$。作者分析了多个预训练 Transformer，观察到这个矩阵总是接近一个非常低秩的矩阵。在自注意力层中（$L_q = L_k$），P 的秩通常接近 $r \approx \log(L_q)$。这意味着可以在不损失太多信息的情况下大幅压缩 P。

如何压缩？可以使用 PCA 压缩 P，但计算 P 的成本正是我们想要降低的部分，这样做会白白浪费计算量并损失准确率。因此，作者提出使用两个在训练中学习的线性投影 E 和 F 缩短键和值。要注意，这会缩短键和值序列的长度，即 token 数量，而不是降低每个 token 的维度。这种低秩注意力机制如下式所示，并见图 17-8。

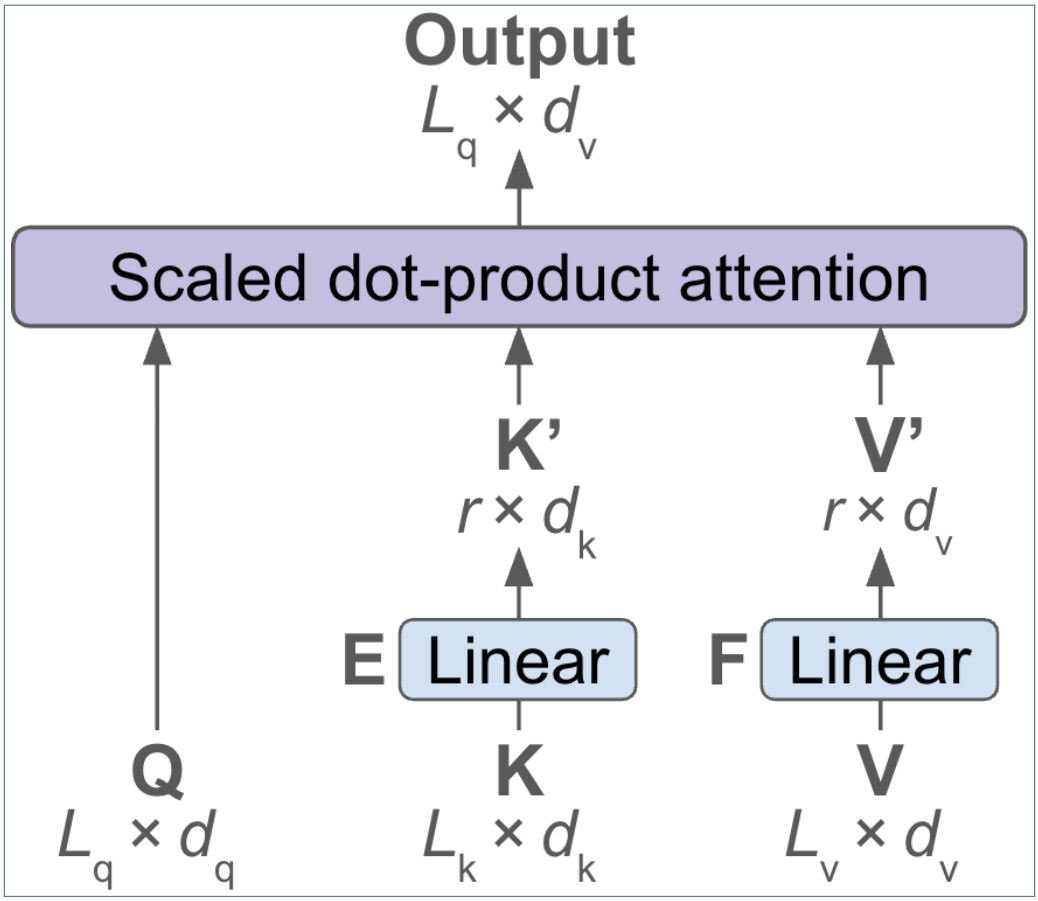

图 17-8: 低秩注意力：使用线性投影缩短键和值（Low-rank attention: the key and value are shortened using linear projections）

$$\text{Attention}(Q, K, V) \approx \text{softmax}\left(\frac{QK'^{\intercal}}{\sqrt{d_k}}\right)V', \qquad K' = EK, \quad V' = FV \tag{17-1}$$

公式 17-1　Linformer 的低秩近似注意力（Linformer's low-rank approximate attention）

其中 $K' = EK$，$V' = FV$。这就是普通的缩放点积注意力公式，只是用压缩后的键 K′ 和值 V′ 替代 K 和 V。在该公式中：

- E 是形状为 $r \times L_k$ 的可训练矩阵。
- F 是形状为 $r \times L_k$ 的可训练矩阵（回顾一下，$L_v = L_k$）。
- r 是压缩后的序列长度，是可以调节的超参数，例如 64。

由于 L_k 会随输入序列变化，作者建议使用具有足够列数、可以覆盖最长序列的可训练矩阵 E′ 和 F′，然后在运行时截断到适当的列数。

作者还尝试了几种参数共享技术，以进一步降低内存使用量：

- 在每层的所有注意力头之间共享 E 和 F。
- 使用同一个投影矩阵 G 同时压缩键和值，即 $K' = GK$、$V' = GV$，并在每层的所有注意力头之间共享 G。
- 在所有注意力头和所有层之间共享同一个用于压缩键和值的矩阵 G。

键和值序列现在都很短，每个只有 r 个 token，因此低秩注意力比普通注意力快得多，也更节省内存。更重要的是，r 是固定的，在训练前确定且不会改变，所以低秩注意力可以随长序列线性扩展，这也解释了 Linformer 名称的含义。

不过，对于给定的 Linformer 模型，虽然 r 固定且可以线性扩展到长序列，但 r 的最优值大致随输入序列长度的对数增长。因此，如果希望长序列上的质量保持可接受，就需要增大 r；也可以说，Linformer 实际上以 $O(L_q \log(L_k))$ 扩展。

总之，Linformer 扩展性好、实现相对简单，只要 r 不太小，准确率就接近 Reformer。它还启发了后续基于其他低秩近似方法的架构，例如 Nyströmformer。接下来介绍使用完全不同数学技术近似注意力的 Performer。

Performer：基于核的近似注意力（Performer: Kernel-Based Approximate Attention）

Performer 由 Google、DeepMind、剑桥大学和 Alan Turing Institute 的研究人员于 2020 年 9 月提出。与 Linformer 一样，Performer 通过快速近似缩放点积注意力公式，实现对长输入序列的线性扩展；但它采用了完全不同的方法。

这一节数学内容较多，如果你对数学细节不感兴趣，可以跳过。简而言之，Performer 使用核技巧快速近似点积注意力，因此可以随输入序列长度线性扩展。

要理解 Performer，先回顾 softmax 函数：它计算输入矩阵中每一项的指数，然后把每一行除以该行的和，从而保证每一行之和为 1。可以写成 $\text{softmax}(M) = D^{-1}A$，其中 $A = \exp(M)$，D 是对角矩阵，其主对角线上的每个值对应 A 中一行的和，D^{-1} 是它的逆矩阵，即把每个对角元素替换为其倒数。

因此，可以把缩放点积注意力改写为 $\text{Attention}(Q, K, V) = \text{softmax}(QK^\intercal)V = D^{-1}AV$，其中 $A = \exp(QK^\intercal)$。为简化公式，这里省略缩放因子 $1/\sqrt{d_k}$：假设 Q 和 K 都乘以该因子的平方根，即 $d_k^{-1/4}$，这样相乘后就得到所需因子 $d_k^{-1/4} \times d_k^{-1/4} = 1/\sqrt{d_k}$。

关键来了。作者定义了一个函数 φ，使得 $A = \exp(QK^\intercal) \approx \phi(Q)\phi(K)^\intercal$，从而得到如下近似注意力公式：

$$\text{Attention}(Q, K, V) \approx D^{-1}\phi(Q)\phi(K)^\intercal V \tag{17-2}$$

公式 17-2　Performer 的基于核的近似注意力（Performer's kernel-based approximate attention）

这里 Q、K 已经按 $d_k^{-1/4}$ 进行了缩放。

这个公式只是多个矩阵的乘积。由于矩阵乘法满足结合律，即 $ABC = (AB)C = A(BC)$，我们可以先计算 $G = \phi(K)^\intercal V$，再计算 $H = \phi(Q)G$，最后计算 Attention $\approx D^{-1}H$。重要的是，所有步骤都只涉及相对较小的矩阵（见图 17-9），完全不需要计算巨大的 $L_q \times L_k$ 矩阵 $QK^\intercal$。

具体来说，$\phi(K)^\intercal$ 是一个较小的 $m \times L_k$ 矩阵（L_k 可以很大，但 m 是可调节的小超参数，例如 256），V 是 $L_k \times d_v$ 矩阵（d_v 是值的维度，例如 64），因此 G 只有 $m \times d_v$ 大小。同样，$\phi(Q)$ 是一个较小的 $L_q \times m$ 矩阵，H 是一个较小的 $L_q \times d_q$ 矩阵。D^{-1} 虽然是大型的 $L_q \times L_q$ 矩阵，但它是对角矩阵，不需要太多计算和内存。总体上，近似注意力的计算量为 $O(Lmd)$，内存空间为 $O(Lm + Ld + md)$，其中 $L = L_k + L_q$，d 是模型维度，因此它对 L 呈线性扩展。

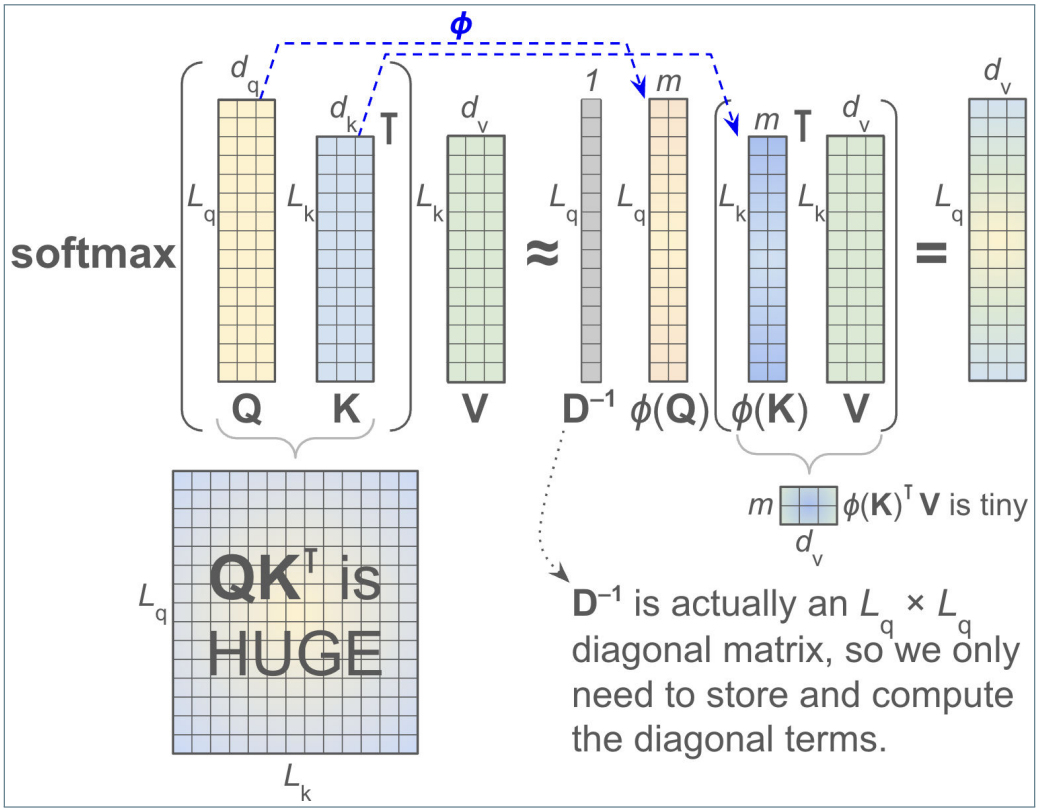

图 17-9: Performer 的基于核的注意力（Performer's kernel-based attention）

如果你读过在线附录 C 中关于支持向量机（SVM）的内容，可能会发现公式 17-2 是一个基于核的公式。核是一个函数 K(x,y)，它计算某个函数 f(x)·f(y) 的点积，其中 f 把输入向量

映射到另一个空间。在 SVM 中，我们通常不会真正计算 f(x) 或 f(y)，因为使用的核可以直接用 x 和 y 表示。事实上，我们甚至不需要知道 f 是什么；它往往只是把输入向量映射到更高维空间（有时甚至是无限维空间）的隐式函数。这样可以避免显式地把 x、y 映射到另一个空间，同时得到同样的结果 f(x)·f(y)，这称为核技巧（kernel trick）。

不过，Performer 恰恰反过来：它显式计算 f(x)·f(y)，以避免计算 K(x,y)。具体来说，它计算 $\phi(Q)\phi(K)^{\mathsf{T}}$，从而不必计算指数核 $K(X,Y) = \exp(QK^{\mathsf{T}})$。

困难在于找到一个函数 φ，使 $\phi(Q)\phi(K)^{\mathsf{T}} \approx \exp(QK^{\mathsf{T}})$。这正是 Performer 作者所做的工作。他们的方法名为 FAVOR+（Fast Attention Via positive Orthogonal Random features），实际上更加通用：它不仅适用于指数函数，还能为多种核函数提供这样的 φ。这里我们只关注指数核。指数核有时也被不太严谨地称为 softmax 核。

为了寻找 φ，作者研究了如下形式的函数族（公式 17-3 是 Performer 论文公式 5 的向量化形式，图 17-10 也表示了这一点）：

$$\phi(X) = h(X)\,[f_1(XW); f_2(XW); \cdots; f_\ell(XW)] \tag{17-3}$$

公式 17-3　Performer 作者研究的特征映射 φ 函数族（向量化形式）（The family of functions explored for the feature map φ）

其中：

- φ(X) 把 d 维空间中的 X 映射到 ℓm 维空间，即 ℓ 个长度为 m 的块；m 是超参数，例如 256，ℓ 是函数 f_1 到 f_ℓ 的数量。
- h(X) 为 X 的每一行返回一个缩放因子。
- f_1 到 f_ℓ 是逐元素应用的实函数，例如 exp。
- W 是任意不可训练的 d×m 矩阵，用于把 X 从 d 维映射到 m 维。W 可以是确定性的，也可以是随机的；如果是随机矩阵，每个列向量都独立地从同一分布中采样，通常使用标准高斯分布。
- **[·;·;⋯;·]** 表示沿水平方向拼接。

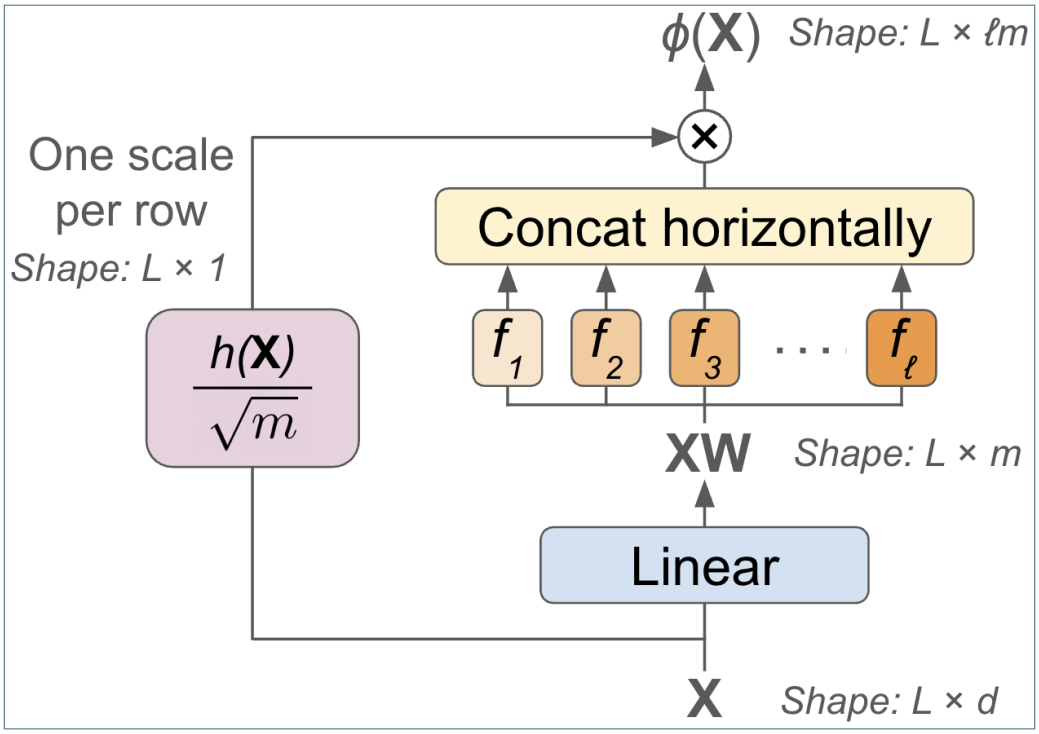

图 17-10: 公式 17-3 的可视化表示（A visual representation of Equation 17-3）

作者从数学上证明，下面这个函数族中的函数可以近似指数核：

- h(X) 对 X 的每一行 x_i 计算 $\exp(-\frac{1}{2}\|x_i\|^2)$，其中 $\|x_i\|$ 是向量长度。
- ℓ=1。
- $f_1(x) = \exp(x)$。
- W 是随机矩阵，每个元素独立地从标准高斯分布中采样：`W = torch.randn(d, m)`。
- m 是可调节的超参数，例如 256。增大 m 会提高近似精度，但也会线性增加内存和计算成本。

作者还提出了第二个函数，效率较低但精度更高：h(X) 对每行计算 $\frac{1}{2}\exp(-\frac{1}{2}\|x_i\|^2)$，$\ell$=2，$f_1(x) = \exp(x)$，$f_2(x) = \exp(-x)$。

W 和 m 与第一个函数相同。

下面使用 PyTorch 实现第一个函数：

```
def phi(X, W):
    squared_norms = X.square().sum(dim=-1, keepdim=True)
    return torch.exp(X @ W - squared_norms / 2) / W.size(-1) ** 0.5
```

这段实现支持超过两个维度的输入，这很有用，因为 Q、K 通常形状为 **[batch size, heads, length, dim]**，W 的形状为 **[heads, length, dim]**，因此可以同时处理批次中的所有序列和所有注意力头。代码还把 **exp(X @ W)** 和 **exp(-squared_norms / 2)** 合并到同一个指数运算中，更加高效。

遗憾的是，这个实现的数值稳定性不够好：如果 XW 中的某个值非常大，指数运算可能溢出为无穷大。解决方法是在计算指数前，从所有值中减去 XW 的最大值 v（对批次中的每个实例和每个注意力头分别处理）。这样输出会相差一个常数因子 **exp(-v)**，但这没有关系，因为后面会对 $\phi(Q)\phi(K)^\intercal$ 进行归一化。这种技术称为安全 softmax（safe softmax）。稳定版本如下：

```
def phi(X, W, dim_subtract_max=(-2, -1)):
    squared_norms = X.square().sum(dim=-1, keepdim=True)
    X_proj = X @ W
    max_vals = X_proj.amax(dim=dim_subtract_max, keepdim=True)
    return torch.exp(X_proj - max_vals - squared_norms / 2) / W.size(-1) ** 0.5
```

默认情况下，会从完整矩阵 XW 中减去最大值（对每个实例和每个注意力头分别计算）。如果把 **dim_subtract_max** 设置为 **-1**，就会针对每一行分别减去最大值，从而保留更多精度。计算 φ(Q) 时可以这样做，因为 $\phi(Q)\phi(K)^\intercal$ 的每一行都会独立归一化。

现在把所有部分组合到 FavorAttention 模块中：

```
class FavorAttention(nn.Module):
    def __init__(self, d_model, n_heads, n_features):
        super().__init__()
        self.d_head = d_model // n_heads
        W = torch.randn(n_heads, self.d_head, n_features)  # h, d, m
        self.register_buffer("W", W)
    def forward(self, Q, K, V):
        scale = self.d_head ** -0.25
        Qp = phi(Q * scale, self.W, dim_subtract_max=-1)
        Kp = phi(K * scale, self.W)
        D = Qp @ Kp.sum(dim=-2).unsqueeze(-1)  # B, h, Lq, 1
        Kp_T_V = Kp.transpose(-2, -1) @ V  # B, h, m, d
        return (Qp @ Kp_T_V) / (D + 1e-6)
```

构造函数生成随机矩阵 W，并把它注册为 buffer，这对保存模型很有用。**forward()** 首先计算标准点积注意力的缩放因子 $d_k^{-1/4}$，再对缩放后的 Q、K 调用 **phi()**；对于 query 使用 **dim_subtract_max=-1**。随后计算 D，它包含 $Q_p K_p^\intercal$ 每一行的和，但不实际构造巨大的

$L_q \times L_k$ 矩阵；最后完成公式 17-2，并除以 D 进行归一化，同时加上很小的 **1e-6**，避免除以零。

FAVOR+ 受到 2007 年一种名为随机 Fourier 特征（random Fourier features）的技术启发。该技术可以使用 $\phi(X) = \sqrt{2/m}\cos(XW + b)$ 近似多种核函数，其中 W 的列向量和向量 b 从标准高斯分布中采样。不过，这种方法仅适用于平移不变核，即 K(x,y) 只依赖 x-y；指数核不是平移不变的。

到这里，我们已经通过随机特征映射获得了快速注意力，这解释了 FAVOR+ 名称中的 FAV 和 R。加号表示 XW 被映射到正值，即使用 **exp(XW)**；由于指数核总是正的，这有助于提高近似精度。O 表示“正交”（orthogonal）：作者证明，把 W 的随机列向量调整为彼此正交，可以显著提高近似精度，尤其是 m 较小时；因此，如果希望进一步节省计算和内存，就可以减小 m。

作者建议使用 Gram-Schmidt 正交化。实现方法之一是先用 QR 分解把 W 分解为 W=QR，其中 Q 包含正交单位向量，R 是上三角矩阵；丢弃 R，只保留 Q；最后把 Q 乘以 $\sqrt{d}$，恢复随机向量原本的平均尺度。由于 d 维空间最多只能容纳 d 个正交向量，所以每次处理 d 列组成的块，使每个块包含 d 个正交向量。PyTorch 实现如下：

```
def orthogonalize(W):
    d_head = W.size(-2)
    W_orth = torch.cat([torch.linalg.qr(W_chunk)[0]
                        for W_chunk in W.split(d_head, dim=-1)], dim=-1)
    return W_orth * d_head ** 0.5
```

只需在构造函数生成 W 后，加入 **W = orthogonalize(W)** 即可。这个技术在数学上颇具挑战，但实现并不复杂，在处理很长的序列时尤其有效。

下面转向另一种 MHA 加速方法：在注意力头之间共享投影。

17.2.3　在注意力头之间共享投影（Sharing Projections Across Attention Heads）

MHA 层接收查询、键和值后，会分别对它们应用线性变换，再把结果分到所有注意力头中（见第 15 章图 15-4）。例如，有 h=8 个注意力头、输入维度 d=512 时，三个线性变换都会产生 512 维输出；把三组输出拆分后，就得到 8 个投影查询、8 个投影键和 8 个投影值，每个都是 64 维。

当我们说 query、key 或 value 时，有时不清楚指的是 MHA 层的输入，还是送入某个注意力头的投影。上下文通常足以说明含义；必要时，本节把 MHA 输入称为输入 query/key/value，记为 XQ、XK、XV；把送入某个注意力头的投影称为投影 query/key/value，记为 Q、K、V（若特指第 i 个头，则记为 Qi、Ki、Vi）。

这一过程依赖三个 d×d 的可训练投影矩阵，会带来大量模型参数和显著的投影计算成本。更重要的是，投影会占用大量内存，尤其是在生成文本的推理阶段：KV 缓存会随每个

新 token 增长，最终可能耗尽 VRAM，导致程序崩溃，或因为内存页交换而严重变慢。此外，为生成每个新 token 而读取完整 KV 缓存，很容易让内存带宽饱和，进一步拖慢模型。

我们真的需要所有这些投影吗？能不能让不同注意力头共享其中一些投影？前面已经看到，Reformer 让 query 和 key 共享同一个投影（Shared-QK），Linformer 作者也提出了多种共享 E、F 投影的方式。本节介绍三种新的投影共享方式：多查询注意力、分组查询注意力和多头潜在注意力。

多查询注意力（Multi-Query Attention，MQA）

2019 年，Noam Shazeer（原始 Transformer 架构的作者之一）发表了第一篇提出跨注意力头共享投影的论文。他建议让所有注意力头共享同一个投影键和投影值，并强调了这对文本生成的好处。由于每个注意力头仍然有不同的投影查询，这种技术被称为多查询注意力（MQA）。

虽然 MQA 在所有注意力头之间共享同一个投影键和同一个投影值，但这仍然是两个不同的投影：K≠V。

MQA 能把 KV 缓存大小减少到原来的 1/h，其中 h 是注意力头数量。不过，它也会显著降低模型的灵活性，对质量产生负面影响。如果希望在 MQA 和 MHA 之间取得折中，可以使用分组查询注意力。

分组查询注意力（Grouped-Query Attention，GQA）

GQA 由 Google 研究人员于 2023 年提出。思想很简单：把注意力头分成多个组，每个组共享同一个投影键和值。实际上，可以把 MQA 看作只有一个组的 GQA。MHA、MQA 和 GQA 如图 17-11 所示。

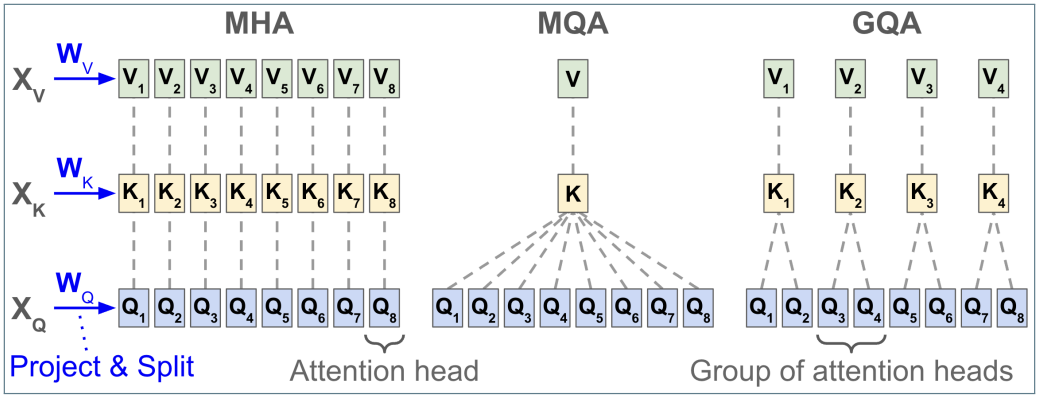

图 17-11: MHA、MQA 和 GQA（MHA, MQA, and GQA）

在 PyTorch 中，只需调用 `F.scaled_dot_product_attention()` 时设置 `enable_gqa=True`，就可以高效实现 MQA 和 GQA。查询张量包含每个注意力头的投影查询，形状为 `[batch size, h, Lq, d]`；键和值张量每个头只包含一个投影，形状为 `[batch size, g, Lk,`

d]，其中 g 是组数。下面的代码使用 8 个注意力头，并将它们分为 2 个组，每组共享投影键和值：

```
batch_size, Lq, Lk, d_head = 32, 100, 90, 64
n_heads = 8
n_groups = 2
query = torch.randn(batch_size, n_heads, Lq, d_head)
key = torch.randn(batch_size, n_groups, Lk, d_head)
value = torch.randn(batch_size, n_groups, Lk, d_head)
attn = F.scaled_dot_product_attention(query, key, value, enable_gqa=True)
```

GQA 是 MHA 和 MQA 之间很好的折中，并且允许灵活选择组数，在效率和质量之间进行平衡。不过，多头潜在注意力似乎始终优于 GQA。

多头潜在注意力（Multi-Head Latent Attention，MLA）

2023 年 12 月，中国 AI 公司 DeepSeek 开始发布令人印象深刻的模型，首先是通用 LLM DeepSeek-LLM。2024 年 1 月，他们发布了面向代码生成和补全的 DeepSeek Coder，随后又发布视觉语言模型 DeepSeek-VL。5 月，他们发布旗舰通用 LLM 的第二代 DeepSeek-V2，该模型采用专家混合和 MLA。此后他们还发布了 Janus 等多模态 Transformer。2025 年 1 月，他们推出了强大的推理模型 DeepSeek-R1，在多个基准上超过 OpenAI o1，训练成本却只占很小一部分；DeepSeek-V3.1 于 2025 年 8 月发布。这是一份令人印象深刻的成绩单！

好消息是，主要 DeepSeek 模型都是开源的，可以免费用于研究和商业用途；DeepSeek 研究人员还公开发表了许多论文和技术报告，分享了以远低于 Google 等巨头预算训练出这些模型的关键创新。

其中一项技术就是 DeepSeek-V2 论文提出的多头潜在注意力（MLA）：它可以像 MQA 一样大幅缩小 KV 缓存，同时不会降低模型质量，甚至还能提高质量。

在自注意力和交叉注意力中，输入的键和值是同一个输入序列，记为 X_{KV}。MLA 的核心思想是：使用可训练投影矩阵 W_{KV}，把这个 d 维序列投影到更小的 d_c 维空间，即潜在空间；再使用压缩后的序列 L_{KV} 为所有注意力头生成键和值投影（见图 17-12）。

在推理时，不再把键和值投影存入 KV 缓存，而是存储 L_{KV}，需要时再用它重新计算投影后的键和值。这样可以节省大量空间，因为无论注意力头数量是多少，都只需保存一个压缩序列。

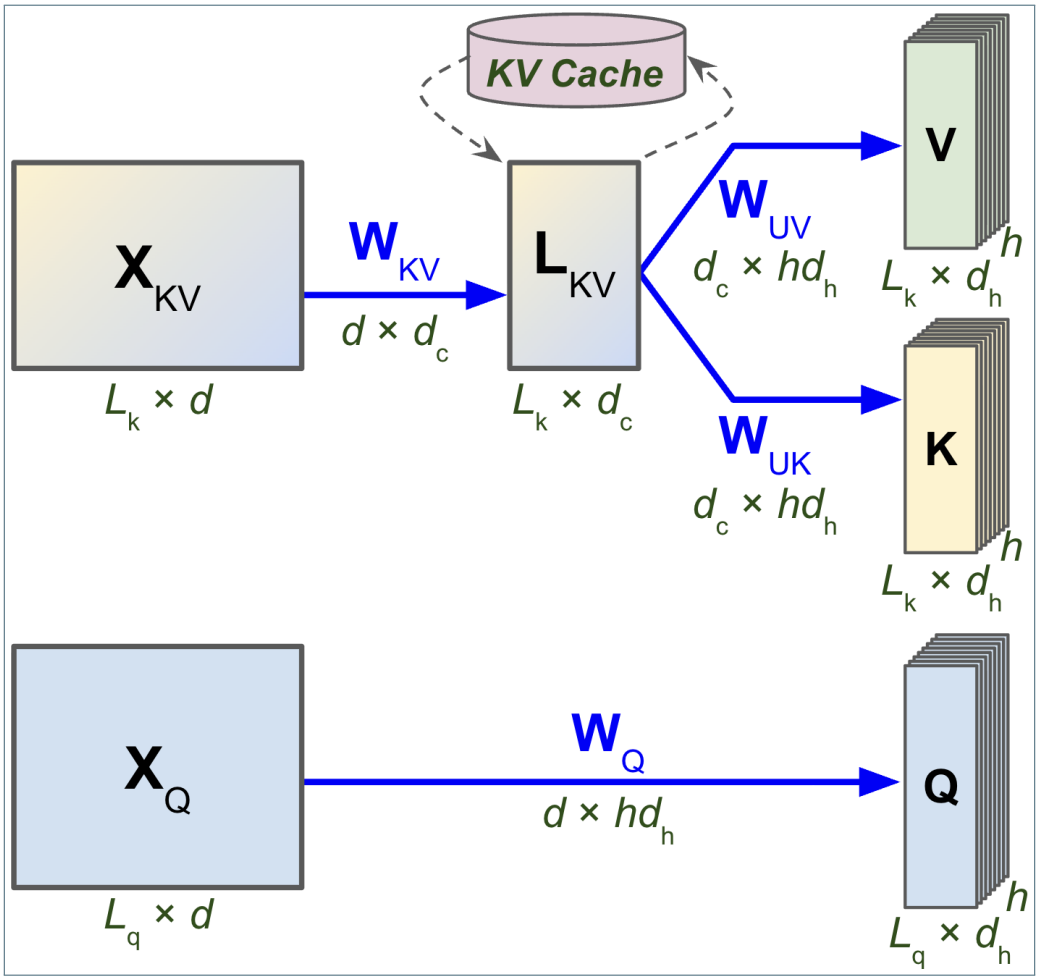

图 17-12: 多头潜在注意力（Multi-head latent attention，MLA）

这种技术可以大幅降低 KV 缓存大小，如表 17-1 所示。该表比较了 MHA、MQA、GQA 和 MLA 的 KV 缓存大小。

表 17-1: MHA、MQA、GQA 和 MLA 的 KV 缓存大小

架构	KV 缓存大小公式	示例
MHA	p × nL × nt × dh × h × 2	3,750 MB
MQA	p × nL × nt × dh × 2	29 MB
GQA	p × nL × nt × dh × g × 2	469 MB
MLA	p × nL × nt × dc	59 MB

其中：

- p 是浮点精度所占的字节数，例如 2。

- n_L 是模型中的 MHA 层数，例如 60。
- n_t 是缓存中存储的 token 总数，例如 1,000。
- d_h 是注意力头维度，例如 128。
- d_c 是潜在空间维度，例如 512。
- h 是注意力头数量，例如 128。
- g 是 GQA 中的组数，例如 16。

在这个基于 DeepSeek-V2 超参数的例子中，MLA 把 KV 缓存缩小了 64 倍！这还不如 MQA 缩小得多，但 MLA 模型的质量高得多，原因有两个：首先，MLA 为每个注意力头保留投影后的键和值，比 MQA 和 GQA 灵活得多；其次，投影键和值时使用的矩阵更小，因此模型更小、更快。得益于这种加速，MLA 在相同训练时间内甚至能达到比 MHA 更好的性能。

实现 MLA 最简单的方式，是复用开源实现，例如 DeepSeek-V3 使用的实现。也可以自己编写模块；好消息是，它可以在每个注意力头内部调用经过优化的 `F.scaled_dot_product_attention()` 函数。

到这里，我们已经介绍了使用稀疏注意力、近似注意力以及跨注意力头共享投影来加速 MHA 的主要技术。下面介绍本章最后一种注意力优化技术：FlashAttention。

17.2.4　FlashAttention：快速且精确的注意力实现（FlashAttention: A Fast and Accurate Attention Implementation）

FlashAttention 是对熟悉的缩放点积注意力的优化实现，其核心计算见公式 17-1。它充分利用现代 GPU 的硬件架构，在不做近似的情况下，速度可以达到以往优化实现的 2 到 4 倍。

它尤其适合较长的输入序列。

FlashAttention 由斯坦福大学和布法罗大学的研究人员于 2022 年 5 月提出。理解 FlashAttention，需要先了解 GPU 中不同类型的内存，它们从最快到最慢依次如下。

寄存器（Registers）

GPU 包含多个运行并行线程的处理核心集群。每个集群称为流式多处理器（streaming multiprocessor，SM）。例如，NVIDIA A100 GPU 包含 108 个 SM。每个 SM 可以直接访问一定数量的 32 位寄存器，例如 $2^{16} = 65,536$ 个寄存器，总容量只有 256 kB。每个线程只能使用属于自己的寄存器集合，例如 255 个，因此数据会分散在线程之间。另一方面，寄存器可以在一个时钟周期内访问，因而拥有最高带宽，例如所有 SM 合计超过 100 TB/s。寄存器用于保存中间结果、计数器，以及矩阵乘法中的部分和等累加器。

共享内存 / L1 缓存（Shared Memory / L1 Cache）

每个 SM 还有一小块由所有线程共享的内存，例如 192 kB。这部分内存位于芯片上，带宽很高，例如 20 TB/s，可以在计算过程中保存数据块。矩阵通常会被切成小块，一次把少量小块存入共享内存；共享内存还可以用于线程间通信。最后，这部分内存中的一部分可以分配为一级缓存（L1 cache），由 GPU 自动缓存频繁访问的数据。

二级缓存（Level 2 Cache）

这部分内存同样位于芯片上，在所有 SM 之间共享，由 GPU 自动管理，用于缓存频繁访问的数据，例如部分模型权重。它容量更大，例如 40 MB，但速度较慢，例如 7 TB/s。

全局内存（Global Memory）

前面几种内存都位于 SRAM（静态随机存取存储器）中，速度快但昂贵。全局内存（例如高带宽内存 HBM）位于芯片外部，使用更便宜的 DRAM（动态随机存取存储器），容量可以大得多，例如 80 GB，但带宽较低，例如 2 TB/s。通常所说的 GPU 内存就是指全局内存：模型存放在这里，新数据批次也会在处理前放到这里。

为了让算法尽可能快，必须尽量减少 GPU 把数据从全局内存传输到 SRAM 的时间，这正是 FlashAttention 所做的事：它是一种具有 I/O 感知的实现。回顾缩放点积注意力公式，它先计算矩阵乘法 $QK^\intercal$，再对输出计算 softmax，最后进行另一次矩阵乘法。如何加速这些操作？

矩阵乘法的加速依靠分块（tiling）：先把待相乘的两个矩阵切成能够放进 SRAM 的小矩阵，即 tile；再将这些 tile 相乘，逐步累加得到输出 tile。图 17-13 的例子把两个 8×8 矩阵切成 4×4 矩阵，然后用这些 tile 计算输出 tile。这样可以让处理核心一直使用能够快速从 SRAM 取出的数据，同时尽量减少从全局内存读取数据的次数。

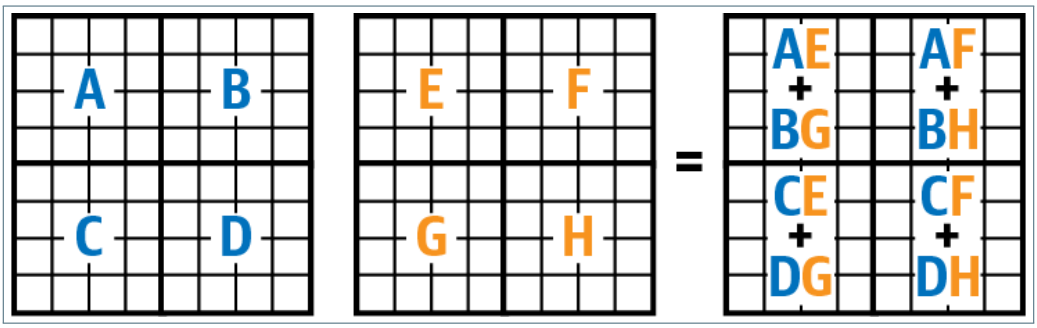

图 17-13: 使用分块进行快速矩阵乘法（Fast matrix multiplication using tiling）

softmax 运算也可以显著加速。朴素实现会先计算 $E = \exp(M)$，再计算每一行的和 $D = E.\text{sum}(\text{dim} = -1)$，最后计算逐元素除法 $O = E \oslash D$。但如果 M 很大，这需要从全局

内存读取 M 来计算 E，把结果写回全局内存；接着读取 E 计算 D、保存 D，再读取 E 和 D 计算 O，最后又把 O 写回全局内存，读写次数很多。

更高效的方法是融合 E 和 D 的计算：遍历 M 的每一行，同时维护该行 $\exp(M_{i,j})$ 的运行总和，这就得到 D；然后第二次遍历 M，计算 $\exp(M_{i,j})/D_i$。由于完全避免读写 E，结果会快得多，这称为在线 softmax（online softmax）。

FlashAttention 更进一步：它使用分块矩阵乘法融合 $QK^\intercal$ 的计算，也融合 SV 的计算；同时在求指数前减去每行的最大值，避免数值溢出，这就是前面介绍的安全 softmax。实现时需要运行估计最大值，并在发现更大最大值时调整运行总和。玩具版 Python 实现见配套 notebook，完整算法见论文第 5 页。

Python 非常适合高级编程，但不适合编写自定义 GPU kernel 这样的底层代码，因此需要切换到 C++ 等系统语言。Julia 和 Mojo 等语言试图解决“两种语言”的问题：它们同时支持简单、富有表现力且动态的高级编程，以及带类型和优化的低级编程；Mojo 甚至希望成为 Python 的超集。

FlashAttention 第一作者 Tri Dao 于 2023 年 7 月发布了第二版 FlashAttention-2，它通过优化 GPU 线程之间的工作划分和并行性，加速了原始算法。FlashAttention 通常只能达到 GPU 25% 到 40% 的利用率，而 FlashAttention-2 可以达到 50% 到 75%。FlashAttention-2 不会忽略 padding token，并且只支持 16 位浮点精度，即 **`torch.float16`** 或 **`torch.bfloat16`**。

一年后，Jay Shah 等人发布 FlashAttention-3，进一步利用 NVIDIA H100 等新 GPU 架构的异步执行和低精度能力优化算法。

如果硬件兼容，PyTorch 的 **`F.scaled_dot_product_attention()`** 会自动使用 FlashAttention-2：它支持兼容 CUDA 版本的 NVIDIA CUDA GPU，也支持通过 ROCm 工具包使用 AMD GPU。若需要更多控制，可以直接使用 **`flash-attn`** 库，安装命令为 **`pip install flash-attn --no-build-isolation`**。

Hugging Face Transformers 中的大多数模型都开箱支持 FlashAttention-2。加载模型时，只需把 **`attn_implementation="flash_attention_2"`** 传给 **`from_pretrained()`**。

FlashAttention（1、2 和 3）彻底改变了注意力加速领域。许多近期架构放弃稀疏和近似注意力，转而使用 FlashAttention，因为它在不损失质量的情况下带来巨大的效率提升。FlashAttention 还可以与 MLA 结合：MLA 在每个注意力头中使用普通的缩放点积注意力，因此同样能受益于 FlashAttention；二者是很好的组合。

回顾一下，我们先介绍了 KV 缓存、推测解码等解码加速技术；然后讨论了稀疏注意力、近似注意力、跨注意力头共享投影和 FlashAttention 等 MHA 加速方法。现在暂时离开多头注意力，转向 Transformer 的另一个主要组件——前馈神经网络，并使用专家混合技术对其进行优化。

17.3　使用专家混合扩展（Scaling Up with Mixture of Experts）

遇到医学问题，我们会问医生；遇到法律问题，我们会问律师。这种把不同请求路由给不同专家的思想，最早由 Robert A. Jacobs、Michael I. Jordan 等人在 1991 年的一篇机器学习论文中提出，他们把这项技术称为专家混合（mixture of experts，MoE）。二十多年里，MoE 一直是小众技术；2017 年 Noam Shazeer 等人把它成功应用于 Transformer，提出稀疏门控 MoE 层（下文简称 MoE 层）后，它才逐渐流行起来。

实际上，正如下面会看到的，“专家”并不像人类专家那样按高层领域专业化，而是以相当低的层次关注输入空间的不同区域，而且专家之间存在很大重叠。此外，一个 MoE Transformer 包含多个 MoE 层，每层都有自己的局部专家集合；不同 token 在每个 MoE 层中可能被路由给不同专家。因此，并不是医学问题由一个医学 Transformer 处理，而是 token 在每个 MoE 层中被拆分到多个专家。于是，“专家混合”这个名称有点误导；或许“拆分子任务的半专业化子网络”更准确，只是这个名称不够朗朗上口。

无论如何，MoE Transformer 已经取得巨大成功，下面看看 MoE 层如何工作。

17.3.1　专家混合层（The Mixture of Experts Layer）

核心思想是：在某些 Transformer block 中（通常每隔 2 到 4 个 block），用 MoE 层替换 FFN。MoE 层由多个 FFN（称为专家）以及一个小型路由器网络组成，路由器决定每个输入 token 应由哪些专家处理（见图 17-14）。

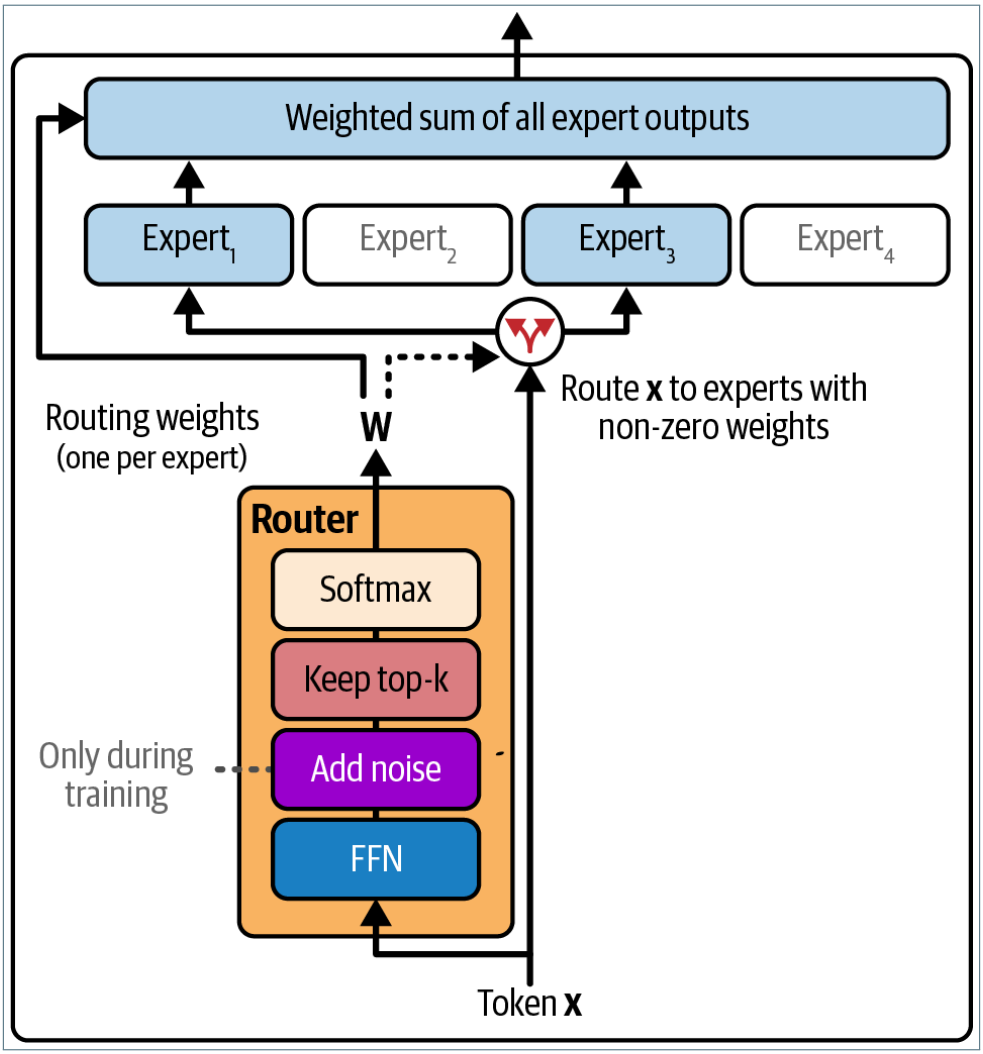

图 17-14: 现代 MoE 层：该层替换每个 Transformer block 中的 FFN 模块（A modern MoE layer replacing the FFN module in each transformer block）

- 每个 token 首先经过路由器网络。路由器只是一个小型 FFN，通常只有一个线性层和 softmax 激活函数。它输出一个向量 w，每个专家对应一个路由权重，所有权重之和为 1。
- 接着找出其中最大的 k 个权重（本例中 k=2），把输入 token x 路由给对应的 k 个专家，其他专家可以休息。
- 最后对这 k 个专家的输出做加权求和：每个输出乘以对应的路由权重，再把所有加权输出相加。在某些 k>1 的架构中，会在计算最终加权和之前先对 top-k 权重归一化。

后续架构逐步改进了 MoE，最重要的包括 GShard、Switch Transformer、Vision-MoE、GLaM 和 ST-MoE，它们由 Google 研究人员在 2020 至 2022 年间发表（当然，此后的研究并不只来自 Google）。

每个专家本身也可以是一个 MoE 层，拥有自己的路由器网络和子专家。这样得到的模型称为层次化 MoE（hierarchical MoE），可以扩展到数千个专家。

17.3.2　MoE 的优势（Benefits of MoE）

把每个 token 只路由给几十个甚至数百个专家中的少数几个，就可以在保持较低计算量的同时大幅提高模型能力。MoE Transformer 通常非常大，往往需要多张 GPU 才能运行；但它的速度远超其规模给人的直觉，因为每个 token 只使用全部模型权重中的一小部分。

这种优势在逐个 token 生成文本的推理阶段尤其明显：MoE 只使用该 token 对应的 top-k 专家，速度非常快，而且只需读取少数模型权重，因此内存带宽开销较小。

不过，在训练阶段和推理时的预填充阶段，MoE 必须处理可能很长的 token 序列，而每个 token 可能被路由给不同专家。因此，很多专家可能需要同时工作，即使每个专家只处理少量 token；这会损失大部分内存带宽优势，但仍能获得很大的计算优势。

可惜没有免费的午餐：构建、训练、运行甚至维护大型 MoE Transformer 都可能很困难且成本高昂。下面看看原因。

17.3.3　MoE 的挑战（The Challenges of MoEs）

MoE 主要有三个困难：在专家之间平衡工作量、减少专家之间的重叠，以及工程挑战。先看负载平衡。

负载平衡（Load Balancing）

第一个主要困难是确保路由器在专家之间均匀分配工作量，至少在平均意义上如此，尤其是在训练期间。如果某个专家在训练的某个阶段表现不如其他专家，路由器就不会选择它；结果它得不到训练，也不会变得更好。这是典型的"强者愈强"反馈循环，在这里会导致专家坍缩（expert collapse）：路由器完全忽略大多数专家，只偏爱少数幸运专家。这会破坏模型准确率，并使训练不稳定。

常见解决方法是加入辅助负载平衡损失，鼓励路由器在专家之间均匀分配工作量；每个 MoE 层的负载平衡损失都会加到总损失中。最初的稀疏门控 MoE 会在计算 softmax 前加入带有每个专家可训练尺度的高斯噪声，损失函数推动模型让每个专家承担相同的期望负载。不过，Switch Transformer 和大多数现代 MoE 架构使用更简单的负载平衡损失，完全不需要加噪声：

$$L_{\text{LB}} = \alpha N \sum_{i=1}^{N} f_i Q_i \tag{17-4}$$

公式 17-4　许多近期 MoE Transformer 使用的辅助负载平衡损失（Auxiliary load-balancing loss）

其中：

- α 是可调节的超参数，决定推动公平负载平衡的力度，例如 0.01。
- N 是专家总数，而不是 top-k 专家数。
- f_i 是当前批次中被分配给第 i 个专家的 token 比例。
- Q_i 是该批次中分配给第 i 个专家的路由权重均值。

这个损失会推动每个 f_i 和 Q_i 接近 $1/N$，从而让每个专家承担相等的负载。

GShard 作者还提出了几种常用技术，可以进一步平衡专家之间的负载。

随机路由（Random Routing）

当 k=2 时，总是选择权重最高的专家，但第二个专家按照其路由权重成比例地随机选择。

专家容量（Expert Capacity）

每个专家在一个批次中最多处理 C 个 token；C 是称为专家容量的超参数，通常略大于“批次 token 数除以专家总数”。当某个专家达到最大容量时，分配给它的 token 通常会被重新路由到其他专家。如果所有候选专家都已满，该 token 通常会跳过 MoE 层：MoE 层输出零，但围绕 MoE 层的跳跃连接允许 token 继续进入下一个 Transformer block。在某些视觉 MoE 架构中，token 甚至可能被完全丢弃，因此输出序列会变短。

接下来讨论 MoE 的第二个主要困难：减少专家之间的重叠。

减少专家之间的重叠（Reducing Overlap Between Experts）

你可能会认为，路由器会把数学问题发给数学专家，把医学问题发给医学专家；但正如前面提到的，路由器似乎更关注低层特征：它可能把标点符号和 emoji 发给一个专家，把形容词和数字发给另一个专家，等等。这总比没有好，但也会导致专家之间大量重叠，因为它们最终都必须学习每个主题。如果所有专家都是通才，就不能真正称为专家了！这种冗余实际上会削弱模型能力，导致准确率下降。

一种提高专家专业化程度的方法是使用更强大的路由器网络，但这会带来额外成本。DeepSeek-V2 提出的另一种方法是增加共享专家，让它们处理每个 token；共享专家倾向于学习最常见的情况，从而鼓励路由专家专门处理更具体的任务。例如，DeepSeek-V3 在每个

MoE 层中使用 1 个共享专家、256 个路由专家，其中每次激活 8 个路由专家。如何进一步改善专家专业化，仍然是活跃的研究方向。

MoE 的工程挑战（Engineering Challenges with MoEs）

无论训练还是推理，MoE 都可能带来很大的工程挑战，尤其是大型模型。下面只是一些可能遇到的工程问题：

- MoE Transformer 可能大到必须把专家切分到多张 GPU，甚至多台服务器上。这意味着每个 MoE 层的输入和输出通常都要通过全互联（all-to-all）通信在设备之间，甚至服务器之间传输。I/O 很容易成为主要瓶颈，而且需要仔细同步，例如必须等待所有专家输出后才能计算加权和。
- 如果模型切分到多台服务器上，基础设施必须检测并处理节点故障，因此监控非常关键。
- 必须动态地把 token 路由给正确的专家，并通过批处理提高 GPU 利用率；但批处理不像稠密模型那样简单。
- 少数过热专家（负载过高）可能拖慢整个模型。可以实现细粒度监控，动态解决问题，例如重新路由 token、复制专家或自动扩展基础设施。

总之，要让大型 MoE 在生产环境中大规模运行，需要复杂且昂贵的工程工作。这也是许多早期 MoE 模型（例如 GLaM 和 Switch）主要停留在研究阶段的原因。

好消息是，现在已经有 DeepSeek-MoE 等多个开源库，可以高效训练和运行 MoE，不需要重复发明所有轮子。还有许多可下载的预训练 MoE 模型（见表 17-2）。

表 17-2: 流行的开源 MoE 模型

模型名称	发布日期	激活/总参数	激活/总专家数（含共享专家）
Mixtral 8×7B	2023 年 12 月	13B / 47B	2 / 8
DBRX Instruct	2024 年 3 月	36B / 132B	4 / 16
Mixtral 8×22B	2024 年 4 月	39B / 141B	2 / 8
DeepSeek-V2	2024 年 5 月	21B / 236B	6 / 160（+2）
OpenMoE-8B	2023 年 11 月	2B / 8B	? / 32
Grok-1	2024 年 3 月	86B / 314B	2 / 8
Jamba-1.5-L	2024 年 8 月	94B / 398B	2 / 16
OLMoE-7B	2024 年 9 月	1B / 7B	8 / 64
Aria（视觉与语言）	2024 年 10 月	3.5B-3.9B / 25B	6 / 64（+2）
DeepSeek-V3	2024 年 12 月	37B / 671B	8 / 256（+1）

如果想进一步了解 MoE 文献，Siyuan Mu 和 Sen Lin 于 2025 年发表的 MoE 综述是很好的入口。

17.4　更快地训练（Faster Training）

Transformer 通常很大，训练速度也很慢，主要有两个原因：首先，大模型通常需要更多计算；其次，大量数据需要在 GPU VRAM 中移动，内存带宽会成为瓶颈。以一个 70 亿参数的模型为例：

17.4.1　模型权重（Model Weights）

使用 16 位浮点精度时，权重占用 14 GB 内存。推理时，这大致就是所需的大部分内存；但训练时还不止这些。使用混合精度训练（见附录 B）时，还需要一份 32 位的可训练权重主副本，额外占用 28 GB。

17.4.2　激活值（Activations）

训练时，必须保存前向传播中的所有激活值，因为反向传播需要它们。具体大小取决于模型架构、批次大小、序列长度和浮点精度；对于 70 亿参数模型，可能约为 40 GB。

17.4.3　梯度（Gradients）

每个可训练参数都需要一个梯度。如果训练全部 70 亿个参数，在 16 位浮点精度下，梯度占用 14 GB。

17.4.4　优化器状态（Optimizer States）

优化器通常需要为每个可训练参数保存额外数据。例如，Adam 为每个参数保存两个运行平均值；在 16 位精度下需要额外 28 GB，但为了精度和训练稳定性，这些状态通常以 32 位精度存储，因此需要 56 GB。

总计，即使使用混合精度训练，一个 70 亿参数模型也需要 152 GB 内存。你的硬件可能没有这么多 VRAM；即使有，内存带宽饱和也会成为噩梦。

一种解决方案是缩小模型，这会同时降低计算成本和内存使用量，附录 B 会讨论这一点。也可以使用前面介绍的技术加速 MHA，或使用 MoE。本节再介绍几种加快 Transformer 训练的方法：

- 参数高效微调（PEFT）减少需要训练的参数数量，从而减少梯度、优化器状态以及混合精度训练中 32 位可训练参数主副本的大小。

- 激活检查点减少反向传播需要保存的激活值；第 12 章已经介绍过，本章重点讨论如何把它应用到 Transformer。
- 序列打包和分桶，消除浪费性的 padding token。
- 梯度累积，在一次优化器更新前处理多个批次，以降低内存负载。
- 并行化，把计算和内存负载拆分到多张 GPU 上。

先从 PEFT 开始。

17.4.5　参数高效微调（Parameter-Efficient Fine-Tuning，PEFT）

有时需要从头训练 Transformer，但更多时候会下载预训练模型，再针对自己的任务进行微调，第 15 章介绍过这一过程。加快微调的一种方法是减少可训练参数数量，从而减少计算和存储可训练权重、梯度及优化器状态的成本，这称为参数高效微调（PEFT）。

PEFT 有多种技术，最流行的方法是使用适配器：冻结大型预训练模型的权重，在模型内部插入若干小型可训练适配器，然后微调得到的适配器模型。由于适配器只有少量可训练参数，效率提升很明显。

适配器有多种插入方式。例如，前缀适配器（prefix adapter）使用可学习的键/值前缀，这与第 15 章简要介绍的提示调优密切相关。不过，最流行的适配器技术是低秩适配（Low-Rank Adaptation，LoRA），由 Microsoft 研究人员于 2021 年提出。

首先冻结预训练模型，并选择希望使用 LoRA 适配器微调的权重矩阵。一般来说，最好至少调整每个 MHA 层中的 query 和 value 投影矩阵。如果需要更高精度，也可以增加适配器，调整每个 MHA 层的 key 和 output 投影矩阵，甚至调整 FFN block 中的权重矩阵。这是效率与准确率之间的权衡。

其次，把涉及某个选定矩阵 W_o 的矩阵乘法 XW_o 替换为 $XW_o + XBA$。其中 X 是输入矩阵，W_o 是冻结的预训练权重矩阵，A、B 是新的小型可训练适配器矩阵。如果 W_o 的形状为 d×k，那么 B 是 d×r 矩阵，A 是 r×k 矩阵，r 是超参数，例如 r=8。实践中，通常会把包含两个线性层的适配器模块附加到要调整的层上，如图 17-15 所示。

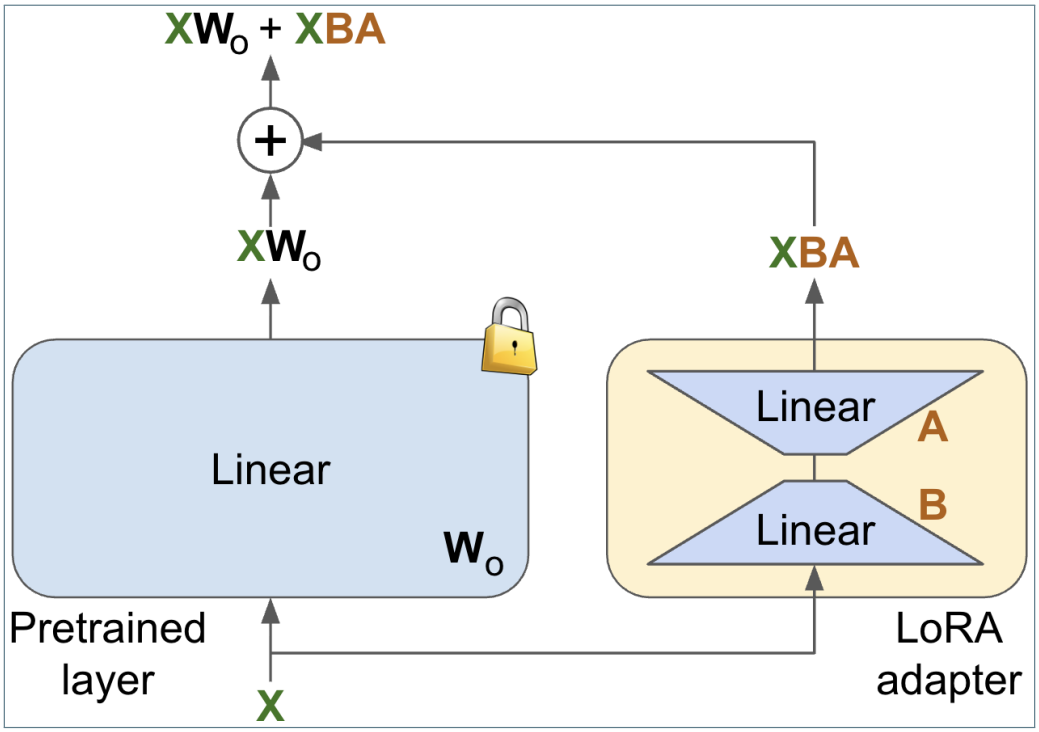

图 17-15: 低秩适配（Low-Rank Adaptation，LoRA）

如果预训练层有 bias 参数，可以选择是否训练它，这是效率与质量之间的常见权衡；LoRA 适配器本身没有 bias 参数，因为那是多余的。

通常可以从 r=8 开始，也可以把它降低到 1 并仍然得到合理结果；如果愿意用速度和内存换取更高质量，也可以增加到 16、32、64 或更高。

这样就可以微调适配后的模型了。为什么 LoRA 既快又能保持质量？速度提升很容易解释：不再微调 d×k 个参数，而是微调 r×(d+k) 个参数。由于 r 相对于 d、k 很小，需要训练的参数少得多，计算量更小，内存带宽饱和的风险也更低。

虽然 BA 是一个很大的 d×k 矩阵，但实际上不需要计算它。计算 XBA 时，先计算 P=XB，得到一个 b×r 矩阵（b 是批次大小），再计算 PA，得到 b×k 矩阵。

LoRA 为什么能产生高质量模型？作者推测，直接微调原始预训练模型（不使用 LoRA）会把 W_o 变成 $W_{tuned} = W_o + W_\Delta$，其中 W_Δ 是原矩阵与微调后矩阵之间的差异。他们假设 W_Δ 具有较低的“内在”秩，因此可以用两个低秩矩阵的乘积很好地近似；换句话说，$W_\Delta \approx BA$。实践证明确实如此：冻结 W_o，只学习 A、B，最终模型的性能几乎与完整微调一样，但训练参数少得多，所需内存也少得多。

使用 Hugging Face 的 Transformers 和 peft 库可以用 LoRA 微调模型。这两个库都预装在 Colab 中。下面使用 Transformers 加载模型，再通过 `peft.get_peft_model()` 创建 LoRA 模型：

```
import peft
from transformers import AutoModelForCausalLM
model_id = "EleutherAI/gpt-neo-125M"  # a small model for this example
model = AutoModelForCausalLM.from_pretrained(model_id, device_map="auto",
                                             dtype=torch.float16)
lora_config = peft.LoraConfig(r=8, target_modules=["q_proj", "v_proj"])
peft_model = peft.get_peft_model(model, lora_config)
peft_model.print_trainable_parameters()
```

这段代码先以 16 位精度加载预训练模型，再创建 LoraConfig，指定秩 r 和需要调整的层名，最后调用 **peft.get_peft_model()**，得到使用 LoRA 适配的参数高效模型。之后就可以像第 15 章一样正常微调。

微调完成后，可以选择移除适配器，把每个原始权重矩阵 Wo 替换为 $W_{LoRA} = W_o + BA$。这样 LoRA 在推理时不会带来任何额外计算或内存成本。只需调用 PEFT 模型的 **merge_and_unload()** 方法：

```
merged_model = peft_model.merge_and_unload()
```

不过，如果原始模型已经量化（见附录 B），保留适配器可能更好。合并适配器需要先反量化权重、加上 BA，再重新量化，这会损失一些精度；由于适配器非常轻量，合并带来的效率收益也很小。

总之，PEFT 减少可训练参数数量，从而降低微调时的计算量和内存需求，并带来显著的速度提升。下面介绍激活检查点，它可以大幅降低训练时激活值的内存使用量，但会略微增加计算成本。

17.4.6　激活检查点（Activation Checkpointing）

第 12 章已经介绍过激活检查点以及在 PyTorch 中的实现方式，但没有介绍 Transformers 库中的实现。本节简要说明：激活检查点用于降低训练时保存激活值的内存使用量。前向传播时不再保存所有激活值，只保存其中一些检查点（例如主要层的输出）；反向传播需要时，再使用检查点重新计算缺失的激活值。结果是内存使用量更低，但计算量更高。

使用 Transformers 实现激活检查点非常简单，只需在传给 Trainer 的 TrainingArguments 对象中设置 **gradient_checkpointing=True**，通常还会和混合精度训练配合使用：

```
from transformers import TrainingArguments
args = TrainingArguments(output_dir="./my_great_model", num_train_epochs=3,
                         per_device_train_batch_size=8,
                         gradient_checkpointing=True, fp16=True)
```

也可以直接在模型上调用 **gradient_checkpointing_enable()** 开启激活检查点，或者在单个模块上开启，以获得更细粒度的控制。

17.4.7　序列打包与分桶（Sequence Packing and Bucketing）

训练可变长度序列时，通常会在创建批次时把它们填充到相同长度。但这会在大量 padding token 上浪费计算和内存，也会产生大小不固定、对硬件不友好的批次。

另一种方法是序列打包：拼接多个输入序列，形成长度为 L 的更长序列。如果打包后的序列不恰好有 L 个 token，可以裁掉多余 token，或者用 padding token 补足 L。也可以把批次中的所有序列打包到一个序列中，不需要裁剪或填充，这称为展平（flattening）。

序列打包可以去除全部 padding token（裁剪或展平时），或去除大部分 padding token（填充时），同时提高 GPU 利用率，从而加速训练。不过实现时需要非常小心：

- 必须调整注意力掩码，确保一个句子中的 token 不能关注另一个句子中的 token。
- 也必须调整位置编码，确保位置相对于每个句子的开头计算。
- 必须对正确的输出计算损失。这主要是仅编码器模型的难点，因为它们常常只有固定数量的输出，例如 `[CLS]` token；因此必须记录这些输出的索引，才能正确计算损失。
- 最后，MHA 实现必须足够智能，能够处理稀疏注意力掩码。朴素实现会遇到二次注意力问题，因为打包后的序列可能非常长，这会完全抵消序列打包的优势；但智能实现可以跳过注意力掩码中全为零的部分。

遗憾的是，PyTorch 没有提供简单的序列打包实现，需要手工处理所有细节。不过，Transformers 库提供了方便的解决方案：把 Trainer 的 **`data_collator`** 参数设置为 **`DataCollatorWithFlattening`** 类的实例，就会自动使用展平策略完成序列打包。它还使用 position IDs 和 FlashAttention-2，高效处理稀疏注意力掩码。

一种更简单但效率较低的方法，常用于仅编码器模型，是把长度相近的序列放到同一批次中，这称为序列分桶（sequence bucketing）。它大幅减少所需 padding token 的数量，也更容易在 PyTorch 中实现。一种方法是根据序列长度把数据集分桶，例如 `[1–10]`、`[11–20]`、`[21–30]` 等；创建批次时随机选一个桶，再从中采样序列。另一种方法是按长度排序数据集；创建批次时随机选择排序后的索引，再采样附近的序列。

图 17-16 比较了普通批处理、序列打包和序列分桶。

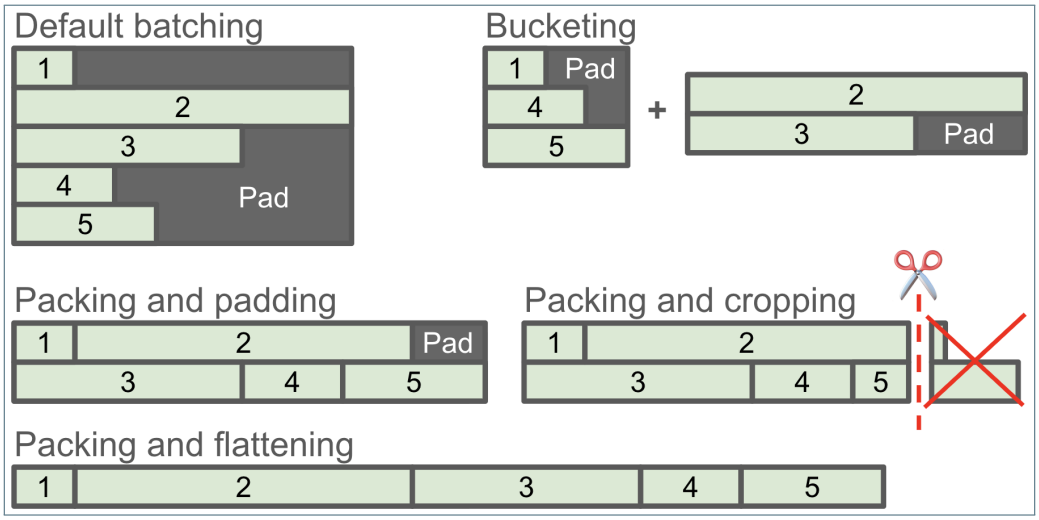

图 17-16: 序列打包和序列分桶与普通批处理的比较（Sequence packing and sequence bucketing versus regular batching）

下面介绍梯度累积，它可以在不增加内存的情况下增大批次大小。

17.4.8　梯度累积（Gradient Accumulation）

不必为每个输入批次都执行一次优化器更新，可以在多个批次之间累积梯度，每 n 个批次才使用累积的梯度执行一次优化器更新。过程如下：

- 把梯度设为零。
- 计算批次 1 的梯度，但不执行优化器更新。
- 对批次 2 到 n 重复计算，并累积梯度。
- 处理完 n 个批次后，使用累积梯度执行一次优化器更新（必要时除以 n，以保持梯度尺度）。
- 对下一组 n 个批次重复上述过程。

这种方法在数学上等价于把批次大小扩大 n 倍，但内存使用量小得多，因此可以使用较大的有效批次大小，而不会遇到内存不足或内存带宽饱和导致的减速。不过，如果内存足够，梯度累积的计算效率仍不如直接增大批次，因为 GPU 无法一次并行处理那么多数据。

PyTorch 中实现梯度累积很简单：

```
[...]  # create the model, optimizer, criterion, and data_loader
accumulation_steps = 4
optimizer.zero_grad()  # reset gradients before starting
for batch_index, (X_batch, y_batch) in enumerate(data_loader):
    X_batch, y_batch = X_batch.to(device), y_batch.to(device)
    y_pred = model(X_batch)
    loss = criterion(y_pred, y_batch)
    loss = loss / accumulation_steps
    loss.backward()
    if (batch_index + 1) % accumulation_steps == 0:
        optimizer.step()
        optimizer.zero_grad()
```

代码基本不难理解，但要注意在训练循环开始前必须把梯度设为零。把损失除以累积步数，也会把梯度除以同一个数，从而保证最终总梯度与使用更大批次时相同（另一种做法是把学习率除以累积步数）。

如果模型使用 batch normalization，或者使用批次级损失（例如 MoE 负载平衡损失），梯度累积就不再等价于增大批次大小。这也是 Transformer 使用 layer normalization 而不是 batch normalization 的部分原因。对于批次级损失，没有简单的修复方法，但训练仍可能正常工作，需要实际测试。

使用 Transformers 库实现梯度累积更简单：只需在 TrainingArguments 中把 **`gradient_accumulation_steps`** 设置为目标值即可。

最后讨论使用多张 GPU（甚至多台服务器）加速训练的方法。

17.5　使用并行化加速训练（Speeding Up Training Using Parallelism）

如果能够在训练中高效并行使用多张 GPU，理论上可以获得与 GPU 数量成比例的线性加速。训练可以沿多个维度并行：沿数据维度的数据并行（data parallelism）、沿特征维度的张量并行（tensor parallelism）、跨模型层的流水线并行（pipeline parallelism）、Transformer 沿序列维度的上下文并行（context parallelism）、MoE 模型跨专家的专家并行（expert parallelism）等。这些技术也可以组合使用。先从相对简单的数据并行开始。

17.5.1　数据并行（Data Parallelism）

数据并行的核心思想是把每个输入批次拆成多个块，在不同设备上并行处理。最常见的策略是在所有设备上复制模型，并让副本保持同步，始终拥有相同的模型参数，这称为镜像策略（mirrored strategy）。

PyTorch 为单机环境提供了基于镜像策略的基本实现：把模型包装在 **torch.nn.DataParallel** 对象中，可选地指定要使用的设备 ID（默认使用所有可用设备），然后像平常一样训练结果模型：

```
dp_model = torch.nn.DataParallel(model, device_ids=[0, 1, 2])
[...] # train dp_model normally
```

DataParallel（DP）包装器会处理所有复杂性。前向传播时，它把输入批次拆到多张 GPU 上，把主 GPU（包装模型所在的 GPU）的最新参数广播给其他 GPU，在所有副本上并行执行前向传播，最后在主 GPU 上收集并拼接输出。训练时，代码用该输出计算损失（通常也在同一设备上）；反向传播时，梯度从所有 GPU 并行传回，在主 GPU 上收集并求和。最后只在主 GPU 上执行梯度下降，并丢弃模型副本，下一次迭代再创建。

这种方法确实有效，而且很容易使用，但效率并不高，因为每次迭代都要把完整模型复制到所有 GPU；主 GPU 的工作量和内存使用量也比其他 GPU 多。幸运的是，PyTorch 提供了更好的实现 Distributed Data Parallel（DDP），它效率高得多，还支持位于不同机器上的 GPU。DDP 使用多进程而不是多线程，因此可以绕过 Python 的全局解释器锁（GIL）。除非只是快速测试，否则强烈建议使用 DDP，而不是 **torch.nn.DataParallel**。

使用 DDP 时（见图 17-17），模型只在启动时复制一次。此外，在每个训练步骤中，所有副本都独立且并行地完成完整前向和反向传播，每个副本处理自己的本地数据块。之后，副本只需同步计算梯度均值；然后每个副本都使用相同的平均梯度执行完全相同的优化器更新，因此无需每次迭代复制完整模型，就能得到相同的更新后参数。

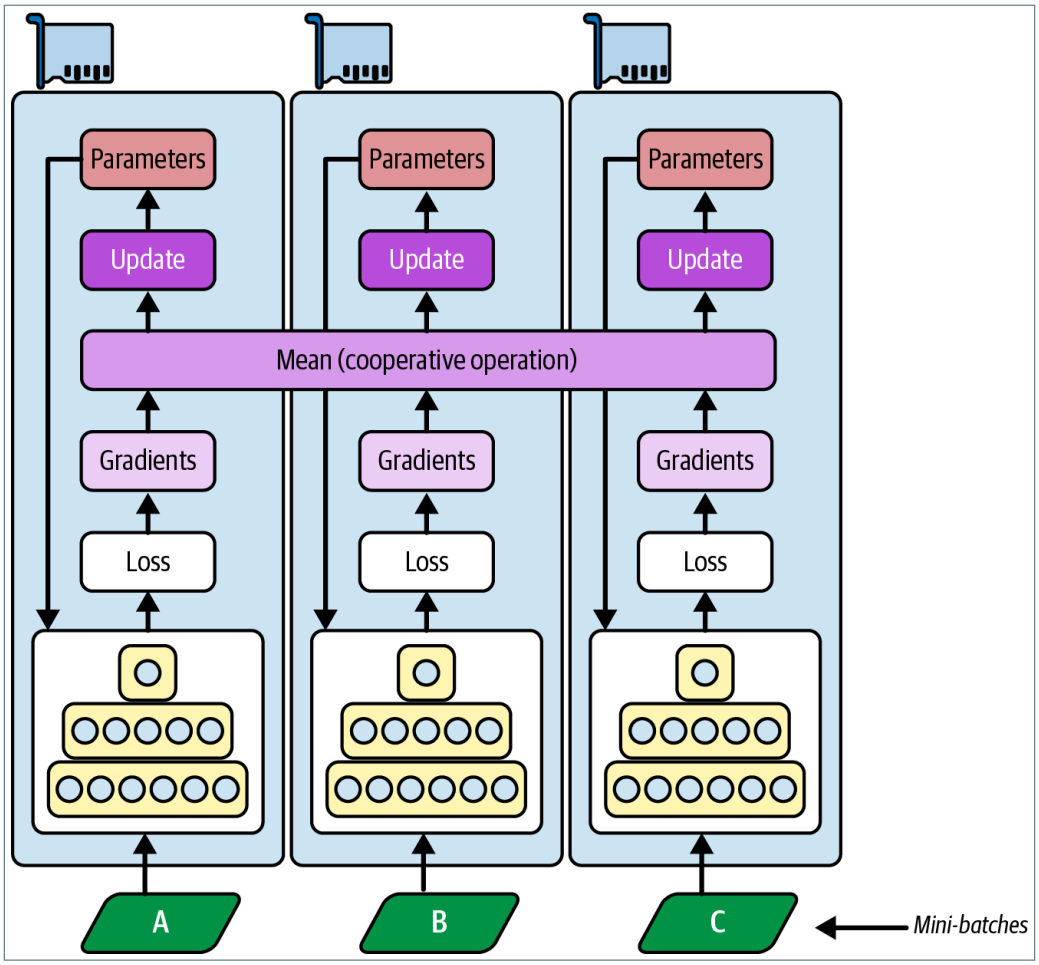

图 17-17: DDP 使用镜像策略进行数据并行：副本通过 all-reduce 操作协作计算平均梯度（DDP implements data parallelism with the mirrored strategy）

DDP 如何高效计算所有设备的梯度均值，并把结果分发给所有设备？它使用 all-reduce 算法。这类算法让多个节点协作执行 reduce 操作，例如计算均值、求和或最大值，同时确保所有节点得到相同的最终结果。NCCL（NVIDIA Collective Communications Library）和 PyTorch Gloo 都提供了现成实现，DDP 会根据硬件在底层使用它们。

镜像策略要求所有 GPU 在每个训练步骤后保持完全同步，即使很小的差异也可能逐步累积并破坏同步。为避免这种情况，all-reduce 必须在所有 GPU 上产生完全相同的结果（NCCL 和 Gloo 可以保证这一点），优化器也必须以完全相同的方式更新每个副本的参数。实践中，这要求所有设备使用一致的硬件和软件环境，并确保优化器步骤只使用确定性操作。

17.5.2　模型并行（Model Parallelism）

数据并行很适合加速训练，但如果完整模型无法放进一张 GPU 呢？许多大型 Transformer 甚至大到最大的 GPU 也无法容纳。这时可以把神经网络本身拆到多张 GPU 上，而不是完整复制模型，这称为模型并行。

拆分神经网络有多种方式，各有不同的权衡。图 17-18 展示了如何把全连接网络横向切分（流水线并行，PP）或纵向切分（张量并行，TP）。在 PP 中，模型被拆成连续的层组，称为 stage，例如每张 GPU 放一个 Transformer block；在 TP 中，每层沿特征维度切分。例如，一个输入和输出都是 512 维的 Linear 层，可以拆成两个输入 512、输出 256 的 Linear 层，分别放在两张 GPU 上。

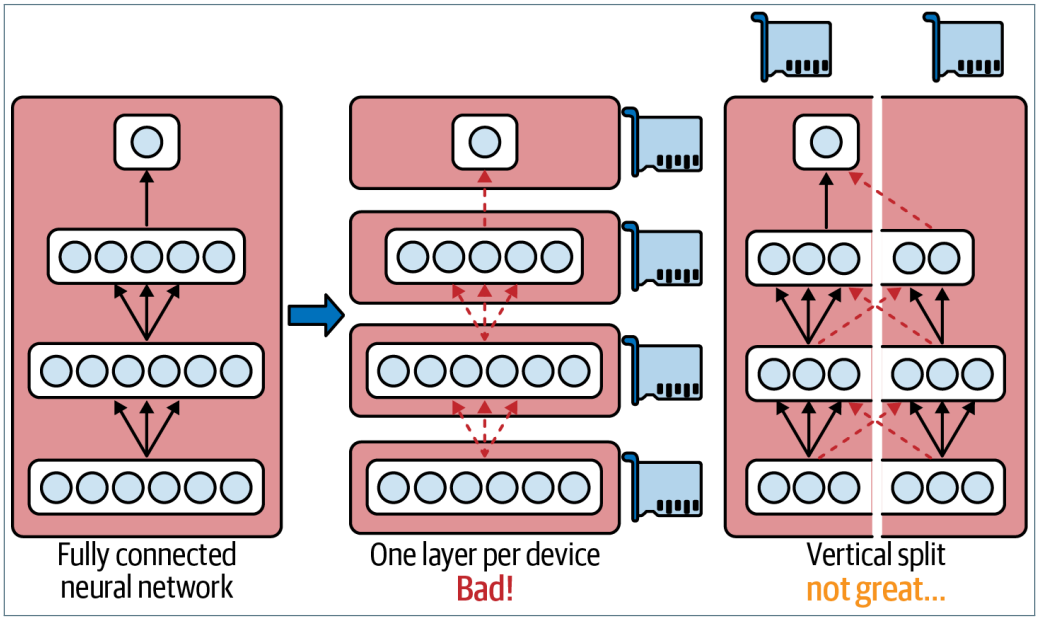

图 17-18: 流水线并行与张量并行（Pipeline parallelism versus tensor parallelism）

TP 的优势是允许训练非常宽的模型，因为每张 GPU 负责不同的垂直切片。不过在前向传播中，每个部分层都必须在下一层开始前把输出发送给其他设备；反向传播也一样。现代互联技术（NVLink、NVSwitch、InfiniBand）具有很高带宽和很低延迟，可以使用优化的 all-gather 操作，高效拼接分布在多个节点上的数据块，并确保每个节点最终都有完整副本。但 I/O 瓶颈仍然可能是主要挑战。

PP 的优势是可以训练非常深的模型，因为每个 stage 位于独立 GPU 上。通常 PP 的 I/O 少于 TP：前向传播每个 stage 只需要两次通信（接收一次、发送一次），反向传播同样两次。不过，朴素实现一次只会使用一张 GPU，因为当前批次必须顺序经过每个 stage。

幸运的是，通过聪明的工程设计，可以让所有 GPU 在不同批次上并行工作。这是数据并行与流水线并行的组合。2018 年，卡内基梅隆大学、斯坦福大学和 Microsoft Research 的研究人员在 PipeDream 系统中首次提出了这种思想：模型被拆成连续 stage，每个设备交替

从输入队列取批次执行前向传播，再从梯度队列取批次执行反向传播（见图 17-19）。这样就形成了异步流水线，所有机器都可以并行工作，空闲时间很少。

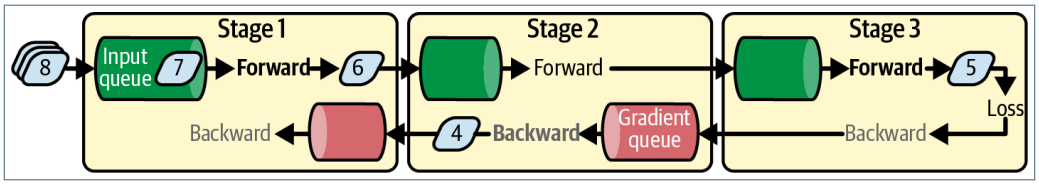

图 17-19: PipeDream 的流水线并行（PipeDream's pipeline parallelism）

不过，图中展示的 PipeDream 还不够理想。以图 17-19 中的 mini-batch #5 为例：它在 stage 1 执行前向传播时，mini-batch #4 的梯度还没有反向传播经过该 stage；但当 #5 的梯度返回 stage 1 时，#4 的梯度已经用于更新模型参数，因此 #5 的梯度会有些过时。这些过时梯度称为陈旧梯度（stale gradients），会减慢收敛、引入噪声和摆动，甚至使训练算法发散。也就是说，如果在计算梯度和应用梯度之间发生了中间权重更新，梯度就可能不再指向正确方向（见图 17-20）。

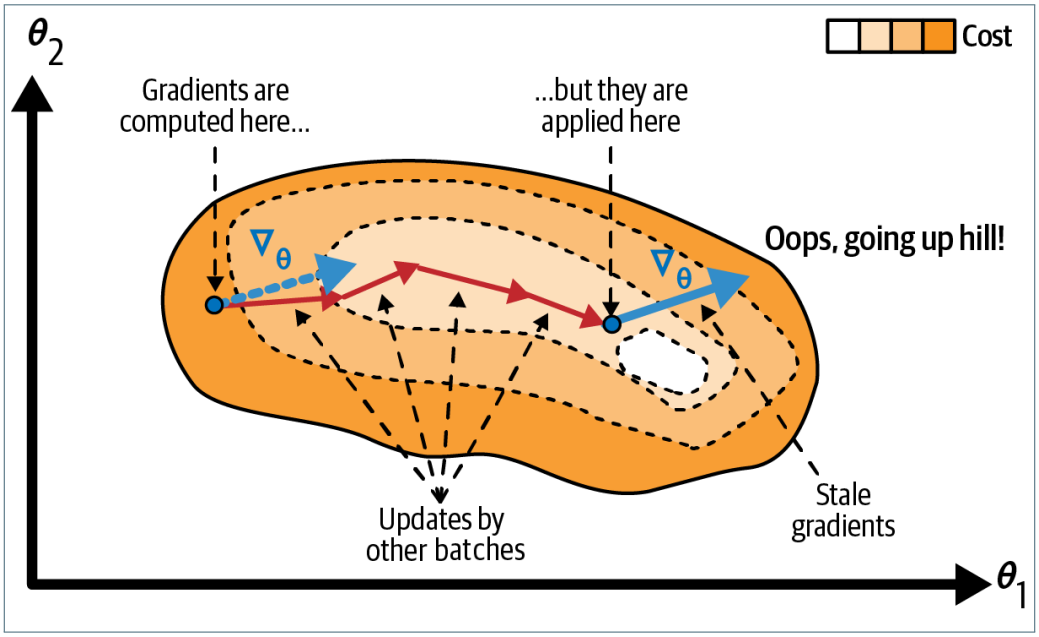

图 17-20: 异步更新中的陈旧梯度（Stale gradients when using asynchronous updates）

stage 越多，问题越严重。不过，论文作者提出了一些缓解方法：例如，每个 stage 在前向传播期间保存权重，在反向传播期间恢复这些权重，确保前向和反向使用同一组权重。这称为权重暂存（weight stashing）。借助这些方法，PipeDream 展现了出色的扩展能力。

如果没有多张 GPU 来训练或微调大型 Transformer，也可以从 Amazon Web Services、Google Cloud Platform、Microsoft Azure、RunPod、Lambda Labs、CoreWeave、Vast.ai、TensorDock、Paperspace 等平台租用 GPU。

Transformer 还可以使用两种方式把计算拆到 GPU 之间。

上下文并行（Context Parallelism）

每张 GPU 只存储和处理每个输入序列在序列维度上的一部分。例如，GPU #1 处理位置 1 到 100 的 token，GPU #2 处理位置 101 到 200 的 token，依此类推。在 MHA 层中，每张 GPU 只为自己的 token 计算 Q、K、V 矩阵；随后所有 GPU 互相共享 K、V 矩阵，因此每张 GPU 都可以为自己的 token 计算注意力分数和输出。这降低了每张 GPU 的激活内存，因为每张 GPU 只需处理一部分 token 激活；也降低了每张 GPU 的计算量，因为每张 GPU 只计算一部分注意力矩阵。

专家并行（Expert Parallelism）

在 MoE Transformer 中，可以把专家分布到不同 GPU 上。主要困难是需要全互联通信，而不仅是 all-gather 或 all-reduce，以便把正确的 token 发送给正确的专家。每个 MoE 层通常要进行两次：一次发送输入 token，一次返回输出。因此，I/O 带宽又会迅速成为主要瓶颈。

另一个可以加速多 GPU 训练的重要技术是零冗余优化器（Zero Redundancy Optimizer，ZeRO），由 Microsoft 研究人员于 2019 年提出。它把优化器状态分片到多个 GPU 上，可以显著降低内存使用量。

这些事情听起来非常困难，而事实确实如此。幸运的是，一些优秀的开源库封装了复杂性，让你只用几行代码，就能跨多台服务器和多个 GPU 高效训练大型 Transformer。主要工具包括：

- **DeepSpeed（Microsoft）**：适合极大规模的多服务器训练，实现了 ZeRO 优化器，支持卸载优化器参数以及许多其他功能。
- **Megatron-LM（NVIDIA）**：更侧重单服务器上的模型并行，但也在逐步支持多服务器；通过高度优化的 kernel、张量并行、流水线并行、混合精度和激活检查点实现高速度。
- **Fully Sharded Data Parallelism（FSDP，PyTorch 核心团队）**：完全集成在 PyTorch 中，提供简洁的 API。FSDP 把模型参数、梯度和优化器状态分片到多个设备；不过功能不如 DeepSpeed 丰富。
- **Accelerate（Hugging Face）**：在 DeepSpeed、FSDP 等库之上提供统一抽象层，支持混合精度、梯度累积等功能。可以先编写简单且与设备无关的 PyTorch 代码，再使用配置 CLI 在任意分布式设置上运行；底层可能使用 DeepSpeed、FSDP 或 PyTorch 内置 DDP。

如果想详细了解 GPU 集群上的大型 Transformer 训练，推荐阅读 Hugging Face 团队编写的《The Ultra-Scale Playbook: Training LLMs on GPU Clusters》。

本章介绍了许多技术！我们先讨论了 KV 缓存、推测解码和其他解码加速方法；然后探索了多种加速多头注意力的方法，包括稀疏注意力、近似注意力、跨注意力头共享投影矩阵，当然还有 FlashAttention；接着了解了如何使用专家混合扩展 Transformer；最后介绍了加速训练的多种方式，包括 LoRA 等参数高效微调、激活检查点、序列打包和分桶、梯度累积，以及跨多张 GPU 分配数据和计算。呼！

下一章（位于主书中）将转向完全不同的主题：使用自编码器、生成对抗网络（GAN）和扩散模型进行表示学习与生成式机器学习。

17.6　练习（Exercises）

1. 什么是 KV 缓存？它用于解决什么问题？它更适合训练还是推理？更适合仅编码器模型还是仅解码器模型？
2. 什么时候应该使用推测解码？它如何工作？草稿模型是否可以任意选择，还是存在限制？
3. 说出几种专门用于并行化文本生成的技术。
4. 所有稀疏注意力方法的核心思想是什么？说出几种最重要的方法，并概括它们的主要思想和优势。
5. 说出一种低秩注意力方法和一种基于核的注意力方法。它们如何工作？
6. 调整 GQA 中的组大小时，需要权衡什么？
7. 多头潜在注意力会增大 KV 缓存吗？为什么？
8. GPU 中主要有哪些类型的内存？
9. 为什么 FlashAttention 比以前的实现快得多？
10. 训练 MoE 模型时有哪些主要挑战？最常见的解决方案是什么？
11. 训练大型 Transformer 时，造成内存饱和的四个主要因素是什么？针对每个因素，可以使用哪些主要技术来降低内存需求？
12. 选择一个大型 Transformer 模型，使用 LoRA 在自己选择的数据集上进行微调。
13. 如果可以使用多张 GPU，尝试使用 Hugging Face Accelerate 在自己选择的数据集上微调大型 Transformer 模型。

17.7　原书超链接资源

下列链接来自用户提供的第 17 章 PDF，保持为可点击外部超链接：

- KV cache
- Speculative decoding
- 并行文本生成综述
- Parallel text generation survey
- Sparse Transformer
- Longformer
- BigBird
- Routing Transformer
- Reformer
- Locality-sensitive hashing
- Angular LSH
- Linformer
- Nyströmformer
- Performer
- 本书在线主页
- Random Fourier features
- Multi-query attention
- Grouped-query attention
- Multi-head latent attention
- FlashAttention
- Julia
- Mojo
- FlashAttention-2
- FlashAttention-3
- Mixture of experts
- 稀疏门控 MoE
- GShard

- Switch Transformer
- Vision-MoE
- GLaM
- ST-MoE
- MoE survey
- LoRA
- Distributed Data Parallel
- PyTorch deterministic behavior
- PipeDream
- ZeRO
- The Ultra-Scale Playbook

第 18 章　自编码器、GAN 和扩散模型

Autoencoders, GANs, and Diffusion Models

自编码器（autoencoder）是能够在无监督条件下学习输入数据稠密表示的人工神经网络。这些表示也称为潜在表示（latent representation）或编码（coding），通常比输入数据的维度低得多，因此自编码器特别适合降维（参见第 7 章），尤其适合可视化。自编码器还可以充当特征检测器，用于深度神经网络的无监督预训练（参见第 11 章），也常用于异常检测。最后，一些自编码器还是生成模型，能够随机生成与训练数据非常相似的新数据。例如，在人脸图片上训练自编码器后，它可以生成新的人脸。

生成式对抗网络（generative adversarial network，GAN）同样是能够生成数据的神经网络。它们可以生成逼真到难以相信的脸部图片；你可以访问 This Person Does Not Exist，亲自看看 StyleGAN 生成的人脸。GAN 曾广泛用于超分辨率、图像着色、强大的图像编辑、把简单草图转换为照片级图像、预测视频下一帧、扩充数据集，以及生成文本、音频和时间序列等数据。

不过，自 2020 年代初起，GAN 在很大程度上被扩散模型（diffusion model）取代。扩散模型比 GAN 更容易训练，还能生成更多样、质量更高的图像；但运行速度慢得多，因此在需要极快生成时，GAN 仍然有用。

自编码器、GAN 和扩散模型都是无监督模型，都能学习潜在表示，也都可以作为生成模型，应用场景也有许多相似之处。不过，它们的工作方式完全不同：

自编码器（Autoencoders）

自编码器只学习把输入复制到输出。这听起来很简单，但如果以各种方式约束网络，任务就会变得非常困难。例如，可以限制潜在表示的大小，或向输入添加噪声并训练网络恢复原始输入。这些约束会阻止自编码器直接把输入原样复制到输出，迫使它学习高效表示数据的方式。简而言之，编码是自编码器在约束条件下学习恒等函数的副产品。

GAN（GANs）

GAN 由两个神经网络组成：生成器（generator）试图生成与训练数据相似的数据，判别器（discriminator）则试图区分真实数据和伪造数据。在训练过程中，生成器与判别器彼此竞争，这称为对抗训练（adversarial training）。生成器常被比作试图制造逼真假钞的罪犯，判别器则像试图辨别真假钞的警察。

扩散模型（Diffusion Models）

扩散模型经过训练，可以逐渐去除图像中的噪声。如果从一张完全由随机噪声组成的图像开始，反复运行扩散模型，就会逐渐得到一张与训练图像相似但并不完全相同的高质量图像。

本章先深入探讨自编码器的工作方式，以及如何将其用于降维、特征提取、无监督预训练和生成模型。这自然会引出 GAN。我们将构建一个简单的 GAN 来生成伪造图像，并讨论对抗训练的主要困难及相应的解决技术。最后，我们将构建并训练一个去噪扩散概率模型（denoising diffusion probabilistic model，DDPM）来生成图像。

让我们从自编码器开始！

18.1　高效的数据表示（Efficient Data Representations）

下面哪组数字序列更容易记忆？

```
40, 27, 25, 36, 81, 57, 10, 73, 19, 68
50, 48, 46, 44, 42, 40, 38, 36, 34, 32, 30, 28, 26, 24, 22, 20, 18, 16, 14
```

乍看之下，第一组更短，似乎应该更容易。但仔细观察第二组就会发现，它只是从 50 倒数到 14 的偶数列表。一旦注意到这个规律，第二组就比第一组容易记忆得多，因为你只需要记住模式（递减的偶数）以及起始和结束数字（50 和 14）。正是因为长序列难以记忆，识别模式才有用，这也说明了为什么在训练过程中约束自编码器会促使它发现并利用数据中的模式。

William Chase 和 Herbert Simon 在 20 世纪 70 年代初研究了记忆、感知和模式匹配之间的关系。他们观察到，顶尖棋手只需看棋盘五秒，就能记住一局棋中所有棋子的位置；但只有当棋子处于真实对局中的合理位置时才是这样，随机摆放棋子则不行。棋手并没有比普通人好得多的记忆力，只是凭借经验更容易看出棋局模式。识别模式可以帮助他们高效存储信息。

与记忆实验中的棋手类似，自编码器观察输入，把输入转换为高效的潜在表示，然后重建出尽可能接近输入的结果。自编码器总是由两部分组成：编码器（encoder，也称识别网络）把输入转换为潜在表示，解码器（decoder，也称生成网络）再把内部表示转换为输出。

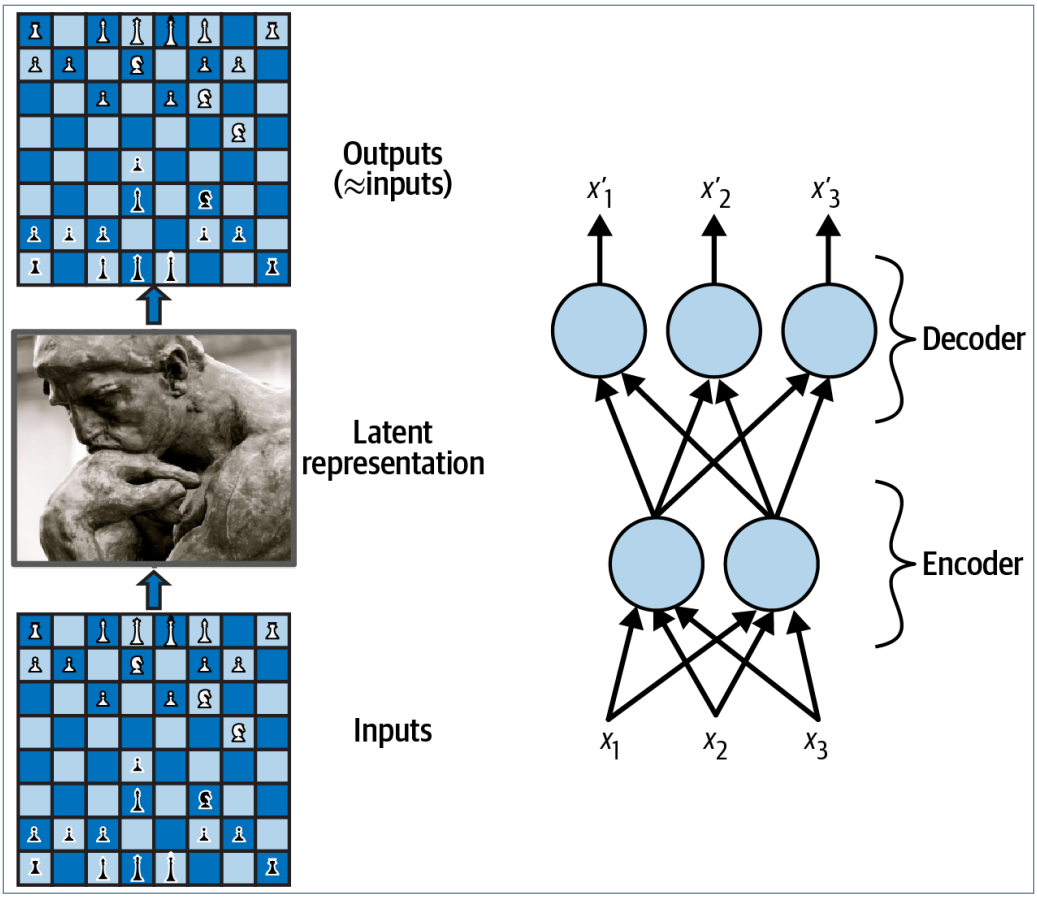

图 18-1: 棋盘记忆实验（左）和简单自编码器（右）（The chess memory experiment and a simple autoencoder）

图 18-1 中的自编码器是一个普通的多层感知机（MLP；参见第 9 章）。由于它必须重建输入，输出层的神经元数量必须等于输入数量（本例为 3）。网络下半部分是编码器（本例中是一个含两个神经元的单层），上半部分是解码器。输出通常称为重建（reconstruction），因为自编码器试图重建输入。代价函数总是包含重建损失，用于惩罚与输入不一致的重建结果。

由于内部表示的维度低于输入数据（本例为二维而不是三维），这种自编码器称为欠完备自编码器（undercomplete autoencoder）。欠完备自编码器不能把输入直接复制到编码中，同时又必须输出输入的副本，因此它被迫压缩数据，学习输入数据中最重要的特征并舍弃不重要的信息。

18.1.1　使用欠完备线性自编码器执行 PCA（Performing PCA with an Undercomplete Linear Autoencoder）

如果自编码器只使用线性激活函数，代价函数使用均方误差（MSE），那么它最终执行的就是主成分分析（PCA；参见第 7 章）。下面的代码构建一个简单的线性自编码器：输入为三维，先投影到二维，再投影回三维。由于训练目标与输入相同，梯度下降会找到最接近训练数据的二维平面，就像 PCA 一样。

```
import torch
import torch.nn as nn
torch.manual_seed(42)
encoder = nn.Linear(3, 2)
decoder = nn.Linear(2, 3)
autoencoder = nn.Sequential(encoder, decoder).to(device)
```

这段代码与前面构建的 MLP 并没有太大区别，但有几点值得注意：

- 我们把自编码器拆成编码器和解码器两个子组件，本例中二者各由一个 `Linear` 层组成；整个自编码器是一个先执行编码器、再执行解码器的 `Sequential` 模型。
- 自编码器的输出数量等于输入数量，即 3。
- 为了执行 PCA，我们不使用激活函数，所有神经元都是线性的，代价函数使用 MSE；这是因为 PCA 是线性变换。

接下来在第 7 章使用过的简单三维数据集上训练模型，并使用编码器将数据编码到二维：

```
from torch.utils.data import DataLoader, TensorDataset
X_train = [...]  # generate a 3D dataset, like in Chapter 7
train_set = TensorDataset(X_train, X_train)  # the inputs are also the targets
train_loader = DataLoader(train_set, batch_size=32, shuffle=True)
```

这里 `X_train` 同时用作输入和目标。接着使用第 10 章介绍的同一个 `train()` 函数训练自编码器：

```
import torchmetrics
optimizer = torch.optim.NAdam(autoencoder.parameters(), lr=0.2)
mse = nn.MSELoss()
rmse = torchmetrics.MeanSquaredError(squared=False).to(device)
train(autoencoder, optimizer, mse, train_loader, n_epochs=20)
```

训练完成后，可以使用编码器把三维输入压缩到二维。例如，压缩整个训练集：

```
codings = encoder(X_train.to(device))
```

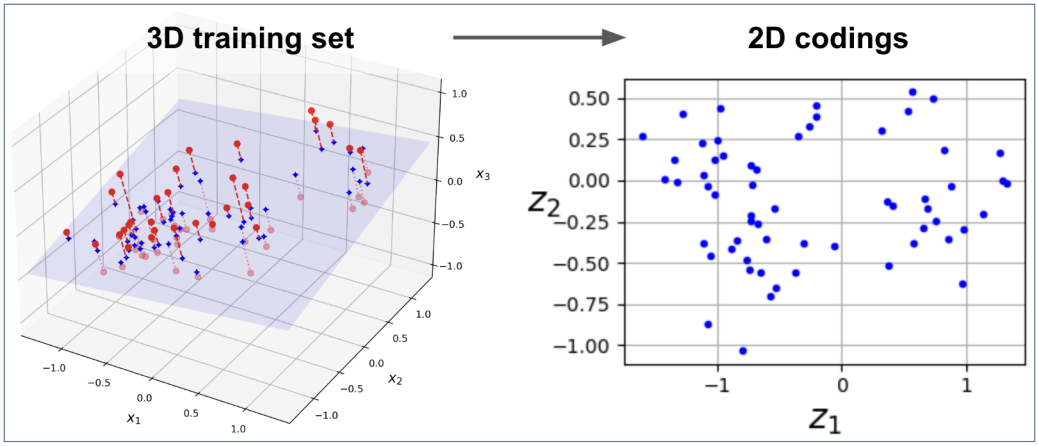

图 18-2: 欠完备线性自编码器执行的近似 PCA（Approximate PCA performed by an undercomplete linear autoencoder）

图 18-2 左侧是原始三维数据集，右侧是自编码器隐藏层（编码层）的输出。可以看到，自编码器找到一个最优二维平面，把数据投影到这个平面上，并尽可能保留数据中的方差，就像 PCA 一样。

> **注意**　可以把自编码器看成一种形式的自监督学习，因为它使用监督学习技术，只是标签是自动生成的（本例中标签恰好等于输入）。

18.2　堆叠自编码器（Stacked Autoencoders）

与前面讨论的其他神经网络一样，自编码器也可以有多个隐藏层。这种自编码器称为堆叠自编码器（stacked autoencoder），也称深度自编码器（deep autoencoder）。增加层数可以让自编码器学习更复杂的编码，但必须注意不要让自编码器过于强大。假设编码器强大到可以把每个输入映射到一个任意的单一数字，解码器再学习逆映射。这样的自编码器可以完美重建训练数据，却没有学到有用的数据表示，因而很可能无法泛化到新实例。

堆叠自编码器通常关于中心隐藏层（编码层）对称，看起来像一个三明治。例如，用于 Fashion MNIST 的自编码器可以有 784 个输入，接着是含 128 个神经元的隐藏层、含 32 个神经元的中心隐藏层、另一个含 128 个神经元的隐藏层，最后是含 784 个神经元的输出层。

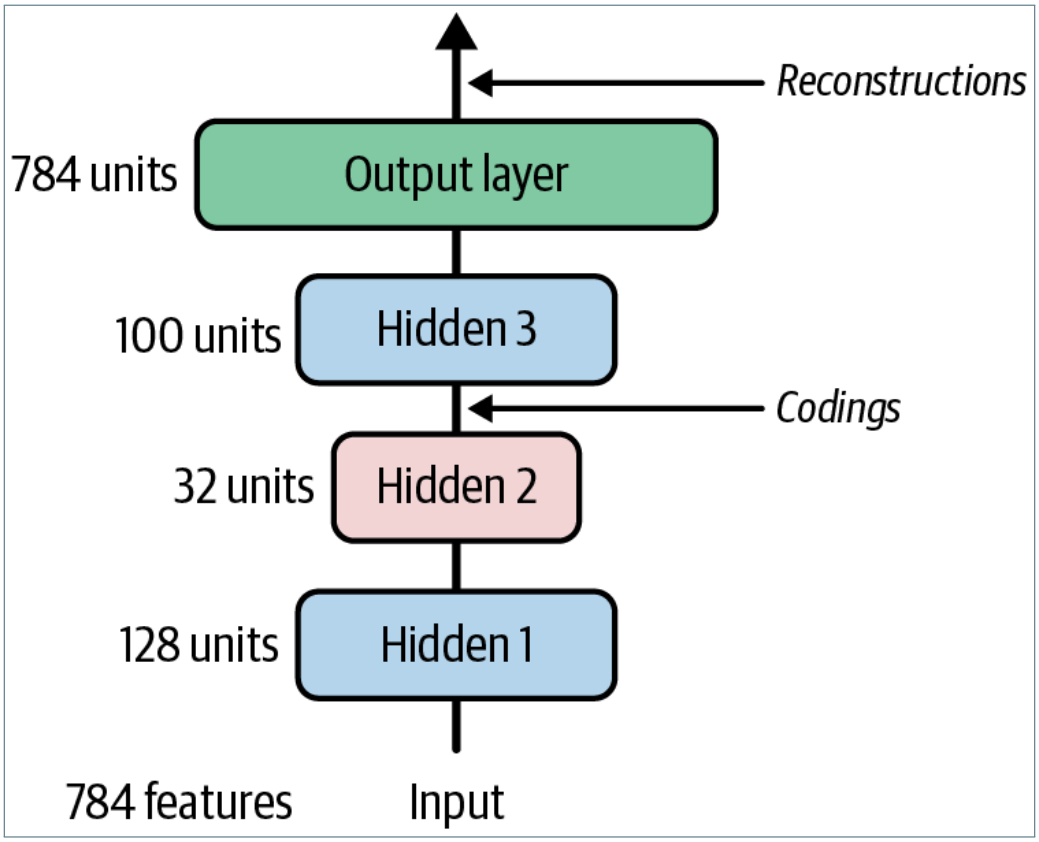

图 18-3: 堆叠自编码器（Stacked autoencoder）

18.2.1　使用 PyTorch 实现堆叠自编码器（Implementing a Stacked Autoencoder Using PyTorch）

下面的自编码器可以处理 Fashion MNIST 图像：

```
stacked_encoder = nn.Sequential(
    nn.Flatten(),
    nn.Linear(1 * 28 * 28, 128), nn.ReLU(),
    nn.Linear(128, 32), nn.ReLU(),
)
stacked_decoder = nn.Sequential(
    nn.Linear(32, 128), nn.ReLU(),
    nn.Linear(128, 1 * 28 * 28), nn.Sigmoid(),
    nn.Unflatten(dim=1, unflattened_size=(1, 28, 28))
)
stacked_ae = nn.Sequential(stacked_encoder, stacked_decoder).to(device)
```

编码器和解码器仍然是两个子模型。编码器接收 28×28 像素的灰度图像，将每张图像展平为长度 784 的向量，然后依次通过大小递减的两个 **Linear** 层（128 个单元，再到 32 个单元），每层后使用 ReLU。每张输入图像最终得到一个长度为 32 的向量。解码器接收长度为 32 的编码，依次通过大小递增的两个 **Linear** 层（128 个单元，再到 784 个单元），然后把最终向量重塑为 1×28×28 数组，使输出形状与输入相同。输出层使用 sigmoid 而不是 ReLU，以确保像素值在 0 到 1 之间。

现在使用 TorchVision 加载 Fashion MNIST 数据集，划分为 **train_data**、**valid_data** 和 **test_data**，再像前一个自编码器一样训练：把输入作为目标，最小化 MSE。请记得修改目标，使其与输入相同；我们训练的是自编码器，而不是分类器。

18.2.2　可视化重建结果（Visualizing the Reconstructions）

训练堆叠自编码器后，可以比较输入和输出来判断模型是否训练良好。下面绘制验证集中的若干图像及其重建结果：

```
import matplotlib.pyplot as plt
def plot_image(image):
    plt.imshow(image.permute(1, 2, 0).cpu(), cmap="binary")
    plt.axis("off")
def plot_reconstructions(model, images, n_images=5):
    images = images[:n_images]
    with torch.no_grad():
        y_pred = model(images.to(device))
    fig = plt.figure(figsize=(len(images) * 1.5, 3))
    for idx in range(len(images)):
        plt.subplot(2, len(images), 1 + idx)
        plot_image(images[idx])
        plt.subplot(2, len(images), 1 + len(images) + idx)
        plot_image(y_pred[idx])
X_valid = torch.stack([x for x, _ in valid_data])
plot_reconstructions(stacked_ae, X_valid)
plt.show()
```

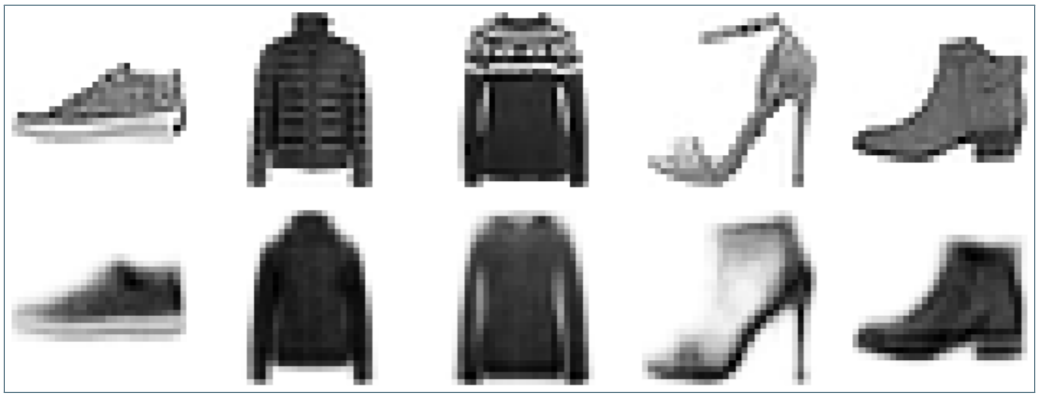

图 18-4: 原始图像（上）和重建图像（下）（Original images (top) and their reconstructions (bottom)）

图 18-4 的重建结果能够辨认，但损失有些大。可以训练更长时间、增强编码器和解码器，或增大编码长度。现在先保留这个模型，看看它还能怎样使用。

18.2.3　使用自编码器进行异常检测（Anomaly Detection Using Autoencoders）

自编码器的一个常见用途是异常检测。如果向自编码器输入一张与训练图像不同的图像（称为分布外图像，out-of-distribution image），重建结果就会很差。例如，图 18-5 展示了在 Fashion MNIST 上训练的模型重建 MNIST 数字的结果；这些重建与输入非常不同，因此重建损失（输入与输出之间的 MSE）会很高。要把模型用于异常检测，只需定义一个阈值，重建损失高于该阈值的图像就可以视为异常。

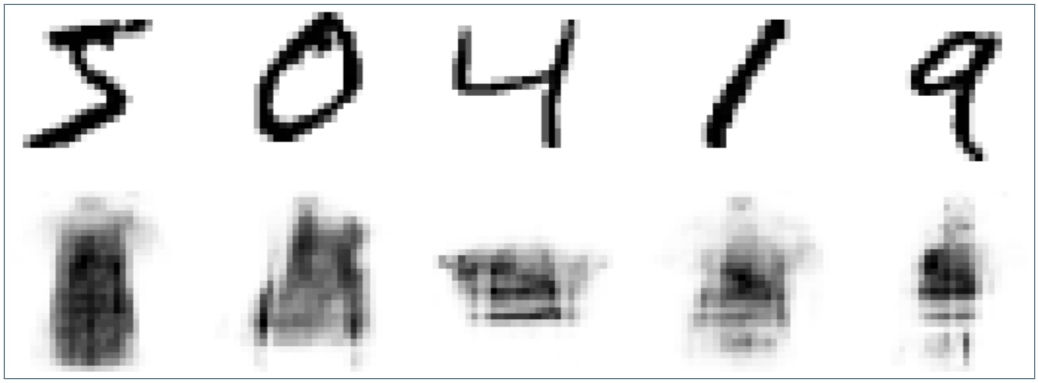

图 18-5: 分布外图像的重建效果很差（Out-of-distribution images are poorly reconstructed）

18.2.4　可视化 Fashion MNIST 数据集（Visualizing the Fashion MNIST Dataset）

欠完备自编码器可以用于降维，但对于大多数数据集，它不能很好地把维度降低到二维或三维，因为需要足够多的维度才能正确重建输入。因此，自编码器通常不会直接用于可

视化。但它们很适合处理超大数据集：可以先用自编码器把维度降低到合理大小，再使用第 7 章介绍的其他降维算法（例如 t-SNE）进行可视化。

下面使用堆叠自编码器的编码器把 Fashion MNIST 降到 32 维，再使用 Scikit-Learn 的 t-SNE 把维度降到 2：

```
from sklearn.manifold import TSNE
with torch.no_grad():
    X_valid_compressed = stacked_encoder(X_valid.to(device))
tsne = TSNE(init="pca", learning_rate="auto", random_state=42)
X_valid_2D = tsne.fit_transform(X_valid_compressed.cpu())
```

```
plt.scatter(X_valid_2D[:, 0], X_valid_2D[:, 1], c=y_valid, s=10, cmap="tab10")
plt.show()
```

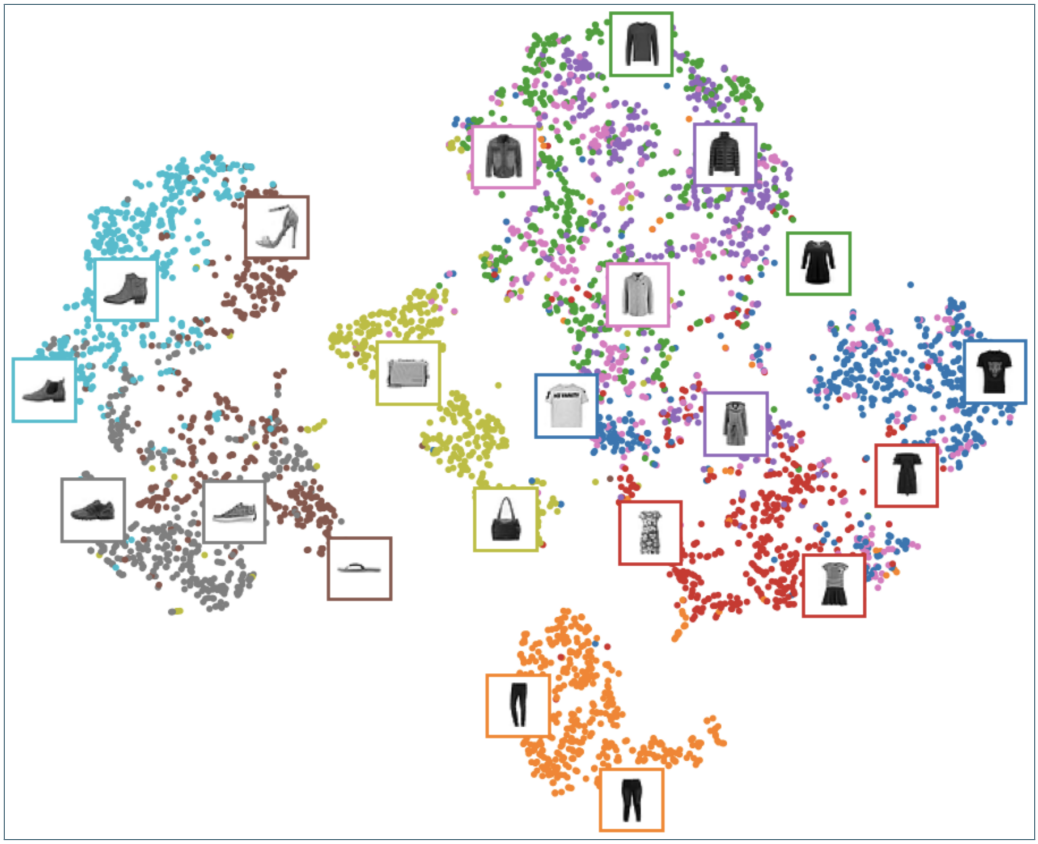

图 18-6: 使用自编码器再结合 t-SNE 可视化 Fashion MNIST（Fashion MNIST visualization using an autoencoder, followed by t-SNE）

图 18-6 中，t-SNE 识别出若干与类别大致对应的簇，每个类别使用不同颜色表示。请注意，如果使用不同随机种子、略有不同的数据或不同平台，t-SNE 的输出可能差异很大。

18.2.5　使用堆叠自编码器进行无监督预训练（Unsupervised Pretraining Using Stacked Autoencoders）

如第 11 章所述，如果要处理复杂的监督任务，却没有很多带标签的训练数据，一个办法是找到执行相似任务的神经网络并复用其低层。这样，网络不需要重新学习所有低级特征，只需复用已有网络学到的特征检测器，就能用较少的训练数据训练高性能模型。

类似地，如果拥有大量数据，但其中大部分没有标签，可以先使用全部数据训练堆叠自编码器，再复用其低层创建实际任务所需的神经网络，并在带标签数据上训练。图 18-7 展示了如何使用堆叠自编码器为分类网络进行无监督预训练。如果带标签数据确实很少，可以冻结预训练层，至少冻结较低的层。

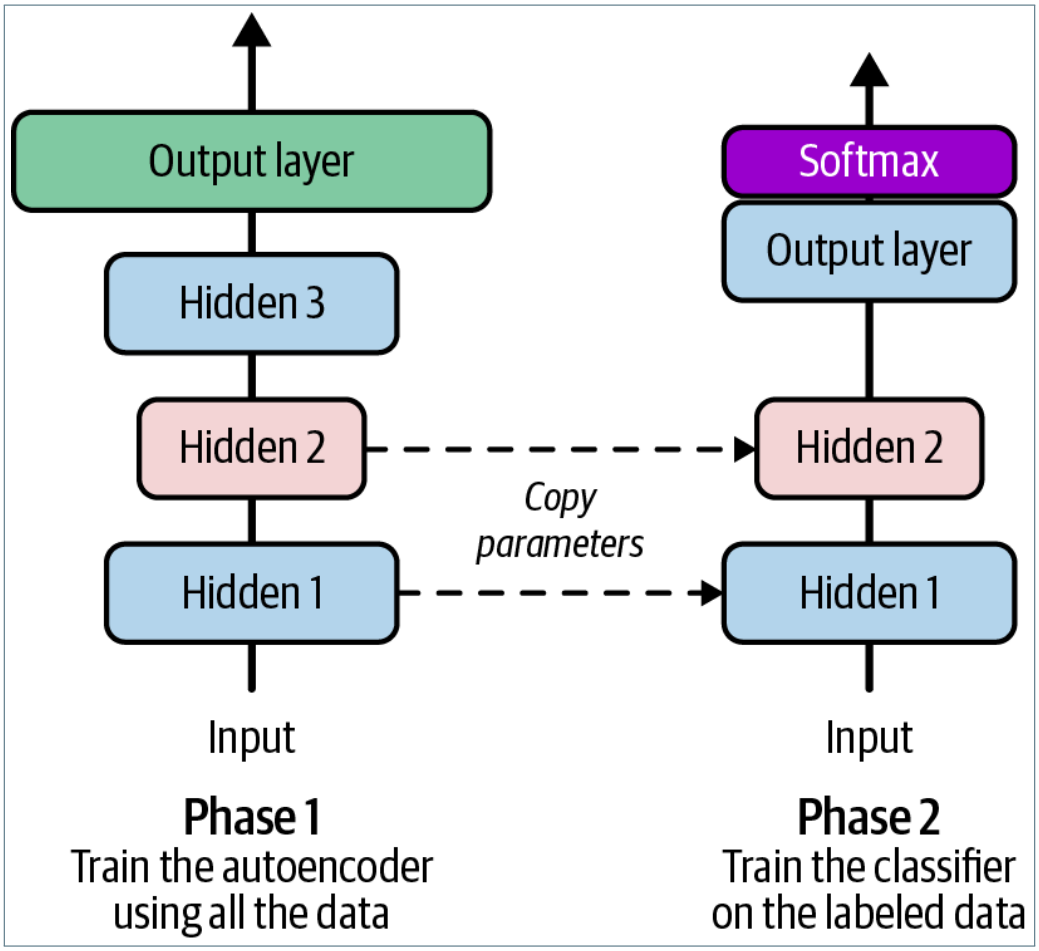

图 18-7: 使用自编码器进行无监督预训练（Unsupervised pretraining using autoencoders）

> **注意**　拥有大量无标签数据、只有少量带标签数据是很常见的情况。构建无标签数据集通常很便宜，例如用简单脚本从互联网下载数百万张图像；但可靠地为这些图像添加标签通常只能由人完成。标注实例既耗时又昂贵，因此只有几千个甚至更少的人工标注实例很正常。不过，使用先进 AI 为数据集自动标注的趋势正在增长。

实现上没有特殊之处：先用全部训练数据（带标签和不带标签）训练自编码器，再复用其编码器层创建新的神经网络，并在带标签实例上训练。下面介绍几种训练堆叠自编码器的技术。

18.2.6　权重绑定（Tying Weights）

当自编码器结构对称时，一个常用技术是把解码器层的权重与编码器层的权重绑定。这样可以把模型中的权重数量减半，从而加快训练并降低过拟合风险。具体来说，假设自编码器共有 N 层（不计输入层），第 L 层的连接权重记为 (W_L)，则解码器层的权重可以定义为 (W_L=W_{N-L+1}^{\{}mathsf{T}})，其中 (L=N/2+1,\{}ldots,N)。

```
import torch.nn.functional as F
class TiedAutoencoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.enc1 = nn.Linear(1 * 28 * 28, 128)
        self.enc2 = nn.Linear(128, 32)
        self.dec1_bias = nn.Parameter(torch.zeros(128))
        self.dec2_bias = nn.Parameter(torch.zeros(1 * 28 * 28))
    def encode(self, X):
        Z = X.view(-1, 1 * 28 * 28)  # flatten
        Z = F.relu(self.enc1(Z))
        return F.relu(self.enc2(Z))
    def decode(self, X):
        Z = F.relu(F.linear(X, self.enc2.weight.t(), self.dec1_bias))
        Z = F.sigmoid(F.linear(Z, self.enc1.weight.t(), self.dec2_bias))
        return Z.view(-1, 1, 28, 28)  # unflatten
    def forward(self, X):
        return self.decode(self.encode(X))
```

这个模型使用大约一半的参数，却比前一个模型获得了更小的重建误差。

18.2.7　一次训练一个自编码器（Training One Autoencoder at a Time）

不必一次性训练整个堆叠自编码器，也可以一次训练一个浅层自编码器，然后把它们堆叠成一个完整的堆叠自编码器，如图 18-8 所示。这种技术称为贪心逐层训练（greedy layer-wise training）。

第一阶段训练第一个自编码器，使它重建输入；然后使用这个自编码器编码整个训练集，得到一个新的压缩训练集。接着在这个新数据集上训练第二个自编码器。最后，把所有自编码器的编码器层按顺序堆叠，再把解码器层按相反顺序堆叠，形成最终的堆叠自编码器。用这种方式还可以继续添加更多自编码器，构建很深的堆叠自编码器。

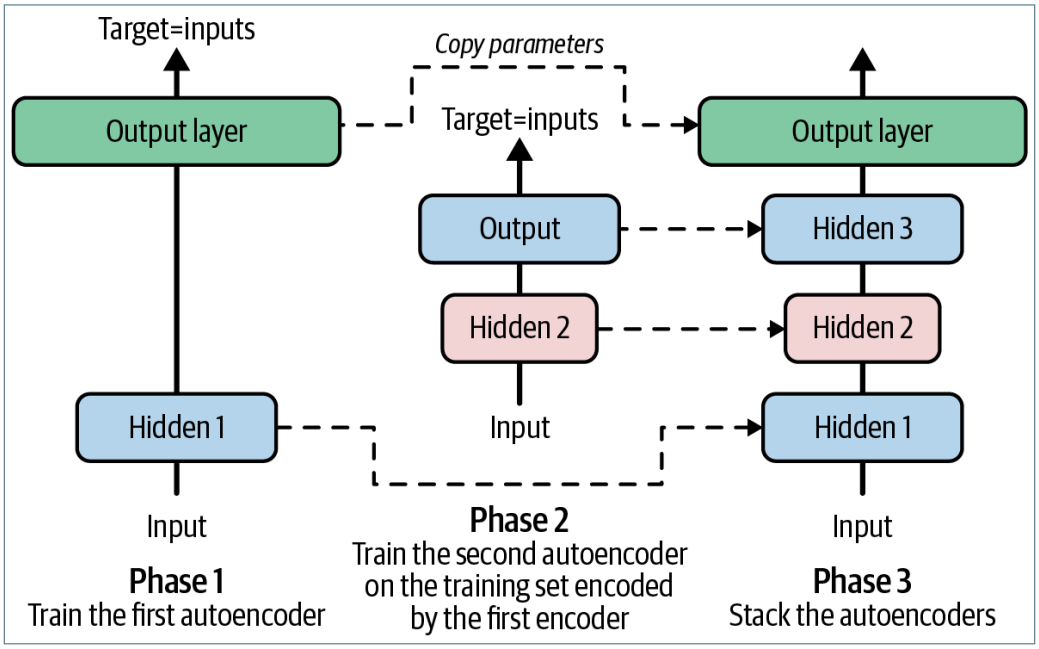

图 18-8: 一次训练一个自编码器（Training one autoencoder at a time）

正如第 11 章提到的，深度学习浪潮的一个触发点，是 Geoffrey Hinton 等人在 2006 年发现可以用这种贪心逐层方法对深度神经网络进行无监督预训练。他们使用受限玻尔兹曼机（RBM）完成这项工作，而 Yoshua Bengio 等人在 2007 年证明自编码器同样有效。多年间，这是训练深度网络的唯一高效方法，直到第 11 章介绍的许多技术使得一次性训练深度网络成为可能。

自编码器并不限于稠密网络，也可以构建卷积自编码器。

18.3　卷积自编码器（Convolutional Autoencoders）

如果处理图像，前面看到的自编码器通常效果不好（除非图像非常小）。如第 12 章所见，卷积神经网络比稠密网络更适合处理图像。用于图像的自编码器，其编码器通常是由卷积层和池化层组成的 CNN，逐渐降低输入的空间维度（高度和宽度），同时增加深度（特征图数量）。解码器执行相反操作：放大图像并把深度恢复到原始维度；可以使用转置卷积层，也可以组合上采样层和卷积层。

下面是一个用于 Fashion MNIST 的基本卷积自编码器：

```
conv_encoder = nn.Sequential(
    nn.Conv2d(1, 16, kernel_size=3, padding="same"), nn.ReLU(),
    nn.MaxPool2d(kernel_size=2),  # output: 16 × 14 × 14
    nn.Conv2d(16, 32, kernel_size=3, padding="same"), nn.ReLU(),
    nn.MaxPool2d(kernel_size=2),  # output: 32 × 7 × 7
    nn.Conv2d(32, 64, kernel_size=3, padding="same"), nn.ReLU(),
    nn.MaxPool2d(kernel_size=2),  # output: 64 × 3 × 3
    nn.Conv2d(64, 32, kernel_size=3, padding="same"), nn.ReLU(),
    nn.AdaptiveAvgPool2d((1, 1)), nn.Flatten())  # output: 32
conv_decoder = nn.Sequential(
    nn.Linear(32, 16 * 3 * 3),
    nn.Unflatten(dim=1, unflattened_size=(16, 3, 3)),
    nn.ConvTranspose2d(16, 32, kernel_size=3, stride=2), nn.ReLU(),
    nn.ConvTranspose2d(32, 16, kernel_size=3, stride=2, padding=1,
                       output_padding=1), nn.ReLU(),
    nn.ConvTranspose2d(16, 1, kernel_size=3, stride=2, padding=1,
                       output_padding=1), nn.Sigmoid())
conv_ae = nn.Sequential(conv_encoder, conv_decoder).to(device)
```

也可以使用其他类型的架构，例如 RNN（参见本章配套笔记本）。到这里，我们已经看过基本、堆叠和卷积自编码器，以及一次性训练或逐层训练的方法；也看过降维、异常检测和无监督预训练等应用。

此前我们通过限制编码层大小，把自编码器限制为欠完备自编码器。实际上还有许多其他约束方式，包括允许编码层与输入一样大，甚至更大，从而得到过完备自编码器（over-complete autoencoder）。下面讨论去噪自编码器、稀疏自编码器和变分自编码器。

18.4　去噪自编码器（Denoising Autoencoders）

迫使自编码器学习有用特征的一种简单方式，是向输入添加噪声，并训练它恢复原始的无噪声输入。这一想法可以追溯到 20 世纪 80 年代；2008 年 Pascal Vincent 等人证明自编码器也可以用于特征提取，2010 年他们又提出了堆叠去噪自编码器。噪声可以是添加到输入上的高斯噪声，也可以像 dropout 一样随机关闭输入。

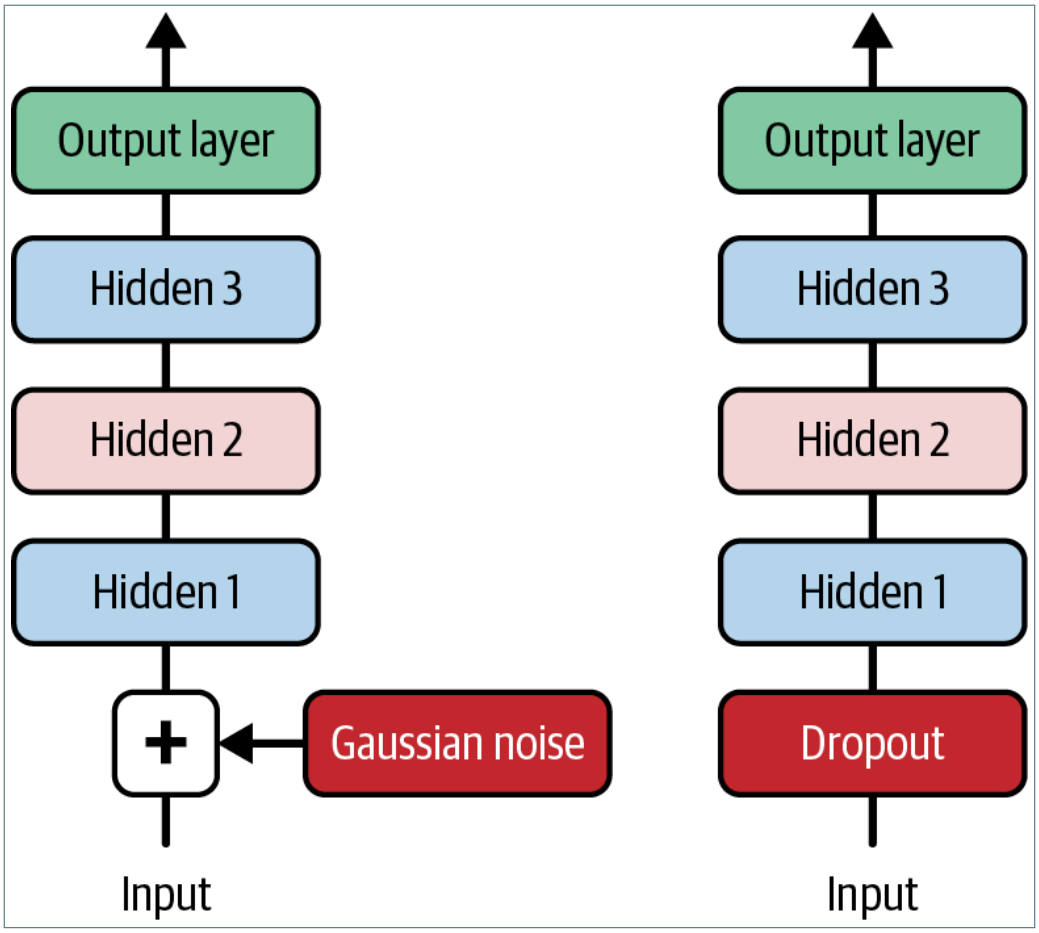

图 18-9: 去噪自编码器：高斯噪声（左）和 dropout（右）（Denoising autoencoders, with Gaussian noise (left) or dropout (right)）

dropout 去噪自编码器的实现很直接：它就是一个普通的堆叠自编码器，只是在编码器输入上额外添加一个 **Dropout** 层（**Dropout** 层只在训练期间激活）。由于噪声已经让重建任务变得困难，编码层不需要把数据压缩得那么厉害：

```
dropout_encoder = nn.Sequential(
    nn.Flatten(),
    nn.Dropout(0.5),
    nn.Linear(1 * 28 * 28, 128), nn.ReLU(),
    nn.Linear(128, 128), nn.ReLU(),
)
dropout_decoder = nn.Sequential(
    nn.Linear(128, 128), nn.ReLU(),
    nn.Linear(128, 1 * 28 * 28), nn.Sigmoid(),
    nn.Unflatten(dim=1, unflattened_size=(1, 28, 28))
)
dropout_ae = nn.Sequential(dropout_encoder, dropout_decoder).to(device)
```

注意　这可能会让你想起 BERT 的 MLM 预训练任务（参见第 15 章）：重建被掩码的输入；区别在于 BERT 没有拆分成编码器和解码器。

图 18-10: 带噪图像（上）及其重建结果（下）（Noisy images (top) and their reconstructions (bottom)）

图 18-10 展示了训练后，dropout 去噪自编码器重建的图像。自编码器甚至会猜测输入中不存在的细节，例如最右侧鞋子的鞋帮。由此可见，去噪自编码器不仅可用于可视化或无监督预训练，也可以简单高效地去除图像噪声。

18.5　稀疏自编码器（Sparse Autoencoders）

另一种经常带来良好特征提取效果的约束是稀疏性：在代价函数中添加适当项，促使编码层中激活的神经元数量减少。这样，自编码器会把每个输入表示为少量激活的组合，因此编码层的每个神经元通常会对应一个有用特征。

一种基本方法是在编码层使用 sigmoid 激活函数，把编码限制在 0 到 1 之间；使用较大的编码层（例如 256 个单元）；再对编码层的激活值添加 ℓ_1 正则化。这等价于把编码的 ℓ_1 范数（绝对值之和）按稀疏性超参数加权后加入损失。稀疏损失会鼓励网络产生接近 0 的编码，但总损失仍包含重建损失，因此模型必须输出至少几个非零值来正确重建输入。与 ℓ_2 范数相比，ℓ_1 范数会促使网络保留最重要的编码，消除对当前输入图像不需要的编码。

另一种通常效果更好的方法，是在每个训练批次中测量编码层每个神经元的平均稀疏度，并在平均稀疏度偏离目标稀疏度（例如 10%）时惩罚模型。批次不能太小，否则平均值不准确。实践中使用 Kullback-Leibler（KL）散度比直接使用平方误差好，因为 KL 散度的梯度更强。

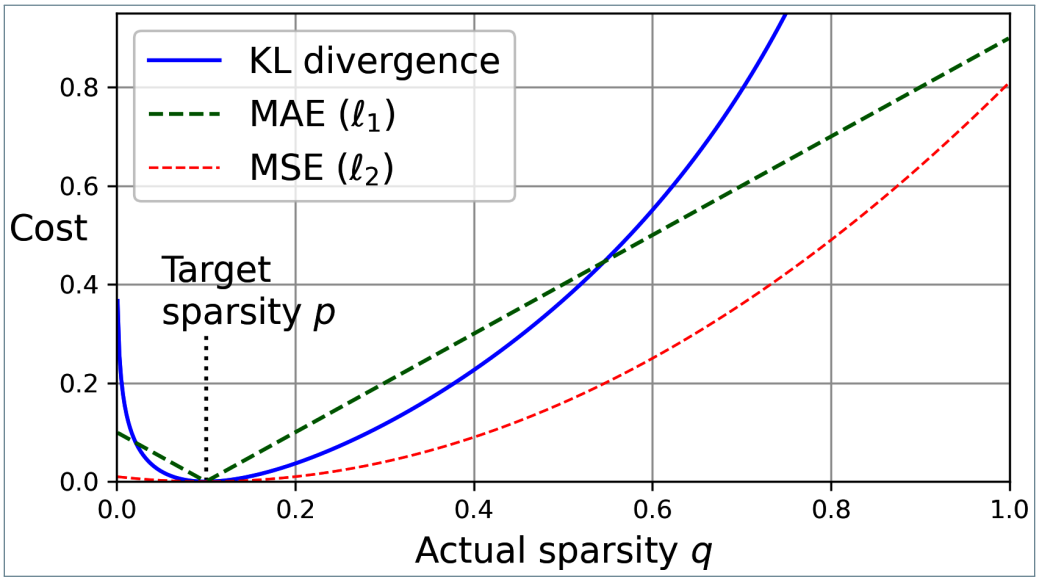

图 18-11: 目标稀疏度 p=0.1 时的稀疏损失（Sparsity loss with target sparsity p = 0.1）

对于两个离散概率分布 P 和 Q，KL 散度 $D_{\text{KL}}(P \parallel Q)$ 可由公式 18-1 计算：

$$D_{\text{KL}}(P \parallel Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)} \tag{18-1}$$

公式 18-1　Kullback-Leibler 散度（Kullback-Leibler divergence）

在本例中，我们要测量编码层某个神经元被激活的目标概率 p 与实际概率 q 之间的散度；q 通过训练批次中的平均激活值估计。因此 KL 散度简化为公式 18-2：

$$D_{\text{KL}}(p \parallel q) = p \log \frac{p}{q} + (1-p) \log \frac{1-p}{1-q} \tag{18-2}$$

公式 18-2　目标稀疏度 p 与实际稀疏度 q 之间的 KL 散度（KL divergence between the target sparsity p and the actual sparsity q）

为了在 PyTorch 中实现这种方法，自编码器的输出必须同时包含重建结果和编码，因为计算损失时二者都需要。这里让自编码器的 **`forward()`** 方法返回包含两个字段的 namedtuple：

```
from collections import namedtuple
AEOutput = namedtuple("AEOutput", ["output", "codings"])
class SparseAutoencoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Flatten(),
            nn.Linear(1 * 28 * 28, 128), nn.ReLU(),
```

```
            nn.Linear(128, 256), nn.Sigmoid())
        self.decoder = nn.Sequential(
            nn.Linear(256, 128), nn.ReLU(),
            nn.Linear(128, 1 * 28 * 28), nn.Sigmoid(),
            nn.Unflatten(dim=1, unflattened_size=(1, 28, 28)))
    def forward(self, X):
        codings = self.encoder(X)
        output = self.decoder(codings)
        return AEOutput(output, codings)
```

> **注意**　可能需要调整训练和评估函数以支持这些 namedtuple 预测。例如，在模型调用之后加入 `y_pred = y_pred.output`。

接着定义损失函数：

```
def mse_plus_sparsity_loss(y_pred, y_target, target_sparsity=0.1,
                           kl_weight=1e-3, eps=1e-8):
    p = torch.tensor(target_sparsity, device=y_pred.codings.device)
    q = torch.clamp(y_pred.codings.mean(dim=0), eps, 1 - eps)  # actual sparsity
    kl_div = p * torch.log(p / q) + (1 - p) * torch.log((1 - p) / (1 - q))
    return mse(y_pred.output, y_target) + kl_weight * kl_div.sum()
```

该函数返回重建损失（MSE）与加权稀疏损失之和。稀疏损失是目标稀疏度与批次平均稀疏度之间的 KL 散度。`kl_weight` 控制鼓励稀疏的程度；`eps` 是避免除以 0 的平滑项。

```
torch.manual_seed(42)
sparse_ae = SparseAutoencoder().to(device)
optimizer = torch.optim.NAdam(sparse_ae.parameters(), lr=0.002)
train(sparse_ae, optimizer, mse_plus_sparsity_loss, train_loader, n_epochs=10)
```

在 Fashion MNIST 上训练后，编码层大约会有 10% 的稀疏度。

> **提示**　稀疏自编码器通常会产生相当可解释的编码，每个分量对应图像中一个可识别的特征。例如，可以绘制第 n 个编码值高于平常（如高于第 90 百分位数）的所有图像，通常会发现它们具有共同点，例如都是鞋子。

18.6　变分自编码器（Variational Autoencoders）

变分自编码器（VAE）由 Diederik Kingma 和 Max Welling 于 2013 年提出，很快成为最流行的自编码器变体之一。

VAE 与前面讨论的自编码器有两点重要区别：

- 它是概率自编码器，即使训练完成，输出也部分由随机性决定；去噪自编码器只在训练期间使用随机性。
- 更重要的是，它是生成式自编码器，可以生成看起来像从训练集采样得到的新实例。

VAE 的结构仍然像一个由编码器和解码器组成的三明治，但编码器不直接输出某个输入的编码，而是输出编码的均值 μ 和标准差 σ。实际编码从均值为 μ、标准差为 σ 的高斯分布中随机采样，然后由解码器正常解码。

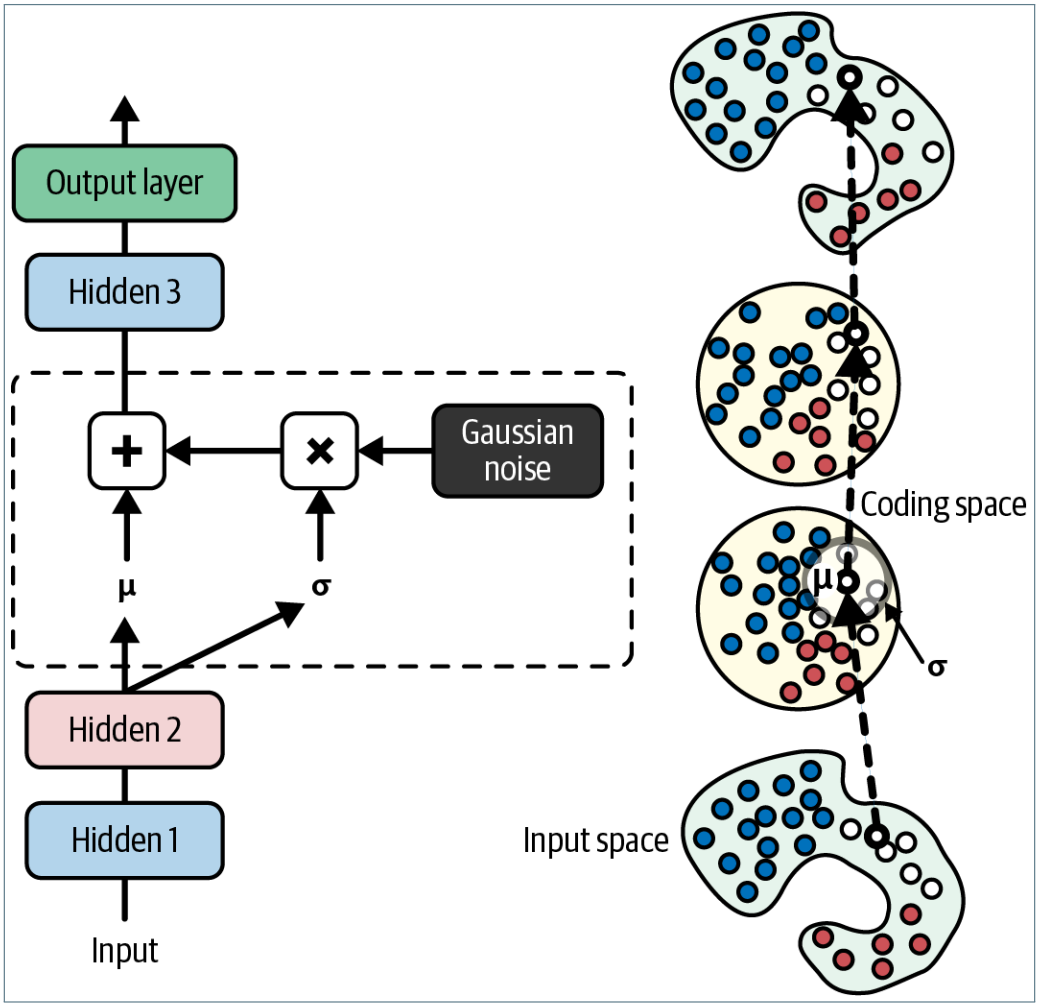

图 18-12: 变分自编码器（左）和一个经过它的实例（右）（A variational autoencoder (left) and an instance going through it (right)）

尽管输入可能具有非常复杂的分布，VAE 往往会产生看起来像从简单高斯分布采样的编码。训练过程中，代价函数会推动编码在潜在空间中逐渐迁移，最终形成一团多维高斯点。训练完成后，只需从高斯分布随机采样一个编码并解码，就可以非常容易地生成新实例。

随机采样不可微，会阻断反向传播。解决办法是重参数化技巧：从 $\mathcal{N}(0,1)$ 采样 ε，然后计算 $\mu+\sigma\otimes\varepsilon$（逐元素乘法）。这与从 $\mathcal{N}(\mu,\sigma^2)$ 采样等价，但把过程拆分为确定性和随机性两部分，使梯度可以通过 μ 和 σ 流回编码器。

代价函数由两部分组成。第一部分是普通的重建损失，可以使用 MSE；第二部分是潜在损失，它推动编码分布接近简单高斯分布，具体是实际编码分布与目标潜在分布之间的 KL 散度。潜在损失可由公式 18-3 计算：

$$\mathcal{L}=-\frac{1}{2}\sum_{i=1}^{n}\left[1+\log\left(\sigma_i^2\right)-\sigma_i^2-\mu_i^2\right] \tag{18-3}$$

公式 18-3　变分自编码器的潜在损失（Variational autoencoder's latent loss）

其中，$\mathcal{L}$ 是潜在损失，n 是编码维度，μ_i 和 σ_i 是编码第 i 个分量的均值和标准差。

一个常见改进是让编码器输出 γ=log(σ²)，而不是 σ。这样数值更稳定，也能加快训练，潜在损失可改写为公式 18-4：

$$\mathcal{L}=-\frac{1}{2}\sum_{i=1}^{n}\left[1+\gamma_i-\exp(\gamma_i)-\mu_i^2\right] \tag{18-4}$$

公式 18-4　使用 γ=log(σ²) 重写的 VAE 潜在损失（Variational autoencoder's latent loss, rewritten using γ = log(σ²)）

下面使用图 18-12 所示的架构构建 Fashion MNIST 的 VAE：

```
VAEOutput = namedtuple("VAEOutput",
                       ["output", "codings_mean", "codings_logvar"])
class VAE(nn.Module):
    def __init__(self, codings_dim=32):
        super(VAE, self).__init__()
        self.codings_dim = codings_dim
        self.encoder = nn.Sequential(
            nn.Flatten(),
            nn.Linear(1 * 28 * 28, 128), nn.ReLU(),
            nn.Linear(128, 2 * codings_dim))  # output both the mean and logvar
        self.decoder = nn.Sequential(
            nn.Linear(codings_dim, 128), nn.ReLU(),
            nn.Linear(128, 1 * 28 * 28), nn.Sigmoid(),
            nn.Unflatten(dim=1, unflattened_size=(1, 28, 28)))
    def encode(self, X):
        return self.encoder(X).chunk(2, dim=-1)  # returns (mean, logvar)
    def sample_codings(self, codings_mean, codings_logvar):
        codings_std = torch.exp(0.5 * codings_logvar)
        noise = torch.randn_like(codings_std)
        return codings_mean + noise * codings_std
    def decode(self, Z):
```

```
        return self.decoder(Z)
    def forward(self, X):
        codings_mean, codings_logvar = self.encode(X)
        codings = self.sample_codings(codings_mean, codings_logvar)
        output = self.decode(codings)
        return VAEOutput(output, codings_mean, codings_logvar)
```

`VAEOutput` 让模型同时输出重建结果、μ（`codings_mean`）和 γ（`codings_logvar`）。编码器的输出大小是编码大小的两倍，因为它输出高斯分布的均值和方差对数，而不是直接输出编码。`encode()` 调用编码器并用 `chunk()` 将输出拆成 μ 和 γ；`sample_codings()` 先计算标准差 σ，再采样标准正态噪声并执行重参数化；`decode()` 产生重建结果；`forward()` 返回计算 VAE 损失所需的全部参数。

接下来定义损失函数，即 MSE 重建损失与 KL 潜在损失之和：

```
def vae_loss(y_pred, y_target, kl_weight=1.0):
    output, mean, logvar = y_pred
    kl_div = -0.5 * torch.sum(1 + logvar - logvar.exp() - mean.square(), dim=-1)
    return F.mse_loss(output, y_target) + kl_weight * kl_div.mean() / 784
```

函数使用公式 18-4 计算批次中每个实例的潜在损失，再对所有实例求平均。重建损失已经对批次中的所有实例和 784 个像素求平均，因此潜在损失除以 784，使两种损失处于相同尺度。

```
torch.manual_seed(42)
vae = VAE().to(device)
optimizer = torch.optim.NAdam(vae.parameters(), lr=1e-3)
train(vae, optimizer, vae_loss, train_loader, n_epochs=20)
```

18.6.1　生成 Fashion MNIST 图像（Generating Fashion MNIST Images）

要使用 VAE 生成服饰图像，只需从均值为 0、方差为 1 的高斯分布中随机采样编码，然后解码：

```
torch.manual_seed(42)
vae.eval()
codings = torch.randn(3 * 7, vae.codings_dim, device=device)
with torch.no_grad():
    images = vae.decode(codings)
```

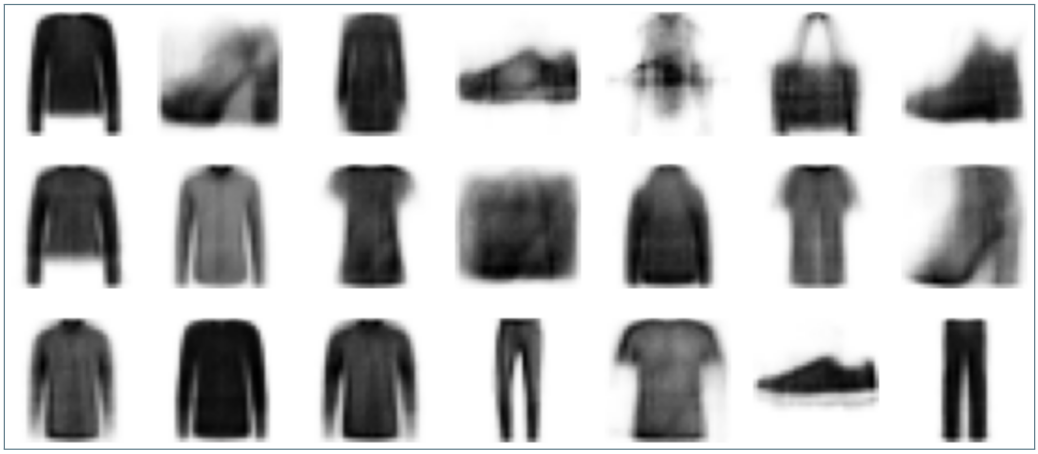

图 18-13: 变分自编码器生成的 Fashion MNIST 图像（Fashion MNIST images generated by the variational autoencoder）

多数图像看起来相当可信，只是有些模糊。其他图像效果不佳，但不要过分苛责这个自编码器：它只训练了几分钟；使用卷积层会得到好得多的结果。

VAE 还可以执行语义插值：不是在像素层面插值（那看起来像把两张图像叠在一起），而是在编码层面插值。下面从两个随机编码出发进行插值，再解码所有中间编码：

```
codings = torch.randn(2, vae.codings_dim)  # start and end codings
n_images = 7
weights = torch.linspace(0, 1, n_images).view(n_images, 1)
codings = torch.lerp(codings[0], codings[1], weights)  # linear interpolation
with torch.no_grad():
    images = vae.decode(codings.to(device))
```

图 18-14: 语义插值（Semantic interpolation）

VAE 还有许多变体，例如使用不同的潜在变量分布。一个重要变体是离散 VAE。

18.7　离散变分自编码器（Discrete Variational Autoencoders）

离散 VAE（dVAE）与 VAE 类似，但编码是离散的而不是连续的：每个编码向量包含潜在代码（latent code，也称类别），每个代码都是 0 到 k-1 之间的整数，其中 k 是潜在代码的数量。编码向量的长度通常记为 d。例如 k=10、d=6 时，共有一百万个可能的编码向量。离散 VAE 非常适合把连续输入 token 化，是 BEiT 和 DALL·E 等模型的核心组件。

将 VAE 离散化最自然的方式，是用分类分布取代高斯分布，这会带来两点变化：首先，编码器必须输出 logits，而不是均值和方差；对于每张输入图像，它输出形状为 `[d, k]` 的张量。其次，分类采样不可微，因此需要再次使用重参数化技巧。最常用的方法是 Gumbel-softmax 技巧：不直接从分类分布采样，而是调用 `F.gumbel_softmax()`，得到分类采样的可微近似。

> **注意**　Gumbel 分布可用于建模另一分布中一组样本的最大值。如果给 logits 加上 Gumbel 噪声，再取结果的 argmax，在数学上等价于分类采样。但 argmax 不可微，因此反向传播时用 softmax 替代它，就得到了可微近似。

这一想法在 2016 年几乎同时由两个独立研究团队提出：一个来自 DeepMind 和牛津大学，另一个来自 Google、剑桥大学和斯坦福大学。

下面为 Fashion MNIST 实现一个 dVAE：

```
DiscreteVAEOutput = namedtuple("DiscreteVAEOutput",
                               ["output", "logits", "codings_prob"])
class DiscreteVAE(nn.Module):
    def __init__(self, coding_length=32, n_codes=16, temperature=1.0):
        super().__init__()
        self.coding_length = coding_length
        self.n_codes = n_codes
        self.temperature = temperature
        self.encoder = nn.Sequential([...])  # outputs [coding_length, n_codes]
        self.decoder = nn.Sequential([...])  # outputs [1, 28, 28]
```

```
    def forward(self, X):
        logits = self.encoder(X)
        codings_prob = F.gumbel_softmax(logits, tau=self.temperature, hard=True)
        output = self.decoder(codings_prob)
        return DiscreteVAEOutput(output, logits, codings_prob)
```

调用 **`F.gumbel_softmax()`** 时设置 **`hard=True`**，确保前向传播使用 Gumbel-argmax，得到采样代码的 one-hot 向量，而反向传播使用 Gumbel-softmax 近似。温度越低，输出分布越接近 one-hot 向量。通常训练初期使用温度 1，然后逐渐降低到 0.1 等较小值。

损失函数也与普通 VAE 类似，是重建损失和加权潜在损失（KL 散度）之和。这里潜在分布是均匀分类分布，每个代码的概率为 1/k；由于 (log(1/k)=-log(k))，计算 KL 散度时可以加入 (log(k))：

```
def d_vae_loss(y_pred, y_target, kl_weight=1.0):
    output, logits, _ = y_pred
    codings_prob = F.softmax(logits, -1)
    k = logits.new_tensor(logits.size(-1))  # same device and dtype as logits
    kl_div = (codings_prob * (codings_prob.log() + k.log())).sum(dim=(1, 2))
    return F.mse_loss(output, y_target) + kl_weight * kl_div.mean() / 784
```

训练时逐渐降低温度，例如：

```
model.temperature = 1 - 0.9 * epoch / n_epochs
```

训练完成后，可以从均匀分布中采样随机编码，把它们转换为 one-hot，再解码：

```
codings = torch.randint(0, d_vae.n_codes,  # from 0 to k - 1
                        (n_images, d_vae.coding_length), device=device)
codings_prob = F.one_hot(codings, num_classes=d_vae.n_codes).float()
with torch.no_grad():
    images = d_vae.decoder(codings_prob)
```

离散 VAE 的另一种方法是向量量化 VAE（VQ-VAE），由 DeepMind 研究人员于 2017 年提出。编码器输出长度为 e 的 d 个嵌入向量，再把每个嵌入映射到可训练嵌入矩阵（形状为 `[k, e]`，称为码本）中最近嵌入的索引，从而得到整数代码。由于最近码本嵌入的替换不可微，反向传播时把码本查找视作恒等函数，让梯度直接通过，这种技巧称为直通估计器（straight-through estimator，STE）。

> **提示**　VQ-VAE 的正确实现可能有些棘手，可以使用 vector-quantize-pytorch 等库。它的优点是训练更稳定，代码也更容易解释。

对于小图像，离散 VAE 效果很好；但对于大图像，局部特征可能不错，整体结构却常常不一致。改进方法是先用训练好的 dVAE 编码整个训练集，使每个实例变成整数序列，再把这些代码当作 token，使用下一个 token 预测来训练 Transformer。直观上，dVAE 学习词汇表，Transformer 学习语法。训练完成后，先用 Transformer 生成代码序列，再把序列交给 dVAE 解码器，就能生成新图像。

训练 Transformer 时还可以加入图像的文本描述作为前缀，从而控制生成过程。这正是第一个 DALL·E 系统的工作方式。实践中，编码器和解码器通常是卷积网络，潜在表示组织成网格；例如编码器输出形状为 `[256, 32, 32]` 的张量，即一个 32×32 网格，每个单元包含一个 256 维嵌入。将嵌入映射到码本索引后，每张图像就表示为 32×32 的整数网格。生成时，Transformer 预测 1,024 个代码，把它们组织成网格，用码本向量替换每个代码，再交给解码器生成最终图像。

> **提示**　为了提高图像质量，还可以堆叠两个或更多 dVAE，每个 dVAE 产生比前一个更小的网格。这称为层次化 VAE（hierarchical VAE，HVAE）。编码器按顺序堆叠，解码器按相反顺序堆叠，所有模型联合训练；损失是一个重建损失加多个 KL 散度损失。

下面转向 GAN。GAN 更难训练，但一旦训练成功，就能生成令人惊叹的图像。

18.8　生成式对抗网络（Generative Adversarial Networks）

生成式对抗网络由 Ian Goodfellow 等人在 2014 年提出。GAN 的思想事后看起来很简单：让神经网络彼此竞争，希望这种竞争推动它们变得更强。如图 18-15 所示，一个 GAN 由两个神经网络组成：

生成器（Generator）　接收随机编码（通常从高斯分布中采样），输出数据，通常是一张图像。编码是待生成图像的潜在表示，因此生成器的功能类似 VAE 的解码器：输入随机向量，输出全新图像；但训练方式完全不同。

判别器（Discriminator）　接收生成器产生的伪造图像或训练集中的真实图像，并判断输入图像是真实还是伪造。

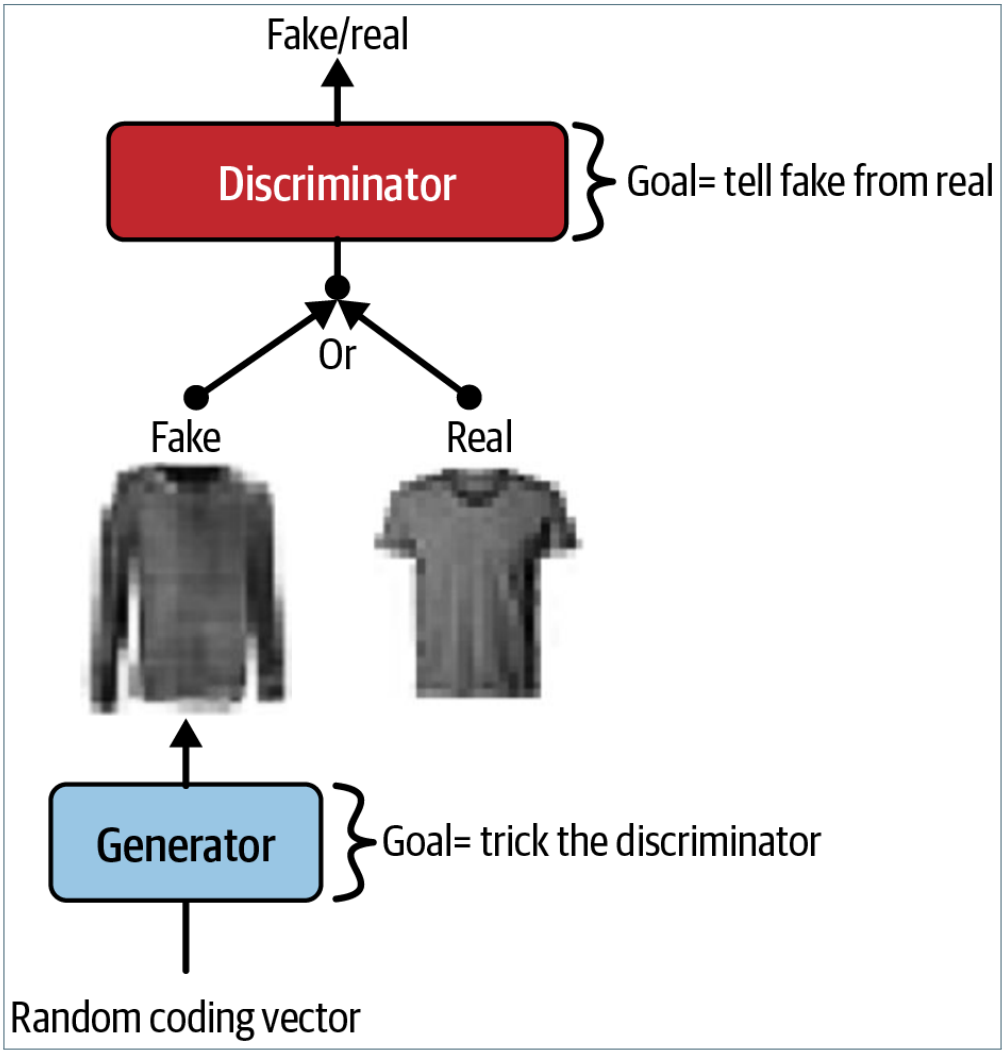

图 18-15: 生成式对抗网络（A generative adversarial network）

训练期间，生成器和判别器的目标相反：判别器试图区分伪造图像和真实图像，生成器则试图产生足够逼真的图像来欺骗判别器。由于两个网络的目标不同，GAN 不能像普通神经网络一样训练；每次训练迭代分为两个阶段。

18.8.1　第一阶段：训练判别器

从训练集采样一批真实图像，再加入生成器产生的同样数量的伪造图像。伪造图像标签设为 0，真实图像标签设为 1，用二元交叉熵损失训练判别器一步。反向传播只优化判别器的权重。

18.8.2　第二阶段：训练生成器

用生成器产生另一批伪造图像，再让判别器判断真假。这次不加入真实图像，所有标签都设为 1；也就是说，希望生成器生成能让判别器错误认为是真实的图像。此时冻结判别器权重，因此反向传播只影响生成器。

> **注意**　生成器从未直接看到真实图像，却能逐渐学会生成逼真的伪造图像；它得到的全部信息都来自通过判别器反向传播的梯度。判别器越强，这些间接梯度包含的真实图像信息就越多。

下面为 Fashion MNIST 构建一个简单 GAN。生成器类似自编码器的解码器，接收编码向量并输出图像；判别器是普通二元分类器，接收图像，最后通过带 sigmoid 激活的单神经元输出：

```
codings_dim = 32
generator = nn.Sequential(
    nn.Linear(codings_dim, 128), nn.ReLU(),
    nn.Linear(128, 256), nn.ReLU(),
    nn.Linear(256, 1 * 28 * 28), nn.Sigmoid(),
    nn.Unflatten(dim=1, unflattened_size=(1, 28, 28))).to(device)
discriminator = nn.Sequential(
    nn.Flatten(),
    nn.Linear(1 * 28 * 28, 256), nn.ReLU(),
    nn.Linear(256, 128), nn.ReLU(),
    nn.Linear(128, 1), nn.Sigmoid()).to(device)
```

由于训练循环不寻常，需要新的训练函数：

```
def train_gan(generator, discriminator, train_loader, codings_dim, n_epochs=20,
              g_lr=1e-3, d_lr=5e-4):
    criterion = nn.BCELoss()
    generator_opt = torch.optim.NAdam(generator.parameters(), lr=g_lr)
    discriminator_opt = torch.optim.NAdam(discriminator.parameters(), lr=d_lr)
    for epoch in range(n_epochs):
        for real_images, _ in train_loader:
            real_images = real_images.to(device)
            pred_real = discriminator(real_images)
            batch_size = real_images.size(0)
            ones = torch.ones(batch_size, 1, device=device)
```

```
            real_loss = criterion(pred_real, ones)
            codings = torch.randn(batch_size, codings_dim, device=device)
            fake_images = generator(codings).detach()
            pred_fake = discriminator(fake_images)
            zeros = torch.zeros(batch_size, 1, device=device)
            fake_loss = criterion(pred_fake, zeros)
            discriminator_loss = real_loss + fake_loss
            discriminator_opt.zero_grad()
            discriminator_loss.backward()
            discriminator_opt.step()
            codings = torch.randn(batch_size, codings_dim, device=device)
            fake_images = generator(codings)
            for p in discriminator.parameters():
                p.requires_grad = False
            pred_fake = discriminator(fake_images)
            generator_loss = criterion(pred_fake, ones)
            generator_opt.zero_grad()
            generator_loss.backward()
            generator_opt.step()
            for p in discriminator.parameters():
                p.requires_grad = True
```

每次迭代先让判别器执行梯度下降，再让生成器执行梯度下降。第一阶段中，真实图像的目标为 1，伪造图像的目标为 0；伪造图像调用 **`detach()`**，避免这一阶段影响生成器。第二阶段中，不对伪造图像调用 **`detach()`**，并暂时把判别器参数的 **`requires_grad`** 设为 **`False`**，使梯度只更新生成器。

训练完成后，可以从高斯分布随机采样编码并交给生成器：

```
generator.eval()
codings = torch.randn(n_images, codings_dim, device=device)
with torch.no_grad():
    generated_images = generator(codings)
```

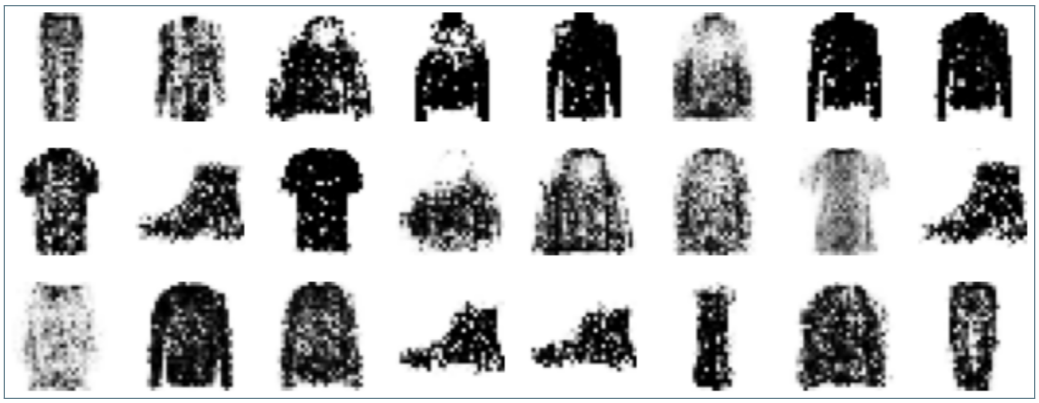

图 18-16: GAN 训练一个 epoch 后生成的图像（Images generated by the GAN after one epoch of training）

一个 epoch 后，生成图像已经开始像带有噪声的 Fashion MNIST 图像；但之后通常不会明显变好，甚至会出现 GAN 忘记已学内容的 epoch。GAN 训练确实很困难。

18.9　GAN 训练的困难（The Difficulties of Training GANs）

训练期间，生成器和判别器不断在零和博弈中相互超越。随着训练进行，博弈可能达到纳什均衡（Nash equilibrium）：在其他参与者策略不变的前提下，没有参与者能通过改变自己的策略获益。例如，如果所有人都靠左行驶，任何一个司机单独改为靠右都不会更好；靠右行驶也存在另一个均衡。不同初始状态和动力学可能把系统带到不同均衡。

对于 GAN，原论文证明它只能达到一个纳什均衡：生成器产生完全逼真的图像，判别器只能随机猜测（真实 50%，伪造 50%）。这听起来很理想，但没有任何保证训练过程一定会到达这个均衡。

最大的困难称为模式崩溃（mode collapse）：生成器输出的多样性逐渐减少。假设生成器比其他类别更擅长生成鞋子，它会更经常生成鞋子，判别器也只会看到鞋子伪造图像，逐渐忘记如何辨别其他类别。之后判别器学会区分真实鞋子与伪造鞋子，生成器又可能转向衬衫，导致双方在少数类别间循环，却始终无法把任何类别做好。

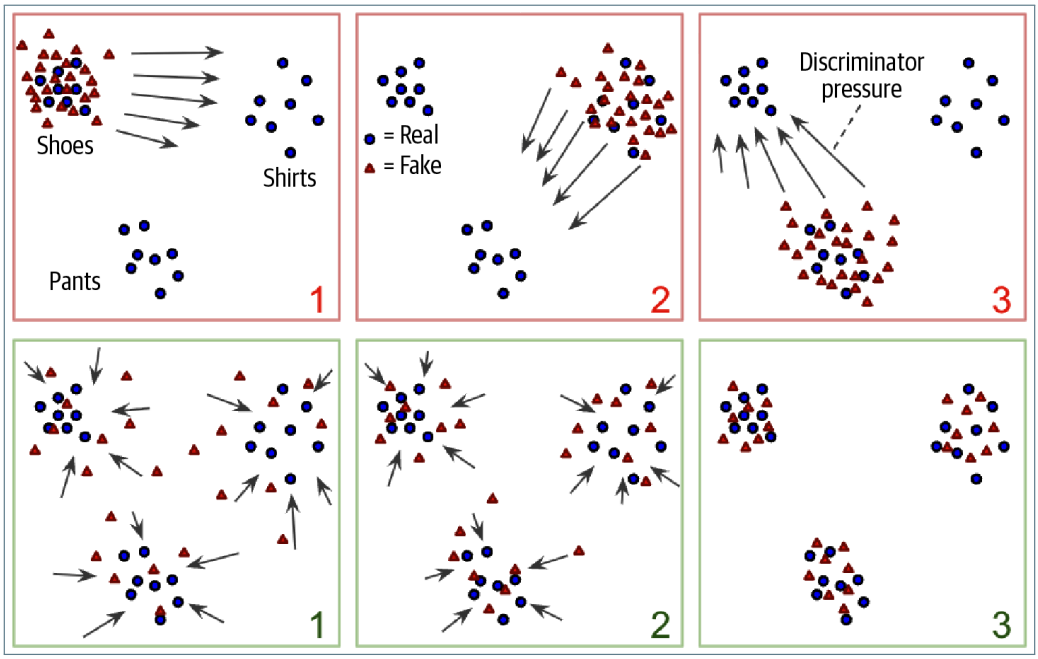

图 18-17: GAN 训练中的模式崩溃（上）与没有模式崩溃的成功训练（下）（Mode collapse while training a GAN (top row) versus successful training without mode collapse (bottom row))

此外，生成器和判别器不断相互对抗，参数可能振荡并变得不稳定。训练可能开始得很好，却因复杂动力学突然发散。GAN 对超参数非常敏感，通常需要大量调参。

研究人员提出了新的损失函数、稳定训练的技术以及避免模式崩溃的方法。例如，经验回放（experience replay）会在每次迭代保存生成器产生的图像到回放缓冲区，逐渐丢弃较旧图像，然后使用真实图像加从缓冲区抽取的伪造图像训练判别器，而不是只使用当前生成器的输出。这可以降低判别器过拟合最新输出的风险。另一个常用技术是小批量判别（mini-batch discrimination）：测量一个批次内图像的相似程度，并把这一统计量提供给判别器，使其能拒绝缺乏多样性的整批伪造图像，从而鼓励生成器产生更多样的图像。

从 DCGAN、渐进式增长 GAN 到 StyleGAN，GAN 曾是非常活跃的研究方向。但扩散模型出现后，凭借更稳定的训练过程和更多样的图像，GAN 很快失去了主流地位。下面转向扩散模型。

18.10　扩散模型（Diffusion Models）

扩散模型的思想已经存在多年，但现代形式由 Jascha Sohl-Dickstein 等人在 2015 年正式提出。他们借用统计力学工具来建模类似牛奶滴在茶中扩散的过程，并训练模型学习逆过程：从完全混合的状态开始，逐渐把牛奶从茶中“分离”出来。

2020 年，Jonathan Ho 等人构建了能够生成高度逼真图像的扩散模型，称为去噪扩散概率模型（DDPM）。几个月后，OpenAI 研究人员分析 DDPM 架构并提出改进，使 DDPM 最终超过 GAN：训练更容易，生成图像更多样、质量更高。主要缺点是生成图像非常慢。

假设从一张猫的图片 x_0 开始，在每个时间步 t 向图像添加均值为 0、方差为 β_t 的高斯噪声；每个像素独立添加相同分布的噪声。经过 $x_1, x_2, \ldots$，直到猫完全被噪声遮住。原始 DDPM 使用 T=1,000 个时间步，改进后的论文把 T 提高到 4,000，并调整了方差调度。这个过程称为前向过程（forward process）。

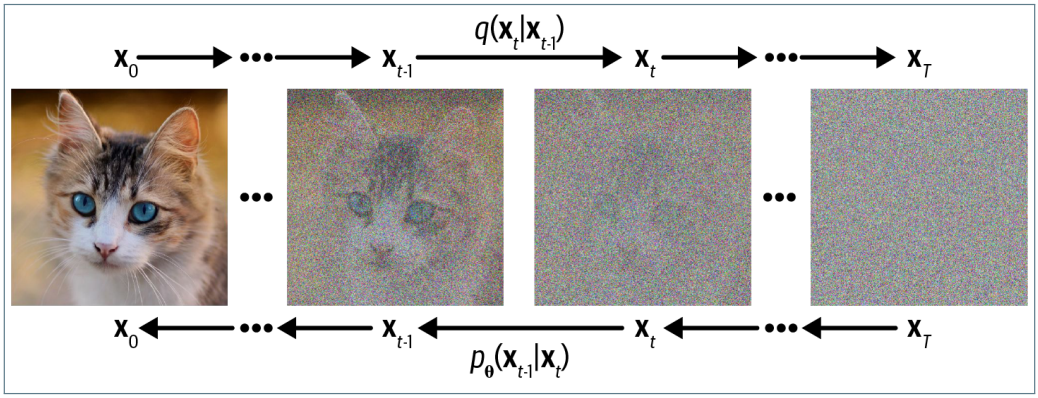

图 18-18: 前向过程 q 与逆过程 p（The forward process q and reverse process p）

每一步还会用 $\sqrt{1-\beta_t}$ 轻微缩放像素值，使均值逐渐接近 0、方差逐渐趋于 1。前向扩散过程可写为公式 18-5，它把 x_t 给定 x_{t-1} 的概率分布 q 定义为高斯分布。

$$q(x_t \mid x_{t-1}) = \mathcal{N}\left(\sqrt{1-\beta_t}\, x_{t-1},\, \beta_t I\right) \tag{18-5}$$

公式 18-5　前向扩散过程的概率分布 q（Probability distribution q of the forward diffusion process）

前向过程有一个快捷方式：不必先计算 $x_1, \ldots, x_{t-1}$，就可以直接从 x_0 采样 x_t。定义 $\alpha_t = 1 - \beta_t$，$\bar{\alpha}_t = \alpha_1 \alpha_2 \cdots \alpha_t$，则可使用公式 18-6。

$$q(x_t \mid x_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_t}\, x_0,\, (1-\bar{\alpha}_t) I\right) \tag{18-6}$$

公式 18-6　前向扩散过程的快捷方式（Shortcut for the forward diffusion process）

我们的目标不是把猫淹没在噪声中，而是生成许多新的猫。为此，训练一个能够执行逆过程的模型，从 x_t 回到 x_{t-1}。模型每次去除少量噪声，重复很多次，直到噪声全部消失。它不是只依赖相邻像素的普通滤波器；去除噪声时，模型会依据训练数据把噪声替换为逼真的像素。

18.10.1　方差调度（Variance Schedule）

在前向过程中，原始猫图像信号的方差在每一步按 $\alpha_t = 1 - \beta_t$ 缩小，经过 t 步后按 $\bar{\alpha}_t$ 缩小。我们希望 $\bar{\alpha}_t$ 在 0 到 T 之间平滑地从 1 降到 0。改进后的 DDPM 使用公式 18-7 的调度方式：

$$\beta_t = 1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}}, \qquad \bar{\alpha}_t = \frac{f(t)}{f(0)}, \qquad f(t) = \cos^2\left(\frac{t/T + s}{1+s}\frac{\pi}{2}\right) \tag{18-7}$$

公式 18-7　前向扩散过程的方差调度公式（Variance schedule equation for the forward diffusion process）

其中，s 是很小的值，用来防止 t=0 附近的 β_t 过小；论文使用 s=0.008。为避免 t=T 附近不稳定，把 β_t 截断为不超过 0.999。

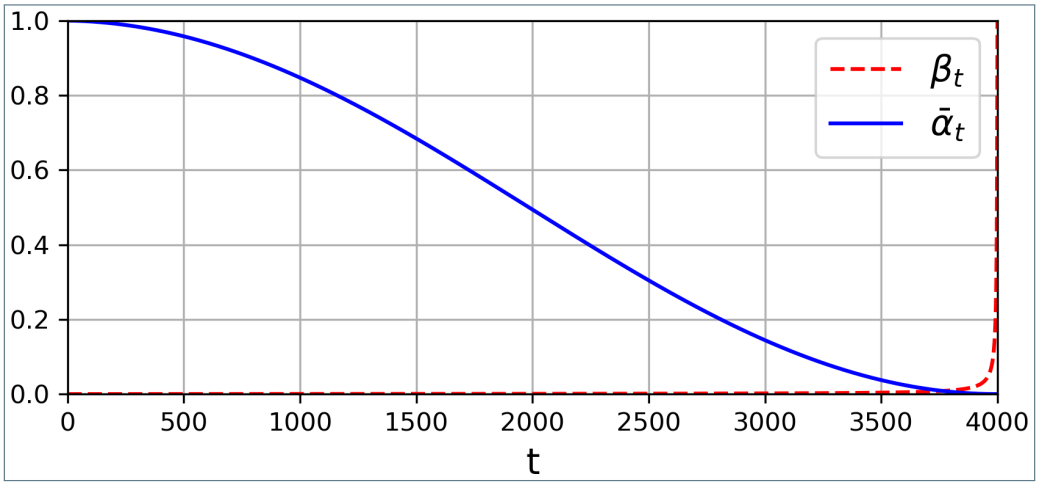

图 18-19: 噪声方差 βt 与剩余信号方差 alpha-bar_t 的调度（Noise variance schedule βt, and the remaining signal variance alpha-bar_t）

下面使用公式 18-7 计算 α_t、β_t 和 $\bar{\alpha}_t$，并令 T=4,000：

```
def variance_schedule(T, s=0.008, max_beta=0.999):
    t = torch.linspace(0, T, T + 1)
    f = torch.cos((t / T + s) / (1 + s) * torch.pi / 2) ** 2
    alpha_bars = f / f[0]
    betas = (1 - (f[1:] / f[:-1])).clamp(max=max_beta)
    betas = torch.cat([torch.zeros(1), betas])  # for easier indexing
    alphas = 1 - betas
    return alphas, betas, alpha_bars
T = 4000
alphas, betas, alpha_bars = variance_schedule(T)
```

为了训练模型逆转扩散过程，需要得到不同时间步的带噪图像。下面使用公式 18-6，接收图像 x_0 和时间步 t，返回 x_t：

```
def forward_diffusion(x0, t):
    eps = torch.randn_like(x0)  # this unscaled noise will be the target
    xt = alpha_bars[t].sqrt() * x0 + (1 - alpha_bars[t]).sqrt() * eps
    return xt, eps
```

模型同时需要带噪图像 x_t 和时间步 t，因此定义一个同时保存二者的类：

```
class DiffusionSample(namedtuple("DiffusionSampleBase", ["xt", "t"])):
    def to(self, device):
        return DiffusionSample(self.xt.to(device), self.t.to(device))
```

下面创建一个数据集包装类，把图像像素值预处理到 -1 到 +1，再使用 **forward_diffusion()** 添加噪声。它返回 **DiffusionSample** 和目标值，即缩放前的噪声 ε：

```
class DiffusionDataset:
    def __init__(self, dataset):
        self.dataset = dataset
    def __getitem__(self, i):
        x0, _ = self.dataset[i]
        x0 = (x0 * 2) - 1  # scale from –1 to +1
        t = torch.randint(1, T + 1, size=[1])
        xt, eps = forward_diffusion(x0, t)
        return DiffusionSample(xt, t), eps
    def __len__(self):
        return len(self.dataset)
train_set = DiffusionDataset(train_data)  # wrap Fashion MNIST
train_loader = DataLoader(train_set, batch_size=32, shuffle=True)
```

预测未缩放的噪声而不是直接预测原图，一方面是因为实验表明这样训练更稳定、效果更好；另一方面，噪声是高斯分布，两个高斯分布之间的 KL 散度与均值的平方距离成正比，因此可以使用简单、快速且稳定的 MSE 损失。

18.10.2　DDPM 模型

DDPM 模型可以是任何接收 **DiffusionSample** 并输出与输入图像形状相同图像的模型。DDPM 原论文使用改进的 U-Net 架构，它与第 12 章语义分割中的全卷积网络有许多相似之处。U-Net 逐渐对输入图像下采样，再逐渐上采样，并通过跳跃连接把下采样部分的每一层连接到对应的上采样层。时间步使用固定的正弦编码，每个 U-Net 层都通过 **Linear** 层接收这些编码；不同层还使用多头注意力。

```
class DiffusionModel(nn.Module):  # see the notebook for full details
    def __init__(self, T=T, embed_dim=64):
        [...]  # create all the required modules to build the U-Net
    def forward(self, sample):
        [...]  # process the sample and predict the noise for each image
```

作者发现 MAE 损失比 MSE 效果更好，也可以使用 Huber 损失：

```
diffusion_model = DiffusionModel().to(device)
huber = nn.HuberLoss()
optimizer = torch.optim.NAdam(diffusion_model.parameters(), lr=3e-3)
train(diffusion_model, optimizer, huber, train_loader, n_epochs=20)
```

18.10.3　DDPM 与 DDIM 采样

模型训练完成后，从均值为 0、方差为 1 的高斯分布中随机采样 (x_T)，使用公式 18-8 得到 (x_{T-1})，再重复 3,999 次直到得到 x_0。

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta(x_t, t)\right) + \sqrt{\beta_t}\,z \tag{18-8}$$

公式 18-8　DDPM 扩散过程中逆向执行一步（Going one step in reverse in the DDPM diffusion process）

其中，$\varepsilon_\theta(x_t, t)$ 是模型根据输入图像和时间步预测的噪声，θ 表示模型参数，z 是均值为 0、方差为 1 的高斯噪声。因此逆过程是随机的，多次运行会得到不同图像。

4,000 次迭代太慢。DDIM（去噪扩散隐式模型）可以一次跨越任意数量的时间步，并使用公式 18-9。DDIM 的训练过程与 DDPM 完全相同，因此可以直接复用训练好的 DDPM 模型。

$$\begin{aligned} x_p &= \sqrt{\bar{\alpha}_p}\,\hat{x}_0 + \sqrt{1-\bar{\alpha}_p-\sigma_t^2}\,\varepsilon_\theta(x_t, t) + \sigma_t z \\ \hat{x}_0 &= \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\varepsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}} \\ \sigma_t^2 &= \eta\frac{1-\bar{\alpha}_p}{1-\bar{\alpha}_t}\beta_t \end{aligned} \tag{18-9}$$

公式 18-9　使用 DDIM 逆向跨越多个时间步（Going multiple steps in reverse with DDIM）

其中，$\varepsilon_\theta(x_t, t)$、θ 和 z 与公式 18-8 中含义相同；p 是 t 之前的任意时间步，例如 p=t-50；η 控制生成时的随机程度，0 表示完全确定性，1 表示与 DDPM 类似。

下面实现逆过程，并生成几张图像：

```
def generate_ddim(model, batch_size=32, num_steps=50, eta=0.85):
    model.eval()
    with torch.no_grad():
        xt = torch.randn([batch_size, 1, 28, 28], device=device)
        times = torch.linspace(T - 1, 0, steps=num_steps + 1).long().tolist()
        for t, t_prev in zip(times[:-1], times[1:]):
            t_batch = torch.full((batch_size, 1), t, device=device)
            sample = DiffusionSample(xt, t_batch)
            eps_pred = model(sample)
            x0 = ((xt - (1 - alpha_bars[t]).sqrt() * eps_pred)
                  / (alpha_bars[t].sqrt()))
            abar_t_prev = alpha_bars[t_prev]
            variance = eta * (1 - abar_t_prev) / (1 - alpha_bars[t]) * betas[t]
            sigma_t = variance.sqrt()
            pred_dir = (1 - abar_t_prev - sigma_t**2).sqrt() * eps_pred
            noise = torch.randn_like(xt)
            xt = abar_t_prev.sqrt() * x0 + pred_dir + sigma_t * noise
        return torch.clamp((xt + 1) / 2, 0, 1)  # from [-1, 1] range to [0, 1]
X_gen_ddim = generate_ddim(diffusion_model, num_steps=500)
```

这次生成只需几秒，会得到如图 18-20 所示的图像。虽然示例并不惊艳，但这里只训练了几分钟；在更大数据集上训练数小时，可以得到更好的结果。

图 18-20: DDIM 加速扩散生成的图像（Images generated by DDIM accelerated diffusion）

18.11　潜在扩散模型（Latent Diffusion Models）

自 2020 年以来，扩散模型取得了巨大进展。2021 年 12 月发表的一篇论文提出了潜在扩散模型：扩散过程不在像素空间进行，而是在潜在空间进行。首先使用强大的自编码器把每张训练图像压缩到更小的潜在空间，在那里执行扩散过程，再用自编码器解压最终潜在表示，生成输出图像。这样可以显著加快图像生成，极大降低训练时间和成本，同时保持出色的图像质量。

研究人员还把各种条件控制技术应用于扩散过程，可以使用文本提示、图像或其他输入引导生成。给定输入图像后，还可以进行扩图（outpainting，扩展图像边界）和内补（inpainting，填充图像中的空洞）。

2022 年 8 月，LMU Munich 与 StabilityAI、Runway 等公司合作，在 EleutherAI 和 LAION 支持下开源了强大的预训练潜在扩散模型 Stable Diffusion。现在，任何人都可以在普通笔记本电脑上免费用几秒生成令人惊叹的图像。下面使用 Hugging Face Diffusers 库加载 Stable Diffusion Turbo，并生成一张“阅读书籍的红毛猩猩”图像：

```
from diffusers import AutoPipelineForText2Image
pipe = AutoPipelineForText2Image.from_pretrained(
    "stabilityai/sd-turbo", variant="fp16", dtype=torch.float16)
pipe.to(device)
prompt = "A closeup photo of an orangutan reading a book"
torch.manual_seed(26)
image = pipe(prompt=prompt, num_inference_steps=1, guidance_scale=0.0).images[0]
```

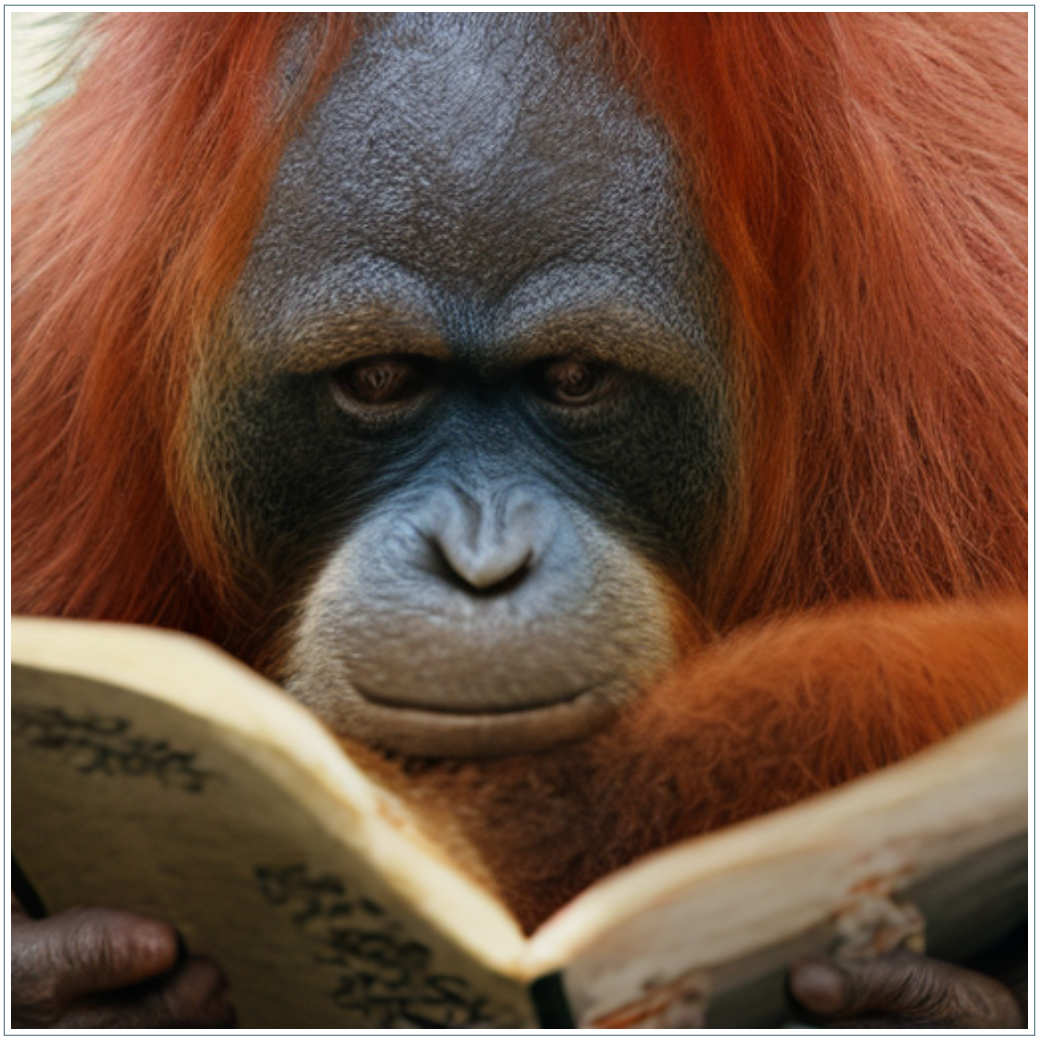

图 18-21: 使用 Diffusers 库由 Stable Diffusion 生成的图像（A picture generated by Stable Diffusion using the Diffusers library）

可能性是无穷的!

下一章将转向深度学习的另一分支：深度强化学习。

18.12　练习（Exercises）

1. 自编码器主要用于哪些任务?
2. 假设你想训练一个分类器，拥有大量无标签训练数据，但只有几千个带标签实例。自编码器如何提供帮助？你会怎样进行?

3. 如果自编码器能够完美重建输入，它是否一定是好的自编码器？如何评估自编码器的性能？
4. 什么是欠完备自编码器和过完备自编码器？过于欠完备的自编码器有什么主要风险？过完备自编码器的主要风险又是什么？
5. 如何在堆叠自编码器中绑定权重？这样做的目的是什么？
6. 什么是生成模型？你能举出一种生成式自编码器吗？
7. 什么是 GAN？请列举几个 GAN 擅长的任务。
8. 训练 GAN 时主要有哪些困难？
9. 扩散模型擅长什么？它的主要局限是什么？
10. 尝试使用去噪自编码器预训练图像分类器。可以使用 MNIST，也可以使用更复杂的 CIFAR-10。无论使用哪个数据集，都按以下步骤操作：
    - 将数据集拆分为训练集和测试集，在完整训练集上训练深度去噪自编码器。
    - 检查图像是否重建得较好；可视化最能激活编码层每个神经元的图像。
    - 构建分类 DNN，复用自编码器的低层，只使用训练集中的 500 张图像训练。比较预训练和不预训练的效果。
11. 在你选择的图像数据集上训练 VAE，并使用它生成图像。也可以选择感兴趣的无标签数据集，尝试生成新样本。
12. 在你选择的图像数据集上训练 DCGAN 并生成图像；加入经验回放，观察它是否有帮助。
13. 在你选择的图像数据集（例如 `torchvision.datasets.Flowers102`）上训练扩散模型并生成高质量图像。然后把图像类别作为额外输入加入模型并重新训练，此时应该能够控制生成图像的类别。

    练习答案位于本章配套笔记本末尾，见练习答案资源。

18.13　原书超链接资源

- This Person Does Not Exist
- 受限玻尔兹曼机资源
- VQ-VAE 的 vector-quantize-pytorch 实现
- 练习答案笔记本

18.14　参考文献说明

1. William G. Chase and Herbert A. Simon, “Perception in Chess”, *Cognitive Psychology* 4, no. 1 (1973): 55-81.
2. 训练自编码器的提示：可以创建自定义 **`AutoencoderDataset`** 类，包装给定数据集并把目标替换为输入。
3. Yoshua Bengio et al., “Greedy Layer-Wise Training of Deep Networks”, *Proceedings of NeurIPS* (2006): 153-160.
4. Jonathan Masci et al., “Stacked Convolutional Auto-Encoders for Hierarchical Feature Extraction”, *Proceedings of ICANN* 1 (2011): 52-59.
5. Pascal Vincent et al., “Extracting and Composing Robust Features with Denoising Autoencoders”, *Proceedings of ICML* (2008): 1096-1103.
6. Pascal Vincent et al., “Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion”, *Journal of Machine Learning Research* 11 (2010): 3371-3408.
7. Diederik Kingma and Max Welling, “Auto-Encoding Variational Bayes”, arXiv:1312.6114 (2013).
8. VAE 的概率性和生成性使其与 RBM 有些相似，但 VAE 更易训练，采样速度也更快。
9. Chris J. Maddison et al., “The Concrete Distribution: A Continuous Relaxation of Discrete Random Variables”, arXiv:1611.00712 (2016).
10. Eric Jang et al., “Categorical Reparameterization with Gumbel-Softmax”, arXiv:1611.01144 (2016).
11. Aaron van den Oord et al., “Neural Discrete Representation Learning”, arXiv:1711.00937 (2017).
12. Ian Goodfellow et al., “Generative Adversarial Nets”, *Proceedings of NeurIPS* 2 (2014): 2672-2680.
13. 关于 GAN 损失函数的比较，原书推荐 Hwal suk Lee 的 GitHub 项目。
14. Mario Lucic et al., “Are GANs Created Equal? A Large-Scale Study”, *Proceedings of NeurIPS* (2018): 698-707.
15. Jascha Sohl-Dickstein et al., “Deep Unsupervised Learning using Nonequilibrium Thermodynamics”, arXiv:1503.03585 (2015).
16. Jonathan Ho et al., “Denoising Diffusion Probabilistic Models”(2020).

17. Alex Nichol and Prafulla Dhariwal, “Improved Denoising Diffusion Probabilistic Models” (2021).
18. Olaf Ronneberger et al., “U-Net: Convolutional Networks for Biomedical Image Segmentation”, arXiv:1505.04597 (2015).
19. Jiaming Song et al., “Denoising Diffusion Implicit Models”, arXiv:2010.02502 (2020).
20. Robin Rombach, Andreas Blattmann, et al., “High-Resolution Image Synthesis with Latent Diffusion Models”, arXiv:2112.10752 (2021).

第 19 章　强化学习

Reinforcement Learning

强化学习（Reinforcement Learning，RL）是当今机器学习领域最令人兴奋的方向之一，也是历史最悠久的方向之一。它自 20 世纪 50 年代起就已经存在，多年来产生了许多有趣的应用，尤其是在博弈（例如西洋双陆棋程序 TD-Gammon）和机器控制方面，只是很少登上新闻头条。

然而，2013 年发生了一场革命：一家名为 DeepMind 的英国初创公司展示了一个系统，它只使用原始像素作为输入，不需要事先知道游戏规则，就能从头学会玩几乎任何 Atari 游戏，最终在其中大多数游戏中超过人类水平。此后又出现了一系列令人惊叹的成果：2016 年，DeepMind 的 AlphaGo 击败了传奇围棋职业棋手李世石；2017 年，它又击败了世界冠军柯洁。此前从未有程序接近过围棋大师，更不用说最顶尖的棋手了。

2020 年，DeepMind 发布 AlphaFold，它能够以前所未有的准确度预测蛋白质的三维结构。这项成果改变了生物学、化学和医学。事实上，DeepMind 的创始人兼首席执行官 Demis Hassabis 和研究负责人 John Jumper 因 AlphaFold 获得了诺贝尔化学奖。

2022 年，DeepMind 发布 AlphaCode，它能够达到具有竞争力的程序设计水平。2023 年，DeepMind 发布 GNoME，它能够预测新的晶体结构，其中包括数十万种预测稳定材料。

那么，DeepMind 的研究人员是如何做到这些的？他们把深度学习的力量应用到了强化学习领域，并且取得了远超预期的效果：深度强化学习由此诞生。如今，虽然 DeepMind 仍然处于领先地位，但许多其他组织也加入了进来，整个领域充满了新思想，并拥有广泛的应用范围。

本章首先解释什么是强化学习以及它适合解决什么问题，然后介绍深度强化学习中最重要的三类技术：策略梯度（policy gradients）、深度 Q 网络（deep Q-networks，包括马尔可夫决策过程），以及演员-评论家（actor-critic）方法，其中包括我们将用来击败一款 Atari 游戏的流行算法 PPO。现在开始。

19.1　什么是强化学习？

在强化学习中，软件智能体（agent）在环境（environment）中进行观测并采取动作，然后从环境中获得奖励。它的目标是学习一种行动方式，使自己随时间获得的期望奖励最大化。如果不介意稍微拟人化一点，可以把正奖励理解为愉快，把负奖励理解为痛苦（这里的“奖励”一词其实有些容易误导）。简言之，智能体在环境中行动，通过试错学习，最大化愉快并最小化痛苦。

这是一个非常宽泛的设定，可以适用于各种各样的任务。下面是几个例子（见图 19-1）：

- 智能体可以是控制机器人的程序。此时环境是真实世界，智能体通过摄像头、触觉传感器等传感器观察环境，动作则是发送信号来启动电机。可以规定：机器人接近目标位置时得到正奖励，浪费时间或走错方向时得到负奖励。
- 智能体可以是控制 Ms. Pac-Man 的程序。此时环境是 Atari 游戏的模拟器，动作是九种可能的摇杆位置（左上、下、中央等），观测是屏幕截图，奖励就是游戏得分。
- 同样，智能体也可以是玩围棋等棋盘游戏的程序。它只有获胜时才得到奖励。
- 智能体不一定要控制一个物理上或虚拟上会移动的对象。例如，它可以是一个智能恒温器：接近目标温度并节省能源时获得正奖励，住户需要手动调节温度时获得负奖励，因此智能体必须学会预测人的需求。
- 智能体可以观察股票市场价格，每秒决定买入或卖出多少。奖励显然就是货币收益和损失。

注意，任务不一定包含正奖励。例如，智能体可能在迷宫中移动，每个时间步都得到负奖励，因此它最好尽快找到出口。强化学习还适用于许多其他任务，例如自动驾驶、推荐系统、网页广告投放，以及控制图像分类系统应该把注意力放在哪里。

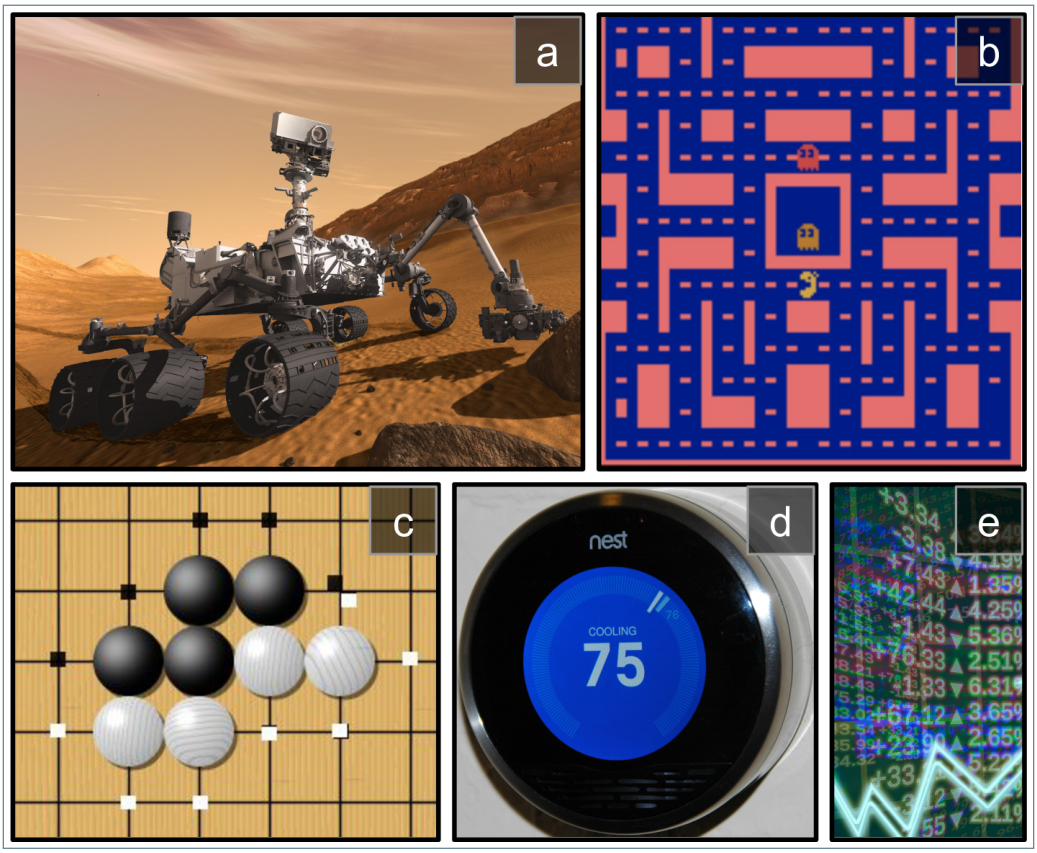

图 19-1: 强化学习示例：（a）机器人，（b）Ms. Pac-Man，（c）围棋棋手，（d）恒温器，（e）自动交易程序

下面转向 RL 算法的一大类：策略梯度。

19.2　策略梯度（Policy Gradients）

软件智能体用来决定采取什么动作的算法称为它的策略（policy）。策略可以是任何你能想到的算法，例如一个把观测作为输入并输出要执行动作的神经网络（见图 19-2）。

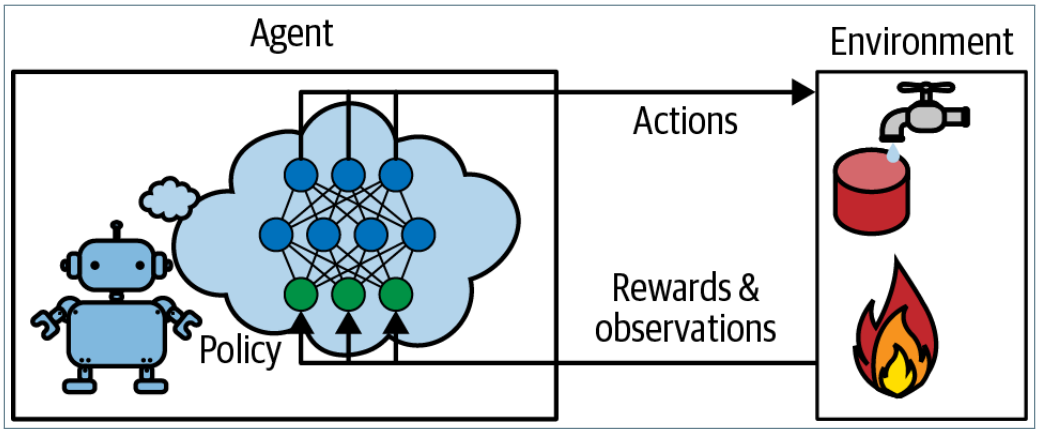

图 19-2: 使用神经网络策略进行强化学习

策略甚至不必是确定性的。事实上，在某些情况下，策略甚至不需要观察环境，只要它能获得奖励即可。想象一个看不见环境的机器人吸尘器，它的奖励是 30 分钟内吸到的灰尘量。它的策略可以是：每秒以概率 p 向前移动，或者以概率 $1-p$ 随机向左或向右转动。旋转角度是在 $-r$ 到 $+r$ 之间的随机角度。由于这个策略包含随机性，因此称为随机策略（stochastic policy）。机器人会沿着不规则的轨迹移动，最终一定能到达所有可达位置并吸走全部灰尘。问题是：它在 30 分钟内能吸到多少灰尘？

怎样训练这样的机器人？你只需要调整两个策略参数：概率 p 和角度范围 r。一种可能的学习算法是尝试许多不同的参数值，然后选择表现最好的组合（见图 19-3）。这是策略搜索（policy search）的一个例子，这里使用的是蛮力方法。当策略空间太大（通常如此）时，用这种方法寻找好的参数就像在巨大的干草堆中寻找一根针。

探索策略空间的另一种方法是使用遗传算法（genetic algorithm）。例如，可以随机创建第一代 100 个策略并测试它们，然后“淘汰”表现最差的 80 个策略，让剩下的 20 个策略各自产生 4 个后代。后代是父代的副本，再加上一些随机变化。幸存策略及其后代共同构成第二代。如此迭代，直到找到一个好的策略。

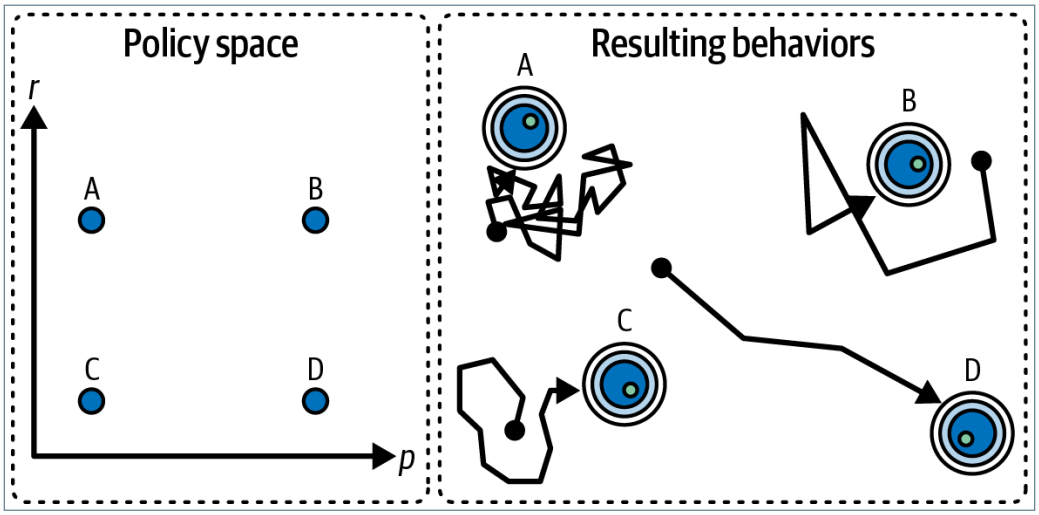

图 19-3: 策略空间中的四个点（左）及智能体对应的行为（右）

还有一种方法是使用优化技术，计算奖励相对于策略参数的梯度，然后沿着指向更高奖励的梯度调整参数。遵循这一策略的算法称为策略梯度（policy gradient，PG）算法。不过，在实现它们之前，我们首先需要为智能体创建一个可以活动的环境——下面介绍 Gymnasium 库。

19.3　Gymnasium 库简介（Introduction to the Gymnasium Library）

强化学习的挑战之一是：为了训练智能体，首先必须拥有一个可工作的环境。如果想编写一个学习玩 Atari 游戏的智能体，就需要 Atari 游戏模拟器；如果想编写一个会走路的机器人，环境就是真实世界，可以直接在其中训练机器人。

然而，真实环境有局限性：如果机器人从悬崖上掉下去，不能简单地点击“撤销”；也不能加速时间——增加计算能力不会让机器人移动得更快；同时，在现实世界中并行训练 1,000 个机器人通常也太昂贵。总之，在真实世界中训练既困难又缓慢，因此通常至少需要一个用于启动训练的模拟环境。例如，可以使用 PyBullet 或 MuJoCo 这样的库进行三维物理仿真。

Gymnasium 是一个开源工具包，提供了各种各样的模拟环境（Atari 游戏、棋盘游戏、二维和三维物理仿真等），可用于训练智能体、比较智能体或开发新的 RL 算法。它是 OpenAI Gym 的后继者，目前由研究人员和开发者社区维护。

在 Colab 上，Gymnasium 已经预安装，同时还安装了 Arcade Learning Environment（ALE）库 **`ale_py`**。它是 Atari 2600 游戏的模拟器，是所有 Atari 环境所必需的；另外还安装了若干需要二维物理的环境所依赖的 Box2D 库。如果是在自己的机器上编码，并且按照安装说明操作，那么应该已经准备就绪。

先导入 Gymnasium 并创建一个环境：

```
import gymnasium as gym
env = gym.make("CartPole-v1", render_mode="rgb_array", max_episode_steps=1000)
```

这里创建了 CartPole 环境（版本 1）。这是一个二维模拟环境：小车可以向左或向右加速，以平衡放在车上的杆（见图 19-4）—-这是一个经典的控制任务。稍后会解释 **render_mode** 和 **max_episode_steps**。

> 提示：**gym.envs.registry** 字典包含所有可用环境的名称和规格。可以使用 **gym.pprint_registry()** 打印一份易读的列表。只有启动 ALE 模拟器后，Atari 环境才会出现。

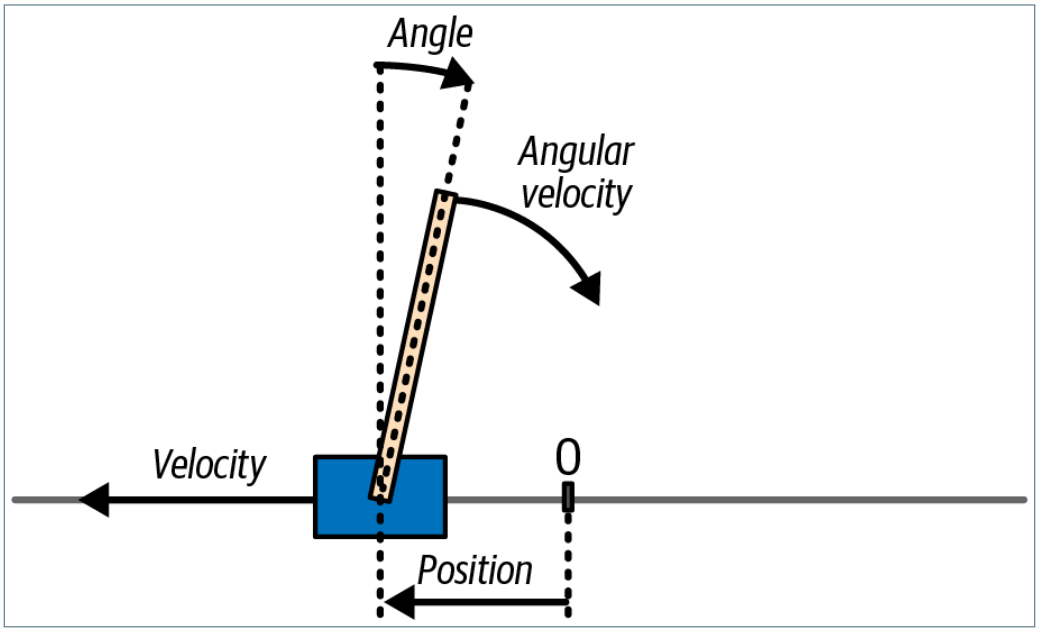

图 19-4: CartPole 环境

创建环境后，必须使用 **reset()** 方法初始化它，也可以选择指定随机种子。该方法返回第一个观测。观测取决于环境类型。对于 CartPole 环境，每个观测都是一个 NumPy 数组，包含四个浮点数，分别表示小车的水平位置（0.0 表示中心）、速度（正值表示向右）、杆的角度（0.0 表示垂直）和角速度（正值表示顺时针）。**reset()** 方法还返回一个字典，其中可能包含额外的环境特定信息，这对于调试、有时也对于训练很有用。

例如，许多 Atari 环境的这个字典包含剩余生命数。但在 CartPole 环境中，该字典为空：

```
>>> obs, info = env.reset(seed=42)
>>> obs
array([ 0.0273956 , -0.00611216,  0.03585979,  0.0197368 ], dtype=float32)
>>> info
{}
```

调用 `render()` 方法可以把环境渲染成图像。由于创建环境时设置了 `render_mode="rgb_array"`，该图像会作为 NumPy 数组返回（随后可以使用 Matplotlib 的 `imshow()` 函数显示它）：

```
>>> img = env.render()
>>> img.shape  # height, width, channels (3 = Red, Green, Blue)
(400, 600, 3)
```

下面询问环境允许哪些动作：

```
>>> env.action_space
Discrete(2)
```

`Discrete(2)` 表示可能的动作是整数 0 和 1，分别代表向左或向右加速。其他环境可能有更多离散动作，或者有其他类型的动作（例如连续动作）。由于杆向右倾斜（`obs[2] > 0`），让小车向右加速：

```
>>> action = 1  # accelerate right
>>> obs, reward, done, truncated, info = env.step(action)
>>> obs
array([ 0.02727336,  0.18847767,  0.03625453, -0.26141977], dtype=float32)
>>> reward, done, truncated, info
(1.0, False, False, {})
```

`step()` 方法执行指定动作，并返回五个值：

- `obs`：新的观测。小车正在向右移动（`obs[1] > 0`）。杆仍然向右倾斜（`obs[2] > 0`），但角速度现在为负（`obs[3] < 0`），因此下一步很可能会向左倾斜。
- `reward`：在此环境中，无论做什么，每一步都得到 1.0 的奖励，因此目标是让回合尽可能长地运行。回合（episode）是环境的一次运行，直到游戏结束或被中断。
- `done`：当回合结束时为 `True`。这会发生在杆倾斜过大或小车离开屏幕之后。此时必须重新设置环境，才能再次使用它。
- `truncated`：当回合被提前中断时为 `True`，通常是因为环境包装器设置了每回合的最大步数。CartPole 的默认环境规格把最大步数设为 500，但我们创建环境时将其改成了

1,000。某些 RL 算法会区别处理被截断的回合和正常结束的回合，但本章把它们视为相同。

- **`info`**：环境特定的信息字典，作用类似于 **`reset()`** 返回的字典。

> 提示：使用完环境（可能是在许多回合之后）后，应调用它的 **`close()`** 方法来释放资源。

先硬编码一个简单策略：当杆向左倾斜时向左加速，当杆向右倾斜时向右加速。运行该策略，查看它在 500 个回合中获得的平均奖励：

```
def basic_policy(obs):
    angle = obs[2]
    return 0 if angle < 0 else 1  # go left if leaning left, otherwise go right
totals = []
for episode in range(500):
    total_rewards = 0
    obs, info = env.reset(seed=episode)
    while True:  # no risk of infinite loop: will be truncated after 1000 steps
        action = basic_policy(obs)
        obs, reward, done, truncated, info = env.step(action)
        total_rewards += reward
        if done or truncated:
            break
    totals.append(total_rewards)
```

这段代码的含义很直观。看看结果：

```
>>> import numpy as np
>>> np.mean(totals), np.std(totals), min(totals), max(totals)
(np.float64(41.698), np.float64(8.389445512070509), 24.0, 63.0)
```

即使尝试了 500 次，这个策略也从未能让杆连续保持直立超过 63 步。效果并不好。如果观察本章笔记本中的模拟，就会发现小车左右摆动得越来越剧烈，直到杆倾斜过度。神经网络可以做得更好。

19.4　神经网络策略（Neural Network Policies）

创建一个神经网络策略。这个神经网络以一个观测为输入，并输出要执行的动作，就像前面硬编码的策略一样。更准确地说，它会为每个动作估计一个概率，然后按照这些估计出的概率随机选择动作（见图 19-5）。

在 CartPole 环境中，只有两个可能的动作（左或右），因此只需要一个输出神经元。它输出执行动作 1（向右）的概率 p，动作 0（向左）的概率自然就是 $1 - p$。例如，如果输出 0.7，就以 70% 的概率选择动作 1，以 30% 的概率选择动作 0；这就是参数为 $p = 0.7$ 的伯努利分布。

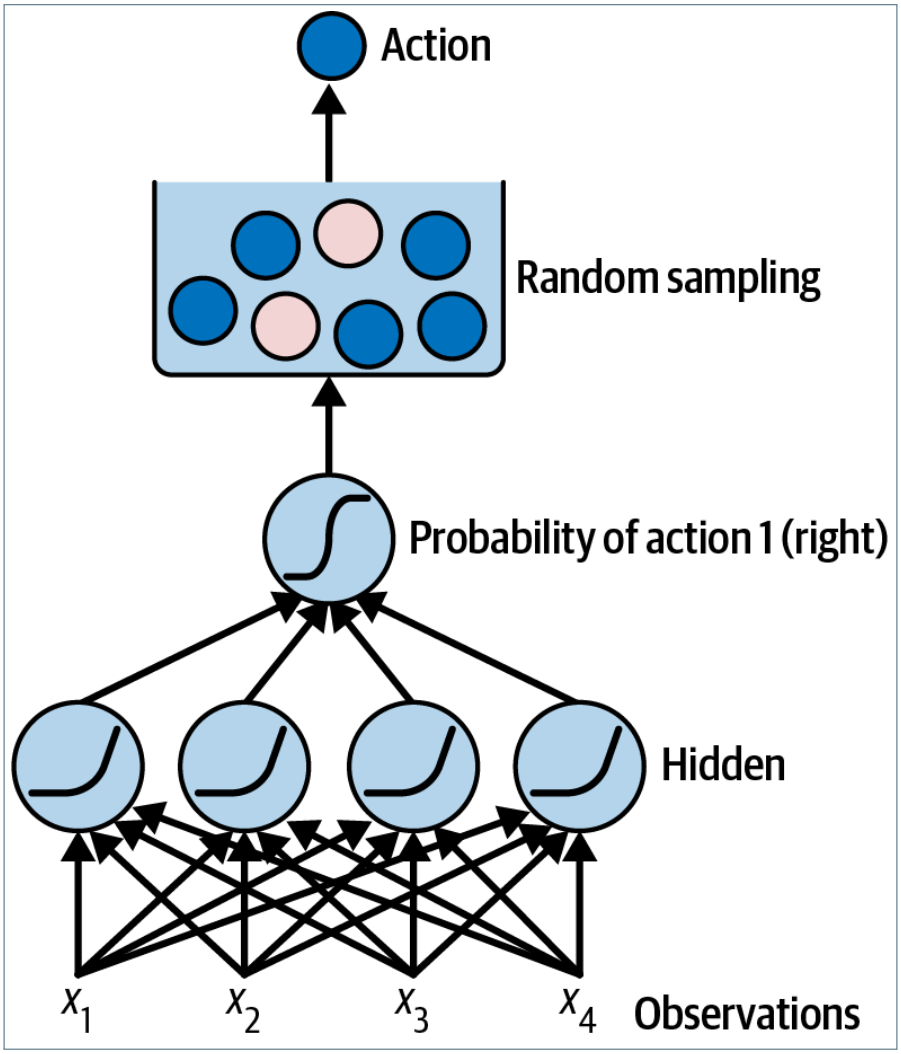

图 19-5: 神经网络策略

你可能会问，为什么根据神经网络给出的概率随机选择动作，而不是直接选择得分最高的动作？这种做法使智能体能够在探索新动作和利用已知有效动作之间找到平衡。

可以打个比方：第一次去一家餐馆时，所有菜看起来都同样诱人，于是随机点一道。如果发现它很好吃，下次可以提高点它的概率，但不应把概率提高到 100%，否则就永远不会尝试其他菜，而其中一些菜可能更好。这种探索/利用（exploration/exploitation）两难是强化学习的核心问题。

还要注意，在这个特定环境中，可以安全地忽略过去的动作和观测，因为每个观测都包含环境的完整状态。如果存在隐藏状态，则可能需要同时考虑过去的动作和观测。例如，如果环境只公开小车的位置而不公开速度，就必须同时考虑当前观测和前一个观测，才能估计当前速度。另一个例子是观测带有噪声；此时通常希望使用最近几次观测，估计最可能的当前状态。CartPole 问题已经尽可能简单：观测没有噪声，并且包含环境的完整状态。

使用 PyTorch 为 CartPole 实现一个基本的神经网络策略：

```
import torch
import torch.nn as nn

class PolicyNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(4, 5), nn.ReLU(), nn.Linear(5, 1))
    def forward(self, state):
        return self.net(state)
```

策略网络是一个很小的多层感知机（MLP），因为任务相当简单。输入数是环境状态的大小；对于 CartPole，它就是单个观测的大小，即 4。这里只有一个包含 5 个单元的隐藏层，已经不需要更多层或单元。最后，需要输出一个概率，因此只有一个输出神经元。

如果可能的动作多于两个，则每个动作使用一个输出神经元。为了获得更好的性能和数值稳定性，末尾不添加 sigmoid 函数，因此网络实际输出的是 logits，而不是概率。

定义一个使用策略网络选择动作的函数：

```
def choose_action(model, obs):
    state = torch.as_tensor(obs)
    logit = model(state)
    dist = torch.distributions.Bernoulli(logits=logit)
    action = dist.sample()
    log_prob = dist.log_prob(action)
    return int(action.item()), log_prob
```

该函数接受单个观测，将其转换成张量并传入策略网络，获得动作 1（向右）的 logit。然后用该 logit 创建伯努利概率分布，并从中采样动作：该分布以 $p = \exp(\text{logit})/(1+\exp(\text{logit}))$ 的概率输出 1，以 $1-p$ 的概率输出 0。如果动作多于两个，则应使用 **`Categorical`** 分布。最后，计算采样动作的对数概率（即 $\log(p)$ 或 $\log(1-p)$），稍后训练时会用到这个对数概率。

> 提示：如果动作空间是连续的，可以使用高斯分布，而不是伯努利分布或分类分布。此时，策略网络必须预测该分布的均值和标准差（或标准差的对数）。标准差的对数通常会被截断，以确保分布既不会太宽，也不会太窄。

现在已经有了一个能够接受环境状态并选择动作的神经网络策略。但怎样训练它呢？

19.5　评估动作：信用分配问题（Evaluating Actions: The Credit Assignment Problem）

如果知道每一步最好的动作，就可以像往常一样，通过最小化估计概率分布和目标概率分布之间的交叉熵来训练神经网络。这就变成了普通的监督学习。然而，在强化学习中，智能体得到的唯一指导是奖励，而奖励通常稀疏且延迟。

例如，如果智能体成功让杆保持平衡 100 步，怎么知道这 100 个动作中哪些是好的，哪些是坏的？它只知道杆在最后一个动作之后倒下了，但最后这个动作显然不应独自承担全部责任。这称为信用分配问题（credit assignment problem）：当智能体得到奖励或惩罚时，很难判断哪些动作应该被表扬、哪些动作应该被责备。想象一只在表现良好数小时后才得到奖励的狗，它能理解自己到底因为什么得到奖励吗？

一种简化信用分配的方法，是根据某个动作之后产生的所有奖励之和来评价该动作，并在每一步应用折扣因子 γ。这个折扣奖励之和称为该动作的回报（return）。考虑图 19-6 中的例子：智能体连续三次向右，第一步之后获得 +10，第二步之后获得 0，第三步之后获得 -50。假设折扣因子 $\gamma = 0.8$，则第一个动作的回报为 $10 + \gamma \times 0 + \gamma^2 \times (-50) = -22$。

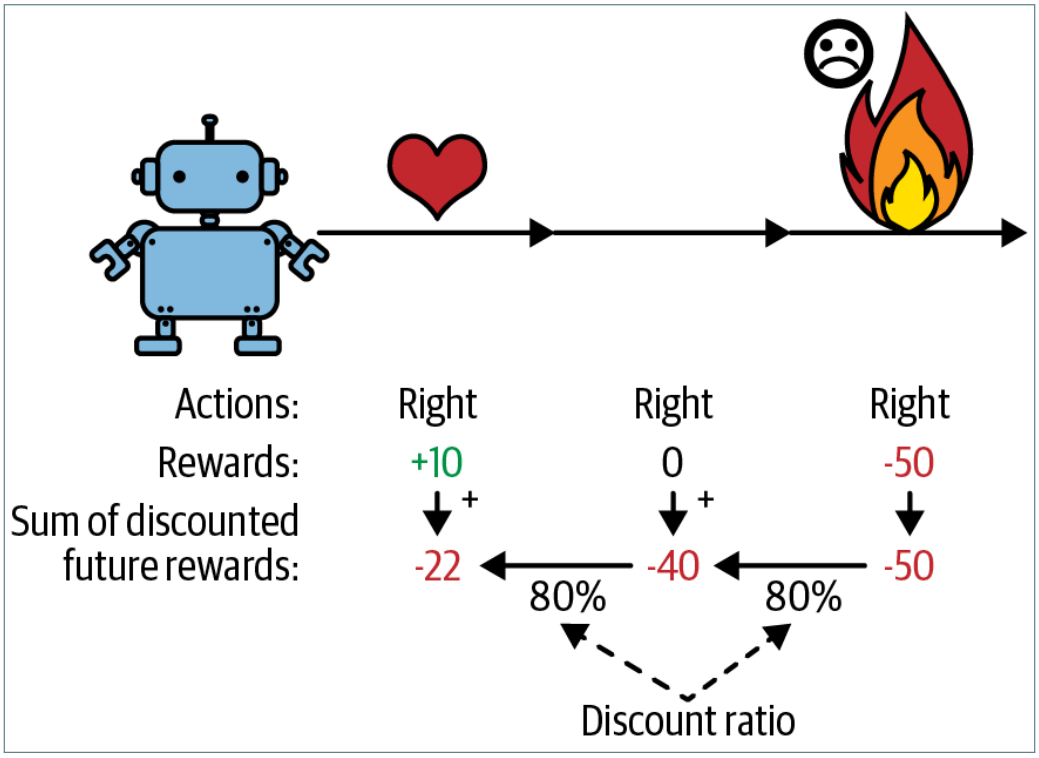

图 19-6: 计算动作回报：折扣未来奖励之和

下面的函数根据奖励和折扣因子计算回报：

```
def compute_returns(rewards, discount_factor):
    returns = rewards[:]  # copy the rewards
    for step in range(len(returns) - 1, 0, -1):
        returns[step - 1] += returns[step] * discount_factor
    return torch.tensor(returns)
```

该函数产生预期结果：

```
>>> compute_returns([10, 0, -50], discount_factor=0.8)
tensor([-22., -40., -50.])
```

如果折扣因子接近 0，未来奖励相对于即时奖励的影响就很小；如果折扣因子接近 1，遥远未来的奖励几乎和即时奖励同样重要。典型的折扣因子在 0.9 到 0.99 之间。折扣因子为 0.95 时，未来 13 步的奖励大约只相当于即时奖励的一半（因为 $0.95^{13} \approx 0.5$）；折扣因子为 0.99 时，则要到未来 69 步才减半。在 CartPole 环境中，动作的影响相对短期，因此选择较低的折扣因子 0.95 比较合理，这会帮助解决信用分配问题，使训练更快、更稳定。

不过，如果折扣因子设置得太低，智能体会学到次优策略，过度关注短期收益。

19.6　使用策略梯度解决 CartPole（Solving the CartPole Using Policy Gradients）

如前所述，策略梯度算法通过沿着奖励增加的方向移动，优化策略参数。一种流行的策略梯度算法称为 REINFORCE（或 Monte Carlo PG），由 Ronald Williams 于 1992 年提出。它有许多变体和改进，但基本原则如下：

1. 让神经网络策略玩完一个回合，记录奖励和估计出的对数概率。
2. 使用上一节定义的函数计算每个动作的回报。
3. 如果动作的回报为正，说明动作可能是好的，希望将来更可能选择它；如果回报为负，则希望降低选择它的概率。为此，可以最小化公式 19-1 所定义的 REINFORCE 损失；这样会最大化期望折扣奖励。

$$\mathcal{L}(\theta) = -\sum_{t} \log \pi_\theta(a_t \mid s_t) \cdot r_t \tag{19-1}$$

公式 19-1　REINFORCE 损失

在这个公式中，$\pi_\theta(a_t \mid s_t)$ 是策略网络在状态 s_t 下对动作 a_t 估计的概率，t 表示时间步；r_t 是该动作观测到的回报；θ 表示模型参数。

使用 PyTorch 实现该算法。首先定义一个让策略网络玩完一回合并记录奖励和对数概率的函数：

```
def run_episode(model, env, seed=None):
    log_probs, rewards = [], []
    obs, info = env.reset(seed=seed)
    while True:  # the environment will truncate the episode if it is too long
        action, log_prob = choose_action(model, obs)
        obs, reward, done, truncated, _info = env.step(action)
        log_probs.append(log_prob)
        rewards.append(reward)
        if done or truncated:
            return log_probs, rewards
```

该函数首先重置环境，开始新回合。为了保证可复现性，将种子传给 **reset()** 方法。接下来是游戏循环：每次迭代把当前环境状态（即上一次观测）传给前面定义的 **choose_action()** 方法，它返回选定动作及其对数概率。随后调用环境的 **step()** 方法执行动作，得到新的观测、奖励、表示游戏结束或被截断的两个布尔值，以及在 CartPole 中可以忽略的 **info** 字典。记录对数概率和奖励，回合结束时返回两个列表。

最后定义训练函数：

```
def train_reinforce(model, optimizer, env, n_episodes, discount_factor):
    for episode in range(n_episodes):
        seed = torch.randint(0, 2**32, size=()).item()
        log_probs, rewards = run_episode(model, env, seed=seed)
        returns = compute_returns(rewards, discount_factor)
        std_returns = (returns - returns.mean()) / (returns.std() + 1e-7)
        losses = [-logp * rt for logp, rt in zip(log_probs, std_returns)]
        loss = torch.cat(losses).sum()
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        print(f"\rEpisode {episode + 1}, Reward: {sum(rewards):.2f}", end=" ")
```

每次训练迭代都会运行一个回合，获得对数概率和奖励，然后计算每个动作的回报。接着对回报进行标准化，即减去平均回报，再除以标准差加上的一个很小的数，以避免除零。标准化是可选的，但它是 REINFORCE 中常用且推荐的改进，可以稳定训练。随后根据公式 19-1 计算损失，并执行优化器步骤以最小化损失。

准备构建和训练策略网络：

```
torch.manual_seed(42)
model = PolicyNetwork()
optimizer = torch.optim.NAdam(model.parameters(), lr=0.06)
train_reinforce(model, optimizer, env, n_episodes=200, discount_factor=0.95)
```

训练不到一分钟。如果使用该策略网络运行一个回合，会看到它能够完美平衡杆。成功。

刚才训练的简单策略梯度算法解决了 CartPole 任务，但不能很好地扩展到更大、更复杂的任务。它的样本效率很低，也就是说，必须探索很长时间，才能取得明显进展。这是因为它的回报估计非常嘈杂，尤其是在好动作和坏动作混杂时。不过，它是更强大算法（例如本章末尾讨论的演员-评论家算法）的基础。

> 提示：研究人员努力寻找这样的算法：即使智能体一开始对环境一无所知，也能工作良好。不过，除非正在写论文，否则不应犹豫向智能体注入先验知识，因为这会大幅加快训练。例如，既然知道杆应该尽可能垂直，就可以加入与杆角度成比例的负奖励。这样能减少奖励稀疏性，加快训练。如果已经有一个相当好的策略（例如硬编码策略），也可以先训练神经网络模仿它，再使用策略梯度改进。

此外，REINFORCE 非常不稳定：智能体可能在训练过程中进步一段时间，然后灾难性地忘记所有内容，再重新学习、再次忘记，反复循环。这很像坐过山车。很大一部分原因是训练样本不是独立同分布（IID）的；训练样本由智能体此刻能够到达的状态组成。随着智能体进步，它探索环境的不同部分，并可能忘记其他部分。例如，一旦学会正确地保持杆直立，它就不再看到倾斜的杆，也会彻底忘记如何处理它们。在更复杂的环境中，这个问题会严重得多。

> 注意：强化学习之所以出了名地困难，很大程度上是因为训练不稳定，而且对超参数值和随机种子的选择极其敏感。正如研究人员 Andrej Karpathy 所说："监督学习希望自己能够工作；RL 必须被迫工作。"你需要时间、耐心、坚持，也许还需要一点运气。这是 RL 没有像常规深度学习那样广泛采用的重要原因之一。

下面考察另一类流行算法：基于价值的方法。

19.7　基于价值的方法（Value-Based Methods）

策略梯度算法直接尝试优化策略，以增加奖励；基于价值的方法则更加间接：智能体学习估计每个状态的价值（即期望回报），或者估计给定状态下每个动作的价值，然后使用这些信息决定如何行动。为了理解这些算法，必须先讨论马尔可夫决策过程（Markov decision process，MDP）。

19.7.1　马尔可夫决策过程（Markov Decision Processes）

20 世纪初，数学家 Andrey Markov 研究了没有记忆的随机过程，称为马尔可夫链（Markov chain）。这类过程拥有固定数量的状态，并且在每一步随机从一个状态转移到另一个状态。从状态 s 转移到状态 s' 的概率是固定的，只取决于 (s, s') 这一对状态，而不依赖过去的状态。因此说这个系统没有记忆。

图 19-7 展示了一个包含四个状态的马尔可夫链。

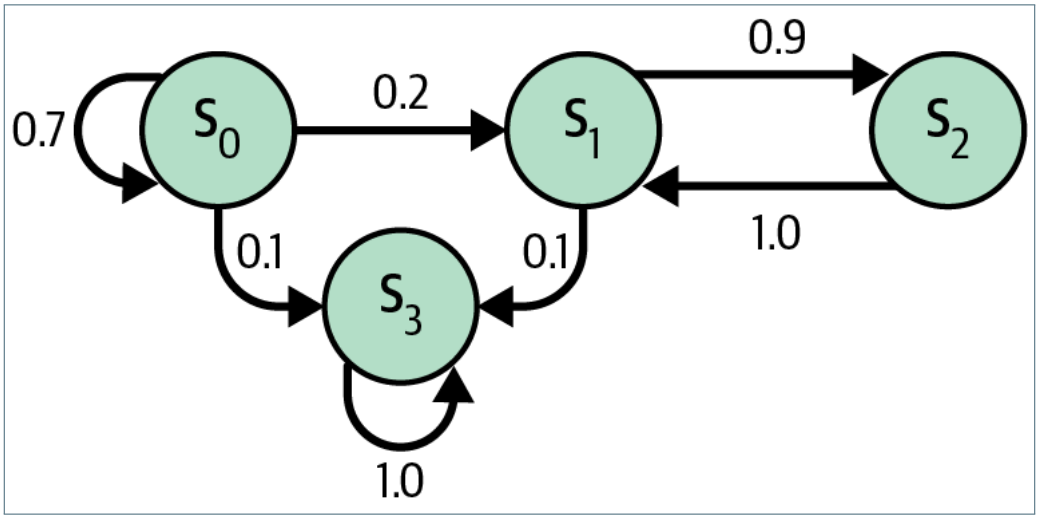

图 19-7: 马尔可夫链示例

假设过程从状态 s_0 开始，下一步仍然留在该状态的概率是 70%。由于没有其他状态指向 s_0，它最终一定会离开并且永远不会回来。如果进入状态 s_1，接下来最可能进入状态 s_2（概率 90%），然后立即回到状态 s_1（概率 100%）。它可能在这两个状态之间交替多次，但最终会进入状态 s_3 并永远停留在那里，因为没有出口；这种状态称为终止状态。

马尔可夫链可以拥有非常不同的动态，常用于热力学、化学、统计学等领域。

马尔可夫决策过程最早由 Richard Bellman 在 20 世纪 50 年代描述。它们类似于马尔可夫链，但有一个变化：每一步中，智能体可以从多个可能动作中选择一个，转移概率取决于所选动作。此外，有些状态转移会产生奖励（正的或负的），智能体的目标是寻找一个策略，使随时间积累的奖励最大化。

例如，图 19-8 所示的 MDP 有三个状态（用圆表示），每一步最多有三个可能的离散动作（用菱形表示）。

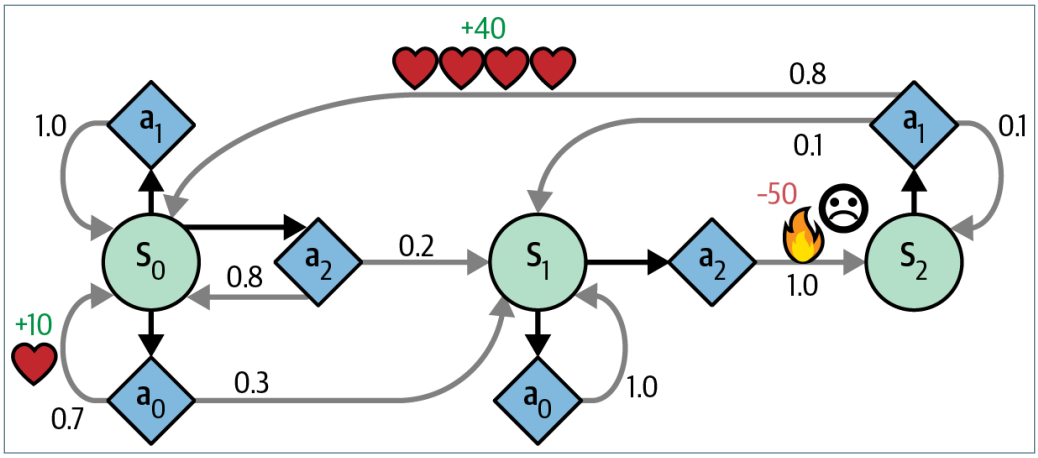

图 19-8: 马尔可夫决策过程示例

如果从状态 s_0 开始，智能体可以在动作 a_0、a_1 和 a_2 中选择。如果选择动作 a_1，就一定留在 s_0，且没有奖励，因此可以永远停留在那里。如果选择动作 a_0，则有 70% 的概率获得 +10 奖励并留在 s_0。它可以一次又一次地尝试获得奖励，但最终会进入状态 s_1。在状态 s_1 中只有两个可能动作：a_0 或 a_2。反复选择 a_0 可以留在原地，也可以选择 a_2 进入状态 s_2，并得到 -50 的负奖励。在状态 s_2 中没有选择，只能执行动作 a_1，它很可能把智能体带回 s_0，途中获得 +40 奖励。由此可以看出：在 s_0 中显然应选择 a_0，在 s_2 中只能选择 a_1；但在 s_1 中，是留在原地还是冒险穿过火焰，并不明显。

Bellman 找到了一种估计任意状态 s 的最优状态价值的方法，记作 $V^*(s)$。它是从状态 s 出发、假设始终采取最优行动时，智能体平均能够获得的折扣未来奖励之和。他证明：如果智能体始终最优行动，就满足 Bellman 最优性方程（见公式 19-2）。该递归方程表示：当前状态的最优价值等于执行一次最优动作后平均得到的奖励，加上该动作可能到达的所有下一状态的期望最优价值。

$$V^*(s) = \max_a \sum_{s'} T(s,a,s')\left[R(s,a,s') + \gamma \cdot V^*(s')\right] \quad \text{for all } s \tag{19-2}$$

公式 19-2　Bellman 最优性方程

在这个公式中：

- $T(s,a,s')$ 是智能体选择动作 a 时从状态 s 转移到状态 s' 的概率。例如，在图 19-8 中，$T(s_2,a_1,s_0) = 0.8$。注意 $\sum_{s'} T(s,a,s') = 1$。
- $R(s,a,s')$ 是智能体选择动作 a 从状态 s 转移到状态 s' 时得到的奖励。例如，$R(s_2,a_1,s_0) = +40$。
- γ 是折扣因子。

> 注意：在 Bellman 方程以及本章其余部分中，最优策略指最大化期望折扣未来奖励之和的策略，因此它取决于折扣因子 γ。不过，在真实任务中，我们通常更关心每个回合的奖励总和，而不进行折扣。为了接近这个目标，通常选择接近 1 的折扣因子，但不能太接近，否则训练会变慢且不稳定。

这个方程直接导出一种可以精确估计所有可能状态最优价值的算法：先把所有状态价值估计初始化为 0，然后使用价值迭代算法反复更新（见公式 19-3）。只要时间足够，这些估计值就一定会收敛到最优状态价值，对应于最优策略。

$$V_{k+1}(s) \leftarrow \max_{a} \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma \cdot V_k(s') \right] \quad \text{for all } s \tag{19-3}$$

公式 19-3　价值迭代算法

这里的 $V_k(s)$ 是算法第 k 次迭代时对状态 s 的估计价值。

> 注意：这个算法是动态规划的一个例子，它把复杂问题分解成可以迭代处理的可解子问题。

知道最优状态价值很有用，尤其可以用来评估策略，但它并不会直接给出智能体的最优策略。幸运的是，Bellman 找到了一个非常相似的算法来估计最优状态-动作价值，通常称为 Q 值（quality value）。状态-动作对 (s,a) 的最优 Q 值记作 $Q^*(s,a)$，它表示：从状态 s 开始、在尚未看到动作 a 的结果之前选择动作 a，并在此后采取最优行动时，智能体平均能够得到的折扣未来奖励之和。

首先把所有 Q 值估计初始化为 0，然后使用 Q 值迭代算法更新它们（见公式 19-4）。

$$Q_{k+1}(s,a) \leftarrow \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma \cdot \max_{a'} Q_k(s',a') \right] \quad \text{for all } (s,a) \tag{19-4}$$

公式 19-4　Q 值迭代算法

得到最优 Q 值后，定义最优策略 $\pi^*(s)$ 很简单：处于状态 s 时，选择该状态对应的最高 Q 值动作。用更正式的数学记号表示：

```
pi_star = argmax_a Q_star(s, a)
```

把这个算法应用到图 19-8 所示的 MDP。首先定义 MDP：

```
transition_probabilities = [  # shape=[s, a, s']
    [[0.7, 0.3, 0.0], [1.0, 0.0, 0.0], [0.8, 0.2, 0.0]],
    [[0.0, 1.0, 0.0], None, [0.0, 0.0, 1.0]],
    [None, [0.8, 0.1, 0.1], None]
]
rewards = [  # shape=[s, a, s']
    [[+10, 0, 0], [0, 0, 0], [0, 0, 0]],
    [[0, 0, 0], [0, 0, 0], [0, 0, -50]],
    [[0, 0, 0], [+40, 0, 0], [0, 0, 0]]
]
possible_actions = [[0, 1, 2], [0, 2], [1]]
```

例如，要获得执行动作 a_1 后从 s_2 转移到 s_0 的概率，可以查询 **transition_probabilities[2][1][0]**，结果为 0.8。相应奖励可以查询 **rewards[2][1][0]**，结果为 +40；状态 s_2 的可能动作列表则是 **possible_actions[2]**，其中只有动作 a_1。

接下来初始化 Q 值；对于不可能的动作，将 Q 值设置为 $-\infty$：

```
Q_values = np.full((3, 3), -np.inf)  # -np.inf for impossible actions
for state, actions in enumerate(possible_actions):
    Q_values[state, actions] = 0.0  # for all possible actions
```

现在运行 Q 值迭代算法。它会对每个状态和每个可能动作反复应用公式 19-4：

```
gamma = 0.90  # the discount factor
for iteration in range(50):
    Q_prev = Q_values.copy()
    for s in range(3):
        for a in possible_actions[s]:
            Q_values[s, a] = np.sum([
                    transition_probabilities[s][a][sp]
                    * (rewards[s][a][sp] + gamma * Q_prev[sp].max())
                for sp in range(3)])
```

结果如下：

```
>>> Q_values
array([[18.91891892, 17.02702702, 13.62162162],
       [ 0.        ,        -inf, -4.87971488],
       [       -inf, 50.13365013,        -inf]])
```

例如，智能体处于状态 s_0 并选择动作 a_1 时，期望折扣未来奖励之和约为 17.0。

对于每个状态，可以找出 Q 值最高的动作：

```
>>> Q_values.argmax(axis=1)  # optimal action for each state
array([0, 0, 1])
```

这就得到了折扣因子为 0.90 时该 MDP 的最优策略：在 s_0 中选择 a_0，在 s_1 中选择 a_0（即留在原地），在 s_2 中选择 a_1（唯一可能的动作）。有趣的是，如果把折扣因子提高到 0.95，最优策略会改变：在 s_1 中，最佳动作变成 a_2，也就是穿过火焰。这很合理，因为越看重未来奖励，就越愿意承受眼前的痛苦来换取未来的幸福。

19.7.2　时序差分学习（Temporal-Difference Learning）

具有离散动作的强化学习问题通常可以建模为马尔可夫决策过程，但智能体一开始并不知道转移概率，也不知道奖励是什么。它至少必须经历每个状态和每次转移一次，才能知道奖励；如果想合理估计转移概率，则还必须多次经历它们。

时序差分（temporal difference，TD）学习算法与 Q 值迭代算法非常相似，但考虑到智能体只掌握 MDP 的部分知识而进行了调整。通常假设智能体一开始只知道可能的状态和动作，其他一无所知。智能体使用探索策略（例如纯随机策略）探索 MDP；随着探索进行，TD 学习算法根据实际观察到的转移和奖励更新状态价值估计（见公式 19-5）。

$$
\begin{aligned}
V_{k+1}(s) &\leftarrow (1-\alpha)V_k(s) + \alpha\left(r + \gamma \cdot V_k(s')\right) \\
&\text{or, equivalently:} \\
V_{k+1}(s) &\leftarrow V_k(s) + \alpha \cdot \delta_k(s, r, s') \\
\delta_k(s, r, s') &= r + \gamma \cdot V_k(s') - V_k(s)
\end{aligned}
\tag{19-5}
$$

公式 19-5　TD 学习算法

在这个公式中：

- α 是学习率（例如 0.01）。
- $r + \gamma \cdot V_k(s')$ 称为 TD 目标。
- $\delta_k(s, r, s')$ 称为 TD 误差。

还可以使用记号 $a \leftarrow_\alpha b$ 更简洁地写出第一种形式，它表示 $a_{k+1} \leftarrow (1-\alpha)a_k + \alpha b_k$。因此，公式 19-5 的第一行也可以写成 $V(s) \leftarrow_\alpha r + \gamma \cdot V(s')$。

> 提示：TD 学习与随机梯度下降有许多相似之处，包括一次处理一个样本。此外，和 SGD 一样，只有逐渐降低学习率，它才会真正收敛；否则会一直在最优 Q 值附近来回跳动。

对于每个状态 s，这个算法都会跟踪智能体离开该状态时获得的即时奖励，以及假设随后采取最优行动时预期得到的奖励之和的运行平均值。

19.7.3　Q 学习（Q-Learning）

类似地，Q 学习算法是 Q 值迭代算法在转移概率和奖励初始未知情况下的改编（见公式 19-6）。Q 学习通过观察智能体行动（例如随机行动），逐渐改进 Q 值估计。一旦 Q 值估计足够准确，最优策略就是选择 Q 值最高的动作，即贪心策略。

$$Q(s,a) \leftarrow_{\alpha} r + \gamma \cdot \max_{a'} Q(s',a') \tag{19-6}$$

公式 19-6　Q 学习算法

对于每个状态-动作对 (s,a)，这个算法都会跟踪智能体在状态 s 执行动作 a 离开该状态时获得的奖励，以及预期得到的折扣未来奖励之和的运行平均值。为了估计未来奖励之和，需要取下一状态 s' 的 Q 值估计最大值，因为假设目标策略从那时起会采取最优行动。

实现 Q 学习。首先定义一步函数，让智能体执行一个动作并获得新的状态和奖励：

```
def step(state, action):
    probas = transition_probabilities[state][action]
    next_state = np.random.choice([0, 1, 2], p=probas)
    reward = rewards[state][action][next_state]
    return next_state, reward
```

再定义智能体的探索策略。由于状态空间很小，简单的随机策略就足够了。只要运行足够长时间，智能体就会多次访问每个状态，也会多次尝试每个可能的动作：

```
def exploration_policy(state):
    return np.random.choice(possible_actions[state])
```

像前面一样初始化 Q 值后，就可以使用学习率衰减运行 Q 学习算法：

```
alpha0 = 0.05  # initial learning rate
decay = 0.005  # learning rate decay
gamma = 0.90  # discount factor
state = 0  # initial state
for iteration in range(10_000):
    action = exploration_policy(state)
    next_state, reward = step(state, action)
    next_value = Q_values[next_state].max()  # greedy policy at the next step
    alpha = alpha0 / (1 + iteration * decay)
    Q_values[state, action] *= 1 - alpha
    Q_values[state, action] += alpha * (reward + gamma * next_value)
    state = next_state
```

该算法会收敛到最优 Q 值，但需要很多次迭代，也可能需要大量超参数调节。如图 19-9 所示，Q 值迭代算法（左）在不到 20 次迭代中就很快收敛，而 Q 学习算法（右）大约需要 8,000 次迭代才收敛。显然，不知道转移概率和奖励会让寻找最优策略变得困难得多。

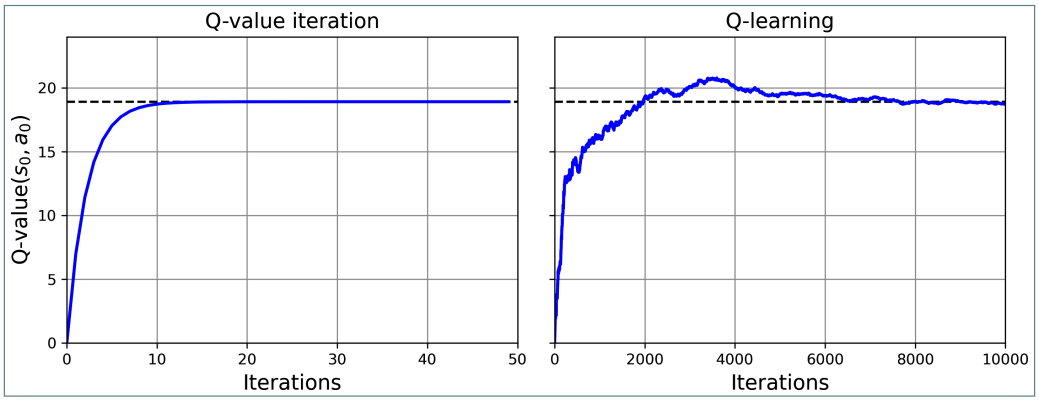

图 19-9: Q 值迭代算法与 Q 学习算法的学习曲线

Q 学习称为离策略（off-policy）算法，因为正在训练的策略不一定是训练过程中使用的策略。在刚才的代码中，实际执行的策略（探索策略）完全是随机的，而正在训练的策略从未被执行。训练后，最优策略对应于系统地选择最高 Q 值动作。相反，REINFORCE 是在策略（on-policy）算法：它使用正在训练的策略探索世界。Q 学习仅通过观察一个随机行动的智能体，就能学到最优策略，这一点相当令人惊讶。想象一下跟着一只蒙着眼睛的猴子学打高尔夫球。有没有更好的办法？

19.7.4　探索策略（Exploration Policies）

Q 学习只有在探索策略足够彻底地探索 MDP 时才能工作。纯随机策略虽然保证最终会多次访问每个状态和每次转移，但可能需要极长时间。因此，更好的选择是使用 ε-贪心策略（ε 读作 epsilon）：每一步以概率 ε 随机行动，以概率 $1-\varepsilon$ 贪心行动，即选择 Q 值最高的动作。

与完全随机策略相比，ε-贪心策略的优点是：随着 Q 值估计越来越准确，它会把越来越多的时间用于探索环境中有趣的区域，同时仍然花一些时间访问 MDP 中未知的区域。通常先使用较大的 ε（例如 1.0），再逐渐降低到 0.05。

另一种方法不是完全依靠概率进行探索，而是鼓励探索策略尝试以前较少尝试的动作。可以将奖励项加入 Q 值估计中，如公式 19-7 所示。

$$Q(s,a) \leftarrow_{\alpha} r + \gamma \cdot \max_{a'} f\left(Q(s',a'), N(s',a')\right) \tag{19-7}$$

公式 19-7　使用探索函数的 Q 学习

其中，$N(s', a')$ 记录在状态 s' 中选择动作 a' 的次数；$f(Q, N)$ 是探索函数，例如 $f(Q, N) = Q + \kappa/(1 + N)$。$\kappa$（kappa）是衡量智能体被未知事物吸引程度的好奇心超参数。

19.7.5　近似 Q 学习与深度 Q 学习（Approximate Q-Learning and Deep Q-Learning）

Q 学习的主要问题是无法很好地扩展到拥有大量状态和动作的大型（甚至中型）MDP。例如，假设要用 Q 学习训练一个玩 Ms. Pac-Man 的智能体（见图 19-1）。大约有 240 个可被吃掉的豆子，每个豆子都可能存在或不存在，因此仅豆子状态就有约 $2^{240} \approx 10^{73}$ 种。再加上所有幽灵和 Ms. Pac-Man 位置的组合，可能状态数会超过银河系中的原子数，因此绝不可能为每个 Q 值都维护一个估计。

解决方法是寻找函数 $Q_\theta(s, a)$，近似任意状态-动作对 (s, a) 的 Q 值，其中向量 θ 是函数的参数。这称为近似 Q 学习。过去多年，人们建议使用从状态中提取的手工特征的线性组合（例如最近幽灵的距离和方向）估计 Q 值；但 2013 年，DeepMind 证明深度神经网络效果好得多，尤其适用于复杂问题，而且不需要特征工程。用于估计 Q 值的 DNN 称为深度 Q 网络（deep Q-network，DQN），使用 DQN 进行近似 Q 学习称为深度 Q 学习。

怎样训练 DQN？考虑 DQN 针对某个状态-动作对 (s, a) 计算出的近似 Q 值。根据 Bellman 方程，我们希望这个近似值尽可能接近：在状态 s 执行动作 a 后实际观察到的奖励 r，加上从那以后采取最优行动时的折扣价值。

为了估计未来折扣奖励之和，只需在下一状态 s' 上运行 DQN，计算所有可能动作 a' 的值。选择最高的值（因为假设将采取最优行动），再对其折扣，就得到未来折扣奖励的估计。把奖励 r 与这个未来价值估计相加，就得到状态-动作对 (s, a) 的目标 Q 值 $y(s, a)$（见公式 19-8）。

$$y(s, a) = r + \gamma \cdot \max_{a'} Q_\theta(s', a') \tag{19-8}$$

公式 19-8　目标 Q 值

有了目标 Q 值，就可以使用任意梯度下降算法执行训练步骤。通常最小化估计 Q 值 $Q_\theta(s, a)$ 与目标 Q 值 $y(s, a)$ 之间的平方误差，或者使用 Huber 损失降低算法对大误差的敏感性。这就是深度 Q 学习算法。下面实现它来解决 CartPole 环境。

19.7.6　实现深度 Q 学习（Implementing Deep Q-Learning）

首先需要一个深度 Q 网络。理论上，需要一个以状态-动作对为输入、输出近似 Q 值的神经网络；但实践中，更高效的做法是让网络只接受状态作为输入，并为每个可能动作输出一个近似 Q 值。要解决 CartPole，不需要非常复杂的网络，两个隐藏层就足够：

```
class DQN(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(4, 32), nn.ReLU(),
                                 nn.Linear(32, 32), nn.ReLU(),
                                 nn.Linear(32, 2))
    def forward(self, state):
        return self.net(state)
```

DQN 与前面的策略网络很相似，不同之处在于：它为每个动作输出一个 Q 值，而不是输出 logits。现在定义一个根据 DQN 选择动作的函数：

```
def choose_dqn_action(model, obs, epsilon=0.0):
        if torch.rand(()) < epsilon:  # epsilon greedy policy
            return torch.randint(2, size=()).item()
        else:
            state = torch.as_tensor(obs)
            Q_values = model(state)
            return Q_values.argmax().item()  # optimal according to the DQN
```

这个函数接受环境状态（单个观测），将其传入神经网络来预测 Q 值，然后简单返回预测 Q 值最大的动作（**argmax()**）。为确保智能体探索环境，使用 ε-贪心策略，即以概率 ε 选择随机动作。

> 警告：除非能够把连续空间离散化（且离散后的空间很小），或者把 DQN 与策略梯度结合，否则 DQN 通常不能用于连续动作空间。因为 DQN 智能体必须在每一步找到 Q 值最高的动作；在连续动作空间中，这意味着每一步都要在 Q 值函数上运行一个优化算法，实践中不可行。

不再只根据最近的经验训练 DQN，而是把所有经验存储到回放缓冲区（replay buffer，也称 replay memory）中，每次训练迭代从中随机抽取一个训练批次。这样可以降低训练批次中经验之间的相关性，使数据分布更加一致，从而稳定训练。

每条经验用一个包含六个元素的元组表示：状态 s、智能体采取的动作 a、得到的奖励 r、到达的下一状态 s'、表示回合是否在此结束的布尔值 **done**，以及表示回合是否在此处被截断的另一个布尔值 **truncated**。还需要一个函数，从回放缓冲区采样随机经验批次，并返回包含六个张量的元组，每个字段对应一个张量：

```
def sample_experiences(replay_buffer, batch_size):
    indices = torch.randint(len(replay_buffer), size=[batch_size])
    batch = [replay_buffer[index] for index in indices.tolist()]
    return [to_tensor([exp[index] for exp in batch]) for index in range(6)]
def to_tensor(data):
    array = np.stack(data)
    dtype = torch.float32 if array.dtype == np.float64 else None
    return torch.as_tensor(array, dtype=dtype)
```

sample_experiences() 从回放缓冲区随机采样所需数量的经验元组。然后，对经验元组的六个字段分别进行提取，将每个字段列表使用 **to_tensor()** 转换为张量。最后返回六个张量的列表。除观测张量的形状为 **[batch_size, 4]** 外，其他张量的形状都是 **[batch_size]**。

to_tensor() 接受包含观测、动作、奖励或布尔值的 Python 列表，并返回适当 PyTorch 类型的张量。注意，包含观测的 64 位 NumPy 数组会转换为 32 位张量。

回放缓冲区可以是任何支持追加和索引的数据结构，也可以限制大小，避免训练期间内存不断增长。为简单起见，使用标准 **collections** 包中的 Python **deque**。它是一种双端队列，可以高效地在两端追加或弹出元素。当达到大小限制后，向一端追加元素会自动从另一端弹出元素，因此每条新经验都会替换最旧的经验，这正是我们想要的。

> 提示：在 **deque** 两端追加和弹出很快，但当队列很长（例如 100,000 项或更多）时，随机访问可能变慢。如果需要非常大的回放缓冲区，可以使用循环缓冲区（实现见本章笔记本），或者查看 DeepMind 的 Reverb 库。

再创建一个函数，让 DQN 玩完一个回合，并把产生的经验存储到回放缓冲区。暂时使用 **torch.no_grad()** 的评估模式，因为此时不需要梯度。为了记录日志，这个函数还会计算回合奖励总和并返回：

```
def play_and_record_episode(model, env, replay_buffer, epsilon, seed=None):
    obs, _info = env.reset(seed=seed)
    total_rewards = 0
    model.eval()
    with torch.no_grad():
        while True:
            action = choose_dqn_action(model, obs, epsilon)
            next_obs, reward, done, truncated, _info = env.step(action)
            experience = (obs, action, reward, next_obs, done, truncated)
            replay_buffer.append(experience)
            total_rewards += reward
            if done or truncated:
                return total_rewards
            obs = next_obs
```

接下来定义一个函数，从回放缓冲区采样一批经验，并在这批经验上执行一次梯度下降训练步骤：

```
def dqn_training_step(model, optimizer, criterion, replay_buffer, batch_size,
                      discount_factor):
    experiences = sample_experiences(replay_buffer, batch_size)
    state, action, reward, next_state, done, truncated = experiences
    with torch.inference_mode():
        next_Q_value = model(next_state)
    max_next_Q_value, _ = next_Q_value.max(dim=1)
    running = (~(done | truncated)).float()  # 0 if s' is over, 1 if running
    target_Q_value = reward + running * discount_factor * max_next_Q_value
    all_Q_values = model(state)
    Q_value = all_Q_values.gather(dim=1, index=action.unsqueeze(1))
    loss = criterion(Q_value, target_Q_value.unsqueeze(1))
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

> 提示：`torch.inference_mode()` 上下文类似于 `torch.no_grad()`，但还会让模型在上下文中运行于评估模式，并且新张量不能用于反向传播。

这段代码的过程如下：

- 函数首先从回放缓冲区采样一个经验批次。
- 然后使用 DQN 计算批次中每条经验的目标 Q 值。这实现了公式 19-8：让 DQN 处于推理模式，计算下一状态的所有 Q 值；由于假设智能体之后会最优行动，只保留最大 Q 值，并乘以折扣因子。如果回合已结束（`done` 或 `truncated`），则把折扣后的最大 Q 值

乘以 0，因为之后不会再有奖励；否则乘以 1。最后加上该经验的奖励。所有计算会同时作用于批次中的所有经验。

- 函数再次使用模型（这次处于训练模式）计算当前状态的所有 Q 值，并使用 **gather()** 提取实际选择动作对应的 Q 值。
- 最后计算损失（通常是目标 Q 值和预测 Q 值之间的均方误差），执行优化器步骤以最小化损失。

这是最困难的部分。现在可以写出主要训练函数并运行：

```
from collections import deque
def train_dqn(model, env, replay_buffer, optimizer, criterion, n_episodes=800,
              warmup=30, batch_size=32, discount_factor=0.95):
    totals = []
    for episode in range(n_episodes):
        epsilon = max(1 - episode / 500, 0.01)
        seed = torch.randint(0, 2**32, size=()).item()
        total_rewards = play_and_record_episode(model, env, replay_buffer,
                                                epsilon, seed=seed)
        print(f"\rEpisode: {episode + 1}, Rewards: {total_rewards}", end=" ")
        totals.append(total_rewards)
        if episode >= warmup:
            dqn_training_step(model, optimizer, criterion, replay_buffer,
                              batch_size, discount_factor)
    return totals
torch.manual_seed(42)
dqn = DQN()
optimizer = torch.optim.NAdam(dqn.parameters(), lr=0.03)
mse = nn.MSELoss()
replay_buffer = deque(maxlen=100_000)
totals = train_dqn(dqn, env, replay_buffer, optimizer, mse)
```

训练算法运行 800 个回合。每次训练迭代中，DQN 使用 **play_and_record_episode()** 玩完一个完整回合，然后使用 **dqn_training_step()** 执行一次训练。前若干个 warmup 回合只积累经验，确保回放缓冲区中有足够经验后才开始训练。ε-贪心策略的 epsilon 值从 1.0 线性下降到 500 回合后的 0.01，之后保持为 0.01。这样智能体的行为会逐渐减少随机性，把重点从探索转向利用。函数还记录每个回合的奖励总和，并返回这些总和；它们绘制在图 19-10 中。

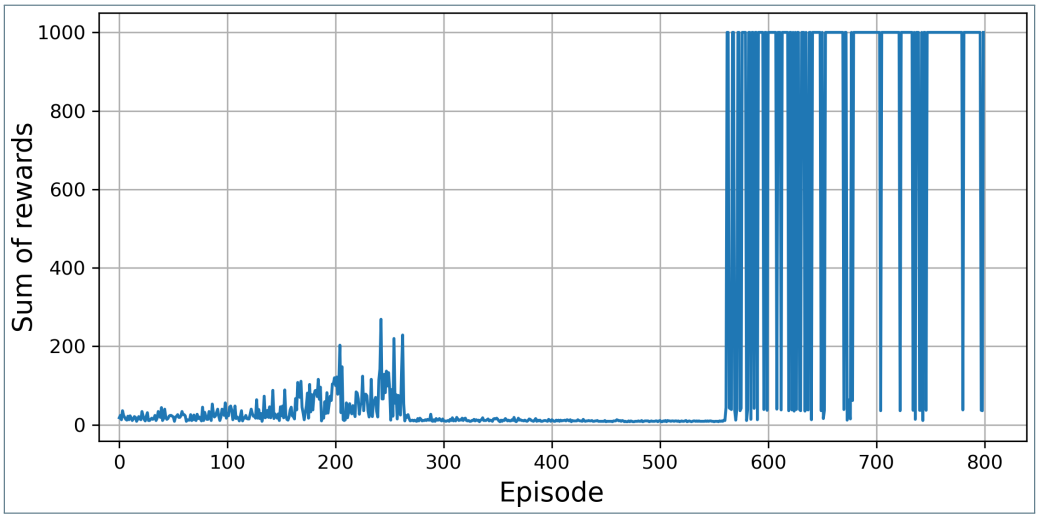

图 19-10: 深度 Q 学习算法的学习曲线

> 注意：为什么不绘制损失？因为损失不是模型性能的可靠指标，最好绘制每个回合的总奖励。损失可能下降而智能体表现变差，例如智能体困在环境的一小块区域中，DQN 开始对该区域过拟合。反过来，损失也可能上升而智能体表现变好，例如 DQN 低估了目标 Q 值，现在开始正确地提高它们。

好消息是算法确实有效：训练后的智能体能够完美地让杆在小车上保持平衡，并达到 1,000 的最大总奖励。坏消息是训练完全不稳定，甚至比 REINFORCE 更不稳定。我花了不少时间调整超参数，才偶然得到这次成功的训练运行。如图所示，智能体大约在 200 个回合后达到 200 分，这并不差，但不久之后它忘记了一切，直到大约第 550 个回合之前表现都很差，随后又迅速解决了问题。

为什么这个 DQN 实现不稳定？可能是数据分布的问题吗？回放缓冲区相当大，因此数据分布确实比 REINFORCE 稳定得多。问题在于：在这个基础深度 Q 学习实现中，模型既用于做预测，又用于设定自己的目标。这会产生类似狗追自己尾巴的情况。这个反馈回路会使网络不稳定：发散、振荡、冻结等。幸运的是，有办法改进它。

19.8　DQN 的改进（DQN Improvements）

在 2013 年的论文中，DeepMind 研究人员提出了通过使用两个 DQN 来稳定 DQN 训练的方法：第一个是在线模型，它在每一步学习，并用于控制智能体；另一个是目标模型，只用于定义目标。目标模型是在线模型的副本，其权重按固定间隔从在线模型复制过来（例如 Atari 模型中每 10,000 步复制一次）。这样 Q 值目标会稳定得多，反馈回路得到抑制，影响也小得多。

他们还结合了其他改进：非常大的回放缓冲区、很小的学习率、很长的训练时间（5,000 万步）、非常缓慢地降低 epsilon（超过 100 万步），以及强大的神经网络（CNN）。

2015 年，DeepMind 研究人员又调整了 DQN 算法，提高了性能并在一定程度上稳定了训练，称为双重 DQN（Double DQN）。这项改进基于一个观察：目标网络容易高估 Q 值。假设所有动作同样好，目标模型估计出的 Q 值理论上应该相同，但由于它们只是近似值，某些值纯粹因为随机性而略高于其他值。目标模型总是选择最大的 Q 值，这个值很可能略高于平均 Q 值，于是会高估真实 Q 值。为修正这一点，选择下一状态最佳动作时使用在线模型，只使用目标模型估计这个最佳动作的 Q 值。

另一个重要改进是优先经验回放（prioritized experience replay，PER）。与从回放缓冲区均匀采样经验不同，为什么不更频繁地采样重要经验？

更具体地说，如果某条经验很可能带来快速的学习进展，就认为它很重要。怎样估计重要性？一种合理方法是测量 TD 误差的大小 $\delta = r + \gamma \cdot V(s') - V(s)$。较大的 TD 误差说明转移 (s, a, s') 很令人意外，因此可能值得学习。当经验首次记录到回放缓冲区时，把它的优先级设为非常大的值，保证至少被采样一次。采样之后（以及每次再次采样时），计算 TD 误差 δ，并把该经验的优先级设为 $p = |\delta|$ 加上一个很小的常数，以确保每条经验都有非零的采样概率。

采样优先级为 p 的经验的概率与 p^ζ 成正比，ζ 是控制重要性采样有多“贪心”的超参数。$\zeta = 0$ 时是均匀采样，$\zeta = 1$ 时是完全的重要性采样。论文中使用 $\zeta = 0.6$，但最佳值取决于任务。

这里有一个问题：由于样本偏向重要经验，训练时必须根据重要性降低这些经验的权重，否则模型会对重要经验过拟合。也就是说，希望重要经验被更频繁采样，但也必须在训练时给它们更低权重。每条经验的训练权重定义为 $w = (nP)^{-\beta}$，其中 n 是回放缓冲区中的经验数量，β 控制对重要性采样偏差的补偿程度（0 表示完全不补偿，1 表示完全补偿）。论文中训练开始时使用 $\beta = 0.4$，最后线性增大到 $\beta = 1$。最优值取决于任务，但增大一个参数时通常也应增大另一个参数。

最后一个值得注意的 DQN 变体是对偶 DQN（dueling DQN，DDQN），不要与双重 DQN 混淆，不过两种技术可以组合。注意，状态-动作对的 Q 值可以表示为 $Q(s, a) = V(s) + A(s, a)$，其中 $V(s)$ 是状态 s 的价值，$A(s, a)$ 是在该状态采取动作 a 相对于其他动作的优势。由于状态价值等于该状态最佳动作 a^* 的 Q 值，$V(s) = Q(s, a^*)$，因此 $A(s, a^*) = 0$。

对偶 DQN 同时估计状态价值和每个可能动作的优势。由于最佳动作的优势应为 0，模型从所有预测优势中减去最大预测优势。算法其余部分与前面相同。

这些技术可以用不同方式组合。DeepMind 在 2017 年的论文中把六种不同技术组合成一个称为 Rainbow 的智能体，它在很大程度上超过了当时的最佳水平。

说到组合不同方法，为什么不把策略梯度与基于价值的方法结合起来，取两者之长？这正是演员-评论家算法的核心思想。

19.9　演员-评论家算法（Actor-Critic Algorithms）

演员-评论家算法是把策略梯度和基于价值的方法结合起来的一类 RL 算法。一个演员-评论家由策略（演员）和价值网络（评论家）组成，两者同时训练。演员依靠评论家估计动作或状态的价值，引导策略更新。由于评论家可以使用很大的回放缓冲区，它能够稳定训练并提高数据效率。这有点像运动员在教练的帮助下学习。

此外，演员-评论家方法和策略梯度一样支持随机策略和连续动作空间，因此确实兼具两类方法的优点。

先实现一个基本的演员-评论家：

```
class ActorCritic(nn.Module):
    def __init__(self):
        super().__init__()
        self.body = nn.Sequential(nn.Linear(4, 32), nn.ReLU(),
                                  nn.Linear(32, 32), nn.ReLU())
        self.actor_head = nn.Linear(32, 1)  # outputs action logits
        self.critic_head = nn.Linear(32, 1)  # outputs state values
    def forward(self, state):
        features = self.body(state)
        return self.actor_head(features), self.critic_head(features)
```

在构造函数中建立演员和评论家网络。这个实现让它们共享底层网络（称为 body）。这是常见做法，因为它减少了参数总数，从而提高数据效率，但也会使训练略微不稳定，因为演员和评论家耦合得更紧密。演员网络接受一批环境状态，并为每个状态输出一个动作 logit（这是动作 1 的 logit，与 REINFORCE 中相同）；评论家网络估计每个给定状态的价值。**`forward()`** 方法把一批状态通过共享主体送入两个网络，并返回动作 logits 和状态价值。

定义选择动作的函数。它与前面为 REINFORCE 策略网络编写的 **`choose_action()`** 相同，只是还返回评论家估计的状态价值，训练时会用到：

```
def choose_action_and_evaluate(model, obs):
    state = torch.as_tensor(obs)
    logit, state_value = model(state)
    dist = torch.distributions.Bernoulli(logits=logit)
    action = dist.sample()
    log_prob = dist.log_prob(action)
    return int(action.item()), log_prob, state_value
```

接着编写一个执行单次训练的函数：

```
def ac_training_step(optimizer, criterion, state_value, target_value, log_prob,
                     critic_weight):
    td_error = target_value - state_value
    actor_loss = -log_prob * td_error.detach()
    critic_loss = criterion(state_value, target_value)
    loss = actor_loss + critic_weight * critic_loss
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

首先计算 TD 误差，即目标价值 $y = r + \gamma V(s')$ 与状态价值 $V(s)$ 之差。演员损失与 REINFORCE 中的相同，但这里用 TD 误差乘以对数概率，而不是使用标准化回报。换句话说，鼓励那些表现好于价值网络预期的动作。评论家损失则鼓励评论家的状态价值估计 $V(s)$ 匹配目标值 y，例如使用 MSE。最后，总损失是演员损失和评论家损失的加权和。为稳定训练，通常应给评论家损失较小的权重。注意调用 **`td_error.detach()`**，因为不希望梯度下降通过演员损失影响评论家网络。

还需要一个计算目标价值的函数：

```
def get_target_value(model, next_obs, reward, done, truncated, discount_factor):
    with torch.inference_mode():
        _, _, next_state_value = choose_action_and_evaluate(model, next_obs)
    running = 0.0 if (done or truncated) else 1.0
    target_value = reward + running * discount_factor * next_state_value
    return target_value
```

这段代码首先用 **`choose_action_and_evaluate()`** 计算 $V(s')$，忽略选出的动作及其对数概率。计算目标时使用推理模式，因为不希望梯度下降影响目标。然后计算目标 $y = r + \gamma V(s')$；如果回合结束，则 $y = r$。

现在已经具备所有组件，可以编写一个运行完整回合、并在每一步训练演员-评论家的函数：

```
def run_episode_and_train(model, optimizer, criterion, env, discount_factor,
                          critic_weight, seed=None):
    obs, _info = env.reset(seed=seed)
    total_rewards = 0
    while True:
        action, log_prob, state_value = choose_action_and_evaluate(model, obs)
        next_obs, reward, done, truncated, _info = env.step(action)
        target_value = get_target_value(model, next_obs, reward, done,
                                        truncated, discount_factor)
        ac_training_step(optimizer, criterion, state_value, target_value,
                         log_prob, critic_weight)
        total_rewards += reward
        if done or truncated:
```

```
            return total_rewards
        obs = next_obs
```

最后编写主要训练函数，它只需多次调用 **run_episode_and_train()**，并返回每个回合的总奖励：

```
def train_actor_critic(model, optimizer, criterion, env, n_episodes=400,
                       discount_factor=0.95, critic_weight=0.3):
    totals = []
    model.train()
    for episode in range(n_episodes):
        seed = torch.randint(0, 2**32, size=()).item()
        total_rewards = run_episode_and_train(model, optimizer, criterion, env,
                                              discount_factor, critic_weight,
                                              seed=seed)
        totals.append(total_rewards)
        print(f"\rEpisode: {episode + 1}, Rewards: {total_rewards}", end=" ")
    return totals
```

运行它：

```
torch.manual_seed(42)
ac_model = ActorCritic()
optimizer = torch.optim.NAdam(ac_model.parameters(), lr=1.1e-3)
criterion = nn.MSELoss()
totals = train_actor_critic(ac_model, optimizer, criterion, env)
```

它确实有效：我们得到一个非常稳定的 CartPole，能够获得最大奖励。不过，这个实现仍然对超参数和随机种子的选择非常敏感，训练也仍然不稳定。幸运的是，研究人员提出了多种稳定演员-评论家的技术，其中包括：

19.9.1　异步优势演员-评论家（A3C）

这是 DeepMind 研究人员于 2016 年提出的重要演员-评论家变体。多个智能体并行学习，探索环境的不同副本。各智能体定期但异步地向主网络推送部分权重更新，然后从主网络拉取最新权重。因此，每个智能体都为改进主网络作出贡献，同时受益于其他智能体学到的内容。

此外，评论家不再估计状态价值甚至 Q 值，而是估计每个动作的优势，这就是名称中第二个 A 的含义，与对偶 DQN 类似。

19.9.2　优势演员-评论家（A2C）

A2C 是 A3C 的一个变体，取消了异步性。所有模型更新都是同步的，因此梯度更新在更大的批次上执行，模型可以更好地利用 GPU。

19.9.3　软演员-评论家（SAC）

SAC 是 Tuomas Haarnoja 和其他加州大学伯克利分校研究人员于 2018 年提出的演员-评论家变体。它不仅学习奖励，还学习如何最大化动作的熵。换句话说，它在尽可能获得更多奖励的同时，尽量让自己的行为不可预测。这会鼓励智能体探索环境，加快训练，并降低评论家估计不准确时反复执行同一动作的可能性。该算法表现出惊人的样本效率，与前面学习很慢的算法相反。

19.9.4　近端策略优化（PPO）

这个算法由 John Schulman 和其他 OpenAI 研究人员提出，建立在 A2C 之上，但会裁剪损失函数，避免权重更新过大（过大的更新通常会导致训练不稳定）。PPO 是此前信赖域策略优化（trust region policy optimization，TRPO）算法的简化版本，同样由 OpenAI 提出。

2019 年 4 月，OpenAI Five 因 PPO 算法登上新闻头条：它击败了多人游戏 Dota 2 的世界冠军。

SAC 和 PPO 是如今最常用的 RL 算法之一，许多库提供了易用且高度优化的实现。例如，下面使用流行的 Stable-Baselines3 库，训练一个玩 Atari Breakout 的 PPO 智能体。

> 提示：应该使用哪种 RL 算法？PPO 是很好的通用 RL 算法，不确定时可以优先选择它。SAC 对连续动作任务的样本效率最高，适合机器人。对于 Atari 游戏或棋盘游戏等离散任务，DQN 仍然很强。

19.10　使用 Stable-Baselines3 的 PPO 实现玩 Atari Breakout（Mastering Atari Breakout Using the Stable-Baselines3 PPO Implementation）

由于 Stable-Baselines3（SB3）默认没有安装在 Colab 中，首先运行`%pip install -q stable_baselines3`，这需要几分钟。如果在自己的机器上运行代码，并按照安装说明操作，则应该已经安装。

接下来创建 ALE 接口：它运行 Atari 2600 模拟器，并让 Gymnasium 与之交互（Atari 游戏会出现在可用环境列表中）：

```
import ale_py
ale = ale_py.ALEInterface()
```

Atari 游戏存储在只读存储器（ROM）卡带中。现在这些 ROM 可以免费下载，用于研究和教育目的。在 Colab 上它们已经预安装；如果按照安装说明在本地运行代码，也应该已经安装。

有了 SB3、ALE 接口和 ROM，就可以创建 Breakout 环境。不过不直接用 Gymnasium 创建，而是使用 SB3 的 **`make_atari_env()`** 函数。它创建一个包含多个并行运行的 Breakout 环境的包装器。包装器环境的每个观测包含一个 Breakout 环境的观测；同样，包装器的 **`step()`** 函数接受每个环境一个动作的数组。此外，包装器还负责预处理图像，把 210×160 的 RGB 图像转换为 84×84 的灰度图像。非常方便。创建包含四个并行 Breakout 环境的 SB3 环境，并重置以获得观测：

```
from stable_baselines3.common.env_util import make_atari_env
envs = make_atari_env("BreakoutNoFrameskip-v4", n_envs=4)
obs = envs.reset()  # a 4 × 84 × 84 × 1 NumPy array (note: no info dict)
```

> 提示：**`env.get_images()`** 方法会返回预处理之前的原始图像（见图 19-11）。

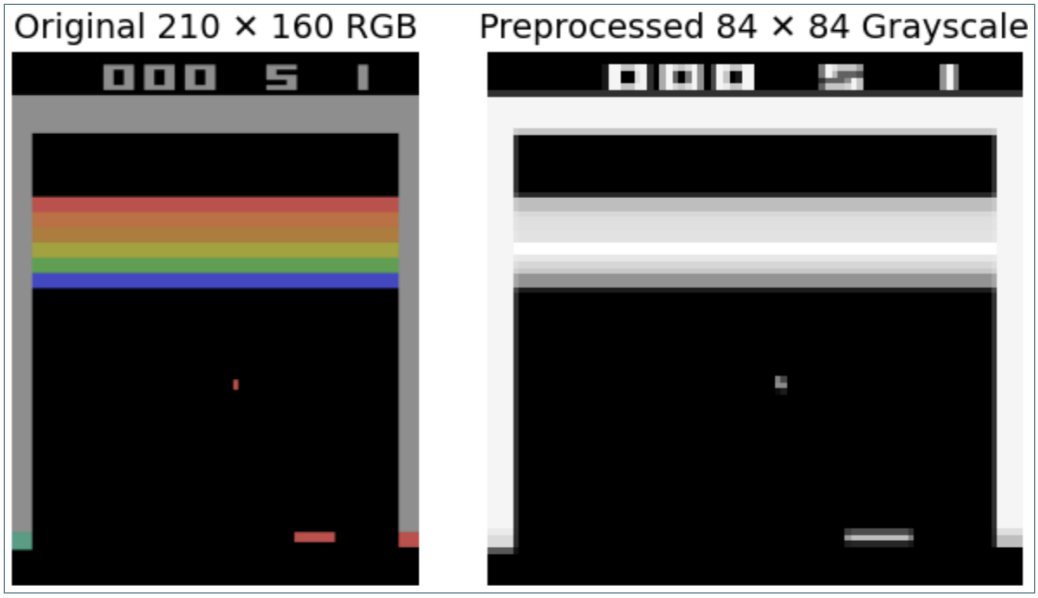

图 19-11: 预处理前（左）和预处理后（右）的 Breakout 帧

ALE 接口以每秒 60 帧运行，非常快，因此连续帧看起来很相似，会浪费计算。为避免这个问题，默认的 Breakout 环境会把每个动作重复四次，只返回最后一个观测，这称为跳帧（frame skipping）。

不过，与其跳过帧，不如把它们堆叠到一张四通道图像中，并把它作为观测。首先要避免跳帧，这就是使用 **BreakoutNoFrameskip-v4** 而不是 **Breakout-v4** 的原因。然后把环境包装在 **VecFrameStack** 中；该包装环境会重复每个动作（这里为四次），并沿通道维度（最后一个维度）堆叠生成的帧：

```
from stable_baselines3.common.vec_env import VecFrameStack
envs_stacked = VecFrameStack(envs, n_stack=4)
obs = envs_stacked.reset()  # returns a 4 × 84 × 84 × 4 NumPy array
```

现在使用一些效果良好的超参数创建 PPO 模型：

```
from stable_baselines3 import PPO
ppo_model = PPO("CnnPolicy", envs_stacked, device=device, learning_rate=2.5e-4,
                batch_size=256, n_steps=256, n_epochs=4, clip_range=0.1,
                vf_coef=0.5, ent_coef=0.01, gamma=0.99, verbose=0)
```

参数很多，下面解释它们：

- 第一个参数是策略网络。由于指定了 **CnnPolicy**，SB3 会根据所选算法（这里是 PPO）和观测空间为我们构建一个合适的 CNN。想进一步了解，可以查看 **ppo_model.policy** 的 CNN 架构：这是一个深度 CNN，具有用于动作 logits 的演员头和用于状态价值的评论家头。可能的动作有四种：向左、向右、发射（发球）和 no-op（什么也不做）。如果要使用自定义神经网络，必须创建 **ActorCriticPolicy** 类的子类，该类位于 **stable_baselines3.common.policies** 模块中。更多细节参见 SB3 文档。
- **env**、**device**、**learning_rate** 和 **batch_size** 的含义很直观。
- **n_steps** 是每次策略更新前、每个环境运行的环境步数。
- **n_epochs** 是优化期间在每个批次上运行的训练步数。
- **clip_range** 限制策略更新的幅度，避免可能导致灾难性遗忘的大变化。
- **vf_coef** 是价值函数损失在总损失中的权重，类似于演员-评论家中的 **critic_weight** 超参数。
- **ent_coef** 是熵项的权重，用于鼓励探索。
- **gamma** 是折扣率。
- **verbose** 是日志详细程度（0 表示静默，1 表示信息，2 表示调试）。

> 提示：对于新任务，PPO 的默认超参数是很好的起点。如果学习太慢，首先尝试增加并行环境数；然后考虑使用更高的学习率或更大的裁剪范围。也可以减小 **n_steps** 或 **batch_size**，但这会带来更嘈杂的梯度。如果学习不稳定，尝试降低学习率或裁剪范围，并使用更大的 rollout（即 **n_steps**）或批次。短时域任务使用接近 0.95 的 gamma，长时域任务使用 0.995 到 0.999。如果希望鼓励更多探索，则增加 **ent_coef**。

现在开始训练。下面的代码会训练模型 3,000 万步，需要很多小时，可能超过 Colab 会话时长（除非使用付费订阅）。笔记本还包含下载训练后模型的代码，可以选择直接下载。长时间训练模型时，应定期保存检查点（例如每调用 **step()** 100,000 次保存一次），避免崩溃或断电后不得不从头开始。创建一个检查点回调，并传给 **learn()** 方法：

```
from stable_baselines3.common.callbacks import CheckpointCallback
cb = CheckpointCallback(save_freq=100_000, save_path="my_ppo_breakout.ckpt")
ppo_model.learn(total_timesteps=30_000_000, progress_bar=True, callback=cb)
ppo_model.save("my_ppo_breakout")  # save the final model
```

> 注意：**save_freq** 参数统计的是 **step()** 方法的调用次数。由于有 4 个并行环境，50,000 次调用对应总共 200,000 个时间步。

要查看训练过程，可以在另一个笔记本中加载最新检查点，用 TensorBoard 可视化学习曲线，尤其是每回合平均奖励。首先在 Colab 或 Jupyter 中激活 TensorBoard 扩展：

```
%load_ext tensorboard
```

然后启动 TensorBoard 服务器，指定日志目录和 TCP 端口。下面的 magic 命令会完成这些工作，并直接在 Colab 或 Jupyter 中打开 TensorBoard 客户端：

```
tensorboard_logdir = "my_ppo_breakout_tensorboard"  # path to the log directory
%tensorboard --logdir={tensorboard_logdir} --port 6006
```

还要在创建模型时告诉 PPO 把 TensorBoard 日志保存在哪里：

```
ppo_model = PPO("CnnPolicy", [...], tensorboard_log=tensorboard_logdir)
```

训练开始后，TensorBoard 界面中的学习曲线大约每 30 秒变化一次（也可以点击刷新按钮）。最重要的指标是 **rollout/ep_rew_mean**，即每回合的平均奖励。它应该缓慢上升，但有时会下降。一百万个总步之后通常只能达到约 20，这还不是很好的智能体；训练 1,000 万步后应该达到人类水平，训练 5,000 万步后通常可以超越人类。

现在可以看看训练出的智能体。可以加载 PPO 模型并收集一个回合中的所有帧：

```
ppo_model = PPO.load("my_ppo_agent_breakout")  # or load the best checkpoint
eval_env = make_atari_env("BreakoutNoFrameskip-v4", n_envs=1, seed=42)
eval_stacked = VecFrameStack(eval_env, n_stack=4)
frames = []
obs = eval_stacked.reset()
for _ in range(5000):  # some limit in case the agent never loses
    frames.append(eval_stacked.render())
    action, _ = ppo_model.predict(obs, deterministic=True) # for reproducibility
    obs, reward, done, info = eval_stacked.step(action)
    if done[0]:  # note: there is no truncated value here
        break
eval_stacked.close()
```

这段代码会捕获一个回合中的所有帧。可以使用 Matplotlib 把它们渲染成动画（示例见笔记本）。如果训练时间足够长，或者使用预训练模型，会看到智能体玩得相当好，甚至学会了在两侧挖掘通道，把球从通道中送过去——这是这个游戏中最好的策略之一。

19.11　一些流行的 RL 算法（Overview of Some Popular RL Algorithms）

结束本章前，简要看一下其他流行算法。

19.11.1　AlphaGo

AlphaGo 使用基于深度神经网络的蒙特卡洛树搜索（Monte Carlo tree search，MCTS）变体，在围棋比赛中击败人类冠军。MCTS 由 Nicholas Metropolis 和 Stanislaw Ulam 于 1949 年发明。它通过进行大量模拟来选择最佳着法：从当前位置开始反复探索搜索树，把更多时间花在最有希望的分支上。当到达以前没有访问过的节点时，随机行棋直到游戏结束，并根据最终结果更新访问过的节点（不包括随机着法）的估计值。

AlphaGo 遵循相同原则，但使用策略网络选择着法，而不是随机行棋。这个策略网络使用策略梯度训练。最初的算法还包含三个额外神经网络，比较复杂；AlphaGo Zero 论文将其简化为使用单个神经网络，同时选择着法并评估游戏状态。AlphaZero 论文进一步推广了该算法，使其不仅能处理围棋，还能处理国际象棋和将棋。最后，MuZero 论文继续改进这一算法，虽然智能体一开始甚至不知道游戏规则，却超过了之前的版本。

> 注意：AlphaGo 的游戏规则是硬编码的。相比之下，MuZero 会逐渐学习环境模型：给定状态 s 和动作 a，它学习预测奖励 r 和到达状态 s' 的概率。拥有环境模型（无论是硬编码的还是学习得到的）能让这些算法提前规划，例如使用 MCTS。因此，这些算

法都属于基于模型的 RL 算法。相比之下，策略梯度、基于价值的方法和演员-评论家方法都是无模型 RL 算法：它们拥有策略模型和/或价值模型，但没有环境模型。

19.11.2　基于好奇心的探索（Curiosity-Based Exploration）

RL 中一个反复出现的问题是奖励稀疏，这会让学习非常缓慢且低效。Deepak Pathak 及其加州大学伯克利分校的研究人员提出了一种有趣的解决方案：为什么不忽略奖励，让智能体对探索环境极其好奇？这样奖励就变成智能体的内在奖励，而不是来自环境的奖励。类似地，激发孩子的好奇心，往往比单纯因为取得好成绩而奖励孩子更有效。

它如何工作？智能体持续尝试预测自己动作的结果，并寻找结果与预测不匹配的情形。换句话说，它想感到惊讶。如果结果可预测（无聊），它就去别处；但如果结果不可预测、同时智能体发现自己无法控制结果，它也会在一段时间后感到无聊。仅使用好奇心，作者就成功训练了许多电子游戏智能体：即使输掉游戏没有惩罚，智能体也会觉得输掉很无聊，因为游戏会重新开始，于是它学会避免失败。

19.11.3　开放式学习（Open-Ended Learning，OEL）

OEL 的目标是训练能够无休止地学习新的、有趣任务的智能体，这些任务通常以程序方式生成。我们还没有完全实现这个目标，但过去几年已经取得了惊人进展。

例如，Uber AI 的研究团队在 2019 年发表的论文介绍了 POET 算法。它生成带有凸起和坑洞的多个二维模拟环境，并为每个环境训练一个智能体。智能体的目标是在避开障碍物的同时尽可能快地行走。

算法从简单环境开始，随着时间推移逐渐变难，这称为课程学习（curriculum learning）。此外，虽然每个智能体只在一个环境中训练，但它必须定期与所有环境中的其他智能体比赛。在每个环境中，获胜者会被复制过去，替换原来的智能体。这样，知识会定期在环境之间转移，并筛选出适应能力最强的智能体。

最终，这些智能体比只在单个任务上训练的智能体行走得好得多，也能处理困难得多的环境。这个原则当然也可以应用到其他环境和任务。如果想深入了解 OEL，可以查看 Enhanced POET 论文，以及 DeepMind 2021 年关于这一主题的论文。

提示：如果想进一步学习强化学习，可以阅读 Phil Winder 的《Reinforcement Learning》（O'Reilly）。

19.12　总结

本章涉及了许多主题。我们学习了策略梯度方法；使用 Gymnasium 实现了 REINFORCE 算法来解决 CartPole 问题；探索了马尔可夫链和马尔可夫决策过程，由此引出了基于价值的方法；实现了一个深度 Q 学习模型。随后讨论了演员-评论家方法，并使用 Stable-Baselines3 库实现 PPO 模型，击败了 Atari 游戏 Breakout。最后还浏览了 RL 的其他方向，包括基于模型的 RL 等。

强化学习是一个庞大而令人兴奋的领域，每天都会出现新的思想和算法。希望本章激发了你的好奇心。还有一个完整的世界等待探索。

19.13　练习（Exercises）

1. 你会如何定义强化学习？它与常规的监督学习或无监督学习有什么不同？
2. 能否想出本章未提到的三个 RL 应用？对于每个应用，环境是什么？智能体是什么？可能的动作有哪些？奖励是什么？
3. 什么是折扣因子？如果修改折扣因子，最优策略可能改变吗？
4. 如何衡量强化学习智能体的性能？
5. 什么是信用分配问题？它何时出现？如何缓解？
6. 使用回放缓冲区的意义是什么？
7. 什么是离策略 RL 算法？它有哪些好处？
8. 什么是基于模型的 RL 算法？能举出一些例子吗？
9. 使用策略梯度解决 Gymnasium 的 LunarLander-v2 环境。
10. 使用你选择的 RL 算法解决 BipedalWalker-v3 环境。
11. 如果有大约 100 美元的预算，可以购买一块 Raspberry Pi 3 以及一些便宜的机器人组件，在树莓派上安装 PyTorch，然后尽情尝试。可以从简单目标开始，例如让机器人转向最亮的角度（如果有光线传感器）或最近的物体（如果有声纳传感器），并向那个方向移动。然后开始使用深度学习：如果机器人有摄像头，可以尝试实现目标检测算法，让它检测人并向人移动；也可以使用 RL，让智能体自己学会如何控制电机来实现目标。祝你玩得开心。

这些练习的解答位于本章笔记本的末尾，可在 https://homl.info/colab-p 获取。

19.14　致谢（Thank You!）

在结束本书最后一章之前，我想感谢你一直读到最后一段。真心希望你阅读本书时获得的快乐，和我写作本书时一样多，也希望它能对你的项目——无论大小——有所帮助。

如果发现错误，请发送反馈。更广泛地说，我很想知道你的想法，因此欢迎通过 O'Reilly 或 **`ageron/handson-mlp`** GitHub 项目联系我。

接下来，我最重要的建议是不断练习：尝试完成所有练习（如果还没有完成），运行 Jupyter 笔记本，加入 Kaggle.com 或其他机器学习社区，观看机器学习课程，阅读论文，参加会议并结识专家。拥有一个具体项目也会极大帮助你，无论它是工作项目还是兴趣项目（最好两者兼具）。如果一直梦想构建某样东西，就试着做出来。循序渐进，不要一开始就追求登月，但要专注于项目，一点一点地构建。需要耐心和毅力，但当你拥有一个会走路的机器人、一个能工作的聊天机器人，或任何你想构建的东西时，回报会非常丰厚。

我最大的希望是，本书能激励你构建一个有益于我们所有人的精彩机器学习应用。它会是什么？

——Aurélien Géron
2025 年 10 月 22 日

19.15　原书超链接资源

- Gymnasium 安装说明：https://homl.info/install-p
- REINFORCE 相关资源：https://homl.info/132
- MDP 与策略梯度资源：https://homl.info/133
- Atari 深度强化学习论文：https://homl.info/dqn
- 人类水平控制论文：https://homl.info/dqn2
- DeepMind 游戏学习视频：https://homl.info/dqn3
- DQN 改进资源：https://homl.info/doubledqn、https://homl.info/prioreplay、https://homl.info/ddqn、https://homl.info/rainbow
- 演员-评论家算法资源：https://homl.info/a3c、https://homl.info/a2c、https://homl.info/sac、https://homl.info/ppo
- Stable-Baselines3 资源：https://homl.info/sb3
- AlphaGo、AlphaGo Zero、AlphaZero 与 MuZero：https://homl.info/alphago、https://homl.info/alphagozero、https://homl.info/alphazero、https://homl.info/muzero
- 好奇心探索：https://homl.info/curiosity

- 开放式学习：https://homl.info/poet、https://homl.info/epoet、https://homl.info/oel2021、https://homl.info/rlbook
- 练习解答：https://homl.info/colab-p

19.16　章节注释

1. 更详细的内容可以参考 Richard Sutton 和 Andrew Barto 的《Reinforcement Learning: An Introduction》（MIT Press）。
2. DeepMind 于 2014 年被 Google 以超过 5 亿美元收购。
3. Volodymyr Mnih 等人的“Playing Atari with Deep Reinforcement Learning”,arXiv:1312.5602（2013），论文链接。
4. Volodymyr Mnih 等人的“Human-Level Control Through Deep Reinforcement Learning”，Nature 518（2015）：529-533，https://homl.info/dqn2。
5. DeepMind 的系统学习玩 Space Invaders、Breakout 等游戏的视频：https://homl.info/dqn3。
6. 图 19-1 中的部分图片属于公共领域；Ms. Pac-Man 截图依据本章的合理使用原则使用；围棋图片来自 Wikipedia，由用户 Stevertigo 制作并以 Creative Commons BY-SA 2.0 发布。
7. 为表现较差者保留少量生存机会通常更好，这样可以保留“基因库”的多样性。
8. 如果只有一个父代，这称为无性繁殖；有两个或更多父代时，称为有性繁殖。后代的基因组（此处是一组策略参数）由父代基因组的部分内容随机组合而成。
9. 一个有趣的强化学习遗传算法例子是 NEAT(NeuroEvolution of Augmenting Topologies)。也可以了解 2025 年提出的进化策略优化（EPO）。
10. 这称为梯度上升，与梯度下降相似，但方向相反：它最大化而不是最小化。
11. Ronald J. Williams，“Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning”，Machine Learning 8（1992）：229-256。
12. 每个回合使用 **`torch.randint()`** 生成新的随机种子。这样每个回合都不同，同时如果在调用 **`train_reinforce()`** 前设置 PyTorch 随机种子，整个训练过程仍然可复现。
13. Alex Irpan 在 2018 年发表的一篇文章很好地梳理了 RL 最大的困难和局限。
14. Richard Bellman，“A Markovian Decision Process”，Journal of Mathematics and Mechanics 6，第 5 期（1957）：679-684。

15. Hado van Hasselt 等人的“Deep Reinforcement Learning with Double Q-Learning”，AAAI 第 30 届人工智能会议论文集（2015）：2094-2100。
16. Tom Schaul 等人的“Prioritized Experience Replay”，arXiv:1511.05952（2015）。
17. 也可能只是奖励带有噪声，在这种情况下有更好的方法估计经验的重要性。
18. Ziyu Wang 等人的“Dueling Network Architectures for Deep Reinforcement Learning”，arXiv:1511.06581（2015）。
19. Matteo Hessel 等人的“Rainbow: Combining Improvements in Deep Reinforcement Learning”，arXiv:1710.02298（2017）：3215-3222。
20. Volodymyr Mnih 等人的“Asynchronous Methods for Deep Reinforcement Learning”，第 33 届国际机器学习会议论文集（2016）：1928-1937。
21. Tuomas Haarnoja 等人的“Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor”，第 35 届国际机器学习会议论文集（2018）：1856-1865。
22. John Schulman 等人的“PPO (Proximal Policy Optimization)”，arXiv:1707.06347（2017）。
23. John Schulman 等人的“Trust Region Policy Optimization”，第 32 届国际机器学习会议论文集（2015）：1889-1897。
24. “v4”后缀是版本号，与跳帧或并行环境数量无关。
25. David Silver 等人的“Mastering the Game of Go with Deep Neural Networks and Tree Search”，Nature 529（2016）：484-489。
26. David Silver 等人的“Mastering the Game of Go Without Human Knowledge”，Nature 550（2017）：354-359。
27. David Silver 等人的“Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm”，arXiv:1712.01815。
28. Julian Schrittwieser 等人的“Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model”，arXiv:1911.08265（2019）。
29. Deepak Pathak 等人的“Curiosity-Driven Exploration by Self-Supervised Prediction”，第 34 届国际机器学习会议论文集（2017）：2778-2787。
30. Rui Wang 等人的“Paired Open-Ended Trailblazer (POET): Endlessly Generating Increasingly Complex and Diverse Learning Environments and Their Solutions”，arXiv:1901.01753（2019）。
31. Rui Wang 等人的“Enhanced POET: Open-Ended Reinforcement Learning Through Unbounded Invention of Learning Challenges and Their Solutions”，arXiv:2003.08536（2020）。

32. Open-Ended Learning Team 等人的“Open-Ended Learning Leads to Generally Capable Agents”，arXiv:2107.12808（2021）。

附录 A：自动微分

Automatic Differentiation

本附录解释 PyTorch 的自动微分（autodiff）功能如何工作，并将它与其他方案进行比较。

假设定义了函数 $f(x,y)=x^2y+y+2$，需要计算偏导数 $\partial f/\partial x$ 和 $\partial f/\partial y$，通常是为了执行梯度下降或其他优化算法。主要方案包括手动微分、有限差分近似、正向模式自动微分和反向模式自动微分。PyTorch 实现的是反向模式自动微分，但要真正理解它，先看看其他方案会很有帮助。

手动微分

计算导数的第一种方法是拿起铅笔和纸，利用微积分知识推导出适当的方程。对于前面定义的函数 $f(x,y)$，这并不困难，只需要使用以下五条规则：

1. 常数的导数是 0。
2. λx 的导数是 λ，其中 λ 是常数。
3. x^λ 的导数是 $\lambda x^{\lambda-1}$，因此 x^2 的导数是 $2x$。
4. 函数和的导数等于这些函数导数之和。
5. 常数 λ 与某个函数的乘积，其导数等于 λ 乘以该函数的导数。

利用这些规则，可以得到公式 A-1。

$$\frac{\partial f}{\partial x}=2xy,\qquad \frac{\partial f}{\partial y}=x^2+1 \tag{A-1}$$

公式 A-1　函数 f(x,y) 的偏导数

对于更复杂的函数，这种方法会变得非常繁琐，并且容易出错。幸运的是，还有其他方案。先来看看有限差分近似。

有限差分近似

回忆一下，函数 $h(x)$ 在点 x_0 处的导数 $h'(x_0)$ 是该点处函数曲线的斜率。更准确地说，导数定义为：过点 x_0 和曲线上另一个点 x 的直线斜率，在 x 无限接近 x_0 时的极限（见公式 A-2）。

$$h'(x_0) = \lim_{x \to x_0} \frac{h(x) - h(x_0)}{x - x_0} = \lim_{\varepsilon \to 0} \frac{h(x_0 + \varepsilon) - h(x_0)}{\varepsilon} \tag{A-2}$$

公式 A-2　函数 h(x) 在点 x_0 处的导数定义

如果要计算 $x = 3, y = 4$ 处 $f(x, y)$ 关于 x 的偏导数，可以计算 $f(3 + \varepsilon, 4) - f(3, 4)$，再除以一个非常小的 ε。这种数值导数近似称为有限差分近似，具体方程称为牛顿差商。下面的代码实现了这一计算：

```
def f(x, y):
    return x**2*y + y + 2
def derivative(f, x, y, x_eps, y_eps):
    return (f(x + x_eps, y + y_eps) - f(x, y)) / (x_eps + y_eps)
df_dx = derivative(f, 3, 4, 0.00001, 0)
df_dy = derivative(f, 3, 4, 0, 0.00001)
```

结果并不精确，而且函数越复杂，误差通常越严重。正确结果分别是 24 和 10，而上面的代码得到的结果大约是：

```
>>> df_dx
24.000039999805264
>>> df_dy
10.000000000331966
```

注意，为了计算两个偏导数，至少需要调用 $f()$ 三次（前面的代码调用了四次，但还可以优化）。如果有 1,000 个参数，至少需要调用 $f()$ 1,001 次。对于大型神经网络，这种有限差分近似的效率太低。不过，它非常容易实现，因此是检查其他方法是否正确的好工具：如果有限差分结果与手动推导的函数不一致，函数很可能存在错误。

正向模式自动微分

图 A-1 展示了正向模式自动微分如何作用于更简单的函数 $g(x, y) = 5 + xy$。左侧表示该函数的计算图；经过正向模式自动微分后，右侧得到表示偏导数 $\partial g/\partial x = 0 + (0 \times x + y \times 1) = y$ 的计算图（同样也可以得到关于 y 的偏导数）。

算法从输入向输出遍历计算图，因此称为“正向模式”。它先取得叶节点的偏导数：常数节点 5 返回常数 0，变量 x 返回常数 1，变量 y 返回常数 0（如果求关于 y 的偏导数，情况相反）。然后根据乘法法则向上计算，最后根据加法法则得到正确的偏导数。

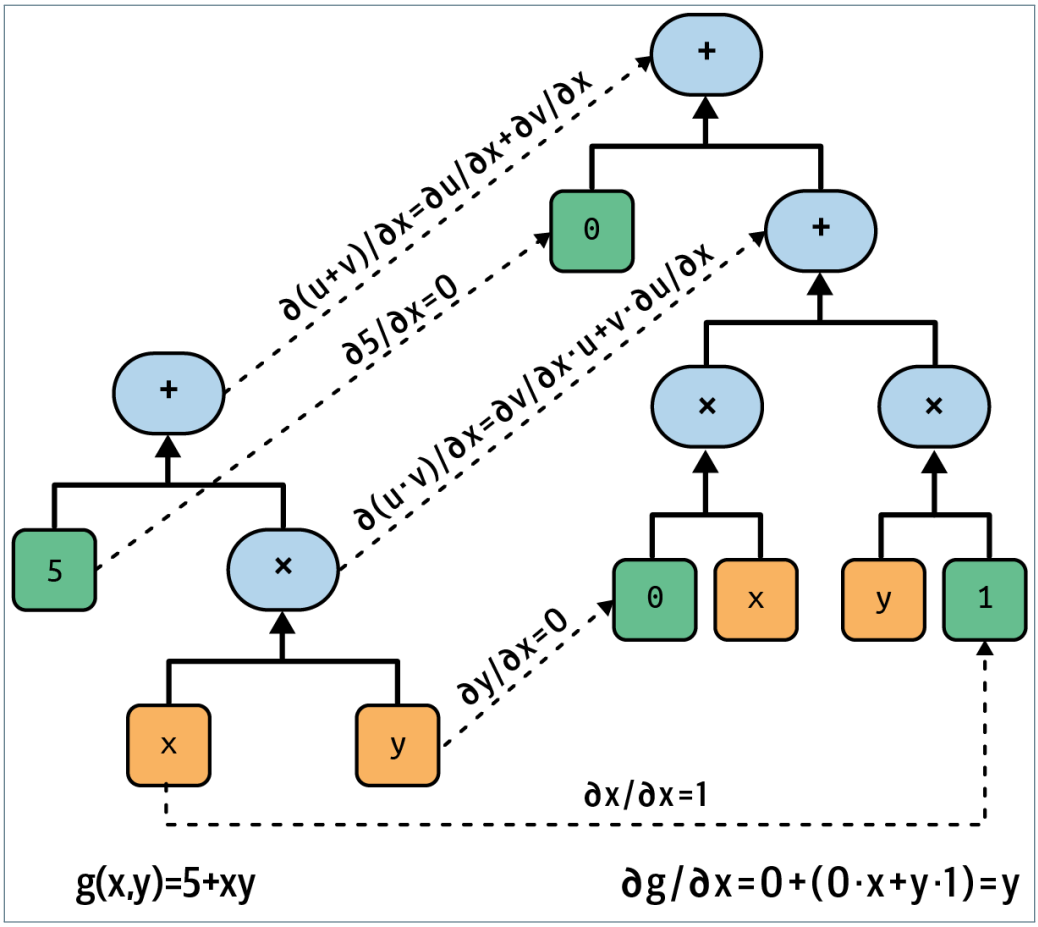

图 A-1：正向模式自动微分（Forward-mode autodiff）

通过剪枝去除不必要的运算后，计算图可以简化为只有一个节点：$\partial g/\partial x = y$。对于更复杂的函数，正向模式自动微分可能生成很大的计算图，难以简化，性能也可能不理想。

前面从一个计算图得到另一个计算图，这称为符号微分。符号微分有两个优点：一旦生成导数的计算图，就可以对任意 x 和 y 重复使用；还可以再次对结果图应用正向模式自动微分，从而得到二阶、三阶甚至更高阶导数。

也可以不构造计算图，而是在运行时计算中间结果。这称为数值正向模式自动微分。一种实现方式是使用对偶数：形如 $a + b\varepsilon$ 的数，其中 a, b 是实数，$\varepsilon^2 = 0$ 但 $\varepsilon \neq 0$。例如，对偶数 $42 + 24\varepsilon$ 可以在内存中表示为数对 (42.0, 24.0)。

对偶数可以执行加法、乘法等运算。最重要的是，可以证明 $h(a+b\varepsilon) = h(a)+b\times h'(a)\varepsilon$，因此计算 $h(a+\varepsilon)$ 就能一次得到 $h(a)$ 和 $h'(a)$。图 A-2 展示了如何用对偶数计算 $x = 3, y = 4$ 处的 $\partial f/\partial x$。

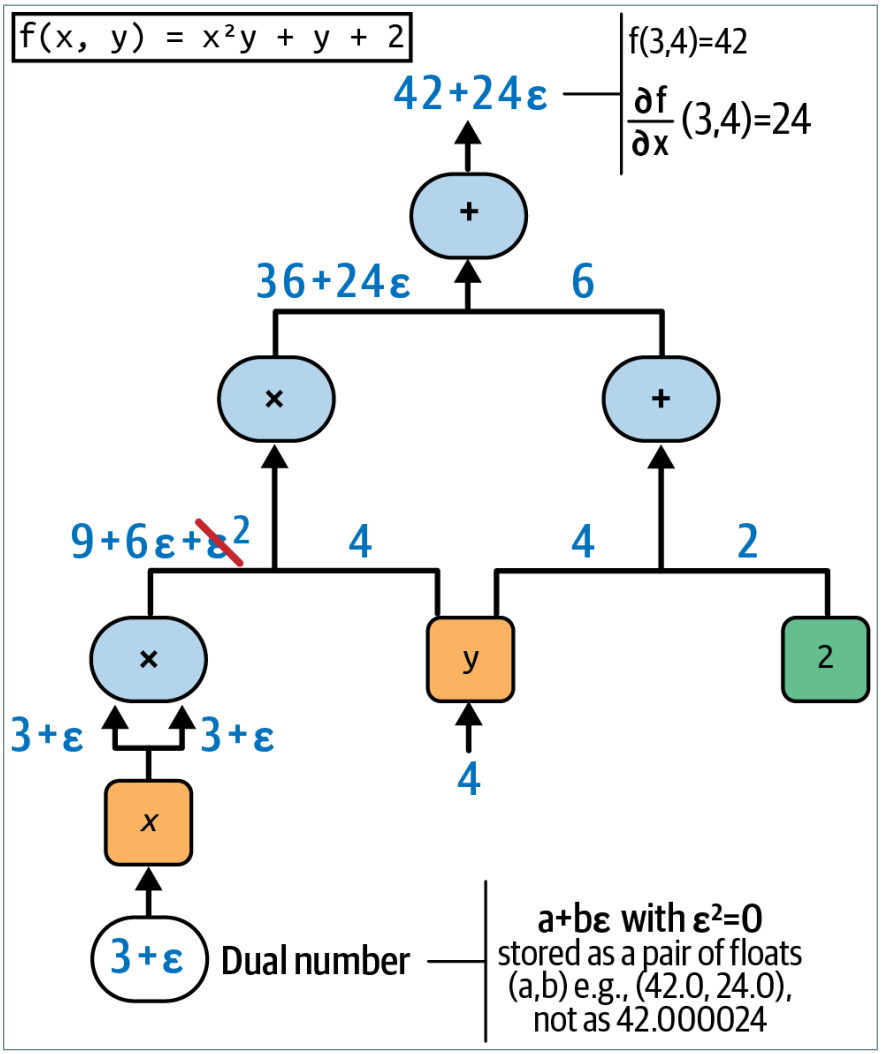

图 A-2：使用对偶数的正向模式自动微分（Forward-mode autodiff using dual numbers）

要计算 $\partial f/\partial y(3,4)$，需要再次遍历计算图，这次令 $x=3, y=4+\varepsilon$。因此，正向模式自动微分比有限差分近似精确得多，但在输入多、输出少（神经网络正是这种情况）时仍有一个主要缺陷：如果有 1,000 个参数，就需要遍历计算图 1,000 次才能计算所有偏导数。

反向模式自动微分

反向模式自动微分正是 PyTorch 使用的方案。它先沿正向方向（从输入到输出）遍历计算图，计算每个节点的值；然后沿反向方向再次遍历，计算所有偏导数。“反向模式”这个名称来自第二次遍历：梯度沿着与正向相反的方向流动。

图 A-3 展示了第二次遍历。第一次遍历从 $x = 3, y = 4$ 开始，已经计算出每个节点的值；为便于说明，节点标记为 n_1 到 n_7，输出节点为 $n_7 = f(3, 4) = 42$。

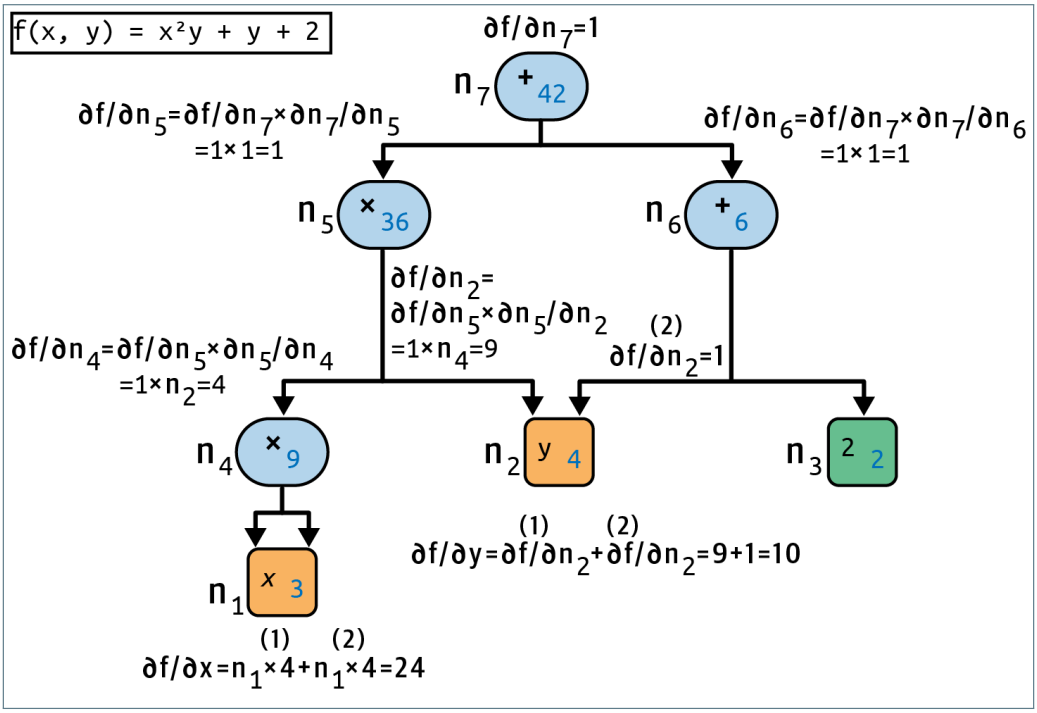

图 A-3：反向模式自动微分（Reverse-mode autodiff）

算法逐步向下遍历计算图，计算 $f(x, y)$ 关于每个连续节点的偏导数，直到到达变量节点。它大量依赖链式法则：

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial n_i} \times \frac{\partial n_i}{\partial x} \tag{A-4}$$

公式 A-4　链式法则

由于 n_7 是输出节点，所以 $f = n_7$，$\partial f/\partial n_7 = 1$。继续向下到 n_5，有 $\partial f/\partial n_5 = (\partial f/\partial n_7)(\partial n_7/\partial n_5) = 1 \times 1 = 1$。再到 n_4，因为 $n_5 = n_4 \times n_2$，所以 $\partial n_5/\partial n_4 = n_2 = 4$，从而 $\partial f/\partial n_4 = 1 \times 4 = 4$。继续这个过程，最终在 $x = 3, y = 4$ 处得到 $\partial f/\partial x = 24$ 和 $\partial f/\partial y = 10$。

当输入多、输出少时，反向模式自动微分尤其强大且精确：对于每个输出，只需要一次正向遍历和一次反向遍历，就可以得到所有输出关于所有输入的偏导数。训练神经网络时，通常要最小化损失函数，而损失只有一个输出，因此只需要两次遍历计算图即可得到梯度。

PyTorch 在每次正向遍历期间动态构建新图。当对 requires_grad=True 的张量执行操作时，PyTorch 计算结果张量，并把它的 grad_fn 属性设置为与操作对应的对象，使 PyTorch 能

够沿操作反向传播梯度。由于计算图是动态构建的，代码可以包含循环和条件分支，自动微分仍然能够正常工作。

反向模式自动微分也能处理不是处处可微的函数，只要要求它在可微点计算偏导数即可。

提示 从零开始为少量操作实现一个小型自动微分框架，是掌握自动微分的优秀练习。如果遇到困难，可以查看项目中的 extra_autodiff.ipynb notebook，并在 Colab 上运行：https://homl.info/colab-p。还可以观看 Andrej Karpathy 构建 micrograd 库的优秀 YouTube 视频。

附录 B：混合精度与量化

Mixed Precision and Quantization

默认情况下，PyTorch 使用 32 位浮点数表示模型参数，每个参数占 4 字节。如果模型有 10 亿个参数，仅保存模型就至少需要 4 GB 内存。推理时还要保存激活值；训练时还要保存反向传播所需的中间激活值和优化器参数（例如 Adam 会为每个模型参数额外保存两个参数，也就是再增加 8 GB）。这不仅需要大量内存，也会带来 CPU 与 GPU 之间的数据传输、存储空间、下载时间和能耗问题。

减少模型大小的一个简单方法是使用低精度浮点表示，通常把 32 位浮点数改为 16 位浮点数。如果先用 32 位训练模型，再在训练后缩小到 16 位，模型大小可以减半，而质量几乎不受影响。

但如果直接使用 16 位浮点数训练，可能遇到收敛问题。因此，常见策略是混合精度训练（mixed-precision training，MPT）：训练期间保留 32 位权重和权重更新，但其他计算使用 16 位精度；训练完成后再把权重缩小到 16 位。最后，还可以进一步量化：把参数离散化为 8 位整数，甚至 4 位或更少。量化更困难，也会对模型质量产生更大影响，但可以把模型缩小 4 倍以上，并显著加速推理。

本附录介绍低精度、混合精度训练和量化。为了理解这些技术，先讨论机器学习中常用的数字表示方法。

常见数字表示

默认情况下，PyTorch 根据 IEEE 浮点算术标准（IEEE 754）使用 32 位浮点数表示权重和激活值。它能够表示很小和很大的数，也能表示特殊值，例如 $+0$、-0、无穷大和 NaN（Not a Number）。

float32 数据类型（简称 fp32）可以表示约 $\pm 1.4\text{e}{-45}$ 到 $\pm 3.4\text{e}38$ 的数。它由一个符号位 S、8 个指数位 E 和 23 个分数位 F 组成。图 B-1 展示了常见的浮点格式：指数位决定可表示的范围，分数位决定精度。

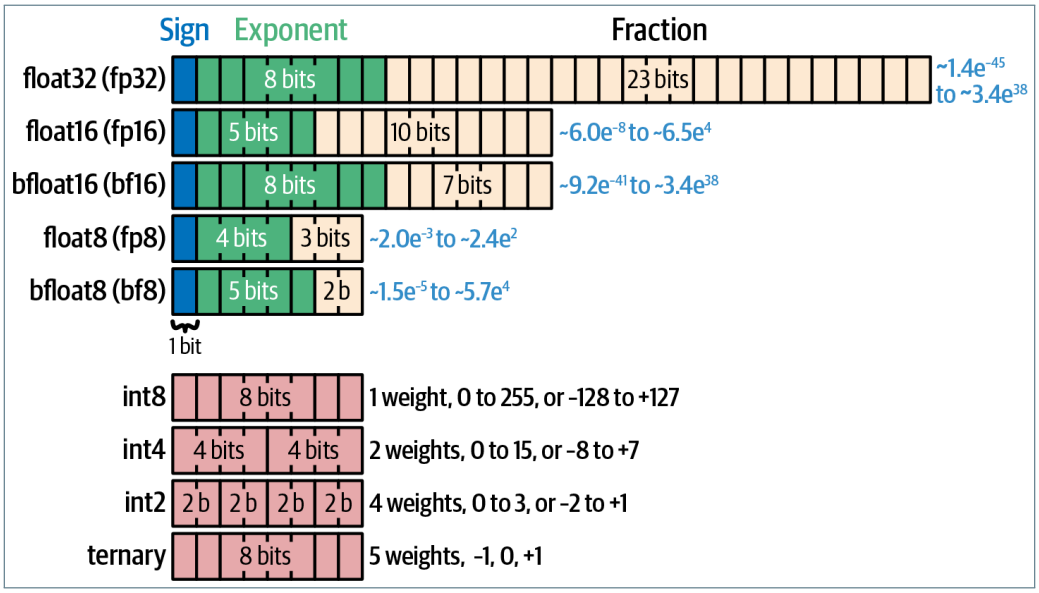

图 B-1：机器学习中的常见数字表示（Common number representations in machine learning）

bfloat16 和 bfloat8 格式由 Google Brain 提出（因此名称中有字母 b），它们用更低的精度换取更宽的数值范围。整数常用 64 位表示；无符号整数范围为 0 到 $2^{64}-1$，有符号整数范围约为 $-4.3e9$ 到 $+4.3e9$。量化经常使用 8 位整数，既可以是无符号的，也可以是有符号的。

低精度模型

将已经训练好的模型从 32 位缩小到 16 位很简单。可以调用模型的 half() 方法：

```
model = model.half()
```

此后需要给模型输入 16 位张量，模型也会输出 16 位张量：

```
X = torch.rand(3, 10, dtype=torch.float16)  # some 16-bit input
y_pred = model(X)  # 16-bit output
```

也可以在创建张量或带参数的模块时直接设置 dtype=torch.float16：

```
model = nn.Sequential(
    nn.Linear(10, 100, dtype=torch.float16),
    nn.ReLU(),
    nn.Linear(100, 1, dtype=torch.float16)
)
```

不过，16 位浮点数的范围比 32 位小得多，因此容易出现下溢和上溢。下溢会把非常小的数变为零，上溢会把过大的数变为无穷大或 NaN。如果损失变成 NaN，训练通常就会失败。

bfloat16 使用与 float32 相同数量的指数位，因此范围相近，但只保留 7 个分数位，精度低于 float16。在许多深度学习任务中，较大的范围比额外的精度更有价值。下载预训练模型时，可以把 Transformers 库的 from_pretrained() 方法的 dtype 设置为 auto，让库根据硬件选择最合适的浮点表示。

混合精度训练

混合精度训练由 Baidu 和 Nvidia 的研究人员于 2017 年提出，用于解决 16 位训练中经常出现的问题。其工作方式如下：

- 模型参数保留一个 32 位主副本；每次训练迭代开始时，根据主副本创建一个 16 位副本。
- 使用 16 位副本执行正向传播和反向传播。
- 使用得到的 16 位梯度更新 32 位主副本。
- 更新后的 32 位主副本再次转换为 16 位副本，进入下一次迭代。

图 B-2 展示了这一流程。32 位浮点数有较长的分数部分，因此在转换为 16 位时可以准确表示大多数参数。另一方面，梯度往往非常小，转换为 16 位后可能下溢为零。常见的解决方法是在计算损失前放大损失，反向传播得到的梯度也随之放大；更新权重前再把梯度缩小回来。这称为损失缩放。

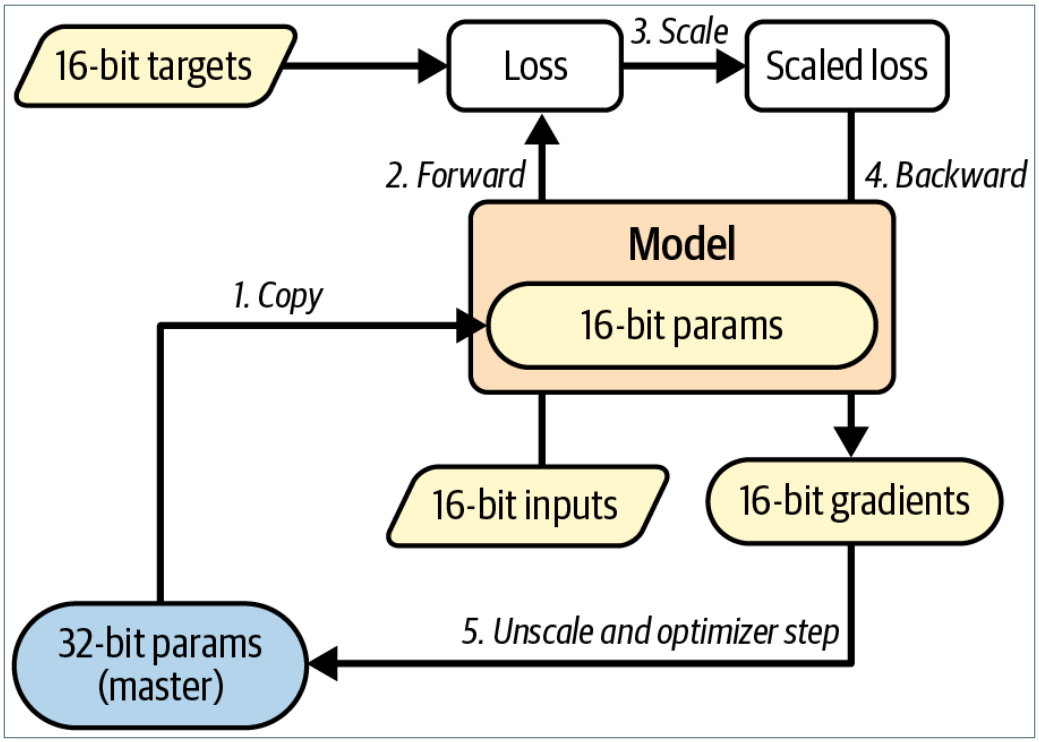

图 B-2：混合精度训练（Mixed-precision training）

如果检测到梯度中出现无穷大或 NaN，说明损失缩放因子过大，应跳过本次更新，并降低缩放因子。如果没有检测到上溢，则可以逐渐增加缩放因子。PyTorch 提供了 torch.amp.GradScaler 来实现这一策略，也提供了 torch.autocast()，在其上下文中自动把适合的操作切换到 16 位精度。

下面是一个简化的混合精度训练循环：

```
from torch.amp import GradScaler

def train_mpt(model, optimizer, criterion, train_loader, n_epochs,
              device="cuda"):
    scaler = GradScaler("cuda")
    model.train()
    for epoch in range(n_epochs):
        for X_batch, y_batch in train_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            optimizer.zero_grad()
            with torch.autocast(device_type="cuda", dtype=torch.float16):
                y_pred = model(X_batch)
                loss = criterion(y_pred, y_batch)
            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()
```

如果使用 Hugging Face 的 Transformers 库，可以在 TrainingArguments 中启用混合精度训练。训练 Transformer 时，混合精度通常能够显著降低内存占用并提高速度。

量化

量化把浮点值映射到有限的整数集合。量化后的模型通常占用更少的内存，运行速度更快，尤其是在硬件支持整数运算时。8 位量化通常能把模型大小缩小到接近四分之一；实际加速和节能效果取决于硬件和实现。

线性量化

最简单的量化方式是线性量化。给定浮点张量 w 的最小值 a 和最大值 b，用 n 位无符号整数表示它。整数张量的范围是 0 到 $2^n - 1$。尺度 s 和零点 z 的计算方式见公式 B-1，量化过程也可以写成 $q_i = \text{round}(w_i/s) + z$。

$$q_i = \text{round}\left(\frac{w_i - a}{s}\right), \qquad s = \frac{b - a}{2^n - 1}, \qquad z = \text{round}\left(-\frac{a}{s}\right) \tag{B-1}$$

公式 B-1　非对称线性量化

其中，w_i 是原始浮点张量中的第 i 个元素，q_i 是量化后的整数，a 和 b 是浮点张量的最小值与最大值，z 是量化偏置或零点。反量化时使用 $w_i \approx s(q_i - z)$。量化参数也可以按张量、通道或更小的分组分别计算；粒度越细，精度通常越高，但需要保存更多参数。

另一种常见方式是对称量化：整数范围围绕零对称，零点固定为 0。公式 B-2 给出了这种量化的定义。

$$q_i = \text{round}\left(\frac{w_i}{s}\right), \qquad s = \frac{\max_i |w_i|}{2^{n-1} - 1} \tag{B-2}$$

公式 B-2　对称线性量化

对称量化实现简单，也能让整数乘法更高效，但它可能浪费一部分可用范围，尤其是在原始分布明显偏向正数或负数时。

例如，一个卷积层有 64 个卷积核，每个卷积核大小为 3×3，并且有 32 个输入通道，那么共有 $64 \times 32 \times 3 \times 3 = 18,432$ 个参数。使用 32 位浮点数时需要 73,728 字节；量化为 8 位整数后只需要 18,432 字节，另外再保存尺度和零点所需的少量空间。

下面展示对称量化的一个简单计算：

```
>>> w = torch.tensor([0.0, -0.94, 0.92, 0.93])
>>> s = w.abs().max() / 127.
>>> q = torch.round(w / s)
>>> w_dequantized = q * s
```

量化粒度越细，通常精度越高，但量化参数会占用更多空间。按张量量化最简单，按通道量化通常能得到更好的效果。

使用 torch.ao.quantization 进行训练后量化

torch.ao 包含剪枝、稀疏化和量化等架构优化工具。torch.ao.quantization 为训练好的模型提供动态量化和静态量化两种主要方案。

动态量化

动态量化在推理期间根据每个输入批次的激活范围计算量化参数。它不需要校准数据，使用方便，但每次推理都要计算范围：

```
import platform
import torch
import torch.nn as nn

machine = platform.machine().lower()
engine = "qnnpack" if ("arm" in machine or "aarch64" in machine) else "x86"

model = nn.Sequential(
    nn.Linear(10, 100), nn.ReLU(), nn.Linear(100, 1)
)
quantized_model = torch.ao.quantization.quantize_dynamic(
    model, {nn.Linear}, dtype=torch.qint8
)
```

动态量化对包含大量线性层的模型特别有用，例如 Transformer 的语言模型。它只量化权重，激活值在运行时量化和反量化。

静态量化

静态量化在推理前使用代表性数据校准激活值范围。模型需要在输入端和输出端放置 QuantStub 与 DeQuantStub：

```
class QuantizedModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.quant = torch.ao.quantization.QuantStub()
        self.net = nn.Sequential(
            nn.Linear(10, 100), nn.ReLU(), nn.Linear(100, 1)
        )
        self.dequant = torch.ao.quantization.DeQuantStub()

    def forward(self, X):
        X = self.quant(X)
        return self.dequant(self.net(X))

model = QuantizedModel()
model.qconfig = torch.ao.quantization.get_default_qconfig(engine)
torch.ao.quantization.prepare(model, inplace=True)

for X_batch, _ in calibration_loader:
    model(X_batch)

torch.ao.quantization.convert(model, inplace=True)
```

流程是：先创建带有量化和反量化桩的 32 位模型；然后正常训练或加载预训练模型；设置 qconfig；调用 prepare() 插入权重和激活观察器；使用代表性输入批次运行模型，让观察器记录范围；最后调用 convert()，移除观察器并把层替换为量化版本。静态量化速度较快，但如果校准数据不能代表生产数据，精度可能下降。

> **注** 观察器有多种选择：MinMaxObserver 记录每个张量的最小值和最大值，PerChannelMinMaxObserver 按通道记录，HistogramObserver 则记录直方图并寻找能最小化量化误差的范围。默认观察器通常已经足够。

训练后量化可以显著缩小并加速模型，但也会降低准确率。量化到 4 位或更低时，精度损失往往更明显；静态量化通常比动态量化更容易出现这种问题。如果精度下降不可接受，可以使用量化感知训练。

量化感知训练

量化感知训练（quantization-aware training，QAT）由 Google 研究人员于 2017 年提出。核心思想是在训练期间加入“伪量化噪声”，让模型学会适应量化带来的误差；训练结束后再真正量化模型，因此可以保持更高的准确率，也可以更激进地量化到 4 位或更低。

加入伪量化噪声的方法很简单：先将权重量化，再立即反量化。例如，权重 0.42 可能被量化为 4 位整数 7，再反量化为 0.39。这样就模拟了真实量化的噪声。但四舍五入操作

的梯度为零（或在整数边界处未定义），梯度下降无法前进。解决方法是使用直通估计器（straight-through estimator，STE）：反向传播时假装伪量化操作是恒等函数，让梯度不变地穿过它。

PyTorch 中的 QAT 实现如下：

```
from torch.ao.quantization import get_default_qat_qconfig

model = nn.Sequential(
    nn.Linear(10, 100), nn.ReLU(), nn.Linear(100, 1)
)
model.qconfig = get_default_qat_qconfig(engine)
torch.ao.quantization.prepare_qat(model, inplace=True)
train(model, optimizer, [...])  # train the model normally
torch.ao.quantization.convert(model.eval(), inplace=True)
```

QAT 不必覆盖整个训练过程。也可以使用较低的学习率，对预训练模型进行几个 epoch 的 QAT 微调，以避免破坏已有权重。

使用 bitsandbytes 量化 LLM

bitsandbytes（简称 bnb）由 Tim Dettmers 创建，旨在帮助用户在显存有限的 GPU 上训练和运行大型模型。它提供 4 位量化、分块量化、节省内存的 8 位 Adam/AdamW 优化器，以及针对 8 位和 4 位量化模型优化的 CUDA 内核。

> **警告** bitsandbytes 主要为 Nvidia GPU 设计，对 CPU 和 AMD GPU 只有有限支持。

下面的示例使用 Transformers 和 bitsandbytes，把一个 TinyLlama 模型以 4 位加载：

```
from transformers import AutoModelForCausalLM, BitsAndBytesConfig

model_id = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16,
)
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    device_map="auto",
    quantization_config=bnb_config,
)
```

Transformers 会在模型加载到 GPU 时直接量化权重，不需要额外步骤。推理期间，需要的权重会临时反量化到 bfloat16，在更高精度下完成计算，用完后立即释放，因此内存使用量保持较低。NF4（4 位 Normal Float）是一种非线性 4 位方案，16 个整数值分别对应 -1 到 $+1$ 之间的浮点值；这些值按以零为中心的正态分布分位数排列，在零附近更密集。

NF4 是 QLoRA 技术的一部分。QLoRA 使用 NF4 量化冻结的预训练模型，再用 LoRA 适配器进行微调，并结合激活检查点技术。作者展示了如何只使用一块 48 GB 显存的 GPU 微调 650 亿参数模型，同时只损失少量准确率。QLoRA 还使用分页优化器利用 Nvidia 统一内存，在需要时把数据页从 GPU 显存移动到 CPU 内存，并使用双重量化进一步压缩量化参数。

使用预量化模型

许多流行的预训练模型已经完成量化，并发布在 Hugging Face Hub 上。现代量化方法包括：

- 生成式预训练量化（GPTQ）：把量化视为优化问题，逐层优化 4 位权重，使量化前后的层输出均方误差最小。GPTQ 只量化权重，适合推理而不适合训练。
- 激活感知权重量化（AWQ）：通过校准数据找出最重要的权重，并设法保留它们的精度，同时对其他权重进行分块量化。
- Llama.cpp 的 GGUF：一种高效保存大型语言模型的二进制格式，包含权重、tokenizer、特殊 token、模型架构、词表大小和其他元数据。格式名中的 Q4_K_M 表示 4 位量化、按块量化以及中等大小和精度。

在 Hugging Face Hub 中，每个仓库都由 Git 支持，因此包含分支和提交。调用 from_pretrained() 时，默认从 main 分支下载；量化模型有时位于其他分支，可以通过 revision 指定分支、标签或提交哈希。GGUF 模型还需要通过 gguf_file 指定文件名。

其他模型压缩方法

低精度、混合精度训练和量化是让大型模型在有限硬件上运行的最重要工具，但还可以使用以下方法：

- 在训练前调整模型架构，减少层数或每层的神经元数量，也可以跨层共享权重，例如 ALBERT。
- 删除训练好模型中的部分权重，例如幅度最小或对损失影响最小的权重；也可以删除整个通道、层或注意力头。这称为模型剪枝，可使用 torch.nn.utils.prune 或 Hugging Face Optimum 实现。
- 使用大型训练模型作为教师，训练一个更小的模型，这称为知识蒸馏。
- 融合冗余层。例如，批归一化层执行线性操作，如果紧跟在线性层之后，就可以把两层融合为单个线性层；卷积层和批归一化层也可以类似融合。融合应在量化之前进行，因为层数越少，量化噪声越少。
- 使用低秩近似，把大型矩阵替换为两个较小矩阵的乘积。例如，使用 Linear(10_000, 20_000) 时，可以替换为 Linear(10_000, 100) 和 Linear(100, 20_000)。这会把参数量从约 2 亿减少到约 300 万，也会大幅减少计算量。中间维度是一个可调超参数，可以平衡准确率与模型大小；训练后还可以使用 SVD 对权重矩阵进行因子分解。

不妨尝试这些技术，把模型缩小吧！在线附录、代码和相关论文链接仍可通过 https://homl.info 获取。

术语索引

Index

本索引按照中文译稿的重排结构编制。由于译稿不再沿用原书分页，索引使用章节和主题定位；英文术语、库名、函数名与缩写保留原文，便于在 PDF 中搜索。章节编号和图表编号均可通过目录或书签跳转。

符号与数字

- *运算符、* 运算符——多输入模型。
- @ 运算符（矩阵乘法）——正规方程。
- α（alpha）超参数——岭回归；Leaky ReLU；ELU 和 SELU。
- β（beta）超参数——GELU、Swish、SwiGLU、Mish 和 RELU²。
- β 动量系数——动量法和 Adam。
- γ（gamma）折扣因子——强化学习中的动作评估。
- ε（容差）——批量梯度下降。
- ε-greedy 策略——探索策略和深度 Q 学习。
- ε 邻域——DBSCAN。
- ℓ1、ℓ2 范数和正则化——性能指标与正则化。
- χ^2 检验——正则化超参数。

A

- A2C（Advantage Actor-Critic，优势演员–评论家）——第 19 章。
- A3C（Asynchronous Advantage Actor-Critic，异步优势演员–评论家）——第 19 章。
- AdaBoost、AdaGrad、AdaMax、Adam、AdamW——第 6 章和第 11 章。
- ALBERT——第 15 章；模型压缩。

- AlphaFold、AlphaGo、AlphaCode——强化学习与生成式模型。
- anomaly detection（异常检测）——第 1 章、第 8 章。
- attention（注意力）——第 13 至 16 章。
- attention heads（注意力头）——多头注意力和 Transformer。
- autocorrelation function，ACF（自相关函数）——时间序列。
- autodiff（自动微分）——附录 A。
- autoencoder（自编码器）——第 8 章和第 18 章。
- AWQ（Activation-aware Weight Quantization，激活感知权重量化）——附录 B。

B

- bagging（装袋法）——第 6 章。
- batch normalization（批归一化）——第 11 章。
- batch gradient descent（批量梯度下降）——第 4 章。
- Bayesian Gaussian mixture（贝叶斯高斯混合）——第 8 章。
- bfloat16——附录 B。
- bias/variance trade-off（偏差–方差权衡）——第 4 章。
- bitsandbytes——附录 B。
- BPE（Byte-Pair Encoding，字节对编码）——第 14 章。
- boosting（提升法）——第 6 章。
- BERT、DeBERTa、RoBERTa——第 15 章。

C

- calibration（校准）——分类概率与量化。
- CART（Classification and Regression Tree）——第 5 章。
- causal language model（因果语言模型）——第 15 章。
- checkpointing（检查点）——训练与 GPU 内存。

- CLIP——第 16 章。
- clustering（聚类）——第 8 章。
- convolutional layer（卷积层）——第 12 章和附录 B。
- cross-attention（交叉注意力）——第 15、16 章。
- cross-validation（交叉验证）——第 2、3 章。

D

- DALL·E——第 18 章。
- DBSCAN——第 8 章。
- decision tree（决策树）——第 5 章。
- denoising autoencoder（去噪自编码器）——第 8 章。
- diffusion model（扩散模型）——第 18 章。
- dimensionality reduction（降维）——第 7 章。
- DINO——第 16 章。
- dropout——第 11 章。
- DQN（Deep Q-Network，深度 Q 网络）——第 19 章。

E

- early stopping（提前停止）——第 4、11 章。
- ELU、SELU、GELU、Leaky ReLU、PReLU、RReLU——第 9、11 章。
- embedding（嵌入）——第 14、15 章。
- encoder-decoder（编码器–解码器）——第 14、15 章。
- ensemble learning（集成学习）——第 6 章。
- epsilon-greedy（ε-贪心）——第 19 章。
- explained variance（解释方差）——第 7 章。

F

- F1 score——第 3 章。
- Fashion MNIST——第 10、12 章。
- feature engineering（特征工程）——第 2 章。
- feature scaling（特征缩放）——第 2、4 章。
- fine-tuning（微调）——第 14 至 17 章。
- finite difference approximation（有限差分近似）——附录 A。
- FlashAttention——第 17 章。
- forward-mode autodiff（正向模式自动微分）——附录 A。

G

- GAN（生成对抗网络）——第 18 章。
- Gaussian mixture model，GMM（高斯混合模型）——第 8 章。
- GELU——第 11 章。
- GGUF——附录 B。
- gradient clipping（梯度裁剪）——第 11、13 章。
- gradient descent（梯度下降）——第 4 章。
- gradient boosting（梯度提升）——第 6 章。
- Gymnasium——第 19 章。

H

- hidden layer（隐藏层）——第 9、10 章。
- Hugging Face Hub——第 14 至 17 章、附录 B。
- hyperparameter（超参数）——第 2、4、11 章。
- hyperparameter search（超参数搜索）——第 2、4 章。

I

- image classification（图像分类）—-第 3、10、12、16 章。
- imputation（缺失值填补）—-第 2 章。
- inference（推理）—-第 10、14、15、17 章。
- initialization（初始化）—-第 4、9、11 章。
- Isolation Forest—-第 8 章。

K

- k-means—-第 8 章。
- k-nearest neighbors，k-NN（k 近邻）—-第 1、3 章。
- KL divergence，KLD（KL 散度）—-第 18 章。
- knowledge distillation（知识蒸馏）—-第 15、17 章和附录 B。

L

- Lasso regression—-第 4 章。
- latent space（潜在空间）—-第 8、16、18 章。
- layer normalization（层归一化）—-第 11、15 章。
- LeNet-5—-第 12 章。
- linear regression（线性回归）—-第 4 章。
- LoRA（Low-Rank Adaptation，低秩适配）—-第 17 章。
- logistic regression（逻辑回归）—-第 4 章。
- loss function（损失函数）—-第 4、9、11、15、18、19 章。
- LSTM（长短期记忆）—-第 13 章。

M

- MAE（Mean Absolute Error，平均绝对误差）——第 2、4 章。
- MLP（Multilayer Perceptron，多层感知机）——第 9、10 章。
- mixed-precision training（混合精度训练）——附录 B。
- momentum（动量）——第 4、11 章。
- MoE（Mixture of Experts，专家混合）——第 17 章。
- model pruning（模型剪枝）——第 17 章和附录 B。
- model selection（模型选择）——第 2、4 章。
- model-based learning（基于模型的学习）——第 19 章。

N

- NaN——附录 B。
- natural language processing，NLP（自然语言处理）——第 13 至 15 章。
- neural network（神经网络）——第 9 至 11 章。
- Newton's difference quotient（牛顿差商）——附录 A。
- normalization（归一化）——第 2、11 章。

O

- object detection（目标检测）——第 16 章。
- one-hot encoding——第 2、3 章。
- optimizer（优化器）——第 4、11、15、19 章。
- overfitting（过拟合）——第 1、4、9 章。

P

- PCA（Principal Component Analysis，主成分分析）——第 7 章。

- policy gradient（策略梯度）——第 19 章。
- positional encoding（位置编码）——第 15、17 章。
- precision/recall（精确率/召回率）——第 3 章。
- pretraining（预训练）——第 14 至 17 章。
- PyTorch——前言、第 10 至 19 章。

Q

- Q-learning——第 19 章。
- QAT（Quantization-Aware Training，量化感知训练）——附录 B。
- QLoRA——第 17 章和附录 B。
- quantization（量化）——附录 B。

R

- random forest（随机森林）——第 6 章。
- reinforcement learning（强化学习）——第 19 章。
- replay buffer（回放缓冲区）——第 19 章。
- residual connection（残差连接）——第 11、15、16 章。
- reverse-mode autodiff（反向模式自动微分）——附录 A。
- Ridge regression（岭回归）——第 4 章。
- ROC curve——第 3 章。
- RNN（循环神经网络）——第 13、14 章。

S

- sampling bias（抽样偏差）——第 1 章。
- Scikit-Learn——前言、第 1 至 8 章。

- self-attention（自注意力）—-第 14、15、16 章。
- semantic segmentation（语义分割）—-第 16 章。
- sigmoid—-第 4、9、11 章。
- softmax—-第 4、9、15 章。
- sparse autoencoder（稀疏自编码器）—-第 8 章。
- static quantization（静态量化）—-附录 B。
- state-space model，SSM（状态空间模型）—-第 15、17 章。
- stacking（堆叠法）—-第 6 章。
- stochastic gradient descent，SGD（随机梯度下降）—-第 4、11 章。
- Swin Transformer—-第 16 章。

T

- temporal-difference learning（时序差分学习）—-第 19 章。
- tokenization（分词）—-第 14、15 章。
- transfer learning（迁移学习）—-第 10、12、15、16 章。
- Transformer—-第 14 至 17 章。
- t-SNE—-第 7 章。

U

- underfitting（欠拟合）—-第 1、4 章。
- unsupervised learning（无监督学习）—-第 1、7、8 章。
- utility function（效用函数）—-第 19 章。

V

- validation set（验证集）—-第 2、4 章。

- variational autoencoder，VAE（变分自编码器）—-第 18 章。
- variance（方差）—-第 4、5、6 章。
- vector quantization（向量量化）—-第 18 章。
- vision transformer，ViT（视觉 Transformer）—-第 16 章。
- VQ-VAE—-第 18 章。

W-Z

- weight decay（权重衰减）—-第 11 章。
- weight-only quantization（仅权重量化）—-附录 B。
- Wide & Deep—-第 10 章。
- WordPiece—-第 14 章。
- YOLO（You Only Look Once）—-第 16 章。
- zero-shot learning（零样本学习）—-第 15、16 章。

作者介绍

About the Author

Aurélien Géron 是机器学习顾问和讲师，曾在 Google 工作，并领导 YouTube 的视频分类团队。他曾创办并担任多家公司的首席技术官：Wifirst 是法国领先的无线互联网服务提供商；Polyconseil 是专注于电信、媒体和战略的咨询公司；Geron AI 则专注于机器学习咨询。

在此之前，Aurélien 曾在多个领域担任工程师，包括金融（J.P. Morgan 和 Société Générale）、国防（加拿大国防部）和医疗保健（血液输注）。他还出版过几本技术图书，主题涉及 C++、Wi-Fi 和互联网架构，并曾在多所大学授课。

一些有趣的小事：他教三个孩子用手指以二进制计数，最高可以数到 1,023；在进入软件工程之前，他曾学习微生物学和进化遗传学；而且他第二次跳伞时，降落伞没有打开。

版本说明

Colophon

《Hands-On Machine Learning with Scikit-Learn and PyTorch》封面上的动物是婆罗洲猩猩（Pongo pygmaeus）。猩猩属于人科（Hominidae），与大猩猩、黑猩猩和人类同属类人猿。

婆罗洲猩猩原产于婆罗洲的热带雨林。它们是树栖动物，但为了寻找食物也会在地面上移动很远的距离。食物包括水果、种子、花、藤蔓和鸟蛋。它们在树上搭建精巧的巢夜间休息；幼猩猩六个月大时开始练习搭巢，三岁左右就能熟练搭建。

婆罗洲猩猩具有明显的两性异形。雄性比雌性大，拥有更大的颊垫（称为面颊肉垂）、更长的犬齿和更多的面部毛发。雄性平均身高约 4 英尺 9 英寸，体重约 165 磅；雌性平均身高约 3 英尺 7 英寸，体重约 85 磅。猩猩的手臂很长，适合在树上攀爬；皮肤呈灰色，被覆蓬松的红棕色毛发。

婆罗洲猩猩比其他猩猩物种更加独居。它们通常不被视为具有领地意识，但雄性相遇时会表现出威胁行为。雄性只与雌性交往和交配。雌性大约 15 岁达到性成熟，每 8 或 9 年生育一个幼崽。猩猩拥有所有类人猿中最长的幼体发育期：幼崽大约 4 岁断奶，约 7 岁完全独立，寿命为 35 至 45 年。

野外大约有 105,000 只婆罗洲猩猩。森林砍伐和捕猎使它们越来越濒危。虽然捕猎猩猩是违法行为，但犯罪者很少受到起诉。由于栖息地遭到破坏，它们在婆罗洲的分布已经变得零散。此外，岛上的降雨量减少，食物变得更加稀缺，也影响了猩猩的出生率。它们在 IUCN 红色名录中的等级为“极危”（Critically Endangered）。O’Reilly 的封面动物中有许多都濒临灭绝；它们对整个世界都很重要。

封面插画由 José Marzan Jr. 绘制，依据 Friedrich Specht 于 1888 年创作的一幅古老线刻画改编。系列设计由 Edie Freedman、Ellie Volckhausen 和 Karen Montgomery 完成。封面字体为 Gilroy Semibold 和 Guardian Sans；正文使用 Adobe Minion Pro，标题使用 Adobe Myriad Condensed，代码使用 Dalton Maag 的 Ubuntu Mono。

原书超链接对照表

Original Link Catalog

本表从原 PDF 的可点击外部链接中提取并去重，按原 PDF 物理页首次出现的位置分组。正文中的链接仍保留在原对应位置；本表用于补足因中文重排、文本清理或 OCR 导致未能放回正文的链接。点击“原文链接”即可打开与原 PDF 相同的 URL。

版权页与出版信息

- 原 PDF 第 4、11、12 页：原文链接
- 原 PDF 第 4 页：原文链接
- 原 PDF 第 4 页：原文链接

前言（Preface）

- 原 PDF 第 5 页：原文链接
- 原 PDF 第 5 页：原文链接
- 原 PDF 第 5 页：原文链接
- 原 PDF 第 6、46、113 页：原文链接
- 原 PDF 第 6、539 页：原文链接
- 原 PDF 第 6、242、244、267、539、560 页：原文链接
- 原 PDF 第 6、449 页：原文链接
- 原 PDF 第 6 页：原文链接
- 原 PDF 第 6 页：原文链接
- 原 PDF 第 6 页：原文链接
- 原 PDF 第 7 页：原文链接
- 原 PDF 第 7 页：原文链接

- 原 PDF 第 7 页：原文链接
- 原 PDF 第 7 页：原文链接
- 原 PDF 第 7 页：原文链接
- 原 PDF 第 7 页：原文链接
- 原 PDF 第 7 页：原文链接
- 原 PDF 第 9 页：原文链接
- 原 PDF 第 9、18、88、104、216、223、386、424、477、502、585 页：原文链接
- 原 PDF 第 9 页：原文链接
- 原 PDF 第 9 页：原文链接
- 原 PDF 第 9 页：原文链接
- 原 PDF 第 9 页：原文链接
- 原 PDF 第 10、87 页：原文链接
- 原 PDF 第 10 页：原文链接
- 原 PDF 第 10 页：原文链接
- 原 PDF 第 10 页：原文链接
- 原 PDF 第 11 页：原文链接
- 原 PDF 第 11 页：原文链接
- 原 PDF 第 11 页：原文链接
- 原 PDF 第 12 页：原文链接
- 原 PDF 第 12 页：原文链接
- 原 PDF 第 12 页：原文链接
- 原 PDF 第 12 页：原文链接
- 原 PDF 第 12 页：原文链接
- 原 PDF 第 12 页：原文链接

第 1 章

- 原 PDF 第 27 页：原文链接
- 原 PDF 第 27 页：原文链接
- 原 PDF 第 33 页：原文链接
- 原 PDF 第 33 页：原文链接
- 原 PDF 第 39 页：原文链接
- 原 PDF 第 40、46、88、112、143、147、155、174、190、217、241、273、308、324、355、384、420、466、500、534、565、572 页：原文链接

第 2 章

- 原 PDF 第 41 页：原文链接
- 原 PDF 第 41 页：原文链接
- 原 PDF 第 41 页：原文链接
- 原 PDF 第 41 页：原文链接
- 原 PDF 第 41 页：原文链接
- 原 PDF 第 41 页：原文链接
- 原 PDF 第 41 页：原文链接
- 原 PDF 第 41 页：原文链接
- 原 PDF 第 41 页：原文链接
- 原 PDF 第 41 页：原文链接
- 原 PDF 第 42、105 页：原文链接
- 原 PDF 第 48 页：原文链接
- 原 PDF 第 50 页：原文链接
- 原 PDF 第 65 页：原文链接
- 原 PDF 第 68 页：原文链接
- 原 PDF 第 74 页：原文链接
- 原 PDF 第 88 页：原文链接

- 原 PDF 第 88 页：原文链接

第 3 章

- 原 PDF 第 112 页：原文链接
- 原 PDF 第 112 页：原文链接
- 原 PDF 第 112 页：原文链接

第 4 章

- 原 PDF 第 133 页：原文链接
- 原 PDF 第 142 页：原文链接
- 原 PDF 第 144 页：原文链接
- 原 PDF 第 144 页：原文链接
- 原 PDF 第 144、565 页：原文链接
- 原 PDF 第 144、190、241 页：原文链接

第 5 章

- 原 PDF 第 145 页：原文链接
- 原 PDF 第 155 页：原文链接

第 6 章

- 原 PDF 第 156 页：原文链接
- 原 PDF 第 159 页：原文链接
- 原 PDF 第 159 页：原文链接
- 原 PDF 第 162 页：原文链接
- 原 PDF 第 162 页：原文链接

- 原 PDF 第 162 页：原文链接
- 原 PDF 第 162 页：原文链接
- 原 PDF 第 164 页：原文链接
- 原 PDF 第 166 页：原文链接
- 原 PDF 第 167 页：原文链接
- 原 PDF 第 170 页：原文链接
- 原 PDF 第 170 页：原文链接
- 原 PDF 第 170 页：原文链接
- 原 PDF 第 170 页：原文链接
- 原 PDF 第 170 页：原文链接
- 原 PDF 第 174 页：原文链接
- 原 PDF 第 174 页：原文链接

第 7 章

- 原 PDF 第 180 页：原文链接
- 原 PDF 第 187 页：原文链接
- 原 PDF 第 187 页：原文链接
- 原 PDF 第 189 页：原文链接
- 原 PDF 第 190 页：原文链接
- 原 PDF 第 190 页：原文链接
- 原 PDF 第 191 页：原文链接
- 原 PDF 第 191 页：原文链接

第 8 章

- 原 PDF 第 194 页：原文链接
- 原 PDF 第 198 页：原文链接

- 原 PDF 第 198 页：原文链接
- 原 PDF 第 198 页：原文链接

第 9 章

- 原 PDF 第 220 页：原文链接
- 原 PDF 第 228 页：原文链接
- 原 PDF 第 238 页：原文链接
- 原 PDF 第 238 页：原文链接
- 原 PDF 第 238 页：原文链接
- 原 PDF 第 239 页：原文链接
- 原 PDF 第 239、300 页：原文链接
- 原 PDF 第 239 页：原文链接
- 原 PDF 第 239 页：原文链接
- 原 PDF 第 241 页：原文链接
- 原 PDF 第 241 页：原文链接

第 10 章

- 原 PDF 第 266 页：原文链接
- 原 PDF 第 267 页：原文链接
- 原 PDF 第 267 页：原文链接
- 原 PDF 第 267 页：原文链接
- 原 PDF 第 273、289、512 页：原文链接
- 原 PDF 第 273 页：原文链接

第 11 章

- 原 PDF 第 274 页：原文链接
- 原 PDF 第 275 页：原文链接
- 原 PDF 第 276 页：原文链接
- 原 PDF 第 277 页：原文链接
- 原 PDF 第 278 页：原文链接
- 原 PDF 第 279 页：原文链接
- 原 PDF 第 279 页：原文链接
- 原 PDF 第 280 页：原文链接
- 原 PDF 第 280 页：原文链接
- 原 PDF 第 280 页：原文链接
- 原 PDF 第 280 页：原文链接
- 原 PDF 第 280 页：原文链接
- 原 PDF 第 281 页：原文链接
- 原 PDF 第 284 页：原文链接
- 原 PDF 第 285 页：原文链接
- 原 PDF 第 290 页：原文链接
- 原 PDF 第 291 页：原文链接
- 原 PDF 第 292 页：原文链接
- 原 PDF 第 293 页：原文链接
- 原 PDF 第 295 页：原文链接
- 原 PDF 第 295 页：原文链接
- 原 PDF 第 300 页：原文链接
- 原 PDF 第 302 页：原文链接
- 原 PDF 第 302 页：原文链接
- 原 PDF 第 304 页：原文链接
- 原 PDF 第 305 页：原文链接

- 原 PDF 第 309 页：原文链接
- 原 PDF 第 309 页：原文链接
- 原 PDF 第 309 页：原文链接
- 原 PDF 第 309 页：原文链接

第 12 章

- 原 PDF 第 310 页：原文链接
- 原 PDF 第 310 页：原文链接
- 原 PDF 第 310 页：原文链接
- 原 PDF 第 311 页：原文链接
- 原 PDF 第 311 页：原文链接
- 原 PDF 第 323 页：原文链接
- 原 PDF 第 323 页：原文链接
- 原 PDF 第 324 页：原文链接
- 原 PDF 第 324 页：原文链接
- 原 PDF 第 326 页：原文链接
- 原 PDF 第 326 页：原文链接
- 原 PDF 第 329 页：原文链接
- 原 PDF 第 331 页：原文链接
- 原 PDF 第 331 页：原文链接
- 原 PDF 第 331 页：原文链接
- 原 PDF 第 333 页：原文链接
- 原 PDF 第 335 页：原文链接
- 原 PDF 第 335 页：原文链接
- 原 PDF 第 335 页：原文链接
- 原 PDF 第 335 页：原文链接
- 原 PDF 第 335 页：原文链接

- 原 PDF 第 335 页：原文链接
- 原 PDF 第 335 页：原文链接
- 原 PDF 第 336 页：原文链接
- 原 PDF 第 338 页：原文链接
- 原 PDF 第 340、350 页：原文链接
- 原 PDF 第 341 页：原文链接
- 原 PDF 第 345 页：原文链接
- 原 PDF 第 345 页：原文链接
- 原 PDF 第 347 页：原文链接
- 原 PDF 第 348 页：原文链接
- 原 PDF 第 348 页：原文链接
- 原 PDF 第 350 页：原文链接
- 原 PDF 第 350 页：原文链接
- 原 PDF 第 350 页：原文链接
- 原 PDF 第 350 页：原文链接
- 原 PDF 第 350 页：原文链接
- 原 PDF 第 350 页：原文链接
- 原 PDF 第 351 页：原文链接
- 原 PDF 第 351 页：原文链接
- 原 PDF 第 354 页：原文链接
- 原 PDF 第 354 页：原文链接
- 原 PDF 第 355 页：原文链接
- 原 PDF 第 355 页：原文链接
- 原 PDF 第 355 页：原文链接

第 13 章

- 原 PDF 第 359 页：原文链接
- 原 PDF 第 360 页：原文链接
- 原 PDF 第 360 页：原文链接
- 原 PDF 第 377、420 页：原文链接
- 原 PDF 第 377 页：原文链接
- 原 PDF 第 377 页：原文链接
- 原 PDF 第 377 页：原文链接
- 原 PDF 第 377 页：原文链接
- 原 PDF 第 379 页：原文链接
- 原 PDF 第 382 页：原文链接
- 原 PDF 第 384 页：原文链接
- 原 PDF 第 384 页：原文链接
- 原 PDF 第 384 页：原文链接
- 原 PDF 第 384 页：原文链接
- 原 PDF 第 384 页：原文链接
- 原 PDF 第 384 页：原文链接
- 原 PDF 第 384 页：原文链接
- 原 PDF 第 385 页：原文链接

第 14 章

- 原 PDF 第 386 页：原文链接
- 原 PDF 第 386 页：原文链接
- 原 PDF 第 387 页：原文链接
- 原 PDF 第 390、404 页：原文链接
- 原 PDF 第 391 页：原文链接

- 原 PDF 第 394 页：原文链接
- 原 PDF 第 394 页：原文链接
- 原 PDF 第 395 页：原文链接
- 原 PDF 第 395 页：原文链接
- 原 PDF 第 397 页：原文链接
- 原 PDF 第 398 页：原文链接
- 原 PDF 第 398 页：原文链接
- 原 PDF 第 398 页：原文链接
- 原 PDF 第 399 页：原文链接
- 原 PDF 第 404 页：原文链接
- 原 PDF 第 404 页：原文链接
- 原 PDF 第 404 页：原文链接
- 原 PDF 第 404 页：原文链接
- 原 PDF 第 407 页：原文链接
- 原 PDF 第 408 页：原文链接
- 原 PDF 第 409 页：原文链接
- 原 PDF 第 409 页：原文链接
- 原 PDF 第 411 页：原文链接
- 原 PDF 第 411 页：原文链接
- 原 PDF 第 414 页：原文链接
- 原 PDF 第 414 页：原文链接
- 原 PDF 第 414 页：原文链接
- 原 PDF 第 416 页：原文链接
- 原 PDF 第 416 页：原文链接
- 原 PDF 第 418 页：原文链接
- 原 PDF 第 420 页：原文链接
- 原 PDF 第 421 页：原文链接

第 15 章

- 原 PDF 第 422 页：原文链接
- 原 PDF 第 423 页：原文链接
- 原 PDF 第 429 页：原文链接
- 原 PDF 第 433 页：原文链接
- 原 PDF 第 438 页：原文链接
- 原 PDF 第 439 页：原文链接
- 原 PDF 第 440 页：原文链接
- 原 PDF 第 445 页：原文链接
- 原 PDF 第 446 页：原文链接
- 原 PDF 第 449 页：原文链接
- 原 PDF 第 450 页：原文链接
- 原 PDF 第 455 页：原文链接
- 原 PDF 第 455 页：原文链接
- 原 PDF 第 455 页：原文链接
- 原 PDF 第 455 页：原文链接
- 原 PDF 第 455 页：原文链接
- 原 PDF 第 455 页：原文链接
- 原 PDF 第 455 页：原文链接
- 原 PDF 第 456 页：原文链接
- 原 PDF 第 456 页：原文链接
- 原 PDF 第 457 页：原文链接
- 原 PDF 第 461 页：原文链接
- 原 PDF 第 463 页：原文链接
- 原 PDF 第 463 页：原文链接
- 原 PDF 第 463 页：原文链接
- 原 PDF 第 463 页：原文链接

- 原 PDF 第 464 页：原文链接
- 原 PDF 第 464 页：原文链接
- 原 PDF 第 464 页：原文链接
- 原 PDF 第 464 页：原文链接
- 原 PDF 第 464 页：原文链接
- 原 PDF 第 464 页：原文链接
- 原 PDF 第 464 页：原文链接
- 原 PDF 第 464 页：原文链接
- 原 PDF 第 464 页：原文链接
- 原 PDF 第 465 页：原文链接
- 原 PDF 第 466 页：原文链接
- 原 PDF 第 466 页：原文链接
- 原 PDF 第 466 页：原文链接

第 16 章

- 原 PDF 第 469 页：原文链接
- 原 PDF 第 470 页：原文链接
- 原 PDF 第 470 页：原文链接
- 原 PDF 第 470 页：原文链接
- 原 PDF 第 474 页：原文链接
- 原 PDF 第 475 页：原文链接
- 原 PDF 第 476 页：原文链接
- 原 PDF 第 477 页：原文链接
- 原 PDF 第 478 页：原文链接
- 原 PDF 第 478 页：原文链接
- 原 PDF 第 479 页：原文链接
- 原 PDF 第 479 页：原文链接

- 原 PDF 第 479 页：原文链接
- 原 PDF 第 479 页：原文链接
- 原 PDF 第 479 页：原文链接
- 原 PDF 第 479 页：原文链接
- 原 PDF 第 479 页：原文链接
- 原 PDF 第 479 页：原文链接
- 原 PDF 第 479 页：原文链接
- 原 PDF 第 481 页：原文链接
- 原 PDF 第 483 页：原文链接
- 原 PDF 第 484 页：原文链接
- 原 PDF 第 486 页：原文链接
- 原 PDF 第 489 页：原文链接
- 原 PDF 第 489 页：原文链接
- 原 PDF 第 489 页：原文链接
- 原 PDF 第 490 页：原文链接
- 原 PDF 第 492 页：原文链接
- 原 PDF 第 493 页：原文链接
- 原 PDF 第 493 页：原文链接
- 原 PDF 第 494 页：原文链接
- 原 PDF 第 494 页：原文链接
- 原 PDF 第 494 页：原文链接
- 原 PDF 第 494 页：原文链接
- 原 PDF 第 494 页：原文链接
- 原 PDF 第 495 页：原文链接
- 原 PDF 第 499 页：原文链接
- 原 PDF 第 499 页：原文链接
- 原 PDF 第 499 页：原文链接

- 原 PDF 第 500 页：原文链接
- 原 PDF 第 500 页：原文链接
- 原 PDF 第 500 页：原文链接
- 原 PDF 第 500 页：原文链接
- 原 PDF 第 500 页：原文链接
- 原 PDF 第 501 页：原文链接
- 原 PDF 第 501 页：原文链接
- 原 PDF 第 501 页：原文链接

第 18 章

- 原 PDF 第 503 页：原文链接
- 原 PDF 第 503 页：原文链接
- 原 PDF 第 504 页：原文链接
- 原 PDF 第 512 页：原文链接
- 原 PDF 第 512 页：原文链接
- 原 PDF 第 513 页：原文链接
- 原 PDF 第 513 页：原文链接
- 原 PDF 第 513 页：原文链接
- 原 PDF 第 517 页：原文链接
- 原 PDF 第 518 页：原文链接
- 原 PDF 第 521 页：原文链接
- 原 PDF 第 521 页：原文链接
- 原 PDF 第 522 页：原文链接
- 原 PDF 第 522 页：原文链接
- 原 PDF 第 523 页：原文链接
- 原 PDF 第 527 页：原文链接
- 原 PDF 第 527 页：原文链接

- 原 PDF 第 527 页：原文链接
- 原 PDF 第 527 页：原文链接
- 原 PDF 第 530 页：原文链接
- 原 PDF 第 531 页：原文链接
- 原 PDF 第 532 页：原文链接
- 原 PDF 第 534 页：原文链接
- 原 PDF 第 534 页：原文链接

第 19 章

- 原 PDF 第 539 页：原文链接
- 原 PDF 第 539 页：原文链接
- 原 PDF 第 544 页：原文链接
- 原 PDF 第 547 页：原文链接
- 原 PDF 第 552、565 页：原文链接
- 原 PDF 第 554 页：原文链接
- 原 PDF 第 556 页：原文链接
- 原 PDF 第 557 页：原文链接
- 原 PDF 第 557 页：原文链接
- 原 PDF 第 557 页：原文链接
- 原 PDF 第 559 页：原文链接
- 原 PDF 第 559 页：原文链接
- 原 PDF 第 559 页：原文链接
- 原 PDF 第 560 页：原文链接
- 原 PDF 第 560 页：原文链接
- 原 PDF 第 561 页：原文链接
- 原 PDF 第 563 页：原文链接
- 原 PDF 第 563 页：原文链接

- 原 PDF 第 563 页：原文链接
- 原 PDF 第 563 页：原文链接
- 原 PDF 第 563 页：原文链接
- 原 PDF 第 564 页：原文链接
- 原 PDF 第 564 页：原文链接
- 原 PDF 第 564 页：原文链接
- 原 PDF 第 564 页：原文链接
- 原 PDF 第 565 页：原文链接
- 原 PDF 第 565 页：原文链接
- 原 PDF 第 565 页：原文链接
- 原 PDF 第 565 页：原文链接
- 原 PDF 第 566 页：原文链接

附录 A

- 原 PDF 第 572 页：原文链接

附录 B

- 原 PDF 第 576 页：原文链接
- 原 PDF 第 581 页：原文链接
- 原 PDF 第 582 页：原文链接
- 原 PDF 第 583 页：原文链接
- 原 PDF 第 583 页：原文链接
- 原 PDF 第 584 页：原文链接
- 原 PDF 第 584 页：原文链接
- 原 PDF 第 584 页：原文链接
- 原 PDF 第 584 页：原文链接

- 原 PDF 第 584 页：原文链接
- 原 PDF 第 584 页：原文链接

共收录 397 个不同的原 PDF 外部 URL；页码是原 PDF 物理页码，中文 PDF 已重新排版。